



Hyperautomation with **Camunda**, **Flowable**, **Agentic AI**, and the **Cloud**

Building Intelligent, Scalable, and
Future-Ready Automation Ecosystems



Kennedy Chengeta, Ph.D. >



CLOUD-NATIVE
ARCHITECTURES



INTELLIGENT
AUTOMATION



AGENTIC AI
INTEGRATION



SCALABLE
OUTCOMES



INTELLIGENT
AUTOMATION

Orchestrate and automate
end-to-end business
processes.



CLOUD-NATIVE
ARCHITECTURES

Design resilient, flexible
systems built for
the cloud.



AGENTIC AI
INTEGRATION

Empower autonomous
decisions with
AI-driven intelligence.



SCALABLE
OUTCOMES

Drive growth and innovation
with measurable,
scalable results.

CENTURY PUBLISHERS

Hyperautomation with Camunda, Agentic AI, and the Cloud

A Technical Approach

Orchestrated Process Modelling for
Banking and Insurance

A Practitioner's Manuscript

Kennedy Chengeta

Architecture, Engineering, and Process Design

Hyperautomation with Camunda, Agentic AI, and the Cloud: A Technical Approach

Copyright © 2026 Kennedy Chengeta. All rights reserved.

Printed for internal technical and educational use.

Published by Century Publishers.

Contributors: Justice Muguda.

While the author has used good-faith efforts to ensure that the information and instructions contained in this work are accurate, the author disclaims all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. The architectural patterns and process models described here must be validated against the regulatory environment and risk appetite of the institution that adopts them.

First Edition: 2026

Document reference: HYP-CAM-AGI-CLD-01

About the Author

Kennedy Chengeta is a process-automation architect and Camunda Chapter Leader whose career spans three decades of enterprise platform engineering across Southern Africa's largest financial institutions.

His journey began with IBM Lotus Domino and progressed through IBM Business Automation Workflow, Oracle Fusion Middleware, and Software AG webMethods — a cross-platform path that gave him a rare understanding not only of how each platform works but of how institutions migrate between them and why they do.

Since adopting Camunda, Kennedy has led major hyperautomation initiatives at Standard Bank of South Africa, Nedbank, Discovery, CBZ Holdings, ZB Financial Holdings, Vodacom, and ACSA (Airports Company South Africa), among others, spanning retail and corporate banking, insurance, payments, and telecommunications. In 2023 he became a Camunda Chapter Leader, bridging the platform's engineering with the realities of regulated industries.

This manuscript draws on every stage of that career. It is written not from theory but from the concrete experience of building, migrating, and operating process platforms inside institutions where the consequences of getting it wrong are measured in regulatory findings, customer harm, and real money.

*To the process owners, solution architects, and risk officers
patiently turning tangled operations into models that can be governed.*

Preface

Hyperautomation is an unfortunate word for an important idea. The word suggests a faster version of the robotic process automation that banks and insurers spent the last decade deploying — more bots, more screen-scraping, more brittle scripts. The idea is almost the opposite. Hyperautomation, done well, is the disciplined orchestration of every capability an institution can bring to a business process: deterministic workflow, business rules, human judgement, machine-learning models, and — increasingly — autonomous agents that can reason over a task and decide how to complete it. The goal is not to remove people from the process. The goal is to make the process itself an explicit, observable, governable artefact.

This manuscript is about how to build that in a regulated financial institution, using three things together: Camunda as the process and decision orchestration layer, agentic AI as the reasoning layer, and the cloud as the elastic substrate that makes the whole thing operable. None of the three is sufficient alone. Camunda without agents automates only what can be fully specified in advance. Agents without an orchestrator are ungovernable — they cannot be audited, versioned, or paused. The cloud without the other two is just somewhere to run the problem. The argument of the manuscript is that the combination, assembled in the right order with the right controls, is what finally makes intelligent automation safe enough for a bank or an insurer to put into production.

Who this is for

The manuscript is written for the people who have to make hyperautomation real: solution architects designing the target state, process engineers modelling the work in BPMN and DMN, platform engineers running Camunda and the agent runtime on cloud infrastructure, and the risk, compliance, and model-governance officers who decide whether any of it is allowed into production. It assumes the reader is comfortable with the idea of a business process but not necessarily with BPMN notation, with the idea of a machine-learning model but not necessarily with the operational mechanics of running one. Every concept the argument depends on is built up from the ground.

How this manuscript is organised

The manuscript is ordered so that each part builds on the ones before it. Part I establishes the groundwork, including the modelling notations — BPMN and DMN — and carries decision modelling with DMN through to depth, since decisions are foundational to every process that follows. Part II steps back to the architectural essentials — what a workflow engine is and what orchestration means, the architectural difference between Camunda 7 and Camunda 8, the distinction between human workflows and straight-through processing, the orchestration of microservices, and event-driven process automation — so the reader has the architectural vocabulary before the detail. Part III sets out a reference architecture and Part IV builds the architecture proper. Parts V through VII then teach the platform itself: Part V the concrete components of a Camunda 8 platform, Part VI connectors, and Part VII the infrastructure the platform runs on. Part VIII treats running Camunda on Kubernetes in concrete depth — the

Helm deployment, the cluster's anatomy, and the architectural choice of running it with or without a Kafka event backbone. Parts IX and X extend that platform knowledge into the harder modelling territory — the advanced workflow patterns, and the batch-processing patterns. Part XI then compares the workflow platforms an institution chooses between, now that the reader can judge them against a real understanding of Camunda. Part XII treats migration to Camunda from proprietary suites, and Part XIII migration from the older BPEL standard. Part XV treats the relationship between Camunda and the low-code platforms an institution runs alongside it, Liferay among them. The next five parts treat the systems and content a financial process must integrate with: Part XVI the ECM and document-management systems that hold the process's documents; Part XVII core banking systems — Temenos, Oracle FLEXCUBE, Mambu, SAP; Part XVIII insurance systems, including Oracle's insurance platforms; Part XIX CRM and master data — Salesforce, Microsoft Dynamics, and the MDM golden record; and Part XX payment systems — the card schemes, processors, and PSPs such as PayPal. Part XIV is the deep technical guide to implementing the platform in Java. Part XXI works through applications as concrete process models, and Part XXII turns to the cross-cutting operational concerns. Part XXIII treats the platform as a producer of data; Part XXIV treats large language models and retrieval-augmented generation; and Part XXV treats agentic AI in depth. The final parts treat security as a whole posture (Part XXVI), the management of API authentication and digital tokens with Camunda (Part XXVII), and ad-hoc processes — work whose path is not known in advance (Part XXVIII).

Every chapter ends with two recurring elements. A *worked example* takes a concrete, numerical version of a problem from the chapter and works it through in full — problem, setup, working, and discussion — so that the reasoning is grounded rather than abstract. A *chapter in brief* consolidates the key takeaways. Many chapters also include a *practical next steps* section: a small, concrete exercise an architecture or process team can carry out in their own organisation, usually small enough to fit inside a working week.

Tip

Cross-references to other chapters appear in **red**, e.g. **Chapter 4**. *Italic* indicates new terms. `Constant width` is used for BPMN element types, code, and configuration keys.

A note on scope

This manuscript is not jurisdiction-specific, and it is not a compliance manual. It draws on practice across several banking and insurance markets and flags where the regulatory environment materially shapes what can be built, but the reader in any specific jurisdiction must verify the applicability of any specific guidance to their own context. Nor is it a Camunda product manual or an agent-framework tutorial; product mechanics change faster than print. The aim throughout is the working practitioner's question: what process can be automated, how should it be modelled, and what will be allowed to run inside a regulated financial institution.

Agentic components introduce a failure mode that classical automation does not have: a system that is confidently wrong. Every pattern in this manuscript that places an agent in a process also specifies the deterministic control — a rule, a boundary, a human task — that contains that failure mode. Treat those controls as part of the design, not as optional

hardening.

Acknowledgments

This manuscript owes its shape to the architects and process engineers who explained why early designs would never survive an operational-risk review, and to the model-risk and compliance reviewers who explained why they were right. The errors that remain are the author's own.

The Author
2026

Contents

Preface	v
Introduction: Lessons from the Age of Machines	9
I Foundations	13
1 What Hyperautomation Really Means	14
2 Process Modelling for Banking and Insurance	21
3 BPMN as the Language of Orchestration	28
4 DMN: Separating Decisions from Flow	33
5 The Rules Language: Fundamentals	37
6 Decision Tables in Depth	41
7 Decision Models: the DRD and Decision Composition	45
8 DMN and BPMN: the Decision in the Process	49
II Workflow Engines and Process Architecture	53
9 Workflow Engines and Process Orchestration	54
10 Camunda 7 versus Camunda 8 Architecture	57
11 Human Workflows versus Straight-Through Processing	61
12 Microservices Orchestration with Camunda	65
13 Event-Driven Process Automation	69
III A Reference Architecture	73
14 What Goes Where: A Modern Hyperautomation Architecture	74
15 AWS Step Functions and Camunda Compared	80
16 The Java Saga Pattern and Where It Fits	85
17 Enterprise Architecture: Where a Process Engine Fits	90
18 Solution Architecture and the Workflow	94
19 Technology Architecture and the Workflow	98
20 BIAN, the Service Landscape, and the Workflow	102

IV	Architecture	106
21	The Camunda Orchestration Core	107
22	Decision Services and Predictive Models	112
23	The Agentic Layer	120
24	The Cloud Platform	125
25	Integration with Core Banking and Insurance Systems	129
26	Data, Identity, and the Security Model	135
V	Key Camunda Components	140
27	The Engine and the Runtime	141
28	Modelling, Operating, and Connecting	146
29	Routing: Gateways and the Control of Flow	151
30	Task Allocation and the Management of Human Work	157
31	Java Integration: Delegates and the Embedded Engine	164
32	Job Workers and the External-Task Pattern	170
33	Connectors in Depth	176
34	Event-Driven Process Design	182
35	Security and Identity: Authentication and Authorization	188
36	The Process API	199
37	Starting a Process	204
38	The Task API	209
39	Listeners: Hooking into the Lifecycle	215
40	Asynchronous Continuations and the Transaction Boundary	220
41	Error Handling in the Worker	225
42	Camunda Forms: the Human Interface	231
43	Service Levels, Timers, and Calendar Management	235
44	Camunda Patterns	240
45	Camunda Antipatterns	244
46	Advanced Task Allocation	248
47	Starting Camunda Processes in Practice	253
48	Completing, Searching for, and Reassigning Tasks	257
49	Call Activities: Composing Processes from Processes	268
50	Subprocesses: Structure, Scope, and Boundary	273

51	Parallel Work and Multi-Instance Activities	279
52	The Camunda 8 Components in Depth	284
VI	Connectors in Depth	289
53	The Connector Model and the Connector Types	290
54	The Key Connectors in Practice	294
55	Building a New Connector	300
VII	Infrastructure	304
56	Where the Platform Runs: Deployment and Topology	305
57	Scaling, Resilience, and Backup	310
58	Infrastructure as a Governed Artefact	315
VIII	Running Camunda on Kubernetes	319
59	Camunda on a Kubernetes Cluster	320
60	The Anatomy of a Camunda Kubernetes Cluster	324
61	The Cluster With and Without Kafka	328
IX	Advanced Workflow Patterns	332
62	Parallelization: Doing Work at Once	333
63	Compensation: Undoing Work That Cannot Be Rolled Back	337
64	Callbacks, Correlation, and Waiting for the World	341
X	Batch Processing Workflow Patterns	345
65	Batch Processing in a Process-Orchestration World	346
66	Batch Workflow Architecture Patterns	350
67	Batch and Real-Time: the Boundary	353
XI	Comparing Workflow Platforms	357
68	Choosing an Orchestration Approach	358
69	Camunda as the Baseline	362
70	Flowable: The Closest Relative	366
71	AWS Step Functions and the Java Saga, Revisited	369
72	IBM Business Automation Workflow and the Proprietary Suites	374

73	webMethods and the Integration-Platform Approach	380
XII	Migrating to Camunda	384
74	The Anatomy of a BPM Migration	385
75	Migrating from IBM Business Automation Workflow	391
76	Migrating the BPMN Process Model	399
77	Migrating Coaches and Coach Views	403
78	Migrating Integration and System Services	409
79	Migrating Server-Side Script	414
80	Migrating Business Objects	419
81	Migrating Toolkits and Shared Assets	423
82	Migrating the WebSphere Runtime	430
83	Migrating In-Flight Process Instances	435
84	Migrating BAW Team Services	440
85	Migrating ECM Integration	444
86	Migrating from webMethods and Oracle	450
87	Migrating from jBPM and Closing the Migration	455
XIII	BPEL and the Migration to BPMN	459
88	What BPEL Is, and Why It Still Matters	460
89	Migrating BPEL Processes to BPMN in Camunda	464
90	Migrating BPEL and SCA Modules	469
XIV	Java Implementation for Camunda	474
91	Java Design Patterns and SOLID Principles for Camunda	475
92	The Java Toolchain and the Spring Boot Starter	479
93	Building Job Workers in Java	484
94	The Java Client: Driving the Engine from Code	490
95	Testing Camunda Processes in Java	498
96	Production-Grade Java: Hardening the Implementation	503
97	Troubleshooting and Operating Running Instances	509

XV Camunda and Low-Code Platforms	516
98 The Low-Code Landscape and Where Camunda Fits	517
99 Integrating Camunda with Liferay and the Experience Layer	521
100 Governing the Low-Code and Orchestration Estate	524
 XVI Integrating with ECM and Document Management	 527
101 Documents, Content, and the Process	528
102 Connecting Camunda to a Content Repository	533
103 Governing Documents Across the Process Estate	538
 XVII Integrating with Core Banking Systems	 542
104 The Core Banking System as a System of Record	543
105 Integrating with Temenos and Oracle FLEXCUBE	548
106 Integrating with Mambu and SAP	552
107 Core Banking Integration Patterns	556
 XVIII Integrating with Insurance Systems	 560
108 The Policy Administration System as a System of Record	561
109 Integrating with Oracle Insurance and the Major Suites	565
110 Underwriting, Claims, and the Insurance Process Estate	569
111 Insurance Integration Patterns	573
 XIX Integrating with CRM and Master Data	 576
112 The CRM and the Customer View	577
113 Master Data Management and the Golden Record	581
114 Integrating with Salesforce and Microsoft Dynamics	585
115 Customer Data Integration Patterns	589
 XX Integrating with Payment Systems	 593
116 The Payments Landscape	594
117 Authorization, Clearing, and Settlement	598
118 Integrating with Schemes, Processors, and PayPal	602
119 Payment Integration Patterns	606

XXI Applications	610
120 Customer Onboarding and Identity	611
121 Lending and Credit Origination	616
122 Payments, Disputes, and Exceptions	620
123 Insurance Underwriting and Claims	625
124 Servicing and the Ongoing Relationship	630
 XXII Operations	 634
125 Observability and Process Monitoring	635
126 Testing, Validation, and Model Risk	640
127 Regulation, Fairness, and Explainability	644
128 The Economics of Hyperautomation	649
129 The Operating Model and the Road Ahead	654
 XXIII Camunda and Data	 659
130 The Data Architecture of a Camunda Platform	660
131 Process Mining	664
132 Camunda Optimize	668
133 Streaming Process Data with Kafka	672
 XXIV Large Language Models and RAG with Camunda	 676
134 Large Language Models as a Process Capability	677
135 Retrieval-Augmented Generation in the Process	681
136 Prompts, Context, and the Cost of Language Models	685
 XXV Agentic AI	 689
137 What an Agent Is, and Why It Needs a Process	690
138 Agentic BPMN: Modelling the Agent in the Process	695
139 Multi-Agent Processes and the Future	699
 XXVI Security	 706
140 The Security Posture of a Process Platform	707
141 Protecting the Data the Platform Holds	711
142 Secrets, Threats, and the Secure Operating Discipline	715

XXVII Security and API Token Management with Camunda	719
143 Authenticating to Camunda: the Token Model	720
144 API Clients, Credentials, and the Token Lifecycle	724
145 Token Management as a Security Discipline	728
XXVIII Ad-hoc Processes	732
146 When the Path Is Not Known in Advance	733
147 Ad-hoc Work, Agents, and the Edge of Automation	738
A A Modelling Reference for BPMN and DMN	742
B Implementation Patterns	745
C A Regulatory and Governance Reference	748
Glossary	751

Introduction: Lessons from the Age of Machines

In 1842, in a cramped London office at Drummond Street, opposite Euston station, a small army of clerks sat bent over ruled ledgers, reconciling the tickets and revenues of nine independent railway companies whose tracks, rolling stock, and passengers crossed one another's boundaries thousands of times a day. No single company could account for the whole. No diagram described the complete journey. The only thing that held the system together was the office itself — the Railway Clearing House — and the painstaking, governed, auditable process it ran.

A hundred and eighty years later, a bank's account-opening process spans five microservices, three external APIs, a fraud-screening model, and a human approval step. No single service can account for the whole. No log describes the complete journey. The only thing that holds the system together is the workflow engine, and the painstaking, governed, auditable process model it runs.

The parallels are not accidental. The problems at the heart of enterprise automation — coordinating work across autonomous participants, governing decisions whose consequences are irreversible, detecting instability before it becomes catastrophe, and knowing, at every moment, where every case stands — are among the oldest problems in organised industry. They were met, with extraordinary ingenuity, by the engineers and institution-builders of the eighteenth and nineteenth centuries, using the materials they had: punched pasteboard, copper telegraph wire, cast-iron governors, and paper ledgers. The solutions they found are, in their essential architecture, the solutions this manuscript builds in BPMN, Camunda, and code — and understanding them is not a historical indulgence but a direct route to understanding why the modern platform is shaped the way it is.

This introduction draws six of those parallels. Each one names a principle — separation of model from machine, real-time distributed coordination, mechanical feedback, cross-boundary orchestration, the cost of overengineering, and the power of the division of labour — that the rest of the manuscript develops in technical depth. The problems are old. The tools are new. The architecture, as it turns out, has barely changed.

The Jacquard Loom: a model that directs the machine

In 1801 Joseph Marie Jacquard demonstrated a loom in Lyon that wove complex patterns automatically; by 1804 he had patented the attachment that bears his name. The breakthrough was not the loom itself — looms existed — but the *punch card*: a stiff card with holes punched in it, each hole directing a hook to lift or lower a thread. A sequence of cards, threaded together, was, in effect, a *program*: a declarative description of the pattern the machine should produce, separate from the machine that produced it. Change the cards and the same loom wove a different pattern. The pattern was not in the machine; it was in the model.

Two centuries later, a BPMN process model is a Jacquard card. It is a declarative description of the work — the steps, the decisions, the routing — separate from the engine that

executes it. Change the model and the same engine runs a different process. The separation Jacquard achieved between *what the machine does* (the cards) and *the machine itself* (the loom) is the separation this manuscript insists on between the *process model* and the *workflow engine*. The model is the institution's intellectual property — its description of how work is done — and it must be readable, versionable, and changeable independently of the engine. Jacquard showed, with punched holes in pasteboard, that a machine directed by an external model is fundamentally more governable than one whose behaviour is woven into its own mechanism.

The Telegraph Network: real-time signalling across a distributed world

Before the electric telegraph, information moved at the speed of a horse or a ship. A message from London to Edinburgh took days. The telegraph, developed through the 1830s and 1840s by Cooke, Wheatstone, and Morse, changed that utterly: a signal sent from one station arrived at another, hundreds of miles away, in seconds. By the 1860s a network of telegraph lines connected cities and nations, and trained *operators* at each station received signals, interpreted them, and routed them onward — a distributed network of human and machine, reacting in real time to messages arriving from anywhere.

A telegraph office received a signal, determined what it meant and who needed it, and dispatched the appropriate response — a message forwarded, an instruction relayed, an alarm raised. This is exactly what an event-driven hyperautomation platform does. Kafka events, external-task workers, APIs, and workflow triggers allow an enterprise to react instantly to operational events arriving from distributed systems. Camunda acts as a modern digital telegraph office: it receives signals from across the institution's landscape — a payment received, a policy lapsed, a fraud alert fired — and coordinates responses across multiple business domains in real time. The telegraph operators are the job workers; the telegraph lines are the event backbone; and the telegraph office's routing book is the BPMN process model that says which signal triggers which response. The nineteenth-century insight — that a network of stations, each reacting to signals and routing them according to a protocol, can coordinate action across a continent — is the architectural insight of event-driven process automation, expressed in copper wire rather than Kafka topics.

The Steam Engine Governor: feedback that prevents runaway

James Watt's centrifugal governor, designed in 1788, solved a problem that threatened to make the steam engine useless: the engine's own power could destroy it. Without regulation, a steam engine under light load would accelerate until it shook itself apart. Watt's governor was a mechanical feedback loop: two weighted arms, spinning with the engine, rose outward as speed increased, and their rising progressively closed the steam valve, reducing the power. If the engine slowed, the arms dropped, the valve opened, and power increased. The governor did not drive the engine; it *regulated* it — automatically, continuously, without human intervention — keeping speed within a safe range and preventing the runaway that would otherwise be inevitable.

Hyperautomation governance plays exactly this role in an AI-driven enterprise. As autonomous workflows accelerate operations — processing claims faster, scoring applications instantly, routing work without human delay — the institution requires controls that prevent

the acceleration from becoming a runaway. Camunda's process monitoring is the governor's spinning arms: it observes the engine's speed. Incident handling is the closing of the steam valve: when a process fails, the incident mechanism halts the affected instance and raises it for attention rather than letting it propagate. SLA timers are the governor's feedback loop: they detect when a process is taking too long and trigger an escalation before the delay compounds. And compensation flows are the governor's emergency stop: when a process that has already taken irreversible steps must be unwound, the compensation mechanism undoes them in the defined order. Together, these are the enterprise's "governors" — mechanisms that regulate speed, detect instability, and prevent the operational runaway that unchecked automation would, like an ungoverned steam engine, inevitably produce.

The Railway Clearing House: one orchestrator for many independent companies

In 1842, British railway companies faced a coordination crisis. Dozens of independent companies operated their own lines, their own rolling stock, and their own tickets — but passengers and goods regularly travelled across multiple companies' networks in a single journey. Who owed what to whom? How was a ticket sold by Company A, for a journey partly on Company B's track and partly on Company C's, reconciled and settled? The answer was the *Railway Clearing House*, established in London: a central body that received records from every company, reconciled the cross-company journeys, calculated the inter-company debts, and settled them. The Clearing House did not run the trains; it *orchestrated the settlement* across autonomous companies that ran their own operations.

This is the microservices orchestration problem, solved in 1842. A modern bank's account-opening process spans five independent services: validate the customer, check credit, screen for fraud, create the account, fund it. Each service is autonomous — independently deployed, independently owned — but the business process that spans them needs a single component that knows the whole journey: which service has been called, which is next, what happens if one fails and the earlier steps must be compensated. Camunda is the Railway Clearing House: it does not do the work — the services do the work, as the railway companies ran the trains — but it holds the model of the cross-service journey, reconciles the outcomes, and settles the process. The Railway Clearing House showed, in an age of steam and paper ledgers, that autonomous operators can coordinate complex cross-boundary work if, and only if, a single orchestrating body holds the model of the whole. That is the architectural argument for a workflow engine in a microservices estate, stated 180 years before anyone coined the word "microservices."

The NASA Pen: the cost of overengineering

A popular story claims that NASA, facing the problem that ballpoint pens do not work in zero gravity, spent millions of dollars developing a pressurised pen that could write in space — while Soviet cosmonauts simply used pencils. The story is partly mythologised (the Fisher Space Pen was privately developed, not a NASA expenditure), but the lesson it symbolises is real and recurs in enterprise technology with depressing regularity: organisations build massive architectures, governance committees, and expensive platforms when a simpler operational fix would solve the real problem more effectively.

The parallel to hyperautomation is direct. An institution that builds a sprawling AI-governance

framework, a multi-year platform programme, and a bespoke integration layer — when the actual problem is that three manual steps in a claims process need to be automated with a simple workflow and a rules table — has spent millions on the pen when a pencil would do. The discipline this manuscript argues for is not to build the largest possible platform but to build the *right-sized* one: to start from the problem, choose the simplest tool that solves it, and add complexity only when the problem genuinely demands it. Every architecture chapter that follows includes the question “is this component needed, or is it the pen?” — and the honest answer, more often than institutions care to admit, is the pencil.

The Pin Factory: specialisation and the division of labour

In 1776, Adam Smith opened *The Wealth of Nations* with a description of a pin factory. One worker, doing every step alone — drawing the wire, straightening it, cutting it, pointing it, grinding the head — could make perhaps twenty pins a day. But the same factory, with each step assigned to a specialist worker, produced forty-eight thousand. The division of labour — breaking a complex task into narrow, specialised steps and assigning each to the worker best suited to it — multiplied productivity by orders of magnitude.

The thin-worker pattern of this manuscript is Adam Smith’s pin factory. A Camunda job worker does one thing: it receives a job, calls a service, returns the result. It does not know the process; it does not decide what happens next; it does not hold state. Like Smith’s pin-maker who only points the wire, the worker’s narrowness is its strength — it is simple, testable, independently deployable, and replaceable. The process model, like the factory foreman, holds the sequence and the decisions; the workers, like the specialised pin-makers, execute their one step. An institution that builds fat workers — workers that contain process logic, make routing decisions, and hold state — has, in Smith’s terms, gone back to one person making the whole pin. The division of labour the pin factory demonstrated in 1776 is the division of responsibility the Camunda architecture insists on: the model orchestrates, the workers execute, and neither takes the other’s job.

These six parallels — the Jacquard card, the telegraph network, the steam governor, the Railway Clearing House, the NASA pen, and the pin factory — are not decorations. Each names a principle the manuscript develops in full: the model separated from the engine, event-driven real-time coordination, feedback and governance that prevent runaway, a single orchestrator for autonomous services, right-sizing over overengineering, and the division of labour between orchestration and execution. The engineers of the eighteenth and nineteenth centuries met these problems with the materials they had — pasteboard, copper wire, cast iron, paper ledgers — and the solutions they found are, in their essential architecture, the solutions this manuscript builds in BPMN, Camunda, and code. The problems are old; the tools are new; the principles have not changed.

PART I

Foundations

CHAPTER 1

What Hyperautomation Really Means

1.1 The word and the idea

The word *hyperautomation* entered the vocabulary of banking and insurance through analyst reports, and it arrived carrying a misleading promise. Read quickly, it suggests an intensification of what robotic process automation already did: if a software robot could click through a screen to move data between two systems, then hyperautomation would simply mean more robots, clicking faster, across more screens. That reading is not merely incomplete. It points an institution in precisely the wrong direction, because the brittle, unobservable, ungoverned bot is the problem that hyperautomation exists to solve, not the thing it scales.

The idea worth keeping is narrower and more demanding. Hyperautomation is the practice of treating a business process as an explicit, executable, observable model, and then orchestrating against that model every capability the institution can bring to bear: deterministic workflow, codified business rules, human judgement, predictive machine-learning models, and autonomous agents that can reason over an open-ended task. The emphasis falls on *orchestration* and on *model*. A process that has been hyperautomated is one whose structure is written down in a form a machine can execute and a person can audit, and whose every step — whether performed by a rule, a model, an agent, or a human — is visible, versioned, and governed.

This distinction is not academic. It determines what an institution builds, who owns it, and whether a regulator will allow it to run. An RPA bot automates a task; nobody can easily say what the bot will do when the screen changes. A hyperautomated process automates a *process*; the model says exactly what happens, the orchestrator enforces it, and the deviation of reality from the model is itself measured. The first is a labour-saving device. The second is an operating capability.

1.2 Why banking and insurance

Banking and insurance are not arbitrary settings for this manuscript. They are the settings where the argument is sharpest, for four structural reasons.

First, the core product of both industries *is* a process. A loan is not a physical object; it is the outcome of an origination process, serviced by another process, and closed by a third. An insurance policy is an underwriting decision followed by a servicing relationship and, eventually, a claims process. The institution's product quality and its process quality are very nearly the same thing. Improving the process is improving the product.

Second, both industries are document-heavy and decision-heavy in a way that suits the three technologies this manuscript combines. Bank statements, payslips, identity documents, medical reports, loss adjuster notes, policy wordings — the raw material of financial services is unstructured text and images that classical automation handles poorly and agentic AI han-

dles well. The decisions taken on that material — approve or decline, the price, the limit, the reserve — are exactly the points where a decision model belongs.

Third, both industries are heavily regulated, which is usually described as a constraint but is better understood as a forcing function. Regulation demands that decisions be explainable, that processes be auditable, that models be governed, and that a human be accountable. Those demands are uncomfortable for an unstructured pile of bots. They are natural for an orchestrated process model. The regulated industries are where hyperautomation, done properly, has the clearest advantage over hyperautomation done as RPA-at-scale.

Fourth, the economics are large enough to matter. Onboarding, lending, payments, underwriting, claims, and servicing are high-volume processes whose unit costs, cycle times, and error rates translate directly into the cost ratios that determine an institution's competitiveness. A modest improvement in a process touched by millions of cases a year is a material number.

Tip

A useful test of whether a candidate process belongs in a hyperautomation programme: can you draw it on a whiteboard as a sequence of steps and decisions? If you can, it can be modelled in BPMN and DMN. If you cannot — if the process is genuinely a matter of unstructured expert judgement with no describable structure — it is a poor first candidate, whatever its cost.

1.3 The three layers and why all three are needed

The subtitle of this manuscript names three things — Camunda, agentic AI, and the cloud — and the central claim is that they are necessary together. It is worth stating plainly why no two of them suffice.

Camunda alone — the orchestration layer — automates only what can be fully specified in advance. A BPMN model is a precise instruction: do this, then evaluate that rule, then route here or there. This is exactly right for the deterministic skeleton of a process, and it is auditable and governable by construction. But a BPMN model cannot, by itself, read a loss adjuster's free-text note and decide that the described damage is inconsistent with the claimed cause. It can call something that does, but it cannot be that thing. Orchestration without reasoning automates the structured and stops at the edge of the unstructured.

Agentic AI alone — the reasoning layer — can read the note, weigh the evidence, and propose a decision. What it cannot do, alone, is be governed. An agent invoked from a script, with no surrounding process model, leaves no audit trail that a regulator recognises, has no version that risk can sign off, cannot be paused when something goes wrong, and has no defined point at which a human takes accountability. Reasoning without orchestration is capability without control — and in a regulated institution, capability without control does not reach production.

The cloud alone is just somewhere to run the problem. It contributes elasticity, managed services, and the operational substrate on which the other two layers become economical to run and to scale. It is essential infrastructure and it is not, by itself, an automation strategy.

The combination is what works. The orchestrator provides the governable skeleton; the agents provide reasoning at the points where the skeleton meets unstructured reality; the cloud makes the assembly operable at the scale a bank or insurer needs. [Chapter 21](#) through [Chapter 24](#) build each layer in turn; the rest of the manuscript shows the combination at work.

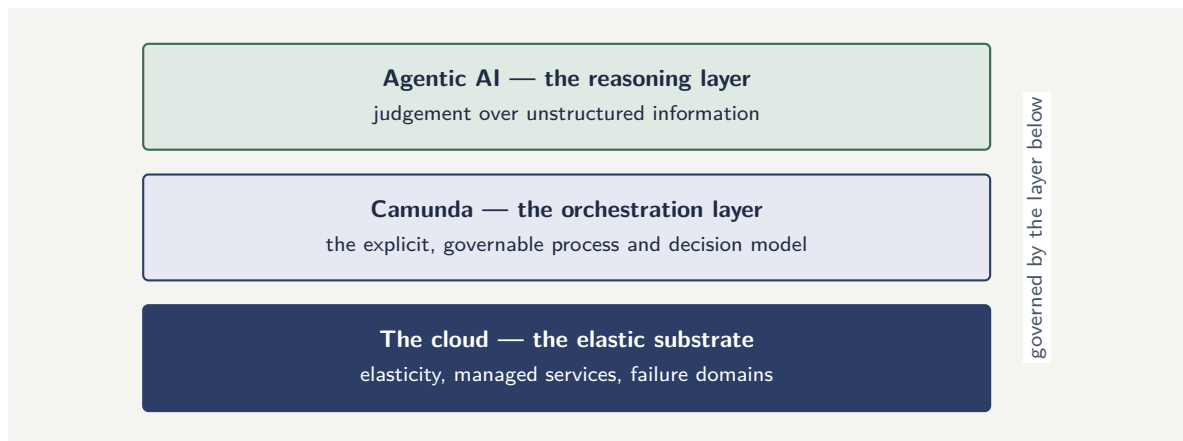


Figure 1.1. The three layers of a hyperautomation platform. Each layer is necessary and none is sufficient alone: the cloud makes the platform operable, the orchestration layer makes it governable, and the agentic layer extends it to unstructured work — but only because the governable structure beneath it contains it.

Figure 1.1 shows the relationship as a stack. It is deliberately drawn with the cloud at the base, the orchestration layer in the middle, and the agentic layer at the top, because that is the order of *dependence*: each layer is made safe and operable by the one beneath it. The agentic layer sits highest not because it is most important but because it is most dependent — it is governable only because the orchestration layer below it is, and operable only because the cloud below that makes it so.

1.4 What hyperautomation is not

It is worth being equally clear about what the term does not mean here.

It does not mean removing humans from processes. The mature pattern places humans deliberately — at points of accountability, judgement, and exception — and uses the model to make those placements explicit rather than accidental. A well-designed hyperautomated process often has *more* clearly defined human touchpoints than the manual process it replaced, because the model forces the question of who is accountable for each decision.

It does not mean a single platform that does everything. Hyperautomation is an integration discipline. The orchestrator, the decision engine, the agent runtime, the document services, the core banking or policy administration systems, and the data platform are distinct components with distinct owners. The manuscript treats their seams as first-class design concerns.

It does not mean a project with an end date. A hyperautomated process is a living artefact that drifts as the business, the data, and the regulation change. [Chapter 17](#) treats the operating model that sustains it.

1.5 A maturity progression, not a switch

It is tempting to treat hyperautomation as a destination an institution either has or has not reached. It is more useful, and more honest, to treat it as a progression through recognisable stages, because that framing tells an institution where it actually stands and what the next genuine step is.

At the first stage the institution has *point automations*: individual bots, scripts, and integra-

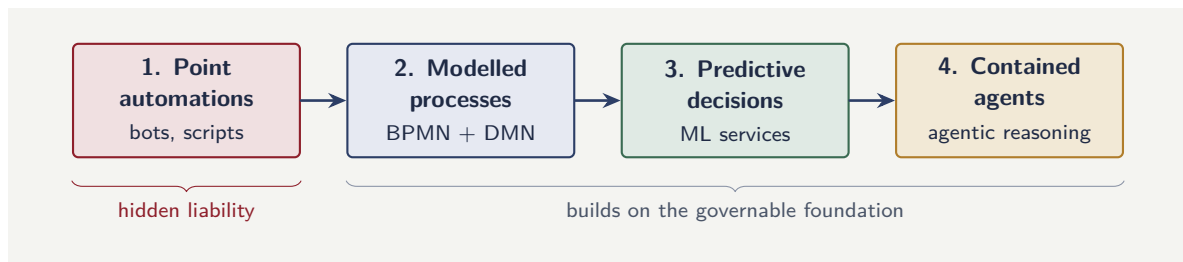


Figure 1.2. The hyperautomation maturity progression. The stages cannot be safely skipped: stages 3 and 4 add intelligence, but only the modelling of stage 2 makes that intelligence governable. An institution that jumps from stage 1 straight to agents builds capability on no foundation.

tions, each solving a local problem. There is no process model. This stage delivers real local savings and accumulates a hidden liability — a sprawl of brittle, unowned, unobservable automations that collectively describe nothing.

At the second stage the institution has *modelled processes*: the work is described in explicit BPMN and DMN models, orchestrated by an engine. The process is now visible, governable, and observable, even if every step within it is still a deterministic rule or a human task. This stage is the one most institutions underrate, because it is unglamorous — it adds no artificial intelligence — and yet it is the stage that delivers the governance, the auditability, and the operational control that everything else depends on. An institution that reaches this stage well has done the hardest and most valuable part.

At the third stage the institution adds *predictive decisions*: machine- learning models take their place, as governed decision services, at the points in the modelled process where a prediction beats a rule. The process gains quantified judgement without losing its governable structure.

At the fourth stage the institution adds *contained agents*: agentic reasoning is placed, with the containments of [Chapter 23](#), at the points where the modelled process meets genuinely unstructured work. The institution can now automate things that could not be specified in advance — and it can do so safely, because the agents sit inside a structure that was made governable two stages earlier.

The progression matters because the stages cannot be safely skipped. An institution that jumps from point automations straight to agents — adding intelligence to a process it never modelled — has built capability on no foundation: it has agents it cannot govern, observe, or explain, because the governable structure was never created. The order is the lesson. Model the process first; add prediction where it helps; add agents where they are needed; and never add a layer of intelligence to a process that the institution cannot already see.

Figure 1.2 draws the progression and marks its two critical features. The brace beneath stages 3 and 4 records that they rest on the governable foundation laid at stage 2; the marker beneath stage 1 records that point automations, left alone, are not a neutral starting point but an accumulating liability.

Tip

Locate your institution honestly on this progression, process by process. Most institutions are a mixture — a few processes well modelled, many still a sprawl of point automations — and the right next step is almost never “add agents.” For a process still at the point-automation stage, the highest-value next step is to model it, because the

modelling is what makes everything after it safe.

Worked example: sizing a hyperautomation candidate

Problem. A retail bank runs a personal-loan origination process that handles 240,000 applications a year. Today each application takes, on average, 38 minutes of staff handling time spread across intake, document checks, and a credit decision, at a fully-loaded cost of \$54 per staff-hour. A hyperautomation redesign is expected to remove 70% of the handling time on the 80% of applications that are “clean” (no exceptions), while the remaining 20% keep their current handling time. The redesign is estimated to cost \$1.6M to build and \$0.35M a year to run. Estimate the annual saving and the simple payback period.

Setup. The annual saving is the reduction in handling cost less the annual run cost. Handling cost is volume times time times hourly rate. Only the clean applications see a time reduction, and only 70% of their time is removed.

Working. Current annual handling cost:

$$240,000 \times \frac{38}{60} \text{ h} \times \$54 = 240,000 \times 0.6333 \times 54 \approx \$8.21\text{M}.$$

Clean applications are 80% of volume, i.e. 192,000 cases. Their current handling cost is

$$192,000 \times 0.6333 \times 54 \approx \$6.57\text{M},$$

and the redesign removes 70% of it: a gross reduction of $0.70 \times \$6.57\text{M} \approx \4.60M . The exception applications are unchanged. Net annual saving:

$$\$4.60\text{M} - \$0.35\text{M} = \$4.25\text{M}.$$

Simple payback period:

$$\frac{\$1.6\text{M}}{\$4.25\text{M per year}} \approx 0.38 \text{ years, about 4.5 months.}$$

Discussion. A payback under five months looks decisive, and for a process this size it often genuinely is — but the calculation rests on two assumptions worth stress-testing. The first is the 80% clean rate; if intake data quality is poorer than believed and only 60% of applications run clean, the saving falls to roughly \$3.1M and the payback extends past six months — still strong, but a different proposition. The second is the 70% time reduction on clean cases, which assumes the orchestrated process truly removes the steps rather than merely relocating them into exception handling. The disciplined version of this estimate writes both assumptions down explicitly, ties them to a measurement that will be taken after go-live, and treats the build as justified only if the measured clean rate and time reduction stay within a defined band of the estimate.

Worked example: where the unstructured edge actually is

Problem. An institution is told that automation can handle “most” of a claims process and wants a sharper number. The process has 100,000 claims a year. Of these, 71% arrive with clean, structured data and follow a path that pure rules and orchestration can fully automate. A further 22% arrive with some unstructured content — a free-text description, a photograph

— that must be interpreted before the structured path can continue; this interpretation is the work an agentic or model step does. The remaining 7% are genuinely novel or contested cases that need human judgement. The institution currently staffs the process as though every claim needs human handling. Quantify how the work divides across the three layers, and what it means for staffing.

Setup. We split the 100,000 claims into the three bands — pure orchestration, orchestration-plus-reasoning, and human judgement — and consider what each implies.

Working. The three bands:

pure orchestration: $100,000 \times 0.71 = 71,000$,

orchestration + a reasoning step: $100,000 \times 0.22 = 22,000$,

genuine human judgement: $100,000 \times 0.07 = 7,000$.

So 93,000 claims a year — the first two bands together — can run without a human making the core decision, and 7,000 genuinely need one.

Discussion. The number that matters is not the headline “most” but the structure underneath it, and the worked example exists to show that the three layers of this chapter are not an abstraction — they are a partition of the actual work. The 71% is the domain of pure orchestration: structured data, a knowable path, rules and a process engine and nothing more. The 22% is exactly the *unstructured edge* the chapter named — claims that orchestration alone cannot finish, because something in them must first be *interpreted*, and interpretation is what the agentic and model layer adds. The 7% is the genuine residue of human judgement, and it does not vanish — the chapter was explicit that hyperautomation does not remove humans. The staffing consequence is stark: an institution staffing as though all 100,000 claims need human handling is staffing for fourteen times the genuine human-judgement load. But the deeper point is directional. The 22% band is where the institution’s investment in reasoning capability pays off, and its size is worth measuring precisely, because it is the band that pure RPA-style automation cannot touch and that orchestration-plus-reasoning can. Knowing that the split is 71/22/7, rather than a vague “most,” tells the institution exactly how much of its process is pure orchestration, exactly how much needs the reasoning layer, and exactly how much will always be human — and those three numbers, not the headline, are what an honest hyperautomation plan is built on.

Practical next steps

Pick one high-volume process in your institution and write a single page that states: its annual case volume, its current average handling time, the fraction of cases that run without exception, and who is accountable for its core decision today. Do not estimate savings yet. The exercise will reveal whether your organisation can even measure these four numbers — and if it cannot, that is the first finding of your hyperautomation programme.

Chapter in brief

Hyperautomation is not RPA at scale; it is the practice of treating a business process as an explicit, executable, governable model and orchestrating every capability — rules, models, agents, and humans — against it. Banking and insurance are the natural proving grounds because their products *are* processes, their raw material is unstructured, they are heavily regulated, and their process economics are large. The three layers —

Camunda for orchestration, agentic AI for reasoning, the cloud for elastic operation — are necessary together: orchestration without reasoning stops at the unstructured edge, reasoning without orchestration cannot be governed, and the cloud alone is only infrastructure. Hyperautomation does not remove humans, is not a single platform, and is not a project with an end date.

CHAPTER 2

Process Modelling for Banking and Insurance

2.1 Why model the process at all

A surprising number of automation programmes never produce a process model. They produce a collection of automations — a bot here, an integration there, a script somewhere else — each solving a local problem, none of them adding up to a description of how the work actually flows. The institution ends up automated but not modelled, and the difference is the subject of this chapter.

A *process model* is a precise, shared, executable description of how a unit of work moves from initiation to completion: the steps, the decisions, the actors, the events that can interrupt it, and the data that flows along it. It is precise because it uses a defined notation rather than prose. It is shared because the business, the architects, the engineers, and the risk function all read the same diagram. It is executable because the same model that documents the process also runs it — there is no separate, drifting implementation. These three properties are why modelling is the foundation of hyperautomation and not merely documentation overhead.

In banking and insurance the case for modelling is stronger still. A regulator examining a lending or claims process will ask to see how it works, will ask where the decisions are made, will ask who is accountable, and will ask how the institution knows the process is operating as intended. An institution with an executable process model answers all four questions by pointing at the model and its runtime data. An institution with a pile of bots answers them with a reconstruction — an after-the-fact guess at what the automation collectively does. The first is governance. The second is hope.

2.2 The shape of a financial process

Before introducing notation it helps to see what financial processes have in common, because the shape recurs and the modelling notations are designed around it.

A financial process is almost always *case-centric*: there is a thing — an application, a policy, a claim, a payment, a dispute — that moves through stages and accumulates data and decisions as it goes. It is *long-running*: it spans minutes to months, it waits on external parties, and it must survive system restarts without losing its place. It is *decision-dense*: at several points the process must evaluate a rule or a model and route accordingly. It is *exception-prone*: the happy path is a minority of real traffic, and the value of the process often lies in how cleanly it handles the unhappy paths. And it is *multi-actor*: customers, staff, third parties, automated services, and — now — agents all act on the same case.

Any modelling notation worth adopting must represent all five properties directly: the case and its data, the long-running waiting, the decisions, the exceptions, and the multiple actors. The pair of notations this manuscript uses — BPMN for the flow and DMN for the decisions — was designed for exactly this shape, which is why [Chapter 3](#) and [Chapter 4](#) treat

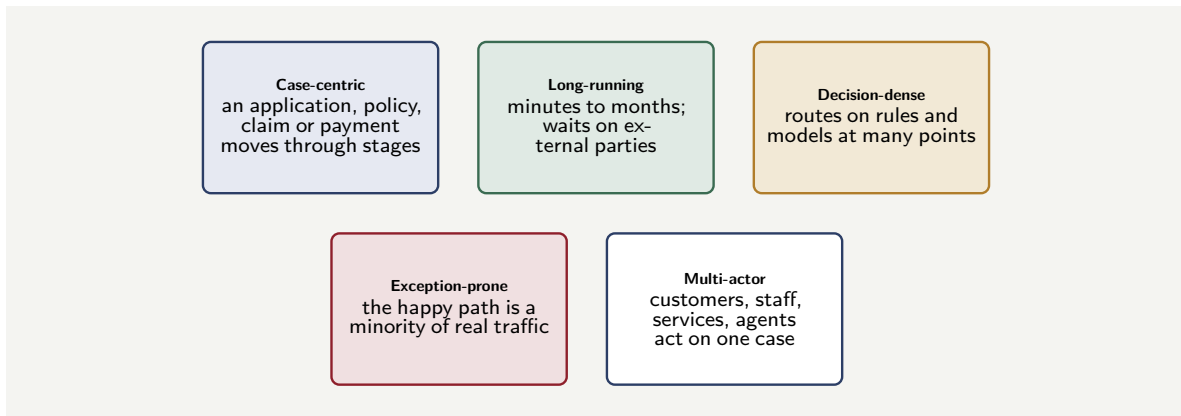


Figure 2.1. The five recurring properties of a financial process. Any modelling notation worth adopting must represent all five directly — and the BPMN/DMN pair was designed for exactly this shape.

them in depth.

Figure 2.1 collects the five properties. They are worth memorising, because they form a quick test: a candidate process that exhibits all five is squarely in the territory BPMN and DMN were built for, and one that exhibits none — a single, instantaneous, decision-free, single-actor step — probably does not need a process model at all.

2.3 Levels of modelling: from value stream to executable model

It is a common and expensive mistake to begin modelling at the wrong altitude — either so abstract that the model documents nothing useful, or so detailed that the model is unreadable and unmaintainable. It helps to recognise three distinct levels and to keep them distinct.

The *value-stream level* describes the end-to-end journey in a handful of stages: for a mortgage, perhaps *apply, assess, offer, complete*. This level is for executives and for scoping; it has no executable detail and is not meant to.

The *process level* describes one stage as a sequence of activities, decisions, and events — the level at which a process owner recognises their process and at which an architect can reason about it. This is the level most of this manuscript’s diagrams occupy.

The *executable level* adds everything the orchestration engine needs: the technical type of each task, the data mappings, the timer durations, the error definitions, the rule references. The same diagram carries both the process-level meaning and the executable detail, which is the central virtue of BPMN — but the modeller must consciously decide how much executable detail to expose to a business reader.

The most common modelling pathology in financial-services automation is the *executable model masquerading as documentation*: a diagram so dense with technical tasks, gateways, and error events that no business stakeholder can read it, yet not complete enough to actually run. It governs nothing and documents nothing. Decide the level of each model deliberately, and keep a readable process-level view alongside the executable one.

2.4 Value streams: the journey before the process

The value-stream level deserves more than a passing mention, because it is where hyperautomation should *begin* — and an institution that skips it tends to automate the wrong things well.

A *value stream* is the end-to-end sequence of activities through which an institution delivers something of value to a customer. It is described from the *customer's* point of view, in the customer's terms, and it spans whatever organisational boundaries it must. “Obtain a mortgage,” from the moment a customer first enquires to the moment the funds complete, is a value stream. “Make a claim,” from the first notification of loss to the moment the claim is settled, is a value stream. “Open and fund an account,” “be serviced through a life event,” “resolve a dispute” — each is a value stream. The defining quality is that a value stream is whole and customer-centred: it is the entire journey that delivers the value, not a department's slice of it.

This is precisely what makes the value stream different from a process. A process, as this manuscript models it, is typically owned by one function and bounded by that function's responsibility — the underwriting process, the payments process. A value stream cuts *across* processes and functions: the mortgage value stream runs through a sales process, an underwriting process, a valuation process, a completion process, each owned by a different team. The value stream is the customer's journey; the processes are the institution's internal segments of it. A customer does not experience the underwriting process and the completion process as separate things — they experience “getting a mortgage,” and that whole experience is the value stream.

Why value streams matter for hyperautomation

Beginning with value streams, rather than with individual processes, changes what an institution automates and in what order — and the change is for the better, for three reasons.

The first is that value streams reveal *where the customer's value actually is*, and therefore where automation is worth doing. A process viewed in isolation can be optimised heavily while the customer's overall journey barely improves — because the time and friction the customer experiences live in the *gaps between* processes, the hand-offs, the waits, which no single process owner sees. The value stream makes those gaps visible. An institution that maps the mortgage value stream often finds that the customer's longest wait is not inside any process at all but in the hand-off between two of them — and that is the thing most worth automating, and the thing a process-by-process view would never have surfaced.

The second is that value streams give automation the right *scope and sequence*. Hyperautomation done well is not a scramble to automate whatever process is loudest; it is a deliberate programme that takes whole value streams and improves them end to end. The value stream is the natural unit of an automation roadmap: “this year we improve the claims value stream,” not “this year we automate fourteen unrelated processes.” Scoping the programme by value stream is what keeps it aligned to customer value rather than to internal convenience.

The third is that value streams connect automation to *business strategy*. An institution's strategy is usually expressed in terms of customer outcomes — faster onboarding, a better claims experience, a particular product proposition. Those are value streams. When the automation programme is organised around value streams, the line from strategy to automation is direct and legible: the strategy names the value streams that matter, and the programme improves them. When automation is organised around individual processes, that line is broken, and the programme drifts into automating what is technically easy rather than what is

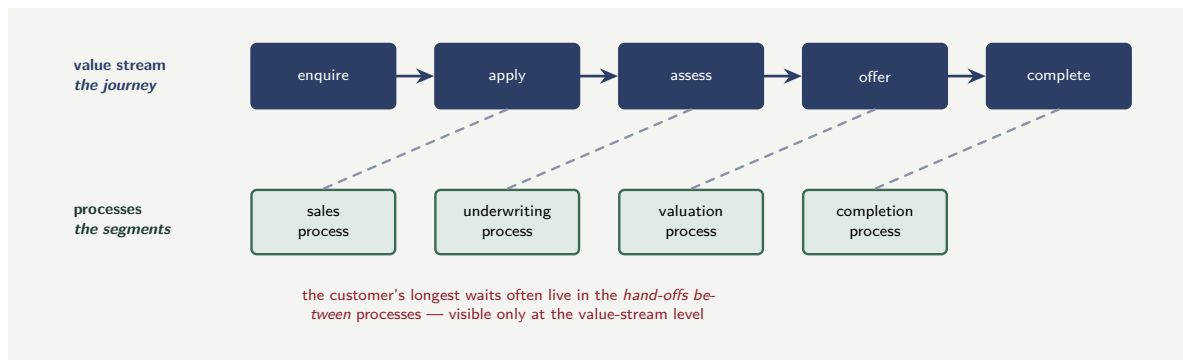


Figure 2.2. A value stream and the processes beneath it. The value stream is the customer's whole journey — enquire, apply, assess, offer, complete — while the processes are the institution's internal, function-owned segments of it. The friction a customer feels often lives in the hand-offs between processes, which only the value-stream view makes visible.

strategically wanted.

Figure 2.2 draws a value stream above the processes that realise it.

Value streams and the model levels

The value stream is the top of the three modelling levels this section opened with, and the relationship between the levels is now clearer. The value stream is the *customer's journey*, modelled in a handful of stages. Each stage is realised by one or more *processes*, modelled at the process level. Each process is made executable at the *executable level* in the engine. The levels nest: a value stream contains processes; a process contains executable detail.

The discipline this implies is to keep the value stream as a living artefact, not a one-off slide. The value stream is what the automation programme is *accountable to*: every process the institution automates should be traceable to the value stream it serves and the customer outcome that value stream delivers. A process that cannot be traced to a value stream is a process the institution should ask hard questions about — it may be automating internal activity that delivers no customer value at all.

Tip

Begin a hyperautomation programme by mapping value streams, not by listing processes. A value stream — “obtain a mortgage,” “make a claim” — is the customer's whole journey, and it is where customer value, and therefore the case for automation, actually lives. Processes are the institution's internal segments of value streams; automating them in isolation can optimise a segment while the customer's journey barely improves. Scope the programme by value stream, and trace every automated process back to the value stream it serves.

Worked example: the optimised process and the unchanged journey

Problem. A bank sets out to improve its mortgage business and chooses to automate the underwriting process, which its underwriting managers have long complained is slow. The automation is a success on its own terms: average underwriting time falls from 3 days to

4 hours. Yet six months later, customer satisfaction with “getting a mortgage” has barely moved, and the average time from a customer’s application to a completed mortgage is almost unchanged. The mortgage value stream, mapped end to end, has five stages with these average elapsed times: enquire-to-apply, 2 days; apply-to-assess hand-off, 9 days; assess (underwriting), formerly 3 days, now 4 hours; assess-to-offer, 1 day; offer-to-complete, 11 days. Explain what went wrong, using the value stream.

Setup. We compare the time the automation removed against the end-to-end value stream, to see how much of the customer’s journey actually changed.

Working. The total value-stream time before the automation, in days:

$$2 + 9 + 3 + 1 + 11 = 26 \text{ days.}$$

The automation reduced the assess stage from 3 days to 4 hours — a saving of about

$$3 \text{ days} - 4 \text{ hours} \approx 2.8 \text{ days.}$$

The new end-to-end time is therefore about

$$26 - 2.8 \approx 23.2 \text{ days.}$$

The customer’s journey improved by 2.8 days out of 26 — roughly an 11% reduction — while the underwriting process itself was made about 18 times faster. Meanwhile the two largest items in the value stream are the *hand-offs*: the 9-day apply-to-assess gap and the 11-day offer-to-complete gap, which together are

$$9 + 11 = 20 \text{ of the 26 days,}$$

and the automation touched neither.

Discussion. The bank automated a real process well and changed the customer’s experience almost not at all, and the worked example exists to show that this is the predictable result of choosing the automation target at the process level rather than the value-stream level. The underwriting managers complained, so underwriting was automated — a process-level view, driven by who was loudest. But the customer experiences the *value stream*, and at the value-stream level the underwriting stage was never the problem: it was 3 days out of 26, and the real time lived in the two hand-offs, 20 days between them, which no single process owner was accountable for and which therefore no one had proposed to fix. Those 20 days are exactly what the opening of this section called the friction *between* processes — invisible to a process-by-process view, glaring at the value-stream level. Had the bank begun by mapping the mortgage value stream, the 9-day and 11-day hand-offs would have been the obvious targets, and automating them — coordinating the hand-off, removing the wait — would have moved the customer’s 26-day journey far more than making an already-modest 3-day stage into a 4-hour one. The lesson is the section’s central discipline: the value stream is where customer value lives, so the value stream is where automation should be aimed. A programme that picks its targets process by process will reliably automate things well without improving the journey, because the journey’s problems hide in the gaps the process view does not show. Begin with the value stream, and the bank would have spent its effort on the 20 days, not the 3.

Practical next steps

Take one customer journey your institution cares about — obtaining a product, making a claim, resolving a problem — and map it as a value stream: the whole journey, in the customer’s terms, in a handful of stages, with the elapsed time of each stage *and each hand-off*

between stages. The largest numbers are very often the hand-offs, not the processes — and that is where a value-stream-led automation programme would aim first.

A process model with no owner decays. The chapter closes with the organisational point because it is the one most often skipped. Each model needs a named *process owner* from the business who is accountable for what the process is supposed to achieve; a *solution architect* accountable for how the model is structured; and, once the model contains agents or predictive models, a *risk owner* accountable for the governance of those components. **Chapter 17** develops the operating model in full; the point here is only that ownership is a property of the model, decided when the model is created, not an afterthought.

Worked example: counting the states of a claims process

Problem. A motor-claims process is described informally as having the stages *registered*, *triaged*, *assessing*, *approved*, *settled*, and *closed*, plus *rejected* and *withdrawn* as terminal outcomes. The team wants to know how many distinct process states a monitoring dashboard must track, and how many *transitions* between states the model must define if the only legal moves are forward through the main sequence, plus rejection from *triaged* or *assessing*, plus withdrawal from any non-terminal state.

Setup. States are simply the named stages including terminal outcomes. Transitions are ordered pairs of states between which a move is legal. We count the forward transitions, the rejection transitions, and the withdrawal transitions separately and sum them.

Working. The states are *registered*, *triaged*, *assessing*, *approved*, *settled*, *closed*, *rejected*, *withdrawn* — 8 states in total.

Forward transitions along the main sequence (*registered* → *triaged* → *assessing* → *approved* → *settled* → *closed*): that is 5 transitions.

Rejection transitions from *triaged* and from *assessing* to *rejected*: 2 transitions.

Withdrawal transitions from each non-terminal state to *withdrawn*. The non-terminal states are *registered*, *triaged*, *assessing*, *approved*, *settled* — the 5 states that are not *closed*, *rejected*, or *withdrawn*. That is 5 transitions.

Total transitions:

$$5 + 2 + 5 = 12.$$

Discussion. Eight states and twelve transitions is a small model, and that is the point of counting them: a process that can be enumerated this cleanly is an excellent modelling candidate, because every state is a dashboard metric and every transition is a place where a rule, a timer, or an SLA can attach. The exercise also exposes hidden complexity. “Withdrawal from any non-terminal state” sounds simple in prose but becomes five separate transitions in the model, each of which must define what happens to work in progress — a reserved amount, a pending payment, an assigned adjuster. Counting forces those questions into the open before the model is built, which is exactly when they are cheapest to answer.

Practical next steps

Take one process in your institution and list its states and its legal transitions as in the worked example. Then ask, for each transition, what triggers it and who or what is accountable for the move. The transitions you cannot cleanly answer for are the parts of the process that are not yet understood well enough to automate.

Chapter in brief

A process model is a precise, shared, executable description of how work flows; modelling, not automating, is the foundation of hyperautomation. Financial processes share a recurring shape — case-centric, long-running, decision-dense, exception-prone, multi-actor — and the BPMN/DMN pair is built for it. Model deliberately at three distinct levels — value stream, process, executable — and never let an unreadable executable model pose as documentation. The *value stream* — the customer's whole journey, such as “obtain a mortgage” — is where customer value lives and where a hyperautomation programme should begin; processes are the institution's internal segments of value streams, and the friction a customer feels often hides in the hand-offs between them. Every model needs a named process owner, a solution architect, and, once it contains agents or predictive models, a risk owner.

CHAPTER 3

BPMN as the Language of Orchestration

3.1 Why a standard notation matters

It would be possible to orchestrate processes in general-purpose code — state machines written in a programming language, with the flow expressed as branches and loops. Many institutions started there. The reason to move to the Business Process Model and Notation, *BPMN*, is not aesthetic. It is that BPMN is a standard, graphical, and executable at once, and those three properties together solve a governance problem that code cannot.

Because BPMN is a *standard*, the same diagram means the same thing to a process owner in the business, an architect, an engineer, a risk reviewer, and an auditor — and to the orchestration engine. There is one artefact, not a business diagram plus a separate and silently diverging implementation. Because it is *graphical*, a non-technical stakeholder can read the process and challenge it, which is the precondition for the business actually owning the process rather than delegating it to whoever wrote the code. Because it is *executable*, the diagram is not a description of the system; it *is* the system's control flow. The model that the auditor reviews is the model that runs.

3.2 The core BPMN vocabulary

BPMN has a large specification, but a disciplined financial-services model uses a small, deliberately restricted subset. The elements below are enough to model almost every process in this manuscript.

Events mark something that happens. A `start` event begins a process instance; an `end` event completes a path. Intermediate events occur during the process — most importantly the `timer` event, which waits for a duration or until a date, and the `message` event, which waits for something external to arrive. Events attached to the boundary of an activity — `boundary events` — are how the model expresses “while this is happening, something else might interrupt it,” which is the natural way to model SLAs and exceptions.

Activities are units of work. A `task` is a single step; the types that matter here are the `user task` (work for a human, placed on a worklist), the `service task` (work for an automated component — an integration, a rule evaluation, an agent invocation), and the `business rule task` (a decision delegated to a DMN model, the subject of [Chapter 4](#)). A `call activity` invokes another process model as a subprocess, which is how large processes are decomposed into governable pieces.

Gateways control flow. An `exclusive gateway` chooses exactly one outgoing path based on a condition — the workhorse of process routing. A `parallel gateway` splits flow into concurrent paths and later joins them. An `event-based gateway` waits for whichever of several events occurs first — the natural way to model “wait for the customer to respond, or for the deadline, whichever comes first.”

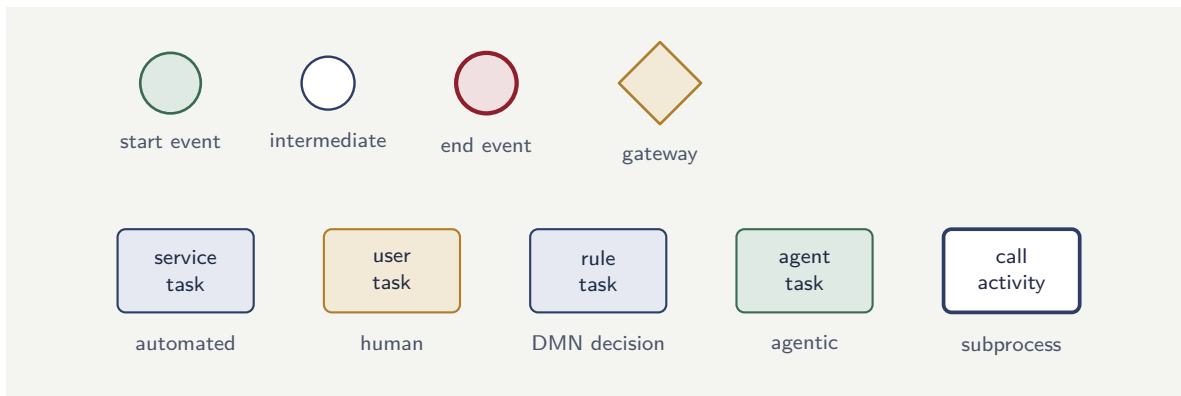


Figure 3.1. The recommended BPMN subset for financial-services modelling. Events mark what happens, tasks are units of work (automated, human, decision, or agentic), the gateway routes flow, and the call activity invokes a subprocess. A model built from these elements stays readable by the business and reviewable by risk.

Sequence flows connect the elements and carry the order of execution. *Pools and lanes* group activities by participant, which is how the model shows who — customer, staff, automated service, agent — is responsible for each part.

Tip

Restraint is a feature. A financial-services modelling standard should explicitly list the BPMN elements it permits and forbid the rest. A model built from start and end events, user, service, and business-rule tasks, call activities, exclusive, parallel, and event-based gateways, and timer, message, and error events can express almost everything — and stays readable by the business and reviewable by risk. Exotic elements buy little and cost clarity.

3.3 Modelling the long-running, waiting process

The single most important thing BPMN does for financial services, and the thing hand-written code does worst, is represent processes that *wait*.

A loan application waits for the customer to upload a document. A claim waits for a loss adjuster's report. A payment waits for a counterparty bank. These waits last hours or weeks; the process must survive restarts and deployments without losing its place; and the waiting must be interruptible — a timer must be able to escalate a wait that runs too long, and a cancellation must be able to end it.

BPMN expresses all of this directly. The wait is an intermediate message event or an event-based gateway. The escalation is a boundary timer event on the waiting activity. The cancellation is a boundary error or message event. The orchestration engine persists the state of every waiting instance, so a million claims can sit waiting for adjuster reports across a weekend and a platform restart, and each resumes exactly where it paused. Modelling the wait explicitly — rather than burying it in a polling loop somewhere in code — is what makes the long-running process governable: every waiting instance is visible, every SLA is a timer on the diagram, and every escalation is a path a reviewer can see.

3.4 Decomposition with call activities

Real financial processes are too large to model as a single readable diagram. A mortgage origination touches dozens of activities; drawn flat, it is unreadable and ungovernable. The discipline is decomposition: model the top-level process as a short sequence of call activities, each invoking a self-contained subprocess — *verify identity*, *assess affordability*, *value property*, *produce offer* — that is itself a readable model with its own owner.

Decomposition is not only a readability device. Each subprocess is a unit of governance: it can be versioned, reviewed, and reused independently. The *verify identity* subprocess that serves mortgage origination can serve account opening and credit-card application too, reviewed once and reused three times. [Chapter 21](#) returns to decomposition as an architectural principle.

3.5 Process variables and the flow of data

A BPMN diagram shows the flow of *control* — the order in which things happen — but a process also has a flow of *data*, and modelling that data deliberately is as important as modelling the control flow. A process instance carries a set of *process variables*: the working data the instance accumulates and acts on as it moves. An onboarding instance carries the application reference, the extracted document data, the classification result, the analyst’s decision; a claims instance carries the claim reference, the severity estimate, the fraud score, the adjuster’s findings.

Three disciplines govern process variables, and neglecting any of them produces a recognisable kind of trouble.

The first is *minimalism*. [Chapter 26](#) will make the security and privacy case for it, but the modelling case stands on its own: a process instance should carry the data it genuinely needs to make its decisions and route itself, and no more. An instance that accumulates every document and every intermediate value it ever touched becomes heavy, slow to persist, and hard to reason about. The authoritative detail belongs in the systems of record ([Chapter 25](#)); the instance carries a lean working set and references.

The second is *explicit mapping at every boundary*. When the process invokes a decision service, an agent, or an integration, the data passed *in* and the data captured *out* should be explicitly mapped, not implicitly assumed. A service task that silently reads and writes whatever process variables it likes is a hidden coupling: a change to one part of the process can break another through a shared variable nobody declared. An explicit input and output mapping for each task makes the data dependency visible, which is what makes the process safe to change.

The third is *typing and validation*. A process variable should have a known type and, where it matters, known constraints — a claim amount is a non-negative number, a classification result is one of a defined set of values. A process that discovers at a gateway that a variable it expected to be one of three values is in fact empty, or text where a number was expected, fails in a way that is hard to diagnose. Validating data as it enters the process, and at the boundaries where it is produced, turns a deep, late failure into a shallow, early, diagnosable one.

Tip

Treat the data a process carries with the same discipline as the control flow. For each

process, write down its variables, their types, and — for each task — what it reads and what it writes. The exercise routinely uncovers variables that nothing produces, variables that nothing consumes, and two tasks silently coupled through a shared variable. Each of those is a defect that an explicit data model surfaces before it becomes a production incident.

3.6 BPMN as a regulated artefact

In banking and insurance, a BPMN process model is not documentation — it is a *regulated artefact*. A regulator may ask to see the process, and the model must be the answer: accurate, current, and executable. This means the model must be maintained as a source of truth, versioned, reviewed before deployment, and never allowed to diverge from what the engine actually runs. A model that is drawn once and forgotten is worse than no model at all, because it gives the false assurance of governance while the real process drifts.

Worked example: reading an onboarding model

Problem. A customer-onboarding process is modelled as follows. A `start` event (application submitted) leads to a `service` task that extracts data from uploaded documents, then a `business rule` task that classifies the application as *straight-through*, *review*, or *decline*. An exclusive gateway routes on that result: *decline* goes to an end event; *straight-through* goes to a `service` task that opens the account; *review* goes to a `user` task for an analyst. The analyst's `user` task has a boundary timer event of 24 hours that escalates to a supervisor. After either the straight-through `service` task or the analyst's decision, an end event completes the process. The team wants to know how many distinct paths a process instance can take from start to end, treating the timer escalation as creating an additional path.

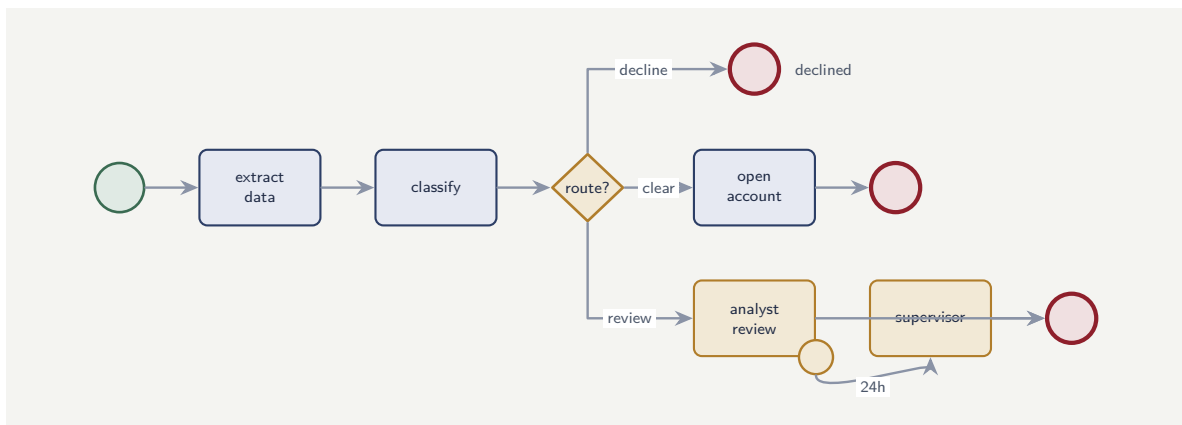


Figure 3.2. The onboarding model of the worked example. The exclusive gateway produces three branches, and the boundary timer on the analyst's user task adds a fourth path — the escalation to a supervisor. Four distinct paths means four test cases.

Figure 3.2 draws the model. Counting paths is now a matter of tracing the diagram rather than parsing the prose.

Setup. A path is a distinct route from the start event to an end event. We enumerate by the routing decision and then by whether the timer fires on the review path.

Working. The exclusive gateway produces three branches.

Branch 1 — *decline*: start → extract → classify → gateway → end. One path.

Branch 2 — *straight-through*: start → extract → classify → gateway → open account → end. One path.

Branch 3 — *review*: the analyst user task is reached. Either the analyst completes it within 24 hours (no escalation) or the boundary timer fires and the case escalates to a supervisor. That is two paths.

Total distinct paths:

$$1 + 1 + 2 = 4.$$

Discussion. Four paths through a process this small is a manageable number, and enumerating them is not busywork — it is the basis of testing. Each of the four paths is a test case the model must be exercised against before it goes live, and the boundary-timer path in particular is the one teams most often forget, because it does not appear when the process is run interactively at normal speed. The count also makes a design question visible: the escalated and non-escalated review branches both end the process, but do they end it in the *same* state? If the supervisor’s decision and the analyst’s decision feed the same downstream systems, the model is consistent; if they do not, the diagram has hidden a discrepancy that path-counting has just surfaced.

Practical next steps

Take an existing process diagram in your institution — or sketch one for a process you know — and enumerate every distinct path from start to end, including every boundary-event path. The number you get is the minimum number of test cases the process needs. If the number surprises you, the process is more complex than its owners believe.

Chapter in brief

BPMN is the language of orchestration because it is standard, graphical, and executable at once: one artefact that the business can read, risk can review, and the engine can run. A disciplined financial-services model uses a small, declared subset — start/end events, user/service/business-rule tasks, call activities, exclusive/parallel/event-based gateways, and timer/message/error events. BPMN’s decisive advantage is representing long-running, waiting, interruptible processes directly, with the engine persisting every waiting instance. Large processes are decomposed with call activities into independently governable subprocesses, and every distinct path through a model is a test case.

CHAPTER 4

DMN: Separating Decisions from Flow

4.1 The case for separating the decision

A process model expresses *flow* — the order in which things happen. A great deal of what a financial process actually does, however, is not flow but *decision*: given this applicant's data, what is the credit grade? Given this claim, which fraud-review queue? Given this policy, what is the premium loading? It is tempting to bury these decisions inside the process model, as conditions on gateways or as logic inside a service task. Resisting that temptation is the subject of this chapter, and the discipline has a name: the Decision Model and Notation, *DMN*.

The case for separating decisions from flow is governance again. A pricing rule, an eligibility rule, a risk-tiering rule — these change far more often than the process that uses them, they are owned by different people (underwriters, credit officers, pricing actuaries rather than process engineers), and they are scrutinised differently by regulators. If the rule is tangled into the BPMN model, every rule change is a process change: a redeployment, a regression test of the whole flow, a process-level review for what was really a pricing tweak. If the rule lives in a separate DMN model, invoked by the process through a `business rule` task, then the rule has its own version, its own owner, its own review cycle, and its own audit trail. The process asks the question; the decision model answers it; and the two evolve independently.

4.2 Decision tables and the DMN model

The heart of DMN is the *decision table*: a tabular representation of a decision in which each row is a rule, the input columns are the conditions, and the output columns are the results. A credit-tiering table might have input columns for income band, existing-debt ratio, and credit-bureau score, and an output column for the assigned tier. Each row says: when the inputs fall in these ranges, the tier is this.

The decision table is powerful precisely because it is boring. An underwriter or credit officer can read it, check it, and challenge it without being able to read code or BPMN. A regulator can be handed the table as the literal, complete statement of the decision logic. And the table has formal properties the tooling can check: *completeness* (is every combination of inputs covered by some rule?) and *consistency* (do any two rules contradict each other?). A decision tangled into code has neither readability nor those guarantees.

It is worth seeing an actual table, because the abstract description undersells how concrete the artefact is. Table 4.1 is a small claim-routing decision: two inputs — the estimated claim value and whether the policy is flagged for prior fraud — and one output, the handling queue.

The table repays a close reading. Each row is one rule; the dash in rules 1 and 4 means the prior-fraud flag is irrelevant to those rules. The input ranges on claim value are half-open intervals placed end to end — $[0, 1000)$, $[1000, 10000)$, $[10000, \infty)$ — so they partition the input: contiguous, non-overlapping, exhaustive. Because the ranges partition, and the

Table 4.1. A small DMN decision table for claim routing. The hit policy is `Unique`: the input ranges partition the input space, so exactly one rule matches any claim.

Rule	Claim value	Prior-fraud flag	Queue
1	[0, 1,000)	–	fast-track
2	[1,000, 10,000)	false	standard
3	[1,000, 10,000)	true	investigation
4	[10,000, ∞)	–	investigation

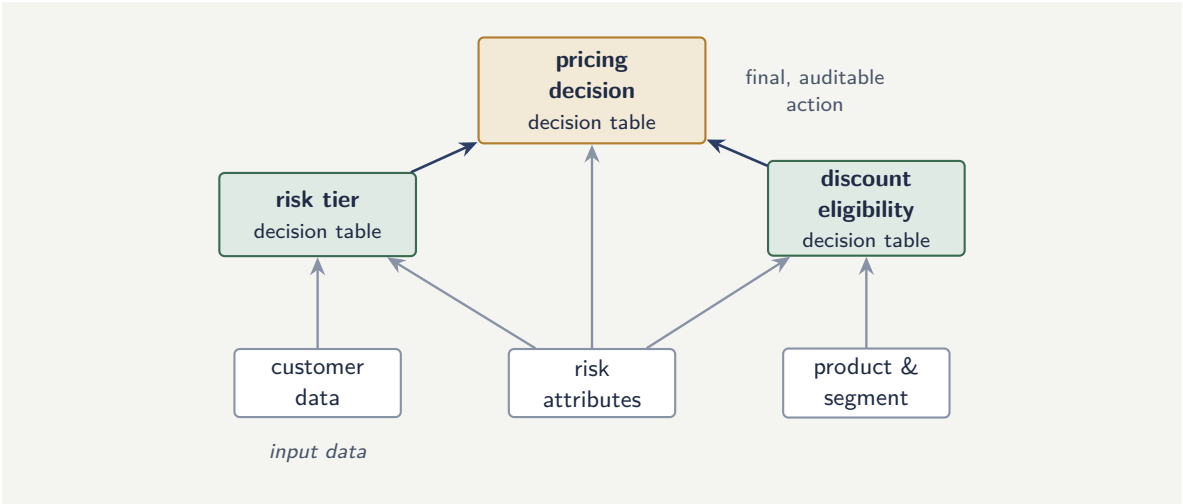


Figure 4.1. A decision requirements diagram. Each box is a separate decision table; arrows are information requirements. The final pricing decision depends on two intermediate decisions and on input data, and every table can be owned, versioned, and checked independently.

fraud flag is a simple true/false split where it appears, exactly one rule matches any possible claim, which is what the `Unique` hit policy requires. A regulator handed this table sees the entire routing logic; an underwriter can challenge a specific row; and the DMN tooling can verify mechanically that there is no gap and no overlap. That combination — human-readable and machine-checkable — is the whole reason the decision belongs in a table rather than in code.

A full DMN model is more than one table. A *decision requirements diagram* shows how decisions depend on one another and on input data: the final *pricing* decision might depend on a *risk tier* decision and a *discount eligibility* decision, each its own table, each fed by named input data. The diagram makes the decision logic itself a modelled, governable structure — the decision-side counterpart of the BPMN process model.

Figure 7.2 shows the structure for a pricing decision. Each table is a separate, independently governable artefact, and the diagram makes the dependencies between them — and the input data each consumes — explicit.

Tip

The *hit policy* of a decision table states what happens when several rules match. `Unique` means rules must not overlap — the safest choice for regulated decisions, because the tooling then enforces that no two rules can contradict. `First` and `Collect` have their uses, but a financial-services standard should default to `Unique` and require a documented

justification for anything else.

4.3 What belongs in DMN and what does not

Not every decision belongs in a decision table, and the boundary matters.

DMN is the right home for decisions that are *rule-based and stable enough to be written down*: eligibility, tiering, routing thresholds, premium loadings from a rating manual, document-checklist logic, escalation thresholds. These are decisions a human expert can articulate as rules, that must be auditable, and that change on a business cadence rather than a modelling cadence.

DMN is the wrong home for two kinds of decision. The first is the genuinely *predictive* decision — the probability that a claim is fraudulent, the expected severity of a loss — which is not a rule but a model trained on data; this belongs to a machine-learning model invoked as a service, treated in [Chapter 22](#). The second is the genuinely *open-ended reasoning* task — judging whether a free-text adjuster note is internally consistent — which belongs to an agent, treated in [Chapter 23](#). The mature pattern uses all three: the predictive model produces a score, the agent produces a reasoned judgement, and a DMN table consumes those outputs alongside hard rules to produce the final, auditable, deterministic decision. DMN is frequently the last, governing step — the place where probabilistic inputs are turned into a defensible action.

4.4 The decision as an audit artefact

A regulated institution must be able to answer, for any individual decision it made, the question: *why*? Why was this applicant declined, this claim routed to investigation, this premium loaded? When the decision lives in a DMN model, the answer is exact: the orchestration engine can record, for that specific case, which rule in which table fired and on what inputs. The explanation is not reconstructed; it is logged. This per-case decision trace is one of the most valuable things the Camunda-plus-DMN combination produces, and [Chapter 127](#) builds on it heavily when it treats explainability and regulation.

4.5 The DMN discipline in a regulated institution

In a regulated institution the value of DMN is not convenience but *auditability*. A decision expressed as a DMN table can be shown to a regulator, validated by a business analyst, versioned like any other model, and tested mechanically before deployment. A decision buried in code can do none of these. The discipline this chapter demands is that every business decision whose outcome a regulator might question — a credit decision, a pricing tier, a risk classification — be expressed in DMN and managed as a first-class artefact, owned by the business, not hidden inside a developer's code. DMN is not a modelling nicety; it is the institution's ability to show, prove, and change its decisions under control.

Worked example: checking a decision table for completeness

Problem. A claim-routing decision table has a single input — estimated claim value — and routes each claim to one of three queues. The three rules are: value *up to* \$1,000 routes to *fast-*

track; value *from* \$1,000 to \$10,000 routes to *standard*; value *above* \$25,000 routes to *complex*. The hit policy is `Unique`. A reviewer is asked to check the table for completeness and consistency over all non-negative claim values.

Setup. Completeness means every possible input value is matched by at least one rule. Consistency under a `Unique` hit policy means no input value is matched by more than one rule. We check the number line of non-negative values against the three rules' ranges.

Working. The three rules cover the ranges:

$$[0, 1,000], \quad [1,000, 10,000], \quad (25,000, \infty).$$

Consistency: the value \$1,000 lies in both the first range ($[0, 1000]$) and the second ($[1000, 10000]$). Two rules match a single input, which violates the `Unique` hit policy at the boundary.

Completeness: the interval $(10,000, 25,000]$ — claims worth more than \$10,000 but not more than \$25,000 — is matched by no rule at all. The table is incomplete over that range.

So the table fails both checks: an overlap at \$1,000 and a gap on $(10,000, 25,000]$.

Discussion. Both defects are the kind that never appear in casual testing and reliably appear in production. The overlap at exactly \$1,000 is a boundary error — the classic off-by-one of decision tables — and under a `Unique` policy the DMN engine will reject a claim valued at exactly \$1,000 rather than route it. The gap on $(10,000, 25,000]$ is worse: claims in that band match nothing, and depending on the engine's configuration they either error or fall through silently to a default, which means a whole class of mid-value claims is mishandled. The fix is to make the ranges *partition* the input: half-open intervals $[0, 1000)$, $[1000, 10000)$, $[10000, \infty)$ — contiguous, non-overlapping, and exhaustive. This is exactly the check DMN tooling can perform automatically, which is the entire argument for putting the decision in a table rather than burying it in code, where neither defect would have been caught at all.

Practical next steps

Take one decision in your institution that is currently expressed in code, configuration, or a spreadsheet, and rewrite it as a DMN decision table with explicit input ranges. Then check the ranges partition the input space — contiguous, non-overlapping, exhaustive. The gaps and overlaps you find are defects that are live in production today.

Chapter in brief

DMN separates decisions from flow so that rules — which change often, are owned by underwriters and credit officers, and are scrutinised by regulators — have their own version, owner, review cycle, and audit trail, invoked by the process through a business-rule task. The decision table is powerful because it is boring: readable by non-technical experts and formally checkable for completeness and consistency, especially under a `Unique` hit policy. DMN is the right home for rule-based, stable decisions, the wrong home for predictive and open-ended reasoning tasks — but it is frequently the final, governing step that turns probabilistic model and agent outputs into a deterministic, auditable action.

CHAPTER 5

The Rules Language: Fundamentals

5.1 Why decisions deserve a part

Chapter 4 introduced DMN — Decision Model and Notation — and made the foundational case: decisions belong in an explicit, separately-owned artefact, not buried in process flow or in code. That chapter established *why*. This part establishes *how*, in the depth a practitioner needs: the rules language DMN is built on, the structure of decision tables, the larger decision models that connect them, and the disciplines that keep a decision estate governable.

Decisions deserve a part of their own because, in a banking or insurance platform, the decisions *are* much of the business. Whether to approve a loan, what premium to charge, which transactions to flag, how to tier a customer — these are decisions, made millions of times, each one a place the institution’s policy meets a specific case. The process orchestrates; but it is the decisions that embody what the institution will actually *do*. An institution that models its processes carefully and leaves its decisions scattered and ungoverned has governed the easier half.

5.2 What a rules language is

At the bottom of DMN is a *rules language* — a precise, formal way to express the logic of a decision. Before decision tables, before decision models, there is the question of how a single rule is even written, and a practitioner must understand the rules language because every decision table cell, every condition, every computed output is an expression in it.

A rules language exists to express three kinds of thing precisely. It expresses *conditions* — tests against the inputs of a decision: is the applicant’s age over 25, is the loan amount in a certain band, is the country on a list. It expresses *outcomes* — the values a decision produces: an approval status, a rate, a tier. And it expresses *computations* — values derived from the inputs: a debt-to-income ratio computed from income and obligations, a premium computed from a base rate and factors.

The defining quality a rules language must have, for a financial institution, is that it is *unambiguous and evaluable*. Unambiguous, so that a rule means exactly one thing and two readers — a developer, an auditor — cannot disagree about what it says. Evaluable, so that the engine can execute it deterministically: the same inputs always produce the same output. A rule written in prose — “approve if the applicant is creditworthy and the amount is reasonable” — is neither; a rule written in a formal rules language is both.

5.3 FEEL: the expression language of DMN

DMN’s rules language is called *FEEL* — the Friendly Enough Expression Language — and the name is a deliberate statement of intent. FEEL is designed to be *friendly enough* that a business

analyst, not only a programmer, can read and write it, while remaining formal enough to be unambiguous and executable. That balance — readable by the business, precise enough for the engine — is exactly the balance DMN as a whole is built to strike, and FEEL is where it is struck at the level of the individual expression.

FEEL expresses the three things a rules language must. For *conditions*, FEEL has comparisons and a particularly important construct, the *range*: a band like “[1000..5000)” that means a thousand up to but not including five thousand. Ranges matter enormously in financial decisions, where so much logic is banded — loan amounts, ages, scores, balances all fall into bands — and FEEL’s range syntax expresses a band directly and unambiguously, with the square and round brackets making explicit whether each endpoint is included. For *outcomes*, FEEL has literal values — numbers, strings, dates, the boolean true and false. For *computations*, FEEL has arithmetic, a library of built-in functions, and the ability to work with dates, durations, lists, and structured data.

The practitioner’s discipline with FEEL is restraint. FEEL is capable enough that a determined author can write quite elaborate expressions in it — and an elaborate FEEL expression buried in a decision-table cell is exactly the “decision logic hidden in a hard-to-see place” that DMN exists to prevent. The right use of FEEL is many *simple* expressions, each in its proper place in a decision table or model, not a few clever ones doing too much. FEEL is the language of the cells; the structure of the decision belongs to the table and the model, which the next chapters treat.

Tip

FEEL — the Friendly Enough Expression Language — is DMN’s rules language, designed to be readable by business analysts yet precise enough to execute. Use it for many *simple* expressions, each in its proper cell: a comparison, a range like “[1000..5000)”, a small computation. An elaborate FEEL expression doing too much inside one cell is decision logic hidden where it cannot be reviewed — exactly what DMN exists to prevent. The structure of a decision belongs to the table and the model, not to a clever cell.

Figure 5.1 sets out FEEL and the three things a rules language expresses.

5.4 Applying the chapter in practice

The principles this chapter describes are most valuable when applied deliberately and reviewed periodically. A team that applies them once, at initial design, captures their value at that moment; a team that returns to them at each major change, each annual review, and each post-incident analysis captures their value continuously. The platform evolves; the principles hold; the specific choices they inform must evolve with the platform.

Worked example: the ambiguous rule made precise

Problem. A bank’s lending policy contains the rule, in its written manual: “Customers aged between 25 and 65 with a good income may be fast-tracked.” The bank is moving this rule into DMN. An analyst proposes to write the FEEL condition for age as the range “[25..65)” and asks whether that is correct. A second analyst objects that the policy is not as precise as it looks. Identify the ambiguities the written rule contains, and what FEEL forces the bank to resolve.

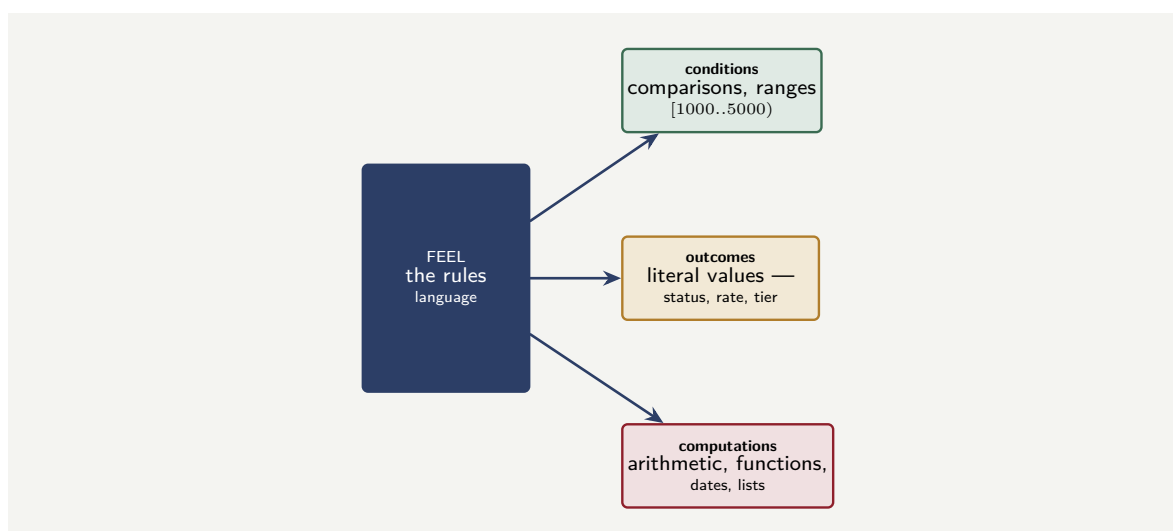


Figure 5.1. FEEL, DMN’s rules language, and the three things it expresses: conditions (comparisons and ranges), outcomes (literal values), and computations (arithmetic, functions, dates, lists). FEEL is friendly enough for a business analyst, precise enough for the engine.

Setup. We examine the written rule word by word for the points where it does not actually specify a precise, evaluable condition.

Working. The written rule has at least three ambiguities. First, “between 25 and 65” — are the endpoints *included*? Is an applicant who is exactly 25, or exactly 65, fast-tracked or not? The prose does not say; FEEL’s range syntax forces the answer, because “[25..65]” (both included) and “(25..65)” (both excluded) and “[25..65)” are three different, explicit rules. Second, “aged 25” — aged 25 in what unit, measured how? Age in completed years at the date of application is the likely intent, but “aged” is a computation — today’s date minus a date of birth — that must be defined. Third, “a good income” — this is not evaluable at all: “good” has no value. FEEL cannot express “good”; it can only express a precise condition, such as income at or above a stated threshold, which the bank must now *decide*.

3 ambiguities the prose hid; FEEL forces all 3 to be resolved.

Discussion. The second analyst is right, and the worked example shows something important about what a rules language *does*. The written policy rule reads as though it is clear — “aged between 25 and 65 with a good income” is a perfectly ordinary English sentence — but it is not precise enough to execute, and the imprecision is not a flaw in this particular rule; it is what prose is like. “Between” does not specify endpoint inclusion; “aged” does not specify the computation; “good” does not specify a value at all. None of this matters when a human underwriter applies the rule, because the underwriter fills the gaps with judgement — but it matters enormously when the rule is automated, because the engine cannot fill gaps, and an unresolved ambiguity becomes either an arbitrary technical default or an inconsistency. The value of FEEL here is not that it is a clever language; it is that it *will not let the ambiguity survive*. To write the rule in FEEL at all, the bank must decide whether 25 and 65 are included, must define how age is computed, and must replace “good income” with a real threshold — and those three decisions are genuine policy decisions that the prose rule had quietly left unmade. This is the deepest reason decisions belong in a rules language: the act of formalising a rule surfaces every place the policy was never actually precise, and forces the institution to decide, deliberately and on the record, what it had previously left to chance.

Practical next steps

Take one rule from your institution's written policy and try to write its conditions in FEEL. Every point where you cannot — an unclear endpoint, an undefined computation, a vague word like "good" or "significant" — is a place the written policy is not actually precise. Those points are not FEEL's difficulty; they are policy ambiguities FEEL has surfaced, and each is a decision the institution should make deliberately.

Chapter in brief

At the bottom of DMN is a rules language, and a practitioner must understand it because every decision-table cell is an expression in it. A rules language expresses conditions, outcomes, and computations, and for a financial institution it must be unambiguous and evaluable — which prose rules are not. DMN's rules language is FEEL, the Friendly Enough Expression Language, designed to be readable by business analysts yet precise enough to execute; its range syntax is especially important for the banded logic pervasive in finance. The discipline with FEEL is restraint — many simple expressions, each in its place, not a few elaborate ones. Formalising a rule in FEEL surfaces every ambiguity the written policy hid, and forces the institution to resolve it deliberately.

CHAPTER 6

Decision Tables in Depth

6.1 The decision table as the unit of decision logic

Chapter 4 introduced the decision table; this chapter treats it in the depth a practitioner building a real decision estate needs. The decision table is the central artefact of DMN — the place where most financial decision logic actually lives — and getting its structure right is most of getting a decision right.

A decision table expresses a decision as a grid. Its columns are of two kinds: *input* columns, each testing one input of the decision, and *output* columns, each producing one part of the decision's result. Its rows are the *rules*: each row says, in effect, “when the inputs match these tests in this row, produce these outputs.” A decision table is read row by row against a case: the case's input values are tested against each row, and the matching row or rows produce the outcome.

This grid form is what makes a decision table so well suited to financial decision logic. The logic of “which rate for which combination of loan-to-value band and credit-score band and term” is, naturally, a grid — and a decision table represents it as a grid, directly, where the same logic in code would be a thicket of nested conditionals that no business reviewer could check. The table is both the specification and the implementation.

6.2 The hit policy: how matching rows resolve

The single most important technical aspect of a decision table — and the one most often misunderstood — is the *hit policy*. The hit policy answers a question every decision table must answer: when a case is tested against the table, what happens if it matches *more than one* row?

DMN defines several hit policies, and a practitioner must choose deliberately. The *Unique* hit policy declares that the rules are written so that any case matches *exactly one* row — the rows do not overlap. This is the safest and clearest policy: each case has one rule, one outcome, no ambiguity, and tooling can verify that the rows genuinely do not overlap. The *First* hit policy declares that if several rows match, the *first* matching row in order wins — which makes the table's correctness depend on row *order*, a more fragile property. Other policies allow multiple rows to match and *collect* their outputs — summing them, listing them — which suits decisions that genuinely accumulate, such as a total of applicable fees.

The discipline is to default to *Unique*, because a Unique table is the one a reviewer and a tool can most easily verify is complete and consistent. A table with a different hit policy should have that policy because the *decision genuinely needs it* — a true accumulation, a deliberate priority order — not because the rows were allowed to overlap carelessly and a forgiving hit policy papered over it.

The hit policy is not a technical detail to set and forget — it determines what a decision table *means* when a case matches several rows. Default to the *Unique* hit policy: it de-

hit policy: Unique	inputs			output
	U	LTV band	credit score	rate
	1	[0..80)	[700..∞)	3.9%
	2	[0..80)	[0..700)	4.5%
	3	[80..∞)	—	5.2%

Figure 6.1. A decision table. Input columns (blue) each test an input; the output column (green) produces the result; each numbered row is a rule. The “U” marks the Unique hit policy — the rows are written not to overlap, so every case matches exactly one rule, and tooling can verify completeness and consistency.

clares the rows do not overlap, gives every case exactly one outcome, and can be verified by tooling. Choose another policy only when the decision genuinely needs it — a real accumulation, a deliberate priority. A *First* policy hiding accidentally-overlapping rows is a table whose correctness silently depends on row order.

6.3 Completeness and consistency

Two properties make a decision table trustworthy, and a practitioner must check both.

A table is *complete* if every possible case matches *at least one* row — there is no combination of inputs the table fails to cover. An incomplete table has a gap: some case falls through, matching no rule, and the decision either fails or falls to an arbitrary default. The banded logic of finance makes gaps easy to create — a range that should have been “[1000..5000)” written as “[1000..5000)” next to “(5000..∞)” leaves the value exactly 5000 uncovered if the first range excludes it and the second also excludes it.

A table is *consistent* if no possible case matches *contradictory* rows — two rows that a case can match at once but that produce conflicting outputs. Under a Unique hit policy, any overlap is an inconsistency by definition.

The decisive advantage of the decision table, stated already in [Chapter 4](#) and worth repeating here as the practitioner’s core discipline, is that completeness and consistency are *checkable* — mechanically, by DMN tooling, before the table is ever deployed. A decision buried in code has no such check; a decision table can be verified to have no gaps and no contradictions. A practitioner who deploys a decision table without running that check has discarded the table’s single greatest advantage.

Figure 6.1 draws a decision table with its inputs, output, rules, and hit policy.

6.4 Decision tables as FEEL: a concrete example

The hit policy and the completeness rules are clearest in a concrete table. Listing 6.1 is a fee-tier decision table in DMN XML: three rules with amount ranges, a UNIQUE hit policy, and the FEEL range expressions that govern each row. A table of this form is what the modeller builds in the decision editor; the XML is the model the engine executes.

Listing 6.1. A DMN fee-tier decision table in XML: UNIQUE hit policy, three rules covering three amount ranges, each producing a fee percentage.

```
1 <decision id="feeDecision" name="Fee Tier">
2   <decisionTable hitPolicy="UNIQUE">
3     <input id="amount" label="Transaction Amount">
4       <inputExpression typeRef="double">
5         <text>amount</text>
6       </inputExpression>
7     </input>
8     <output id="feePct" label="Fee %"
9       typeRef="double"/>
10    <!-- rule 1: up to 1000 -->
11    <rule>
12      <inputEntry><text>[0..1000]</text></inputEntry>
13      <outputEntry><text>0.5</text></outputEntry>
14    </rule>
15    <!-- rule 2: 1001 to 5000 -->
16    <rule>
17      <inputEntry><text>(1000..5000]</text></inputEntry>
18      <outputEntry><text>0.3</text></outputEntry>
19    </rule>
20    <!-- rule 3: above 5000 -->
21    <rule>
22      <inputEntry><text>>5000</text></inputEntry>
23      <outputEntry><text>0.2</text></outputEntry>
24    </rule>
25  </decisionTable>
26 </decision>
```

The FEEL range syntax is directly visible: `[0..1000]` includes both endpoints; `(1000..5000]` excludes the lower endpoint (so 1000 is covered by rule 1 only, not both); `>5000` is an open upper bound. *UNIQUE* means at most one rule matches for any input — and the three ranges are designed to be mutually exclusive and collectively exhaustive. Any completeness gap — an amount the rules do not cover — would return no result, which is what the completeness section warned of.

6.5 The DMN discipline in a regulated institution

In a regulated institution the value of DMN is not convenience but *auditability*. A decision expressed as a DMN table can be shown to a regulator, validated by a business analyst, versioned like any other model, and tested mechanically before deployment. A decision buried in code can do none of these. The discipline this chapter demands is that every business decision whose outcome a regulator might question — a credit decision, a pricing tier, a risk classification — be expressed in DMN and managed as a first-class artefact, owned by the business, not hidden inside a developer’s code. DMN is not a modelling nicety; it is the institution’s ability to show, prove, and change its decisions under control.

Worked example: the gap at exactly five thousand

Problem. A bank’s fee decision table has, among its input columns, one for the transaction amount. Two of its rules test that amount with the ranges `“[0..5000)”` and `“(5000..∞)”`. The table uses the Unique hit policy. A transaction for exactly \$5,000 is processed. The bank runs about 8,000 transactions of exactly \$5,000 a year (it is a common round figure). What happens to those transactions, and what does this reveal?

Setup. We test the value 5000 against the two ranges and see whether it matches any rule.

Working. The first range, `“[0..5000)”`, includes 0 and goes *up to but not including* 5000

— the round bracket excludes the endpoint. So 5000 does *not* match it. The second range, “(5000..∞)”, *excludes* 5000 — the round bracket again — and covers everything above. So 5000 does *not* match it either:

$$5000 \notin [0..5000) \quad \text{and} \quad 5000 \notin (5000..∞).$$

The value exactly 5000 matches *no rule*. The table is *incomplete* — it has a gap of width zero, at exactly one point — and the 8,000 transactions a year at exactly \$5,000 fall into that gap. Under the Unique hit policy, a case matching no rule produces no decision: those transactions fail the fee decision, or fall to whatever default the surrounding system applies.

Discussion. The gap is a single point on the number line — the value 5000 and nothing else — and that is exactly why it is dangerous: it is almost invisible to casual review. An analyst eyeballing the table sees “[0..5000)” and “(5000..∞)” and reads them as “below 5000” and “above 5000,” which *sounds* complete, and the missing point between them does not announce itself. But \$5,000 is a common round transaction amount, so the zero-width gap is hit 8,000 times a year. The worked example’s lesson is precisely the chapter’s discipline on completeness. This gap is not something a practitioner should hope to catch by reading the table carefully — it is something the DMN tooling’s completeness check catches *mechanically*, because a completeness check tests whether every possible input is covered and finds the uncovered point 5000 immediately. The fix is trivial once the gap is known: one of the ranges must include the endpoint — “[0..5000]” or “[5000..∞)” — so that 5000 matches exactly one rule. The deeper point is the one the chapter has built toward: a decision table’s great advantage over decision logic in code is that completeness and consistency can be *verified by tooling*, and a practitioner who deploys a table without running that verification is choosing to find the zero-width gaps in production, 8,000 transactions at a time, instead of finding them in seconds before deployment.

Practical next steps

For every decision table your institution has deployed, confirm that its completeness and consistency were checked by DMN tooling before deployment — not merely eyeballed. The banded, range-based logic of financial decisions makes zero-width gaps and subtle overlaps easy to create and nearly impossible to spot by eye. The tooling finds them in seconds; production finds them one mishandled case at a time.

Chapter in brief

The decision table is DMN’s central artefact, where most financial decision logic lives: a grid of input columns and output columns, with each row a rule. Its grid form suits the banded “which value for which combination” logic pervasive in finance. The hit policy determines what the table means when a case matches several rows — default to *Unique*, which declares the rows do not overlap and can be verified. Two properties make a table trustworthy: completeness (every case matches at least one rule) and consistency (no case matches contradictory rules), and the decisive advantage of the decision table is that both are mechanically checkable by tooling before deployment — an advantage discarded by any practitioner who deploys a table unchecked.

CHAPTER 7

Decision Models: the DRD and Decision Composition

7.1 Beyond the single table

The previous chapter treated the decision table in depth. But a real financial decision is rarely one table. “Should this mortgage be approved” depends on creditworthiness, on affordability, on the property valuation, on policy eligibility — and each of those is itself a decision, with its own inputs and its own logic. A real decision is a *structure of decisions*, and DMN’s way of expressing that structure is the *Decision Requirements Diagram* — the DRD.

7.2 The Decision Requirements Diagram

The DRD is the model that sits above the individual decision tables and shows how they connect. It is, in essence, a diagram of *which decisions depend on which* — and on what data.

A DRD has a small vocabulary. A *decision* — drawn as a rectangle — is a point where logic produces an outcome; it is typically realised by a decision table. An *input data* element — drawn distinctively — is a piece of data the decisions draw on, supplied from outside. And the *requirement* arrows show the dependencies: a decision *requires* the outcomes of the sub-decisions beneath it, and *requires* certain input data. Read top to bottom, a DRD shows a top-level decision resting on sub-decisions, those resting on further sub-decisions or on input data — the whole structure of how a complex decision is composed.

This composition is the same discipline as the process decomposition of [Chapter 3](#), applied to decisions. Just as a large process is decomposed into governable sub-processes, a large decision is decomposed into governable sub-decisions — each a decision table small enough to be read, reviewed, owned, and verified on its own. A mortgage-approval decision built as one monstrous decision table with forty columns is unreadable and unverifiable; the same decision built as a DRD of a dozen modest tables, each doing one coherent thing, is both.

Figure [7.2](#) draws a Decision Requirements Diagram composing a mortgage decision from sub-decisions.

7.3 Decisions, sub-decisions, and ownership

The DRD does for decision *ownership* what process decomposition did for process ownership. Each decision in the DRD — each table — can have its *own owner*, the function genuinely accountable for that piece of policy.

This matters because a complex financial decision crosses ownership boundaries. In the mortgage DRD, the creditworthiness sub-decision is owned by the credit-risk function; the affordability sub-decision by a function accountable for responsible-lending policy; the eligibility sub-decision by the product function. A single monolithic mortgage decision table would have to be “owned” by someone, and that someone would inevitably own policy they

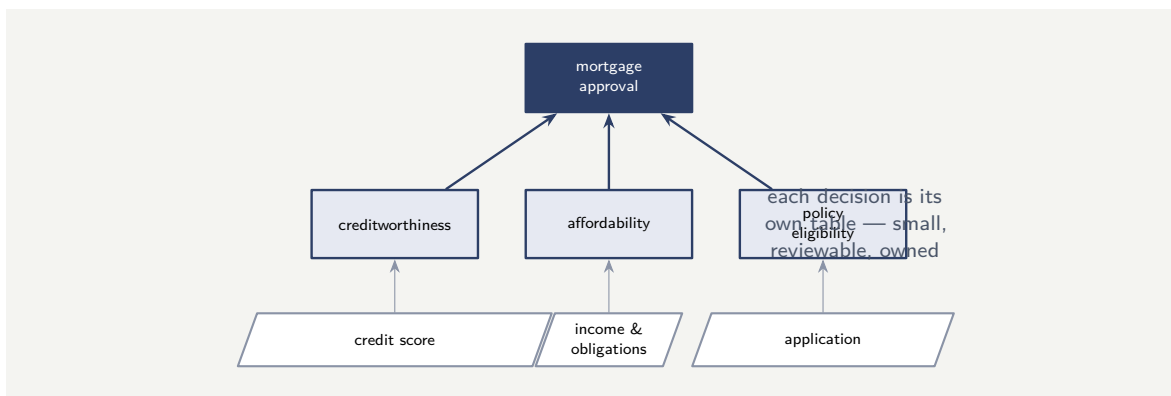


Figure 7.1. A Decision Requirements Diagram. The top-level mortgage-approval decision requires the outcomes of sub-decisions — creditworthiness, affordability, policy eligibility — and each sub-decision requires input data. Each decision is its own modest table; the DRD shows how they compose.

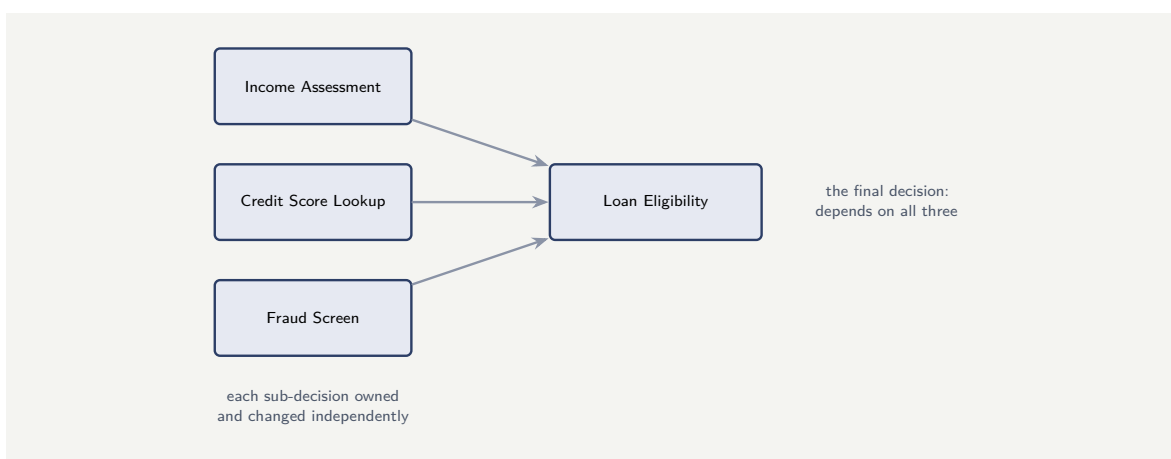


Figure 7.2. A Decision Requirements Diagram for a lending decision. Three sub-decisions — income, credit, fraud — feed the final eligibility decision. Each is a separate table, owned independently, changed without touching the others.

are not actually accountable for. The DRD lets ownership follow the structure: each sub-decision owned by the right function, the top-level decision composing their outcomes.

This is also how a decision estate stays *governable as it grows*. New policy is a new or changed sub-decision, slotted into the DRD, owned by the right function, verified on its own. The DRD keeps a large decision estate as a structured, owned thing rather than a sprawl of tables no one can relate to one another.

7.4 A DRD in concrete form

The DRD is clearest with an example. A lending decision has three required sub-decisions: an income assessment, a credit-score lookup, and a fraud screen. The final eligibility decision depends on all three, and the DRD declares that dependency as a graph rather than burying it in a forty-column table.

Figure 7.2 is that DRD. The three sub-decisions are separate tables; the final Loan Eligibility table consumes their outputs as its own inputs. If the bank's credit-scoring model changes, the Credit Score Lookup table is updated — and only it, without the Income Assessment or

the Fraud Screen needing to change at all, and without anyone touching the process model.

7.5 The DMN discipline in a regulated institution

In a regulated institution the value of DMN is not convenience but *auditability*. A decision expressed as a DMN table can be shown to a regulator, validated by a business analyst, versioned like any other model, and tested mechanically before deployment. A decision buried in code can do none of these. The discipline this chapter demands is that every business decision whose outcome a regulator might question — a credit decision, a pricing tier, a risk classification — be expressed in DMN and managed as a first-class artefact, owned by the business, not hidden inside a developer's code. DMN is not a modelling nicety; it is the institution's ability to show, prove, and change its decisions under control.

Worked example: the forty-column table no one could own

Problem. A bank has implemented its entire mortgage-approval decision as a single DMN decision table. The table has grown to 41 input columns — credit factors, income factors, property factors, policy flags — and several hundred rows. Three things are now true: the credit-risk team wants to change how credit factors are weighted but is afraid to touch the table; a regulator has asked who owns the affordability logic and the bank cannot give a clean answer; and the table's last completeness check timed out. Assess the problem and what a DRD would change.

Setup. We consider the single-table design against the three difficulties, and contrast with the same decision composed as a DRD.

Working. *The single table.* All 41 columns of logic — credit, income, property, policy — are in one artefact. The credit-risk team's change touches columns interleaved with everyone else's logic, so they cannot change their part without risking everyone's:

no isolation — one change risks the whole table.

The regulator's ownership question has no clean answer because the table is one artefact and one artefact has one owner, but the affordability logic is genuinely the responsible-lending function's:

ownership cannot follow the policy — the artefact is monolithic.

And completeness checking a table with 41 input columns is checking an input space of enormous dimensionality:

the verification does not complete.

The same decision as a DRD. Composed as a DRD — a creditworthiness sub-decision, an affordability sub-decision, a property sub-decision, an eligibility sub-decision, each its own modest table feeding a top-level approval decision — the credit-risk team changes only the creditworthiness table; the affordability table is cleanly owned by the responsible-lending function; and each modest table is checkable on its own.

Discussion. The single 41-column table is the decision-modelling equivalent of [Chapter 3](#)'s wall-of-boxes process diagram: it is not wrong in its logic, but it is structured so badly that it cannot be changed safely, owned correctly, or verified at all — and the worked example shows all three failures flowing from the one structural mistake of building a composite decision as a single table. A complex financial decision is *inherently* a structure of sub-decisions

owned by different functions, and a single table flattens that structure into one artefact, which then cannot isolate a change, cannot carry multiple owners, and cannot be verified because its input space is too large. The DRD restores the structure that was always really there: it composes the mortgage decision from sub-decisions, each corresponding to one coherent body of policy, each a modest table that one function owns, that one function can change without disturbing the others, and that tooling can completeness-check on its own. The lesson is the chapter's central discipline: a decision, like a process, must be *decomposed* — and the DRD is the instrument of decomposition. A bank that builds big decisions as big tables will, exactly like the bank in this example, arrive at a decision it is afraid to change, cannot assign ownership for, and cannot verify — and the remedy is not a bigger table or a faster checker, it is the structural decomposition the DRD provides.

Practical next steps

For the largest decision table in your institution, count its input columns. If it has grown beyond what one function can own and one reviewer can hold in mind, it is not really one decision — it is a composite that should be a DRD of sub-decisions. Decompose it: each sub-decision a modest table, owned by the function accountable for that policy, verifiable on its own.

Chapter in brief

A real financial decision is rarely one table — it is a structure of decisions, and DMN expresses that structure with the Decision Requirements Diagram, the DRD. The DRD shows which decisions require which sub-decisions and which input data, composing a complex decision from modest, governable sub-decisions — the decomposition discipline of [Chapter 3](#) applied to decisions. The DRD also lets ownership follow the structure: each sub-decision owned by the function genuinely accountable for that policy, where a single monolithic table forces one owner onto policy crossing many functions. A composite decision built as one giant table cannot be safely changed, correctly owned, or verified — the DRD restores the structure that was always really there.

CHAPTER 8

DMN and BPMN: the Decision in the Process

8.1 Two models, one platform

The manuscript has treated BPMN — the process — and DMN — the decision — as the two modelling notations a hyperautomation platform rests on. This chapter treats how they *join*: how a decision modelled in DMN is invoked from a process modelled in BPMN, and the disciplines that keep the join clean.

The relationship is one of *separation with a defined connection*. The process and the decision are separate models, separately owned, separately versioned, separately verified — that separation is the whole point of [Chapter 4](#)’s argument. But they must also connect, because a process step that needs a decision must be able to get one. The connection is the *business rule task*.

8.2 The business rule task

In BPMN, the step that invokes a DMN decision is the *business rule task* — a task type, drawn distinctively, whose job is precisely to call out to a decision and bring back its outcome. When the process reaches a business rule task, the engine evaluates the referenced DMN decision against the data the task supplies, and the decision’s outcome becomes available to the process for its subsequent steps — typically feeding the gateway that routes on the decision’s result.

This is the clean join. The process does not *contain* the decision logic; it contains a business rule task that *invokes* the decision. The decision lives in its own DMN model. The process model stays readable as a process — its business rule task says “here a decision is made,” not how — and the decision model stays readable as a decision. A reviewer of the process sees where decisions happen; a reviewer of the decision sees the decision logic; neither is forced to wade through the other.

Tip

The clean join between process and decision is the *business rule task*: a BPMN task type that invokes a DMN decision and brings back its outcome. The process must *invoke* the decision through this task, never *contain* the decision logic itself — decision logic written into a process’s gateways and scripts is exactly the buried, ungoverned decision DMN exists to prevent. The business rule task keeps the process readable as a process and the decision readable as a decision.

8.3 What data crosses, and the contract

The business rule task raises the same boundary question a call activity did: what data crosses between the process and the decision. The discipline is the same and worth stating precisely.

A DMN decision has a defined set of *inputs* — the input data of its DRD — and a defined *outcome*. That defined set is the decision’s *contract*: it is exactly what the decision needs to be given, and exactly what it will return. The business rule task’s job is to honour that contract — to map the process’s data into the decision’s declared inputs, and to receive the decision’s outcome back into the process’s variables.

This contract is what makes the process and the decision *independently changeable*, which is the practical payoff of the whole separation. As long as the decision’s contract holds — the same inputs in, the same shape of outcome out — the decision’s internal logic can be changed freely: the credit-risk function can re-weight its creditworthiness table, deploy a new version, and the process does not change, because the process only ever depended on the contract, not on the logic behind it. Equally, the process can be restructured without touching the decision. The business rule task plus the decision’s defined contract is what lets the process and the decision — as [Chapter 4](#) promised — evolve on their own cycles, owned by their own functions.

8.4 When the decision lives outside: external rules engines

Not every decision in the process can be expressed as a DMN table. Some decisions — anti-money-laundering screening, complex insurance underwriting, credit-policy evaluation with hundreds of interdependent rules — live in *external rules engines*: Drools, IBM ODM, FICO Blaze Advisor, or another. The process model must integrate with them cleanly.

The BPMN mechanism is the same business rule task the chapter has described, but instead of invoking the built-in DMN engine, it invokes a *job worker* that calls the external engine’s decision-service API. From the process model’s perspective, the task is identical: it sends input variables and receives a decision result. Whether the rules are evaluated by Camunda’s DMN engine or by a remote Drools KIE Server is invisible to the model.

[Listing 8.1](#) shows a business rule task in BPMN bound, through its job type, to an external rules engine.

Listing 8.1. A business rule task bound to an external rules engine via a job type. The model is identical whether the rules live in DMN, Drools, or ODM.

```
1 <bpmn:serviceTask id="evaluatePolicy"
2   name="Evaluate Credit Policy">
3   <bpmn:extensionElements>
4     <zeebe:taskDefinition
5       type="evaluate-credit-policy"/>
6   </bpmn:extensionElements>
7 </bpmn:serviceTask>
```

The BPMN model does not name the engine — it names a *job type*, and the worker behind that type calls whichever engine holds the rules. This separation lets the institution change engines by replacing a worker, not by rewriting the process. [Chapter 22](#) treats the external rules engines — Drools, IBM ODM, FICO Blaze Advisor — and their integration code in depth.

8.5 The DMN discipline in a regulated institution

In a regulated institution the value of DMN is not convenience but *auditability*. A decision expressed as a DMN table can be shown to a regulator, validated by a business analyst, versioned like any other model, and tested mechanically before deployment. A decision buried

in code can do none of these. The discipline this chapter demands is that every business decision whose outcome a regulator might question — a credit decision, a pricing tier, a risk classification — be expressed in DMN and managed as a first-class artefact, owned by the business, not hidden inside a developer's code. DMN is not a modelling nicety; it is the institution's ability to show, prove, and change its decisions under control.

Worked example: the decision change that did not touch the process

Problem. A bank's loan-origination process, modelled in BPMN, has a business rule task that invokes a DMN creditworthiness decision. The decision takes defined inputs — credit score, recent defaults, existing exposure — and returns a defined outcome: a creditworthiness tier of A, B, C, or D. The credit-risk function needs to substantially change how the tier is calculated: new weightings, a new sub-factor, a re-tuned set of thresholds. The process runs 200,000 loan applications a year and is operated by a different team. Assess what the credit-risk function's change requires, given the business rule task and the decision's contract.

Setup. We consider what the credit-risk change touches, given that the process invokes the decision through a business rule task honouring a defined contract.

Working. The credit-risk function's change is entirely *inside* the creditworthiness DMN decision: new weightings, a new sub-factor, re-tuned thresholds — all of it is the decision's internal logic. The decision's *contract* is unchanged: it still takes credit score, recent defaults, and existing exposure, and it still returns a tier of A, B, C, or D. Because the process depends only on the contract — it supplies those inputs through the business rule task and routes on the A/B/C/D outcome — the process does not need to change:

decision logic: changed; decision contract: unchanged; process: untouched.

The credit-risk function changes and re-deploys the DMN decision; the loan process, and the team that operates it, are not involved.

Discussion. This worked example is the positive counterpart to the buried-decision warnings of [Chapter 4](#) — it shows the separation *paying off*. The credit-risk function makes a substantial policy change — the kind that, if the creditworthiness logic had been written into the loan process's gateways and scripts, would have meant editing, re-reviewing, re-testing, and re-deploying the entire loan-origination process, involving the process team, with all the risk of disturbing a process that runs 200,000 applications a year. Instead, because the decision lives in its own DMN model and the process invokes it through a business rule task honouring a defined contract, the change is contained entirely within the decision. The contract — these inputs, an A/B/C/D outcome — is the interface, and as long as the change respects the interface, the process is genuinely none the wiser. The credit-risk function owns the creditworthiness decision, changes it on its own cycle, verifies it with DMN tooling on its own, and re-deploys it — and the loan process continues unchanged. This is the entire practical purpose of modelling decisions separately and joining them through the business rule task: not architectural tidiness for its own sake, but the concrete ability of the function that owns a piece of policy to change that policy without co-ordinating a change to every process that consumes it. The separation that [Chapter 4](#) argued for and this part has built out is, in the end, what makes a bank's decision policy something it can actually change.

Practical next steps

For one process in your institution that makes a significant decision, check how the decision is invoked. If it is a business rule task calling a DMN decision with a defined contract, the decision can be changed without touching the process — confirm that is so. If the decision logic is instead written into the process's gateways and scripts, every policy change is a process change, and the decision should be extracted into its own DMN model behind a business rule task.

Chapter in brief

BPMN and DMN are separate models — separately owned, versioned, and verified — but they must connect, and the clean connection is the *business rule task*: a BPMN task type that invokes a DMN decision and brings back its outcome. The process invokes the decision through this task; it never contains the decision logic. What crosses the join is governed by the decision's *contract* — its defined inputs and defined outcome — and the business rule task maps the process's data to honour that contract. The contract is what makes process and decision independently changeable: as long as it holds, the function that owns a decision can change its logic freely, without touching any process that invokes it — which is the entire practical payoff of modelling decisions separately.

PART II

Workflow Engines and Process Architecture

Workflow Engines and Process Orchestration

9.1 What a workflow engine is

The foundations have established what a process is and how BPMN and DMN describe it. This part turns to the thing that *runs* a process — the *workflow engine* — and to the architectural choices that surround it, because how an institution orchestrates its processes is a decision made long before any individual process is modelled.

A workflow engine is a piece of software with one defining job: it takes a process definition — a BPMN model — and *executes* it. It creates a process *instance* when work begins, tracks where that instance is, calls out to do the work each step requires, waits when a step waits, routes the flow through gateways, and carries the instance to completion. It is, in the most literal sense, the runtime for a process model.

What makes an engine more than a script runner is *state*. A workflow engine *persists* the state of every running instance: which activity is active, what data the instance carries, which tasks are outstanding. Because the state is persisted, an instance survives a restart, a crash, a deployment; a process that takes three weeks is not lost because a server was patched in week two. This durability is the engine's first essential property, and it is why a long-running, regulated business process needs an engine rather than a program.

9.2 Orchestration: the engine as the single point of truth

The discipline a workflow engine brings is *orchestration*. An orchestrated process has one explicit model — held and executed by the engine — that directs the work: it calls service A, then, on the result, service B or service C, then waits for a human task, then continues. The sequence, the conditions, the waiting are all *in the model*, and the engine is the single component that knows the state of the whole.

The alternative is the absence of orchestration: logic for “what happens next” scattered across the services themselves, each service knowing a little about what should follow it. That arrangement works until it must be *changed* or *understood* — and then no one component holds the process, no diagram describes it, and the true sequence can only be reconstructed by reading every service. Orchestration's value is that the process becomes a *thing* — visible, governable, changeable in one place.

For a bank or an insurer this is not a convenience but a regulatory necessity. A regulated process must be *auditable*: the institution must be able to show what the process does, demonstrate that it did it, and change it under control. An orchestrating workflow engine makes the process an explicit, inspectable, audited artefact; scattered logic makes it a reconstruction.

Figure 9.1 shows the engine as the single point of truth for a process.

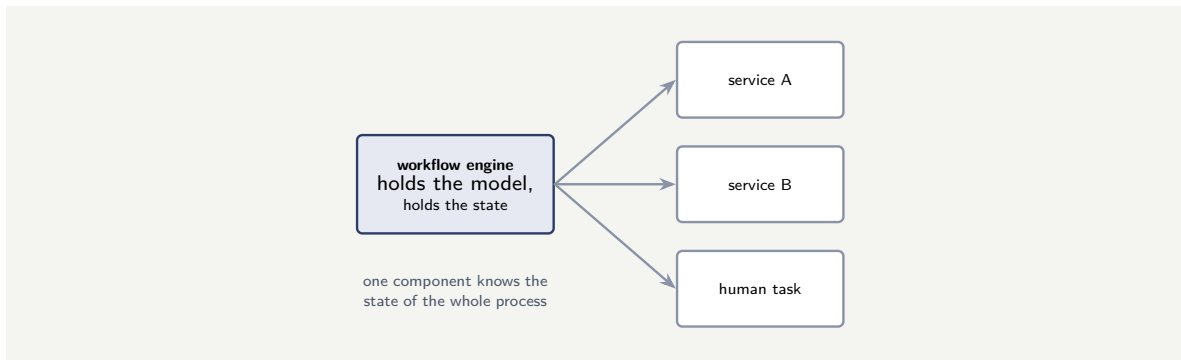


Figure 9.1. The workflow engine as orchestrator. The engine holds the process model and the persisted state of every instance, and directs the services and human tasks that do the work — so one component, not a scatter of services, knows the state of the whole.

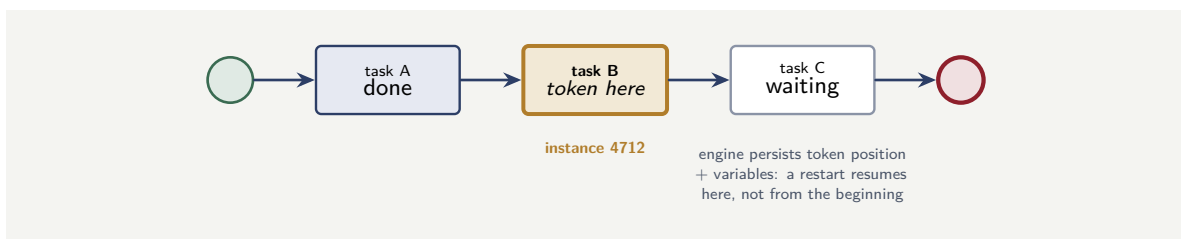


Figure 9.2. A process instance as a token position. Instance 4712 has completed Task A and is currently at Task B; the engine persists this position so a restart resumes at Task B rather than the start.

9.3 The engine's state model: tokens and instances

A workflow engine's persistence rests on a simple model: a *process instance* has one or more *tokens*, each marking where in the process the instance currently is. When a token reaches an activity, the engine executes that activity; when it completes, the token moves to the next element. A parallel gateway creates multiple tokens; a join consumes them. The engine stores the current token positions — and the instance's accumulated variables — in its data store, so every restart knows exactly where each of the thousands of concurrent instances is.

Figure 143.1 shows one instance mid-execution: its token at Task B, Task A completed, Task C not yet reached. This persisted token position is what distinguishes a workflow engine from a script — a script re-runs from the beginning on every restart; an engine resumes from the last persisted token position, with its variables intact.

9.4 The engine as the institution's process memory

A workflow engine's state store is, in a real sense, the institution's *process memory*: it knows, for every case in progress, where it is, what has been done, what is outstanding, and what the data says. This memory survives restarts, deployments, and failures — it is durable, queryable, and auditable. An institution without a workflow engine has process memory only in the heads of the people doing the work, in spreadsheets, and in email threads — none of which survives a staff change, none of which a regulator can query.

Worked example: the process no one could find

Problem. A bank's loan-origination "process" is implemented without a workflow engine: eight microservices, each calling the next directly, the sequence encoded in the services' own code. A regulator asks the bank to produce the loan-origination process — to show, as a single artefact, what it does and prove it is followed. The bank cannot. Assess why.

Setup. We ask where the process actually *is* in this implementation.

Working. There is no workflow engine, so there is no component that holds the process. The sequence is distributed: service 1 knows it calls service 2; service 2 knows it calls service 3 or service 5; and so on. The process as a whole exists only as an *emergent* property of eight services' individual code:

process = the sum of eight services' "what I call next",

and nowhere is it written down, modelled, or held. To answer the regulator the bank must reverse-engineer the process by reading all eight services and tracing the calls — and even then it has a reconstruction, not an authoritative artefact.

Discussion. The bank did not lose its process; it never had its process as a *thing*. This is the cost of building without orchestration, and the worked example shows it is not a modelling nicety but a regulatory exposure. An orchestrating workflow engine would hold the loan-origination model as one explicit, versioned, inspectable artefact: the regulator's request would be answered by opening it. The engine would also hold the execution record — which instances ran, what path each took — so the bank could prove the process was followed, not merely described. Scattered choreography gave the bank none of this: the process is real but invisible, running but unauditably. The lesson is the chapter's central one: a workflow engine's value is not that it runs steps — code can run steps — but that it makes the process an explicit, stateful, governable artefact, which a regulated institution does not merely benefit from but is required to have.

Practical next steps

For any process that matters to a regulator, ask whether an explicit model of it exists and is executed by an engine, or whether the process is only an emergent property of the services that implement it. If the latter, the institution cannot readily show or prove its own process — and an orchestrating workflow engine is what converts a scatter of services into a governable process.

Chapter in brief

A *workflow engine* is the runtime for a process: it executes a BPMN model, creates and tracks instances, and — crucially — *persists* their state, so a long-running process survives restarts and crashes. Its defining discipline is *orchestration*: one explicit model, held and executed by the engine, directs the work, so a single component knows the state of the whole process. The alternative — "what happens next" logic scattered across the services — leaves the process as an emergent property no component holds, visible only by reading every service. For a regulated institution orchestration is a necessity, not a convenience: it makes the process an explicit, inspectable, audited artefact the institution can show, prove, and change under control.

Camunda 7 versus Camunda 8 Architecture

10.1 Two engines, one notation

An institution choosing Camunda is choosing between two architecturally distinct platforms that share a name and a notation but little else underneath. Understanding the difference is essential, because it shapes deployment, scaling, the programming model, and the operational skills a team needs.

Both *Camunda 7* and *Camunda 8* execute BPMN 2.0, and a process modelled for one looks much like a process modelled for the other. But the engine beneath is different. Camunda 7's engine descends from the Activiti project — the open-source BPMN engine Camunda originally forked — and is a mature, relational-database-backed Java engine. Camunda 8's engine is *Zeebe*, a new engine Camunda built from scratch, designed for cloud-native, horizontally-scaled, distributed operation. They are not two versions of one product in the ordinary sense; they are two engines.

10.2 Embedded versus remote: the defining difference

The single most consequential architectural difference is how the engine relates to the application.

Camunda 7 can run as an *embedded* engine. The engine is a Java library placed *inside* the application: it runs in the same JVM, shares the application's thread pool, and — the decisive consequence — can share the application's *database and transaction manager*. Because of that shared transaction, a unit of application work and the corresponding engine state change can commit, or roll back, *together*, in one ACID transaction. Camunda 7 can also run as a standalone server reached over REST, but the embedded mode is the one that defines its character.

Camunda 8 has no embedded mode. Its engine, *Zeebe*, is *always a remote resource*: the application talks to it across the network, through the Zeebe Gateway, over gRPC. The decisive consequence is the mirror image of Camunda 7's: the application and the engine *cannot share an ACID transaction*, because they are different systems on different sides of a network boundary. Camunda 8 instead embraces *eventual consistency* — the application's state and the engine's state converge, but not in one atomic commit — and the programming model is built around that.

This one difference — embedded-and-transactional versus remote-and-eventually-consistent — propagates into everything else.

Figure 10.1 contrasts the two architectures.

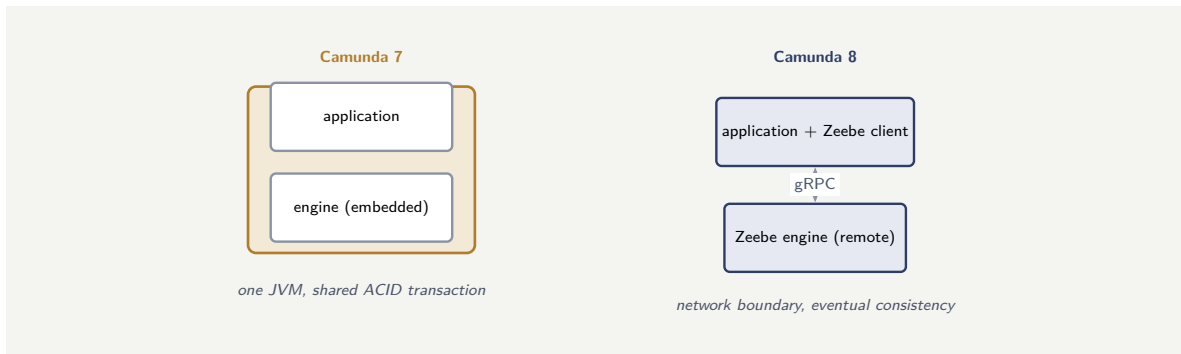


Figure 10.1. The defining architectural difference. Camunda 7 can embed the engine in the application — one JVM, one shared ACID transaction. Camunda 8’s Zeebe engine is always remote, reached over gRPC across a network boundary, so the application and engine cannot share a transaction and the model is eventually consistent.

10.3 What follows from it

Four further differences follow from the embedded-versus-remote split, and an architect should know them as a set.

Data store and scaling. Camunda 7 persists state in a *relational database*, and a Camunda 7 cluster is several engine nodes sharing one database — which makes the database the scaling bottleneck and the single point of failure. Camunda 8’s Zeebe does not use a relational database at all: it is event-sourced, its brokers hold state and replicate it among themselves, and a stream of events is exported to *Elasticsearch* for querying. A Zeebe cluster scales *horizontally* by adding brokers, with no shared database to bottleneck — the default is at least three brokers for resilience.

Communication. Camunda 7 is reached by REST, or by direct Java calls when embedded. Camunda 8 is reached through the Zeebe Gateway over *gRPC*, with client libraries for the common languages.

Deployment. Camunda 7 suits a traditional or embedded deployment — a Java application, an application server, a monolith. Camunda 8 is *cloud-native*: it is designed to run on Kubernetes, as a distributed system of brokers, gateway, and the web components.

The programming model. Camunda 7’s embedded mode invites *Java delegates* — glue code the engine calls in-process. Camunda 8’s remote model makes the *job worker* — the external-task pattern — the normal way to do work: workers poll the engine for jobs and execute them in their own process. Notably, the external-task pattern *exists in Camunda 7 too*, and a Camunda 7 solution built around it is the one that maps most directly onto Camunda 8.

Tip

Choosing between Camunda 7 and Camunda 8 is an architectural decision, not a “newer is better” one. Camunda 8’s Zeebe gives cloud-native horizontal scaling and resilience with no relational-database bottleneck — the right choice for high volume and a Kubernetes-first strategy. Camunda 7 remains a sound choice where the application genuinely needs to *share an ACID transaction* with the engine, where the scale comfortably fits a relational database, and where a team is invested in an embedded Java deployment. Decide on scale, consistency needs, and deployment strategy — not on the version number.

10.4 Choosing between the two: a decision framework

The choice between Camunda 7 and Camunda 8 should be made on three axes, not on the version number. The first axis is *transactional need*: does any process step genuinely require a shared ACID transaction with the engine? If yes, Camunda 7's embedded mode is the only architecture that provides it. The second is *scale*: will the platform serve volumes that a single relational database cannot sustain? If yes, Camunda 8's horizontally-scaled Zeebe cluster is the architecture that removes that bottleneck. The third is *deployment*: is the institution committed to Kubernetes and cloud-native operations? Camunda 8 is designed for that world; Camunda 7 is not. A team that decides on all three axes makes an architectural choice; a team that picks the higher number has made no choice at all.

Worked example: the transaction that could not span the boundary

Problem. A team migrating a process from Camunda 7 to Camunda 8 has a service task whose Java delegate, in the Camunda 7 embedded engine, does two things in *one transaction*: it writes a row to the application's database and it completes the engine's service task. They expect to move the delegate's code to Camunda 8 unchanged. Assess this.

Setup. We test the expectation against the embedded-versus-remote difference.

Working. In Camunda 7 embedded, the delegate and the engine share a JVM, a data-source, and a transaction manager, so "write the row" and "complete the task" commit together — atomically — or roll back together. In Camunda 8, the engine is remote: the job worker that replaces the delegate runs in its own process, and completing the job is a gRPC call across a network boundary:

Camunda 7: one ACID transaction → Camunda 8: two systems, no shared transaction.

The code cannot move unchanged, because the property it relied on — one transaction spanning application and engine — does not exist in Camunda 8.

Discussion. The expectation failed on the platform's defining difference, and the worked example shows why that difference is not a detail but a redesign. In Camunda 8 the worker must handle the two state changes without a shared transaction: it writes its row in its own transaction, then completes the job; and it must be built so that if it crashes between the two, the system still converges correctly — which is what the at-least-once job delivery and idempotent-worker discipline of the components part exist for. The worker becomes *eventually consistent* with the engine rather than atomically consistent. This is not a flaw in Camunda 8; it is the deliberate trade of cloud-native distributed architecture, where a network boundary buys horizontal scaling at the cost of the shared transaction. The lesson is the one the tip box states: the choice and the migration between Camunda 7 and 8 turn on this transactional boundary, and a team that treats the two as interchangeable versions will meet the difference, unplanned, in exactly the code that relied on the embedded engine's shared transaction.

Practical next steps

When choosing between Camunda 7 and 8, or migrating between them, find the code that relies on a shared ACID transaction with the embedded engine. In Camunda 8 that code must be redesigned around a remote engine and eventual consistency — idempotent workers, no assumed atomic commit across the boundary. The transactional boundary is the difference

that governs the rest.

Chapter in brief

Camunda 7 and Camunda 8 share BPMN 2.0 but are architecturally distinct platforms. Camunda 7's engine descends from Activiti, is relational-database-backed, and can run *embedded* — in the application's JVM, sharing its thread pool and, decisively, its transaction manager, so application work and engine state commit in one ACID transaction. Camunda 8's engine, *Zeebe*, has no embedded mode: it is always *remote*, reached over gRPC through the Zeebe Gateway, so application and engine cannot share a transaction and the model is eventually consistent. From that one difference follow the rest: Zeebe is event-sourced with no relational bottleneck and scales horizontally across brokers; it is cloud-native on Kubernetes; and its remote model makes the job worker, not the embedded Java delegate, the normal way to do work. Choose on scale, consistency, and deployment — not the version number.

Human Workflows versus Straight-Through Processing

11.1 Two kinds of process

A workflow engine runs processes, but processes are not all of one kind. A fundamental architectural distinction — one that shapes how a process is designed, staffed, and measured — divides them into *human workflows* and *straight-through processing*.

A *human workflow* is a process in which people do essential work. It has user tasks: a step where an analyst assesses, an underwriter decides, an officer checks. The process advances at the pace of human work, it has worklists and allocation, and its design must account for people — their availability, their skills, the four-eyes controls. Much of banking and insurance is human workflow: a complex claim, a commercial loan, a complaint, a fraud investigation.

Straight-through processing — STP — is a process that runs end to end *without human intervention*. Every step is automated: the process receives its trigger, makes its decisions through rules and services, and completes, with no person in the path. A payment that clears automatically, an instant account decision, an automatic policy renewal — these are STP. The process runs at machine speed, its throughput is limited by systems not staff, and its design goal is to need no one.

The distinction is not a label applied after the fact; it is an architectural decision made early, because the two kinds of process are built, measured, and staffed differently.

Figure 11.1 contrasts the two shapes.

11.2 Why the distinction is architectural

The two kinds of process differ in what their design must optimise, and an architect who blurs them optimises for the wrong thing.

A human workflow's performance is governed by *people*: its throughput is the team's capacity, its delays are queue waits, its design centres on *allocation* — getting the right task to the right person — and on the worklist, the form, the escalation timer. Its key measures are human: utilisation, time-in-queue, tasks per analyst. The platform features that matter are Tasklist, candidate groups, the allocator.

An STP process's performance is governed by *systems*: its throughput is the rate the services and rules can sustain, its delays are system latencies, its design centres on the *decisioning* — the rules and models that replace human judgement — and on resilience under volume. Its key measures are technical: instances per second, automation rate, the proportion that complete without falling out to a human.

That last measure points to the most important practical truth: the two are not a strict binary but a *spectrum*, and the design question is the *automation rate*. Even an STP-intended process usually has an exception path — the case the rules cannot decide — that *falls out* to a human. A well-designed STP process maximises the share that flows straight through and handles

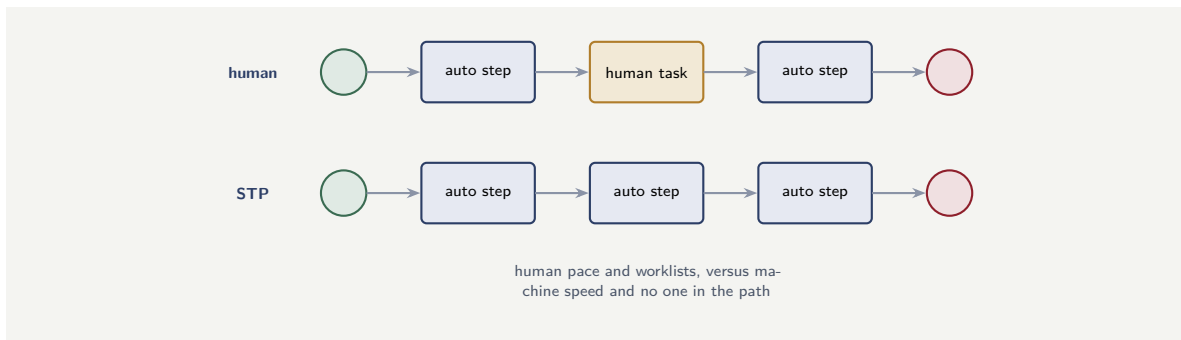


Figure 11.1. Human workflow versus straight-through processing. A human workflow has at least one user task and advances at human pace; STP is automated end to end and runs at machine speed. The distinction shapes design, staffing, and the measures that matter.

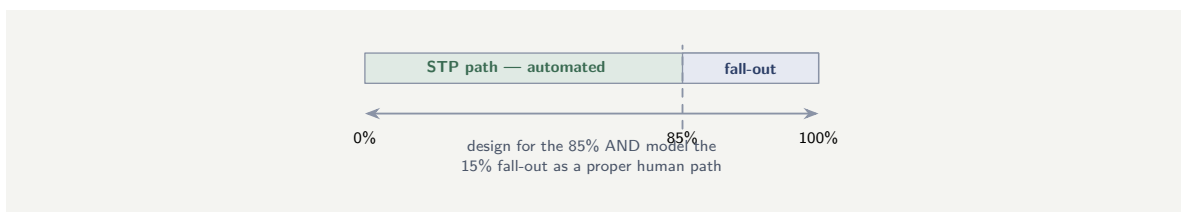


Figure 11.2. The automation-rate spectrum. An STP process running at 85% automation is not a failure; but the 15% fall-out must be modelled as a real human path — not abandoned, not handled by a manual workaround outside the process.

the fall-out cleanly as a small human workflow alongside. Designing this honestly means deciding, deliberately, which decisions are automated and what happens to the residue — not assuming an STP process is one that simply never involves a person.

11.3 Measuring the spectrum: automation rate

The spectrum from human workflow to STP is measured by the *automation rate* — the percentage of cases that complete without any human intervention. An STP-intended process with an automation rate of 100% is rare; most real processes have a residue of exceptions requiring human judgement. Understanding the actual rate — and planning for the fall-out — is the architectural act this chapter argues for.

Figure 11.2 places a typical STP process on the spectrum. The architecture question is not whether to aim for 100% — it is to know honestly what the actual rate is, and to model the fall-out as a first-class part of the process rather than leaving it to improvisation.

11.4 Measuring STP honestly

The automation rate is the measure, and it must be measured *honestly* — against the total population of cases, not against a curated sample. An STP rate of 85% measured against all cases is genuine; a rate of 95% measured after excluding “exceptions” is an exercise in flattery. The honest measure includes every case, counts every fall-out, and forces the institution to face the residue — which is where the real operational cost sits, because the 15% that fall out to human handling are, by definition, the hard cases, and they take disproportionately more effort per case than the 85% that flow through.

Worked example: the STP process designed with no fall-out path

Problem. A bank builds an instant-decision consumer-loan process as pure straight-through processing: rules decide every application, no human tasks, no worklist. In production, 12% of applications hit a condition the rules were not built to decide — unusual income, a data mismatch, a borderline score. The process has nowhere to send them. Assess the design.

Setup. We consider what happens to the 12% under a design with no human path.

Working. The process was designed as if *automation rate would be 100%* — every application decided by rules. The reality is an automation rate of 88%, with 12% requiring judgement the rules do not encode. A pure-STP design has no user task, no worklist, no allocation — so the 12% cannot be routed to a person:

88% flow through + 12% with nowhere to go.

Those applications either fail, or are forced through a rule that should not decide them, or are handled by an unmodelled manual workaround outside the process entirely — and the bank has lost sight of them.

Discussion. The team treated “straight-through processing” as an absolute — a process with no humans, ever — rather than as a point on the human-to-STP spectrum, and the worked example shows the cost. Almost no real STP process has a 100% automation rate; there is nearly always a residue the rules cannot decide, and the architectural question is not *whether* there is fall-out but *what happens to it*. The sound design is an STP process that maximises the straight-through share and routes the exception — the 12% — onto a proper, modelled human path: a user task on a small worklist, worked by people equipped for the hard cases. That is not a failure of the STP ambition; it is the honest form of it. By designing as pure STP with no fall-out path, the bank built a process that handles 88% well and 12% not at all — and the 12%, being invisible to the process, became an unmanaged manual workaround, the very thing a process platform exists to eliminate. The lesson is that human-versus-STP is a spectrum and a design decision: decide the automation rate deliberately, and always model what happens to the residue.

Practical next steps

When designing a straight-through process, do not assume a 100% automation rate. Establish, from data or honest estimate, the share that will fall out to human judgement, and model that exception path explicitly as a small human workflow — a user task, a worklist, allocation. An STP process with no fall-out path does not eliminate the hard cases; it loses them.

Chapter in brief

Processes divide into *human workflows* — where people do essential work, the process advances at human pace, and design centres on allocation, worklists, and escalation — and *straight-through processing* (STP), where the process runs end to end with no human in the path, at machine speed, its design centred on decisioning and resilience. The distinction is architectural: the two are built, measured, and staffed differently, a human workflow by utilisation and time-in-queue, an STP process by throughput and automation rate. Crucially the two are a *spectrum*, not a binary: almost every STP process has a residue the rules cannot decide, and the honest design maximises the

straight-through share while routing the fall-out onto a proper, modelled human path.

Microservices Orchestration with Camunda

12.1 The orchestration problem in a microservices estate

A modern bank or insurer does not run a monolith; it runs a landscape of *microservices* — many small, independently-deployed services, each owning a narrow capability. This architecture has clear benefits, but it creates a specific problem that a workflow engine is built to solve: the problem of *the business process that spans many services*.

A business process — opening an account, originating a loan, settling a claim — is rarely the work of one microservice. It is a *sequence* across many: validate the customer, check credit, screen for fraud, create the account, fund it, notify. Each step is a different service. The question every microservices estate must answer is: *what holds that sequence?* What knows that fraud screening follows credit checking, that funding waits for account creation, that a failure at step five must compensate steps one through four?

There are two answers, and the choice between them is the central architectural decision of this chapter: *orchestration* and *choreography*.

12.2 Orchestration versus choreography

In *choreography*, there is no central coordinator. Each service reacts to events and emits its own: the credit service, seeing a “customer validated” event, does its work and emits “credit checked”; the fraud service reacts to that. The process is *emergent* — it is the sum of the services’ individual reactions, and no component holds it.

In *orchestration*, one component — an *orchestrator* — holds the process as an explicit model and directs the services: it calls the credit service, and on the result calls the fraud service, and so on. The process is *explicit* — modelled, visible, held in one place.

Camunda is an *orchestrator*, and using it to orchestrate microservices is one of its primary roles in a modern estate. The Camunda process is the explicit model of the business process; each service task in the model corresponds to a call to a microservice — through a connector or, more commonly, through a job worker that the service team owns. The BPMN model is the one place the cross-service sequence is written down.

The decisive advantage in a regulated institution is the same as in [Chapter 14](#): the cross-service business process becomes a *governable artefact*. With choreography, “what is our loan-origination process” has no answer but “read the services and infer it.” With Camunda orchestrating, the answer is the model. And the orchestrator is where the hard cross-service concerns — the timeout when a service does not respond, the *compensation* when a later step fails and earlier steps must be undone, the retry, the escalation — are expressed once, in the process, rather than scattered.

Figure [12.1](#) contrasts orchestrating and choreographing a microservices estate.

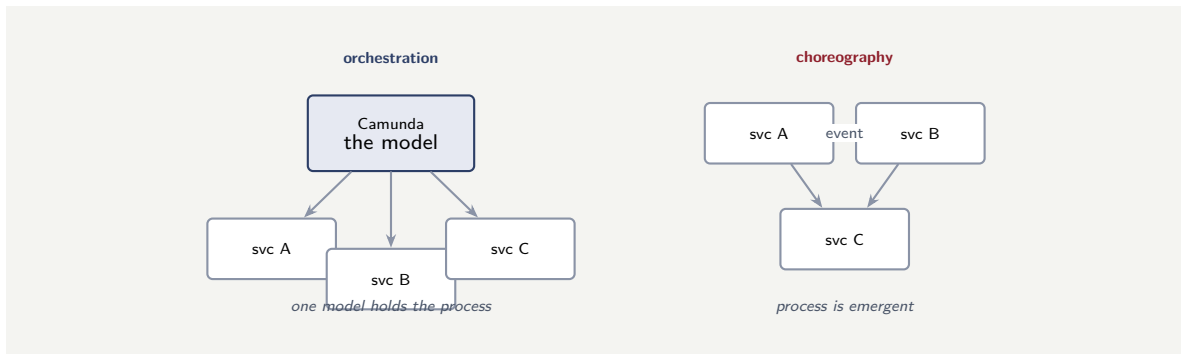


Figure 12.1. Orchestrating microservices. With orchestration, Camunda holds the cross-service process as one explicit model and directs the services. With choreography, services react to one another’s events and the process is emergent — held nowhere, governable by no one.

12.3 The discipline: orchestrate the process, respect the services

Orchestrating microservices well has one governing discipline: the orchestrator owns the *process*, not the services. Camunda directs the sequence; it does not reach inside a service, own its data, or know its internals. Each service stays autonomous — independently deployable, owning its own data — and the Camunda model calls it through a declared interface, exactly as the anti-corruption discipline of [Chapter 14](#) requires. The process becomes explicit; the services stay loosely coupled. An orchestrator that grows tendrils into the services’ internals has rebuilt the monolith with extra steps.

Tip

Orchestrating microservices with Camunda, keep one line firm: Camunda owns the *process* — the cross-service sequence, the timeouts, the compensation, the retries — and each microservice owns *itself* — its logic, its data, its deployment. The process model calls services through declared interfaces and never reaches inside them. Cross the line — let the orchestrator manipulate a service’s data directly — and the services stop being autonomous; the estate becomes a distributed monolith with a workflow engine bolted on.

12.4 Choreography versus orchestration in code

The distinction between choreography and orchestration becomes concrete when expressed in code. In choreography, a service publishes an event and another reacts. In orchestration, the orchestrator calls a service and knows the result. [Listing 12.1](#) shows the two forms for the same credit-check step.

Listing 12.1. Orchestration versus choreography for the same step. The orchestrator calls the service and knows the result; choreography has no one component that knows the whole.

```
1 // ORCHESTRATION: the process calls the credit service
2 // and knows the result -- one place owns the sequence
3 @JobWorker(type = "credit-check")
4 public Map<String, Object> check(ActivatedJob job) {
5     CreditResult r = creditService.check(
```

```

6     job.getVariable("customerId");
7     return Map.of("creditScore", r.score());
8 }
9
10 // CHOREOGRAPHY: the credit service reacts to an event
11 // -- no component knows what happens next
12 @KafkaListener(topics = "customer.validated")
13 public void onCustomerValidated(CustomerEvent event) {
14     // react and emit -- no knowledge of the sequence
15     CreditResult r = creditService.check(
16         event.customerId());
17     kafkaTemplate.send("credit.checked", r);
18 }

```

The orchestration worker knows it is step “credit-check” in a named process; the Camunda engine knows this worker is the third of seven steps. The choreography listener knows only that it reacts to one event and emits another — it has no knowledge of its position in a larger sequence.

12.5 The boundary between orchestration and autonomy

The governing discipline is that the orchestrator owns the *process* and each service owns *itself*. This boundary must be policed. An orchestrator that reads a service’s database directly has violated the service’s autonomy; a service that makes decisions about what happens next in the process has taken over the orchestrator’s job. The boundary is the declared interface — the job type, the input variables, the output variables — and everything on each side of it is private. Crossing it in either direction rebuilds the monolith, one violation at a time.

Worked example: the saga that choreography could not show

Problem. A bank’s account-opening spans five microservices, wired by choreography — each reacts to the previous one’s event. Step four, account-funding, sometimes fails, and when it does, steps one to three (customer record, credit reservation, account creation) must be *compensated* — undone. The compensation logic is spread across the services. A regulator asks the bank to show how a failed account-opening is unwound. Assess the position.

Setup. We ask where the compensation logic lives and whether it can be shown as one thing.

Working. Under choreography no component holds the process, and so no component holds the *compensation* either: each service has its own fragment of “if I hear the failure event, I undo my part.” The unwinding sequence — the *saga* — is, like the process itself, emergent:

compensation = five services’ individual “undo” fragments,

written down nowhere as a whole. To show the regulator how a failed account-opening unwinds, the bank must again read all five services and reconstruct the saga.

Discussion. This is the microservices orchestration problem at its sharpest. A cross-service process that can fail partway needs a *compensation* or *saga* — a defined unwinding of the steps already done — and compensation is precisely the kind of cross-service concern that must be held in one place to be correct and to be shown. Choreography scatters it: each service knows only its own undo, and whether the whole composes into a correct unwinding is an emergent property no one can point to. Camunda orchestrating the account-opening would hold the *saga in the model* — BPMN’s compensation events and boundary handlers

express, explicitly, that funding's failure triggers the undo of account creation, credit reservation, and customer record, in order. The regulator's question is then answered by the model, and — just as important — the bank can be confident the compensation is correct because it is designed as a whole, not assembled from fragments. The lesson sharpens the chapter's point: orchestration's value in a microservices estate is not only that it makes the happy path explicit, but that it gives the *failure* path — the saga, the compensation — a single, governable home, which choreography structurally cannot.

Practical next steps

In a microservices estate, decide orchestration versus choreography deliberately, and for any cross-service process that can fail partway, treat the compensation as decisive: orchestration holds the saga as one explicit model; choreography scatters it across services where its correctness cannot be shown. Let Camunda own the process and the compensation, and let each service stay autonomous behind a declared interface.

Chapter in brief

A microservices estate must answer one question: what holds a business process that spans many services? The two answers are *choreography* — services react to one another's events, and the process is emergent, held nowhere — and *orchestration* — one component holds the process as an explicit model and directs the services. Camunda is an orchestrator, and orchestrating microservices is one of its primary roles: the BPMN model is the one place the cross-service sequence, timeouts, retries, and — decisively — *compensation* are written down, so the process becomes a governable artefact. The governing discipline is that Camunda owns the *process* while each service stays autonomous behind a declared interface; an orchestrator that reaches into services' internals has rebuilt the monolith.

Event-Driven Process Automation

13.1 Processes that react

The orchestration of the previous chapter has the process *reach out* — the model calls a service and waits for the answer. But a great deal of what a bank or insurer must respond to does not work that way: it *arrives*. A payment lands, a customer changes an address, a market threshold is crossed, a fraud alert fires. The process does not call for these; they happen, and the process must *react*. This is *event-driven process automation*, and it is an architectural style a hyperautomation platform must support alongside orchestration.

An *event*, in this sense, is a record that something significant happened — a business fact, stated in the past tense: “payment received,” “address changed,” “loan disbursed,” “policy lapsed.” Event-driven automation is the arrangement in which processes are *triggered and advanced by these events* rather than only by direct calls.

13.2 The event backbone, and how processes use it

Event-driven automation rests on an *event backbone*: a durable, ordered stream onto which the institution’s systems publish their business events, and from which any interested consumer — including the hyperautomation platform — reads. The backbone *decouples* the publisher from the consumer: the system that emits “policy lapsed” does not know or care which processes react to it, and a new reacting process can be added without touching the publisher.

A process uses the backbone in three distinct ways, and naming them gives event-driven design its vocabulary.

An event can *start* a process: the arrival of “payment received” creates a new instance of a payment-handling process. In BPMN this is a message start event, and it is how a process begins by reacting to the world rather than by being called.

An event can *advance* a waiting process: an instance parked at an intermediate catch event resumes when its event arrives — a claims process waiting for “assessment uploaded” continues when that event lands. This is how a long-running process waits for the world without consuming resources while it waits.

An event can *interrupt* a process: an event caught on the boundary of an activity diverts the flow — a “fraud alert” event during account-opening breaks off the normal path and routes to investigation. This is how a process stays responsive to things that cannot wait for its normal sequence.

Figure 13.1 sets out the backbone and the three ways a process uses it.

13.3 The disciplines event-driven automation imposes

Event-driven automation is powerful but exacting, and three disciplines are not optional.

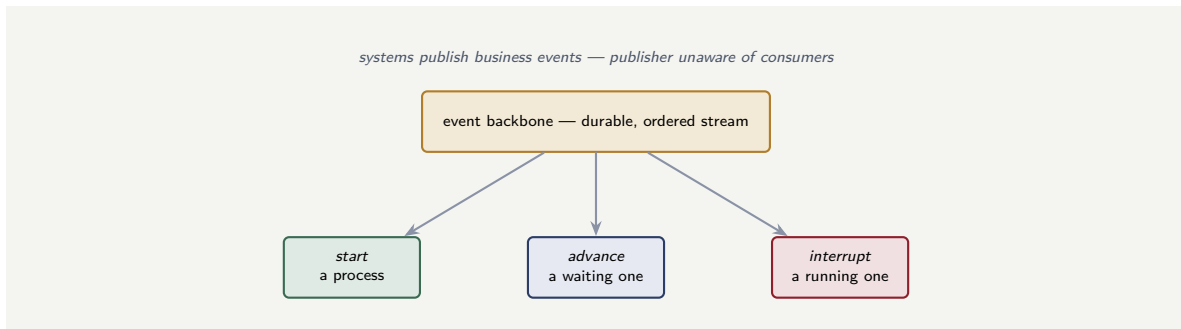


Figure 13.1. Event-driven process automation. Systems publish business events to a durable, ordered backbone; processes consume them in three ways — an event can start a process, advance a waiting one, or interrupt a running one.

Events must be *durable and ordered*: a consumer that is briefly down must resume from where it stopped without losing or reordering events, or a process will miss a fact it needed.

Events should be *business facts*, not technical noise: “loan disbursed” is a business event a process can meaningfully react to; “row inserted in table 17” is a technical change that should not be on the business backbone.

And event delivery is typically *at-least-once*, so a process trigger may fire *twice* — which means a process started by an event must be *idempotent* with respect to that event: receiving “payment received” twice must start one payment process, not two. This is the same idempotency discipline the components part requires of job workers, here at the level of process triggering.

Event-driven automation does not relax the idempotency rule — it sharpens it. An event backbone typically delivers *at least once*, so the same “payment received” event can arrive twice, and a process triggered naively will start twice — two payment processes for one payment. Every event-triggered process must be idempotent on the event: correlate on a business key so the second delivery joins, or is recognised and dropped, rather than starting a duplicate. An event-driven design that assumes exactly-once delivery will, under load or after a failure, double-process real business events.

13.4 The event backbone in a banking context

In a bank, the event backbone carries the institution’s business facts: “loan disbursed,” “payment received,” “customer onboarded,” “claim settled.” These are not technical messages; they are the institution’s operational heartbeat, and every system that needs to react to the business does so by subscribing to the backbone. The hyperautomation platform is one such subscriber — it starts and advances processes from the backbone’s events — but it is also a *publisher*: the process’s own significant events (“application approved,” “case escalated”) are published back to the backbone for other systems to consume. The platform is both listener and speaker on the institution’s event nervous system.

Worked example: the duplicate that became two payments

Problem. A bank automates payment handling event-driven: a “payment-received” event on the backbone starts a payment process. The backbone guarantees at-least-once delivery. After a broker hiccup, a batch of payment-received events is redelivered, and the bank finds it has started — and in some cases completed — *two* payment processes for single payments. Assess the design.

Setup. We test the design against the delivery guarantee it was built on.

Working. The backbone’s guarantee is *at-least-once* — an event may be delivered more than once, and after the broker hiccup, many were. The process was designed as if delivery were exactly-once: each “payment-received” event unconditionally starts a new process instance. So:

one payment → event delivered twice → two payment processes.

The design had no idempotency on the event, so the redelivery — which the guarantee explicitly permits — became duplicate business processing, and some payments were acted on twice.

Discussion. The failure is the warn box’s, realised, and its root is a mismatch between the delivery guarantee and the design’s assumption. The backbone never promised exactly-once; it promised at-least-once, and a robust event-driven design must therefore treat every event as *possibly a repeat*. The fix is idempotency on a business key: the payment-received event carries a payment identifier, and the process is started with that identifier as a *correlation key*, so a second delivery of the same event correlates to the instance already running — and is recognised as a duplicate and dropped — rather than starting a fresh one. With that, redelivery is harmless: the platform sees the same event twice and acts once. This is the same idempotency the components part requires of job workers, applied at the moment of process triggering, and it is not optional in event-driven automation because at-least-once delivery makes redelivery a certainty, not a risk. The lesson is the chapter’s exacting truth: event-driven automation’s power — processes that react to the world — comes with disciplines that must be honoured, and idempotency on the event is the one whose absence turns an ordinary broker hiccup into doubled payments.

Practical next steps

For every event-triggered process, design idempotency on the event from the start: correlate on a business key so a redelivered event joins or is dropped, never starts a duplicate. Assume at-least-once delivery, because that is what event backbones provide — and an event-driven design that assumes exactly-once will double-process real business events the first time a broker redelivers.

Chapter in brief

Event-driven process automation is the style in which processes are triggered and advanced by *events* — records of business facts such as “payment received” or “policy lapsed” — rather than only by direct calls. It rests on an *event backbone*, a durable ordered stream that decouples the systems publishing events from the processes consuming them. A process uses the backbone in three ways: an event can *start* a process, *advance* a waiting one parked at a catch event, or *interrupt* a running one through a

boundary event. The style is exacting: events must be durable, ordered, and genuine business facts — and because backbones deliver *at least once*, every event-triggered process must be idempotent on the event, correlating on a business key so a redelivered event never starts a duplicate.

PART III

A Reference Architecture

What Goes Where: A Modern Hyperautomation Architecture

14.1 Assembling the whole

The manuscript has, part by part, treated the pieces of a hyperautomation platform: the process engine, the decision service, the agentic layer, the cloud, the Java, the connectors, the data. This chapter does something different. It steps back and treats the *assembly* — how the pieces fit together into one coherent architecture, and, above all, the question a practising architect actually faces: *what goes where*.

That question is sharper than it sounds, because a modern financial institution does not have only Camunda. It has Camunda, and it has cloud functions — AWS Lambda and the like — and it has its own Java services, and it has agentic AI, and it has message backbones and databases and existing systems. All of these can, in some sense, “do work,” and an architecture is largely the set of decisions about which kind of component does which kind of work. An architecture that puts the wrong work in the wrong place is not broken — it runs — but it is fragile, expensive, and hard to change, and the fragility traces directly to those placement decisions. This chapter offers a reference architecture and, more importantly, the reasoning by which the placement decisions are made.

14.2 The five kinds of component

A modern hyperautomation architecture, reduced to its essentials, has five kinds of component, and the whole of “what goes where” is the discipline of knowing what each is *for*.

The first is the *process orchestrator* — Camunda. Its job is to hold the long-running, stateful, auditable business process: the sequence of steps, the waiting, the routing, the human tasks, the visible model of how work flows from start to finish. It is the component that *remembers* — that knows, for every instance, where it is and what has happened.

The second is the *decision service* — DMN, as [Chapter 4](#) described. Its job is the deterministic, rule-governed decision: should this loan be approved, what tier is this customer, is this transaction to be flagged. It is consulted by the orchestrator; it does not itself orchestrate.

The third is the *stateless function* — a cloud function such as AWS Lambda. Its job is a discrete, short-lived unit of computation: transform this data, call that API, perform this calculation, react to that event. It holds no long-running state; it runs, produces a result, and is gone.

The fourth is the *service* — a deployed application, very often a Java service in a banking institution. Its job is to own a piece of the institution’s business capability and its data: the customer service, the accounts service, the pricing service. It exposes an API; it owns a database; it embodies a domain.

The fifth is the *agent* — the agentic AI layer of [Part XXV](#). Its job is the flexible, goal-directed reasoning that the other four cannot do: interpreting an ambiguous case, assembling an in-

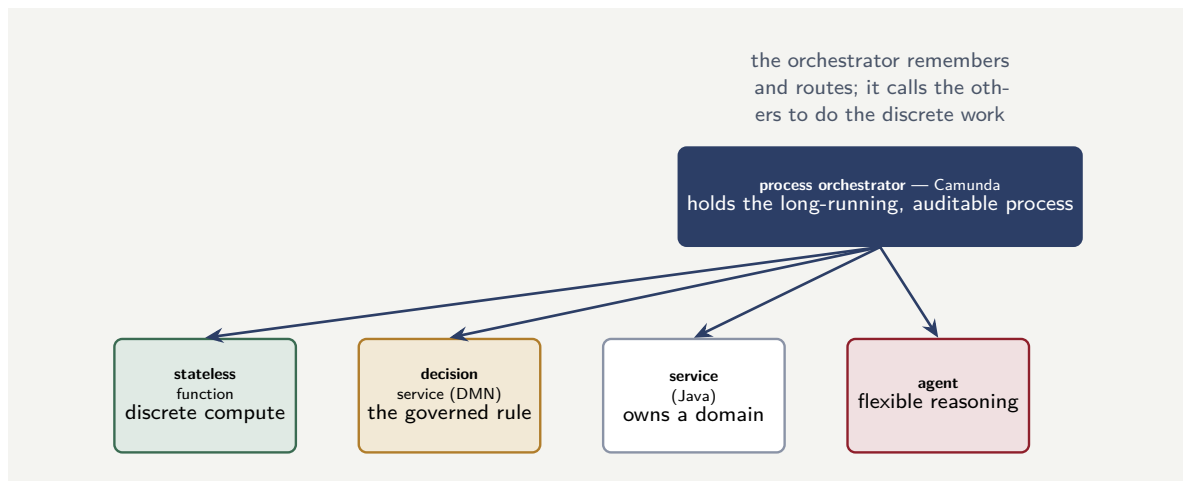


Figure 14.1. The five kinds of component. The process orchestrator holds the long-running, auditable process and calls out to the other four — the decision service for governed rules, stateless functions for discrete compute, services for domain capability, agents for flexible reasoning. Knowing what each is for is the whole of “what goes where”.

vestigation, handling a situation no fixed logic anticipated.

Figure 14.1 draws the five and their relationship: the orchestrator at the centre, calling out to the other four.

14.3 The placement principles

With the five kinds of component named, the architecture’s central question — what goes where — becomes a set of placement principles. Four of them carry most of the weight.

The orchestration logic goes in the orchestrator — nowhere else. The sequence of a business process, its routing, its waiting, its overall shape — this belongs in Camunda, as an explicit model, and must not be scattered into the functions or the services. A function that, after doing its discrete computation, decides what the process should do next and triggers it has taken a piece of orchestration into a place it cannot be seen or governed. The single most common architectural mistake is orchestration logic leaking out of the orchestrator into the components it calls — and the result is a process whose real flow is not in the model but smeared across a dozen functions, where no one can see it.

The discrete computation goes in the functions. A step of a process that is a discrete, stateless unit of work — transform this document, call this external API, compute this figure — is a function. It is not modelled as elaborate logic inside the orchestrator, and it is not built into a heavyweight always-on service. The function is the right home for the short-lived task, and the orchestrator invokes it as a step.

The domain capability and its data go in the services. A coherent piece of the institution’s business — customers, accounts, pricing — with its own data, is a service. The orchestrator does not own customer data; the customer service does, and the orchestrator’s process *calls* the customer service. This keeps the orchestrator free of domain data it should not hold, and keeps each domain’s data under one owner.

The governed decision goes in the decision service, and the flexible reasoning goes in the agent — and never the reverse. A decision that can be expressed as rules belongs in DMN, deterministic and auditable; reasoning that genuinely cannot be reduced to rules belongs in a contained

agent. The error in both directions is real: rules forced into an agent lose their determinism and auditability; genuine judgement forced into a rigid rule table is brittle and wrong at the edges.

Tip

The most common and most damaging architectural mistake is orchestration logic leaking out of the orchestrator. A function or a service that, having done its own job, decides and triggers the process's next step has taken a piece of the process's flow into a place where it cannot be seen, reviewed, or audited. Keep the orchestration in the orchestrator: functions and services do their discrete work and *return*; they do not decide what the process does next.

14.4 How the components communicate

An architecture is not only its components but the *connections* between them, and a few disciplines govern those.

The orchestrator reaches the functions and services through the connectors and workers of [Chapter 33](#) and [Chapter 32](#) — an outbound connector to invoke a function, a job worker for a step the institution's own code performs. These are synchronous-feeling calls within a process step: the orchestrator's step calls out, the function or service does the work, the result returns.

The orchestrator hears about events in the world through the inbound connectors and message events of [Chapter 34](#) — an event on a backbone, a message correlated to a waiting instance. This is how a process started elsewhere, or a thing that happened in another system, reaches the process.

And the architecture should prefer, between its larger parts, *orchestration over choreography* — the lesson of [Chapter 34](#) at architecture scale. The process's flow lives in the orchestrator's explicit model; the components do not coordinate among themselves by a web of events that no single artefact describes. The connections carry work and carry events, but the *flow* — the knowledge of what happens next — stays in the one place built to hold it.

14.5 Architecture as a living artefact

An architecture is not a document produced once and filed; it is a *living artefact* that must be maintained as the platform evolves. The reference architecture of [Chapter 14](#) sets the target; the solution and technology architectures of this part realise it; and all three must be updated as the platform grows, as new integrations are added, and as the institution's regulatory and operational landscape shifts. An architecture diagram that does not match the running platform is not an architecture; it is a historical record of a platform that no longer exists.

Worked example: orchestration leaked into the functions

Problem. A payments platform is built on cloud functions. The flow — validate the payment, check for fraud, debit the account, credit the beneficiary, notify the customer — is implemented as five functions, and each function, when it finishes, decides what comes next and invokes the next function directly. There is no process orchestrator; the flow exists only as the chain of function-to-function invocations. A regulator asks the institution to produce,

for a sample of 40 payments, the exact path each took and where each currently stands. A separate request asks the team to insert a new sanctions-screening step between fraud and debit. Assess what each request costs this architecture, and what a process orchestrator would have changed.

Setup. We consider, for each request, what the function-chain architecture must do, and contrast with an architecture where Camunda holds the flow.

Working. *Producing the path of 40 payments.* The flow is not recorded anywhere as a flow — it exists only as functions having invoked functions. To reconstruct a payment's path, the team must gather and correlate the logs of five separate functions and stitch them into a sequence:

$40 \text{ payments} \times 5 \text{ functions} = 200 \text{ log trails to gather and correlate by hand,}$

and “where does it stand now” has no direct answer at all, because no component holds the current state of a payment. *Inserting the sanctions step.* The flow is encoded in the function-to-function invocations, so inserting a step means changing the code of the fraud function (to call sanctions instead of debit) and the new sanctions function (to call debit), and testing the rewired chain:

a code change across multiple functions, to alter flow.

With a process orchestrator. The flow is one explicit Camunda model. The regulator's question is answered from Operate and the audit record directly — every payment's path and current position, immediately. Inserting the sanctions step is a change to the *model* — a new step on the diagram — reviewed and deployed, with no function rewired.

Discussion. The function-chain architecture works — payments flow through it every day — and that is exactly what makes its flaw dangerous: it is invisible until the day someone asks a question the architecture cannot answer. The two requests in the problem are not exotic; “show me what happened and where it stands” and “insert a step” are the most ordinary demands made of a financial process, and this architecture answers the first only by hours of log archaeology and the second only by a multi-function code change. The root cause is the placement error this chapter has named: the orchestration logic — the knowledge of what step follows what — was put in the functions, each function deciding and triggering the next, and so the flow exists nowhere as a reviewable, queryable thing. It is smeared across five functions' invocation calls. A process orchestrator would have held that flow as one explicit model, and the model is simultaneously the answer to “what is the flow,” the source of “where does each instance stand,” and the place a new step is added. The lesson is the chapter's central principle stated through its violation: the discrete computation rightly lived in the functions, but the *orchestration* should never have. Functions are excellent at being functions — discrete, stateless, scalable units of compute — and they are a poor place to keep a business process's flow, because a flow needs to be remembered, seen, queried, and governed, and a chain of function calls is none of those things. What goes where: the computation in the functions, the flow in the orchestrator.

Worked example: placing five pieces of work

Problem. An architecture team is designing a loan-origination platform and must place five pieces of work, each with the right kind of component. (1) The end-to-end loan journey — intake, checks, underwriting, offer, completion — runs for days and involves human approvals. (2) A document-classification step takes an uploaded PDF and returns its type, in about two seconds. (3) The eligibility decision applies the bank's lending rules to produce an

approve/refer/decline outcome. (4) The customer's accounts and balances must be read and updated by many processes. (5) A complex exception — an unusual application no fixed path anticipates — must be investigated case by case. For each, name the component, and give the reason.

Setup. We match each piece of work to one of the five kinds of component by its defining characteristic — long-running flow, discrete compute, governed rules, domain capability, or flexible reasoning.

Working. (1) *The loan journey* is a long-running, human-involved flow that must be remembered, seen, and audited — the defining work of the **process orchestrator** (Camunda). (2) *Document classification* is a discrete, stateless, seconds-long unit of computation — a **stateless function**, invoked as a step. (3) *The eligibility decision* is governed, rule-based logic with an auditable outcome — a **decision service** (DMN). (4) *Accounts and balances* are a coherent domain with its own data, used by many processes — an **accounts service**, which owns that data and exposes an API. (5) *The complex exception* is flexible, goal-directed reasoning over a case no fixed logic anticipated — a contained **agent**. Five pieces of work, five kinds of component:

journey → orchestrator, classify → function, eligibility → DMN,
accounts → service, exception → agent.

Discussion. Each placement is read directly off the work's defining characteristic, and the worked example exists to show that “what goes where” is not a matter of taste but of matching a recognisable kind of work to the component built for it. The journey *is* a long-running flow, so it goes in the orchestrator; classification *is* discrete compute, so it goes in a function; and so on down the list. The mistakes happen when a team places work by habit or convenience rather than by its characteristic — putting the eligibility rules in a function's code, where they are neither governed nor auditable; or letting the orchestrator hold accounts data, making it a shadow system of record; or forcing the complex exception down a rigid modelled path it does not fit. The discipline is to ask, of every piece of work, the single question this chapter is built around: which *kind* of work is this — a flow, a computation, a governed decision, a domain capability, or flexible reasoning? — because the answer names the component, and an architecture is sound exactly to the degree that its five kinds of work sit in their five kinds of place.

Practical next steps

Look at one end-to-end process in your institution and ask where its *flow* actually lives. Is it an explicit model in an orchestrator — one artefact you can show a regulator and edit to change the process — or is it implicit in a chain of services and functions each invoking the next? If it is the latter, the architecture has put orchestration where computation belongs, and the cost is paid every time someone asks what happened or asks for a change.

Chapter in brief

A modern hyperautomation architecture has five kinds of component, and “what goes where” is the discipline of knowing what each is for: the process orchestrator (Camunda) holds the long-running, auditable flow; the decision service (DMN) holds governed rules; stateless functions do discrete compute; services own domain capabil-

ity and data; agents do flexible reasoning. Four placement principles follow — orchestration logic in the orchestrator and nowhere else, discrete computation in functions, domain capability and data in services, governed decisions in DMN and genuine judgement in agents. The most common and damaging mistake is orchestration logic leaking out of the orchestrator into the components it calls, leaving a process whose real flow is in no single artefact.

AWS Step Functions and Camunda Compared

15.1 Two orchestrators, honestly compared

The previous chapter named the process orchestrator as the component that holds a business process's flow, and treated Camunda as that orchestrator. But Camunda is not the only orchestrator a financial institution might use. The cloud providers offer their own, and the most prominent is *AWS Step Functions*. An architect choosing how to build, or inheriting a mixed estate, needs an honest comparison — not an advocacy piece — and this chapter provides one.

The honest starting point is that Step Functions and Camunda are genuinely the same *kind* of thing. Both are orchestrators: both hold a defined workflow as an explicit artefact, both invoke steps in order, both track the state of each execution, both handle retries and errors, both provide a visual execution history. An architect who knows one will recognise the other. The comparison is therefore not “orchestrator versus not” but a comparison of two orchestrators with different origins, different strengths, and different natural homes.

15.2 What AWS Step Functions is

AWS Step Functions is a *serverless orchestration service*: a managed AWS service that runs workflows — called *state machines* — defined in a JSON-based language, the *Amazon States Language*. A state machine is a sequence of states; each state typically invokes an AWS service, most often a Lambda function, and the state machine manages the transitions, the state, the retries, and the error handling between them.

Step Functions offers two execution modes, and the distinction matters. The *Standard* workflow is built for long-running, durable, auditable processes — it can run for up to a year, retains a full visual execution history, and is the mode for business processes. The *Express* workflow is built for high-volume, short-duration executions — it is cheaper per execution and optimised for throughput, but does not retain the same per-execution audit detail, and is the mode for event-processing rather than business process. An architect using Step Functions for a business process is using Standard workflows.

Step Functions has, natively, much of what [Part V](#) treated in Camunda: built-in retry policies with exponential backoff, error catching and routing, a callback pattern that lets a workflow pause and wait for an external signal — a human approval, an event — and a visual console showing each execution's path. It is, genuinely, a capable orchestrator.

Listing 15.1. A state in the Amazon States Language. The state invokes a Lambda function, retries transient failures with exponential backoff, and catches a remaining failure to a compensation state — the retry-and-catch machinery is built in.

```
1 "ChargePayment": {  
2   "Type": "Task",
```

```

3  "Resource": "arn:aws:lambda:...:function:charge-payment",
4  "TimeoutSeconds": 30,
5  "Retry": [
6    { "ErrorEquals": ["States.TaskFailed"],
7      "IntervalSeconds": 2,
8      "MaxAttempts": 3,
9      "BackoffRate": 2.0 }
10 ],
11 "Catch": [
12   { "ErrorEquals": ["States.ALL"],
13     "Next": "CompensatePayment" }
14 ],
15 "Next": "ReserveInventory"
16 }

```

The listing shows the shape of an Amazon States Language definition: a JSON state that names a Lambda resource, declares its retry policy, and catches a failure onward to a compensation state. It is the JSON-defined counterpart of the BPMN service task with a boundary error event — the same orchestration ideas, in a different notation.

15.3 Where each is naturally at home

The useful comparison is not which orchestrator is “better” — both are capable — but where each is *naturally at home*, because the differences in origin produce real differences in fit.

Step Functions is naturally at home *inside the AWS ecosystem, orchestrating AWS services*. Its deepest strength is integration: it invokes Lambda functions, and a great many other AWS services, directly and natively, with no connector to build. If the work a process must orchestrate is itself mostly AWS services and Lambda functions, Step Functions orchestrates them with the least friction of anything — it is the AWS-native answer, and an institution already deep in AWS will find it sits naturally in that estate. It is serverless, so there is no cluster to operate. And for an architecture whose components are predominantly AWS, Step Functions keeps the orchestration in the same managed, serverless world as the components.

Camunda is naturally at home *orchestrating the business process across a heterogeneous estate*. Its strengths are the ones this manuscript has spent twelve parts on: BPMN as a standard, business-readable modelling notation that a non-technical process owner can read and review — where the Amazon States Language is a JSON definition for engineers; DMN as a governed, first-class decision artefact alongside the process; the human task and worklist machinery of [Part V](#), built deeply for processes where people do much of the work; portability, since Camunda is not tied to one cloud and orchestrates AWS, other clouds, on-premise systems, and legacy estates evenhandedly; and the process-mining and Optimize analytics of [Part XXIII](#). For a complex, human-involved, regulated business process spanning systems that are not all in one cloud, Camunda’s BPMN-and-DMN, process-and-decision, human-task strengths are decisive.

Figure [15.1](#) sets the two side by side — not to crown a winner, but to show where each fits.

15.4 They are not mutually exclusive

An architect’s instinct, given two orchestrators, is to choose one. But a large institution’s reality is often *both*, and the disciplined position is that this can be a sound architecture rather than a confused one — if the boundary is deliberate.

A coherent mixed architecture places Camunda as the orchestrator of the *business process* — the long-running, human-involved, regulated, end-to-end flow — and uses Step Functions,

aspect	Camunda	AWS Step Functions
modelling	BPMN — business-readable	Amazon States Language (JSON)
decisions	DMN, first-class	no native equivalent
human tasks	deep — worklists, allocation	callback pattern only
estate fit	portable — any cloud, on-prem	AWS-native — best within AWS
operations	self-managed or SaaS	fully serverless

Figure 15.1. Camunda and AWS Step Functions compared. Both are capable orchestrators; the differences are of fit. Camunda’s BPMN, DMN, and human-task depth suit complex, human-involved processes across a heterogeneous estate; Step Functions’ serverless, AWS-native integration suits orchestration of AWS services.

where it appears, to orchestrate *technical sequences within the AWS estate* that a Camunda step invokes. The Camunda process has a step “enrich and score the application”; that step calls out to a Step Functions Express workflow that chains six Lambda functions to do the enrichment; the Step Functions workflow returns its result to the Camunda step. Camunda holds the business process; Step Functions orchestrates a technical sub-sequence; the boundary between them is a single, clean call.

What makes this sound rather than confused is that the two orchestrators are not *competing* for the same role. The incoherent version — the one to avoid — has the business process’s flow split arbitrarily between the two, some steps in Camunda and some in Step Functions with no principle, so that no single artefact holds the business process and the [previous chapter](#)’s leaked-orchestration problem appears in a new form. The coherent version has a clear rule: the business process is in Camunda; Step Functions, if used, orchestrates technical AWS sequences *within* a Camunda step. One orchestrator owns the business flow; the other is a tool that flow may call.

Using both Camunda and Step Functions is sound only if the boundary between them is principled. The incoherent architecture splits one business process’s flow arbitrarily across the two, so no single artefact holds the process — the leaked-orchestration problem in a new guise. The coherent architecture gives one orchestrator the business process end to end — Camunda, for a regulated, human-involved process — and lets the other orchestrate technical sequences *within* a step of it. Decide which orchestrator owns the business flow, and do not split it.

15.5 Evaluation criteria for a regulated institution

A financial institution evaluating workflow platforms should weight its criteria differently from a technology startup. The criteria that matter most are *auditability* (can the platform produce, for any past case, a complete record of what happened?), *human-task support* (does the platform have a first-class worklist, task allocation, and escalation model?), and *regulatory*

longevity (will the platform be supported for the decade-long horizon a regulated institution plans on?). Raw throughput, developer experience, and time-to-first-demo matter, but they are secondary to these three. A platform chosen for its demo speed but lacking auditability will fail its first regulatory examination.

Worked example: choosing the orchestrator for two workloads

Problem. An institution, already a significant AWS user, has two new workloads to orchestrate. Workload A is a mortgage origination process: it runs for weeks, involves six human decision points across underwriting and approval, must be readable and reviewable by non-technical process owners and presented to a regulator, and touches a core banking system that is on-premise. Workload B is a document-enrichment pipeline: when a document is uploaded to AWS storage, a sequence of eight Lambda functions extracts, classifies, and indexes it; it runs in seconds, has no human involvement, and is entirely within AWS. The architecture team must choose an orchestrator for each. Reason to the choices.

Setup. We match each workload’s characteristics against where each orchestrator is naturally at home.

Working. *Workload A — mortgage origination.* Its characteristics: long-running (weeks); six human decision points; a need for business-readable, regulator-presentable models; an on-premise system in scope. Against the comparison: human-task depth, BPMN business-readability, regulatory auditability, and portability beyond AWS are all Camunda’s natural strengths, and the on-premise core banking system is outside Step Functions’ AWS-native home. The choice:

Workload A → Camunda.

Workload B — document enrichment. Its characteristics: seconds-long; no human involvement; eight Lambda functions; entirely within AWS. Against the comparison: this is exactly Step Functions’ natural home — a short, technical, human-free sequence of AWS services, where native Lambda integration and serverless operation are decisive and BPMN readability and human-task machinery are simply not needed. The choice:

Workload B → AWS Step Functions.

Discussion. The two workloads go to two different orchestrators, and that is the right answer — not a failure to standardise. The worked example is built to show that “which orchestrator” is not an institution-wide religious choice but a per-workload fit decision, and the fit is read off the workload’s characteristics. Workload A has every characteristic that is Camunda’s home: duration, human involvement, the need for business-readable and regulator-facing models, an estate that reaches beyond one cloud — forcing it onto Step Functions would mean orchestrating six human approvals through the callback pattern, modelling a regulated mortgage process in JSON, and reaching an on-premise system from a service designed for AWS. Workload B has every characteristic that is Step Functions’ home: brief, technical, human-free, all-AWS — and putting it on Camunda would mean operating a process engine, and building connectors, for a job that AWS-native Lambda orchestration does with no friction. The deeper lesson is the [previous chapter](#)’s “what goes where” applied to the orchestrator choice itself: an architecture is a set of placement decisions, and placing each workload with the orchestrator whose strengths its characteristics actually call for produces a sound mixed estate. The mistake would be insisting on one orchestrator for both out of a

wish for uniformity — because uniformity bought by putting a workload somewhere it does not fit is a false economy that the workload’s friction repays, with interest, for as long as it runs.

Practical next steps

For a workload your institution is about to orchestrate, list its characteristics — duration, human involvement, the need for business-readable models, the estate it touches — and match them against where Camunda and Step Functions are each naturally at home. The orchestrator is chosen per workload, by fit; an institution can soundly use both, provided each business process is owned, end to end, by one.

Chapter in brief

AWS Step Functions and Camunda are genuinely the same kind of thing — both orchestrators that hold a workflow as an explicit artefact, track state, and handle retries and errors. Step Functions is a serverless AWS service, defining state machines in the JSON-based Amazon States Language, with Standard workflows for long-running processes and Express for high-volume short ones. The comparison is one of fit: Step Functions is naturally at home orchestrating AWS services within the AWS estate; Camunda is at home orchestrating complex, human-involved, regulated business processes across a heterogeneous estate, with BPMN readability, first-class DMN, and deep human-task machinery. The two are not mutually exclusive — a sound mixed architecture gives one orchestrator the business process and lets the other orchestrate technical sequences within a step.

CHAPTER 16

The Java Saga Pattern and Where It Fits

16.1 Distributed transactions without a transaction

Chapter 50 introduced the saga as the answer to a problem financial systems cannot escape: a business operation that must change several systems — debit here, credit there, update a third — where no single database transaction can span all of them, because they are separate systems with separate databases. The saga’s answer was a sequence of local steps, each with a *compensating action* that undoes it, so that if a later step fails, the compensations of the completed steps run in reverse and the systems are returned to consistency.

That chapter treated the saga as Camunda models it — the transaction subprocess, compensation. This chapter treats the saga as a *Java pattern* implemented without a process engine, because a great many institutions do implement it that way, and an architect must understand both the pattern in itself and when each implementation is right.

16.2 The saga as a hand-built Java pattern

A saga can be built directly in Java, with no orchestrator, and at its core it is not complicated. A *saga coordinator* — an ordinary Java class — holds the sequence of steps. For each step it knows two things: the *forward action* that does the work, and the *compensating action* that undoes it. The coordinator executes the forward actions in order; if one fails, it walks back through the steps already completed and executes their compensating actions in reverse order; and it records, throughout, which steps have completed so it knows what to compensate.

This hand-built pattern is entirely legitimate, and for the right case it is the right tool. It has real virtues. It is *lightweight* — it is a class, not a platform; there is no engine to deploy or operate. It is *fast* — an in-process sequence of method calls has none of an engine’s overhead. It sits *inside a service*, naturally, when the saga is that service’s own concern. For a saga that is short-lived, lives within one service’s responsibility, involves no human steps, and is one piece of a developer’s code rather than a business process in its own right, the Java saga pattern is proportionate and correct, and reaching for a process engine would be over-engineering.

Listing 16.1. The core of a hand-built Java saga coordinator. Each step pairs a forward action with its compensation; on a failure, the completed steps are compensated in reverse.

```
1 public class SagaCoordinator {
2     private final Deque<Runnable> completedCompensations
3         = new ArrayDeque<>();
4
5     public void run(List<SagaStep> steps) {
6         try {
7             for (SagaStep step : steps) {
8                 step.forward().run();           // do the work
9                 // remember how to undo this step
10                completedCompensations.push(step.compensation());
11            }
12        }
13    }
```

```

11     }
12     } catch (RuntimeException failure) {
13         compensateAll();           // walk back
14         throw new SagaFailedException(failure);
15     }
16 }
17
18 private void compensateAll() {
19     // undo completed steps in reverse order
20     while (!completedCompensations.isEmpty()) {
21         completedCompensations.pop().run();
22     }
23 }
24 }

```

16.3 Where the hand-built saga reaches its limit

The hand-built Java saga is right for the case named above — and an architect must equally know where it stops being right, because its limits are the place the process engine earns its keep.

The hand-built saga is *in-memory and process-bound*. The coordinator’s record of which steps have completed lives in the memory of the running Java process. If that process *dies* mid-saga — a crash, a deployment, a machine failure — the record of what has completed dies with it. The saga is then in a terrible position: some forward actions have happened, in real systems, money has moved — and the one thing that knew which, and knew how to compensate, is gone. The hand-built saga has no durable memory of its own state, and a saga whose state is not durable is a saga that a crash can leave permanently, invisibly inconsistent.

The hand-built saga is also *short-lived by nature*. It is a sequence of method calls; it cannot easily wait days for a human approval, or weeks for an external party, because holding an in-memory coordinator alive and uncrashed for days is not realistic. A saga with a human step, or a long wait, is not the hand-built pattern’s territory.

And the hand-built saga is *not visible*. Its state is in memory, its flow is in code; there is no console showing a stuck saga, no audit record of compensations run, nothing to show a regulator. For a saga that is one developer’s internal concern, that is acceptable; for a saga that is a regulated business operation, it is not.

The hand-built Java saga keeps its record of completed steps in the memory of the running process. If that process dies mid-saga — a crash, a deployment — the record is lost: forward actions have executed in real systems, money has moved, and nothing now knows what to compensate. A hand-built saga is sound for short-lived, in-service sagas; it must not be used for a long-running or regulated distributed transaction, because its state is not durable and a crash can leave it permanently and invisibly inconsistent.

16.4 The saga in Camunda, and how to choose

The process engine’s saga — the transaction subprocess and compensation of [Chapter 50](#) — is precisely the hand-built saga with its three limits removed. The engine’s state is *durable*: the saga’s progress is persisted, so a crash loses nothing — a recovering engine knows exactly which steps completed and can drive the compensations. It is *long-lived*: an engine-borne saga can wait days or weeks, for a human or an external party, because the engine holds the

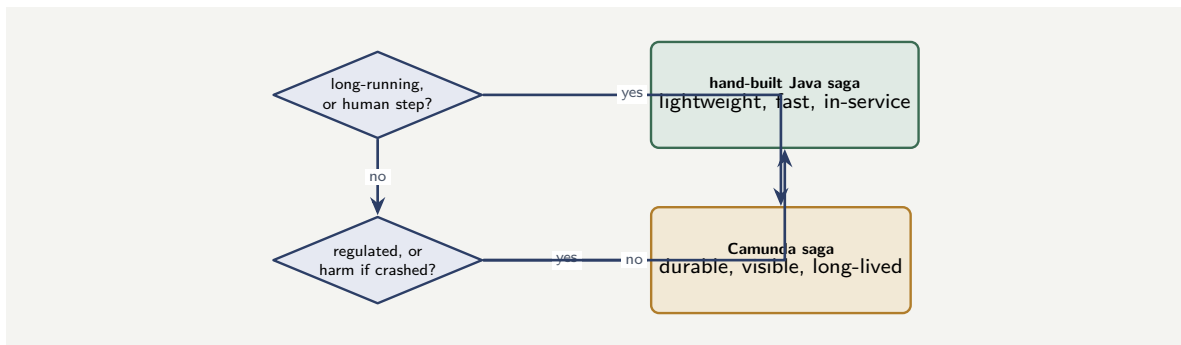


Figure 16.1. Choosing the saga implementation. A saga that is short, technical, human-free, and recoverable if it crashes is well served by a hand-built Java coordinator; a saga that is long-running, human-involved, regulated, or would cause harm if it crashed mid-way requires the durable, visible, engine-borne saga.

state durably while it waits. And it is *visible*: the saga is a model, its execution is in Operate, its compensations are in the audit record.

The choice between the two is therefore not ideological; it follows from the saga’s nature, and four questions decide it. Is the saga *long-running*, or does it complete in moments? Does it involve a *human step* or a long wait? Is it a *regulated business operation* needing visibility and an audit record, or an internal technical concern? Could a crash mid-saga cause *serious harm* — real money left in an inconsistent state? If the saga is short, technical, human-free, in-service, and a crash is recoverable by other means — the hand-built Java saga is proportionate and correct. If the saga is long-running, human-involved, regulated, or a crash mid-saga would be a serious incident — the engine’s durable, visible, long-lived saga is required, and the hand-built pattern is the wrong tool however appealing its lightness.

Figure 16.1 draws the choice as the decision it is.

16.5 The Java discipline for a Camunda platform

The Java code behind a Camunda platform is production code in a regulated institution, and it must be held to production standards: code review, static analysis, dependency scanning, and the testing disciplines this part describes. A worker that “works” but has not been reviewed, tested, and scanned is technical debt from the moment it is deployed, and in a regulated institution technical debt is also regulatory debt — a regulator who asks “how do you ensure the quality of your automation code” expects an answer, and “it works in the demo” is not one.

Worked example: the saga that outgrew its Java coordinator

Problem. A bank builds a fund-transfer saga as a hand-built Java coordinator: three steps — debit the source account, credit the destination, record the transfer — each with a compensation. It runs entirely within the transfers service, completes in well under a second, and for two years works well. The bank then extends the product: a transfer above \$50,000 now requires a human compliance approval, which may take up to two business days, *between* the debit and the credit. The team’s first instinct is to keep the hand-built coordinator and have it wait for the approval. Assess that instinct, and what the extension actually demands.

Setup. We test the extended saga against the hand-built pattern’s limits — in particular, durability and long-livedness — and identify what is required.

Working. The extension inserts, between step 1 (debit) and step 2 (credit), a wait of up to:

2 business days

for a human approval. For the hand-built coordinator to “wait” for that, the Java process holding the coordinator — and its in-memory record that the debit has completed — must stay alive and uncrashed for up to two days. In two days of real operation, that process will, at some point, be restarted: a deployment, a patch, a machine failure. When it is:

the in-memory record that the debit completed is lost,

and the bank has a state where money has left a customer’s account, no credit has been made, and nothing knows a compensation is owed. The extension has taken the saga from short-lived to long-running and from human-free to human-involved — crossing two of the hand-built pattern’s limits at once. The extended saga requires durable, long-lived state:

the engine-borne saga — Camunda’s transaction subprocess.

Discussion. The team’s instinct — keep the coordinator, just have it wait — is the natural one and is wrong, and the worked example is built to show *why* the wrongness is easy to miss. For two years the hand-built saga was the correct choice: short, technical, in-service, sub-second — a crash mid-saga was nearly impossible because the saga lasted milliseconds. The extension does not look, on the surface, like an architectural change — it is “just adding an approval step” — but it silently moves the saga across two of the hand-built pattern’s hard limits. A saga that must wait two days is a saga whose state must survive two days, which means it must survive the restarts that two days of operation guarantee, which means the state must be *durable* — and durable state is exactly what the hand-built coordinator does not have. The human approval compounds it: the hand-built pattern has no notion of a human task, no worklist, no way to present the approval. The lesson generalises into the chapter’s decision rule. The choice between a hand-built Java saga and an engine-borne one is not made once and forever; it must be *re-made whenever the saga’s nature changes*, because a change that looks like a small feature — a human approval, a longer wait — can move the saga from the territory where the lightweight pattern is correct into the territory where only the durable, visible, long-lived engine saga will do. The bank’s extended transfer is now a long-running, human-involved, regulated distributed transaction handling real money, and that is, precisely, Camunda’s transaction subprocess — not because the engine is grander, but because the saga’s nature now demands durability the hand-built pattern cannot give.

Practical next steps

For any saga — any multi-step distributed transaction with compensations — in your institution, apply the four questions: is it long-running, does it involve a human step, is it a regulated operation needing an audit record, would a mid-saga crash cause serious harm? A saga that answers no to all four is well served by a hand-built Java coordinator. A saga that answers yes to any needs the durable, visible, engine-borne saga — and a saga whose nature is changing should be re-assessed against these questions, not assumed to still fit the choice made when it was first built.

Chapter in brief

The saga — a sequence of local steps each with a compensating action — solves the distributed-transaction problem, and it can be built directly in Java: a coordinator class that runs forward actions and, on failure, compensates the completed steps in reverse. The hand-built Java saga is lightweight, fast, and proportionate for a short-lived, in-service, human-free saga. But its state is in-memory and process-bound — a crash mid-saga loses the record of what to compensate — and it is short-lived by nature and not visible. Camunda's engine-borne saga removes those three limits: durable state, long-lived waiting, full visibility. The choice follows from the saga's nature — short, technical, recoverable favours the Java pattern; long-running, human-involved, regulated, or harmful-if-crashed requires the engine.

Enterprise Architecture: Where a Process Engine Fits

17.1 The widest view

The chapters so far in this part have treated architecture at the level of a platform — what goes where among orchestrator, functions, services, and agents. This chapter steps back to the widest level an architect works at: *enterprise architecture*, the discipline that describes the institution as a whole — all its capabilities, all its major systems, all its data, and how they fit together to serve the business strategy.

The question this chapter answers is the one an enterprise architect must settle before any platform is built: *at the enterprise level, what role does a process engine play?* It is a more fundamental question than “which process engine,” and it has, broadly, two answers. The process engine can be the institution’s *primary orchestration capability* — the recognised, enterprise-level home of business-process flow — or it can be a *tactical tool* used here and there for particular problems, with no enterprise-level role at all. The difference between those two answers shapes everything downstream, and an institution that has never consciously chosen between them has, in effect, chosen the second by default.

17.2 What enterprise architecture is

Enterprise architecture is the practice of describing, and deliberately shaping, the structure of an entire institution’s business and technology. It works in *layers* — conventionally the business layer (capabilities, processes, organisation), the application layer (the systems and their interactions), the data layer (the information and its ownership), and the technology layer (the infrastructure beneath). An enterprise architect’s job is to keep these layers coherent: to ensure the applications genuinely support the business capabilities, that data has clear ownership, that the whole moves toward the strategy rather than sprawling.

The enterprise architecture is also where *cross-cutting decisions* are made — decisions no single project would make for itself but that every project must respect. Standards for integration. The principle that customer data has one master. The decision that the institution will, or will not, have an enterprise orchestration capability. These are enterprise-architecture decisions, and the role of a process engine is exactly one of them.

17.3 The process engine as a recognised enterprise capability

The central proposition of this chapter is that, in a modern financial institution, *process orchestration deserves to be a recognised enterprise-architecture capability* — named, owned, and standardised at the enterprise level, exactly as data management or integration is.

The reasoning follows from what the manuscript has argued throughout. A bank or in-

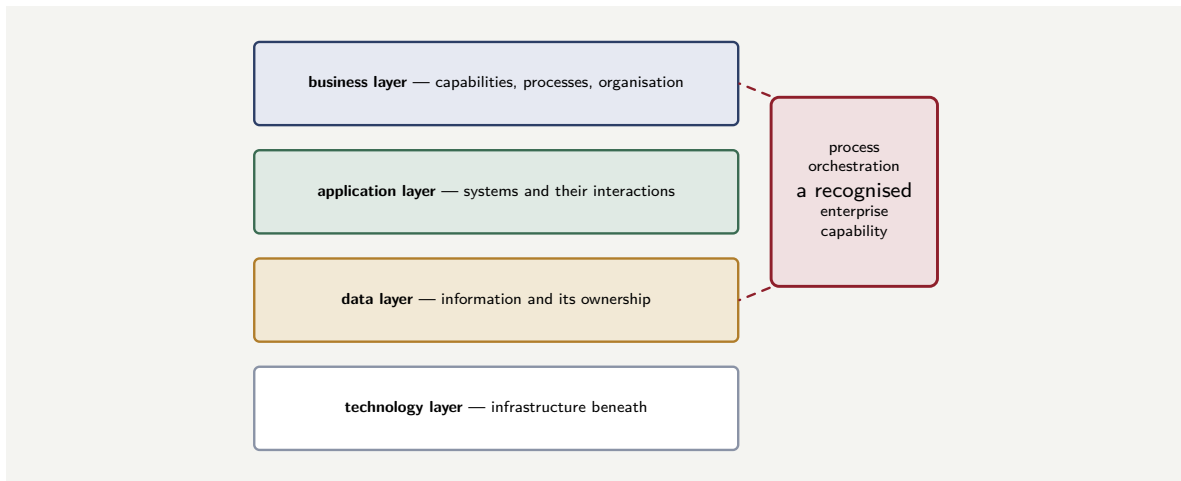


Figure 17.1. The enterprise-architecture layers, and process orchestration as a recognised capability spanning the business and application layers. An enterprise architect’s contribution is to put process orchestration on the map — named, owned, and standardised — rather than leaving it absent and improvised per project.

surer’s products *are* processes; its regulatory obligations are substantially obligations about how processes run and are evidenced; its operational cost is largely process cost. An institution that treats process orchestration as a recognised enterprise capability gives that reality an architectural home: a standard way processes are modelled, a standard engine, an enterprise-level owner, a coherent picture of how the institution’s processes connect. An institution that does not — that lets each project pick its own way to coordinate work — gets what [Chapter 1](#) called a pile of bots: many local automations, no enterprise picture, no standard, no coherent home for the thing that most defines the business.

This does not mean a single engine runs literally every process. It means process orchestration is *on the enterprise architecture’s map* — a named capability with a deliberate strategy — rather than absent from it. The enterprise architect’s contribution is to put it on the map.

Figure 17.1 places process orchestration on the enterprise-architecture map.

17.4 Primary orchestrator, or exception handler only

Granting that process orchestration belongs on the enterprise map, the enterprise architect still faces the chapter’s opening question in its sharpest form: in what *role* is the engine placed? Two enterprise-level patterns are worth naming precisely, because they are genuinely different architectures.

In the *primary-orchestrator* pattern, the process engine is the enterprise’s principal home of business-process flow. The major end-to-end journeys — onboarding, lending, claims, servicing — are modelled in the engine, and the engine coordinates the systems, the decisions, the people, and the agents that each journey touches. The engine is, at the enterprise level, *where the processes live*.

In the *exception-handler* pattern, the institution’s straight-through processing happens *inside the core systems* — the core banking system, the policy administration system run their standard flows — and the process engine is brought in only for what those systems handle badly: the *exceptions*, the cases that fall out of straight-through processing, the cross-system coordination the cores cannot do, the human-judgement work. The engine is, at the enterprise level,

the handler of what the cores cannot do alone.

Neither pattern is wrong, and the honest enterprise architect chooses deliberately rather than drifting. The primary-orchestrator pattern gives the institution the fullest version of everything this manuscript has argued — one explicit, governed home for process flow, end-to-end visibility, the process as a first-class managed asset — but it is a larger commitment, and it asks the core systems to cede the orchestration role they often historically held. The exception-handler pattern is a smaller, lower-risk commitment that delivers real value where the cores are weakest, and it is often the right *first* enterprise role for an engine — but an institution that stops there keeps its core processes locked inside systems that are hard to see into and hard to change.

Tip

At the enterprise level, decide deliberately between two roles for the process engine: the *primary orchestrator*, where the major end-to-end journeys live in the engine; or the *exception handler*, where straight-through processing stays in the core systems and the engine handles only the fall-outs and cross-system coordination. Both are legitimate. The mistake is not choosing — letting the engine's role emerge project by project, so the institution ends up with neither a coherent orchestration capability nor a clear division of labour with the cores.

17.5 Architecture as a living artefact

An architecture is not a document produced once and filed; it is a *living artefact* that must be maintained as the platform evolves. The reference architecture of [Chapter 14](#) sets the target; the solution and technology architectures of this part realise it; and all three must be updated as the platform grows, as new integrations are added, and as the institution's regulatory and operational landscape shifts. An architecture diagram that does not match the running platform is not an architecture; it is a historical record of a platform that no longer exists.

Worked example: two banks, two enterprise roles

Problem. Two banks each adopt Camunda. Bank A places it as the primary orchestrator: over three years it models its onboarding, lending, payments, and servicing journeys in Camunda, and the core banking system becomes a system of record that Camunda processes call. Bank B places it as an exception handler: the core banking system continues to run straight-through processing for all four journeys, and Camunda handles only the exceptions — the cases that fall out — plus some cross-system coordination. Five years on, both banks are asked by a regulator to demonstrate end-to-end control of their lending process and to make a specific change to it. Compare what each bank can do.

Setup. We consider, for each bank, where the lending process's flow lives and what that means for the regulator's two requests — demonstrating control and making a change.

Working. *Bank A — primary orchestrator.* The lending journey is an explicit Camunda model. Demonstrating end-to-end control means showing the model, the decisions, and the audit record — one coherent artefact and its history. Making the change means editing the model, reviewing it, and deploying it:

control: shown from one model; change: a governed model edit.

Bank B — exception handler. The lending journey's main flow lives *inside the core banking system*; Camunda holds only the exception paths. Demonstrating end-to-end control means reconstructing the picture from the core system's internal logic — as visible or invisible as that system allows — *plus* the Camunda exception models, two different places with two different levels of transparency. Making the change, if it falls in the straight-through path, means a change *to the core banking system*:

control: reconstructed from two places; change: a core-system change.

Discussion. Both banks made a legitimate choice, and the worked example is not an argument that Bank B chose wrongly — it is an argument that the choice has consequences an enterprise architect must see clearly in advance. Bank A's primary-orchestrator role means the lending process is, at the enterprise level, an explicit and governed asset: visible, auditable, and changeable as a model. That is the full realisation of the manuscript's argument, and it cost Bank A a larger, multi-year commitment and the displacement of the core system from the orchestration role. Bank B's exception-handler role was cheaper and lower-risk, and it delivered genuine value on the exception work — but five years on, Bank B's lending process is still substantially locked inside a core system, and the regulator's two ordinary requests are correspondingly harder: control must be reconstructed from two places, and a straight-through change is a core-system change with all the cost and slowness that implies. The enterprise-architecture lesson is that the role of the process engine is a *strategic* decision with a long shadow. An institution choosing the exception-handler role should do so knowing it is choosing to leave its core processes inside the cores — which may be the right call for now — and should revisit the decision deliberately, rather than discovering years later that “where do our processes live” has an awkward answer it never consciously chose.

Practical next steps

For your institution, establish which enterprise role its process engine actually plays today — primary orchestrator, exception handler, or an unplanned mixture — and whether that role was ever consciously decided. Then ask the enterprise-architecture question: is that the role the institution *wants* the engine to play, given that its products are processes? The answer may be to stay where you are; the discipline is to have chosen.

Chapter in brief

Enterprise architecture describes and shapes an entire institution in layers — business, application, data, technology — and makes the cross-cutting decisions every project must respect. In a financial institution, whose products are processes, process orchestration deserves to be a recognised enterprise-architecture capability — named, owned, standardised — rather than absent from the map and improvised per project. The enterprise architect must then choose the engine's role deliberately: the *primary orchestrator*, where the major end-to-end journeys live in the engine; or the *exception handler*, where straight-through processing stays in the core systems and the engine handles only fall-outs and cross-system coordination. Both are legitimate; not choosing is the mistake.

Solution Architecture and the Workflow

18.1 From the enterprise to the solution

The previous chapter worked at the enterprise level — the institution as a whole. This chapter narrows to the level most architects spend most of their time at: *solution architecture*, the design of one specific solution to one specific business problem. Where enterprise architecture asks “what capabilities does the institution have, and how do they fit,” solution architecture asks “for this particular need — a new lending product, a claims overhaul — what shall we actually build, from which components, in what shape?”

Solution architecture is where the enterprise architecture’s principles meet a real, bounded problem. The enterprise architecture may say “process orchestration is a recognised capability and Camunda is the standard engine”; the solution architecture for the new lending product must then decide how *this* product’s process is modelled, what it calls, where its data sits, how it integrates — a concrete design for a concrete problem, consistent with the enterprise principles but specific to the solution.

18.2 The solution architecture's workflow view

Every solution that automates or coordinates work has, somewhere in it, a *workflow* — a flow of steps from a beginning to an end. The solution architect’s task, for that workflow, is a sequence of deliberate decisions, and naming them turns “design the solution” into a tractable set of questions.

Where does the workflow live? The first decision is the one the enterprise chapter framed: is this solution’s workflow an explicit model in the process engine, or is it embedded in a system, or implicit in a chain of calls? A solution consistent with an enterprise orchestration capability puts its workflow in the engine, as an explicit model.

What does each step do, and what kind of component does it? This is [Chapter 14](#)’s “what goes where,” applied within the solution: each step of the workflow is a human task, a decision, a call to a service, an invocation of a function, or an agent step — and the solution architect assigns each deliberately.

What does the workflow integrate with? The solution does not stand alone; its workflow calls systems of record, external services, other processes. The solution architect identifies every such integration and chooses its style — [Chapter 25](#)’s synchronous, asynchronous, event, or batch.

Where does the workflow’s data live, and what does it carry? The solution architect decides what the process instance carries as variables and what it references in systems of record — the data-minimisation discipline applied to this solution.

How is the workflow operated, secured, and evidenced? The solution must be observable, its access controlled, its decisions auditable — the solution architect designs these in, rather

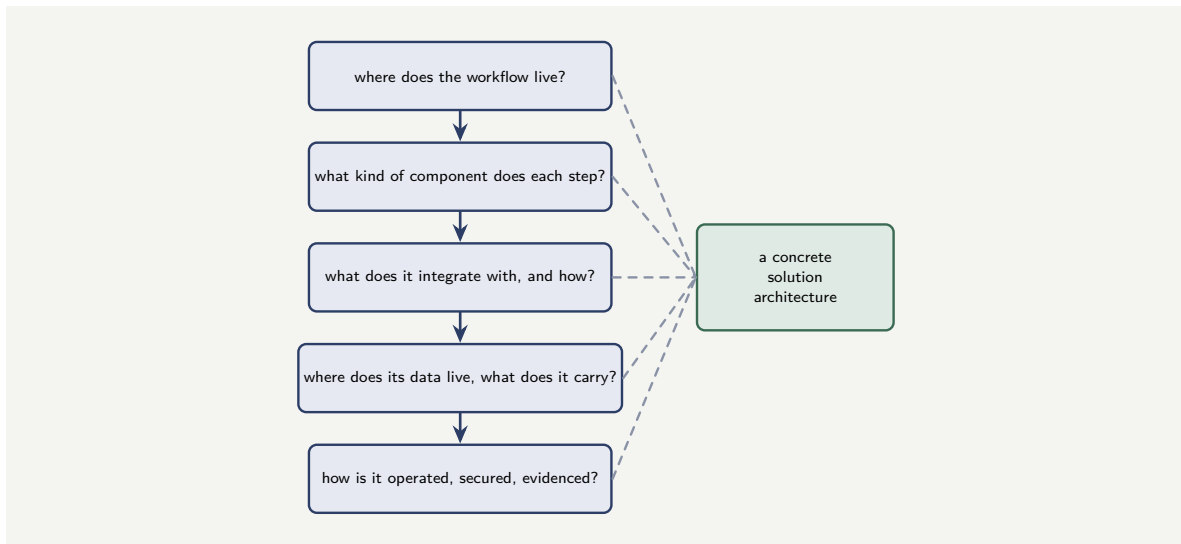


Figure 18.1. The solution architect’s workflow decisions. For the workflow at the heart of a solution, five deliberate decisions — where it lives, what components its steps use, what it integrates with, where its data sits, and how it is operated — turn “design the solution” into a tractable, answerable sequence.

than leaving them to be retrofitted.

Figure 18.1 sets out the workflow decisions a solution architecture must make.

18.3 Consistency with the enterprise, fitness for the problem

A solution architecture is pulled in two directions, and holding both is the solution architect’s central discipline.

It must be *consistent with the enterprise architecture*. The enterprise’s standards, its chosen engine, its integration principles, its data-ownership rules — the solution respects all of them, because a solution that quietly ignores the enterprise architecture is how the enterprise architecture decays into fiction, one expedient solution at a time.

And it must be *fit for the specific problem*. The enterprise principles are general; this problem is particular, and the solution architect must design genuinely for it — its volumes, its latencies, its human-work shape, its regulatory specifics — rather than applying a template. A solution that is consistent with the enterprise but does not actually fit the problem is as much a failure as one that fits the problem but violates the enterprise.

The resolution is that the enterprise architecture should constrain the solution at the level of *principle* — use the standard engine, honour data ownership, integrate in the standard styles — while leaving the solution architect free at the level of *design* — how this specific workflow is shaped, decomposed, and tuned. When the enterprise architecture tries to over-specify the solution, solutions stop fitting their problems; when it specifies too little, solutions stop being consistent. The workflow is usually where this balance is struck: the enterprise says “the workflow lives in the engine, modelled in BPMN, governed”; the solution architect decides what the workflow actually is.

18.4 Architecture as a living artefact

An architecture is not a document produced once and filed; it is a *living artefact* that must be maintained as the platform evolves. The reference architecture of [Chapter 14](#) sets the target; the solution and technology architectures of this part realise it; and all three must be updated as the platform grows, as new integrations are added, and as the institution's regulatory and operational landscape shifts. An architecture diagram that does not match the running platform is not an architecture; it is a historical record of a platform that no longer exists.

Worked example: the solution that ignored the enterprise standard

Problem. An institution's enterprise architecture establishes Camunda as the standard orchestration engine and BPMN as the standard process notation. A solution team, building a new trade-finance product under time pressure, finds it faster to implement the product's workflow as orchestration logic inside a custom Java application — no process engine, no BPMN model, the flow expressed in code. The solution ships on time and works. Two years later, the institution undertakes a programme to give every major process end-to-end observability through the standard tooling, and to retrain its process analysts on a single notation. Assess the position the trade-finance solution is now in.

Setup. We consider what the solution's departure from the enterprise standard costs once the institution acts on the standard at scale.

Working. The trade-finance workflow lives in custom Java code, outside the standard engine and not expressed in BPMN. When the institution rolls out end-to-end observability through the standard tooling, that tooling — built for processes in the standard engine — cannot see the trade-finance workflow:

the trade-finance process is invisible to the enterprise tooling.

When the institution retrains its analysts on the standard notation, those analysts cannot read or maintain the trade-finance workflow, because it is not in that notation — it is in Java:

the trade-finance process is unmaintainable by the standard team.

The solution that shipped on time is now an island: outside the enterprise's observability, outside its skill base, a special case requiring special handling indefinitely.

Discussion. The solution team made a locally rational choice — the code-based workflow genuinely was faster to build under time pressure — and the worked example is about the gap between local rationality and enterprise consistency. At the moment of the decision, the cost of ignoring the enterprise standard was invisible: the solution worked, it shipped, nothing broke. The cost materialised two years later, when the institution did exactly what an enterprise architecture exists to enable — act on its standards at scale — and the trade-finance solution turned out to be an island the enterprise programmes could not reach. This is the precise reason solution architecture must be consistent with enterprise architecture: the enterprise standard is not bureaucracy, it is the thing that makes enterprise-wide capabilities — common observability, a common analyst skill base, common governance — possible at all, and every solution that quietly departs from it removes itself from those capabilities and quietly erodes them for everyone. The discipline is not that the solution team should never have felt the time pressure; it is that the right response to the pressure was to build the workflow in

the standard engine and BPMN — which is consistent *and* fit for the problem — rather than to buy a few weeks by creating a permanent island. A solution architecture's consistency with the enterprise is not a constraint on the solution; it is the solution's ticket to every enterprise capability the institution will ever build.

Practical next steps

For a solution your institution has recently built, check whether its workflow is consistent with the enterprise architecture's orchestration standard — the standard engine, the standard notation — or whether time pressure produced an island. Islands are invisible to enterprise tooling and unmaintainable by the standard team. The cost is not felt on the day the island is built; it is felt every time the institution tries to do something enterprise-wide.

Chapter in brief

Solution architecture designs one specific solution to one specific business problem, where enterprise architecture's principles meet a real, bounded need. Every solution has a workflow, and the solution architect makes five deliberate decisions about it: where it lives, what kind of component does each step, what it integrates with and how, where its data sits, and how it is operated, secured, and evidenced. A solution architecture is pulled two ways — it must be consistent with the enterprise architecture and fit for the specific problem — and the balance is struck by letting the enterprise constrain at the level of principle while the solution architect designs freely at the level of detail. A solution that departs from the enterprise standard becomes an island, invisible to enterprise tooling and unmaintainable by the standard team.

Technology Architecture and the Workflow

19.1 The layer beneath the design

The previous two chapters worked at the level of *what* is built — the enterprise’s capabilities, the solution’s design. This chapter treats the layer beneath: *technology architecture*, the discipline of the concrete technologies, platforms, and infrastructure on which everything runs, and the standards that govern them.

Technology architecture is where the abstract becomes the operational. The solution architecture may specify “the workflow runs in the standard process engine”; the technology architecture decides what that means in running-system terms — which version of the engine, on which runtime platform, with which databases, networking, identity infrastructure, and operational tooling, conforming to which technology standards. Much of this manuscript’s **Part VII** was technology architecture for the engine specifically; this chapter treats how the workflow fits into the institution’s *technology architecture as a whole*.

19.2 The workflow's technology footprint

A workflow, however cleanly designed at the solution level, has a real and specific *technology footprint* — a set of concrete technologies it requires — and the technology architect’s job is to know that footprint and fit it into the institution’s technology estate.

The footprint of a process-engine workflow includes: the *engine runtime* itself, with its stateful and stateless components; the *datastores* it depends on — the engine’s own storage and the secondary store; the *runtime platform* it is deployed on, typically Kubernetes; the *networking* that connects its components and exposes its interfaces; the *identity infrastructure* it authenticates against; and the *operational tooling* — monitoring, logging, alerting — that watches it. Every one of these is a technology choice, and every one must be consistent with the institution’s technology standards: its standard Kubernetes platform, its standard databases, its standard identity provider, its standard observability stack.

The technology architect’s discipline is to recognise that a workflow is not weightless. A solution architecture that adds “a process engine” adds, in technology terms, a substantial running system with the footprint above — and that footprint must be planned, standardised, and fitted, not discovered after the solution is approved.

Figure 19.1 sets out the technology footprint a workflow brings.

19.3 Standards, and the cost of the one-off

The technology architecture’s most valuable contribution is *standardisation* — and the workflow is a good place to see why.

If the institution standardises the technology of its process workflows — one standard

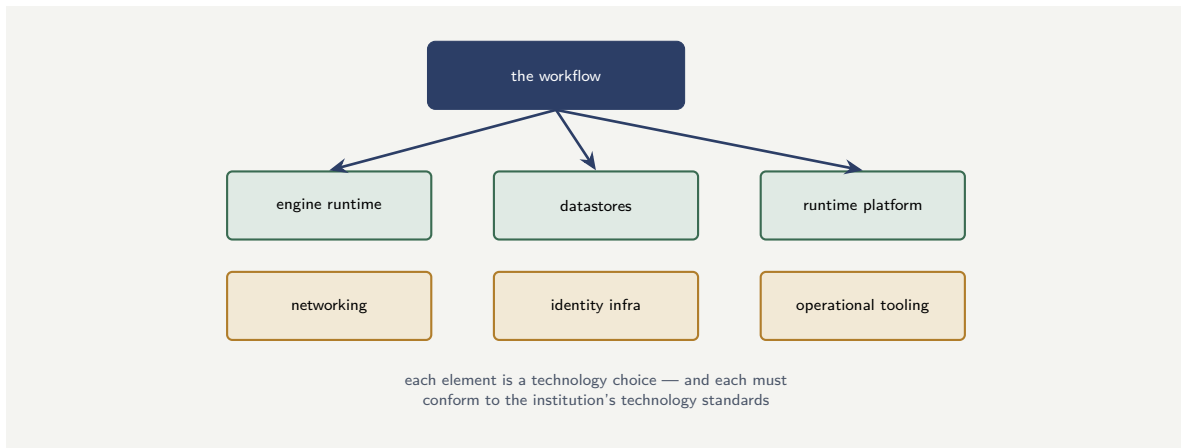


Figure 19.1. A workflow’s technology footprint. A process-engine workflow is not weightless: it requires an engine runtime, datastores, a runtime platform, networking, identity infrastructure, and operational tooling — each a concrete technology choice that must conform to the institution’s technology standards.

engine version, one standard way to deploy it on the standard runtime platform, one standard secondary store, one standard observability integration — then every workflow solution draws on a known, supported, repeatable technology base. The platform team builds the operational expertise once. Upgrades, security patches, and infrastructure improvements apply across all workflows at once. A new workflow solution is a known quantity to operate.

If the institution does not — if each workflow solution chooses its own engine version, its own deployment approach, its own datastores — then every workflow is a technology one-off. Each must be operated specially, patched specially, understood specially; the platform team’s expertise does not compound; and the institution accumulates a sprawl of slightly-different process technology that is expensive to run and dangerous to leave un-patched. The technology architect’s standardisation is what converts “many workflows” from a sprawl into a fleet.

This connects directly to [Chapter 58](#)’s argument that infrastructure must be a governed artefact. Technology standardisation is that governance at the enterprise scale: not only is each workflow’s infrastructure declared and versioned, but the *standard* the infrastructure conforms to is itself an enterprise-governed decision.

Without a technology standard for process workflows, each workflow solution becomes a technology one-off — its own engine version, deployment, datastores, tooling. The institution then cannot patch, upgrade, or operate its workflows as a fleet; the platform team’s expertise does not compound; and an un-patchable sprawl of slightly-different process technology accumulates. Standardise the workflow’s technology footprint at the enterprise level — one engine, one deployment pattern, one observability integration — so that many workflows are a fleet, not a sprawl.

19.4 Architecture as a living artefact

An architecture is not a document produced once and filed; it is a *living artefact* that must be maintained as the platform evolves. The reference architecture of [Chapter 14](#) sets the target;

the solution and technology architectures of this part realise it; and all three must be updated as the platform grows, as new integrations are added, and as the institution's regulatory and operational landscape shifts. An architecture diagram that does not match the running platform is not an architecture; it is a historical record of a platform that no longer exists.

Worked example: the workflow version sprawl

Problem. An institution has, over six years, built fourteen process workflows across eleven teams, with no enterprise technology standard for them. The result: the fourteen workflows run across six different engine versions, four different deployment approaches, and three different secondary-store technologies. A serious security vulnerability is then announced in the process engine, requiring an urgent patch. Assess what the institution must now do, and what a technology standard would have changed.

Setup. We consider the patching effort the sprawl creates and contrast it with a standardised estate.

Working. The urgent patch must reach all fourteen workflows. Because they run across six engine versions and four deployment approaches, the patch is not one operation but many: the patch must be obtained or back-ported for each of the

6 engine versions,

and applied through each of the

4 deployment approaches,

by teams whose process technology differs workflow to workflow. Some old engine versions may no longer receive the patch at all, leaving those workflows

vulnerable with no straightforward fix.

With a technology standard. All fourteen workflows would run one standard, supported engine version on one deployment pattern. The patch is *one* operation, applied once to the standard, rolling out to all fourteen — and no workflow is stranded on an unsupported version.

Discussion. The sprawl did not announce itself as a problem during the six years it accumulated — each of the fourteen workflows worked, and each team's local choice of engine version and deployment was locally reasonable. The cost arrived all at once, on the day of the security patch, and the worked example is built to show that the technology architecture's standardisation is precisely an investment against that day. A standardised estate turns the urgent patch into a single, well-understood operation; the sprawl turns it into fourteen special cases across six versions, with some workflows possibly stranded on engine versions too old to patch — a genuine security exposure created not by any single bad decision but by the *absence* of a technology standard. The lesson generalises beyond patching: every enterprise-wide technology action — upgrades, infrastructure migration, observability roll-out — is cheap against a standardised fleet and agonising against a sprawl. The technology architect's contribution is unglamorous and easy to defer, because its value is invisible until an enterprise-wide action is needed — but a financial institution *will* need to patch its engine urgently, will need to upgrade, will need to migrate, and the technology standard is what makes those survivable. Standardising the workflow's technology footprint is not tidiness; it is the institution's ability to act on its process estate as one thing.

Practical next steps

Find out how many distinct process-engine versions, deployment approaches, and datastore technologies your institution's workflows actually run on. If the answer is more than one of each, the institution has a technology sprawl rather than a fleet — and the next urgent patch or upgrade will be paid for in proportion to that sprawl. A technology standard for the workflow footprint is the investment that makes enterprise-wide action on the estate possible.

Chapter in brief

Technology architecture is the discipline of the concrete technologies, platforms, and infrastructure on which everything runs. A workflow has a real technology footprint — engine runtime, datastores, runtime platform, networking, identity infrastructure, operational tooling — and each element is a technology choice that must conform to the institution's standards. The technology architecture's most valuable contribution is standardisation: a standard engine version, deployment pattern, secondary store, and observability integration turn many workflows from an un-patchable sprawl into an operable fleet. Without it, every workflow is a technology one-off, and every enterprise-wide action — patching, upgrading, migrating — becomes many special cases instead of one operation.

BIAN, the Service Landscape, and the Workflow

20.1 An industry reference model for banking

The three preceding chapters treated architecture at the enterprise, solution, and technology levels in general terms. This chapter treats a specific, banking-industry instrument that an architect in a bank will very likely encounter: *BIAN* — the Banking Industry Architecture Network — and its *Service Landscape*. The chapter explains what BIAN is, and, in keeping with the part, where a process engine and a workflow fit in relation to it.

BIAN is an industry body that has produced a *reference model* for the business architecture of a bank: a standard, non-proprietary description of the capabilities a bank has and how they are organised. It exists so that banks, their software vendors, and their partners can share a common, consistent vocabulary for what a bank *does* — rather than every bank and every vendor inventing its own.

20.2 Service domains and the Service Landscape

The core of BIAN is the *Service Landscape*, and its central building block is the *service domain*.

A *service domain* is, in BIAN's terms, a discrete, non-overlapping building block of business capability — the most fine-grained capability in the model. Each service domain represents one coherent thing a bank can do: *Customer Offer*, *Payment Execution*, *Consumer Loan*, *Collateral Administration*, *Fraud Resolution*. The defining discipline is that the service domains do *not overlap* — each capability of the bank has exactly one home — and that they offer their services to one another, so a service domain can fulfil its own work partly by calling others.

The *Service Landscape* is the organised whole: the full collection of service domains, grouped — into business domains, and broader business areas — so that the complete functional picture of a bank can be navigated. BIAN is explicit that the Service Landscape is a *reference framework*, not a design blueprint for any particular bank — it is a map of the territory from which a given bank derives its own specific architecture.

BIAN also standardises how service domains *interact*: through *semantic APIs*, a consistent style of interface so that a service in one domain can be called the same way regardless of which domain or which vendor provides it. And in implementation terms, a service domain maps naturally onto a microservice — one service domain, one service, owning its data.

Figure 20.1 shows the workflow orchestrating across BIAN service domains.

20.3 Where the workflow fits: BIAN says what, the engine says how

BIAN and a process engine are often spoken of as alternatives, as if an institution must choose between “a BIAN architecture” and “a process-orchestration architecture.” That framing is

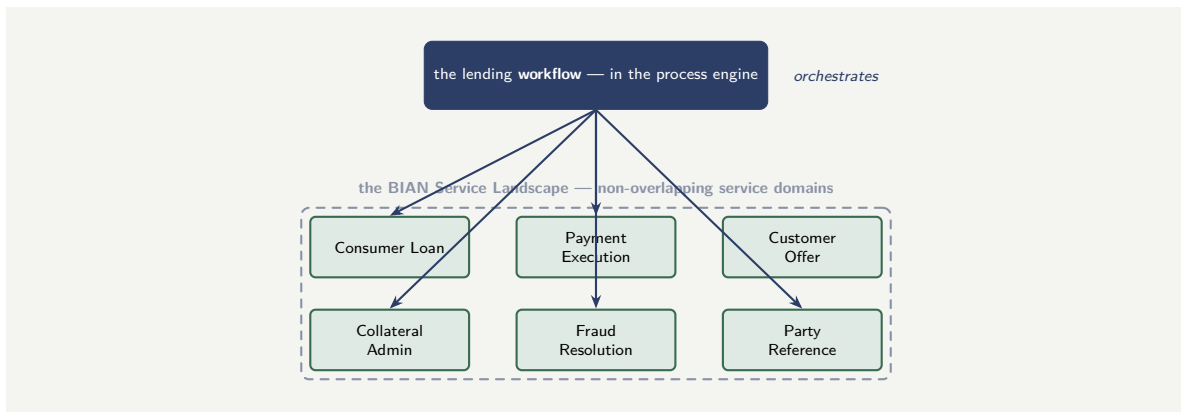


Figure 20.1. BIAN’s Service Landscape and the workflow. The service domains are discrete, non-overlapping building blocks of banking capability; the workflow, held in the process engine, orchestrates them — calling each service domain to do its part of an end-to-end journey. BIAN says what the capabilities are; the workflow says how a journey traverses them.

wrong, and correcting it is the heart of this chapter. BIAN and the process engine answer *different questions*, and a sound architecture uses both.

BIAN answers *what the bank’s capabilities are and how they are divided*. It gives the institution a disciplined, non-overlapping decomposition of itself into service domains — a clean answer to “what are the building blocks, and which block owns what.” What BIAN deliberately does *not* do is describe the *end-to-end business processes* — the journeys that cut across many service domains. A mortgage origination touches Customer Offer, Consumer Loan, Collateral Administration, Party Reference, Fraud Resolution, and more — and the *flow* across those domains, the sequence and the routing and the waiting and the human steps, is not what a service-domain decomposition expresses.

That flow is exactly what the process engine and the workflow express. The workflow is the *orchestration across service domains*: the mortgage process, modelled in the engine, calls Customer Offer, then Consumer Loan, then Collateral Administration, in the order and with the routing the journey requires. BIAN tells the architect *what* the well-formed building blocks are; the workflow says *how* an end-to-end journey traverses them.

This is the cleanest possible division of labour, and it makes the two mutually reinforcing. A workflow that orchestrates BIAN service domains is calling well-defined, non-overlapping, semantically-standardised capabilities — the best possible things for a workflow to call. And a set of BIAN service domains gains its end-to-end business value only when something orchestrates them into journeys — which is the workflow’s whole purpose. BIAN without orchestration is a tidy set of capabilities that no artefact assembles into a customer journey; orchestration without a BIAN-like decomposition is a workflow calling a tangle of overlapping, inconsistently-defined systems. Together — the workflow orchestrating across clean service domains — they are the disciplined architecture each on its own only half-describes.

Tip

BIAN and a process engine are not alternatives — they answer different questions. BIAN’s Service Landscape says *what* a bank’s capabilities are, decomposing the bank into discrete, non-overlapping service domains. The workflow in the process engine says *how* an end-to-end journey traverses those domains — the flow, routing, waiting,

and human steps that a service-domain decomposition deliberately does not express. Use BIAN to define the building blocks and the process engine to orchestrate across them; the two are mutually reinforcing, not competing.

20.4 Architecture as a living artefact

An architecture is not a document produced once and filed; it is a *living artefact* that must be maintained as the platform evolves. The reference architecture of [Chapter 14](#) sets the target; the solution and technology architectures of this part realise it; and all three must be updated as the platform grows, as new integrations are added, and as the institution's regulatory and operational landscape shifts. An architecture diagram that does not match the running platform is not an architecture; it is a historical record of a platform that no longer exists.

Worked example: mapping a mortgage journey onto BIAN

Problem. A bank has adopted BIAN and decomposed itself into service domains. An architect must design the mortgage-origination solution. A colleague argues that because the bank “has a BIAN architecture,” the mortgage process does not need a separate orchestration layer — the service domains can simply call one another as the process requires. The architect disagrees. Set out the architect's reasoning, using the structure of a mortgage journey.

Setup. We consider what a mortgage journey requires and whether service domains calling one another can provide it.

Working. The mortgage journey, in outline, traverses several service domains: *Customer Offer* (present and agree the mortgage offer), *Party Reference* and identity verification, *Consumer Loan* (set up the loan), *Collateral Administration* (register the property as security), *Fraud Resolution* (screen the application), and others. The journey is not a single call — it is a *flow*: a sequence, with routing (a referral if the application is non-standard), with waiting (for a valuation, for a human underwriting decision, which may take days), with parallel work (several checks at once), and with compensation (if a late step fails, undo the earlier ones). If the service domains simply call one another:

the flow is smeared across the service domains' code,

exactly the leaked-orchestration problem of [Chapter 14](#) — no single artefact holds the mortgage journey, no one can see where an application stands, and changing the journey means changing several service domains. The journey needs an *orchestrator*:

the workflow, in the engine, holds the mortgage flow; it calls the service domains.

Discussion. The architect is right, and the colleague's error is the exact misconception this chapter exists to correct: treating BIAN as if a decomposition into capabilities were also a description of processes. It is not, and BIAN itself does not claim it is — the Service Landscape is explicitly a reference framework for *capabilities*, not a blueprint of *journeys*. A mortgage origination is a long-running, routed, waiting, human-involved flow across a dozen service domains, and that flow has to live *somewhere*. The colleague's proposal — service domains calling one another — puts the flow nowhere: it is smeared across the domains' code, which is the leaked-orchestration anti-pattern, and it leaves the bank's most important journey with no explicit model, no visibility, no governed home. The architect's design keeps BIAN doing

what BIAN is excellent at — defining clean, non-overlapping service domains for the workflow to call — and adds the process engine doing what only an orchestrator can: holding the mortgage journey as one explicit, governed, visible model. The worked example's lesson is the chapter's central point made concrete: BIAN and the workflow are not competitors, and an architecture that has BIAN service domains but no orchestration has a beautiful set of building blocks and no description of how the bank's journeys are actually built from them. The mortgage process is the flow across the domains, and the flow belongs in the engine.

Practical next steps

If your bank uses BIAN, look at one major customer journey and check where its *flow* across the service domains lives. BIAN defines the service domains well — but the journey across them is a process, and if that process is not an explicit model in an orchestrator, it is smeared across the service domains' code. BIAN says what the capabilities are; make sure something says how the journeys traverse them.

Chapter in brief

BIAN — the Banking Industry Architecture Network — provides a standard reference model for a bank's business architecture. Its core is the Service Landscape, built from *service domains*: discrete, non-overlapping building blocks of banking capability, interacting through semantic APIs and mapping naturally onto microservices. BIAN and a process engine are not alternatives: BIAN says *what* the bank's capabilities are and how they divide, while the workflow in the engine says *how* an end-to-end journey traverses those capabilities — the flow, routing, waiting, and human steps a service-domain decomposition deliberately does not express. The disciplined architecture uses both: the workflow orchestrating across clean BIAN service domains, each defining what the other only half-describes.

PART IV

Architecture

The Camunda Orchestration Core

21.1 What the orchestrator is responsible for

The orchestration core is the component that holds the process model and runs it. Everything else in a hyperautomation architecture — the decision engine, the agent runtime, the integrations, the user interfaces — is something the orchestrator calls or hands work to. It is worth being precise about what the core is responsible for, because architectures fail when responsibilities leak across the boundary.

The orchestrator owns four things. It owns *process state*: for every live instance, which token sits at which activity, what data the instance carries, and what it is waiting for. It owns *flow control*: deciding, when an activity completes, which sequence flow to follow, which gateway condition to evaluate, which path to take. It owns *task distribution*: placing human work on the right worklist and offering automated work to the right worker. And it owns the *audit record*: the durable, ordered history of everything that happened to every instance.

The orchestrator is deliberately *not* responsible for business logic. It does not contain the credit rules — those are in DMN. It does not contain the fraud model — that is a service. It does not contain the reasoning that reads an adjuster's note — that is an agent. The core's job is to know the process, hold the state, and route the work. Keeping it that thin is what keeps it governable: a thin orchestrator with externalised logic can be reasoned about, while a fat orchestrator with business logic smeared through it becomes the unmaintainable monolith that hyperautomation was supposed to replace.

Figure 21.1 places the orchestrator at the centre with the components it calls around it. The shape of the picture is the architecture's central principle made visible: the core is connected to everything and contains none of it.

21.2 The execution model: tokens, jobs, and persistence

It helps to understand, at least in outline, how a BPMN model becomes running behaviour, because the mechanics explain the operational properties that matter.

A running process instance is represented by one or more *tokens*. A token sits on an element of the model; when that element completes, the token moves along a sequence flow to the next element. A parallel gateway creates several tokens; a joining gateway consumes them. The set of token positions is the instance's state, and the engine *persistent* that state durably — to a database or a comparable store — so that an instance which is waiting, or an instance caught mid-flight by a restart, survives without loss.

Automated work is handled through *jobs*. When a token reaches a service task, the engine creates a job — a unit of work — and makes it available. A *job worker*, a separate process, picks the job up, does the work (calls an integration, evaluates a rule, invokes an agent), and reports completion back to the engine, which then moves the token on. This separation matters: the

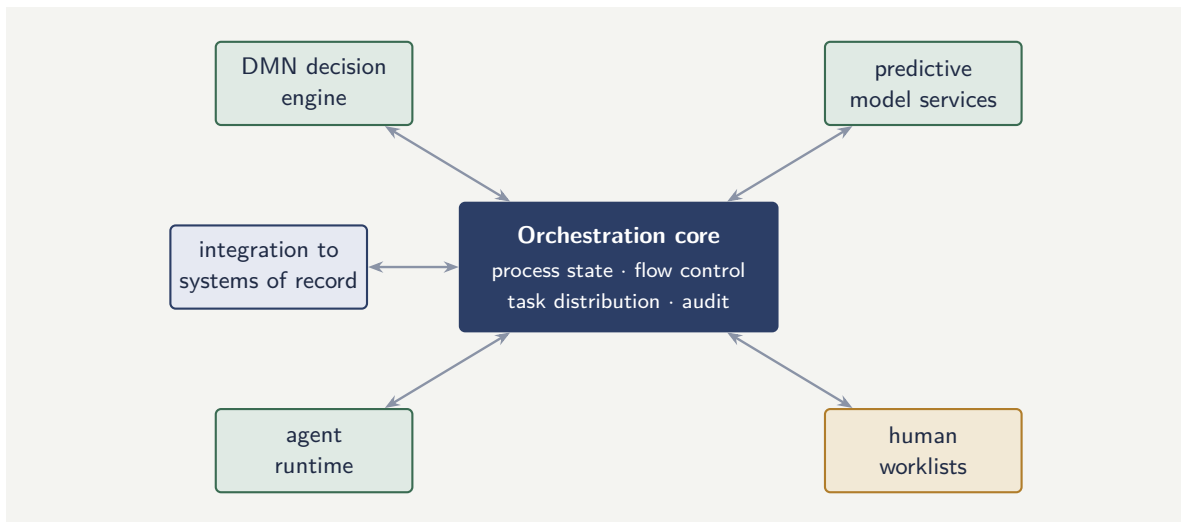


Figure 21.1. The orchestration core and the components around it. The core holds the process and routes the work; every piece of business logic — decisions, models, agents — lives in a surrounding component the core calls. Keeping the core thin is what keeps it governable.

engine is not blocked while the work runs, the worker can be scaled independently, and a worker crash simply means the job is retried rather than the process being lost.

Tip

The job-worker model is why an orchestrated architecture degrades gracefully. If an integration is slow or down, jobs queue rather than processes failing; when the integration recovers, the workers drain the queue. Design every automated touchpoint as a job with a defined retry policy and a defined *failure* path, and the platform absorbs transient faults instead of amplifying them.

21.3 Modern orchestration: from embedded engine to platform

Orchestration technology has moved through two generations, and the distinction shapes the architecture.

The first generation embedded the workflow engine as a library inside an application: the engine and the application shared a process and a database. This was simple to start with and scaled poorly — the engine could not be scaled independently of the application, and the shared database became the bottleneck.

The current generation runs orchestration as a distinct, horizontally scalable platform. The engine is a clustered service; job workers connect to it over the network; the model is deployed to the platform rather than compiled into an application. This is the form that suits a cloud deployment and the scale of a bank or insurer, and it is the form this manuscript assumes. [Chapter 23](#) treats how agents connect to such a platform, and [Chapter 24](#) treats running it on cloud infrastructure.

21.4 Decomposition as architecture

Chapter 3 introduced call activities as a readability device. At the architecture level, decomposition is more than that — it is how the orchestration estate is structured for governance and reuse.

The principle is to model the enterprise's processes as a layered set of models, not a flat sprawl. At the top, a small number of *journey* models describe end-to-end customer journeys — account opening, borrowing, claiming. Each journey is a short model of call activities. Each call activity invokes a *capability* model — verify identity, assess affordability, detect fraud, disburse funds — that is self-contained, separately owned, separately versioned, and reusable across journeys. The capability models in turn may call shared *utility* models for cross-cutting steps such as notification or document generation.

This layering gives the estate the property that matters most: *changeability with bounded blast radius*. A change to the affordability-assessment capability is reviewed and redeployed as one capability model; the journeys that call it are unaffected unless its interface changes. The alternative — one giant model per journey, with logic duplicated across them — means every change is an enterprise-wide change. **Chapter 120** onward models real journeys this way.

21.5 Versioning and the problem of the in-flight instance

A process model changes over its life: a step is added, a rule is moved, a path is rerouted. The orchestration core must therefore version process models — and the genuinely difficult question is not how to deploy a new version but what happens to the instances already running under the old one.

A financial process is long-running, as **Chapter 3** established. When a new version of a lending process is deployed, there may be tens of thousands of applications part-way through the previous version: some waiting for documents, some at a credit decision, some at an offer. They cannot simply be abandoned, and they cannot all be forced instantly onto the new model, because an instance halfway through the old model may be at a step the new model no longer has.

The orchestration core's answer is that a deployed model version is *immutable*, and a running instance is bound to the version it started under. New instances begin on the new version; existing instances continue, to completion, on the version they began. The two versions run side by side until the last old-version instance finishes. This is the safe default, and for most changes it is the right one: it guarantees that no instance ever finds itself stranded at a step that no longer exists.

Sometimes, though, an institution must do more — a regulatory change may require that even in-flight instances adopt new logic immediately. The orchestrator then supports a deliberate *migration*: a controlled, explicitly mapped move of running instances from one version to another, specifying for each old-model state where its instances land in the new model. Migration is powerful and is treated as a governed operation in its own right, because a careless migration can place an instance into an inconsistent state. The discipline is to prefer the safe default — let old instances drain on their old version — and to treat migration as an exceptional, carefully designed and reviewed action, not a routine one.

Tip

When you change a process model, your first question should be not “how do I deploy this?” but “what happens to the instances already running?” For most changes, the answer is to let them complete on the version they began — deploy the new version for new instances and let the old one drain. Reserve explicit instance migration for the cases, usually regulatory, where in-flight instances genuinely must change behaviour, and govern that migration as the consequential operation it is.

21.6 The engine as the institution's process memory

A workflow engine's state store is, in a real sense, the institution's *process memory*: it knows, for every case in progress, where it is, what has been done, what is outstanding, and what the data says. This memory survives restarts, deployments, and failures — it is durable, queryable, and auditable. An institution without a workflow engine has process memory only in the heads of the people doing the work, in spreadsheets, and in email threads — none of which survives a staff change, none of which a regulator can query.

Worked example: sizing the orchestration cluster

Problem. An insurer expects its orchestration platform to run, at peak, 50 process instances started per second. Each instance, on average, traverses 40 BPMN elements over its life, and each element transition costs the engine roughly one persisted state update. The engine's data store is benchmarked to sustain 12,000 state updates per second per node with acceptable latency. The team wants the number of engine nodes needed at peak, with 40% headroom above the raw requirement.

Setup. The raw load is instances per second times elements per instance times updates per element. The required capacity is the raw load divided by per-node capacity, then multiplied by the headroom factor, and finally rounded up to a whole number of nodes.

Working. Raw state-update load at peak:

$$50 \frac{\text{instances}}{\text{s}} \times 40 \frac{\text{elements}}{\text{instance}} \times 1 \frac{\text{update}}{\text{element}} = 2,000 \frac{\text{updates}}{\text{s}}.$$

Nodes needed before headroom:

$$\frac{2,000}{12,000} \approx 0.167 \text{ nodes.}$$

Apply the 40% headroom factor (multiply by 1.40):

$$0.167 \times 1.40 \approx 0.233 \text{ nodes.}$$

Rounding up to a whole node gives **1** engine node to meet the throughput requirement.

Discussion. The arithmetic delivers a deliberately uncomfortable result: a single node carries the throughput with room to spare. The lesson is that for most banking and insurance workloads the orchestration engine is *not* throughput-bound — the state-update rates involved are modest by database standards. Yet no sensible production deployment runs a single node, and the reason is the second requirement that the calculation does not capture: *availability*. A single node is a single point of failure; the platform needs at least two, and commonly three, nodes not for throughput but so that the loss of one node does not stop every

process in the institution. This is a recurring pattern in **Chapter 24**: cloud sizing for these platforms is driven far more by resilience and failure-domain spread than by raw capacity, and a sizing exercise that reports only throughput has answered the easy half of the question.

Practical next steps

Estimate the peak instances-per-second and average elements-per-instance for the busiest process you intend to orchestrate. Carry out the calculation above. Then write the *availability* requirement separately: how many node and zone failures must the platform survive, and what is the maximum tolerable recovery time? The second number, not the first, will size your cluster.

Chapter in brief

The orchestration core owns process state, flow control, task distribution, and the audit record — and deliberately owns no business logic, which lives in DMN, in services, and in agents. It runs BPMN through a token-and-job model: token positions are the persisted state, and independently scalable job workers do automated work, which is what makes the platform degrade gracefully under load. Modern orchestration is a clustered platform rather than an embedded library. Decomposition into layered journey, capability, and utility models gives the estate changeability with bounded blast radius. Orchestration clusters are sized by availability and failure-domain spread, not by raw throughput.

Decision Services and Predictive Models

22.1 Two kinds of decision, one integration pattern

Chapter 4 drew the line between a *rule-based* decision, which belongs in a DMN table, and a *predictive* decision, which belongs in a machine-learning model. This chapter is about how the predictive model takes its place in the architecture, and the central point is that — from the orchestrator’s perspective — both kinds of decision look the same.

The orchestrator reaches a point in the process where a question must be answered. It does not care whether the answer comes from a decision table evaluating hand-written rules or from a neural network scoring a feature vector. In both cases the orchestrator’s behaviour is identical: it provides the inputs, it invokes a *decision service*, it receives a structured result, and it routes on that result. The rule engine and the predictive model are both decision services behind a common contract. This uniformity is what keeps the process model clean: the BPMN diagram shows a decision being made, not the machinery behind it, and the machinery can change — a rule table replaced by a model, a model retrained — without the process model changing.

22.2 What a predictive decision service must provide

A predictive model used inside a regulated process is not just a function from features to a score. To be usable as a decision service it must provide several things beyond the score itself.

It must provide a *calibrated* score wherever the score will be treated as a probability. A fraud model that outputs 0.8 should mean that, among cases it scores 0.8, about 80% truly are fraud. An uncalibrated score can still rank cases, but it cannot be combined with monetary amounts or fed into a DMN threshold sensibly, because the number does not mean what it appears to mean.

It must provide a *version*. Every score the model returns must be attributable to a specific, immutable version of the model, because the institution will later need to explain a specific past decision, and the explanation depends on which model made it.

It must provide *inputs-as-recorded*. The exact feature values the model saw must be capturable alongside the score, because an explanation or a challenge six months later cannot rely on reconstructing what the data looked like at decision time.

And it should provide a *reason or attribution* where the downstream process or a regulator will need one — an indication of which features drove the score — so that the decision is not merely a number but a number with a rationale. **Chapter 127** treats explainability in depth; the architectural point here is that these obligations are part of the decision service’s contract, not optional extras.

A predictive model wired directly into a gateway condition — “route to investigation if fraud score above 0.7” — buries a model inside the process model and inherits none of the governance DMN provides. Prefer the pattern where the model service returns the score, and a DMN decision table consumes that score alongside other inputs to produce the routing. The model predicts; the table, which is versioned, auditable, and owned, decides.

22.3 The model lifecycle behind the service

A decision service backed by a predictive model is the visible tip of a lifecycle that the architecture must support, even though the orchestrator never sees it.

A model is *trained* on historical data, *validated* against held-out data and against fairness and stability criteria, *reviewed* by a model-risk function, *deployed* as a service version, *monitored* in production for drift and degradation, and eventually *retired* or *retrained*. Each stage produces governance artefacts. The platform must hold those artefacts and link them to the model version, so that any decision the orchestrator records can be traced to a model, and the model to its validation and approval. [Chapter 126](#) treats this lifecycle as the discipline of model risk; the architecture’s responsibility is to make the lifecycle’s outputs first-class, queryable data rather than documents in a folder.

22.4 Features: the data a model actually sees

A predictive model does not consume raw records; it consumes *features* — the derived quantities computed from raw data that the model was trained on and scores against. A credit-risk model does not see a customer’s transaction history as such; it sees features computed from it: the average inflow over three months, the count of missed payments in a year, the ratio of committed outgoings to income. The quality and the consistency of those features determine the quality of the model’s score, and the architecture must treat feature management as a first-class concern, because two failure modes follow directly from neglecting it.

The first is *training-serving skew*. A feature is computed one way when the model is trained — often in an analytical environment, over historical data, by a data scientist — and must be computed the *identical* way when the model is scored in production, inside the running process. If the two computations differ even slightly — a different handling of missing values, a different time window, a subtly different definition — the model in production sees features unlike those it learned from, and its scores are quietly wrong. The defence is a single, shared definition of each feature, used by both training and serving, so that the feature a model was trained on and the feature it scores are provably the same computation.

The second is *point-in-time correctness*. A model scoring a case must see the features as they stood *at the moment of the decision*, not as they stand now. A model that, in training, was accidentally given a feature computed from data that only became available after the outcome it was predicting — a subtle form of leakage — will appear accurate in validation and fail in production. And a model rescoreing a historical case for an explanation must see the features as they were then. The architecture’s answer is a feature store or an equivalent discipline: features are computed, versioned, and recorded with the time they were valid, so that both training and explanation can reconstruct the exact inputs a decision was made on.

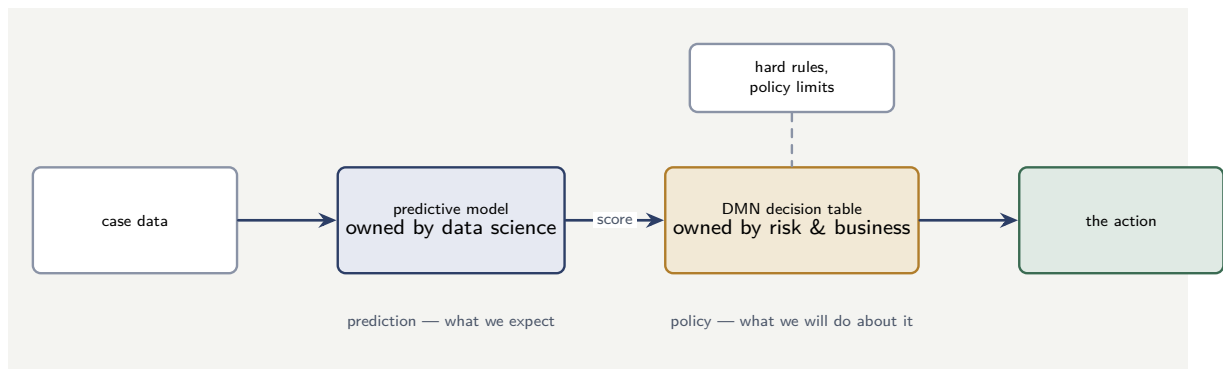


Figure 22.1. The layered decision. A predictive model turns case data into a score — the prediction the institution could not derive from rules — and a DMN decision table consumes that score together with hard rules and policy limits to produce the action. The model contributes prediction; the table contributes governed, auditable policy.

Tip

Insist on one definition of each feature, shared between training and production scoring, and on point-in-time correctness for every feature a model consumes. Most “the model worked in validation but fails in production” incidents are not model failures at all — they are feature failures: training-serving skew or leakage. The model is only as sound as the features beneath it, and features are an architecture responsibility, not a detail left to whoever builds each model.

22.5 Where the score meets the rule

The most robust pattern in financial-services hyperautomation is the layered decision: a predictive model produces a score; a DMN table consumes the score together with hard rules and policy constraints; the table produces the action. The model contributes *prediction* — a quantified expectation the institution could not derive from rules. The table contributes *policy* — the deterministic, auditable, owned logic that says what the institution will actually do given that prediction.

This separation is not bureaucratic. It places each kind of logic where it can be governed by the people who own it. The data science function owns the model and its accuracy. The business and risk functions own the table and the policy. A regulator examining the decision sees a clear two-part answer: here is what we predicted, and here is the explicit rule by which we acted on the prediction. [Chapter 122](#) and [Chapter 123](#) use this layered pattern repeatedly in underwriting and claims.

Figure 22.1 draws the layered decision — model into table into action.

22.6 External rules engines and the process

Not every decision in a banking or insurance process is expressed in DMN. Many institutions run dedicated *rules engines* — platforms whose sole purpose is to evaluate complex business rules at scale — and a Camunda process must integrate with them rather than replace them.

The landscape of external rules engines

Three external rules engines dominate the financial-services landscape, and a practitioner integrating Camunda must know each.

Drools (Red Hat) is the leading open-source rules engine, part of the KIE (Knowledge Is Everything) platform. It uses its own rule language (DRL) or decision tables, supports forward- and backward-chaining inference, and runs inside the JVM. Many institutions use Drools alongside Camunda: the process orchestrates the journey, and a service task invokes the Drools rule session for a complex decision that exceeds what a single DMN table can express — anti-money-laundering screening with hundreds of interdependent rules, for example, or insurance underwriting with actuarial tables that change monthly.

IBM Operational Decision Manager (ODM) is the enterprise rules platform in the IBM ecosystem. ODM separates rules into a *decision service* deployed on its own runtime, reached over REST, with a rule-management UI that lets business analysts author and test rules without a developer. Institutions running IBM middleware — BAW, MQ, DataPower — often have ODM already, and the Camunda integration is a REST call from a job worker to the ODM decision service endpoint.

FICO Blaze Advisor (now FICO Platform) is the rules engine most common in credit decisioning and fraud detection. It is optimised for high-throughput, low-latency rule evaluation against large rule sets — exactly the profile of a credit-scoring or fraud-screening decision that must evaluate in milliseconds. Many banks' credit-decision and fraud-detection rules live in Blaze Advisor, and the Camunda integration reaches it through its decision service API.

Other engines a practitioner may encounter include *Oracle Policy Automation* (OPA), used in insurance and government; *Pegasystems'* rules engine, embedded in the Pega platform; and *InRule*, a .NET-native rules engine used in some North American institutions.

Integrating a Camunda process with an external rules engine

The integration pattern is the same regardless of the engine: a *service task* in the BPMN model calls a *job worker*, and the worker invokes the external rules engine's decision service, passing the input data and receiving the decision result.

Listing 22.1 is a job worker calling a Drools KIE Server decision service. The worker sends the application data, the KIE Server evaluates the rules, and the worker returns the decision to the process.

Listing 22.1. A job worker invoking a Drools KIE Server decision service: the worker sends the input, Drools evaluates the rules, the worker returns the result.

```
1 @JobWorker(type = "evaluate-aml-rules")
2 public Map<String, Object> evaluate(ActivatedJob job) {
3     // build the KIE Server command
4     KieServicesClient client = KieServicesFactory
5         .newKieServicesClient(kieConfig);
6     RuleServicesClient rules = client
7         .getServicesClient(RuleServicesClient.class);
8
9     // the fact object the rules evaluate against
10    AmlScreeningRequest req = new AmlScreeningRequest(
11        job.getVariable("customerId"),
12        job.getVariable("transactionAmount"),
13        job.getVariable("countryCode"));
14
15    // fire the rules
16    KieContainerResource container =
```



```

17     new KieContainerResource("aml-rules-1.0");
18     ExecutionResults results = rules.executeCommands(
19         "aml-rules-1.0",
20         BatchExecutionFactory.newBatchExecution(
21             List.of(CommandFactory.newInsert(req),
22                 CommandFactory.newFireAllRules())));
23
24     AmlScreeningRequest fired =
25         (AmlScreeningRequest) results
26             .getValue("req");
27
28     return Map.of(
29         "amlRiskLevel", fired.getRiskLevel(),
30         "amlAlertRaised", fired.isAlertRaised());
31 }

```

Listing 22.2 is the same pattern against IBM ODM: the worker calls the ODM decision service over REST, sends the input as JSON, and receives the decision.

Listing 22.2. A job worker calling an IBM ODM decision service over REST: the same integration pattern, different engine.

```

1  @JobWorker(type = "evaluate-credit-policy")
2  public Map<String, Object> evaluate(ActivatedJob job) {
3      // ODM decision service reached over REST
4      String payload = objectMapper.writeValueAsString(
5          Map.of("borrower", Map.of(
6              "income", job.getVariable("annualIncome"),
7              "score", job.getVariable("creditScore"),
8              "dti", job.getVariable("debtToIncome"))));
9
10     HttpResponse<String> r = httpClient.send(
11         HttpRequest.newBuilder()
12             .uri(URI.create(odmEndpoint
13                 + "/DecisionService/rest/v1/"
14                 + "CreditPolicyRuleset/1.0/"
15                 + "CreditPolicyRuleset/1.0"))
16             .header("Content-Type", "application/json")
17             .POST(HttpRequest.BodyPublishers
18                 .ofString(payload))
19             .build(),
20         HttpResponse.BodyHandlers.ofString());
21
22     JsonNode decision = objectMapper
23         .readTree(r.body());
24
25     return Map.of(
26         "policyDecision",
27         decision.path("decision").asText(),
28         "maxApprovedAmount",
29         decision.path("maxAmount").asDouble());
30 }

```

The two listings make the architectural point concrete: the Camunda process does not know *which* rules engine evaluates the decision; it knows only that a service task of type “evaluate-aml-rules” or “evaluate-credit-policy” produces a result. Whether the rules live in Drools, ODM, Blaze Advisor, or the built-in Camunda DMN engine is a deployment decision, not a process-model decision — and that separation is the anti-corruption discipline of Chapter 25 applied to the decision layer.

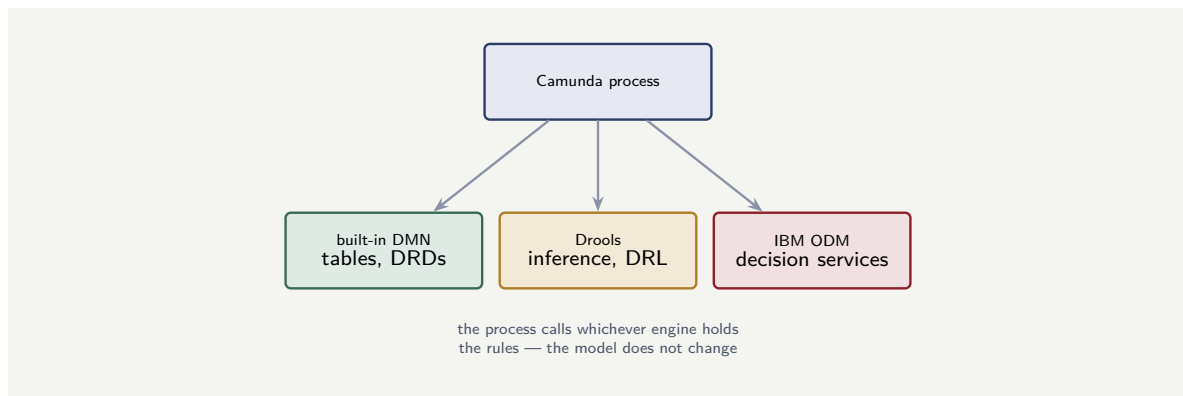


Figure 22.2. The process and the rules engines. The Camunda process calls whichever engine holds the rules for a given decision — the built-in DMN engine, Drools, IBM ODM, or another — through a job worker. The process model is engine-agnostic; changing the rules engine changes the worker, not the model.

Tip

When integrating an external rules engine — Drools, IBM ODM, FICO Blaze Advisor — keep the integration boundary clean. The job worker calls the engine’s decision-service API; the process model names a job type, not an engine. This lets the institution change engines — migrate from ODM to Drools, or from Drools to the built-in DMN engine — by replacing a worker, not by rewriting every process that makes a decision.

22.7 When to use the built-in DMN engine versus an external engine

The choice is governed by three factors.

Rule complexity. The built-in Camunda DMN engine handles decision tables and DRDs expressed in DMN and FEEL. For decisions expressible as tables — fee tiers, routing rules, eligibility checks — it is the right choice, because the rules are inside the same governance model as the process. For decisions that require hundreds of interdependent rules, forward-chaining inference, or actuarial-table lookups, an external engine (Drools for inference, Blaze Advisor for high-throughput scoring) is the right tool.

Ownership. If the rules are owned and maintained by a dedicated rules team with its own lifecycle — its own testing, its own deployment pipeline, its own release schedule — an external engine with its own management UI (ODM’s Decision Center, Drools’ Business Central) is the right home, because the rules team needs tools the process team does not provide. If the rules are owned by the same team that owns the process, the built-in DMN engine keeps everything in one place.

Existing estate. If the institution already runs an external engine with thousands of rules in production, those rules are not moving. The integration is the job-worker pattern this section described: the Camunda process calls the existing engine. Replacing a working, production-tested rules estate with newly-written DMN tables is a migration, not an integration, and it carries all the migration risks of [Chapter 76](#).

Figure 22.2 shows the architecture: the process model is engine-agnostic; the job worker is the integration seam; and the rules engine is a deployment choice.

22.8 Applying the chapter in practice

The principles this chapter describes are most valuable when applied deliberately and reviewed periodically. A team that applies them once, at initial design, captures their value at that moment; a team that returns to them at each major change, each annual review, and each post-incident analysis captures their value continuously. The platform evolves; the principles hold; the specific choices they inform must evolve with the platform.

Worked example: calibrating a fraud score against a threshold

Problem. A claims-fraud model outputs a score intended as a probability of fraud. On a validation sample, claims scored in the band 0.70 to 0.80 numbered 4,000, and of those, 2,600 were later confirmed fraudulent. The process routes a claim to manual investigation when the score exceeds 0.75. Investigation costs \$120 per claim; an undetected fraudulent claim costs the insurer an average of \$3,000. For claims in this score band, is routing to investigation justified on expected-cost grounds, and is the model well calibrated in this band?

Setup. Calibration in the band is checked by comparing the observed fraud rate with the band's nominal probability range. The investigation decision is justified if the expected loss avoided exceeds the investigation cost. Expected loss avoided per claim is the probability of fraud times the cost of an undetected fraudulent claim.

Working. Observed fraud rate in the 0.70–0.80 band:

$$\frac{2,600}{4,000} = 0.65.$$

The band's nominal range is 0.70 to 0.80, with midpoint 0.75. The observed rate of 0.65 sits *below* the band, so the model is *over-confident* here — it scores these claims as more likely fraud than they are.

Expected fraud cost per claim in the band, using the observed rate:

$$0.65 \times \$3,000 = \$1,950.$$

Since \$1,950 vastly exceeds the \$120 investigation cost, routing to investigation is justified by a wide margin.

Discussion. The two findings pull in different directions and both matter. On pure expected cost, investigation is overwhelmingly worthwhile — the \$120 check guards against a \$1,950 expected exposure, a ratio of more than fifteen to one — so the routing decision itself is sound. But the calibration finding is a warning that should not be lost in the comfortable economics: a model that claims 0.75 and delivers 0.65 is miscalibrated, and while the error is harmless at this threshold, the same over-confidence near a more finely-balanced threshold — say a pricing or a credit-limit decision — would cause real mistakes. The disciplined response is to recommend investigation routing now, on the economics, and *separately* to flag the model for recalibration before its score is reused in any decision where the exact probability, rather than merely its ordering, matters.

Practical next steps

For one predictive model already in use in your institution, pull a recent validation sample, pick the score band around the threshold that actually drives a decision, and compare the observed outcome rate with the band. If they diverge, the model is making decisions on a

number that does not mean what the threshold assumes — a finding worth more than a small accuracy improvement.

Chapter in brief

From the orchestrator's view, a rule-based DMN decision and a predictive model are the same thing: a decision service behind a common contract, invoked with inputs and returning a structured result. A predictive decision service must provide more than a score — a calibrated probability, a model version, the inputs as recorded, and an attribution — because those are the contract a regulated process needs. Behind the service sits a full model lifecycle whose governance artefacts the platform must keep as queryable data. The most robust pattern is layered: the model predicts, and a versioned, owned DMN table decides what to do with the prediction.

The Agentic Layer

23.1 What an agent adds that a model does not

A predictive model answers a fixed question: given these features, what is the score? It is invoked, it returns a number, and it has no latitude in how it gets there. An *agent*, in the sense this manuscript uses, is different in kind. An agent is given a *task* and a set of *tools*, and it decides — step by step, reasoning as it goes — how to use the tools to complete the task.

The difference matters at exactly the points where financial processes meet unstructured reality. Consider the task: *review this submitted claim pack and assess whether the described incident is internally consistent*. A predictive model cannot be pointed at this; there is no fixed feature vector and no single score. A human can do it, by reading the first-notification text, checking it against the policy wording, looking at the photographs, perhaps querying the prior-claims history, and forming a judgement. An agent can do the same: given the task, given tools to retrieve each of those documents, and given the ability to reason over what it retrieves, it can work through the task in steps and produce a reasoned assessment.

What the agent adds, then, is *open-ended task completion*: the ability to handle a step whose sub-structure is not known in advance, that requires pulling together heterogeneous information, and that ends in a judgement rather than a lookup. That is a genuinely new capability in the automation toolkit, and it is also, for exactly the same reasons, the most dangerous one.

23.2 Why the agent must be contained

An agent's strength — deciding for itself how to complete a task — is inseparable from its central weakness: it can decide wrongly, and it can do so *confidently and plausibly*. A miscalibrated predictive model returns a number that is wrong; a misfiring agent returns a fluent, well-argued, entirely incorrect assessment. In a regulated financial process, an automation that is confidently and plausibly wrong is more dangerous than one that simply fails, because the failure is not obvious.

This is why the architectural treatment of agents is dominated by *containment*. An agent is never the whole of a process and never the final word. It is placed inside an orchestrated process, where four containment mechanisms surround it.

Bounded scope. The agent is given one task with a defined input and a defined output schema, not an open mandate. It assesses consistency; it does not “handle the claim.”

Bounded tools. The agent can use only the tools the process gives it, and those tools are themselves governed — a retrieval tool that can read documents, not a tool that can move money. The agent's reach is the toolset, and the toolset is a design decision.

Deterministic adjudication. The agent's output is an input to a subsequent deterministic step — a DMN table, a rule, a human task — which decides what actually happens. The agent recommends; something governable disposes.

Human accountability. For any consequential decision, a human task sits on the path, and a named human is accountable for the outcome. The agent informs that human; it does not replace them.

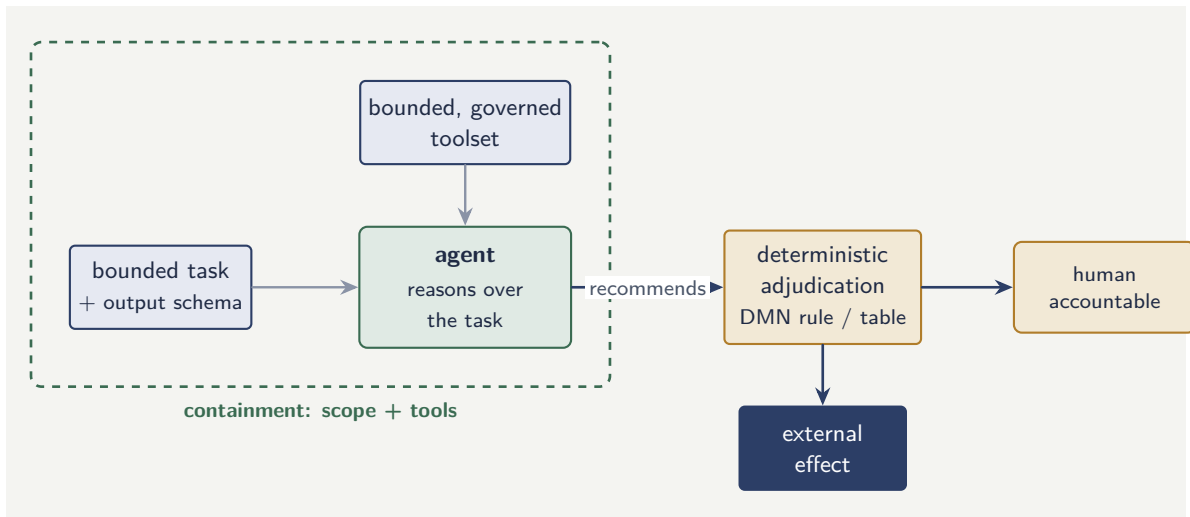


Figure 23.1. The four containments around an agent. Bounded scope and a bounded toolset constrain what the agent can do; a deterministic adjudication step consumes its recommendation; and a human is accountable for the consequential outcome. No external effect is ever the agent’s output directly.

The single most important rule in this manuscript: an agent’s output is never directly an action with external consequence. It is always an input to a deterministic step — a rule, a decision table, or a human task — that adjudicates it. An agent that can move money, bind cover, or close a claim without a deterministic gate between its reasoning and the effect is an ungoverned agent, and an ungoverned agent does not belong in a regulated process.

Figure 23.1 draws the four containments together. The dashed frame marks what the agent is bounded *by* — its scope and tools — and the path on the right shows that nothing the agent produces reaches an external effect except through a deterministic step and, for consequential decisions, a human.

23.3 How the orchestrator invokes an agent

With containment established, the integration itself is straightforward, and deliberately so. From the orchestrator’s perspective, an agent invocation is a service task — the same construct used for any automated work. The process reaches the task; a job is created; a job worker hands the task and its inputs to the agent runtime; the agent does its reasoning; a structured result comes back; the token moves on.

Two properties of this invocation deserve emphasis. First, it is *schema-bounded on both ends*: the agent receives a defined input and is required to return a defined output structure, so that the next BPMN step — a DMN table or a gateway — has something deterministic to consume. An agent that returns free prose has not completed the task in a way the process can use. Second, it is *traced*: the agent’s inputs, the tools it invoked, the intermediate steps it took, and its final output are all recorded against the process instance, because [Chapter 125](#) will need that trace for observability and [Chapter 127](#) for explainability.

23.4 Single agents and multi-agent processes

It is tempting, having one agent, to build a society of them: a triage agent that hands to an assessment agent that hands to a settlement agent. The manuscript counsels caution. Each agent is a source of confident error, and chaining agents directly compounds those errors while obscuring where a fault arose.

The disciplined pattern keeps the orchestrator, not an agent, as the thing that connects agents. Where a process genuinely needs several agentic steps, each is a separate, bounded, contained service task in the BPMN model, with deterministic steps — rules, tables, sometimes human tasks — between them. The process model, which is governable, is the integration fabric; the agents are contained components within it. “Multi-agent” becomes a property of the process model, not an unsupervised conversation among agents. This keeps every agent individually contained and every hand-off individually visible, which is the only form of multi-agent system that survives an operational-risk review.

23.5 Tools, retrieval, and the agent's working knowledge

An agent is only as useful as the tools it can invoke and the information it can reach, and the design of that toolset deserves its own attention, because it is where an agent's usefulness and its safety are jointly decided.

A *tool*, in this setting, is a defined capability the process grants the agent: retrieve a specified document, look up a customer's prior-claims history, read a policy wording, query a structured data source. Each tool is a deliberate grant. The agent cannot reach anything the process did not give it a tool for, which means the toolset is simultaneously the agent's reach and the boundary of its reach — the same fact seen from two sides. Designing the toolset is therefore designing the containment: a tool added for usefulness is also a surface added for risk, and [Chapter 26](#) returns to this as a security matter.

The most important category of tool is *retrieval*: the agent's ability to bring relevant information into its reasoning. A claims-assessment agent asked whether an incident is internally consistent needs to retrieve the first-notification narrative, the policy wording, the photographs, perhaps the prior-claims history. The quality of the agent's assessment depends heavily on whether retrieval brings it the *right* information — complete enough to judge, current enough to be correct, and scoped tightly enough that the agent is not reasoning over a mass of irrelevant material.

Retrieval also introduces a specific discipline. The information an agent retrieves should be *authoritative* — drawn from the systems of record of [Chapter 25](#) or from governed document stores, not from an uncontrolled source — because an agent reasoning over stale or wrong information will reason fluently to a wrong conclusion. And the retrieved information should be *recorded* as part of the agent's trace, because [Chapter 127](#)'s explainability requirement extends to what the agent knew: an explanation of the agent's assessment that cannot say what information the assessment was based on is not a complete explanation.

Tip

Design an agent's toolset as the minimum that lets it complete its bounded task, and treat every tool as both a capability and a risk surface. Favour retrieval tools scoped to specific, authoritative sources over broad, general-purpose access. An agent that can reach exactly what its task requires, and no more, is both easier to make accurate and

easier to contain — the two goals point the same way.

23.6 AI within governance, not outside it

The integration of AI capabilities — language models, agents, retrieval systems — into a regulated process must happen *within* the process's governance, not outside it. An LLM call from inside a governed process step is auditable: the input, the output, the model version, the timestamp are all part of the process record. An LLM call from an ungoverned script, triggered by a cron job or a manual action, has none of these properties. The process model is the governance frame, and every AI capability must sit inside it, as a step whose inputs and outputs the institution can show, explain, and defend.

Worked example: the expected cost of an agent in the loop

Problem. An insurer considers placing a consistency-checking agent into its claims-triage process. Without the agent, triage is fully manual at \$9 per claim. With the agent, the agent runs first at a compute cost of \$0.40 per claim and correctly clears 60% of claims for straight-through handling; the other 40% still go to a manual reviewer at \$9. However, the agent is wrong on 3% of the claims it clears — it clears a claim that should have been reviewed — and each such error costs the insurer an average of \$700 in downstream loss. On 100,000 claims a year, what is the annual cost with and without the agent, and is the agent justified?

Setup. Without the agent, cost is volume times manual cost. With the agent, cost is the agent compute cost on all claims, plus manual cost on the 40% not cleared, plus the expected error cost on the cleared claims. We compute both and compare.

Working. *Without the agent:*

$$100,000 \times \$9 = \$900,000.$$

With the agent. Agent compute on all claims:

$$100,000 \times \$0.40 = \$40,000.$$

Claims still reviewed manually (40%): 40,000 claims at \$9:

$$40,000 \times \$9 = \$360,000.$$

Claims cleared by the agent (60%): 60,000 claims. Of these, 3% are wrongly cleared:

$$60,000 \times 0.03 = 1,800 \text{ erroneous clearances,}$$

each costing \$700:

$$1,800 \times \$700 = \$1,260,000.$$

Total with the agent:

$$\$40,000 + \$360,000 + \$1,260,000 = \$1,660,000.$$

Discussion. The agent makes the process *worse* — \$1.66M against \$0.9M — and the worked example exists to make the reason unmistakable. The processing saving is real: replacing \$9 of manual review with \$0.40 of compute on 60,000 claims looks like a clean win.

But the 3% error rate on cleared claims, at \$700 each, generates \$1.26M of downstream loss that swamps the entire saving. This is the agentic failure mode quantified: the agent is fast and cheap and confidently wrong often enough to be a net negative. The decision is not “agent or no agent.” It is that the agent cannot be allowed to clear claims unsupervised at a 3% error rate. The fix is containment, exactly as this chapter argued: the agent should not *clear* claims but *recommend* clearance, with a deterministic check — a DMN rule on claim value, or a sampled human review — catching the costly errors before they become losses. Run the numbers again with a containment step that intercepts, say, 90% of the erroneous clearances, and the agent moves firmly back into positive territory. The agent is justified; the *ungoverned* agent is not.

Practical next steps

Identify one process step in your institution that is currently manual because it needs judgement over unstructured information — the kind of step a predictive model cannot do. For that step, write down the four containment mechanisms explicitly: what is the agent’s bounded task, what tools may it use, what deterministic step adjudicates its output, and which named human is accountable. If you cannot fill in all four, the step is not yet ready for an agent.

Chapter in brief

An agent is given a task and tools and decides for itself how to complete it, which adds open-ended task completion — judgement over unstructured information that a predictive model cannot provide. That same latitude makes the agent confidently and plausibly wrong, the most dangerous failure mode in a regulated process. Agents are therefore governed by containment: bounded scope, bounded tools, deterministic adjudication of their output, and human accountability. The orchestrator invokes an agent as a schema-bounded, traced service task, and where several agents are needed the governable process model — not an unsupervised agent conversation — connects them.

CHAPTER 24

The Cloud Platform

24.1 What the cloud is for in this architecture

The cloud is the third named element of the manuscript’s title, and it is worth being honest that it is the least conceptually interesting of the three. Camunda contributes orchestration; agentic AI contributes reasoning; the cloud contributes none of the process logic. What it contributes is the substrate that makes the other two operable at the scale and the reliability a financial institution requires — and that contribution, though unglamorous, is what determines whether the architecture runs on a slide or in production.

The cloud provides four things the architecture depends on. It provides *elasticity*: the orchestration job workers, the agent runtime, and the decision services all have load that varies with the business day and the business cycle, and the cloud lets capacity follow that load rather than being provisioned for the peak and idle the rest of the time. It provides *managed services*: the databases, message infrastructure, object storage, and identity services that the platform needs but that the institution should not want to operate by hand. It provides *failure domains*: physically separated zones and regions across which the platform can be spread so that a localised failure does not become a total one. And it provides a *control plane*: a uniform way to describe, deploy, observe, and secure the whole estate as code.

24.2 The shape of the deployment

A hyperautomation platform on the cloud has a recognisable shape, and naming its tiers helps reason about it.

The *orchestration tier* runs the clustered process engine and its job workers. It is stateful in the sense that it depends on a durable store, and it is sized — as [Chapter 21](#) established — for availability rather than raw throughput.

The *decision and model tier* runs the DMN evaluation and the predictive model services. These are typically stateless request-response services, which makes them straightforward to scale horizontally with load.

The *agentic tier* runs the agent runtime and the tools agents are permitted to use. Its load is spiky and its per-request cost is high, which makes it the tier where elasticity matters most and where cost control, treated in [Chapter 128](#), bites hardest.

The *data tier* holds the durable process state, the audit and trace records, the document store, and the feature data the models and agents draw on. It is the tier with the strongest data-residency and protection obligations.

The *integration tier* connects the platform to the institution’s systems of record — core banking, policy administration, payments, customer data — treated in its own right in [Chapter 25](#).

Figure [24.1](#) names the five tiers and stacks them. The picture is a useful reasoning aid because each tier has a different scaling profile — the agentic tier spiky and expensive, the decision tier statelessly scalable, the data tier residency-constrained — and a platform design must address each tier on its own terms.

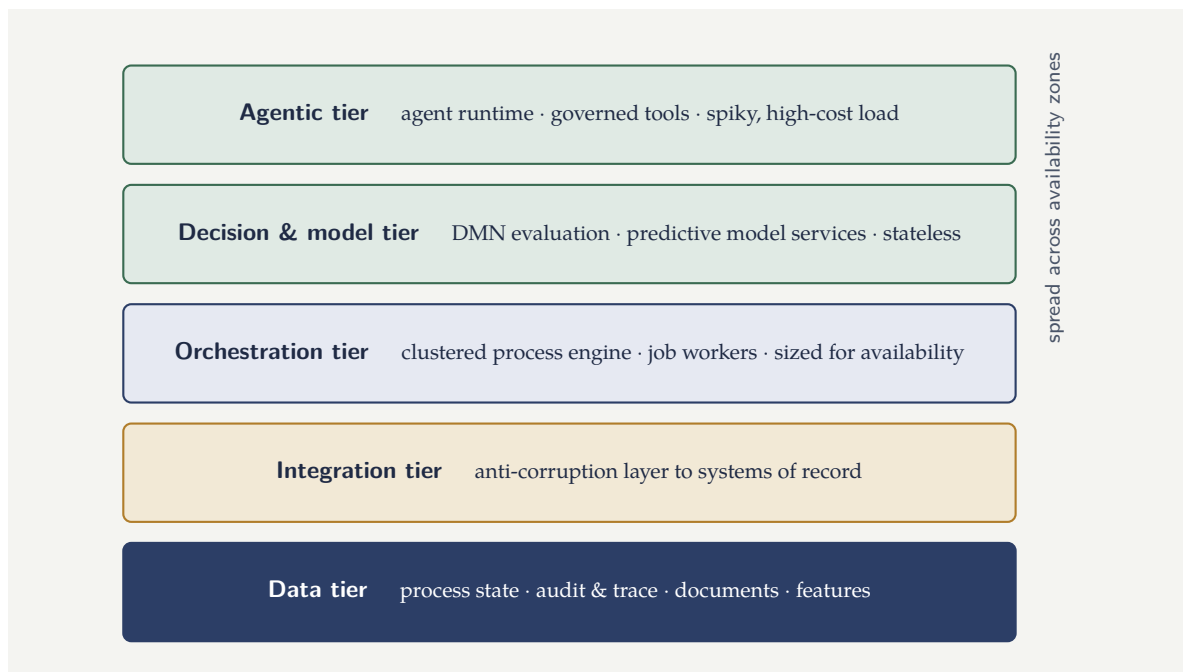


Figure 24.1. The five tiers of a hyperautomation platform on the cloud. Each tier has a distinct scaling and resilience profile, and the whole is spread across availability zones so that the loss of one zone degrades but does not stop the platform.

Tip

Describe the whole platform as code — infrastructure, configuration, network, and access policy in version-controlled definitions, deployed through a pipeline. In a regulated institution this is not merely an engineering convenience: it makes every change to the platform reviewable, auditable, and reproducible, which is exactly what an examiner of the platform will ask for. A platform assembled by hand cannot answer the question “how do you know it is configured as approved?”

24.3 Resilience: designing for failure

Financial institutions do not tolerate their lending, payments, or claims platforms being unavailable, and the cloud does not make components stop failing — it makes failure *plannable*. Resilience is therefore a design activity, organised around a few principles.

The platform is spread across *multiple availability zones* so that the loss of one zone — a data-centre-scale failure — degrades but does not stop the platform. Stateful components keep synchronous replicas across zones; stateless components run active instances in each.

The platform’s behaviour under partial failure is *designed, not discovered*. Chapter 21 noted that an integration outage should cause jobs to queue, not processes to fail; that property is a design choice — retry policies, timeouts, circuit breakers, and explicit BPMN failure paths — not an accident.

The platform’s recovery is *measured against objectives*. For each process, the institution states a recovery-time objective (how quickly must it be running again after a failure) and a recovery-point objective (how much in-flight state may be lost). The architecture is then built and, crucially, *tested* against those numbers — including by deliberately injecting failures, so that

the recovery behaviour is known rather than hoped for.

24.4 Cloud, region, and the regulatory geography

A financial institution does not have a free hand in where its platform runs. Data-residency rules, regulatory expectations about concentration risk, and the institution's own resilience standards all constrain the cloud deployment, and these constraints are architecture inputs, not afterthoughts.

Data-residency obligations may require that certain customer data — and therefore the process state and audit records that contain it — never leave a particular jurisdiction, which constrains region choice for the data tier. Supervisory attention to *concentration risk* — the systemic exposure created when many institutions depend on one cloud provider — pushes some institutions toward architectures that avoid deep, unportable coupling to a single provider's proprietary services. And the institution's own standards will set out how many simultaneous failures the platform must survive. The practical consequence is that the cloud deployment is designed jointly with the institution's risk and compliance functions, and [Chapter 18](#) returns to the regulatory dimension in full.

24.5 Form design for regulated processes

A form in a regulated process is not just a user interface; it is a *data-capture point* whose design directly affects data quality, compliance, and the auditability of the decision that follows. Fields should be validated at input, not corrected downstream. Required fields should be genuinely required, not optional-then-manually-checked. And the form should capture exactly the data the process step needs — no more, no less — because every unnecessary field is a data-protection exposure and every missing field is a decision made on incomplete information.

Worked example: availability of a multi-zone deployment

Problem. An orchestration tier is to be deployed across three availability zones. The platform is designed so that it remains fully operational as long as at least two of the three zones are up. Each zone has an availability of 0.99 — that is, each zone is independently down 1% of the time. Assuming zone failures are independent, what is the availability of the three-zone platform, and how does it compare with running in a single zone?

Setup. The platform is up when at least two zones are up. We compute the probability of that event as the probability all three are up plus the probability exactly two are up. Each zone is up with probability $p = 0.99$ and down with probability $q = 0.01$, independently.

Working. Probability all three zones are up:

$$p^3 = 0.99^3 = 0.970299.$$

Probability exactly two zones are up (one specific zone down, chosen 3 ways):

$$\binom{3}{2} p^2 q = 3 \times 0.99^2 \times 0.01 = 3 \times 0.9801 \times 0.01 = 0.029403.$$

The platform is operational in both cases, so its availability is the sum:

$$0.970299 + 0.029403 = 0.999702.$$

Single-zone availability is just $p = 0.99$. The three-zone, two-of-three design reaches 0.999702.

Discussion. The improvement is large and worth stating in the units that matter operationally. A single zone at 0.99 availability is unavailable about 1% of the time — roughly 3.65 days a year, which no institution would accept for a lending or claims platform. The three-zone design at 0.999702 is unavailable about 0.03% of the time — roughly 2.6 hours a year, an order-of-magnitude class better, achieved purely by spreading the same workload across independent failure domains. Two cautions belong with the number, though. First, the calculation assumes *independent* zone failures; a correlated failure — a shared network plane, a provider-wide control-plane incident, a region-level event — breaks the independence assumption and the real availability is lower, which is the architectural argument for considering a second region for the most critical processes. Second, availability of the orchestration tier alone is not availability of the *process*: if a single-zone data tier or a single integration endpoint sits behind this resilient orchestration tier, the platform is only as available as that weakest component. The calculation sizes one tier; the end-to-end number requires the same analysis applied to every tier on the path.

Practical next steps

For your most critical process, write down its recovery-time and recovery-point objectives as explicit numbers agreed with the business. Then, tier by tier — orchestration, decision and model, agentic, data, integration — identify the weakest component on the process's path. Your end-to-end availability is governed by that component, not by the most resilient tier. The exercise usually finds the weak point in the integration or data tier, not the orchestration tier.

Chapter in brief

The cloud contributes no process logic; it contributes the substrate that makes orchestration and agents operable at scale — elasticity, managed services, failure domains, and a code-driven control plane. A hyperautomation platform has five recognisable tiers: orchestration, decision and model, agentic, data, and integration. Resilience is a design activity — multi-zone spread, designed behaviour under partial failure, and recovery tested against explicit objectives — not a property the cloud confers. Region and provider choices are constrained by data residency, concentration risk, and the institution's resilience standards, so the cloud deployment is designed jointly with risk and compliance.

Integration with Core Banking and Insurance Systems

25.1 The unglamorous half of the architecture

A hyperautomation platform that could not reach the institution's systems of record would be an elegant machine connected to nothing. The orchestrator can run a beautiful lending process, but the loan must ultimately be booked in the core banking system; the agent can assess a claim, but the reserve must be set in the policy administration system and the payment made through the payments rails. Integration — the connection between the new orchestration platform and the institution's existing systems — is half the real work of a hyperautomation programme, and it is the half most often underestimated.

It is underestimated because it is unglamorous and because it is constrained. The systems of record in a bank or insurer are typically old, were not designed to be orchestrated from outside, are business-critical and therefore change-averse, and are frequently the institution's least flexible assets. The hyperautomation platform must integrate with them as they are, not as an architect would wish them to be. This chapter is about doing that well.

25.2 Systems of record and the integration boundary

It clarifies the architecture to name the systems involved and to be precise about the boundary.

A bank's *systems of record* typically include a core banking system that holds accounts and balances and posts transactions; a payments infrastructure; a customer master that holds the authoritative customer record; a product or pricing system; and a general ledger. An insurer's include a policy administration system that holds policies and coverages; a claims system; a billing system; a rating engine; and an actuarial reserving environment. These systems are authoritative: they hold the truth about accounts, policies, balances, and claims, and the hyperautomation platform holds the truth about *process* but not about those entities.

The integration boundary is therefore a boundary of *authority*, and the governing principle is simple: the hyperautomation platform orchestrates and decides, but it does not duplicate the systems of record. It reads from them and instructs them; it does not become a second, divergent copy of the account or the policy. A process instance carries a *reference* to the loan or the claim and the working data it needs, not a shadow ledger.

The most damaging integration anti-pattern is the *shadow system of record*: the orchestration platform gradually accumulating its own copy of balances, policy details, or claim reserves, which then drifts out of step with the real system. When the two disagree, nobody can say which is right, and reconciliation becomes a permanent operational burden. The platform holds process state and references; it does not hold a second copy of the truth.

25.3 Integration styles and when each fits

There is no single way to integrate, and a mature platform uses several styles deliberately.

Synchronous request-response — the platform calls a system's interface and waits for an answer — suits reads and quick commands: fetch the customer record, check a balance, post a single transaction. It is simple and immediate, and it couples the process's progress to the called system's availability and speed, which is acceptable for fast, reliable interfaces and dangerous for slow or flaky ones.

Asynchronous messaging — the platform emits a message and continues, receiving a result later — suits commands to systems that are slow, batch-oriented, or unreliable. It fits the orchestrator's job-and-wait model naturally: the BPMN process sends, then waits at a message event, and the engine persists the waiting instance until the result arrives. This style absorbs the slowness and unreliability of legacy systems instead of being infected by it.

Event consumption — the platform reacts to events the systems of record emit — suits processes that should be *triggered* by something happening in a core system: a payment received, a policy lapsed, a threshold breached. The event starts or advances a process instance.

Batch and file-based integration — still pervasive in banking and insurance — suits the reality that some legacy systems exchange data only as periodic files. The platform must accommodate it: a process step that waits for the overnight file, or a batch that initiates many process instances from a file's contents.

A real platform uses all four. The integration architecture's job is to choose the right style for each system of record and, importantly, to keep the choice *behind a stable interface* so that the process model is not rewritten when an integration style changes.

25.4 The anti-corruption layer

The systems of record speak their own languages: their data models, their codes, their quirks, their decades of accumulated special cases. If those languages leak into the process models and decision tables, the hyperautomation platform becomes as rigid as the systems it was meant to modernise — every legacy quirk reproduced in the new layer.

The defence is an *anti-corruption layer*: a deliberate translation tier between the systems of record and the orchestration platform. It exposes the core systems to the platform through clean, stable, business-meaningful interfaces — “get customer,” “book loan,” “set reserve” — and it absorbs, on the inside, all the translation, the legacy codes, the protocol adaptation, and the quirks. The process models and decision tables are written against the clean interfaces and never see the legacy detail.

The anti-corruption layer has a second, strategic value. Because the platform is coupled to the clean interface and not to the legacy system, the legacy system can be modernised or replaced — a core banking migration, a new policy administration system — with the change absorbed inside the anti-corruption layer and the process models untouched. The layer that looks like overhead is the layer that makes the institution's modernisation survivable.

Figure 25.1 draws the anti-corruption layer between the platform and the legacy systems of record.

25.5 The event backbone

The integration styles described above are mostly *point-to-point*: the process calls a system, or sends it a message, or waits for a file from it. A mature platform usually also has a different

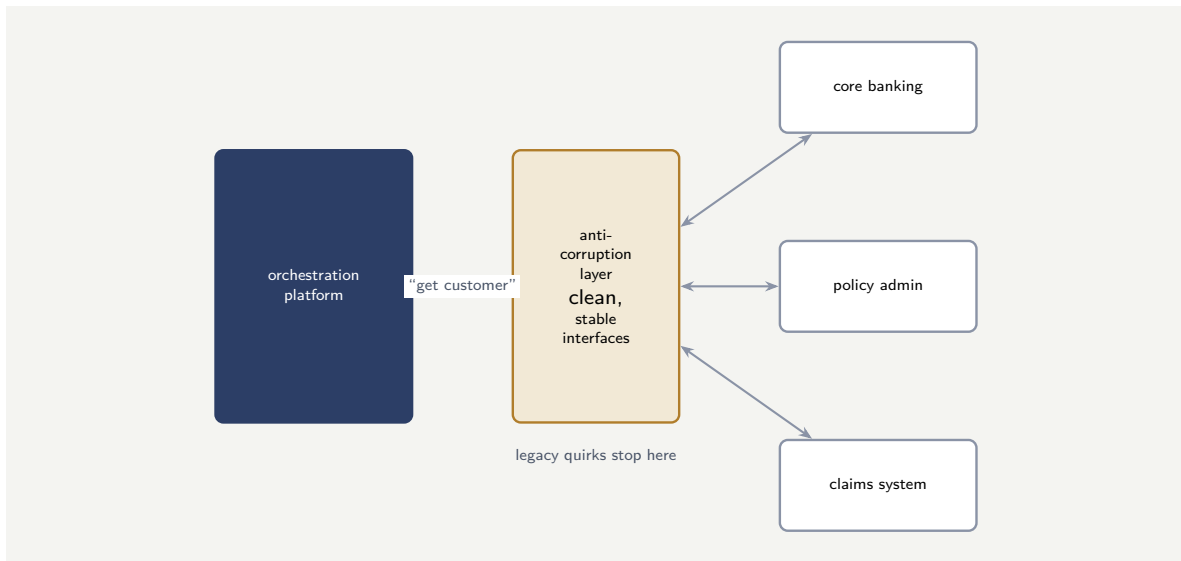


Figure 25.1. The anti-corruption layer. It exposes the legacy systems of record to the platform through clean, stable, business-meaningful interfaces, so the legacy systems’ data models and quirks never leak into the process models — and a legacy system can later be modernised behind the layer without touching the processes.

integration substrate running underneath — an *event backbone* — and it is worth treating on its own because it changes how processes are triggered and how the institution’s systems stay consistent.

An event backbone is a durable, ordered stream of business events: a payment was received, a policy lapsed, a customer’s address changed, a loan was booked, a claim was settled. Systems publish events to the backbone when something significant happens, and other systems — including the hyperautomation platform — subscribe to the events they care about. The backbone decouples the publisher from the subscriber: the system that emits “policy lapsed” does not know or care which processes react to it, and a new reacting process can be added without touching the publisher.

For a hyperautomation platform the event backbone does three things that point-to-point integration does poorly. It is the natural way to *trigger* processes: a process that should begin when a payment is received subscribes to the payment-received event, and the event starts a process instance — the platform reacts to the business rather than polling for changes. It is the natural way to *advance* a waiting process: a claims instance waiting for a third party’s response resumes when the corresponding event arrives, which is exactly the message event of [Chapter 3](#) sourced from the backbone. And it is the natural way to keep the institution’s systems *loosely consistent*: each system updates itself in reaction to the events it cares about, rather than through a web of direct, brittle, point-to-point updates.

The event backbone also imposes its own disciplines. Events must be *durable and ordered*, so that a subscriber that was briefly unavailable can resume from where it stopped without losing or reordering events. Events should be *meaningful business facts* — “loan disbursed” — rather than low-level technical change notifications, because the subscribers reason in business terms. And, returning to the discipline of [Chapter 122](#), an event may be *delivered more than once*, so a process triggered or advanced by an event must be idempotent: receiving the same “payment received” event twice must start one process instance, not two.

Tip

Treat the event backbone as a first-class part of the integration architecture, not as plumbing. The events an institution chooses to publish become a shared, durable description of what is happening across the business — and a well-chosen set of business events is one of the most reusable assets a hyperautomation programme creates, because every future process that needs to react to one of those facts simply subscribes.

25.6 The anti-corruption discipline for core banking

The core banking system's data model is its own: decades of accumulated structure, vendor-specific field names, legacy encodings. A process that binds directly to that model inherits all of it, and every change the vendor makes to the core's API ripples into every process. The anti-corruption discipline requires a deliberate translation layer: the worker maps the core's fields into the process's own vocabulary, and the process model upstream of the worker never sees a vendor-specific field name. The translation costs a small amount of code; the coupling it prevents costs a large amount of rework every time the core's API changes.

Worked example: the latency budget of an integrated process step

Problem. A real-time account-opening process must complete a synchronous identity-and-eligibility check in no more than 2,000 milliseconds end to end, because the customer is waiting at a screen. The check, as designed, makes three sequential synchronous calls: to the customer master (mean 250 ms), to an external identity service (mean 600 ms), and to a sanctions-screening service (mean 450 ms). The orchestration and decision logic around the calls adds a fixed 150 ms. Does the design meet the budget, and how much margin is there for the variability of the slowest call?

Setup. Because the three calls are sequential, the mean end-to-end time is the sum of the three call means plus the fixed overhead. The margin is the budget minus that mean. We then express the margin relative to the slowest call, since real call times vary around their means and the slowest call is the likeliest to overrun.

Working. Mean end-to-end time:

$$250 + 600 + 450 + 150 = 1,450 \text{ ms.}$$

Margin against the 2,000 ms budget:

$$2,000 - 1,450 = 550 \text{ ms.}$$

The slowest call is the identity service at a mean of 600 ms. The 550 ms margin, expressed relative to that call's mean:

$$\frac{550}{600} \approx 0.92.$$

Discussion. On mean times the design passes with 550 ms to spare, but the worked example is really about the difference between a mean and a budget. A 2,000 ms *budget* is almost never a promise about the average case — it is a promise about nearly every case, the 95th or 99th percentile. Real service-call times have long upper tails: an identity service with a 600 ms mean may have a 99th-percentile time well above 1,200 ms. The 550 ms of margin is

only 0.92 of the slowest call's *mean*, which means a single bad-tail response from the identity service can consume the entire margin and breach the budget. Two design responses follow. First, the calls that are genuinely independent — the customer-master read and the sanctions screening do not depend on each other — should be issued *in parallel* rather than in sequence, which the orchestrator's parallel gateway expresses directly and which removes the smaller of the two from the critical path. Second, the design needs an explicit timeout-and-fallback path: a BPMN boundary timer on the slow call, and a defined behaviour when it fires — proceed with a degraded result, or route the case to a non-real-time path — so that a tail-latency event becomes a designed outcome rather than a breached promise and a customer staring at a frozen screen.

Practical next steps

Take one synchronous, customer-facing process step in your institution and write its latency budget. List every call it makes, with mean *and* upper-percentile times. Sum the sequential ones. If the upper-percentile sum breaches the budget, identify which calls are independent and could run in parallel, and define the timeout-and-fallback path for the slowest.

25.7 The insurance process estate

An insurer's process estate is distinctive in its *breadth* and its *timescales*. The breadth: an insurer runs processes across underwriting, policy administration, claims, renewals, complaints, and regulatory reporting — more process categories than most banks. The timescales: a claim can remain open for years, a policy for decades, and the processes that manage them must be designed for lifetimes measured in years, not days. These two properties — breadth and longevity — make the insurer's process estate one of the most demanding environments for a hyperautomation platform, and every architectural choice in this chapter must be evaluated against both.

The chapter's principles interact with the platform's operational model in a way worth making explicit. The *deployment* of the artefacts this chapter describes must go through a governed pipeline — versioned, reviewed, tested, and traceable. An artefact deployed by a manual action, outside the pipeline, is an artefact whose presence in production cannot be explained, and in a regulated institution an unexplained artefact is a finding.

The *monitoring* of the artefacts must cover both their functional correctness and their operational health. A process step that executes but produces wrong results is harder to detect than one that fails outright, because it does not raise an error — it raises a consequence, and the consequence may not be discovered until a customer complains or a regulator asks a question. The observability model should therefore include not only error rates and latencies but also *outcome monitoring*: checks that the results the process produces are within expected ranges, flagging anomalies for review.

Chapter in brief

Integration with the institution's systems of record is half the real work of a hyperautomation programme and the most underestimated half. The integration boundary is a boundary of authority: the platform orchestrates process and holds references, but never becomes a shadow copy of the core systems' truth. A mature platform uses

synchronous calls, asynchronous messaging, event consumption, and batch integration deliberately, each behind a stable interface. An anti-corruption layer translates the legacy languages into clean, business-meaningful interfaces, keeping process models free of legacy quirks and making core-system modernisation survivable. Synchronous customer-facing steps need latency budgets sized against upper-percentile, not mean, call times.

Data, Identity, and the Security Model

26.1 Why security is an architecture chapter

A hyperautomation platform in a bank or insurer is, by its nature, a concentration of sensitive material. It touches customer identity documents, financial positions, medical information in the case of health and life insurers, transaction histories, and the decisions made about all of them. It connects to every important system of record. It now also contains agents that can reason over that material and tools those agents can invoke. A platform with that reach is an attractive target and a significant point of risk, and its security model is therefore a first-class part of the architecture — designed alongside the orchestration and the integration, not bolted on after them.

This chapter treats the security model under three headings: the data the platform holds and how it is protected, the identities — human and machine — that act on the platform, and the specific new surface that agents and their tools introduce.

26.2 Data: classification, protection, and minimisation

The starting point is to know what data the platform holds and how sensitive each kind is. Every category of data the platform touches — identity documents, financial data, health data, decision records, audit traces — should be *classified*, and the classification should drive the controls: where the data may reside, who and what may read it, how long it is kept, and how strongly it is protected.

Three protection principles then apply throughout. Data is *encrypted* both in transit between components and at rest in every store — the process state, the document store, the audit records, the feature data. Access to data follows *least privilege*: each component and each person can read and write only the data their role genuinely requires, and no more. And the platform practises *data minimisation*: a process instance carries the data it actually needs to make its decisions and no more, and data is retained only as long as there is a defined reason — regulatory, operational, or contractual — to keep it.

Minimisation deserves emphasis because it is the principle most often neglected. A process instance that accumulates every document and every field it ever touched, and keeps them forever, is both a larger breach exposure and a compliance problem. The disciplined process carries a lean working set and relies on the systems of record — [Chapter 25](#) — for the authoritative detail it does not need to hold.

A classification, to be useful, must be concrete: a named set of categories, each tied to the controls it demands. [Table 26.1](#) is such a classification for the data a banking or insurance process platform typically holds — the artefact a security architect produces early and the rest of the platform's controls then follow.

The table makes the principle operational: the controls are not applied uniformly — health

data category	sensitivity	controls it demands
identity documents	high	encrypt; strict access; short retention
financial & account data	high	encrypt; least privilege; system-of-record holds master
health data (insurance)	highest	encrypt; minimal access; special-category handling
decision records	medium	integrity-protected; retained per regulation
audit traces	medium	append-only; tamper-evident; long retention
operational metrics	low	standard protection; aggregate where possible

Figure 26.1. A data classification for a process platform. Each category of data the platform holds is rated for sensitivity and tied to the controls it demands — and the classification, produced early, is what the platform’s encryption, access, and retention controls are then derived from.

data and operational metrics do not warrant the same handling — but are *derived* from each category’s sensitivity. A platform that has not classified its data has no principled basis for deciding how strongly to protect any of it.

Figure 26.2 draws the three principles applied to classified data.

Tip

Treat data retention as a modelled property of each process, not a database default. For every category of data a process holds, state explicitly why it is kept and for how long, and build the deletion or archival into the process or platform. “We keep everything indefinitely because deleting is hard” is not a retention policy a regulator or a privacy authority will accept.

26.3 Identity: humans, services, and least privilege

Everything that acts on the platform has an *identity*, and the platform’s security depends on those identities being well managed.

Human identities — the staff who perform user tasks, the administrators who operate the platform, the analysts who review decisions — are authenticated through the institution’s identity provider, and their permissions are governed by role. A claims analyst can act on claims tasks; a platform administrator can deploy process models; a model-risk reviewer can read decision traces. The roles are defined deliberately and reviewed regularly, because permission sprawl — accounts that accumulate access and never lose it — is one of the commonest real-world security weaknesses.

Service identities — the job workers, the decision services, the integration components —

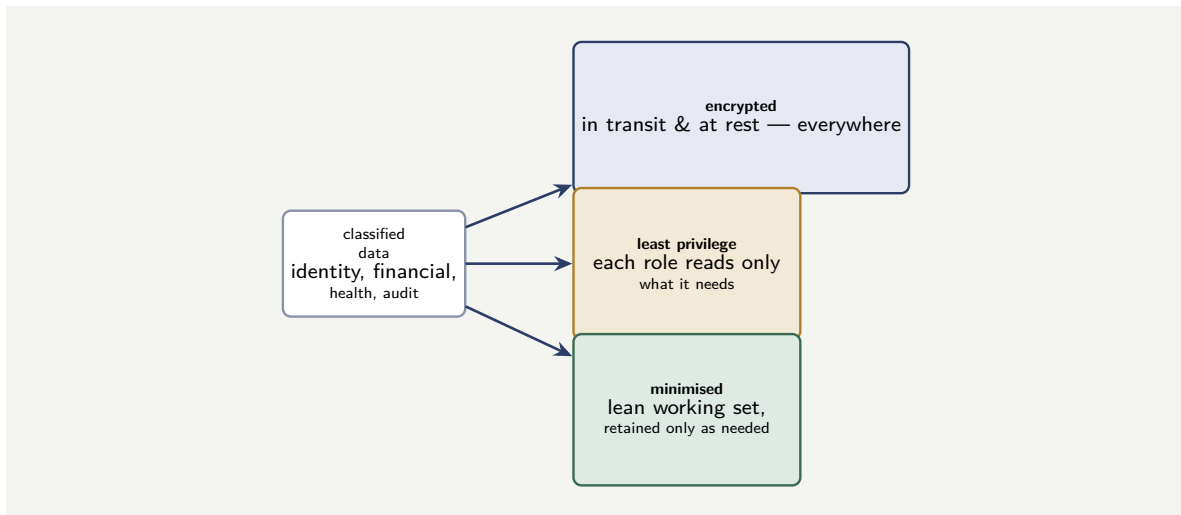


Figure 26.2. The three data-protection principles. Classified data is encrypted in transit and at rest everywhere; access follows least privilege, each role reading only what it needs; and data is minimised — a lean working set, retained only as long as a defined reason requires.

also have identities, and they too are governed by least privilege. The job worker that books loans can call the core banking “book loan” interface and nothing else. Machine identities are credentials, and credentials are managed: issued, rotated, and revocable, held in a secrets-management service rather than embedded in code or configuration.

The accountability link. Because the platform records, for every action, the identity that performed it, the audit trail answers not only “what happened” but “who or what did it.” For a human task, the trail names the person. For an automated step, it names the service identity and the human role accountable for that automation. This link between every action and an accountable identity is what makes the platform auditable in the sense a regulator means.

26.4 The audit record as a controlled asset

Chapter 21 listed the audit record as one of the four things the orchestration core owns, and the preceding section showed that it links every action to an accountable identity. The audit record deserves a section of its own in the security model, because it is simultaneously the platform’s most valuable evidentiary asset and a concentration of sensitive information — and those two facts pull in different directions.

The audit record is valuable because it is the raw material of everything Part XIV will ask of the platform: the explainability of Chapter 127, the observability of Chapter 125, the incident diagnosis, the supervisory answer. An institution that can produce, for any past decision, an exact and trustworthy account of how it was made has a genuine asset. The qualifier is *trustworthy*: an audit record is only worth what its integrity is worth.

Three properties make an audit record trustworthy. It must be *complete*: every consequential action — every decision, every model score, every agent assessment, every human action, every state transition — is recorded, because a record with gaps cannot be relied on and the gaps are exactly where a dispute will land. It must be *tamper-evident*: it must not be possible to alter a past record without the alteration being detectable, because an audit record that could have been quietly edited proves nothing. The standard technique is append-only storage

with cryptographic chaining, so that each record is bound to its predecessors and any change breaks the chain. And it must be *access-controlled*: the audit record contains sensitive customer and decision data, so reading it is itself a governed action, subject to the least-privilege discipline and — fittingly — itself audited.

Tip

Hold the audit record append-only and tamper-evident, and treat read access to it as a governed, logged action in its own right. An audit record that the institution itself could have quietly altered is not evidence — and a supervisor, or a court, will treat it accordingly. The integrity of the audit record is what converts the platform’s transparency from a claim into proof.

26.5 The agentic security surface

Agents introduce a security surface that classical automation does not have, and it must be treated explicitly.

An agent acts by reasoning over inputs and invoking tools. Two new risks follow. The first is that an agent’s behaviour is influenced by the *content* it processes: a document an agent reads could contain text crafted to manipulate the agent into doing something other than its task — the agentic analogue of an injection attack. The second is that an agent’s reach is the set of tools it can invoke: a tool that can move money or change a record, placed within an agent’s reach, is a far more serious exposure than the same tool behind a deterministic service.

The architecture contains these risks with the same discipline [Chapter 23](#) applied to agentic correctness. An agent is given the *minimum* toolset its bounded task requires, and the tools themselves are governed — a retrieval tool that can read a defined class of documents, never a general-purpose capability. Tools that have external consequence — moving money, binding cover, changing a system of record — are not given to agents at all; those effects happen only through the deterministic steps that adjudicate an agent’s output. And the content an agent processes is treated as untrusted input: the agent’s task and tool boundaries are enforced by the surrounding process and the runtime, not left to the agent’s own judgement to respect. The agent is contained by construction, so that even an agent successfully manipulated by malicious content cannot exceed the reach the architecture gave it.

Worked example: least privilege and the blast radius of a credential

Problem. A platform has 30 service components. Under the current design, all 30 share a single broad credential that grants access to every system of record and every data store. A security review proposes replacing it with per-component least-privilege credentials, after which each component can reach, on average, only 3 of the 30 possible targets. Treating the “blast radius” of a compromised credential as the number of targets it can reach, how much does the proposal reduce the blast radius?

Setup. The blast radius of one credential is the number of targets it can reach. With a single shared credential, a compromise reaches all targets. With per-component credentials, a single compromised credential reaches only that component’s targets.

Working. *Before — single shared credential.* One credential, blast radius 30. A compromise reaches all 30 targets. Blast radius: 30.

After — per-component credentials. Each credential has an average blast radius of 3. A single compromised credential reaches, on average, only 3 targets. The reduction in single-compromise blast radius:

$$\frac{30 - 3}{30} = 0.90, \quad \text{a 90\% reduction.}$$

The summed credential surface across the estate rises from 30 (one credential reaching 30) to $30 \times 3 = 90$ target-reaches, but spread across 30 separately compromisable credentials.

Discussion. The numbers capture the real trade in least-privilege design and why it is still strongly worthwhile. The summed surface actually *rises* from 30 to 90 — there are more credentials, so there is more total credential surface to attack — and a naive reading might call that worse. It is not, and the reason is that security is governed by blast radius, not by summed surface. Under the shared credential, *any* successful compromise is catastrophic: the attacker reaches everything. Under least-privilege, a successful compromise is contained: the attacker reaches three targets, a 90% smaller blast radius, and reaching the rest requires compromising further independent credentials. The design trades a small increase in the number of things that can be attacked for a large decrease in what any single successful attack achieves. The worked example also points at the operational cost the proposal carries: 30 credentials must be issued, rotated, and revoked, which is why [Chapter 58](#)'s insistence on a secrets-management service and infrastructure-as-code is not optional — least privilege is only sustainable if the credentials are managed by machinery rather than by hand.

Practical next steps

Inventory the credentials your current or planned platform uses, and for each write down its blast radius — the set of systems and data stores it can reach. Find the credential with the largest blast radius. That credential is your platform's single worst-case compromise, and narrowing it is the highest-value security change available to you.

Chapter in brief

A hyperautomation platform concentrates sensitive data and broad system reach, so its security model is a first-class part of the architecture. Data is classified, and the classification drives encryption in transit and at rest, least-privilege access, and genuine data minimisation — including modelled retention. Every actor, human and machine, has a managed identity governed by least privilege, and every action is linked to an accountable identity, which is what makes the platform auditable. Agents add a new surface: they can be manipulated by the content they read, and their reach is their toolset — so agents get minimum, governed tools, never tools with external consequence, and are contained by construction.

PART V

Key Camunda Components

CHAPTER 27

The Engine and the Runtime

27.1 From principle to product

The three preceding chapters of Part IV described the orchestration core, the agentic layer, and the cloud platform as *architectural* ideas — what each must do, and why. This part turns from the architecture to the concrete: the actual components an institution assembles when it builds the platform on Camunda. The manuscript is not a product manual, and product detail dates quickly, so the treatment stays at the level of *what each component is for* and *how it maps onto the architecture and the disciplines* the rest of the manuscript has built. The components change faster than print; their roles in a governed hyperautomation platform are stable, and it is the roles this part fixes.

A Camunda-based platform is not a single program. It is a set of cooperating components, each owning one concern, and a recurring source of confused designs is treating “Camunda” as one thing rather than understanding which component does what. This chapter treats the components that *run* the platform — the engine and the runtime around it. [Chapter 12](#) treats the components people *use* — to model, to operate, to monitor, and to connect.

27.2 Zeebe: the process engine

At the centre of a Camunda 8 platform is *Zeebe*, the process engine. It is the concrete realisation of the orchestration core that [Chapter 21](#) described in the abstract: the component that holds the deployed BPMN models, creates and advances process instances, persists their state, evaluates the routing at gateways, and makes automated work available to be done.

Two design choices in Zeebe are worth understanding, because they explain the operational properties [Chapter 21](#) and [Chapter 24](#) relied on.

The first is that Zeebe is *distributed and horizontally scalable by design*. It runs as a cluster of nodes — brokers — across which the processing of instances is partitioned, so that throughput grows by adding nodes rather than by buying a larger single machine. This is the concrete basis for [Chapter 24](#)’s point that the orchestration tier is a clustered platform; Zeebe is built to be one.

The second is that Zeebe does *not* keep its live process state in a traditional relational database. It is built on an event-streamed, append-only log: every change to an instance is an event appended to a replicated log, and the current state of an instance is the result of replaying its events. This is why Zeebe survives node and zone failure cleanly — the log is replicated across nodes, so the loss of a node loses no committed state — and it is the concrete mechanism behind [Chapter 24](#)’s availability arithmetic. The event log is also, incidentally, the natural source of the audit record: an append-only log of every state change is most of what [Chapter 26](#) asked an audit record to be.

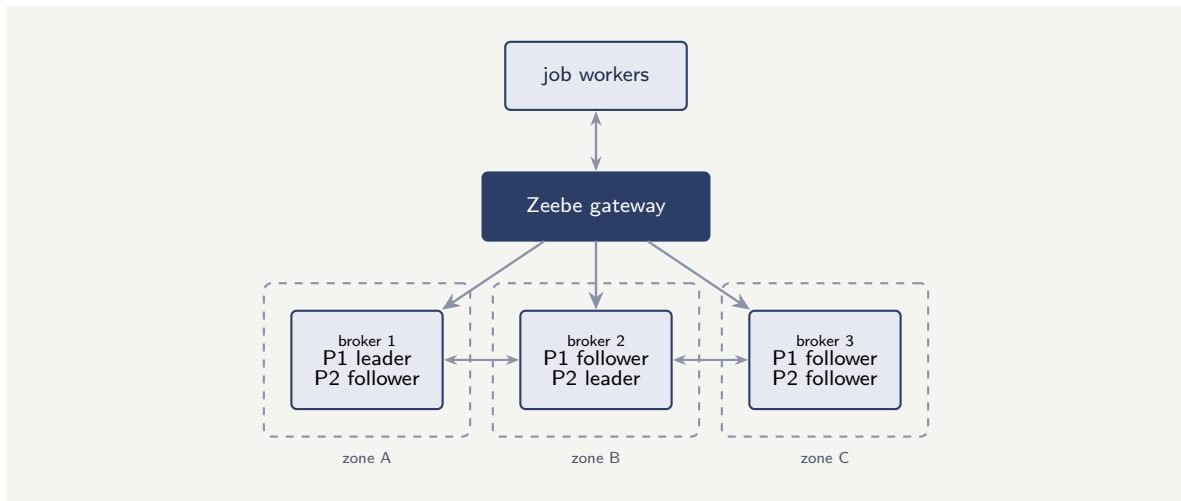


Figure 27.1. A Zeebe cluster: three brokers, one per availability zone, with two partitions (P1, P2) each replicated across all three. The loss of any one zone leaves a complete, functioning set of partition replicas. Job workers reach the cluster through the gateway.

Tip

When reasoning about a Camunda 8 platform’s resilience and scale, reason about Zeebe’s cluster: how many brokers, spread across how many availability zones, with what replication. [Chapter 24](#)’s lesson holds concretely here — the cluster is sized for availability and failure-domain spread first, and raw throughput is rarely the binding constraint for banking and insurance workloads.

Figure 27.1 draws the cluster the chapter’s worked example sizes: brokers spread one per zone, each partition replicated across all three, so that a zone failure is survivable. It is [Chapter 8](#)’s availability picture in Zeebe’s own terms.

27.3 The gateway and the clients

A running process reaches the engine through the *Zeebe gateway*: the entry point that accepts requests — deploy this model, start an instance, this job is complete — and routes them into the broker cluster. The gateway is what the rest of the platform talks to, and separating it from the brokers lets the cluster’s internal topology change without the things that call it needing to know.

The components that connect to the gateway and do automated work are *clients* — and in the architecture’s vocabulary, the realisation of the *job workers* of [Chapter 21](#). A job worker is a piece of code that connects to the gateway, asks for jobs of a particular type — “evaluate this credit rule,” “call the core banking interface,” “invoke this agent” — does the work, and reports completion. Camunda provides client libraries in common languages so that a job worker is straightforward to build, but the important point is architectural, not technical: the worker is a separate process from the engine, it scales independently of the engine, and its failure is contained — a crashed worker means a retried job, as [Chapter 21](#) established.

This separation — engine in the cluster, workers outside it connecting through the gateway — is the concrete shape of the “thin orchestrator with externalised logic” principle. The engine knows the process; the workers do the work; the gateway is the disciplined boundary between

them.

27.4 FEEL: the expression language

A process model is not purely a picture. Gateways have conditions; tasks have input and output mappings; timers have durations; decision tables have expressions. All of these are written in a small expression language called *FEEL* — Friendly Enough Expression Language — which is part of the DMN standard and is used throughout Camunda for both BPMN and DMN.

FEEL matters to this manuscript for one reason of principle. It is deliberately a *restricted* expression language, not a general-purpose programming language. It can compare values, do arithmetic, evaluate ranges, and test conditions; it cannot open a network connection, loop indefinitely, or reach arbitrary code. That restriction is a feature. The conditions on a gateway and the expressions in a decision table are logic that a business reviewer and a risk reviewer must be able to read and trust, and a restricted language keeps that logic legible and bounded. When a modeller finds FEEL insufficient for a step, that is usually the signal that the step is not an expression at all — it is real work, and belongs in a service task, a decision model, or an agent, where it can be governed as such.

Resist the temptation to push complex logic into FEEL expressions on gateways and mappings because it is convenient. Logic buried in expressions is logic that is not in a governable place: it is not a process step a reviewer sees, not a decision table risk can check, not a service that can be tested. Keep FEEL expressions simple — conditions and mappings — and move anything more substantial into a component built to be governed.

27.5 The runtime in concrete form

The engine, the gateway, the clients, and FEEL are clearer seen as the artefacts an engineer actually writes. This section gives three short ones.

A Camunda 8 cluster is configured, as [Chapter 10](#) established, through a Helm *values file* — and the values that define the engine itself are the ones from this chapter’s discussion of partitioning and replication. [Listing 27.1](#) is the core of it.

Listing 27.1. The core of a Zeebe cluster’s Helm values — the broker count, the partition count, and the replication factor that this chapter’s worked example sizes.

```
1 # values.yaml -- the Zeebe engine cluster
2 zeebe:
3   clusterSize: 3      # number of brokers
4   partitionCount: 6   # work is sharded across partitions
5   replicationFactor: 3 # each partition replicated 3x
6   pvcSize: 32Gi      # persistent volume per broker
7   zeebe-gateway:
8     replicas: 2       # the stateless entry point
```

A process reaches the engine by deploying a BPMN model whose service tasks declare a *job type* — the name a worker subscribes to. The engine does not run the work itself; it creates a job of that type and waits for a worker to take it. [Listing 27.2](#) is the defining element.

Listing 27.2. A BPMN service task declaring its Zeebe job type. The engine creates a job of this type; a worker subscribed to it does the work.

```
1 <bpmn:serviceTask id="scoreCredit" name="Score Credit">
2   <bpmn:extensionElements>
3     <zeebe:taskDefinition type="credit-scoring"
4       retries="3"/>
5   </bpmn:extensionElements>
6 </bpmn:serviceTask>
```

And FEEL — the restricted expression language — is what a gateway condition or an output mapping is written in. Listing 27.3 shows the kind of expression FEEL is *for*: a gateway condition, a range test, an output mapping. Each is a single legible line — which is the point.

Listing 27.3. FEEL expressions in their proper use — a gateway condition, a range test, an output mapping. Each is one legible line; logic that needs more is not an expression.

```
1 // a gateway condition
2 = application.amount > 50000
3
4 // a range test
5 = application.score in [600..720]
6
7 // an output mapping shaping a result variable
8 = { band: if score >= 700 then "A" else "B" }
```

The three listings together are the runtime in miniature: the values file sizes the engine, the service task hands work to it by job type, and FEEL carries the small decisions inside the model — with everything larger pushed out to the governed components the rest of this part describes.

27.6 Applying the chapter in practice

The principles this chapter describes are most valuable when applied deliberately and reviewed periodically. A team that applies them once, at initial design, captures their value at that moment; a team that returns to them at each major change, each annual review, and each post-incident analysis captures their value continuously. The platform evolves; the principles hold; the specific choices they inform must evolve with the platform.

Worked example: partitioning a Zeebe cluster for a workload

Problem. A bank’s platform must run a peak of 30 process instances started per second. Zeebe distributes processing across *partitions*, and the team’s benchmarking shows one partition comfortably handles 25 instances per second of this workload. For resilience, each partition is *replicated* three times — a leader and two followers — and the platform runs across 3 availability zones with one replica of each partition in each zone. If each broker node hosts at most 4 partition replicas, how many partitions are needed for throughput, and how many broker nodes does the resulting cluster require?

Setup. The number of partitions is the peak throughput divided by per-partition capacity, rounded up. The total number of partition replicas is the number of partitions times the replication factor. The number of broker nodes is the total replicas divided by the per-node replica limit, rounded up — and then checked against the requirement that the cluster spread across 3 zones.

Working. Partitions needed for throughput:

$$\frac{30 \text{ instances/s}}{25 \text{ instances/s per partition}} = 1.2 \Rightarrow 2 \text{ partitions.}$$

Total partition replicas, at replication factor 3:

$$2 \text{ partitions} \times 3 = 6 \text{ replicas.}$$

Broker nodes needed, at 4 replicas per node:

$$\frac{6}{4} = 1.5 \Rightarrow 2 \text{ nodes by capacity.}$$

But the resilience requirement is one replica of each partition in each of 3 zones, which means at least 3 broker nodes — one per zone — so that each zone holds a full set of replicas. Three nodes also comfortably hold the 6 replicas (2 per node, within the limit of 4).

Discussion. The cluster is sized at three broker nodes, and the worked example exists to show *which* requirement set that number. The throughput math asked for only two partitions and would have been satisfied by two nodes — echoing [Chapter 21](#)’s lesson that banking and insurance orchestration workloads are rarely throughput-bound. The binding constraint was resilience: spreading a replica of every partition across three availability zones requires at least three nodes, one per zone, so that the loss of any single zone leaves a complete, functioning set of replicas. This is [Chapter 24](#)’s availability argument made concrete in Zeebe’s own vocabulary of partitions and replicas. A team that sized this cluster from the throughput number alone would have built a two-node cluster that cannot survive a zone failure — correctly sized for the easy requirement and wrongly sized for the one that matters.

Practical next steps

For the busiest process you intend to run on Camunda, estimate its peak instances-per-second, and separately write down its resilience requirement — how many zones, what replication. Size the partition count from the first and the broker-node count from the second. If the two numbers disagree, the resilience requirement wins, and the gap between them tells you how far your platform is from being throughput-bound.

Chapter in brief

A Camunda 8 platform is a set of cooperating components, not one program. Zeebe is the process engine — the concrete orchestration core — and it is distributed and horizontally scalable by design, built on a replicated, append-only event log rather than a relational database, which is what gives it clean survival of node and zone failure. Processes reach the engine through the Zeebe gateway; job workers are separate, independently scaling clients that connect through it and do the automated work. FEEL is the deliberately restricted expression language for conditions and mappings — anything more substantial belongs in a governable component. A Zeebe cluster is sized by resilience and zone spread, not by throughput.

Modelling, Operating, and Connecting

28.1 The components people use

Chapter 27 treated the components that run the platform. This chapter treats the components people interact with directly: the tools for modelling processes, for operating and monitoring them, for the human work inside them, for analysing how they perform, and for connecting them to the outside world. Each maps onto a discipline the manuscript has already established, and seeing the mapping is the point — the components are not a miscellany but the concrete handles for the architecture's principles.

Figure 28.1 places the components around the engine. The sections that follow take each in turn, and the recurring move is to name, for each component, the discipline from earlier in the manuscript that it makes concrete.

28.2 The Modeler: where models are built

A process model has to be created somewhere, and in Camunda that place is the *Modeler*. It exists in two forms — a desktop application and a collaborative web application — and both produce the same artefacts: BPMN process models, DMN decision models, and the forms that human tasks present.

The Modeler matters to this manuscript for more than the obvious reason. It is the tool in which the central discipline of Part I — that the process model is an explicit, shared, readable artefact — either holds or fails. A Modeler that only an engineer can use produces models only an engineer can read, and the business ownership that **Chapter 2** required quietly evaporates. The collaborative web form of the Modeler is significant precisely because it lets a process owner, an architect, and an engineer work on the *same* model: the diagram a business stakeholder reviews is the diagram that will be deployed and run. The Modeler is where the “one artefact, not a business diagram plus a diverging implementation” principle of **Chapter 3** is kept or lost.

The Modeler is also where the executable detail of **Chapter 2**'s three modelling levels is added: the technical type of each task, the input and output mappings of **Chapter 3**'s data-flow discipline, the FEEL expressions, the rule references. A model leaves the Modeler ready to deploy to Zeebe.

Tip

Treat the Modeler as a shared workspace, not an engineer's tool. If the people who own a process cannot open, read, and challenge its model in the Modeler, the model has failed the readability test that justifies using BPMN at all — and the institution has, without noticing, gone back to delegating its processes to whoever wrote the implementation.

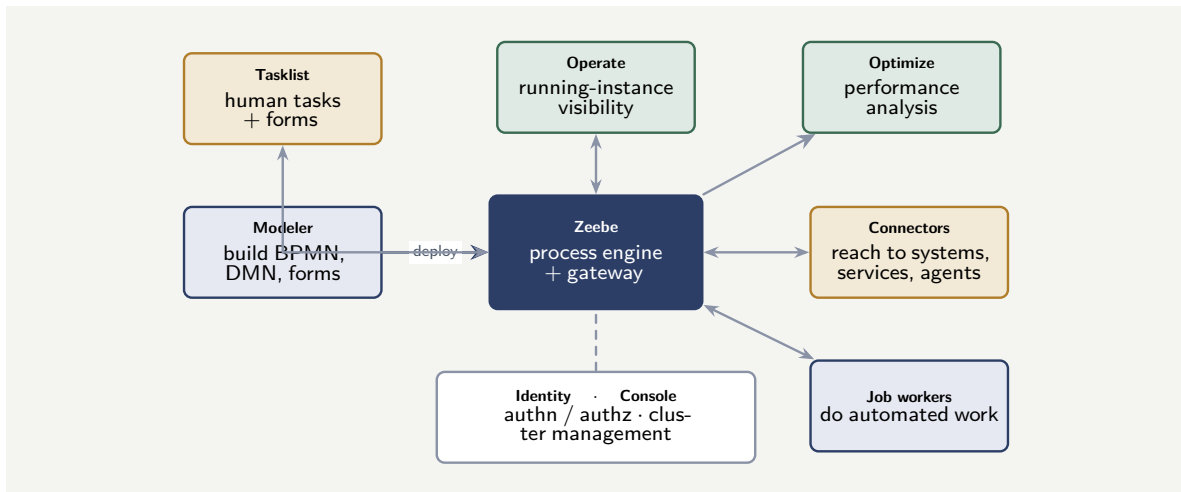


Figure 28.1. The Camunda 8 components and how they relate. Zeebe runs the processes; the Modeler builds the models deployed to it; Tasklist carries the human work; Operate and Optimize provide running visibility and performance analysis; Connectors and job workers reach the outside; and Identity and Console govern access and the cluster.

28.3 Operate: running-process visibility

Once models are deployed and instances are running, the institution needs to see what is happening — and *Operate* is the component that provides it. Operate is the concrete realisation of the *process observability* layer that [Chapter 125](#) of Part XIV will describe: it shows, for every deployed process, the instances that are running, where each instance’s token currently sits, which paths instances have taken, where instances are waiting, and — importantly — where instances have failed and why.

That last capability deserves emphasis. When a process instance hits a problem — a service task whose job has failed all its retries, an *incident* in Camunda’s vocabulary — Operate is where an operator sees it, understands it, and resolves it: correcting the data, or retrying the job, once the underlying cause is fixed. This is the concrete machinery behind [Chapter 125](#)’s “observe, signal, respond”: Operate is the signal and part of the response.

Operate is, in short, the window onto the running platform. An institution that has deployed processes to Zeebe but given its operators no Operate-style visibility has built the orchestration and discarded the observability — which [Chapter 125](#) identified as discarding the platform’s main advantage over a pile of bots.

28.4 Tasklist and forms: the human work

[Chapter 3](#) introduced the *user task* as the modelled point where human judgement enters a process. *Tasklist* is the component where that human work actually happens: it presents each person the tasks assigned to them, shows the relevant context and the form to complete, and returns the result to the engine so the process can continue.

Tasklist works hand in hand with *forms* — the structured screens, built in the Modeler, that a user task presents. A form gives the person the work instruction, shows them the information the decision needs, and collects their input in a structured, validated shape that the process can act on.

The architectural significance is that Tasklist is where the manuscript’s recurring discipline

— a named, accountable human at every consequential decision — becomes a concrete, usable thing. [Chapter 23](#) placed a human task on the path of every consequential agentic decision; [Chapter 127](#) of Part XIV insisted that the human be supported, not merely nominally responsible. Tasklist is the support: it is how the accountable human is brought the right case, with the right context, in a form they can act on. A platform that automates heavily but gives its human tasks a poor or contextless interface has made its accountability an inconvenience rather than a genuine control.

28.5 Optimize: analysing how processes perform

Operate shows what is happening *now*; *Optimize* analyses what has happened *over time*. It draws on the history of process execution to answer the questions a process owner asks of a process as a whole: where is the bottleneck, how long does each stage really take, which paths are most common, where do instances most often wait or fail, are the SLAs being met, and how is all of this trending.

Optimize is the realisation of the analytical side of [Chapter 125](#)'s observability — the dashboards and reports through which a process owner exercises the accountability that [Chapter 2](#) assigned them. It is also the natural home for some of [Chapter 125](#)'s *decision observability*: the distribution of decision outcomes over time, the drift in how often a process takes each path, the kind of shift that [Chapter 125](#)'s worked example flagged. Where Operate is the operator's tool for the individual troubled instance, Optimize is the process owner's tool for the health and the trend of the process as a population.

28.6 Connectors: the reach to the outside

A process is almost never self-contained; it must reach external systems, and *Connectors* are the components that provide that reach. A connector is a reusable, configurable integration — to a system of record, to an external service, to a messaging system, to a document service — that a service task can use without bespoke integration code being written for every process.

Connectors map directly onto [Chapter 25](#)'s treatment of integration. They are, in effect, pre-built job workers for common integration needs, and the integration styles of [Chapter 25](#) — synchronous call, asynchronous message, event subscription — appear as different kinds of connector. Two of [Chapter 25](#)'s disciplines apply to them directly. First, a connector to a core banking or policy administration system should sit behind the *anti-corruption layer*: the connector handles the mechanics of reaching the system, but the clean, business-meaningful interface the process model sees is still the institution's design, not the legacy system's data model leaking through. Second, a connector that has an external effect — moving money, writing to a system of record — must honour the *idempotency* discipline of [Chapter 122](#), because the job behind it will be retried.

A particular kind of connector matters for the agentic layer: a connector to a large-language-model or agent service is how an agentic step, in [Chapter 23](#)'s sense, is reached from a process. Camunda's support for *agentic BPMN* — modelling reasoning steps, with memory and human-in-the-loop boundaries, in the same executable model as the deterministic flow — is the product expression of this manuscript's central argument: the agent is a contained component inside a governable process model, not a thing that runs outside it. The connector, and the modelled boundaries around it, are where [Chapter 23](#)'s four containments are concretely enforced.

A connector makes reaching an external system easy — and easy reach is exactly what [Chapter 25](#) and [Chapter 120](#) warned can go wrong. A connector to a system of record still belongs behind a clean interface, not wired raw into process models; a connector with an external effect still must be idempotent; and a connector to an agent or model service is still subject to [Chapter 23](#)'s containment. The component makes the integration convenient; it does not make the discipline optional.

28.7 Identity and the supporting components

Two further components complete the picture. *Identity* is the component that manages authentication and authorisation — who, and what, may do what on the platform. It is the concrete realisation of the identity model of Part IV's security chapter: the human roles, the service identities, and the permissions that govern them are configured and enforced here, and the least-privilege discipline of that chapter is applied through Identity. *Console* is the component through which clusters themselves are configured and managed — the administrative control plane of [Chapter 24](#)'s “platform as code” principle.

These components are easy to treat as background plumbing, and that is a mistake. Identity is where the accountability link of Part IV's security chapter — every action traceable to an accountable identity — is either genuinely enforced or quietly left full of holes. The supporting components are where a good architecture is made real or undermined.

28.8 Applying the chapter in practice

The principles this chapter describes are most valuable when applied deliberately and reviewed periodically. A team that applies them once, at initial design, captures their value at that moment; a team that returns to them at each major change, each annual review, and each post-incident analysis captures their value continuously. The platform evolves; the principles hold; the specific choices they inform must evolve with the platform.

Worked example: mapping a claims process onto the components

Problem. An insurer is building the claims process of [Chapter 123](#) on Camunda. The process, as modelled, has: a BPMN model with the *notify-triage-validate cover-assess-decide-settle* flow; a DMN model for the cover rules; a fraud model and a severity model invoked as services; an agent that assesses claim evidence; human tasks for adjuster review and for the disposition of suspected fraud; and integrations to the policy administration and payments systems. The architecture team wants a single statement of which Camunda component is responsible for each of these, as a check that nothing has been left without a home.

Setup. We take each element of the modelled process in turn and name the component that owns it, then check the list against the disciplines the manuscript has established.

Working. Element by element:

The *BPMN model* and the *DMN cover-rules model* are built in the *Modeler* and deployed to and run by *Zeebe*.

The *fraud* and *severity models* are invoked as service tasks; the work is done by *job workers* (clients) connecting through the *Zeebe gateway*, or by model-service *connectors*.

The *agent* that assesses evidence is reached by a *connector* to an agent service — the agentic-BPMN step — and is contained by the modelled boundaries around that connector.

The *human tasks* for adjuster review and fraud disposition are presented and completed in *Tasklist*, using *forms* built in the Modeler.

The *integrations* to policy administration and payments are *connectors*, sitting behind the anti-corruption layer; the payments connector is built to be idempotent.

Running-instance visibility and incident resolution are provided by *Operate*; the process owner's analysis of cycle times, bottlenecks, and decision-rate trends is provided by *Optimize*.

Authentication, the adjusters' and operators' roles, and the service identities are managed by *Identity*; the cluster itself is managed through *Console*.

Every element has a home, and the check passes.

Discussion. The exercise looks like bookkeeping and is in fact a genuine design control. Its value is that it forces two questions that troubled platforms answer badly. The first is *has anything been left homeless?* — a step in the model that no component is clearly responsible for is a step that will be improvised at build time, usually as logic stuffed somewhere it should not be. The second, and subtler, is *is each element in the component that makes it governable?* Notice that the mapping above is not merely a list of components; at each line it carries a discipline — the agent behind a connector *with its containment*, the payments connector *idempotent*, the integrations *behind the anti-corruption layer*, the human tasks given *real context* in Tasklist. A claims process where every element has a home *and* each home enforces the relevant discipline is a process that has been designed, not assembled. The component map is how an architecture review confirms, before a line of the platform is built, that the architecture of Parts I and II has actually been applied.

Practical next steps

For one process your institution intends to build on Camunda, write the component map of the worked example: every modelled element, the component responsible, and the discipline that component must enforce for that element. The elements you cannot cleanly place, and the disciplines you cannot cleanly attach, are the parts of the design that are not yet finished — and finding them on one page is far cheaper than finding them in production.

Chapter in brief

The Camunda components people use map directly onto the manuscript's disciplines. The Modeler is where the explicit, shared, readable model of Part I is built or lost. Operate is the realisation of process observability — running-instance visibility and incident resolution. Tasklist and forms are where the accountable human of every consequential decision is genuinely supported. Optimize analyses how processes perform over time, serving the process owner's accountability and decision observability. Connectors provide the reach of [Chapter 25](#)'s integration — still subject to the anti-corruption layer, idempotency, and, for agent connectors, containment. Identity enforces the security model's accountability link. A component map of a process is a real design control.

Routing: Gateways and the Control of Flow

29.1 Routing as the heart of a process

A process model is, at bottom, two things: a set of activities and a set of *routing* decisions that determine which activities a given instance visits and in what order. [Chapter 3](#) introduced the gateway as one BPMN element among several; this chapter treats routing in the depth it deserves, because the correctness of a process is very largely the correctness of its routing. An activity that does the wrong thing is a bug in one place; a gateway that routes wrongly sends entire classes of cases down the wrong path, and in a lending or claims process that is the difference between a decision made correctly and a decision made not at all.

Routing in BPMN is expressed through gateways, through conditional sequence flows, and through events. This chapter takes each mechanism in turn, with the emphasis throughout on the disciplines that keep routing correct, auditable, and readable — the same three properties the manuscript has demanded of every modelled artefact.

29.2 The exclusive gateway and the discipline of the default

The *exclusive gateway* — the data-based XOR gateway — is the workhorse of process routing. It has one incoming flow and several outgoing flows, each outgoing flow guarded by a condition, and it routes the instance down exactly one of them: the first whose condition evaluates true.

The phrase “the first whose condition is true” carries a warning. If the conditions on a gateway’s outgoing flows are not mutually exclusive, two of them may both be true, and which one fires then depends on their evaluation order — a fragile and usually unintended dependency. And if the conditions are not exhaustive, an instance may arrive at the gateway with *no* outgoing condition true, at which point the engine raises an error and the instance stops. Both failure modes are routing defects, and both are preventable by a single discipline.

The discipline is the *default flow*. Every exclusive gateway has one outgoing flow marked as the default — taken when, and only when, no other flow’s condition is true. With a default flow, the gateway is exhaustive by construction: there is always somewhere for the instance to go. The conditions on the non-default flows should still be written to be mutually exclusive, but the default flow guarantees that the case the modeller did not foresee — the unexpected input value, the data that should have been impossible — reaches a defined path rather than halting the instance.

What the default flow should route *to* is itself a design decision. In a financial process the safe target is rarely “carry on as normal.” It is usually a path that stops, escalates, or routes to a human: an instance whose data does not match any expected case is, almost by definition, an instance the automated process does not understand, and the right response to

not understanding a case is to refer it, not to guess.

Tip

Give every exclusive gateway a default flow, and make the default route to a human or an exception path — never to the normal continuation. The non-default conditions express what the process expects; the default catches what it did not. A gateway with no default flow is a gateway that halts the instance on the first input its modeller failed to anticipate, and in a high-volume process that input *will* arrive.

29.3 The parallel gateway and the join

The *parallel gateway* — the AND gateway — expresses concurrency. In its splitting form it has one incoming flow and several outgoing flows, and it activates *all* of them: the instance now has several tokens, proceeding concurrently. In its joining form it has several incoming flows and one outgoing flow, and it *waits* — the token does not pass until every incoming flow has arrived.

The parallel gateway is how a process does genuinely independent work at the same time. [Chapter 25](#)'s latency worked example made the case: an identity check, a sanctions screen, and an affordability calculation that do not depend on one another should run concurrently, not in sequence, so that the process waits once for the slowest rather than once for the sum.

Two disciplines govern parallel gateways. The first is that the split and the join must be *balanced*: tokens created by a parallel split should be consumed by a corresponding parallel join. An unbalanced parallel structure — a split whose branches do not all reach a join, or a join that waits for a token that a routing decision upstream means will never arrive — produces either an instance that completes with tokens still live, or an instance stuck forever at a join waiting for a token that is not coming. The second discipline is that the branches must genuinely be *independent*: parallel routing is correct only when the concurrent branches do not need each other's results and do not write the same process variables. Branches that share a variable are a race condition, exactly as in any concurrent program.

29.4 The event-based gateway

The *event-based gateway* routes not on data but on *what happens first*. It has one incoming flow and several outgoing flows, each leading to an event — a message, a timer, another signal — and the instance waits at the gateway until one of those events occurs, then takes that event's flow.

This is the natural model for a process that waits on the outside world with more than one possible outcome. [Chapter 121](#)'s lending offer was the example: after an offer is issued the process waits, and exactly one of three things happens first — the customer accepts, the customer rejects, or the offer expires. An event-based gateway with a message event for accept, a message event for reject, and a timer event for expiry models this directly and correctly: whichever occurs first wins, the others are discarded, and the instance proceeds down one path.

The event-based gateway is what makes the long-running, waiting, interruptible process of [Chapter 3](#) expressible. Without it, “wait for the customer, but not forever” has to be simulated with polling and timers in application code; with it, the wait and its time limit are one explicit, readable construct on the diagram.

29.5 Conditional flows and inclusive routing

Two further routing mechanisms deserve brief, careful treatment.

A *conditional sequence flow* is a flow that carries a condition without a gateway in front of it. It is occasionally convenient, but it tends to hide routing logic — a reviewer scanning for gateways will miss it — and the manuscript’s recommendation is to prefer explicit gateways, so that every routing decision is visible as a gateway on the diagram.

The *inclusive gateway* — the OR gateway — activates *every* outgoing flow whose condition is true, which may be one, several, or all. It is genuinely expressive: a process that must perform some subset of several checks, depending on the case, can say so in one gateway. But its joining behaviour is subtle — the join must wait for exactly the branches the split activated, no more — and that subtlety is a frequent source of stuck instances. The manuscript’s guidance follows [Chapter 3](#)’s principle of a restricted, declared element set: prefer to express “some subset of these checks” as explicit exclusive and parallel gateways, or as a decomposed subprocess, and reach for the inclusive gateway only when the alternative is genuinely less clear — and then review its join with particular care.

29.6 Writing routing conditions in FEEL

[Chapter 27](#) introduced FEEL as the restricted expression language Camunda uses throughout. Routing is where FEEL does most of its work: every condition on a gateway’s outgoing flow is a FEEL expression that evaluates to true or false, and writing those expressions well is a practical skill worth treating concretely.

A routing condition reads process variables and tests them. The simplest are direct comparisons. [Listing 29.1](#) shows the conditions on the three outgoing flows of a claim-routing gateway.

Listing 29.1. FEEL conditions on the outgoing flows of an exclusive gateway. Each is evaluated against the instance’s process variables; the first true condition wins, the default catches the rest.

```
1 // flow 1 -- fast-track
2 =claimValue < 1000
3
4 // flow 2 -- standard handling
5 =claimValue >= 1000 and claimValue < 10000 and priorFraudFlag = false
6
7 // flow 3 -- investigation
8 =claimValue >= 10000 or priorFraudFlag = true
9
10 // the default flow carries no condition -- it is taken
11 // when none of the above is true
```

Three properties of these expressions are worth drawing out. They are *side-effect free*: a FEEL condition reads variables and computes a boolean; it cannot change anything, which is what makes evaluating a gateway safe and repeatable. They are *total over the expected inputs*: taken together with the default flow, every possible combination of `claimValue` and `priorFraudFlag` reaches exactly one flow — the partition discipline of [Chapter 4](#), applied to sequence flows rather than a decision table. And they are *legible*: a business reviewer can read `claimValue >= 1000 and claimValue < 10000` and check it against the intended policy without being able to program.

FEEL also offers constructs that make conditions cleaner than a chain of comparisons. The *range test*, `claimValue in [1000..10000)`, expresses a half-open interval directly. The *list membership test*, `riskGrade in ["A","B","C"]`, replaces a string of or-comparisons. The *null-safe*

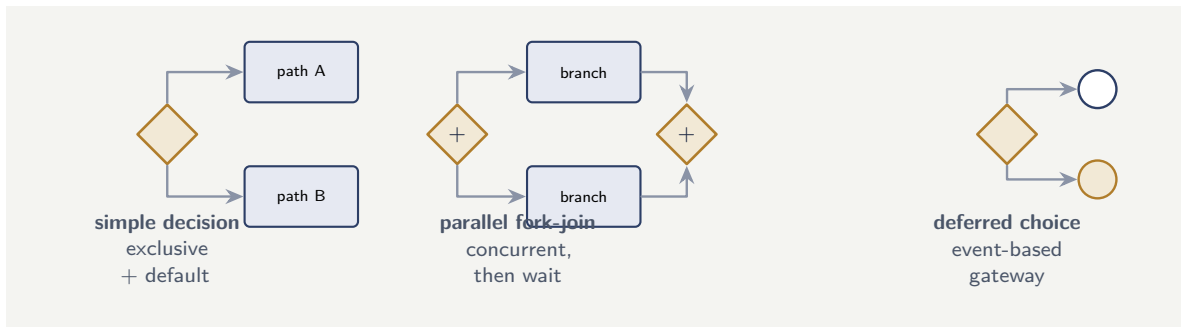


Figure 29.1. Three of the five recurring routing patterns. The simple decision routes on data; the parallel fork-join does independent work concurrently; the deferred choice waits for the world to decide. Exception interruption and the bounded loop complete the set.

access pattern matters in a regulated process: a condition that assumes a variable is present will fail if it is absent, so a robust condition either tests presence explicitly or is guarded by the data-validation discipline of [Chapter 3](#).

Tip

Keep routing conditions to comparisons, ranges, and membership tests over validated variables — the things FEEL does cleanly and a reviewer can check. If a gateway condition is growing into a long, nested FEEL expression, that is the signal that the decision is no longer a simple route but a genuine decision, and it belongs in a DMN table reached by a business-rule task — where it can be governed, versioned, and checked for completeness, as [Chapter 4](#) required.

29.7 A catalogue of routing patterns

Routing constructs combine into a small number of recurring *patterns*, and naming them helps a modeller reach for the right one and a reviewer recognise it. Five cover the great majority of financial-process routing.

The *simple decision* is one exclusive gateway with a default: the process evaluates a condition and takes one of several paths. It is the workhorse — a classification result routes a case, a threshold sends a claim one way or another.

The *parallel fork-join* is a parallel split, several independent branches, and a parallel join: genuinely independent work done concurrently, as in [Chapter 25](#)'s latency example. The discipline is balance and independence.

The *deferred choice* is an event-based gateway: the process reaches a point and waits for the world to decide which way it goes — accept, reject, or expire. The choice is made by what happens, not by data the process already holds.

The *exception interruption* is a boundary event on an activity: while the activity runs, a timer or an error or a message may interrupt it and divert the token down an exception flow. It is how an SLA, a cancellation, or a failure is modelled without cluttering the main flow.

The *loop* is a routing path that returns to an earlier point: a document rejected for poor quality routes back to a re-upload step, a review that requests more information routes back to the customer. A loop must always have a bounded exit — a counter, a timer, an escalation — so that a case cannot circle indefinitely.

Figure 29.1 draws three of the five. A modeller who recognises that a routing requirement *is* one of these patterns — rather than inventing a bespoke arrangement of gateways — produces models that other people can read, because the patterns are a shared vocabulary, and a reviewer who sees a routing structure that is *none* of the five has found something worth a second look.

Worked example: the cost of a missing default flow

Problem. A lending process has an exclusive gateway that routes on a `riskGrade` variable, with outgoing flows for grades A, B, C, and D. The gateway has no default flow. A change upstream introduces a fifth grade, E, for a small segment of applications — 0.8% of the 300,000 applications a year. Every grade-E application reaches the gateway, matches no condition, and the instance halts with an error requiring manual recovery at a cost of \$45 per instance. The defect runs for two months before it is noticed. What does the missing default flow cost, and what would the default flow have done instead?

Setup. The number of affected instances is the annual volume times the grade-E fraction times the two-month exposure (one sixth of a year). The cost is that count times the manual recovery cost. We then consider what a default flow would have changed.

Working. Grade-E applications in a full year:

$$300,000 \times 0.008 = 2,400.$$

Over a two-month exposure (one sixth of a year):

$$2,400 \times \frac{1}{6} = 400 \text{ halted instances.}$$

Manual recovery cost:

$$400 \times \$45 = \$18,000.$$

Discussion. The direct cost is \$18,000 of manual recovery, but the worked example is about more than the number. Consider what actually happened to those 400 applicants: their applications did not get a wrong decision — they got *no* decision, halting silently until someone noticed two months of accumulating errors. For a customer waiting on a loan, a silent halt is a poor experience and, depending on the product and jurisdiction, potentially a conduct concern. Now consider what a default flow would have done. Routed to a human-review path, the 400 grade-E applications would have been handled — by a person, correctly, the moment the first one arrived — and the unexpected grade would have surfaced as “why are these going to manual review?” within days, not months. The default flow does not merely avoid the \$18,000; it converts a silent, accumulating failure into a visible, handled exception. The arithmetic understates the case, which is the real lesson: the cost of a missing default flow is not only the recovery bill but the time the defect runs undetected, and a default flow that routes to a human is what keeps that time short.

Practical next steps

Audit the exclusive gateways in one of your process models. For each, check that a default flow exists and that it routes to an exception or human path, not to the normal continuation. Every gateway without a default flow is an instance halt waiting for the first input its conditions did not anticipate — and in a high-volume process, that input is a matter of when, not whether.

Chapter in brief

The correctness of a process is very largely the correctness of its routing. The exclusive gateway routes on data and demands a default flow — routing to a human or exception path — so that an unforeseen input reaches a defined path rather than halting the instance. The parallel gateway expresses genuine concurrency and requires balanced splits and joins over independent branches. The event-based gateway routes on what happens first and is what makes the long-running, interruptible wait expressible. Conditional flows hide routing and the inclusive gateway's join is subtle — prefer explicit exclusive and parallel gateways so every routing decision is visible.

Task Allocation and the Management of Human Work

30.1 Why allocation is a first-class concern

A hyperautomated process does not remove human work; it places it deliberately. [Chapter 3](#) introduced the user task as the modelled point where human judgement enters a process, and [Chapter 28](#) introduced Tasklist as the component where that work happens. This chapter treats the question those two left open: once a user task is created, *who* does it, and how is that decided? This is the problem of *task allocation*, and it is a first-class concern because a process that generates human tasks but allocates them badly does not perform, however well its automated parts are built.

Allocation has two sides. There is the modelled side — how the process model expresses who a task is for — and the operational side — how a population of tasks is balanced across a population of people so that work is done promptly, fairly, and by someone competent to do it. A serious treatment needs both.

30.2 Assignment, candidate groups, and candidate users

BPMN, as implemented in Camunda, gives a user task three related ways to say who it is for.

A task may have an *assignee*: a single, named person who owns the task. An assigned task appears on exactly that person's list and is theirs to complete. Direct assignment is appropriate when the process genuinely knows the right individual — the relationship manager named on the account, the underwriter who began the case — but it is a strong commitment, because an assigned task waiting on an absent person waits indefinitely unless something reassigns it.

A task may instead name *candidate groups*: one or more roles — “credit analysts,” “senior underwriters,” “fraud investigators” — any member of which *may* take the task. A task with candidate groups appears on the lists of everyone in those groups, unclaimed, until one of them *claims* it, at which point it becomes theirs. This is the most common and most robust pattern: the process commits to the *role* that should handle the task, not to an individual, and the work is resilient to any one person's absence.

A task may name *candidate users*: a specific set of individuals, any of whom may claim it. This sits between the other two — more specific than a group, less brittle than a single assignee.

The disciplined default is candidate groups. The process should reason in terms of *roles* — the competence and authority a task requires — because roles are stable, while individuals come and go. Direct assignment should be used where the process truly needs a named person, and even then the model should consider what happens when that person is unavailable.

Tip

Model user tasks against *roles*, through candidate groups, not against named individuals. A task assigned to a role is resilient: anyone with the role can pick it up, and the process does not stall when one person is on leave. Reserve direct assignment for the cases where the identity of the individual genuinely matters — and even there, model the fallback for when that individual is unavailable.

30.3 The claim, the worklist, and the four-eyes principle

The lifecycle of a group-allocated task is worth stating precisely, because its stages are where allocation discipline is enforced. The task is *created* when the instance reaches the user task; it is *available* — visible on the worklists of the candidate group, unclaimed; it is *claimed* when one person takes it, which removes it from others' worklists and prevents two people doing the same work; it is *completed* when that person submits the result, returning control to the process. A claimed task can also be *unclaimed* — returned to the group — if the person cannot complete it.

The claim step is what makes group allocation safe: without an atomic claim, two analysts could open the same case and both work it. With it, the first to claim owns it and the others see it disappear.

The claim mechanism also enables a control that regulated processes depend on: the *four-eyes principle*, the requirement that certain decisions be made or checked by two different people. A four-eyes control is modelled as two user tasks — a maker task and a checker task — with a constraint that the person who completes the checker task must not be the person who completed the maker task. The process model expresses the constraint; the allocation mechanism enforces it, by excluding the maker from the checker task's candidates. Four-eyes is pervasive in banking and insurance — in payment authorisation, in large-claim approval, in credit decisions above a limit — and it is fundamentally an allocation rule.

30.4 Allocation strategies: pull, push, and balanced

How tasks get from a worklist to a person is the operational side of allocation, and there are three broad strategies.

In a *pull* model, tasks sit in the group worklist and people claim them — they pull work when ready. Pull is simple and respects individual pace, but it has two failure modes: people may cherry-pick the easy tasks, leaving the hard ones to age, and an urgent task may sit unclaimed simply because no-one happened to look.

In a *push* model, the platform assigns each task to a specific person when it is created — it pushes work to people. Push gives control over who gets what and prevents cherry-picking, but a naive push can assign work to someone absent or already overloaded.

In a *balanced* model — the mature pattern — the platform pushes, but intelligently: it assigns each task to a member of the candidate group chosen by current workload, skill, availability, and the task's urgency. A balanced allocator levels load across the team, routes a task needing a particular competence to someone who has it, and escalates an urgent task rather than letting it wait.

Allocation strategy interacts with the SLA timers of [Chapter 3](#). A task that is approaching its deadline should not be sitting politely in a pull queue; the boundary timer should fire, and the response — escalation, reassignment, prioritisation — is itself an allocation decision.

Allocation and the process model's timers are two halves of one mechanism for getting the right work done in time.

30.5 Implementing an allocator: round-robin and load-balancing

The previous section described the balanced strategy in principle; this section shows where it lives in code and gives two concrete algorithms. Both are short enough to read in full, and the point of showing them is that “intelligent allocation” is not a vague aspiration — it is a small, ordinary piece of software at a well-defined seam.

30.5.1 Where the allocator runs

The natural seam is the *creating* user-task listener of [Chapter 28](#). In Camunda 8 a task listener is implemented as a job worker, and a *creating* listener fires the moment the engine creates a user task — before anyone sees it. A creating-listener job worker may return *corrections* to the task, and one of the attributes it may correct is the *assignee*. That is exactly the hook an allocator needs: when a task is created with a candidate group but no assignee, the creating listener computes *which* member of the group should get it and corrects the assignee to that person. The process model stays clean — it names a candidate group — and the allocator, attached as a listener, turns the group into a specific assignment.

The two algorithms below are the body of that listener. Each takes the candidate group's members and chooses one; the listener then returns the chosen user as the assignee correction.

30.5.2 Round-robin allocation

Round-robin is the simplest balanced allocator: it hands successive tasks to successive members of the group, cycling back to the start, so that over time every member receives an equal *count* of tasks. It needs only a single piece of remembered state — a counter — per group.

Listing 30.1. A round-robin allocator. A counter per candidate group cycles through the group's members; each new task goes to the next member in turn.

```
1 public class RoundRobinAllocator {
2
3     // one counter per candidate group, shared safely
4     private final Map<String, AtomicInteger> counters
5         = new ConcurrentHashMap<>();
6
7     /** Choose the next member of the group, in turn. */
8     public String allocate(String group,
9                             List<String> members) {
10         if (members.isEmpty()) {
11             return null; // no-one to assign to
12         }
13         AtomicInteger counter = counters.computeIfAbsent(
14             group, g -> new AtomicInteger(0));
15
16         // getAndIncrement wraps naturally via modulo
17         int next = Math.floorMod(
18             counter.getAndIncrement(), members.size());
19         return members.get(next);
20     }
21 }
```

Round-robin's virtue is that it is trivial, predictable, and free of any dependency on live data. Its limitation is just as clear: it equalises the *number* of tasks, not the *work*. If one task is a two-minute check and the next a two-hour investigation, round-robin still alternates them

evenly, and the member who drew the long tasks ends the day overloaded while the member who drew the short ones is idle. Round-robin is the right choice when tasks are reasonably uniform in effort; when they are not, the allocator must look at load.

30.5.3 Load-balancing allocation

A *load-balancing* allocator assigns each new task to the group member who currently has the *least outstanding work* — not the fewest tasks, but the least estimated effort. It needs live state: for each member, the sum of the effort of the tasks they currently hold. That figure rises when a task is assigned and falls when one is completed — which is why the same allocator must also be invoked from a *complete* listener, to decrement the load.

Listing 30.2. A least-loaded allocator. Each new task goes to the available member carrying the least estimated outstanding effort.

```
1 public class LoadBalancingAllocator {
2
3     // current outstanding effort (minutes) per member
4     private final Map<String, Integer> load
5         = new ConcurrentHashMap<>();
6
7     /** Assign to the available member with the least load. */
8     public synchronized String allocate(
9         List<String> members,
10        Set<String> available,
11        int taskEffortMinutes) {
12
13        String chosen = null;
14        int lowest = Integer.MAX_VALUE;
15
16        for (String member : members) {
17            if (!available.contains(member)) {
18                continue; // skip absent members
19            }
20            int current = load.getOrDefault(member, 0);
21            if (current < lowest) {
22                lowest = current;
23                chosen = member;
24            }
25        }
26        if (chosen != null) {
27            // reserve the load now, on assignment
28            load.merge(chosen, taskEffortMinutes, Integer::sum);
29        }
30        return chosen;
31    }
32
33    /** Called from the complete listener when a task ends. */
34    public synchronized void release(String member,
35        int taskEffortMinutes) {
36        load.merge(member, -taskEffortMinutes,
37            (a, b) -> Math.max(0, a + b));
38    }
39 }
```

The load-balancing allocator does three things round-robin cannot. It accounts for *effort*, not count, so a member who drew long tasks is not given more until their load falls back. It accounts for *availability* — the `available` set excludes anyone on leave or off-shift, so work is never assigned to someone who is not there, the failure mode the previous section warned of in a naive push. And, because `release` is called when a task completes, the load figure tracks reality rather than drifting.

A production allocator extends this skeleton in ways the chapter has already motivated: a *skill* filter, so a task needing a particular competence is only offered to members who have it; a *priority* input, so an urgent task can pre-empt; and the audit hook of [Chapter 28](#)'s listeners, so every automated assignment is recorded. But the core is exactly what is shown — a few lines that turn a candidate group into the right specific assignee.

A load-balancing allocator holds live state — each member's outstanding load — and that state is only correct if it is updated on *both* sides: incremented when a task is assigned and decremented when one completes. An allocator wired only to the *creating* listener, with no *complete* listener calling `release`, has built a load figure that only ever rises — and within a day it will believe the whole team is at capacity and allocate blindly. Wire both listeners, or the balancer silently degrades to a worse allocator than round-robin.

30.6 Task data, forms, and the cost of context

Allocating a task to the right person is half the problem; the other half is giving that person what they need to do it well. A user task is not just an item on a list — it carries *data*, presents a *form*, and the quality of both decides whether the human decision is fast and sound or slow and error-prone.

The data a task carries is drawn from the process variables of [Chapter 3](#). A well-designed user task is configured with explicit *input mappings* — it receives exactly the variables the decision needs — and explicit *output mappings* — the person's decision is written back to named variables the downstream gateway will route on. This is the data-flow discipline applied to human work: a task that simply exposes the entire process state to the person is both a security concern, in the sense of Part IV's minimisation principle, and a usability one, because the relevant fact is buried among irrelevant ones.

The form is what the person actually sees. [Chapter 28](#) introduced forms as structured screens built in the Modeler; the allocation chapter's concern is that the form present the *right* context. A claims adjuster deciding a case needs the claim, the policy terms, the evidence summary, and the model and agent outputs — assembled onto one screen. Forcing the adjuster to gather that context from five systems is the hidden cost that makes a 25-minute task into a 40-minute one, and it is a cost the process model controls, by deciding what data the task carries and the form presents.

Camunda also provides *task listeners*: hooks that fire at points in the task's lifecycle — on creation, on assignment, on completion. A listener is how cross-cutting allocation logic is attached without cluttering the process model: a create-listener that records when the task entered the queue for SLA measurement, an assignment-listener that notifies the person, a complete-listener that captures the decision into the audit record of Part IV's security chapter. Listeners are the technical seam where allocation, observability, and audit meet the individual task.

Figure [30.1](#) draws the lifecycle with its listener hooks and the SLA timer. The picture makes a design point visible: the period between *available* and *claimed* is queue time, and the period between *claimed* and *completed* is handling time. They are different problems — queue time is an allocation and capacity issue, handling time is a context and form issue — and the task listeners are what let a platform measure each separately rather than seeing only the undifferentiated total.

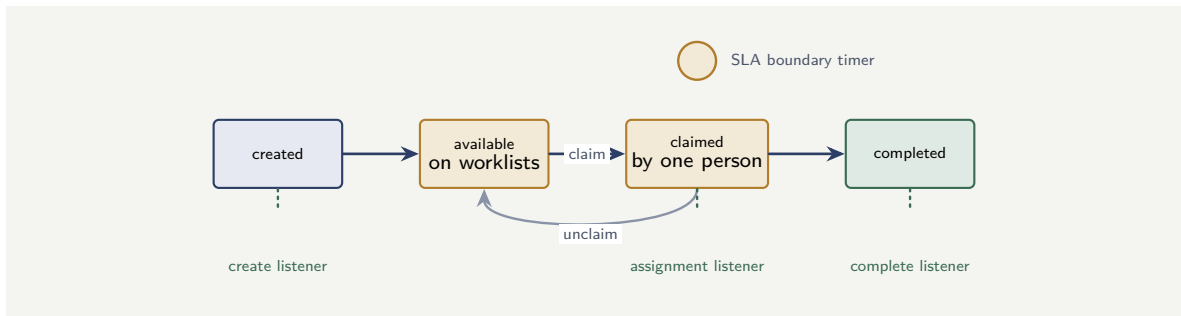


Figure 30.1. The user-task lifecycle, with task listeners and the SLA timer. A task is created, becomes available on the candidate group’s worklists, is claimed by one person, and is completed — and may be unclaimed back to the group. Listeners fire at each transition; the boundary timer guards the deadline.

Tip

Design what a user task *carries* and what its form *shows* as deliberately as you design who it is allocated to. A correctly allocated task with a poor, contextless form is still a slow, error-prone task. Use input and output mappings so the task carries exactly the decision’s data, and use task listeners to measure queue time and handling time separately — they are different problems with different fixes.

Worked example: sizing a team against task arrival and handling time

Problem. A claims operation has a user task — adjuster review — that arrives at a rate of 480 tasks per working day. Each task takes an adjuster, on average, 25 minutes to complete. An adjuster has 6.5 productive hours in a working day. The operation wants the team sized so that, on average, tasks do not accumulate — the team’s daily capacity at least matches the daily arrival — and then wants to know the further allowance needed so that the team is no more than 85% utilised, since a team run at 100% of capacity has no slack and queues explode.

Setup. The daily work arriving is the task count times the handling time. One adjuster’s daily capacity is their productive hours. The minimum team size is arriving work divided by per-adjuster capacity. The 85%-utilisation team size is the minimum divided by 0.85.

Working. Daily work arriving:

$$480 \text{ tasks} \times 25 \text{ min} = 12,000 \text{ minutes} = 200 \text{ hours.}$$

One adjuster’s daily capacity: 6.5 hours. Minimum team size to match arrival:

$$\frac{200}{6.5} \approx 30.8 \Rightarrow 31 \text{ adjusters.}$$

Team size for no more than 85% utilisation:

$$\frac{30.8}{0.85} \approx 36.2 \Rightarrow 37 \text{ adjusters.}$$

Discussion. The two numbers — 31 and 37 — are both correct answers to different questions, and the gap between them is the point. Thirty-one adjusters is the size at which daily capacity exactly matches daily arrival, and a team sized at 31 will, on a spreadsheet, “keep

up.” In practice it will not. Task arrivals are not smooth — they cluster — and handling times vary, and a team run at 100% utilisation has no slack to absorb either kind of variation: a busy morning produces a queue that the rest of the day cannot drain, and the backlog ratchets up. This is not a staffing opinion; it is the behaviour of queues, and it is why the operation asked for the 85% figure. The six extra adjusters — 37 rather than 31 — are not overstaffing; they are the slack that keeps task waiting times stable instead of explosive. The lesson for allocation is twofold. First, a team must be sized against arrival *and* variability, not against the average alone. Second — and this is where allocation strategy earns its place — a balanced allocator that levels load and escalates urgent tasks effectively raises the utilisation at which the team stays stable, because it removes the self-inflicted variability of cherry-picking and idle queues. Good allocation is, in part, a way to buy back some of those six adjusters.

Practical next steps

For one user task in your institution, measure its arrival rate and its average handling time, compute the minimum team size and the 85%-utilisation team size, and compare both with the team you actually have. If you are staffed at or below the minimum, your task waiting times are not stable, and no allocation strategy will fully fix an under-sized team — though a balanced allocator will make a correctly-sized one perform far better.

Chapter in brief

Task allocation is a first-class concern: a process that allocates human work badly does not perform. A user task may name an assignee, candidate groups, or candidate users — and the disciplined default is candidate groups, modelling work against stable roles rather than individuals. The claim step makes group allocation safe and enables the four-eyes principle, which is fundamentally an allocation rule. Allocation strategies are pull, push, and balanced; the balanced allocator levels load, matches skill, and escalates urgency. It is implemented concretely as a *creating* task-listener job worker that corrects the assignee: a *round-robin* allocator cycles a counter through the group’s members, equalising task count, while a *load-balancing* allocator assigns each task to the available member with the least outstanding effort — and must be wired to both the creating and completing listeners so its load figures track reality. Teams must be sized against arrival and variability, not the average — and good allocation raises the utilisation at which a team stays stable.

Java Integration: Delegates and the Embedded Engine

31.1 Two generations, two integration models

A process model does the routing and holds the state, but the actual automated work — calling a system, evaluating logic, transforming data — is done in code, and in the Camunda world that code is most often Java. How that Java code connects to the engine is the subject of this chapter and the two that follow, and it is worth being clear at the outset that there are two distinct models, belonging to two generations of the technology.

The first model is the *embedded engine* with *Java delegates*. Here the process engine runs as a library *inside* a Java application; a service task in the model names a Java class — a delegate — and the engine calls that class directly, in the same process, when an instance reaches the task. This is the model of Camunda 7 and of the first generation of orchestration that [Chapter 21](#) described.

The second model is the *external worker*: the engine runs as a separate, clustered service, and the Java code connects to it over the network as a client, pulling jobs and reporting results. This is the model of Camunda 8 and Zeebe, and it is the subject of [Chapter 32](#).

This chapter treats the embedded model and the Java delegate. It does so for two reasons. Many institutions are running it now and will be for years, so it is live, practical knowledge. And understanding it makes clear *why* the external-worker model exists — the delegate's limitations are the external worker's reason for being.

Figure [31.1](#) contrasts the embedded delegate model with the external-worker model, and the contrast is the chapter's whole argument in one picture.

31.2 The Java delegate

A *Java delegate* is a class that implements a single method the engine will call. In its most common form it implements the `JavaDelegate` interface, whose one method, `execute`, receives a `DelegateExecution` — a handle onto the process instance through which the delegate reads and writes process variables.

Listing [31.1](#) shows a delegate for an affordability check in a lending process.

Listing 31.1. A Java delegate implementing an affordability check. The engine calls `execute` when an instance reaches the service task wired to this class.

```
1 @Component("affordabilityCheck")
2 public class AffordabilityCheckDelegate implements JavaDelegate {
3
4     private final AffordabilityService affordabilityService;
5
6     public AffordabilityCheckDelegate(AffordabilityService service) {
7         this.affordabilityService = service;
8     }
9 }
```

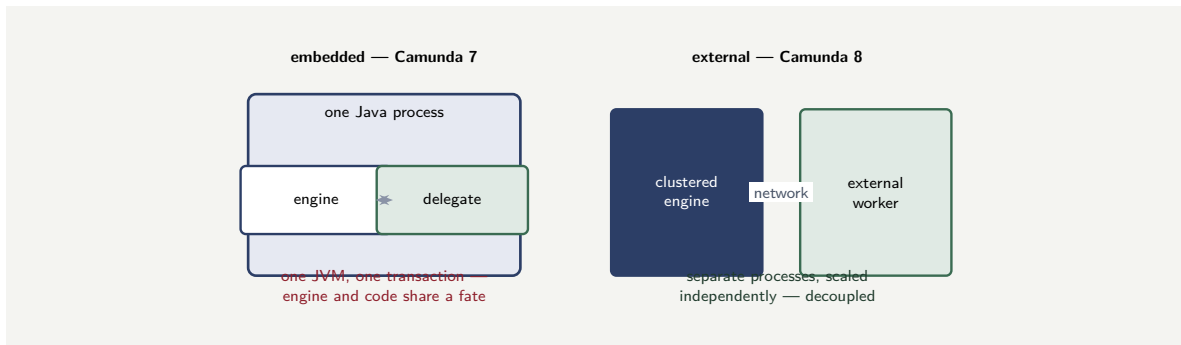



Figure 31.1. The two integration models. In the embedded model of Camunda 7, the engine and the Java delegate run in one Java process and share a transaction. In the external model of Camunda 8, the clustered engine and the worker are separate processes connected over the network — decoupled, and scaled independently.

```

10  @Override
11  public void execute(DelegateExecution execution) {
12      // read process variables set earlier in the flow
13      BigDecimal income = (BigDecimal) execution.getVariable("netIncome");
14      BigDecimal repay = (BigDecimal) execution.getVariable("repayment");
15
16      // perform the work
17      AffordabilityResult result =
18          affordabilityService.assess(income, repay);
19
20      // write results back for the next gateway to route on
21      execution.setVariable("affordable", result.isAffordable());
22      execution.setVariable("affordabilityRatio", result.getRatio());
23  }
24  }

```

The service task in the BPMN model is wired to this class — in Camunda 7, by a `camunda:delegateExpression` attribute referencing the Spring bean named `affordabilityCheck`. When an instance reaches that task, the engine invokes `execute`, the delegate does its work, and the instance proceeds.

Several disciplines, all of them consequences of the manuscript's earlier chapters, govern a well-written delegate.

The delegate should be *thin*. It reads the variables it needs, calls a proper service class to do the actual work, and writes the results back. The business logic belongs in that service class — testable on its own, free of any engine dependency — not in the delegate. A delegate stuffed with business logic is the chapter-5 anti-pattern, the fat orchestrator, in miniature.

The delegate's variable access should be *disciplined*: it reads named variables of known types and writes named variables of known types, which is the data-flow discipline of [Chapter 3](#) applied at the code boundary. A delegate that reads and writes process variables ad hoc is the hidden-coupling failure that chapter warned of.

And the delegate must be written with its *transactional behaviour* in mind, which is the subject of the next section.

31.3 The same work as an external worker

The delegate is the Camunda 7, embedded model. It is worth seeing the same piece of work written the other way — as a Camunda 8 *job worker* — so the two integration models stand in direct comparison rather than abstract description.

Listing 31.2 does what the delegate of Listing 31.1 did — score a credit application — but as a job worker. The differences are not cosmetic. There is no `JavaDelegate` interface to implement and no `execute(DelegateExecution)` method called by the engine in-process; instead a method is annotated with the *job type* it serves, and the Zeebe client invokes it when it has polled a job of that type from the remote engine. The variables arrive through the `ActivatedJob`, and the method's return value becomes the job's result variables — the engine is updated by a network call, not a shared transaction.

Listing 31.2. The credit-scoring work of Listing 31.1, written as a Camunda 8 job worker rather than an embedded delegate. The method is bound to a job type; its return value becomes the result variables.

```
1  @Component
2  public class CreditScoringWorker {
3
4      private final ScoringService scoringService;
5
6      public CreditScoringWorker(ScoringService s) {
7          this.scoringService = s;
8      }
9
10     // bound to the job type, not the engine's call
11     @JobWorker(type = "credit-scoring")
12     public Map<String, Object> score(ActivatedJob job) {
13         // variables arrive through the job
14         var amount = job.getVariable("amount");
15         var customerId = job.getVariable("customerId");
16
17         // the real work is in a plain, testable service
18         var result = scoringService.score(
19             customerId, amount);
20
21         // the return value becomes result variables
22         return Map.of(
23             "creditScore", result.score(),
24             "scoreBand",    result.band());
25     }
26 }
```

Two things in the listing carry the chapter's argument. First, the worker is still *thin* — it reads its variables, calls a plain `ScoringService`, and returns the result — exactly the discipline the delegate section insisted on; the integration model changed, the thinness discipline did not. Second, and decisively, the worker runs in its *own* process and updates the engine across the network: there is no shared transaction with the engine, which is the Camunda 7-versus-8 difference of [Chapter 10](#) seen in code, and the reason the next section's treatment of the embedded transaction is specifically a *Camunda 7* concern.

31.4 The engine, the delegate, and the transaction

The defining property of the embedded model — and the source of both its convenience and its danger — is that the engine and the delegate run in the *same transaction*.

When the engine advances an instance, it does so in a database transaction: the state change is written, the delegate runs, and the transaction commits. If the delegate runs cleanly, everything commits together — the state advance and whatever the delegate did. If the delegate throws an exception, the transaction *rolls back* — the state advance is undone, and the instance is left as if the step had not run.

This is convenient because it gives a clean all-or-nothing step: the process does not advance past a delegate that failed. But it is dangerous for one specific, important reason. If the

delegate's work includes a call to an external system — moving money, writing to a system of record — that external effect is *not* part of the engine's transaction and cannot be rolled back. If the delegate calls the payments system, the payment succeeds, and then the delegate throws before the transaction commits, the engine rolls back its state — but the money has moved. The process now believes the payment step did not happen; reality disagrees.

This is the embedded model's central hazard, and it is exactly the idempotency problem of [Chapter 31](#), met here at the level of Java code. The disciplines that contain it are firm: a delegate that has an irreversible external effect should do *only* that, kept in its own service task so the transactional boundary is as tight as possible; the external call must be idempotent, so that the retry which follows a rollback does not double the effect; and long-running or unreliable external work should not be done in an embedded delegate at all — it should be an asynchronous step, which is part of what the external-worker model of [Chapter 32](#) exists to provide.

An embedded Java delegate runs inside the engine's database transaction — but the external systems it calls do not. If a delegate moves money or writes to a system of record and then fails before the transaction commits, the engine rolls back its state while the external effect stands, and the process's view of the world is now wrong. Keep irreversible external effects in their own narrow service task, make them idempotent, and prefer an asynchronous, external-worker step for anything slow or unreliable.

31.5 Asynchronous continuations and transactions

The embedded model gives the modeller explicit control over where one transaction ends and the next begins, through *asynchronous continuations* — in Camunda 7, the `camunda:asyncBefore` and `camunda:asyncAfter` flags on an activity.

An asynchronous continuation tells the engine to commit the transaction at that point and continue the instance in a fresh transaction, picked up by a background job executor. This does two valuable things. It *bounds* the transaction: without async points, a long stretch of the process runs in one transaction, and a failure anywhere in that stretch rolls back the whole stretch. With an async point after each significant step, a failure rolls back only the current step, and the engine retries from there — the job-and-retry resilience of [Chapter 21](#), configured explicitly. And it provides *save points*: the instance's progress is durably committed at each async point, so a crash loses at most the work since the last one.

Placing async boundaries is therefore a real design activity in the embedded model. The guidance is to commit *after* any step with an external effect — so the effect and the state that records it are durable together, before anything else can fail — and to place async points frequently enough that no single transaction spans more than one risky operation. In the external-worker model of [Chapter 32](#), this becomes simpler, because every job is naturally its own transactional unit; but in the embedded model it is the modeller's explicit responsibility.

Worked example: the blast radius of a transaction boundary

Problem. An embedded lending process has a stretch of five sequential service tasks: *validate*, *score credit*, *call core banking to reserve funds*, *generate documents*, *notify customer*. In design

A, the whole stretch runs in a single transaction — no async continuations. In design B, there is an asynchronous continuation after every task. The *generate documents* task fails transiently on 2% of the 200,000 instances a year. When it fails, the transaction rolls back; the work that is rolled back must be redone on retry. Reserving funds in core banking takes, on average, 1.5 seconds of engine-blocking time, and re-doing it is the expensive part. How much redundant core-banking work does design A cause compared with design B?

Setup. The failing task is *generate documents*, the fourth of five. In design A, a failure there rolls back the whole single transaction — including the core-banking reserve, the third task — so the reserve is redone on retry. In design B, the async continuation after the core-banking task has already committed it; a failure in *generate documents* rolls back only that task, and the reserve is *not* redone. We count the failing instances and the redundant core-banking calls.

Working. Failing instances per year:

$$200,000 \times 0.02 = 4,000.$$

Design A: each failure rolls back the whole stretch, so the core-banking reserve — 1.5 seconds of work — is redone for every failing instance:

4,000 redundant reserves $\times$ 1.5 s = 6,000 seconds = 1.67 hours of redundant core-banking work.

Design B: the reserve was committed at the async point before *generate documents* ran; a failure there redoes only document generation. Redundant core-banking reserves: 0.

Discussion. Design A does 1.67 hours a year of completely redundant core-banking work, and design B does none — but the hours are not the real point. The real point is correctness, and it is sharper than the time figure suggests. The core-banking reserve is an *external effect*: redoing it on retry does not just waste 1.5 seconds, it issues a *second* reserve instruction to the core banking system. Unless that instruction is idempotent — [Chapter 31](#)’s discipline — design A risks reserving the funds twice. Design B, by committing the transaction at an async continuation immediately after the core-banking task, ensures the reserve happens once and stays done: a later failure cannot roll it back, because it is already committed. This is why async-boundary placement is a correctness decision, not a performance tuning knob. The rule the worked example illustrates is the one the chapter stated: commit immediately after any step with an external effect, so that the effect and the state recording it become durable together, before anything downstream can fail and trigger a rollback.

Practical next steps

Take one embedded process model and mark every service task that has an external effect — a call that moves money, writes to a system of record, or sends something irreversible. Check that an asynchronous continuation commits the transaction immediately after each one. Every external-effect task not followed by an async commit is a task whose effect can be stranded by a downstream rollback.

31.6 Choosing between the two models

The chapter has described the embedded model in detail and pointed repeatedly to the external-worker model of [Chapter 32](#) as its successor. It is worth drawing the comparison together explicitly, because an institution with a Camunda estate will often run both and must know which suits a given situation. [Table 31.1](#) sets the two side by side.

The table makes the trade-off plain. The embedded model’s shared transaction is a genuine convenience for purely internal, fast logic — a calculation, a data transformation — where

Table 31.1. The embedded-delegate model and the external-worker model compared. The external-worker model removes the embedded model’s central hazards at the cost of the embedded model’s transactional convenience.

Aspect	Embedded delegate	External worker
Engine location	library inside the application	separate clustered service
Direction of call	engine calls the code	code calls the engine
Transaction	code shares the engine’s transaction	code has its own, separate transaction
Scaling	engine and code scale together	code scales independently of the engine
Failure of the code	rolls back the engine’s state too	a job failure; the engine is unaffected
External-effect hazard	high — stranded effect on roll-back	contained — job and effect commit together
Language	Java, in the engine’s process	any language, anywhere
Best suited to	fast, in-process, transactional logic	calls to external systems, slow or unreliable work

all-or-nothing with the engine’s state advance is exactly what is wanted. But for the work that dominates a financial platform — calls to core banking, to payment rails, to identity services, to model and agent services — the shared transaction is the hazard this chapter has dwelt on, and the external-worker model’s separate transaction, independent scaling, and contained failure are decisively better. The practical guidance: on a Camunda 8 platform, use external workers as the default and reach for an in-process approach only for genuinely internal, transactional logic; on a Camunda 7 estate, treat every external-effect delegate with the transaction discipline this chapter has set out, and treat that discipline’s difficulty as a reason to plan the move to the external-worker model.

Chapter in brief

Camunda has two integration models: the embedded engine with Java delegates (Camunda 7), and the external worker (Camunda 8). A Java delegate implements `execute`, reading and writing process variables through a `DelegateExecution` — and should be thin, delegating real logic to a testable service class. The defining hazard of the embedded model is that the delegate runs inside the engine’s transaction but the external systems it calls do not, so a failed delegate can leave an external effect stranded while the engine rolls back. Asynchronous continuations bound the transaction and provide save points; commit immediately after any external effect. The external-worker model removes these hazards through a separate transaction and independent scaling, and is the default on Camunda 8; the embedded model suits only fast, internal, transactional logic.

Job Workers and the External-Task Pattern

32.1 The pattern and why it exists

[Chapter 31](#) ended with a list of the embedded delegate’s hazards: it runs inside the engine’s transaction, it couples the engine’s availability to the work’s, and it scales only as the engine scales. The *external-task pattern* — and the *job worker* that implements it — exists to resolve all three, and it is the model on which Camunda 8 is built. This chapter treats it in the depth its centrality deserves.

The idea is a clean inversion of the embedded model. Instead of the engine calling code, code calls the engine. When an instance reaches a service task, the engine does not invoke anything; it simply records that a *job* of a particular type is available. Separate processes — job workers — connect to the engine, ask for jobs of the types they handle, do the work in their own process and their own transaction, and report the result back. The engine and the worker are decoupled in every dimension that mattered in [Chapter 31](#): different processes, different transactions, independent scaling, independent failure.

32.2 The job lifecycle

A job moves through a precise lifecycle, and understanding it is understanding the pattern.

The job is *created* when an instance reaches a service task; it carries a *type* — a string such as *credit-scoring* or *payment-execution* — that says what kind of work it is, and the process variables the work needs.

A worker *activates* the job: it asks the engine for jobs of a type it handles, and the engine hands it one, with a *lock* for a defined period. The lock is essential — it ensures that while this worker holds the job, no other worker is given the same one, which is what makes it safe to run many workers of the same type concurrently.

The worker does the work and then *completes* the job, passing back any result variables; the engine records them and advances the instance. Or, if the work cannot be done, the worker *fails* the job, optionally with a retry count; the engine will make it available again until the retries are exhausted. Or the worker raises a *BPMN error*, which is different from a technical failure — it is a business outcome (“card declined,” “identity not verified”) that the process model is expected to catch and route on with a boundary error event.

If a worker activates a job and then crashes without completing or failing it, the *lock expires* and the engine simply makes the job available again for another worker. This is the property that makes the pattern resilient: a worker crash costs one retry, never a lost instance — the resilience [Chapter 21](#) promised, delivered by the lock-and-expiry mechanism.

32.3 Writing a job worker

A job worker, in Camunda 8, is ordinary Java code annotated to say which job type it handles. Listing 32.1 shows one for credit scoring.

Listing 32.1. A Camunda 8 job worker. The annotation binds the method to a job type; the method's return value becomes the job's result variables.

```
1  @Component
2  public class CreditScoringWorker {
3
4      private final CreditBureau bureau;
5
6      public CreditScoringWorker(CreditBureau bureau) {
7          this.bureau = bureau;
8      }
9
10     @JobWorker(type = "credit-scoring")
11     public Map<String, Object> score(ActivatedJob job) {
12         // read the inputs the job carries
13         String customerId = job.getVariable("customerId");
14
15         // do the work in this worker's own transaction
16         BureauReport report = bureau.pull(customerId);
17
18         // the returned map becomes the job's result variables
19         return Map.of(
20             "creditScore", report.getScore(),
21             "scoreBand",   report.getBand()
22         );
23     }
24 }
```

The contrast with the delegate of [Chapter 31](#) is instructive. There is no `DelegateExecution` reaching into the engine's internals; the worker receives an `ActivatedJob` — a self-contained package of inputs — and returns a map of outputs. The worker does not share a transaction with the engine; its work commits, or not, on its own. And the worker is just a client: it can run anywhere, be deployed independently, and be scaled by running more copies.

The same disciplines as for the delegate apply, and one is new. The worker should be *thin*, delegating real logic to a testable service class. Its variable access should be *disciplined* — named inputs, named outputs. And, the new one: because a job may be delivered more than once — a worker that completes a job just as its lock expires, a retry after an ambiguous failure — the worker's work must be *idempotent*. This is [Chapter 31](#)'s discipline once more, and in the external-task pattern it is not optional fine print; it is a structural property the at-least-once delivery of jobs requires.

A job worker can be handed the same job more than once — after a lock expiry, after an ambiguous failure, during a retry. The external-task pattern guarantees *at-least-once* execution, not exactly-once. Every job worker whose work has an external effect must therefore be idempotent: completing the same job twice must produce the same result as completing it once. A worker that is not idempotent is a double payment, or a double-booked loan, waiting for an unlucky lock expiry.

32.4 Tuning the worker fleet

A population of job workers — a *fleet* — has a handful of parameters that govern its behaviour under load, and they are worth understanding because they are where the pattern’s scalability is realised or squandered.

The *number of worker instances* of a given type sets the concurrency for that type of work. Because workers are independent clients, this scales by simply running more of them — the elasticity of [Chapter 8](#) applied to the work, not the engine.

The *lock duration* must be comfortably longer than the work takes. Too short, and the lock expires mid-work, the job is handed to a second worker, and the work is done twice — tolerable only because the worker is idempotent, but wasteful. Too long, and a genuinely crashed worker’s job sits locked and unworked until the long lock expires.

The *number of jobs a worker fetches at once* trades latency against throughput: fetching a batch is efficient but means jobs wait in the worker’s local buffer; fetching one at a time is responsive but chattier.

The *back-off and retry policy* governs what happens when work fails: how many times to retry, and how long to wait between attempts — with increasing back-off so that a struggling downstream system is not hammered.

32.5 Failure, errors, and incidents

A job worker must distinguish three quite different things that can go wrong, because the process model treats each differently and confusing them is a common and costly modelling error.

A *technical failure* is the work failing for an infrastructure reason: the downstream system timed out, the network dropped, the database was briefly unavailable. The right response is to *fail the job with a retry*: the engine will reoffer it, and a later attempt — perhaps after a back-off — may well succeed. A technical failure is not a business outcome; it is a transient fault, and the process model should not have a path for it. It is handled by retrying, and only if the retries are exhausted does it become an incident.

A *business error* is a legitimate, expected outcome of the work that happens to be negative: the card was declined, the identity could not be verified, the account does not exist. This is not a fault — nothing is broken, and retrying would change nothing. The worker raises a *BPMN error*, with an error code, and the process model is expected to *catch* it with a boundary error event and route on it. A declined card is a path through the process, not an exception to it.

An *incident*, in Camunda’s vocabulary, is what a technical failure becomes when its retries are exhausted. The job cannot be completed, the instance cannot proceed, and a human operator must intervene — through Operate, as [Chapter 28](#) described — to diagnose the cause, fix it, and retry the job. An incident is the process saying, honestly, “this instance is stuck and needs a person.”

[Listing 32.2](#) shows a worker that makes the distinction explicitly.

Listing 32.2. A job worker distinguishing a business error from a technical failure. The business error is thrown as a BPMN error for the model to catch; the technical failure is allowed to propagate, becoming a job failure the engine will retry.

```
1 @JobWorker(type = "card-payment")
2 public Map<String, Object> pay(ActivatedJob job) {
3     PaymentRequest req =
4         job.getVariablesAsType(PaymentRequest.class);
```

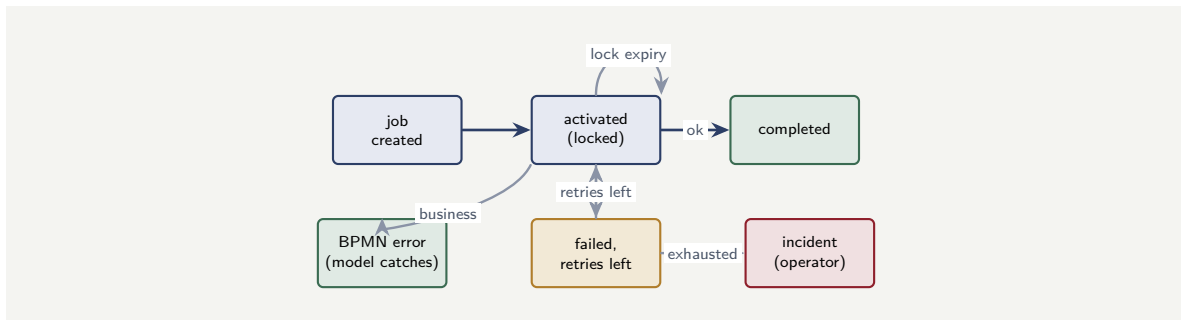


Figure 32.1. The job lifecycle and its three ways of not completing. A technical failure with retries left returns the job for another attempt; exhausted retries become an operator-visible incident; a business error is raised for the process model to catch and route. A lock expiry quietly returns the job.

```

5
6   PaymentResult result = paymentRail.execute(req); // may throw
7
8   if (result.isDeclined()) {
9       // a business outcome -- the model must catch and route this
10      throw new BpmnError("CARD_DECLINED", result.getReason());
11  }
12  return Map.of("paymentId", result.getId());
13
14  // a technical failure -- a timeout or rail outage -- is
15  // NOT caught here: it propagates, the job fails, the
16  // engine retries, and only exhausted retries raise an
17  // operator-visible incident.
18 }

```

Figure 32.1 draws the lifecycle with all three non-completion paths. The single most common worker bug is collapsing the distinction: treating a declined card as a technical failure — so the engine pointlessly retries a decision that will never change, and eventually raises a spurious incident — or treating a genuine outage as a business error — so the model routes a transient fault down a permanent path. The worker decides which is which, and it must decide correctly.

Do not confuse a technical failure with a business error. A technical failure — a timeout, an outage — is transient: fail the job and let the engine retry. A business error — a declined card, an unverified identity — is a real outcome: raise a BPMN error for the model to catch and route. Retrying a business error wastes effort and raises false incidents; routing a technical failure down a business path sends a transient fault to a permanent conclusion. The worker is where the two are told apart.

Worked example: sizing a worker fleet against a job rate

Problem. A payments process generates payment-execution jobs at a peak rate of 90 per second. Each job takes a worker, on average, 400 milliseconds to complete — mostly waiting on the payments rail. The team wants the fleet sized so it can clear jobs at the peak rate, with 30% headroom above the raw requirement. Each worker instance processes jobs one at a time. How many worker instances are needed, and what lock duration is sensible?

Setup. A single worker, processing jobs serially at 400 ms each, completes $1/0.4 = 2.5$ jobs per second. The raw fleet size is the peak job rate divided by per-worker throughput. The

fleet size with headroom is the raw size divided by $(1 - 0.30)$, or equivalently times 1.30 — we multiply by 1.30. The lock duration must exceed the handling time with a safe margin.

Working. Per-worker throughput:

$$\frac{1}{0.4 \text{ s}} = 2.5 \text{ jobs per second.}$$

Raw fleet size to clear 90 jobs per second:

$$\frac{90}{2.5} = 36 \text{ worker instances.}$$

With 30% headroom:

$$36 \times 1.30 = 46.8 \Rightarrow 47 \text{ worker instances.}$$

Lock duration: the work takes 400 ms on average, but its upper tail — a slow response from the payments rail — is far longer. A lock of 400 ms would expire constantly. A sensible lock is well above the tail of the handling time: of the order of 30 to 60 seconds, long enough that a lock expiry means a genuinely stuck worker, not a merely slow one.

Discussion. Forty-seven worker instances clear the peak with headroom, and the calculation is routine — but the worked example’s lessons are in what surrounds it. First, notice that the fleet scales by *instance count*: needing 47 workers is simply a matter of running 47 copies, with no change to the engine, which is the external-task pattern’s whole promise — the work scales independently of the orchestrator. Second, the lock-duration reasoning matters more than it first appears. The naive instinct is to set the lock close to the 400 ms handling time; that instinct is wrong, because handling time has a long tail and a lock just above the *mean* will expire on every slow job, handing live jobs to second workers and doing the work twice. The lock must clear the *tail*, which is why 30 to 60 seconds — two orders of magnitude above the mean — is right: a lock expiry should be a rare signal of a genuinely dead worker, not a routine consequence of a slow payments rail. And third, this is exactly why the [Chapter 31](#) discipline of idempotency is structural here: even a well-tuned lock will occasionally expire on a real outlier, the job will be done twice, and only an idempotent worker makes that harmless. Sizing the fleet is the easy part; tuning the lock and guaranteeing idempotency are what make the fleet correct.

Practical next steps

For one high-volume service task in your platform, estimate its peak job rate and average handling time, and size the worker fleet as in the worked example. Then look at the *tail* of the handling time, not the mean, and set a lock duration well above it. If your lock duration is anywhere near your mean handling time, your platform is doing a quiet, wasteful amount of duplicate work.

Chapter in brief

The external-task pattern inverts the embedded model: code calls the engine, not the reverse. The engine records available jobs; job workers — separate, independently scalable clients — activate jobs under a lock, do the work in their own transaction, and complete, fail, or raise a BPMN error. A crashed worker’s lock expires and the job is reoffered, so a crash costs one retry, never a lost instance. A worker is thin Java code

bound to a job type; because delivery is at-least-once, every worker with an external effect must be idempotent. A worker fleet scales by instance count, and the lock duration must clear the tail of the handling time, not the mean.

Connectors in Depth

33.1 From bespoke worker to reusable connector

Chapter 32 described the job worker: Java code that does the work behind a service task. Writing a job worker is the right answer when the work is specific to one institution — a call into a particular core banking system, a piece of bespoke logic. But a great deal of integration work is *not* institution-specific. Calling a REST service, sending an email, putting a message on a queue, reading from a database, invoking a large-language-model service — these are the same everywhere, and writing a bespoke worker for each, in each process, is wasteful and error-prone.

A *connector* is the answer: a reusable, configurable integration component that does a common kind of work, so that a service task can use it by *configuration* rather than by code. Chapter 28 introduced connectors among the components; this chapter treats them in the depth a serious platform needs, because the connector is where most of a real platform’s integration surface lives.

33.2 Outbound and inbound connectors

Connectors come in two directions, and the distinction is fundamental because it corresponds to the two ways a process touches the outside world — *acting on it* and *reacting to it*.

An *outbound connector* is used by a service task to *call out*: the process reaches a point, the connector makes the call — a REST request, an email, a database write, a model invocation — and the result comes back into the process. An outbound connector is, in effect, a pre-built, configurable job worker: the engine creates a job, the connector’s runtime activates it, performs the configured call, and completes it. Everything Chapter 32 said about job workers — the lock, the at-least-once delivery, the need for idempotency on external effects — applies to outbound connectors unchanged, because an outbound connector *is* a job worker, just one written once and reused by configuration.

An *inbound connector* works the other way: it lets something outside *start or advance* a process. An inbound connector subscribes to an external source — a webhook, a message queue, a polled endpoint — and when something arrives, it starts a new process instance or correlates a message to a waiting one. The inbound connector is the concrete realisation of the event-driven triggering that Chapter 25’s event backbone described and that Chapter 34 will treat as a design style: it is how an event in the world becomes a running process.

Recognising which direction a connector is determines how it is reasoned about. An outbound connector is a step *within* a process and is governed like any service task. An inbound connector is an *entry point* to the platform, and entry points are a security surface — which the next section takes up.

Figure 33.1 draws the two directions: the process calling out, and an external source triggering the process in.

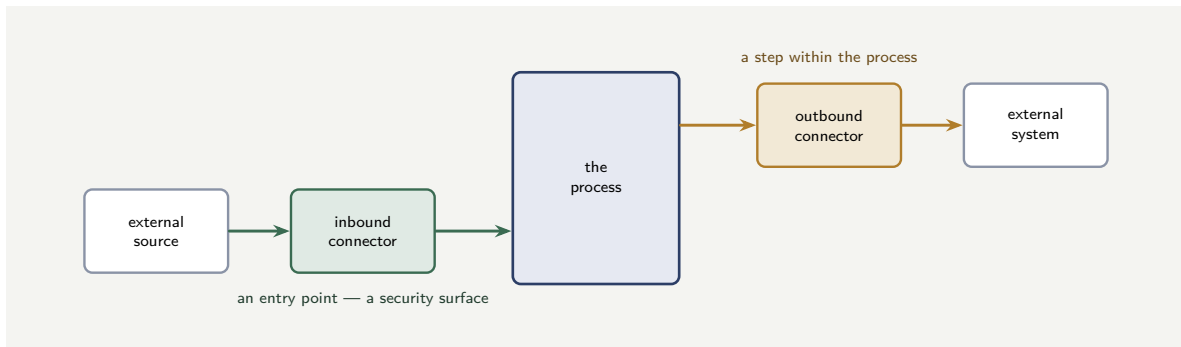


Figure 33.1. The two directions of a connector. An outbound connector is a step within the process, calling out to an external system, governed like any service task. An inbound connector is an entry point — an external source triggers a process through it — and so it is part of the platform’s security surface.

33.3 Configuring, securing, and governing a connector

A connector’s convenience — integration by configuration — carries its own disciplines, and a platform that adopts connectors casually inherits problems that bespoke code would have made visible.

Configuration is logic. A connector is configured: the URL it calls, the payload it sends, the mapping from process variables to request and from response to process variables. That configuration is as much a part of the process’s behaviour as any FEEL expression or delegate, and it deserves the same governance — versioned with the model, reviewed, and tested. A connector reconfigured quietly is a process changed quietly.

Credentials are not configuration. A connector that calls a secured system needs credentials, and those credentials must not sit in the connector configuration in the model. They belong in the secrets-management service of [Chapter 24](#), referenced by the connector, never embedded. This is the least-privilege discipline of Part IV’s security chapter applied at the connector: the connector gets exactly the credential its one integration needs, held securely, rotated centrally.

Inbound connectors are an attack surface. An inbound connector — especially a webhook — is a door into the platform from outside. It must authenticate what calls it, validate what it receives, and be protected against abuse, because an unauthenticated inbound connector is a way for anything on the network to start process instances in a bank. The entry point must be as governed as the process behind it.

Failure is the connector’s responsibility too. A connector’s call can fail, time out, or return an error, and the process model must handle that — a boundary error event on the connector’s service task, routing to a defined failure path — exactly as for any service task. A connector does not make the failure-path discipline of [Chapter 21](#) optional; it just does the calling.

Tip

Treat a connector’s configuration as governed process logic — versioned, reviewed, tested — not as a quick setting. Keep its credentials in the secrets service, never in the model. And treat every inbound connector as an authenticated, validated, rate-limited entry point into the platform. The connector makes integration convenient; it does not make any of the integration disciplines optional.

33.4 The agent connector and agentic BPMN

One kind of outbound connector deserves its own treatment: the connector to a large-language-model or agent service. It is the concrete mechanism by which an agentic step, in [Chapter 23](#)'s sense, is reached from a process.

The agent connector is configured with the task the agent is to perform, the inputs it is given, the tools it may use, and the schema its output must take. That configuration *is* the containment of [Chapter 23](#) made concrete: the bounded task, the bounded toolset, the defined output schema are all connector configuration, and configuring them carelessly is exactly how the contained agent becomes the ungoverned one.

Camunda's *agentic BPMN* extends this further: the reasoning loop, the agent's memory, the human-in-the-loop boundary, and the deterministic adjudication of the agent's output are modelled as BPMN structure around the agent connector. This is the product realisation of the manuscript's central argument — the agent is a contained component inside a governable process model. The agent connector reaches the agent; the modelled BPMN structure around it enforces the four containments of [Chapter 23](#); and the whole remains an explicit, auditable, executable model. An agent reached through a configured connector inside a modelled containment is governable. An agent reached by bespoke code that bypasses the model is the ungoverned agent the manuscript has warned against from [Chapter 23](#) onward.

33.5 Building a custom connector

The connector catalogue covers the common cases, but an institution will sometimes have an integration that is common *within that institution* — the same idiosyncratic core-banking call needed by a dozen processes — and for that, the right answer is neither a bespoke worker per process nor a mismatched generic connector, but a *custom connector*: the institution builds its own reusable connector once, and every process configures it.

A connector, structurally, is three things: the integration logic itself, a *template* that declares what the connector can be configured with, and the binding that ties a configured service task to the logic. The integration logic of an outbound connector is, as the chapter has stressed, a job worker — so building a custom connector begins with writing one, exactly as [Chapter 32](#) described, but parameterised so that its behaviour is driven by configuration rather than hard-coded. Listing 33.1 shows the shape.

Listing 33.1. The core of a custom outbound connector. The connector function receives the configured inputs, performs the parameterised integration, and returns a result — reusable by any process that configures it.

```
1  @OutboundConnector(  
2      name = "CoreBankingPosting",  
3      inputVariables = {"accountRef", "amount", "postingType"},  
4      type = "io.bank.connector:core-posting:1")  
5  public class CorePostingConnector  
6      implements OutboundConnectorFunction {  
7  
8      @Override  
9      public Object execute(OutboundConnectorContext context) {  
10         // bind the configured inputs to a typed request object  
11         PostingRequest req =  
12             context.bindVariables(PostingRequest.class);  
13  
14         // credentials come from the secrets store, never the model  
15         var creds = context.getSecret("CORE_BANKING_CREDENTIAL");  
16     }
```



```

17 // the parameterised integration -- the reusable logic
18 PostingResult result = corePostingClient.post(req, creds);
19
20 // the result is mapped back into process variables
21 return new PostingResponse(result.getPostingId(),
22                             result.getStatus());
23 }
24 }

```

The *template* is the other half. It is a declaration — consumed by the Modeler — of the fields a modeller will fill in when they drop this connector onto a service task: which process variable supplies the account reference, which supplies the amount, where the result is written, which secret holds the credential. The template is what turns the Java logic above into something a modeller uses by configuration, without writing code, and it is what makes the connector genuinely reusable across processes.

Two disciplines govern a custom connector, and both are the manuscript’s recurring themes met once more. First, a custom connector is a *governed, versioned artefact* in its own right — note the version in its type, `core-posting:1`. A change to the connector’s logic is a new version, and processes adopt it deliberately, exactly as [Chapter 27](#) required of any reused component. A custom connector silently changed is many processes silently changed. Second, a custom connector to a system of record still belongs *behind the anti-corruption layer* of [Chapter 25](#): the connector’s template should expose clean, business-meaningful inputs — “account reference,” “amount” — and absorb the legacy system’s idiosyncrasies inside, so that the processes configuring it never see the core system’s raw data model.

Tip

When the same idiosyncratic integration is needed by several processes, build a custom connector once rather than a bespoke worker many times — but treat that connector as a versioned, governed component, and let its template expose clean, business-meaningful fields so it doubles as the anti-corruption layer. A custom connector is the institution turning its own recurring integration into reusable, governed infrastructure.

33.6 Connector governance

A connector is reusable integration, and reusable integration must be governed. Each connector the platform uses — REST, messaging, cloud, calendar — should be inventoried, its configuration reviewed, its credentials rotated, and its error behaviour understood. A connector deployed and forgotten is a connector whose credentials expire silently, whose endpoint changes without anyone updating the configuration, and whose error handling has never been tested under the conditions that will trigger it. Govern connectors as you govern any shared infrastructure component: inventoried, reviewed, and maintained.

Worked example: build-versus-connector for an integration estate

Problem. A platform must integrate with 40 distinct external touch points. Analysis shows 28 of them are *standard* — REST calls, message queues, email, database access — each of which a connector can handle by configuration, and 12 are *bespoke* — calls into legacy core systems with idiosyncratic protocols — needing a hand-written worker. A bespoke worker costs, on

average, 12 engineer-days to build and 3 engineer-days a year to maintain. Configuring a connector for a standard touch point costs 2 engineer-days and 0.5 engineer-days a year to maintain. The team is tempted to hand-write all 40 for “consistency.” Compare the two approaches over a three-year horizon.

Setup. For each approach we total the one-off build cost and three years of maintenance. Approach 1 hand-writes all 40. Approach 2 uses connectors for the 28 standard touch points and bespoke workers for the 12.

Working. *Approach 1 — hand-write all 40.* Build: $40 \times 12 = 480$ engineer-days. Maintenance: $40 \times 3 \times 3 = 360$ engineer-days. Total: $480 + 360 = 840$ engineer-days.

Approach 2 — connectors for 28, bespoke for 12. Connectors — build: $28 \times 2 = 56$; maintenance: $28 \times 0.5 \times 3 = 42$. Connector total: 98 engineer-days. Bespoke workers — build: $12 \times 12 = 144$; maintenance: $12 \times 3 \times 3 = 108$. Bespoke total: 252 engineer-days. Approach 2 total: $98 + 252 = 350$ engineer-days.

Difference: $840 - 350 = 490$ engineer-days saved over three years — the hand-write-everything approach costs roughly 2.4 times as much.

Discussion. The connector-where-you-can approach saves 490 engineer-days over three years, and the instinct toward hand-writing everything “for consistency” is revealed as expensive. But the worked example is not an argument for connectors *everywhere*, and that is its real lesson. Notice that the 12 bespoke touch points still account for 252 of Approach 2’s 350 days — the genuinely idiosyncratic legacy integrations are expensive whichever way the estate is built, because a connector cannot configure away a protocol it does not understand. The discipline is to use the right tool for each touch point: a connector where the integration is standard and configuration genuinely suffices, and a bespoke worker — behind the anti-corruption layer of [Chapter 25](#) — where the integration is idiosyncratic. “Consistency” achieved by hand-writing the 28 standard integrations is consistency bought at 2.4 times the cost; the consistency that matters is not uniform implementation but uniform *governance* — every integration, connector or bespoke, versioned, secured, and given a failure path. Approach 2 delivers that governance and the saving together.

Practical next steps

Inventory your platform’s external touch points and classify each as standard — a connector can handle it by configuration — or bespoke. Resist both extremes: hand-writing the standard ones wastes the connector’s leverage, and forcing a connector onto a genuinely idiosyncratic legacy system produces a fragile configuration that a proper worker behind an anti-corruption layer would have done cleanly. The classification is the design decision.

Chapter in brief

A connector is a reusable, configurable integration component — integration by configuration rather than code. Outbound connectors call out and are, in effect, pre-built job workers, inheriting every job-worker discipline including idempotency. Inbound connectors let the outside start or advance a process and are an authenticated, validated entry point into the platform. A connector’s configuration is governed process logic; its credentials belong in the secrets service; its failures need modelled failure paths. The agent connector reaches an agentic step, and agentic BPMN models the four containments around it. Choose connectors for standard integrations and bespoke workers for

idiosyncratic ones — the consistency that matters is uniform governance, not uniform implementation.

Event-Driven Process Design

34.1 Two ways a process relates to the world

Every chapter of this part so far has, implicitly, treated a process as something that is *started* and then *runs* — a request arrives, an instance begins, it proceeds through its activities to an end. That is the *command-driven* view, and it is correct as far as it goes. But a process in a bank or an insurer does not only run; it also *waits* and *reacts*. It waits for a customer to respond, for a payment to clear, for a counterparty to confirm. It reacts to things that happen elsewhere — a policy lapses, a threshold is breached, a fraud alert fires. This is the *event-driven* view, and a process designed without it is a process that can act but cannot listen.

This chapter treats event-driven process design as a discipline in its own right. It draws together threads from across the manuscript — the message and timer events of [Chapter 3](#), the event-based gateway of [Chapter 29](#), the event backbone of [Chapter 25](#), the inbound connectors of [Chapter 33](#) — and shows them as one coherent way of thinking: the process not as a script that runs start to finish, but as a participant in a flow of events.

34.2 Events in BPMN: the vocabulary of reaction

BPMN has a rich event vocabulary, and a process designed to be event-driven uses it deliberately. It is worth setting the pieces out together.

A *message event* represents the arrival of something specific and addressed — a particular customer’s response, a particular payment confirmation. A message-start event begins a new instance when the message arrives; an intermediate message-catch event, mid-process, pauses the instance until the message arrives; a message-throw event sends one.

A *timer event* represents the passage of time — a duration elapsing, a date arriving, a cycle recurring. As a boundary event on an activity it is the SLA mechanism of [Chapter 3](#); as an intermediate event it is a deliberate wait; as a start event it begins instances on a schedule.

A *signal event* represents a broadcast — something announced to whoever is listening, rather than addressed to one instance. Where a message goes to one recipient, a signal goes to all: a “market closed” or “system-wide freeze” signal can be caught by every instance that cares.

An *error event* represents a business error thrown by an activity — the BPMN error of [Chapter 32](#) — caught by a boundary error event and routed as an exception path.

The *message-correlation* mechanism is what makes message events work in a high-volume setting. When a payment confirmation arrives, the platform must deliver it not to *a* claims instance but to *the* instance that is waiting for that specific payment. Correlation is the matching of an incoming message to the one waiting instance it belongs to, by a correlation key — the payment reference, the claim number. Event-driven design lives or dies on correct correlation: a message that correlates to the wrong instance, or to none, is a process that reacts to the wrong case or fails to react at all.

The hardest defect in event-driven process design is mis-correlation: an incoming event matched to the wrong waiting instance, or to none. A correlation key must be genuinely unique to the instance and present on every event that must reach it. A key that is not unique delivers an event to several instances; a key that is missing or wrong strands the event and leaves the instance waiting forever. Design the correlation key with the same care as the routing logic — it *is* routing, for events.

34.3 Choreography and the event backbone

Chapter 25 introduced the event backbone — the durable, ordered stream of business events that systems publish and subscribe to. Event-driven process design is what makes full use of it, and it introduces a distinction worth naming: *orchestration* versus *choreography*.

In *orchestration*, a central process model directs the work: it calls this, waits for that, routes here. Most of this manuscript has described orchestration, and it is the right default — it gives the explicit, governable, observable model that Part I argued for.

In *choreography*, there is no central director. Several processes each react to events and emit events, and the overall behaviour *emerges* from their interaction: process A emits “loan approved,” process B reacts by starting fulfilment, process B emits “funds disbursed,” process C reacts by beginning servicing. No single model contains the whole; each process knows only its own events in and out.

Choreography has real strengths — the processes are loosely coupled, each can change independently, new reactors can be added without touching the emitters. But it has a serious cost, and the manuscript is direct about it: *no single artefact describes the end-to-end behaviour*. The whole exists only as an emergent property of many processes and their events, which is precisely the property that makes it hard to govern, hard to audit, and hard to explain — the three things Part I insisted a process must be.

The disciplined position is therefore a deliberate balance. Use orchestration for the end-to-end business process the institution must govern as a whole — the loan, the claim, the onboarding journey — so that there is one explicit, auditable model of it. Use event-driven choreography *between* those orchestrated processes, where one major process needs to trigger or inform another, so that the large-grained pieces are loosely coupled. Orchestrate within the governed unit; choreograph between the units. A platform that is pure choreography has loose coupling and no governable whole; a platform that orchestrates everything into one vast model has governance and no flexibility. The balance is the design.

Figure 34.1 sets the two side by side. The point the picture makes is that they are not rivals to choose between but tools for different scales: orchestration governs the inside of a business process, choreography loosely couples one business process to the next.

34.4 Designing a process to be event-driven

Pulling the threads together, designing a process to be event-driven means making a few questions explicit at modelling time.

How does it start? A command-driven process starts on request; an event-driven one may start on a message — an inbound connector catching a webhook, a backbone event — and the start event should say which.

What does it wait for, and what interrupts the wait? Every point where the process depends on the outside world is a message or event-based gateway, and every wait that must not be

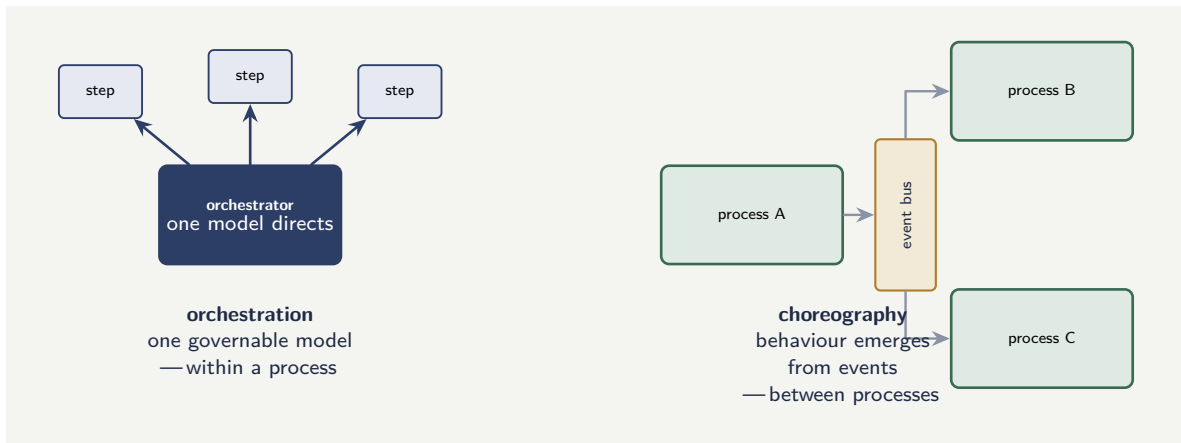


Figure 34.1. Orchestration versus choreography. Orchestration — one model directing the steps — gives a governable whole and is used *within* a business process. Choreography — processes reacting to events on the backbone — gives loose coupling and is used *between* processes. The balance is the design.

unbounded carries a boundary timer — [Chapter 3](#)’s long-running, interruptible process, designed deliberately.

What does it emit? A process that other processes react to must emit its significant business events — “application submitted,” “loan disbursed,” “claim settled” — to the backbone, as deliberate, well-named facts.

How are its messages correlated? For every message the process catches, the correlation key must be identified and must be present on the event — the discipline of the warning above.

A process for which these four questions have explicit, modelled answers is a process that can listen as well as act — and in a bank or insurer, where so much of what matters is something happening elsewhere, a process that can only act is only half a process.

34.5 The reliability of a connector

A connector is the seam between the platform and an external system, and seams are where reliability is won or lost. A connector that works in a demo and fails under production conditions is one of the more common disappointments in a platform’s first months, and the failures are predictable enough to be designed out in advance.

An outbound connector inherits, in full, the failure discipline of [Chapter 41](#), because an outbound connector *is*, under its reusable surface, a job worker calling an external system. It can meet a transient failure — a timeout, a brief outage — and it can meet a permanent one, and it can meet a legitimate business error from the system it calls. The connector must classify these exactly as [Chapter 41](#) required: a transient failure is retried with back-off, a permanent failure raises an incident, and a business error — the core banking system declined the posting — is raised as a BPMN error the process model catches and routes. A connector that lets every failure propagate as one undifferentiated error is the same defect that chapter named, wearing a connector’s clothes.

The point that deserves special emphasis is *idempotency*, because a connector’s reusability makes it tempting to forget. A connector that performs an irreversible external effect — moves money, binds cover, posts a transaction — will, by the at-least-once delivery of [Chapter 32](#), sometimes be invoked twice for one logical step. The connector must therefore carry an idem-

potency key, derived from the process instance and the element, so the external system recognises and ignores the repeat. And here is the subtlety: because a connector is reused across many processes, its idempotency is not one team's concern but every consuming team's concern at once — a non-idempotent connector that posts payments is a latent double-payment defect replicated into every process that reuses it. The connector's idempotency is part of its contract, and the team that builds the connector owes that guarantee to everyone who will ever configure it.

A connector with an irreversible external effect must be idempotent, and because a connector is *reused*, its idempotency is owed to every process that will ever configure it. A non-idempotent payment connector is not one team's bug — it is a double-payment defect copied into every process on the platform that posts a payment through it. Build idempotency into the connector, treat it as part of the connector's published contract, and test it before the connector is offered for reuse.

An inbound connector has a reliability concern of its own: it must not *lose* an event. An external source fires an event — a webhook, a message on a queue — and the inbound connector must reliably turn it into a started process or a correlated message, even across a moment when the platform is briefly busy or restarting. A durable queue between the source and the inbound connector, rather than a fire-and-forget webhook the connector might miss, is the difference between an entry point that is reliable and one that silently drops the occasional claim. The reliability of the inbound connector is the reliability of the platform's ability to hear the world.

34.6 The event backbone in a banking context

In a bank, the event backbone carries the institution's business facts: "loan disbursed," "payment received," "customer onboarded," "claim settled." These are not technical messages; they are the institution's operational heartbeat, and every system that needs to react to the business does so by subscribing to the backbone. The hyperautomation platform is one such subscriber — it starts and advances processes from the backbone's events — but it is also a *publisher*: the process's own significant events ("application approved," "case escalated") are published back to the backbone for other systems to consume. The platform is both listener and speaker on the institution's event nervous system.

Worked example: the latency of polling versus an event

Problem. A claims process must react when a counterparty's payment clears. Two designs are considered. In design A, the process *polls*: a service task checks the payment status, and if it is not yet cleared, a timer waits 30 minutes and the check repeats. In design B, the process *waits on an event*: it pauses at a message-catch event, and an inbound connector delivers the "payment cleared" event from the backbone the moment it happens. Payments clear, on average, 90 minutes after the process begins waiting, but the spread is wide. The institution handles 100,000 such claims a year. Compare the average reaction delay and the wasted work of the two designs.

Setup. For design A, the reaction delay is the time from the actual clearing to the next poll — on average half the 30-minute polling interval — plus the polling itself does work

whether or not the payment has cleared. For design B, the reaction delay is essentially zero — the event arrives when the clearing happens. We compute the average added delay and the polling work.

Working. *Design A — polling.* A payment clears at some moment between two polls; on average it clears halfway through a 30-minute interval, so the average delay between clearing and the process noticing is

$$\frac{30}{2} = 15 \text{ minutes.}$$

The number of poll checks per claim is the 90-minute average wait divided by the 30-minute interval:

$$\frac{90}{30} = 3 \text{ checks per claim (on average),}$$

of which only the last finds the payment cleared — so 2 of every 3 checks are wasted work. Across the book:

$$100,000 \times 3 = 300,000 \text{ poll checks a year, } 200,000 \text{ of them wasted.}$$

Design B — event. The “payment cleared” event arrives when the clearing happens, so the average reaction delay is effectively **0 minutes**, and there are **no** poll checks — the process does no work at all while it waits.

Discussion. Design B reacts 15 minutes faster on average and does 200,000 fewer units of wasted work a year, and the worked example is built to show that this is not a marginal tuning gain but a difference in kind. Polling is the command-driven habit applied to a problem that is inherently event-driven: it repeatedly *asks* “has it happened yet?” because it has not been told. Every poll that finds nothing is pure waste — compute, load on the payments system, and an instance doing busywork — and the reaction is always late by, on average, half the polling interval. Shortening the interval reduces the delay but multiplies the wasted checks; lengthening it cuts the checks but worsens the delay. There is no good point on that trade-off, because the trade-off itself is the symptom of the wrong design. The event-driven design B has no trade-off to tune: the process waits at a message-catch event, costing nothing — the engine simply persists the waiting instance, as [Chapter 21](#) described — and reacts the instant the event arrives. The lesson generalises well beyond payments: wherever a process is polling to find out whether something happened elsewhere, that is a process that should be waiting on an event instead, and the event backbone of [Chapter 25](#) is what makes the event available to wait on.

Practical next steps

Search one of your process models for polling loops — a check, a timer, a re-check. Each one is a place where the process is asking a question it could instead be told the answer to. For each, identify the business event that would let the process wait rather than poll, and whether your event backbone already carries that event. The polling loops are your event-driven design debt, made visible.

Chapter in brief

A process relates to the world in two ways: command-driven — started and run — and event-driven — waiting and reacting. A process designed without the second can act but cannot listen. BPMN’s event vocabulary — message, timer, signal, error events — is the vocabulary of reaction, and message correlation, matching an incoming event to

the one waiting instance it belongs to, is where event-driven design succeeds or fails. Orchestrate within the governed end-to-end process so there is one auditable model; choreograph between major processes for loose coupling. Designing a process to be event-driven means explicit answers to how it starts, what it waits for, what it emits, and how its messages correlate — and replacing polling loops with waits on events.

Security and Identity: Authentication and Authorization

35.1 Why a platform needs an identity provider

Every chapter of this part so far has described *what* the platform does — it runs processes, allocates tasks, calls workers, reacts to events. None has yet asked the question that a regulated institution must answer before any of that is allowed to run: *who is permitted to do it?* A platform that orchestrates lending, claims, and payments is a platform on which the wrong person starting the wrong process, or completing the wrong task, is a serious incident. Security is not a feature added to the platform; it is the condition of the platform being usable at all.

Part IV's security chapter set out the principles — classified data, managed identities, least privilege, every action linked to an accountable identity. This chapter makes them concrete in the Camunda setting. The central component is *Identity*, and the central idea is that the platform does not invent its own notion of users and credentials. It delegates authentication to an *identity provider* — an external system whose job is to establish, by some trusted means, who a user or a calling application is — and consumes the result. The institution almost always already has such a provider: a corporate directory, an OIDC service, an enterprise single-sign-on. The platform connects to it rather than competing with it.

The standard the platform speaks is *OpenID Connect* — OIDC — an authentication layer built on OAuth 2.0. Two providers dominate in banking and insurance settings, and this chapter uses both as worked examples: *Keycloak*, the open-source identity and access provider that Camunda can also run as a bundled internal component, and *Microsoft Entra ID*, the cloud directory and identity service many institutions already run as their corporate identity backbone.

35.2 Authentication, authorization, and the token

Two words that are casually treated as one must be kept firmly apart, because the platform treats them as two distinct mechanisms.

Authentication establishes *who* the caller is. It is the identity provider's job: the user signs in, or the calling application presents its credentials, and the provider — having checked them — issues a *token* asserting the established identity.

Authorization establishes *what* that caller may do. It is the platform's job: given an authenticated identity, Identity and the orchestration cluster decide whether that identity is permitted the specific action it is attempting — start this process, claim that task, query those instances.

The artefact that connects the two is the *token* — specifically a JSON Web Token, a JWT. A JWT is a compact, signed assertion: it carries *claims* about the caller — who they are, which

client issued the token, what scopes and audience it is valid for, when it expires — and it is cryptographically signed by the identity provider so that the platform can verify it was genuinely issued and not tampered with. The platform does not re-check a password on every request; it checks the token’s signature and its claims, and trusts the identity provider that issued it.

One claim deserves specific mention because it is a frequent source of misconfiguration: the *audience* claim, *aud*. It states which system the token is *for*. The orchestration cluster validates that an incoming token’s audience matches its own configured audience, so that a token issued for one system cannot be replayed against another. An identity provider that does not emit an *aud* claim, or emits the wrong one, produces tokens the platform will reject — and “the provider is not issuing the audience claim” is one of the most common setup failures.

Tip

Keep *authentication* and *authorization* firmly distinct. Authentication — who is this? — is delegated to the identity provider and asserted by a signed token. Authorization — what may they do? — is the platform’s own decision, made by Identity and the cluster against the authenticated identity. A platform that conflates the two tends to grant access on the strength of mere authentication, which is how an authenticated but unauthorised caller ends up starting a process they should never have reached.

35.3 The two ways a caller authenticates

A caller of the platform is one of two things, and the OIDC standard handles each with a different *flow*.

A caller may be a *human user* — a claims adjuster opening Tasklist, an operator opening Operate. The human authenticates interactively: the component redirects the browser to the identity provider, the user signs in there — with a password, a second factor, whatever the provider enforces — and the provider redirects back with a token. This is the *authorization-code flow*, and its defining property is that the platform never sees the user’s password; only the identity provider does.

A caller may instead be a *machine* — a job worker, a backend service, an integration calling the Process API. There is no human and no browser. The calling application has been registered with the identity provider as a *client*, with a client ID and a client secret, and it authenticates by presenting those directly to the provider’s token endpoint and receiving a token in return. This is the *client-credentials flow* — machine-to-machine, or M2M, authentication — and it is how every programmatic caller in the chapters that follow obtains the token it needs.

Listing 35.1 shows the client-credentials flow as a raw token request, which is the foundation everything in [Chapter 36](#), [Chapter 37](#), and [Chapter 38](#) builds on.

Listing 35.1. Requesting a machine-to-machine token with the client-credentials grant. The client ID and secret identify the calling application; the response is a signed JWT the platform will accept.

```
1 # request a token from the identity provider's token endpoint
2 curl --request POST \
3   'https://<IDP_HOST>/realms/<REALM>/protocol/openid-connect/token' \
4   --header 'Content-Type: application/x-www-form-urlencoded' \
5   --data-urlencode 'grant_type=client_credentials' \
6   --data-urlencode 'client_id=${CLIENT_ID}' \
7   --data-urlencode 'client_secret=${CLIENT_SECRET}' \
```

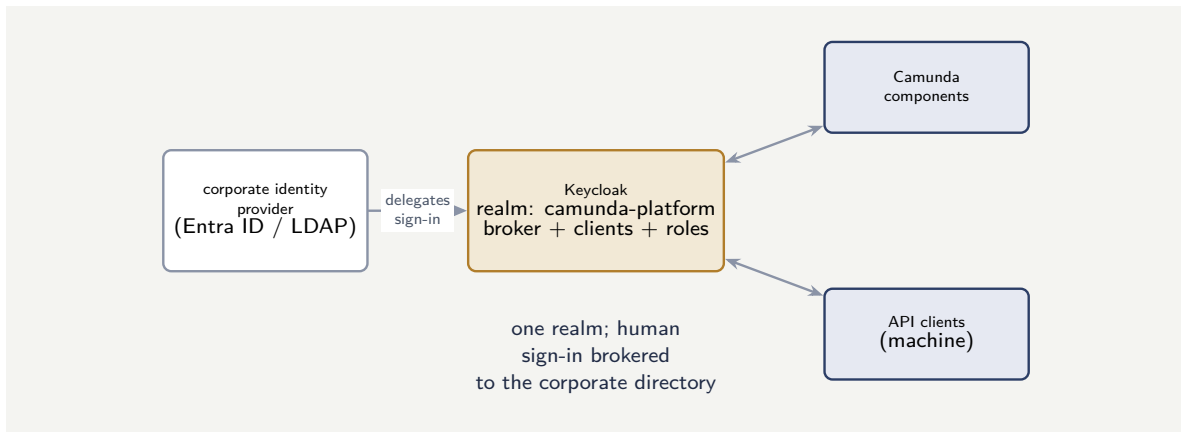



Figure 35.2. Keycloak as an identity broker. Camunda’s realm holds the clients and roles the platform needs, but human authentication is delegated onward to the institution’s existing corporate identity provider — so the institution keeps one source of truth for who its people are.

Tip

Keycloak as an *identity broker* is the pattern that fits most institutions: Camunda’s internal Keycloak provides the realm, clients, and roles the platform needs, but it delegates the actual sign-in of human users to the institution’s existing corporate identity provider. The institution keeps one source of truth for who its people are; the platform gets the realm and client model it needs; and the two are joined by Keycloak’s broker and attribute mappers rather than by a second, divergent copy of the staff directory.

35.5 Setup with Microsoft Entra ID

Many institutions do not want to run an identity provider at all; their corporate identity already lives in *Microsoft Entra ID*, the cloud directory and identity service, and they want the platform to use it directly. Camunda supports this: Identity can connect to Entra ID as its OIDC provider, and the bundled Keycloak is then not needed.

The setup follows the same OIDC shape, expressed in Entra’s terms. Each Camunda component, and each machine caller, is registered in Entra as an *app registration* — Entra’s name for an OIDC client — with an application ID and, for confidential callers, a client secret or certificate. Two Entra-specific details matter and are common stumbling points. First, Camunda requires the *v2.0* token endpoint, so the configured issuer URL must end in */v2.0*; the *v1.0* endpoint issues tokens of a shape Camunda does not accept. Second, Entra must be configured to emit access tokens carrying the *aud* audience claim Camunda validates — in the app registration’s manifest, the *requestedAccessTokenVersion* is set so that *v2.0* access tokens are issued. An Entra integration that fails to authenticate is, very often, failing on exactly one of these two points.

The institution’s staff, their groups, and their multi-factor and conditional-access policies all then live in Entra, unchanged, and the platform consumes the identities Entra asserts. The roles that govern authorization — who is a claims analyst, who is a platform administrator — are mapped from the groups Entra places in the token’s claims onto the platform’s own roles.

The two most common Microsoft Entra ID integration failures are both silent-looking misconfigurations, not code faults. If the issuer URL does not end in `/v2.0`, Camunda receives tokens it cannot process. If the app registration is not configured to emit the audience claim in its access tokens, Camunda rejects every token as not intended for it. Verify both before assuming the integration itself is wrong — they account for the majority of “Entra will not authenticate” incidents.

35.6 Connecting to the corporate directory: LDAP

Both the Keycloak and the Entra arrangements rest on a fact worth treating directly: an institution’s staff identities almost never originate in the process platform. They live in a *corporate directory* — and in a great many banks and insurers that directory is, at its root, an *LDAP* directory, most often *Microsoft Active Directory*. LDAP — the Lightweight Directory Access Protocol — is the long-established standard for querying a directory of users, groups, and organisational structure, and it is where “who works here, and what are they a member of” is authoritatively held.

A process platform must not become a second, divergent copy of that directory. The discipline is *federation*: the platform reads identities from the LDAP directory rather than re-entering them, so the directory stays the single source of truth and a lever removed there is a lever everywhere.

With *Keycloak*, this is done through Keycloak’s *LDAP user federation*. Keycloak is configured with a federation provider pointing at the directory, and from then on the directory’s users appear in the Camunda realm without being copied into it. The configuration is a small, well-defined set of values, shown in Listing 35.2: the directory’s connection URL, a bind account Keycloak authenticates as to read the directory, and the *search bases* — the points in the directory tree under which users and groups are found.

Listing 35.2. Keycloak LDAP user-federation configuration: Keycloak reads users and groups from the corporate directory rather than holding its own copy.

```
1 # Keycloak LDAP user federation (realm: camunda-platform)
2 vendor: "Active Directory"
3 connectionUrl: "ldaps://ad.bank.example:636"
4 bindDn: "CN=svc-keycloak,OU=Service,DC=bank,DC=example"
5 bindCredential: "${LDAP_BIND_SECRET}"
6 usersDn: "OU=Staff,DC=bank,DC=example"
7 usernameLDAPAttribute: "sAMAccountName"
8 # group mapper -- LDAP groups become Keycloak groups
9 groupsDn: "OU=Groups,DC=bank,DC=example"
10 groupObjectClasses: "group"
```

Two parts of that configuration carry the real weight. The first is `ldaps://` on port 636 rather than plain `ldap://` on 389: the connection to a directory of staff identities must be *TLS-encrypted*, because a bind travels credentials and the directory’s contents are sensitive. The second is the *group mapper*: an LDAP directory’s *groups* — “Claims-Analysts”, “Senior-Underwriters”, “Platform-Admins” — are mapped into Keycloak groups, and from there onto the platform roles that govern authorization. The institution’s existing organisational structure thus *becomes* the platform’s role structure, with no second list to maintain.

With *Microsoft Entra ID*, the LDAP directory is reached one step further back. Entra ID is itself, in most institutions, synchronised from the on-premises Active Directory — through Microsoft’s directory-synchronisation tooling — so the on-prem AD remains the root source

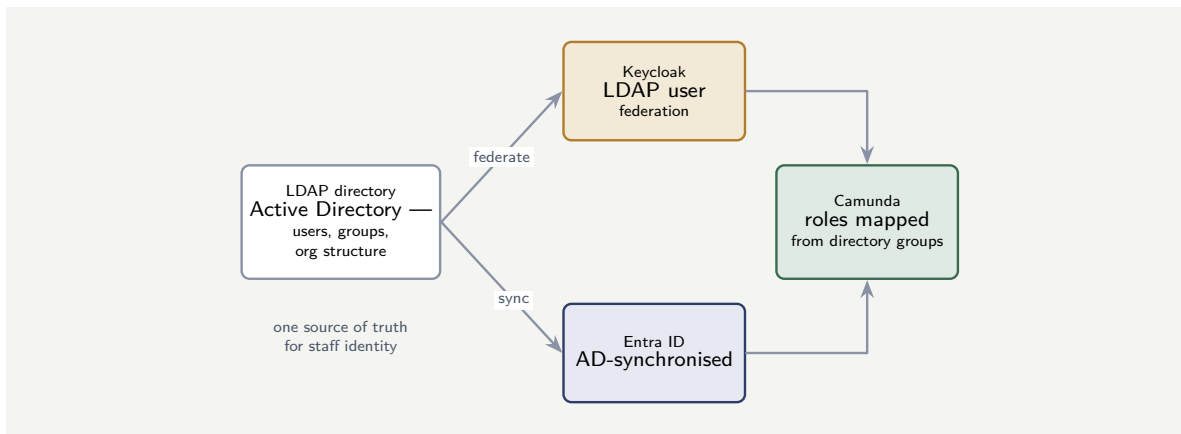


Figure 35.3. Two paths from the corporate LDAP directory to Camunda. Keycloak reaches the directory directly through LDAP user federation; Entra ID reaches it as the cloud projection of an AD-synchronised directory. Either way the LDAP directory stays the single source of truth, and its groups become the platform’s roles.

of truth and Entra is its cloud projection. The platform connects to Entra over OIDC, as the previous section described, and Entra carries forward the users and groups that originated in the on-premises LDAP directory. The platform engineer does not configure LDAP directly in this arrangement; they ensure the directory *groups* the platform’s roles depend on are in scope of the AD-to-Entra synchronisation and are emitted in the token’s group claims.

Figure 35.3 sets the two paths side by side. In both, the principle is identical: the LDAP directory is the single source of truth for staff identity, and the platform federates from it rather than copying it.

Tip

Whether identities arrive through Keycloak’s LDAP user federation or through an AD-synchronised Entra ID, never let the process platform hold its own separate list of staff. Federate from the corporate LDAP directory so that joining, moving, and — critically — *leaving* are reflected automatically: a person removed from the directory loses platform access without anyone remembering to remove them. A platform with its own user list will, sooner or later, grant a former employee a task.

35.7 Picking up users and attributes from Active Directory and Entra

Federation establishes that the directory *is* the source of identity. The practical question that follows is concrete: how does a specific user, and the specific *attributes* that user carries in Active Directory, actually reach a Camunda process? A process often needs more than a username — it needs the user’s *department*, their *branch*, their *approval limit*, their *manager* — and all of those are attributes held against the person in AD.

The mechanism has two stages: the directory’s attributes are mapped into the identity provider, and the identity provider places them in the *token* the platform receives.

Mapping directory attributes through Keycloak

When Keycloak federates an LDAP directory, each directory attribute the platform needs is declared as an *LDAP mapper*: a small configuration that says “this AD attribute becomes this Keycloak user attribute.” Listing 35.3 declares mappers for the attributes a banking process commonly needs.

Listing 35.3. Keycloak LDAP attribute mappers: each Active Directory attribute the platform needs is mapped to a Keycloak user attribute.

```
1 # Keycloak LDAP attribute mappers (realm: camunda-platform)
2 - name: "department"
3   mapper-type: "user-attribute-ldap-mapper"
4   ldap.attribute: "department"
5   user.model.attribute: "department"
6 - name: "branch"
7   mapper-type: "user-attribute-ldap-mapper"
8   ldap.attribute: "physicalDeliveryOfficeName"
9   user.model.attribute: "branch"
10 - name: "approval-limit"
11   mapper-type: "user-attribute-ldap-mapper"
12   ldap.attribute: "extensionAttribute1"
13   user.model.attribute: "approvalLimit"
14 - name: "manager"
15   mapper-type: "user-attribute-ldap-mapper"
16   ldap.attribute: "manager"
17   user.model.attribute: "managerDn"
```

A second kind of mapper — a *protocol mapper* — then places those attributes into the token, so each one travels, as a *claim*, in the JWT the platform receives. With Microsoft Entra ID the arrangement is the equivalent: the application registration is configured with *optional claims* and group claims, so the user’s attributes and group memberships — sourced, ultimately, from the AD the Entra directory is synchronised from — are emitted in the token.

Reading the attributes in a process

Once the attributes are claims in the token, a Camunda worker can read them. Listing 35.4 is a job worker that picks up the authenticated user’s directory attributes from the token and uses them in the process — here, checking a transaction against the user’s AD-held approval limit.

Listing 35.4. A job worker reading Active Directory attributes from the authenticated user’s token — the department, branch, and approval limit travel as token claims.

```
1 @JobWorker(type = "check-approval-authority")
2 public Map<String, Object> check(ActivatedJob job,
3   @AuthenticationPrincipal Jwt token) {
4   // attributes sourced from AD, carried as token claims
5   String department = token.getClaimAsString("department");
6   String branch     = token.getClaimAsString("branch");
7   long limit       = token.getClaim("approvalLimit");
8
9   long amount = job.getVariable("transactionAmount");
10
11   // the AD-held approval limit governs the decision
12   boolean withinAuthority = amount <= limit;
13
14   return Map.of(
15     "approverDepartment", department,
16     "approverBranch",     branch,
17     "withinAuthority",    withinAuthority);
18 }
```

The worker never queries Active Directory itself — it reads the attributes from the *token*, where the identity provider placed them, having sourced them from AD. This is the disciplined arrangement: the directory is queried once, by the identity provider, at authentication; the attributes then travel in the signed token; and the process reads them from there. A process that reached out to AD directly on every step would couple itself to the directory's availability and protocol, and would lose the signature guarantee the token gives — the attributes in the token are vouched for by the identity provider, where attributes fetched ad hoc are not.

Read directory attributes from the *token*, not by querying Active Directory from inside a worker. The attributes in a signed token are vouched for by the identity provider and travel with the request; a worker that queries AD directly on every step couples the process to the directory's uptime and protocol, multiplies load on the directory, and trusts data with no signature behind it. Configure the attributes the platform needs as mappers and claims once — then every process reads them from the token it already has.

35.8 Authorization: from authenticated identity to permitted action

Once a caller is authenticated — a token in hand, its signature and audience verified — the platform must decide what that caller may *do*. This is authorization, and the orchestration cluster does it through a *resource-based* model.

The model has three parts. A *resource* is a kind of thing the platform governs — a process definition, a user task, a decision definition, a deployment. A *permission* is an action on that kind of resource — the process-definition resource has a `CREATE_PROCESS_INSTANCE` permission, a `READ` permission, and others; the user-task resource has permissions to read, claim, and complete. An *authorization* grants a specific permission on a specific resource to a specific identity or role.

The consequence is fine-grained and exactly what a regulated institution needs. A machine client that starts onboarding processes can be granted `CREATE_PROCESS_INSTANCE` on the onboarding process definition and *nothing else* — it cannot start a lending process, cannot complete a task, cannot query unrelated instances. A claims analyst's role can be granted the permissions to read and complete claims user tasks and nothing touching payments. This is the least-privilege principle of Part IV's security chapter, realised concretely: the authorization model is where "each identity can do only what its role requires" stops being a principle and becomes a configuration.

35.9 Roles and security on human-in-the-loop tasks

The authorization model just described applies to every resource the platform governs, but it deserves its own treatment for one resource in particular: the *user task* — the human-in-the-loop step. Human tasks are where a regulated process most visibly meets people, and getting their authorization wrong is not an abstract risk; it means the wrong person sees, claims, or completes a piece of regulated work.

A user task, as a governed resource, carries a small set of distinct permissions, and naming them precisely is the start of getting this right. The permission to *read* a task — to see it exists,

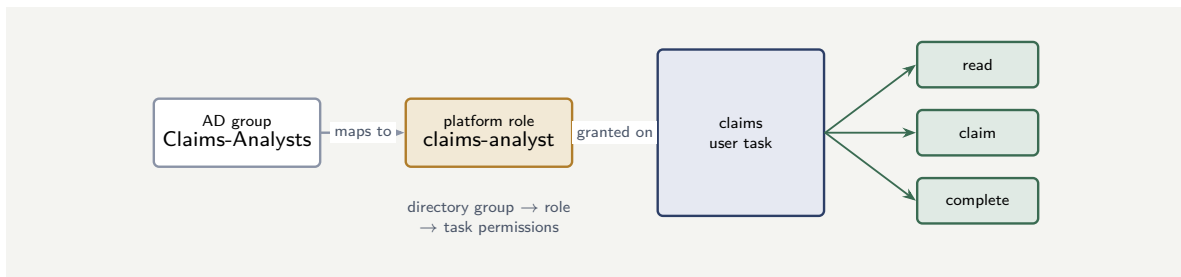


Figure 35.4. Role-based security on a human task. A corporate-directory group maps to a platform role, and the role is granted specific permissions — read, claim, complete — on a class of user task. The three permissions are separate acts of trust, granted separately.

on a worklist — is distinct from the permission to *claim* it — to take it as one’s own — which is distinct again from the permission to *complete* it — to submit its outcome and advance the process. These are separate permissions because they are separate acts of trust: an auditor might be permitted to read a queue of approval tasks without ever being permitted to claim or complete one; a trainee might claim and work a task whose completion is gated behind a review. Authorization on a human task is therefore not one switch but several.

These permissions are granted to *roles*, not to individuals — the disciplined default of [Chapter 30](#)’s task allocation, seen now from the security side. A role such as `claims-analyst` is granted read, claim, and complete on the claims user tasks; a `claims-auditor` role is granted read alone; a `senior-underwriter` role is granted the tasks an analyst cannot complete. Because, as the LDAP section established, these roles are mapped from the corporate directory’s groups, the chain is complete and auditable: a person is in an Active Directory group, that group maps to a platform role, that role carries specific permissions on specific user tasks — and the institution can answer, for any task, exactly who may act on it and why.

Figure 35.4 traces the chain from directory group to task permission.

Two task-level security disciplines complete the picture, and both are properties of the process model rather than the authorization configuration alone. The first is the *candidate group*: a well-modelled user task names the role that may work it — its candidate group — directly in the BPMN, so the model itself declares who the task is for, and the authorization configuration and the model agree. The second is the *four-eyes principle* of [Chapter 30](#): a task whose completion must be checked by a different person from the one who did the prior step is an *authorization* rule — the checker task excludes, from its candidates, the identity that completed the maker task. Four-eyes is not a workflow decoration; it is task-level security, enforced by the same model that grants the permissions.

On a human task, *read*, *claim*, and *complete* are separate permissions and must be granted separately. Granting a role blanket access to a class of task — because separating the three “seemed like detail” — collapses real distinctions: an auditor who should only *see* an approval queue can suddenly *complete* approvals; a trainee who should only *work* a task can submit its regulated outcome unchecked. In a regulated process the three permissions correspond to three different levels of trust; grant each to the roles that genuinely warrant it, and no more.

Worked example: the blast radius of an over-scoped client

Problem. A platform exposes its Process API to 6 machine clients — 6 separate backend systems that each start one specific kind of process. Under the current setup, all 6 share a single registered client with a single secret, authorized for `CREATE_PROCESS_INSTANCE` on *all* 20 process definitions on the platform. A security review proposes giving each backend system its own registered client, each authorized for `CREATE_PROCESS_INSTANCE` on only the 1 process definition it actually starts. Treating the “blast radius” of a leaked client secret as the number of process definitions an attacker could start instances of, compare the two setups.

Setup. The blast radius of one leaked secret is the number of process definitions that secret is authorized to start. We compute it for the shared client and for a per-system client, and consider what the per-system setup costs.

Working. *Current setup — one shared client.* The single secret is authorized on all 20 process definitions. A leak of that one secret lets an attacker start instances of

20 process definitions — blast radius 20.

Proposed setup — per-system clients. Each of the 6 clients is authorized on exactly 1 process definition. A leak of any one secret lets the attacker start instances of

1 process definition — blast radius 1.

The reduction in single-leak blast radius:

$$\frac{20 - 1}{20} = 0.95, \quad \text{a 95\% reduction.}$$

Discussion. The per-system setup cuts the blast radius of a leaked secret by 95%, and the worked example is the security chapter of Part IV met in concrete Camunda configuration. Under the shared client, a single leaked secret — copied from a misconfigured log, a compromised host, a careless repository commit — lets an attacker start instances of every process on the platform: lending, claims, payments, all of it. Under per-system clients, the same leak reaches exactly one process definition, and starting the others requires compromising further independent secrets. The cost of the proposal is real but modest: 6 clients to register and 6 secrets to rotate instead of 1, which is precisely why [Chapter 24](#)’s secrets-management service and infrastructure-as-code are not optional — per-system authorization is only sustainable when the clients and secrets are managed by machinery. And note what the calculation does *not* capture: the shared client is also unauditably in a way the per-system clients are not. When all 6 systems call as one client, the audit record cannot say *which* system started a given instance; with per-system clients, every instance is attributable to the one accountable caller — the accountability link of Part IV’s security chapter, which the shared client quietly breaks.

Practical next steps

Inventory the machine clients registered against your platform’s identity provider. For each, list the resources and permissions it is authorized for. Find the client authorized for the most — the widest blast radius — and ask whether every one of those permissions is genuinely needed by that one caller. The client that can do the most is your platform’s worst-case credential leak, and narrowing it is the highest-value authorization change available.

Chapter in brief

A regulated platform must answer who may do what before it runs. The platform delegates *authentication* — who is this? — to an external identity provider speaking OIDC, and makes *authorization* — what may they do? — its own decision. A signed JWT carries the authenticated identity and its claims, including the *aud* audience claim the cluster validates. Human users authenticate by the authorization-code flow; machine callers by the client-credentials flow, presenting a client ID and secret for a token. Keycloak and Microsoft Entra ID are the two common providers, and both rest on the corporate *LDAP* directory — Active Directory — as the single source of truth for staff identity: Keycloak reaches it through LDAP user federation, Entra ID as the cloud projection of an AD-synchronised directory, and in both the directory's groups become the platform's roles. Authorization is resource-based — permissions on resources, granted to roles — and on the *human task* the permissions to *read*, *claim*, and *complete* are separate acts of trust, granted separately to the roles that warrant them.

CHAPTER 36

The Process API

36.1 Why a process needs an API

A process model deployed to the engine is inert until something interacts with it. Some of that interaction is the human kind — a person in Tasklist, an operator in Operate — but a great deal of it is programmatic: another system needs to deploy a model, start an instance, ask how an instance is progressing, read or update its variables, or cancel it. The *Process API* is how that programmatic interaction happens, and this chapter treats it in the depth a platform integrator needs.

In Camunda 8, this API is part of the unified *Orchestration Cluster REST API* — a single, consistent HTTP interface to the cluster, covering process instances, user tasks, decisions, and more. This chapter treats the process-oriented part of it; [Chapter 38](#) treats the task-oriented part. The API is HTTP and JSON, every call is authenticated by a bearer token in the sense of [Chapter 35](#), and every call is subject to the resource-based authorization of that chapter — so the chapter that precedes this one is its precondition.

36.2 The shape of the API

The Process API is organised around a small set of resources and the operations on them, and the naming is deliberately consistent — a discipline worth knowing because it makes the whole API predictable once any part of it is learned.

The central resources are the *process definition* — a deployed model, a particular version of it — and the *process instance* — one running execution of a definition. The operations divide into three kinds.

Deployment operations put resources onto the cluster: a BPMN model, a DMN model, a form. A deployment makes a new process definition available to be instantiated.

Lifecycle operations act on instances: create one, cancel one, and — the subject of [Chapter 37](#) — the several ways of starting one. They change what exists and what is running.

Query operations ask questions without changing anything: search for process instances matching some criteria, fetch one instance by its key, read an instance's variables, retrieve the incidents on an instance. They are how a calling system observes the platform.

Every resource the API returns is identified by a *key* — a `processInstanceKey`, a `processDefinitionKey` — a unique technical identifier the caller holds onto and uses in subsequent calls. The naming convention is uniform: a unique technical identifier is always suffixed `key`, which is a small thing that makes the API markedly easier to work with.

Figure [36.1](#) draws the API's shape — the three operation kinds, the two central resources, and the authorization gate every call passes. The picture is worth holding in mind through the rest of the chapter, because each section that follows treats one slice of it.

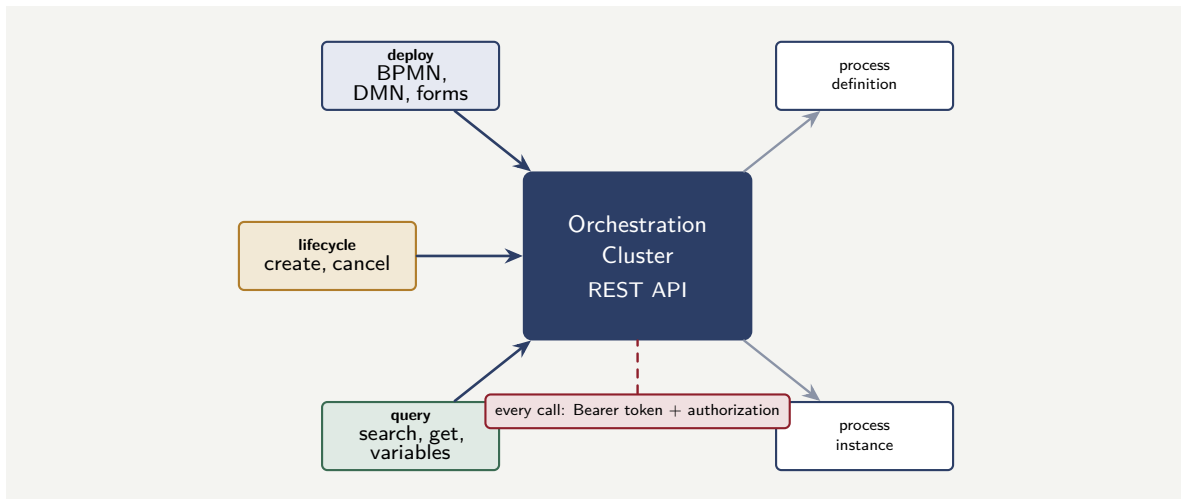


Figure 36.1. The Process API: three kinds of operation — deployment, lifecycle, and query — acting on two resources, the process definition and the process instance. Every call passes the same gate: a bearer token and a resource-authorization check.

36.3 Deploying a process

Before a process can be started it must be deployed. A deployment call sends one or more resources — the BPMN model, any DMN models it references, any forms — to the cluster, and the cluster makes them available, returning the keys of the definitions it created.

Deployment has a property that matters for governance: [Chapter 27](#)’s versioning. Deploying a model that already exists, changed, creates a *new version* of that process definition; it does not overwrite the old one. Instances already running on the old version continue on it, as [Chapter 27](#) described, and new instances use the new version unless told otherwise. The deployment call is therefore not a destructive operation, and a disciplined platform treats deployment as a governed step in a release pipeline — the model deployed is a versioned, reviewed artefact — not an ad-hoc action.

[Listing 36.1](#) is the deployment call through the Java client. It sends the BPMN resource to the cluster and receives back the key and version of the definition the cluster created.

[Listing 36.1](#). Deploying a process through the Java client. The call returns the key and version of the created definition — a new version, never an overwrite.

```

1 DeploymentEvent deployment = zeebeClient
2   .newDeployResourceCommand()
3   .addResourceFromClasspath("loan-origination.bpmn")
4   .send()
5   .join();
6
7 // the cluster returns the created definition's metadata
8 Process def = deployment.getProcesses().get(0);
9 log.info("deployed {} version {}",
10    def.getBpmnProcessId(), def.getVersion());

```

36.4 Querying process instances

Much of what a calling system needs from the Process API is not to change anything but to *know* something: is this instance still running, where has it got to, what are its variables, has it

hit an incident. The query operations serve this, and they are the programmatic counterpart of the Operate visibility of [Chapter 28](#).

A *search* operation finds instances matching criteria — all running instances of the lending process, all instances that started in a date range, all instances currently carrying an incident — and returns them in pages. A *get-by-key* operation fetches the full detail of one instance. A *variables* operation reads the process variables an instance carries.

Listing 36.2 shows a search call and the shape of what it returns.

Listing 36.2. Searching the Process API for running instances of a process. The bearer token authenticates the call; the filter narrows the result; the response is a page of matching instances.

```
1 curl --request POST \  
2   'https://<CLUSTER>/v2/process-instances/search' \  
3   --header "Authorization: Bearer ${ACCESS_TOKEN}" \  
4   --header 'Content-Type: application/json' \  
5   --data '{  
6     "filter": {  
7       "processDefinitionId": "lending-origination",  
8       "state": "ACTIVE"  
9     },  
10    "page": { "from": 0, "limit": 50 }  
11  }'  
12  
13 # the response is a page of matching instances  
14 # {  
15 #   "items": [  
16 #     { "processInstanceKey": "2251799813686019",  
17 #       "processDefinitionId": "lending-origination",  
18 #       "processDefinitionVersion": 4,  
19 #       "state": "ACTIVE",  
20 #       "startDate": "2026-05-20T09:14:22.000Z" }  
21 #   ],  
22 #   "page": { "totalItems": 1 }  
23 # }
```

One discipline governs the query operations and is worth stating plainly. A query is authorized, like every call, by the model of [Chapter 35](#): a caller can only find and read instances of process definitions it is authorized to read. A query is not a way around authorization; it is subject to it. This matters because process instances carry customer data, and an unauthorized caller querying instances would be a data-protection breach in the sense of Part IV's security chapter — so the platform makes the query see only what the caller's authorization permits.

A Process API query returns customer data — the variables an instance carries, the state of a real lending or claims case. Treat the query operations with the same data-protection seriousness as any access to that data: queries are authorized per process definition, callers see only what their authorization permits, and a broad query authorization is a broad data exposure. “It is only a read” is not a reason to scope a query client loosely.

36.5 Applying the chapter in practice

The principles this chapter describes are most valuable when applied deliberately and reviewed periodically. A team that applies them once, at initial design, captures their value at that moment; a team that returns to them at each major change, each annual review, and each

post-incident analysis captures their value continuously. The platform evolves; the principles hold; the specific choices they inform must evolve with the platform.

Worked example: paging through a large instance query

Problem. A reporting system must, each night, retrieve every completed instance of the claims process from the previous day — on a busy day, 24,000 instances. The Process API returns search results in pages, and the platform’s configured maximum page size is 1,000 items. Each page request takes, on average, 250 milliseconds. The reporting system’s nightly window is tight. How many page requests are needed, how long does the retrieval take, and what is the disciplined way to structure it?

Setup. The number of pages is the total instance count divided by the page size, rounded up. The total time is the page count times the per-request time. We then consider how the retrieval should be structured.

Working. Pages required to retrieve 24,000 instances at 1,000 per page:

$$\frac{24,000}{1,000} = 24 \text{ page requests.}$$

Total retrieval time:

$$24 \times 250 \text{ ms} = 6,000 \text{ ms} = 6 \text{ seconds.}$$

Discussion. Six seconds for 24 sequential page requests is unremarkable, and that is partly the point: the worked example is less about the arithmetic than about the discipline of *paged* retrieval. A naive integrator’s instinct is to ask the API for “all completed claims instances” in one call, and a well-designed API will simply refuse — the configured maximum page size of 1,000 is a deliberate limit, because an unbounded result set is a denial-of-service risk against the cluster and a memory risk in the caller. The disciplined retrieval pages explicitly: request page one, process it, request page two, and so on, holding only one page in memory at a time. Two further disciplines belong with it. The paging must be *stable* — the search must be ordered by a stable key, typically the `processInstanceKey`, so that instances do not shift between pages as the underlying data changes mid-retrieval. And the retrieval must be *resumable* — if it fails at page 15 of 24, it should resume from page 15, not restart, which the stable ordering makes possible. A nightly bulk retrieval that is unpagged, unstable, or unresumable works in testing against a hundred instances and fails in production against twenty-four thousand. The six-second figure is the easy part; paging it correctly is the engineering.

Practical next steps

For one integration that reads from your Process API, check how it handles volume. Does it page explicitly, or does it assume one call returns everything? Is its paging ordered by a stable key? Can it resume a failed bulk retrieval part-way, or must it restart? An integration that fails any of these three works until the data grows — and then fails at the worst time, in a nightly batch.

36.6 Operating a running instance

Querying observes; the Process API also *operates* — it changes a running instance. Two operations matter most, and both are powerful enough to deserve care.

To *cancel* an instance is to end it before it completes — a process started in error, a duplicate, a case withdrawn. Listing 36.3 is the call.

Listing 36.3. Cancelling a running process instance through the Java client — ending it before completion.

```
1 zeebeClient
2   .newCancelInstanceCommand(processInstanceKey)
3   .send()
4   .join();
```

To *modify* an instance is more surgical: it moves an instance's token, activating one element and terminating another, without cancelling the whole process. It is how an operator rescues a stuck instance or corrects one that took a wrong path — and it is the same mechanism the in-flight migration of [Chapter 83](#) uses. Listing 36.4 moves a stuck instance forward.

Listing 36.4. Modifying a running instance: moving the token by activating one element and terminating another — a surgical correction, not a cancellation.

```
1 zeebeClient
2   .newModifyProcessInstanceCommand(processInstanceKey)
3   .activateElement("manual-review")
4   .and()
5   .terminateElement(stuckElementInstanceKey)
6   .send()
7   .join();
```

Instance cancellation and modification are operational power tools, and every use is a deviation from the process as modelled — an instance ended early, or a token moved by hand. Each must be authorized, audited, and rare. A platform on which operators routinely cancel and modify instances is not being operated; it is being overridden, and the process model has stopped describing what actually happens. Treat these calls as exceptional interventions, recorded and reviewed — not as a normal part of running the platform.

Chapter in brief

The Process API is how systems interact programmatically with deployed processes. In Camunda 8 it is part of the unified Orchestration Cluster REST API — HTTP and JSON, every call bearer-token authenticated and resource-authorized. It is organised around process definitions and process instances, with deployment, lifecycle, and query operations, every resource identified by a uniformly named key. Deploying a changed model creates a new version rather than overwriting. Query operations observe the platform without changing it, but return customer data and are authorized per process definition. Large queries must be retrieved in stable, resumable pages.

CHAPTER 37

Starting a Process

37.1 The most important call there is

Of all the operations on a process, one is foundational: *starting* an instance. Every running process began with a start, every other operation presupposes an instance that a start created, and the way a process is started shapes how it is governed, how it is triggered, and how it relates to the systems around it. This chapter gives the start its own treatment because the question “how does this process begin?” has more answers, and more consequence, than it first appears.

Chapter 34 drew the distinction between a process that is *commanded* to start — a system explicitly creates an instance — and a process that is *triggered* to start — something happens, and an instance begins in reaction. This chapter treats both, and the several concrete mechanisms within each, because choosing the right start mechanism is a genuine design decision.

37.2 The explicit start: creating an instance

The most direct way a process begins is that a caller explicitly creates an instance through the Process API of Chapter 36. The caller names the process definition — by its ID, or by a specific processDefinitionKey for an exact version — supplies the initial process variables, and the cluster creates a running instance and returns its processInstanceKey.

Listing 37.1 shows the call. It is the concrete realisation of the *command-driven* start.

Listing 37.1. Starting a process instance explicitly through the Process API. The caller names the definition, supplies the initial variables, and receives the key of the created instance.

```
1  curl --request POST \  
2    'https://<CLUSTER>/v2/process-instances' \  
3    --header "Authorization: Bearer ${ACCESS_TOKEN}" \  
4    --header 'Content-Type: application/json' \  
5    --data '{  
6      "processDefinitionId": "lending-origination",  
7      "variables": {  
8        "applicationRef": "APP-2026-44817",  
9        "channel": "mobile",  
10       "requestedAmount": 15000  
11     }  
12   }'  
13  
14   # the response identifies the created instance  
15   # {  
16   #   "processDefinitionId": "lending-origination",  
17   #   "processDefinitionKey": "2251799813685249",  
18   #   "processInstanceKey": "2251799813686019",  
19   #   "processDefinitionVersion": 4  
20   # }
```

Two choices in this call are design decisions, not details. The first is *which version*: naming only the processDefinitionId starts the latest deployed version, which is usually right; naming a specific processDefinitionKey pins the start to an exact version, which a caller does

when it must be certain which model logic runs. The second is the *initial variables*: these are the process's starting data, and the data-flow discipline of [Chapter 3](#) begins here — the caller should supply exactly the variables the process needs to begin, with known names and types, validated, and no more.

The same call through the Java client is [Listing 37.2](#). The client gives a fluent, typed builder over the same operation the REST call performs — naming the definition, supplying the variables, and returning the created instance's key.

[Listing 37.2](#). Starting a process instance through the Java client — the typed equivalent of the REST call, naming the latest version of the definition.

```
1 ProcessInstanceEvent instance = zeebeClient
2   .newCreateInstanceCommand()
3   .bpmnProcessId("lending-origination")
4   .latestVersion()
5   .variables(Map.of(
6     "applicationRef", "APP-2026-44817",
7     "channel",        "mobile",
8     "requestedAmount", 15000))
9   .send()
10  .join();
11
12 long instanceKey = instance.getProcessInstanceKey();
```

The Process API is not Java-only. Camunda provides clients for several languages, and a great many institutions run their integration code on Node.js. [Listing 37.3](#) is the same process start through the Node client — the same operation, the same fluent shape, in JavaScript.

[Listing 37.3](#). Starting the same process through the Camunda Node.js client — the Process API reached from JavaScript rather than Java.

```
1 // Node.js -- the Camunda JavaScript client
2 const { Camunda8 } = require('@camunda8/sdk');
3 const camunda = new Camunda8();
4 const zeebe = camunda.getZeebeGrpcApiClient();
5
6 const instance = await zeebe.createProcessInstance({
7   bpmnProcessId: 'lending-origination',
8   variables: {
9     applicationRef: 'APP-2026-44817',
10    channel:        'mobile',
11    requestedAmount: 15000,
12  },
13 });
14
15 console.log(instance.processInstanceKey);
```

The Java and the Node calls are deliberately alike: both name the process by its `bpmnProcessId`, both pass the same initial variables, both return the created instance's key. The Process API is one API; the client library is a convenience over it in whichever language the integration team works in, and nothing about the process, the variables, or the semantics changes between them. An institution with Java services and Node services reaches the same platform, the same way, from both.

37.3 Start and await: the synchronous variant

Usually, starting a process and learning its outcome are separate concerns: the caller starts an instance, receives its key, and the process runs on independently — the caller does not

wait. This is the right default, because a process is typically long-running and the caller has no reason to block.

But some cases genuinely need the outcome *now*. A real-time account-eligibility check, called from a screen the customer is waiting at, must start a short process and return its result before the screen can proceed. For this, the Process API offers a *start-and-await* variant — `awaitCompletion` — where the call does not return when the instance is created but when the instance *completes*, carrying back the process’s final variables.

This is powerful and must be used with discipline. Start-and-await is appropriate only for processes that are genuinely short and bounded — the caller is holding a connection open for the whole run, and a process that might take minutes, or wait on a human or a third party, must never be started this way. **Chapter 25**’s latency-budget thinking applies directly: a start-and-await call is on a customer’s critical path, and the process behind it must complete within the budget, with a timeout-and-fallback for the case where it does not.

The start-and-await variant holds the caller’s connection open until the process completes. It is correct only for genuinely short, bounded processes — a synchronous eligibility or validation check. Never start a long-running process, or one that waits on a human task or a third party, with `awaitCompletion`: the caller will block for minutes or forever, the connection will time out, and the caller will not know whether the process is still running. Long-running processes are started fire-and-forget; only the caller observes their progress through the query API.

37.4 The triggered start: messages, timers, and events

Many processes are not commanded to start by a caller at all; they *begin in reaction* to something, and BPMN’s start events — introduced in **Chapter 3** and **Chapter 34** — are how the model expresses this.

A *message start event* begins an instance when a matching message arrives. The message may come from an inbound connector catching a webhook — **Chapter 33** — or from the event backbone of **Chapter 25**. A process modelled to start on a “payment-received” message begins an instance every time that event occurs, with no system explicitly calling a start: the event *is* the start.

A *timer start event* begins instances on a schedule — a daily reconciliation process that starts itself every night, a process that starts on a fixed date. The schedule is in the model, and the engine creates the instances when the time comes.

A *signal start event* begins instances on a broadcast — a platform-wide signal that several different processes all react to by starting.

The triggered start is what makes a process a participant in the event-driven world of **Chapter 34** rather than merely a thing systems call. A well-designed process makes its start mechanism explicit and deliberate: the modeller decides whether this process is commanded or triggered, and if triggered, by what — and the start event on the diagram states the decision.

37.5 Starting from within: call activities and subprocesses

A process can also be started by *another process*. The call activity of **Chapter 3** — the mechanism of decomposition — starts an instance of a subprocess each time a parent instance reaches it. From the subprocess’s point of view this is a start like any other; from the plat-

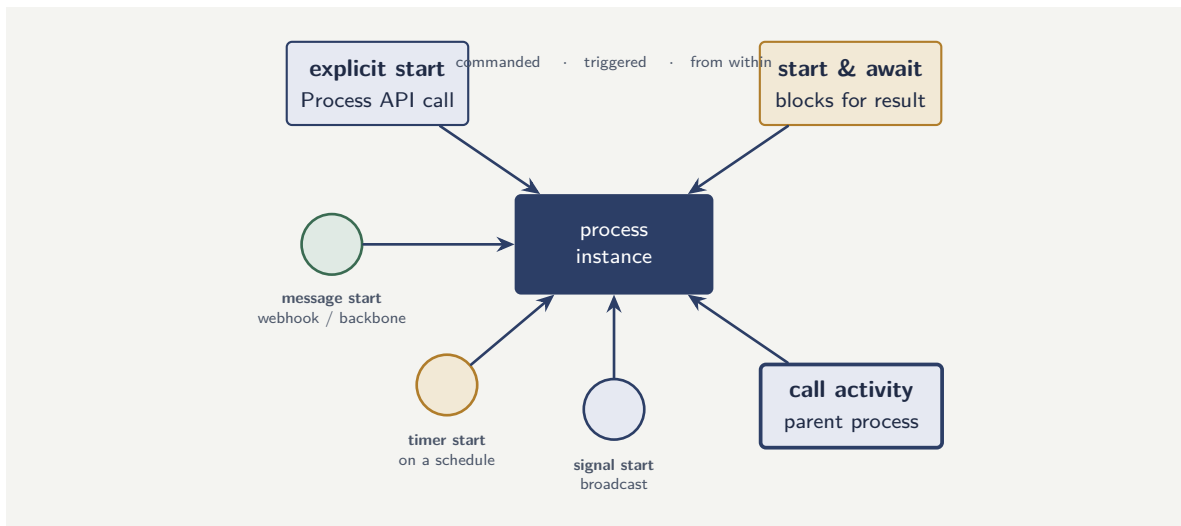


Figure 37.1. The ways a process instance begins. A commanded start — an explicit Process API call, optionally awaiting the result — a triggered start — a message, timer, or signal start event reacting to the world — or a start from within, by a parent process’s call activity. The right mechanism is chosen per process.

form’s point of view it is how the layered journey-and-capability structure of [Chapter 21](#) actually runs. The onboarding journey starting its *verify identity* capability is one process starting another, and every reusable capability of [Chapter 29](#) is, in execution, started this way.

Figure [37.1](#) collects the mechanisms. The worked example below walks through choosing among them for four real processes — and the lesson there is exactly that the choice is different for each.

37.6 Applying the chapter in practice

The principles this chapter describes are most valuable when applied deliberately and reviewed periodically. A team that applies them once, at initial design, captures their value at that moment; a team that returns to them at each major change, each annual review, and each post-incident analysis captures their value continuously. The platform evolves; the principles hold; the specific choices they inform must evolve with the platform.

Worked example: choosing a start mechanism

Problem. An insurer is modelling four processes and must choose a start mechanism for each. (1) A *quote* process: a customer on the website requests a quote and waits a few seconds for the price. (2) A *renewal* process: every policy must begin its renewal workflow 30 days before its expiry date. (3) A *first-notification-of-loss* process: it must begin whenever a claim is reported, through any channel — web, phone, broker. (4) A *verify-cover* capability: it is needed as a step inside the claims process. For each, identify the right start mechanism and the reason.

Setup. We match each process to the start mechanism the chapter has described — explicit start, start-and-await, message start, timer start, signal start, or call activity — by asking what actually causes the process to begin and whether the caller needs the result synchronously.

Working. (1) *Quote*. A customer is waiting at a screen for a price; the process is short and

bounded. This is an *explicit start with awaitCompletion* — the website calls the Process API, and the call returns when the quote process completes, carrying the price.

(2) *Renewal*. The process must begin on a date relative to each policy's expiry. Nothing calls it; time causes it. This is a *timer start event* — the schedule is in the model, and the engine creates each renewal instance when its date arrives.

(3) *First notification of loss*. A claim can be reported through several channels, and the process must begin however it arrives. This is a *message start event* — each channel publishes a "claim-reported" message, through an inbound connector or the event backbone, and the message starts the process. Modelling it as an explicit API start would force every channel to call the API directly and couple them all to it; the message start decouples them.

(4) *Verify cover*. It is not a standalone process at all; it is a step inside the claims process. It is started by a *call activity* in the claims model — the reusable-capability pattern of [Chapter 29](#).

Discussion. Four processes, four different start mechanisms — and that is the lesson. The instinct of an inexperienced team is to start everything the same way, usually by an explicit API call, because it is the mechanism they learned first. The worked example shows why that instinct misleads. The renewal process started by an API call would need some *other* system to track every policy's expiry and remember to make the call — rebuilding, badly, the timer the engine already provides. The first-notification process started by an API call would couple every reporting channel directly to that call; modelled as a message start, the channels just publish an event and know nothing of the process. The quote process started *without* `awaitCompletion` would leave the customer's screen with no price to show. And the verify-cover capability is not started by anything external at all. The start mechanism is a genuine design decision, made per process by asking one question — what actually causes this process to begin? — and the answer is, correctly, different for different processes.

Practical next steps

For each process your institution runs or plans, write down its start mechanism and the reason. Look especially for processes started by an explicit API call that would more naturally be timer-started or message-started — a process that some other system has to remember to start is a process whose start mechanism is wrong, and the fix removes both the coupling and the other system's hidden responsibility.

Chapter in brief

Starting an instance is the foundational process operation, and how a process starts is a genuine design decision. An explicit start creates an instance through the Process API, naming the definition and supplying initial variables; the start-and-await variant blocks for the result and suits only short, bounded processes. A triggered start — message, timer, or signal start event — begins an instance in reaction to the world, decoupling the process from any caller. A call activity starts a subprocess, which is how the layered capability structure runs. The right mechanism is chosen per process by asking what actually causes it to begin.

CHAPTER 38

The Task API

38.1 Human work, reached programmatically

[Chapter 30](#) treated task allocation — how a user task finds the right person — and [Chapter 28](#) introduced Tasklist as the component where human work happens. But Tasklist is not the only way human work is reached. An institution frequently needs to embed human tasks in its *own* applications — a claims adjuster working in the insurer’s own claims workbench, not in a separate Tasklist window — and for that it needs to reach tasks programmatically. The *Task API* is how, and this chapter treats it in detail.

The Task API, like the Process API of [Chapter 36](#), is part of the unified Orchestration Cluster REST API: HTTP and JSON, every call bearer-token authenticated per [Chapter 35](#), every call resource-authorized. Where the Process API acts on process instances, the Task API acts on *user tasks* — querying them, claiming them, completing them, updating them.

38.2 The token-based task model

A point of discipline runs through this chapter, and it is worth stating first. Every Task API call is authenticated by a token — the bearer token of [Chapter 35](#) — and that token does two things at once. It authenticates the *caller*: which application is making the request. And, when the call is made on behalf of a person, the identity in the token is the *person* whose action this is.

This matters because human tasks are where accountability is most concrete. [Chapter 30](#)’s four-eyes principle, the audit record of Part IV’s security chapter, the “named accountable human at every consequential decision” of the whole manuscript — all of them depend on the platform knowing *which person* claimed a task and *which person* completed it. The token-based model is what carries that identity: when an adjuster, working in the institution’s own claims workbench, completes a task, the workbench’s call to the Task API carries a token identifying that adjuster, and the platform records the completion against them. A Task API integration that completes tasks under a single shared service identity, losing the individual person, has broken the accountability link — it has built a faster claims workbench and a worse audit record.

Every Task API call carries a token, and when the call is a person’s action, the token must carry the *person’s* identity — not a shared service account. A claims workbench that completes every adjuster’s tasks under one service identity records every decision as made by “the workbench,” which is no accountability at all. The token-based task model exists precisely so that the individual accountable person is on every claim and every completion. Preserve the user’s identity through to the Task API call; do not collapse it into a service account.

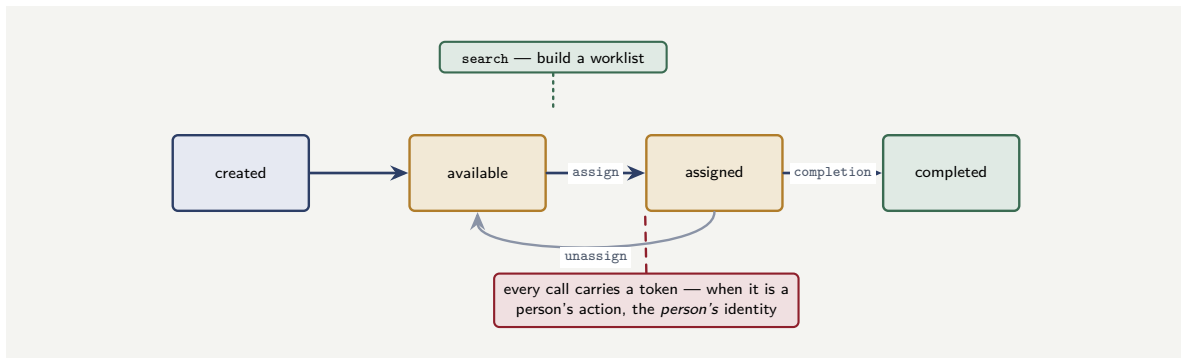


Figure 38.1. The Task API and the user-task lifecycle. The *search*, *assign*, *unassign*, and *completion* operations drive the transitions; *search* builds the worklist. Every call carries a token — and when the call is a person's action, that token carries the person's identity, which is what makes the completion accountable.

38.3 The task lifecycle through the API

Chapter 30 described the user-task lifecycle — created, available, claimed, completed. The Task API is how each transition is driven programmatically, and the operations correspond directly to the lifecycle.

A *search* operation queries tasks: the tasks available to a particular candidate group, the tasks assigned to a particular person, the tasks for a particular process instance. This is how an institution's own application builds a worklist — it searches the Task API for the tasks its current user may work, and presents them.

An *assign* (or claim) operation gives a task to a person — removing it from others' worklists, the atomic claim of **Chapter 30** that prevents two people working the same case. An *unassign* operation returns it to the group.

A *complete* operation finishes a task: it carries back the variables representing the person's decision, and the process advances. This is the operation that the data-flow discipline of **Chapter 3** governs — the completion carries named, typed, validated output variables that the next gateway will route on.

An *update* operation changes a task's attributes — its due date, its priority — without completing it.

Figure 38.1 maps the API operations onto the lifecycle of **Chapter 30**. The red annotation is the chapter's discipline made visual: the token on the *completion* call is not just authentication but the carrier of the accountable individual's identity.

Listing 38.1 shows the two operations at the heart of an embedded worklist: searching for a person's tasks, and completing one.

Listing 38.1. Two Task API calls. The first finds the tasks a candidate group may work; the second completes a task, carrying the person's decision as output variables. Both are bearer-token authenticated.

```

1 # search -- find available tasks for a candidate group
2 curl --request POST \
3   'https://<CLUSTER>/v2/user-tasks/search' \
4   --header 'Authorization: Bearer ${ACCESS_TOKEN}' \
5   --header 'Content-Type: application/json' \
6   --data '{
7     "filter": { "candidateGroup": "claims-adjusters",
8               "state": "CREATED" },

```

```

9     "page": { "from": 0, "limit": 25 }
10   }'
11
12   # complete -- finish a task, carrying back the decision
13   curl --request POST \
14     'https://<CLUSTER>/v2/user-tasks/${userTaskKey}/completion' \
15     --header "Authorization: Bearer ${ACCESS_TOKEN}" \
16     --header 'Content-Type: application/json' \
17     --data '{
18       "variables": {
19         "claimDecision": "APPROVED",
20         "approvedAmount": 4200,
21         "adjusterNote": "Damage consistent with reported incident."
22       }
23     }'
```

38.4 Claiming and assigning through the API

Search and completion are the two ends of the task lifecycle; between them sit the operations that govern *who holds* a task — *claim*, *unclaim*, and *assign* — and a workbench application reaches each through the Task API.

To *claim* a task is for a person to take a group task as their own; to *unclaim* is to return it to the group; to *assign* is for the platform, or a supervisor, to set a task’s assignee directly. Listing 38.2 shows the claim and assignment calls.

Listing 38.2. Task API calls that govern who holds a task: a person claims a group task; a supervisor assigns one directly. Both are bearer-token authenticated.

```

1  # claim -- the authenticated person takes a group task
2  curl --request POST \
3    'https://<CLUSTER>/v2/user-tasks/${userTaskKey}/assignment' \
4    --header "Authorization: Bearer ${ACCESS_TOKEN}" \
5    --header 'Content-Type: application/json' \
6    --data '{ "assignee": "adjuster.okafor",
7              "allowOverride": false }'
8
9  # unassign -- return the task to its candidate group
10 curl --request DELETE \
11   'https://<CLUSTER>/v2/user-tasks/${userTaskKey}/assignee' \
12   --header "Authorization: Bearer ${ACCESS_TOKEN}"
```

One field in the claim call is a real safeguard: `allowOverride`. With it `false`, the call claims the task only if it is *unassigned* — so two adjusters clicking “claim” on the same task at the same moment do not both succeed; the second call fails because the task is no longer free. This is the atomic claim that [Chapter 30](#) described, enforced at the API. A workbench that ignored it — that set the assignee unconditionally — would let two people both believe they own one task.

The same lifecycle is reached through the Java client, which gives a typed command for each operation. Listing 38.3 completes a task through the client, carrying the person’s decision as result variables.

Listing 38.3. Completing a user task through the Java client — the typed equivalent of the REST completion call, carrying the decision as variables.

```

1  camundaClient
2    .newUserTaskCompleteCommand(userTaskKey)
3    .variables(Map.of(
4      "claimDecision", "APPROVED",
5      "approvedAmount", 4200))
```

```
6     .send()
7     .join();
```

The Task API, like the Process API, is reachable from the Node.js client — the choice for a workbench whose back end runs on Node. Listing 38.4 completes the same user task in JavaScript.

Listing 38.4. Completing a user task through the Camunda Node.js client — the Task API reached from JavaScript.

```
1 // Node.js -- completing a user task
2 const { Camunda8 } = require('@camunda8/sdk');
3 const camunda = new Camunda8();
4 const tasklist = camunda.getCamundaRestClient();
5
6 await tasklist.completeUserTask({
7   userTaskKey,
8   variables: {
9     claimDecision: 'APPROVED',
10    approvedAmount: 4200,
11  },
12 });
```

A workbench application is exactly the kind of system that is often built on Node, with a JavaScript or TypeScript front end and a Node back end, so the Node client is not an afterthought for the Task API — it is, for many institutions, the primary one. The operation is identical to the Java form: the same task key, the same decision variables, the same completion semantics.

Whether reached over REST or through the client, the calls are the same lifecycle — search, claim, complete — and the same authorization, examined next, governs every one of them.

One search is worth showing on its own: the *personal worklist*. The lifecycle's search found the tasks a candidate *group* may work; a person's own application asks a narrower question — what is assigned to *me*, right now — and Listing 38.5 is that query, filtering on the authenticated user as assignee.

Listing 38.5. Querying a personal worklist: the tasks assigned to the authenticated user, the query a person's own task application makes.

```
1 # the tasks assigned to me, not yet completed
2 curl --request POST \
3   'https://<CLUSTER>/v2/user-tasks/search' \
4   --header "Authorization: Bearer ${ACCESS_TOKEN}" \
5   --header 'Content-Type: application/json' \
6   --data '{
7     "filter": { "assignee": "adjuster.okafor",
8               "state": "CREATED" },
9     "sort": [ { "field": "creationDate",
10               "order": "ASC" } ],
11     "page": { "from": 0, "limit": 25 }
12   }'
```

The two searches — by candidate group and by assignee — are how a workbench builds its two views: the *group queue*, the unclaimed work a person may take, and the *personal worklist*, the work they have already claimed. Together they are the everyday surface of the Task API.

38.5 Authorization and the task

Task API authorization is more intricate than process authorization, and the intricacy is deliberate, because task access is two questions, not one.

The first question is process-level: may this caller see and act on tasks of *this kind of process* at all? That is a permission on the process definition, exactly as in [Chapter 35](#) — a caller with no authorization for the claims process sees none of its tasks.

The second question is task-level: of the tasks this caller may see in principle, which *specific* ones may they act on? This is where [Chapter 30](#)’s allocation meets [Chapter 35](#)’s authorization. A task assigned to one adjuster should not be completable by another; a task in a candidate group should be claimable only by members of that group; the four-eyes constraint means the person who completed the maker task must be excluded from the checker task. These are task-level authorization rules, and the Task API enforces them — so that the allocation discipline of [Chapter 30](#) is not merely modelled but enforced on every programmatic call.

The combination — process-level permission plus task-level rules — is what lets an institution embed tasks in its own applications without weakening control. The claims workbench reaches tasks through the Task API; the API, checking the adjuster’s token against both levels of authorization, ensures the adjuster sees and completes exactly the tasks they are entitled to, and no others.

Worked example: the audit consequence of a shared task identity

Problem. An insurer embeds claims tasks in its own workbench through the Task API. Two designs are on the table. In design A, the workbench holds *one* service-account token and completes every adjuster’s tasks under it. In design B, each adjuster’s own identity is carried through to the Task API on every call. The insurer handles 60,000 claims tasks a month. A regulator later investigates a sample of 200 claim decisions and, for each, asks the institution to name the individual adjuster accountable. For how many of the 200 can each design answer, and what does the gap mean?

Setup. The design can answer for a sampled decision if the audit record attributes that decision’s task completion to a named individual. We determine, for each design, what the audit record contains.

Working. *Design A — shared service identity.* Every one of the 60,000 monthly task completions is recorded against the single service account. For the regulator’s sample of 200, the audit record names, in every case, the service account — “claims-workbench” — and not a person. Decisions the institution can attribute to the accountable individual:

0 of 200.

Design B — individual identity carried through. Every task completion is recorded against the adjuster whose token carried it. For all 200 sampled decisions, the audit record names the individual.

200 of 200.

Discussion. The arithmetic is stark — 0 against 200 — and it is meant to be, because the gap is not a degradation but a categorical failure. Design A’s workbench works: adjusters see their tasks, complete them, the processes advance. It is, functionally, indistinguishable from design B on any ordinary day. The difference appears only under exactly the scrutiny a

regulated institution must withstand — and then design A cannot answer the single most basic question a supervisor asks, *who decided this?*, for *any* of its 60,000 monthly decisions. This is the warning of this chapter quantified: the token-based task model exists so that the accountable individual is on every completion, and a workbench that collapses every adjuster into one service account has not made a small audit compromise — it has unbuilt the accountability link that Part IV’s security chapter, and [Chapter 17](#)’s treatment of accountability, identified as the deepest regulatory principle of the whole platform. Design B costs nothing extra in running the workbench; it requires only that the integration carry the user’s identity through to the Task API rather than discarding it. The lesson is that an accountability failure does not announce itself in testing or in daily operation. It waits, silent, until the one occasion — the investigation, the dispute, the examination — when the institution needs the answer and discovers design A never recorded it.

Practical next steps

If your institution embeds tasks through the Task API, check one thing: when a user completes a task, does the API call carry that user’s identity, or a shared service identity? Pull a recent task completion from the audit record and see whose name is on it. If it is a service account, your platform is generating decisions that no individual is recorded as accountable for — and that is a finding to fix before a regulator finds it for you.

Chapter in brief

The Task API reaches user tasks programmatically, letting an institution embed human work in its own applications. Part of the unified Orchestration Cluster REST API, it is bearer-token authenticated and resource-authorized. The token-based model is the chapter’s discipline: when a call is a person’s action, the token must carry that person’s identity, because the audit record, the four-eyes principle, and the manuscript’s accountable-human principle all depend on knowing which individual claimed and completed each task. The API’s operations — search, assign, complete, update — drive the task lifecycle. Task authorization is two-level: a process-definition permission plus task-level rules that enforce allocation and four-eyes.

Listeners: Hooking into the Lifecycle

39.1 Cross-cutting logic and where it belongs

Every process carries logic that is not part of its business flow but must nonetheless run alongside it: recording when a step started for a service-level measurement, writing an entry to the audit record, notifying a person that a task is now theirs, validating that a completion carries sensible data. This is *cross-cutting* logic — it cuts across many steps and many processes — and the question of where it belongs has a wrong answer that is tempting and a right answer that is disciplined.

The wrong answer is to scatter it through the process: a notification service task here, an audit service task there, a validation gateway somewhere else. That works, but it clutters the model — the business flow becomes hard to see among the plumbing — and it is fragile, because the cross-cutting concern is now reimplemented at every point it is needed. The right answer is the *listener*: a hook that fires automatically at a defined point in an element's lifecycle, carrying the cross-cutting logic without placing it on the diagram as a flow step.

Camunda provides two kinds of listener, and this chapter treats both: *execution listeners*, which fire on the lifecycle of any BPMN element, and *user task listeners*, which fire on the lifecycle of a user task specifically. [Chapter 30](#) mentioned task listeners in passing; this chapter gives listeners the full treatment, because a platform that uses them well keeps its process models clean and its cross-cutting concerns consistent.

39.2 Execution listeners

An *execution listener* fires on the lifecycle of a BPMN element — a task, an event, a gateway, a subprocess, or the process itself. It has two event types. A *start* listener fires *before* the element is processed — useful for setting up variables, recording a start time, checking a precondition. An *end* listener fires *after* the element is processed — useful for cleanup, for post-processing, for emitting a “step completed” record.

A point of architecture matters here, and it is a consequence of [Chapter 32](#)'s external-task pattern. In Camunda 8, an execution listener is *not* a special in-engine callback; it is implemented as a *job worker*, exactly like a service task. The engine, reaching a listener point, creates a job; a worker activates it, runs the listener logic, and completes it; only then does the element proceed. This means everything [Chapter 32](#) established about job workers applies to listeners unchanged — they are thin, they are idempotent, they fail and retry like any job — which is a considerable simplification: a listener is not a new concept to learn but a job worker bound to a lifecycle point rather than a service task.

If several listeners of the same event type are defined on one element, they run *sequentially*, in the order the model declares them. That ordering is deterministic and modelled, which matters when one listener depends on another — a listener that records a start time should

run before a listener that notifies someone the step has begun.

Because a Camunda 8 execution listener *is* a job worker, its code is a job worker — annotated with the listener’s job type rather than a service task’s. Listing 39.1 is an end-listener that records a “step completed” audit entry.

Listing 39.1. An execution listener implemented as a job worker: an end-listener that records a step-completed audit entry, then lets the element proceed.

```
1 @JobWorker(type = "audit-step-completed")
2 public void onStepEnd(ActivatedJob job) {
3     // an end listener -- fires after the element is done
4     auditService.record(
5         job.getProcessInstanceKey(),
6         job.getElementId(),
7         "COMPLETED",
8         Instant.now());
9     // returning normally lets the element proceed
10 }
```

The listener is bound to the element in the BPMN model, which names the job type the engine creates a job for. Listing 39.2 is that binding — an `executionListener` of type `end` on a service task.

Listing 39.2. Binding the execution listener to an element in BPMN: an end-type listener naming the job type its worker subscribes to.

```
1 <bpmn:serviceTask id="disburseLoan" name="Disburse Loan">
2   <bpmn:extensionElements>
3     <zeebe:executionListeners>
4       <zeebe:executionListener
5         eventType="end"
6         type="audit-step-completed"/>
7     </zeebe:executionListeners>
8   </bpmn:extensionElements>
9 </bpmn:serviceTask>
```

The two listings show the listener for what it is: a job worker (Listing 39.1) bound to a lifecycle point by a small piece of model configuration (Listing 39.2). The worker is thin — it records the audit entry and returns — exactly as the external-task discipline requires, and the engine holds the element at the listener point until the worker completes the job.

Tip

Use execution listeners for genuinely cross-cutting concerns — audit records, timing measurements, technical notifications — and keep them *off* the business flow of the diagram. A listener is the right home for “something that must happen at this lifecycle point but is not a business step.” But remember the limit: a listener is a job worker, so logic that is a real business step, with its own success and failure paths, belongs as a service task on the flow, not hidden in a listener.

39.3 User task listeners

A *user task listener* fires on the lifecycle of a user task specifically, and because the user-task lifecycle is richer than a plain element’s — created, assigned, completed, and so on, as Chapter 30 described — the user task listener can do more than an execution listener, in two important ways.

First, a user task listener has *access to user-task-specific data*: the assignee, the candidate groups, the due date, the priority. A listener firing on assignment can read who the task was assigned to and act on it — notify that person, with the specific context of their task.

Second, and more powerfully, a user task listener can *correct task data* and can *deny a life-cycle transition*. A listener firing on assignment can adjust the due date or the priority before the assignment settles — dynamic, rule-driven adjustment of the task. And a listener firing on completion can *deny* the completion, rolling the task back to its previous state, if the completion is invalid — the person tried to complete the task with variables that do not pass a validation rule.

That denial capability is the one to dwell on, because it is how a regulated process enforces correctness at the human boundary. **Chapter 3**'s data-flow discipline asked that a task's completion carry named, typed, validated output. A user task listener on completion is where that validation is enforced: it inspects the variables the person submitted, and if they are inconsistent — an approved claim with no approved amount, a decision outside the permitted set — it denies the transition, and the task returns to the person to be corrected. The validation is not a hopeful convention; it is an enforced gate.

Listing 39.3. A user task listener on the completing event. It validates the person's submitted decision and denies the completion if it is inconsistent — enforcing the data-flow discipline at the human boundary.

```
1 @JobWorker(type = "validate-claim-completion")
2 public void onCompleting(ActivatedJob job,
3                           JobClient client) {
4     var task = job.getUserTask();
5     Map<String, Object> vars = job.getVariablesAsMap();
6
7     String decision = (String) vars.get("claimDecision");
8     Object amount = vars.get("approvedAmount");
9
10    boolean invalid =
11        "APPROVED".equals(decision) && amount == null;
12
13    if (invalid) {
14        // deny the transition: the task returns to the person
15        client.newDenyCommand(job.getKey())
16            .reason("Approved claims require an approved amount.")
17            .send().join();
18    } else {
19        // allow the completion to proceed
20        client.newCompleteCommand(job.getKey()).send().join();
21    }
22 }
```

Figure 39.1 shows the listeners on the lifecycle and marks their two distinctive powers — correction and denial.

39.4 When to use this mechanism

This chapter's mechanism is powerful but not universal. Use it when the process genuinely needs it — when the cross-cutting concern cannot be expressed any other way, when the lifecycle hook is the right place for the logic, when the timer or the error boundary is the right tool for the requirement. Do not use it by default, because every mechanism adds a piece of logic that is not visible on the main process flow, and a process laden with hidden mechanisms is a process that no one can fully understand by reading its diagram. The diagram should tell the story; the mechanisms should handle the edges.

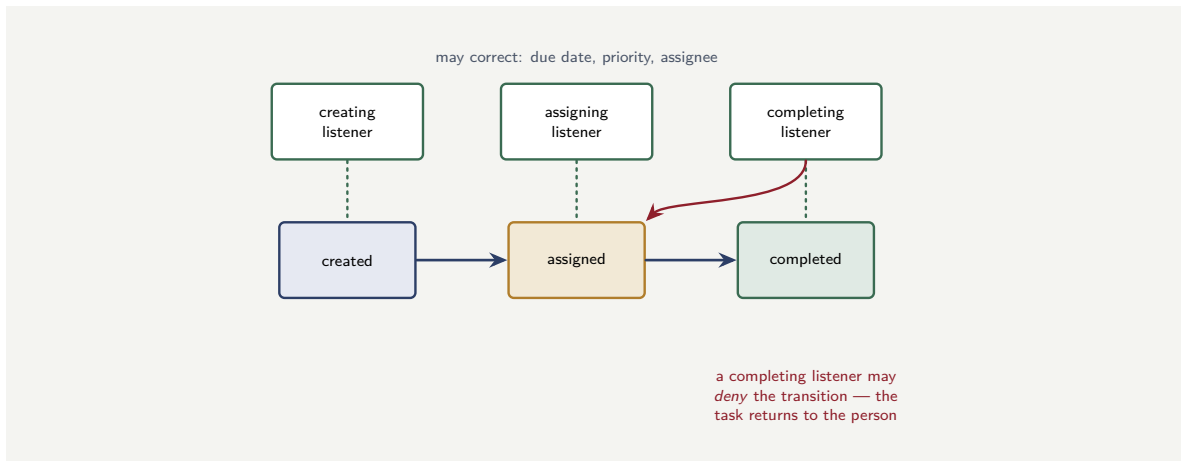


Figure 39.1. User task listeners on the task lifecycle. Each listener fires at a named transition and can read task data; the assigning listener may correct attributes such as due date and priority; the completing listener may deny an invalid completion, returning the task to the person.

Worked example: enforcing four-eyes through a task listener

Problem. A payments process has a four-eyes control: [Chapter 30](#)’s maker–checker pair, where the person who completes the *authorise* task must not be the person who completed the *prepare* task. The team has modelled the two user tasks but is enforcing the four-eyes rule by a convention — training the operators not to check their own work. An audit finds that, over a year of 90,000 payments, the convention was breached on 0.4% of payments because operators were rushed or the maker and checker were the same on-call person at night. Each breach is a control failure the regulator counts. How many breaches occurred, and how does a task listener eliminate them?

Setup. The number of breaches is the annual volume times the breach rate. We then consider what a completing listener on the *authorise* task does to that number.

Working. Breaches under the convention:

$$90,000 \times 0.004 = 360 \text{ control failures in the year.}$$

Now add a user task listener on the *completing* event of the *authorise* task. The listener reads two things: the identity of the person completing the *authorise* task — available, because the token of [Chapter 38](#) carries it — and the identity recorded against the *prepare* task, held in a process variable. If they are the same person, the listener *denies* the completion. Breaches that can reach a regulator:

$$0 \text{ — the listener makes the breach impossible.}$$

Discussion. The convention produced 360 control failures a year; the listener produces none, and the difference is the difference between a control that is *hoped for* and a control that is *enforced*. Under the convention, four-eyes depended on every operator, on every payment, including the rushed ones and the lone night-shift ones, remembering and honouring a rule — and [Chapter 122](#)’s lesson about the underfunded operating model has a sibling here: a control that depends on human diligence alone will be breached at some rate, and that rate times a year’s volume is never zero. The task listener moves the control from convention

to mechanism. The completing listener inspects the two identities and, finding them equal, denies the transition — the task simply will not complete, and the operator sees why. The four-eyes rule is no longer a thing operators must remember; it is a thing the platform will not let them violate. This is the deeper value of the listener's denial capability: it converts a procedural control into a structural one, and a structural control does not have a breach rate.

Practical next steps

Find a control in one of your processes that currently depends on a human convention — “operators must not approve their own work,” “the reviewer must check the attachment is present.” Ask whether a user task listener on the relevant transition could enforce it instead, by denying the completion when the rule is not met. Every control you move from convention to listener is a control whose breach rate drops to zero.

Chapter in brief

Listeners are hooks that carry cross-cutting logic at lifecycle points, keeping it off the business flow of the diagram. An execution listener fires on the start or end of any BPMN element; in Camunda 8 it is implemented as a job worker, inheriting every job-worker discipline. A user task listener fires on the user-task lifecycle, can read task-specific data such as the assignee, and has two distinctive powers: it can correct task attributes like due date and priority, and it can deny a lifecycle transition — which is how a regulated process enforces completion validity and controls such as four-eyes as structural mechanisms rather than human conventions.

Asynchronous Continuations and the Transaction Boundary

40.1 Why this chapter is deep and narrow

This chapter treats one topic — the asynchronous continuation, the `asyncBefore` and `asyncAfter` markers and what they mean — in more depth than any other mechanism in the manuscript. The narrowness is deliberate. The asynchronous continuation is the single most misunderstood detail of building on Camunda, the misunderstanding causes a specific and serious class of production defect, and the topic also marks the sharpest difference between the two generations of the engine. A platform engineer who understands this chapter understands why processes are durable; one who does not will, sooner or later, ship a process that loses work.

Chapter 31 introduced asynchronous continuations briefly, in the context of the embedded Java engine. This chapter goes to the bottom of the topic and treats both generations: the explicit `asyncBefore` model of Camunda 7, where the modeller places the boundaries, and the Camunda 8 model, where the engine places them for you — and where, crucially, the defaults are different and the reasoning must adjust.

40.2 What a transaction boundary is, and why a process needs them

A process instance is a long-lived thing. It may run for milliseconds or for months, and across that life the engine must be able to do two things: persist the instance's progress durably, so that a crash or a restart does not lose it, and retry a failed step without redoing the whole process. Both depend on the process having *transaction boundaries* — points at which the instance's state so far is committed, durably, and beyond which a later failure cannot roll back.

Think of the boundaries as save points in the run of the instance. Between two save points, the engine advances the instance in one unit of work; if that unit fails, everything since the last save point is undone and retried; if it succeeds, the new state is committed and becomes a fresh save point. The placement of those save points is therefore not a performance detail. It determines two things of real consequence: how much work is lost and redone when a step fails, and — the hazard Chapter 31 dwelt on — whether an external effect can be stranded by a rollback.

Some points in a process are *always* save points, in both generations of the engine. A *wait state* — a user task, a message catch event, a timer — is inherently a boundary: the instance stops there, durably, by its nature. The interesting question is about the points *between* wait states: the runs of automated steps, and whether each is one long unit of work or several short ones.

40.3 The Camunda 7 model: `asyncBefore` and `asyncAfter`

In the embedded Camunda 7 engine, the modeller controls the boundaries explicitly, with two flags on an activity.

`asyncBefore=true` places a save point *before* the activity: the engine commits the instance's state, and then a separate job-executor thread, in a fresh transaction, performs the activity. `asyncAfter=true` places a save point *after* the activity: the activity runs, and then the engine commits before continuing.

Without either flag, an activity runs in the *same* transaction as whatever preceded it. A run of five service tasks with no async flags is one transaction: the engine advances through all five, and only at the next wait state does it commit. If the fourth task fails, the transaction rolls back — all five tasks are undone — and the whole run is retried from the start.

The discipline of placing async flags in Camunda 7, then, is the discipline of deciding how much work shares a fate. [Chapter 31](#)'s worked example showed the cost of getting it wrong: a long single transaction means a late failure redoes early, expensive, externally-affecting work. The rule that chapter gave — place an async boundary immediately after any step with an external effect — is, precisely, the rule for placing `asyncAfter`.

In Camunda 7, an activity with no `asyncBefore` or `asyncAfter` flag shares a transaction with its neighbours. A long run of un-flagged service tasks is one large transaction: a failure anywhere in it rolls back the entire run, and any external effect within it that has already happened is now stranded against an engine state that has been undone. The absence of an async flag is itself a decision — and usually the wrong one for any run containing an external effect.

40.4 The Camunda 8 model: always asynchronous

Camunda 8, with the Zeebe engine, changes the model in a way that a Camunda 7 engineer must consciously absorb, because the old flags do not merely behave differently — they are gone.

In Zeebe, *every task is implicitly `asyncBefore` and `asyncAfter`*. There is no flag to set, because the engine already treats every task as its own committed unit. When Zeebe processes an instance, it advances it token by token, and each step the token takes is a committed record in the engine's replicated event log — the append-only log of [Chapter 27](#). The save points the Camunda 7 modeller placed by hand are, in Zeebe, everywhere by default.

This is a genuine simplification, and it removes an entire category of Camunda 7 defect — the accidentally-too-long transaction. But it must be understood correctly, and two points are easy to get wrong.

The first: a Camunda 7 engineer migrating a model must *not* look for the async flags, find them absent, and conclude the model has no save points. The opposite is true — the model has save points everywhere. The diagram converter of [Chapter 28](#) simply drops the `asyncBefore` and `asyncAfter` attributes on migration, because in Zeebe they are not configuration; they are the default and only behaviour.

The second, and more important: the Zeebe model does not make [Chapter 31](#)'s external-effect hazard disappear — it relocates it. Because every task is its own committed unit, a job worker's work commits, or fails, as its own transaction; the engine's state advance and the worker's external effect are no longer in one shared transaction that can roll back together.

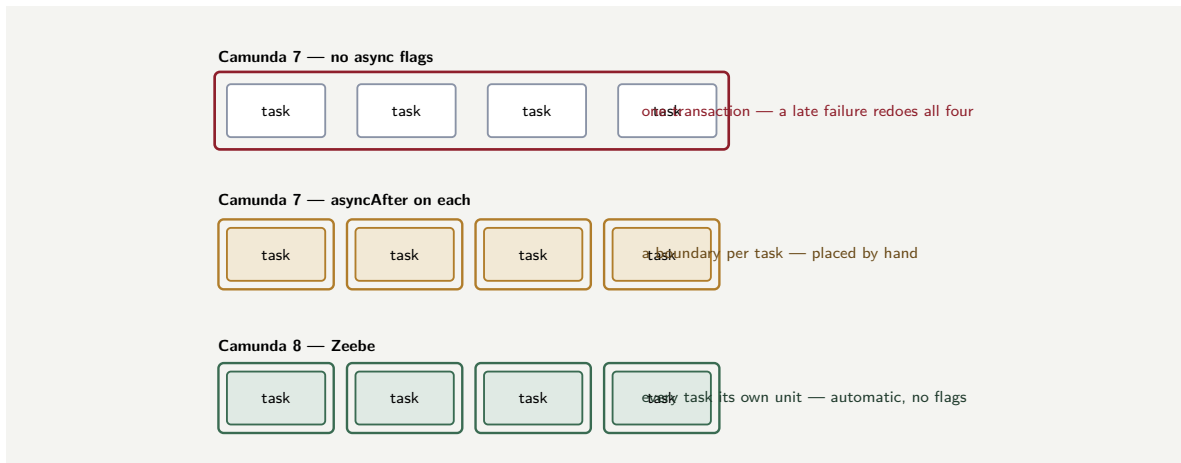


Figure 40.1. Transaction boundaries across the two engine generations. In Camunda 7 an un-flagged run of tasks shares one transaction — a late failure rolls back the whole run; `asyncAfter` flags, placed by hand, give each task its own boundary. In Camunda 8, Zeebe makes every task its own committed unit automatically — the boundaries the modeller once placed are everywhere by default.

But the worker can still be handed the same job twice — the at-least-once delivery of [Chapter 32](#) — and so the discipline shifts from “place the transaction boundary correctly” to “make the worker idempotent.” The hazard is the same hazard; the engine has moved where the engineer must address it.

Tip

Migrating from Camunda 7 to Camunda 8, do not read the absence of `asyncBefore` flags as the absence of save points. Zeebe makes every task an implicit save point — the boundaries the Camunda 7 modeller placed by hand are now automatic and everywhere. The engineering effort that went into placing `async` boundaries correctly should be redirected, not abandoned: in Camunda 8 it goes into making every externally-affecting worker idempotent, because that is where the same old hazard now lives.

Figure 40.1 contrasts the three cases: the Camunda 7 long transaction, the hand-flagged Camunda 7 model, and the automatically-bounded Camunda 8 model.

40.5 When to use this mechanism

This chapter’s mechanism is powerful but not universal. Use it when the process genuinely needs it — when the cross-cutting concern cannot be expressed any other way, when the lifecycle hook is the right place for the logic, when the timer or the error boundary is the right tool for the requirement. Do not use it by default, because every mechanism adds a piece of logic that is not visible on the main process flow, and a process laden with hidden mechanisms is a process that no one can fully understand by reading its diagram. The diagram should tell the story; the mechanisms should handle the edges.

Worked example: lost work under three boundary designs

Problem. A run of automated steps in a process performs six service tasks in sequence; the sixth fails transiently on 5% of runs, and the process runs 100,000 times a year. Each service task takes 200 milliseconds of work. Consider three designs. *Design A (Camunda 7, no flags)*: the six tasks share one transaction. *Design B (Camunda 7, flagged)*: an `asyncAfter` boundary follows every task. *Design C (Camunda 8)*: the engine makes every task its own committed unit. When the sixth task fails and the run is retried, how much work is redone under each design?

Setup. On a failure of task 6, the work redone is whatever shares a transaction with task 6 and is therefore rolled back with it. We count the redone work per failing run, and across the 5% of 100,000 runs that fail.

Working. Failing runs per year:

$$100,000 \times 0.05 = 5,000.$$

Design A — one shared transaction. Task 6's failure rolls back all six tasks; all six are redone:

$$6 \times 200 \text{ ms} = 1,200 \text{ ms redone per failing run,}$$

$$5,000 \times 1,200 \text{ ms} = 6,000 \text{ s} = 1.67 \text{ hours of redundant work a year.}$$

Design B — boundary after every task. Task 6's failure rolls back only task 6; only task 6 is redone:

$$1 \times 200 \text{ ms} = 200 \text{ ms per failing run,}$$

$$5,000 \times 200 \text{ ms} = 1,000 \text{ s} = 0.28 \text{ hours a year.}$$

Design C — Camunda 8, always async. Every task is its own committed unit by default — identical in effect to design B, with *no flags to set*: 0.28 hours a year, achieved automatically.

Discussion. Designs B and C redo a sixth of the work design A redoes, and that arithmetic is the visible lesson — but the deeper one is in the contrast between B and C. Design B achieves the good outcome through *engineering effort*: a Camunda 7 modeller had to understand transaction boundaries, decide where the save points belong, and place an `asyncAfter` flag on every task — and a modeller who forgot, or who did not understand, would silently ship design A and its 1.67 hours of redundant work, or worse, its stranded external effects. Design C achieves the same good outcome *by default*: Zeebe's always-async model means the sixth-of-the-work result is what an engineer gets without knowing the concept exists. This is real progress, and it is why the Camunda 8 model is the better one. But the worked example's sting is in the tail: design C still redoes task 6, once, on every failing run — 5,000 times a year — and if task 6 has an external effect, that effect happens again on the retry. The engine has made the transaction boundary automatic; it has not made the retry disappear. The 0.28 hours is free in Camunda 8; the idempotency that makes the retry safe is not, and remains the engineer's responsibility.

Practical next steps

If you run Camunda 7, audit one process for runs of un-flagged service tasks and, in particular, for any external-effect task not immediately followed by an `asyncAfter` boundary. If you run Camunda 8, accept that the boundaries are automatic — and redirect the attention that would have gone to placing them onto the question that still matters: is every externally-effecting worker idempotent against a retry?

Chapter in brief

A process needs transaction boundaries — save points — so that progress is durable and a failed step is retried without redoing the whole process. Wait states are always boundaries. In Camunda 7, the modeller places the boundaries between wait states explicitly with `asyncBefore` and `asyncAfter`; an un-flagged run of tasks shares one transaction, and a late failure rolls the whole run back. In Camunda 8, Zeebe makes every task an implicit save point — the boundaries are automatic and everywhere, removing the too-long-transaction defect. But the always-async model relocates rather than removes the external-effect hazard: the discipline shifts from placing boundaries to making every externally-affecting worker idempotent.

CHAPTER 41

Error Handling in the Worker

41.1 The worker is where failure is met

Chapter 32 introduced the job worker and named, briefly, the three ways a job can fail to complete: a technical failure, a business error, and the incident that an exhausted technical failure becomes. That chapter's purpose was the worker as a pattern; this chapter's purpose is the worker as a place where failure must be *handled well*, because handling it badly is one of the most common and most damaging defects in a real platform.

The premise is simple and uncomfortable: in a financial platform, things fail constantly. Core banking systems time out. Payment rails reject. Identity services are briefly unavailable. Networks drop. None of this is exceptional; it is the normal weather of an integrated platform, and the worker is where that weather is met. A worker that handles failure well makes the platform resilient; a worker that handles it badly turns every transient hiccup into a stuck process, a stranded effect, or a false alarm.

41.2 The taxonomy of failure, precisely

Chapter 32 drew the three-way distinction; this chapter sharpens it, because the boundaries between the categories are where workers go wrong.

A *transient technical failure* is a failure that retrying might fix, because nothing is fundamentally wrong: a timeout, a momentary network drop, a downstream system briefly overloaded. The defining test is *would the identical call, made again in a minute, plausibly succeed?* If yes, it is transient. The correct response is to *fail the job with retries remaining*, so the engine reoffers it after a back-off.

A *permanent technical failure* is a failure that retrying cannot fix, because something is genuinely broken: a malformed request the downstream system will always reject, a configuration error, a missing credential, a bug. The defining test is the same one, answered *no* — the identical call will fail identically forever. Retrying a permanent failure is pure waste; the correct response is to stop retrying and *raise an incident* immediately, so a human is brought in.

A *business error* is not a failure at all. It is a legitimate, expected, negative *outcome*: the card was declined, the identity did not match, the account is closed. Nothing is broken, retrying is meaningless, and the process model has — or should have — a path for it. The correct response is to raise a *BPMN error*, which the model catches with a boundary error event and routes.

The single most consequential worker bug is misclassifying across these boundaries. Treat a transient failure as permanent, and the platform raises incidents and summons humans for hiccups that would have cleared themselves. Treat a permanent failure as transient, and the platform retries a hopeless call for hours, burning resources and delaying the incident that a human needed to see immediately. Treat a business error as a technical failure, and the platform retries — pointlessly — a decision that will never change, and eventually raises a spurious incident for what was simply a declined card. Treat a technical failure as a business

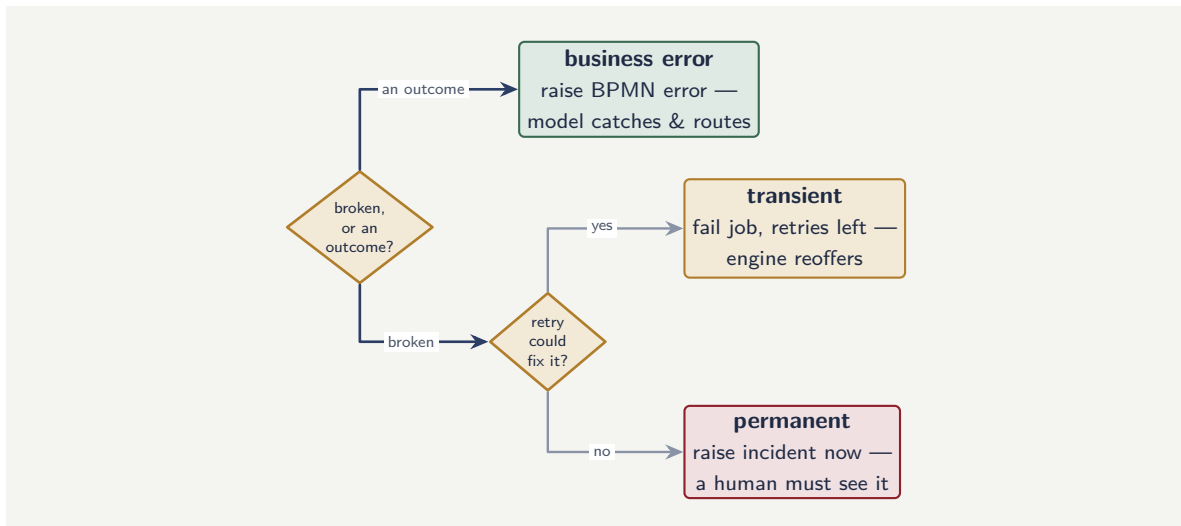


Figure 41.1. The worker’s failure-classification decision. Two questions resolve every failure: is something broken, or is this a legitimate negative outcome? And if broken, could a retry plausibly fix it? The three leaves are the three correct responses — and misclassifying across them is the most common worker bug.

error, and the model routes a transient network blip down a permanent “card declined” path, telling a customer their card was refused when in truth the rail timed out.

Figure 41.1 draws the classification as the two questions it really is. A worker should make this decision explicitly — in code a reader can see — not by accident of which exception happens to propagate.

41.3 Retries, back-off, and the limit

When a worker fails a job as transient, it does not retry blindly. Two parameters govern the retry, and both are design decisions.

The *retry count* is how many attempts the job gets before its technical failure is reclassified as an incident. Too low, and a downstream system’s brief wobble becomes an incident a human must clear; too high, and a genuinely broken call is retried far too long before anyone is told. The count should be set to the number of attempts across which a genuinely transient fault would plausibly clear — typically a small number, three to five.

The *back-off* is the delay between attempts, and it should *grow* with each attempt — exponential back-off. The reason is consideration for the downstream system. A downstream system that is failing because it is overloaded is a system that retrying immediately and repeatedly makes *worse*: the retries are load, and the load is the problem. Exponential back-off — wait one second, then four, then sixteen — gives the struggling system room to recover, and is the difference between a worker that helps a downstream system recover and one that holds it down.

There is a subtler point about the retry limit, and it connects to [Chapter 34](#)’s event-driven thinking. A worker calling a downstream system that is *completely* down does not benefit from any number of immediate retries. Past a few attempts, the right response is not to keep retrying but to stop, raise the incident, and let the platform’s operators — through Operate, as [Chapter 28](#) described — decide when the downstream system is back and the jobs should be retried as a batch. The retry limit is the boundary between “this will probably clear itself”

and “a human needs to know.”

A worker that retries a failing downstream system immediately and without limit does not improve its own chances — it amplifies the downstream system’s problem, because the retries are themselves load on a system that is failing under load. Always use exponential back-off, and always set a finite retry limit. Past that limit, the right action is an incident and a human decision, not more retries. A retry storm against a struggling core banking system has turned a worker’s error handling into a denial-of-service attack on the institution’s own infrastructure.

41.4 The error handling in code

The taxonomy — transient, business, and the retry discipline — is concrete in the worker’s code. Listing 41.1 is a job worker that classifies its failures the three ways this chapter has named.

Listing 41.1. A job worker classifying its failures: a transient fault is re-thrown for retry, a business fault becomes a `BpmnError`, an exhausted retry count becomes an explicit incident.

```
1 @JobWorker(type = "credit-bureau-score")
2 public Map<String, Object> score(ActivatedJob job) {
3     try {
4         var result = bureauClient.score(
5             job.getVariable("applicationRef"));
6         return Map.of("creditScore", result.score());
7     }
8     } catch (BureauUnavailable e) {
9         // transient: re-throw so Camunda retries it,
10        // with the back-off configured on the job
11        throw e;
12    }
13    } catch (BureauDeclined e) {
14        // business: a BPMN error the model routes
15        throw new BpmnError("BUREAU_DECLINED",
16            e.getReason());
17    }
18 }
```

The worker above lets Camunda’s job mechanism handle the retry count and back-off, which is the usual choice. But a worker can also manage retries *explicitly* — failing the job with a chosen remaining-retry count and a back-off duration — and Listing 41.2 shows that form, using the lower-level client to set an exponential back-off as it fails the job.

Listing 41.2. A worker failing a job with an explicit remaining-retry count and an exponential back-off duration — giving a struggling downstream system room to recover.

```
1 // in the catch for a transient fault:
2 int remaining = job.getRetries() - 1;
3 Duration backoff = Duration.ofSeconds(
4     (long) Math.pow(4, 3 - remaining)); // 1s, 4s, 16s
5
6 zeebeClient.newFailCommand(job.getKey())
7     .retries(remaining)
8     .retryBackoff(backoff)
9     .errorMessage("bureau unavailable, will retry")
10    .send()
11    .join();
```


Two things in these listings are the chapter’s argument made executable. In the first, the *catch blocks are* the taxonomy: the type of the exception caught decides whether the failure is re-thrown as transient or raised as a `BpmnError` the process model routes — the classification is not a comment, it is the control flow. In the second, the back-off is computed to *grow* with each attempt — one second, four, sixteen — so that a worker failing against an overloaded downstream system gives it room rather than adding to its load. A worker that caught every exception the same way, or failed jobs with no back-off, would have neither property.

41.5 The worker and the stranded effect

One failure mode deserves separate, careful treatment, because it is the one that does real financial damage: the *stranded external effect*.

The scenario is precise. A worker calls a downstream system — it moves money, say — and the call *succeeds*. Then, before the worker can report the job complete to the engine, the worker crashes, or the network between the worker and the engine drops. The engine never hears that the job completed. Its lock expires; the job is reoffered; another worker picks it up and moves the money *again*.

This is not a hypothetical. It is the ordinary consequence of the at-least-once delivery that [Chapter 32](#) established, and no amount of careful error classification prevents it, because nothing failed in a way the worker could classify — the work succeeded; only the report of it was lost. The only defence is the one [Chapter 31](#) and [Chapter 32](#) named and this chapter underlines: *idempotency*. The worker’s external call must carry a key — an idempotency key, derived from the job — that lets the downstream system recognise a repeated call and decline to apply it twice. With the key, the second “move money” call is recognised by the payment system as the same instruction it already executed, and ignored. Without it, the customer is debited twice.

The discipline is therefore absolute: a worker whose call has an irreversible external effect must make that call idempotent, and idempotency is not the worker’s own cleverness but a contract with the downstream system — the worker sends a key, the downstream system honours it. A worker that cannot make its effect idempotent has not finished handling its errors, whatever its classification logic looks like.

41.6 When to use this mechanism

This chapter’s mechanism is powerful but not universal. Use it when the process genuinely needs it — when the cross-cutting concern cannot be expressed any other way, when the lifecycle hook is the right place for the logic, when the timer or the error boundary is the right tool for the requirement. Do not use it by default, because every mechanism adds a piece of logic that is not visible on the main process flow, and a process laden with hidden mechanisms is a process that no one can fully understand by reading its diagram. The diagram should tell the story; the mechanisms should handle the edges.

Worked example: the cost of an unclassified failure

Problem. A worker calls an identity-verification service. On 3% of the 200,000 calls a year, the service returns an outcome the worker does not explicitly classify — it simply lets whatever exception arises propagate as a generic technical failure. Of those unclassified outcomes, analysis shows half are genuine transient timeouts and half are the service’s legitimate “iden-

tity could not be verified” business response. The worker is configured with 5 retries. A genuine business response, retried, still returns the same business response. Each retry costs a call to the identity service, billed at \$0.12. How much does the misclassification cost in wasted identity-service calls a year, and what else is wrong?

Setup. The misclassified business responses are the ones that cost waste: each is retried 5 times, every retry a billed call that cannot change the outcome. We count those calls. We then consider the non-monetary damage.

Working. Unclassified outcomes a year:

$$200,000 \times 0.03 = 6,000.$$

Half are genuine business responses misclassified as technical failures:

$$6,000 \times 0.5 = 3,000 \text{ misclassified business responses.}$$

Each is retried 5 times — 5 wasted billed calls that cannot change the outcome:

$$3,000 \times 5 = 15,000 \text{ wasted calls,}$$

$$15,000 \times \$0.12 = \$1,800 \text{ a year in wasted identity-service billing.}$$

Discussion. The \$1,800 is the visible cost, and it is the least of the problem. Consider what else the misclassification does. Each of those 3,000 misclassified business responses, after exhausting its 5 retries, does not route down the model’s “identity not verified” path — it raises an *incident*. So the platform generates 3,000 spurious incidents a year, each summoning a human operator through Operate to investigate what was never a fault at all but a perfectly ordinary “this person’s identity did not check out.” That is the real damage: 3,000 false alarms bury the genuine incidents an operator must not miss, and an operator who learns that incidents are usually nothing is an operator who will, eventually, be slow to the one incident that is something — the alert-fatigue failure of [Chapter 125](#)’s observability, caused here by a worker that would not classify its failures. And there is a customer-facing harm too: a genuine “identity not verified” is a real outcome the process should handle gracefully — perhaps routing to a manual identity check — and instead those 3,000 customers’ applications simply stall in an incident queue. The fix is small and the chapter’s whole point: the worker must *explicitly classify* the identity service’s response — a verification failure is a BPMN error, a timeout is a transient technical failure — rather than letting every non-success propagate as the same generic exception. Explicit classification costs a few lines of code; the failure to do it costs \$1,800, three thousand false incidents, and three thousand stalled customers.

Practical next steps

Take one worker in your platform and read its error handling. Does it explicitly classify what comes back — this is a business error, this is transient, this is permanent — or does it let a single generic exception path absorb everything? A worker with one undifferentiated failure path is a worker generating false incidents and stranding real outcomes. Explicit classification is the fix, and it is a few lines of code.

Chapter in brief

The worker is where a financial platform's constant failures are met, and handling them well is what makes the platform resilient. Failure has three kinds: a transient technical failure, which a retry might fix — fail the job with retries left; a permanent technical failure, which a retry cannot fix — raise an incident now; and a business error, a legitimate negative outcome — raise a BPMN error for the model to route. Misclassifying across these boundaries is the most damaging worker bug. Retries use exponential back-off and a finite limit, lest they become a denial-of-service on a struggling downstream system. The stranded external effect — a call that succeeded but whose completion was lost — is defended against only by idempotency.

Camunda Forms: the Human Interface

42.1 The form is where the process meets the person

[Chapter 30](#) treated task allocation — how a user task finds the right person — and noted, in passing, that the form a task presents matters as much as the allocation. This chapter gives the form its due. *Camunda Forms* are the structured screens a user task presents: they show the person the work, give them the context the decision needs, collect their input, and return it to the process. A process's automated parts can be flawless, but if its human steps present poor forms, the process is slow, error-prone, and disliked by the people who must use it — and the form is where that is decided.

A form is built in the Modeler of [Chapter 28](#), as a first-class artefact alongside the BPMN and DMN models, and a user task is configured to present it. The form is, deliberately, not free-form code: it is a declared structure of fields, laid out, typed, and validated — the same philosophy of a restricted, governable artefact that [Chapter 3](#) applied to BPMN and [Chapter 4](#) to DMN, applied now to the human screen.

42.2 What a form is made of

A Camunda Form is a set of *components* arranged on a screen, and the components fall into three kinds.

Input components collect the person's entry: text fields, number fields, date pickers, checkboxes, radio groups, select lists. Each input component is bound to a process variable — the *key* of the component is the variable it reads and writes — and this binding is the form's half of the data-flow discipline of [Chapter 3](#). A number field bound to `approvedAmount` reads that variable to pre-fill, and writes it when the person submits.

Presentation components show the person information they need but do not edit: text blocks, headings, the value of a process variable displayed read-only, an image. These are how the form gives the person *context* — the claim summary, the policy terms, the model's recommendation — assembled onto the one screen where the decision is made.

Structural components organise the screen: groups, separators, dynamic lists. They are how a form with many fields stays readable rather than becoming an undifferentiated wall of inputs.

A property runs through all of them and is worth stating: a form component is *declared*, not programmed. The form designer places a component, binds it to a variable, sets its type, and sets its validation rules — and the platform renders and enforces it. There is no hand-written screen code to drift from the model, no bespoke validation to test separately. The form is a governable artefact in the same sense as the process model.

Figure [42.1](#) draws the three component kinds and the way input and presentation components relate to the process variables.

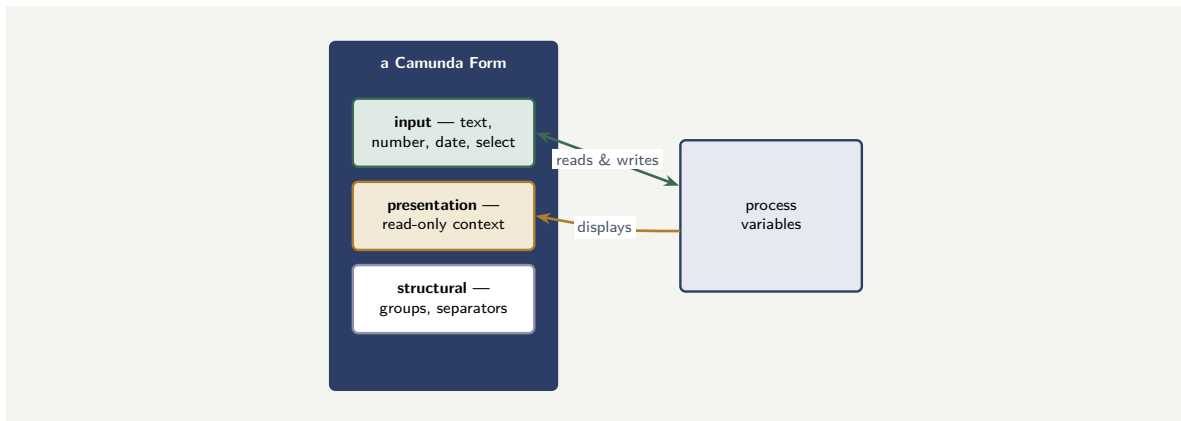


Figure 42.1. The three kinds of Camunda Form component. Input components are bound to process variables and read and write them; presentation components display variables read-only as context; structural components organise the screen. Every component is declared, not programmed.

42.3 Validation: the form as the first gate

A form does not merely collect input; it *validates* it, and the validation is the first of two gates that protect the process from bad human data.

The form's validation is declared per component: a field is required or optional, a number has a minimum and a maximum, a text field matches a pattern, a date falls in a range. The platform enforces these as the person fills the form — a required field left empty, a number out of range, and the form will not submit. This catches the large class of simple errors — the missing field, the implausible value — at the point of entry, before they ever reach the process.

But the form's validation has a limit, and recognising it is the design point. A form validates each field against *static* rules — this field is required, this number is positive. It cannot easily validate a field against *another* field, or against process state, or against a business rule that depends on the case. That cross-field, stateful, rule-based validation is the job of the *second* gate: the user task listener of [Chapter 39](#), firing on completion, which can see all the submitted variables together and the process state, and can deny the completion if they are jointly inconsistent.

The disciplined division is therefore: the form validates each field at the point of entry — required, typed, in range — and the completion listener validates the submission as a whole — coherent, rule-consistent, valid against the case. The form catches the careless error early and cheaply; the listener catches the subtle error that only the whole picture reveals. A process that relies on the form alone has no defence against an internally inconsistent submission; a process that relies on the listener alone makes the person submit a whole bad form before learning a single field was wrong.

Tip

Use both validation gates and use each for what it is good at. The form's per-field validation — required, typed, ranged — catches the careless error at the point of entry, cheaply and immediately. The completion listener of [Chapter 39](#) catches the cross-field and rule-based error that only the whole submission reveals. Neither alone is enough: the form cannot see the whole picture, and the listener should not be the first thing that

tells a person a required field was blank.

42.4 The form and the embedded application

Chapter 38's Task API treated the case where an institution embeds human tasks in its *own* application rather than using Tasklist. The form has a part in that story worth drawing out.

When a task is worked in Tasklist, Tasklist renders the Camunda Form directly — the form designer's screen is what the person sees. When a task is worked in the institution's own application — the bespoke claims workbench of Chapter 38 — the institution has a choice. It can fetch the form's *schema* through the API and render the Camunda Form inside its own application, keeping one declared form definition that both Tasklist and the workbench honour. Or it can build a fully bespoke screen in the workbench and use only the Task API's variable contract — the named input and output variables — as the boundary.

The first option is usually better, and for the manuscript's recurring reason: one declared artefact, governed once. A Camunda Form fetched and rendered by the workbench is the same form, with the same fields and the same validation, that a reviewer approved and that Tasklist would show — there is no second, divergent screen to govern. A fully bespoke workbench screen is sometimes justified by genuine user-experience needs, but it is a second artefact, and it must be governed, tested, and kept consistent with the process's variable contract as its own discipline.

42.5 Form design for regulated processes

A form in a regulated process is not just a user interface; it is a *data-capture point* whose design directly affects data quality, compliance, and the auditability of the decision that follows. Fields should be validated at input, not corrected downstream. Required fields should be genuinely required, not optional-then-manually-checked. And the form should capture exactly the data the process step needs — no more, no less — because every unnecessary field is a data-protection exposure and every missing field is a decision made on incomplete information.

Worked example: the handling-time cost of a poor form

Problem. A claims process has an adjuster-review user task. In the current form, the adjuster sees the claim's basic fields but must open three other systems — the policy system, the document store, the fraud-model dashboard — to gather the context the decision needs. A redesigned form uses presentation components to assemble all of that context onto the one screen. Measurement shows the current form takes an adjuster 18 minutes per task, of which 7 minutes is gathering context from the other systems; the redesigned form is expected to remove that 7 minutes. The operation handles 140,000 adjuster-review tasks a year, and a loaded adjuster-hour costs \$58. What does the form redesign save a year?

Setup. The saving per task is the context-gathering time the redesign removes. The annual saving is that per-task saving, converted to hours, times the annual task volume, times the hourly cost.

Working. Time saved per task: 7 minutes. In hours:

$$\frac{7}{60} = 0.1167 \text{ hours per task.}$$

Across 140,000 tasks a year:

$$140,000 \times 0.1167 = 16,333 \text{ adjuster-hours saved.}$$

At \$58 an hour:

$$16,333 \times \$58 \approx \$947,000 \text{ saved a year.}$$

Discussion. A form redesign saves the operation nearly a million dollars a year, and the figure is worth pausing on because it is so easily overlooked. The form is, to a casual eye, a cosmetic layer — fields on a screen — and form design is exactly the work a hyperautomation programme is tempted to skimp on, having spent its attention on the orchestration, the decision models, the agents. The worked example shows the skimping is expensive. The 7 minutes per task is not the adjuster making the decision — it is the adjuster doing the platform’s job, manually assembling context the platform already holds and could have placed on the form with a handful of presentation components. Across 140,000 tasks that clerical gathering is 16,000 hours and nearly a million dollars, and it is pure waste: the policy terms, the documents, the model’s score are all data the process already has. The redesign does not make adjusters faster at deciding; it stops making them slow at gathering. And note the saving compounds with a quality effect the arithmetic does not capture: an adjuster deciding from one assembled screen, rather than from memory of three other systems’ tabs, makes fewer errors. The form is not cosmetic. It is, for a high-volume human task, one of the largest and most overlooked sources of cost and quality on the whole platform.

Practical next steps

For one high-volume user task in your institution, measure how much of the handling time is the person *deciding* and how much is the person *gathering context* from other systems. The gathering time is what a better form, using presentation components to assemble that context, can remove — and multiplied by the task volume, it is very often a far larger number than anyone expected.

Chapter in brief

Camunda Forms are the structured screens a user task presents — where the process meets the person. A form is a Modeler artefact of declared components: input components bound to process variables, presentation components that assemble context, and structural components that keep the screen readable. The form validates each field at the point of entry — required, typed, ranged — while the completion listener of [Chapter 39](#) validates the submission as a whole; both gates are needed. An institution embedding tasks in its own application should usually render the same Camunda Form through the API rather than build a divergent screen. For a high-volume task, form design is a major and overlooked source of cost and quality.

Service Levels, Timers, and Calendar Management

43.1 Time as a first-class concern

A financial process runs in time, and against time. A loan decision has a service-level commitment — a regulator’s expectation, a customer promise — to complete within so many days. A claim has a deadline by which the insurer must acknowledge it. A human task has a point past which, undone, it becomes a breach. [Chapter 3](#) introduced the timer event as the BPMN mechanism for time; this chapter treats time as a first-class operational concern — service-level agreements, the timers that track them, and the *calendar* reasoning that real deadlines require.

The chapter has a particular practical thread: real service levels are not counted in raw elapsed hours but in *business* time, and computing business time correctly means knowing about weekends, public holidays, and working hours. That is calendar management, and it is where a platform either integrates with the institution’s real calendar — through Google Calendar or Microsoft Outlook — or quietly computes its deadlines wrongly.

43.2 The SLA and the timer that tracks it

A *service-level agreement* on a process step is a commitment that the step — or the whole process — completes within a stated time. Modelling it has two parts: detecting the breach, and responding to it.

Detection is a *boundary timer event*, in the sense of [Chapter 3](#), attached to the activity the SLA governs. The timer is set to the SLA duration; if the activity is still running when the timer fires, the SLA has been breached, and the timer’s flow is taken. This is the direct, modelled expression of the SLA: it is on the diagram, a reviewer sees it, and it is not a separate monitoring system bolted on but part of the process itself.

Response is whatever the timer’s flow does, and it is a design decision with real range. The mildest response is to *escalate* — raise the task’s priority, notify a supervisor, move it up a queue — while leaving the activity running. A firmer response *interrupts* the activity and routes the case down an expedited or exception path. The right choice depends on the SLA: a soft, internal target warrants escalation; a hard, regulatory deadline may warrant interruption and a different handling path entirely.

A subtlety from [Chapter 3](#) returns here. A boundary timer can be *interrupting* or *non-interrupting*. A non-interrupting timer fires its flow — the escalation — while leaving the original activity running, so the work can still be completed by the person even as the escalation proceeds. An interrupting timer cancels the activity. An SLA escalation is almost always *non-interrupting*: the point is to raise an alarm and add urgency, not to snatch the task away from the person who may be moments from finishing it.

Tip

Model an SLA as a boundary timer on the activity it governs, and choose the timer's nature deliberately. An SLA *escalation* — notify a supervisor, raise priority — should almost always be a *non-interrupting* timer, so the alarm is raised without snatching the task from a person who may be about to complete it. Reserve interrupting timers for the case where the deadline genuinely means the work must stop and the case must take a different path.

The SLA timer in code

The SLA model is two artefacts. The first is the BPMN: a non-interrupting boundary timer on the activity the SLA governs, with its flow leading to the escalation. Listing 43.1 is that timer.

Listing 43.1. A non-interrupting boundary timer expressing a four-hour SLA: if the activity still runs when it fires, the escalation flow is taken — the activity continues.

```
1 <bpmn:boundaryEvent id="slaBreach"
2   attachedToRef="assessClaim"
3   cancelActivity="false">
4   <bpmn:timerEventDefinition>
5     <bpmn:timeDuration>PT4H</bpmn:timeDuration>
6   </bpmn:timerEventDefinition>
7 </bpmn:boundaryEvent>
```

The `cancelActivity="false"` is the non-interrupting choice the tip box insists on, and the `PT4H` is an ISO-8601 duration — four hours. The second artefact is the worker the escalation flow reaches. Listing 43.2 is that worker: it raises the task's priority and notifies a supervisor, while the assessment task — untouched — continues.

Listing 43.2. The escalation worker the SLA timer's flow reaches: it raises priority and notifies a supervisor, while the original activity keeps running.

```
1 @JobWorker(type = "sla-escalate")
2 public void escalate(ActivatedJob job) {
3   String caseRef = job.getVariable("caseReference");
4
5   // raise urgency -- the task itself still runs
6   worklistService.raisePriority(caseRef);
7   notificationService.notifySupervisor(
8     caseRef,
9     "SLA breached: claim assessment over 4h");
10 }
```

The two listings are the SLA as the chapter describes it: detection is the boundary timer in the model, visible to any reviewer; response is the worker, doing exactly what the escalation requires and no more. And because the timer is non-interrupting, the escalation worker and the assessment task run at the same time — the alarm is raised without the work being taken away.

43.3 Business time, not elapsed time

Here is the practical heart of the chapter. An SLA of “two business days” is not a timer of 48 hours. A claim that arrives at 4 p.m. on the Friday before a long weekend, with a two-business-day SLA, is not due at 4 p.m. on Sunday — it is due partway through the following Wednesday, once the intervening Saturday, Sunday, and Monday public holiday are excluded.

A timer set to a raw 48 hours would declare a breach that has not happened, escalate cases that are perfectly on time, and — across a year — generate a steady stream of false SLA alarms that train operators to ignore SLA alarms.

Computing time correctly therefore means computing in *business time*: elapsed time with non-working periods excluded. And business time depends on three things the process engine does not inherently know. It depends on the *working week* — which days are working days, which are weekend. It depends on *public holidays* — which vary by country, by region, and by year, and cannot be hard-coded because next year's are not yet known in this year's code. And it depends, for hour-level SLAs, on *working hours* — a “four business hours” SLA counts only the hours the operation is open.

The institution already has this information. Its working days, its holidays, its office hours live in a *calendar* — typically a corporate Google Calendar or a Microsoft Outlook calendar. The disciplined approach is for the platform to *integrate with that calendar* rather than maintain its own divergent copy of which days are holidays.

43.4 Calendar integration with Google Calendar and Outlook

The integration follows the connector pattern of [Chapter 17](#). The platform reaches the institution's calendar — Google Calendar through the Google Calendar API, Microsoft Outlook through the Microsoft Graph API — by a connector, and uses what it finds there to compute deadlines and to schedule.

Two distinct uses are worth separating.

The first is *business-time computation*. To set a boundary timer for a “two business day” SLA, the platform must convert two business days into an actual target date and time, and that conversion needs the calendar: it reads the working week and the public holidays — the latter often maintained as a dedicated holiday calendar that Google Calendar and Outlook both support subscribing to — and counts forward two working days from the start, skipping the non-working ones. The timer is then set to fire at that computed date, and it fires when the SLA is genuinely breached, not when a naive elapsed-hours count would have wrongly said so.

The second is *scheduling against people's real availability*. A process that must book a human activity — a customer appointment, a review meeting, a callback — can read the relevant person's or team's calendar to find genuine free time, create the appointment as a calendar event, and let the calendar's own reminders and the person's own tooling carry it. This is the calendar as a two-way integration: the platform reads availability and writes appointments, and the institution's people see the process's appointments in the same Outlook or Google Calendar they already live in, rather than in a separate system they must remember to check.

Figure [43.1](#) draws the integration. The point the picture makes is that the timer is set to a *computed* date — the output of a calendar-aware calculation — not to a raw duration.

An SLA timer set to raw elapsed time — 48 hours for “two business days” — will fire wrongly: it will declare breaches over weekends and public holidays that have not occurred. A steady stream of false SLA alarms is worse than useless: it trains operators to ignore SLA alarms, so that when a real breach is escalated, it is dismissed like all the false ones. Compute SLA deadlines in business time, against the institution's real calen-

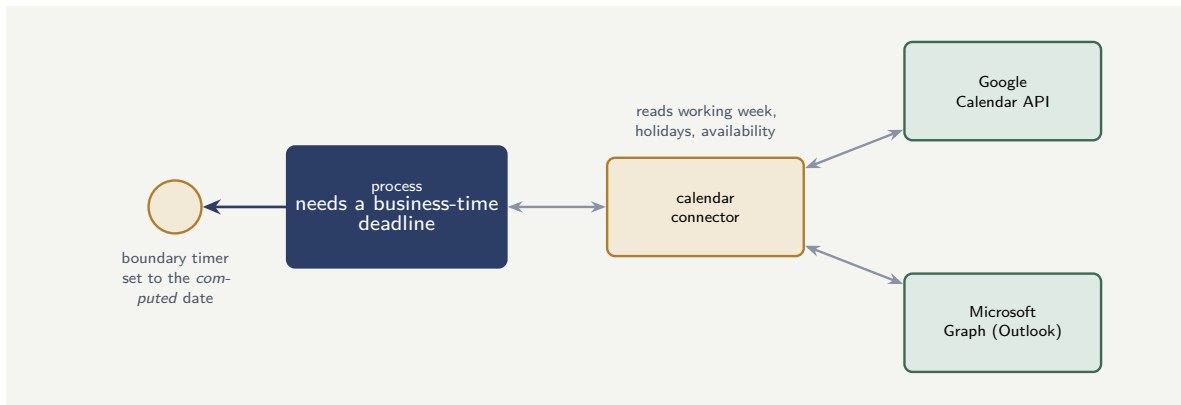


Figure 43.1. Calendar integration for business-time SLAs. The process asks a calendar connector to convert a business-time SLA — “two business days” — into a real target date, using the working week and public holidays from Google Calendar or Outlook. The boundary timer is then set to the computed date, so it fires on a genuine breach, not a naive elapsed-hours count.

dar — the false-alarm rate of a naively-timed SLA quietly destroys the credibility of the SLA mechanism itself.

43.5 When to use this mechanism

This chapter’s mechanism is powerful but not universal. Use it when the process genuinely needs it — when the cross-cutting concern cannot be expressed any other way, when the lifecycle hook is the right place for the logic, when the timer or the error boundary is the right tool for the requirement. Do not use it by default, because every mechanism adds a piece of logic that is not visible on the main process flow, and a process laden with hidden mechanisms is a process that no one can fully understand by reading its diagram. The diagram should tell the story; the mechanisms should handle the edges.

Worked example: the false-breach rate of a naive SLA timer

Problem. A claims-acknowledgement step has a “one business day” SLA. The team has implemented it as a boundary timer of a raw 24 hours. Claims arrive uniformly across the 7 days of the week. The operation’s working week is 5 days; there are, additionally, 9 public holidays in the year. A claim whose 24-hour raw timer spans any non-working day fires a *false* breach — the timer reports a breach when, in business time, the claim is on time. Estimate what fraction of the year’s 250,000 claims get a false breach from the naive timer, and what the business-time computation would do.

Setup. A claim gets a false breach if its 24-hour window crosses into a non-working day. With claims arriving uniformly, the fraction whose next-day window lands on a non-working day is, to a good approximation, the fraction of days that are non-working. We compute the non-working-day fraction and apply it.

Working. Non-working days in the year: 2 weekend days per week across 52 weeks, plus 9 public holidays:

$$(2 \times 52) + 9 = 104 + 9 = 113 \text{ non-working days.}$$

As a fraction of the 365-day year:

$$\frac{113}{365} = 0.3096, \quad \text{about 31\%}.$$

A claim arriving such that its next calendar day is a non-working day gets a false breach; with uniform arrival, that is roughly 31% of claims:

$$250,000 \times 0.31 \approx 77,500 \text{ false breaches a year.}$$

With business-time computation against the calendar, the timer is set to the next *working* day, and the false-breach count is:

0.

Discussion. The naive timer generates on the order of 77,500 false SLA breaches a year — nearly a third of all claims wrongly flagged as late — and the business-time computation generates none. The arithmetic is striking, but the operational consequence is the real lesson. Seventy-seven thousand false breaches is not a minor inaccuracy; it is the complete destruction of the SLA mechanism’s usefulness. An operations team that sees a third of all claims flagged as SLA breaches very quickly learns that the SLA flag means nothing — and once they have learned that, the flag means nothing even on the claims that genuinely breached. The naive timer does not just mismeasure; it trains the organisation to ignore the measurement, which is [Chapter 125](#)’s alert-fatigue failure reached by a different route. And the fix is not exotic: the institution’s working week and its 9 public holidays are already in its corporate calendar, and a calendar connector that reads them lets the timer be set to the genuine next-working-day deadline. The business-time computation is a modest piece of integration; the naive timer it replaces is a slow poisoning of the institution’s trust in its own service-level reporting.

Practical next steps

Check how one SLA in your platform computes its deadline. Is the timer set to a raw duration — 24 hours, 48 hours — or to a date computed in business time against a real calendar? If it is a raw duration, estimate its false-breach rate from your non-working-day fraction. The number is usually large enough to justify the calendar integration on its own — and the integration also buys you genuine appointment scheduling against people’s real availability.

Chapter in brief

A financial process runs against time, and time is a first-class concern. An SLA is modelled as a boundary timer on the activity it governs — usually non-interrupting, so an escalation raises an alarm without snatching the task. The practical heart is that real SLAs count *business* time, not raw elapsed time: “two business days” must exclude weekends and public holidays, which the engine does not inherently know. The institution’s calendar — Google Calendar or Microsoft Outlook, reached by a connector — supplies the working week, the holidays, and people’s availability, so timers fire on genuine breaches and appointments are booked against real free time. A naively-timed SLA generates false breaches that destroy trust in the SLA mechanism itself.

CHAPTER 44

Camunda Patterns

44.1 Why name the patterns

The preceding chapters of this part have, between them, described every mechanism a Camunda platform offers — the engine, the components, routing, allocation, workers, connectors, events, listeners, forms, timers. This chapter and the next stand back from the mechanisms and look at how they *combine*. A *pattern* is a combination of mechanisms that recurs, that solves a problem well, and that — because it recurs — deserves a name, so that a designer can reach for it and a reviewer can recognise it.

Patterns matter for the same reason a shared vocabulary always matters. A team that knows the patterns does not reinvent a solution to a solved problem, and a reviewer who sees a familiar pattern in a model can assess it quickly. The patterns below are not exhaustive — every domain grows its own — but they are the ones that recur across banking and insurance platforms, and a designer who has these in hand has most of what the common cases need.

44.2 The orchestrator-with-workers pattern

The most fundamental pattern is the one the whole manuscript has built toward: a thin process model holds the flow and the state, and the actual work is done by external job workers the model invokes through service tasks. The model orchestrates; the workers execute; the engine, the gateway, and the workers are decoupled exactly as [Chapter 32](#) described.

It is worth naming as a pattern, rather than treating it as simply “how Camunda works,” because naming it makes its discipline explicit: the model contains no business logic, every worker is thin and idempotent, and the seam between them is a typed variable contract. A platform that follows this pattern everywhere has a uniform, governable shape; a platform that sometimes follows it and sometimes smuggles logic into the model has lost the uniformity that makes it reviewable.

44.3 The layered-decision pattern

[Chapter 22](#) introduced it and [Chapter 31](#)’s lending and [Chapter 123](#)’s claims applied it: a decision that combines probabilistic inputs with deterministic policy is modelled in layers. A predictive model produces a score; an agent may produce an assessment; and a DMN decision combines those signals with hard rules to produce the actual, owned, auditable decision. The probabilistic components advise; the deterministic DMN decides.

As a pattern, its value is that it places the boundary of accountability in exactly one spot — the DMN table — and keeps the probabilistic components on the advisory side of it. Whenever a process must make a decision that mixes a model or an agent with policy, this is the pattern, and a model that wires a predictive score straight into a gateway, with no DMN layer between, has skipped it and lost the auditable decision point.

44.4 The contained-agent pattern

[Chapter 23](#) built it: an agent is reached through a connector, given a bounded task and a bounded toolset, its output consumed by a deterministic adjudication step, and a human made accountable for any consequential outcome. [Chapter 33](#)'s agent connector and agentic BPMN are how it is built in Camunda.

Named as a pattern, it is the answer to “how do we use an agent here?” that is always correct: never the bare agent, always the agent inside its four containments. A reviewer who sees an agentic step should look for the pattern — the bounded task, the bounded tools, the adjudication, the accountable human — and treat its absence as the defect [Chapter 23](#) named.

44.5 The long-running-wait pattern

A process that must wait for the outside world — a customer to respond, a counterparty to confirm, a payment to clear — with a deadline, uses the combination [Chapter 3](#) and [Chapter 34](#) described: an event-based gateway, or a message catch event, with a boundary timer for the deadline. The process waits, costing nothing while it does, and proceeds on whichever of the awaited event or the timer comes first.

As a pattern it is the answer to “the process needs to wait here”: not a polling loop, not application-code timers, but the modelled wait-with-deadline. [Chapter 34](#)'s worked example quantified why the polling alternative is the wrong one.

44.6 The synchronous-facade pattern

Sometimes a caller needs a process's outcome immediately — the real-time eligibility check of [Chapter 37](#), called from a screen the customer is waiting at. The pattern is a short, bounded process started with [Chapter 37](#)'s start-and-await variant, presenting a synchronous face — call, get answer — over an engine that is asynchronous underneath. The discipline, from [Chapter 37](#), is that only a genuinely short, bounded process may wear this facade; a long-running process behind a synchronous facade is the antipattern the next chapter names.

44.7 The saga pattern: compensation over a long transaction

One pattern this part has not yet named deserves a full treatment, because it is how a financial process handles an all-or-nothing requirement that spans many steps and external systems.

Consider a process that reserves funds, books a loan, and registers a collateral charge — three steps, three different systems. The institution needs all-or-nothing: either all three succeed, or none stands. But the three are separate external systems with no shared transaction — the technical transaction of [Chapter 40](#) cannot span them. The answer is the *saga*: each step has a defined *compensating action* that undoes it, and if a later step fails, the process runs the compensating actions of the earlier steps in reverse. Reserve funds; book loan; register charge — and if registering the charge fails, the process compensates: un-book the loan, release the funds. The all-or-nothing is achieved not by one transaction but by a modelled sequence of forward steps and, on failure, modelled compensation.

BPMN expresses the saga directly: a *compensation boundary event* on each step names that step's compensating action, and a *compensation throw event* on the failure path triggers the compensations. The saga is the pattern for “these steps must all hold or all be undone, and

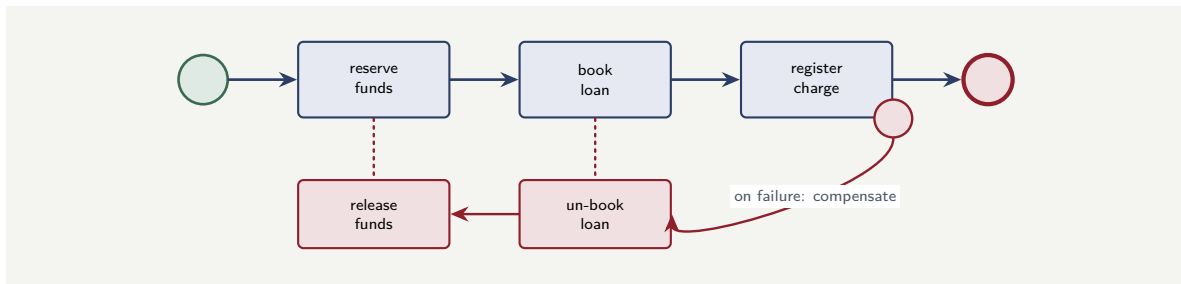


Figure 44.1. The saga pattern. Three forward steps across three systems; each reversible step carries a compensating action. If a later step fails, the process runs the earlier steps’ compensations in reverse — achieving all-or-nothing without a transaction that could span the three systems.

they span systems no single transaction can cover” — which, in banking and insurance, is a common requirement.

Figure 44.1 draws the saga: forward steps along the top, their compensating actions below, and the failure path that triggers compensation in reverse.

44.8 When to use this pattern

Every pattern this part describes is a tool, not a default. The parallel gateway is for genuinely independent work; using it for dependent work produces race conditions. The compensation pattern is for irreversible external effects; using it for in-memory state that can simply be rolled back is overengineering. The correlation pattern is for matching an incoming event to a waiting instance; using it when a direct call would suffice adds complexity for no benefit. The discipline is to reach for each pattern when the work genuinely needs it, and to use the simpler construct when it does not.

Worked example: recognising the pattern a requirement needs

Problem. A design team has four requirements and is about to design a bespoke solution for each. (1) A mortgage process must reserve funds, book the loan, and register a charge — all-or-nothing, across three systems. (2) A process must call a fraud model and route on its score, with the routing auditable. (3) A process must wait for a customer to accept an offer, but not forever. (4) A quote process is called from a website that needs the price back in the same request. For each, name the pattern that already solves it.

Setup. We match each requirement to one of the chapter’s patterns by its defining shape.

Working. (1) All-or-nothing across systems no single transaction covers — the *saga pattern*: forward steps with compensating actions, compensation on failure. (2) A probabilistic input that must yield an auditable decision — the *layered-decision pattern*: the model scores, a DMN table decides. (3) A bounded wait on the outside world — the *long-running-wait pattern*: an event-based gateway with a boundary timer. (4) A synchronous answer over an asynchronous engine — the *synchronous-facade pattern*: a short process started with start-and-await.

Discussion. Four requirements, four patterns already named — and not one needs a bespoke solution invented for it. That is the chapter’s whole argument in miniature. The design team’s instinct, to design each requirement afresh, would produce four solutions that work

but that no reviewer has seen before, that no other team can reuse, and that each contain, somewhere, a subtle error the pattern would have avoided — an unbalanced compensation, a predictive score wired straight to a gateway, a polling loop, a long process behind a synchronous call. The patterns are not academic taxonomy; they are the accumulated, corrected experience of many platforms, and reaching for the named pattern is how a team inherits that experience instead of re-earning it. The skill the worked example exercises — looking at a requirement and seeing which pattern it *is* — is among the most valuable a platform designer develops, because it converts every requirement from a design problem into a recognition problem.

Practical next steps

Take the requirements for a process your team is about to build, and before designing anything, try to name the pattern each requirement needs from this chapter. The requirements you can name are mostly solved already. The requirements you cannot name are the genuinely novel parts of the design — and knowing which those are is itself worth the exercise.

Chapter in brief

A pattern is a recurring combination of mechanisms that solves a problem well and deserves a name, so designers can reach for it and reviewers recognise it. The core Camunda patterns are: orchestrator-with-workers — a thin model, external workers; layered-decision — probabilistic inputs, a DMN decision; contained-agent — an agent inside its four containments; long-running-wait — an event-based gateway with a boundary timer; synchronous-facade — a short process started with start-and-await; and the saga — forward steps with compensating actions, achieving all-or-nothing across systems no single transaction can span. Recognising which pattern a requirement needs converts a design problem into a recognition problem.

CHAPTER 45

Camunda Antipatterns

45.1 Learning from what goes wrong

The previous chapter named the combinations of mechanism that work. This one names the combinations that do not — the *antipatterns*: arrangements that recur, that seem reasonable at the moment they are chosen, and that cause a predictable, serious problem later. [Chapter 122](#) listed the ways a hyperautomation *programme* fails; this chapter is narrower and more technical — the ways a Camunda *platform* is built wrongly, at the level of the model and the code.

An antipattern is worth naming for the same reason a pattern is. A named antipattern is one a designer can recognise and avoid, and one a reviewer can spot and challenge. Each of the antipatterns below has appeared, as a warning, somewhere earlier in the manuscript; gathering them here, named, makes them a checklist.

45.2 The fat process model

The *fat process model* puts business logic into the BPMN model itself — long, complex FEEL expressions on gateways, decision logic spread across many exclusive gateways, calculations buried in mappings — rather than in DMN tables, service tasks, and workers where it belongs. It is tempting because, in the moment, adding one more condition to a gateway is quicker than creating a DMN table.

The problem it causes is the loss of governability that [Chapter 21](#) and [Chapter 29](#) both warned of. Logic in the model is logic that is not in a governable place: not a DMN table risk can check for completeness, not a service a tester can exercise, not a worker that can be versioned independently. The fat process model is unreviewable and untestable in proportion to how much logic it has absorbed. The correction is the discipline of the whole manuscript: the model routes and holds state; logic lives in DMN, services, and workers.

45.3 The chatty orchestration

The *chatty orchestration* decomposes a process into far too many tiny service tasks, each making one trivial call, so that a single business step becomes a dozen engine round-trips. It often comes from misapplying the decomposition advice of [Chapter 21](#) — decomposition is good, so more must be better.

The problem is performance and noise. Every service task is an engine interaction, a job created and activated and completed; a process modelled at too fine a grain spends more effort being orchestrated than doing work, and its Operate view becomes an unreadable sea of tiny steps. The correction is to model service tasks at the grain of a *meaningful unit of work* — one call to a system, one coherent piece of logic — not at the grain of every individual statement.

45.4 The synchronous long-runner

The *synchronous long-runner* starts a long-running process with [Chapter 37](#)'s start-and-await variant, or otherwise holds a caller blocked waiting for a process that is not short and bounded. It is the explicit antipattern [Chapter 37](#) warned of.

The problem is that the caller blocks for minutes, or forever; the connection times out; and the caller is left not knowing whether the process is still running. The correction is [Chapter 37](#)'s discipline: long-running processes are started fire-and-forget, and the caller observes their progress through the query API. Only a genuinely short, bounded process may wear the synchronous facade.

45.5 The non-idempotent worker

The *non-idempotent worker* performs an external effect — moves money, writes to a system of record — without an idempotency key, on the assumption that each job runs exactly once. It is the antipattern [Chapter 32](#), [Chapter 40](#), and [Chapter 41](#) all warned of, from three angles.

The problem is that jobs are delivered *at least once*, not exactly once: a lock expiry, a lost completion, a retry, and the job runs twice. A non-idempotent worker that runs twice doubles its effect — a double payment, a double-booked loan. The correction is absolute and was stated three times already: every worker with an irreversible external effect must carry an idempotency key the downstream system honours.

45.6 The ungoverned agent

The *ungoverned agent* gives an agent a tool with a direct external effect, or omits the deterministic adjudication step, so that the agent's probabilistic output reaches a real consequence unmediated. It is the antipattern [Chapter 23](#) exists to prevent.

The problem is that an agent's confident, plausible errors become real — money moved, cover bound, a claim closed — with no deterministic gate and no accountable human between the reasoning and the effect. The correction is the contained-agent pattern of the previous chapter: the four containments, every time.

45.7 The shared service identity

The *shared service identity* has many callers — many backend systems, or many human users working through an embedded application — act under one shared client or service account, rather than each carrying its own identity. It is the antipattern [Chapter 35](#) and [Chapter 38](#) warned of.

The problem is twofold: the blast radius of a leaked credential is every process the shared identity can touch, and — worse — the audit record cannot attribute an action to the individual caller or person responsible, breaking the accountability link [Part IV's](#) security chapter and [Chapter 17](#) hold to be the platform's deepest obligation. The correction is per-caller identity: each backend its own client, each person's own identity carried through.

45.8 The naive timer

The *naive timer* sets an SLA boundary timer to a raw elapsed duration — 48 hours for “two business days” — ignoring weekends, public holidays, and working hours. It is the antipat-

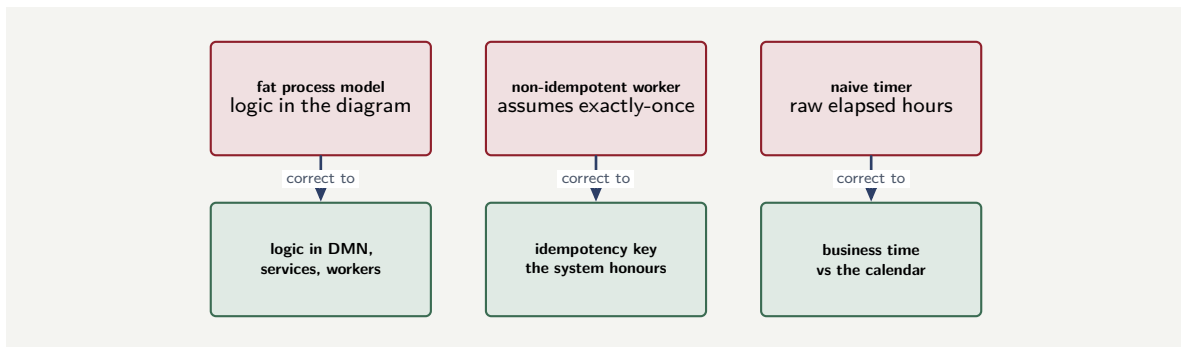


Figure 45.1. Three of the antipatterns and their corrections. Each antipattern is a tempting shortcut with a predictable later cost; each correction is a discipline named earlier in the manuscript. The full set forms a review checklist.

tern [Chapter 43](#) warned of.

The problem is a steady stream of false breaches that trains the organisation to ignore the SLA mechanism entirely. The correction is business-time computation against the institution's real calendar.

Figure 45.1 shows three antipattern-and-correction pairs as a sample of the set; the chapter's value is that the whole set, named, becomes a checklist a reviewer runs against any model.

45.9 When to use this pattern

Every pattern this part describes is a tool, not a default. The parallel gateway is for genuinely independent work; using it for dependent work produces race conditions. The compensation pattern is for irreversible external effects; using it for in-memory state that can simply be rolled back is overengineering. The correlation pattern is for matching an incoming event to a waiting instance; using it when a direct call would suffice adds complexity for no benefit. The discipline is to reach for each pattern when the work genuinely needs it, and to use the simpler construct when it does not.

Worked example: auditing a model against the antipatterns

Problem. A reviewer is given a payments process model to assess before it goes live. A walkthrough reveals: (1) the main routing gateway carries a 40-line nested FEEL expression; (2) the payment-execution worker makes its call to the rail with no idempotency key; (3) the process is started by the website with `awaitCompletion`, and the process includes a user task for manual review of flagged payments; (4) the SLA timer on the acknowledgement step is set to 4 hours flat. Identify the antipattern in each finding and its correction.

Setup. We match each finding to a named antipattern from the chapter and state the correction.

Working. (1) The 40-line FEEL expression on a gateway is the *fat process model*: business logic in the diagram. Correction: move the routing logic into a DMN decision table reached by a business-rule task, where it is checkable and governable. (2) The payment worker with no idempotency key is the *non-idempotent worker*. Correction: the rail call must carry an idempo-

tency key derived from the job, so an at-least-once redelivery cannot double the payment. (3) A process started with `awaitCompletion` that contains a *user task* is the *synchronous long-runner*: the website will block on a process that may wait hours for a human reviewer. Correction: start the process fire-and-forget; if the website needs an immediate response, the process must return a provisional “received” result and the manual review must be asynchronous. (4) The 4-hour flat SLA timer is the *naive timer*. Correction: compute the deadline in business hours against the calendar.

Discussion. Four findings, four named antipatterns — and the worked example shows what the named set is *for*. A reviewer without the vocabulary would still feel that something was wrong with each of these, but would have to reason each one out from first principles, and might miss one — the synchronous long-runner in finding 3 is genuinely easy to overlook, because the `awaitCompletion` and the *user task* are in different parts of the model and only the combination is the defect. A reviewer *with* the named set runs it as a checklist: fat model? chatty? synchronous long-runner? non-idempotent worker? ungoverned agent? shared identity? naive timer? — and each finding announces itself. None of the four findings is exotic; every one is a tempting shortcut that a rushed team takes and a checklist catches. That is the deeper point: antipatterns are not rare pathologies but the *default* mistakes, the things a platform drifts into under delivery pressure unless a reviewer, armed with the named set, is deliberately looking for them. The chapter turns “review the model” from an act of open-ended judgement into the running of a concrete, named checklist — and a checklist catches what judgement, tired on a Friday, lets through.

Practical next steps

Take the seven antipatterns of this chapter — fat process model, chatty orchestration, synchronous long-runner, non-idempotent worker, ungoverned agent, shared service identity, naive timer — and turn them into a literal review checklist. Run it against one model your team considers finished. An antipattern found in review is an hour’s correction; the same antipattern found in production is an incident.

Chapter in brief

An antipattern is a recurring, tempting arrangement that causes a predictable serious problem — the default mistakes a platform drifts into under delivery pressure. The core Camunda antipatterns are: the fat process model — logic in the diagram; the chatty orchestration — too many trivial service tasks; the synchronous long-runner — a caller blocked on a long process; the non-idempotent worker — an external effect assuming exactly-once delivery; the ungoverned agent — probabilistic output reaching a real effect unmediated; the shared service identity — many callers under one identity, breaking accountability; and the naive timer — a raw-duration SLA ignoring the calendar. Named, the set becomes a review checklist, and a checklist catches what tired judgement lets through.

Advanced Task Allocation

46.1 When simple allocation is not enough

Chapter 30 treated task allocation in its essential form: a user task names an assignee, or candidate groups, or candidate users, and a person claims and completes it. That treatment is correct and sufficient for most processes. But a large financial operation — a claims department of several hundred adjusters, a lending operation routing thousands of applications a day — runs into a set of allocation problems that the simple model does not address, and this chapter treats them.

The advanced problems share a character: they are about allocating a *population* of tasks across a *population* of people, over time, well. The simple model answers “who may do this task?” The advanced questions are “which of the eligible people should get it?”, “what happens when the team is overloaded?”, “how is a specialist’s scarce time protected?”, and “how is fairness, across people and across customers, actually achieved?” These are operational questions, and getting them wrong does not break the process — it makes the process slow, unfair, and unable to scale.

46.2 Skills-based routing

The candidate-group model of **Chapter 30** treats every member of a group as interchangeable: any claims adjuster may take any claims task. In a real operation this is too coarse. Adjusters have specialisms — motor, property, liability, complex bodily injury — and a complex bodily injury claim routed to a motor specialist is either done badly or, more likely, returned to the queue having wasted the specialist’s time looking at it.

Skills-based routing refines allocation by matching a task’s required competence to a person’s actual competence. The task carries an attribute — derived from its data, perhaps by a DMN decision — stating what skill it needs; each person has a recorded set of skills; and the allocator routes the task only to people whose skills include the task’s requirement. The candidate group becomes a starting set, narrowed by skill to the people who can genuinely do the work.

The discipline skills-based routing demands is that the skill model be *maintained*. A person’s skills change — they are trained, they move teams, they gain authority — and a skill model that is not kept current routes work on the basis of who could do it a year ago. The skill model is a governed artefact, owned by the operation’s management, not a field set once and forgotten.

46.3 Workload balancing

Even among people who can do a task, the question remains of *which one*. Allocating every task to the first eligible person, or at random, ignores that the eligible people are not equally free: one has two open cases, another has fifteen. *Workload balancing* routes each task to the eligible person with the lightest current load, so that work spreads evenly and no individual

becomes a bottleneck while colleagues sit idle.

Workload is itself a question of definition, and the definition matters. The crudest measure is the *count* of open tasks, but a count treats a five-minute task and a five-hour task as equal. A better measure is the estimated *remaining effort* — the sum, over a person's open tasks, of each task's expected handling time — so that balancing equalises real workload rather than task count. The best operations balance on effort, and estimate each task's effort from its type and data, often with the same kind of predictive model [Chapter 22](#) described.

46.4 Priority, urgency, and the queue discipline

Not all tasks are equal in importance, and an allocator must respect that. A task carries a *priority* — set when the task is created, perhaps by a DMN decision over the case — and the allocator presents or assigns higher-priority tasks first.

Priority must be distinguished from *urgency*, and conflating them is a common error. Priority is an intrinsic property of the task — a vulnerable-customer complaint is intrinsically high-priority. Urgency is a property of *time* — a low-priority task whose SLA deadline is two hours away has become urgent regardless of its priority. A good allocator orders work by a combination: a task's effective ranking rises as its deadline approaches, so that a routine task does not sit forgotten until it breaches merely because higher-priority work kept arriving. The *ageing* of a task — its rising urgency as it waits — is what stops a pure-priority queue from starving its low-priority tasks forever.

46.5 Reserved capacity and the specialist

A scarce specialist — the one adjuster qualified for the most complex claims, the senior underwriter authorised above a high limit — presents an allocation problem of its own. If the specialist is in the candidate group for ordinary work as well as complex work, ordinary work will fill their queue, and when a complex case arrives there is no specialist capacity free for it.

The answer is *reserved capacity*: a portion of the specialist's time is held back from ordinary allocation, available only for the work that genuinely needs them. The allocator, routing ordinary work, treats the specialist as unavailable until the ordinary queue is severe; the specialist's reserved capacity waits for the complex case it exists to serve. This is the allocation counterpart of an idea from [Chapter 30](#)'s team-sizing worked example: a resource run at 100% utilisation has no slack, and a specialist with no reserved capacity is a specialist who is never free when the hard case arrives.

A scarce specialist placed in the ordinary candidate group will have their queue filled by ordinary work, and will not be free when the complex case — the case only they can handle — arrives. Reserve a portion of every specialist's capacity for the work that genuinely requires them. An allocator that optimises only for keeping everyone busy will keep the specialist busy on work anyone could do, which is the most expensive way to waste the platform's scarcest skill.

46.6 Fairness, and the two directions it runs

Fairness in allocation runs in two directions, and an advanced allocator must serve both.

Fairness *to staff* means work is distributed equitably across the team: no individual is sys-

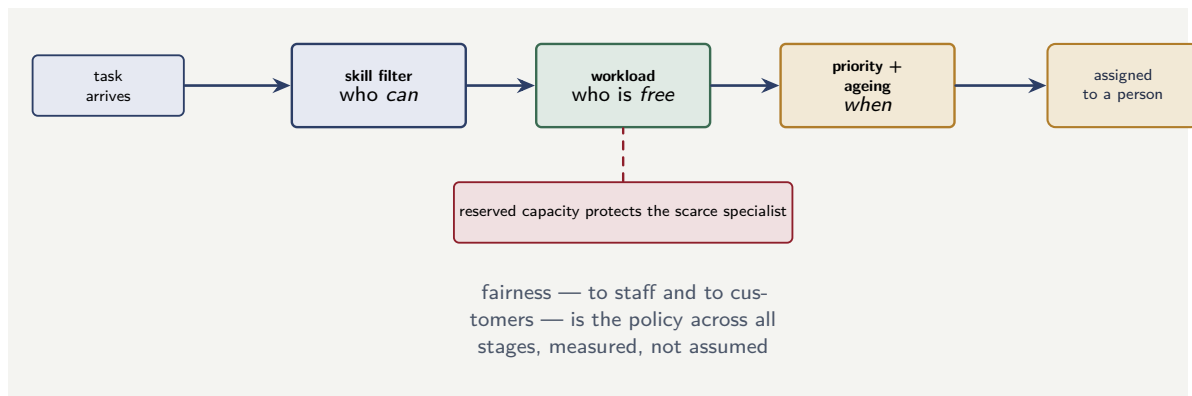


Figure 46.1. Advanced task allocation as a pipeline. A skill filter narrows to who *can* do the task; workload balancing chooses who is *free*; priority and ageing decide *when*. Reserved capacity protects the specialist, and fairness — to staff and customers — is the policy designed across the whole pipeline.

tematically given the hardest cases, the dullest cases, or simply too many cases, while others are spared. Workload balancing serves part of this, but genuine staff fairness also means balancing the *mix* of work — the desirable and undesirable cases — not only the volume.

Fairness *to customers* means a customer's case is handled in a time, and to a standard, that does not depend on the luck of which queue it landed in or how it was routed. A customer whose claim sat for a week because it was routed to an overloaded specialist while a generalist sat idle has been treated unfairly, and — the connection to [Chapter 127's](#) treatment of conduct — inconsistent handling times across customers can be a fairness and conduct concern, not merely an efficiency one.

The two fairnesses can pull against each other — protecting staff from a brutal day may slow some customers' cases — and the allocator's policy is where the institution's chosen balance is made explicit. The point is that fairness is not an accident of the allocation mechanism; it is a property the allocation policy must be deliberately designed to produce, and then measured to confirm.

Figure 46.1 draws the advanced allocator as the pipeline it is: skill, then workload, then priority and ageing, with reserved capacity and fairness as policies across the whole.

46.7 Applying the chapter in practice

The principles this chapter describes are most valuable when applied deliberately and reviewed periodically. A team that applies them once, at initial design, captures their value at that moment; a team that returns to them at each major change, each annual review, and each post-incident analysis captures their value continuously. The platform evolves; the principles hold; the specific choices they inform must evolve with the platform.

Worked example: the cost of count-based balancing

Problem. A claims operation balances workload by *task count*: each new task goes to the adjuster with the fewest open tasks. Two task types share the team: *simple* claims, averaging 20 minutes each, and *complex* claims, averaging 3 hours each. On a given day, adjuster A has been allocated 8 complex claims and adjuster B has been allocated 12 simple claims — so count-based balancing considers B the busier and sends the next task to A. Compare the

two adjusters' actual remaining workload, and determine what effort-based balancing would have done.

Setup. Each adjuster's real workload is the sum of the expected handling times of their open tasks. We compute both, compare them with what the count-based measure concluded, and state what effort-based balancing does.

Working. Adjuster A's workload — 8 complex claims at 3 hours:

$$8 \times 3 = 24 \text{ hours of work.}$$

Adjuster B's workload — 12 simple claims at 20 minutes:

$$12 \times \frac{20}{60} = 12 \times 0.333 = 4 \text{ hours of work.}$$

Count-based balancing saw $A = 8$ tasks and $B = 12$ tasks, judged B the busier, and sent the next task to A. But A has 24 hours of work queued and B has 4. The count-based measure routed the new task to the adjuster who was, in reality, six times more loaded.

Effort-based balancing compares 24 hours against 4 hours, sees B as far freer, and sends the next task to B — the correct decision.

Discussion. Count-based balancing did not merely make a marginally suboptimal choice; it made the *opposite* of the right choice, and the worked example is built to show why the error is systematic rather than unlucky. The flaw is that a task count treats a 20-minute claim and a 3-hour claim as one unit each, and so an adjuster handling genuinely heavy work *looks* idle next to a colleague handling many trivial ones. Over a day, count-based balancing therefore drifts predictably: it steers work toward the adjusters doing complex cases, because their low task counts read as spare capacity, until those adjusters are catastrophically overloaded and their colleagues on simple work are under-occupied. The customers whose complex claims sit in adjuster A's 24-hour queue wait far longer than the platform's reporting — looking at task counts — will suggest. The fix is to balance on *effort*: estimate each task's handling time from its type and data, and balance the sum. The estimate need not be perfect; even a rough effort weight — “complex claims count as nine simple ones” — corrects the systematic error that a raw count guarantees. The lesson generalises beyond balancing: any allocation measure that treats unequal tasks as equal will, given a mix of task sizes, drift in a predictable and damaging direction.

Practical next steps

Check how your platform measures workload for allocation. If it balances on a count of open tasks, and your processes have a mix of short and long tasks, your allocator is systematically overloading the people doing the heavy work. Estimate a rough effort weight per task type and balance on the sum — it is a small change that corrects a systematic, customer-visible error.

Chapter in brief

A large operation runs into allocation problems the simple candidate-group model does not address — allocating a population of tasks across a population of people, well. Skills-based routing matches a task's required competence to a person's actual skills. Workload balancing routes to the freest eligible person — and must balance on estimated *effort*, not a task count, or it systematically overloads those doing heavy work. Priority is intrinsic; urgency is a property of time, and ageing stops a priority

queue starving its low-priority tasks. Reserved capacity protects the scarce specialist. Fairness runs in two directions — to staff and to customers — and is a policy that must be designed across the whole allocation pipeline and then measured.

Starting Camunda Processes in Practice

47.1 From the mechanism to the practice

Chapter 37 treated the *mechanisms* by which a process begins — the explicit start, the start-and-await variant, the triggered starts, the call activity. This chapter is its practical companion: it works through, with concrete examples, how a process is actually started in a running banking or insurance platform, and the operational concerns that surround the start once it is more than a single call in a tutorial.

The distinction is worth making because the mechanism and the practice are different kinds of knowledge. Knowing that a message start event exists is the mechanism; knowing how an institution’s twelve claim-reporting channels are wired to one message start event without coupling them, how the start payload is validated, how a start failure is handled, and how a flood of starts is absorbed — that is the practice, and it is what this chapter supplies.

47.2 Starting a process the explicit way: a worked walk-through

Consider a concrete case: a digital onboarding journey, started when a customer submits an application through the bank’s mobile app. The mobile back-end is the caller; it holds the application data; it must start the onboarding process and obtain the instance key so it can later report progress to the customer.

The practical sequence has four steps, and each carries a discipline. First, the back-end *validates* the application data — before starting anything. A process started with malformed or incomplete data is a process that will fail at its first real step, having consumed an instance, an entry in Operate, and an audit record; validating at the door is cheaper than cleaning up after. Second, the back-end *starts* the process, naming the definition and supplying the validated data as initial variables. Third, it *records* the returned instance key against the customer’s application in its own database — the key is how every later interaction with this instance will be addressed. Fourth, it *handles the failure case*: a start call can fail — the cluster briefly unreachable, the caller’s authorization wrong — and the back-end must have a defined response, not an unhandled exception.

Listing 47.1. Starting an onboarding process in practice. The data is validated first; the instance key is captured and recorded; the start failure has a defined handler.

```

1 public OnboardingHandle startOnboarding(Application app) {
2     // 1. validate BEFORE starting -- a bad start is wasted
3     validator.check(app);           // throws on invalid input
4
5     try {
6         // 2. start, supplying validated initial variables
7         var event = camunda.newCreateInstanceCommand()

```

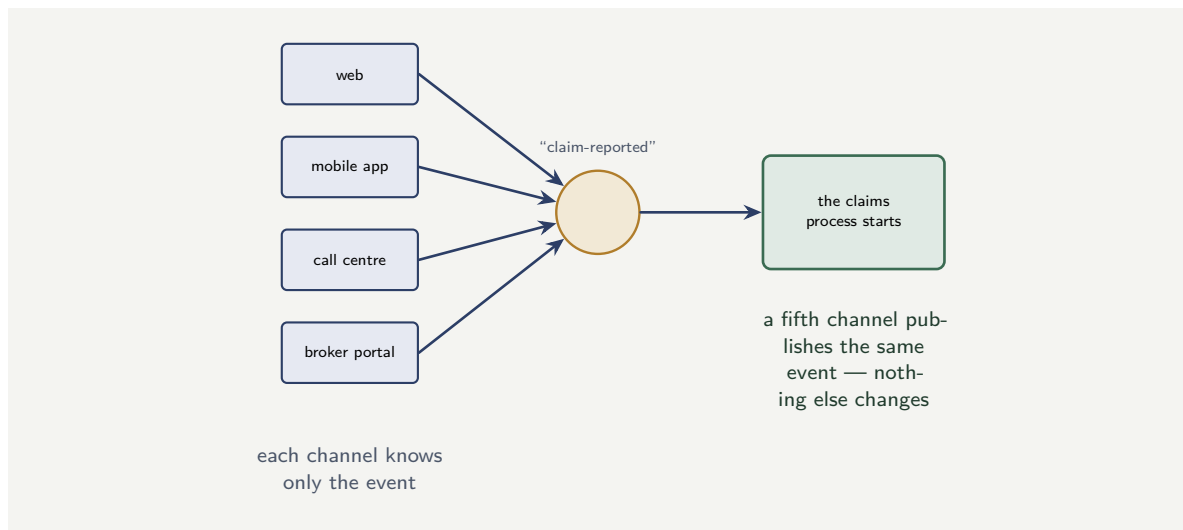


Figure 47.1. Starting one process from many channels. Rather than each channel calling the create-instance API — which couples all of them to the process — every channel publishes the same business event, and a message start event begins the process. The channels are decoupled from the process and from each other.

```

8      .bpmnProcessId("customer-onboarding")
9      .latestVersion()
10     .variables(Map.of(
11         "applicationRef", app.reference(),
12         "channel",        "mobile",
13         "customerData",   app.toProcessPayload())
14     .send().join();
15
16     // 3. record the key -- the address for all later calls
17     long key = event.getProcessInstanceKey();
18     applicationStore.recordProcessKey(app.reference(), key);
19     return new OnboardingHandle(app.reference(), key);
20
21 } catch (ClientException startFailed) {
22     // 4. a defined failure response -- not an unhandled throw
23     applicationStore.markStartFailed(app.reference());
24     throw new OnboardingUnavailableException(startFailed);
25 }
26 }

```

47.3 Starting from many channels without coupling

A common practical problem: a process must be startable from several channels. A claim can be reported through the web, the mobile app, a call-centre system, or a broker portal — four callers, one process.

The naive practice has all four channels call the Process API’s create-instance operation directly. It works, but it couples all four channels to the Process API, to the exact process definition ID, and to the shape of the start payload — and a change to any of those means changing four systems. The disciplined practice, which [Chapter 37](#) and [Chapter 34](#) both pointed to, is a *message start event*: each channel publishes a “claim-reported” event, through an inbound connector or the event backbone, and the message starts the process. The channels know only the event; they know nothing of the process. A fifth channel is added by having it publish the same event — no existing system changes.

Figure 47.1 draws the multi-channel start: four channels, one shared event, one process, no coupling.

Tip

When a process must be startable from several channels, resist having every channel call the create-instance API directly — that couples each channel to the process definition and the start payload. Use a message start event: each channel publishes a business event, and the message starts the process. The channels are decoupled from the process and from each other, and a new channel is added without touching any existing one.

47.4 Absorbing a flood of starts

A practical concern the mechanism chapter did not dwell on: what happens when starts arrive faster than the platform can comfortably handle them. A marketing campaign drives a surge of applications; a market event triggers a spike of trades; a storm produces a flood of claims. The start rate spikes far above normal.

The platform's behaviour under a start flood is, in fact, sound by design: Chapter 27's Zeebe engine decouples the *creation* of work from the *performing* of it, so a flood of starts creates a flood of instances that then wait, durably, for workers to process them at the workers' sustainable rate. The instances are not lost; they queue. But two practical disciplines make the difference between a graceful absorption and a poor customer experience. The starting channel should apply *back-pressure sensibly* — if the start API is momentarily slow under the surge, the channel should queue and retry, not fail the customer outright. And the process should be modelled so that a long wait for a worker is *visible to the customer* as a defined "received, in progress" state, not as silence. A flood of starts is survivable; a flood of starts that leaves customers staring at a spinner is a reputational event.

47.5 Applying the chapter in practice

The principles this chapter describes are most valuable when applied deliberately and reviewed periodically. A team that applies them once, at initial design, captures their value at that moment; a team that returns to them at each major change, each annual review, and each post-incident analysis captures their value continuously. The platform evolves; the principles hold; the specific choices they inform must evolve with the platform.

Worked example: validating before the start

Problem. An onboarding platform starts a process for every application, then validates the application data as the process's first service task. Of 400,000 applications a year, 6% have data so malformed that the process fails immediately at that first task and must be cleaned up: the instance cancelled, the Operate incident resolved, the audit record annotated — an average of 4 minutes of operations effort per failed instance, at a loaded \$62 an operations-hour. A proposed change validates the data in the calling channel *before* the process is started, so a malformed application never becomes an instance. What does validating-first save, and what is the deeper benefit?

Setup. The cost avoided is the cleanup effort on the malformed applications that, under validate-first, never become instances. We compute the failed-instance count, the cleanup

effort, and its cost.

Working. Malformed applications a year:

$$400,000 \times 0.06 = 24,000.$$

Under validate-after, each becomes an instance that fails and needs 4 minutes of cleanup:

$$24,000 \times 4 \text{ min} = 96,000 \text{ min} = 1,600 \text{ hours}.$$

At \$62 an hour:

$$1,600 \times \$62 = \$99,200 \text{ a year in cleanup effort}.$$

Under validate-first, the 24,000 malformed applications are rejected at the channel and never become instances — the cleanup cost is \$0.

Discussion. Validating before the start saves just under \$100,000 a year in cleanup effort, but the cleanup bill is not the deepest point. Consider what each malformed application that becomes an instance actually does. It consumes an instance key; it appears in Operate; it raises an incident that an operator must triage; it generates audit records; and — because it failed at its first step — it has done all of that to accomplish nothing. Twenty-four thousand such instances a year are 24,000 entries of pure noise in the platform’s operational surface: incidents that are not really incidents, instances that never had a chance, audit records of non-events. That noise has the same corrosive effect this manuscript has noted before — it buries the genuine incidents an operator must not miss. Validating at the channel, before the start, means a malformed application is rejected as what it is — a bad input — at the door, with a clear message to whoever submitted it, rather than being admitted into the platform to fail expensively and noisily inside. The discipline is general: the start of a process is a threshold, and a threshold is the cheapest place to turn away what should not come in. A platform that validates after starting has put its filter on the wrong side of the door.

Practical next steps

For one process your platform starts programmatically, find where its input data is validated — before the start, in the calling channel, or after, as the process’s first step. If it is after, estimate the rate of malformed input and the cleanup cost, and consider moving the validation to the channel. The start is a threshold; validate at the threshold, not inside.

Chapter in brief

This chapter is the practical companion to [Chapter 37](#)’s mechanisms of starting a process. In practice, an explicit start is a sequence: validate the input first, start the process, record the returned instance key, and handle the start failure with a defined response. A process startable from several channels should be started by a message start event each channel publishes — not by every channel calling the create-instance API, which couples them all. A flood of starts is absorbed by design, because Zeebe decouples work creation from work performance — but the channel should apply sensible back-pressure and the customer should see a defined “in progress” state. Validate before the start: the start is a threshold, and a threshold is the cheapest place to turn bad input away.

CHAPTER 48

Completing, Searching for, and Reassigning Tasks

48.0.1 The completion is the point of the task

A user task exists to be completed. Everything [Chapter 30](#) said about allocation, everything [Chapter 42](#) said about forms, is in service of the moment a person finishes the task and the process moves on. This chapter treats that moment in practical detail: how a task is completed, what the completion carries, what can go wrong, and the disciplines that keep the completion correct.

The completion matters more than its brevity suggests, because it is the point at which a human decision *enters the process as data*. Before the completion, the person's judgement is in their head; after it, it is a set of process variables that gateways will route on, that workers will act on, that the audit record will preserve. The completion is the conversion of human judgement into process state, and a conversion done carelessly corrupts everything downstream of it.

48.0.2 What a completion carries

A task completion carries *output variables* — the structured result of the person's work. [Chapter 38](#)'s Task API showed the mechanism; this section treats the discipline.

The output variables a completion carries should be *exactly* those the process needs, named as the model expects, and typed correctly. A claims adjuster completing a review task submits, say, a `claimDecision` of `APPROVED` or `DECLINED`, an `approvedAmount` if approved, and an `adjusterNote`. Those three variables are the decision; they are what the next gateway routes on and what the audit record preserves. A completion that carries more — the adjuster's scratch calculations, UI state, half-considered intermediate values — pollutes the process state with data no step needs and the audit record with noise. A completion that carries less, or carries it mis-typed, leaves the downstream gateway unable to route.

The output variables are, in the language of [Chapter 3](#), the task's half of the data-flow contract. The user task declares what variables it produces; the completion must deliver exactly those.

Listing 48.1. Completing a user task through the Task API. The completion carries exactly the named, typed output variables the process needs — the structured result of the person's decision.

```
1 curl --request POST \  
2   'https://<CLUSTER>/v2/user-tasks/${userTaskKey}/completion' \  
3   --header "Authorization: Bearer ${ACCESS_TOKEN}" \  
4   --header 'Content-Type: application/json' \  
5   --data '{  
6     "variables": {  
7       "claimDecision": "APPROVED",  
8       "approvedAmount": 4200,  
9       "adjusterNote": "Damage consistent with the reported incident."  
10    }  
11  }'
```

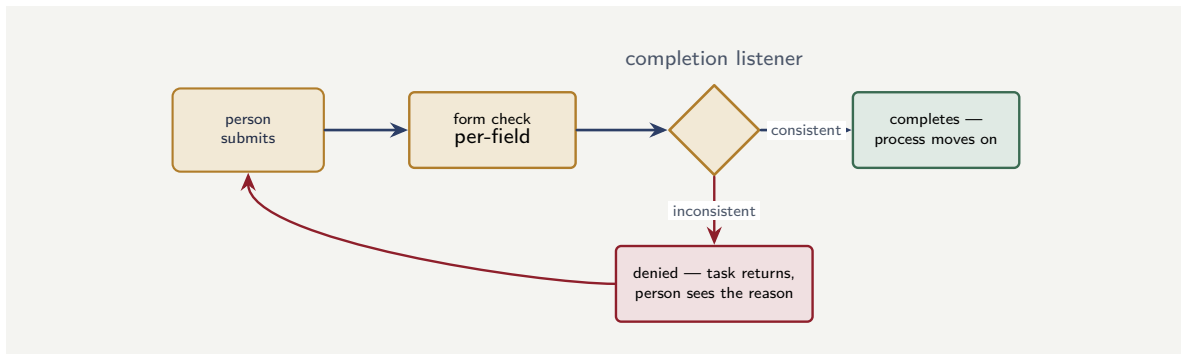


Figure 48.1. Completion as a guarded transition. The form’s per-field check catches careless errors at entry; the completion listener then validates the submission as a whole and either lets the task complete or denies it — returning the task to the person with the reason. An application driving the completion must handle the denial.

48.0.3 The completion as a guarded transition

A completion is not an unconditional act. [Chapter 39](#) established that a user task listener on the completing event can *validate* the submitted variables and *deny* the completion if they are inconsistent. The practical consequence, for anyone building task completion into an application, is that a completion can be *refused*, and the application must handle that.

When a completion is denied — the listener found the submission inconsistent, an approved claim with no amount, a decision outside the permitted set — the task does not complete; it returns to its prior state, and the person must correct and resubmit. An application that calls the completion API and assumes it always succeeds will, on a denial, leave the person believing their work is done when it is not. The practical discipline is that the application treats the completion call as it would any guarded operation: it checks the outcome, and on a denial it shows the person the reason and keeps them in the task.

This is the two-gate validation of [Chapter 42](#) seen from the completion side: the form’s per-field validation should catch most errors before the person ever submits, and the completion listener catches the cross-field and rule-based errors — but either way, the application driving the completion must be ready for the completion to be refused.

Figure 48.1 draws the completion’s two gates and the path a denied completion takes back to the person.

A task completion can be denied — by a completion listener that found the submitted variables inconsistent. An application that calls the completion API and assumes success will, on a denial, tell the person their task is done when it is not, and the work is silently lost. Always check the completion’s outcome; on a denial, show the person why and keep them in the task. A completion is a guarded transition, not an unconditional one.

48.0.4 Who completed it: the completion and accountability

A practical point that [Chapter 38](#) established and that bears restating in the completion context: the completion must be attributed to the *individual person* who made the decision.

When a task is completed through Tasklist, this is automatic — the person is signed in, and Tasklist completes the task as them. When a task is completed through an institution’s

own application, it depends on the application carrying the person's identity, in the token, through to the completion call. [Chapter 38](#)'s worked example quantified what is lost when it does not: a completion attributed to a shared service account is a decision no individual is recorded as accountable for, and in a regulated process that is a finding waiting to be made. The completion is the moment accountability is fixed; the individual's identity must be on it.

Worked example: the cost of an over-stuffed completion

Problem. A claims workbench completes adjuster-review tasks. The adjuster's decision is three variables — decision, amount, note — totalling about 200 bytes. But the workbench, for convenience, completes each task with the *entire* working state of its review screen: the decision, plus all the context it loaded for the adjuster to see, plus UI state — about 40 kilobytes per completion. The claims process runs 600,000 review tasks a year. The context and UI state are never read by any later step. Process variables set by a completion are persisted in the instance and preserved in the audit record. Quantify the unnecessary data, and explain the harm.

Setup. The unnecessary data per completion is the over-stuffed payload minus the 200 bytes genuinely needed. We compute the annual volume and consider the harms.

Working. Unnecessary data per completion:

$$40 \text{ KB} - 0.2 \text{ KB} \approx 39.8 \text{ KB}.$$

Across 600,000 completions a year:

$$600,000 \times 39.8 \text{ KB} \approx 23,900,000 \text{ KB} \approx 23.9 \text{ GB a year}$$

of review-screen context and UI state written into process instances and the audit record, that no step ever reads.

Discussion. Twenty-four gigabytes a year of unread data in the process state and the audit record is the measurable harm, and like the over-returning worker of [Chapter 93](#), the volume is only part of it. Consider what the over-stuffed completion does to the audit record in particular. The audit record exists so that, later, the institution can say exactly what a decision was and who made it — it must be precise and trustworthy. A completion that writes 40 kilobytes of UI state and loaded context into the record buries the three variables that *are* the decision under a mass of irrelevant data, and makes the record both harder to search and larger to retain. The adjuster's actual decision — approved, \$4,200, this note — is in there, but it is in there surrounded by the scroll position of a panel and the contents of a context pane. The discipline is the one this chapter has pressed: a completion carries *exactly* the named output variables the decision consists of, and nothing else. The context the adjuster saw was input to their decision; it does not belong in the output of it. The fix is for the workbench to send, on completion, only the three decision variables — which is also, not coincidentally, all the Task API and the process model ever asked for.

48.1 Searching for Tasks

48.1.1 Finding the work

Before a person can complete a task, the task must be *found* — surfaced onto a worklist, presented to the right person, in the right order. Task search is the operation that does this, and it is worth a chapter because an institution that embeds human work in its own applications,

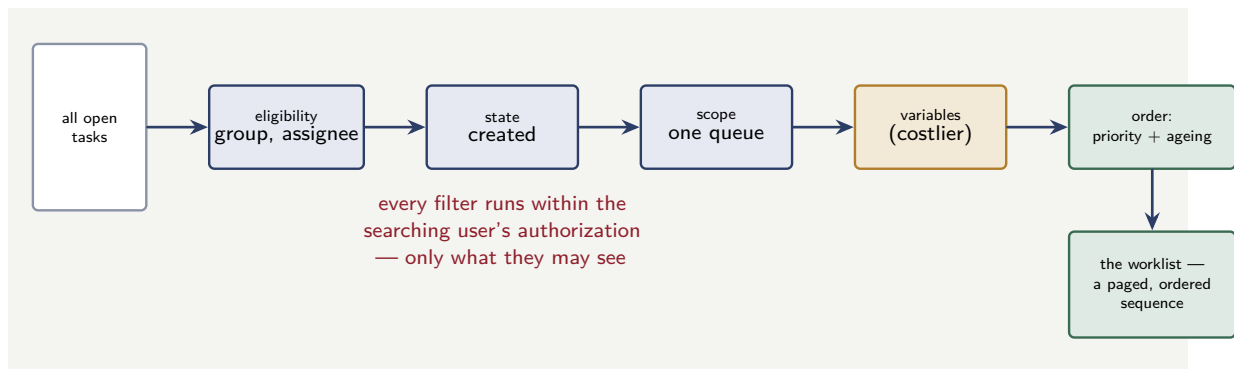


Figure 48.2. The worklist search as a filtering pipeline. Eligibility, state, and scope narrow the open tasks cheaply on metadata; a variable filter narrows further but at greater cost. The result is ordered by priority with ageing into a paged sequence — and the whole search runs inside the user’s authorization.

as [Chapter 38](#) described, builds its worklists on task search, and a worklist built on a poor search is a worklist that shows people the wrong work, in the wrong order, slowly.

Task search is the query side of the Task API. [Chapter 38](#) introduced the search operation; this chapter treats it in the depth a worklist actually needs: what to filter on, how to order, how to page, and the disciplines that keep a search both correct and fast.

48.1.2 What a worklist search filters on

A worklist answers a specific question: *what should this person work on now?* The task search that builds it filters on several dimensions, and choosing them well is the design of the worklist.

The first filter is *eligibility* — which tasks this person may work at all. That is the candidate-group and assignee filtering of [Chapter 30](#): a claims adjuster’s worklist searches for tasks whose candidate group includes the adjuster’s groups, plus tasks already assigned to them. The second filter is *state*: a worklist usually wants tasks that are `CREATED` — available or assigned and not yet completed. The third filter is *scope*: a worklist may be narrowed to one process, one product, one queue — the motor-claims worklist, the high-value-payments worklist.

A search may also filter on *task data* — the process variables a task carries. A worklist of “claims over \$50,000 awaiting review” filters on a variable. This is powerful and is how specialised worklists are built, but it carries a performance discipline the next section treats.

Figure 48.2 draws the search as the filtering pipeline it is, from all open tasks down to the ordered, paged worklist.

48.1.3 Ordering: the worklist is not a set but a sequence

A worklist is not just a set of tasks; it is an *ordered* sequence, because the person works it top-down, and the order is therefore a real decision about which work is done first.

The ordering should reflect the allocation thinking of [Chapter 46](#). A worklist ordered purely by task age — oldest first — ignores priority. A worklist ordered purely by priority starves its low-priority tasks, as [Chapter 46](#) warned. The practical worklist orders by a combination: priority first, but with the ageing that makes a low-priority task rise as its deadline approaches, so the sequence the person sees is genuinely the sequence they should work. The task search’s sort criteria are where this ordering is expressed, and a worklist whose search re-

turns tasks in an arbitrary or purely chronological order has left the most important question — what first? — to chance.

48.1.4 Paging and the performance of search

A task search must *page*, for exactly the reasons [Chapter 36](#)’s process query had to: a candidate group may have thousands of open tasks, and a search that tried to return them all in one response would be slow, memory-hungry, and fragile. The worklist requests a page — the first twenty-five tasks, say — and requests more only as the person scrolls or pages.

Two performance disciplines matter. The search should be ordered by a *stable* key as well as by the worklist’s display order, so that tasks do not shift between pages as the underlying data changes. And a search that filters on task *variables* — the “claims over \$50,000” worklist — should be understood to be more expensive than one filtering only on candidate group and state, because variable filtering examines task data rather than task metadata. A worklist that filters on many variables, refreshed frequently, for many concurrent users, is a real load on the platform, and its design should be deliberate: filter on what genuinely narrows the worklist, not on everything that might.

Tip

A worklist’s task search should page, be ordered by both its display order and a stable key, and filter deliberately. Filtering on candidate group and state is cheap — it examines task metadata. Filtering on task variables is more expensive — it examines task data. A worklist that filters on many variables, refreshes every few seconds, and serves hundreds of users is a substantial load; design the filter to narrow the list genuinely, and refresh no more often than the work changes.

48.1.5 The search and authorization

A practical discipline easy to overlook: task search is subject to the authorization of [Chapter 35](#) and [Chapter 38](#). A search does not return every task matching its filter; it returns every task matching its filter *that the searching identity is authorized to see*.

This matters for worklist correctness. An institution might be tempted to build a worklist by searching broadly and then filtering the results in the application for what the user should see. That is both slower — it fetches tasks only to discard them — and dangerous, because it puts the who-may-see-what decision in application code rather than in the platform’s authorization model. The disciplined practice is to let the search itself be authorized: the searching identity is the user’s identity, the platform returns only what that user may see, and the application does not re-implement, possibly wrongly, an access-control decision the platform already makes correctly.

48.1.6 The stale worklist and the double-claim

A worklist is a *snapshot* — the result of a search run at a moment — and the work it shows changes continuously as colleagues claim and complete tasks. Between the moment a worklist is fetched and the moment a person acts on it, the world has moved, and a worklist design that ignores this produces a specific, irritating, and avoidable failure: the double-claim.

The scenario is ordinary. Two adjusters, looking at worklists fetched seconds apart, both see the same unclaimed high-priority task at the top. Both reach for it. One claims it; the other’s claim arrives a moment later, against a task that is no longer available. If the worklist

application has not anticipated this, the second adjuster gets an error they did not expect, or — worse — the application’s stale view lets them open and begin work on a task that is already someone else’s, and the duplication is discovered only when both submit.

The discipline has two parts. The first is that the *claim* is the authoritative moment, not the *display*: a worklist showing a task is not a reservation of it, and the application must treat a claim as an operation that can fail because someone got there first — and handle that failure gracefully, refreshing the worklist and letting the person pick again, rather than presenting it as an error. The second is that the worklist’s refresh cadence should be fast enough that the window for a double-claim is small, but — as the worked example below explores — not so fast that the refresh load becomes its own problem. The worklist is a snapshot; the application’s job is to make acting on a slightly-stale snapshot safe, not to pretend the snapshot is always current.

Worked example: the refresh rate of a heavy worklist

Problem. A claims department’s worklist application runs a task search for each of 300 concurrent adjusters. Each search filters on candidate group, state, and three task variables, and the application refreshes every adjuster’s worklist every 5 seconds. Each variable-filtered search costs the platform, on average, 45 milliseconds of query work. A proposed change refreshes every 30 seconds instead, and uses a lighter search — candidate group and state only, with variable-based narrowing done once and cached — costing 12 milliseconds. Compare the query load the two designs place on the platform.

Setup. The query load is the number of searches per second times the cost per search. We compute it for both designs.

Working. *Current design.* Searches per second across 300 adjusters refreshing every 5 seconds:

$$\frac{300}{5} = 60 \text{ searches per second.}$$

At 45 ms of query work each, the query load is:

$$60 \times 45 \text{ ms} = 2,700 \text{ ms of query work per second}$$

— that is, 2.7 seconds of query work demanded for every second of real time, meaning the search load alone needs roughly 3 concurrent query workers just to keep up.

Proposed design. Searches per second, refreshing every 30 seconds:

$$\frac{300}{30} = 10 \text{ searches per second.}$$

At 12 ms each:

$$10 \times 12 \text{ ms} = 120 \text{ ms of query work per second}$$

— a fortieth of the same demand.

Discussion. The proposed design reduces the worklist’s query load on the platform by a factor of about 22 — from 2,700 to 120 milliseconds of query work per second — and the worked example’s lesson is about a cost that is easy to create without noticing. The current design’s two extravagances each seemed reasonable in isolation. A 5-second refresh feels responsive — why not keep the worklist fresh? Filtering on three variables feels precise — why not let the search do the narrowing? But the costs multiply: 300 users, times a 5-second refresh, times a heavy variable-filtered search, is 2.7 seconds of query work per second of real time, a continuous load that scales with every adjuster added. The proposed design questions

both extravagances. A worklist does not need to refresh every 5 seconds — claims tasks do not arrive that fast, and a 30-second refresh is still entirely responsive to the pace at which the work actually changes. And the variable-based narrowing need not be redone on every refresh — it can be done once and cached, leaving the frequent refresh to the cheap metadata-only search. The lesson generalises: the load a worklist places on the platform is the product of the user count, the refresh rate, and the search cost, and a design that is casual about any of the three creates a continuous, scaling load that no single decision looks responsible for. Refresh as often as the work changes — not as often as the screen could.

48.2 Reassigning Tasks

48.2.1 Why allocation is not final

[Chapter 30](#) and [Chapter 46](#) treated how a task is allocated to a person. But an allocation, once made, is not permanent, and the practical operation of a financial back-office depends on the ability to *reassign* a task — to move it from the person it is with to someone else, or back to a group. This chapter treats reassignment: why it happens, how it is done, and the disciplines — particularly the accountability disciplines — that govern it.

Reassignment is not an edge case. It is a constant, ordinary part of running an operation. People go on leave, fall ill, reach the end of a shift. Workloads become unbalanced and a supervisor rebalances them. A task turns out to need a specialist the original assignee is not. An escalation, fired by the SLA timer of [Chapter 43](#), moves a task to someone more senior. All of these are reassignment, and a platform that cannot reassign cleanly is a platform whose human work seizes up the first time someone is unexpectedly absent.

48.2.2 The forms of reassignment

Reassignment takes several forms, and distinguishing them is the start of treating it well.

A task may be *reassigned from one person to another* — a supervisor moves a specific case from an absent adjuster to a present one. A task may be *returned to its candidate group* — unassigned from the person who held it, becoming available again for anyone in the group to claim; this is the *unclaim* of [Chapter 30](#), and it is the right move when the question is not “who specifically” but simply “not this person.” A task may be *reassigned in bulk* — when a person leaves or a shift ends, all their open tasks are moved at once. And a task may be reassigned *automatically* — by the escalation logic of [Chapter 27](#), or by a user task listener of [Chapter 39](#) firing on a timeout.

Each form is, mechanically, an operation on the task’s assignee through the Task API: clearing it, setting it to a new person, or returning the task to its group. [Chapter 38](#)’s assign and unassign operations are the mechanism; the forms above are the practical uses.

Figure [48.3](#) draws the four forms of reassignment converging on the task, with the audit obligation that governs all of them.

Listing 48.2. Reassigning a task through the Task API. The first call moves a task to a specific new assignee; the second clears the assignee, returning the task to its candidate group.

```
1 # reassign a task to a specific new person
2 curl --request POST \
3   'https://<CLUSTER>/v2/user-tasks/${userTaskKey}/assignment' \
4   --header "Authorization: Bearer ${ACCESS_TOKEN}" \
5   --header 'Content-Type: application/json' \
6   --data '{ "assignee": "adjuster.patel" }'
7
```

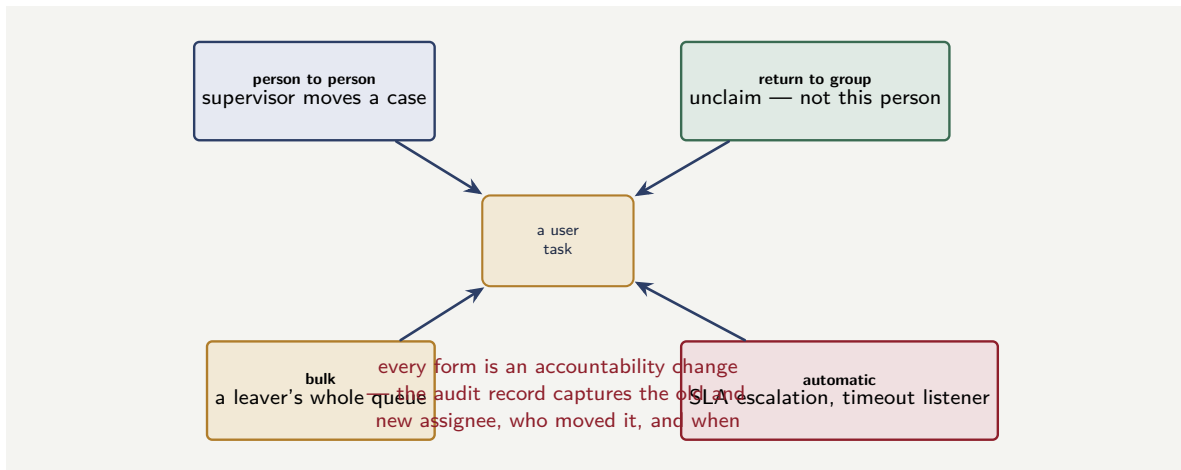


Figure 48.3. The four forms of reassignment. A task may be moved person-to-person, returned to its candidate group, reassigned in bulk when someone leaves, or reassigned automatically by escalation or a timeout listener. Each is a change in who is accountable, and each must be captured in the audit record.

```

8 # return a task to its candidate group (unassign)
9 curl --request DELETE \
10 'https://<CLUSTER>/v2/user-tasks/${userTaskKey}/assignee' \
11 --header "Authorization: Bearer ${ACCESS_TOKEN}"

```

48.2.3 Reassignment and accountability

Here is the discipline that makes reassignment more than a mechanical operation. Every reassignment is an event the audit record must capture — because reassignment touches the accountability that this manuscript has held, from Part IV onward, to be the platform’s deepest obligation.

Consider why. The audit record must be able to answer not only “who completed this task” but, where it matters, “who held it, and when, and who moved it.” If a task sat too long, the record should show whether it sat with one person or was passed between several. If a four-eyes control is in play, the record must show that reassignment did not route the checker task to the very person who did the maker task. If a decision is later questioned, the chain of custody of the task — who had it, who reassigned it, on whose authority — is part of the answer. A reassignment that the audit record does not capture is a gap in that chain.

So a reassignment is recorded: the task, the old assignee, the new assignee, the person or rule that performed the reassignment, and the time. A platform where tasks can be moved silently — with no record of the movement — has a worklist that works but an audit trail that cannot account for how work flowed, and [Chapter 123](#)’s regulator will ask exactly that.

Every reassignment must be captured in the audit record — the task, the old and new assignee, who performed the reassignment, and when. Reassignment is a change in who is accountable for a piece of work, and an accountability change that leaves no trace is a gap in the chain of custody. A platform that lets tasks be moved silently has a worklist that functions and an audit trail that cannot answer who held the work, when, and on whose authority — which is among the first things an investigation asks.

48.2.4 Reassignment and the four-eyes constraint

Reassignment interacts with the four-eyes principle of [Chapter 30](#) in a way that needs explicit care, because reassignment is exactly how a four-eyes control can be accidentally defeated.

Recall the control: the person who completes a checker task must not be the person who completed the corresponding maker task. The allocation of the checker task is set up to exclude the maker. But *reassignment* can override that allocation — a supervisor, rebalancing workload, reassigns the checker task, and if nothing prevents it, reassigns it to the very person who did the maker task. The four-eyes control, correctly modelled, is silently broken by an ordinary reassignment.

The disciplined platform enforces the four-eyes constraint on reassignment as well as on initial allocation: the reassignment operation itself checks that the new assignee of a checker task is not the maker, and refuses the reassignment if it is — the structural enforcement of [Chapter 39](#)'s listeners, applied to the reassignment event. A platform that enforces four-eyes only at initial allocation, and not on reassignment, has a control with a hole in it exactly the size of one supervisor's rebalancing.

48.2.5 Reassignment must respect the destination's load

A reassignment moves a task *to* someone, and that someone has a workload of their own — a fact a careless reassignment ignores, with consequences worth naming.

Consider the ordinary case: a supervisor, seeing an absent adjuster's queue, reassigns its tasks to whichever colleagues come to mind. The reassignment succeeds; the tasks move. But if the supervisor moves them without regard to those colleagues' existing load, the supervisor has not solved an overload — they have relocated it, and possibly concentrated it, dropping fifteen tasks onto a colleague who already held twenty. The absent adjuster's queue is now empty and a present adjuster's queue is now unmanageable, and the second problem is worse than the first because it is now affecting someone who is actually at work.

The discipline is that reassignment is itself an act of allocation, and so the allocation thinking of [Chapter 46](#) applies to it in full. A reassignment — particularly a bulk one — should route the moved tasks the way any allocation should: to the people with genuine capacity, balanced on *effort* rather than count, respecting skills, respecting reserved specialist capacity. The best platforms do not make a bulk reassignment a blunt “move all of these to that person” operation; they make it a *re-allocation* — the tasks return to their candidate groups, or are distributed by the same effort-balancing logic that allocates new work, so the moved tasks land where there is room for them.

This connects reassignment back to fairness. A reassignment that dumps a departed colleague's queue onto one unlucky present colleague is unfair to that colleague in exactly the sense [Chapter 46](#) named — it gives one person too much work through no fault of their own — and it is unfair to the customers whose cases now sit in a suddenly-overloaded queue. Reassignment done well is reassignment that respects where the work is landing; reassignment done as a blunt move is a fairness problem created in the act of solving a coverage problem.

Worked example: the bulk reassignment of a departed employee

Problem. An adjuster leaves the company. At the moment of departure they hold 34 open claims tasks. Two procedures are compared. In procedure A, the 34 tasks are left assigned to the departed adjuster, and the operations team works through them individually over the

following days as they are noticed. In procedure B, a bulk reassignment immediately returns all 34 tasks to their candidate groups the moment the departure is processed. Claims tasks have a 2-business-day SLA. The individual cleanup in procedure A takes, on average, 3 days to complete — during which the affected claims sit unworked. Of the 34 claims, how many breach their SLA under each procedure, and what is the lesson?

Setup. A claim breaches if it sits unworked past its 2-business-day SLA. Under procedure A, the tasks sit with a departed employee until the 3-day individual cleanup reaches them. Under procedure B, they return to active groups immediately. We reason about the breaches under each.

Working. *Procedure A.* The 34 tasks are assigned to a person who is gone; they appear on no active worklist and are claimed by no one. They sit until the operations team’s individual cleanup, averaging 3 days, reaches each. A 3-day delay against a 2-business-day SLA means essentially all of the tasks not cleaned up on the first day breach. With cleanup spread over 3 days, of the order of

$$34 \times \frac{2}{3} \approx 23 \text{ claims breach their SLA}$$

— the two-thirds not reached within the first day.

Procedure B. The 34 tasks return to their candidate groups immediately, appear on active adjusters’ worklists at once, and are worked within the normal SLA like any other task. Breaches caused by the departure:

0.

Discussion. Procedure A breaches the SLA on roughly 23 of the 34 claims; procedure B breaches none — and the difference is a single operational discipline: a departing employee’s open tasks are bulk-reassigned the moment the departure is processed, not cleaned up individually afterwards. The reason procedure A fails is subtle enough to be worth naming. The 34 tasks assigned to the departed adjuster are not visibly a problem — they are assigned, they are not incidents, they do not raise alarms. They are simply invisible: assigned to someone who will never open a worklist again, appearing on no active queue, quietly ageing toward breach while the platform’s operational surface shows nothing wrong. That invisibility is what makes the individual cleanup too slow — the operations team is working through tasks that nothing flags as urgent. Procedure B’s bulk reassignment converts 34 invisible, stranded tasks into 34 ordinary tasks on active worklists, where the normal allocation and SLA machinery handles them like any other work. The lesson is that a personnel event — a departure, a long leave, a shift change — is also a task-allocation event, and must be handled as one, promptly and in bulk. A task assigned to a person who is not coming back is not a safe task; it is a silent breach accumulating, and bulk reassignment is the discipline that prevents it.

Practical next steps

Ask how your operation handles a departing employee’s open tasks. Is there a bulk reassignment, triggered the moment the departure is processed, that returns the tasks to active groups? Or are they left assigned and cleaned up individually? If it is the latter, every departure is a cluster of silent SLA breaches — and the fix is a defined, prompt, bulk reassignment as part of the leaver process.

Chapter in brief

This chapter treats the three operations that surround the working life of a user task — completing it, finding it, and reassigning it. *Completion* is the point of the task: it must carry the task's output as governed process data, must be attributed to the individual who made the decision, and ends the task's claim on a person's attention. *Searching* is how the work is found before it can be completed: a worklist surfaces tasks by querying the Task API, and must respect the access control of the underlying engine rather than re-implementing it, while bounding the platform load of users times refresh rate times search cost. *Reassignment* exists because allocation is not final — people leave, workloads rebalance, escalation intervenes — and every reassignment must be captured in the audit record, must continue to enforce the four-eyes constraint, and must treat a personnel event as a task-allocation event so a departed employee's tasks do not become silent SLA breaches.

Call Activities: Composing Processes from Processes

49.1 The process that is too large for one diagram

A real banking or insurance process, modelled honestly, is large. A mortgage origination process touches application intake, identity verification, credit assessment, valuation, underwriting, offer, and completion — and each of those is itself a substantial piece of flow. Drawn as one diagram, the result is a wall of boxes no reviewer can hold in their head, no team can own in parts, and no two processes can share. The *call activity* is the BPMN mechanism that solves this, and this chapter treats it in the depth a large estate of processes needs.

A call activity is, in one sentence, a step in a process that *invokes another, separately-deployed process* as a unit of work. The calling process reaches the call activity, a new instance of the called process is created and runs to completion, and only then does the calling process move past the call activity to its next step. The called process is a complete, independent BPMN process — deployed on its own, versioned on its own, owned on its own — and the call activity is how one process composes it into a larger whole.

49.2 Why a call activity, and not a subprocess

The next chapter treats the embedded subprocess, and it is worth marking the distinction now because choosing wrongly between them is a common modelling error. An embedded subprocess is drawn *inside* its parent's diagram — it is part of the same BPMN file, deployed and versioned with the parent. A called process is *externalized* — a separate BPMN file, separately deployed, with its own lifecycle.

The distinguishing question is *reuse*. If a piece of flow is used by one process only, and is separated merely to keep the parent diagram readable, an embedded subprocess is right — it is a visual grouping, nothing more. If a piece of flow is, or could be, used by *several* processes — an identity-verification flow that mortgage, account-opening, and lending all need — then it must be a separate process invoked by a call activity, because only a separate process can be shared. A subprocess is owned by one parent; a called process is owned by no parent and reused by many.

Tip

Choose between an embedded subprocess and a call activity by asking one question: is this flow used by more than one process, now or plausibly soon? If yes, it must be a separate process invoked by a call activity — only a separate, externalized process can be reused. If it is used by exactly one parent and separated only for readability, an embedded subprocess is the lighter, correct choice. Reuse is the deciding criterion.

49.3 The variable boundary of a call activity

The most important technical aspect of a call activity — and the one most often misunderstood — is how variables cross between the calling process and the called process.

When a call activity creates an instance of the called process, that instance has its own variable scope. It is not automatically given the calling process's variables, and it does not automatically write back. What crosses, in each direction, is a *deliberate mapping*: the call activity declares which of the calling process's variables are passed *in* to the called process, and which of the called process's result variables are passed *out* to the caller.

This mapping is not an inconvenience to be worked around — it is the call activity's most valuable property, and it should be used deliberately. Camunda permits a call activity to simply propagate *all* the parent's variables into the child, and it is tempting, because it saves the work of declaring the mapping. Resist it. A call activity that passes everything makes the called process depend, invisibly, on the entire variable shape of every process that calls it — and the called process can then never be changed, or reused by a new caller, without checking what every existing caller happens to have in scope. A call activity with an *explicit* input and output mapping has a clear, narrow contract: these variables in, these variables out. That contract is what makes the called process genuinely reusable and independently governable.

The discipline is the data-flow discipline of [Chapter 3](#), applied at the process boundary: the called process declares exactly what it consumes and exactly what it produces, and the call activity's mapping honours that contract explicitly.

Listing 49.1. A call activity invoking a reusable identity-verification process. The binding type pins which version is called; the explicit mapping — not blanket propagation — defines the variable contract.

```
1 <bpmn:callActivity id="VerifyIdentity" name="Verify Identity">
2   <bpmn:extensionElements>
3     <zeebe:calledElement
4       processId="identity-verification"
5       bindingType="deployment" />
6     <!-- explicit input mapping: only what the child needs -->
7     <zeebe:ioMapping>
8       <zeebe:input source="customerRef" target="subjectRef" />
9       <zeebe:input source="documentSet" target="documents" />
10      <!-- explicit output: only what the parent needs back -->
11      <zeebe:output source="verificationResult"
12        target="identityOutcome" />
13    </zeebe:ioMapping>
14  </bpmn:extensionElements>
15 </bpmn:callActivity>
```

49.4 Versioning: which called process runs

A call activity raises a question an embedded subprocess does not: when the called process exists in several deployed versions, *which version* does the call activity invoke? This is a real decision, and Camunda makes it an explicit one through the call activity's *binding type*.

The call activity can bind to the *latest* deployed version of the called process — so improvements to the called process are picked up automatically by every caller. It can bind to the version that was current *at the time the calling process was deployed* — so the caller and the called process move together as a tested pair, and a later change to the called process does not silently alter the caller's behaviour. Or it can bind to a specific *tagged* version — pinning

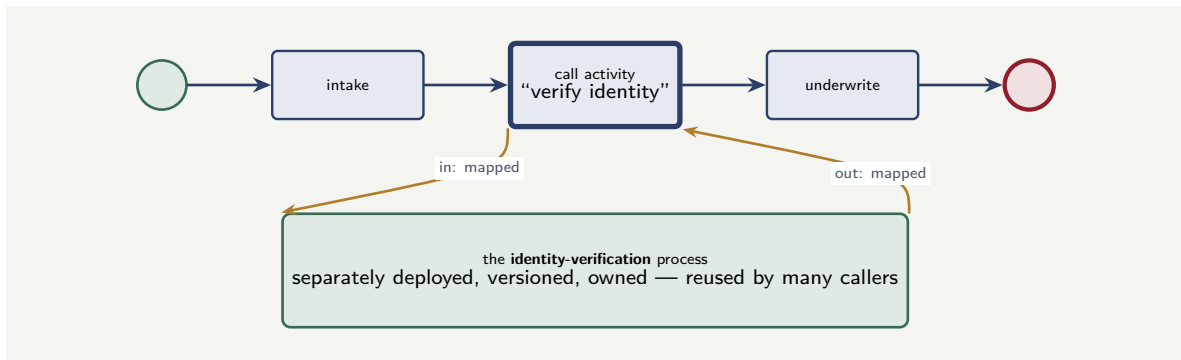


Figure 49.1. A call activity. The thick-bordered step in the parent process invokes a separately-deployed, reusable process; variables cross the boundary by an explicit input and output mapping, not by blanket propagation. The called process is owned independently and reused by many callers.

the caller to a known, named version of the called process.

The choice matters for a regulated institution, and it is the [Chapter 37](#) versioning question in a new place. Latest-version binding is convenient but means a change to a shared called process instantly changes the behaviour of every process that calls it — which, for a called process that embodies a regulated control, may be exactly what a governance team does *not* want to happen without deliberate re-deployment. Deployment-time binding makes the caller and called process a governed, tested pair. The right choice depends on the called process: a shared utility may want latest; a called process that embodies a control usually wants the pinned, deliberate binding.

Figure 49.1 draws the call activity invoking its reusable called process across an explicit variable boundary.

49.5 Errors across the call boundary

One more technical point completes the picture: what happens when the called process fails. If the called process ends in a BPMN error — a business error of [Chapter 41](#)’s kind — that error propagates *up* to the call activity, and the calling process can catch it with a boundary error event on the call activity and route accordingly. This is the right and governable behaviour: the called process declares its error outcomes, and the caller decides how to handle them, exactly as a service task’s business errors are caught and routed.

The discipline this implies is that a reusable called process should have a *defined set of error outcomes* — named BPMN errors — as part of its contract, just as it has a defined set of input and output variables. A caller then knows not only what variables to pass and expect, but what can go wrong and how it will be told. A called process whose failure modes are undocumented is a called process no caller can safely handle.

49.6 Choosing the right subprocess type

The choice between an embedded subprocess, an event subprocess, and a transaction subprocess is a design decision, and each has a clear use case. An embedded subprocess *groups* related steps and provides a scope for local variables and shared boundary events. An event subprocess handles *interruptions* that can happen at any point during the parent’s life. A transaction subprocess provides an *all-or-nothing* boundary with compensation. Using the wrong

type — a transaction subprocess where a simple grouping would suffice, or an embedded subprocess where compensation is needed — either overcomplicates the model or fails to provide the safety the business requires.

Worked example: the called process changed under callers

Problem. An identity-verification process is built as a reusable called process and is invoked, by call activities, from four parent processes: mortgage, current-account opening, lending, and credit-card application. All four call activities use *latest-version* binding. The identity team deploys a new version of the called process that adds a required input variable, `jurisdictionCode`, which the verification logic now depends on. Three of the four parent processes happen to have that variable in scope and pass it; the credit-card process does not. The credit-card process runs 4,000 times a month. From the moment the new version is deployed, what happens to the credit-card process, and what does this reveal?

Setup. With latest-version binding, every caller switches to the new called-process version the instant it is deployed — with no redeployment of the caller and no test of the pair. We reason about the credit-card process, which does not supply the new required variable.

Working. The moment the new version is deployed, all four call activities — bound to latest — begin invoking it. The mortgage, account-opening, and lending processes supply `jurisdictionCode` and are unaffected. The credit-card process does not supply it; its call activity now invokes a called process that requires a variable the caller never maps. Every credit-card identity verification now fails:

4,000 credit-card applications a month

begin failing at the identity step — immediately, with no code change to the credit-card process and no warning to the credit-card team.

Discussion. A team that does not own the credit-card process, did not touch the credit-card process, and was not in contact with the credit-card team has broken the credit-card process — 4,000 applications a month — by deploying a change to a *different* process. The worked example is built to show that this is not a freak event but the predictable consequence of two choices made together. The first is the blanket coupling: had the call activities used *explicit* variable mappings throughout, the dependency of the called process on each input would be visible in each caller's model, and a reviewer of the new called-process version would see exactly which callers mapped the new variable and which did not. The second is latest-version binding: it propagated the change to every caller *instantly*, with no moment at which the credit-card pairing could be tested before going live. Each choice alone is survivable; together they make a change to one process a silent break of another. The fix is the chapter's discipline. Explicit variable mappings make the contract visible. And for a called process that many parents depend on, deployment-time or tagged binding — not latest — means a new version of the called process reaches a caller only when that caller is deliberately redeployed, at which point the pairing is tested. A reusable called process is a powerful thing, but reuse is coupling, and coupling that is not explicit and not version-governed is how one team's improvement becomes another team's incident.

Practical next steps

For one reusable called process in your estate, list its callers and check two things: does each call activity use an explicit variable mapping, or blanket propagation; and does each bind to latest, or to a pinned version? Blanket propagation plus latest binding is the combination

that lets a change to the called process silently break a caller. Make the mappings explicit, and choose the binding deliberately for what the called process is.

Chapter in brief

A call activity is a step that invokes another, separately-deployed process as a unit of work — the mechanism for composing a large process from independently-owned parts. It differs from an embedded subprocess by being *reusable*: a called process is externalized and can be invoked by many parents, where a subprocess belongs to one. Variables cross the call boundary by an explicit input and output mapping, which is the called process's contract — blanket propagation of all variables destroys that contract and the called process's reusability. The binding type — latest, deployment-time, or tagged — decides which version of the called process runs, and for a called process embodying a control the pinned binding is usually right. Business errors propagate up to the call activity for the caller to catch.

Subprocesses: Structure, Scope, and Boundary

50.1 Structure within one process

The previous chapter treated the call activity — the invocation of a *separate* process. This chapter treats the *embedded subprocess* — structure *within* a single process — and the related event subprocess. Together they are how a process is given internal shape: not broken into separate deployable pieces, but organised, scoped, and made able to respond to events, all within one BPMN model.

An embedded subprocess is, at its simplest, a box on the diagram that contains a piece of flow — its own start, its own steps, its own end — drawn inside the parent process. It is part of the same BPMN file, deployed and versioned with the parent. But it is not merely a visual grouping; the subprocess boundary is a real boundary, and three things it does — scoping, event handling, and transactional grouping — make it a genuine modelling tool rather than a cosmetic one.

50.2 The subprocess as a scope

The first thing a subprocess boundary does is create a *scope*.

Variables can be defined *local* to a subprocess — they exist while the subprocess runs and are gone when it completes, not polluting the parent process's variable space. A subprocess that does some intermediate calculation, needing working variables that mean nothing outside it, should hold those variables locally; they are then visibly the subprocess's own, and the parent's variable space stays clean and meaningful. This is the data-minimisation thinking of Part IV applied to the internal structure of a process: a variable should exist in the narrowest scope that needs it.

A scope also bounds *events*. A boundary event on a subprocess — a timer, an error catch — applies to the whole subprocess as a unit. A timer on a subprocess boundary sets a deadline for the entire enclosed piece of flow; an error boundary event on a subprocess catches an error arising anywhere within it. This is powerful: it lets a deadline or an exception handler apply to a *group* of steps, expressed once on the group's boundary, rather than repeated on each step inside.

Figure 50.1 draws the subprocess boundary as a scope — bounding both variables and events.

50.3 The event subprocess

A particular and valuable kind of embedded subprocess is the *event subprocess*: a subprocess that is not on the normal flow at all, but sits inside its parent waiting to be *triggered by an event*.

An event subprocess has an event start event — a message, a timer, an error, an escala-

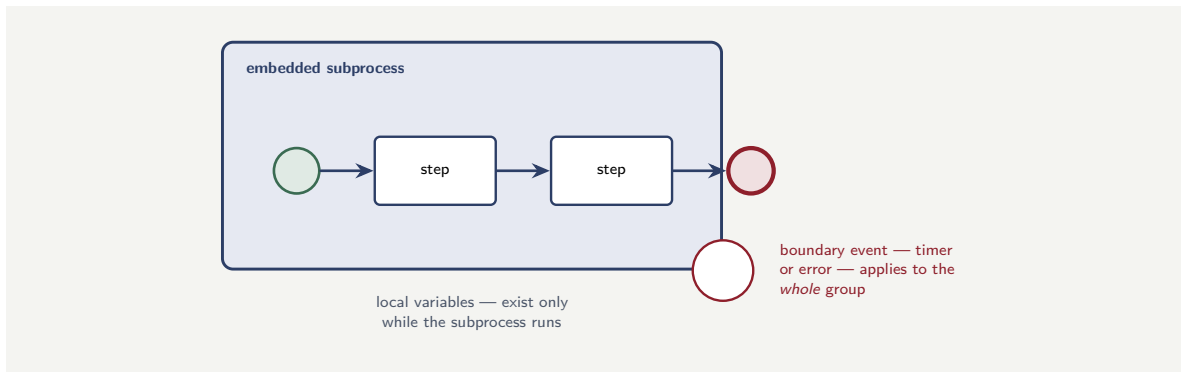


Figure 50.1. The embedded subprocess as a scope. Variables declared local to the subprocess exist only while it runs and never reach the parent’s variable space; a boundary event on the subprocess edge — a timer, an error catch — applies to the whole enclosed group of steps at once, expressed once rather than repeated on each step.

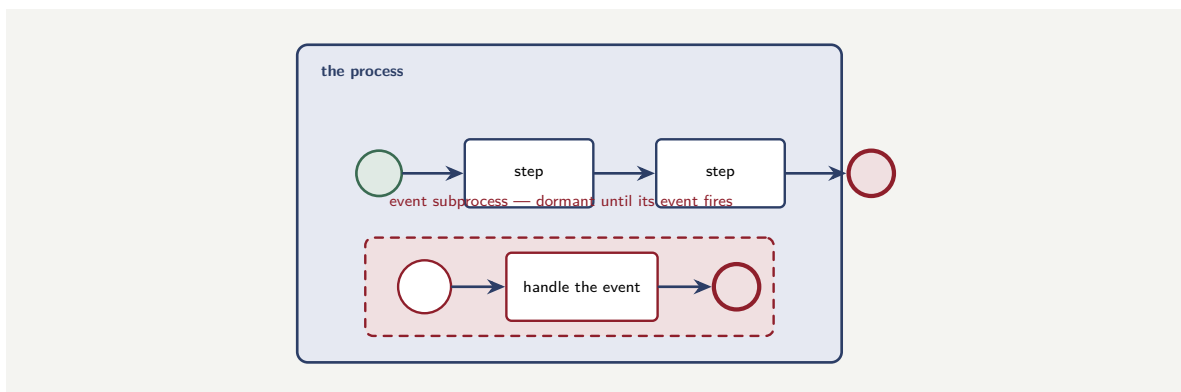


Figure 50.2. An event subprocess. Inside the parent process, alongside the main flow, a dormant event subprocess waits for its triggering event — a cancellation, a fraud signal, a regulatory hold. When the event fires at any point in the parent’s life, the event subprocess runs, interrupting the parent or running alongside it.

tion — and it lies dormant until that event occurs within its parent’s scope. Then it runs. It comes in two forms. An *interrupting* event subprocess, when triggered, cancels the rest of its parent’s flow — the parent stops what it was doing and the event subprocess takes over. A *non-interrupting* event subprocess runs *alongside* the parent’s continuing flow — the parent carries on, and the event subprocess handles the event in parallel.

The event subprocess is the right tool for a class of requirement that is otherwise awkward to model: *something that may happen at any point during a process, and must be handled when it does*. A customer cancellation that may arrive at any stage of an application; a fraud signal that may fire at any point in a payment’s life; a regulatory hold that may land at any time. Trying to model these as explicit flow — a gateway after every step asking “did a cancellation arrive?” — produces an unreadable tangle. An event subprocess expresses it cleanly: one dormant handler, triggered by the event whenever in the parent’s life it occurs.

Figure 50.2 draws an event subprocess dormant within its parent, waiting for the event that may arrive at any point.

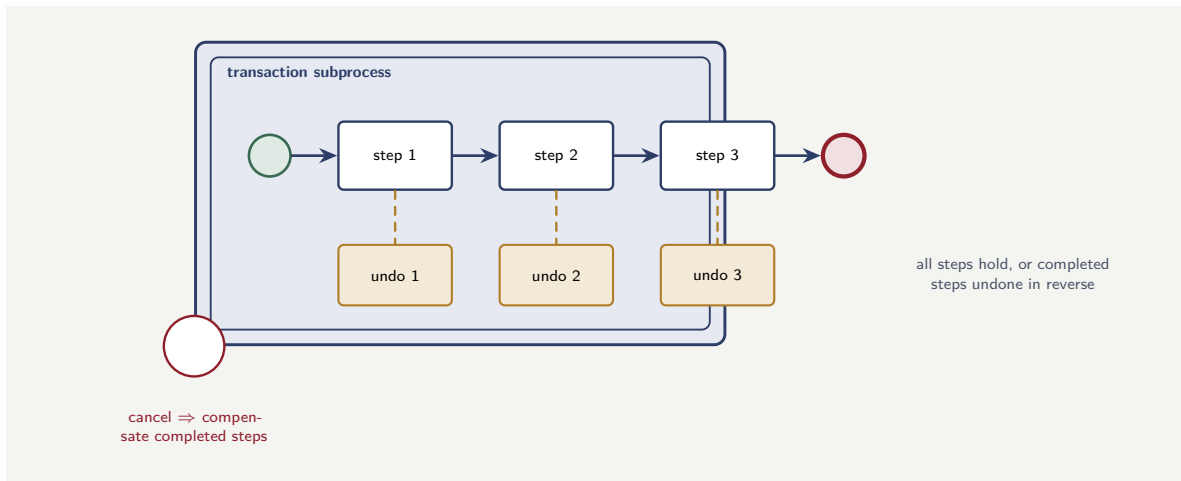


Figure 50.3. The transaction subprocess and compensation. The forward steps each carry a paired compensating action; if the subprocess is cancelled, the engine runs the compensations of the steps that had completed, in reverse. The transaction boundary is the concrete, reviewable form of the saga’s all-or-nothing outcome.

A requirement of the form “X may happen at any point in this process and must be handled” — a cancellation, a fraud hold, a complaint — should not be modelled as a gateway check repeated after every step. That produces an unreadable diagram and, worse, a check that someone will forget to add after a newly-inserted step, leaving a gap. Model it as an event subprocess: one dormant handler, triggered whenever in the parent’s life the event occurs, with no gap and no repetition.

50.4 The transaction subprocess and compensation

A subprocess can also be marked as a *transaction* — and this connects to the saga pattern of [Chapter 44](#). A transaction subprocess groups a set of steps for which an all-or-nothing business outcome is wanted: either the whole subprocess completes successfully, or, if it is cancelled, the *compensation* of its already-completed steps is triggered, undoing them in reverse.

The transaction subprocess is the modelled home of the saga. The steps inside it each carry their compensating actions; if the subprocess is cancelled — by an error, by a cancellation event — the engine runs those compensating actions for the steps that had completed. The all-or-nothing boundary that [Chapter 44](#) described is, concretely, the transaction subprocess’s boundary. It is how “these five steps across four systems must all hold or all be undone” becomes a single, reviewable element of the model.

Figure 50.3 draws the transaction subprocess — the forward steps, their paired compensations, and the cancel boundary that triggers the undo.

50.5 The subprocess in BPMN and in code

The three subprocess variants — embedded, event, transaction — share one characteristic in BPMN: they are each a single element on the parent process’s diagram, and the structure they contain is held inside that element. What varies is the *trigger* and the *boundary*. A concrete

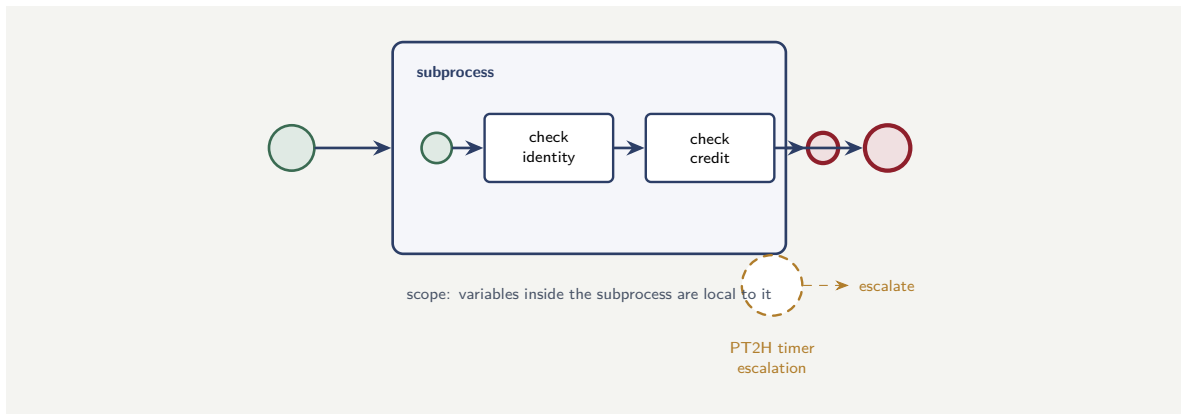


Figure 50.4. An embedded subprocess with a non-interrupting boundary timer. The subprocess groups two inner tasks; its boundary is a scope for local variables. If the timer fires, the escalation flow is taken while the inner steps continue — the `cancelActivity="false"` of Listing 50.1.

BPMN fragment and a Java worker show what this means in practice.

Listing 50.1 is the core of an embedded subprocess in BPMN: a subprocess element that wraps a sequence of tasks, with a timer boundary event on the subprocess boundary that fires if the whole group has not completed within the allotted time.

Listing 50.1. An embedded subprocess with a non-interrupting boundary timer. The subprocess groups the two inner tasks; if they take too long the timer fires and the escalation flow is taken while the subprocess continues.

```

1 <bpmn:subProcess id="verifyApplication"
2   name="Verify Application">
3   <!-- the inner steps -->
4   <bpmn:serviceTask id="checkIdentity"
5     name="Check Identity">
6     <bpmn:extensionElements>
7       <zeebe:taskDefinition type="identity-check"/>
8     </bpmn:extensionElements>
9   </bpmn:serviceTask>
10  <bpmn:serviceTask id="checkCredit"
11    name="Check Credit">
12    <bpmn:extensionElements>
13      <zeebe:taskDefinition type="credit-check"/>
14    </bpmn:extensionElements>
15  </bpmn:serviceTask>
16 </bpmn:subProcess>
17
18 <!-- non-interrupting boundary timer on the subprocess -->
19 <bpmn:boundaryEvent id="verifyTimer"
20   attachedToRef="verifyApplication"
21   cancelActivity="false">
22   <bpmn:timerEventDefinition>
23     <bpmn:timeDuration>PT2H</bpmn:timeDuration>
24   </bpmn:timerEventDefinition>
25 </bpmn:boundaryEvent>

```

Figure 50.4 shows the subprocess model — the grouping, the inner steps, and the boundary timer’s non-interrupting flow.

The subprocess’s *scope* property makes its local variables worth showing in code. A worker running inside the subprocess declares its variables as *local* — they exist only inside the subprocess’s life and are not propagated to the parent. Listing 50.2 is an identity-check worker that returns a `rawCheckResult` variable, which the subprocess uses internally but the parent

process never needs to carry.

Listing 50.2. A worker inside a subprocess returning a local variable: the result is used inside the subprocess but does not propagate to the parent, keeping the parent's variable space clean.

```
1 @JobWorker(type = "identity-check")
2 public Map<String, Object> check(ActivatedJob job) {
3     String customerId = job.getVariable("customerId");
4
5     IdentityResult result = identityService.check(
6         customerId);
7
8     // rawCheckResult is a subprocess-local variable:
9     // the parent process does not see it
10    return Map.of(
11        "identityVerified", result.passed(),
12        "rawCheckResult", result.rawCode());
13 }
```

Worked example: the cancellation modelled two ways

Problem. A loan-application process has eleven sequential steps. The business requires that a customer cancellation, which may arrive at any point, stops the process and runs a defined cleanup. Two teams model this. Team A adds, after each of the eleven steps, an exclusive gateway that checks a cancelled flag and routes to cleanup if set. Team B models the cleanup as a single interrupting event subprocess triggered by a cancellation message. Some months later, three new steps are inserted into the process. Compare what each team's design requires, then and at the later change, and the risk each carries.

Setup. We count the modelling elements each design needs, and consider what the later insertion of three steps demands of each.

Working. *Team A* — a gateway after every step. The initial model needs, for the cancellation handling alone:

11 gateways + 11 routing flows + 1 cleanup flow,

woven through the diagram. When three steps are later inserted, three more gateways must be added — and, critically, each must be added *correctly*, by whoever inserts the steps, or a gap appears: a step after which a cancellation is not checked.

3 new gateways required, each a chance to forget.

Team B — one event subprocess. The initial model needs:

1 event subprocess,

dormant, triggered by the cancellation message wherever it arrives. When three steps are later inserted:

0 changes to the cancellation handling — it already covers any point.

Discussion. Team A's design works on the day it is built and is a latent defect from that day forward; team B's design works and stays correct through the later change without being touched. The worked example's lesson is about *where correctness lives*. In team A's design, the correctness of the cancellation control is distributed across eleven — then fourteen — gateways, and is only as good as the diligence of every person who ever inserts a step. The control

is correct by *repetition*, and repetition is fragile: the engineer who inserts the three new steps is thinking about the new steps, not about the cancellation control, and the gateway they forget to add is a silent gap through which a cancellation arriving at exactly that step is missed. In team B's design, the correctness lives in *one* element whose entire purpose is "handle a cancellation whenever it occurs." The event subprocess does not need to know how many steps the process has, or which were inserted when — it covers any point in the parent's scope by construction. Inserting three steps cannot create a gap, because there were never per-step checks to keep complete. This is the deeper value of the subprocess as a scope: it lets a concern that applies to a *whole region* of a process be expressed once, on the region's boundary, rather than woven — fragilely, repetitiously — through every step inside it.

Practical next steps

Look through one of your larger process models for a cross-cutting concern — a cancellation, a hold, a timeout — handled by a check or a gateway repeated at several points. Each repetition is a place a future edit can leave a gap. Consider whether an event subprocess, or a boundary event on an embedded subprocess, could express the concern once on a scope boundary instead.

Chapter in brief

An embedded subprocess is structure within one process — part of the same BPMN file — and its boundary is a real boundary that does three things. It creates a *scope*: variables can be local to it, and a boundary event — a timer, an error catch — applies to the whole enclosed group. The *event subprocess* is a dormant handler, triggered by an event that may occur at any point in the parent's life — the clean way to model a cancellation, a fraud hold, a regulatory hold — either interrupting the parent or running alongside it. The *transaction subprocess* is the modelled home of [Chapter 44](#)'s saga: an all-or-nothing boundary that triggers compensation of its completed steps if cancelled.

Parallel Work and Multi-Instance Activities

51.1 When work need not be sequential

Most of the process flow this manuscript has drawn is sequential: one step, then the next. But a great deal of real banking and insurance work is not inherently sequential — it is merely *drawn* sequentially out of habit, and drawing it so makes the process slower than it needs to be. This chapter treats the two BPMN mechanisms for work that happens in parallel: the *parallel gateway*, for a fixed set of branches that run at once, and the *multi-instance activity*, for a step that runs many times over a collection.

The motivation is partly speed and partly honesty. Speed, because parallel work finishes in the time of its longest branch rather than the sum of all branches. Honesty, because a process drawn sequentially when its steps are truly independent is *misrepresenting* the work — it implies an ordering constraint that does not exist, and a model should represent the real shape of the work, not an arbitrary linearisation of it.

51.2 The parallel gateway

The *parallel gateway* splits one flow into several that run simultaneously, and — in its closing form — joins them back together. A parallel split activates all its outgoing branches at once; the corresponding parallel join waits for *all* its incoming branches to arrive before the flow continues.

The classic banking case is the independent check. An onboarding process must run an identity check, a sanctions screening, and a credit check. None depends on the others; all must complete before underwriting. Drawn sequentially, the process takes the sum of the three durations. Drawn with a parallel gateway — split into three branches, joined after — it takes the duration of the *longest* of the three, because they run at once. For checks that each take a second or two against external systems, the difference is the difference between an onboarding that feels instant and one that feels slow.

The discipline of the parallel gateway is in the *join*. The join waits for all branches, and that is usually what is wanted — but it means the flow proceeds at the pace of the slowest branch, and a branch that hangs hangs the whole join. The parallel join is a synchronisation point, and its correctness depends on every branch genuinely completing. A branch that can fail must fail in a way the join can still resolve — an error caught and routed, not a branch that simply never arrives.

Figure 51.1 draws the parallel split and join around three independent checks.

Figure 51.2 makes the time saving concrete: the same three checks sequential versus parallel, with the 60% reduction visible. The saving grows with each additional independent branch, and a process designer who defaults to sequential because sequential is easier to draw has, in effect, charged the business for an ordering constraint that serves no purpose.

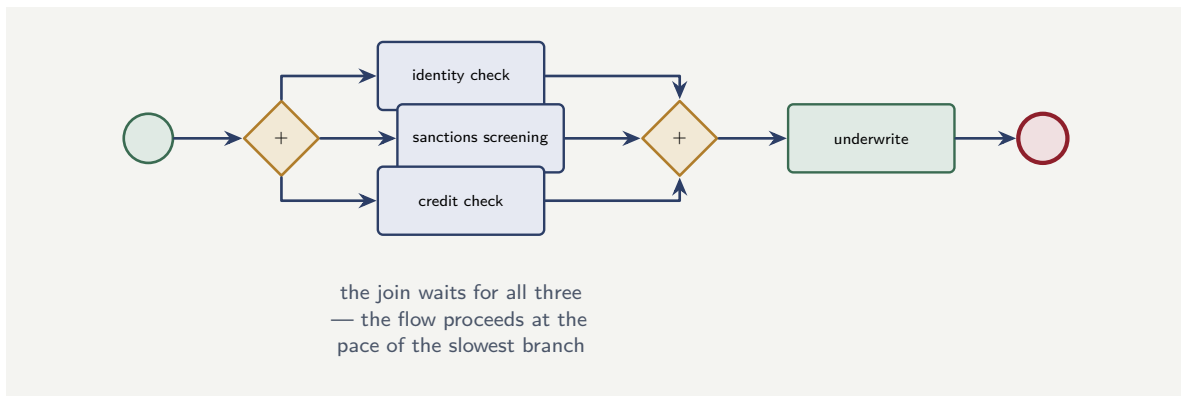


Figure 51.1. A parallel gateway. The split activates three independent checks at once; the join waits for all three before underwriting proceeds. The parallel work finishes in the time of the longest branch, not the sum — and the model honestly shows that the three checks have no ordering between them.

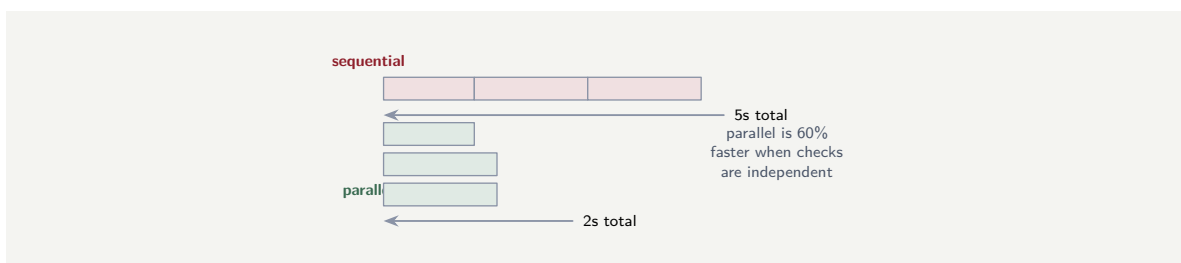


Figure 51.2. Sequential versus parallel for the same three checks. Sequential takes the *sum* of all durations ($2 + 1.5 + 1.5 = 5\text{s}$); parallel takes only the *longest* (2s) — a 60% reduction. Parallelism’s value grows with the number of independent branches.

51.3 The multi-instance activity

The parallel gateway handles a *fixed* set of branches known when the model is drawn. But often the parallel work is over a *collection* whose size is not known until the process runs: a syndicated loan with a list of participating banks, each needing the same approval step; a claim with several damaged items, each needing the same assessment; a customer with a list of accounts, each needing the same review. The *multi-instance activity* is the mechanism for this — a single activity, drawn once, that runs once for *each element of a collection*.

A multi-instance activity is configured with an *input collection* — the process variable holding the list to iterate over — and it creates one instance of the activity per element. It can run those instances two ways. A *sequential* multi-instance runs them one at a time, each after the last. A *parallel* multi-instance creates them all at once and runs them concurrently. And it gathers results: an *output collection* collects each instance’s result into a list the process can use after the multi-instance activity completes.

The choice between sequential and parallel multi-instance is a real one. Parallel is faster — the collection is processed in the time of its slowest element. But sequential is right when the instances must not run at once: when each calls a downstream system that cannot take the concurrent load, or when the work has an ordering. And a multi-instance can carry a *completion condition* — a boolean expression checked as each instance completes, which, when true, finishes the multi-instance activity early without waiting for the remaining instances. A multi-instance “check each of these documents” might complete as soon as one is found invalid, if one invalid document is enough to decide the case.

The two constructs in code

Both constructs are clearest seen in the BPMN model. Listing 51.1 is a parallel gateway: a fork gateway splits the flow into concurrent branches, and a join gateway — the same element type — waits for all of them before the flow continues.

Listing 51.1. A parallel gateway in BPMN: a fork splits the flow into concurrent branches; a join waits for every branch before continuing.

```
1 <bpmn:parallelGateway id="fork"/>
2 <bpmn:parallelGateway id="join"/>
3
4 <!-- the fork feeds three concurrent branches -->
5 <bpmn:sequenceFlow sourceRef="fork" targetRef="creditCheck"/>
6 <bpmn:sequenceFlow sourceRef="fork" targetRef="fraudScreen"/>
7 <bpmn:sequenceFlow sourceRef="fork" targetRef="amlCheck"/>
8 <!-- the join waits for all three -->
9 <bpmn:sequenceFlow sourceRef="creditCheck" targetRef="join"/>
10 <bpmn:sequenceFlow sourceRef="fraudScreen" targetRef="join"/>
11 <bpmn:sequenceFlow sourceRef="amlCheck" targetRef="join"/>
```

A multi-instance activity is a single task carrying a `multiInstanceLoopCharacteristics` element. Listing 62.1 configures one: it names the *input collection* to iterate over, the variable each instance receives as its element, the *output collection* that gathers the results, and — through the `isSequential` flag — whether the instances run one at a time or all at once.

Listing 51.2. A parallel multi-instance activity: one task that runs once per element of an input collection, gathering each result into an output collection.

```
1 <bpmn:serviceTask id="assessParticipant" name="Assess">
2   <bpmn:multiInstanceLoopCharacteristics
3     isSequential="false">
4     <bpmn:extensionElements>
5       <zeebe:loopCharacteristics
6         inputCollection="=participants"
7         inputElement="participant"
8         outputCollection="assessments"
9         outputElement="=assessmentResult"/>
10    </bpmn:extensionElements>
11    <!-- finish early once any assessment fails -->
12    <bpmn:completionCondition>
13      =assessmentResult.status = "FAIL"
14    </bpmn:completionCondition>
15  </bpmn:multiInstanceLoopCharacteristics>
16 </bpmn:serviceTask>
```

Three parts of that configuration are the ones that matter. The `inputCollection` is a FEEL expression — `=participants` — evaluated against the process variables, so the collection is whatever the process computed at runtime, not a fixed list. The `isSequential` flag is the sequential-versus-parallel choice the prose described, expressed as one boolean. And the `completionCondition` is the early-exit rule: a FEEL expression checked as each instance finishes, here ending the activity the moment any participant’s assessment fails.

Tip

Choose parallel multi-instance for speed when the instances are genuinely independent — but choose sequential when each instance calls a downstream system that cannot absorb the whole collection’s worth of concurrent calls. A parallel multi-instance over a 500-element collection makes 500 simultaneous calls to whatever each instance invokes;

if that is a core banking system, the parallel multi-instance has just become a load spike the downstream system may not survive. The choice between sequential and parallel is also a choice about the load you place on everything the instances call.

51.4 When to use this pattern

Every pattern this part describes is a tool, not a default. The parallel gateway is for genuinely independent work; using it for dependent work produces race conditions. The compensation pattern is for irreversible external effects; using it for in-memory state that can simply be rolled back is overengineering. The correlation pattern is for matching an incoming event to a waiting instance; using it when a direct call would suffice adds complexity for no benefit. The discipline is to reach for each pattern when the work genuinely needs it, and to use the simpler construct when it does not.

Worked example: sequential versus parallel for a syndicated loan

Problem. A syndicated-loan process must obtain an approval from each participating bank. A typical syndication has 8 participating banks; the approval step for each takes, on average, 6 hours of elapsed time (the partner bank's own turnaround). The approval step is modelled as a multi-instance activity over the list of banks. Compare the elapsed time of the syndication approval under a sequential multi-instance and a parallel one, and note when each is nonetheless the right choice.

Setup. A sequential multi-instance runs the 8 approvals one after another; a parallel multi-instance runs all 8 at once. We compute the elapsed time of each.

Working. *Sequential multi-instance.* The 8 approvals run one at a time:

$$8 \times 6 \text{ hours} = 48 \text{ hours of elapsed time.}$$

Parallel multi-instance. All 8 approvals run at once; the step completes when the slowest finishes:

$$\max(8 \text{ approvals}) \approx 6 \text{ hours of elapsed time.}$$

The parallel multi-instance completes the syndication approval

$$\frac{48}{6} = 8 \text{ times faster.}$$

Discussion. The parallel multi-instance turns a two-day approval stage into a six-hour one, and for a syndicated loan — where the participating banks genuinely approve independently, with no ordering between them — it is clearly the right choice: the sequential design's 48 hours is not 48 hours of necessary sequencing, it is 48 hours of an arbitrary linearisation of work that was never sequential. The eight banks do not wait for each other in reality, so a model that makes them wait for each other is, in the language of this chapter, *dishonest* about the work — and it costs the customer two days. But the worked example's discussion must also mark the boundary, because parallel is not always right. Suppose the per-bank approval step were not a wait on a partner bank but a call to the institution's *own* overloaded internal pricing service. Then a parallel multi-instance over a large syndication — or worse, over a 200-name collection in some other process — fires the whole collection's calls at that service

simultaneously, and the speed-up is bought by a load spike that may bring the service down. The discipline is to ask, for each multi-instance, two questions: are the instances genuinely independent, so parallel is *correct*; and can everything the instances call absorb the whole collection's concurrency, so parallel is *safe*. When both answers are yes — as for the syndicated loan's partner banks — parallel multi-instance is a large, honest speed-up. When the second answer is no, sequential, or a controlled batch size, is the price of not turning a process into a denial-of-service on its own infrastructure.

Practical next steps

Find one process in your estate that processes a collection — items on a claim, accounts on a customer, names on a syndication — with a sequential loop or a sequence of repeated steps. Ask the two questions: are the elements genuinely independent, and can the downstream systems absorb the concurrency? If both are yes, a parallel multi-instance is a large speed-up the process is currently leaving on the table.

Chapter in brief

Much banking work drawn sequentially is not inherently sequential, and drawing it so makes the process needlessly slow and misrepresents the work. The *parallel gateway* splits flow into a fixed set of branches that run at once and joins them — the right tool for independent checks like identity, sanctions, and credit; the parallel work finishes in the time of the longest branch. The *multi-instance activity* runs one activity once per element of a collection whose size is unknown until runtime, with an input collection and a gathered output collection. Multi-instance can be sequential or parallel: parallel is faster, but a parallel multi-instance fires the whole collection's concurrency at whatever the instances call — so it must be both genuinely independent and safe for the downstream systems.

The Camunda 8 Components in Depth

52.1 The platform as a set of cooperating components

[Chapter 28](#) introduced the Camunda components people use — the Modeler, Operate, Tasklist, Optimize, Connectors — at the level needed to understand the platform’s shape. This chapter returns to them with the depth a platform engineer actually needs: what each component *is*, technically; how it relates to the others; how it is configured and scaled; and the component-specific facts that decide whether a deployment is sound. Where the earlier chapter answered “what are the components for,” this one answers “how does each component actually work, and what must I know to run it.”

The single most important framing is this: a Camunda 8 platform is not one program. It is a set of distinct, cooperating components, each with its own job, its own resource profile, its own scaling behaviour, and its own failure modes. An engineer who treats the platform as a monolith will size it wrongly, scale it wrongly, and be surprised by its failures. An engineer who knows the components individually can reason about each. This chapter is that component-by-component knowledge.

52.2 Zeebe: the engine, the brokers, and the gateway

At the centre of every Camunda 8 platform is *Zeebe* — the workflow engine — and Zeebe is itself not one thing but three roles worth distinguishing.

The *brokers* are the heart. A broker is a process that holds and advances process-instance state: it runs the BPMN, moves the tokens, manages the timers, and writes the event stream. [Chapter 27](#) described their internals — the partitioned, replicated, append-only log. The operationally decisive fact, repeated here because it governs everything, is that brokers are *stateful*: each broker holds real execution state, the brokers are not interchangeable, and so they run on persistent storage with stable identity. The broker count and the partition count are the engine’s capacity, and the partition count is fixed at cluster creation.

The *gateway* is the brokers’ front door. Client applications — workers, the applications that start processes — do not talk to brokers directly; they talk to the gateway, which routes their requests to the right broker. The gateway is *stateless*: it holds no execution state, it merely routes, and so — unlike the brokers — it is scaled freely by running more copies. A platform engineer should know that the gateway speaks a deliberate mix of protocols: a gRPC interface, which requires HTTP/2 transport, and a REST interface served over ordinary HTTP. The job-worker traffic of [Chapter 32](#) flows through the gateway, so the gateway must be sized for the worker fleet’s request volume, not just for the process-starting traffic.

The *clients* are not a server component at all — they are libraries embedded in the institution’s own applications, as [Chapter 44](#) described for Java. They connect to the gateway. The architectural consequence, and it is a valuable one, is that the institution’s worker and client

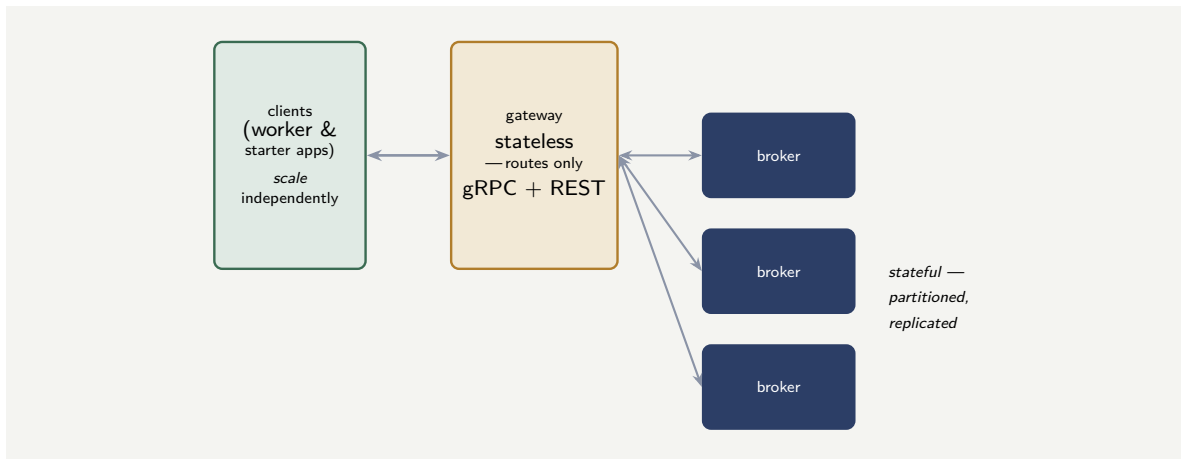


Figure 52.1. Zeebe’s three roles. The institution’s client applications connect to the stateless gateway, which routes their gRPC and REST requests to the stateful, partitioned, replicated brokers. The clients scale independently of the brokers — a worker fleet can grow without touching the engine.

applications scale *independently* of Zeebe: the worker fleet can be scaled up against a load spike without touching the brokers, and the brokers can be scaled without redeploying the workers.

Figure 52.1 draws Zeebe’s three roles and their relationship.

52.3 Operate and Tasklist: the runtime-facing components

Operate and *Tasklist* are the two components through which people interact with running processes, and although they look like simple web applications, a platform engineer should understand what they actually are.

Both are, in essence, *query applications over the platform’s data*. They do not hold process state themselves and they do not drive the engine; they read the platform’s data and present it. *Operate* presents it to operators: every running and completed instance, where each instance’s token sits, which instances have raised incidents and why — and it offers the operational actions of [Chapter 28](#), resolving an incident or retrying a job. *Tasklist* presents the human-task side: the worklists of [Chapter 30](#) and the forms of [Chapter 26](#), the interface where a person claims and completes a user task.

The component-specific fact that matters most is *where they read from*, and it is a fact in motion. In the Camunda 8 architecture an engineer is most likely to meet, *Operate* and *Tasklist* read from the *secondary store* — the Elasticsearch or OpenSearch of [the data part](#) — fed from the engine by an *exporter*. This has a visible operational consequence: there is a small *delay* between something happening in the engine and its appearing in *Operate*, because the data must be exported and indexed first. An operator watching *Operate* is watching a view that is current to within seconds, not instantaneous — usually fine, occasionally a thing to know. Camunda has been actively reducing this delay: more recent releases move toward a unified *Camunda Exporter* and an architecture that streams data to the web components more directly, cutting the latency that older exporter-and-importer designs could accumulate. A platform engineer should check, for the specific version deployed, how *Operate* and *Tasklist* are fed — because that determines both the data-freshness behaviour and the infrastructure the components need.

Tip

Operate and Tasklist are query applications over the platform's data, not the engine itself — and in the common architecture they read a secondary store fed by an exporter. This means the view they show is current to within seconds, not instantaneous: an operator resolving an incident is acting on data exported a moment ago. Know, for your deployed version, how the web components are fed — the exporter architecture has been changing release to release, and it determines both data freshness and the infrastructure the components require.

52.4 The Connector runtime: a component in its own right

[Chapter 33](#) treated connectors as a concept — the reusable, configurable integration. This section adds the component fact: the connectors are executed by a distinct, deployable component called the *Connector runtime*, and a platform engineer must treat it as a component to be deployed, configured, and scaled, not as something the engine does internally.

The Connector runtime is, in effect, a specialised host for connector logic. For *outbound* connectors, it behaves as a job worker: it activates the jobs of connector-typed service tasks and runs the connector function. For *inbound* connectors, it does something the engine cannot do alone — it listens: it polls or subscribes to the external sources that webhook and message-start connectors depend on, and turns an arriving external event into a started process or a correlated message. The runtime discovers which inbound connectors to run from the deployed process models.

Two component-specific facts follow. First, the Connector runtime is its own *deployment* with its own resources and its own scaling — a platform heavy on connector traffic must size the runtime for it, and a runtime that is under-resourced becomes a bottleneck on every connector-using process. Second, the runtime is where connector *secrets* live at execution time: it is configured with access to the secrets the connectors need, injected per [Chapter 141](#)'s discipline, so the runtime is also a security-sensitive component whose configuration must be handled with care.

52.5 Identity, Modeler, and Console

Three more components complete the platform, and each has its specific role.

Identity is the component that manages authentication and authorization — who may sign in, what each user and application may do. It is the concrete machinery behind [Chapter 35](#)'s security model, and it typically integrates with the institution's own identity provider rather than being a separate directory. Every other component relies on Identity, which makes it both essential and security-critical.

The *Modeler* — in its Web and Desktop forms — is where BPMN, DMN, and forms are built. The Web Modeler is a collaborative, server-deployed component; the Desktop Modeler is a local application. The component fact worth knowing is that the Modeler is a *design-time* component: it is not in the runtime path, so its availability does not affect running processes, but it is part of the platform's change pipeline and is governed accordingly.

Console is the management component — creating, configuring, and monitoring clusters, and administering organisational settings. It is the operator's control panel for the platform itself, as distinct from Operate, which is the operator's window onto the *processes* the platform runs.

52.6 How the components are packaged and deployed

A final, practical component fact: these components are not installed individually by hand. As [Chapter 56](#) described, a self-managed Camunda 8 platform is deployed onto Kubernetes through a *Helm chart* that installs the set of components together, configured to cooperate. The platform engineer chooses which components the chart installs and how each is configured and scaled — and the chart’s structure mirrors the component structure of this chapter: the stateful engine in a *StatefulSet*, the stateless components as ordinary scalable deployments, the secondary store as its own infrastructure. Knowing the components individually, as this chapter has set them out, is exactly what makes the Helm chart’s configuration legible rather than mysterious: each value in the chart configures a component whose job the engineer now understands.

Worked example: sizing the components against a workload

Problem. A platform team is deploying Camunda 8 for a workload with these characteristics: a steady 60 process instances started per second; a worker fleet making about 900 job activations per second across all service tasks; heavy connector use, with roughly 40% of all service tasks being connector tasks; and 400 operators using *Operate* and 1,200 staff using *Tasklist*. The team’s first plan is to deploy “Camunda” on three identical nodes and split everything evenly. Explain why component-aware sizing is needed instead, and which components carry which part of this load.

Setup. We attribute the workload’s four characteristics to the specific components that bear them, and show why even-split sizing misjudges the platform.

Working. The load divides across components by their distinct jobs. The *brokers* carry the 60 instances per second of execution work and all the token movement it implies — this is stateful engine load, sized by broker count and partition count. The *gateway* carries the 900 job activations per second *plus* the 60 starts per second *plus* the operators’ and *Tasklist* users’ API traffic — because all client and worker requests pass through it; the gateway, though stateless, is on the path of the platform’s busiest traffic. The *Connector runtime* carries the connector share — on the order of

$$900 \times 0.40 = 360 \text{ connector job activations per second}$$

— and that is a substantial, distinct load on a component the even-split plan did not even name. *Operate* carries the 400 operators’ queries, and *Tasklist* the 1,200 staff’s worklist traffic — both are query load on the secondary store, not on the engine.

Discussion. The even-split plan — “deploy Camunda on three identical nodes” — is wrong not because three nodes is the wrong number but because “Camunda” is not one thing to split. The workload has at least five distinct loads, and they fall on five distinct components with five different scaling behaviours. The brokers’ 60-instances-per-second load is stateful and sized by partition count, a decision fixed at cluster creation. The gateway’s load is the largest single traffic stream — 900-plus activations a second — and the gateway is stateless, so it is sized by running enough copies, a completely different lever from the brokers’. The Connector runtime’s 360 activations a second is real load on a component the plan forgot exists, and an under-resourced runtime would bottleneck 40% of all service tasks. *Operate* and *Tasklist* place query load on the secondary store, which is its own infrastructure again. The worked example’s lesson is the chapter’s central one: a Camunda 8 platform

is a set of cooperating components, and sizing it means sizing each component against the part of the workload it actually bears. The team that knows the components can read this workload and place each load correctly; the team that treats “Camunda” as a monolith will over-provision some components, starve others — most likely the unnamed Connector runtime — and discover the imbalance only when a production load spike finds the weakest component. Component-aware knowledge is not academic; it is what makes a deployment correctly sized.

Practical next steps

For your institution’s Camunda 8 platform — or the one it is planning — list the components separately: brokers, gateway, Connector runtime, Operate, Tasklist, Identity, the secondary store. For each, ask what part of the workload it bears and how it is scaled. If the platform is sized as one undifferentiated block, some component is wrongly provisioned — and it is worth finding which before a load spike does.

Chapter in brief

A Camunda 8 platform is not one program but a set of cooperating components, each with its own job, scaling behaviour, and failure modes. Zeebe has three roles: stateful, partitioned brokers; a stateless routing gateway speaking gRPC and REST; and clients embedded in the institution’s own applications, scaling independently of the engine. Operate and Tasklist are query applications over the platform’s data — commonly a secondary store fed by an exporter, which makes their view current to within seconds. The Connector runtime is a distinct deployable component hosting connector logic and listening for inbound events. Identity, the Modeler, and Console complete the set. The whole is deployed by a Helm chart whose structure mirrors the components — and sizing the platform means sizing each component against the load it actually bears.

PART VI

Connectors in Depth

The Connector Model and the Connector Types

53.1 Why connectors deserve a part

Part V treated the components and mechanics of a Camunda platform, and connectors appeared there as one capability among many. This part returns to connectors and treats them in the depth a practitioner integrating a real financial platform needs — because connectors are how a process actually reaches the world.

A process, on its own, does nothing outside itself. Every time a Camunda process must call a core banking system, screen a customer against a sanctions list, send a notification, read a document store, or react to an external event, it does so through an *integration* — and the connector is Camunda’s reusable, configurable mechanism for that integration. An institution that understands connectors deeply integrates cleanly; an institution that does not falls back on ad-hoc, hand-coded integration for everything and loses the reusability and the consistency connectors were built to provide.

53.2 What a connector is

A connector, in Camunda, is a reusable integration component: a packaged, configurable way to connect a process to a particular kind of external system, which a process developer can drop into a model and configure, rather than writing integration code from scratch each time.

A connector has, in essence, two parts, and a practitioner must understand both. The first is the *connector function* — the actual code, typically Java, that does the integration: it takes the inputs the process supplies, talks to the external system, and returns a result. The second is the *element template* — a definition, expressed in JSON, of how the connector appears in the Modeler and how a process developer configures it: which fields the developer fills in, what they are called, how they are validated. The function is the behaviour; the element template is the user-facing surface.

This two-part structure is what makes a connector reusable. The function is written once. The element template means that every developer who uses the connector sees the same consistent, guided configuration surface — they do not need to know the integration’s internals, only how to fill in the template’s fields. A connector is, in this sense, an integration turned into a reusable building block with a friendly face.

53.3 Outbound and inbound connectors

The first and most important distinction among connectors is the *direction*: outbound versus inbound. A practitioner must know which kind a given integration needs, because they have different shapes and different lifecycles.

An *outbound* connector is used when something must happen in an external system *because*

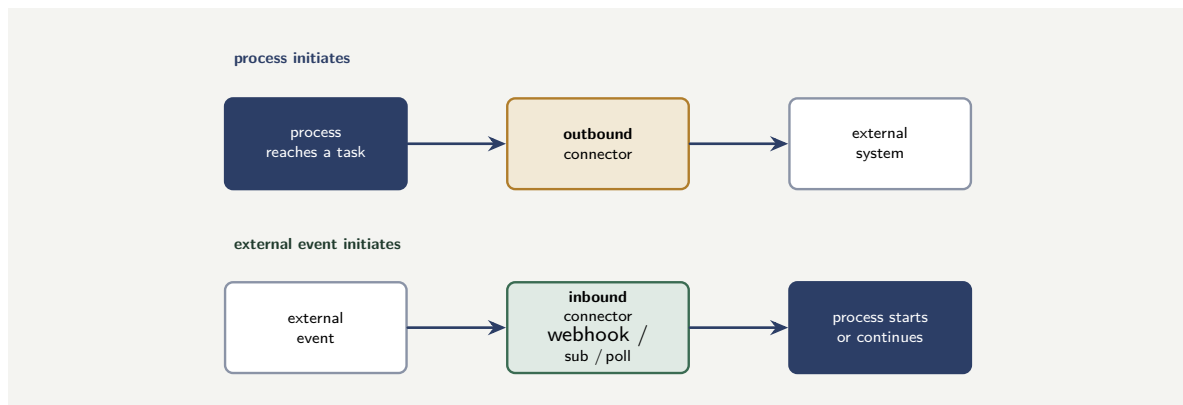


Figure 53.1. The two connector directions. An outbound connector fires when the process reaches a task and calls an external system; an inbound connector — webhook, subscription, or polling — reacts to an external event and starts or continues a process. The direction is set by who initiates.

the process reached a point. The process reaches a service task; the outbound connector fires; it calls the external system — a REST endpoint, a message to send, a record to write — and brings back a result. The process is the initiator; the external system is called. Most integrations a process makes are outbound.

An *inbound* connector is used the other way: when something must happen *in the process because of an external event*. An external system does something — a message arrives, a webhook is called, a queue receives an item — and the inbound connector causes a process to start, or a waiting process to continue. The external system is the initiator; the process reacts.

Inbound connectors come in distinct sub-types, by *how* they learn of the external event. A *webhook* connector exposes an endpoint that the external system calls. A *subscription* connector subscribes to a message queue. A *polling* connector periodically polls an external system for new data. The sub-type is chosen by how the external system makes its events available — does it push to a webhook, publish to a queue, or only expose data to be pulled?

Figure 53.1 contrasts outbound and inbound connectors.

53.4 Protocol connectors and the layered model

A second distinction is between connectors for a *specific service* and connectors for a *general protocol*. A connector for a specific service — a particular sanctions-screening provider, a particular payment network — is preconfigured for exactly that system. A *protocol connector* is generic: it implements a broad integration protocol — HTTP REST, SOAP, GraphQL — and can talk to anything that speaks that protocol.

The REST protocol connector deserves particular note, because it is the workhorse: an enormous fraction of modern integrations are REST API calls, and the REST connector handles them generically — and, as the next chapter shows, it is also the foundation many custom connectors are built on.

This gives Camunda’s connector model a useful *layering*, which a practitioner should hold in mind. At one layer are ready-made connectors for common systems. Below that are the generic protocol connectors. Below that is the Connector SDK for building entirely custom connectors. An institution integrating a new system should descend this layering only as far as it must: use a ready-made connector if one fits; otherwise a protocol connector; otherwise build a custom one. Building a custom connector is the right choice when nothing above it

fits — not the first move.

Tip

Camunda's connectors are layered, and an integration should descend the layers only as far as it must. First, a ready-made connector for the specific system, if one exists. Failing that, a generic protocol connector — REST, SOAP, GraphQL — configured for the target. Only when nothing above fits, a custom connector built on the Connector SDK. Building custom is a legitimate choice when the layers above genuinely do not fit — but it is the last resort, not the first, because every custom connector is code the institution must then own.

53.5 Connector governance

A connector is reusable integration, and reusable integration must be governed. Each connector the platform uses — REST, messaging, cloud, calendar — should be inventoried, its configuration reviewed, its credentials rotated, and its error behaviour understood. A connector deployed and forgotten is a connector whose credentials expire silently, whose endpoint changes without anyone updating the configuration, and whose error handling has never been tested under the conditions that will trigger it. Govern connectors as you govern any shared infrastructure component: inventoried, reviewed, and maintained.

Worked example: choosing the connector for four integrations

Problem. An institution is building a customer-onboarding process and must integrate with four external systems. (1) A public REST API for postcode lookup — a standard, well-documented REST service. (2) The institution's core banking system, which exposes only an old SOAP interface. (3) A specialised identity-verification SaaS product that is popular enough that Camunda offers a ready-made connector for it. (4) A bespoke internal fraud-scoring service with an unusual, non-standard binary protocol of the institution's own devising. For each, say which layer of the connector model to use.

Setup. We descend the connector layering — ready-made, protocol connector, custom SDK — for each of the four integrations, stopping at the first layer that fits.

Working. (1) *Postcode REST API*. It is a standard REST service; the generic **REST protocol connector** handles it directly, configured with the API's URL and parameters — no custom code. (2) *Core banking SOAP interface*. No ready-made connector for this institution's specific core system, but it speaks a standard protocol — SOAP — so the generic **SOAP protocol connector** fits. (3) *Identity-verification SaaS*. Camunda offers a **ready-made connector** for this specific product — the top layer fits, and it should be used: it is preconfigured and needs the least effort. (4) *Bespoke fraud service, non-standard protocol*. No ready-made connector, and no standard protocol — REST, SOAP, GraphQL — describes its unusual binary interface. Only here must the institution descend to the **Connector SDK** and build a custom connector.

ready-made : (3); protocol connector : (1), (2); custom SDK : (4).

Discussion. The worked example shows the connector layering doing its job: of four integrations, only *one* genuinely needs a custom connector, and the other three are served by layers that require no bespoke code at all. This is the discipline the chapter's tip states. A

team that did not think in layers might build custom connectors for all four — four bodies of integration code to write, test, and own forever — when in fact the postcode API is just REST, the core banking interface is just SOAP, and the identity SaaS already has a connector someone else maintains. Each layer descended is more effort and more ownership, so the right move is always to stop at the highest layer that fits: a ready-made connector if one exists, a protocol connector if the system speaks a standard protocol, and the SDK only for the genuinely unusual case — here, the bespoke fraud service with its non-standard binary protocol, which nothing above the SDK can handle. The lesson is that “we need to integrate with X” should never lead straight to “so we build a connector for X.” It should lead to a descent through the layers, and for most integrations — as for three of these four — the descent stops well before any custom code is written. Building custom is for the genuine last case, and recognising which integrations are *not* that case is most of integrating a platform efficiently.

Practical next steps

For each external system your institution’s processes integrate with, identify which connector layer it uses — ready-made, protocol connector, or custom. If custom connectors have been built where a protocol connector would have served, the institution is owning integration code it did not need to write. The layering exists so that custom code is the exception; check that it is being treated as one.

Chapter in brief

Connectors are how a Camunda process reaches the world, and they deserve depth. A connector is a reusable integration component with two parts: the connector function — the Java code that does the integration — and the element template — the JSON definition of how it is configured in the Modeler. The first distinction is direction: *outbound* connectors fire when the process reaches a task and call an external system; *inbound* connectors — webhook, subscription, or polling — react to an external event and start or continue a process. The second is generality: ready-made connectors for specific systems, generic protocol connectors (REST, SOAP, GraphQL), and the Connector SDK for custom ones. The layers should be descended only as far as an integration genuinely requires.

The Key Connectors in Practice

54.1 The connectors a financial platform leans on

The previous chapter set out the connector model; this chapter treats the specific connectors a banking or insurance platform actually uses most, and the practical disciplines each one demands. The point is not to catalogue every connector — the set grows, and the Camunda Marketplace adds more — but to treat the recurring ones with enough depth that a practitioner uses them well.

54.2 The REST connector: the workhorse

The *REST connector* is, for most platforms, the connector used more than any other, because most modern systems — internal services, SaaS products, public APIs — expose REST interfaces. It is an outbound protocol connector: the process supplies a URL, a method, headers, and a body; the connector makes the HTTP request; the response comes back into the process.

The disciplines the REST connector demands are the disciplines of any remote call, and a financial platform must apply them. A REST call can be *slow* — so the connector's timeout must be set deliberately, not left to chance. A REST call can *fail transiently* — so retry behaviour matters, and the connector runtime supports configurable retries, including a retry backoff so that attempts are spaced rather than hammering a struggling service. A REST call returns *errors* — and the process must decide which response codes mean success, which mean a retryable failure, and which mean a business error that should be handled in the model. The REST connector makes the *call* easy; the practitioner still owns the *discipline* around the call.

54.3 Messaging, cloud, and human-channel connectors

Beyond REST, a financial platform leans on several recurring categories.

Messaging connectors integrate the process with the event backbone of [Chapter 25](#) — queues and topics. An outbound messaging connector publishes an event; an inbound subscription connector consumes one, starting or continuing a process. These are how a process participates in the institution's asynchronous, event-driven fabric rather than only making synchronous calls.

Cloud-service connectors integrate with the managed services of the cloud platform — a function-as-a-service, a managed queue, an object store, a cloud AI service. For an institution on a particular cloud, these connectors turn the cloud's services into things a process can use directly.

Human-channel connectors integrate the process with the channels people actually use — email, and messaging platforms for notifications and simple interactions. These are how a process communicates outward to customers and staff, distinct from the human *task* work that flows through Tasklist.

The practitioner's discipline across all of these is the same and worth stating plainly: a connector makes an integration *easy to invoke*, but it does not make the integration *free of con-*

sequence. Every connector call is a call to a system that can be slow, can fail, can be unavailable — and the process model must be designed for those realities, with timeouts, retries, error handling, and the asynchronous patterns of [Chapter 25](#), exactly as it would be for any integration. The connector removes the boilerplate, not the engineering judgement.

A connector makes an integration easy to invoke — it does not make the integrated system reliable. Every connector call reaches a system that can be slow, fail transiently, or be unavailable, and the process model must be designed for that: deliberate timeouts, configured retries with backoff, explicit handling of error responses, and asynchronous patterns where the called system is slow or flaky. A platform that treats connector calls as instant and infallible because the connector made them easy to add has mistaken convenience for reliability.

54.4 A worked connector: calendar integration end to end

The categories above are easier to grasp through one connector built out in full. A recurring need in banking and insurance processes is *calendar integration* — booking an appointment for a customer: a mortgage advisor meeting, a claims assessment visit, an account-review call. The two calendar systems an institution almost always faces are *Google Calendar* and *Microsoft Outlook* (the Microsoft 365 calendar, reached through the Microsoft Graph API). This section shows a complete integration with each — Google Calendar from a Java job worker, Outlook from a Node.js job worker — so the connector category becomes concrete code rather than description.

Google Calendar, from a Java worker

A process step that books an appointment is a service task whose job a worker fulfils. Listing 54.1 is the worker in full: it authenticates to Google with a service account, builds a calendar event from the process variables, inserts it into the calendar, and returns the created event's identifier to the process.

Listing 54.1. A complete Google Calendar job worker in Java: it authenticates with a service account, builds an event from process variables, inserts it, and returns the event id.

```
1  @Component
2  public class CalendarBookingWorker {
3
4      private final Calendar calendar;
5
6      public CalendarBookingWorker() throws Exception {
7          // authenticate with a Google service account
8          GoogleCredentials creds = GoogleCredentials
9              .fromStream(new FileInputStream(
10                  "service-account.json"))
11              .createScoped(List.of(
12                  "https://www.googleapis.com/auth/calendar"));
13
14          this.calendar = new Calendar.Builder(
15              GoogleNetHttpTransport.newTrustedTransport(),
16              GsonFactory.getDefaultInstance(),
17              new HttpCredentialsAdapter(creds))
18              .setApplicationName("camunda-platform")
19              .build();
20      }
21  }
```

```

22  @JobWorker(type = "book-appointment")
23  public Map<String, Object> book(ActivatedJob job)
24      throws Exception {
25      // build the event from process variables
26      Event event = new Event()
27          .setSummary(job.getVariable("appointmentTitle"))
28          .setLocation(job.getVariable("branchName"))
29          .setDescription(job.getVariable("caseReference"));
30
31      EventDateTime start = new EventDateTime()
32          .setDateTime(new DateTime(
33              (String) job.getVariable("startTime")));
34      EventDateTime end = new EventDateTime()
35          .setDateTime(new DateTime(
36              (String) job.getVariable("endTime")));
37      event.setStart(start).setEnd(end);
38
39      // the customer and advisor as attendees
40      event.setAttendees(List.of(
41          new EventAttendee().setEmail(
42              job.getVariable("customerEmail")),
43          new EventAttendee().setEmail(
44              job.getVariable("advisorEmail"))));
45
46      // insert into the calendar
47      Event created = calendar.events()
48          .insert("primary", event)
49          .execute();
50
51      // return the reference for the process to carry
52      return Map.of(
53          "calendarEventId", created.getId(),
54          "eventStatus",    created.getStatus());
55  }
56  }

```

The worker is long only because it is complete; its shape is the familiar one. It authenticates once, in the constructor, with a *service account* — a machine identity, its credentials held in a secret, exactly the discipline of [Chapter 26](#). Each job then builds an *Event* from process variables, inserts it, and — the point that matters — returns only the *calendarEventId*, a reference, into the process. The process carries the pointer to the appointment, not a copy of the calendar entry; if a later step must reschedule or cancel, it has the id to do so.

Microsoft Outlook, from a Node.js worker

The same booking against *Microsoft Outlook* goes through the *Microsoft Graph* API. Listing 54.2 is the equivalent worker in Node.js: it authenticates with the Microsoft identity platform, constructs the event, posts it to the user's calendar through Graph, and returns the event id.

Listing 54.2. A complete Microsoft Outlook job worker in Node.js: it authenticates to Microsoft Graph, builds the event, posts it to the calendar, and returns the event id.

```

1  const { Camunda8 } = require('@camunda8/sdk');
2  const { ClientSecretCredential } = require(
3      '@azure/identity');
4  const { Client } = require(
5      '@microsoft/microsoft-graph-client');
6  require('isomorphic-fetch');
7
8  // authenticate to Microsoft Graph with an app identity
9  const credential = new ClientSecretCredential(
10     process.env.MS_TENANT_ID,
11     process.env.MS_CLIENT_ID,
12     process.env.MS_CLIENT_SECRET);

```

```

13
14 const graph = Client.initWithMiddleware({
15   authProvider: {
16     getAccessToken: async () => (
17       await credential.getToken(
18         'https://graph.microsoft.com/.default')).token,
19   },
20 });
21
22 const camunda = new Camunda8();
23 const zeebe = camunda.getZeebeGrpcApiClient();
24
25 zeebe.createWorker({
26   taskType: 'book-appointment',
27   taskHandler: async (job) => {
28     const v = job.variables;
29
30     // build the Outlook event
31     const event = {
32       subject: v.appointmentTitle,
33       location: { displayName: v.branchName },
34       body: { contentType: 'text',
35         content: v.caseReference },
36       start: { dateTime: v.startTime,
37         timeZone: 'UTC' },
38       end: { dateTime: v.endTime,
39         timeZone: 'UTC' },
40       attendees: [
41         { emailAddress: { address: v.customerEmail },
42           type: 'required' },
43         { emailAddress: { address: v.advisorEmail },
44           type: 'required' },
45       ],
46     };
47
48     // post it to the advisor's calendar via Graph
49     const created = await graph
50       .api(`/users/${v.advisorEmail}/events`)
51       .post(event);
52
53     // complete the job, returning the event reference
54     return job.complete({
55       calendarEventId: created.id,
56       eventStatus:      created.responseStatus.response,
57     });
58   },
59 });

```

The Node worker is the same integration against the other calendar system. It authenticates with an *application identity* through the Microsoft identity platform — the Outlook counterpart of the Google service account — builds the event from the job’s variables, posts it through the Graph API, and completes the job carrying back the `calendarEventId`. The two workers, Java-against-Google and Node-against-Outlook, are deliberately parallel: the same job type, book-appointment; the same variables in; the same reference out. A process with a “book appointment” service task does not know or care which calendar system, or which worker, is behind it — the anti-corruption discipline of [Chapter 25](#) applied to the calendar integration.

Tip

When integrating a calendar — Google or Outlook — have the worker return only the *event identifier* to the process, never a copy of the calendar entry. The calendar system is

the system of record for the appointment; the process carries the id so a later step can reschedule or cancel, and nothing more. And authenticate with a *machine identity* — a Google service account, a Microsoft application registration — with its credentials in a secret store, never a real person’s calendar login embedded in the worker.

Worked example: the connector that hammered a struggling service

Problem. A bank’s process uses a REST connector to call an internal credit-reference service on every loan application — about 3,000 calls an hour at peak. The connector was added with default settings: no explicit timeout, and retries that fire *immediately* on failure. One afternoon the credit-reference service becomes slow and starts failing intermittently under load. Within minutes, its operators report it has gone from degraded to completely down. Explain how the connector configuration turned a degradation into an outage.

Setup. We trace what the default retry behaviour does when the called service is already struggling.

Working. The credit-reference service is degraded — slow, failing some requests. Each failed call from the bank’s process triggers the connector’s retry, and the retries fire *immediately*, with no backoff. So a failing call becomes, in quick succession, a second call, a third, a fourth. With 3,000 calls an hour already, and each failure now multiplying into rapid retries, the load on the struggling service *increases* sharply at exactly the moment it can least handle it:

degraded service + immediate retries → more load → harder failure → more retries.

The retries form a feedback loop — a retry storm — that drives the degraded service the rest of the way down.

Discussion. The connector did not fail; it did exactly what it was configured to do — and that is the worked example’s point. The REST connector made calling the credit-reference service trivially easy, and the team, having added it with default settings, never applied the discipline a remote call requires. The default of immediate retries is benign when a failure is a one-off blip, but actively dangerous when a service is degraded, because immediate retries multiply load precisely when the service needs load to *drop* — the classic retry storm, where the caller’s well-meaning retries finish off the service. The fixes are exactly the disciplines the chapter named: a retry *backoff*, so that attempts are spaced — thirty seconds apart, say — rather than immediate, which gives a struggling service room to recover; a deliberate *timeout*, so a slow call fails cleanly instead of hanging; and a sensible *cap* on retries. None of this is exotic — it is ordinary remote-call engineering — but the ease of adding the connector let the team skip it, and “the connector made it easy” quietly became “nobody designed the integration.” The lesson is the chapter’s warning made concrete: a connector removes the boilerplate of *making* a call, but the engineering of calling a fallible system well — timeouts, spaced retries, error handling — is exactly as necessary as it ever was, and a platform that skips it because the connector was easy to drop in has built a retry storm waiting for the first bad afternoon.

Practical next steps

Audit the connector calls in your institution's processes for their timeout and retry configuration. Any connector left on default immediate-retry behaviour is a retry storm waiting to happen the next time the called service degrades. Configure deliberate timeouts and spaced retry backoff — the connector made the call easy, but only you can make it safe.

Chapter in brief

A financial platform leans on a recurring set of connectors. The REST connector is the workhorse, because most systems expose REST — and it demands the disciplines of any remote call: deliberate timeouts, configured retries with backoff, and explicit error handling. Messaging connectors integrate the process with the event backbone — publishing and consuming queue events; cloud-service connectors turn a cloud's managed services into things a process can use; human-channel connectors reach people through email and messaging. The discipline common to all is that a connector makes an integration easy to invoke but does not make the integrated system reliable — timeouts, retries, and error handling remain the practitioner's responsibility.

CHAPTER 55

Building a New Connector

55.1 When the platform must be extended

The connector layering of [Chapter 53](#) ended with a clear rule: build a custom connector only when nothing above it fits. This chapter is for exactly that case. When a financial institution must integrate with a system that has no ready-made connector and speaks no standard protocol — a bespoke internal service, a niche industry system, a partner’s idiosyncratic interface — it builds its own connector, and this chapter sets out how, and how to do it well.

Building a connector is, done properly, not a one-off hack but the creation of a *reusable asset*. The institution that builds a connector for its core-banking system once, well, gives every future process a clean, consistent, governed way to reach that system. The chapter’s framing is that a custom connector is worth building as a *first-class component*, not improvised.

55.2 The two things a connector is made of

[Chapter 53](#) said a connector has two parts; building one means building both, and a practitioner should hold them clearly distinct.

The first is the *connector function* — the code. Camunda provides a *Connector SDK* for this: a custom outbound connector is, at heart, a Java class implementing the SDK’s connector interface, with one method that receives the inputs, does the integration work — the actual call to the external system — and returns the result. An inbound connector is similar in spirit but has a lifecycle suited to long-running listening — waiting on a webhook, holding a queue subscription — rather than a single call-and-return. The SDK provides the structure; the institution writes the integration logic specific to its system.

The second is the *element template* — the connector’s face in the Modeler. This is a JSON document that declares the connector’s properties: what a process developer must supply to use the connector, what each field is called, how it is grouped and validated, and — critically — the *binding* that ties the template to the connector function, so the engine knows which connector code to run for a task using this template. The element template is what turns raw connector code into something a process developer can use through a guided panel rather than by knowing the code’s internals.

A good practice the SDK supports: the element template can be *generated* from the connector’s input definition, so the two parts stay consistent rather than drifting — the template always reflects what the function actually expects.

Figure [55.1](#) draws the two parts of a connector and how they relate.

55.3 Build on a protocol connector where you can

One practice deserves its own emphasis, because it saves a great deal of effort and is easy to miss: a custom connector need not be built from nothing. If the system to integrate with is reached over a standard protocol — most often REST — the custom connector can be built *on top* of the REST connector.

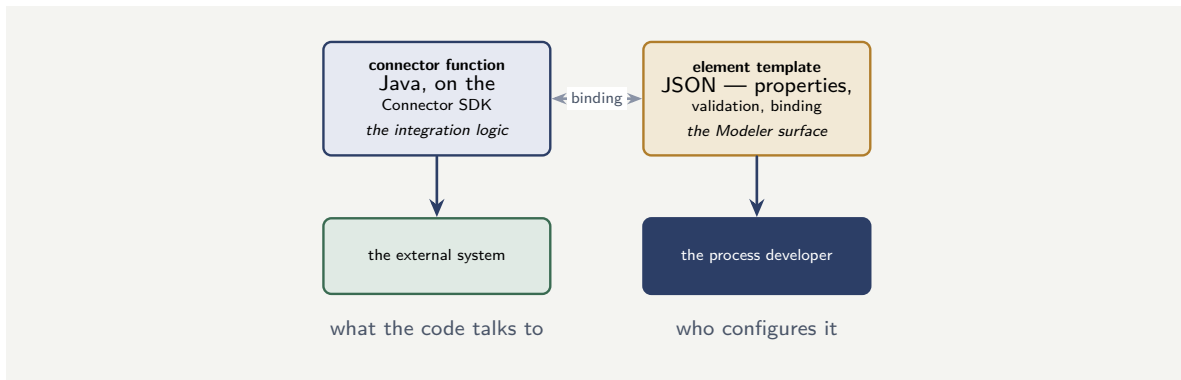


Figure 55.1. The two parts of a connector. The connector function — Java on the Connector SDK — holds the integration logic that talks to the external system. The element template — JSON — is the Modeler surface the process developer configures. The binding ties them together; generating the template from the function’s inputs keeps them consistent.

The pattern is this: rather than writing the HTTP handling, the custom connector reuses the REST connector’s protocol machinery and adds only the *service-specific* part — the particular endpoints, the particular authentication, the particular request and response shapes of the target system, and an element template that presents the developer with that system’s concepts rather than raw REST fields. The result is a connector that *feels* purpose-built for the system — a developer configures it in the system’s own terms — but did not cost the effort of building protocol handling from scratch.

This is the connector-building equivalent of the layering discipline: even when building custom, build on the highest foundation that fits. A custom connector from the bare SDK is for the genuinely non-standard system; a custom connector *on the REST connector* is the right, lighter choice whenever the target speaks REST but deserves a friendlier, preconfigured face than the generic REST connector gives.

55.4 A connector is a governed component

A built connector is code the institution now owns, and it must be treated with the discipline of any owned component — this is the chapter’s closing point.

A connector has an owner. It is versioned — and because processes depend on its element template, a change to that template is a change to an interface many processes rely on, to be versioned and managed as carefully as any contract. It is tested. It is documented, so the developers who use it know what it does and what it expects. And it is, ideally, shared — a connector built once for the institution’s core banking system should be *the* way every process reaches that system, not one of several competing integrations.

A connector built and then left ungoverned — unversioned, untested, owned by no one, known only to its original author — becomes exactly the kind of fragile, mysterious integration that connectors were meant to replace. The value of building a connector is realised only if the connector is governed as the reusable, first-class component it was built to be.

Tip

A custom connector is code the institution owns — govern it as a first-class component. It has an owner, a version, tests, and documentation. Its element template is an interface

that processes depend on, so changes to it are interface changes, versioned and managed with care. And it should be shared and canonical — the one way every process reaches that system. A connector built well but left ungoverned becomes the fragile, mysterious integration connectors exist to replace.

55.5 Connector governance

A connector is reusable integration, and reusable integration must be governed. Each connector the platform uses — REST, messaging, cloud, calendar — should be inventoried, its configuration reviewed, its credentials rotated, and its error behaviour understood. A connector deployed and forgotten is a connector whose credentials expire silently, whose endpoint changes without anyone updating the configuration, and whose error handling has never been tested under the conditions that will trigger it. Govern connectors as you govern any shared infrastructure component: inventoried, reviewed, and maintained.

Worked example: build from scratch, or on the REST connector

Problem. An institution must build a custom connector for a partner insurer's claims-data service. The service is reached over a standard REST/JSON interface, but it has a dozen specific endpoints, its own API-key authentication scheme, and request and response shapes particular to the claims domain. A developer proposes to build the connector from the bare Connector SDK, writing all the HTTP handling directly. A colleague proposes building it on top of the REST connector instead. Compare the two approaches.

Setup. We consider what each approach has to build, given that the partner service speaks standard REST.

Working. *From the bare SDK.* The developer writes everything: the HTTP request construction, the connection handling, timeout and retry behaviour, response parsing — *plus* the service-specific part, the dozen endpoints and the claims-domain shapes. The generic HTTP machinery is built from scratch even though the partner service speaks ordinary REST:

build = HTTP protocol handling + service-specific logic.

On the REST connector. The partner service speaks standard REST, so the REST connector's protocol machinery — HTTP handling, timeouts, retries — is reused. The developer builds only the service-specific part: the endpoints, the API-key authentication, the claims-domain request and response shapes, and an element template presenting the partner's concepts:

build = service-specific logic only.

Discussion. The colleague is right, and the worked example applies the chapter's central building practice. The partner claims service is not a non-standard system — it speaks ordinary REST/JSON — and that fact is decisive: the only thing genuinely particular about it is the *service-specific* layer, the dozen endpoints, the API-key scheme, the claims-domain shapes. Building from the bare SDK means re-implementing all the generic HTTP machinery — request construction, timeouts, retries, parsing — that the REST connector already provides, tested and maintained, which is wasted effort and a fresh body of protocol code for the institution to own. Building on the REST connector reuses that machinery and confines the work

to the part that is actually unique. The result is the same from the process developer's point of view — a connector that presents the partner's claims service in its own terms, configured through a guided template — but it cost a fraction of the effort and carries far less owned code. The deeper point is that the layering discipline does not stop applying once an institution has decided to build custom: even within “build a custom connector,” there is a choice between the bare SDK and building on a protocol connector, and the same rule holds — build on the highest foundation that fits. The bare SDK is for the genuinely non-standard system; a partner service that speaks REST should have its custom connector built on the REST connector, every time.

Practical next steps

If your institution is building a custom connector, first establish whether the target system speaks a standard protocol. If it speaks REST — as most modern services do — build the connector on top of the REST connector, not from the bare SDK, and confine your effort to the service-specific layer. And whatever you build, govern it: an owner, a version, tests, documentation, and a place as the canonical way to reach that system.

Chapter in brief

A custom connector is built only when no ready-made or protocol connector fits, and it should be built as a reusable, first-class component. It has two parts: the connector function — Java on the Connector SDK, holding the integration logic — and the element template — a JSON definition of the Modeler-facing properties, tied to the function by a binding, ideally generated from the function's inputs so the two stay consistent. Where the target system speaks a standard protocol such as REST, the custom connector should be built *on top of* the REST connector, reusing its protocol machinery and adding only the service-specific layer. And a built connector is owned code: it needs an owner, versioning, tests, documentation, and a place as the canonical integration — ungoverned, it becomes the fragile integration connectors exist to replace.

PART VII

Infrastructure

Where the Platform Runs: Deployment and Topology

56.1 The platform is also infrastructure

Every part so far has treated Camunda as software — processes, components, code, data. This part treats it as *infrastructure*: a running system that occupies machines, consumes resources, must be installed, sized, scaled, backed up, and kept available. The distinction matters because a platform that is architecturally sound and operationally well-run can still fail if its infrastructure is wrong — under-provisioned, poorly laid out, or unable to survive the loss of a machine. A bank’s hyperautomation platform is, in the end, something running on computers, and this part treats the computers.

This first chapter treats *where* and *how* the platform runs: the deployment model, the topology of its parts, and the decisions a platform team must make before the first process is ever deployed.

56.2 Self-managed and the managed service

The first infrastructure decision is whether the institution runs the platform itself — *self-managed* — or consumes it as a managed cloud service. The manuscript does not prescribe one; it sets out what the choice means.

The managed service removes the infrastructure burden: the provider runs the engine, the cluster, the secondary store, the upgrades, the backups. The institution builds processes and workers and consumes the platform through its APIs. This is the lighter operational path, and for many institutions it is the right one — the institution’s scarce platform engineers spend their effort on processes, not on operating clusters.

Self-managed means the institution runs the platform on its own infrastructure — its own cloud account, or its own data centre. It is the heavier path: the institution’s team installs, sizes, scales, patches, backs up, and keeps the platform available. Banking and insurance institutions often have reasons to choose it nonetheless — data-residency requirements that mandate where process data physically sits, security policies that require the platform inside the institution’s own network perimeter, regulatory expectations about control of critical systems. The choice is frequently made *for* the platform team by the institution’s regulatory and security posture, and this part is written mostly for the self-managed case, because it is the case with infrastructure to discuss.

56.3 Kubernetes as the home of a self-managed platform

A self-managed Camunda 8 platform runs, in the overwhelmingly common and recommended case, on *Kubernetes*. Kubernetes is the container orchestration platform that runs the platform’s many parts — the engine brokers, the gateway, the web components, the connectors,

the institution’s own worker applications — as managed, scheduled, health-checked workloads.

The reason Kubernetes is the recommended home is worth stating, because it connects to [Chapter 24](#)’s cloud principles. A hyperautomation platform is not one process on one machine; it is a dozen cooperating components, each needing to be deployed, scaled, restarted on failure, and connected. Kubernetes is the layer that does exactly that — schedules the components onto machines, restarts a component that dies, scales a component that is loaded, and gives the components a stable way to find one another. Doing all of that by hand, on bare machines, is possible and is how platforms were run before; doing it on Kubernetes is how a modern self-managed platform is run, and the institution’s existing Kubernetes practice — most large institutions have one — applies directly.

The platform is installed onto Kubernetes through *Helm* — a packaging mechanism that installs the whole set of cooperating components, configured together, from one published chart. The platform team does not install a dozen components by hand and wire them together; it configures one Helm chart and installs the set. The chart is the platform’s infrastructure expressed as a configurable, versioned artefact — which is itself the discipline this part will return to: infrastructure as something declared and governed, not assembled by hand.

56.4 The topology: what runs where

A Camunda platform is not one thing but a topology of parts, and a platform team must understand the topology to size it, secure it, and operate it.

At the centre is the *orchestration cluster* — the Zeebe engine brokers and the gateway through which clients reach them. This is the platform’s beating heart, and it has a property the rest of the topology does not: it is *stateful*. The brokers hold the live execution state of every running process instance, and so they cannot be treated as interchangeable, disposable units the way a stateless component can. They need stable identity and persistent storage — a point the next section develops.

Around the cluster are the *stateless components*: the web applications of [Chapter 28](#) — Operate, Tasklist — the connector runtime, and the institution’s own worker applications. These are stateless in the sense that matters for infrastructure: any instance of them is as good as any other, one can be killed and replaced freely, and they are scaled simply by running more of them.

Outside the cluster sits the *secondary store* — the Elasticsearch or OpenSearch of [the data architecture chapter](#). It is its own substantial piece of infrastructure, with its own sizing, its own storage, its own scaling, and the disciplined topology places it outside the orchestration cluster — often as a separately-managed datastore — precisely because its heavy, analytical load should not share machines with the latency-sensitive engine.

Figure [56.1](#) draws the topology — stateful core, stateless components around it, secondary store outside.

56.5 Stateful and stateless: why the distinction governs everything

The single most important infrastructure idea in this chapter is the distinction the topology section drew: the orchestration cluster is *stateful*; almost everything else is *stateless*. Nearly

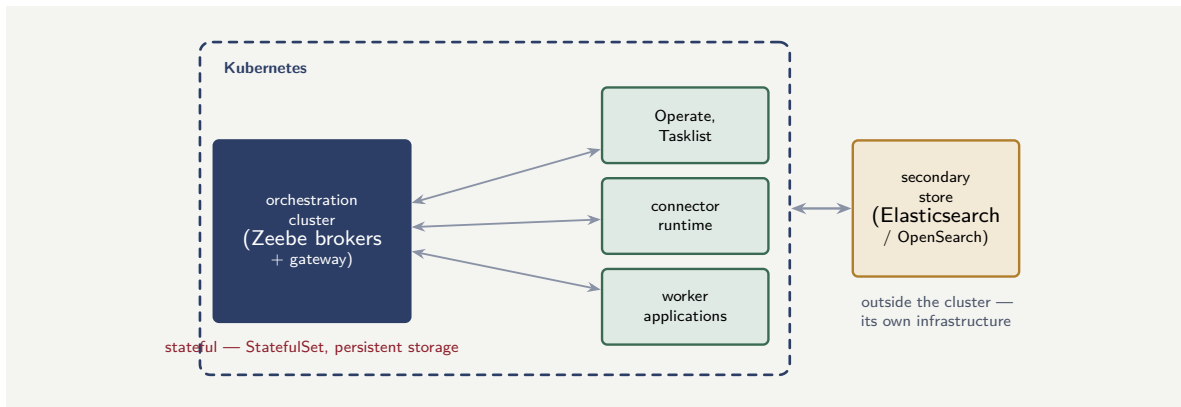


Figure 56.1. The topology of a self-managed Camunda platform. The stateful orchestration cluster and the stateless components — web applications, connectors, workers — run on Kubernetes; the secondary store sits outside, as its own infrastructure, so its heavy analytical load does not share machines with the latency-sensitive engine.

every infrastructure decision flows from it.

A stateless component is easy infrastructure. It holds nothing that matters; it can be killed and replaced freely; it is scaled by running more copies; a new copy needs no data to start. Operate, the connector runtime, the worker applications — these are scaled, redeployed, and recovered with little ceremony, because losing one loses nothing.

The stateful orchestration cluster is demanding infrastructure. Its brokers hold the live state of every running process; a broker is not interchangeable with a fresh empty one; and so the brokers need *persistent storage* that survives a broker restarting, and *stable identity* so a restarted broker rejoins as the broker it was. In Kubernetes terms, the brokers run as a *StatefulSet* with persistent volumes — not as the ordinary, disposable Deployment the stateless components use. And the next chapter’s whole subject — scaling, replication, backup — is demanding precisely because the cluster is stateful: you cannot back up, or carelessly scale, a thing that holds live state the way you can a thing that holds nothing.

The orchestration cluster’s brokers hold the live state of every running process instance. They must run on *persistent storage* that survives a restart, and that storage must not be configured to be reclaimed and deleted when a broker’s volume claim is removed — a default in some Kubernetes distributions. A broker brought back with empty storage has lost the execution state it held: running process instances, gone. The persistence configuration of the stateful cluster is not a detail; it is the line between a restart and a data-loss incident.

56.6 Infrastructure as a team discipline

Infrastructure management is not a solo activity; it is a team discipline. The values file is under version control, changes are reviewed, and deployments go through a pipeline. A single engineer making ad-hoc changes to the running cluster is a single point of failure and a governance gap. The infrastructure discipline this chapter describes works only if it is a *team* discipline: every change reviewed by a second pair of eyes, every deployment traceable to a reviewed commit, every incident followed by a blameless review that improves the process.

Worked example: sizing the first cluster

Problem. A bank is deploying its first self-managed Camunda platform. The platform team must choose the orchestration cluster's size. They expect a steady-state load of 40 process instances started per second, with daily peaks around 110 per second, and they need the platform to survive the loss of any single broker without interruption. They are choosing between a single-broker cluster and a three-broker cluster with a replication factor of three. Reason about which meets the requirement, and what the single-broker option would actually cost.

Setup. The requirement has two parts: enough capacity for the peak load, and survival of a single broker's loss. We assess each option against both.

Working. *Single-broker cluster.* One broker holds all the platform's state and does all its processing. On capacity, a single broker may or may not carry the 110-per-second peak — but on *resilience* it fails the requirement outright: there is no second broker, so the loss of the one broker is the loss of the entire platform and, if its storage is damaged, the loss of all in-flight process state. Survival of a single broker's loss:

not possible — the one broker is a single point of failure.

Three-broker cluster, replication factor three. The platform's state is partitioned across the brokers and each partition is replicated to all three. The loss of any one broker leaves its partitions' data intact on the other two, and the cluster continues:

survives the loss of one broker — the requirement is met.

The three brokers also share the processing load, comfortably carrying the 110-per-second peak that a single broker might not.

Discussion. The three-broker cluster is not merely the better option; it is the only one that meets the stated requirement, and the worked example exists to make a point about how infrastructure requirements are really decided. The bank's requirement included "survive the loss of any single broker" — and that single clause eliminates the single-broker cluster completely, before capacity is even discussed, because a single broker *is* a single point of failure and no amount of making it a powerful broker changes that. This is the character of resilience requirements: they are not satisfied by a bigger version of a non-resilient design; they are satisfied only by a design that has no single point of failure, which means replication, which means more than one broker. The three-broker, replication-three configuration is the standard small production cluster for exactly this reason — it is the smallest topology that holds every partition's data in more than one place and so survives a broker's loss. The single-broker option is appropriate for a developer's laptop and nowhere else; choosing it for a bank's production platform would be choosing, knowingly or not, that the loss of one machine is the loss of the bank's running processes. Infrastructure sizing begins with the resilience requirement, because resilience is the property that a design either structurally has or structurally lacks — capacity can be added to a running platform, but a single point of failure can only be designed out.

Practical next steps

For your institution's platform — or the one it is planning — find out the orchestration cluster's broker count and replication factor. A production cluster should have enough brokers, and a replication factor, to survive the loss of at least one broker with no data loss and no interruption. If it is a single broker, it has a single point of failure, and that is an infrastructure decision worth raising before it is tested by a real failure.

Chapter in brief

A hyperautomation platform is also infrastructure — a running system on machines that must be installed, sized, and kept available. The first decision is self-managed versus a managed service; banking institutions often choose self-managed for data-residency and security reasons. A self-managed Camunda 8 platform runs on Kubernetes, installed through a Helm chart that packages its many components. The topology has a stateful core — the orchestration cluster of Zeebe brokers, which needs persistent storage and stable identity, run as a StatefulSet — surrounded by stateless components and an external secondary store. The stateful/stateless distinction governs every infrastructure decision, and a production cluster needs enough brokers and replication to survive the loss of one.

Scaling, Resilience, and Backup

57.1 Keeping the platform up and right-sized

The previous chapter established where the platform runs and the stateful/stateless distinction. This chapter treats the three infrastructure disciplines that keep it running well over time: *scaling* it to its load, making it *resilient* to failure, and *backing it up* against disaster. These are the operational realities of a platform that is not a demonstration but a system a bank depends on, every day, for years.

57.2 Scaling the stateless and the stateful

Scaling a Camunda platform means scaling its parts to their load — and because of the previous chapter’s distinction, the stateless parts and the stateful core scale very differently.

The stateless components scale *easily and elastically*. The connector runtime, the worker applications, the web components — under load, run more copies; under light load, run fewer. This is the elastic scaling of [Chapter 24](#), and for the stateless parts it is straightforward: a new copy needs no state to start, so scaling up is just starting more, and Kubernetes can even do it automatically in response to load. The institution’s worker fleet, in particular, is scaled this way — [Chapter 32](#)’s fleet sizing is realised by running the right number of stateless worker copies.

The stateful orchestration cluster scales *deliberately and with constraints*. Its capacity is set by its broker count and how its work is *partitioned* across those brokers — [Chapter 27](#)’s partitioning. And here is the constraint a platform team must know in advance: the *partition count is fixed when the cluster is created* and cannot be changed later by ordinary scaling. The number of partitions is a decision made at the cluster’s birth, sized for the load the platform is expected to grow into — because, unlike adding stateless worker copies, it is not a dial that can simply be turned up on a running cluster. A platform team that sizes the partition count only for today’s load, with no headroom, has built a ceiling into the platform that a future growth will hit.

Tip

The stateless components of a Camunda platform scale elastically — run more copies under load. But the stateful cluster’s partition count is fixed at cluster creation and cannot be raised by ordinary scaling afterwards. Size the partition count at the start for the load the platform is expected to grow into over its life, with real headroom — it is not a dial that can be turned up on a Friday when load rises. The partition count is a long-horizon decision made once.

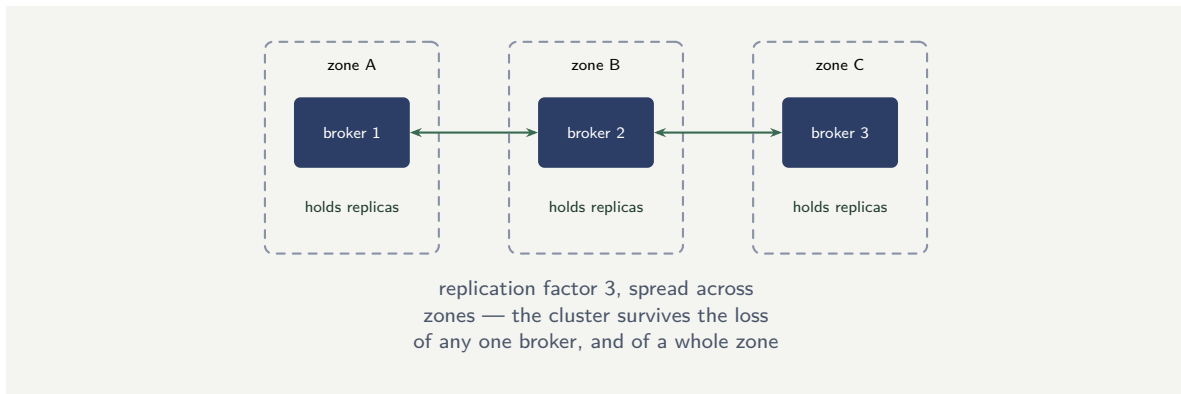


Figure 57.1. A resilient stateful cluster. Three brokers, one per data-centre zone, with each partition replicated to all three. The loss of any one broker loses no data; the loss of a whole zone loses one broker's worth, which replication covers — the platform continues.

57.3 Resilience: surviving the loss of a part

Resilience is the property that the platform survives the failure of one of its parts — and *something always eventually fails*: a machine, a disk, a network link, a whole data-centre zone. A resilient platform is not one in which nothing fails; it is one in which a failure does not become an outage.

For the stateless components, resilience is simple, and it is the same property as scaling: run more than one copy. If two copies of the connector runtime are running and one dies, the other carries the load while Kubernetes starts a replacement. A stateless component run as a single copy is a single point of failure for no reason — the fix costs only the running of a second copy.

For the stateful cluster, resilience is *replication*, and it is the reason [Chapter 27](#)'s replication factor exists. Each partition's data is held not on one broker but on several — a replication factor of three means three brokers hold each partition. The loss of one broker then loses no data and stops nothing: the partitions it held are intact on the other brokers, and the cluster continues. The replication factor is precisely the number of broker failures the cluster can survive, and for a bank's production platform it is chosen so the platform survives the failures that realistically happen — a single broker certainly, and ideally a whole zone.

This connects to a topology decision worth naming: *zone distribution*. A cluster whose three brokers all run in one data-centre zone survives the loss of a broker but not the loss of the zone. A cluster whose brokers are spread across zones survives a whole zone's loss. For a critical banking platform, spreading the stateful cluster across zones is how resilience is extended from "survives a machine" to "survives a data-centre incident."

Figure 57.1 draws the zone-distributed, replicated cluster — resilience to a machine and to a zone.

57.4 Backup: the defence of last resort

Replication protects against the loss of a *broker*. It does not protect against everything: a software fault that corrupts data, an operator error that deletes the wrong thing, a disaster that takes the whole region. Against those, the defence is a *backup* — a consistent, restorable copy of the platform's state, taken regularly and stored somewhere separate.

A Camunda platform's backup has two parts, because its state has two homes. The or-

chestration cluster's state — the brokers' data — is backed up to durable storage, typically an object store such as cloud blob storage held separately from the cluster. The secondary store has its own backup mechanism, to its own separate storage. A complete platform backup is a *coordinated* backup of both, taken so that the two are mutually consistent — a backup of the engine state and a backup of the secondary store from wildly different moments do not restore into a coherent platform.

Two disciplines make a backup real rather than nominal. The first: the backup must be stored *separately* from what it protects — a backup of the cluster held on the same infrastructure as the cluster is no defence against that infrastructure's loss. The second, and the one institutions most often neglect: the *restore must be tested*. A backup that has never been restored is a backup that is assumed to work, and an assumption is not a recovery capability. A platform team that has actually performed a restore — in a drill, into a separate environment — knows its backups work and knows how long a restore takes; a team that has only ever taken backups knows neither, and will discover both during the real disaster, which is the worst possible time.

A backup that has never been restored is not a recovery capability — it is an assumption. The disaster is the wrong time to discover that a backup is incomplete, inconsistent between the engine and the secondary store, or takes far longer to restore than the institution's recovery-time obligation allows. Test the restore: perform a real restore, into a separate environment, as a drill, and measure how long it takes. Until a restore has actually been done, the institution does not know whether it has a recovery capability or only a collection of files.

57.5 Infrastructure as a team discipline

Infrastructure management is not a solo activity; it is a team discipline. The values file is under version control, changes are reviewed, and deployments go through a pipeline. A single engineer making ad-hoc changes to the running cluster is a single point of failure and a governance gap. The infrastructure discipline this chapter describes works only if it is a *team* discipline: every change reviewed by a second pair of eyes, every deployment traceable to a reviewed commit, every incident followed by a blameless review that improves the process.

Worked example: the untested backup

Problem. A bank's platform team takes a backup of the orchestration cluster every night and of the secondary store every night, and has done so for two years without incident. The two backups are scheduled independently: the cluster backup at 01:00, the secondary-store backup at 03:00. The bank's regulatory recovery-time objective for the platform is four hours. A serious storage failure then forces a full restore. The team discovers, during the restore, two problems: the cluster backup from 01:00 and the secondary-store backup from 03:00 are two hours apart in the state they capture, so process instances that started between 01:00 and 03:00 are in the secondary store's backup but not the cluster's; and, never having timed a restore, the team finds the full restore takes nine hours. Assess what went wrong and what the team should have known.

Setup. We examine the two failures — the inconsistency between the two backups, and the restore duration against the four-hour objective — and identify the disciplines that would

have prevented each.

Working. *The inconsistency.* The cluster backup and the secondary-store backup were taken two hours apart and were never coordinated. The restored platform therefore has an engine state from 01:00 and a secondary store from 03:00 — and process instances started in that two-hour window exist in one and not the other. The restored platform is internally inconsistent: Operate, reading the secondary store, shows instances the engine has no record of.

the two backups do not restore into a coherent platform.

The restore duration. The recovery-time objective is 4 hours; the actual restore takes 9 hours.

9 hours > 4 hours — the regulatory obligation is breached by more than a working day's half.

Discussion. The team did the visible part of backup diligently — two years of nightly backups, never missed — and failed at the two parts that are invisible until a restore. The worked example's lesson is that *taking* a backup and *having a recovery capability* are not the same thing, and the gap between them is exactly the two disciplines this chapter named. The inconsistency was guaranteed by scheduling the two backups independently: a platform's state lives in two stores, and a backup that captures them two hours apart captures two moments, not one, so it cannot restore a coherent platform — the backups had to be *coordinated* to a consistent point. The nine-hour restore was guaranteed to be a surprise because the restore had never been performed: a team that had once done a real restore drill would have discovered the nine hours in the drill, two years ago, in calm conditions, and either accepted it, or invested to bring it under the four-hour objective, or at least told the regulator the true recovery time rather than the assumed one. Instead the team discovered it mid-disaster, while breaching a regulatory obligation, with no time to do anything about it. Both failures were prevented by disciplines that cost a little effort up front and were skipped because the backups “obviously” worked — and “obviously” is precisely the word that should never appear near a recovery capability. A backup is not real until it has been restored, and a platform's backup is not coherent until its two stores are backed up together.

Practical next steps

Ask your platform team two questions. Are the engine backup and the secondary-store backup coordinated to a mutually consistent point, or scheduled independently? And has a full restore ever actually been performed, end to end, with its duration measured against the institution's recovery-time objective? If the answer to either is no, the institution has backups but has not confirmed it has a recovery capability — and the disaster is the wrong place to find out.

Chapter in brief

Three infrastructure disciplines keep a platform running well. *Scaling*: the stateless components scale elastically by running more copies, but the stateful cluster's partition count is fixed at creation and must be sized at the start for the platform's expected growth. *Resilience*: stateless components survive failure by running more than one copy; the stateful cluster survives by replication — the replication factor is the number of broker losses it survives — and spreading brokers across zones extends this to surviving a whole zone. *Backup*: the defence against corruption, operator error, and disaster — it must coordinate the engine and secondary-store backups to a consistent point, store

them separately, and, above all, be tested by a real restore, because an untested backup is an assumption, not a recovery capability.

Infrastructure as a Governed Artefact

58.1 The last place discipline is allowed to lapse

This manuscript has, part after part, made the same argument in different domains: the process must be an explicit, governed, versioned artefact; so must the decision; so must the prompt; so must the agent. This short chapter makes the argument one final time, about the infrastructure itself — because the infrastructure is the foundation everything else stands on, and it is, too often, the one layer a hyperautomation programme leaves ungoverned, configured by hand, known only to whoever set it up.

The argument is the same and the stakes are, if anything, higher. A process configured by hand and undocumented is one bad process. An *infrastructure* configured by hand and undocumented is the whole platform resting on something no one can reliably reproduce, review, or reason about.

58.2 Infrastructure as code

The discipline is *infrastructure as code*: the platform's infrastructure — the Kubernetes resources, the Helm chart configuration, the cluster sizing, the networking, the storage — is defined in *declared, versioned files*, not assembled by hand through a console and a sequence of commands no one wrote down.

The benefits are exactly the benefits this manuscript has claimed for every other governed artefact, and they are worth naming once more in this setting. A declared infrastructure is *reproducible*: the same files produce the same platform, so a test environment genuinely matches production and a recovery rebuilds what was lost. It is *reviewable*: a change to the infrastructure is a change to a file, which a second engineer can read and challenge before it is applied — where a change made by hand in a console is seen by no one. It is *versioned*: the infrastructure's history is recorded, a bad change can be identified and reverted, and the question “what changed?” has an answer. And it is *auditable*: a regulator asking how the platform is configured, and how that configuration is controlled, can be shown the files and their change history — a far better answer than “the engineer who built it knows.”

The Helm chart configuration of **the deployment chapter** is the centre of this: the platform's infrastructure expressed as a configurable, versioned artefact, held in the same kind of version control, subject to the same review, as the institution's code. Infrastructure as code is not a separate practice the platform team must invent; it is the manuscript's governance argument, applied to the layer the platform stands on.

Figure 58.1 draws the four properties a declared infrastructure gains.

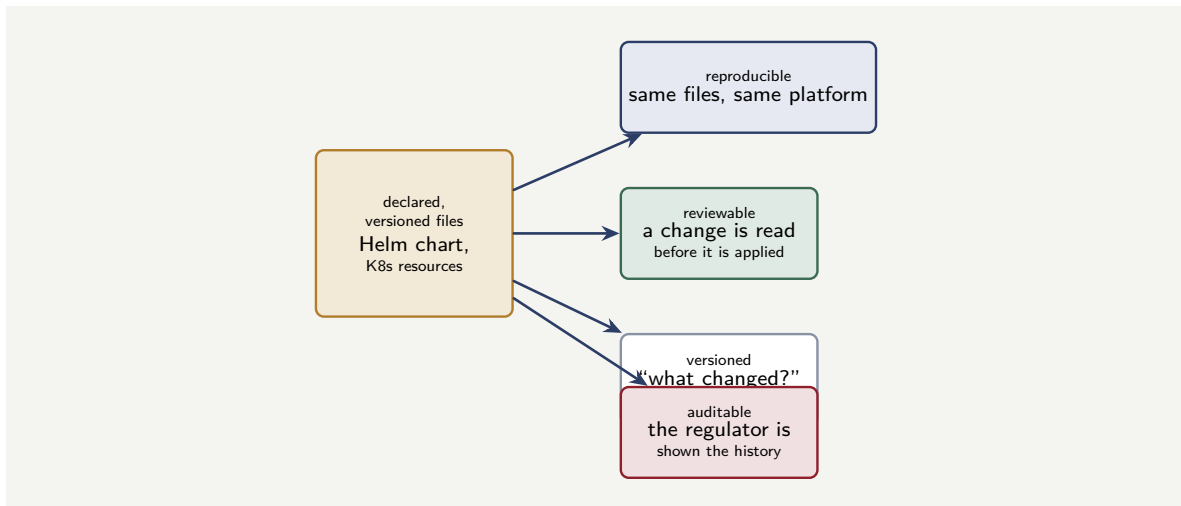


Figure 58.1. Infrastructure as code. When the platform’s infrastructure is defined in declared, versioned files, it gains the four properties of any governed artefact: it is reproducible, reviewable before a change is applied, versioned so “what changed?” has an answer, and auditable.

58.3 The environments and their parity

A governed infrastructure makes possible something a hyperautomation programme genuinely needs: a set of *environments* that genuinely resemble each other.

A platform is not one installation but several — development, testing, perhaps staging, production — and the value of the non-production ones depends entirely on how faithfully they resemble production. A test environment that differs from production in its sizing, its configuration, its topology is a test environment that can pass a process the production environment will then fail — the integration faults of [Chapter 95](#)’s testing, discovered in production because the test environment was not production-like.

Infrastructure as code is what makes environment *parity* achievable. Because the environments are built from the same declared files — differing only in deliberate, declared, reviewable ways, such as scale — a test environment is genuinely a smaller production, not a different thing that happens to also run Camunda. The parity is not maintained by hand and hope; it is a property of building every environment from one governed definition.

Tip

The value of a test or staging environment is exactly its fidelity to production. An environment built by hand, separately from production, drifts from it — and a process that passes there can still fail in production because the environments differ. Build every environment from the same infrastructure-as-code definition, differing only in declared, reviewed ways. Environment parity is not a thing maintained by diligence; it is a property of a governed infrastructure.

58.4 Infrastructure as a team discipline

Infrastructure management is not a solo activity; it is a team discipline. The values file is under version control, changes are reviewed, and deployments go through a pipeline. A single engineer making ad-hoc changes to the running cluster is a single point of failure and

a governance gap. The infrastructure discipline this chapter describes works only if it is a *team* discipline: every change reviewed by a second pair of eyes, every deployment traceable to a reviewed commit, every incident followed by a blameless review that improves the process.

Worked example: the change no one could explain

Problem. A bank’s Camunda platform suffers an incident: process throughput drops sharply one morning, and a number of processes begin missing their SLAs. The platform team investigates. The engine is healthy, the workers are healthy, but the orchestration cluster is performing far below its normal capacity. After several hours they find the cause: three weeks earlier, an engineer had reduced the CPU resource allocation of the broker pods — by hand, directly, to free capacity for an unrelated test — and had never reverted it. Because the change was made directly and not through any versioned definition, there was no record that it had happened, no review that might have caught it, and nothing for the investigating team to compare against. Assess what the incident cost beyond the outage, and what would have changed it.

Setup. We consider the diagnosis difficulty the un-governed change created, and contrast with what an infrastructure-as-code discipline would have produced.

Working. *What actually happened.* The change — a reduction in broker CPU — was made directly to the running cluster. It left:

no versioned record, no review, nothing to diff against.

The investigating team therefore could not answer “what changed?” by *looking*; they had to find the change by *searching*, comparing the live cluster’s every setting against what they believed it should be — several hours of work. *What infrastructure as code would have produced.* The broker CPU allocation would have been a value in a versioned file. Changing it would have been a change to that file — visible in the change history, and, before it was ever applied, in front of a second engineer for review. The investigating team’s first action — “what changed in the last month?” — would have answered itself in:

minutes, from the change history,

and, more likely, the harmful change would never have reached production at all, because a reviewer would have asked why a production cluster’s broker CPU was being reduced.

Discussion. The outage was the visible cost; the worked example is about the invisible one. An un-governed change does not merely risk causing an incident — it makes every incident, including incidents it did not cause, harder to diagnose, because it destroys the platform team’s ability to answer the single most useful question in any investigation: *what changed?* When infrastructure is governed as code, that question is answered by reading a change history; when infrastructure is configured by hand, it can only be answered by exhaustive comparison, and the bank paid for that difference in hours of outage while a team searched. And the deeper failure is the one before the incident: the harmful change was made directly, by one engineer, seen by no one, with no review — and a review is exactly what would have stopped it, because “reduce a production cluster’s broker CPU” is a change a second engineer would have questioned. This is the manuscript’s governance argument in its final form. Every reason this book has given for modelling processes explicitly, governing decisions, versioning prompts — reproducibility, review, a change history, auditability — applies with full force to the infrastructure, and an institution that governs all of those but

leaves its infrastructure configured by hand has left the foundation of the whole platform as the one ungoverned thing. The foundation is the last place the discipline should be allowed to lapse, because it is the place everything else stands on.

Practical next steps

Ask how your institution's platform infrastructure is defined. Is it in declared, versioned files — reviewed when changed, with a readable history — or is some of it configured directly, by hand, known to whoever set it up? Every part of the infrastructure that is not governed as code is a part where “what changed?” has no quick answer and a harmful change meets no review. Bringing the infrastructure under the same governance as the institution's code is the completion of the discipline this whole manuscript has argued for.

Chapter in brief

The infrastructure is the foundation the whole platform stands on, and it must be a governed artefact like every other — not configured by hand and known only to whoever set it up. Infrastructure as code defines the platform's infrastructure in declared, versioned files, making it reproducible, reviewable, versioned, and auditable — the same benefits this manuscript has claimed for every governed artefact. It is also what makes environment parity real: every environment built from one definition, so a test environment is genuinely a smaller production. An un-governed infrastructure change destroys the ability to answer “what changed?” — the most useful question in any incident — and meets no review. The foundation is the last place the discipline should be allowed to lapse.

PART VIII

Running Camunda on Kubernetes

Camunda on a Kubernetes Cluster

59.1 Why Kubernetes is the deployment of record

The infrastructure part established what a platform’s deployment must provide; this part treats the platform Camunda 8 is, in practice, deployed on: *Kubernetes*. It deserves a part of its own because Camunda 8 is a cloud-native, distributed system, and Kubernetes is the environment that system is designed for and the one Camunda recommends for production.

Camunda 8 Self-Managed is, recall from [Chapter 10](#), not one process; it is a set of cooperating components — the Zeebe brokers, the Zeebe Gateway, the web applications, a data store. Running that set by hand — starting each component, wiring them together, restarting one when it fails, adding a broker when load grows — is exactly the work Kubernetes exists to automate. Kubernetes is a *container orchestrator*: it runs containerised components, keeps the declared number of each running, restarts failures, schedules them across machines, and gives them stable network identities and storage. A distributed system deployed on Kubernetes is a distributed system whose operational toil is handled by the platform.

Camunda deploys to Kubernetes through *Helm* — a package manager for Kubernetes. Camunda publishes an official Helm *chart*, the `camunda-platform` chart, which describes the whole set of components and their relationships; an institution installs it with one command and configures it through a *values file*. The chart can be deployed to any conformant Kubernetes — a cloud provider’s managed Kubernetes, an on-premises cluster, Red Hat OpenShift — because Helm and the chart are not bound to any one provider.

Figure [59.1](#) shows the chart and the cluster.

59.2 Installing the chart

Installing Camunda on Kubernetes is, at its simplest, three commands: add the Camunda Helm repository, update it, and install the chart into a namespace. Listing [59.1](#) is the whole of a basic install.

Listing 59.1. Installing Camunda 8 on Kubernetes: add the Helm repository and install the chart into a namespace with a values file.

```

1 # add Camunda's official Helm chart repository
2 helm repo add camunda https://helm.camunda.io
3 helm repo update
4
5 # install the platform into its own namespace
6 helm install camunda camunda/camunda-platform \
7   --namespace camunda \
8   --create-namespace \
9   --values values.yaml

```

The `values.yaml` named in that command is where the deployment is shaped — the broker

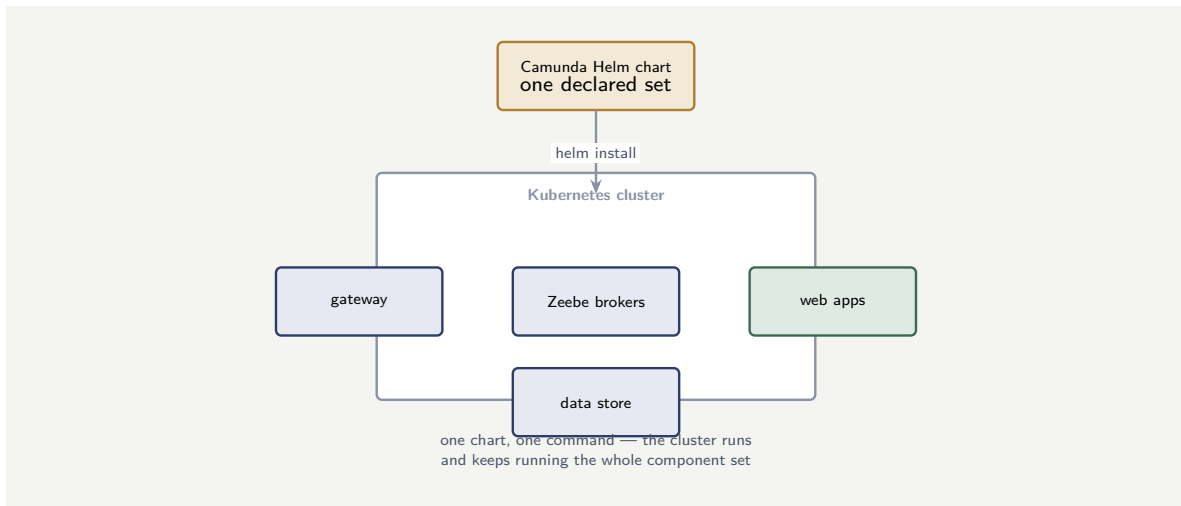


Figure 59.1. Camunda on Kubernetes. The official Helm chart describes the whole component set; Kubernetes runs it, keeps each component at its declared count, restarts failures, and schedules the components across the cluster’s machines.

count, the partition count, the resource requests, the data store, whether each web application is enabled. The chart ships with sensible defaults; the values file is the institution’s deliberate overrides, and it is a *governed artefact* in the sense of [Chapter 58](#): versioned, reviewed, and applied through a pipeline, because it *is* the platform’s definition.

Tip

Treat the Helm `values.yaml` as the platform’s source of truth, under version control and code review, applied only through a pipeline. It defines the broker count, the partitioning, the data store, the resource limits — the entire shape of the running platform. A cluster changed by ad-hoc `kubectl` edits instead of by a reviewed values file is a cluster whose real configuration no one can state with certainty.

59.3 Organising the cluster: namespaces and resource limits

A production Camunda deployment separates its components from the rest of the cluster’s workloads using a *namespace*, and protects the cluster’s nodes from any one component consuming all available resources by setting *resource requests and limits*. Both are configured in the values file. Listing 59.2 shows the namespace-scoped resource configuration for the Zeebe gateway, which carries all client traffic and must be explicitly sized.

Listing 59.2. Resource requests and limits for the Zeebe Gateway in the values file: the gateway is stateless and scaled horizontally, but each replica must be explicitly sized.

```

1 # values.yaml -- gateway resource sizing
2 zeebe-gateway:
3   replicas: 2
4   resources:
5     requests:
6       cpu: "1"
7       memory: 1Gi
  
```

```

8     limits:
9       cpu: "2"
10      memory: 2Gi
11     env:
12     - name: JAVA_TOOL_OPTIONS
13       value: "-Xms512m -Xmx1g"

```

Setting both `requests` and `limits` is the Kubernetes resource discipline: `requests` are what the scheduler uses to place the pod; `limits` are the ceiling that prevents one misbehaving component from consuming the node's resources. A gateway with no limit can, under an unusual load spike, starve the Zeebe brokers running on the same node — a production cluster sizes each component explicitly to prevent this.

59.4 Kubernetes operations for a Camunda platform team

A Camunda platform team operating on Kubernetes needs three operational disciplines beyond ordinary Kubernetes practice. First, *broker awareness*: the Zeebe brokers are stateful, and a rolling update of the StatefulSet must respect the replication protocol — updating one broker at a time, waiting for partition replicas to rebalance before proceeding. Second, *storage monitoring*: the brokers' persistent volumes grow with the event stream, and a volume that fills causes the broker to stop — monitoring free space and configuring compaction are not optional. Third, *export-lag monitoring*: the exporter that feeds Operate and Tasklist can lag behind the engine under load, and a growing lag means the web applications show stale data. Monitor the export position against the engine's position, and alert when the gap exceeds a defined threshold.

Worked example: the platform changed outside the values file

Problem. A bank runs Camunda on Kubernetes, installed from a `values.yaml` held in version control. Under load pressure, an engineer raises the Zeebe brokers' memory directly with a live `kubectl` edit on the running resource. Months later the cluster is rebuilt in a new region by re-applying the `values.yaml` — and the new cluster is under-resourced and unstable. Assess what happened.

Setup. We compare what the values file says with what the running cluster had become.

Working. The cluster was installed from the values file, but then changed *outside* it: the live `kubectl` edit raised broker memory on the running resource without changing the `values.yaml`. From that moment the file and the cluster disagreed:

`values.yaml`: original memory $\neq$ running cluster: raised memory.

The running cluster worked because of a change recorded nowhere. When the cluster was rebuilt from the file, the rebuild faithfully reproduced the file — the original, lower memory — and so reproduced an under-resourced cluster. The fix the engineer had applied was lost, because it had never been written where the platform's definition lives.

Discussion. The failure is the tip box's, realised. The `values.yaml` is only the platform's source of truth if it is *actually* the source — if every change goes through it. The live `kubectl` edit was a reasonable firefight in the moment, but leaving the change only on the running resource made the file a lie: it no longer described the platform. Kubernetes makes this failure particularly easy, because a live edit *works* — the cluster accepts it and runs — so nothing visibly breaks until the cluster is rebuilt and the undocumented change vanishes. The discipline

is the one [Chapter 58](#) sets out: the values file is infrastructure-as-code, and infrastructure-as-code only holds if the code is the single way infrastructure changes. The engineer should have changed the `values.yaml` and re-applied it — through the pipeline, reviewed — so the raised memory was part of the platform’s definition and survived the rebuild. The lesson is that on Kubernetes the gap between “what the values file says” and “what the cluster is” opens silently with every ad-hoc edit, and the only defence is the rule that there are no ad-hoc edits.

Practical next steps

Run Camunda on Kubernetes from a version-controlled Helm `values.yaml` and change the cluster only by changing that file and re-applying it through a pipeline. Audit for drift — live `kubectl` edits that the values file does not reflect — because each one is a change that will vanish the next time the cluster is rebuilt.

Chapter in brief

Camunda 8 Self-Managed is a cloud-native distributed system — Zeebe brokers, a gateway, web applications, a data store — and *Kubernetes*, a container orchestrator that runs components, keeps them at their declared counts, restarts failures, and schedules them across machines, is the environment it is designed for and the one Camunda recommends for production. Camunda deploys to Kubernetes through *Helm*: the official `camunda-platform` chart describes the whole component set, installed with one command and shaped by a `values.yaml`. That values file — broker count, partitioning, data store, resource limits — is the platform’s definition, and must be a governed artefact: version-controlled, reviewed, and applied only through a pipeline, because a cluster changed by ad-hoc `kubectl` edits is one whose real configuration no one can state and whose undocumented changes vanish on rebuild.

CHAPTER 60

The Anatomy of a Camunda Kubernetes Cluster

60.1 What the chart deploys

The previous chapter installed the chart with one command; this chapter opens that command up — what, concretely, is now running in the cluster, and how the pieces are arranged — because operating the platform means knowing its parts.

The `camunda-platform` chart deploys, by default, the *orchestration cluster* and its supporting components. The *Zeebe brokers* are the engine itself — the component of [Chapter 27](#) — and they run as a Kubernetes *StatefulSet*, because each broker has durable identity and durable storage. The *Zeebe Gateway* is the entry point, the gRPC endpoint clients and workers reach; it is stateless and runs as an ordinary scalable *Deployment*, by default with two replicas. The web applications — *Operate* for monitoring instances, *Tasklist* for human work, *Optimize* for analysis — run as Deployments. And the platform needs a *data store*: Zeebe exports its event stream to *Elasticsearch* or *OpenSearch*, and Operate and Tasklist read from it. *Identity* and *Keycloak*, with a database, provide the authentication of [Chapter 30](#).

The architecturally important distinction among these is *stateful* versus *stateless*. The brokers and the data store are stateful — they hold data, they need durable storage, they cannot simply be replaced. The gateway, the web applications, and the workers are stateless — they hold no durable data, and Kubernetes can add, remove, or restart them freely. This is the [Chapter 57](#) distinction, and on Kubernetes it is concrete: the stateful components are *StatefulSets* with persistent volumes, the stateless ones are *Deployments*.

Figure [60.1](#) divides the cluster into its stateful and stateless halves.

60.2 High availability across the cluster

A production Camunda Kubernetes cluster is arranged for *high availability*, and the arrangement uses Kubernetes' own scheduling.

The Zeebe brokers — by default three — are spread across the cluster's *availability zones*, the independent failure domains a cloud region provides, so that the loss of one zone leaves a working majority of brokers. Zeebe's partitions are *replicated* across the brokers, as [Chapter 27](#) described, so each partition survives the loss of a broker. The stateless components run with multiple replicas, also spread across zones. The result is a cluster with no single point of failure: any one node, or any one zone, can be lost and the platform continues.

Listing [60.1](#) shows the part of a `values.yaml` that arranges this — the broker count, the partition replication, the persistent volume sizing, and the spreading.

Listing 60.1. The high-availability core of a Camunda Kubernetes values file: three replicated brokers, persistent storage, and the resource requests Kubernetes schedules against.

```
1 # values.yaml -- the orchestration cluster
2 zeebe:
```

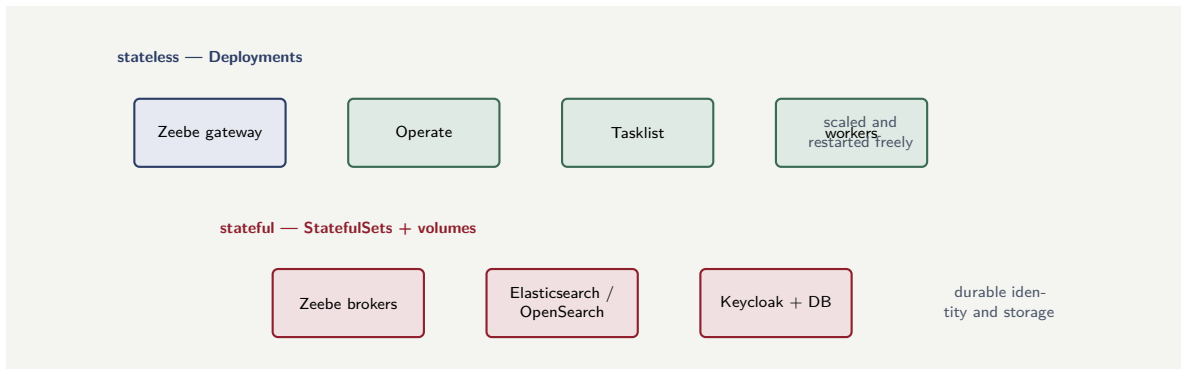


Figure 60.1. The anatomy of a Camunda Kubernetes cluster. The stateless components — gateway, web applications, workers — run as Deployments, scaled and restarted freely. The stateful components — brokers, the data store, Keycloak’s database — run as StatefulSets with persistent volumes.

```

3  clusterSize: 3
4  partitionCount: 6
5  replicationFactor: 3      # each partition on 3 brokers
6  pvcSize: 64Gi           # persistent volume per broker
7  resources:
8    requests:
9      cpu: "2"
10     memory: 4Gi
11 zeebe-gateway:
12   replicas: 2              # stateless -- scaled freely
13 elasticsearch:
14   master:
15     replicaCount: 3        # the data store, also HA

```

The Zeebe brokers are a *StatefulSet* with persistent volumes, and two of their settings are fixed at the cluster’s birth: the `partitionCount` and the `replicationFactor` cannot be changed by ordinary scaling afterwards. Size them at install for the load the platform must grow into over its life, with real headroom. A cluster created with a partition count sized only for today’s load has a ceiling built in, and raising it later means standing up a new cluster and migrating to it — not editing a value.

60.3 Verifying the cluster

Once the chart is installed, a small set of `kubectl` checks confirms the cluster is healthy before any process is deployed. Listing 60.2 is the verification sequence: pods running, StatefulSets ready, and a quick topology check on the Zeebe gateway.

Listing 60.2. Verifying a Camunda Kubernetes cluster after installation: pods running, StatefulSets ready, and gateway topology responding.

```

1  # all pods running in the camunda namespace
2  kubectl get pods -n camunda
3
4  # zeebe brokers: 3/3 ready
5  kubectl get statefulsets -n camunda
6
7  # gateway reports the broker topology
8  kubectl exec -n camunda \
9    deployment/camunda-zeebe-gateway -- \

```



```
10 zbctl status --insecure
```

The `zbctl status` output reports each broker and its partition assignments. A healthy three-broker cluster shows all three brokers as leaders or followers for each partition — a broker reported as disconnected before the platform carries any workload is a configuration problem to resolve before any process is deployed.

60.4 Kubernetes operations for a Camunda platform team

A Camunda platform team operating on Kubernetes needs three operational disciplines beyond ordinary Kubernetes practice. First, *broker awareness*: the Zeebe brokers are stateful, and a rolling update of the StatefulSet must respect the replication protocol — updating one broker at a time, waiting for partition replicas to rebalance before proceeding. Second, *storage monitoring*: the brokers' persistent volumes grow with the event stream, and a volume that fills causes the broker to stop — monitoring free space and configuring compaction are not optional. Third, *export-lag monitoring*: the exporter that feeds Operate and Tasklist can lag behind the engine under load, and a growing lag means the web applications show stale data. Monitor the export position against the engine's position, and alert when the gap exceeds a defined threshold.

Worked example: the cluster that lost a zone

Problem. A bank runs a Camunda Kubernetes cluster with three Zeebe brokers, `replicationFactor 3`, the brokers spread one per availability zone across a three-zone region. A zone fails entirely — one broker, and its node, are lost. The platform stays up. The operations team asks why, and whether a second zone failure would be survived too.

Setup. We count the brokers and partition replicas surviving one zone loss, then two.

Working. With three brokers, one per zone, and each partition replicated across all three brokers, the loss of one zone removes one broker and one replica of each partition. Two brokers and two replicas of each partition remain — a *majority* — so each partition keeps a working replica set and the platform continues:

$$3 \text{ brokers} - 1 \text{ zone} = 2 \text{ remaining} = \text{majority}.$$

A second zone failure would remove a second broker, leaving one:

$$3 - 2 = 1 = \text{not a majority of } 3.$$

One broker is not a majority of a three-broker cluster, so a second simultaneous zone loss would not be survived — the partitions would lose their quorum.

Discussion. The platform survived the first zone failure for a precise reason: a three-broker cluster with replication across three zones keeps a majority — two of three — when any one zone is lost, and a majority is what each partition needs to keep accepting work. This is why the chapter's HA arrangement spreads the brokers across zones rather than packing them in one: the zone is the failure domain, and spreading makes a zone loss survivable. The honest answer to the second question, though, is that this cluster tolerates *one* zone failure, not two — two simultaneous zone losses drop it below the majority a three-broker cluster needs. An institution that must survive two simultaneous zone failures needs a larger broker count — five brokers tolerate two losses — sized, like the partition count, at the cluster's birth. The lesson is that Kubernetes makes high availability achievable but does not make it automatic

or unlimited: the degree of failure a cluster tolerates is a deliberate choice, set by the broker count and the zone spread, and an institution should know which failures its chosen numbers actually survive — and not discover the limit during the second zone outage.

Practical next steps

For a Camunda Kubernetes cluster, decide explicitly how many simultaneous failure-domain losses the platform must survive, and size the broker count for it — three brokers tolerate one zone loss, five tolerate two. Spread the brokers across availability zones so the zone is the unit of failure, and fix the partition and replication settings at install for the platform’s whole life.

Chapter in brief

The `camunda-platform` Helm chart deploys the orchestration cluster: the *Zeebe brokers* as a *StatefulSet* with durable storage, the *Zeebe Gateway* and the web applications — Operate, Tasklist, Optimize — as stateless *Deployments*, a *data store* (Elasticsearch or OpenSearch) Zeebe exports to, and Identity with Keycloak. The architecturally central division is stateful versus stateless: brokers and data store are *StatefulSets* with persistent volumes, the rest are freely-scaled *Deployments*. A production cluster is arranged for high availability — brokers spread across availability zones, partitions replicated across brokers — so any one zone can be lost and the platform continues. The broker count and partition settings are fixed at the cluster’s birth, so they must be sized for the platform’s whole life, and for the number of simultaneous failures it must survive.

The Cluster With and Without Kafka

61.1 The exporter, and where the event stream goes

The previous chapters deployed a Camunda Kubernetes cluster whose data store is Elasticsearch or OpenSearch. This chapter treats a deliberate architectural choice on top of that: whether the cluster also publishes its event stream to *Kafka*, and what each option is for.

The choice rests on a Zeebe mechanism, the *exporter*. The Zeebe engine, as [Chapter 10](#) established, is event-sourced: everything that happens — an instance started, a task created, a job completed — is a record on an ordered internal stream. An *exporter* is a pluggable component that reads that stream and publishes it onward, and Zeebe’s exporter mechanism is the seam at which an institution decides where the engine’s events go.

In the default cluster, there is one exporter: it writes the event stream to *Elasticsearch* or *OpenSearch*, and that is what Operate and Tasklist read to show running instances and worklists. This is the cluster the previous chapter deployed, and for a great many institutions it is the whole story: the engine runs, its events populate the data store, and the web applications and the platform’s own APIs serve every need.

61.2 Without Kafka: the self-contained cluster

The cluster *without* Kafka is the default, and it is a deliberate, defensible choice, not merely the absence of a feature.

A cluster without Kafka is *self-contained*: the engine, its data store, and the web applications form a closed system, and the platform’s events are consumed only within it — by Operate, by Tasklist, by Optimize, and by applications that call the platform’s APIs. Nothing outside the cluster subscribes to the engine’s event stream. This is simpler to deploy, simpler to operate, and has fewer moving parts and fewer failure modes; for an institution whose process platform does not need to broadcast its events to a wider event-driven landscape, it is the right choice, and adding Kafka would be adding infrastructure for no purpose.

The signal that a cluster needs more than this is *external* demand: when systems *outside* the platform need to react, in near-real-time, to what the processes do — when “loan disbursed” or “claim settled” must reach a data platform, a notification service, a fraud system, an analytics pipeline, as it happens. The platform’s APIs can be polled, but polling is not the same as a system being *pushed* the events as they occur. That external, push-based demand is what Kafka answers.

61.3 With Kafka: the cluster on the event backbone

The cluster *with* Kafka adds a second exporter, or a streaming integration, that publishes the engine’s business events to *Apache Kafka* — the durable, ordered event-streaming platform

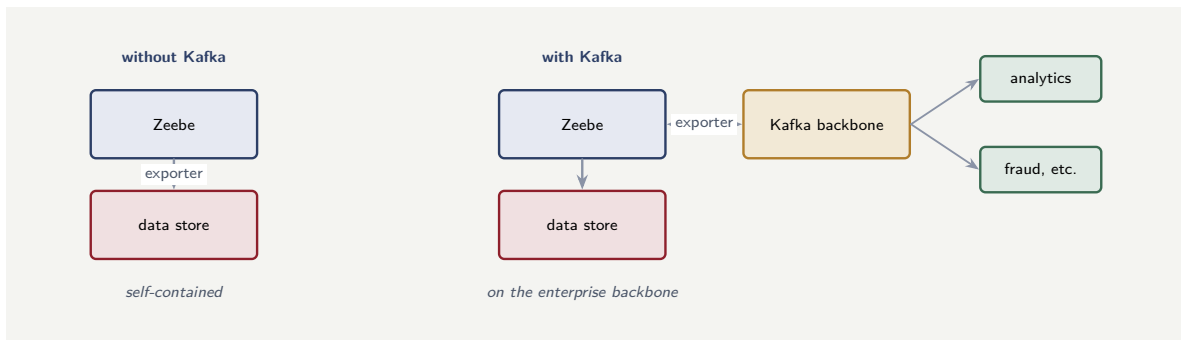


Figure 61.1. The cluster with and without Kafka. Without Kafka the cluster is self-contained — the engine exports only to its data store. With Kafka a second exporter publishes the engine’s events to the enterprise event backbone, where any system may consume them.

that, in many institutions, *is* the enterprise event backbone of [Chapter 13](#).

With this arrangement, the engine’s significant events — a process completed, a milestone reached, a business fact established — become messages on Kafka topics, and any system in the institution may subscribe. The process platform becomes a *publisher on the enterprise event backbone*: its events are available, in near-real-time, to data platforms, analytics, fraud monitoring, customer notification, other process applications — without any of them calling the Camunda APIs, and without Camunda knowing or caring who consumes. And the relationship runs both ways: Kafka topics carry events *into* the platform too, starting and advancing processes as [Chapter 13](#)’s event-driven automation describes, so the cluster both publishes to and consumes from the backbone.

On Kubernetes this is an architectural addition, not a rewrite. Kafka itself runs — as its own StatefulSet, often in the same Kubernetes cluster or a neighbouring one — and the Camunda deployment is configured with the exporter or streaming component that bridges Zeebe’s event stream to Kafka topics. The engine, the brokers, the data store are unchanged; what is added is the bridge to the backbone.

Figure [61.1](#) contrasts the two cluster shapes.

61.4 Kubernetes operations for a Camunda platform team

A Camunda platform team operating on Kubernetes needs three operational disciplines beyond ordinary Kubernetes practice. First, *broker awareness*: the Zeebe brokers are stateful, and a rolling update of the StatefulSet must respect the replication protocol — updating one broker at a time, waiting for partition replicas to rebalance before proceeding. Second, *storage monitoring*: the brokers’ persistent volumes grow with the event stream, and a volume that fills causes the broker to stop — monitoring free space and configuring compaction are not optional. Third, *export-lag monitoring*: the exporter that feeds Operate and Tasklist can lag behind the engine under load, and a growing lag means the web applications show stale data. Monitor the export position against the engine’s position, and alert when the gap exceeds a defined threshold.

Worked example: deciding whether the cluster needs Kafka

Problem. Two institutions deploy Camunda on Kubernetes. Bank A runs a set of internal approval processes; the only consumers of process state are Operate, Tasklist, and one

workbench application that calls the Task API. Insurer B runs claims processes whose every settlement must, in near-real-time, reach a fraud-analytics platform, a customer-notification service, a regulatory reporting pipeline, and a data lake. Each institution asks whether its cluster needs Kafka.

Setup. We apply the chapter's criterion — external, push-based demand for the engine's events — to each.

Working. Bank A's consumers of process events are all *inside* the platform's own boundary: Operate and Tasklist read the data store, and the workbench calls the Task API. Nothing outside the cluster needs to be pushed the engine's events:

Bank A: consumers = {Operate, Tasklist, API caller} $\Rightarrow$ no external push demand.

Insurer B has four *external* systems — fraud, notification, regulatory, data lake — that must each react to settlements as they happen:

Insurer B: 4 external systems need push $\Rightarrow$ external push demand.

By the chapter's criterion, Bank A's cluster does not need Kafka and Insurer B's does.

Discussion. The two institutions reach opposite answers from the same criterion, which is the point: Kafka is not a maturity badge a platform should acquire, it is an architectural component that answers a specific need, and the need is external systems requiring near-real-time push of the engine's events. Bank A has no such need — every consumer of its process state is inside the platform boundary, served by the data store and the APIs — so adding Kafka would add a StatefulSet, an exporter, and a class of failure modes for nothing, and the self-contained cluster is the correct, simpler choice. Insurer B has four external consumers that must react as settlements happen; polling four pipelines against the Camunda APIs would be fragile and laggy, and the right architecture is the Kafka exporter publishing settlements to the backbone where all four subscribe. The lesson is the chapter's: the with-Kafka and without-Kafka clusters are both correct designs, for different institutions, and the deciding question is neither scale nor ambition but whether systems outside the platform need to be pushed its events — answer that honestly, and the architecture follows.

Practical next steps

Decide the Kafka question by one test: do systems *outside* the Camunda platform need to react, in near-real-time, to the engine's business events? If not, deploy the self-contained cluster — it is simpler and has fewer failure modes. If so, add the Kafka exporter and put the cluster on the enterprise event backbone. Do not add Kafka as a default or a badge; add it to meet a real external demand.

Chapter in brief

Zeebe is event-sourced, and an *exporter* is the pluggable component that reads the engine's event stream and publishes it onward — the seam at which an institution decides where the engine's events go. The cluster *without* Kafka has one exporter, to Elasticsearch or OpenSearch; it is self-contained, simpler, with fewer failure modes, and right whenever the platform's events are consumed only within it — by Operate, Tasklist, and API callers. The cluster *with* Kafka adds an exporter that publishes the engine's business events to Apache Kafka, the enterprise event backbone, so any external system — data platforms, fraud monitoring, notification, analytics — may subscribe

without calling Camunda's APIs. The deciding question is neither scale nor ambition: it is whether systems outside the platform need a near-real-time push of the engine's events.

PART IX

Advanced Workflow Patterns

Parallelization: Doing Work at Once

62.1 Patterns, gathered

The manuscript has, in many chapters, used particular workflow constructs — a parallel gateway here, a compensation there, a message correlation elsewhere. This part gathers the most important of them and treats them as *patterns*: recurring, named shapes of workflow, each with a problem it solves, a way it is built, and a discipline for using it well. An architect who knows the patterns by name designs faster and designs better, because the patterns are the vocabulary of advanced workflow design.

This first chapter treats *parallelization* — the family of patterns for doing work at once rather than in sequence. [Chapter 51](#) introduced the parallel gateway and the multi-instance activity; this chapter treats parallelization as a design discipline, with the patterns and the pitfalls that the earlier mechanical treatment did not.

62.2 Why parallelize, and the honest accounting

The reason to parallelize is straightforward: parallel work finishes in the time of its slowest branch, not the sum of all branches, so a process with genuinely independent steps is faster — often dramatically faster — when those steps run at once. For a customer-facing process, the difference between sequential and parallel checks can be the difference between a response that feels instant and one that feels slow.

But parallelization must be accounted for honestly, because it is not free, and three costs are real. It adds *coordination*: parallel branches must be joined, and the join is a synchronisation point that must be designed. It adds *load*: parallel branches place their work on downstream systems *simultaneously* rather than spread over time, and a downstream system that comfortably handles sequential calls may not handle the same calls arriving all at once. And it adds *failure complexity*: when several branches run at once, several can fail at once, and the process must handle the combination. Parallelization is a powerful tool, and like every powerful tool it is misused by those who see only its benefit.

62.3 The parallelization patterns

Parallelization is not one pattern but a small family, and naming them sharpens design.

The *parallel split-and-join* is the basic pattern: a fixed set of branches, known at modelling time, that run at once and are joined before the flow continues — [Chapter 51](#)’s parallel gateway. It is the pattern for the fixed, independent checks: identity, sanctions, credit.

The *multi-instance* pattern parallelizes over a *collection* whose size is unknown until runtime — one activity run once per element, all at once — [Chapter 51](#)’s parallel multi-instance. It is the pattern for the syndication’s banks, the claim’s damaged items.

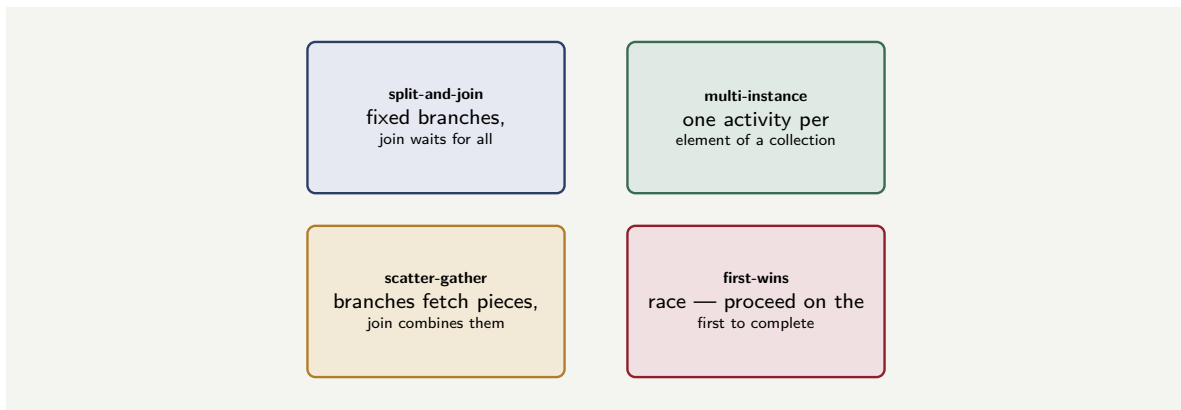


Figure 62.1. The parallelization patterns. A fixed set of branches is split-and-join; a runtime collection is multi-instance; branches that fetch pieces of a composite result are scatter-gather; a race where the first answer wins is first-wins. Naming the pattern is most of designing it well.

The *scatter-gather* pattern is parallel split-and-join with an explicit emphasis on the gather: several branches each fetch or compute a piece, and the join *combines* the pieces into a composite result. A pricing process that gathers, in parallel, a credit component, a risk component, and a market component, and combines them into a price, is scatter-gather.

And the *parallel-with-first-wins* pattern — built with an event-based gateway rather than a parallel join — runs several branches but proceeds as soon as the *first* completes, abandoning the rest. It is the pattern for “ask three credit bureaus, use whichever answers first.”

Knowing which pattern a situation calls for is most of designing the parallelization well: a fixed set of checks is split-and-join; a collection is multi-instance; a composite result is scatter-gather; a race is first-wins.

Figure 62.1 sets out the four patterns of parallelization.

Listing 62.1. A parallel multi-instance activity in BPMN. The input collection is iterated; one instance of the activity runs per element, all concurrently; each result is gathered into the output collection.

```

1 <bpmn:serviceTask id="ApproveByBank" name="Approve by bank">
2   <bpmn:multiInstanceLoopCharacteristics isSequential="false">
3     <bpmn:extensionElements>
4       <zeebe:loopCharacteristics
5         inputCollection="participatingBanks"
6         inputElement="bank"
7         outputCollection="approvals"
8         outputElement="approvalResult" />
9     </bpmn:extensionElements>
10  </bpmn:multiInstanceLoopCharacteristics>
11 </bpmn:serviceTask>

```

The listing shows the configuration that turns one modelled activity into a parallel multi-instance: `isSequential="false"` makes the instances run concurrently, the input collection names the list to iterate, and the output collection gathers each instance’s result — the syndicated loan approval of [Chapter 48](#), expressed in BPMN.

62.4 The disciplines of parallel work

Three disciplines keep parallelization from becoming a problem.

The join must handle the failure of a branch. A parallel join waits for all its branches — and so a branch that fails, or hangs, and never arrives at the join, hangs the whole process at

the join. Every parallel branch that can fail must fail in a way the join can still resolve: an error caught on the branch and routed, a branch that completes-with-failure rather than never completing. A parallelization whose branches can silently fail to arrive is a parallelization that can deadlock.

The downstream load must be deliberate. As [Chapter 37](#) warned of multi-instance, parallel branches fire their work at downstream systems at once. An architect parallelizing must ask what concurrency the parallelization creates and whether the downstream systems can absorb it — and, where they cannot, must bound the parallelism: a multi-instance with a limited batch size, a deliberate cap on how many branches run at once. Parallelization that ignores downstream capacity converts a process speed-up into a downstream outage.

Parallelize only what is genuinely independent. The deepest discipline: two steps may be run in parallel only if they truly do not depend on each other — neither needs the other’s result, neither’s correctness depends on ordering. Steps parallelized that are *not* independent produce a process that is sometimes wrong, in ways that depend on timing and are agonising to diagnose. Before parallelizing two steps, the question is not “would it be faster” but “are these genuinely independent” — and only if the answer is a confident yes does the speed-up become available.

Tip

Before parallelizing two steps, ask not “would it be faster” but “are these two steps genuinely independent — does neither need the other’s result, and does neither’s correctness depend on the order?” Parallelizing genuinely independent steps is a free speed-up. Parallelizing steps that are not independent produces a process that is intermittently, timing-dependently wrong — the hardest kind of defect to find. Independence is the precondition; speed is only the reward.

62.5 When to use this pattern

Every pattern this part describes is a tool, not a default. The parallel gateway is for genuinely independent work; using it for dependent work produces race conditions. The compensation pattern is for irreversible external effects; using it for in-memory state that can simply be rolled back is overengineering. The correlation pattern is for matching an incoming event to a waiting instance; using it when a direct call would suffice adds complexity for no benefit. The discipline is to reach for each pattern when the work genuinely needs it, and to use the simpler construct when it does not.

Worked example: the parallelization that created a race

Problem. A claims process has two steps that each took about 3 seconds: step X retrieves the policy details and writes them into the process variables; step Y calculates the claim reserve. The team, to speed the process, puts X and Y in a parallel split so they run at once — expecting to save 3 seconds. But step Y’s reserve calculation *reads* the policy details that step X writes. After the change, the process is correct most of the time but, on about 30% of claims, the reserve is calculated wrongly — using stale or missing policy data. The process runs 200,000 claims a year. Assess what went wrong and what the team failed to check.

Setup. We examine whether X and Y were genuinely independent — the precondition for parallelizing — and what their parallelization produced.

Working. Step Y reads the policy details that step X writes. That is a *dependency*: Y needs X's result. The two steps are therefore *not* independent, and the precondition for parallelizing them is not met. Run in parallel, X and Y race: sometimes X happens to finish writing the policy details before Y reads them, and the reserve is correct; sometimes Y reads before X has written, and the reserve is calculated on stale or missing data. The outcome depends on the timing of the two branches, which is why it is wrong:

about $200,000 \times 0.30 = 60,000$ claims a year

with a reserve calculated on bad data — a number that will drift as system timing changes, and that is intermittent enough to be very hard to diagnose.

Discussion. The team saw a 3-second saving and took it, and in doing so introduced a defect that miscalculates 60,000 claim reserves a year — and the worked example's lesson is that the entire error was a failure to ask one question before parallelizing. The team asked “would it be faster” — and the answer was yes, which is exactly what made the mistake tempting. They did not ask “are X and Y genuinely independent” — and the answer was no, because Y reads what X writes. A sequential process had hidden this dependency harmlessly: run one after the other, X always finishes writing before Y reads, and the dependency, though real, never bites. Parallelizing the two exposed the dependency as a race, and a race produces the worst class of defect — intermittent, timing-dependent, correct in testing often enough to ship, wrong in production often enough to matter, and miserable to diagnose because it cannot be reliably reproduced. The fix is not a clever synchronisation; it is to recognise that X and Y were never candidates for parallelization at all, because they are not independent, and a dependent pair must run in sequence — X then Y — so that Y always has X's result. The lesson is the chapter's central discipline stated through its violation: parallelization is a free speed-up *only* for genuinely independent work, and “genuinely independent” is a property to be verified — by tracing what each step reads and writes — not assumed because parallelizing would be faster. The 3 seconds the team wanted to save were, for these two steps, not available; the price of taking them anyway was 60,000 wrong reserves a year.

Practical next steps

Look at any parallel split in your process models and, for each pair of branches, trace what each branch reads and writes. If one branch reads a variable another branch writes, the branches are not independent and the parallelization is a latent race — intermittently, timing-dependently wrong. Genuinely independent branches are safe to parallelize; dependent ones must be returned to sequence.

Chapter in brief

Parallelization — doing work at once rather than in sequence — makes a process finish in the time of its slowest branch, but it is not free: it adds coordination, concentrates downstream load, and complicates failure. It is a family of named patterns: split-and-join for a fixed set of branches, multi-instance for a runtime collection, scatter-gather for branches that combine into a composite result, first-wins for a race. Three disciplines govern it: the join must handle a branch that fails or hangs; the concentrated downstream load must be deliberate and, where needed, bounded; and — the deepest — only genuinely independent work may be parallelized, because parallelizing dependent steps produces an intermittent, timing-dependent race.

Compensation: Undoing Work That Cannot Be Rolled Back

63.1 The problem compensation solves

Chapter 50 and the architecture part's saga chapter both treated compensation as part of the saga. This chapter treats compensation as a pattern in its own right, because it is one of the most important and most subtle patterns in financial workflow, and it deserves direct, full treatment.

The problem compensation solves is this. A long-running process performs a series of steps, and each step, as it completes, has a *real effect in the world*: an account is debited, a reservation is made, a letter is sent, a ledger entry is posted. Then, later, the process must *not continue* — a later step fails, the customer cancels, a check comes back bad. The earlier steps' effects are already real. They cannot be rolled back by a database transaction, because they were never inside one — they happened in separate systems, at separate times, and some of them happened in the physical world. The process needs a way to *undo* effects that are not technically reversible. That way is compensation.

63.2 Compensation is not rollback

The single most important idea in this chapter is the distinction between *rollback* and *compensation*, because confusing them is the source of most compensation mistakes.

A *rollback* undoes a change by erasing it — the database transaction aborts, and it is as if the change never happened. Rollback requires the change to have been inside a transaction, and it leaves no trace.

A *compensation* undoes a change by performing a *new, opposite action* — a second real action that counteracts the first. The original effect is not erased; it happened, and it remains in the record. A debit is compensated not by erasing the debit but by performing a *credit* of the same amount. A sent letter is compensated not by un-sending it — impossible — but by sending a *correction*. A made reservation is compensated by *cancelling* the reservation. Compensation is forward motion that counteracts; it is not erasure.

This distinction has real consequences. Because a compensation is a new action, it is visible: the debit and its compensating credit both appear in the account history, which is correct — the account history should show that money moved and then moved back. Because a compensation is a new action, it can itself fail, and the process must consider that. And because a compensation counteracts rather than erases, it must be *designed* — for each forward action, someone must work out what real opposite action genuinely counteracts it. The compensation for a step is part of the step's design, not an afterthought.

Figure 63.1 draws the forward chain and its compensations — paired, and run in reverse.

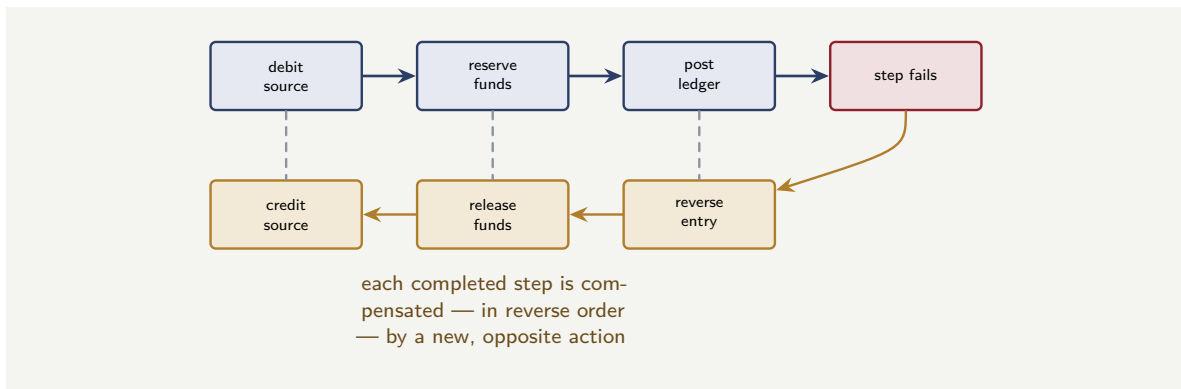


Figure 63.1. Compensation. When a step fails, the completed steps are compensated in reverse order — each by a new, opposite action: a debit by a credit, a reservation by a release, a ledger posting by a reversing entry. Compensation counteracts; it does not erase.

63.3 The disciplines of compensation

Compensation done well follows several disciplines, each learned from compensation done badly.

Compensation runs in reverse order. The completed steps are compensated last-completed-first, because the steps may depend on each other and undoing them in forward order can be wrong — you release a reservation before reversing the thing that depended on it. Reverse order is the safe order.

Every step with a real effect needs a designed compensation — or an explicit decision that none is possible. For each forward action, the question must be asked and answered: what real action counteracts this? Most actions have a compensation. A few genuinely do not — some effects, once done, cannot be meaningfully undone — and where that is so, it must be *known*, because a saga containing an uncompensatable step is a saga that, past that step, cannot fully unwind, and the process must be designed so that the uncompensatable step is placed as late as possible, after everything that might fail.

A compensation can fail, and that must be handled. A compensation is a real action against a real system, and that system can be down. A compensation that fails leaves the process in a partially-compensated state, which is a genuine incident needing a defined response — a retry, an escalation to a human, an alert. A saga that assumes its compensations always succeed has an unhandled failure mode exactly where it can least afford one.

Compensation must be idempotent. Like any action the engine may retry, a compensation may run more than once — and a compensation that runs twice must not credit the account twice. The idempotency discipline of [Chapter 41](#) applies to compensations with full force.

A saga may contain a step whose effect genuinely cannot be compensated — some real-world actions, once done, cannot be meaningfully undone. Such a step must be identified, and the process designed so it is placed *as late as possible* — after every step that might fail and trigger an unwind. A saga with an uncompensatable step early in its sequence is a saga that, if a later step fails, cannot return to a consistent state. Know which of your steps cannot be undone, and order the saga around them.

63.4 When to use this pattern

Every pattern this part describes is a tool, not a default. The parallel gateway is for genuinely independent work; using it for dependent work produces race conditions. The compensation pattern is for irreversible external effects; using it for in-memory state that can simply be rolled back is overengineering. The correlation pattern is for matching an incoming event to a waiting instance; using it when a direct call would suffice adds complexity for no benefit. The discipline is to reach for each pattern when the work genuinely needs it, and to use the simpler construct when it does not.

Worked example: the uncompensatable step placed too early

Problem. An insurance new-policy process is built as a saga with four steps: (1) take the customer's first premium payment, (2) register the policy with a regulator-mandated central register — an action that, once done, *cannot* be reversed, as the register has no withdrawal mechanism, (3) run a final fraud check, (4) issue the policy documents. The fraud check at step 3 fails on about 2% of applications. The process runs 500,000 new policies a year. Assess the consequence of the saga's ordering, and how it should be reordered.

Setup. We trace what happens when step 3 fails, given that step 2 is uncompensatable and placed before it, and then consider the corrected order.

Working. On the 2% of applications where the step 3 fraud check fails, the saga must unwind — compensate steps 1 and 2. Step 1, the premium payment, is compensatable: refund the premium. Step 2, the central-register registration, is *not* compensatable — the register has no withdrawal mechanism. So for:

$$500,000 \times 0.02 = 10,000 \text{ applications a year,}$$

the saga unwinds to find that a fraudulent or ineligible application has been *permanently registered* on a regulator-mandated central register, with no mechanism to undo it — an irreversible, regulator-visible consequence of a fraud the process itself then detected. *The correct order.* The uncompensatable step must be placed *last*, after every step that can fail and trigger an unwind. The fraud check (step 3) can fail; the registration (the uncompensatable step) must come after it. Reordered: take payment, run the fraud check, *then* register with the central register, then issue documents — so registration happens only once everything that could fail has passed.

Discussion. The original saga registers 10,000 fraudulent or ineligible applications a year, permanently and visibly, on a regulator's central register — and the cause is purely the *order* of the steps, not any step's correctness. Each step does its job; the saga's compensation logic is sound for the three steps that can be compensated; the defect is that an *uncompensatable* step was placed *before* a step that can fail. The worked example's lesson is the chapter's sharpest discipline. Compensation lets a saga unwind — but only across steps that *can* be compensated, and a saga is only as unwindable as its least-reversible step. An uncompensatable step is a point of no return: once the saga passes it, the saga cannot fully unwind, so everything that might trigger an unwind must happen *before* that point. The original order put the point of no return — registration — second, before the fraud check that, in 2% of cases, demands an unwind, and so those 2% unwind into a state the saga cannot fix. The correction is not new machinery; it is reordering, placing the irreversible registration last, after the fraud check, so that by the time the saga reaches the point of no return, nothing remains that could send it

back. The general principle: in any saga, identify the steps that cannot be compensated, and order the saga so they come after every step that might fail — the irreversible action goes last, when the saga is committed to going forward. A saga that places a point of no return before a step that can fail has, by its ordering alone, guaranteed a population of unrecoverable cases.

Practical next steps

For any saga in your institution, identify each step's compensation — and find the steps that have *none*, because their effect genuinely cannot be undone. Then check the order: every uncompensatable step must come after every step that could fail and trigger an unwind. An uncompensatable step placed early is a guaranteed population of cases the saga cannot recover — and the fix is usually only a reordering.

Chapter in brief

Compensation undoes the effects of completed steps when a long-running process must not continue. It is not rollback: a rollback erases a change inside a transaction, while a compensation performs a new, opposite action that counteracts the original — a debit undone by a credit, a reservation by a cancellation — so the original effect happened and remains in the record. Its disciplines: compensation runs in reverse order; every real-effect step needs a designed compensation or an explicit acknowledgement that none is possible; a compensation can itself fail and needs a defined response; and compensations must be idempotent. A saga is only as unwindable as its least-reversible step, so uncompensatable steps must be ordered after everything that might fail.

Callbacks, Correlation, and Waiting for the World

64.1 The process that must wait

A great many financial processes spend most of their elapsed life *doing nothing* — waiting. Waiting for a payment to clear, for a human to approve, for a partner to respond, for a document to be uploaded, for a regulator’s register to confirm. The work of the process is brief; the waiting is long. This final chapter of the part treats the patterns for waiting well — the *callback* pattern and the *correlation* that makes it work — because a process that waits badly is a process that is either stuck, or polling wastefully, or losing the events it waited for.

64.2 The callback pattern

The *callback* pattern is the disciplined way for a process to wait for something that happens outside it. The shape is this: the process reaches a point where it needs something from the outside world — an approval, a partner’s response. Rather than *polling* — repeatedly asking “is it done yet?” — the process *pauses at a waiting state* and registers, in effect, a callback: when the awaited thing happens, the outside world will *send a message* to the process, and that message resumes it.

The callback pattern is the right way to wait for three reasons. It is *efficient*: a paused process consumes nothing while it waits — no polling, no held thread, no load — where polling consumes resources proportional to how often it asks. It is *prompt*: the process resumes the moment the awaited message arrives, where polling resumes only at the next poll, on average half a poll-interval late. And it is *durable*, in an engine-borne process: the paused process’s state is persisted, so it can wait for days or weeks — through restarts, through anything — and still be there, waiting, when the message comes.

This is the BPMN message catch event and the receive task of [Chapter 34](#), seen now as a pattern: the process pauses at a message catch, the external world sends the message, the process resumes. It is also exactly the “callback pattern” that [the architecture part’s](#) comparison noted Step Functions provides — because waiting for the world is a universal need of orchestrators, and the callback is the universal answer.

64.3 Correlation: delivering the message to the right instance

The callback pattern raises a question that is its whole subtlety: when the outside world sends the resuming message, *which paused process instance does it resume?* At any moment a bank may have fifty thousand process instances paused, waiting for a payment confirmation. A confirmation arrives. It must resume *one* of those fifty thousand — the right one — and not the other 49,999. The mechanism that achieves this is *correlation*.

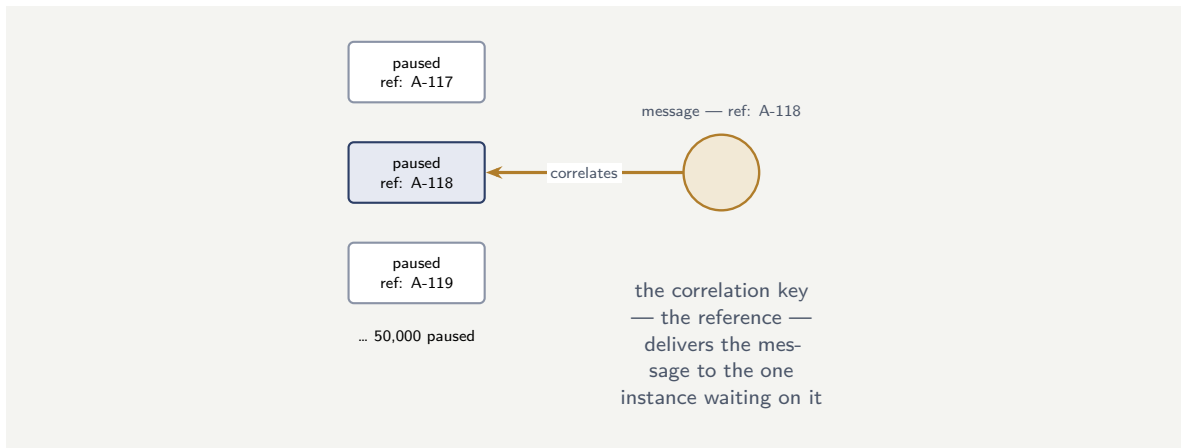


Figure 64.1. Correlation. Among fifty thousand paused instances, an arriving message must resume exactly the right one. The correlation key — here a reference carried on the message — matches the message to the single instance waiting on that value.

Correlation works by a *correlation key* — a piece of data that uniquely identifies which instance a message belongs to. When a process instance pauses at a message catch event, it declares the correlation key it is waiting on — say, a particular payment reference. When a message arrives, it carries a value for that key — the payment reference the confirmation concerns — and the engine matches the message to the one paused instance waiting on that value. The payment reference is the thread that connects the outside event back to the one process instance it concerns.

Correlation is the subtle, failure-prone part of the callback pattern, and it fails in two characteristic ways. The correlation key may be *missing or wrong on the message* — the external system sends the confirmation without the payment reference, or with a reference in a different form — and then the message matches no instance and the awaited resumption never happens; the process waits forever. Or the correlation key may be *not unique* — two paused instances are waiting on the same value — and then the message is ambiguous, and the wrong instance, or both, may be resumed. A callback pattern is only as reliable as its correlation, and its correlation is only as reliable as the uniqueness and the presence of the key.

Figure 64.1 draws correlation — a message finding its one instance among many.

64.4 Waiting safely: the timeout

A process that waits for the world must also confront the possibility that the world *never answers*. The partner system fails permanently; the human never approves; the confirmation is lost. A callback that waits purely for its message, with no other provision, is a callback that — if the message never comes — waits *forever*, and a process instance stuck forever is a silent failure.

The discipline is that a wait is almost always paired with a *timeout*. The process pauses for the awaited message *or* a timer, whichever comes first — the event-based gateway of [Chapter 34](#). If the message arrives, the process resumes normally. If the timer fires first, the process takes a defined timeout path: it escalates, it alerts, it retries, it routes to a human — something, rather than nothing. The timeout converts “waits forever” into “waits a defined time, then acts,” and that conversion is what makes waiting safe. A callback without a timeout is an unbounded wait, and an unbounded wait in a financial process is a defect waiting for the day

the world does not answer.

Tip

A process that waits for an external message must almost always wait for that message *or* a timeout, whichever comes first. A callback with no timeout waits forever if the message never arrives — and the partner system that fails permanently, the human who never approves, the lost confirmation, all guarantee that someday it will not. Pair every wait with a timer and a defined timeout path. “Waits forever” is never an acceptable process behaviour; “waits a defined time, then escalates” is.

64.5 When to use this pattern

Every pattern this part describes is a tool, not a default. The parallel gateway is for genuinely independent work; using it for dependent work produces race conditions. The compensation pattern is for irreversible external effects; using it for in-memory state that can simply be rolled back is overengineering. The correlation pattern is for matching an incoming event to a waiting instance; using it when a direct call would suffice adds complexity for no benefit. The discipline is to reach for each pattern when the work genuinely needs it, and to use the simpler construct when it does not.

Worked example: the callback that waited forever

Problem. A trade-settlement process pauses at a callback, waiting for a settlement-confirmation message from an external settlement system. The confirmation is correlated by the trade reference. The process has no timeout on the wait — the team reasoned that the settlement system “always confirms.” Over a year, the settlement system fails to send a confirmation for about 1 in 4,000 trades — a lost message, a system glitch. The process runs 2,000,000 trades a year. Assess what happens to the un-confirmed trades, and what the missing discipline was.

Setup. We consider the fate of the trades whose confirmation never arrives, given a callback with no timeout, and what a timeout would have changed.

Working. The number of trades a year whose confirmation is never sent:

$$\frac{2,000,000}{4,000} = 500 \text{ trades a year.}$$

For each of those 500 trades, the process is paused at a callback waiting for a message that will never arrive, and — because there is no timeout — it waits:

forever — the instance is stuck, silently, indefinitely.

Five hundred trade-settlement processes a year become permanently stuck instances, each a real trade in an unresolved state, and nothing in the process ever raises an alarm, because waiting is not an error — the process is doing exactly what it was modelled to do, which is wait. *With a timeout.* The callback would wait for the confirmation *or* a timer — say, the settlement system’s confirmation SLA plus a margin. For the 500 trades, the timer fires, and the process takes a defined timeout path: escalate the un-confirmed trade to the settlements operations team, who investigate and resolve it. The 500 trades become 500 *handled exceptions*, not 500 silent stuck instances.

Discussion. The team’s reasoning — the settlement system “always confirms” — is the precise belief the timeout discipline exists to override, and the worked example shows why.

“Always” is never true of an external system over a year of operation; 1-in-4,000 is a small failure rate, but across two million trades it is 500 real trades a year, and a callback with no timeout converts each of those into a permanently stuck process instance. The particular danger is that the stuck instances are *silent*: a process paused at a callback is not in an error state — it is waiting, which is normal, modelled behaviour — so nothing flags it, no incident is raised, and the 500 stuck trades accumulate unnoticed until something else — a reconciliation, a customer query, an auditor — stumbles on them. The timeout is what converts this silent, unbounded failure into a visible, bounded one. A callback paired with a timer cannot wait forever: either the confirmation arrives, or the timer fires and the process *does something* — escalates, alerts, routes to a human. Five hundred trades a year still go unconfirmed by the settlement system either way; the discipline does not prevent the external failure. What it changes is whether those 500 trades are 500 silent stuck instances or 500 promptly-escalated exceptions a team resolves — and in trade settlement, the difference between those two is the difference between an operational process and a regulatory incident. The lesson is the chapter’s firmest: a wait for the world is always a wait for the world *or* a timeout, because the world, often enough to matter, does not answer.

Practical next steps

Find every point in your process models where the process waits for an external message — a confirmation, an approval, a partner response. For each, check whether the wait is paired with a timeout and a defined timeout path. Any callback waiting purely for its message, with no timer, becomes a permanently stuck instance on the day that message is lost — and external messages are, eventually, always lost. Pair every wait with a timeout.

Chapter in brief

Many financial processes spend most of their life waiting — for a payment, an approval, a partner. The callback pattern is the disciplined way to wait: the process pauses at a waiting state, and the external world sends a message that resumes it — efficient, prompt, and, in an engine-borne process, durable for days. Its subtlety is correlation: an arriving message must resume the one right instance among thousands paused, matched by a correlation key that must be present and unique. And every wait must be paired with a timeout: a callback with no timeout waits forever if its message never arrives, and external messages are eventually always lost — a timeout converts a silent, unbounded stuck instance into a visible, bounded, escalated exception.

PART X

Batch Processing Workflow Patterns

Batch Processing in a Process-Orchestration World

65.1 Why batch deserves a part

The preceding integration parts repeatedly met a recurring fact: financial institutions run a great deal of work as *batch*. The core banking system of [Chapter 107](#) runs overnight batch; payments clear and settle in batches; reconciliation is a batch activity. This final part treats batch processing directly — the architecture patterns for batch work in a world where process orchestration, the subject of this whole manuscript, is the dominant paradigm.

Batch deserves a part because it is easy, in a manuscript devoted to process orchestration, to treat batch as a relic — the old way, to be replaced by real-time, event-driven processes. That view is wrong, and acting on it causes real architectural mistakes. Batch processing is not a relic; it is a *permanent and appropriate* pattern for a large class of financial work, and an architect needs to know when batch is right, how it relates to process orchestration, and how to architect it well.

65.2 What batch processing is

Batch processing is the processing of work in *groups*, on a *schedule*, rather than each item individually as it arrives. The overnight interest calculation of a core banking system is batch: at a set time, the whole population of accounts is processed together. End-of-day reconciliation is batch. Statement generation is batch.

Batch contrasts with *real-time* or *event-driven* processing, in which each item of work is handled individually, as it arrives — the payment processed when the customer makes it, the application handled when it is submitted. Much of this manuscript has concerned event-driven, per-instance process orchestration: a process instance per customer, per application, per claim.

The two are not rivals; they are suited to different work. The architect's task is not to replace one with the other but to know which is appropriate for which work.

Figure [65.1](#) shows the batch loop — chunk, process, write, loop. The chunk boundary is the key property: a batch job that fails mid-run can restart from the last committed chunk rather than from the beginning of the entire dataset, which is what makes batch processing tractable over millions of records.

65.3 When batch is the right pattern

Batch is the right pattern for work with a particular shape, and an architect should recognise it.

Batch suits work that is naturally *periodic*: it genuinely happens at a point in time, not continuously — end-of-day, month-end, the regulatory reporting cycle. Forcing such work

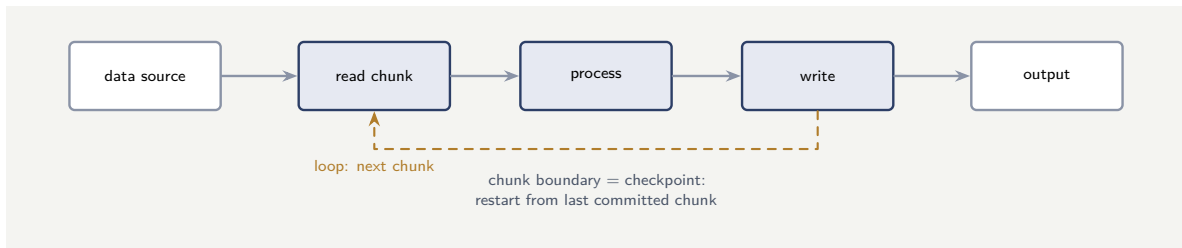


Figure 65.1. The batch processing loop. Work arrives in a large set; the job reads it in chunks, processes each, writes results, and loops. The chunk boundary is the restart checkpoint.

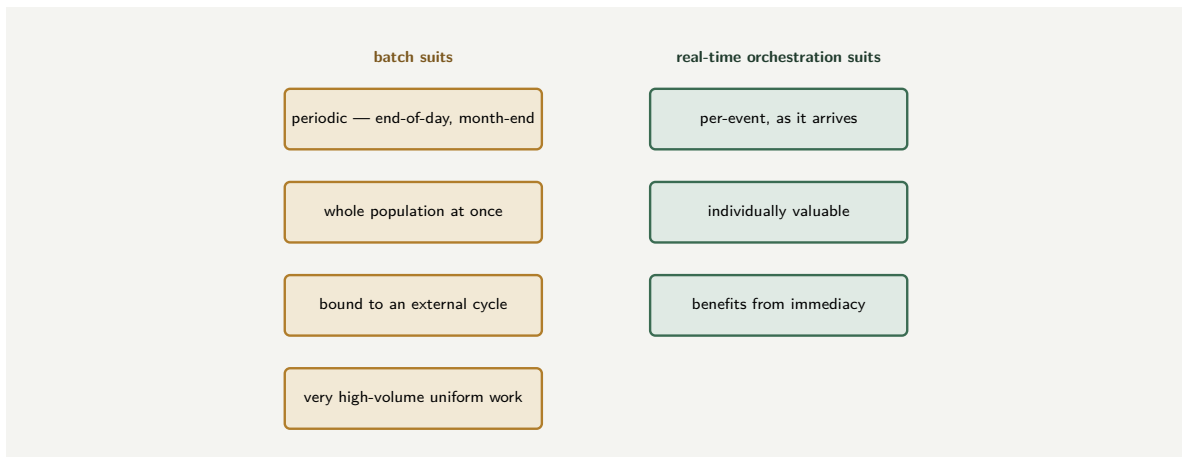


Figure 65.2. Matching the pattern to the work. Batch suits periodic, population-wide, externally-cycled, high-volume uniform work; real-time process orchestration suits per-event, individually-valuable work that benefits from immediacy. They are not rivals — they are suited to different work.

into a real-time model is artificial.

Batch suits work done over a *whole population at once*: calculating interest for every account, revaluing every position, generating every statement. When the work is inherently set-wide, batch expresses it directly.

Batch suits work bounded by an *external cycle the institution does not control*: the payment scheme’s settlement cycle, the market’s close, another institution’s batch. A process cannot make an external batch cycle real-time; it must work with it.

And batch suits *very high volumes* of uniform work where processing in bulk is more efficient than instance-by-instance handling.

Conversely, batch is the *wrong* pattern for work that is genuinely per-event and benefits from being handled when it arrives — a customer’s payment, an application, a claim notification. The architect’s discipline is to match the pattern to the work: periodic, population-wide, externally-cycled, high-volume-uniform work is batch; per-event, individually-valuable work is real-time orchestration.

Figure 65.2 contrasts the work that suits batch with the work that suits real-time orchestration.

65.4 The engine as the institution's process memory

A workflow engine’s state store is, in a real sense, the institution’s *process memory*: it knows, for every case in progress, where it is, what has been done, what is outstanding, and what

the data says. This memory survives restarts, deployments, and failures — it is durable, queryable, and auditable. An institution without a workflow engine has process memory only in the heads of the people doing the work, in spreadsheets, and in email threads — none of which survives a staff change, none of which a regulator can query.

Worked example: the real-time rewrite that should not have happened

Problem. A bank, modernising onto Camunda, has a long-standing month-end batch job: on the last day of each month it calculates and posts interest for every savings account — some millions of accounts. An enthusiastic team, believing batch is the old way and real-time is the modern way, proposes to replace it with an event-driven design: an individual Camunda process instance per account, triggered to recalculate and post interest. The architect challenges this. Assess the proposal.

Setup. We hold the month-end interest work against the criteria for when batch is the right pattern.

Working. We test the interest-posting work against the shape that suits batch. Is it *periodic*? Yes — it genuinely happens at month-end, by the nature of how interest accrues; it is not a continuous stream of events. Is it *population-wide*? Yes — it is performed over every savings account at once. Is it *high-volume and uniform*? Yes — millions of accounts, the same calculation each. So on every criterion:

periodic + population-wide + high-volume uniform $\Rightarrow$ batch is the right pattern.

The proposed real-time design — millions of individual process instances — would create enormous numbers of process instances to do work that is inherently a single periodic, population-wide activity, and there is no per-event trigger for it: nothing *happens* to an account to occasion the interest posting; it is simply month-end. The real-time rewrite manufactures an event-driven shape for work that is not event-driven.

Discussion. The architect is right to challenge the proposal, and the worked example shows the concrete cost of the belief that batch is merely the old way. Month-end interest posting is the textbook case of work that *should* be batch: it is periodic by the nature of interest, it is performed over the whole account population, and it is millions of uniform calculations. Re-casting it as millions of individual Camunda process instances does not modernise it — it mismodels it: it manufactures an event-driven architecture for work that has no events, creating immense instance volumes and operational overhead to express what is genuinely one scheduled, population-wide activity. The team's error is the one the chapter opens by warning against — treating batch as a relic to be eliminated rather than a permanent, appropriate pattern. The sound approach keeps the interest posting as batch, and uses Camunda for what Camunda is for: orchestrating the *per-event*, individually-valuable processes — onboarding, lending, claims — while the periodic, population-wide interest run remains batch, with the process platform *relating* to that batch as the next chapters describe. The lesson is the part's governing principle: modernisation is not the replacement of all batch with real-time; it is matching each kind of work to the right pattern, and periodic, population-wide work is, and should remain, batch — the maturity is in knowing the difference, not in eliminating one side of it.

Practical next steps

Review any plans in your institution to “modernise” batch jobs into real-time processes. For each, test the work against the criteria: is it periodic, population-wide, externally-cycled, high-volume uniform? If so, it is genuinely batch work, and recasting it as per-instance processes mismodels it. Modernise the platform around it, not the pattern of the work itself.

Chapter in brief

Financial institutions run a great deal of work as batch — overnight core processing, payment settlement, reconciliation — and batch is not a relic to be replaced by real-time processes; it is a permanent, appropriate pattern. *Batch processing* handles work in groups, on a schedule, rather than each item as it arrives. It is the right pattern for work that is periodic, performed over a whole population at once, bound to an external cycle, or very high-volume and uniform — and the wrong pattern for per-event, individually-valuable work that benefits from immediacy. Batch and real-time orchestration are not rivals; they suit different work, and the architect’s discipline is to match the pattern to the work rather than recast all batch as real-time.

Batch Workflow Architecture Patterns

66.1 Architecting batch well

The previous chapter established when batch is the right pattern. This chapter treats how to *architect* batch work well — because batch, done badly, is a brittle, opaque, hard-to-recover thing, and the patterns that make it sound are worth setting out.

66.2 The batch as an orchestrated process

The first and most important pattern is a reframing: a batch run is itself a *process*, and it benefits from being *orchestrated* as one.

A naive batch job is a monolithic program: it starts, it churns through its work, it ends — and if it fails partway, it is opaque about where, and hard to resume. A batch run architected as an *orchestrated process* is different. The batch becomes a modelled flow: a step that prepares and identifies the work; a step that processes it — often the parallel processing of [Chapter 62](#)'s patterns, applied to the batch's items; a step that handles the items that failed; a step that finalises and reports. Modelled this way, the batch run has the visibility, the error handling, and the recoverability that [Chapter 27](#)'s engine gives any process.

This does not mean every item in a million-item batch is a process instance — the previous chapter warned against that. It means the batch *run* — the orchestration of preparing, processing, handling failures, and finalising — is a process, even though the bulk processing of items within it is done in bulk. The process orchestrates the batch; it does not necessarily instantiate per item.

Figure 66.1 draws a batch run as an orchestrated process.

66.3 Chunking, restartability, and the failed item

Three further patterns make a batch architecture sound.

Chunking: a large batch is processed in *chunks* — manageable groups of items — rather than as one indivisible mass. Chunking gives the batch progress points, bounds the work that must be redone if something fails, and makes the batch's progress observable.

Restartability: a batch architecture must answer the question “what happens if the batch fails partway.” A well-architected batch is *restartable*: it records its progress — which chunks are done — so that a restart resumes from where it stopped rather than redoing completed work or, worse, leaving the batch in an unknown half-done state. A batch that must be restarted from the beginning after any failure is a batch that, for a large population, may not fit in its window.

The failed item: in any large batch, some items will fail — a record is malformed, a dependency is unavailable for that one item. A sound batch architecture does not let one failed item

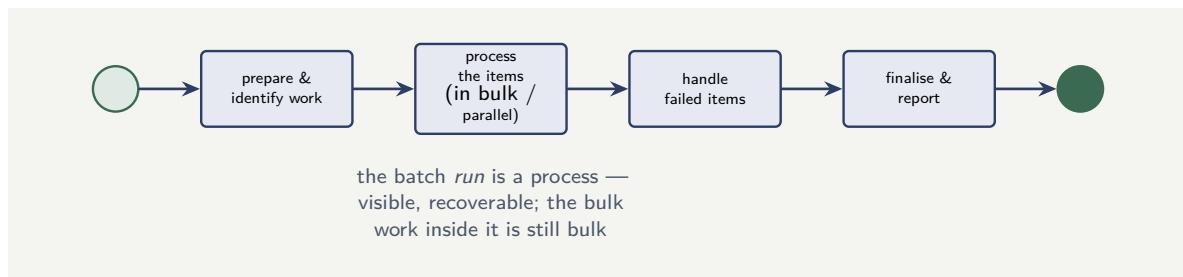


Figure 66.1. A batch run architected as an orchestrated process: modelled steps to prepare and identify the work, process it in bulk, handle failed items, and finalise and report. The run gains visibility, error handling, and recoverability — without making every item a process instance.

halt the whole batch, nor silently discard it. It *isolates* the failed item — sets it aside, records it — lets the rest of the batch complete, and routes the failed items to a defined handling path: a report, a retry, a human review. The failed-item path is part of the batch architecture, designed in advance.

Tip

A sound batch architecture is chunked, restartable, and explicit about failed items. *Chunking* processes the population in manageable groups, giving progress points and bounding rework. *Restartability* records progress so a failed batch resumes where it stopped, not from the beginning — essential when the population is too large to re-process within the window. And the *failed item* gets a designed path: isolated, recorded, and routed to retry or review, so one bad record neither halts the batch nor vanishes silently.

66.4 Batch and the process platform

The relationship between batch processing and a process platform is complementary, not competitive. The process platform orchestrates the *business process* — the sequence, the decisions, the human steps. Batch processing executes the *bulk data work* — the millions of records, the reconciliation, the interest accrual. A sound architecture uses both: the process platform reaches a step whose work is bulk, and that step launches a batch job and waits for its outcome. The process keeps the orchestration; the batch framework keeps the throughput.

Worked example: the batch that restarted from the beginning

Problem. A bank's overnight batch revalues every position in a large portfolio — a long-running job that takes most of the available overnight window. The batch is architected as a single monolithic run with no chunking and no progress recording. One night, about 80% of the way through, the batch fails — a transient infrastructure problem. The operations team restarts it. The restart begins again *from the first position*, and the batch does not finish before the window closes and the business day begins. Assess the architectural failure.

Setup. We consider what the monolithic, no-progress design forces a restart to do.

Working. The batch records no progress: it does not track which positions it has revalued.

So when it fails at 80% and is restarted, the restart has no knowledge of the 80% already done — it can only begin again from the first position:

no progress recording $\Rightarrow$ restart redoes everything, including the 80% already complete.

The batch had taken most of the window to reach 80%; restarting from zero cannot complete within what remains of the window. The revaluation does not finish before the business day starts — a serious operational failure — purely because the restart could not resume.

Discussion. The batch's logic was not at fault — it revalued positions correctly. The failure was *architectural*: the batch was built as a monolithic, unchunked, progress-blind run, and that design has no answer to the entirely ordinary event of a transient failure partway through. Because nothing recorded which positions were done, the only possible restart was from the beginning, and redoing 80% of a window-long job does not fit in the window that remains. The chapter's patterns are precisely the remedy. *Chunking* would have divided the portfolio into groups processed in sequence. *Restartability* — recording which chunks completed — would have let the restart skip the 80% already done and resume at the failed chunk, finishing comfortably within the window. The transient infrastructure problem would have been a minor interruption rather than a missed overnight deadline. The lesson is the chapter's: a batch architecture must be designed for the failure that will eventually happen partway through, and that means chunking and recorded progress so the batch is *restartable*. A batch that can only restart from the beginning is a batch that has not been architected for failure — and for a large, window-bound job, the first significant partway failure turns that omission into a missed deadline and a business-day impact. Restartability is not an optional refinement of a batch; for any batch large enough to matter, it is part of getting the batch right.

Practical next steps

For each significant batch job your institution runs, ask one question: if it fails 80% of the way through, does the restart resume from there, or from the beginning? If from the beginning, the batch is not architected for failure — and a window-bound batch that restarts from zero will eventually miss its deadline. Introduce chunking and recorded progress so the batch is genuinely restartable.

Chapter in brief

Batch, architected badly, is brittle and opaque; sound patterns make it robust. The foremost is to treat the batch *run* as an orchestrated process — modelled steps to prepare the work, process it in bulk, handle failed items, and finalise — gaining the visibility, error handling, and recoverability the engine gives any process, without making every item a process instance. Three further patterns: *chunking*, processing the population in manageable groups with progress points; *restartability*, recording progress so a failed batch resumes where it stopped rather than restarting from the beginning; and the explicit *failed-item* path, which isolates and records a failed item and routes it to retry or review, so one bad record neither halts the batch nor vanishes.

Batch and Real-Time: the Boundary

67.1 The two worlds meet

The first chapter of this part placed batch and real-time orchestration as suited to different work; the second treated batch architecture. This final chapter — the last of the manuscript’s technical parts — treats the *boundary* between the two, because in a real financial institution batch and real-time do not run in isolation: they meet, they hand work to each other, and the boundary between them must be architected deliberately.

67.2 How batch and real-time interact

Batch and real-time orchestration interact in recurring ways, and an architect should recognise the patterns.

A real-time process may *wait for a batch*. A process orchestrating something that depends on an overnight run — a settlement, an interest posting, an end-of-day position — reaches a point where it must wait for that batch to complete before continuing. This is a process waiting on an event, and the event is “the batch finished.”

A real-time process may *feed a batch*. Through the day, real-time processes accumulate items — transactions, instructions — that an overnight batch will later process together. The real-time processes are producers; the batch is the periodic consumer.

A batch may *trigger real-time processes*. A batch run may identify a population of items each needing individual follow-up — accounts flagged for review, exceptions needing handling — and for each it can start a real-time Camunda process. Here the batch is the producer of work, and the per-instance processes are the consumers.

The boundary, in other words, has defined crossing points: the process that waits for a batch, the process that feeds a batch, the batch that spawns processes. Each crossing must be designed — with the events, the signals, the correlation that the manuscript’s patterns provide.

Figure 67.1 sets out the crossing points between batch and real-time.

67.3 The architect's discipline at the boundary

The discipline the whole part has built toward is this: an institution’s work is neither all batch nor all real-time, and a sound architecture does not pretend otherwise. It places each kind of work in the pattern that suits it — periodic, population-wide work as batch; per-event work as real-time orchestration — and it architects the *boundary* between them deliberately, with explicit, designed crossing points rather than fragile, implicit assumptions.

The failure to avoid is the architecture that treats the boundary as accidental: a real-time process that simply *assumes* a batch has finished by the time it runs, with no actual wait or

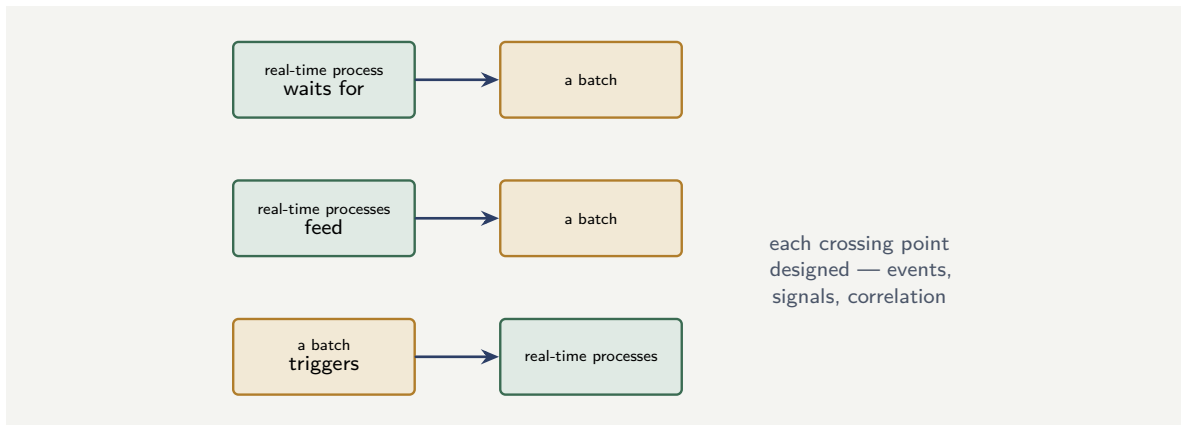


Figure 67.1. The boundary between batch and real-time. A real-time process may wait for a batch, real-time processes may feed a batch, and a batch may trigger real-time processes. Each crossing point is designed deliberately, with the events, signals, and correlation the platform provides.

check; a batch that assumes its input is ready with nothing ensuring it; a hand-off held together by timing coincidence rather than by an event. Such a boundary works until the batch runs late, the volume grows, the timing shifts — and then it fails silently and confusingly. A deliberate boundary — the process genuinely waits for the batch’s completion event, the batch genuinely confirms its input — holds.

The most fragile thing in a mixed batch-and-real-time architecture is a boundary held together by timing coincidence: a real-time process that assumes a batch has finished because it “usually has by now,” a batch that assumes its input is ready. Such a hand-off works until the batch runs late or the volume grows, then fails silently. Architect every batch–real-time crossing deliberately: the process waits for an actual batch-completion event, the batch confirms its input is genuinely present. Designed boundaries hold; assumed ones eventually do not.

67.4 Batch and the process platform

The relationship between batch processing and a process platform is complementary, not competitive. The process platform orchestrates the *business process* — the sequence, the decisions, the human steps. Batch processing executes the *bulk data work* — the millions of records, the reconciliation, the interest accrual. A sound architecture uses both: the process platform reaches a step whose work is bulk, and that step launches a batch job and waits for its outcome. The process keeps the orchestration; the batch framework keeps the throughput.

Worked example: the process that assumed the batch had finished

Problem. A bank has an overnight batch that posts the previous day’s transactions and finalises balances; it normally completes by about 3 a.m. The bank also has a Camunda process that produces and sends customers’ daily balance notifications, scheduled to run at 6 a.m. — comfortably after the batch “always” finishes. The 6 a.m. process does not check whether the

batch has completed; it simply runs and reads balances. One night, the transaction volume is unusually high and the batch does not finish until 7 a.m. Describe what happens and the boundary error.

Setup. We consider what the 6 a.m. process reads when the batch is still running.

Working. The notification process runs at 6 a.m. and reads balances — but on this night the batch is still running and will not finish until 7 a.m. The balances at 6 a.m. are therefore *not finalised* — the batch has not finished posting the previous day's transactions:

process runs at 6 a.m.+batch finishes at 7 a.m. $\Rightarrow$ notifications sent from unfinalised balances.

The bank sends every customer a daily balance notification containing a *wrong* balance, because the notification process read the data before the batch that produces it had finished.

Discussion. The boundary between the batch and the notification process was never actually architected — it was *assumed*, and the worked example shows exactly how an assumed boundary fails. The notification process and the batch have a real dependency: the notifications depend on the batch's output, the finalised balances. But that dependency was expressed only as a *timing coincidence* — “the batch finishes by 3 a.m., so a 6 a.m. process is safely after it” — with nothing in the architecture actually *enforcing* the ordering. For as long as the batch finished on time the coincidence held and the boundary appeared to work; the first night the batch ran late, the coincidence broke, and the unenforced dependency produced wrong balance notifications sent to the whole customer base. This is precisely the fragile, implicit boundary the chapter warns against. The correct design makes the crossing point explicit: the notification process must *genuinely wait for the batch's completion* — the batch emits a completion event or signal, and the notification process waits for that event before reading balances, regardless of the clock. Then a late batch simply delays the notifications until the data is genuinely ready, rather than causing wrong ones to be sent. The lesson is the part's closing discipline: a financial institution's architecture is a mixture of batch and real-time, the two have real dependencies at their boundary, and those dependencies must be architected with explicit events and waits — not left to the assumption that the clock will keep them in order. A boundary held by timing coincidence is not an architecture; it is a coincidence, and coincidences end.

Practical next steps

Find every place in your institution's architecture where a real-time process depends on a batch having finished, or a batch depends on its input being ready. For each, check whether the dependency is enforced by an actual event and wait, or merely assumed from timing. Every timing-assumed boundary is a silent failure waiting for the batch to run late. Make the crossing explicit: real waits on real completion events.

Chapter in brief

In a real financial institution batch and real-time orchestration do not run in isolation — they meet, and the boundary must be architected deliberately. The crossings recur: a real-time process may wait for a batch, real-time processes may feed a batch, and a batch may trigger real-time processes. Each crossing should be designed with explicit events, signals, and correlation. The failure to avoid is the boundary held together by timing coincidence — a process that assumes a batch “has finished by now,” a batch that assumes its input is ready — which works until the batch runs late or volumes grow,

then fails silently. A sound architecture places each kind of work in the pattern that suits it and architects the boundary between them with real waits on real completion events.

PART XI

Comparing Workflow Platforms

Choosing an Orchestration Approach

68.1 Why a comparison part

The preceding part set out a reference architecture and named Camunda as the process orchestrator at its centre. This part does something an honest architecture book must: it puts that choice under scrutiny. Camunda is not the only way to orchestrate work, and an architect — or an institution inheriting a mixed estate — needs a clear-eyed comparison of the realistic alternatives, not advocacy.

The chapters that follow compare five approaches an institution genuinely chooses between: *Camunda* itself; *Flowable*, its closest open-heritage relative; *AWS Step Functions*, the cloud-native orchestrator; the *Java Saga pattern*, the hand-built no-engine approach; and *IBM Business Automation Workflow*, the proprietary enterprise suite. Each chapter describes the approach fairly, on its own terms, and then says where it genuinely fits and where it does not. This first chapter sets up the comparison: the dimensions along which orchestration approaches actually differ, so that the chapters that follow compare like with like.

68.2 The dimensions that actually matter

Workflow platforms are marketed on long feature lists, most of which do not decide anything. An architect comparing approaches should hold to a small set of dimensions that genuinely separate them.

The modelling notation and its readability. Does the approach use a standard, business-readable notation — BPMN, DMN — that a non-technical process owner can review, or a proprietary or developer-only representation? For a regulated institution whose processes must be reviewed by business and risk, readability is not cosmetic.

The execution model and scaling. How does the engine execute and persist process state, and how does it scale — vertically only, or horizontally; to ordinary loads or to very high throughput?

Openness and lock-in. Is the approach open and portable, or does it tie the institution to one vendor or one cloud? Lock-in is a cost paid for years.

The operational model. Is the platform self-managed, a managed service, or embedded in the institution's own application — and what operational burden does each imply?

The human-task and decision capability. How deeply does the approach support human work — worklists, allocation, forms — and governed decisions, which financial processes lean on heavily?

The cost shape and the total cost of ownership. Licence cost, run cost, the cost of the skills required, the cost of lock-in — the honest total, not the sticker.

Figure 68.1 sets out the six dimensions the rest of the part compares on.

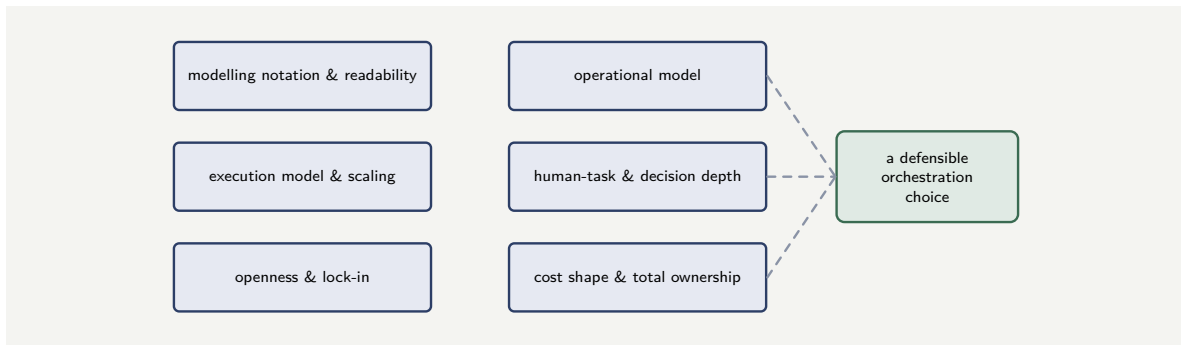


Figure 68.1. The six dimensions along which orchestration approaches genuinely differ. A defensible choice is made by comparing approaches on these — not on marketed feature lists, most of which decide nothing.

68.3 There is rarely one right answer

The most important framing for the whole part is that the comparison rarely yields a single universal winner. Each approach has a genuine home — a context where it is the right choice — and the architect’s task is not to crown one platform but to *match the approach to the context*.

A high-throughput, cloud-native institution; a regulated bank with deep human-work processes; a team building one small saga inside a microservice; an enterprise with a large existing investment in a proprietary suite — these contexts genuinely call for different answers. The chapters that follow each end by naming the context their approach fits. An architect who reads the part looking for “the best workflow platform” has asked the wrong question; the right question is “the best workflow platform *for this institution, this workload, this constraint set*” — and that is a question only the comparison, applied to a real context, can answer.

Tip

There is rarely a single best workflow platform — there is a best platform for a given institution, workload, and constraint set. Compare approaches on the six dimensions that matter — notation and readability, execution and scaling, openness, operational model, human-task and decision depth, and total cost — and match the approach to the context. An architect looking for a universal winner has asked the wrong question.

68.4 The engine as the institution's process memory

A workflow engine’s state store is, in a real sense, the institution’s *process memory*: it knows, for every case in progress, where it is, what has been done, what is outstanding, and what the data says. This memory survives restarts, deployments, and failures — it is durable, queryable, and auditable. An institution without a workflow engine has process memory only in the heads of the people doing the work, in spreadsheets, and in email threads — none of which survives a staff change, none of which a regulator can query.

Worked example: the same decision, two institutions

Problem. Two institutions ask the same question — “which workflow platform should we adopt?” — in the same month. Institution A is a digital-only challenger bank, entirely on

one public cloud, with very high transaction volumes, a strong engineering culture, and few long-running human-judgement processes. Institution B is a established multi-line insurer with a large on-premise estate, deeply human-involved underwriting and claims processes, a strong regulatory burden, and a modest engineering team. Should the comparison give them the same answer?

Setup. We hold the two institutions against the six dimensions and see whether the dimensions point the same way for both.

Working. For *Institution A*, the dimensions point toward scaling and cloud-native fit: very high throughput makes the execution-and-scaling dimension decisive, the single-cloud commitment lowers the weight of the lock-in dimension, and the thin human-task needs make that dimension nearly irrelevant. For *Institution B*, the same dimensions point elsewhere: deep human-involved processes make the human-task-and-decision dimension decisive, the on-premise estate and regulatory burden make openness and portability weigh heavily, and ordinary process volumes make extreme scaling nearly irrelevant. The two institutions weight the six dimensions so differently that:

the comparison cannot, and should not, give them the same answer.

Discussion. The worked example exists to make the part's framing concrete before the platform chapters begin. Institution A and Institution B asked the identical question and a sound comparison gives them different answers — not because the comparison is unreliable, but because the answer genuinely depends on the context, and the six dimensions are how the context enters the decision. For Institution A, an architect would weight scaling and cloud-native fit heavily and might land on a cloud-native orchestrator or Camunda 8; for Institution B, an architect would weight human-task depth, openness, and regulatory auditability, and would land somewhere those strengths are deepest. The mistake the part is written to prevent is the architect who has a favourite platform and recommends it to both institutions — because a recommendation that does not change when the context changes is not a comparison, it is a preference. The five chapters that follow each describe one approach honestly and name the context it fits; the reader's job is to hold those descriptions against their own institution's weighting of the six dimensions. The answer is real, but it is local.

Practical next steps

Before reading the platform chapters, write down how your institution weights the six dimensions: which two or three genuinely matter most, given your volumes, your human-work, your cloud posture, your regulatory burden, your team. The platform chapters will be far more useful read against an explicit weighting than read in search of a universal winner.

Chapter in brief

An honest architecture book must scrutinise its own central choice, so this part compares five orchestration approaches — Camunda, Flowable, AWS Step Functions, the Java Saga pattern, and IBM Business Automation Workflow. They genuinely differ on six dimensions: modelling notation and readability; execution model and scaling; openness and lock-in; operational model; human-task and decision depth; and total cost of ownership. Marketed feature lists mostly decide nothing; these six dimensions decide everything. And the comparison rarely yields a universal winner — each approach has a real home, and the architect's task is to match the approach to the institu-

tion, the workload, and the constraints, not to crown one platform.

Camunda as the Baseline

69.1 The platform this book has described

This part compares five approaches, and a fair comparison needs a baseline described as plainly as the alternatives. That baseline is Camunda — the platform the rest of this manuscript treats in depth — and this short chapter sets it out against the six dimensions, neither advocating nor apologising, so the chapters that follow have a clear thing to compare against.

69.2 Camunda against the six dimensions

Modelling notation and readability. Camunda’s central commitment is to the open standards: processes in BPMN, decisions in DMN, both business-readable and reviewable by non-technical process owners. This is, for a regulated institution, Camunda’s defining strength — the same artefact the engine executes is the artefact business and risk review.

Execution model and scaling. Camunda exists in two lineages an architect must not conflate. *Camunda 7* is the established Java-based engine, embeddable in an application, proven for ordinary process loads. *Camunda 8* is the newer, cloud-native platform built on the Zeebe engine of [Chapter 27](#), designed to scale horizontally to very high throughput. An institution choosing Camunda is really choosing between these two, and the choice turns on whether cloud-native horizontal scale is needed.

Openness and lock-in. Camunda is built on open standards and is portable across clouds and on-premise — it does not tie the institution to one infrastructure provider. The honest qualification is commercial: Camunda 8’s production components are a commercial product, so while the standards and the portability are real, the platform is not cost-free.

Operational model. Camunda 8 can be consumed self-managed — the institution runs it, per [Part VII](#)’s infrastructure chapters — or as a managed cloud service. The institution chooses its operational burden.

Human-task and decision depth. This is, with notation, Camunda’s other defining strength. The human-task machinery of [Chapter 30](#) — worklists, allocation, forms — and first-class DMN decisioning are deep and mature, which is exactly what financial processes, heavy with human judgement and governed decisions, lean on.

Cost shape. Camunda’s total cost is the commercial licence or subscription, the run cost of the infrastructure, and the cost of the skills — offset against the absence of cloud lock-in and the value of a standard, transferable skill base.

Tip

“Camunda” is two platforms, and an architect must not conflate them. *Camunda 7* is the established, embeddable Java engine for ordinary process loads; *Camunda 8* is the cloud-native, horizontally-scaling platform on the Zeebe engine. Many comparisons go wrong by comparing an alternative against “Camunda” without saying which — the

scaling story, the operational model, and the cost shape all differ between the two.

69.3 Where Camunda fits

Camunda's natural home follows directly from its strengths. It fits the institution that wants its processes as open, business-readable, governed BPMN and DMN artefacts; that has deep human-involved and decision-dense processes — which is most banks and insurers; that wants portability rather than commitment to one cloud; and that is willing to either run the platform or consume it as a service. For a regulated financial institution orchestrating complex, human-and-decision-heavy, end-to-end business processes, Camunda is a strong default — which is why this manuscript treats it as the baseline. The chapters that follow test that default against four alternatives, each of which fits some context better.

69.4 The baseline in concrete form: a minimal Camunda process

To ground the comparison, here is the simplest non-trivial Camunda 8 process: a service task, a gateway, a human task. This is the baseline against which every other platform in this part is measured.

Listing 69.1. A minimal Camunda 8 process: a service task that scores a case, a gateway that routes on the result, and a human task for borderline cases.

```
1 <bpmn:process id="baseline" isExecutable="true">
2   <bpmn:serviceTask id="score" name="Score">
3     <bpmn:extensionElements>
4       <zeebe:taskDefinition type="score-case"/>
5     </bpmn:extensionElements>
6   </bpmn:serviceTask>
7   <bpmn:exclusiveGateway id="route"/>
8   <bpmn:userTask id="review" name="Human Review"/>
9   <bpmn:sequenceFlow sourceRef="score" targetRef="route"/>
10  <bpmn:sequenceFlow sourceRef="route" targetRef="review">
11    <bpmn:conditionExpression>
12      =score < 600
13    </bpmn:conditionExpression>
14  </bpmn:sequenceFlow>
15 </bpmn:process>
```

The listing shows the three properties the chapter evaluates: the modelling notation (BPMN 2.0, standard), the execution mechanism (a job worker, the `zeebe:taskDefinition` binding), and the human-task model (a BPMN user task, rendered by Tasklist). Every comparison that follows measures the other platform against these three properties — and a platform that matches them is a viable alternative; one that does not is a different tool for a different purpose.

69.5 Evaluation criteria for a regulated institution

A financial institution evaluating workflow platforms should weight its criteria differently from a technology startup. The criteria that matter most are *auditability* (can the platform produce, for any past case, a complete record of what happened?), *human-task support* (does the platform have a first-class worklist, task allocation, and escalation model?), and *regulatory*

longevity (will the platform be supported for the decade-long horizon a regulated institution plans on?). Raw throughput, developer experience, and time-to-first-demo matter, but they are secondary to these three. A platform chosen for its demo speed but lacking auditability will fail its first regulatory examination.

Worked example: which Camunda, for which load

Problem. An institution has decided, in principle, on Camunda. It has two workloads. Workload one is a set of internal back-office processes — moderate volume, perhaps 5 process instances per second at peak, heavily human-involved, run on the institution’s own servers. Workload two is a customer-facing transaction-orchestration process expected to peak at 1,200 instances per second, cloud-deployed. The team assumes “we chose Camunda” is the whole decision. Explain what remains to be decided.

Setup. We hold each workload against the Camunda 7 / Camunda 8 distinction.

Working. *Workload one* — 5 instances per second, human-heavy, on-premise — is an ordinary process load well within the proven range of the established Java engine; an embeddable Camunda 7-lineage engine suits it, and the cloud-native horizontal scaling of Camunda 8 is capability it would not use. *Workload two* — 1,200 instances per second, cloud — is exactly the high-throughput, cloud-native case Camunda 8’s Zeebe engine was built for, and is beyond what the classic embeddable engine is designed to carry. So:

workload one → the established engine, workload two → Camunda 8 / Zeebe.

Discussion. The team’s assumption — “we chose Camunda, the decision is made” — is the exact conflation the chapter’s tip warned against. “Camunda” names two platforms with different execution models, and the two workloads land on different sides of the distinction: a moderate, human-heavy, on-premise load is well served by the established embeddable engine, while a 1,200-per-second cloud load needs Camunda 8’s horizontally-scaling Zeebe architecture. The worked example’s lesson is that even after an institution has chosen Camunda, the comparison is not over — the execution-and-scaling dimension still has to be applied, workload by workload, and an institution with both kinds of load may well run both lineages. Choosing the platform family is the first decision; matching the workload to the right engine within it is the second, and skipping the second is how a team either under-serves a high-throughput process or over-engineers a modest one.

Practical next steps

If your institution uses or is considering Camunda, be explicit about which lineage — the established embeddable engine or the cloud-native Camunda 8 — each workload calls for, judged on its throughput and deployment model. “We use Camunda” is not a complete architectural statement until that is settled.

Chapter in brief

Camunda is this part’s baseline, described against the six dimensions. Its defining strengths are open, business-readable BPMN and DMN modelling and deep human-task and decision capability — exactly what regulated, human-heavy financial processes need. It is open and portable, not tied to one cloud, though Camunda 8’s production components are commercial. Crucially, “Camunda” is two platforms: the estab-

lished embeddable Java engine for ordinary loads, and the cloud-native, horizontally-scaling Camunda 8 on Zeebe for high throughput — and an architect must say which. Camunda’s natural home is the regulated institution orchestrating complex, human-and-decision-heavy end-to-end processes, which is why the manuscript treats it as the default the alternatives are tested against.

Flowable: The Closest Relative

70.1 A shared ancestry

Of the alternatives in this part, *Flowable* is the one closest to Camunda — close enough that the comparison is genuinely subtle, and an architect should understand why. The closeness is not coincidence: it is *shared ancestry*. Both Camunda’s classic engine and Flowable descend from the same open-source project, Activiti — Camunda forked from it, and Flowable later forked from it as well. The two are, in a real sense, cousins, and the family resemblance runs deep.

70.2 What Flowable is

Flowable is an open-source, Java-based BPM engine. Like Camunda’s classic lineage, it implements the BPMN 2.0 standard for processes and DMN for decisions, it is embeddable in a Java application, and it executes and persists process instances against a relational database. An architect who knows Camunda 7 will find Flowable’s model of the world immediately familiar — the shared Activiti ancestry means the core concepts, and much of the developer experience, rhyme closely.

Flowable has its own emphases. It supports *CMMN* — Case Management Model and Notation — a standard for case-style work that Camunda chose, deliberately, not to support. It positions itself strongly as a flexible open-source engine that embeds neatly into an existing Java stack. And it is frequently chosen as an *embedded* workflow component inside a larger application — including, in banking, inside core systems — where its low integration friction with a Java codebase is a real advantage.

70.3 Flowable against the six dimensions

On *notation*, Flowable and Camunda are close peers: both BPMN 2.0 and DMN, both business-readable; Flowable additionally offers CMMN. On *execution and scaling*, Flowable is, like Camunda’s classic engine, a solid relational-database-backed engine well suited to ordinary process loads — it does not have a direct equivalent of Camunda 8’s cloud-native, horizontally-scaling Zeebe architecture, so for extreme-throughput, cloud-native workloads the comparison favours Camunda 8. On *openness*, Flowable’s open-source core is a genuine strength, and for an institution wary of commercial lock-in this weighs in Flowable’s favour. On the *operational model*, Flowable’s most natural mode is embedded in the institution’s own application. On *human-task and decision depth*, both are capable; Flowable’s CMMN support gives it an additional case-management story. On *cost*, Flowable’s open-source core can lower licence cost, though as always the total cost includes skills and operations.

Figure 70.1 draws the shared ancestry and the divergence.

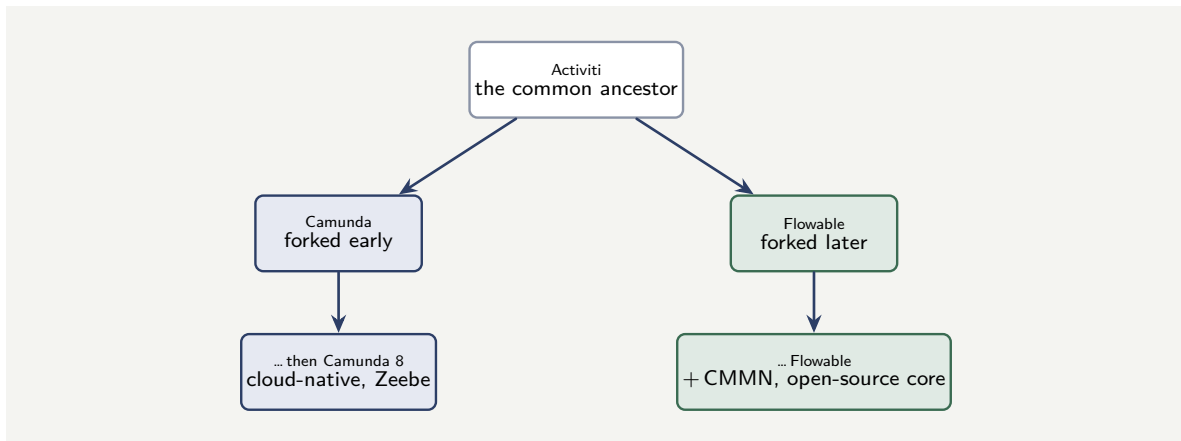


Figure 70.1. The shared ancestry of Camunda and Flowable. Both descend from the Activiti project; the family resemblance is deep. They diverged afterwards — Camunda toward the cloud-native Camunda 8, Flowable toward an open-source core with CMMN case-management support.

70.4 Where Flowable fits

Flowable’s natural home is the institution that wants a capable, standards-based, open-source BPM engine embedded in its own Java application — particularly where an open-source core matters for cost or governance reasons, where CMMN case management is genuinely wanted, and where the workload is ordinary process load rather than extreme cloud-native throughput. For a bank embedding workflow inside a core system’s Java stack, Flowable is a serious and well-fitted choice. Where the institution needs cloud-native horizontal scaling, the deepest commercial human-task tooling, or simply the larger surrounding ecosystem, the comparison tilts back toward Camunda — but Flowable is the closest alternative in the part, and for an embedded, open-source, ordinary-load case it can be the better fit.

70.5 Evaluation criteria for a regulated institution

A financial institution evaluating workflow platforms should weight its criteria differently from a technology startup. The criteria that matter most are *auditability* (can the platform produce, for any past case, a complete record of what happened?), *human-task support* (does the platform have a first-class worklist, task allocation, and escalation model?), and *regulatory longevity* (will the platform be supported for the decade-long horizon a regulated institution plans on?). Raw throughput, developer experience, and time-to-first-demo matter, but they are secondary to these three. A platform chosen for its demo speed but lacking auditability will fail its first regulatory examination.

Worked example: embedded engine or platform

Problem. A bank is building a new loan-servicing application as a single, self-contained Java application that the bank will run on its own infrastructure. The application needs workflow — a moderate-volume, human-involved servicing process — and the architecture team wants the workflow engine to sit *inside* the application, sharing its database and deployment, with the least possible integration friction. They are comparing Flowable and Camunda 8. Which way does the comparison point, and why?

Setup. We weight the six dimensions for this specific context — embedded, ordinary load,

Java, open-source-friendly.

Working. The decisive dimension here is the *operational model*. The team explicitly wants the engine embedded inside one Java application, sharing its database and deployment. Flowable's most natural mode is exactly that — an embeddable Java engine with low friction against an existing Java stack. Camunda 8, by contrast, is a cloud-native *platform* — a separate set of running components the application connects to, not a library embedded inside it; embedding is not its model. The workload is moderate, so Camunda 8's horizontal-scaling advantage is capability this case would not use. So for this context:

the comparison points to Flowable.

Discussion. This is the worked example the part's framing predicts: the same six dimensions that would favour Camunda 8 for a high-throughput, cloud-native customer-facing workload favour Flowable here, because the context is different. The team wants an *embedded* engine, and embedding inside a single Java application is Flowable's natural mode and not Camunda 8's — Camunda 8 is a platform of separate components, architecturally a different shape. The moderate volume removes the one dimension — extreme scaling — on which Camunda 8 would have pulled the decision back. The lesson is not that Flowable is better than Camunda 8; it is that “better” is meaningless without the context, and in the embedded, ordinary-load, Java-stack context, Flowable's strengths line up with what the team actually needs. An architect who recommended Camunda 8 here — because it is newer, or because it is this book's baseline — would be fitting the institution to the platform rather than the platform to the institution. The closest relative wins when the context is its home.

Practical next steps

If your institution embeds workflow inside its own Java applications, give Flowable a genuine evaluation rather than assuming the baseline. Its open-source core, its Activiti-shared familiarity, its CMMN support, and its natural fit as an embedded engine make it a real contender for that context — and the comparison should be decided on the dimensions, not on which platform is most discussed.

Chapter in brief

Flowable is Camunda's closest relative — both descend from the Activiti open-source project, and the family resemblance is deep. Flowable is an open-source, Java-based, BPMN 2.0 and DMN engine, embeddable in a Java application, and it additionally supports CMMN case management, which Camunda deliberately does not. Against the six dimensions it is a close peer of Camunda's classic engine, without a direct equivalent of Camunda 8's cloud-native horizontal scaling. Flowable's natural home is the institution wanting a capable, standards-based, open-source engine embedded in its own Java application, at ordinary process loads — where an open-source core, CMMN, and low embedding friction matter. For that context the closest relative can be the better fit.

AWS Step Functions and the Java Saga, Revisited

71.1 Two approaches already met

Two of this part's five approaches have already been treated in depth: [Chapter 15](#) compared AWS Step Functions with Camunda, and [Chapter 16](#) treated the Java Saga pattern. This chapter does not repeat those treatments; it *places* both approaches into this part's six-dimension comparison, so the part's picture is complete and the two sit beside Flowable and IBM in one consistent frame.

71.2 AWS Step Functions in the comparison

AWS Step Functions, as [Chapter 15](#) set out, is a serverless orchestration service: state machines defined in the JSON-based Amazon States Language, with Standard and Express workflow modes, native integration with Lambda and other AWS services, built-in retry and a callback pattern.

Against the six dimensions: on *notation*, Step Functions uses the Amazon States Language — a JSON definition for engineers, not a business-readable standard like BPMN, which is a real difference for a regulated institution whose process owners must review the model. On *execution and scaling*, it is serverless and scales as an AWS service. On *openness*, it is the sharp point: Step Functions ties the institution to AWS — it is the least-portable approach in the part. On the *operational model*, it is fully managed — no cluster to run, the lowest operational burden of any approach here. On *human-task depth*, it has the callback pattern but nothing like Camunda's worklist machinery. On *cost*, it is pay-per-use, with no licence, but the cost of the AWS lock-in is real and paid in flexibility.

Its home, as [Chapter 15](#) concluded, is orchestrating technical sequences of AWS services within an institution already committed to AWS — not the regulated, human-heavy, portable-by-requirement business process.

One property deserves its own note, because it is a genuine strength and a point of comparison — and because it is the source of a common confusion. AWS Step Functions is a *durable* orchestrator: it persists the state of a running workflow, so that the workflow survives failures and can suspend for long periods, waiting, without consuming compute, and resume exactly where it left off. This durability is real and is necessary for any serious orchestrator — it is the same long-running, waiting, crash-surviving property of [Chapter 3](#).

It is important, though, not to conflate AWS Step Functions with a second, distinct AWS capability that an architect will increasingly encounter: *AWS Lambda durable functions*. These are not the same product, and the difference matters. AWS Step Functions, as described, is a *platform-level orchestrator* — the workflow is defined as a graph in the JSON-based Amazon States Language, separate from application code, and the service coordinates across many AWS services. AWS Lambda durable functions, introduced much more recently, is a *code-*

first capability: it brings durable execution *inside* Lambda function code, using a checkpoint-and-replay SDK so that ordinary code — written in a standard programming language, with familiar *await*-style syntax — can checkpoint its progress, suspend, and resume. The two solve overlapping problems — reliable, long-running, stateful execution — but they are different things: Step Functions is the JSON-defined orchestration *graph* across services; Lambda durable functions is durable execution *written as code* within a function. AWS positions them for different development styles, and an architect comparing “AWS” against Camunda must be precise about *which* AWS capability is meant.

For this part’s comparison, the point holds for both. Whether durability is delivered by Step Functions’ managed orchestration graph or by Lambda durable functions’ in-code checkpointing, it is delivered *inside the AWS-locked* envelope — and Step Functions additionally inside the JSON-defined, human-task-thin envelope the dimensions already described, while Lambda durable functions, being code, has no business-readable notation at all. Durable execution is necessary for a serious orchestrator; it is not, by itself, sufficient to make either AWS approach fit for a regulated, portable, human-heavy business process.

71.3 The Java Saga in the comparison

The *Java Saga pattern*, as [Chapter 16](#) set out, is not a platform at all — it is a hand-built coordinator class, with no engine: a sequence of steps each paired with a compensating action, run in code.

Against the six dimensions it is deliberately the minimal approach. On *notation*, there is none — the flow is Java code, readable only by developers. On *execution*, the state lives in the running process’s memory, which is the pattern’s defining limit: a crash mid-saga loses the record of what to compensate. On *openness*, it is just code, so there is no lock-in at all. On the *operational model*, there is no platform to operate — it lives inside an existing service. On *human-task depth*, there is none — it is for short, technical, human-free sagas. On *cost*, it is the cheapest to start and carries no licence, but it buys that cheapness by having no durability, no visibility, and no governance.

Its home, as [Chapter 16](#) concluded, is the short-lived, in-service, human-free, technical saga where an engine would be over-engineering — and emphatically not the long-running, regulated, human-involved distributed transaction.

71.4 Activiti in the comparison

One more approach belongs in this comparison, because it is part of Camunda’s own family: *Activiti*. [Chapter 14](#) described the shared Activiti ancestry of Camunda and Flowable; here Activiti is treated as a candidate platform in its own right, because it is one — a live, maintained, open-source BPM engine an institution can genuinely choose.

Activiti is a lightweight, Java-centric, open-source BPMN 2.0 engine, released under the Apache licence. Like Camunda’s classic engine and Flowable — its relatives — it executes business processes defined in BPMN 2.0, it is embeddable in a Java application, and it persists process state in a relational database, which makes it genuinely *durable*: a running Activiti process survives a crash, because its state is in the database, not in memory. It provides a process engine, task management, and process modelling, and the project has extended toward the cloud with *Activiti Cloud*, a set of cloud-native building blocks.

Against the six dimensions, Activiti looks much like its close relative Flowable of [Chapter 14](#): BPMN-based and business-readable on *notation*; a solid relational-database-backed,

dimension	AWS Step Functions	Java Saga pattern	Activiti
notation	JSON (ASL)	none — Java code	BPMN 2.0
scaling	serverless	in-memory, fragile	ordinary loads
durability	durable — managed	none — lost on crash	durable — DB-backed
openness	AWS lock-in	none — just code	open source
human tasks	callback only	none	yes — BPM engine

Figure 71.1. AWS Step Functions, the Java Saga pattern, and Activiti in the part’s frame. AWS Step Functions is a durable, serverless, AWS-locked orchestrator — distinct from AWS Lambda durable functions, the separate code-first capability discussed in the text; the Java Saga is an undurable in-code pattern; Activiti is an open-source, durable, BPMN-based engine of Camunda’s own family. Each has a genuine home, and each is a poor fit for some contexts.

durable engine for ordinary loads on *execution*, without a direct equivalent of Camunda 8’s cloud-native horizontal scaling; genuinely open on *openness*; naturally embedded on the *operational model*; and a real BPM engine on *human-task depth*, with task management built in. Its home is the institution wanting a lightweight, open-source, embeddable, durable BPMN engine for ordinary process loads — close to Flowable’s home, and the comparison between Activiti, Flowable, and Camunda’s classic engine turns on the finer distinctions of community, momentum, ecosystem, and the surrounding tooling rather than on a sharp difference of kind, since all three descend from the same Activiti root.

Figure 71.1 places the three further approaches in the part’s comparison frame.

71.5 Evaluation criteria for a regulated institution

A financial institution evaluating workflow platforms should weight its criteria differently from a technology startup. The criteria that matter most are *auditability* (can the platform produce, for any past case, a complete record of what happened?), *human-task support* (does the platform have a first-class worklist, task allocation, and escalation model?), and *regulatory longevity* (will the platform be supported for the decade-long horizon a regulated institution plans on?). Raw throughput, developer experience, and time-to-first-demo matter, but they are secondary to these three. A platform chosen for its demo speed but lacking auditability will fail its first regulatory examination.

Worked example: four approaches, one workload

Problem. An institution must orchestrate a regulated, long-running mortgage-origination process: it runs for weeks, involves six human approvals, must be reviewable by non-technical process owners and a regulator, and the institution wants to avoid commitment to any single cloud. A team proposes, variously, AWS Step Functions, the Java Saga pattern, Flowable, and Camunda. Use the six dimensions to eliminate the poor fits.

Setup. We test each approach against the workload’s decisive dimensions: business-readable notation, human-task depth, long-running durability, and portability.

Working. The *Java Saga pattern* fails immediately: it has no business-readable notation, no human-task capability, and no durable state for a weeks-long process — it is for short, technical, human-free sagas, and this process is none of those.

Java Saga: eliminated.

AWS Step Functions fails on two decisive dimensions: its Amazon States Language is a JSON definition, not a business-readable notation a process owner and regulator can review, and it ties the institution to AWS, which the institution explicitly wants to avoid.

AWS Step Functions: eliminated.

That leaves *Flowable* and *Camunda* — both BPMN-based, business-readable, both capable of long-running human-involved processes, both portable. The decision between the two then turns on the finer dimensions of [Chapter 70](#): embedded versus platform, the depth of commercial human-task tooling, cloud-native scaling.

Flowable or Camunda: the real comparison.

Discussion. The worked example shows the six dimensions doing exactly the work the part is built around: not crowning a winner, but *eliminating the poor fits* so the genuine comparison is clear. The Java Saga and AWS Step Functions are not bad approaches — earlier chapters were at pains to give each its real home — but for *this* workload, a regulated, long-running, human-heavy, portability-required business process, each fails on dimensions that are decisive: the Saga on notation, human tasks, and durability all at once; Step Functions on notation and openness. Eliminating them is not a verdict on the approaches; it is a verdict on their fit to this context. What remains — Flowable and Camunda — are the two approaches whose strengths actually line up with the workload, and the choice between them is the genuine, finer comparison. This is how the part is meant to be used: run the candidate approaches against the dimensions the specific workload makes decisive, let the poor fits fall away, and spend the real deliberation on the approaches that survive.

Practical next steps

For a workload your institution must orchestrate, list the two or three dimensions the workload makes *decisive* — and run all candidate approaches against just those. Approaches that fail a decisive dimension are eliminated regardless of their other merits; the real comparison is among the survivors.

Chapter in brief

This chapter places two approaches already treated in depth — AWS Step Functions and the Java Saga pattern — into the part’s six-dimension frame. Step Functions is a serverless AWS orchestrator: JSON-defined, not business-readable; fully managed, lowest operational burden; but the least-portable approach, tying the institution to AWS. Its home is AWS-native technical orchestration. The Java Saga is no platform at all — a hand-built coordinator in code: no notation, in-memory fragile state, no human tasks, but no lock-in and the cheapest start. Its home is short, in-service, human-free

sagas. Both are legitimate for their context and poor fits for the regulated, human-heavy, portable business process — and the six dimensions are how an architect eliminates the poor fits and finds the real comparison.

IBM Business Automation Workflow and the Proprietary Suites

72.1 The incumbent in many institutions

The four approaches compared so far are, in different ways, modern: open engines, a cloud service, a code pattern. But a great many established banks and insurers run their processes today on something different — a *proprietary enterprise BPM suite*, bought years ago, deeply embedded, and very much still running. The most prominent in financial services is *IBM Business Automation Workflow* — IBM BAW — the current name for the lineage that earlier institutions knew as IBM BPM and, before that, as other IBM workflow products. This chapter compares it honestly, because for many institutions IBM BAW is not a hypothetical alternative — it is the incumbent the institution is comparing *everything else against*.

The chapter speaks of IBM BAW specifically but its observations apply broadly to the class of proprietary enterprise BPM suites — Pega, Appian, Oracle’s BPM, TIBCO’s — which share a common shape and a common set of trade-offs.

72.2 What IBM BAW is, fairly stated

IBM BAW is a mature, comprehensive, proprietary enterprise BPM suite. It is genuinely capable, and a fair comparison must say so. It provides process automation, case management, human-task management, business rules, forms, and extensive tooling, integrated into one vendor-supported product. It has decades of investment behind it, deep capabilities, and — for the institutions that run it — years of accumulated processes, skills, and operational knowledge. An architect who dismisses it as merely “legacy” is not comparing honestly: IBM BAW does real work, reliably, in many institutions, today.

72.3 The architecture of IBM BAW, in technical terms

A fair comparison needs more than a characterisation — it needs the actual shape of the platform, because the comparison’s dimensions only become concrete against the real architecture.

IBM BAW is, by origin, a *unification of two formerly separate IBM products*: IBM Business Process Manager — IBM BPM, the structured-process engine — and IBM Case Manager — ICM, the case-management product. BAW merged them so that one platform offers both *structured BPMN processes* and *unstructured cases*, and lets a solution sit anywhere on the spectrum between the two rather than being forced into one model. This dual heritage is visible throughout the product and is genuinely one of its strengths.

The platform’s principal components are worth naming, because they are what an institu-

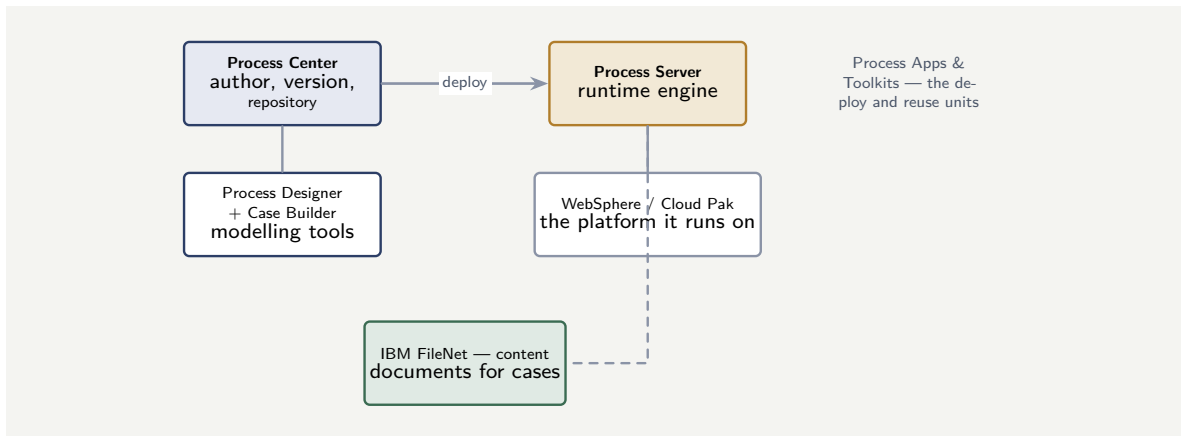


Figure 72.1. The principal components of IBM BAW. The Process Center is the design-time repository; the Process Server is the runtime, traditionally on WebSphere and, in current form, optionally on the Kubernetes-based Cloud Pak; Process Designer and Case Builder are the modelling tools; and IBM FileNet provides the content layer. Process Applications are the deployable unit, Toolkits the reuse mechanism.

tion running BAW actually operates and what a migration actually has to account for.

The *Process Center* is the development and repository hub — the shared environment where process applications are authored, versioned, and stored, and from which they are deployed. It is the design-time heart of the platform.

The *Process Server* is the runtime — the engine that executes the deployed process applications and cases. In a traditional BAW topology the Process Server runs on *IBM WebSphere Application Server*, the Java EE application server that underpins much of IBM’s middleware; this is a significant technical fact, because it ties BAW’s operational model to WebSphere administration, WebSphere clustering, and WebSphere’s resource model.

The *Process Designer* is the web-based modelling tool in which structured processes are authored, and *Case Builder* is the tooling for designing case solutions — the unstructured side of the platform.

Process logic is packaged into *Process Applications* — the deployable unit — and shared logic is factored into *Toolkits*, BAW’s mechanism for reuse across process applications. An architect comparing BAW to Camunda should note the parallel and the difference: a Toolkit is a reuse unit, as a shared BPMN process is in Camunda, but it is a *suite-specific* construct, authored and consumed only within BAW’s tooling.

For content and documents, BAW integrates with *IBM FileNet* — the Content Platform Engine — as its document and content layer; case solutions in particular lean on this content integration, and in many deployments the FileNet content engine is a separate, separately-licensed component alongside BAW itself.

Finally, in its current positioning BAW is offered as part of *IBM Cloud Pak for Business Automation* — IBM’s containerised automation platform — so a modern BAW deployment may run on a Kubernetes-based Cloud Pak rather than a traditional WebSphere installation. This matters to the comparison: it is IBM’s own move to modernise the operational model, and an architect comparing BAW to Camunda should compare against the deployment form the institution actually runs, not an assumed one.

Figure 72.1 sets out BAW’s principal components.

72.4 IBM BAW against the six dimensions

The honest comparison is not that IBM BAW is incapable; it is that its trade-offs differ sharply from the open approaches on dimensions a modern institution increasingly weights heavily.

On *notation*, IBM BAW supports BPMN, though, like several proprietary suites, with vendor-specific extensions and tooling; its models are authored in Process Designer and bound to BAW's own constructs — coach views for forms, the Toolkit reuse model, BAW's specific service and variable types — so a process is not as cleanly portable as the BPMN standard implies, and a BAW process is not, in practice, a process another engine can simply run. On *execution and scaling*, it is a capable enterprise engine, but traditionally architected on WebSphere Application Server, with scaling achieved through WebSphere clustering and vertical capacity rather than the cloud-native, horizontally-scaling design newer platforms emphasise — the Cloud Pak packaging is IBM's move to address exactly this, and an architect should establish which model the institution actually runs. On *openness*, this is the decisive dimension and IBM BAW's hardest: it is a proprietary suite, and an institution running it is, to a real degree, *locked in* — the processes, the coach-view forms, the Toolkits, the skills, the WebSphere and FileNet operational knowledge, the integrations are all suite-specific, and that lock-in is the cost the institution has been paying, often without naming it, for years. On the *operational model*, it is a heavyweight platform: a traditional deployment is a WebSphere installation with the Process Center, one or more Process Servers, a database, and — for case solutions — a FileNet content engine, typically run by a specialist team holding WebSphere, BAW, and FileNet skills together. On *human-task and decision depth*, it is genuinely strong — comprehensive, mature human-workflow, the structured-process and unstructured-case duality inherited from the BPM-and-Case-Manager merger, and rules capability are exactly what the proprietary suites were built for. On *cost*, this is the other hard dimension: proprietary enterprise suites carry substantial licence costs — and, in BAW's case, the FileNet content engine and other components may be separately licensed — so the total cost of ownership — licence, specialist WebSphere-and-BAW-and-FileNet skills, the lock-in — is high, which is precisely what drives many institutions to evaluate alternatives.

The cost of a proprietary BPM suite is not only its licence. It is the licence *plus* the lock-in — processes, skills, tooling, and integrations all specific to one vendor — and lock-in is a cost paid silently, every year, in reduced flexibility and constrained negotiating position. An institution comparing IBM BAW (or any proprietary suite) against the open alternatives must count the lock-in as a real cost, not treat it as free because no invoice names it.

72.5 The architectural contrast that shapes a migration

Beyond the six dimensions, two architectural differences between IBM BAW and a platform like Camunda are technical enough, and consequential enough, to deserve their own statement — because they are what make a BAW-to-Camunda migration a genuine re-architecture rather than a file conversion.

The first is the *coach view* and the embedded user interface. BAW's human tasks are presented through *coach views* — BAW's own UI technology, in which the task forms and screens are built inside the BAW tooling and bound to the process. The user interface is, to a significant degree, *inside* the BAW platform. A platform like Camunda takes the other approach:

the process engine exposes tasks through an API, and the user interface is a separate concern — a Tasklist, or a custom application built with ordinary web technology. This is a real architectural divergence: BAW processes carry their UI with them; a migration must therefore separate the genuine process from the coach-view UI and rebuild the interface as an independent layer, which is design work, not translation.

The second is the *unit of execution and the runtime model*. A BAW process application is a deployable artefact bound to the Process Server and, traditionally, to WebSphere — it runs as part of a heavyweight, vertically-scaled application-server installation. Camunda 8's runtime is the cloud-native Zeebe engine of [Chapter 27](#), designed for horizontal scaling, with process applications and connectors as independently deployable pieces. Moving from one to the other is not redeploying the same thing on new infrastructure; it is moving to a different runtime model, with different scaling, different operational tooling, and a different topology.

These two differences are the reason [Chapter 75](#)'s treatment of migrating from IBM BAW is a substantial chapter and not a paragraph: the BPMN diagram may look superficially translatable, but the coach views, the Toolkits, the WebSphere-bound runtime, and the FileNet content integration are all genuine architecture that a migration must consciously re-home.

Tip

A BAW-to-Camunda migration is not a BPMN file conversion. BAW carries its user interface inside the platform as *coach views*, factors reuse into suite-specific *Toolkits*, runs its *Process Server* on WebSphere (or Cloud Pak), and integrates *FileNet* for content. Each of these is real architecture with no automatic Camunda equivalent — the UI becomes a separate layer, the runtime becomes cloud-native Zeebe, the content integration is re-designed. Scope a migration against the whole architecture, not the diagrams alone.

The honest conclusion is balanced. For an institution that *already runs* IBM BAW, that has years of working processes on it, a skilled team, and no acute pain, the suite's maturity and the sunk investment are real, and ripping it out for its own sake is not sound architecture. IBM BAW's natural home is the institution already committed to it and content with the trade-off.

But the comparison also explains a clear industry direction. The dimensions on which the proprietary suites are weakest — openness and total cost — are exactly the dimensions modern institutions increasingly weight most heavily, as they tire of lock-in and licence cost and want open standards, portability, and a transferable skill base. That is why so many institutions running IBM BAW are evaluating, or undertaking, a move to an open platform such as Camunda — and why this manuscript's next part is devoted entirely to migration. The proprietary suite is the incumbent; the open platform is, for many institutions, the deliberate destination; and the journey between them is involved enough to need its own part.

72.6 Evaluation criteria for a regulated institution

A financial institution evaluating workflow platforms should weight its criteria differently from a technology startup. The criteria that matter most are *auditability* (can the platform produce, for any past case, a complete record of what happened?), *human-task support* (does the platform have a first-class worklist, task allocation, and escalation model?), and *regulatory longevity* (will the platform be supported for the decade-long horizon a regulated institution plans on?). Raw throughput, developer experience, and time-to-first-demo matter, but they

are secondary to these three. A platform chosen for its demo speed but lacking auditability will fail its first regulatory examination.

Worked example: the lock-in that did not appear on an invoice

Problem. A bank runs its core processes on a proprietary BPM suite. Asked by its board to compare the cost of staying versus moving to an open platform, the finance team produces a straightforward figure: the suite's annual licence is \$2.0M, and a move would cost an estimated \$6M one-off, so — they conclude — staying is cheaper for at least three years. An architect argues the comparison is incomplete. What has the finance team's figure omitted?

Setup. We identify the costs of the proprietary suite that do not appear as a licence line.

Working. The licence is \$2.0M a year and is visible. The costs the figure omits are the ones the chapter's warning names. The bank's processes, tooling, integrations, and staff skills are all *suite-specific* — so the bank pays a *lock-in cost*: specialist staff command a premium and are hard to replace; the bank has little negotiating leverage at licence renewal, because the vendor knows moving is hard; process change is constrained to what the suite allows and priced accordingly; and the bank cannot easily adopt innovations outside the suite. None of these appears on an invoice, but each is real:

$$\text{true cost of staying} = \$2.0\text{M licence} + \text{lock-in cost (unpriced)}.$$

The finance team compared a fully-costed move against a *partially*-costed stay, and a comparison of a complete figure with an incomplete one is not a comparison.

Discussion. The architect is right, and the worked example exists to make the chapter's central caution concrete. The finance team did nothing dishonest — they costed what was visible, and a licence figure is very visible. But the proprietary suite's true cost is the licence *plus* the lock-in, and the lock-in is invisible precisely because no invoice itemises “constrained negotiating position” or “skills that do not transfer.” By comparing a fully-loaded migration cost against a licence-only cost of staying, the team's three-year conclusion is built on an unequal comparison, and the real picture — once the lock-in is counted — may well favour the move sooner than three years, or may not, but cannot be known until both sides are costed the same way. The lesson generalises beyond this bank: every comparison involving a proprietary suite must put the lock-in on the books, as a real if hard-to-quantify cost, or the proprietary option will always look artificially cheap, because its biggest cost is the one that never sends an invoice. An architecture decision made on visible costs alone systematically favours lock-in — and that is exactly the bias an honest comparison must correct.

Practical next steps

If your institution runs a proprietary BPM suite, press for a comparison that costs the *lock-in* — the skills premium, the renewal leverage lost, the constrained pace of change — alongside the licence. The licence is the easy number; the lock-in is the large one. A stay-versus-move decision made on the licence alone is built on an unequal comparison.

Chapter in brief

IBM Business Automation Workflow — IBM BAW, formerly IBM BPM — is the incumbent proprietary enterprise BPM suite in many financial institutions, and a fair comparison must grant that it is genuinely capable: comprehensive, mature human-workflow, case, and rules capability, with years of working processes behind it. Its trade-offs, against the six dimensions, are sharp on two: *openness*, where it is a proprietary suite carrying real lock-in; and *cost*, where the licence plus the unpriced lock-in make total cost of ownership high. Its home is the institution already committed to it and content with the trade-off — but the dimensions on which it is weakest are exactly the ones modern institutions weight most heavily, which is why so many are moving to open platforms, and why the next part is devoted to migration.

webMethods and the Integration-Platform Approach

73.1 A different kind of competitor

The previous chapter compared IBM Business Automation Workflow — a dedicated *BPM suite*. This chapter compares a platform of a different character again: *Software AG's webMethods*. webMethods belongs in this comparison part because, for a great many established financial institutions, it is a genuine incumbent and a genuine alternative — but it is not, at heart, a BPM platform. It is an *integration* platform that also does process management, and that distinction is the whole point of the chapter.

An architect who compares “webMethods versus Camunda” as though comparing two BPM engines will compare them wrongly. The two are not the same kind of thing, and a fair comparison must begin by being clear about what webMethods actually is.

73.2 What webMethods is

webMethods is an *integration platform*. Its centre of gravity — its origin and its defining strength — is the *Enterprise Service Bus*: the machinery for connecting systems, applications, data sources, B2B partners, and APIs across an enterprise. At its core sits the webMethods Integration Server, a Java-based, multi-platform enterprise integration server.

Around that integration core, webMethods is sold as a *unified suite*: the ESB, plus API management, plus B2B integration, plus managed file transfer — *and* business process management. The BPM capability is tightly integrated with the ESB and the other modules, which is genuinely a strength for orchestrating end-to-end processes across many systems. webMethods has a large library of connectors — on the order of hundreds — to enterprise systems, ERPs, databases, cloud applications, and legacy systems.

Two further facts shape the comparison. First, webMethods has historically used a *proprietary orchestration language* — “Flow” — alongside its support for standards, so its processes are not purely standard-notation artefacts. Second, its development environment, webMethods Designer, is powerful but widely regarded as having a *steep learning curve* and demanding strong integration and Java experience — it is an integration developer's tool, not a business analyst's.

73.3 webMethods against the six dimensions

Against the comparison's six dimensions, webMethods' integration-first nature shows clearly.

On *notation and readability*, webMethods is the weaker of the comparison for a regulated institution's purpose: its heritage includes the proprietary Flow language, its processes are authored in an integration developer's tool, and the artefacts are not the business-readable, business-reviewable BPMN-and- DMN models that are Camunda's defining strength. A pro-

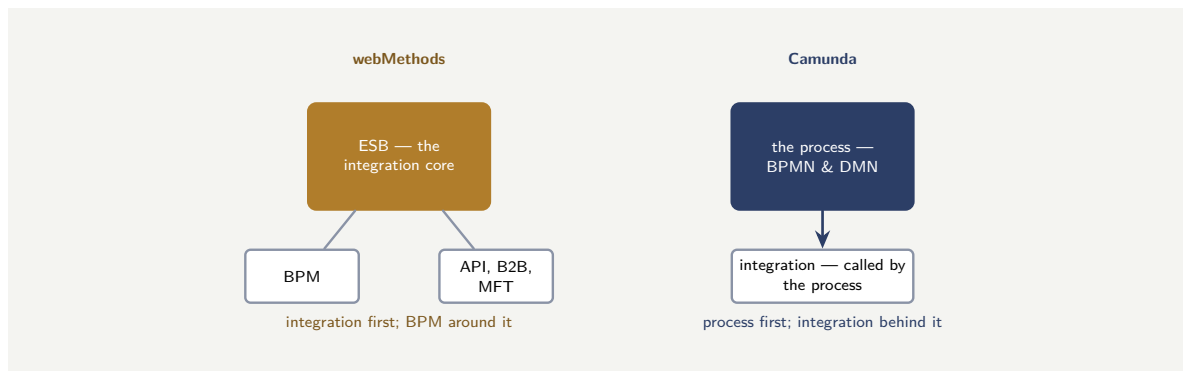


Figure 73.1. webMethods and Camunda are different kinds of platform. webMethods is integration-first — the ESB at the centre, BPM as one module around it. Camunda is process-first — the business-readable BPMN-and-DMN process at the centre, integration a layer the process calls.

cess owner or risk reviewer cannot read a webMethods integration the way they can read a BPMN model.

On *execution and scaling*, webMethods is a proven, enterprise-grade platform, built to carry serious integration loads — this is its home ground.

On *openness*, webMethods is a proprietary commercial platform, and an institution running it is, as with any proprietary suite, subject to vendor lock-in — the processes, the Flow services, the connectors, the skills are all webMethods-specific.

On the *operational model*, webMethods is a heavyweight platform: its installation, configuration, and upgrades are substantial undertakings requiring deep specialist knowledge.

On *human-task and decision depth*, webMethods' BPM module provides human-workflow capability, but human-centric business process management is not where the platform's centre of gravity lies — it is an integration platform first.

On *cost*, webMethods carries the licence cost of a comprehensive proprietary enterprise suite, plus the cost of the specialist skills its steep learning curve demands, plus the lock-in.

Figure 73.1 contrasts the integration-first and process-first shapes.

73.4 Where webMethods fits

The honest conclusion is balanced and follows from what webMethods is.

webMethods' natural home is the institution whose dominant problem is *integration* — connecting a large, heterogeneous estate of systems, B2B partners, and APIs — and which wants that integration capability and its process management in one unified, vendor-supported suite. For deep enterprise integration, webMethods is a serious and capable platform, and an institution already running it for that purpose has a real asset.

But for the problem this manuscript is about — orchestrating complex, governed, human-and-decision-heavy, business-readable end-to-end *business processes* — the comparison favours a process-first platform. The business-readable BPMN-and-DMN models, the human-task depth, the openness, and the analyst-accessible tooling are exactly the dimensions on which an integration-first platform is weakest. This is the same conclusion Chapter 86's migration treatment reached from the other direction: an institution moving from webMethods to Camunda is, rightly, separating the genuine business process — which belongs in a process-first platform — from the integration logic — which belongs in an integration layer. The comparison and the migration tell one story: webMethods is an integration platform, Camunda is a

process orchestrator, and an institution should be clear about which problem it is solving.

Tip

“webMethods versus Camunda” is not a comparison of two BPM engines — webMethods is an *integration platform* (an ESB at its core) that also does BPM, and Camunda is a *process orchestrator*. Compare them by the problem being solved: for connecting a large heterogeneous estate of systems and B2B partners, webMethods’ integration heritage is a genuine strength; for orchestrating governed, business-readable, human-and-decision-heavy end-to-end business processes, a process-first platform wins on notation, human-task depth, openness, and analyst-accessible tooling.

73.5 Evaluation criteria for a regulated institution

A financial institution evaluating workflow platforms should weight its criteria differently from a technology startup. The criteria that matter most are *auditability* (can the platform produce, for any past case, a complete record of what happened?), *human-task support* (does the platform have a first-class worklist, task allocation, and escalation model?), and *regulatory longevity* (will the platform be supported for the decade-long horizon a regulated institution plans on?). Raw throughput, developer experience, and time-to-first-demo matter, but they are secondary to these three. A platform chosen for its demo speed but lacking auditability will fail its first regulatory examination.

Worked example: the integration platform asked to be a BPM platform

Problem. A bank runs webMethods, originally bought to integrate its large estate of systems — and it does that job well. A new programme needs to build the bank’s customer-onboarding and lending processes: complex, long-running, regulated, human-heavy, and required to be reviewable by business process owners and risk. A team proposes to build these in webMethods’ BPM module, reasoning “we already own webMethods, and it has BPM.” An architect challenges the reasoning. Assess it.

Setup. We separate what webMethods is genuinely strong at from what the new programme actually needs, using the six dimensions.

Working. webMethods is genuinely strong at *integration* — the ESB, the connectors, the heterogeneous-estate connectivity — and the bank rightly values it for that. But the new programme’s processes — onboarding, lending — are defined by a different set of needs: they are complex, regulated, human-heavy, and must be *business-readable and reviewable*. Testing those needs against webMethods:

business-readable notation: weak — Flow heritage, integration developer’s tooling;

human-task depth: present, but not the platform’s centre;

analyst-accessible review: weak.

The team’s reasoning — “we own it, it has BPM” — weighs only *sunk ownership*, and ignores that the dimensions the new programme makes decisive are exactly the ones an integration-first platform is weakest on.

Discussion. The architect is right to challenge the reasoning, and the worked example shows the specific error: “we already own it, and it has the feature” is not a platform decision, it is the absence of one. webMethods genuinely has a BPM module, and the bank genuinely owns webMethods — both facts are true, and neither answers the actual question, which is whether an integration-first platform’s BPM module is the right home for a set of complex, regulated, human-and-decision-heavy, business-readable processes. By the six dimensions, it is not: the onboarding and lending processes need business-readable BPMN-and-DMN artefacts that process owners and risk reviewers can read, deep human-task capability, and analyst-accessible tooling — and those are precisely the dimensions on which webMethods, an integration platform authored in an integration developer’s tool with a proprietary-language heritage, is weakest. The sound architecture keeps each platform on its home ground: webMethods continues to do what it is genuinely excellent at — integrating the bank’s large system estate — and the new business processes are built in a process-first orchestrator, which then *calls* webMethods for the integration. That is not a rejection of webMethods; it is using it for what it is. The lesson is the chapter’s: webMethods and Camunda are different kinds of platform, the choice between them is decided by the nature of the problem, and “we already own one of them” is a statement about cost, not about fit — and a complex regulated business process placed on an integration platform because the licence was already paid is a process placed by accounting, not by architecture.

Practical next steps

If your institution runs webMethods, be precise about what it is being asked to do. For integration — connecting the system estate, B2B, APIs — it is on its home ground. For building complex, regulated, business-readable end-to-end business processes, test the need against the six dimensions rather than defaulting to “we already own it.” The two platforms are different kinds of thing, and fit, not sunk cost, should decide.

Chapter in brief

Software AG’s webMethods is a genuine incumbent in many financial institutions, but it is not, at heart, a BPM platform — it is an *integration platform*, with the Enterprise Service Bus at its core, sold as a unified suite of ESB, API management, B2B, managed file transfer, *and* BPM. Its integration heritage — hundreds of connectors, a Java-based integration server — is a real strength. But against the six dimensions its integration-first nature shows: a proprietary Flow-language heritage and integration-developer tooling make it weak on business-readable notation; it carries proprietary lock-in and a heavyweight operational model; and human-centric BPM is not its centre of gravity. webMethods’ home is deep enterprise integration; for governed, business-readable, human-heavy end-to-end business processes, a process-first orchestrator wins.

PART XII

Migrating to Camunda

The Anatomy of a BPM Migration

74.1 Why migration deserves a part

The comparison part ended on a clear observation: many institutions running a proprietary BPM suite are deciding to move to an open platform, and the journey is involved enough to need its own treatment. This part is that treatment. It is about *migrating* an institution's processes from a legacy or proprietary BPM platform to Camunda — and it exists because migration is where a great many hyperautomation programmes either succeed or quietly fail.

Migration deserves a part of its own for a simple reason: it is not the same activity as building. Building a new process on Camunda is the subject of the rest of this manuscript. Migrating an *existing* process — one that runs in production, that has live instances, that the business depends on, that was built in another platform's idioms — is a different and harder discipline, with its own pitfalls. An institution that approaches migration as “just building, but starting from an existing diagram” will be surprised, expensively.

74.2 What a migration actually moves

The first clarifying idea is that a BPM migration is not one thing being moved but *several distinct things*, each with its own difficulty, and a plan that does not separate them will underestimate the work.

A migration moves, first, the *process definitions* — the models themselves, the diagrams and their logic. Second, it moves the *implementation behind the steps* — the service integrations, the scripts, the data mappings, the forms, all the executable substance that made the old models actually run. Third, and hardest, it must deal with the *in-flight process instances* — the thousands of cases that are *currently running* on the old platform and cannot simply be abandoned. Fourth, it moves the *surrounding assets* — the business rules, the user interfaces, the reports. And fifth, it must move the *people* — the skills, the operational practices, the team's mental model.

These five differ enormously in difficulty. The *process definitions* are often the most visible part and, surprisingly, not the hardest. The *in-flight instances* are usually the hardest and the most underestimated. A migration plan that says “we will migrate the processes” without separating these five has not yet engaged with the actual work.

Figure 74.1 sets out the five distinct things a migration moves.

74.3 The mechanics of migrating an in-flight instance

Because the in-flight instances are the hardest of the five, it is worth setting out, technically, what migrating one actually involves — because the mechanism is specific, and a plan that has not understood it cannot estimate it.

An in-flight process instance is not just a row in a database; it is a *live execution state*. Concretely, it consists of: one or more *tokens*, each marking where in the process the instance currently is — which activity is active; the instance's *variables* — the data it has accumulated;

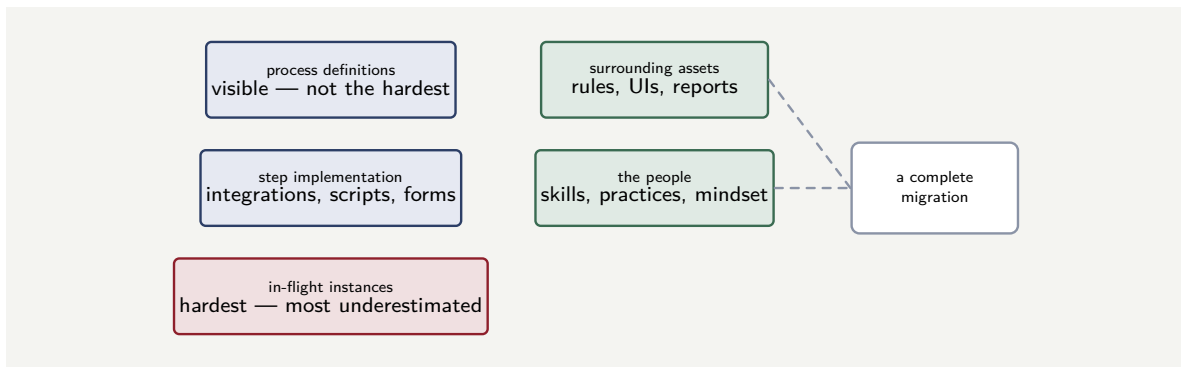


Figure 74.1. The five things a BPM migration moves. They differ enormously in difficulty: the process definitions are visible but not hardest; the in-flight instances are the hardest and most underestimated. A plan that does not separate the five has not engaged with the real work.

and, for the activities currently active, their own state — a user task that is claimed and partly filled, a service task whose job is in progress, a timer that is partway to firing. To migrate the instance is to recreate that exact state in a process running on the new engine.

Camunda’s mechanism for this is the *migration plan*, built from *mapping instructions*. A migration plan maps elements of the source process definition to elements of the target process definition: it says, in effect, “an instance currently active at activity A in the old process should, after migration, be active at activity A’ in the new process.” The engine then takes a running instance, applies the plan, and the instance *continues* on the new definition from the equivalent point — its variables intact, its active tasks preserved. A claimed, partly-completed user task is not reset by a well-formed migration; the assignee can finish, after migration, the task they started before it.

Three technical constraints on this mechanism shape what a migration can and cannot do, and an architect must know them.

First, *only running instances can be migrated* — instances in an active or incident state. Completed instances are history, not live state, and are not migrated by this mechanism.

Second, *an element can only be mapped to an element of the same type*. An active service task maps to a service task; a user task to a user task. A mapping instruction cannot turn an active user task into a service task, because that would change the kind of state the engine must hold.

Third, the mechanism maps *semantically equivalent* activities cleanly — but where the new process is genuinely different at the point an instance sits, mapping is not enough. For those cases, migration is combined with *process-instance modification* — cancelling an activity instance and starting another — to move the token deliberately to where the new process needs it. This is the technically demanding case, and it is the one that makes migrating instances across *substantially redesigned* processes hard: every distinct place an instance might currently sit, for which the old and new processes do not correspond, needs its own deliberate modification.

Figure 74.2 shows a mapping instruction carrying a token from source to target.

74.4 The three migration strategies

There is no single way to migrate, and an institution should choose its strategy deliberately. Three are worth naming.

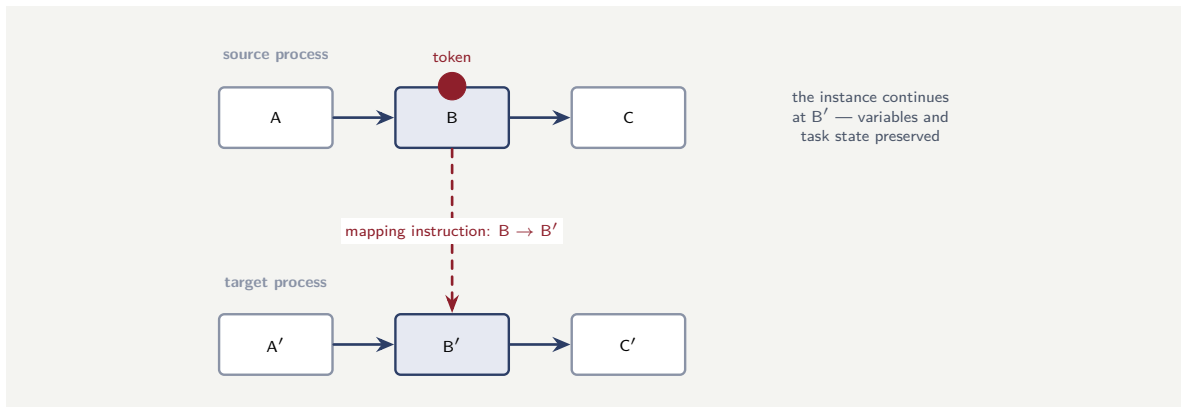


Figure 74.2. Migrating an in-flight instance. A migration plan’s mapping instruction maps the element where the instance’s token currently sits — here, B — to the equivalent element in the target process, B’. The instance continues from the equivalent point, its variables and active-task state preserved. Where source and target do not correspond, mapping is combined with process-instance modification.

The *big-bang* strategy moves everything at once: on a chosen date, the old platform is switched off and the new one takes over. It is conceptually simple and it ends the cost of running two platforms quickly — but it concentrates all the risk on one date, and for a large process estate that concentration is dangerous.

The *phased* strategy moves the estate process by process, or domain by domain, over an extended period. The two platforms run in parallel during the transition, which costs more and is more complex to operate — but the risk is spread, each phase’s lessons inform the next, and a problem in one phase does not imperil the whole estate. For most large financial institutions, phased migration is the disciplined default.

The *strangler* strategy — borrowing the well-known pattern’s name — migrates by building all *new* processes on Camunda and moving existing ones only as they need significant change, so the old platform is gradually “strangled” as its processes age out or are rewritten. It is the lowest-risk and the slowest, and it suits an institution whose legacy platform is stable enough to be left running while the centre of gravity shifts.

Tip

Choose the migration strategy deliberately. *Big-bang* — everything on one date — is simple but concentrates all risk on that date; it is rarely right for a large estate. *Phased* — process by process, two platforms running in parallel — costs more but spreads the risk and lets each phase teach the next; it is the disciplined default for most institutions. *Strangler* — new processes on Camunda, old ones moved only as they change — is lowest-risk and slowest. The wrong choice is no choice: a migration that drifts without a named strategy gets big-bang’s risk and phased’s cost at once.

74.5 The tooling, and the runtime-versus-history distinction

A migration is not done by hand, instance by instance, and an architect should know the technical tooling that exists and — just as important — a distinction it forces.

For the definitions, *converter* tooling exists: for a Camunda 7-to-8 migration, for instance,

a BPMN converter maps the source models toward the target form, translating the BPMN elements it can and flagging those it cannot. A converter does real work, but its output is a *starting point* that must be reviewed and completed — it does not, by itself, produce a correct target process, because the constructs that do not map automatically are exactly the ones that need a human decision.

For the instances, *data-migration* tooling exists: a data migrator that can run in a *running-instance* mode — carrying live instances across — and is operated as part of a carefully sequenced cutover: stop the old solution, run the migrator over the running instances, start the new solution so the migrated instances continue immediately, keeping downtime to a minimum.

This tooling forces a distinction the migration plan must make explicit: the difference between *runtime data* and *history data*. Runtime data is the live state of the in-flight instances — the tokens, variables, and active tasks of the previous section — and it is what must be carried across so the instances can continue. History data is the record of *completed* process instances — what ran, when, with what outcome. The two are treated differently: runtime state is migrated so execution continues, but the old engine's *historic* data often cannot simply be moved into the new engine's history, and a migration must decide deliberately what happens to it — archived from the old platform, kept queryable in a reporting or analytics store, or accepted as lost. A migration plan that has not distinguished runtime from history has not decided what happens to the institution's process *record*, and for a regulated institution that record may be something a regulator can ask for.

A migration must distinguish *runtime data* — the live state of in-flight instances, which is migrated so they can continue — from *history data*, the record of completed instances. The old engine's history often cannot be moved into the new engine's history, so the plan must decide deliberately: archive it from the old platform, keep it queryable in a reporting store, or accept its loss. For a regulated institution this is not a technicality — the historic process record is something a regulator may later ask to see, and “we switched the old platform off” is not an answer.

74.6 The hidden cost of a migration

Every migration has a cost the plan does not show: the cost of the *knowledge* that lives in the old platform and in the heads of the people who run it. The old platform's configuration, its workarounds, its undocumented behaviours, its operational playbooks — all of these are knowledge that was bought with years of production experience, and none of it transfers automatically. A migration plan that costs only the artefacts — the process models, the integration code, the UI — and not the operational knowledge is undercosted, and the gap will appear in the months after go-live, when the team discovers behaviours the old platform handled that the new one does not yet, because no one remembered to transfer the knowledge.

Worked example: the underestimated in-flight instances

Problem. A bank plans to migrate its loan-origination process from a legacy BPM suite to Camunda. The project plan allocates effort as: 60% to migrating the process definitions, 30% to re-implementing the step integrations, 10% to “cutover.” On the legacy platform, at any given moment, there are about 12,000 loan applications *in flight* — partway through the

process, some of them weeks from completion. The plan's "cutover" line assumes these will simply finish on the old platform while new applications start on Camunda. Six weeks before go-live, the team realises the legacy platform is contracted to be switched off on the cutover date. Assess the problem the plan has walked into.

Setup. We consider what must happen to the 12,000 in-flight instances if the old platform is switched off, and whether the plan has budgeted for it.

Working. The plan assumed in-flight instances would "finish on the old platform." But the old platform is switched off on the cutover date, and some of the 12,000 instances are weeks from completing — so they cannot all finish first. The 12,000 in-flight instances must therefore be *migrated*: each one's current state — where its token sits, its variables, its history — must be carried across to a running Camunda process. That is the hardest category of migration work, and the plan allocated to it:

0% — it is hidden inside a 10% "cutover" line.

Migrating live process instances — mapping each one's exact state into the new engine's model, for 12,000 cases, correctly — is a substantial engineering effort, and the project has six weeks and no budget for it.

Discussion. The plan made the exact error this chapter is written to prevent: it treated migration as essentially the moving of *definitions* — 60% of the effort — and folded the genuinely hard part, the in-flight instances, into a 10% line called "cutover" that quietly assumed the problem away. The assumption "they will finish on the old platform" is the seductive one, and it collapses the moment the old platform has a switch-off date, which it almost always does — a licence ends, a contract closes, a data centre is decommissioned. Twelve thousand live loan applications cannot be abandoned and cannot all finish in time, so they must be migrated as running instances, and that is the hardest of the five things a migration moves: each instance's precise state has to be mapped into the new engine. The lesson is the chapter's central one. A migration moves five distinct things, and the in-flight instances are routinely the hardest and the most underestimated — and a plan that does not give them their own line, their own budget, and their own design has not planned the migration, it has planned only the visible part of it. Had the bank separated the five from the start, the in-flight instances would have been costed honestly, a strategy chosen for them — migrate them, or start the cutover early enough to drain most of them first — and the six-weeks-out discovery would never have happened.

Practical next steps

For any BPM migration your institution is planning, find the line in the plan that covers *in-flight process instances* — the live cases running on the old platform at cutover. If there is no such line, or it is hidden inside a vague "cutover" item, the plan has underestimated its hardest component. Separate the five things a migration moves, and give the in-flight instances a budget of their own.

Chapter in brief

Migrating existing processes to Camunda is a different and harder discipline than building new ones, and it deserves its own treatment. A migration moves five distinct things: the process definitions; the implementation behind the steps; the in-flight process instances; the surrounding assets; and the people — and they differ enormously in diffi-

culty, with the in-flight instances the hardest and most underestimated. Migrating an in-flight instance means recreating its live state — tokens, variables, active-task state — on the new engine, done through a *migration plan* of *mapping instructions* that map source elements to same-type target elements, combined with process-instance modification where the processes do not correspond. Converter and data-migration tooling exists but produces starting points, not finished migrations, and the plan must distinguish *runtime* data, which is carried across, from *history* data, whose fate must be decided deliberately. Three strategies are available — big-bang, phased, and strangler — and the strategy must be chosen deliberately, since drift gets big-bang's risk and phased's cost at once.

Migrating from IBM Business Automation Workflow

75.1 The most common migration in banking

Of all the migrations a financial institution undertakes toward Camunda, the move from *IBM Business Automation Workflow* — IBM BAW, and its earlier incarnation IBM BPM — is among the most common, because, as the comparison part noted, IBM’s suites are deeply embedded in banking. This chapter treats that specific migration: what makes it hard, and how the discipline of the previous chapter applies to it.

75.2 The BPMN-export trap

The first thing an architect must know about migrating from IBM BAW is a specific, concrete trap, because teams walk into it constantly.

IBM BAW is BPMN-based, and so the natural first assumption is comforting: both the old platform and Camunda speak BPMN, so the process definitions should simply export from one and import into the other. They do not. IBM’s BPMN exports, in practice, *omit the diagram-interchange information* — the data that describes where each shape sits on the canvas. A BPMN file without that information is logically complete but visually empty: imported into Camunda’s Modeler, it produces no diagram to display. The models are not genuinely portable just because both tools claim BPMN.

The lesson generalises beyond this one detail. “Both platforms support BPMN” is never sufficient grounds to assume models will move cleanly. Vendors omit parts of the standard, extend it with proprietary constructs, and store vendor-specific configuration outside the standard’s reach. Migration tooling exists — including tools specifically for reconstructing diagram information from IBM exports — and helps considerably, but the architect must begin from the realistic premise that the process definitions will need genuine *translation*, not a clean export-import.

“Both platforms support BPMN, so the models will just transfer” is one of the most expensive false assumptions in a BPM migration. IBM BAW’s BPMN exports, in particular, omit diagram-interchange information — import them into Camunda’s Modeler and you get no diagram at all. More broadly, vendors omit, extend, and sidestep the standard. Budget for genuine translation of the process definitions, assisted by migration tooling — never for a clean export-import.

75.3 The BAW-specific artefacts a migration must translate

To estimate a BAW migration concretely, an architect must know the specific IBM BAW constructs that have no direct Camunda equivalent — because each is a distinct piece of translation work, and the comparison part’s architecture chapter named them: Process Applications, Toolkits, coach views, the WebSphere-bound runtime, and FileNet content. This section treats what each means for the migration.

Coach views. BAW’s human-task user interfaces are built as *coaches* — composed of *coach views*, BAW’s own UI component technology — and they live *inside* the BAW process application, bound to the human tasks. Camunda makes the opposite choice: the engine exposes a task through an API, and the user interface is a separate concern — a Tasklist, or a custom application in ordinary web technology. There is no “convert a coach view” operation, because the two models are architecturally different. Every coach in the BAW estate is a screen that must be *rebuilt* as a Camunda form or as part of a separate task application. For a large estate this is often the single largest line of migration effort, and it is invisible if the plan looks only at the process diagrams.

Toolkits. BAW factors shared logic — common subprocesses, shared services, shared coach views — into *Toolkits*, its reuse unit. A Toolkit is referenced by many process applications. In migration this matters twice over: a Toolkit cannot be migrated in isolation from the process applications that depend on it, so the migration must map the dependency graph; and the Toolkit’s contents must be re-homed into Camunda’s own reuse mechanisms — shared BPMN processes invoked by call activity, shared connectors, a shared form library — which are organised differently. A migration plan that inventories process applications but not the Toolkits beneath them has missed a layer of the estate.

The BPEL and SCA heritage. Older parts of an IBM estate — particularly where IBM BPM grew from the earlier WebSphere Process Server — may contain *BPEL* processes and *SCA* (Service Component Architecture) modules rather than only BPMN. These do not map to Camunda BPMN at all in the ordinary sense; they are the subject of [Part XIII](#)’s treatment of BPEL migration, and an architect must check whether the BAW estate is purely BPMN or carries this older substrate, because the two need different translation approaches.

The runtime and operational artefacts. A BAW process runs on the Process Server on WebSphere; its operational configuration — the WebSphere data sources, the security realms, the cluster configuration — is BAW-and-WebSphere specific and does not migrate as such. The Camunda equivalent is a different runtime model entirely — the cloud-native Zeebe engine of [Chapter 27](#) — so the operational layer is rebuilt, not ported, and the operations team’s WebSphere knowledge does not directly transfer.

Tip

Inventory a BAW estate at four levels, not one. The *process diagrams* are the visible level. Beneath them: the *coach views* (every human-task screen, rebuilt as a Camunda form or task-app screen — often the largest single effort); the *Toolkits* (shared logic, re-homed into Camunda’s reuse mechanisms, with their dependency graph mapped); and any *BPEL/SCA* substrate from the WebSphere Process Server heritage, which needs [Part XIII](#)’s approach. A plan costed on the diagrams alone has seen a quarter of the estate.

IBM BAW construct	Camunda equivalent	migration effort
BPMN process model	BPMN model — re-drawn; logic translated	moderate
coach / coach view (human-task UI)	Camunda form or custom task application	high — rebuilt
integration / system service	connector or job worker	high — rewritten
server-side JavaScript (script task)	job worker or FEEL expression	moderate
business object (data type)	process variable (JSON-shaped)	low-moderate
Toolkit (shared logic)	shared called process / connector library	moderate
BPEL / SCA module (legacy substrate)	BPMN — via Part XIII's BPEL approach	high
WebSphere runtime / config	Zeebe runtime — rebuilt, not ported	high
in-flight process instances	drained, or instance-migrated	highest

Figure 75.1. The BAW-to-Camunda construct-mapping table. Each IBM BAW construct has a distinct Camunda target and a distinct migration effort; the coach views, the integration services, the legacy BPEL/SCA substrate, the runtime, and above all the in-flight instances are the high-effort rows — and none of them is visible if the plan looks only at the process diagrams.

75.4 The construct-mapping table

The artefacts of the previous section can be set out as a *mapping table* — the single most useful planning artefact a BAW migration produces, because it converts “migrate from BAW” into an itemised list of distinct translation tasks, each with a known target and a known difficulty. Table 75.1 gives the mapping for the principal BAW constructs.

The table makes a planning point the prose cannot make as sharply: the process *diagram* — the most visible artefact — is only a *moderate*-effort row, while the high-effort rows are the coach views, the integration services, the BPEL/SCA substrate, the runtime, and the in-flight instances. A migration estimate that weights the diagrams heavily and the rest lightly has inverted the real distribution of work.

Each of the table’s nine rows is a translation discipline of its own, and the nine chapters that follow this one open each row in technical detail — the process model, coaches, integration services, server-side script, business objects, Toolkits, the BPEL and SCA substrate, the WebSphere runtime, and the in-flight instances — one chapter per construct. The remainder of this chapter sets out what the table cannot: the deeper difficulties and the shape of the project as a whole.

75.5 What is hard beyond the diagrams

The diagram trap is the visible difficulty; the deeper ones follow the previous chapter's five-part anatomy.

The *step implementation* rarely transfers at all. IBM BAW's service integrations, its scripting, its data handling are expressed in IBM's idioms and technologies; the equivalent behaviour must be *rebuilt* on Camunda's terms — as job workers, connectors, and the Java integration of **Part XIV**. The logic is reused; the implementation is rewritten. Concretely, a BAW *integration service* or *general system service* becomes a Camunda connector or job worker; BAW's server-side *JavaScript* in script tasks becomes a job worker or a FEEL expression; BAW's *business objects* — its structured data types — become Camunda process variables, typically JSON-shaped, with the data model re-expressed; and BAW *Toolkit* services become shared connectors or called processes. None of this is a translation in the file-conversion sense; each is a re-implementation against a different engine's integration model, and the estimate must be built service by service.

The *in-flight instances* are the chapter's hardest problem, as the anatomy chapter warned. A bank running loan or onboarding processes on IBM BAW has thousands of live cases; their state must be either drained — allowed to complete on IBM BAW while new cases start on Camunda — or genuinely migrated, instance by instance, into running Camunda processes.

And IBM BAW carries a particular shape of *surrounding asset*: its coaches and case tooling, its built-in user interfaces. Camunda's model, where human work flows through Tasklist and forms, is different, and the user-facing layer is substantially rebuilt rather than moved.

75.6 The shape of a BAW migration project

The construct-mapping table itemises *what* must be translated; a migration project also needs a *shape* — an order in which the work is done — and a BAW migration has a characteristic one, worth setting out so a plan can follow it rather than improvising.

A sound BAW migration runs as a sequence of workstreams, each depending on the ones before. *Inventory and assessment* comes first: catalogue the process applications, the Toolkits, the coaches, the services, the BPEL/SCA substrate, and the live-instance volumes — the four-level inventory the tip box demands — so the scope is genuinely known. *Model translation* follows: the BPMN diagrams are translated and re-drawn on Camunda, with diagram-interchange information reconstructed. *Implementation rebuild* runs alongside and after it: the integration services, the script logic, the business objects are re-implemented as connectors, job workers, and process variables. *UI rebuild* — the coaches becoming Camunda forms or a task application — is its own substantial workstream, and because it is the largest single effort it must start early, not be left to the end. *Integration and test* brings the translated model, the rebuilt implementation, and the rebuilt UI together and proves them against the old platform's behaviour. And *cutover* — the per-process drain-or-migrate decision of the next section, executed — closes the project.

The dependency that most often breaks a plan is the UI rebuild: because the coaches are invisible on the process diagram, a plan that scopes from the diagrams discovers the UI workstream late, when there is no longer time for the largest single piece of work. The shape exists precisely to prevent that.

Figure 75.2 sets out the workstreams and their dependencies.

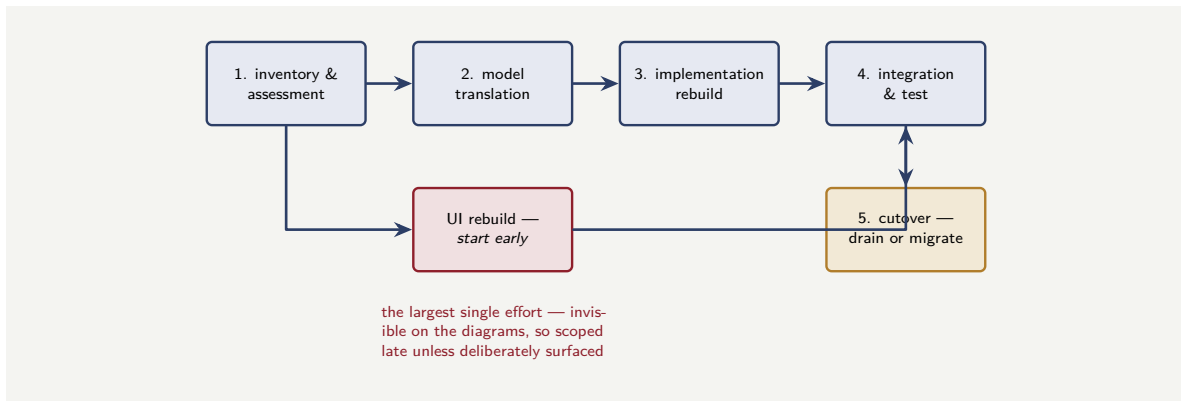


Figure 75.2. The shape of a BAW migration project. Five sequential workstreams — inventory, model translation, implementation rebuild, integration and test, cutover — with the UI rebuild as a parallel workstream that must start early, since the coaches are the largest single effort and are invisible on the process diagrams.

75.7 Migration strategies: how to sequence the cutover

The workstream shape is *what* a migration does; a migration strategy is *how* it moves the estate from BAW to Camunda — and a BAW migration has several strategies to choose between, not one. The choice is consequential, and a plan should make it deliberately.

The *big-bang* strategy migrates the whole estate and cuts over in one event: every process, every coach, every integration is rebuilt, tested, and then BAW is switched off and Camunda switched on together. Its appeal is a clean break — no period of running both — but its risk is concentrated: everything is unproven until the single cutover, and there is no incremental learning. Big-bang suits only a small estate, or one where running two platforms in parallel is genuinely impossible.

The *phased* strategy is the usual sound choice: the estate is divided into batches — by process, by business line, by Toolkit grouping — and migrated batch by batch, each batch cut over before the next begins. BAW and Camunda *run side by side* for the duration, each owning the processes assigned to it. Phasing spreads the risk, lets each batch teach the next, and lets the team build delivery rhythm — at the cost of a transition period in which two platforms must be operated and, where processes interact, integrated.

The *pilot-first* strategy is a refinement of phasing: the first batch is chosen deliberately as a *pilot* — a process real enough to prove the toolchain, the patterns, and the team's estimates, but contained enough that its risk is bounded. The pilot's purpose is learning; its results recalibrate the estimate for everything after. A BAW migration that does not pilot is estimating the whole estate from theory.

Two further strategies are patterns applied *within* a phased migration. The *strangler-fig* strategy migrates a large process incrementally from the edges: new work, or a slice of the process, is routed to Camunda while the rest still runs on BAW, and the Camunda share grows until BAW handles nothing and is removed — the new system grows around the old until the old can be cut away. It suits a process too large or too critical to rebuild in one move. And the *coexistence* or *API-facade* strategy, for the transition period, puts a stable interface in front of both platforms so that callers do not need to know which platform currently runs a given process — the facade routes to BAW or Camunda, and the routing changes as processes migrate, without the callers changing at all.

Figure 75.3 sets the strategies side by side.

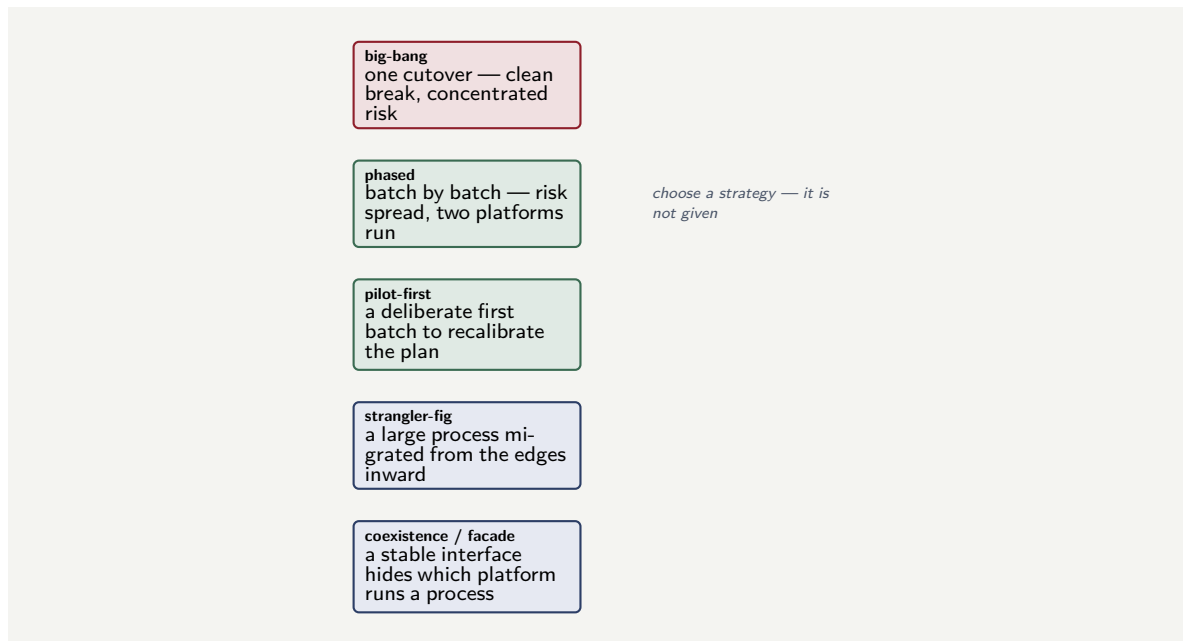


Figure 75.3. Strategies for a BAW migration. Big-bang, phased, and pilot-first are whole-estate strategies; strangler-fig and the coexistence facade are patterns applied within a phased migration. The phased, pilot-first approach is the usual sound choice for a substantial estate.

The strategies are not mutually exclusive: a realistic BAW migration is *phased*, opens with a deliberate *pilot*, applies the *strangler-fig* pattern to whichever individual processes are too large to move in one batch, and runs a *coexistence facade* for the transition so the rest of the institution is insulated from the migration's progress. The one strategy a substantial BAW estate should not choose is big-bang — the estate is too large, and the construct-mapping table's lesson is precisely that a BAW estate holds more, and more varied, work than a diagram count suggests, which is exactly the condition under which a single all-or-nothing cutover is most dangerous.

Tip

Choose a BAW migration strategy deliberately; it is not implied by the decision to migrate. For a substantial estate the sound default is *phased and pilot-first*: a deliberate pilot process to prove the toolchain and recalibrate the estimate, then batch-by-batch migration with BAW and Camunda running side by side. Apply the *strangler-fig* pattern to any single process too large to move in one batch, and run a *coexistence facade* so the rest of the institution need not track the migration. Reserve *big-bang* for a small estate — for a large one it concentrates all the risk at the worst moment.

Problem. A bank is migrating its account-opening process from IBM BAW to Camunda. The process takes, on average, 5 working days end to end, and the bank starts about 800 new account openings a day. At any moment roughly 4,000 cases are in flight. The bank must choose, for these in-flight cases, between two approaches: *drain* — stop starting new cases on IBM BAW, run new cases on Camunda, and let IBM BAW empty out — or *migrate* — carry the 4,000 live cases' state across to Camunda. The bank's IBM BAW licence ends in 4 weeks. Reason about the choice.

Setup. We consider how long draining would take against the licence deadline, and what

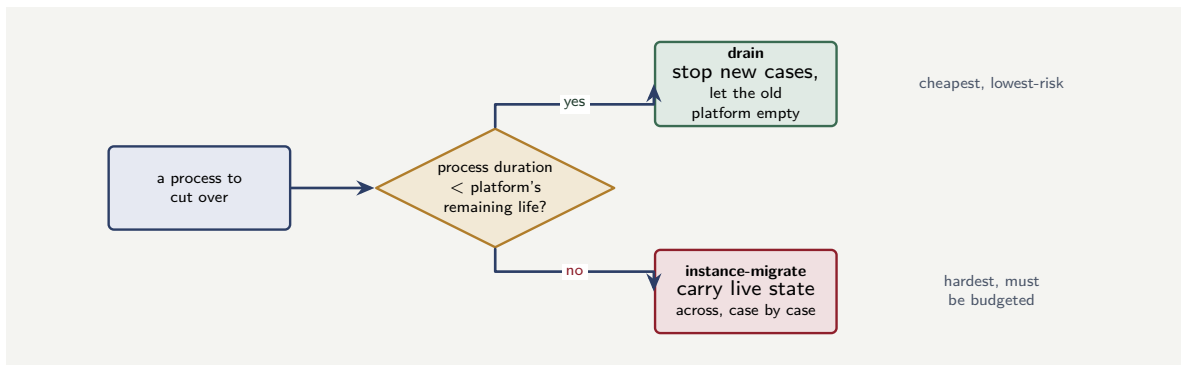


Figure 75.4. The drain-versus-migrate decision, made per process. If a process’s typical duration is shorter than the time the old platform can be kept running, its in-flight cases can be *drained*; if longer, they must be *instance-migrated*. Most banks run both kinds and so do both.

migrating would instead require.

Working. *Draining.* If the bank stops starting new cases on IBM BAW today, the in-flight cases finish at the rate the process completes them. With a 5-working-day average process, the bulk of the 4,000 cases drain within about

$\approx 1\text{--}2$ weeks,

and almost all within

≈ 3 weeks (allowing for the slow tail of long-running cases),

which is *inside* the 4-week licence deadline. *Migrating.* Carrying 4,000 live cases’ exact state across to Camunda is the hardest category of migration work and a substantial engineering effort — effort that, here, draining makes unnecessary.

Discussion. For this process, drain is clearly the right choice, and the worked example shows that the drain-versus-migrate decision is a genuine calculation, not a default. The decisive facts are the process’s *duration* and the *deadline*: a 5-day process empties its in-flight population in two to three weeks, comfortably inside the 4-week licence window, so the bank can simply stop feeding IBM BAW, run new openings on Camunda, and let the old platform empty itself — avoiding entirely the hardest and riskiest part of the migration. But the calculation is specific. Change the facts and the answer flips: a process that runs for *months* — a mortgage, a complex claim — cannot drain inside any reasonable licence window, and for it the 4,000 in-flight cases would have to be genuinely migrated, because waiting for them to finish is not an option. The lesson is that “drain or migrate” is decided per process, by comparing the process’s duration against the time the old platform can be kept running. Short processes drain; long processes must be migrated. An institution with both — most banks have both — will drain some of its IBM BAW processes and instance-migrate others, and the migration plan must make that determination process by process rather than adopting one approach for the whole estate.

Figure 75.4 sets out the per-process drain-or-migrate decision.

Practical next steps

For each process in an IBM BAW estate being migrated, compare the process’s typical duration against how long IBM BAW can be kept running. Processes that complete well inside that window can be drained — the cheapest, lowest-risk path. Processes that run longer than

the window must have their in-flight instances genuinely migrated, and that work must be budgeted. The determination is per process, not per estate.

Chapter in brief

Migrating from IBM Business Automation Workflow — IBM BAW — is among the most common migrations in banking. Its signature trap is the assumption that because both platforms support BPMN the models will simply transfer: IBM's BPMN exports omit diagram-interchange information, importing as no diagram at all. The chapter's central planning artefact is the *construct-mapping table*: each BAW construct — BPMN model, coach view, integration service, script, business object, Toolkit, BPEL/SCA module, runtime, in-flight instances — has a distinct Camunda target and a distinct migration effort, and the table shows that the visible process diagram is only a moderate-effort row while the coach views, the integration rebuild, the legacy substrate, the runtime, and the in-flight instances are the high-effort rows. A BAW migration project runs as five workstreams — inventory, model translation, implementation rebuild, integration and test, cutover — with the UI rebuild a parallel workstream that must start early, and is sequenced by a deliberately chosen *strategy*: phased and pilot-first for a substantial estate, with strangler-fig and a coexistence facade applied within it, big-bang reserved for a small one. And the in-flight instances are the hardest: the drain-versus-migrate choice is a per-process calculation comparing process duration against the old platform's remaining life.

Migrating the BPMN Process Model

76.1 What a BAW process definition actually is

The construct-mapping table of [Chapter 75](#) gave each IBM BAW construct a row; this chapter and the eight that follow open each row into the technical detail a migration plan needs. The first row, and the most visible, is the *process model* itself.

In IBM BAW the structured process is a *Business Process Definition* — a BPD, in the older IBM BPM vocabulary — or a BPMN process authored in Process Designer. It is BPMN-shaped, and that shape is genuine: activities, gateways, sequence flows, and events, arranged as a graph. But a BAW process definition is *not* a neutral, portable BPMN file. It carries three things that bind it to BAW.

It carries *IBM-specific extension attributes* — the namespaced properties BAW hangs on the standard BPMN elements to record its own configuration. It carries *IBM's particular service and gateway configurations* — the way a BAW activity is wired to a BAW service, the way a BAW gateway's conditions are expressed. And it carries *references into the surrounding process application* — to coaches, to services, to Toolkit elements — so the process definition is not self-contained; it is one node in a web of process-application artefacts.

And, as [Chapter 75](#)'s BPMN-export trap established, an export of a BAW process typically *drops the diagram-interchange information* — the data describing where each shape sits on the canvas — so even the visible structure does not survive an export cleanly.

76.2 The three parts of the translation

Translating a BAW process model to Camunda is not one operation but three distinct ones, and an estimate must treat them separately.

The *topology* — the activities, gateways, sequence flows, and events, and the way they connect — usually carries over in *structure*, because both BAW and Camunda are genuinely BPMN. A five-activity process with two gateways in BAW is a five-activity process with two gateways in Camunda. This is the part of the translation that is real reuse, and it is why the row is not rated *high*.

The *diagram interchange* — the visual layout — must be *reconstructed*. Because the export drops it, the model arrives in Camunda Modeler with correct logic but no layout, and a process whose elements are piled at the origin is logically sound and humanly unreadable. Reconstruction is done by auto-layout tooling where the process is simple enough, and by hand where it is not — and for a regulated process that must be *reviewable*, a readable layout is not cosmetic, it is a requirement.

The *element configuration* — what each activity actually *does* — does *not* carry over. A BAW activity's binding to a BAW service must become a Camunda task's binding to a job type or a connector; a BAW gateway's condition must become a Camunda FEEL expression. This is

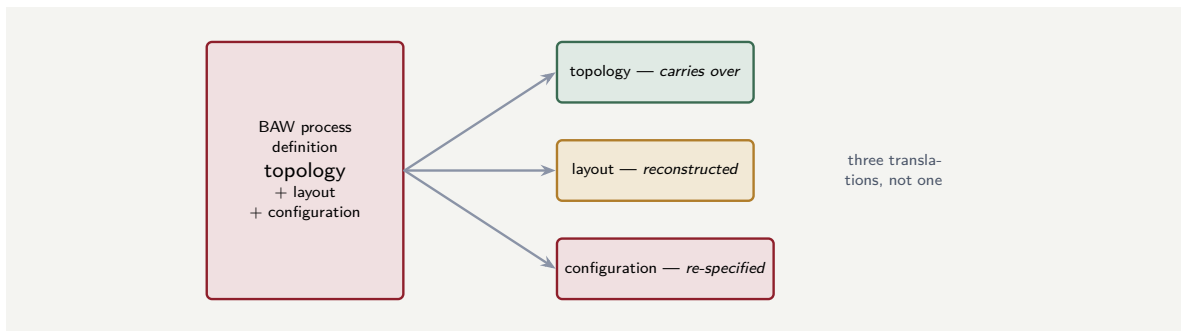


Figure 76.1. Translating a BAW process model is three distinct operations: the topology carries over in structure, the diagram layout must be reconstructed because the export drops it, and the element configuration must be re-specified element by element. Only the first is genuine reuse.

re-specification, element by element, and it is the part of the model translation that consumes the real effort.

Figure 76.1 separates the three parts of the model translation.

76.3 The export and re-draw in practice

The three parts of the translation — topology, diagram interchange, element configuration — are concrete in a before-and-after pairing. Listing 76.1 sketches what a BAW BPMN export gives: the structure is present but the IBM-specific extensions and the diagram layout are not portable.

Listing 76.1. A BAW BPMN export fragment: the activity topology is present, but IBM-specific extension attributes and missing diagram-interchange data mean it cannot be imported cleanly into Camunda.

```

1  <!-- exported from IBM BAW -->
2  <bpmn2:process id="onboarding">
3    <bpmn2:serviceTask id="scoreApp"
4      name="Score Application"
5      ibm:calledService="CreditScoreService"
6      ibm:toolkit="CreditToolkit"/>
7    <bpmn2:userTask id="review"
8      name="Human Review"
9      ibm:coachId="ReviewCoach"/>
10 </bpmn2:process>
11 <!-- no bpmndi:BPMNDiagram section -->

```

After re-drawing in Camunda Modeler, the same process has clean Camunda bindings and a proper diagram-interchange section. The topology — the activities and flows — carried over; everything else was rebuilt.

76.4 The hidden cost of a migration

Every migration has a cost the plan does not show: the cost of the *knowledge* that lives in the old platform and in the heads of the people who run it. The old platform's configuration, its workarounds, its undocumented behaviours, its operational playbooks — all of these are knowledge that was bought with years of production experience, and none of it transfers automatically. A migration plan that costs only the artefacts — the process models, the integration code, the UI — and not the operational knowledge is undercosted, and the gap will appear in

the months after go-live, when the team discovers behaviours the old platform handled that the new one does not yet, because no one remembered to transfer the knowledge.

Worked example: the model that imported with no diagram

Problem. A bank exports a 40-activity BAW loan process as BPMN XML and imports it into Camunda Modeler, expecting to see the familiar diagram. Instead the Modeler shows all 40 activities stacked at the top-left corner, overlapping, with the sequence flows a tangle of crossing lines. The team concludes the import “failed” and considers re-drawing the whole process from scratch. Assess this.

Setup. We separate what actually transferred from what did not, using the chapter’s three-part split.

Working. The import did not fail. The BPMN XML carried the *topology* — all 40 activities, the gateways, the sequence flows, their connections — and that is intact: the logical process is fully present. What is missing is the *diagram interchange*, the layout data, which the BAW export dropped. So:

logic transferred + layout dropped = a correct process that looks like a tangle.

Re-drawing the whole process from scratch would discard the topology that *did* transfer — 40 activities’ worth of correct structure — and redo it by hand for nothing.

Discussion. The team mistook a missing layout for a failed import, and the worked example shows why the distinction is worth real money. The 40-activity topology is the part of the model that genuinely carries over, and it transferred correctly; throwing it away and re-drawing from scratch would spend effort recreating what the bank already has. The actual problem is narrow and known: the diagram-interchange information is gone, and the layout must be reconstructed — by auto-layout tooling, which can untangle a 40-node graph into a readable arrangement in seconds, with hand-adjustment for the parts that matter most to a reviewer. That is a far smaller task than re-drawing. The element configuration still needs the element-by-element re-specification the chapter describes — that work is real and unavoidable — but it is not what the tangle is about. The lesson is the chapter’s three-part split used as a diagnostic: when a BAW model imports as a tangle, the right response is not despair and a rewrite but recognition — topology intact, layout to reconstruct, configuration to re-specify — and a plan that costs each of the three correctly.

Practical next steps

When a BAW process model imports into Camunda Modeler as an unreadable tangle, do not conclude the import failed. Check the three parts separately: the topology has almost certainly transferred; the layout must be reconstructed, which auto-layout tooling does cheaply; and the element configuration must be re-specified. Re-drawing from scratch discards the one part that genuinely transferred.

76.5 Migration risk and mitigation

Every migration carries risk, and the risk is managed by acknowledging it explicitly rather than hoping it will not materialise. The principal risks are *functional regression* (the migrated process does not behave as the old one did), *data loss* (in-flight instances or their variables are lost or corrupted), and *performance degradation* (the new platform is slower under the same load). Each is mitigated by a specific discipline: functional regression by parallel running and

comparison testing against the old platform; data loss by verifiable, checkpointed migration with rollback capability; performance degradation by load testing against realistic volumes before cutover. A migration plan that does not name its risks and their mitigations is a plan that has not been thought through.

The chapter's principles interact with the platform's operational model in a way worth making explicit. The *deployment* of the artefacts this chapter describes must go through a governed pipeline — versioned, reviewed, tested, and traceable. An artefact deployed by a manual action, outside the pipeline, is an artefact whose presence in production cannot be explained, and in a regulated institution an unexplained artefact is a finding.

The *monitoring* of the artefacts must cover both their functional correctness and their operational health. A process step that executes but produces wrong results is harder to detect than one that fails outright, because it does not raise an error — it raises a consequence, and the consequence may not be discovered until a customer complains or a regulator asks a question. The observability model should therefore include not only error rates and latencies but also *outcome monitoring*: checks that the results the process produces are within expected ranges, flagging anomalies for review.

Chapter in brief

A BAW process model is a Business Process Definition authored in Process Designer — BPMN-shaped, but not a neutral BPMN file: it carries IBM-specific extension attributes, IBM's service and gateway configurations, and references into the surrounding process application, and a BAW export typically drops the diagram-interchange layout information. Translating it to a Camunda BPMN 2.0 model is three distinct operations: the *topology* carries over in structure because both are genuinely BPMN; the *diagram layout* must be reconstructed, by tooling or by hand, or the model imports as an unreadable tangle; and the *element configuration* — what each activity does — does not carry over and must be re-specified element by element. The row is moderate because the topology is real reuse; it is not low because the layout and configuration are not.

Migrating Coaches and Coach Views

77.1 What a coach is

The second construct, and the one a migration most often under-scopes, is the human-task user interface. In IBM BAW that interface is a *coach*.

A coach is a BAW human-task screen — what the person sees when they open a task. It is assembled from *coach views*: reusable UI components, built in BAW's own widget technology, composed together to form a screen and bound to a BAW user task. A coach is not just a layout; it carries real logic. It includes the *layout* of fields and sections; the *data binding* that connects each field to a process variable; *client-side validation* — the rules that check the person's input before it is accepted; and *visibility rules* — the logic that shows or hides parts of the screen depending on the data. All of this is expressed in BAW's coach framework and, historically, in its associated client-side scripting.

The crucial fact for a migration is architectural: in BAW, this user interface lives *inside the process application*, bound to the engine's user task. The UI and the process are one artefact.

77.2 Why the Camunda target is a rebuild

Camunda makes the opposite architectural choice, and that is why this row is rated *high*. Camunda does not embed the user interface in the engine. The engine exposes a task through an API; the user interface is a *separate concern*.

So a coach has no like-for-like Camunda equivalent. The Camunda target is one of two things. For a straightforward task screen, it is a *Camunda form* — built in the form editor, rendered by Tasklist, a declarative description of fields and their bindings. For a rich or highly-customised screen, it is a view in a *custom task application* — built with ordinary web technology against the Tasklist API, with no limit on what it can do.

Either way, the translation is a genuine *rebuild*, in three parts. The form's *fields and data bindings* are re-expressed against Camunda's form schema or the task application's framework. The *validation and visibility rules* are re-implemented — in the form schema where it supports them, in the task application's code where it does not. And the coach view's *reusable-component structure* — the fact that a coach view was a shared widget used by many coaches — must be re-thought in the target's terms, as a shared form or a shared component library.

Figure 77.1 contrasts the embedded coach with Camunda's separate UI layer.

77.3 The rebuild in code

A coach is a rebuild rather than a port, and the rebuild is concrete when the two artefacts are seen side by side. This section follows one human-task screen — a loan-approval form — from a BAW coach to a Camunda form.

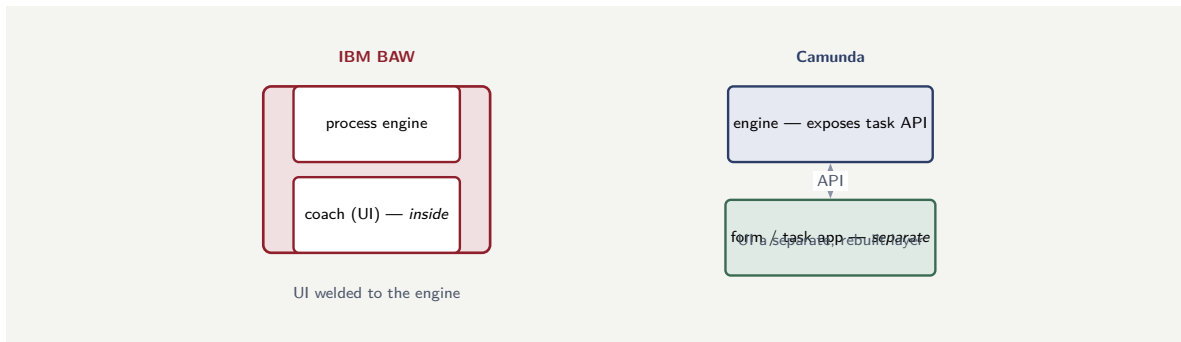


Figure 77.1. Why a coach is a rebuild, not a port. In BAW the coach — the human-task UI — lives inside the process application, welded to the engine. Camunda exposes the task through an API and the UI is a separate layer: a Camunda form or a custom task application. There is no like-for-like conversion.

A BAW coach is built from coach views in the BAW tooling; what it amounts to is a layout of fields, each *bound* to a process variable, with validation and visibility rules attached. Listing 77.1 sketches that structure for a small approval coach.

Listing 77.1. A BAW coach: coach-view controls bound to process variables, with a validation rule — the human-task UI, held inside the process application.

```

1  <!-- BAW coach: LoanApprovalCoach -->
2  <coach name="LoanApprovalCoach">
3    <coachView control="Text" label="Applicant"
4      binding="tw.local.application.name"
5      readOnly="true"/>
6    <coachView control="Decimal" label="Amount"
7      binding="tw.local.application.amount"
8      readOnly="true"/>
9    <coachView control="Select" label="Decision"
10     binding="tw.local.decision"
11     options="APPROVE,DECLINE,REFER"/>
12    <coachView control="Text" label="Rationale"
13     binding="tw.local.rationale"
14     required="true"/>
15  </coach>

```

The Camunda target is a *form* — a declarative description of the same fields, built in the form editor and rendered by Tasklist. A Camunda form is JSON: each field names a *key*, which is the process variable it binds to, and carries its own type, label, and validation. Listing 77.2 is the same approval screen as a Camunda form.

Listing 77.2. The Camunda form equivalent: the same fields as a declarative form schema, each key binding to a process variable, rendered by Tasklist as a separate UI layer.

```

1  {
2    "components": [
3      { "type": "textfield", "key": "application.name",
4        "label": "Applicant", "readonly": true },
5      { "type": "number", "key": "application.amount",
6        "label": "Amount", "readonly": true },
7      { "type": "select", "key": "decision",
8        "label": "Decision",
9        "values": [ { "value": "APPROVE", "label": "Approve" },
10                   { "value": "DECLINE", "label": "Decline" },
11                   { "value": "REFER", "label": "Refer" } ] },
12      { "type": "textfield", "key": "rationale",
13        "label": "Rationale",

```

```
14     "validate": { "required": true } }  
15   ]  
16 }
```

The two listings carry the chapter's argument exactly. The *fields and their bindings* translate closely — the coach's binding to a process variable becomes the form's key, and the read-only and required flags carry over — which is why a straightforward screen is a manageable rebuild. But the translation is still a *rebuild*: the coach lived inside the BAW process application, and the Camunda form is a separate artefact rendered by Tasklist, a distinct UI layer. And the part the listings cannot show is where the real effort sits — a coach with rich client-side logic, conditional visibility, custom coach-view components, has logic that a declarative form schema cannot express, and that screen becomes a view in a custom task application built against the Tasklist API, not a form at all.

Tip

Triage a BAW coach estate before estimating it. A coach that is a layout of fields with simple validation translates closely to a declarative Camunda form — key-by-key, a manageable rebuild. A coach with rich client-side scripting, conditional visibility, or custom coach-view components cannot become a form; it must be rebuilt as a view in a custom task application against the Tasklist API, several times the effort. The estimate depends entirely on which kind each coach is, so inspect them — a coach count alone tells you nothing.

77.4 The spectrum of targets: from form to micro-frontend

A Camunda form and a single custom task application are the two targets the chapter has named so far, but they are the ends of a *spectrum* of coach migration options, and a large estate is usually migrated to several points on it at once. Naming the full spectrum lets a migration choose deliberately rather than defaulting.

The *embedded form* is the simplest target: a declarative Camunda form, built in the form editor, rendered inside Tasklist. It suits the field-and-validation coaches the previous section described, and it is the cheapest target — but it is bounded by what a declarative schema can express.

The *single custom task application* is the next: one web application, built by one team, that renders the task screens for the whole estate against the Tasklist API. It can do anything web technology can do, so it handles the rich coaches a form cannot — but it is one application and one codebase, and in a large institution that becomes a bottleneck: every team whose process has a task screen must change the same application, and its releases couple unrelated processes together.

The *micro-frontend* target removes that bottleneck, and it deserves emphasis because it is the option a large BAW estate most often genuinely needs. In a micro-frontend architecture, the user-facing application is not one monolith but an *assembly of independently-built, independently-deployed front-end pieces* — each one owned by the team that owns the corresponding process. A claims team builds and deploys the claims task screens; a lending team builds and deploys the lending task screens; a thin *shell* application composes them into one experience for the user, and each task screen is fetched and rendered as the micro-frontend its owning team published.

The fit with Camunda's architecture is exact. Camunda already separates the engine from

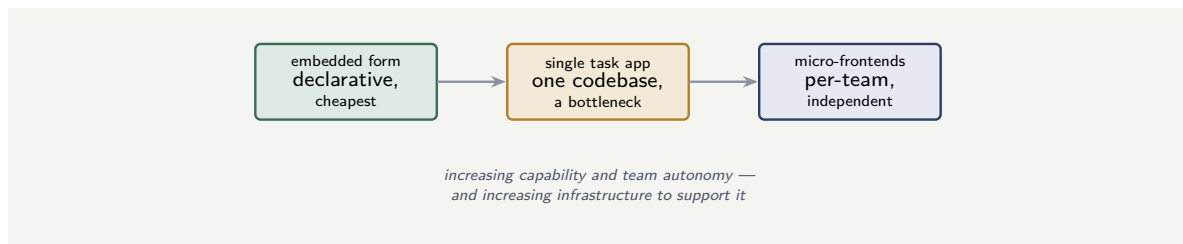


Figure 77.2. The spectrum of coach migration targets. A coach can be re-homed as a declarative embedded form, as a view in a single custom task application, or as an independently-owned micro-frontend — increasing capability and team autonomy, at increasing infrastructural cost.

the user interface — the engine exposes the task through the Tasklist API, and the UI is a separate concern. A micro-frontend is simply that separation taken to its conclusion: not one separate UI but *many*, each aligned to a process and a team. The task screen a micro-frontend renders still binds to the same task and the same process variables; what changes is that it is built, owned, versioned, and deployed by the process's own team, independently of every other.

Figure 77.2 places the three targets on one spectrum.

The micro-frontend target is not free. It needs a *shell* and a composition mechanism; it needs the design discipline of a shared component library so the independently-built screens still look like one institution; and it needs the deployment infrastructure to publish many front-end pieces. For a small estate that overhead is not worth it, and a form or a single task app is right. But for a large BAW estate — dozens of process applications, hundreds of coaches, many owning teams — the micro-frontend target turns the coach migration from one team's monolithic rebuild into *parallel work across the process teams*, each rebuilding its own screens, which both removes the bottleneck and aligns the new UI ownership with the process ownership.

Tip

For a large BAW estate, consider migrating coaches to *micro-frontends* rather than to one custom task application. A single task app makes every process team change one shared codebase — a bottleneck that worsens with scale. Micro-frontends let each team rebuild and deploy its own task screens independently, against the same Tasklist API, composed by a thin shell. It costs shell-and-composition infrastructure and a shared component library, so it is not for a small estate — but for a large one it converts a monolithic UI rebuild into parallel, team-aligned work.

77.5 The hidden cost of a migration

Every migration has a cost the plan does not show: the cost of the *knowledge* that lives in the old platform and in the heads of the people who run it. The old platform's configuration, its workarounds, its undocumented behaviours, its operational playbooks — all of these are knowledge that was bought with years of production experience, and none of it transfers automatically. A migration plan that costs only the artefacts — the process models, the integration code, the UI — and not the operational knowledge is undercosted, and the gap will appear in the months after go-live, when the team discovers behaviours the old platform handled that the new one does not yet, because no one remembered to transfer the knowledge.

Worked example: the migration that costed only the diagrams

Problem. A bank scopes a BAW migration by counting process definitions: 60 processes, estimated at three days of model-translation work each, for 180 days. The plan is approved on that basis. Midway through, the team realises the 60 processes contain, between them, around 340 distinct coaches, none of which was in the estimate. Assess the scale of the omission.

Setup. We consider what the diagram-based count saw and what it did not, and roughly what the coaches add.

Working. The estimate counted *process definitions* — the diagrams — because those are what is visible when an architect opens the BAW tooling and looks at the process list. The coaches are not on that list; they are attached to the user tasks *inside* the processes, invisible to a diagram count. There are around 340 of them, and each is a genuine UI rebuild — fields, bindings, validation, visibility — not a port:

180 days estimated vs. 180 days + (~ 340 coach rebuilds).

Even at a conservative half-day per coach, the coaches add on the order of 170 days — roughly doubling the project — and a rich coach is more than a half-day. The omission is not a detail; it is comparable in size to the entire estimated project.

Discussion. This is the failure [Chapter 75](#)'s construct-mapping table exists to prevent, seen in numbers. The team scoped from the diagrams because the diagrams are what a process list shows, and the coaches — the highest-effort UI rebuild — were invisible to that view, so a project that is really two comparable halves was costed as one. The plan was not slightly optimistic; it was missing a whole category of work the same order of magnitude as the work it counted. The fix is the discipline the table chapter set out: inventory the estate at four levels, and count the coaches explicitly — every user task across every process is a coach to rebuild — before the estimate is set. The lesson is sharp and worth carrying: in a BAW migration the user interface is a separate, high-effort workstream that is structurally invisible to a diagram-based scope, and a plan that does not deliberately go and count the coaches will, predictably, discover halfway through that it has under-costed the project by something close to a factor of two.

Practical next steps

Before a BAW migration estimate is set, count the coaches, not just the processes. Every user task across every process has a coach behind it, and each is a UI rebuild against a Camunda form or task application. A plan costed on the process diagrams alone has omitted what is often the largest single workstream.

Chapter in brief

A coach is a BAW human-task screen, assembled from reusable *coach view* widgets and carrying real logic — layout, data binding, client-side validation, visibility rules — all living *inside* the process application, welded to the engine. Camunda makes the opposite choice: the engine exposes the task through an API and the UI is a separate concern, so a coach has no like-for-like equivalent. The Camunda target is a *Camunda form* for a straightforward screen or a *view* in a *custom task application* for a rich one,

and the translation is a genuine rebuild — fields and bindings re-expressed, validation and visibility re-implemented, the shared-component structure re-thought. This is the high-effort row a diagram-based scope misses, because a coach is invisible on the process diagram, and a large estate may hold hundreds.

Migrating Integration and System Services

78.1 What a BAW service is

The third construct is the platform's outbound reach: how a BAW process calls the systems around it. In BAW this is done through *integration services* and *general system services*.

These are BAW service constructs — units of work, defined in the BAW tooling, that a process activity invokes. An integration service typically invokes a web service, runs an *adapter* to a packaged system, or calls a REST endpoint; a general system service executes server-side logic. Both take inputs from the process, do their work, and return data into the process variables. They are *configured* in the BAW tooling — endpoints, mappings — and, in part, *scripted* there, and they run inside the BAW Process Server, in the Process Server's transaction and threading context.

So a BAW service is two things at once: an *intent* — call system X with these inputs, map these outputs back — and an *implementation* — the BAW-specific configuration and script that realises the intent inside the Process Server.

78.2 Why the Camunda target is a rewrite

The Camunda target for a BAW service is either a *connector* or a *job worker*, and the choice between them follows [Part VI](#)'s distinction.

For a call over a standard protocol — REST, a messaging system, a common cloud service — the target is a *connector*, configured through an element template, with little or no code. For anything needing real code — a non-trivial transformation, a call to a system with no off-the-shelf connector, logic that must run in a particular way — the target is a *job worker*, written in Java or another client language against the Camunda job API.

Either way the translation is a *rewrite*, and it is important to be precise about what is and is not preserved. The *intent* is preserved: the new connector or worker calls the same system X, passes the same inputs, maps the same outputs. The *implementation* is built afresh, on Camunda's job-and-connector model — because the BAW service's implementation was BAW-Process-Server-specific and none of it ports. And three things in particular are re-expressed in Camunda's terms: *authentication* — through the token discipline of [Part XXVII](#) rather than BAW's mechanisms; *retry* — through Camunda's job-retry model; and *error handling* — through Camunda's incidents and BPMN error events.

Because each BAW service is its own small rewrite, the estimate for this row cannot be a single number; it must be built *service by service*, since a trivial REST call and a complex adapter integration are very different amounts of work.

Figure [78.1](#) shows the connector-or-worker decision for a BAW service.

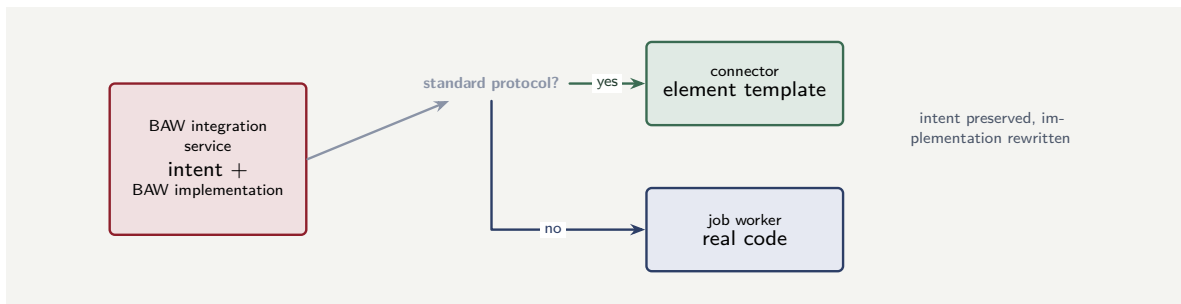


Figure 78.1. Migrating a BAW service. Each BAW integration or system service becomes a Camunda connector — for a standard protocol, configured by an element template — or a job worker — for anything needing real code. The intent is preserved; the implementation is rewritten on Camunda’s model.

78.3 The rewrite in code

The rewrite is concrete when the BAW service and its Camunda replacement are placed side by side. This section follows one integration service — a REST call to a credit bureau — through both targets.

In BAW, the integration service is defined in the tooling: an endpoint, an input mapping, an output mapping, bound to a process activity. Listing 78.1 sketches what that service definition holds.

Listing 78.1. A BAW integration service: a REST call to a credit bureau, defined in the BAW tooling with input and output mappings.

```

1 <!-- BAW integration service: CreditBureauLookup -->
2 <integrationService name="CreditBureauLookup">
3   <restBinding method="POST"
4     url="https://bureau.example/score"/>
5   <input name="ssn" variable="tw.local.customer.ssn"/>
6   <input name="amount" variable="tw.local.loan.amount"/>
7   <output name="score" variable="tw.local.bureauScore"/>
8 </integrationService>
  
```

If the credit bureau is reached over plain REST — a standard protocol — the Camunda target is a *connector*, configured by an element template with no code at all. The service task names the REST connector and supplies the same endpoint and mappings, as in Listing 78.2.

Listing 78.2. The Camunda REST connector equivalent: a service task bound to the REST connector template, with the endpoint and mappings as configuration — no code.

```

1 <bpmn:serviceTask id="creditBureau" name="Credit Bureau">
2   <bpmn:extensionElements>
3     <zeebe:taskDefinition type="io.camunda:http-json:1"/>
4     <zeebe:ioMapping>
5       <zeebe:input source="POST" target="method"/>
6       <zeebe:input source="=&#34;https://bureau...&#34;"
7         target="url"/>
8       <zeebe:output source="=body.score"
9         target="bureauScore"/>
10    </zeebe:ioMapping>
11  </bpmn:extensionElements>
12 </bpmn:serviceTask>
  
```

If instead the integration needs real code — an adapter to a packaged system, a non-trivial transformation, particular error handling — the target is a *job worker*. Listing 78.3 is the same

credit-bureau call as a worker, and it shows what the connector hides: the authentication, the call, and the explicit error handling that becomes a BPMN error.

Listing 78.3. The same integration as a Camunda job worker — the target when the call needs real code. Authentication, the call, and error handling are explicit.

```
1 @JobWorker(type = "credit-bureau-lookup")
2 public Map<String, Object> lookup(ActivatedJob job) {
3     String ssn    = job.getVariable("ssn");
4     long  amount = job.getVariable("amount");
5
6     try {
7         // token from the secrets store, not embedded
8         BureauResult r = bureauClient.score(ssn, amount);
9         return Map.of("bureauScore", r.score());
10    } catch (BureauUnavailable e) {
11        // a transient failure: let Camunda retry
12        throw e;
13    } catch (BureauRejected e) {
14        // a business error: a BPMN error, not a retry
15        throw new BpmnError("BUREAU_REJECTED",
16                            e.getMessage());
17    }
18 }
```

The three listings make the chapter's point precise. The BAW service's *intent* — call the bureau, pass ssn and amount, return the score — is identical in all three; what changes is the *implementation*. And the worker listing shows the part of the rewrite that is genuinely new work: the distinction between a *transient* failure, which is re-thrown so Camunda's job-retry mechanism handles it, and a *business* failure, which is raised as a `BpmnError` the process model catches and routes. The BAW service expressed error handling in BAW's terms; the worker expresses it in Camunda's, and that re-expression is the work the estimate must carry.

When rewriting a BAW integration service as a Camunda job worker, do not throw every failure the same way. A *transient* failure — the system is briefly down — must be re-thrown so Camunda's job retries handle it; a *business* failure — the system definitively said no — must be raised as a `BpmnError` the process model catches and routes. A worker that treats a business rejection as a retryable error will retry a call that will never succeed; one that treats a transient outage as a business error will abandon a call that would have succeeded on the next attempt.

78.4 The hidden cost of a migration

Every migration has a cost the plan does not show: the cost of the *knowledge* that lives in the old platform and in the heads of the people who run it. The old platform's configuration, its workarounds, its undocumented behaviours, its operational playbooks — all of these are knowledge that was bought with years of production experience, and none of it transfers automatically. A migration plan that costs only the artefacts — the process models, the integration code, the UI — and not the operational knowledge is undercosted, and the gap will appear in the months after go-live, when the team discovers behaviours the old platform handled that the new one does not yet, because no one remembered to transfer the knowledge.

Worked example: the adapter with no connector

Problem. A BAW estate has 90 integration services. The migration team, having learned that Camunda connectors are configured with little code, estimates all 90 as connector configurations, half a day each — 45 days. An architect reviewing the inventory finds that 30 of the 90 are adapter integrations to a packaged core system for which no off-the-shelf Camunda connector exists. Assess the estimate.

Setup. We separate the services that map to a configured connector from those that need a coded job worker.

Working. The team's estimate assumed every BAW service maps to a *connector* — a configured, low-code element. But the Camunda target depends on whether the call is over a standard protocol with an available connector. For 60 of the services — standard REST and messaging calls — that holds: they are connector configurations. For the 30 adapter integrations to the packaged core, there is *no off-the-shelf connector*, so each must be a *job worker* — real code against the system's interface:

$$60 \text{ connector configs} + 30 \text{ coded job workers} \neq 90 \text{ connector configs.}$$

A coded job worker — with its own authentication, error handling, and testing — is several times the effort of a connector configuration, so the 30 are not 15 days of the estimate; they are a large multiple of it, and the 45-day figure is badly low.

Discussion. The team made the right general observation — Camunda connectors are low-code — and then over-applied it, and the worked example shows the cost of not building the estimate service by service. “Migrate the integration services” is not one kind of work: a service that calls a standard REST endpoint genuinely is a connector configuration, but a service that drives an adapter to a packaged system with no off-the-shelf connector is a coded job worker, and the two differ by a large factor. By estimating all 90 at the connector rate, the team costed the 30 hard ones as if they were the easy kind. The discipline the chapter insists on is to inventory the services and classify each: which map to an existing connector, which need a new connector built, which need a job worker, and how complex each worker is — and to sum the classified estimate, not apply one rate to a count. The lesson is the row's defining point: each BAW service is its own small rewrite, the rewrites are not uniform, and an estimate that does not go service by service will be wrong by whatever proportion of the estate turns out to be the hard kind — here, a third of it.

Practical next steps

Inventory the BAW integration and system services and classify each by its Camunda target: an existing connector, a connector to be built, or a coded job worker — and rate the workers by complexity. Sum the classified estimate rather than applying a connector rate to a service count. The services that need coded workers are several times the effort of the ones that do not.

78.5 Migration risk and mitigation

Every migration carries risk, and the risk is managed by acknowledging it explicitly rather than hoping it will not materialise. The principal risks are *functional regression* (the migrated process does not behave as the old one did), *data loss* (in-flight instances or their variables are lost or corrupted), and *performance degradation* (the new platform is slower under the same load). Each is mitigated by a specific discipline: functional regression by parallel running and

comparison testing against the old platform; data loss by verifiable, checkpointed migration with rollback capability; performance degradation by load testing against realistic volumes before cutover. A migration plan that does not name its risks and their mitigations is a plan that has not been thought through.

The chapter's principles interact with the platform's operational model in a way worth making explicit. The *deployment* of the artefacts this chapter describes must go through a governed pipeline — versioned, reviewed, tested, and traceable. An artefact deployed by a manual action, outside the pipeline, is an artefact whose presence in production cannot be explained, and in a regulated institution an unexplained artefact is a finding.

The *monitoring* of the artefacts must cover both their functional correctness and their operational health. A process step that executes but produces wrong results is harder to detect than one that fails outright, because it does not raise an error — it raises a consequence, and the consequence may not be discovered until a customer complains or a regulator asks a question. The observability model should therefore include not only error rates and latencies but also *outcome monitoring*: checks that the results the process produces are within expected ranges, flagging anomalies for review.

Chapter in brief

A BAW process reaches other systems through *integration services* and *general system services* — BAW service constructs that invoke a web service, run an adapter, call a REST endpoint, or execute server-side logic, configured and partly scripted in the BAW tooling and run inside the Process Server. The Camunda target is a *connector* — for a standard protocol, configured by an element template — or a *job worker* — for anything needing real code against the job API. The translation is a rewrite: the *intent* is preserved, but the implementation is built afresh on Camunda's model, with authentication, retry, and error handling re-expressed in Camunda's terms. Because each service is its own small rewrite and the rewrites are not uniform, the estimate must be built service by service.

Migrating Server-Side Script

79.1 The inline script estate

The fourth construct is the logic a BAW estate accumulates not as services but as *inline script*. BAW lets process logic be written as *server-side JavaScript* in script activities — short pieces of code, embedded directly in the process flow, that manipulate process variables, shape data, or make a decision inline.

Individually each script is small. Collectively they are not. Over years of a BAW estate's life, script activities accumulate: a date calculation here, a field-mapping there, a small branching decision somewhere else, each added because, at the moment, writing three lines of JavaScript in the flow was the quickest thing to do. A mature BAW process can be threaded through with dozens of these, and the estate as a whole holds a great deal of inline script — script that is real logic, that the process depends on, and that a migration must account for.

79.2 Classify, then re-home

The Camunda target for a script activity is not a single thing; it depends on what the script actually *does*, and the translation work is therefore a two-step discipline: *classify*, then *re-home*.

The classification sorts each script into one of two kinds. The first kind is *trivial*: pure data shaping, a simple calculation, a straightforward mapping from one variable to another — logic with no external call and no real complexity. The second kind is *substantial*: real logic, an external call, a non-trivial transformation, a decision involved enough to deserve testing in its own right.

The re-homing then follows the classification. A *trivial* script becomes a *FEEL expression* — Camunda's expression language, evaluated inline by the engine, with no code and no separate deployment. The three-line date calculation becomes a FEEL expression in the element that needs it, and it simply disappears as a separate artefact. A *substantial* script becomes a *job worker* — the same target as an integration service of [Chapter 78](#) — because real logic deserves to be real, tested, deployed code, not script buried in a diagram.

The row is rated *moderate* because much inline script is genuinely trivial and collapses cleanly into FEEL — but a migration plan must still *read* every script to classify it, and must budget properly for the substantial minority that become workers. The danger is a plan that assumes all inline script is trivial; the reality is that some of it is a worker in disguise.

79.3 The two translations in code

The classify-then-re-home discipline is clearest seen in code. The two listings below take real BAW server-side script and show its Camunda target — one trivial, collapsing into FEEL; one substantial, becoming a job worker.

A *trivial* script is the common case. Listing [79.1](#) shows a typical BAW script activity: a few lines of server-side JavaScript that derive a value from existing process variables — here, classifying a loan by amount.

Listing 79.1. A trivial BAW server-side script activity: pure data shaping from existing variables, no external call.

```
1 // BAW script activity (server-side JavaScript)
2 var amount = tw.local.application.amount;
3
4 if (amount >= 1000000) {
5     tw.local.application.band = "large";
6 } else if (amount >= 100000) {
7     tw.local.application.band = "standard";
8 } else {
9     tw.local.application.band = "small";
10 }
```

This script has no external call and no real complexity — it is pure data shaping. Its Camunda target is not code at all but a *FEEL expression*, set as an output mapping or in the element that needs the value. Listing 79.2 is the whole of it.

Listing 79.2. The Camunda equivalent of the trivial script: a FEEL expression, evaluated inline by the engine, with no code and no deployment.

```
1 // Camunda FEEL expression (an output mapping on the element)
2 if      application.amount >= 1000000 then "large"
3 else if application.amount >= 100000  then "standard"
4 else                                     "small"
```

The script has disappeared as a separate artefact: there is no code to deploy, no worker to run, just an expression the engine evaluates. This is what *moderate* effort looks like for the trivial majority — a near-mechanical collapse.

A *substantial* script is the case a plan must not miss. Listing 79.3 sketches a BAW script that is doing far more: it calls an external service, parses a response, and handles failure — logic that deserves to be real, tested code.

Listing 79.3. A substantial BAW script activity: an external call, response parsing, and error handling — a worker in disguise.

```
1 // BAW script activity -- but really a worker
2 var svc = tw.local.rateServiceUrl;
3 var resp = new Packages.HttpClient().get(
4     svc + "?ccy=" + tw.local.currency);
5 if (resp.status != 200) {
6     tw.local.rateError = true;           // ad-hoc handling
7 } else {
8     var rate = JSON.parse(resp.body).rate;
9     tw.local.converted = tw.local.amount * rate;
10 }
```

This is not data shaping — it makes an HTTP call, parses JSON, and handles an error case. Its Camunda target is a *job worker*: real code, against the job API, with Camunda's retry and incident handling instead of the script's ad-hoc `rateError` flag. Listing 79.4 is the skeleton.

Listing 79.4. The Camunda equivalent of the substantial script: a job worker, with the external call as real code and failure handled by Camunda's job retries and incidents.

```
1 @JobWorker(type = "currency-conversion")
2 public Map<String, Object> convert(ActivatedJob job) {
3     String currency = job.getVariable("currency");
4     double amount   = job.getVariable("amount");
5
6     // a failed call throws; Camunda retries, then
7     // raises an incident -- no ad-hoc error flag
8     BigDecimal rate = rateService.rateFor(currency);
```

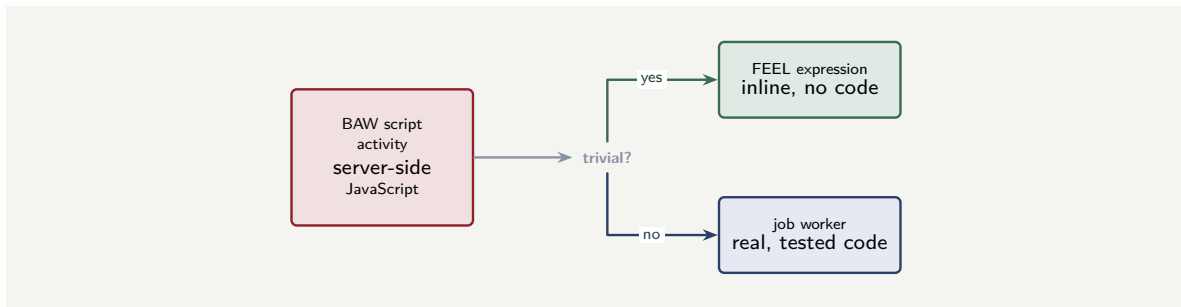


Figure 79.1. Re-homing a BAW script activity. Each inline server-side script is classified: trivial data-shaping collapses into a FEEL expression with no code; substantial logic becomes a job worker — real, tested, deployed code.

```

9
10     return Map.of(
11         "converted", BigDecimal.valueOf(amount)
12             .multiply(rate));
13 }
  
```

The contrast between the two pairs is the chapter’s whole point in concrete form. The trivial script collapsed to three lines of FEEL with nothing to deploy; the substantial one became a deployed, tested worker with its own error semantics. They are the same kind of artefact in BAW — a script activity — and entirely different amounts of work in Camunda, which is exactly why every script must be read and classified rather than counted.

Figure 79.1 shows the classify-then-re-home decision.

79.4 The hidden cost of a migration

Every migration has a cost the plan does not show: the cost of the *knowledge* that lives in the old platform and in the heads of the people who run it. The old platform’s configuration, its workarounds, its undocumented behaviours, its operational playbooks — all of these are knowledge that was bought with years of production experience, and none of it transfers automatically. A migration plan that costs only the artefacts — the process models, the integration code, the UI — and not the operational knowledge is undercosted, and the gap will appear in the months after go-live, when the team discovers behaviours the old platform handled that the new one does not yet, because no one remembered to transfer the knowledge.

Worked example: the script that was really a worker

Problem. A BAW process has a script activity the team’s plan lists, with all the others, as “trivial inline script — collapses to FEEL.” On inspection, the script: calls an external currency-rate service over HTTP, parses the response, applies a multi-step rounding and fee calculation, and handles the case where the service is unavailable. Assess the classification.

Setup. We test the script against the chapter’s two-kind classification.

Working. The chapter’s trivial kind is pure data shaping or a simple calculation with *no external call and no real complexity*. This script makes an *external HTTP call*, parses a response, runs a *multi-step* calculation, and has *error handling*:

external call + multi-step logic + error handling $\Rightarrow$ substantial, not trivial.

It is, by the chapter’s definition, a *substantial* script, and its correct Camunda target is a *job*

worker — not a FEEL expression. The plan has mis-classified it, and a FEEL expression cannot, in any case, make an HTTP call and handle a service outage.

Discussion. The mis-classification matters because the two targets are not close. A FEEL expression is an inline, no-code, no-deployment artefact estimated in minutes; a job worker that calls an external service is real code with its own authentication, error handling, retry, tests, and deployment — a different order of effort entirely. By listing this script as “trivial, collapses to FEEL,” the plan costed a job worker as a one-line expression. The error is the one the chapter’s row warns of directly: assuming *all* inline script is trivial. Much of it genuinely is — a date calculation, a field copy — and that much does collapse cleanly into FEEL. But a script activity is just a box in a diagram, and a box can contain three trivial lines or a small application; the only way to know is to *read* it. The discipline is the chapter’s two steps applied honestly: read every script, classify each as trivial or substantial against the external-call-and-complexity test, and re-home accordingly — FEEL for the trivial majority, a budgeted job worker for each substantial one. The lesson is that the script estate cannot be estimated by counting boxes; it must be estimated by reading them, because the box gives no hint of whether it holds a FEEL expression or a worker.

Practical next steps

Do not estimate a BAW script estate by counting script activities. Read each script and classify it: trivial data-shaping that collapses to a FEEL expression, or substantial logic — external calls, multi-step computation, error handling — that must become a budgeted job worker. A script activity’s box gives no hint of which it is.

79.5 Migration risk and mitigation

Every migration carries risk, and the risk is managed by acknowledging it explicitly rather than hoping it will not materialise. The principal risks are *functional regression* (the migrated process does not behave as the old one did), *data loss* (in-flight instances or their variables are lost or corrupted), and *performance degradation* (the new platform is slower under the same load). Each is mitigated by a specific discipline: functional regression by parallel running and comparison testing against the old platform; data loss by verifiable, checkpointed migration with rollback capability; performance degradation by load testing against realistic volumes before cutover. A migration plan that does not name its risks and their mitigations is a plan that has not been thought through.

The chapter’s principles interact with the platform’s operational model in a way worth making explicit. The *deployment* of the artefacts this chapter describes must go through a governed pipeline — versioned, reviewed, tested, and traceable. An artefact deployed by a manual action, outside the pipeline, is an artefact whose presence in production cannot be explained, and in a regulated institution an unexplained artefact is a finding.

The *monitoring* of the artefacts must cover both their functional correctness and their operational health. A process step that executes but produces wrong results is harder to detect than one that fails outright, because it does not raise an error — it raises a consequence, and the consequence may not be discovered until a customer complains or a regulator asks a question. The observability model should therefore include not only error rates and latencies but also *outcome monitoring*: checks that the results the process produces are within expected ranges, flagging anomalies for review.

Chapter in brief

BAW lets process logic be written as *server-side JavaScript* in script activities — short pieces of code embedded in the flow — and over years an estate accumulates a great deal of it. The Camunda target depends on what the script does, so the translation is two steps: *classify*, then *re-home*. A *trivial* script — pure data shaping, a simple calculation, no external call — collapses into a *FEEL expression*, inline and code-free. A *substantial* script — real logic, external calls, non-trivial transformation — becomes a *job worker*, real tested code. The row is moderate because much inline script is genuinely trivial, but a plan must read every script to classify it, and must budget for the substantial minority that are workers in disguise.

CHAPTER 80

Migrating Business Objects

80.1 What a business object is

The fifth construct is the shape of the data a process carries. In IBM BAW that shape is defined by *business objects*.

A business object is a structured data type — BAW’s way of declaring that “a loan application” has these typed fields, with this nested structure: an applicant, who has a name and an address; an amount; a term; a set of documents. The process variables in a BAW process are *typed* against these business objects: a variable is not just “some data,” it is a `LoanApplication`, and the BAW tooling knows its shape.

A mature BAW estate has a library of business objects, often layered — common objects shared across processes, specific ones for particular processes — and the processes are written against them throughout.

80.2 The data-modelling translation

The Camunda target is the *process variable* as Camunda holds it. Camunda process variables are typically *JSON-shaped* data — structured, nested, but not declared against a separate strongly-typed schema in the way a BAW business object is.

So the translation is a *data-modelling exercise*, in two parts. First, each business object’s structure is *re-expressed* as the JSON shape the Camunda process will carry — the same fields, the same nesting, rendered as the JSON object the process variables will hold. Second, every place the old process *read or wrote* a business-object field must be *re-pointed* at the new variable structure — every input mapping, every output mapping, every expression that navigated into the business object.

The row is rated *low-to-moderate*, and the range is the honest answer. It is not *high* because the data model itself usually carries over *conceptually*: a loan application has the same fields whichever platform holds it, and re-expressing a clean, well-structured business object as JSON is mechanical. But it is not *low* either, for two reasons. The *typing and access pattern change* — BAW’s strongly-typed business object becomes Camunda’s more loosely-typed JSON, and the process’s every access to the data is re-pointed. And a *sprawling or inconsistent* business-object estate — the same concept modelled three different ways in three processes, fields that are typed loosely, deep nesting accumulated over years — makes the re-expression real, investigative work rather than a mechanical one.

80.3 The translation in code

The data-modelling translation is concrete when seen as the two artefacts side by side. Listing 80.1 is a BAW business object: a strongly-typed structure, with named fields, declared types, and a nested object — BAW’s declaration of what “a loan application” is.

Listing 80.1. A BAW business object: a strongly-typed structured data type, with named fields, declared types, and nested structure.

```
1 // BAW business object: LoanApplication
2 LoanApplication
3     applicant : Person          // nested object
4     amount    : Decimal
5     term      : Integer
6     documents : List<Document>
7
8 // the nested Person business object
9 Person
10     fullName : String
11     dateOfBirth : Date
12     address   : Address
```

The Camunda target is the *process variable* as Camunda holds it — JSON-shaped data. Listing 80.2 is the same loan application re-expressed as the JSON object a Camunda process will carry: the same fields, the same nesting, but no separate strongly-typed schema declared against it.

Listing 80.2. The Camunda equivalent: the same structure re-expressed as the JSON-shaped process variable a Camunda process carries.

```
1 {
2   "applicant": {
3     "fullName": "Jordan Mensah",
4     "dateOfBirth": "1985-04-12",
5     "address": { "line1": "...", "postcode": "..." }
6   },
7   "amount": 250000,
8   "term": 240,
9   "documents": [ { "type": "payslip" } ]
10 }
```

Two differences between the listings are the substance of the row. The first is the *typing change*: the BAW object declares amount as a Decimal and term as an Integer, and the tooling enforces those types; the JSON simply holds 250000 and 240, and nothing in the variable itself guarantees their types — so where the BAW process relied on the type system, the Camunda process must validate, in a form, a connector, or a FEEL expression. The second is the *access pattern*: every place the BAW process referred to `tw.local.application .applicant.fullName` must be re-pointed at the JSON path `application .applicant.fullName` in Camunda's terms. For one clean object this is near-mechanical — which is the *low* end of the rating — but, as the worked example below shows, an estate where the same concept is shaped three different ways turns this into genuine data-modelling design.

Figure 80.1 shows the business-object-to-JSON translation.

80.4 The hidden cost of a migration

Every migration has a cost the plan does not show: the cost of the *knowledge* that lives in the old platform and in the heads of the people who run it. The old platform's configuration, its workarounds, its undocumented behaviours, its operational playbooks — all of these are knowledge that was bought with years of production experience, and none of it transfers automatically. A migration plan that costs only the artefacts — the process models, the integration code, the UI — and not the operational knowledge is undercosted, and the gap will appear in the months after go-live, when the team discovers behaviours the old platform handled that the new one does not yet, because no one remembered to transfer the knowledge.

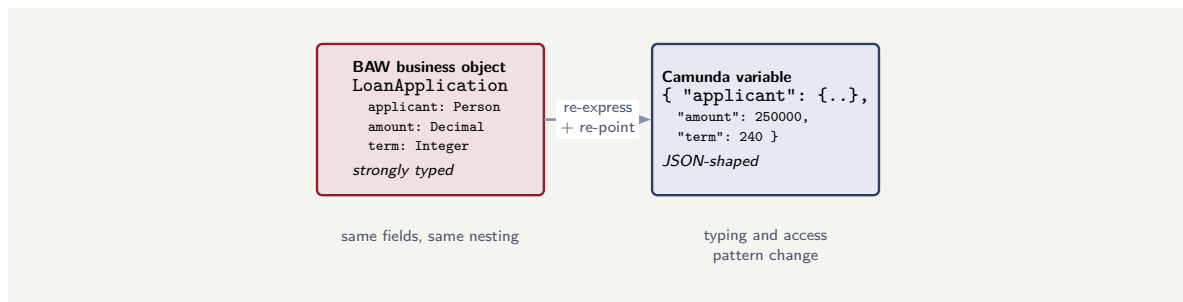


Figure 80.1. Migrating a business object. The strongly-typed BAW business object is re-expressed as the JSON shape a Camunda process variable carries — the same fields and nesting — and every place the process read or wrote a field is re-pointed at the new structure.

Worked example: the customer modelled three ways

Problem. A bank’s BAW estate has three processes that each handle “the customer.” On inspection, the three define the customer differently: one business object calls it `Customer` with a nested `Address`; another calls it `Party` with address fields flattened to the top level; the third calls it `ClientRecord` with the address as a separate linked object. The migration plan budgeted business-object translation as “low — mechanical re-expression as JSON.” Assess this.

Setup. We consider whether the estate is the clean kind the low rating assumes or the sprawling kind the moderate end describes.

Working. The low end of the rating assumes the data model carries over conceptually — the same concept, modelled consistently, mechanically re-expressed. This estate does not meet that assumption: one real-world concept, the customer, is modelled *three different ways*, with three names, three structures, three treatments of the address:

one concept → three inconsistent business objects.

Re-expressing each as JSON is still mechanical *per object*, but the migration now faces a prior question the low rating ignored: should the three become one consistent Camunda data shape, or three? That is data-modelling *design* work — reconciling the inconsistency — not mechanical translation, and it is exactly the “sprawling or inconsistent estate” the moderate end of the rating describes.

Discussion. The plan applied the low rating, and the worked example shows that the rating’s range — low-to-moderate — is not vagueness but a genuine fork that depends on the estate. A clean estate, where each concept is modelled once and consistently, genuinely is the low case: re-expressing well-structured business objects as JSON is mechanical. But this estate is the moderate case, and the tell is the inconsistency: three names and three structures for one concept means the migration cannot just translate, it must first *decide* — reconcile the three into one coherent customer shape, or deliberately keep them separate — and that decision is design work, with consequences for every process that touches a customer. The plan’s “low — mechanical” rating silently assumed the clean case without checking which case the estate actually was. The lesson is to assess the business-object estate before rating it: a consistent, well-structured estate is genuinely low-effort, but a sprawling one — the same concept modelled many ways, loose typing, deep accreted nesting — carries a real data-modelling design task, and the migration that does not see this inherits the old inconsistency or pays, unbudgeted, to resolve it.

Practical next steps

Before rating business-object translation, inspect the estate for consistency. If each real-world concept is modelled once, coherently, the re-expression as JSON is genuinely low-effort. If the same concept is modelled several ways across processes, the migration carries a data-modelling design task — to reconcile or deliberately keep separate — that must be budgeted, not assumed away.

80.5 Migration risk and mitigation

Every migration carries risk, and the risk is managed by acknowledging it explicitly rather than hoping it will not materialise. The principal risks are *functional regression* (the migrated process does not behave as the old one did), *data loss* (in-flight instances or their variables are lost or corrupted), and *performance degradation* (the new platform is slower under the same load). Each is mitigated by a specific discipline: functional regression by parallel running and comparison testing against the old platform; data loss by verifiable, checkpointed migration with rollback capability; performance degradation by load testing against realistic volumes before cutover. A migration plan that does not name its risks and their mitigations is a plan that has not been thought through.

The chapter's principles interact with the platform's operational model in a way worth making explicit. The *deployment* of the artefacts this chapter describes must go through a governed pipeline — versioned, reviewed, tested, and traceable. An artefact deployed by a manual action, outside the pipeline, is an artefact whose presence in production cannot be explained, and in a regulated institution an unexplained artefact is a finding.

The *monitoring* of the artefacts must cover both their functional correctness and their operational health. A process step that executes but produces wrong results is harder to detect than one that fails outright, because it does not raise an error — it raises a consequence, and the consequence may not be discovered until a customer complains or a regulator asks a question. The observability model should therefore include not only error rates and latencies but also *outcome monitoring*: checks that the results the process produces are within expected ranges, flagging anomalies for review.

Chapter in brief

A BAW *business object* is a structured data type — BAW's way of declaring the typed, nested shape of the data a process carries — and BAW process variables are strongly typed against these objects. The Camunda target is the *process variable* as Camunda holds it: typically JSON-shaped data. The translation is a data-modelling exercise — each business object re-expressed as a JSON shape, and every read and write of a field re-pointed at the new structure. The row is low-to-moderate: low because the data model usually carries over conceptually, with the same fields and nesting; but not zero, because the typing and access pattern change, and a sprawling or inconsistent business-object estate turns the re-expression into real data-modelling design work.

Migrating Toolkits and Shared Assets

81.1 What a Toolkit is

The sixth construct is BAW's mechanism for *reuse*. In a BAW estate of any maturity, common logic is not copied into every process that needs it; it is factored into a *Toolkit*.

A Toolkit is BAW's unit of reuse: a package of shared assets — subprocesses, services, coach views, business objects — that multiple process applications reference. A mature BAW estate is therefore *layered*: common Toolkits sit underneath, and the process applications sit on top of them, each application drawing on the Toolkits it needs. A single Toolkit — say, a shared “customer-identity” Toolkit — may be referenced by a dozen applications.

This layering is good engineering in BAW, and it is exactly what makes the Toolkit a distinct migration problem: a Toolkit is not migrated for its own sake but because applications depend on it, and those dependencies form a graph that the migration must understand.

81.2 The fan-out, and the dependency graph

There is *no single Camunda construct called a Toolkit*. Camunda's reuse is provided by several different mechanisms, and a Toolkit's contents *fan out* into them by kind.

A shared *subprocess* becomes a *shared process invoked by a call activity* — the mechanism of [Chapter 49](#). A shared *service* becomes a *shared connector or connector template*. A shared *coach view* becomes a component in a *shared form or task-application library*. A shared *business object* becomes a *shared data-shape definition*. One Toolkit, in other words, does not become one Camunda thing; its contents scatter into four different reuse mechanisms according to what each asset is.

So the translation has two parts. The second part is *re-homing* each kind of Toolkit content into its matching Camunda mechanism — the fan-out just described. But the *first* part, and the one a plan most often omits, is *mapping the dependency graph*: establishing which applications depend on which Toolkits, and which Toolkits depend on other Toolkits. This must come first, because a Toolkit cannot be migrated independently of the applications that use it — migrating a Toolkit's shared subprocess means every application that called it must now call the new Camunda shared process. The row is rated *moderate*, and its hidden cost is precisely this dependency mapping: a plan that inventories applications but never maps the Toolkit dependencies beneath them has not seen the shape of its own estate.

Figure 81.1 shows the Toolkit fan-out and the dependency mapping that must precede it.

81.3 Re-homing a Toolkit asset in code

The fan-out is concrete when a single shared asset is followed from BAW to Camunda. This section takes the most common case — a shared *subprocess* in a Toolkit — and shows the

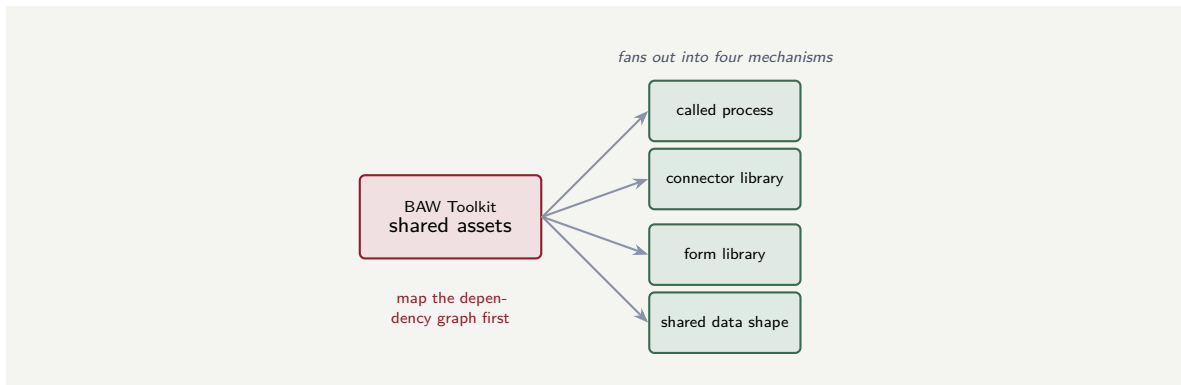


Figure 81.1. Migrating a Toolkit. There is no Camunda “Toolkit”; the Toolkit’s contents fan out by kind into Camunda’s several reuse mechanisms — called processes, connector and form libraries, shared data shapes. The hidden first step is mapping which applications depend on which Toolkits.

re-homing in code, because this is the case a migration most often gets subtly wrong.

In BAW, a Toolkit’s shared subprocess is a piece of flow defined once in the Toolkit and called, in place, by every process application that needs it. Suppose a “CustomerIdentity” Toolkit contains a shared subprocess `VerifyIdentity`, used by the onboarding, lending, and account-opening applications. In each calling process, the BAW model embeds a reference to that Toolkit subprocess, as sketched in Listing 81.1.

Listing 81.1. A BAW process application calling a shared subprocess from the CustomerIdentity Toolkit. Each calling application carries a reference of this kind.

```

1 <!-- inside the onboarding process application -->
2 <callActivity id="verifyId"
3   name="Verify Identity"
4   calledElement="VerifyIdentity"
5   toolkit="CustomerIdentity"/>
6 <!-- the same reference appears in the lending and
7   account-opening applications -->
  
```

The Camunda re-homing has two halves, and both must be done — doing only the first is the mistake. The *first* half: the shared subprocess becomes a *standalone, independently-deployed BPMN process* — the called process of Chapter 49. It is migrated *once*, given its own process ID, and deployed on its own. Listing 81.2 is its defining element.

Listing 81.2. The shared subprocess re-homed as a standalone Camunda process — migrated once, deployed on its own, with its own process ID.

```

1 <!-- verify-identity.bpmn -- a process in its own right -->
2 <bpmn:process id="verify-identity"
3   name="Verify Identity" isExecutable="true">
4   <!-- ... the verification flow, migrated once ... -->
5 </bpmn:process>
  
```

The *second* half: every application that used the Toolkit subprocess now calls that standalone process through a *call activity*, naming it by its process ID. Listing 81.3 is the call activity each of the three applications carries — and the critical point is that all three point at the *same* processId, so there is still exactly one `verify-identity` process, shared, as the Toolkit intended.

Listing 81.3. The call activity each migrated application uses. All callers name the same process ID, so the reuse the Toolkit provided is preserved — one shared process, many callers.

```
1 <!-- inside each migrated calling process -->
2 <bpmn:callActivity id="verifyId" name="Verify Identity">
3   <bpmn:extensionElements>
4     <zeebe:calledElement
5       processId="verify-identity"
6       propagateAllChildVariables="false"/>
7   </bpmn:extensionElements>
8 </bpmn:callActivity>
```

The two halves of re-homing a shared Toolkit subprocess must *both* be done, and in this order: migrate the subprocess once as a standalone process, *then* point every calling application's call activity at that one process ID. A migration that re-homes the subprocess separately inside each application — a copy per caller — produces several independent processes where the Toolkit deliberately had one. The reuse silently vanishes, and the copies drift. One shared Toolkit asset must become one shared Camunda asset, never one-per-caller.

81.4 Re-homing a shared service: the second fan-out path

A shared subprocess is one kind of Toolkit content; a shared *service* is another, and it re-homes by a different path — worth showing in code because the “migrate once, reference many” discipline applies to it just as strictly, through a different Camunda mechanism.

Suppose the CustomerIdentity Toolkit also contains a shared *service* — LookupSanctionsList — that every calling application invokes to check a customer against a sanctions register. In BAW it is a Toolkit service, referenced by an activity in each application, as in Listing 81.4.

Listing 81.4. A BAW process activity invoking a shared service from the CustomerIdentity Toolkit. As with the shared subprocess, every caller carries a reference of this kind.

```
1 <!-- inside each calling process application -->
2 <serviceTask id="sanctionsCheck"
3   name="Lookup Sanctions List"
4   calledService="LookupSanctionsList"
5   toolkit="CustomerIdentity"/>
```

A shared service does not become a called *process* — it is not a piece of flow, it is a unit of integration. It re-homes to a *shared job worker* addressed by a *job type*: the worker is implemented and deployed *once*, and every calling process reaches it by naming the same job type on a service task. Listing 81.5 is the worker — the single, shared implementation.

Listing 81.5. The shared service re-homed as a single Camunda job worker, deployed once and addressed by its job type. One implementation, reused by every caller.

```
1 @Component
2 public class SanctionsLookupWorker {
3
4   private final SanctionsRegister register;
5
6   public SanctionsLookupWorker(SanctionsRegister r) {
7     this.register = r;
8   }
9
10  // every caller's service task names this job type
```

Toolkit	depended on by
CustomerIdentity	onboarding, lending, account-opening (3 apps)
NotificationKit	onboarding, lending, claims, servicing (4 apps)
CommonData	all apps + CustomerIdentity, NotificationKit (Toolkit-on-Toolkit)

Figure 81.2. A Toolkit dependency map. Each row records a Toolkit and everything that depends on it — applications, and other Toolkits. CommonData is depended on by another Toolkit as well as by applications, so it must be migrated before the Toolkits that use it.

```

11  @JobWorker(type = "lookup-sanctions-list")
12  public Map<String, Object> lookup(ActivatedJob job) {
13      String customerId = job.getVariable("customerId");
14      SanctionsHit hit = register.check(customerId);
15      return Map.of(
16          "sanctionsMatch", hit.isMatch(),
17          "sanctionsRef", hit.reference());
18  }
19  }

```

Each migrated application’s service task then simply names that job type, as in Listing 81.6. There is no copy of the lookup logic in any application — only a reference to the one shared worker, exactly as the BAW Toolkit intended.

Listing 81.6. The service task each migrated application carries. All callers name the same job type, so the one shared worker serves them all.

```

1  <!-- inside each migrated calling process -->
2  <bpmn:serviceTask id="sanctionsCheck"
3      name="Lookup Sanctions List">
4      <bpmn:extensionElements>
5          <zeebe:taskDefinition type="lookup-sanctions-list"/>
6      </bpmn:extensionElements>
7  </bpmn:serviceTask>

```

The two fan-out paths now stand side by side. A shared *subprocess* re-homes to a standalone *called process*, invoked by a call activity naming its process ID; a shared *service* re-homes to a shared *job worker*, invoked by a service task naming its job type. The mechanisms differ, but the discipline is identical: the shared asset is migrated *once*, and every caller is pointed at that one thing. A Toolkit’s contents fan out into different Camunda mechanisms by kind — and each kind keeps the “one shared asset, many references” rule that makes it shared at all.

Before any of that re-homing can be done safely, the dependency graph must be mapped — and it is worth seeing what that artefact concretely looks like, because “map the dependency graph” is otherwise an instruction with no shape.

A dependency map is, at its simplest, a table: each Toolkit, and the applications — and other Toolkits — that depend on it. Table 81.2 is such a map for a small estate.

Table 81.2 makes two things visible that an application-only inventory never does. First, the *shared* Toolkits — CustomerIdentity is used by three applications, NotificationKit by four — so each must be migrated *once* and the migrated applications pointed at it, exactly the pattern

the previous section’s code showed. Second, the *Toolkit-on-Toolkit* dependency: `CommonData` is depended on not only by applications but by the other two Toolkits, which means `CommonData` must be migrated *before* `CustomerIdentity` and `NotificationKit`, and those before the applications. The dependency map is what yields that ordering; without it, a migration picks an order by guesswork and discovers the constraints by hitting them.

Worked example: the migration order read off the dependency map

Problem. A bank’s BAW estate is the one in Table 81.2: three Toolkits — `CommonData`, `CustomerIdentity`, `NotificationKit` — and several applications. A migration team, eager to show progress, proposes to start with `CustomerIdentity` because it is the “most important” Toolkit. An architect says the dependency map dictates a different starting point. Who is right, and what is the correct order?

Setup. We read the dependency constraints off Table 81.2 and derive an order that respects them.

Working. The map records that `CustomerIdentity` and `NotificationKit` both depend on `CommonData` — the Toolkit-on-Toolkit row. A Camunda asset cannot be built against a shared dependency that does not yet exist, so:

`CommonData` must be migrated before `CustomerIdentity` and `NotificationKit`.

And every Toolkit must exist before the applications that call it, since the applications’ call activities and connector references must point at already-migrated shared assets:

Toolkits before applications.

Reading these constraints off the map yields the order: `CommonData` first; then `CustomerIdentity` and `NotificationKit` (in either order, or in parallel — the map shows no dependency between them); then the applications. Starting with `CustomerIdentity`, as the team proposed, would begin migrating a Toolkit before the `CommonData` Toolkit it depends on exists.

Discussion. The architect is right, and the worked example shows the dependency map earning its place. The team’s instinct — start with the “most important” Toolkit — chose an order by *salience*, which is not a technical constraint at all; importance does not determine what can be built first, dependency does. `CustomerIdentity` depends on `CommonData`, so beginning with `CustomerIdentity` means building a Camunda asset whose own shared dependency is not yet there — the migration stalls immediately, or, worse, the team builds `CustomerIdentity` against a temporary stand-in for `CommonData` and has to redo it. The dependency map removes the guesswork: it states, in a form anyone can read, that `CommonData` is depended on by the other Toolkits and must come first, that the Toolkits must precede the applications, and that `CustomerIdentity` and `NotificationKit` have no dependency between them and so may go in parallel — which is genuinely useful, because it tells the team where it *can* safely parallelise. The lesson is the chapter’s central one made operational: “map the dependency graph first” is not bureaucratic thoroughness; it is what converts a Toolkit estate into a correct migration *order*, and a team that migrates by salience, or by whichever Toolkit someone opened first, will hit the dependency constraints the hard way — mid-migration, as rework.

Practical next steps

Before migrating a Toolkit estate, build the dependency map: for each Toolkit, record every application and every other Toolkit that depends on it. Read the migration order off the map — Toolkits depended on by other Toolkits first, all Toolkits before applications, independent Toolkits in parallel. A team that picks an order by importance or convenience will discover the real constraints as rework.

Worked example: the Toolkit migrated in isolation

Problem. A migration team, working application by application, migrates an application that uses a shared “customer-identity” Toolkit. They migrate the Toolkit’s contents along with it. Two weeks later, migrating a second application, they find it uses the *same* Toolkit — and a third and fourth application do too. Each was migrated, or is being migrated, with its *own* copy of the Toolkit’s logic. Assess what has happened.

Setup. We consider what the team did not establish before migrating the first application.

Working. The team migrated application by application, and migrated each application’s Toolkit content *as part of* that application. But the “customer-identity” Toolkit is shared — four applications depend on it — and migrating it four times, once per application, produces four separate copies of what was, in BAW, deliberately *one* shared thing:

one shared Toolkit → four independent copies.

The reuse the Toolkit existed to provide is lost on the Camunda side, and the bank now has four copies of customer-identity logic to maintain and keep consistent — the very duplication the Toolkit was created to prevent.

Discussion. The team skipped the chapter’s mandatory first step — mapping the dependency graph — and the worked example shows the concrete cost. Working application by application is a reasonable migration rhythm, but it only works if the *shared* layer is recognised first: a Toolkit is, by definition, depended on by many applications, and it must be migrated *once*, as a shared Camunda asset — a shared called process, a shared library — that the migrated applications then reference. By migrating the Toolkit inside the first application, and again inside the second, the team turned one shared asset into four divergent copies, recreating on Camunda exactly the duplication BAW’s Toolkit mechanism was designed to eliminate. Worse, the four copies will now drift, because nothing ties them together. The fix is the chapter’s ordering: before migrating any application, map the dependency graph — which applications use which Toolkits — then migrate each Toolkit once into a shared Camunda asset, and only then migrate the applications, each pointing at the shared asset. The lesson is that a Toolkit cannot be migrated as a by-product of the first application that happens to use it; it is shared infrastructure, and shared infrastructure must be migrated once, deliberately, with its dependents known — which is why the dependency mapping is the row’s real, and most-omitted, work.

Practical next steps

Before migrating BAW applications one by one, map the Toolkit dependency graph — which applications depend on which Toolkits. Migrate each shared Toolkit once, into a shared Camunda asset, and have the migrated applications reference it. A Toolkit migrated as a by-product of each application that uses it becomes several divergent copies of what was deliberately one shared thing.

Chapter in brief

A *Toolkit* is BAW's unit of reuse — a package of shared subprocesses, services, coach views, and business objects that multiple process applications reference, so a mature estate is layered with Toolkits beneath applications. There is no single Camunda construct called a Toolkit; the contents *fan out* by kind into Camunda's several reuse mechanisms — a shared subprocess becomes a called process, a shared service a connector library, a shared coach view a form-library component, a shared business object a shared data shape. The translation has two parts, and the one a plan omits comes first: mapping the *dependency graph*, since a Toolkit cannot be migrated independently of the applications that use it, and a Toolkit migrated per-application becomes several divergent copies of one shared thing.

Migrating the WebSphere Runtime

82.1 What the runtime actually is

The eighth construct is not a process artefact at all — it is the *platform the processes run on*. A BAW migration that translates every process and rebuilds every coach has still not finished, because the processes need somewhere to run.

A BAW process runs on the *Process Server*, hosted — in a traditional deployment — on IBM *WebSphere Application Server*, the Java EE application server that underpins much of IBM's middleware. The operational substance of a BAW platform is therefore *WebSphere-shaped*. It is WebSphere data sources defining the database connections; WebSphere JMS resources for messaging; WebSphere security realms for authentication; WebSphere cluster topology for scaling and availability; WebSphere deployment artefacts; and the WebSphere administration model — the consoles, the scripts, the operational practices — that an operations team uses to run all of it.

None of this is visible on a process diagram, and none of it is optional: it is the running platform, and a migration must provide its equivalent.

82.2 Rebuilt, not ported

The Camunda target is the cloud-native *Zeebe* runtime of [Chapter 27](#) — and the essential word for this row is that the runtime is *rebuilt, not ported*.

Zeebe is a fundamentally different runtime from the WebSphere-hosted Process Server. It is designed to *scale horizontally* — by adding nodes that share the load — rather than by the vertical-capacity-and-clustering model of a WebSphere installation. It is deployed and operated differently: as a cloud-native system, typically containerised, often on Kubernetes, rather than as an application deployed into an application server. Nothing of the WebSphere configuration *ports*: a WebSphere data source, a WebSphere security realm, a WebSphere cluster definition has no meaningful “equivalent file” on the Camunda side — the concepts themselves are different.

So this row is an *infrastructure and operations workstream in its own right*: defining the Camunda deployment topology, its data layer, its scaling model, its monitoring, its security integration — the subject of the manuscript's infrastructure part, applied to this migration. The row is rated *high*, and it carries a quiet cost that the construct rows do not: the *people* cost. The operations team's expertise is WebSphere expertise — years of it — and that expertise does not transfer to a cloud-native Zeebe platform. The team must learn a new operational model, and the migration plan must account for that learning, not assume the people who ran the old platform can run the new one unchanged.

82.3 The configuration contrast in code

“Rebuilt, not ported” is most concrete when the two platforms’ configuration artefacts are placed side by side. They are not two dialects of one thing; they are different kinds of artefact describing different kinds of system.

A traditional BAW deployment configures its resources *inside the WebSphere application server*. A data source — the database connection the Process Server uses — is a WebSphere-administered object, defined in WebSphere’s configuration, as sketched in Listing 82.1.

Listing 82.1. A WebSphere data source — the database connection a BAW Process Server uses — defined as a WebSphere-administered resource.

```
1 <!-- WebSphere Application Server: resources.xml -->
2 <resources.jdbc:DataSource name="BPMDB"
3   jndiName="jdbc/BPMDB"
4   providerType="DB2 Universal JDBC Driver Provider">
5   <connectionPool maxConnections="50"
6     minConnections="10"/>
7   <propertySet>
8     <resourceProperties name="databaseName"
9       value="BPMDB"/>
10    <resourceProperties name="serverName"
11      value="db-host"/>
12  </propertySet>
13 </resources.jdbc:DataSource>
```

This artefact has no Camunda equivalent — not a different file, but no equivalent at all — because Camunda 8’s runtime is not an application server and has no concept of a JNDI-bound, server-administered data source. The cloud-native Camunda runtime is deployed instead through a *Helm chart* onto *Kubernetes*, and its configuration is a *values file* that describes the cluster’s shape. Listing 82.2 is the equivalent concern — “how is the engine cluster sized and stored” — expressed the cloud-native way.

Listing 82.2. The Camunda 8 Self-Managed equivalent concern: a Helm values file describing the Zeebe cluster’s size, partitioning, and persistent storage — a cloud-native artefact, not an app-server resource.

```
1 # Camunda 8 Self-Managed: values.yaml (Helm)
2 zeebe:
3   clusterSize: 3           # three brokers
4   partitionCount: 3
5   replicationFactor: 3
6   pvcSize: 32Gi           # persistent volume per broker
7 zeebe-gateway:
8   replicas: 2
9 elasticsearch:
10  enabled: true            # the runtime data store
```

The two listings describe genuinely different worlds. The WebSphere data source is one resource inside one application server; the Helm values file describes a *cluster* — three Zeebe brokers, deployed by Kubernetes as a *StatefulSet* with a persistent volume each, behind two gateway replicas. The WebSphere file is administered through the WebSphere console; the values file is applied with `helm install` and the cluster is then operated with `kubectl`. Nothing maps element to element, because the scaling model itself differs: WebSphere’s `maxConnections` tunes one server vertically, while `clusterSize` and `partitionCount` scale the engine horizontally across brokers.

The application’s own connection to the engine moves the same way. Where a BAW arte-

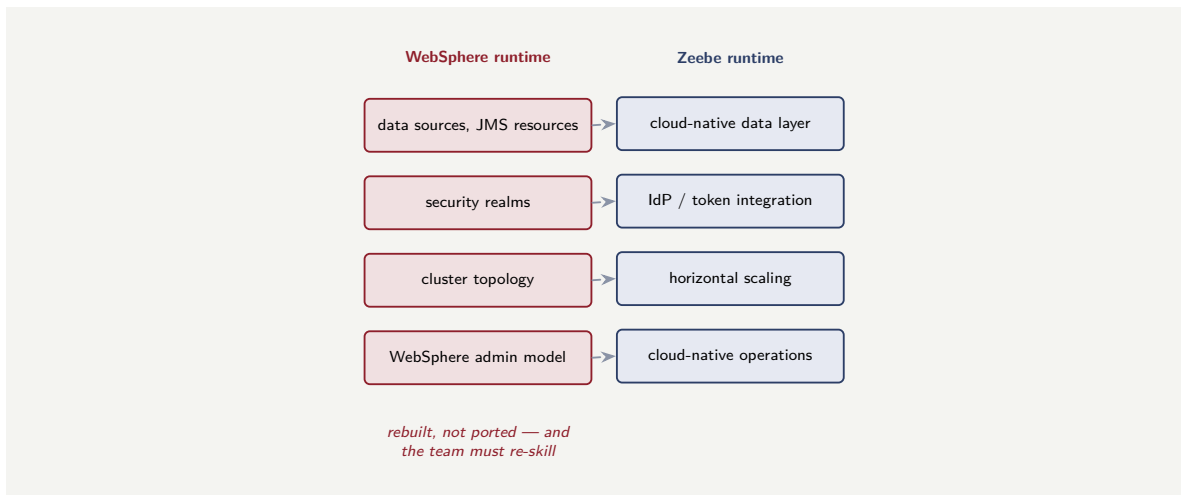


Figure 82.1. The runtime is rebuilt, not ported. Each WebSphere operational concept — data sources, security realms, cluster topology, the admin model — has no port to the cloud-native Zeebe runtime; the equivalent is rebuilt, and the operations team’s WebSphere expertise does not transfer unchanged.

fact bound to jdbc/BPMDB through JNDI, a Camunda job-worker application — a Spring Boot application — is configured with a plain properties file, as in Listing 82.3.

Listing 82.3. A Camunda job-worker application’s configuration: a Spring Boot properties file pointing at the Zeebe gateway and its OAuth credentials — the connection concern, cloud-native.

```

1 # application.yaml -- a Camunda worker application
2 camunda:
3   client:
4     mode: saas      # or self-managed
5   zeebe:
6     grpc-address: https://my-cluster.zeebe:443
7   auth:
8     client-id: ${ZEEBE_CLIENT_ID}
9     client-secret: ${ZEEBE_CLIENT_SECRET}

```

The contrast across the three listings is the row’s whole argument in concrete form. The WebSphere data source, the Helm values file, and the Spring Boot properties file are not three versions of one artefact; they belong to three different operational models — an application-server resource, a Kubernetes cluster definition, and a twelve-factor application config. A migration does not *translate* the first into the others; it *rebuilds* the runtime in the new model, and — the chapter’s quiet point — the operations team must learn to read and operate the new artefacts, because fluency in `resources.xml` is not fluency in Helm and `kubectl`.

Tip

When a BAW migration plan reaches the runtime, ask to see the target’s configuration artefacts concretely — the Helm values file, the Kubernetes manifests, the worker application’s properties. If the plan cannot show them, it has not yet done the runtime row; it has only assumed it. “Rebuilt, not ported” means there are real new artefacts to author and a team that must become fluent in them — both belong in the estimate.

Figure 82.1 sets the WebSphere operational concepts against their rebuilt Zeebe counterparts.

82.4 The hidden cost of a migration

Every migration has a cost the plan does not show: the cost of the *knowledge* that lives in the old platform and in the heads of the people who run it. The old platform's configuration, its workarounds, its undocumented behaviours, its operational playbooks — all of these are knowledge that was bought with years of production experience, and none of it transfers automatically. A migration plan that costs only the artefacts — the process models, the integration code, the UI — and not the operational knowledge is undercosted, and the gap will appear in the months after go-live, when the team discovers behaviours the old platform handled that the new one does not yet, because no one remembered to transfer the knowledge.

Worked example: the operations team that was assumed unchanged

Problem. A BAW migration plan accounts in detail for translating processes, rebuilding coaches, and rewriting services. For the runtime, it says: “the existing operations team will run the new platform.” The team is six WebSphere administrators with, between them, decades of WebSphere experience. Assess the assumption.

Setup. We consider what running the new platform actually requires, against what the team's expertise actually is.

Working. The team's expertise is *WebSphere* expertise: WebSphere consoles, WebSphere clustering, WebSphere data sources and security realms, the WebSphere administration model. The new platform is the cloud-native *Zeebe* runtime — containerised, horizontally scaling, typically on Kubernetes, operated through an entirely different model:

team's skill: WebSphere administration; new platform: cloud-native Zeebe operations.

“The existing team will run the new platform” is true only in the sense that the same six people will be assigned to it. Their *expertise* does not transfer: running a Kubernetes-based cloud-native system is not running a WebSphere installation, and the plan has budgeted zero for the gap.

Discussion. The plan made the construct rows visible and the *people* cost invisible, and the worked example shows that this row's quiet cost is exactly the one most easily assumed away. The runtime row is not only an infrastructure-build task — defining the Camunda deployment, its data layer, its scaling — it is also a *re-skilling* task, because the operations team that will run the platform holds the wrong kind of expertise for it. WebSphere administration and cloud-native Zeebe operations are genuinely different disciplines; the six administrators are capable people, but capability is not the same as currency, and the plan that says “the existing team will run it” has silently assumed a transfer of expertise that does not happen by itself. The honest plan budgets the runtime row twice over: the infrastructure build, and the team's learning — training, ramp-up time, very possibly a period of external cloud-native expertise alongside the team while they re-skill. The lesson generalises the row's defining point: the runtime is rebuilt, not ported, and “rebuilt” applies to the operational *capability* as much as to the configuration — a migration that ports neither the WebSphere config nor, realistically, the WebSphere skills, and must budget for building both anew.

Practical next steps

In a BAW migration plan, treat the runtime as two budget lines, not zero: the infrastructure build of the cloud-native Camunda platform, and the *re-skilling* of the operations team, whose

WebSphere expertise does not transfer to a cloud-native Zeebe runtime. “The existing team will run the new platform” assumes a transfer of expertise that does not happen on its own.

82.5 Migration risk and mitigation

Every migration carries risk, and the risk is managed by acknowledging it explicitly rather than hoping it will not materialise. The principal risks are *functional regression* (the migrated process does not behave as the old one did), *data loss* (in-flight instances or their variables are lost or corrupted), and *performance degradation* (the new platform is slower under the same load). Each is mitigated by a specific discipline: functional regression by parallel running and comparison testing against the old platform; data loss by verifiable, checkpointed migration with rollback capability; performance degradation by load testing against realistic volumes before cutover. A migration plan that does not name its risks and their mitigations is a plan that has not been thought through.

The chapter’s principles interact with the platform’s operational model in a way worth making explicit. The *deployment* of the artefacts this chapter describes must go through a governed pipeline — versioned, reviewed, tested, and traceable. An artefact deployed by a manual action, outside the pipeline, is an artefact whose presence in production cannot be explained, and in a regulated institution an unexplained artefact is a finding.

The *monitoring* of the artefacts must cover both their functional correctness and their operational health. A process step that executes but produces wrong results is harder to detect than one that fails outright, because it does not raise an error — it raises a consequence, and the consequence may not be discovered until a customer complains or a regulator asks a question. The observability model should therefore include not only error rates and latencies but also *outcome monitoring*: checks that the results the process produces are within expected ranges, flagging anomalies for review.

Chapter in brief

A BAW process runs on the *Process Server*, hosted traditionally on *IBM WebSphere Application Server*, so the operational substance of a BAW platform is WebSphere-shaped — data sources, JMS resources, security realms, cluster topology, the WebSphere administration model. The Camunda target is the cloud-native *Zeebe* runtime, and the essential word is that it is *rebuilt*, not *ported*: Zeebe scales horizontally and is operated as a cloud-native, typically containerised system, and no WebSphere configuration concept has a meaningful port. The row is a full infrastructure-and-operations workstream, and it carries a quiet *people* cost — the operations team’s WebSphere expertise does not transfer, and the plan must budget the re-skilling as well as the build.

Migrating In-Flight Process Instances

83.1 The processes that are already running

The ninth and final construct is the one [Chapter 75](#)'s table rated *highest* — and it is the one that is hardest to see, because it is not an artefact in a repository at all. It is the live state of the processes *currently running* on BAW.

At the moment a migration cuts over, a BAW platform is not idle. It has, very often, thousands of process instances *partway through*: loan applications halfway down the approval flow, claims awaiting an assessor, onboarding cases waiting on a document. Each of these in-flight instances is a piece of *live state*: a token position — where in the process the instance currently is; a set of *variables* — the data the instance has accumulated; and the state of its active tasks — a claimed, partly-completed human task, a service call in progress.

These running instances cannot simply be abandoned, and they are not in any export of process definitions. They are the work the institution has in progress, and a migration must decide, deliberately, what happens to them.

83.2 Drain or migrate

There are two ways to deal with the in-flight instances, and the choice — made, as [Chapter 75](#) established, *per process* — is the substance of this row.

To *drain* a process is to stop starting *new* instances of it on BAW, route new work to Camunda, and let the instances already running on BAW *finish there*, on the old platform, in their own time. The old platform is kept alive until its last in-flight instance completes, and then switched off. Draining is the cheaper and lower-risk option — no live state is moved at all — and it is available whenever the process is *short enough*: if the instances will all complete within the time the old platform can be kept running, draining simply works.

To *instance-migrate* is to carry the live state across: to take each running BAW instance and recreate its state — token, variables, task state — in a running Camunda instance, through the migration-plan-and-mapping- instruction mechanics of [Chapter 74](#). Instance migration is necessary when a process is *long-running* — a mortgage, a multi-year policy, a complex claim — because its instances will not drain within any reasonable window, and waiting years for them to finish is not an option. Instance migration is the hardest category of migration work, and the row is rated *highest* for it — not because any one instance is hard, but because correctness must hold across *every* instance, and there may be tens of thousands.

Most institutions have both kinds of process and so do both: they drain the short processes and instance-migrate the long ones, and the migration plan must make that determination process by process rather than adopting one approach for the whole estate.

Figure [83.1](#) sets out the decision that governs this row.

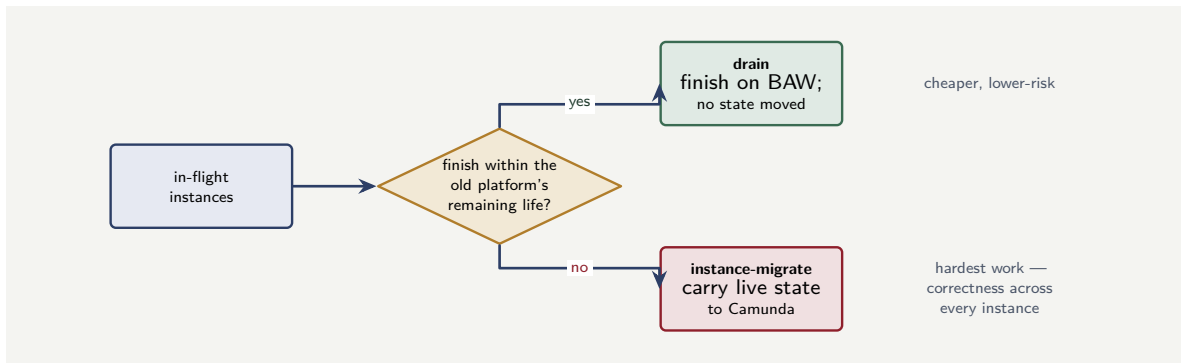


Figure 83.1. The per-process drain-or-migrate decision for in-flight instances. A process whose instances finish within the old platform’s remaining life can be drained — no live state moved; a long-running process must be instance-migrated, its live state carried to Camunda. Most estates need both.

83.3 Instance migration in code

When the decision is to *migrate* an in-flight instance rather than drain it, the carry-across is done through Camunda 8’s process-instance migration API, and it is worth seeing concretely, because the mechanism is specific.

A migration is described by a *migration plan*: it names the target process definition and lists *mapping instructions*, each saying that an instance currently at one activity should, after migration, be at the corresponding activity in the new definition. Listing 83.1 builds such a plan with the Zeebe client and applies it to a running instance.

Listing 83.1. A Camunda 8 process-instance migration: a migration plan of mapping instructions, carrying a running instance from the old process definition to the new one.

```

1 // migrate one running instance to the new definition
2 zeebeClient.newMigrateProcessInstanceCommand(processInstanceKey)
3   .migrationPlan(newProcessDefinitionKey)
4   // map each active element to its new counterpart
5   .addMappingInstruction("review-application",
6     "review-application-v2")
7   .addMappingInstruction("await-credit-check",
8     "await-credit-check-v2")
9   .send()
10  .join();
  
```

Two things in the listing are the heart of doing this correctly. The mapping instructions must cover *every element an instance might currently be waiting at* — a user task, a message catch event, a timer — because an instance parked at an unmapped element has nowhere to go. And the mapping is only valid between elements of the *same type*: a user task maps to a user task, a catch event to a catch event.

Where the old and new processes genuinely differ at the point an instance sits — the new process has no counterpart for where the instance is — mapping alone is not enough, and migration is combined with *process-instance modification*: the instance is moved deliberately, by cancelling the stranded activity and activating the right one in the new process. Listing 83.2 shows that modification.

Listing 83.2. Process-instance modification: for an instance stranded where the new process has no counterpart, the token is moved deliberately — cancel the old activity, activate the new one.

```

1 // move a stranded instance: cancel old, activate new
  
```

```

2 zeebeClient.newModifyProcessInstanceCommand(processInstanceKey)
3   .activateElement("new-verification-step")
4   .and()
5   .terminateElement(strandedElementInstanceKey)
6   .send()
7   .join();

```

For an estate, this is not run instance by instance by hand. A migration is a scripted, batched operation: the running instances of the process are queried, and the migration command is applied across them, in controlled batches, as part of the sequenced cutover. The two listings are the per-instance mechanics; the migration of an estate wraps them in a loop over the instances the cutover must carry.

A process-instance migration plan must have a mapping instruction for *every* element a running instance could currently be waiting at — every user task, every catch event, every timer in the old definition that an instance might be parked on. An instance sitting at an element the plan did not map cannot be migrated: the command fails for that instance. Before migrating an estate, enumerate the wait states of the old process and confirm the plan covers all of them — a plan that maps the elements the team happened to think of will strand the instances sitting everywhere else.

83.4 The hidden cost of a migration

Every migration has a cost the plan does not show: the cost of the *knowledge* that lives in the old platform and in the heads of the people who run it. The old platform's configuration, its workarounds, its undocumented behaviours, its operational playbooks — all of these are knowledge that was bought with years of production experience, and none of it transfers automatically. A migration plan that costs only the artefacts — the process models, the integration code, the UI — and not the operational knowledge is undercosted, and the gap will appear in the months after go-live, when the team discovers behaviours the old platform handled that the new one does not yet, because no one remembered to transfer the knowledge.

Worked example: the estate migrated by one rule

Problem. A bank's BAW estate has two kinds of process: a set of short-lived service requests — typically complete within a week — and a set of long-running products including mortgages, which run for many months. The migration team, wanting a simple plan, proposes a single rule for in-flight instances: instance-migrate everything, so the cutover is clean and the old platform can be switched off at once. Assess this.

Setup. We test the single rule against the two kinds of process the estate actually contains.

Working. Instance migration is the *hardest* category of migration work — carrying live state across, correctly, for every instance. For the *long-running* processes — the mortgages — it is unavoidable: their instances will not drain within any reasonable window, so their live state must be migrated. But for the *short-lived* service requests, which complete within a week, instance migration is unnecessary effort:

short processes: could drain in ~ 1 week;

single rule: instance-migrate them anyway — hardest work, for nothing.

The single rule applies the hardest technique to the entire estate, including the large part of it that could be handled by simply waiting a week.

Discussion. The team optimised for a simple plan and chose the wrong simplification. A single rule *is* simpler to state, but “instance-migrate everything” commits the estate to its most expensive and riskiest technique universally — including for the short-lived processes where draining would achieve the same outcome by keeping BAW alive a single extra week and letting those instances finish themselves. The genuinely simple-*and*-correct plan is the per-process determination [Chapter 75](#) describes: compare each process’s duration against the time the old platform can be kept running, drain the ones that finish inside that window, and reserve the hard work of instance migration for the long-running processes that genuinely need it. That is barely more complex to decide — it is one comparison per process — and it removes the hardest category of work from however much of the estate is short-lived. The opposite single rule, “drain everything,” fails the other way: the mortgages would keep BAW alive for years. The lesson is the row’s defining point: in-flight instances are handled per process, not per estate, because the estate contains processes of very different durations, and the cheap, low-risk option is available for the short ones and the hard option is forced only for the long ones — a plan that picks one rule for all either does the hardest work needlessly or keeps the old platform alive far too long.

Practical next steps

For the in-flight instances of a BAW estate, do not adopt one rule for the whole estate. Determine, process by process, whether the process’s instances will finish within the old platform’s remaining life — drain those, and reserve instance migration for the long-running processes that genuinely need it. One rule for all either applies the hardest technique needlessly or keeps the old platform alive for years.

83.5 Migration risk and mitigation

Every migration carries risk, and the risk is managed by acknowledging it explicitly rather than hoping it will not materialise. The principal risks are *functional regression* (the migrated process does not behave as the old one did), *data loss* (in-flight instances or their variables are lost or corrupted), and *performance degradation* (the new platform is slower under the same load). Each is mitigated by a specific discipline: functional regression by parallel running and comparison testing against the old platform; data loss by verifiable, checkpointed migration with rollback capability; performance degradation by load testing against realistic volumes before cutover. A migration plan that does not name its risks and their mitigations is a plan that has not been thought through.

The chapter’s principles interact with the platform’s operational model in a way worth making explicit. The *deployment* of the artefacts this chapter describes must go through a governed pipeline — versioned, reviewed, tested, and traceable. An artefact deployed by a manual action, outside the pipeline, is an artefact whose presence in production cannot be explained, and in a regulated institution an unexplained artefact is a finding.

The *monitoring* of the artefacts must cover both their functional correctness and their operational health. A process step that executes but produces wrong results is harder to detect than one that fails outright, because it does not raise an error — it raises a consequence, and the consequence may not be discovered until a customer complains or a regulator asks a question. The observability model should therefore include not only error rates and latencies but also

outcome monitoring: checks that the results the process produces are within expected ranges, flagging anomalies for review.

Chapter in brief

The final and highest-effort construct is the live state of the processes *currently running* on BAW — thousands of in-flight instances, each with a token position, accumulated variables, and active-task state, none of it in any export of definitions. There are two ways to handle them, chosen *per process*. To *drain* is to stop new instances, route new work to Camunda, and let the running instances finish on BAW — cheaper and lower-risk, available whenever the process is short enough to complete within the old platform's remaining life. To *instance-migrate* is to carry each instance's live state to Camunda through [Chapter 74](#)'s mapping mechanics — necessary for long-running processes, and the hardest category of migration work, because correctness must hold across every one of tens of thousands of instances. Most estates need both.

CHAPTER 84

Migrating BAW Team Services

84.1 What a Team service is

The nine construct chapters covered the artefacts the construct-mapping table names. Two further BAW constructs deserve their own chapters because a migration regularly meets them and they fit none of the nine rows cleanly. The first is the *Team service*.

In IBM BAW, a *Team* models a group of people who do work — the BAW notion that sits behind task assignment. But a Team is rarely a fixed list. BAW computes team membership dynamically through two service constructs. A *Team Retrieval Service* computes *who is on a team* at runtime — querying a directory, applying organisational rules, resolving a manager’s reports — so the team is determined when the process needs it, not hard-wired at design time. A *Team Filter Service* then *narrows* a team for a particular task — excluding the person who did the previous step, restricting to those with a skill, removing anyone unavailable.

Together these are how BAW does *dynamic, computed task allocation*: the process does not name individuals; it names a Team, and the Team services compute, at runtime, exactly who is eligible. A migration that treats task assignment as a static list will miss this entirely — the logic of *who gets the work* is in the Team services, not in the process diagram.

84.2 The Camunda target: candidate groups and the allocator

Camunda has no construct literally called a Team service, but it has the mechanism the Team services exist to feed: *candidate-group resolution* and the *task listener allocator* of [Chapter 30](#).

The translation has two parts. A BAW *Team Retrieval Service* — “compute who is on the team” — becomes the resolution of a Camunda user task’s *candidate group*, or a FEEL expression and a job worker that computes the candidate set from the same directory or organisational rules. Where BAW computed a team, Camunda computes a candidate group. A BAW *Team Filter Service* — “narrow the team for this task” — becomes part of the *creating task-listener allocator*: the listener that fires when a task is created and corrects the assignee can equally apply the filtering rules — exclude the four-eyes maker, require a skill, drop the unavailable — that the Team Filter Service expressed.

So the Team services do not vanish; their logic moves into the allocation seam Camunda provides. Listing 84.1 sketches a Team Filter Service’s logic re-homed as a creating-listener job worker — the same shape as the allocators of [Chapter 30](#), now carrying the filter rules.

Listing 84.1. A BAW Team Filter Service’s logic re-homed as a Camunda creating-listener job worker: it computes the candidate set, applies the filter rules, and corrects the task.

```
1 @JobWorker(type = "team-filter-listener")
2 public Map<String, Object> filter(ActivatedJob job) {
3     // the BAW Team Retrieval equivalent: compute the team
4     List<String> team = directory.membersOf(
5         job.getVariable("teamName"));
6 }
```

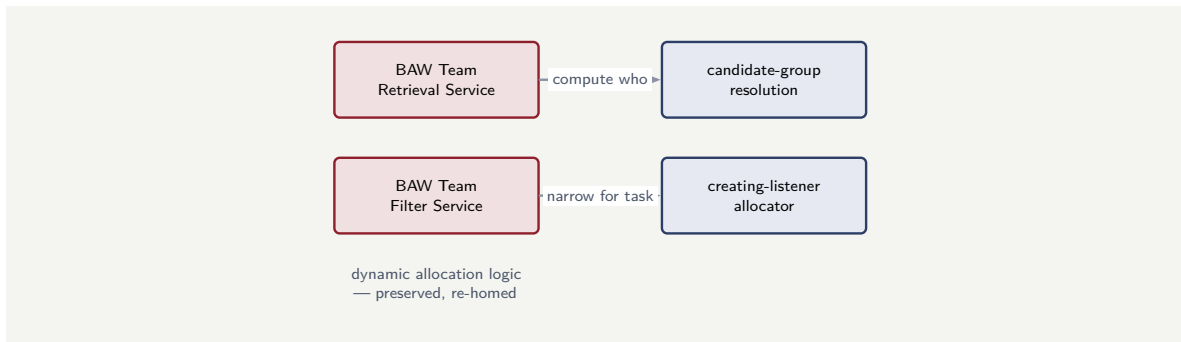


Figure 84.1. Migrating BAW Team services. A Team Retrieval Service — “compute who is on the team” — becomes Camunda candidate-group resolution; a Team Filter Service — “narrow the team for this task” — becomes part of the creating task-listener allocator. The dynamic-allocation logic is preserved, not lost.

```

7 // the BAW Team Filter equivalent: narrow it
8 String maker = job.getVariable("makerUserId");
9 List<String> eligible = team.stream()
10     .filter(u -> !u.equals(maker)) // four-eyes
11     .filter(directory::isAvailable) // on shift
12     .toList();
13
14 // correct the task's candidate users
15 return Map.of("candidateUsers", eligible);
16 }

```

Figure 84.1 maps the two Team service kinds onto their Camunda homes.

84.3 The hidden cost of a migration

Every migration has a cost the plan does not show: the cost of the *knowledge* that lives in the old platform and in the heads of the people who run it. The old platform’s configuration, its workarounds, its undocumented behaviours, its operational playbooks — all of these are knowledge that was bought with years of production experience, and none of it transfers automatically. A migration plan that costs only the artefacts — the process models, the integration code, the UI — and not the operational knowledge is undercosted, and the gap will appear in the months after go-live, when the team discovers behaviours the old platform handled that the new one does not yet, because no one remembered to transfer the knowledge.

Worked example: the assignment logic that was not in the diagram

Problem. A bank migrates a BAW approval process. The team translates the BPMN diagram, rebuilds the coach, and rewrites the integration services. At test, the approvals all land in one unfiltered pool: the rule that an approval must not go to the person who raised the request, and must go only to approvers in the right region, has been lost. The team is puzzled — they migrated everything on the diagram. Assess what happened.

Setup. We ask where the missing assignment rules lived in the BAW process.

Working. The rules — exclude the requester, restrict by region — are *allocation* logic, and in BAW allocation logic does not live on the process diagram; it lives in the *Team services*. The process named a Team; a Team Retrieval Service computed the regional approver group, and

a Team Filter Service excluded the requester:

diagram migrated + Team services not migrated = approvals in one unfiltered pool.

The team migrated every artefact that was *visible on the diagram* and none that was not — and the Team services, like coaches, are invisible there.

Discussion. The puzzle resolves the moment one sees that “everything on the diagram” is not everything in the process. BAW deliberately keeps the *who-gets-the-work* logic out of the diagram and in the Team services, so a process diagram shows a task assigned to “a Team” and reveals nothing about how that team is computed or filtered. The team’s translation was complete for the artefacts it could see and empty for the Team services it could not, and the symptom — approvals in one unfiltered pool — is exactly what remains when the candidate-set computation is dropped. The fix is to recognise the Team services as a migration construct in their own right: inventory them, and re-home each one — Team Retrieval into candidate-group resolution, Team Filter into the creating-listener allocator of [Chapter 30](#), exactly as [Listing 84.1](#) shows. The lesson generalises the coach lesson of [Chapter 77](#): a BAW process keeps several kinds of essential logic off the diagram — the UI in coaches, the allocation in Team services — and a migration scoped from the diagram will, predictably, lose exactly those, and discover the loss only when the behaviour is visibly wrong in test.

Practical next steps

When migrating a BAW estate, inventory the Team services as a construct in their own right. For each, identify whether it is a Team Retrieval Service — re-homed as candidate-group resolution — or a Team Filter Service — re-homed into the creating task-listener allocator. The logic of who gets the work lives here, not on the process diagram.

84.4 Migration risk and mitigation

Every migration carries risk, and the risk is managed by acknowledging it explicitly rather than hoping it will not materialise. The principal risks are *functional regression* (the migrated process does not behave as the old one did), *data loss* (in-flight instances or their variables are lost or corrupted), and *performance degradation* (the new platform is slower under the same load). Each is mitigated by a specific discipline: functional regression by parallel running and comparison testing against the old platform; data loss by verifiable, checkpointed migration with rollback capability; performance degradation by load testing against realistic volumes before cutover. A migration plan that does not name its risks and their mitigations is a plan that has not been thought through.

The chapter’s principles interact with the platform’s operational model in a way worth making explicit. The *deployment* of the artefacts this chapter describes must go through a governed pipeline — versioned, reviewed, tested, and traceable. An artefact deployed by a manual action, outside the pipeline, is an artefact whose presence in production cannot be explained, and in a regulated institution an unexplained artefact is a finding.

The *monitoring* of the artefacts must cover both their functional correctness and their operational health. A process step that executes but produces wrong results is harder to detect than one that fails outright, because it does not raise an error — it raises a consequence, and the consequence may not be discovered until a customer complains or a regulator asks a question. The observability model should therefore include not only error rates and latencies but also *outcome monitoring*: checks that the results the process produces are within expected ranges, flagging anomalies for review.

Chapter in brief

In IBM BAW a *Team* models a group of people, but team membership is rarely fixed: a *Team Retrieval Service* computes who is on a team at runtime, and a *Team Filter Service* narrows a team for a particular task — excluding the four-eyes maker, requiring a skill, dropping the unavailable. Together they are how BAW does dynamic, computed task allocation, and crucially this logic lives off the process diagram. Camunda has no Team service construct, but it has the mechanism they feed: a Team Retrieval Service re-homes as *candidate-group resolution*, and a Team Filter Service re-homes into the *creating task-listener allocator*. The logic is preserved, not lost — but only if the migration recognises the Team services as a construct to inventory, since a diagram-scoped migration cannot see them.

Migrating ECM Integration

85.1 Why BAW reaches into a content system

The second further construct is the BAW estate's integration with an *enterprise content management* system — and it deserves its own chapter because content integration is pervasive in banking and insurance processes and because it migrates by a path of its own.

A great many BAW processes do not only move data; they move *documents*. An account-opening process collects identity documents; a lending process gathers pay slips, statements, and a signed agreement; a claim handles photographs, reports, and correspondence. In a BAW estate these documents do not live in the process — they live in an *ECM* system, most often *IBM FileNet* (the Content Platform Engine), and the BAW process *integrates* with it: storing a document, retrieving one, advancing its lifecycle, reading its properties.

BAW provides this through its content integration — an ECM toolkit and content-aware coach controls — frequently speaking the *CMIS* standard (Content Management Interoperability Services) to the repository. The crucial point for a migration is that the documents, and the integration with the system that holds them, are a distinct concern: not a process artefact, not an ordinary integration service, but a content integration with its own shape.

85.2 The two halves of an ECM migration

Migrating ECM integration separates cleanly into two halves, and conflating them is the common mistake.

The *first half* is the *repository*. The ECM system itself — the FileNet content engine, the documents it holds, its folders and metadata — is a system of record, and like a core banking system it usually *stays*. A process migration is not a content migration: the disciplined default is to *leave the documents where they are*, in the existing repository, and have the new Camunda process integrate with it. Moving the content itself is a separate, large, and often unnecessary project — and [Part XVI](#) treats ECM integration as an architecture in its own right.

The *second half* is the *integration* — and this is what the migration actually rebuilds. Every point where the BAW process called the content system — store this document, fetch that one, read these properties, move it through its lifecycle — becomes a Camunda integration: a connector or a job worker, exactly as an integration service does in [Chapter 78](#), but speaking to the content repository. Where the BAW process spoke CMIS through its ECM toolkit, the Camunda process speaks CMIS — or the repository's API — through a connector or worker. [Listing 85.1](#) sketches a content-store call re-homed as a job worker.

Listing 85.1. A BAW ECM content-store operation re-homed as a Camunda job worker speaking to the content repository — the documents stay in the repository; the integration is rebuilt.

```

1 @JobWorker(type = "store-document")
2 public Map<String, Object> store(ActivatedJob job) {
3     byte[] content = job.getVariable("documentBytes");
4     String caseRef = job.getVariable("caseReference");
5 }

```



```

6 // the repository stays; only the call is rebuilt
7 ContentId id = contentRepository.store(
8     content,
9     Map.of("caseReference", caseRef,
10         "docType", job.getVariable("docType")));
11
12 // the process carries only the reference
13 return Map.of("documentId", id.value());
14 }

```

One detail in the listing is the heart of doing this well: the worker returns only the *documentId* — a reference — into the process. The Camunda process variable carries a pointer to the document in the repository, not the document itself. This is the same discipline a well-built BAW process followed, and it is treated in full in [Part XVI](#); a migration must preserve it, because a process that pulls whole documents into its variables inflates its state and its audit store with content that belongs in the ECM system.

85.3 The migration in code: before and after

The integration rebuild is clearest seen as the BAW artefact and its Camunda replacement side by side. This section follows three ECM operations — store, fetch, and a property update — from one to the other.

In BAW, a content operation is a step in the process application that calls the ECM toolkit. Listing 85.2 sketches a BAW content-store operation as the BAW tooling expresses it: a service that hands content and metadata to the FileNet integration and receives an identifier back.

Listing 85.2. A BAW content-store operation, as the BAW ECM toolkit expresses it: a service activity bound to the FileNet integration.

```

1 <!-- BAW process application: ECM toolkit service -->
2 <serviceTask id="filremDoc"
3     name="Store Document"
4     calledService="ECM.CreateDocument"
5     toolkit="ContentIntegration">
6     <input name="content" variable="tw.local.docBytes"/>
7     <input name="folder" variable="tw.local.caseFolder"/>
8     <output name="objectId" variable="tw.local.docId"/>
9 </serviceTask>

```

The Camunda replacement is the job worker already shown in Listing 85.1. The pairing is the point: the BAW service's *intent* — store content, get an identifier — is preserved, and its *implementation* is rebuilt against the repository's API. The BAW toolkit binding is gone; a Camunda job worker takes its place.

The *fetch* operation migrates the same way. Where BAW retrieved content through the toolkit, the Camunda process carries the *documentId* and a worker fetches the bytes only at the step that needs them — Listing 85.3.

Listing 85.3. The fetch operation as a Camunda job worker. The process carries the *documentId* reference; the worker resolves it to content only when a step needs the bytes.

```

1 @JobWorker(type = "fetch-document")
2 public Map<String, Object> fetch(ActivatedJob job) {
3     String documentId = job.getVariable("documentId");
4
5     // resolve the reference to content, at this step
6     Document doc = contentRepository.getById(documentId);
7
8     return Map.of(

```

```

9      "contentBytes", doc.getContentStream(),
10     "mimeType",     doc.getProperty("mimeType"));
11 }

```

Many content operations do not move the document at all — they read or update its *meta-data*. A BAW step that set a document's status becomes a Camunda worker that updates the property through CMIS or the repository's API, carrying no content, as in Listing 102.3.

Listing 85.4. A property-update worker. Many ECM operations touch only metadata — here a document's status — and move no content at all.

```

1  @JobWorker(type = "update-document-status")
2  public void updateStatus(ActivatedJob job) {
3      String documentId = job.getVariable("documentId");
4      String newStatus = job.getVariable("status");
5
6      // a metadata update -- no content transferred
7      contentRepository.updateProperties(
8          documentId,
9          Map.of("status", newStatus));
10 }

```

For repositories reached over the CMIS standard — the portable interface **Part XVI** describes — the worker's repository client is a CMIS client, and the same worker code serves any CMIS-compliant repository. Listing 85.5 shows the connection the workers share: a CMIS session, configured once, against which store, fetch, and property operations all run.

Listing 85.5. A CMIS session the ECM workers share. Because CMIS is a standard, the same integration code serves FileNet, OpenText, or any CMIS-compliant repository.

```

1  // a CMIS session -- configured once, shared by workers
2  Map<String, String> params = new HashMap<>();
3  params.put(SessionParameter.ATOM_PUB_URL,
4      ecmConfig.getCmisUrl());
5  params.put(SessionParameter.USER, ecmConfig.getUser());
6  params.put(SessionParameter.PASSWORD, ecmConfig.getSecret());
7  params.put(SessionParameter.BINDING_TYPE,
8      BindingType.ATOMPUB.value());
9
10 Session session = SessionFactory
11     .createSession(params);

```

The four listings together are the ECM migration in miniature: the BAW toolkit service becomes a job worker; store, fetch, and property operations are each re-homed; and a CMIS session gives the workers a standard, portable connection to the repository — which stays exactly where it was.

Tip

When migrating ECM integration, build the Camunda workers against *CMIS* where the repository supports it. CMIS is a standard, so a CMIS-based worker is not bound to FileNet — it serves any CMIS-compliant repository, which protects the migration's integration code against a future change of content platform. Reach for the repository's proprietary API only where CMIS genuinely cannot express what the process needs.

Figure 85.1 draws the repository-stays, integration-rebuilt split.

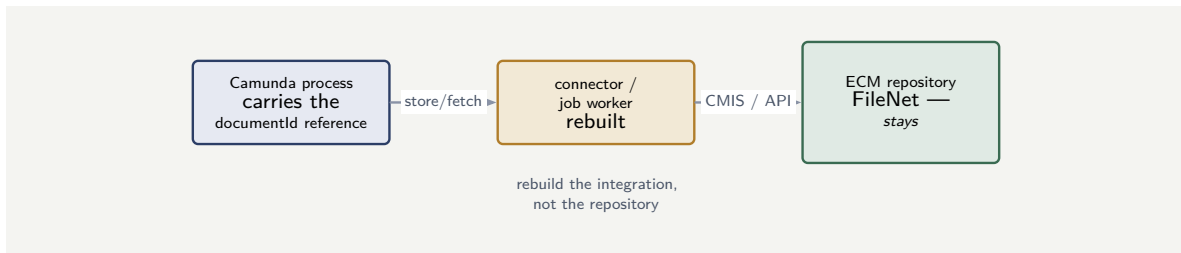


Figure 85.1. The two halves of an ECM migration. The content repository — the FileNet engine and its documents — stays, like any system of record; only the integration is rebuilt, as a connector or job worker. The Camunda process carries a document reference, never the document itself.

85.4 The hidden cost of a migration

Every migration has a cost the plan does not show: the cost of the *knowledge* that lives in the old platform and in the heads of the people who run it. The old platform’s configuration, its workarounds, its undocumented behaviours, its operational playbooks — all of these are knowledge that was bought with years of production experience, and none of it transfers automatically. A migration plan that costs only the artefacts — the process models, the integration code, the UI — and not the operational knowledge is undercosted, and the gap will appear in the months after go-live, when the team discovers behaviours the old platform handled that the new one does not yet, because no one remembered to transfer the knowledge.

Worked example: the migration that planned to move the documents

Problem. A bank’s BAW estate integrates with a FileNet repository holding eight years of account, lending, and claims documents — tens of millions of files. The migration plan, aiming for “a clean break from IBM,” proposes to migrate the documents themselves out of FileNet into a new content store as part of the Camunda migration. Assess this.

Setup. We separate the two halves of an ECM migration and ask which the plan has actually taken on.

Working. An ECM migration has two halves: the *integration*, which the process migration genuinely must rebuild, and the *repository*, which — like a core banking system — is a system of record and normally stays. The plan has bundled the second into the first:

process migration + content migration of tens of millions of files = two projects, costed as one.

Moving eight years of documents is a large content-migration project in its own right — with its own integrity, metadata, retention, and regulatory-evidence concerns — and it is *not required* for the process migration: a Camunda process can integrate with the existing FileNet repository exactly as the BAW process did.

Discussion. The plan let an ambition — “a clean break from IBM” — override the architecture, and the worked example shows why the two halves must be kept apart. The process migration’s real ECM work is the *integration*: every content call rebuilt as a connector or job worker, as Listing 85.1 shows. The *repository* is a different matter entirely; FileNet holding eight years of regulated documents is a system of record, and a system of record is left in place and integrated with, not swept up into a process migration — exactly as a core banking system is in Part XVI. Bundling a tens-of-millions-of-files content migration into the Camunda project

multiplies its size and risk for a goal — removing FileNet — that is not a process-migration goal at all. If the bank genuinely wants to retire FileNet, that is a deliberate, separately-scoped content-migration programme, decided on its own merits and its own timeline, not a rider on the workflow migration. The lesson is the chapter's central split: migrate the ECM *integration*, leave the ECM *repository*, and treat any decision to move the content itself as the separate major project it is.

Practical next steps

For a BAW estate that integrates with an ECM system, scope the two halves separately. The *integration* — every content store, fetch, and property call — is rebuilt as Camunda connectors or job workers, and belongs in the migration. The *repository* and its documents are a system of record; leave them in place and integrate with them. Treat any move of the content itself as a separate, deliberately-scoped project.

85.5 Migration risk and mitigation

Every migration carries risk, and the risk is managed by acknowledging it explicitly rather than hoping it will not materialise. The principal risks are *functional regression* (the migrated process does not behave as the old one did), *data loss* (in-flight instances or their variables are lost or corrupted), and *performance degradation* (the new platform is slower under the same load). Each is mitigated by a specific discipline: functional regression by parallel running and comparison testing against the old platform; data loss by verifiable, checkpointed migration with rollback capability; performance degradation by load testing against realistic volumes before cutover. A migration plan that does not name its risks and their mitigations is a plan that has not been thought through.

The chapter's principles interact with the platform's operational model in a way worth making explicit. The *deployment* of the artefacts this chapter describes must go through a governed pipeline — versioned, reviewed, tested, and traceable. An artefact deployed by a manual action, outside the pipeline, is an artefact whose presence in production cannot be explained, and in a regulated institution an unexplained artefact is a finding.

The *monitoring* of the artefacts must cover both their functional correctness and their operational health. A process step that executes but produces wrong results is harder to detect than one that fails outright, because it does not raise an error — it raises a consequence, and the consequence may not be discovered until a customer complains or a regulator asks a question. The observability model should therefore include not only error rates and latencies but also *outcome monitoring*: checks that the results the process produces are within expected ranges, flagging anomalies for review.

Chapter in brief

Many BAW processes move *documents*, not just data, and those documents live in an ECM system — most often IBM FileNet — which the process integrates with, frequently over the CMIS standard. Migrating ECM integration separates into two halves. The *repository* — the content engine and its documents — is a system of record and normally *stays*, exactly as a core banking system does; moving the content is a separate major project, not part of the process migration. The *integration* is what the migration

rebuilds: every content store, fetch, and property call becomes a Camunda connector or job worker, with the process carrying a document *reference*, never the document itself. Migrate the integration, leave the repository — and Part XVI treats ECM integration as an architecture in its own right.

Migrating from webMethods and Oracle

86.1 When the source is an integration platform

The previous chapter migrated from a dedicated BPM suite. This chapter treats two migrations with a different character, because their source platforms have a different nature: *Software AG's webMethods* and *Oracle's BPM*, part of the Oracle Fusion Middleware family. Both are widely embedded in financial institutions, and both differ from IBM BAW in a way that shapes the migration: they are, at heart, *integration platforms* that also do workflow, rather than workflow platforms first.

86.2 Migrating from webMethods

Software AG's webMethods is, in its origins and its centre of gravity, an *integration platform* — an enterprise service bus and integration suite — that also provides business-process and workflow capability. An institution running processes on webMethods is typically running them *intertwined with* a great deal of integration logic: the process flow and the system-to-system plumbing are woven together in the same platform.

This shapes the migration. The challenge is less about translating a clean process model — there may not be one — and more about *separating* the genuine business-process flow from the integration logic it is tangled with. The migration to Camunda becomes an exercise in disentanglement: identifying what is genuinely the *process* — the orchestration, the human steps, the business sequence, which belongs in Camunda — and what is genuinely *integration* — the message routing, the protocol mediation, the transformation, which belongs in an integration layer Camunda *calls*, exactly the architecture of [Chapter 25](#).

A webMethods migration is therefore as much an *architecture* clarification as a move: it forces the institution to draw, often for the first time clearly, the line between its processes and its integration. Done well, the result is better than a faithful reproduction would have been — the process emerges as an explicit, governed Camunda model, and the integration sits cleanly behind it. Done badly — by trying to reproduce the tangled whole in Camunda — the migration carries the old entanglement into the new platform.

86.3 Migrating from Oracle

Oracle's BPM, within the Oracle Fusion Middleware stack, carries a different shaping characteristic: *deep integration with the surrounding Oracle ecosystem* — the database, the SOA suite, the identity stack, the broader Fusion Middleware. An institution's Oracle BPM processes are typically bound tightly to that ecosystem.

The migration challenge is correspondingly about *dependency*. The process logic itself may translate in the ordinary way, but the process's many points of contact with the Oracle ecosys-

tem must each be re-examined: a step that relied on tight Oracle-stack coupling becomes, on Camunda, an explicit integration — a connector, a job worker — to that Oracle system, now treated as an external system of record like any other. The migration loosens a tight coupling into an explicit, visible integration, which is architecturally healthier but must be done deliberately, dependency by dependency.

As with webMethods, the disciplined migration treats this as a clarification. The Oracle systems do not have to be removed — the database, the systems of record can remain — but the *process* stops being entangled in the Oracle stack and becomes an explicit Camunda model that *integrates with* the Oracle systems through clean, declared interfaces.

86.4 The specific constructs each migration must translate

Disentanglement is the strategy; to estimate the work, an architect needs the specific technical constructs each source platform presents.

webMethods, concretely. A webMethods estate is built largely in *Flow* — webMethods' proprietary graphical service language — and in *Flow services* and *Java services* running on the *Integration Server*. The process and the integration are both expressed in these same artefacts, which is exactly why they are tangled. A migration must sort the Flow-service estate into two piles: the services that are genuinely *process orchestration* — the business sequence — which become the Camunda BPMN model; and the services that are genuinely *integration* — adapter calls, document mapping, the pub-sub messaging over the *Universal Messaging* broker, protocol mediation — which become a distinct integration layer. webMethods *adapters* (the connectors to SAP, databases, and so on) become Camunda connectors or job workers, or are kept as an integration tier the Camunda process calls. There is no automated path for this; the sorting is analytical work, service by service.

Oracle BPM, concretely. Oracle BPM is part of Oracle SOA Suite, and this gives the migration a precise shape. An Oracle BPM project deploys as an *SOA composite application*, and that composite typically bundles several *service components*: BPMN processes, BPEL processes, *human tasks*, *business rules* (decision services), and the *Oracle Mediator* for routing and transformation — all running on the Oracle BPM engine, which itself contains separate BPMN and BPEL engines over a shared process core, on Oracle WebLogic Server. A migration must decompose the composite: the BPMN and BPEL processes become Camunda BPMN — with the BPEL parts following [Part XIII](#)'s approach; the *human task* components, presented through the *Oracle BPM Worklist*, become Camunda user tasks surfaced through Tasklist or a custom application; the *business rules* become Camunda DMN of [Part I](#); and the *Oracle Mediator* routing and transformation moves into the integration layer. The single Oracle composite, in other words, fans out into several distinctly-typed Camunda artefacts, and the migration plan must map that fan-out component by component.

Figure [86.1](#) shows the Oracle composite fanning out into Camunda artefacts.

Figure [86.2](#) draws the disentanglement that a webMethods or Oracle migration really performs.

86.5 Evaluation criteria for a regulated institution

A financial institution evaluating workflow platforms should weight its criteria differently from a technology startup. The criteria that matter most are *auditability* (can the platform

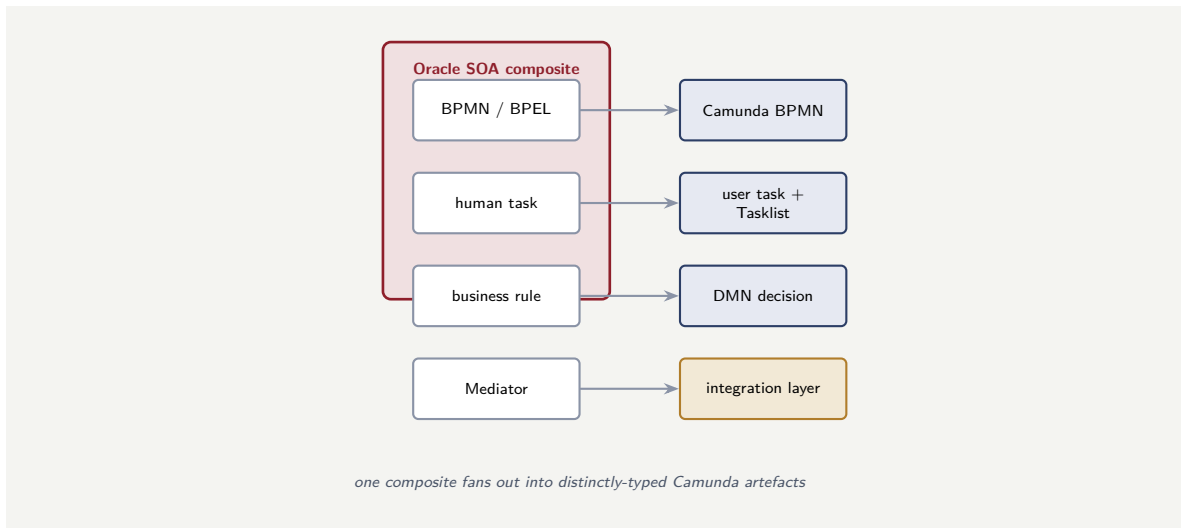


Figure 86.1. Decomposing an Oracle SOA composite. A single Oracle BPM project bundles several service components — BPMN/BPEL processes, human tasks, business rules, the Mediator — and a migration fans them out into the correspondingly distinct Camunda artefacts: BPMN models, user tasks with Tasklist, DMN decisions, and an integration layer.

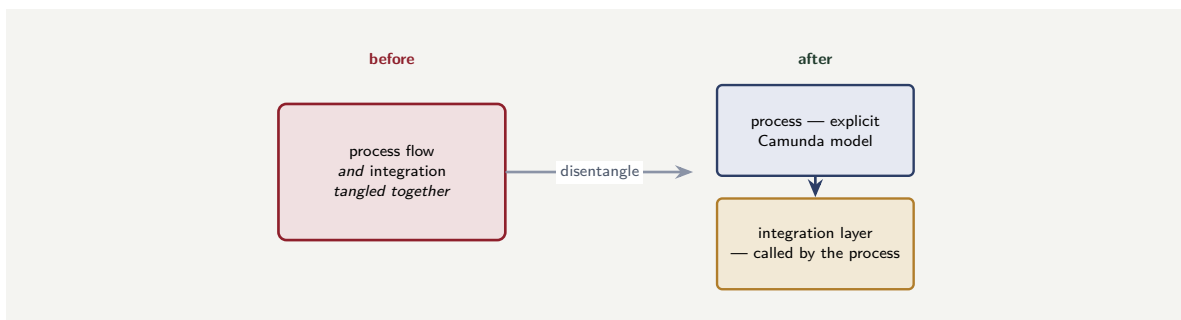


Figure 86.2. Migrating from an integration-centred platform. The migration’s real work is disentanglement: separating the genuine business-process flow — which becomes an explicit Camunda model — from the integration logic it was tangled with, which becomes a layer the process cleanly calls.

produce, for any past case, a complete record of what happened?), *human-task support* (does the platform have a first-class worklist, task allocation, and escalation model?), and *regulatory longevity* (will the platform be supported for the decade-long horizon a regulated institution plans on?). Raw throughput, developer experience, and time-to-first-demo matter, but they are secondary to these three. A platform chosen for its demo speed but lacking auditability will fail its first regulatory examination.

Worked example: reproducing the tangle, or clarifying it

Problem. A bank migrating from webMethods has a payment-processing process in which, on the old platform, the business flow (validate, authorise, post, notify) and the integration logic (routing messages between six systems, transforming formats, mediating protocols) are thoroughly intertwined — there is no clean separation in the source. Two migration teams propose different approaches. Team X will reproduce the whole thing in Camunda as faithfully as possible, tangle and all, to minimise change. Team Y will first separate the process

flow from the integration, putting only the flow in Camunda and the integration logic in a separate integration layer the Camunda process calls. Assess the two approaches.

Setup. We consider what each approach produces on Camunda and what it means for the future.

Working. *Team X — reproduce the tangle.* The migration is, in the short term, more mechanical: copy the combined behaviour across. But the result is a Camunda process model with the integration logic still embedded inside it — message routing, format transformation, protocol mediation all sitting in the process model:

a Camunda model that is part process, part plumbing — unreadable as a process.

The leaked-orchestration and shadow-logic problems of [Chapter 14](#) are reproduced intact on the new platform. *Team Y — disentangle.* The migration does more analytical work up front — separating flow from integration — but produces a Camunda model that is *purely the payment process* — validate, authorise, post, notify — calling a separate integration layer:

a clean, readable, governable process model + a clean integration layer.

Discussion. Team Y is right, and the worked example captures the defining choice of an integration-platform migration. Team X's instinct — reproduce faithfully, minimise change — sounds prudent and is in fact the costly path: it carries the old platform's worst property, the entanglement of process and integration, onto the new platform, so the bank spends a migration budget and ends up with a Camunda model that is not really a process model at all but a tangle wearing BPMN. Every future reader of that model, every future change to it, pays for the tangle that was migrated rather than resolved. Team Y treats the migration as what it should be — an opportunity to draw the line the old platform never drew — and does the analytical work to separate the genuine process from the integration plumbing. The result is the architecture of [Chapter 25](#): an explicit, readable process model that *calls* a clean integration layer. The lesson is the chapter's central one: migrating from an integration-centred platform is not a copying exercise, it is a disentanglement, and the migration that tries hardest to “change nothing” reproduces precisely the entanglement worth leaving behind. A migration is the rare moment when an institution can clarify its architecture; reproducing the tangle wastes that moment.

Practical next steps

If your institution is migrating from webMethods, Oracle BPM, or any integration-centred platform, establish early that the migration's real task is disentanglement — separating genuine process flow from integration logic — not faithful reproduction. Resist the “change nothing” instinct: it carries the old entanglement onto the new platform. The migration is the moment to clarify the architecture, and that moment does not come again cheaply.

86.6 Migration risk and mitigation

Every migration carries risk, and the risk is managed by acknowledging it explicitly rather than hoping it will not materialise. The principal risks are *functional regression* (the migrated process does not behave as the old one did), *data loss* (in-flight instances or their variables are lost or corrupted), and *performance degradation* (the new platform is slower under the same load). Each is mitigated by a specific discipline: functional regression by parallel running and comparison testing against the old platform; data loss by verifiable, checkpointed migration

with rollback capability; performance degradation by load testing against realistic volumes before cutover. A migration plan that does not name its risks and their mitigations is a plan that has not been thought through.

The chapter's principles interact with the platform's operational model in a way worth making explicit. The *deployment* of the artefacts this chapter describes must go through a governed pipeline — versioned, reviewed, tested, and traceable. An artefact deployed by a manual action, outside the pipeline, is an artefact whose presence in production cannot be explained, and in a regulated institution an unexplained artefact is a finding.

The *monitoring* of the artefacts must cover both their functional correctness and their operational health. A process step that executes but produces wrong results is harder to detect than one that fails outright, because it does not raise an error — it raises a consequence, and the consequence may not be discovered until a customer complains or a regulator asks a question. The observability model should therefore include not only error rates and latencies but also *outcome monitoring*: checks that the results the process produces are within expected ranges, flagging anomalies for review.

Chapter in brief

Migrating from webMethods or Oracle BPM has a different character from a dedicated-BPM-suite migration, because both source platforms are integration-centred. A webMethods estate has the business-process flow intertwined with extensive integration logic — built in the proprietary *Flow* language as Flow and Java services on the Integration Server — and the migration's real work is *disentanglement*: sorting the services into the genuine process, which becomes a Camunda BPMN model, and the integration, which becomes a separate layer. An Oracle BPM estate deploys as *SOA composite applications* bundling several service components — BPMN and BPEL processes, human tasks, business rules, the Oracle Mediator — and a migration fans that single composite out into correspondingly distinct Camunda artefacts: BPMN models, user tasks with Tasklist, DMN decisions, and an integration layer. In both cases the migration is an architecture clarification, and the “change nothing” instinct is the trap — it reproduces the old entanglement on the new platform.

Migrating from jBPM and Closing the Migration

87.1 Migrating from jBPM

The final specific migration this part treats is from *jBPM* — the open-source BPM project long associated with the JBoss and Red Hat ecosystem. A jBPM migration has a different character again from the proprietary-suite and integration-platform migrations, and the difference is mostly favourable.

jBPM is, like Camunda's classic engine and Flowable, a standards-based, Java-centred, open-source BPM engine: it implements BPMN, it is embeddable in Java applications, and an institution running jBPM is typically running it close to its own code. Because jBPM and Camunda share the open, standards-based, Java-centred worldview, the conceptual distance is smaller than in an IBM BAW or webMethods migration — the process definitions are genuine BPMN, the team's mental model already fits an open Java engine, and there is no proprietary suite's worth of vendor-specific tooling to escape.

The work that remains is real but more contained. jBPM's process definitions still need translation — BPMN portability between any two engines is never perfect, and jBPM, like every engine, has its own extensions and execution nuances. jBPM's distinctive close integration with the *Drools* rules engine means that an institution's jBPM processes often lean on Drools-based business rules, and the migration must decide how those rules are handled on Camunda — re-expressed as DMN, where they fit DMN's model, or kept as an external rules service the Camunda process calls. And the step implementation and in-flight instances follow the previous chapters' patterns. But overall, a jBPM-to-Camunda migration is, of the four this part treats, usually the least disruptive — it is a move between two members of the same open, standards-based family.

Tip

A jBPM-to-Camunda migration is usually the least disruptive of the common migrations, because jBPM and Camunda share an open, standards-based, Java-centred heritage — the process definitions are genuine BPMN and the team's mental model already fits. The distinctive item to plan for is jBPM's close coupling to the Drools rules engine: decide deliberately whether each body of Drools rules becomes DMN on Camunda or stays as an external rules service the process calls.

87.2 What every migration shares

Across the four specific migrations — IBM BAW, webMethods, Oracle, jBPM — and the anatomy chapter that opened the part, a set of common lessons emerges, and gathering them is the right way to close.

A migration is a translation, not a transfer. No two BPM platforms are close enough that processes simply move. Every migration — even the gentlest, from jBPM — requires genuine translation of definitions, rebuilding of step implementation, and deliberate handling of what does not have a clean equivalent. Budget for translation.

The in-flight instances are the hardest part. In every migration, the live cases running on the old platform at cutover are the most underestimated problem. Decide, per process, whether to drain them or migrate them, and budget the answer.

A migration is an architecture opportunity. Every migration is a chance to leave behind what was wrong with the old platform — the entanglement, the leaked orchestration, the proprietary lock-in — and arrive at the clean architecture this manuscript has argued for. The instinct to “change nothing” wastes that chance and carries the old problems forward.

The people migrate too. A migration that moves every process perfectly and does not move the institution’s skills, operational practices, and mental model has not finished. The team that operated the old platform must become a team that can build and run Camunda — and that transition is part of the migration, to be planned and resourced, not assumed.

The most dangerous migration is the one declared finished when the last process definition is moved. A migration is not complete until the in-flight instances are resolved, the step implementations genuinely work in production, the surrounding assets are rebuilt, *and* the institution’s people can build and operate the new platform without the old one’s crutches. A migration that moves the processes but not the people has produced a Camunda estate the institution cannot actually run.

87.3 The hidden cost of a migration

Every migration has a cost the plan does not show: the cost of the *knowledge* that lives in the old platform and in the heads of the people who run it. The old platform’s configuration, its workarounds, its undocumented behaviours, its operational playbooks — all of these are knowledge that was bought with years of production experience, and none of it transfers automatically. A migration plan that costs only the artefacts — the process models, the integration code, the UI — and not the operational knowledge is undercosted, and the gap will appear in the months after go-live, when the team discovers behaviours the old platform handled that the new one does not yet, because no one remembered to transfer the knowledge.

Worked example: the migration declared finished too early

Problem. A bank completes what it calls a successful jBPM-to-Camunda migration: all 30 process definitions are translated, deployed, and running on Camunda, and the project is declared complete and closed. Three months later, two problems are evident. First, the operations team — who knew jBPM well — are struggling: incidents on Camunda take far longer to resolve than they used to, because the team does not yet know Operate, the worklist tooling, or Camunda’s operational model. Second, several processes are running but performing poorly, because their step implementations were translated literally from jBPM idioms rather than rebuilt for Camunda. Assess what the “successful” migration actually omitted.

Setup. We hold the declared-complete migration against the five things a migration moves.

Working. The migration moved one of the five things well — the process *definitions*, all 30 translated and deployed. Of the other four: the *step implementation* was translated literally

rather than rebuilt, so it runs but runs poorly; the *people* were not migrated at all — the operations team’s skills still fit jBPM, not Camunda. So the migration that was declared complete had genuinely finished:

1 of the 5 things a migration moves,

and the project closed with the definitions moved and the implementation, the operational capability, and the people’s skills all unfinished.

Discussion. The bank declared victory at the most visible milestone — all the process definitions are on Camunda — and the worked example shows why that milestone is not the finish line. Definitions are the visible part of a migration and, as the anatomy chapter said, not the hardest part; a migration measured by definitions moved is measuring the easy 20% and ignoring the rest. The two problems that surfaced at three months are precisely the two unmoved things: the step implementations were *translated* rather than *rebuilt*, carrying jBPM idioms that run poorly on Camunda; and the *people* were never migrated, so the operations team is slow because their hard-won jBPM expertise does not transfer to Camunda’s Operate and worklist model. Both were entirely predictable from the five-part anatomy, and both were invisible at the moment the project closed because neither shows up in a count of deployed definitions. The lesson, and the right note to close the part on, is that a migration is complete only when all five things have moved — the definitions, the implementation working properly in production, the in-flight instances, the surrounding assets, and the people’s skills and practices. A bank that defines “migration complete” as “definitions deployed” will reliably declare victory three months early and spend those three months, and more, discovering the parts it did not plan. The migration is finished when the institution can build and run its Camunda estate as confidently as it once ran the old one — and not before.

Practical next steps

Define, for any migration your institution undertakes, what “complete” means — and make the definition cover all five things a migration moves, not just the deployed process definitions. In particular, ensure the plan funds the rebuilding (not mere translation) of step implementations and the retraining of the operations and development teams. A migration declared finished at “definitions deployed” has finished only its most visible fifth.

87.4 Migration risk and mitigation

Every migration carries risk, and the risk is managed by acknowledging it explicitly rather than hoping it will not materialise. The principal risks are *functional regression* (the migrated process does not behave as the old one did), *data loss* (in-flight instances or their variables are lost or corrupted), and *performance degradation* (the new platform is slower under the same load). Each is mitigated by a specific discipline: functional regression by parallel running and comparison testing against the old platform; data loss by verifiable, checkpointed migration with rollback capability; performance degradation by load testing against realistic volumes before cutover. A migration plan that does not name its risks and their mitigations is a plan that has not been thought through.

The chapter’s principles interact with the platform’s operational model in a way worth making explicit. The *deployment* of the artefacts this chapter describes must go through a governed pipeline — versioned, reviewed, tested, and traceable. An artefact deployed by a manual action, outside the pipeline, is an artefact whose presence in production cannot be explained, and in a regulated institution an unexplained artefact is a finding.

The *monitoring* of the artefacts must cover both their functional correctness and their operational health. A process step that executes but produces wrong results is harder to detect than one that fails outright, because it does not raise an error — it raises a consequence, and the consequence may not be discovered until a customer complains or a regulator asks a question. The observability model should therefore include not only error rates and latencies but also *outcome monitoring*: checks that the results the process produces are within expected ranges, flagging anomalies for review.

Chapter in brief

Migrating from jBPM is usually the least disruptive of the common migrations: jBPM and Camunda share an open, standards-based, Java-centred heritage, so the conceptual distance is small — the main distinctive item is deciding how jBPM’s closely-coupled Drools rules become DMN or an external rules service. Across all four migrations, common lessons hold: a migration is a translation, not a transfer; the in-flight instances are the hardest, most underestimated part; a migration is an architecture opportunity that the “change nothing” instinct wastes; and the people migrate too. A migration is complete only when the definitions, the working implementation, the in-flight instances, the surrounding assets, and the institution’s skills have all moved — not when the last definition is deployed.

PART XIII

**BPEL and the Migration to
BPMN**

What BPEL Is, and Why It Still Matters

88.1 An older standard, still in production

The previous part treated migration from proprietary BPM suites. This part treats a migration of a different kind — not from a vendor’s product, but from an older *standard*: *BPEL*, the Business Process Execution Language. A great many financial institutions have processes, often critical ones, that run today as BPEL — and an architect modernising toward Camunda and BPMN will, sooner or later, meet them.

BPEL deserves a part of its own, separate from the proprietary-suite migrations, because it is a genuinely different thing. IBM BAW or webMethods is a *product*; BPEL is an open *standard* — and the processes written in it are not tied to one vendor’s tooling in the same way. The migration from BPEL is therefore less about escaping a vendor and more about moving from one process *standard* to a newer one — from BPEL to BPMN — and understanding why that move is worth making, and what it involves, is this part’s subject.

88.2 What BPEL is

BPEL — in its mature form, the OASIS standard WS-BPEL 2.0 — is an XML-based language for specifying business processes as orchestrations of *web services*. Its historical context explains its shape. BPEL emerged in the era of *service-oriented architecture* and web services, and it was designed for exactly that world: a process, in BPEL, is a way of assembling a set of discrete web services — described in WSDL, invoked over SOAP — into an end-to-end flow. BPEL was, for years, the standard for web-service orchestration, and it was widely implemented; the most prominent platform in financial services is *Oracle’s BPEL Process Manager*, part of the Oracle SOA Suite.

A BPEL process is defined entirely in XML. It has activities — invoke a service, receive a message, assign a value, wait — and structured constructs that compose them: sequences, parallel flows, conditional branches, loops, and scopes with fault handlers. A BPEL engine executes this XML directly. There is no separate diagram in the way BPMN has one: BPEL’s primary and authoritative form is the XML itself.

88.3 Block-structured, not graph-structured

The single most important thing to understand about BPEL — the fact that explains both its character and the difficulty of migrating from it — is that BPEL is *block-structured*, while BPMN is *graph-structured*.

A block-structured language composes a process the way a conventional computer program is composed: out of nested blocks. A sequence contains activities; a flow contains parallel branches; an *if* contains conditional branches; a *while* contains a loop body. Everything

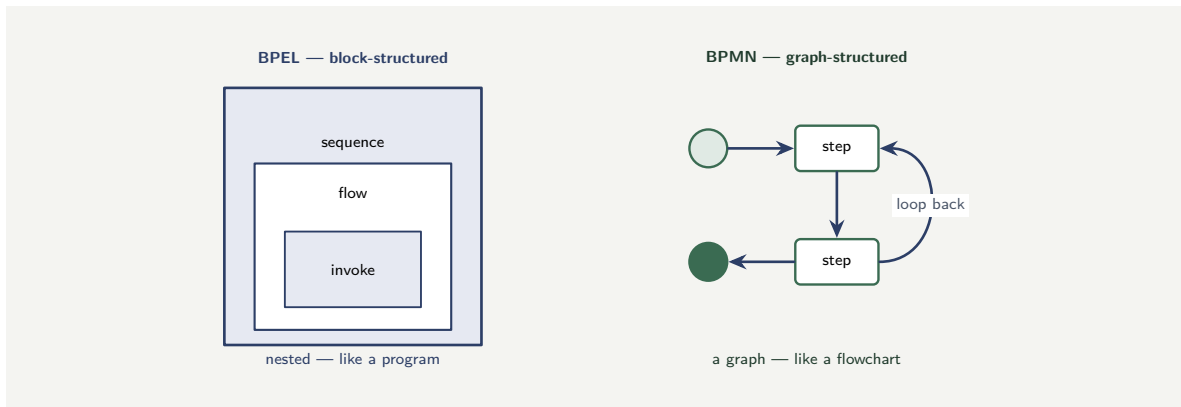


Figure 88.1. BPEL is block-structured — activities nest inside sequences, flows, and scopes, like the braces of a program. BPMN is graph-structured — nodes connected by arrows that can loop back or cross, like a flowchart. The two describe overlapping but not identical sets of process shapes.

nests cleanly inside something else, like the braces of a programming language. This is BPEL.

A graph-structured language composes a process the way a flowchart is composed: out of nodes connected by arrows that can go anywhere. A sequence flow in BPMN can loop back to an earlier node, jump across the diagram, converge from several places — the structure is a graph, not a nesting. This is BPMN.

This difference is not cosmetic, and it is the root of why BPEL and BPMN do not cleanly convert into each other. A block-structured process is, in a sense, a *restricted* shape: it cannot express an arbitrary loop-back or an arbitrary crossing flow, because those do not nest. A graph-structured process is more free. The two notations describe overlapping but not identical sets of process shapes — which [Chapter 21](#) will show is the central challenge of the migration.

Figure 88.1 contrasts BPEL’s nested blocks with BPMN’s graph.

88.4 The BPEL legacy in context

BPEL’s legacy is not merely historical; it is *present*. Many institutions still run BPEL-substrate processes, often inside IBM BAW or Oracle SOA Suite, and those processes are in production, handling real work. A migration from BPEL is therefore a migration from a running system, not an archaeological exercise, and the running system’s behaviour — including its undocumented behaviours and its accumulated workarounds — must be understood, tested against, and faithfully reproduced or deliberately changed in the migration. A BPEL process that has run for a decade has, in effect, been tested by a decade of production traffic; its replacement starts with zero production history and must earn its trust.

Worked example: why the process would not convert

Problem. An institution’s integration team, told that both BPEL and BPMN are open process standards, assumes that migrating its Oracle BPEL processes to Camunda will be a format conversion — run the BPEL XML through a converter, get BPMN out. An architect says it is not that simple, and points to one specific process: a document-collection process that, in its BPMN design, loops back — if a submitted document is rejected, flow returns to an earlier “request document” step, and this can repeat any number of times. Explain why this process

resists clean conversion.

Setup. We consider whether the loop-back can be expressed in a block-structured language.

Working. The document-collection process, as a BPMN graph, has a sequence flow from the “review document” step back to the earlier “request document” step — an arrow that jumps backward across the diagram. BPEL is block-structured: it composes processes only out of cleanly nested blocks — sequences, flows, scopes, and structured loops like `while`. An arbitrary backward jump to a named earlier activity is not a nested block:

a free graph-style loop-back $\notin$ the block-structured shapes.

To express the document-collection logic in BPEL at all, it would have to be *restructured* — reshaped into a `while` loop whose body contains the request-and-review steps, because a `while` is a block and the free loop-back is not. The process cannot simply be converted; its *shape* must be changed to fit the target language.

Discussion. The integration team’s assumption — two open standards, therefore a format conversion — is the exact misconception this chapter is built to correct, and the loop-back is the cleanest illustration of why. BPEL and BPMN are both process standards, but they are not two encodings of the same thing: one is block-structured and one is graph-structured, and that is a difference in the very *set of process shapes each can express*. A BPMN process with a free loop-back has a shape that BPEL’s nested blocks cannot directly hold, so converting that process is not a matter of translating symbols — it is a matter of *restructuring* the process into a shape the target language permits, in this case folding the loop-back into a structured `while`. Restructuring is real design work, it requires understanding what the process means, and it can change the process subtly. The lesson, which the next chapter builds on, is that a BPEL-to-BPMN migration — or the reverse — is never a pure conversion. It is a translation between two languages whose expressible shapes only overlap, and the places where they differ are places the migration must consciously reshape the process. An institution that budgets a BPEL migration as “run the converter” has mis-budgeted it; the converter handles the overlap, and the architect handles the difference.

Practical next steps

If your institution runs BPEL processes, resist the assumption that moving them to BPMN is a format conversion. Identify, early, the processes whose shape — free loop-backs, crossing flows — may not have lived naturally in BPEL’s block structure, and the BPMN designs that will not reduce to BPEL blocks. Those are the processes the migration must consciously reshape, and they are where the real effort lies.

Chapter in brief

BPEL — the Business Process Execution Language, in its mature form WS-BPEL 2.0 — is an older but still widely-running open standard for orchestrating web services, defined entirely in XML, prominent in financial services through Oracle’s BPEL Process Manager. The defining fact about BPEL is that it is *block-structured* — processes composed of cleanly nested blocks, like a program — whereas BPMN is *graph-structured* — nodes and arrows that can loop back and cross, like a flowchart. The two describe overlapping but not identical sets of process shapes, which is why BPEL and BPMN do not cleanly convert into each other: a migration between them is a translation that

must consciously reshape the processes where the two languages differ.

Migrating BPEL Processes to BPMN in Camunda

89.1 The shape of the migration

The previous chapter established what BPEL is and why a BPEL-to-BPMN move is a translation, not a conversion. This chapter treats the migration itself: how an institution moves its BPEL processes onto Camunda as BPMN, and the disciplines that make the move an improvement rather than a lossy copy.

The migration follows the five-part anatomy of [Chapter 74](#) — definitions, step implementation, in-flight instances, surrounding assets, people — but the *definitions* step has the particular character the previous chapter described: it is a translation between a block-structured and a graph-structured language, and the block-to-graph translation is the migration’s signature work.

89.2 Translating the block structure to a graph

The good news about the block-to-graph direction is that it is the *easier* of the two directions. A block-structured BPEL process is a restricted shape; translating it to a graph-structured BPMN process means expressing those nested blocks as graph constructs, and the graph can always express what the blocks could — BPMN is the more expressive notation. The difficulty of the previous chapter’s worked example was BPMN-to-BPEL, the *hard* direction; BPEL-to-BPMN, the migration’s actual direction, is the direction that does not lose anything.

The translation is, in outline, a mapping of constructs. A BPEL `sequence` becomes a chain of BPMN activities connected by sequence flow. A BPEL `flow` — parallel branches — becomes a pair of BPMN parallel gateways with the branches between them. A BPEL `if` becomes BPMN exclusive gateways. A BPEL `while` becomes a BPMN loop — and here the graph’s freedom helps, because the loop can be drawn naturally. A BPEL `invoke` of a web service becomes a BPMN service task. A BPEL `scope` with a fault handler becomes a BPMN subprocess with a boundary error event. A BPEL `receive` or `pick` becomes a BPMN message event.

Tooling exists to assist this mapping, and it handles the mechanical bulk. But the architect’s judgement is still required, because a faithful mechanical mapping produces a BPMN diagram that is *correct* but often *shaped like BPEL* — deeply nested, structured as the XML was, rather than drawn as a process analyst would naturally draw it.

Figure [89.1](#) sets out the BPEL-to-BPMN construct mapping.

89.3 The mapping in code

The construct mapping is concrete when a BPEL fragment and its BPMN equivalent are placed side by side. Listing [89.1](#) is a small BPEL fragment: a sequence containing an `invoke` of a web service, wrapped in a `scope` with a fault handler.

BPEL construct	BPMN equivalent
sequence	chain of activities + flow
flow (parallel)	parallel gateways
if	exclusive gateways
while	BPMN loop
invoke	service task
scope + fault handler	subprocess + boundary error event
receive / pick	message event

Figure 89.1. The construct mapping at the heart of a BPEL-to-BPMN migration. Each block-structured BPEL construct has a BPMN equivalent — tooling handles the mechanical mapping, but the architect must still reshape the result so it reads as a process, not as BPEL XML drawn out.

Listing 89.1. A BPEL fragment: a scope with a fault handler, containing a sequence that invokes a web service.

```
1 <scope name="ScoreApplication">
2   <faultHandlers>
3     <catch faultName="bureauFault">
4       <invoke partnerLink="logger"
5         operation="recordFailure"/>
6     </catch>
7   </faultHandlers>
8   <sequence>
9     <invoke partnerLink="creditBureau"
10       operation="score"
11       inputVariable="application"
12       outputVariable="score"/>
13   </sequence>
14 </scope>
```

Listing 89.2 is the same fragment translated to BPMN: the BPEL scope becomes a subprocess, the faultHandlers catch becomes a *boundary error event* on that subprocess, and the invoke becomes a service task.

Listing 89.2. The BPMN equivalent: the BPEL scope becomes a subprocess, the fault handler a boundary error event, the invoke a service task.

```
1 <bpmn:subProcess id="ScoreApplication">
2   <bpmn:serviceTask id="score" name="Score Application">
3     <bpmn:extensionElements>
4       <zeebe:taskDefinition type="credit-bureau-score"/>
5     </bpmn:extensionElements>
6   </bpmn:serviceTask>
7 </bpmn:subProcess>
8
9 <!-- the BPEL fault handler becomes a boundary event -->
10 <bpmn:boundaryEvent id="onBureauFault"
11   attachedToRef="ScoreApplication">
12   <bpmn:errorEventDefinition errorRef="bureauFault"/>
13 </bpmn:boundaryEvent>
```


The pairing shows both what is straightforward and what is not. The mapping itself is clean — scope to subprocess, fault handler to boundary event, invoke to service task — and a tool performs it mechanically. But notice what the BPMN version has gained: the fault handler, buried *inside* the BPEL scope's XML, is now a *boundary event drawn on the edge of the subprocess*, visible on the diagram. That is the migration working as it should — BPEL's nested, textual structure becoming BPMN's visible graph. The risk the section warned of is the opposite outcome: a mechanical translation that keeps the BPEL nesting, producing a BPMN subprocess buried five levels deep because the BPEL was, with no analyst reshaping it into the flat, readable graph BPMN exists to provide.

89.4 From integration script to readable process

The deepest opportunity in a BPEL migration is not the construct mapping — it is the chance to recover a *readable business process* from what had become an integration script.

Recall what BPEL is for: orchestrating web services in a service-oriented architecture. A BPEL process, in practice, is often heavy with the technical detail of that orchestration — WSDL bindings, SOAP faults, XML assignments — and light on business legibility. It was authored by integration developers, in XML, and a business process owner cannot read it. Over the years it has often accumulated, like the webMethods estates of [Chapter 86](#), a tangle of integration logic and business flow.

The migration to BPMN is the chance to fix this. Done well, it does for the BPEL process what [Chapter 86](#)'s disentanglement did for the webMethods process: it separates the genuine *business process* — which becomes a clean, business-readable BPMN model — from the *integration detail* — which becomes connectors and job workers behind the process, the architecture of [Chapter 25](#). The web services that the BPEL process invoked do not disappear; they become systems the BPMN process integrates with through clean service tasks. What changes is that the process itself stops being an integration script only a developer can read and becomes an explicit, governed, business-legible model — which is the entire reason BPMN displaced BPEL as the process standard, and the entire reason the migration is worth doing.

Tip

The prize in a BPEL-to-BPMN migration is not the construct mapping — tooling largely handles that — it is recovering a *readable business process* from an integration script. A BPEL process is typically XML authored by integration developers, heavy with WSDL and SOAP detail, unreadable by a business owner. Migrate it as [Chapter 86](#) migrated webMethods: separate the genuine business flow, which becomes a clean BPMN model, from the integration detail, which becomes connectors and workers behind it. A migration that produces BPMN no business owner can read has copied the BPEL, not improved on it.

89.5 The hidden cost of a migration

Every migration has a cost the plan does not show: the cost of the *knowledge* that lives in the old platform and in the heads of the people who run it. The old platform's configuration, its workarounds, its undocumented behaviours, its operational playbooks — all of these are knowledge that was bought with years of production experience, and none of it transfers automatically. A migration plan that costs only the artefacts — the process models, the integration

code, the UI — and not the operational knowledge is undercosted, and the gap will appear in the months after go-live, when the team discovers behaviours the old platform handled that the new one does not yet, because no one remembered to transfer the knowledge.

Worked example: the faithfully migrated BPEL that no one could read

Problem. A bank migrates a BPEL payment-orchestration process to Camunda. The migration team uses a conversion tool and is pleased: every BPEL construct mapped to a BPMN equivalent, the resulting process is logically identical, and it runs correctly on Camunda. But when the bank's payments process owner — a business manager, not a developer — is shown the new BPMN diagram, she finds it as unreadable as the BPEL XML was: it has 9 levels of nested subprocesses, service tasks named after WSDL operations, and gateways mediating SOAP faults. The migration is declared a technical success and a business disappointment. Assess what was missed.

Setup. We compare what the migration produced against what a BPMN model is supposed to give the institution.

Working. The migration achieved a faithful *construct mapping*: BPEL sequences, flows, scopes, and invokes each became their BPMN equivalent, and the logic is preserved. But faithful mapping preserved the BPEL process's *shape* and *character*: 9 levels of nesting because the BPEL XML was nested 9 deep; service tasks named after WSDL operations because the BPEL invokes were; gateways mediating SOAP faults because the BPEL scopes did. The result is BPMN that is:

logically correct but shaped and named like BPEL XML.

The one thing a BPMN model is *for* — being readable by the business process owner — was not delivered, because the migration translated the *constructs* but not the *character*: it did not separate business flow from integration detail, and did not reshape the deeply-nested structure into a process a business reader would recognise.

Discussion. The migration team measured success as “the constructs mapped and the logic runs,” and by that measure they succeeded — but that measure is the wrong one, and the worked example exists to show why. BPMN displaced BPEL as the process standard for one reason above all: BPMN is business-readable and BPEL is not. So a BPEL-to-BPMN migration whose output is unreadable by the business has migrated the format and missed the entire point. The 9 levels of nesting are the giveaway: BPEL is block-structured and nests deeply, and a faithful mapping carries that nesting straight into the BPMN diagram, where — because BPMN is a graph and did not need to nest — it is pure inherited clutter. A process analyst drawing the payment process fresh would draw it flat and legible, naming steps for what they do in business terms, with the SOAP-fault mediation tucked into the integration layer, not on the process canvas. That is the reshaping the migration skipped. The lesson is the chapter's central discipline: a BPEL migration's success is measured not by whether the constructs mapped but by whether the result is a process the business can read and own. The conversion tool produces correct BPMN; the architect must then do what the tool cannot — separate flow from integration, flatten the inherited nesting, rename for business meaning — so that the migration delivers the readable, governable process model that was the whole reason to leave BPEL.

Practical next steps

If your institution is migrating BPEL processes, define migration success as “a business owner can read and own the resulting BPMN” — not as “the constructs mapped and it runs.” Budget, explicitly, the reshaping work after the conversion tool: separating integration detail from business flow, flattening BPEL’s inherited deep nesting, and renaming technical steps in business terms. That reshaping is where the value of leaving BPEL is actually realised.

89.6 Migration risk and mitigation

Every migration carries risk, and the risk is managed by acknowledging it explicitly rather than hoping it will not materialise. The principal risks are *functional regression* (the migrated process does not behave as the old one did), *data loss* (in-flight instances or their variables are lost or corrupted), and *performance degradation* (the new platform is slower under the same load). Each is mitigated by a specific discipline: functional regression by parallel running and comparison testing against the old platform; data loss by verifiable, checkpointed migration with rollback capability; performance degradation by load testing against realistic volumes before cutover. A migration plan that does not name its risks and their mitigations is a plan that has not been thought through.

The chapter’s principles interact with the platform’s operational model in a way worth making explicit. The *deployment* of the artefacts this chapter describes must go through a governed pipeline — versioned, reviewed, tested, and traceable. An artefact deployed by a manual action, outside the pipeline, is an artefact whose presence in production cannot be explained, and in a regulated institution an unexplained artefact is a finding.

The *monitoring* of the artefacts must cover both their functional correctness and their operational health. A process step that executes but produces wrong results is harder to detect than one that fails outright, because it does not raise an error — it raises a consequence, and the consequence may not be discovered until a customer complains or a regulator asks a question. The observability model should therefore include not only error rates and latencies but also *outcome monitoring*: checks that the results the process produces are within expected ranges, flagging anomalies for review.

Chapter in brief

Migrating BPEL processes to BPMN on Camunda follows the five-part migration anatomy, with the definitions step having a special character: a translation from a block-structured to a graph-structured language. The block-to-graph direction is the easier one — BPMN can always express what BPEL’s nested blocks could — and the constructs map systematically: sequences to flows, `flow` to parallel gateways, `scope` to subprocess with boundary error event, `invoke` to service task. Tooling handles the mechanical mapping. But the real prize is recovering a readable business process from an integration script: separating genuine business flow from WSDL-and-SOAP integration detail, and reshaping BPEL’s inherited deep nesting into a model a business owner can read — which is the entire reason BPMN displaced BPEL.

Migrating BPEL and SCA Modules

90.1 The older substrate beneath BAW

The seventh construct is not, strictly, a BAW construct at all — it is the *older substrate* that a BAW estate may be sitting on, and a migration that does not look for it will be surprised by it.

IBM BAW, and IBM BPM before it, did not appear from nothing. Parts of an IBM automation estate descend from the older *WebSphere Process Server*, and where they do, the estate may contain artefacts that are not BPMN at all. It may contain *BPEL* processes — the Business Process Execution Language, the older standard for process orchestration — and *SCA* modules: Service Component Architecture, IBM's older component-assembly model.

This matters because BPEL is a fundamentally *different execution model* from BPMN. BPMN is *graph-structured*: a process is a graph of nodes and arrows, and the flow goes wherever the arrows lead. BPEL is *block-structured*: a process is a nesting of blocks — a sequence, a flow, a while, a pick — like the nested structure of a program. The two are not cosmetic variants; they are different ways of expressing what a process *is*.

90.2 Detection, and the conversion that is not the ordinary one

The Camunda target for a BPEL process is still a Camunda BPMN model — but the translation is *not* the ordinary BPMN-to-BPMN translation of [Chapter 76](#). Because BPEL is block-structured and BPMN is graph-structured, the translation must *re-express the block structure as graph structure*: a BPEL sequence becomes a chain of flow; a BPEL flow becomes parallel branches with gateways; a BPEL pick becomes an event-based gateway. This block-to-graph re-expression is a specific discipline, and it is the specific subject of [Part XIII](#), which this manuscript devotes to BPEL migration in its own right.

The row is rated *high*, but its defining planning point is not effort — it is *detection*. The danger is a migration plan built on the assumption “BAW means BPMN.” A plan that assumes the whole estate is BPMN will translate it with the ordinary BPMN-to-BPMN approach, and will simply not have *budgeted* for the BPEL conversion at all — until the BPEL processes are discovered, mid-migration, with no allowance for them. The architect's task in this row is therefore an *inventory* task done early: establish, during assessment, whether the estate carries this older BPEL and SCA substrate at all. If it does not, the row is empty and costs nothing. If it does, the BPEL processes are routed to [Part XIII](#)'s approach and budgeted accordingly. What must not happen is discovering the substrate after the plan is set.

90.3 Block-to-graph: the translation in code

The block-to-graph re-expression is abstract until it is seen as code. This section shows three BPEL constructs and their Camunda BPMN equivalents, so the shape of the translation —

and why it is not the ordinary BPMN-to-BPMN copy — is concrete.

Consider first a BPEL sequence: a block that runs its children one after another. Listing 90.1 is a small BPEL sequence — receive a request, invoke a service, reply.

Listing 90.1. A BPEL sequence: a block-structured construct whose children run in order. The nesting, not arrows, expresses the flow.

```
1 <sequence>
2   <receive partnerLink="client" operation="submit"
3     variable="request" createInstance="yes"/>
4   <invoke partnerLink="scoring" operation="score"
5     inputVariable="request"
6     outputVariable="score"/>
7   <reply partnerLink="client" operation="submit"
8     variable="score"/>
9 </sequence>
```

In BPEL the order is expressed by *nesting* — the three children sit inside the sequence, and that containment *is* the sequencing. The Camunda equivalent expresses the same order as an explicit *graph*: a start event, three tasks, an end event, joined by sequence flows. Listing 90.2 is the BPMN.

Listing 90.2. The Camunda BPMN equivalent: the same order expressed as an explicit graph of elements joined by sequence flows.

```
1 <bpmn:startEvent id="start"/>
2 <bpmn:serviceTask id="receiveReq" name="Receive"/>
3 <bpmn:serviceTask id="score" name="Score"/>
4 <bpmn:serviceTask id="reply" name="Reply"/>
5 <bpmn:endEvent id="end"/>
6 <bpmn:sequenceFlow sourceRef="start" targetRef="receiveReq"/>
7 <bpmn:sequenceFlow sourceRef="receiveReq" targetRef="score"/>
8 <bpmn:sequenceFlow sourceRef="score" targetRef="reply"/>
9 <bpmn:sequenceFlow sourceRef="reply" targetRef="end"/>
```

The difference is the whole point of the row. BPEL's sequence has no arrows — order is containment; BPMN's flow has no containment — order is arrows. Translating one to the other is not editing attributes on the same shape; it is re-expressing the process in a different structural model.

The contrast is sharper for a BPEL *flow*, which runs its children *in parallel*. Listing 90.3 shows a flow with two parallel branches.

Listing 90.3. A BPEL flow: a block whose children run concurrently. Again, structure is expressed by nesting.

```
1 <flow>
2   <invoke partnerLink="credit" operation="check"
3     inputVariable="app" outputVariable="credit"/>
4   <invoke partnerLink="fraud" operation="screen"
5     inputVariable="app" outputVariable="fraud"/>
6 </flow>
```

The Camunda equivalent cannot express concurrency by nesting; it must use an explicit *parallel gateway* to split the flow into branches and another to join them, as in Listing 90.4.

Listing 90.4. The Camunda BPMN equivalent of the BPEL flow: a parallel gateway splits the graph into concurrent branches and a second gateway joins them.

```
1 <bpmn:parallelGateway id="fork"/>
2 <bpmn:serviceTask id="creditCheck" name="Credit Check"/>
```

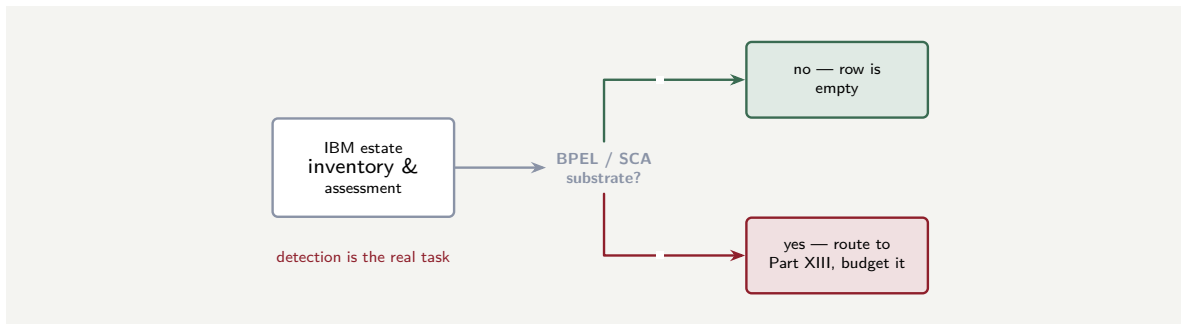


Figure 90.1. The BPEL/SCA row is a detection problem. During inventory an architect must establish whether the IBM estate carries the older WebSphere Process Server substrate. If not, the row costs nothing; if so, the BPEL processes are routed to Part XIII’s block-to-graph approach and budgeted — not discovered after the plan is set.

```

3 <bpmn:serviceTask id="fraudScreen" name="Fraud Screen"/>
4 <bpmn:parallelGateway id="join"/>
5 <bpmn:sequenceFlow sourceRef="fork" targetRef="creditCheck"/>
6 <bpmn:sequenceFlow sourceRef="fork" targetRef="fraudScreen"/>
7 <bpmn:sequenceFlow sourceRef="creditCheck" targetRef="join"/>
8 <bpmn:sequenceFlow sourceRef="fraudScreen" targetRef="join"/>

```

A BPEL flow — a single block — has become *four* BPMN elements: two gateways and the structure that joins them. This is why the row is rated *high* and why it is not the ordinary translation: a construct that is one nested block in BPEL fans into a small graph in BPMN, and the conversion must introduce elements — gateways — that have no one-to-one source. The same is true of a BPEL pick, which waits for one of several events and becomes a Camunda *event-based gateway*, and of *while*, which becomes a loop expressed with a gateway and a flow back. Part XIII treats the full set; the point here is that every BPEL structural construct is a *re-expression*, not a relabelling.

Tip

When estimating a BPEL conversion, do not count BPEL constructs as if each maps to one BPMN element. A BPEL *sequence* expands into a chain of elements and flows; a *flow* expands into branches plus *two* gateways; a *pick* into an event-based gateway and its events. The element count grows in translation, and — more importantly — the translation is a re-expression in a different structural model, which is design work, not attribute editing. Budget it as such.

Figure 90.1 frames the row as the detection decision it is.

90.4 The hidden cost of a migration

Every migration has a cost the plan does not show: the cost of the *knowledge* that lives in the old platform and in the heads of the people who run it. The old platform’s configuration, its workarounds, its undocumented behaviours, its operational playbooks — all of these are knowledge that was bought with years of production experience, and none of it transfers automatically. A migration plan that costs only the artefacts — the process models, the integration code, the UI — and not the operational knowledge is undercosted, and the gap will appear in the months after go-live, when the team discovers behaviours the old platform handled that

the new one does not yet, because no one remembered to transfer the knowledge.

Worked example: the substrate found after the plan was signed

Problem. A bank's BAW migration plan is built, costed, and approved on the stated basis "the estate is BPMN; both platforms are BPMN; the model translation is BPMN-to-BPMN." Two months in, the team finds that 18 of the processes are not BPMN at all — they are BPEL processes inherited from a WebSphere Process Server era predating IBM BPM. Assess the position.

Setup. We consider what the plan assumed and what the 18 BPEL processes actually require.

Working. The plan assumed a uniform estate — all BPMN — and budgeted a uniform translation: BPMN-to-BPMN, the ordinary kind. The 18 BPEL processes break that assumption. They are block-structured, and translating them is not BPMN-to-BPMN; it is the block-to-graph re-expression of [Part XIII](#), a different and harder discipline:

plan budgeted: BPMN-to-BPMN for all; 18 processes need: BPEL block-to-graph.

The 18 processes have *no budget line at all* — not an underestimate, but an absence — because the plan's assumption excluded the possibility of their existing.

Discussion. The failure here is not one of estimation but of *inventory*, and the worked example shows why this row's planning point is detection rather than effort. The plan did not under-cost the BPEL processes; it did not know they existed, because it assumed "BAW means BPMN" and never checked. An IBM automation estate is often geologically layered — BAW on top, IBM BPM beneath, and, where the lineage runs deep, WebSphere Process Server BPEL and SCA at the bottom — and the older strata do not announce themselves on a modern process list. The only defence is the chapter's discipline: during inventory and assessment, *deliberately look* for the older substrate — ask whether any part of the estate predates IBM BPM, examine the older processes' actual form — so that the BPEL processes are found *before* the plan is costed, routed to [Part XIII](#)'s approach, and given a real budget line. The lesson is that an unexamined assumption about the estate's uniformity is the most expensive kind of assumption: a plan that assumes all BPMN and meets BPEL has not made a small error, it has a whole category of work with no allowance, discovered at the worst time — after the commitment is made.

Practical next steps

During the inventory of an IBM BAW estate, deliberately check for an older BPEL and SCA substrate from the WebSphere Process Server era — do not assume "BAW means BPMN." If the substrate exists, route those processes to [Part XIII](#)'s block-to-graph approach and budget them explicitly. The cost of this row is set by whether it is detected before the plan is signed or after.

Chapter in brief

Where an IBM estate descends from the older *WebSphere Process Server*, parts of it may not be BPMN at all but *BPEL* processes and *SCA* modules — a different, older, *block-structured* execution model, against BPMN's graph structure. The Camunda target is

still a BPMN model, but the translation is not the ordinary BPMN-to-BPMN one: the block structure must be re-expressed as graph structure, the specific subject of **Part XIII**. The row is high-effort, but its defining planning point is *detection*: a plan that assumes “BAW means BPMN” will have no budget line at all for the BPEL conversion, so an architect must establish during inventory whether the older substrate exists — before the plan is costed, not after.

PART XIV

Java Implementation for Camunda

Java Design Patterns and SOLID Principles for Camunda

91.1 Why patterns and principles belong in a Camunda book

A Camunda platform is, at its implementation level, a Java application — and the quality of its Java determines the quality of the platform. A well-designed worker is simple, testable, and independently deployable; a poorly designed one is brittle, untestable, and tightly coupled to the engine it should be separated from. The difference is not talent; it is the application of established design patterns and principles that the Java community has refined over decades. This chapter names the patterns and principles that matter most for Camunda development and shows how each applies.

91.2 The SOLID principles, applied to Camunda

The five SOLID principles — originally articulated by Robert C. Martin — are a set of design guidelines that, when followed, produce code that is easier to understand, test, and change. Each has a specific, concrete application in a Camunda platform.

Single Responsibility Principle

A class should have one reason to change. For a Camunda job worker, this means the worker class does *one thing*: it receives a job, delegates to a service, and returns the result. It does not also parse configuration, manage database connections, or decide what happens next in the process. A worker that violates the Single Responsibility Principle is a worker that must change whenever *any* of its multiple responsibilities changes — and in a regulated platform, every change is a review, a test, and a deployment.

Listing 91.1. A worker that respects the Single Responsibility Principle: it does one thing — delegates to the scoring service and returns the result.

```
1 @JobWorker(type = "score-application")
2 public Map<String, Object> score(ActivatedJob job) {
3     // one responsibility: delegate and return
4     ScoringResult result = scoringService.score(
5         job.getVariable("applicationRef"));
6     return Map.of("creditScore", result.score(),
7                 "scoreBand",    result.band());
8 }
```

Open/Closed Principle

A class should be open for extension but closed for modification. In Camunda terms, a worker's behaviour should be extendable — through configuration, through injected strategies, through new implementations of an interface — without modifying the worker class

itself. The worker calls an interface; new behaviour is a new implementation of that interface, deployed alongside, not a change to the worker's code.

Liskov Substitution Principle

Subtypes must be substitutable for their base types. When a Camunda platform uses an interface for a service — a `ScoringService` interface with implementations for different credit bureaus — any implementation must be safely swappable without the worker knowing. A `EquifaxScoringService` and an `ExperianScoringService` must both honour the contract of `ScoringService`; a worker that receives either must produce correct results. This is the core-agnostic worker pattern of [Chapter 106](#), stated as a principle.

Interface Segregation Principle

No client should be forced to depend on methods it does not use. A worker that integrates with a core banking system should depend on a narrow interface — `fetchBalance(accountId)` — not on a fat client that exposes every operation the core provides. A fat dependency couples the worker to the core's entire API surface; a narrow one couples it only to the operation it needs.

Dependency Inversion Principle

High-level modules should not depend on low-level modules; both should depend on abstractions. A Camunda worker (high-level) should depend on an injected *interface* (an abstraction), not on a concrete HTTP client or a specific vendor SDK (a low-level module). The concrete implementation is provided by Spring's dependency injection at runtime. This is what makes the worker testable — the test supplies a stub; production supplies the real implementation — and it is the foundation of the thin-worker pattern this part develops throughout.

Listing 91.2. Dependency Inversion: the worker depends on an injected interface, not a concrete implementation. A test supplies a stub; production supplies the real service.

```
1  @Component
2  public class BalanceWorker {
3      private final AccountService accountService;
4
5      // injected -- the worker never constructs its own
6      public BalanceWorker(AccountService accountService) {
7          this.accountService = accountService;
8      }
9
10     @JobWorker(type = "fetch-balance")
11     public Map<String, Object> fetch(ActivatedJob job) {
12         BigDecimal bal = accountService.getBalance(
13             job.getVariable("accountId"));
14         return Map.of("balance", bal);
15     }
16 }
```

91.3 Key Java design patterns for Camunda

Beyond SOLID, three design patterns recur in well-built Camunda platforms.

The Strategy pattern

A worker that must perform the same logical operation against different backends — different credit bureaus, different core banking systems, different payment gateways — uses the *Strategy pattern*: one worker class, with the backend-specific logic injected as a strategy implementation. The worker delegates to whichever strategy Spring injects; a new backend is a new strategy class, not a change to the worker.

Listing 91.3. The Strategy pattern: the worker delegates to an injected strategy. A new backend is a new strategy class, not a modification to the worker.

```
1 public interface PaymentGateway {
2     PaymentResult initiate(String ref, BigDecimal amt);
3 }
4
5 @Component @Profile("paypal")
6 public class PayPalGateway implements PaymentGateway {
7     public PaymentResult initiate(String ref,
8         BigDecimal amt) {
9         return paypalClient.createOrder(ref, amt);
10    }
11 }
12
13 @JobWorker(type = "initiate-payment")
14 public Map<String, Object> pay(ActivatedJob job) {
15     // the injected strategy determines the backend
16     PaymentResult r = gateway.initiate(
17         job.getVariable("paymentRef"),
18         job.getVariable("amount"));
19     return Map.of("transactionId", r.id());
20 }
```

The Adapter pattern

A worker integrating with an external system — a core banking API, a CRM, a legacy mainframe — wraps the external system's interface in an *Adapter*: a class that translates the external system's data model into the process's own vocabulary. The worker calls the adapter; the adapter calls the external system; and the process model upstream never sees a vendor-specific field name. This is the anti-corruption layer of [Chapter 25](#), expressed as a Java pattern.

The Template Method pattern

Many workers share a common structure: fetch variables, call a service, handle errors, return results. The *Template Method* pattern extracts that common structure into a base class, with the service-specific step left as an abstract method that each concrete worker implements. This reduces boilerplate and ensures that every worker in the fleet handles errors, logs, and returns results the same way — the consistency a platform team needs when it operates dozens of workers.

91.4 Patterns to avoid

Two anti-patterns are worth naming because they are common and costly.

The *fat worker* puts business logic, process-routing decisions, and state management inside the worker class. It violates Single Responsibility, is untestable without engine machinery, and makes the worker a hidden piece of the process that no BPMN diagram shows.

The *God service* injects a single service class that does everything — calls the core, calls the

CRM, sends notifications, and manages its own transactions. It violates Interface Segregation, couples the worker to every system the service touches, and makes the worker's blast radius enormous: a change to the notification logic forces a redeployment of the worker that calls the core.

Both are avoided by applying the principles this chapter has described: single responsibility, narrow interfaces, injected abstractions, and the strategy pattern for backend variation.

Worked example: the worker that was four workers

Problem. A team builds a single job worker that, depending on a process variable `action`, calls the core banking system, the CRM, the notification service, or the document store. The worker has four responsibilities, four sets of dependencies, and a `switch` statement that routes between them. Assess this design against the chapter's principles.

Setup. We evaluate the worker against Single Responsibility and Interface Segregation.

Working. The worker has four reasons to change: a change to any of the four backends forces a change to the worker. It violates Single Responsibility (four responsibilities in one class) and Interface Segregation (the worker depends on four service interfaces it uses one at a time). Its test requires stubs for all four services, even when testing only one path. And its deployment couples four independent integrations: a fix to the notification logic forces a redeployment that also carries the core-banking, CRM, and document-store code.

Discussion. The sound design is four separate workers, one per action, each with a single responsibility and a single injected service. The process model dispatches to the right worker by using four service tasks, each with its own job type, rather than one task with a routing variable. The four workers are independently deployable, independently testable, and independently changeable. The "one worker, four actions" design saved a few minutes of modelling and cost the team months of coupling.

Practical next steps

Review the workers in your Camunda platform against the SOLID principles. For each worker, ask: does it have one responsibility? Does it depend on a narrow interface? Is its service injected, not constructed? Can it be tested without engine machinery? Where the answer is no, refactor — the cost is small and the return is every subsequent change, test, and deployment made easier.

Chapter in brief

The quality of a Camunda platform's Java determines the quality of the platform, and the SOLID principles provide the design discipline. *Single Responsibility*: one worker, one job; *Open/Closed*: extend through injected strategies, not by modifying worker code; *Liskov Substitution*: backend implementations are interchangeable behind their interface; *Interface Segregation*: depend on narrow interfaces, not fat clients; *Dependency Inversion*: the worker depends on an injected abstraction, making it testable with a stub. Three design patterns recur: *Strategy* for multi-backend workers, *Adapter* for anti-corruption translation, and *Template Method* for consistent worker structure. Two anti-patterns to avoid: the *fat worker* (business logic and routing inside the worker) and the *God service* (one service class that does everything).

The Java Toolchain and the Spring Boot Starter

92.1 Why a part on Java

The manuscript has, until now, kept its treatment of code deliberately light. [Chapter 31](#) showed a Java delegate, [Chapter 32](#) a job worker, and the API chapters showed calls as raw HTTP — but each was an illustration in service of a concept, not a guide to building. This part is the guide to building. It treats, in deep technical detail, how a Camunda platform is actually implemented in Java: the toolchain, the worker code, the client, the tests, and the production-hardening that separates a demonstration from a system a bank can run.

The choice of Java is not arbitrary. Java with Spring Boot is the default technology stack for building on Camunda, the combination the platform’s own tooling and documentation assume, and the one the overwhelming majority of banking and insurance institutions already build on. A reader whose institution uses another language will find the concepts transfer — a job worker is a job worker in any language — but the concrete detail here is Java, because that is where the detail is most useful.

A word on what this part is not. It is not a Java tutorial; it assumes the reader can read and write Java. And it is not a substitute for the platform’s own reference documentation, which is versioned and current in a way print cannot be. It is the bridge between the two: the architecture and disciplines of Parts I through III, expressed as the Java a team actually writes, with the production concerns a reference manual does not emphasise made central.

92.2 The Spring Boot Starter

A Java application that connects to a Camunda 8 platform — to run job workers, to start processes, to interact with the engine — is built on the *Camunda Spring Boot Starter*. It is the supported library that wires a Spring Boot application to the orchestration cluster: it manages the connection, the authentication, the job-worker machinery, and the client, so that the application code is business logic and annotations rather than plumbing.

A point of currency matters here, and the manuscript states it plainly because it is exactly the kind of detail that dates. The Camunda Java tooling has evolved through more than one generation — an earlier community Spring Zeebe SDK, then the supported Spring Boot Starter — and the artifact names, versions, and some package names change between releases. The reader should take the artifact identifiers in this part as illustrative of *shape*, and confirm the exact current identifiers against the platform’s documentation for their target version. What is stable — and what this part teaches — is the structure: a Spring Boot application, a starter dependency, a YAML configuration, annotated worker methods. The names on the parts move; the shape does not.

The dependency is added to the build — a Maven `pom.xml` or a Gradle build file — as a single starter artifact, in the `io.camunda` group. [Listing 92.1](#) shows the shape in Maven.

Listing 92.1. The Camunda Spring Boot Starter dependency in a Maven `pom.xml`. The exact artifact id and version move between releases — confirm them against the documentation for your target platform version; the shape is stable.

```
1 <dependencies>
2   <!-- the Camunda Spring Boot Starter -->
3   <dependency>
4     <groupId>io.camunda</groupId>
5     <artifactId>spring-boot-starter-camunda-sdk</artifactId>
6     <version><!-- target version --></version>
7   </dependency>
8   <!-- standard Spring Boot starter -->
9   <dependency>
10    <groupId>org.springframework.boot</groupId>
11    <artifactId>spring-boot-starter</artifactId>
12  </dependency>
13 </dependencies>
```

One build-level detail is worth singling out because omitting it causes a confusing failure: the Java compiler should be run with the `-parameters` flag enabled. The worker machinery, as the next chapter shows, binds process variables to method parameters *by name*, and the parameter names survive compilation into the bytecode only if that flag is set. Without it, the binding fails in a way whose cause is not obvious from the error. It is a one-line build configuration, and it belongs in every Camunda Java project.

92.3 Connecting to the cluster

The application’s connection to the orchestration cluster is configured, in Spring Boot fashion, in an `application.yaml` file — and, crucially, *not* in code. Configuration externalised into YAML can differ between environments — a developer’s local cluster, a test environment, production — without the application being rebuilt, which is the twelve-factor discipline and the [Chapter 24](#) principle of configuration as a separate, environment-specific concern.

Listing 92.2 shows the shape of the configuration for a self-managed cluster with OIDC authentication — the authentication of [Chapter 35](#), expressed as Spring configuration.

Listing 92.2. The shape of an `application.yaml` connecting to a self-managed cluster with OIDC authentication. The credentials themselves are not here — they are injected from the environment or a secrets service.

```
1 camunda:
2   client:
3     mode: self-managed
4     # the cluster's gRPC and REST addresses
5     zeebe:
6       grpc-address: https://cluster.internal:26500
7       rest-address: https://cluster.internal:8080
8     # OIDC authentication -- the client-credentials flow
9     auth:
10      issuer: https://idp.internal/realms/camunda-platform
11      client-id: ${CAMUNDA_CLIENT_ID}
12      client-secret: ${CAMUNDA_CLIENT_SECRET}
13      audience: orchestration-cluster
```

Note what is, and is not, in that file. The cluster addresses, the issuer, the audience — the non-secret configuration — are stated directly. The client ID and secret are *not*: they appear as `${...}` placeholders, injected at runtime from environment variables or, better, from the secrets-management service of [Chapter 24](#). A credential committed into a configuration file in a repository is the leaked-secret incident of [Chapter 35](#) waiting to happen, and the discipline

of keeping secrets out of the configuration file is absolute.

Tip

Keep the connection configuration in `application.yaml`, not in code, so it can differ between environments without a rebuild — but keep the *secrets* out of the YAML too. The client ID and secret belong in environment variables injected from a secrets service; the YAML carries only a placeholder. A repository’s history is forever, and a secret committed once is a secret leaked permanently.

92.4 The shape of a Camunda Java application

With the starter on the build path and the connection configured, a Camunda Java application has a recognisable, simple shape: an ordinary Spring Boot application, started in the ordinary way, whose components include annotated job workers. Listing 92.3 shows the skeleton.

Listing 92.3. The skeleton of a Camunda Spring Boot application. It is an ordinary Spring Boot application; the starter wires it to the cluster, and the `@Deployment` annotation deploys process models at startup.

```
1 @SpringBootApplication
2 @Deployment(resources = "classpath*:bpmn/**/*.bpmn")
3 public class LendingPlatformApplication {
4
5     public static void main(String[] args) {
6         SpringApplication.run(
7             LendingPlatformApplication.class, args);
8     }
9 }
```

Two things in that short listing carry weight. The application is, simply, a Spring Boot application — the team’s existing Spring knowledge, tooling, testing, and operational practice all apply unchanged, which is a large part of why Java and Spring are the recommended stack. And the `@Deployment` annotation deploys the process models found on the classpath when the application starts — convenient for development, but a practice to reconsider for production, where, as Chapter 36 argued, deployment should be a governed pipeline step rather than a side effect of an application booting. The annotation is a good servant in a development loop and a questionable one in production; the team should decide deliberately.

Figure 92.1 draws the application, the starter, and the two directions of its relationship with the cluster.

92.5 The Java discipline for a Camunda platform

The Java code behind a Camunda platform is production code in a regulated institution, and it must be held to production standards: code review, static analysis, dependency scanning, and the testing disciplines this part describes. A worker that “works” but has not been reviewed, tested, and scanned is technical debt from the moment it is deployed, and in a regulated institution technical debt is also regulatory debt — a regulator who asks “how do you ensure the quality of your automation code” expects an answer, and “it works in the demo” is not one.

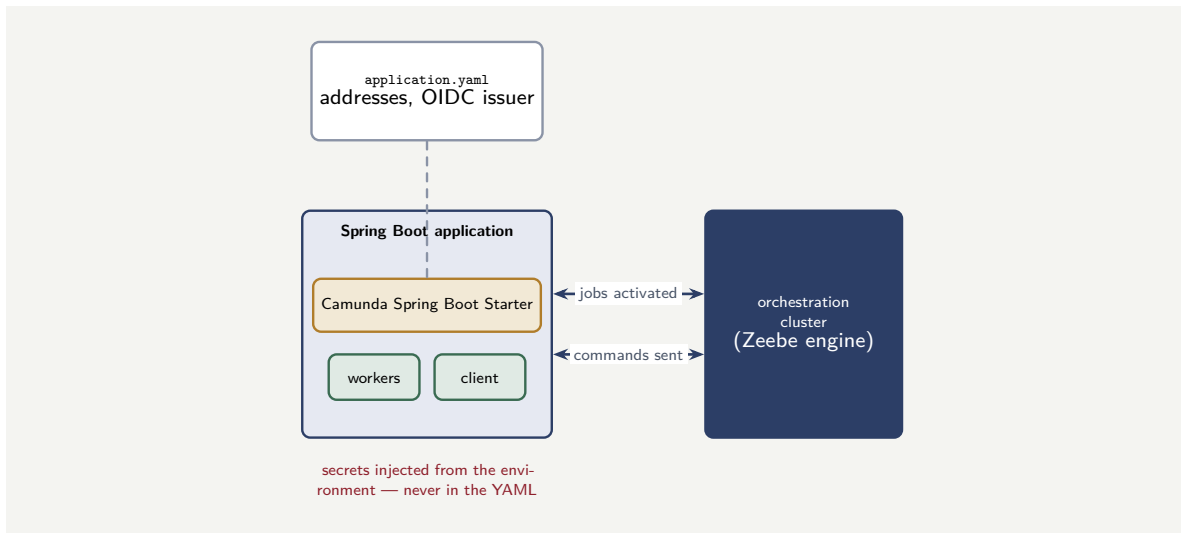


Figure 92.1. A Camunda Java application and its connection to the cluster. The Spring Boot Starter, configured from `application.yaml`, wires the application’s workers and client to the orchestration cluster — workers receive jobs, the client sends commands. The non-secret configuration is in the YAML; the credentials are injected from the environment.

Worked example: the cost of a missing compiler flag

Problem. A team builds a Camunda Java service with twelve job workers, each binding several process variables to method parameters by name. The build does not enable the `-parameters` compiler flag. The service compiles cleanly, starts cleanly, and connects to the cluster — but every worker fails at runtime, because the parameter names needed for variable binding were not retained in the bytecode. The team spends half a day, across three engineers, diagnosing it before finding the one-line fix. Their loaded cost is \$95 an engineer-hour. What did the missing flag cost, and what is the lesson beyond the number?

Setup. The direct cost is the diagnosis time — half a day across three engineers — at the loaded hourly rate. We compute it, then consider why the failure was disproportionately hard to diagnose.

Working. Diagnosis effort: half a working day is 4 hours, across 3 engineers:

$$4 \times 3 = 12 \text{ engineer-hours.}$$

At \$95 an hour:

$$12 \times \$95 = \$1,140 \text{ of diagnosis time.}$$

Discussion. The \$1,140 is real but modest; the lesson is in *why* a one-line build setting consumed twelve engineer-hours. The failure had every property that makes a defect expensive to diagnose. It was *silent at every stage the team naturally checks*: the code compiled, the application started, the connection to the cluster succeeded — every green light was green. It surfaced only at the moment a worker actually tried to bind a variable, far from its cause. And its cause — a missing compiler flag — is not in the application’s own code, where an engineer looks first, but in the build configuration, where an engineer looks last. The combination — silent until late, and rooted in the build rather than the code — is exactly the profile of a defect that consumes a disproportionate amount of time relative to the triviality of its fix. The lesson is the value of knowing the toolchain’s non-obvious requirements *before* they bite: the `-parameters` flag, the secrets kept out of YAML, the artifact versions confirmed against the

target platform. None of these is intellectually hard; all of them, omitted, produce a confusing failure; and a team that sets up its Camunda Java toolchain from a known checklist pays the few minutes that saves the twelve hours.

Practical next steps

If your team builds on Camunda in Java, audit one project's build configuration against the toolchain requirements: is the `-parameters` flag enabled, is the starter version confirmed against your target platform, are secrets kept out of every configuration file? These checks take minutes and pre-empt exactly the silent, late, hard-to-diagnose failures this chapter's worked example described.

Chapter in brief

Java with Spring Boot is the default stack for building on Camunda, and this part is the deep technical guide to that build. The Camunda Spring Boot Starter wires a Spring Boot application to the orchestration cluster — artifact names and versions move between releases and should be confirmed against the target documentation, but the shape is stable: a starter dependency, a YAML configuration, annotated workers. The `-parameters` compiler flag must be enabled, or by-name variable binding fails. The cluster connection lives in `application.yaml`, externalised per environment, with secrets injected from a secrets service, never committed. A Camunda Java application is an ordinary Spring Boot application, which is much of why the stack is recommended.

Building Job Workers in Java

93.1 From pattern to production code

Chapter 32 established the job worker as a pattern and showed a short example; Chapter 41 treated how a worker must handle failure. This chapter treats the job worker as *Java code* — how it is written, in detail, so that it is correct, testable, and fit for a bank's production. The worker is where most of a Camunda platform's bespoke code lives, and writing workers well is the single most exercised Java skill on the platform.

93.2 The annotated worker method

A job worker, in a Camunda Spring Boot application, is a method — on an ordinary Spring component — annotated to declare which job type it handles. The starter's machinery does the rest: it activates jobs of that type from the cluster, invokes the method, and — on a clean return — completes the job. Listing 93.1 shows a worker written with the care a production worker needs.

Listing 93.1. A production-shaped job worker. The annotation declares the job type; `@Variable` binds named process variables to parameters; the real work is delegated to a service; the return value becomes the job's output variables.

```

1  @Component
2  public class ReserveFundsWorker {
3
4      private static final Logger log =
5          LoggerFactory.getLogger(ReserveFundsWorker.class);
6
7      private final CoreBankingService coreBanking;
8
9      public ReserveFundsWorker(CoreBankingService coreBanking) {
10         this.coreBanking = coreBanking; // injected, testable
11     }
12
13     @JobWorker(type = "reserve-funds")
14     public Map<String, Object> reserveFunds(
15         @Variable String accountRef,
16         @Variable long amountMinor,
17         @Variable String processInstanceRef) {
18
19         log.info("reserve-funds for {}", processInstanceRef);
20
21         // the idempotency key -- derived from the instance,
22         // stable across retries of the same job
23         String idempotencyKey = "reserve-" + processInstanceRef;
24
25         ReserveResult result = coreBanking.reserve(
26             accountRef, amountMinor, idempotencyKey);
27
28         // the returned map becomes the job's output variables
29         return Map.of(
30             "reservationId", result.id(),
31             "reserved", true);

```

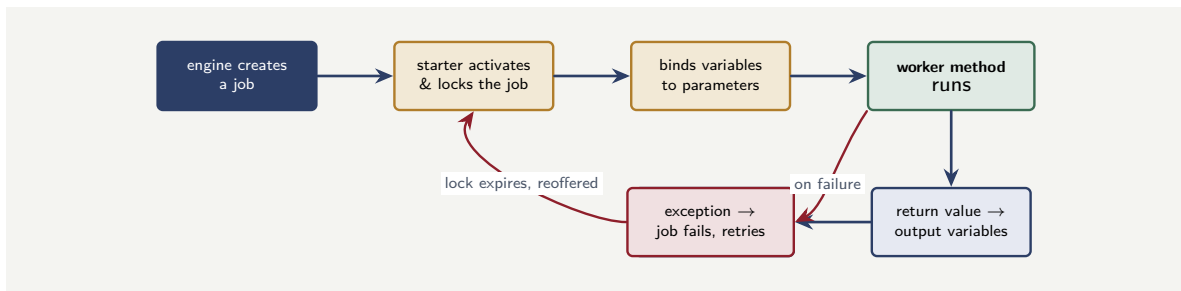


Figure 93.1. The life of a job, from the worker’s side. The engine creates a job; the starter activates and locks it, binds its variables to the method’s parameters, and invokes the worker method; a clean return becomes the job’s output variables and the engine completes it. An exception fails the job, which — after its lock expires — is reoffered, which is why the worker must be idempotent.

```

32     }
33 }

```

That listing is short, but every line of it reflects a discipline from earlier in the manuscript, and walking through them is the point of the chapter.

The worker is *thin*. It reads its inputs, calls a `CoreBankingService` to do the actual work, and returns its outputs. The business logic — how a reservation is actually made — is in that service class, not in the worker. This is [Chapter 32](#)’s thin-worker discipline, and it has a concrete payoff the next chapter draws on: the service class, free of any engine dependency, is unit-testable on its own.

The dependency is *injected* through the constructor, not constructed inside the worker. This is ordinary Spring practice, and it is what makes the worker testable — a test can supply a stand-in `CoreBankingService`, as [Chapter 95](#) will show.

The variable access is *disciplined*. The `@Variable` annotation binds named process variables to typed method parameters — the data-flow discipline of [Chapter 3](#) expressed in Java. The worker declares exactly the variables it needs, with their types; it does not reach into an untyped bag of all process state.

The work is *idempotent*. The worker derives an idempotency key from the process instance — stable across every retry of this job — and passes it to the core-banking call. This is the discipline of [Chapter 16](#), [Chapter 40](#), and [Chapter 41](#), realised in the one line that derives the key. The at-least-once delivery of jobs makes this not optional.

Figure 93.1 draws the job’s life from activation to completion, including the retry loop that makes idempotency mandatory.

93.3 Reading and writing process variables

The worker’s interface to the process instance’s data deserves its own treatment, because it is where a subtle class of defect enters.

The recommended way to read variables is the `@Variable` annotation shown above: each named variable is bound to a typed parameter, and the binding is explicit, visible in the method signature, and checked. A worker that instead takes the whole `ActivatedJob` and reaches into it for variables by string key is a worker whose data dependencies are hidden in its body rather than declared in its signature — the hidden-coupling failure of [Chapter 3](#), in Java form.

For a worker that needs several related variables, binding them to a typed object is cleaner

still: a small Java record, its fields named for the process variables, populated by the starter from the instance's data. The worker then receives one typed, validated object rather than a handful of loose parameters, and the record's type *is* the worker's input contract, written down.

Writing variables back follows the same discipline in reverse. The worker returns a map — or a typed object — of output variables, and those, and only those, are merged into the process instance. The worker should return exactly the variables the downstream model needs, named as the model expects. A worker that writes back a large, vague bag of state is the data-minimisation failure of Part IV met at the code boundary.

A worker that reads variables by reaching into the raw job for string keys, and writes back an undifferentiated bag of state, has hidden its data contract inside its code. The next person to change the process cannot see, from the worker's signature, what it consumes and produces — and will couple something to it by accident. Bind inputs with `@Variable` or to a typed record, and return exactly the named outputs the model needs. The worker's signature should *be* its data contract.

93.4 Failure handling in the worker method

Chapter 41 set out the taxonomy of failure — transient, permanent, business error — and the responses. This chapter shows the Java.

A *business error* is raised by throwing the dedicated BPMN-error type, with an error code the process model catches. A *transient technical failure* is, in the Spring worker model, handled by letting the exception propagate — the starter then fails the job, and the engine, respecting the job's retry configuration, reoffers it. A *permanent technical failure* is one where retrying is pointless, and the worker signals it by failing the job with the retries set to zero, so an incident is raised at once rather than after pointless attempts.

Listing 93.2 shows the three handled explicitly — the explicit classification **Chapter 41** insisted on.

Listing 93.2. Explicit failure classification in a worker method. A business outcome becomes a BPMN error; a permanent fault fails the job with no retries; a transient fault is allowed to propagate for the engine to retry.

```
1 @JobWorker(type = "verify-identity", autoComplete = false)
2 public void verifyIdentity(ActivatedJob job, JobClient client) {
3     try {
4         IdentityResult r = identityService.verify(
5             job.getVariablesAsType(IdentityRequest.class));
6
7         if (!r.verified()) {
8             // a business outcome -- the model catches and routes it
9             throw new BpmnError("IDENTITY_NOT_VERIFIED",
10                                 r.reason());
11         }
12         client.newCompleteCommand(job)
13             .variables(Map.of("identityVerified", true))
14             .send().join();
15     } catch (MalformedRequestException permanent) {
16         // retrying cannot fix a malformed request -- incident now
17         client.newFailCommand(job).retries(0)
18             .errorMessage("malformed: " + permanent.getMessage())
19             .send().join();
20     }
```



```

21     }
22     // a transient exception (timeout, outage) is left to propagate:
23     // the starter fails the job, the engine retries per its config
24 }

```

93.5 Worker configuration that matters

A worker has configuration — some on the annotation, some in `application.yaml` — and a few settings have enough operational consequence to name here, each connecting to a discipline established earlier.

The *job type* string is the contract between the model and the worker: the service task in the BPMN model declares a type, and the worker declares the same type. A mismatch — a typo, a renamed task — means jobs pile up unworked and the process silently stalls. The job type is a contract, and like any contract it must be kept consistent on both sides.

The *timeout* — the job's lock duration — must exceed the worst-case handling time of the worker, with margin, as [Chapter 32](#)'s worked example established. Set near the mean, it expires on every slow job; set far above the tail, a genuinely dead worker's job is reoffered promptly.

The *number of jobs activated* and the *number of worker threads* together set the worker's concurrency, and they are how the fleet of [Chapter 32](#) is realised in one application: more threads and a larger activation batch raise throughput at the cost of memory and in-flight work.

93.6 The worker and its variables: a typed contract

A theme of [Chapter 3](#) returns here with a concrete Java expression worth dwelling on, because it is where a worker's maintainability is decided.

A worker's relationship to process variables can be written two ways, and they look similar but age very differently. The loose way takes the raw `ActivatedJob` and pulls variables out of it by string key inside the method body — `job.getVariablesAsMap().get("accountRef")` and so on. It compiles, it runs, and it is a quiet liability: the worker's data dependencies are now invisible, scattered through its body as string literals, and the next person to change the process has no way to see, from the worker's signature, what it consumes. A typo in a key is found at runtime, not at compile time. A renamed variable breaks the worker silently.

The disciplined way binds the variables the worker needs to a typed Java record — a small class whose fields are named for the process variables — and lets the starter populate it. The worker method then receives one typed object, and that object's type *is* the worker's input contract, written down, compiler-checked, and visible at a glance. A field renamed in the process is a compile error in the record, found immediately, not a silent runtime failure found by a customer. The record costs a few lines to declare; it buys a worker whose data contract cannot drift unnoticed.

The same holds in reverse for outputs. A worker that returns a typed object, its fields named for the output variables the model expects, has a checked, visible output contract; a worker that assembles an untyped map of loosely-named keys has an output contract that exists only in the programmer's memory and the model's hope. The worker's signature — its typed input and its typed output — should *be* its contract with the process, and a worker written that way is one a stranger can safely change.

Tip

Bind a worker's inputs and outputs to typed Java records, not loose maps of string keys. The record's type becomes the worker's data contract — visible in the signature, checked by the compiler, and impossible to drift unnoticed. A process variable renamed becomes a compile error found in seconds, rather than a silent runtime failure found, eventually, by a customer.

Worked example: a worker that returns too much

Problem. A document-extraction worker reads a customer's uploaded documents and returns the extracted fields. As written, it returns the extracted fields *and*, for convenience, the full raw text of every document — on average 90 kilobytes of text per instance — as process variables. The process runs 400,000 times a year. The raw text is never read by any later step in the model. Process variables are persisted in the engine's state and carried with the instance. Estimate the unnecessary data the worker writes into the engine across a year, and explain the harms beyond the raw volume.

Setup. The unnecessary data is the raw text the worker returns but no step consumes, times the annual instance count. We compute the volume and then consider the non-volume harms.

Working. Unnecessary data per instance: 90 kilobytes. Across 400,000 instances a year:

$$400,000 \times 90 \text{ KB} = 36,000,000 \text{ KB} = 36 \text{ GB a year}$$

of document text written into the engine's state as process variables that no step in any model ever reads.

Discussion. Thirty-six gigabytes a year of dead weight in the engine's state is the visible harm, and on its own it is reason enough to fix the worker — it bloats the engine's storage, slows every operation that loads or persists an instance, and enlarges every backup. But the volume is not the worst of it. Recall the data-minimisation principle of Part IV's security chapter: process variables are carried with the instance, included in its state, and visible wherever the instance is. Ninety kilobytes of raw customer document text, per instance, carried in process variables, is a large body of sensitive personal data spread far wider than it needs to be — into the engine's state store, into Operate's view of the instance, into every backup, into the audit trail. The worker that returned the raw text “for convenience” has, without intending to, taken sensitive data the institution holds in one controlled place — the document store — and copied it into many less controlled ones. The fix is the discipline of this chapter: a worker returns *exactly* the named outputs the model needs — here, the extracted fields — and nothing else. The raw document stays in the document store, referenced by an identifier if the process needs to reach it again. “Return it in case it is useful” is how a worker quietly becomes both a performance problem and a data-protection one.

Practical next steps

Take one worker in your platform and list what it returns. For each returned variable, find the step in the model that consumes it. Any variable no step consumes is dead weight the worker should not return — and if that variable carries customer data, it is also a data-minimisation failure. The worker's outputs should be exactly what the model reads, and confirming that is a quick, high-value audit.

Chapter in brief

The job worker is where most of a Camunda platform's bespoke Java lives. A worker is an annotated method on a Spring component: the annotation declares the job type, `@Variable` binds named process variables to typed parameters, and the return value becomes the job's outputs. A well-written worker is thin — real logic in an injected, testable service — accesses variables by explicit typed binding, and derives an idempotency key for any external effect. Failure is classified explicitly: a business error is thrown as a BPMN error, a permanent fault fails the job with zero retries, a transient fault is left to propagate for the engine to retry. A worker returns exactly the named outputs the model needs — no convenience extras.

CHAPTER 94

The Java Client: Driving the Engine from Code

94.1 The other direction

[Chapter 93](#) treated the worker — code the *engine* calls. This chapter treats the *client* — code that *calls the engine*: that deploys models, starts process instances, publishes messages, queries state, completes user tasks. [Chapter 36](#) and [Chapter 38](#) described these operations as REST APIs; this chapter shows them as Java, because a Java application interacts with the cluster not by hand-assembling HTTP but through a client object the starter provides.

The two directions — worker and client — are the whole of a Java application’s relationship with the engine. The worker receives work; the client issues commands and questions. A real application is usually both: a lending service runs workers for its automated steps *and* acts as a client to start instances when applications arrive.

Figure [94.1](#) draws the two directions: the engine calling in to workers, and the application calling out as a client.

94.2 Obtaining and using the client

In a Spring Boot application built on the starter, the client is a Spring bean: the application declares it as a dependency and the starter supplies it, configured and authenticated from the same `application.yaml` of [Chapter 92](#). The application code does not build a connection or manage a token — it injects the client and calls methods on it.

The client’s calls follow a consistent *fluent* shape: a command is built up by chained method calls naming its parameters, and then sent. Listing [94.1](#) shows the operation at the heart of most client code — starting a process instance, the Java form of [Chapter 21](#)’s explicit start.

Listing 94.1. Starting a process instance through the Java client. The client is an injected Spring bean; the command is built fluently and sent; the returned event carries the new instance’s key.

```
1 @Service
2 public class LendingApplicationService {
3
4     private final CamundaClient client; // injected by the starter
5
6     public LendingApplicationService(CamundaClient client) {
7         this.client = client;
8     }
9
10    public long startLendingProcess(LendingApplication app) {
11        var event = client.newCreateInstanceCommand()
12            .bpmnProcessId("lending-origination")
13            .latestVersion()
14            .variables(Map.of(
15                "applicationRef", app.reference(),
16                "channel", app.channel(),
17                "requestedAmount", app.amountMinor()))
```

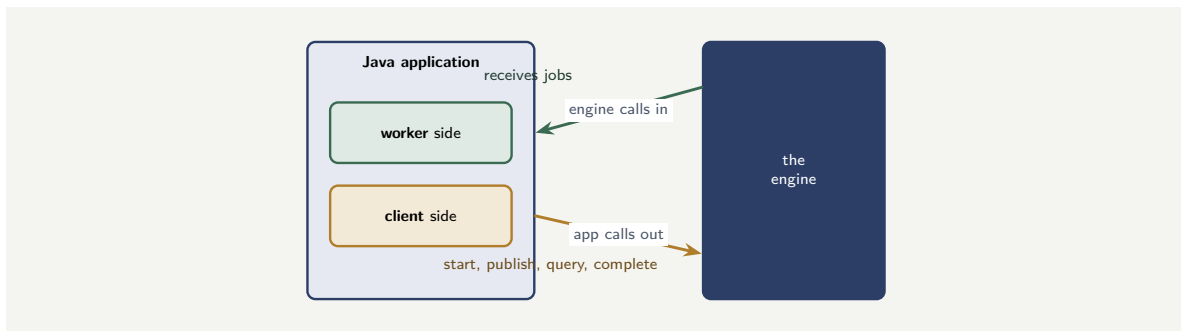


Figure 94.1. The two directions of a Java application’s relationship with the engine. On the worker side, the engine calls in — the application receives and performs jobs. On the client side, the application calls out — starting instances, publishing messages, querying state, completing tasks. A real application is usually both at once.

```

18     .send()
19     .join(); // wait for the command to be acknowledged
20
21     return event.getProcessInstanceKey();
22 }
23 }

```

Two details of that listing carry the chapter’s lessons.

The command ends with `.send()` and then `.join()`. `.send()` dispatches the command and returns immediately, with a future; `.join()` waits for that future to resolve. The separation is deliberate and important: a client that needs to issue many commands can `.send()` them all and then await them, rather than blocking on each in turn. The simple code joins immediately; high-throughput code need not.

The command names `.latestVersion()`. This is the [Chapter 37](#) version decision in Java: `latestVersion` starts the newest deployed version, which is usually right; a specific version can be named instead where the caller must pin the model logic. The choice is explicit in the code, as it should be.

94.3 Starting and awaiting a result

[Chapter 37](#) drew the distinction between starting a process fire-and-forget and starting it and awaiting its outcome. In Java, the distinction is one method on the command: a plain create-instance command returns when the instance is *created*; a create-instance command with the await-result option returns when the instance *completes*, carrying its final variables.

The discipline of [Chapter 37](#) applies unchanged and bears restating in the Java context because the await-result method is so easy to call: it is correct *only* for genuinely short, bounded processes. A Java service that calls the await-result command on a long-running process holds a thread blocked for the whole life of that process — and threads are a finite resource. A handful of such calls against slow processes can exhaust the service’s thread pool and stall it entirely. The await-result command is a sharp tool, and [Chapter 37](#)’s rule — short, bounded processes only — is, in Java, also a rule about not exhausting the thread pool.

94.4 Querying the engine

A client also *asks* the engine things — the query operations of [Chapter 36](#) and [Chapter 38](#). In Java these are fluent commands too: a query is built up with filter and paging criteria and sent, and the result is a typed page of items.

The disciplines are those [Chapter 36](#) established, now as Java concerns. A query must *page* — the Java client returns a page, and the application must loop, requesting successive pages, never assuming one call returns everything. A bulk query should be *stably ordered* and *resumable*. And a query is *authorized* — the client’s identity, from its configured credentials, determines what the query may see. A Java application that loads “all instances” into a list in memory works in a test against a hundred instances and exhausts its heap in production against a million; the paging discipline is not optional politeness but a correctness requirement.

Tip

The Java client’s query methods return a *page*, not a complete result set. Always loop over pages with explicit, stable, resumable paging — never accumulate “all instances” into one in-memory list. The query that works in a test against a hundred instances is the query that exhausts the heap in production against a million, and the failure arrives at the worst time, in a bulk job, under load.

Worked example: the thread-pool exhaustion of a blocking client

Problem. A Java service exposes a synchronous REST endpoint that, for each incoming request, starts a Camunda process with the *await-result* command and returns the process’s outcome. The service’s thread pool has 200 threads. The process being started is believed to be quick, but in fact, when a downstream system is slow, it can take up to 90 seconds. During one such slow period, requests arrive at 5 per second. How long does it take for the slow process to exhaust the entire thread pool, and what does the service do then?

Setup. Each incoming request, under the *await-result* design, occupies one thread for the whole duration of the process — up to 90 seconds during the slow period. Threads are occupied at the arrival rate and released only as processes complete. We find when occupied threads reach the pool size of 200.

Working. During the slow period, a request occupies its thread for the full 90 seconds. Requests arrive at 5 per second, so threads are occupied at:

5 threads per second.

No thread is released for the first 90 seconds — the first process has not finished. Threads occupied reach the pool size of 200 after:

$$\frac{200}{5} = 40 \text{ seconds.}$$

After 40 seconds, every one of the 200 threads is blocked inside an *await-result* call, waiting on a slow process. The service can accept no further requests — new requests have no thread to run on. The service is, for practical purposes, down: not crashed, but unable to serve anything, including its health checks and any unrelated endpoint sharing the pool.

Discussion. In forty seconds, a slow downstream system has taken down a service that never failed and was never overloaded in any ordinary sense — and the worked example exists to show that the cause is a design decision, not bad luck. The `await-result` command is the synchronous-facade pattern of [Chapter 44](#), and [Chapter 37](#) stated its rule plainly: it is for genuinely short, bounded processes only. This service broke the rule — it put the `await-result` command in front of a process that was usually quick but not *bounded*, with a 90-second tail — and the Java thread pool is where breaking the rule was punished. Each blocked `await-result` call holds a thread hostage for the process’s whole duration; a modest arrival rate against a long tail fills the pool; and a full pool is a dead service. The deeper lesson is that “usually fast” is not the test — [Chapter 37](#)’s rule says *bounded*, and a process with a 90-second tail is not bounded however good its average. The correct design starts the process fire-and-forget, returns a provisional “received” response immediately, freeing the thread, and delivers the eventual outcome asynchronously — a callback, an event, a status the client polls. The thread pool is a finite resource, and a blocking client against an unbounded process is a design that spends that resource faster than it returns it.

94.5 Camunda among the frameworks a Java team already runs

The chapters so far built the Java of a Camunda platform with the Camunda client and the Spring Boot starter. But a Java team integrating Camunda rarely works on a blank canvas: it already runs an estate of established Java frameworks, and a process platform must sit *along-side* them, not ask the team to abandon them. This chapter treats three of the most common — Spring Batch, Apache Camel, and the scheduling frameworks — and shows how each meets Camunda.

The unifying principle is the one the whole part has rested on: the job worker is the seam. A worker is ordinary Java, running in an ordinary Java application, and so it can call — or be called by — any other Java framework in that application. Integrating Camunda with the Java ecosystem is, in almost every case, a matter of wiring one of these frameworks to a job worker.

94.6 Spring Batch: heavy batch work behind a process step

Spring Batch is the established Java framework for large-scale batch processing — reading, processing, and writing very large volumes of records, with chunking, restartability, and a job-and-step model of its own. Banking and insurance are full of such work: end-of-day reconciliation, interest accrual across millions of accounts, regulatory-extract generation.

The integration question is how a Camunda process and a Spring Batch job relate. The answer follows from what each is *for*. Camunda orchestrates the *business process* — the sequence, the decisions, the human steps. Spring Batch executes *bulk record processing*. They are not competitors; they are different tools, and the sound design uses each for its purpose: the Camunda process reaches a step whose work is a bulk batch run, and that step *launches a Spring Batch job*.

Concretely, the batch step in the BPMN model is a service task, and its job worker launches the Spring Batch job through Spring Batch’s `JobLauncher`. [Listing 94.2](#) is that worker.

[Listing 94.2.](#) A job worker launching a Spring Batch job. The Camunda process orchestrates; the worker hands the bulk work to Spring Batch and reports the result back.


```

1  @Component
2  public class ReconciliationWorker {
3
4      private final JobLauncher jobLauncher;
5      private final Job reconciliationJob;
6
7      public ReconciliationWorker(JobLauncher launcher,
8          @Qualifier("reconciliationJob") Job job) {
9          this.jobLauncher = launcher;
10         this.reconciliationJob = job;
11     }
12
13     @JobWorker(type = "run-reconciliation-batch")
14     public Map<String, Object> run(ActivatedJob job)
15         throws Exception {
16         // pass process data into the batch job
17         JobParameters params = new JobParametersBuilder()
18             .addString("businessDate",
19                 job.getVariable("businessDate"))
20             .addLong("runId", job.getKey())
21             .toJobParameters();
22
23         // launch the Spring Batch job
24         JobExecution exec = jobLauncher.run(
25             reconciliationJob, params);
26
27         // report the batch outcome back to the process
28         return Map.of(
29             "batchStatus", exec.getStatus().toString(),
30             "recordsRead", exec.getStepExecutions()
31                 .iterator().next().getReadCount());
32     }
33 }

```

Two design points matter. The `runId` is set from the job key, giving the batch run an idempotency anchor — a retried worker launches a run the batch framework can recognise rather than blindly re-running millions of records. And the worker reports the batch *outcome* — status, record counts — back as process variables, so the process can branch on whether the batch succeeded. The process orchestrates and decides; Spring Batch does the bulk work.

Tip

Do not rebuild bulk record processing as a Camunda multi-instance activity over a million-element collection — that makes a million process-engine operations out of work a batch framework does in one tuned job. Let Camunda orchestrate and let Spring Batch process: a service task whose worker launches a Spring Batch job, reporting the outcome back. The process keeps the decisions and the human steps; the batch framework keeps the chunking, restartability, and throughput it was built for.

94.7 Apache Camel: the integration routes beside the process

Apache Camel is the established Java integration framework — a library of *components* for talking to virtually any system or protocol, and a routing language for moving and transforming messages between them. Many institutions already run Camel as their integration layer.

Camunda and Camel meet cleanly, and in fact there is an official Camel component for Camunda. The relationship can run in either direction. A Camunda *worker* can *invoke* a Camel *route* — handing the route a message and letting Camel's components do a complex protocol

integration the process should not itself contain. Or, through the `camunda` Camel component, a Camel route can act as a *job worker* — a route declared with a `camunda://worker` endpoint consumes jobs of a type, processes them through the route, and completes them. Listing 94.3 shows the second form: a Camel route that is, itself, a Camunda job worker.

Listing 94.3. An Apache Camel route acting as a Camunda job worker: the route consumes jobs of a type, processes them through Camel’s components, and completes them.

```
1 public class DocumentArchiveRoute extends RouteBuilder {
2     @Override
3     public void configure() {
4         // the route IS a job worker: it consumes jobs
5         from("camunda://worker?jobType=archive-document")
6             .log("archiving ${header.CamelCamundaJobKey}")
7             // Camel components do the integration work
8             .setHeader("CamelFileName",
9                 simple("${header.documentId}.pdf"))
10            .to("sftp://archive@vault.example/documents")
11            // a Map body becomes the job's output vars
12            .setBody(constant(Map.of(
13                "archived", true)));
14    }
15 }
```

The route reaches an SFTP archive through Camel’s SFTP component — one of hundreds Camel provides — and because it is declared as a `camunda://worker`, the engine’s job of type `archive-document` is consumed and completed by the route. The institution’s existing Camel expertise and its existing routes become directly usable as Camunda workers; the process gains access to every protocol Camel speaks without the worker code containing any of that protocol detail.

94.8 Scheduling: starting processes on a cadence

The third framework category is *scheduling*. A workflow engine has its own timers — the timer events of Chapter 3, which fire *within* a running process. But a different need is common: starting a process *on a cadence* — a reconciliation process every night at 2 a.m., a regulatory-report process on the first of each month, a review process every quarter. The trigger here is not an event within a process; it is the clock, starting a new process instance.

This is the job of a *scheduling framework* — Spring’s own `@Scheduled` mechanism, or Quartz Scheduler for richer scheduling — and the integration is direct: a scheduled method, firing on its cadence, uses the Camunda client of Chapter 94 to start a process instance. Listing 94.4 is that integration with Spring’s scheduler.

Listing 94.4. A scheduled method starting a Camunda process on a cadence: Spring’s scheduler fires on a cron expression and uses the Camunda client to start the process.

```
1 @Component
2 public class NightlyProcessScheduler {
3
4     private final CamundaClient camunda;
5
6     public NightlyProcessScheduler(CamundaClient c) {
7         this.camunda = c;
8     }
9
10    // fire every night at 02:00
11    @Scheduled(cron = "0 0 2 * * *")
12    public void startReconciliation() {
```

```

13     camunda.newCreateInstanceCommand()
14         .bpmnProcessId("nightly-reconciliation")
15         .latestVersion()
16         .variables(Map.of(
17             "businessDate", LocalDate.now()
18                 .minusDays(1).toString()))
19         .send()
20         .join();
21     }
22 }

```

One discipline matters here, and it is a division of responsibility. The scheduler's job is *only* to start the process — to be the clock that fires. Everything after that — the steps, the error handling, the retries, the human escalations — belongs *in* the process model, where it is visible and governed. A scheduler that did more than start the process — that tried to do the work itself on a timer — would be process logic hidden in a cron job, exactly the ungoverned-logic problem the book has warned of throughout. The scheduler starts; the process does.

A scheduling framework should do exactly one thing for a Camunda platform: *start a process instance* on its cadence. The work the process then does — the steps, the decisions, the error handling — must live in the BPMN model, not in the scheduled method. A scheduled job that fetches data, calls services, and handles failures itself has become an unmodelled, ungoverned process running in a cron entry, invisible to Operate and to every reviewer. Let the scheduler be the clock, and let the process be the process.

Worked example: three frameworks, one nightly run

Problem. A bank must run a nightly account-reconciliation: at 2 a.m., reconcile every account against the core banking system, archive a reconciliation report, and — if discrepancies are found — raise a human review task. A team proposes to build the whole thing as one large Spring Batch job with the scheduling, the archiving, and the exception handling all inside it. Assess this, and describe the sound design.

Setup. We separate the work into its kinds and match each to the framework built for it.

Working. The nightly run is not one kind of work; it is four. There is a *cadence* — fire at 2 a.m. There is *bulk record processing* — reconcile every account. There is an *integration* — archive the report to a document vault. And there is a *business process with a human step* — if discrepancies are found, route a review task to a person. The team's proposal puts all four inside Spring Batch, but Spring Batch is built for only the second:

one batch job $\neq$ cadence + bulk + integration + human process.

A human review task inside a Spring Batch job has no home — Spring Batch has no concept of a worklist, an assignee, or an escalation; and the cadence and the archiving are equally not what Spring Batch is for.

Discussion. The sound design uses each framework for its purpose, and the chapter has shown all four pieces. A *scheduler* — Spring `@Scheduled` on a 2 a.m. cron — starts a Camunda *process*. That process orchestrates the run: a service task whose worker launches the *Spring Batch* reconciliation job; then a service task whose worker — or a *Camel* route — archives the report; then an exclusive gateway on the batch's discrepancy count, and where discrepancies were found, a *user task* for human review, with all the allocation and escalation a human step needs. Each framework does what it is built for: the scheduler is the clock, Spring Batch

does the bulk reconciliation, Camel does the archive integration, and Camunda orchestrates the whole and owns the human step. The team's one-big-batch-job design fails not because Spring Batch is weak but because three of the four kinds of work are not batch work at all — and the human review task, in particular, simply cannot live in a batch job. The lesson is the chapter's organising principle: Camunda does not replace the Java ecosystem, it *orchestrates* it, and a process that reaches a batch step, an integration step, or a human step should hand each to the framework — Spring Batch, Camel, the engine's own human-task machinery — built for it.

Practical next steps

Search your Java services for synchronous endpoints that start a Camunda process and await its result on the calling thread. For each, ask the [Chapter 37](#) question — is the process genuinely *bounded*, or merely usually fast? Any await-result call in front of an unbounded process is a thread-pool exhaustion waiting for the downstream system to have a slow day. Convert it to a fire-and-forget start with an asynchronous result.

Chapter in brief

The client is code that calls the engine — deploying models, starting instances, publishing messages, querying state, completing tasks — the counterpart of the worker. In a Spring Boot application the client is an injected bean, configured from the same YAML. Its commands are fluent: built by chained calls, then `.send()` to dispatch and `.join()` to await. Starting a process is the Java form of [Chapter 37](#)'s explicit start, with an explicit version choice; the await-result variant blocks a thread for the process's whole life and is, by [Chapter 37](#)'s rule, for bounded processes only — or it exhausts the thread pool. Queries return pages and must be looped, never accumulated whole in memory. The chapter closes by placing Camunda among the Java frameworks a team already runs: it does not replace them, it orchestrates them through the job-worker seam. *Spring Batch* does bulk record processing, launched from a worker through the `JobLauncher`; *Apache Camel* does integration, with a route able to be a job worker itself through the official camunda component; and a *scheduling framework* — Spring `@Scheduled`, Quartz — starts processes on a cadence. Each framework does what it is built for; the process model keeps the orchestration, the decisions, and the human steps.

Testing Camunda Processes in Java

95.1 What it means to test a process

A process model is executable, and anything executable can be wrong — a gateway condition that routes incorrectly, a worker that mishandles an input, a path that was never exercised. [Chapter 126](#) treats testing as a discipline of the whole platform; this chapter treats the concrete Java practice of testing a Camunda process, because a process that has not been tested in code is a process whose correctness is a hope.

Testing a Camunda process in Java has three layers, and a serious platform uses all three. The *unit* layer tests the worker logic in isolation. The *process* layer tests the model itself — the routing, the flow — against a test engine. The *integration* layer tests the assembled whole. This chapter takes each in turn.

Figure 95.1 draws the three layers as the pyramid they form — broad and fast at the base, narrow and heavy at the top.

95.2 Unit-testing the worker's logic

The thin-worker discipline of [Chapter 93](#) pays its first dividend here. Because a well-written worker delegates the real logic to an injected service class, that service class has no dependency on the engine, on jobs, or on process variables — it is ordinary Java — and so it is tested with ordinary unit tests. No engine, no cluster, no Camunda machinery: the `CoreBankingService` or `IdentityService` behind the worker is constructed in a test, given inputs, and its outputs asserted, exactly as any Java class is tested.

This is worth stating as a discipline because it is so easily lost. If the business logic is in the worker method itself — the fat-worker antipattern — testing it requires standing up engine machinery to deliver a job to the method. If the logic is in an injected service, testing it requires nothing but the service. The thin worker is not only cleaner architecture; it is the difference between business logic that is trivially unit-testable and business logic that is awkward to reach.

Listing 95.1 is such a test: it exercises the `ScoringService` behind a worker with an ordinary unit test — no engine, no job, no Camunda machinery at all.

Listing 95.1. A plain unit test of the service behind a worker. Because the service has no engine dependency, the test needs no Camunda machinery — it is ordinary Java testing.

```

1  @Test
2  void declinesWhenScoreBelowThreshold() {
3      // no engine, no job -- just the service
4      ScoringService service = new ScoringService(
5          new StubBureauClient(/* returns score 540 */));
6
7      ScoringResult result = service.score(

```

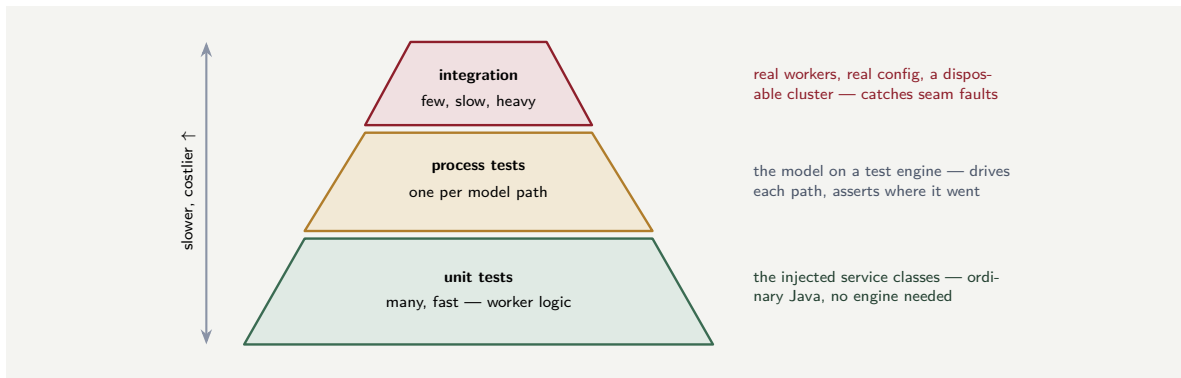


Figure 95.1. The testing pyramid for a Camunda platform. Many fast unit tests exercise the worker logic; a middle layer of process tests runs the model on a test engine, one per distinct path; a few slow, heavy integration tests assemble the real parts and catch the seam faults the lower layers cannot.

```

8      "APP-2026-1001", 15000);
9
10     // the business logic, asserted directly
11     assertThat(result.decision()).isEqualTo("DECLINE");
12     assertThat(result.band()).isEqualTo("D");
13 }

```

The test reaches the business logic directly — construct the service, give it inputs, assert its outputs — and that directness is the dividend of the thin worker. Had the scoring logic lived in the worker method, this same test would have needed a job to activate, an `ActivatedJob` to construct or mock, and engine machinery to deliver it; the thin worker reduces all of that to `new ScoringService(...)`.

95.3 Process-testing against the test engine

Unit tests confirm the worker logic; they do not confirm the *model*. To test that the BPMN model itself routes correctly — that a declined application reaches the declined end event, that a high-value claim is routed to investigation, that the boundary timer escalates — the test must run the model on an engine.

Camunda provides a *test engine* for exactly this: a lightweight engine, runnable inside a Java test, against which a process can be deployed, started, driven, and asserted. A process test, in shape, deploys the model to the test engine, starts an instance with chosen variables, drives it — completing jobs and user tasks, as the test directs — and asserts where the instance went: which path it took, which elements it visited, which end event it reached, what variables it carried.

The crucial capability of a process test is that it can *drive the process down a chosen path*. By completing a classification job with a chosen result, the test steers the gateway; by letting a timer fire, the test exercises the escalation path. This is how the path-counting of [Chapter 3](#)'s worked example becomes a test suite: each distinct path through the model is one process test, driving the instance down that path and asserting it arrives correctly. A model with four paths needs four process tests, and a model whose paths are untested is a model whose routing is unverified.

One technical note the manuscript states because it catches teams out: the test engine runs the process *asynchronously*, as the real engine does, so a test must *wait* for the process to reach the state it intends to assert before asserting it. A test that asserts immediately after starting

an instance, without waiting, asserts against a process that has not yet run — and such a test is flaky, passing or failing on timing rather than on correctness. The test engine provides the means to wait for the process to be idle; a process test must use it.

Listing 95.2. The shape of a process test. The model is deployed to the test engine, an instance is started and driven down a chosen path, and the test waits for idleness before asserting where the instance went.

```
1  @Test
2  void declinedApplicationReachesDeclinedEnd() {
3      // start an instance with variables that should route to decline
4      var instance = client.newCreateInstanceCommand()
5          .bpmnProcessId("lending-origination")
6          .latestVersion()
7          .variables(Map.of("preScreenResult", "DECLINE"))
8          .send().join();
9
10     // wait for the asynchronous engine to settle
11     engine.waitForIdleState(Duration.ofSeconds(10));
12
13     // assert the instance took the declined path and ended there
14     assertThat(instance)
15         .hasPassedElement("gateway-prescreen")
16         .hasPassedElement("end-declined")
17         .isCompleted();
18 }
```

The test engine runs a process asynchronously, just as the real engine does. A process test that asserts immediately after starting an instance — without waiting for the engine to settle — is testing a process that has not finished running. Such a test is *flaky*: it passes or fails on timing, not on correctness, and a flaky test is worse than no test, because it trains the team to ignore failures. Always wait for the idle state before asserting.

95.4 The path coverage that matters

A process-test suite's job is to cover the model's *paths*, and the discipline is the path-counting of [Chapter 3](#): enumerate every distinct route from start to end — every gateway branch, every boundary event, every exception path — and write a test that drives the instance down each.

The paths teams remember to test are the happy ones; the paths that matter most are the ones they forget. The boundary-timer escalation, the exception route, the rejection branch, the path taken only when an upstream worker raises a particular business error — these are the paths that are rare in production, hard to trigger by hand, and therefore both under-tested and, when they fire, operating on real cases for the first time. A process test can trigger them trivially — let the timer fire, complete the job with the error — and so the process test is the *only* cheap place to exercise them. A model whose exception paths are covered by process tests is a model whose rare paths have been run before a customer's case runs them.

95.5 Integration testing the assembled platform

The third layer tests the assembled whole: the real workers, the real configuration, the model, and as much of the surrounding integration as can be stood up, exercised together. Where the process test uses a test engine and test-driven jobs, an integration test runs the actual worker

code against a real — if disposable — cluster, often one started in a container for the test and discarded after.

Integration tests are slower and heavier than unit and process tests, and so a platform has few of them relative to the many fast tests below — the ordinary testing pyramid. Their distinct value is that they catch what the lower layers cannot: a job-type string that the model and the worker spell differently, a configuration that is wrong, an authentication that fails, a variable contract that the worker and the model disagree on. These are integration faults — the parts are individually correct and the seam between them is wrong — and only a test that assembles the parts finds them.

95.6 The Java discipline for a Camunda platform

The Java code behind a Camunda platform is production code in a regulated institution, and it must be held to production standards: code review, static analysis, dependency scanning, and the testing disciplines this part describes. A worker that “works” but has not been reviewed, tested, and scanned is technical debt from the moment it is deployed, and in a regulated institution technical debt is also regulatory debt — a regulator who asks “how do you ensure the quality of your automation code” expects an answer, and “it works in the demo” is not one.

Worked example: the untested exception path

Problem. A claims process model has 9 distinct paths from start to end: 6 happy-ish routing branches and 3 exception paths — a boundary-timer escalation, a fraud-investigation route, and a path taken when the valuation worker raises a business error. The team’s process-test suite has 6 tests, one per happy branch; the 3 exception paths are untested. Over the first year in production, the three exception paths are taken by, respectively, 2%, 0.5%, and 1% of the 250,000 annual claims. A defect is later found in the fraud-investigation route. How many real claims went down each untested path before any test would have exercised it, and what is the lesson?

Setup. For each untested exception path, the number of real claims that traversed it in the year is the annual volume times that path’s frequency. We total them and consider what the missing tests meant.

Working. Claims down each untested path in the year:

$$\text{timer escalation: } 250,000 \times 0.02 = 5,000,$$

$$\text{fraud investigation: } 250,000 \times 0.005 = 1,250,$$

$$\text{valuation error: } 250,000 \times 0.01 = 2,500.$$

Total real claims down an untested path in the year:

$$5,000 + 1,250 + 2,500 = 8,750.$$

Discussion. Eight thousand seven hundred and fifty real customers’ claims travelled a path that no test had ever exercised — and the fraud-investigation route, where the defect was later found, carried 1,250 of them down a path that was not merely untested but *defective*. The worked example’s lesson is about where testing effort is misdirected. The team’s six tests covered the six happy branches, and that feels like diligence — six tests, one per branch.

But the happy branches are the paths most exercised in production and therefore the ones a defect would surface in fastest anyway; the *exception* paths are rare, and rarity is precisely what makes them dangerous to leave untested — a defect on a 0.5% path may run for months before enough cases accumulate for anyone to notice, and every one of those cases is a real claim mishandled. The cost of the missing tests was not the three tests' worth of effort saved; it was 8,750 claims on unverified paths and 1,250 on a broken one. And the bitter detail is how cheap the missing tests were: a process test can trigger a boundary-timer escalation by letting the timer fire, and a business-error path by completing a job with that error — each exception path is a few lines of test. The exception paths are the hardest to trigger in production and the easiest to trigger in a process test, which is exactly why the process test is where they must be covered. A test suite that covers only the happy paths has tested the cases that would have been fine anyway and skipped the cases that needed it.

Practical next steps

Take one of your process models, enumerate its distinct paths as [Chapter 3](#) described — including every boundary event and exception route — and compare the count with the number of process tests in its suite. The gap is your untested paths, and they are almost certainly the exception paths. Each is a few lines of process test, and each is a path real customers' cases will otherwise run for the first time in production.

Chapter in brief

A Camunda process is executable and so can be wrong; testing it in Java has three layers. Unit tests exercise the worker's logic — trivial when the thin-worker discipline has kept that logic in an injected, engine-free service class. Process tests run the model on a lightweight test engine, driving an instance down a chosen path and asserting where it went; they must wait for the asynchronous engine to settle before asserting, or they are flaky. The process-test suite must cover every distinct path — and the exception paths, rare in production and easy to trigger in a test, are the ones that matter most and are most often skipped. Integration tests assemble the real parts and catch the seam faults — a mismatched job type, a wrong configuration — the lower layers cannot.

Production-Grade Java: Hardening the Implementation

96.1 From working to production-ready

The preceding chapters of this part showed how to build the Java of a Camunda platform: the toolchain, the workers, the client, the tests. A platform built from those chapters *works*. This final chapter of the part treats the gap between working and *production-ready* — the hardening that a bank’s or insurer’s platform needs and that a demonstration does not, and that is therefore the part most often skimped under delivery pressure.

The concerns here — observability, resilience, resource management, configuration discipline, graceful lifecycle — are not Camunda-specific in their principles; they are the ordinary concerns of any production Java service. But each has a specific Camunda expression, and gathering those is the purpose of this chapter.

96.2 Observability of the worker fleet

[Chapter 125](#) treats observability as a platform discipline; this section is its narrow Java counterpart — how a Java worker application is made observable.

The starter exposes *metrics* through the standard Spring mechanism — Spring Boot Actuator and Micrometer — and among them are Camunda-specific metrics that matter: counts of job invocations, broken down by job type and by outcome — activated, completed, failed, and BPMN-error. Those four outcomes, per job type, are a precise, continuous picture of the worker fleet’s health. A rising *failed* count for one job type is a downstream system in trouble; a rising *bpmn-error* count is not a fault at all but a shift in business outcomes; the distinction — [Chapter 41](#)’s taxonomy — is visible directly in the metrics if the application exposes them.

The discipline is simply to *expose and watch* them. A worker application that emits these metrics into the institution’s monitoring — the dashboards of [Chapter 125](#) — gives operations a live view of the fleet. One that does not is a fleet running blind, and the first sign of trouble is then a stuck process rather than a rising failure count seen early.

Alongside metrics, *logging* matters, and one discipline makes worker logs useful: every log line a worker emits should carry the *process-instance key* and the *job key*. [Chapter 41](#)’s end-to-end trace depends on a correlation identifier threaded through every component; in the worker, that means putting the instance key on every log line, so that the logs of a single troubled case can be gathered from across the fleet. A worker log line with no instance key is a fact with no case to attach to.

96.3 Resilience in the worker application

[Chapter 41](#) treated failure handling within a worker method; this section treats resilience of the worker *application* as a whole.

A worker application is a client of many things — the cluster, and every downstream system its workers call — and any of them can be slow or absent. Two patterns, ordinary in production Java, are worth applying deliberately.

A *timeout* on every outbound call — to a downstream system, to anything — ensures a worker thread cannot block forever on an unresponsive dependency. A worker call with no timeout is a thread that an unresponsive core banking system can capture permanently, and enough such captures stall the fleet.

A *circuit breaker* around a downstream dependency stops a worker hammering a system that is comprehensively down. When calls to a dependency fail past a threshold, the breaker *opens* — subsequent calls fail fast, without even attempting the dead dependency — and after a pause the breaker tests whether the dependency has recovered. This is the application-level counterpart of [Chapter 41](#)’s point about retry storms: the circuit breaker is what stops a worker fleet’s retries from becoming a sustained denial-of-service against a struggling downstream system.

Tip

Put a timeout on every outbound call a worker makes, and a circuit breaker around every downstream dependency. Without the timeout, one unresponsive system can capture worker threads until the fleet stalls; without the breaker, the fleet’s retries pile load onto a system that is failing under load. Both are ordinary production-Java patterns — the Camunda-specific point is only that a worker fleet, calling many systems at high volume, needs them more than most applications, not less.

A hardened worker in code

The hardening this section describes is concrete in the worker’s code. [Listing 96.1](#) is a job worker built for production: its outbound call carries a timeout, runs behind a circuit breaker, and emits the metrics the observability section requires.

Listing 96.1. A production-hardened job worker: the outbound call has a timeout, runs behind a circuit breaker, and the worker emits success and failure metrics.

```
1  @JobWorker(type = "credit-bureau-score")
2  public Map<String, Object> score(ActivatedJob job) {
3      Timer.Sample sample = Timer.start(meterRegistry);
4      try {
5          // the call runs behind a circuit breaker and
6          // carries a timeout -- it cannot block forever
7          BureauResult r = circuitBreaker.executeSupplier(
8              () -> bureauClient.score(
9                  job.getVariable("applicationRef"),
10                 Duration.ofSeconds(5))); // timeout
11
12         meterRegistry.counter("worker.score.ok").increment();
13         return Map.of("creditScore", r.score());
14     }
15     catch (CallNotPermittedException e) {
16         // breaker is open -- fail fast, retry later
17         meterRegistry.counter("worker.score.breaker")
18             .increment();
19         throw e;
20     }
21     finally {
22         sample.stop(meterRegistry.timer("worker.score.time"));
23     }
24 }
```

Graceful shutdown is the other half of a production worker, and it is configuration as much as code. Listing 96.2 shows the Spring Boot settings that let a worker application, on shutdown, stop taking new jobs and finish the ones in hand before the process exits.

Listing 96.2. Spring Boot configuration for graceful shutdown: the worker application finishes its in-flight jobs before exiting, rather than abandoning them.

```
1 # application.yaml -- graceful lifecycle
2 server:
3   shutdown: graceful
4 spring:
5   lifecycle:
6     timeout-per-shutdown-phase: 30s
7 camunda:
8   client:
9     zeebe:
10      # stop polling for new jobs on shutdown
11      job:
12        pollInterval: 100ms
```

The two listings are the section’s argument in concrete form. The worker code puts the timeout and the circuit breaker exactly where the prose places them — on the outbound call — and adds the metric counters and timer that make the worker observable, so success rate, breaker trips, and call latency are all visible. The configuration makes shutdown *graceful*: without it, a redeployment kills the worker mid-job and abandons whatever was in flight; with it, the worker stops taking new jobs and is given thirty seconds to finish those it holds. A worker that has both — hardened calls and graceful lifecycle — is one a platform team can deploy, scale, and restart without losing work.

A worker application without graceful shutdown loses every in-flight job each time it is redeployed — and a busy platform is redeployed often. The jobs are not lost permanently: Camunda’s job timeout will eventually make them available again. But “eventually” can be minutes, and in the meantime the work is stalled. Configure graceful shutdown so a redeployment finishes its in-flight jobs rather than abandoning them — it is a few lines of configuration, and without it every routine deployment is a small throughput outage.

Figure 96.1 draws the resilience patterns — timeout and circuit breaker on the call path — and the graceful-shutdown sequence.

96.4 Resource management and the worker fleet

A worker application’s resources — its threads, its connections, its memory — are finite, and Chapter 32’s fleet-tuning has a Java expression in how they are managed.

The worker’s *concurrency* — the activation batch size and the thread count of Chapter 93 — must be set with the application’s memory and the downstream systems’ capacity in mind. A worker configured to activate a huge batch and run many threads can overwhelm both its own heap and the downstream system it calls; one configured too conservatively leaves throughput on the table. Chapter 32’s worked example sized the fleet; this is where that sizing becomes concrete configuration.

Connection pools — to databases, to downstream HTTP services — must be sized to the worker’s real concurrency. A worker running 50 threads against a connection pool of 10 has

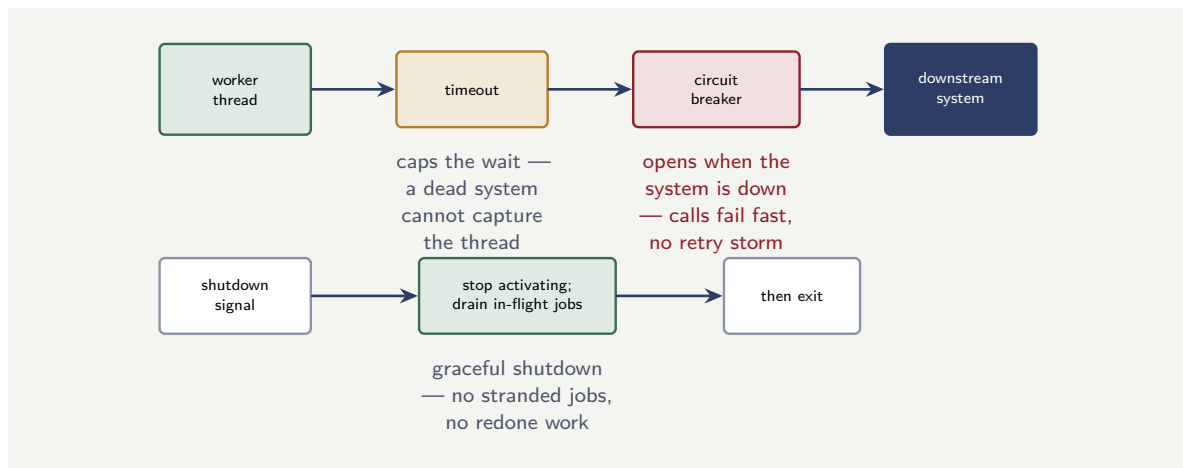


Figure 96.1. The hardening patterns of a production worker application. A timeout on every outbound call stops a dead dependency capturing a thread; a circuit breaker opens when a downstream system is comprehensively down, failing fast instead of a retry storm. Separately, graceful shutdown drains in-flight jobs before the application exits.

40 threads contending for connections, and the contention is invisible until it is a production slowdown. The pool and the worker concurrency are two numbers that must be set consistently.

96.5 Graceful lifecycle: startup and shutdown

A production worker application is deployed, redeployed, and scaled continuously — [Chapter 24](#)’s elastic platform — and how it starts and stops matters.

On *shutdown*, a worker application must stop *gracefully*: it must stop activating new jobs, finish the jobs it currently holds, report their completion, and only then exit. An application killed abruptly mid-job leaves those jobs to their lock expiry — they are reoffered and redone, which is safe only because the workers are idempotent, but is wasteful and, at scale, disruptive. Spring Boot’s graceful-shutdown mechanism, which the starter respects, is what makes shutdown orderly; it must be enabled and given enough time in the deployment configuration for in-flight jobs to drain.

On *startup*, the application should not report itself *ready* — to the platform’s health checks, to a load balancer, to an orchestrator like Kubernetes — until it has genuinely connected to the cluster and is able to work jobs. An application that reports ready before it can work is an application that a deployment system will consider successfully rolled out while it is in fact doing nothing.

Worked example: the redeployment that lost a morning's throughput

Problem. A worker application is redeployed during the working day — a routine rolling update. It is configured *without* graceful shutdown: on redeployment, each old instance is killed immediately. At the moment of redeployment the fleet of 12 application instances is holding, collectively, 360 in-flight jobs (an average of 30 each). Each killed job sits locked until its lock expires; the lock duration is configured at 15 minutes. Each job represents, on

average, 40 seconds of already-completed work that is lost and must be redone. Quantify the disruption, and identify the fix.

Setup. Every one of the 360 in-flight jobs is abandoned at kill. Each is unavailable to be redone until its 15-minute lock expires, and each then costs its 40 seconds of redone work. We compute the delay and the redundant work, and name the fix.

Working. Jobs abandoned at the abrupt kill:

$$12 \text{ instances} \times 30 \text{ jobs} = 360 \text{ jobs.}$$

Each abandoned job is stranded until its lock expires — so 360 process instances are stalled for up to

15 minutes

before their work can even resume. The redundant work, once they resume:

$$360 \times 40 \text{ s} = 14,400 \text{ s} = 4 \text{ hours}$$

of already-completed work thrown away and redone across the fleet.

Discussion. A routine redeployment stalled 360 customer cases for up to fifteen minutes each and discarded four hours of completed work — and the worked example’s point is that none of it was necessary. Nothing failed: the new version was fine, the cluster was healthy, the workers were correctly idempotent so no work was *corrupted*. The damage was purely the cost of an *abrupt* shutdown where a *graceful* one was available. With graceful shutdown, each old application instance, told to stop, would have stopped activating new jobs and spent its last seconds finishing and reporting the jobs it already held — draining its in-flight work — before exiting. The 360 jobs would have been completed by the very instances already working them; nothing would have been stranded for a lock expiry; nothing would have been redone. The fix is configuration, not code: enable Spring Boot’s graceful shutdown, and give the deployment enough termination grace period for the in-flight jobs to drain. The lesson is the chapter’s theme exactly — the application *worked* either way, and a demonstration would never have revealed the difference, because a demonstration is not redeployed under load during the working day. Production-readiness is the set of properties that matter only when the platform is run for real, continuously, at scale — and graceful shutdown is among the cheapest of them and the most visibly missed when it is absent.

Practical next steps

Check one worker application’s shutdown behaviour. Is graceful shutdown enabled, and does the deployment give it enough termination grace period to drain in-flight jobs? Redeploy it under a realistic load in a test environment and watch whether in-flight jobs drain or strand. The redeployment that loses a morning’s throughput is invisible until the first busy-hour redeployment — and that is the worst time to discover it.

Chapter in brief

The gap between a Camunda Java platform that works and one that is production-ready is hardening — the part most skimmed under delivery pressure. A worker application is made observable by exposing the starter’s job metrics, broken down by type and outcome, and by putting the process-instance key on every log line. It is made resilient by a timeout on every outbound call and a circuit breaker around every downstream

dependency. Its resources — worker concurrency, connection pools — must be sized consistently with each other and with downstream capacity. And its lifecycle must be graceful: draining in-flight jobs on shutdown, reporting ready only when genuinely able to work. These properties matter only when the platform is run for real — which is exactly why they must be built before it is.

Troubleshooting and Operating Running Instances

A production Camunda platform has, at any moment, thousands of process instances in flight. Some will get stuck — a downstream system that stopped responding, a bug in a deployed model, a data condition no one anticipated — and the operations team must know how to diagnose and resolve them without restarting the platform or losing work. This section is the operations playbook for the most common production situations.

97.1 Diagnosing a stuck instance

A production Camunda platform has, at any moment, thousands of process instances in flight. Some will get stuck — a downstream system that stopped responding, a bug in a deployed model, a data condition no one anticipated — and the operations team must know how to diagnose and resolve them.

From the Operate UI

In Operate, navigate to the process instance by its key or by filtering the instance list. The instance detail view shows every element, its state, and — crucially — the *token position*: a highlighted element where the instance is currently waiting. Operate distinguishes the states:

An element with an *active job* shows the job type and its state. If the job is active but no worker is consuming it, the worker fleet is down or misconfigured. If the job has *failed* and the retry count is zero, an *incident* has been created — Operate shows the incident with its error message, the failed element, and the timestamp.

An element waiting for a *message* shows the message name and the correlation key the instance expects. If the message has not arrived, the instance will wait indefinitely unless a timer boundary event provides a timeout.

An element waiting for a *timer* shows the timer's due date. If the timer has not fired, the instance is simply waiting — not stuck.

From the REST API

The REST API provides the same diagnostic data programmatically.

Listing 97.1. Querying an instance's state through the REST API: the response shows every active element, its type, and the incident if one exists.

```

1 # search for a specific process instance
2 curl --request POST \
3   'https://<CLUSTER>/v2/process-instances/search' \
4   --header "Authorization: Bearer ${TOKEN}" \
5   --header 'Content-Type: application/json' \
6   --data '{
7     "filter": {
8       "processInstanceKey": 2251799814000042

```

```

9     }
10  }'
11
12  # list incidents for this instance
13  curl --request POST \
14    'https://<CLUSTER>/v2/incidents/search' \
15    --header "Authorization: Bearer ${TOKEN}" \
16    --header 'Content-Type: application/json' \
17    --data '{
18      "filter": {
19        "processInstanceKey": 2251799814000042
20      }
21    }'

```

The incident search returns the error message, the failed element id, and the job key — everything needed to understand why the instance is stuck.

97.2 Moving tokens: modifying a running instance

When an instance is stuck at a step that cannot be fixed in place — a bug in the current model version, a data condition that makes the current step impossible — the operations team can *modify* the instance: move its token to a different element.

From the Operate UI

In Operate, open the stuck instance and click *Modify Instance*. The modification view shows the process diagram. Click on the element to *cancel* (the stuck one) and then click on the element to *activate* (the target — an earlier step to retry or a later step to skip). Operate previews the modification before applying it: the cancelled element turns red, the activated element turns green. Confirm to apply.

From the Java client

Listing 97.2. Moving a stuck instance's token via the Java client: activate a target element and terminate the stuck one.

```

1  zeebeClient
2      .newModifyProcessInstanceCommand(instanceKey)
3      .activateElement("manual-review")
4      .and()
5      .terminateElement(stuckElementInstanceKey)
6      .send()
7      .join();

```

From the REST API

Listing 97.3. Moving a token via the REST API: the same modification expressed as a POST request.

```

1  curl --request POST \
2    'https://<CLUSTER>/v2/process-instances/'${INSTANCE_KEY} '/modification' \
3    --header "Authorization: Bearer ${TOKEN}" \
4    --header 'Content-Type: application/json' \
5    --data '{
6      "activateInstructions": [
7        { "elementId": "manual-review" }
8      ],
9      "terminateInstructions": [
10       { "elementInstanceKey": '${ELEMENT_KEY}' }

```

```
11     ]
12     }'
```

Moving backward — to retry an earlier step — names an earlier element as the activation target. Moving forward — to skip a broken step — names a later element. The engine does not validate that the move is “sensible” — it activates whatever element is named — so the operations team must understand the model before modifying.

Every token move is a deviation from the governed process. Record the instance key, the source element, the target element, the reason, and the operator’s identity. A platform on which token moves are routine is a platform whose process models no longer describe what actually happens.

97.3 Migrating an instance to a new model version

When the fix for a stuck instance is a corrected model — a bug in the BPMN that has been fixed in a new version — the operations team can *migrate* the running instance from the old version to the new one.

From the Operate UI

In Operate, open the instance and click *Migrate Instance*. Operate shows the current model version and lets you select the target version. The UI presents the element-mapping view: each element in the current version is mapped to an element in the target version. Where element ids match between versions, the mapping is pre-populated. Where they do not, the operator must map manually. Confirm to apply the migration.

From the Java client

Listing 97.4. Migrating a running instance to a new model version via the Java client with explicit element mappings.

```
1  long newKey = zeebeClient
2      .newDeployResourceCommand()
3      .addResourceFromClasspath("lending-v2.bpmn")
4      .send().join()
5      .getProcesses().get(0)
6      .getProcessDefinitionKey();
7
8  zeebeClient
9      .newMigrateProcessInstanceCommand(instanceKey)
10     .migrationPlan(newKey)
11     .addMappingInstruction(
12         "scoreApplication", "scoreApplication")
13     .addMappingInstruction(
14         "humanReview", "humanReview")
15     .send()
16     .join();
```

From the REST API

Listing 97.5. Migrating a running instance via the REST API with a migration plan.

```
1  curl --request POST \
```

```

2 'https://<CLUSTER>/v2/process-instances/'${INSTANCE_KEY} '/migration' \
3 --header "Authorization: Bearer ${TOKEN}" \
4 --header 'Content-Type: application/json' \
5 --data '{
6   "targetProcessDefinitionKey": '${NEW_DEF_KEY}',
7   "mappingInstructions": [
8     { "sourceElementId": "scoreApplication",
9       "targetElementId": "scoreApplication" },
10    { "sourceElementId": "humanReview",
11      "targetElementId": "humanReview" }
12   ]
13 }'
```

Tip

Always test a migration plan on a non-production instance first. A mapping that is wrong — an element mapped to the wrong target, or a new element activated without the variables it needs — cannot be undone: the instance is now on the new version, in the mapped position, and a second migration is the only correction.

97.4 Cancelling and terminating an instance

An instance that should not continue — started in error, a duplicate, a case formally withdrawn — is *cancelled*. Cancellation ends the instance immediately; all active tokens are terminated, all active jobs are cancelled.

From the Operate UI

In Operate, open the instance and click *Cancel Instance*. Operate asks for confirmation. Once confirmed, the instance enters the CANCELED state. Cancellation is irreversible from the UI.

From the Java client

Listing 97.6. Cancelling a running instance via the Java client.

```

1 zeebeClient
2   .newCancelInstanceCommand(instanceKey)
3   .send()
4   .join();
```

From the REST API

Listing 97.7. Cancelling a running instance via the REST API.

```

1 curl --request POST \
2   'https://<CLUSTER>/v2/process-instances/'${INSTANCE_KEY} '/cancellation' \
3   --header "Authorization: Bearer ${TOKEN}"
```

A cancelled instance cannot be resumed. If the cancellation was a mistake, the only remedy is to start a new instance with the same data.

97.5 Publishing a message to advance a waiting instance

An instance waiting at a *message catch event* — a receive task, an intermediate message event, a message boundary event — is waiting for a specific message to arrive. If the message should have arrived and has not, the operations team can publish it directly.

From the Operate UI

Operate does not currently provide a UI for publishing messages. This operation is performed through the Java client or the REST API.

From the Java client

Listing 97.8. Publishing a message to advance a waiting instance via the Java client.

```
1 zeebeClient
2   .newPublishMessageCommand()
3   .messageName("paymentReceived")
4   .correlationKey("LOAN-2026-44817")
5   .variables(Map.of(
6       "paymentAmount", 1500,
7       "paymentDate", "2026-05-27"))
8   .timeToLive(Duration.ofMinutes(5))
9   .send()
10  .join();
```

From the REST API

Listing 97.9. Publishing a message via the REST API: the message name and correlation key must match the catch event.

```
1 curl --request POST \
2   'https://<CLUSTER>/v2/messages/publication' \
3   --header "Authorization: Bearer ${TOKEN}" \
4   --header 'Content-Type: application/json' \
5   --data '{
6     "messageName": "paymentReceived",
7     "correlationKey": "LOAN-2026-44817",
8     "variables": {
9       "paymentAmount": 1500,
10      "paymentDate": "2026-05-27"
11    },
12     "timeToLive": "PT5M"
13   }'
```

Two fields must match: the `messageName` must match the name on the catch event in the BPMN model, and the `correlationKey` must match the correlation expression the catch event evaluates. If either does not match, the message expires silently.

Tip

When publishing a message manually, verify the `messageName` and `correlationKey` against the BPMN model and the instance's variables *before* publishing. The most common mistake is a mismatched correlation key — the message expires silently and the instance remains stuck.

97.6 Resolving incidents

An *incident* is a job that has exhausted its retries and cannot proceed. The instance is stuck at the element, and Operate shows the incident with its error message.

From the Operate UI

In Operate, navigate to the Incidents view or to the affected instance. The incident shows the error message, the element, and the job details. To resolve: first fix the root cause (redploy a corrected worker, fix the downstream system, or correct the instance's variables through the Variables panel in Operate — click a variable, edit its value, and save). Then click *Resolve Incident*. Operate retries the job.

From the Java client

Listing 97.10. Resolving an incident via the Java client: fix the variables first, then resolve the incident.

```
1 // fix the variables if the root cause is bad data
2 zeebeClient
3     .newSetVariablesCommand(elementInstanceKey)
4     .variables(Map.of(
5         "accountId", "CORRECTED-ACC-12345"))
6     .send()
7     .join();
8
9 // resolve the incident -- the job retries
10 zeebeClient
11     .newResolveIncidentCommand(incidentKey)
12     .send()
13     .join();
```

From the REST API

Listing 97.11. Correcting variables and resolving an incident via the REST API.

```
1 # step 1: correct the variables
2 curl --request PUT \
3     'https://<CLUSTER>/v2/element-instances/'${ELEM_KEY} '/variables' \
4     --header "Authorization: Bearer ${TOKEN}" \
5     --header 'Content-Type: application/json' \
6     --data '{
7     "variables": {
8         "accountId": "CORRECTED-ACC-12345"
9     }
10 }'
11
12 # step 2: resolve the incident
13 curl --request POST \
14     'https://<CLUSTER>/v2/incidents/'${INCIDENT_KEY} '/resolution' \
15     --header "Authorization: Bearer ${TOKEN}"
```

The two-step pattern — fix the cause, then resolve — is deliberate. Resolving without fixing simply re-creates the incident: the job retries, hits the same error, and fails again.

97.7 The operations playbook as a governed artefact

The operations this chapter describes — moving tokens, migrating instances, cancelling, publishing messages, resolving incidents — are *powerful*, and power must be governed. Each

should be documented in a runbook, authorised by a defined role, auditable in the platform's logs, and reviewed after use.

The playbook is itself a governed artefact: version-controlled, reviewed, and updated as the platform evolves. It names, for each operation, *who* is authorised, *when* it is appropriate, *how* it is performed (from Operate, from the Java client, or from the REST API), and *what* must be recorded afterwards. A team that has this playbook and follows it operates with confidence; a team that does not operates by improvisation, and the first serious incident will expose the gap.

Chapter in brief

A production Camunda platform has thousands of instances in flight, and some will get stuck. This chapter is the operations playbook: *diagnosing* a stuck instance (missing worker, undelivered message, incident from exhausted retries); *moving tokens* with the modify-instance API to skip a broken step or retry an earlier one; *migrating* a running instance to a corrected model version with an explicit element-mapping plan; *cancelling* an instance that should not continue; *publishing a message* to advance an instance waiting at a catch event; and *resolving incidents* with the two-step fix-then-resolve pattern. Each operation is powerful and must be governed: documented in a runbook, authorised by role, auditable, and reviewed after use.

PART XV

Camunda and Low-Code Platforms

The Low-Code Landscape and Where Camunda Fits

98.1 Why low-code deserves a part

The preceding parts treated the systems a financial process integrates with — core banking, insurance, CRM, payments. This part treats a different kind of neighbour: the *low-code platform* an institution runs alongside Camunda, and the question of how they relate.

Low-code deserves a part because it is, for many institutions, a live source of architectural confusion. Low-code platforms can build applications, design forms, and — crucially — automate simple workflows, and an institution that has both a low-code platform and Camunda must answer a real question: which does what? An institution that answers it well gets the benefit of both; one that answers it badly ends up with processes scattered, duplicated, or trapped in the wrong tool — the islands of [Chapter 17](#) in a new form.

98.2 What low-code platforms are

A *low-code platform* is a platform that lets people build software — applications, forms, interfaces, simple automations — with little or no traditional programming, typically through visual, drag-and-drop tooling usable by people who are not professional developers.

Low-code platforms are a broad category. Some are general application builders. Some are *digital experience platforms* — like *Liferay* — that build portals, customer-facing sites, intranets, and the experiences around them. Some specialise in forms and data collection. What they share is the ambition to let business and citizen developers build working software fast, without the full cost of conventional development.

And — this is the point that matters for this part — most low-code platforms include some *workflow* or *process-automation* capability. They let a user define a sequence of steps, route an item for review and approval, assign work to people. Liferay, for example, includes Liferay Workflow: a drag-and-drop way to define review-and-approval workflows, assign steps to roles, and track submissions. This workflow capability is genuine and useful — and it is exactly where the relationship with Camunda must be thought through, because now *two* things in the institution can automate a process.

Figure [98.1](#) draws the overlap between a low-code platform and Camunda.

98.3 The principle: orchestration versus experience

The principle for dividing the work follows the manuscript's whole argument, and it can be stated cleanly.

A low-code platform's strength is building *applications and experiences* quickly — the forms, the portals, the interfaces, the simple departmental apps — and automating the *simple, local* workflows that belong to them: a basic review-and-approval, a single-team process, the kind

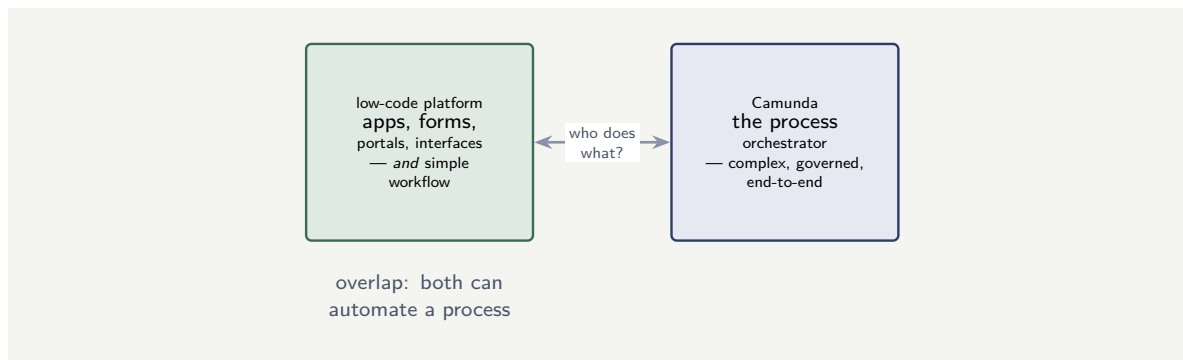


Figure 98.1. The overlap that must be resolved. Most low-code platforms include workflow capability, so an institution with both a low-code platform and Camunda has two tools that can automate a process — and must decide deliberately which does what.

of thing Liferay Workflow is explicitly designed for. For that work, the low-code platform is the right tool, and routing it through Camunda would be needless heaviness.

Camunda's strength is *orchestrating complex, governed, end-to-end business processes* — the cross-system, long-running, regulated, human-and- decision-heavy processes this whole manuscript is about. For that work, Camunda is the right tool, and building it in a low-code platform's simple workflow feature would be the island mistake of [Chapter 17](#) and the CRM-workflow mistake of [Chapter 114](#).

The dividing line, then, is *orchestration versus experience and local automation*. The low-code platform builds the experiences and automates the simple local flows; Camunda orchestrates the complex end-to-end processes. They are not rivals; they are a division of labour — and an institution that draws the line clearly gets a fast experience-building capability and a sound process orchestration capability, each doing what it is best at.

Tip

A low-code platform and Camunda are not rivals — they are a division of labour. The low-code platform builds applications, forms, portals, and experiences, and automates the *simple, local* workflows that belong to them — a basic review-and-approval, a single-team process. Camunda orchestrates the *complex, governed, cross-system, end-to-end* business processes. Draw the line at orchestration versus experience: a complex end-to-end process built in a low-code platform's simple workflow feature is an island; a basic approval routed through Camunda is needless heaviness.

98.4 The boundary between low-code and orchestration

The boundary between a low-code platform and the orchestration engine must be explicit. The low-code platform builds the user-facing experience — the portal, the forms, the self-service screens. The orchestration engine runs the process behind them. The two meet at a defined interface: the low-code application calls the process API to start a process, and the process calls back — through a webhook or an event — when it needs the user. A low-code application that embeds process logic in its own scripting has blurred the boundary; a process that renders its own UI has done the same from the other side. Keep each tool doing what it is built for.

Worked example: two automations, two tools

Problem. A bank runs both a low-code digital experience platform and Camunda. Two pieces of work need automating. Work one: an internal process where a staff member submits a request to update a page on the bank's intranet, and a single content manager reviews and approves it. Work two: the bank's mortgage-origination process — spanning the CRM, the core banking system, credit decisioning, underwriting, and funding, running for weeks, with multiple human approvals and regulatory requirements. A team proposes to build both in whichever tool the requesting department prefers. Assess this, and place each correctly.

Setup. We test each piece of work against the dividing line: orchestration of a complex end-to-end process, or a simple local automation attached to an experience.

Working. *Work one — intranet page approval.* This is a single review-and-approval, one approver, entirely within the intranet content domain — a simple, local workflow attached to an experience the low-code platform already builds. It is precisely what a low-code platform's workflow feature, such as Liferay Workflow, is designed for:

simple, local, experience-attached $\Rightarrow$ the low-code platform.

Work two — mortgage origination. This spans five systems, runs for weeks, has multiple human approvals, and carries regulatory weight — the textbook complex, governed, cross-system end-to-end process:

complex, cross-system, governed, end-to-end $\Rightarrow$ Camunda.

The team's proposal — “whichever tool the department prefers” — ignores the dividing line entirely, and would, on a department's whim, just as easily put the mortgage process in the low-code tool (an island) or the page approval in Camunda (needless heaviness).

Discussion. The team's proposal substitutes *preference* for *principle*, and the worked example shows why that fails. The two pieces of work are not similar things to be assigned by taste; they are different kinds of work, and the dividing line places each unambiguously. The intranet page approval is a simple, local, experience-attached automation — one approver, one domain — and the low-code platform, which already builds the intranet experience, automates that kind of flow quickly and well; routing it through Camunda would wrap an enterprise orchestrator around a one-step approval. Mortgage origination is the opposite in every respect — five systems, weeks long, multiple approvals, regulated — and belongs in Camunda, the orchestrator built for exactly that; building it in the low-code platform's simple workflow feature would create an island, a critical regulated process trapped in a tool not built to govern it, invisible to the enterprise's process tooling. The lesson is the chapter's principle: the choice between a low-code platform and Camunda is not a matter of departmental preference but of the *nature of the work* — experience and simple local automation to the low-code platform, complex governed orchestration to Camunda. An institution that lets the tool be chosen by preference will, sooner or later, put a process on the wrong side of the line, and the expensive version of that mistake is the regulated end-to-end process discovered, later, stranded in a low-code island.

Practical next steps

In your institution, list the workflows currently built in low-code platforms. For each, ask whether it is genuinely a simple, local, experience-attached automation or actually a complex, cross-system process. The latter are islands that belong in Camunda. And establish the

dividing line as a stated principle, so future automations are placed by the nature of the work, not by which team asked.

Chapter in brief

Low-code platforms — general app builders, digital experience platforms such as Liferay, forms tools — let business and citizen developers build software through visual tooling, and most include some workflow capability. That workflow capability creates an overlap with Camunda: an institution with both has two tools that can automate a process, and must divide the work deliberately. The principle is orchestration versus experience: the low-code platform builds applications, forms, portals, and the *simple, local* workflows attached to them, while Camunda orchestrates the *complex, governed, cross-system, end-to-end* processes. They are a division of labour, not rivals — and the choice between them is settled by the nature of the work, never by departmental preference.

Integrating Camunda with Liferay and the Experience Layer

99.1 The two layers working together

The previous chapter divided the work: the low-code platform builds the experience and the simple local automations; Camunda orchestrates the complex processes. This chapter treats how the two then *work together* — because in a well-designed institution they are not separate worlds but two layers of one architecture, and *Liferay* — a digital experience platform with a genuine Camunda partnership — is the concrete case this chapter uses.

99.2 Liferay as the experience layer over Camunda

Liferay is a digital experience platform: it builds the portals, customer sites, customer self-service interfaces, and intranets through which an institution's people and customers actually interact with it. It has its own low-code capabilities — Liferay Objects for data, drag-and-drop interface building, and Liferay Workflow for simple review-and-approval flows.

The architecture that uses Liferay and Camunda together places them as two layers. Liferay is the *experience layer*: it is what the customer or the staff member sees and interacts with — the portal, the forms, the self-service screens. Camunda is the *orchestration layer* beneath: it runs the complex end-to-end process that the experience is a window onto.

Liferay's own documentation makes the division explicit and is worth heeding: Liferay's native workflow is designed for business processes requiring basic review and approval, and for more complex business-process-management needs an institution connects Liferay to Camunda, whose processes use the BPMN and DMN standards this manuscript has built on. Liferay itself, in other words, draws the same line the previous chapter drew — simple approval in Liferay Workflow, complex BPM in Camunda — and the Camunda-Liferay partnership exists precisely to blend the low-code experience layer with enterprise-grade orchestration.

Figure 99.1 draws Liferay and Camunda as two layers.

99.3 How the two layers connect

The connection between the experience layer and the orchestration layer follows the integration patterns this manuscript has built throughout.

The experience layer *starts* processes: a customer completing a form in a Liferay portal, or a staff member submitting a request, causes a Camunda process to start — the experience layer calls Camunda, or raises an event Camunda's inbound connectors react to.

The experience layer *presents* the process's human work: when a Camunda process reaches a user task, that task is surfaced and completed through a Liferay interface — the experience layer is the worklist and the form, the orchestration layer is the process behind it. This is the completing-and-searching machinery of [Chapter 48](#) delivered through a low-code-built

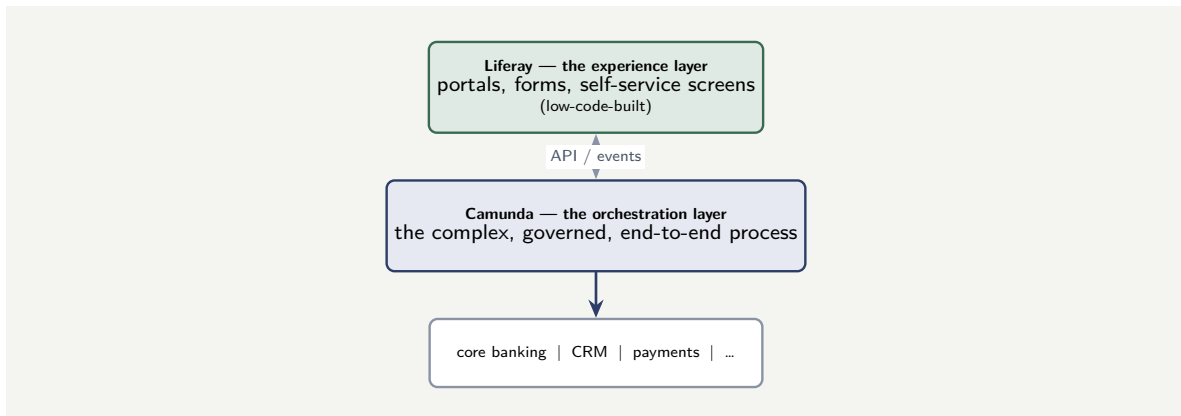


Figure 99.1. Liferay and Camunda as two layers of one architecture. Liferay, the low-code-built experience layer, is what people interact with; Camunda, the orchestration layer beneath, runs the complex end-to-end process; the process in turn integrates with the institution’s systems.

interface rather than the standard Tasklist.

And the two are connected by the ordinary mechanisms of **Part VI**: Liferay can call Camunda’s APIs, Camunda can be connected to Liferay through connector element templates, and events can pass between them. The integration is concrete and supported — but the architectural point is the one that matters: the experience layer and the orchestration layer connect through clean, defined interfaces, each remaining what it is best at, neither absorbing the other’s job.

99.4 The boundary between low-code and orchestration

The boundary between a low-code platform and the orchestration engine must be explicit. The low-code platform builds the user-facing experience — the portal, the forms, the self-service screens. The orchestration engine runs the process behind them. The two meet at a defined interface: the low-code application calls the process API to start a process, and the process calls back — through a webhook or an event — when it needs the user. A low-code application that embeds process logic in its own scripting has blurred the boundary; a process that renders its own UI has done the same from the other side. Keep each tool doing what it is built for.

Worked example: the process logic that crept into the portal

Problem. A bank builds a customer-facing lending journey. The customer-facing screens are built in Liferay; the lending process runs in Camunda. Under deadline pressure, the team starts placing decision logic in the Liferay layer: the Liferay screens themselves decide, based on the customer’s answers, which step comes next, calling individual Camunda services in the order the screens determine. The Camunda side becomes a set of disconnected service calls; the actual flow lives in the portal’s screen logic. Assess what has happened.

Setup. We consider where the lending process’s flow now lives, and what each layer is supposed to be doing.

Working. The intended architecture has Liferay as the experience layer and Camunda as the orchestration layer — Camunda runs the lending *process*, Liferay presents it. But the team

has moved the *flow logic* — which step follows which, the routing decisions — into the Liferay screens. So the lending process's actual orchestration now lives in the experience layer:

flow logic in Liferay screens $\Rightarrow$ the experience layer is doing the orchestration layer's job.

Camunda has been demoted from the orchestrator to a bag of individual services the portal calls. The complex, governed lending process — which should be one reviewable BPMN model — is now scattered across portal screen logic, invisible as a process, ungoverned, and bound to the experience layer.

Discussion. The team has dissolved the two-layer architecture by letting orchestration leak upward into the experience layer, and the worked example shows the cost. The whole value of the layering is that the lending process exists as *one thing* — one governed, reviewable, auditable BPMN model in Camunda — while Liferay provides the screens onto it. By moving the flow logic into the portal, the team destroyed that: there is no longer a lending process model anywhere; there is portal screen logic that happens to call services. The process cannot be reviewed as a process, cannot be governed or audited as one, and cannot be changed without editing the portal — and a regulated lending process that exists only as portal screen logic is precisely the island the low-code relationship is supposed to prevent, now created not by building the process in the wrong tool but by letting orchestration bleed out of the right one. The correct architecture holds the line: the lending *process* — all of its flow, its routing, its sequence — lives in the Camunda BPMN model, the orchestration layer; Liferay presents the current step's screen and sends the customer's input back, but it does not decide what comes next. The lesson is the chapter's: the experience layer and the orchestration layer must each stay what they are, and the most common way the architecture fails is not building a process in the low-code tool outright but letting the low-code experience layer gradually take over the orchestration — until the process has quietly moved into the portal and no longer exists as a governed model at all.

Practical next steps

Where your institution runs a low-code experience layer over Camunda, check where the *flow logic* lives. If the experience-layer screens are deciding which step comes next and calling Camunda services in an order they determine, orchestration has leaked upward and the process no longer exists as a governed model. Keep all flow logic in the Camunda BPMN model; let the experience layer present steps and collect input, nothing more.

Chapter in brief

Liferay and Camunda, used together, are two layers of one architecture: Liferay, the low-code-built *experience layer*, is the portals, forms, and self-service screens people interact with; Camunda, the *orchestration layer* beneath, runs the complex end-to-end process. Liferay's own guidance draws the same line — native Liferay Workflow for basic review and approval, Camunda for complex BPMN-and-DMN business processes — and the Camunda-Liferay partnership exists to blend the two. The layers connect through clean interfaces: the experience layer starts processes and presents their human work; the orchestration layer runs the process. The architecture fails when orchestration leaks upward — flow logic creeping into portal screens — until the process no longer exists as one governed model.

CHAPTER 100

Governing the Low-Code and Orchestration Estate

100.1 The estate as a whole

The first chapter divided the work between low-code platforms and Camunda; the second showed the two working as layers. This chapter treats the *governance* of the resulting estate — because an institution running low-code platforms and Camunda together has a mixed automation estate, and a mixed estate left ungoverned drifts back into exactly the fragmentation the division was meant to prevent.

100.2 The drift back to fragmentation

The danger is gradual and worth naming precisely. The division of [Chapter 98](#) is clear in principle — simple local automation to low-code, complex orchestration to Camunda — but in practice, without governance, automations get built wherever is fastest in the moment. A team under deadline builds a process in the low-code platform because they already have it open. A simple automation grows, over years, into a complex one without ever being moved. A low-code app quietly accumulates workflow logic that has become genuinely cross-system.

The result, untended, is drift: complex processes accumulating in low-code platforms where they have become islands, and the institution losing the clear picture of what runs where. The mixed estate, ungoverned, becomes the sprawl of disconnected automation that [Chapter 17](#) warned of — not because anyone decided to fragment it, but because no one governed against fragmentation.

A mixed low-code-and-Camunda estate drifts toward fragmentation unless it is actively governed. Without governance, automations get built wherever is fastest in the moment, simple low-code workflows grow into complex ones without being moved, and complex processes accumulate in low-code platforms as islands. The drift is gradual and no one decides on it — which is exactly why it needs a deliberate governance practice: periodic review of what runs where, and a defined path to move a process to Camunda when it has outgrown the low-code platform.

100.3 Governing the mixed estate

Governing a mixed automation estate rests on a few deliberate practices.

The institution needs *visibility* of the whole estate: a known, maintained picture of which automations run in low-code platforms and which in Camunda. An estate no one can see whole cannot be governed.

It needs the dividing line of [Chapter 98](#) established as a *stated standard* — not folklore — so

that new automation is placed by principle, and so there is a clear basis for saying a particular automation is in the wrong place.

It needs *periodic review*: the simple low-code workflow that has grown complex must be *noticed*, and there must be a defined path to *migrate* it to Camunda when it has outgrown its origin — a smaller instance of the migration discipline of [Part XII](#). An automation's correct home can change as it grows, and governance is what catches that.

And it needs the low-code platforms themselves brought within the institution's *governance and security* reach — the identity, access, audit, and risk disciplines of the manuscript's security treatment — because a low-code platform building real applications and automating real workflows is a real part of the institution's systems, not an unaccountable corner.

100.4 The engine as the institution's process memory

A workflow engine's state store is, in a real sense, the institution's *process memory*: it knows, for every case in progress, where it is, what has been done, what is outstanding, and what the data says. This memory survives restarts, deployments, and failures — it is durable, queryable, and auditable. An institution without a workflow engine has process memory only in the heads of the people doing the work, in spreadsheets, and in email threads — none of which survives a staff change, none of which a regulator can query.

Worked example: the low-code workflow that should have been migrated

Problem. Three years ago a bank's operations team built, in a low-code platform, a simple workflow: a single-step approval for a routine internal request, entirely within one team. It was correctly placed — a simple, local automation. Over three years, driven by small changes, it has grown: it now has nine steps, calls three other systems, involves four teams, and feeds a regulatory report. It still runs in the low-code platform. A regulatory review asks the bank to demonstrate governance over the process and the bank struggles. Assess what went wrong, given that the original placement was correct.

Setup. We consider how the workflow changed over three years and whether anything was supposed to catch the change.

Working. At its origin, the workflow was genuinely a simple, local, single-team automation, and placing it in the low-code platform was *right* by the [Chapter 98](#) principle. But over three years it crossed the dividing line: nine steps, three systems, four teams, a regulatory report — this is now a complex, cross-system, governed process, which by the same principle belongs in Camunda:

correct placement at origin + three years of growth $\Rightarrow$ now on the wrong side of the line.

Nothing was wrong with the original decision. What was missing was the *governance* to notice that the workflow had outgrown its placement and to migrate it. The workflow silently became an island, and the regulatory review found it there.

Discussion. This worked example makes the chapter's central point: the correct home of an automation is not fixed forever, and an estate governed only at the moment of creation will drift regardless of how good the original decisions were. The operations team did nothing wrong three years ago — a single-step, single-team approval genuinely belonged in the low-code platform. The failure was the absence of *ongoing* governance: no periodic review ever re-examined the workflow against the dividing line, so its slow growth from simple to complex —

nine steps, three systems, four teams, a regulatory report, accreted change by small change — was never noticed, and it was never migrated to Camunda where a complex regulated process belongs and can be governed and audited. By the time the regulator asked, the process was a low-code island, and the bank could not readily demonstrate the governance the regulator expected. The lesson is the chapter's: governing a mixed estate is not a one-time placement decision but a continuous practice — visibility of the whole estate, the dividing line as a stated standard, and above all *periodic review* with a defined migration path, so that an automation which has outgrown its origin is caught and moved. An institution that governs its mixed estate only at creation will, given a few years, accumulate exactly these islands — not through bad decisions, but through good decisions left unreviewed while the work they automated grew.

Practical next steps

Institute a periodic review of your institution's low-code automations against the dividing line — not just new ones, but existing ones, which may have grown past their original placement. For each that has become complex and cross-system, plan its migration to Camunda. The correct home of an automation changes as it grows, and only ongoing governance catches that.

Chapter in brief

An institution running low-code platforms and Camunda together has a mixed automation estate, and a mixed estate drifts back toward fragmentation unless actively governed — automations built wherever is fastest, simple workflows growing complex without being moved, complex processes accumulating in low-code platforms as islands. Governing the estate rests on deliberate practices: *visibility* of what runs where; the orchestration-versus-experience dividing line established as a *stated standard*, not folklore; *periodic review* with a defined path to migrate an automation to Camunda once it has outgrown the low-code platform; and bringing the low-code platforms within the institution's governance and security reach. The correct home of an automation changes as it grows, and only ongoing governance catches the drift.

PART XVI

Integrating with ECM and Document Management

Documents, Content, and the Process

101.1 Why content deserves a part

The preceding integration parts treated the systems a financial process exchanges *data* with — core banking, insurance, CRM, payments. This part treats a different kind of neighbour: the systems that hold the process's *documents*. It deserves a part of its own because banking and insurance processes are, to a degree often underestimated, *document-driven*, and a process platform that integrates with content badly will either drown in documents or lose them.

Consider how much of a regulated process is document. An account opening collects proof of identity and proof of address. A lending decision rests on payslips, bank statements, and a signed agreement. A claim is built from photographs, loss reports, invoices, and correspondence. A mortgage file can run to dozens of documents. These are not incidental attachments — they are the *evidence* on which a regulated decision is made, and which a regulator may later demand to see. The process and its documents are inseparable, and how the process platform relates to the document store is a first-class architectural question.

101.2 What ECM is

The systems that hold these documents are *enterprise content management* systems — ECM. An ECM system is a platform for storing, organising, securing, versioning, and managing the lifecycle of an organisation's documents and unstructured content.

An ECM system does much more than hold files. It provides *metadata* — typed properties describing each document, so a document is not just bytes but a classified, queryable record. It provides *versioning* — the history of a document as it changes. It provides *access control* — who may see or change each document. It provides *retention and disposition* — the rules governing how long a document is kept and when it is destroyed, which in a regulated institution is a legal matter, not a convenience. And it provides *lifecycle* — the states a document moves through, from draft to approved to archived.

In banking and insurance the dominant ECM systems include *IBM FileNet* — the Content Platform Engine — which a great many institutions run, often behind an IBM process platform; *OpenText* content platforms; Microsoft's *SharePoint* for some classes of content; and a range of others. What they share is the role: the ECM system is the institution's *system of record for documents*, as the core banking system is its system of record for accounts.

Figure 101.1 places the process and the content system side by side.

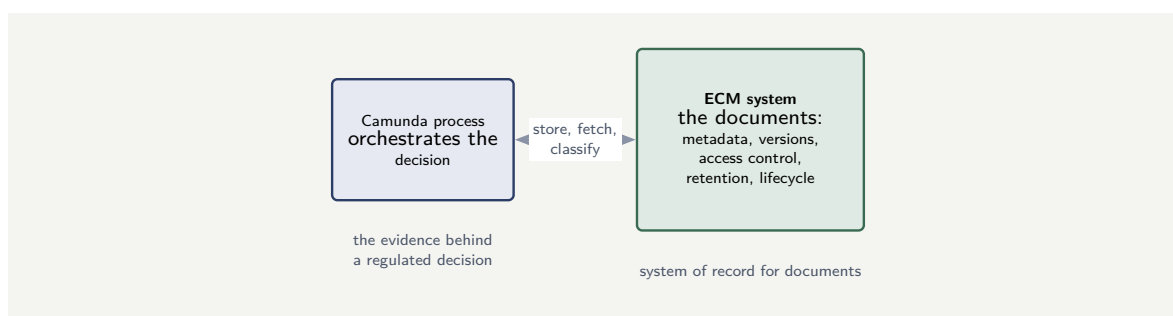


Figure 101.1. The process and the content system. The ECM system is the institution’s system of record for documents — holding metadata, versions, access control, retention rules, and lifecycle. The process orchestrates the decision; the documents that are its evidence live in the ECM system.

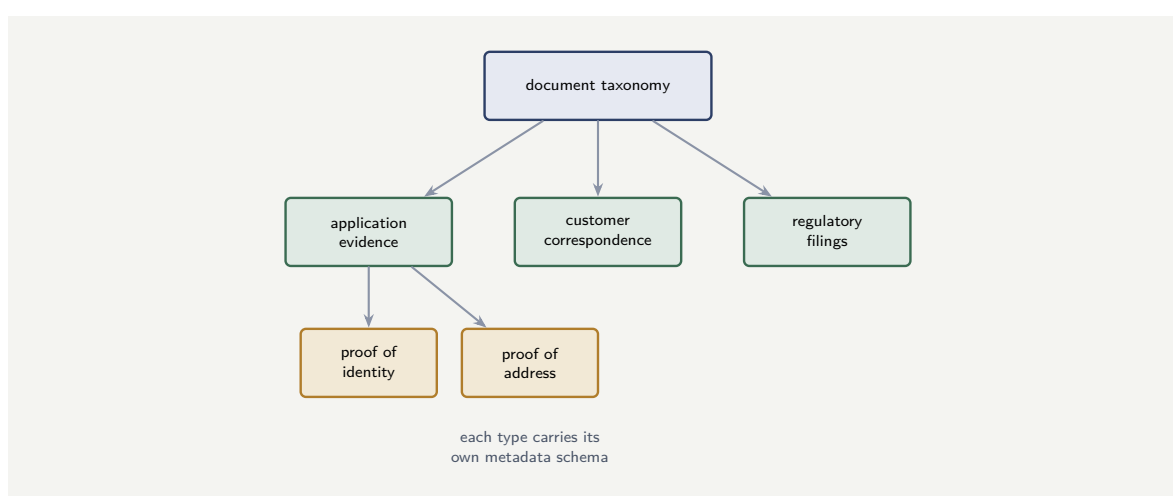


Figure 101.2. A document taxonomy: classes at the top level, types within each class, and a metadata schema governing each type. The taxonomy determines how each document is stored, indexed, retained, and disposed of.

101.3 Document taxonomy: classifying what the process handles

Before a process can manage documents, the institution must know *what kinds* of documents it manages. A document taxonomy is a structured classification of the institution’s document types — not an academic exercise but a practical necessity, because the classification determines how each document is stored, indexed, retained, protected, and disposed of.

A banking or insurance taxonomy typically has three levels. The first is the *document class*: the broad category — customer correspondence, application evidence, regulatory filings, internal memoranda. The second is the *document type* within the class: within “application evidence,” the types might be proof of identity, proof of address, proof of income, signed declarations. The third is the *metadata schema* each type carries: a proof-of-identity document carries the document’s issuing authority, its expiry date, and the identity number it evidences; a regulatory filing carries the filing period, the regulatory body, and the submission reference.

Figure 101.2 shows the three-level structure. The taxonomy is not a content-management concern alone; it is an *input to the process design*, because the process must know what it is handling. A lending process that requests “a document” is underspecified; a lending process

that requests a “proof of income (payslip or bank statement, no older than 90 days)” has named a document *type*, its acceptable subtypes, and its freshness rule — all of which come from the taxonomy.

The taxonomy also governs *retention*: how long each document type must be kept, and when it may or must be destroyed. A proof-of-identity document collected during customer onboarding may need to be retained for seven years after the relationship ends (a common anti-money-laundering requirement); an internal workflow memo may need only two years. Without a taxonomy, retention is a per-document guessing game; with one, it is a rule applied by type, enforced automatically by the ECM system.

Tip

Build the document taxonomy before building the processes that handle documents. The taxonomy defines the document types, their metadata, and their retention rules; the process designs consume those definitions. A process built without a taxonomy will invent its own ad-hoc document classifications — inconsistent across processes, un-governed, and unable to support the institution-wide retention and disposition rules a regulator expects.

101.4 The principle: the process holds the reference, not the document

One principle governs this whole part, and it can be stated now: *the process orchestrates; the ECM system holds the content; and the process variable carries a reference, not the document.*

A Camunda process that needs a customer’s identity document does not load the document’s bytes into a process variable. It carries a *document identifier* — a reference to the document in the ECM system — and fetches the content from the ECM system at the moment a step actually needs it. The document lives in one place, the ECM system built to hold it; the process carries a lightweight pointer.

The reasons are the reasons the rest of this part elaborates. A document held as a process variable is copied into the engine’s state, its history, and every backup — inflating all three with content that the ECM system already stores properly. It escapes the ECM system’s access control, versioning, and retention — the very governance a regulated institution needs. And it conflates two systems of record. The reference-not-document principle keeps each system doing its job: the engine orchestrates, the ECM system governs the content, and the two are joined by a pointer.

Tip

The governing rule of process-and-content integration: a process variable carries a document *reference*, never the document’s bytes. The document lives in the ECM system — which provides the versioning, access control, and retention a regulated institution requires — and the process fetches the content only at the step that needs it. A process that loads documents into its variables inflates its state, history, and backups with content, and escapes the ECM system’s governance.

101.5 Document governance as a regulatory obligation

In a regulated institution, document governance is not a best practice; it is a *regulatory obligation*. A bank must be able to show, for any regulated document, where it is stored, who has accessed it, how long it will be retained, and when it will be disposed of. An ECM system provides these answers; a process that stores documents in its own variables, in shared drives, or in email attachments provides none of them. The integration discipline this chapter describes — the process carries a reference, the ECM system holds and governs the content — is the only architecture that lets the institution answer a regulator’s questions about its documents.

Worked example: the process that swallowed the document store

Problem. A bank builds a claims process in Camunda. Each claim carries, on average, 15 documents — photographs, reports, correspondence — and the team, for simplicity, stores each document’s bytes directly in a process variable. A claim’s documents average 40 MB in total. The process runs 200,000 claims a year, and process data is retained for 10 years. Assess the design.

Setup. We follow where the document content goes under this design, and compare it with the reference principle.

Working. With the documents held *as process variables*, the 40 MB of content per claim is copied into the engine’s runtime state, into its history, and into every backup, and held for the 10-year retention. Over one year:

$$200,000 \text{ claims} \times 40 \text{ MB} = 8 \text{ TB of document content,}$$

propagated into the engine’s data store and history, and — across the 10-year retention — on the order of

$$10 \times 8 \text{ TB} = 80 \text{ TB}$$

of document bytes living inside the process platform’s data layer. Under the reference principle, the process variables would carry only a document identifier — a few bytes each — and the 80 TB of content would live, once, in the ECM system built to hold it.

Discussion. The team optimised for short-term simplicity and built a long-term structural problem, and the worked example puts a number on it. Storing the documents as process variables turns the Camunda engine’s data layer into a document store — one it was never designed to be — and inflates its state, its history, and its every backup with tens of terabytes of content. Worse than the volume is the governance loss: documents in process variables have escaped the ECM system’s versioning, its access control, and its retention and disposition rules, so the bank’s claims evidence is now held somewhere with none of the controls a regulator expects for it. The reference principle avoids both problems at once: the process variable carries only a document identifier, the 80 TB of content lives once in the ECM system that governs it properly, and the engine’s data layer stays light. The lesson is this part’s governing rule, and the worked example shows it is not a stylistic preference but a matter of scale and of compliance: a process that swallows its documents becomes an ungoverned document store of its own, and in a regulated institution that is both an operational and a regulatory failure.

Practical next steps

In any process that handles documents, check where the document *bytes* live. If they are held in process variables, the engine's data layer is becoming an ungoverned document store. Hold a document *reference* in the process and keep the content in the ECM system — the rest of this part shows how.

Chapter in brief

Banking and insurance processes are document-driven: the payslips, statements, photographs, and signed agreements are the *evidence* on which a regulated decision rests. Those documents live in an *enterprise content management* system — IBM FileNet, Open-Text, SharePoint, and others — which is the institution's system of record for documents, providing metadata, versioning, access control, retention and disposition, and lifecycle. The governing principle of this part is that the process *orchestrates*, the ECM system *holds the content*, and the process variable carries a *reference*, never the document's bytes — because a process that loads documents into its variables inflates its state, history, and backups with content and escapes the ECM system's governance.

Connecting Camunda to a Content Repository

102.1 From principle to mechanism

The previous chapter set the principle: the process holds a reference, the ECM system holds the content. This chapter treats the *mechanism* — how a Camunda process actually talks to a content repository, and what the integration concretely looks like.

102.2 CMIS and the repository API

A Camunda process reaches an ECM system the way it reaches any external system — through **Part VI**'s connectors and job workers — but content repositories offer a particular interface worth knowing: *CMIS*.

CMIS — Content Management Interoperability Services — is an open standard for working with content repositories. It defines a common model and a common API for the operations content work needs: creating a document, retrieving its content, reading and updating its properties, navigating folders, versioning, querying. Its value is portability: a CMIS-based integration is, in principle, not bound to one vendor's repository, because FileNet, OpenText, and other major ECM systems expose CMIS interfaces.

A Camunda integration with an ECM system therefore has two honest options. Where the repository exposes CMIS and the operations are standard, the integration speaks *CMIS* — through a connector or a job worker using a CMIS client — and gains the portability the standard offers. Where the process needs something CMIS does not cover, or the repository's native API is richer, the integration speaks the *repository's own API*. Either way the integration is a connector or job worker of the ordinary kind; content integration is not a special mechanism, it is ordinary integration pointed at a content system.

102.3 The content operations a process needs

A process's content integration is built from a small, recurring set of operations, and naming them gives the integration its shape.

The process *stores* a document: it takes content — uploaded by a customer, generated by the process — and creates it in the repository, attaching the metadata that classifies it, and receives back the *identifier* it will carry. The process *fetches* a document: given an identifier, it retrieves the content, at the step that needs it. The process *reads or updates properties*: it queries or sets the document's metadata — its type, its status, its case reference — without moving the content at all. And the process *advances lifecycle*: it moves a document through the ECM system's states — submitted, verified, approved — so the document's status reflects the process's progress.

Listing 102.1 shows the fetch operation as a job worker: given the reference the process

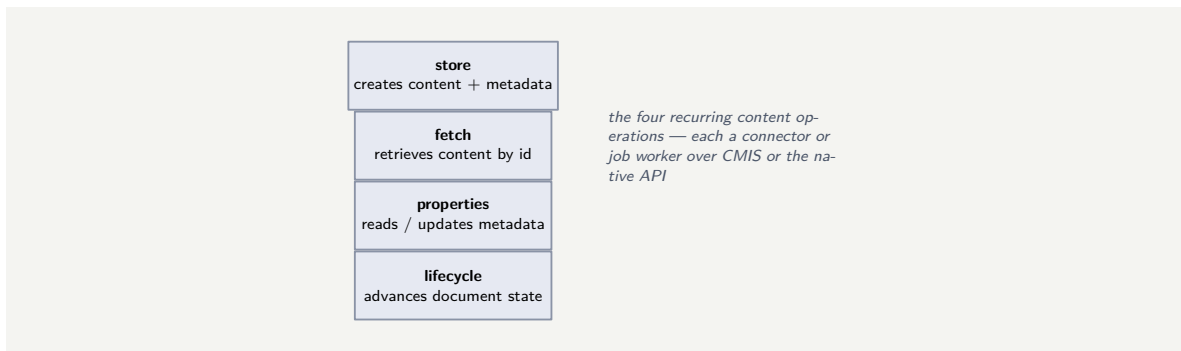


Figure 102.1. The content operations a process needs. A content integration is built from four recurring operations — store, fetch, read/update properties, and advance lifecycle — each implemented as an ordinary connector or job worker, speaking CMIS or the repository’s native API.

carries, it retrieves the content only when a step genuinely needs the bytes.

Listing 102.1. A content-fetch job worker. The process carries a document identifier; the worker retrieves the actual content from the repository only at the step that needs it.

```

1  @JobWorker(type = "fetch-document")
2  public Map<String, Object> fetch(ActivatedJob job) {
3      // the process carries only the reference
4      String documentId = job.getVariable("documentId");
5
6      // fetch the content at the step that needs it
7      Document doc = repository.getById(documentId);
8
9      return Map.of(
10         "contentBytes", doc.getContentStream(),
11         "mimeType",     doc.getProperty("mimeType"));
12 }

```

Figure 102.1 names the four recurring content operations.

The fetch operation was shown in Listing 102.1. The remaining three complete the set.

Listing 102.2 is the *store* operation: the worker receives content — here, a document uploaded by the customer — creates it in the repository with the taxonomy-derived metadata, and returns the identifier the process will carry for the rest of its life.

Listing 102.2. A content-store job worker: it creates the document in the repository with typed metadata and returns the identifier the process will carry.

```

1  @JobWorker(type = "store-document")
2  public Map<String, Object> store(ActivatedJob job) {
3      // build the metadata from the taxonomy
4      Map<String, Object> props = Map.of(
5         "cmis:objectTypeId", "D:bank:proofOfIdentity",
6         "bank:caseRef",     job.getVariable("caseReference"),
7         "bank:docType",     job.getVariable("documentType"),
8         "bank:expiryDate",  job.getVariable("expiryDate"));
9
10     // create in the repository
11     Document created = repository.createDocument(
12         props,
13         contentStream(job.getVariable("fileBytes"),
14             job.getVariable("mimeType")),
15         VersioningState.MAJOR);
16
17     // the process carries only the reference
18     return Map.of(

```

```

19     "documentId", created.getId(),
20     "documentUrl", created.getUrl());
21 }

```

Listing 102.3 is the *properties* operation: a worker that reads or updates the document’s metadata — here, setting the document’s verification status after a human reviewer has checked it — without touching the content bytes at all.

Listing 102.3. A properties-update job worker: it updates the document’s metadata in the repository without touching the content — here, recording the verification outcome.

```

1  @JobWorker(type = "update-document-status")
2  public void updateStatus(ActivatedJob job) {
3      String docId = job.getVariable("documentId");
4      Document doc = repository.getById(docId);
5
6      // update the metadata, not the content
7      doc.updateProperties(Map.of(
8          "bank:verificationStatus",
9          job.getVariable("verificationOutcome"),
10         "bank:verifiedBy",
11         job.getVariable("reviewerIdentity"),
12         "bank:verifiedDate",
13         Instant.now().toString()));
14 }

```

And Listing 102.4 is the *lifecycle* operation: advancing the document through the ECM system’s managed states — here, moving a document from “draft” to “approved,” which in many ECM systems creates a new major version and locks the content from further editing.

Listing 102.4. A lifecycle-advance job worker: it moves the document from draft to approved in the repository, creating a new major version.

```

1  @JobWorker(type = "approve-document")
2  public Map<String, Object> approve(ActivatedJob job) {
3      String docId = job.getVariable("documentId");
4      Document doc = repository.getById(docId);
5
6      // advance the lifecycle: draft -> approved
7      // creates a new major version in the repository
8      ObjectId newVersion = doc.checkIn(
9          true, // major version
10         Map.of("cmis:comment", "approved by process"),
11         "Approved via lending process");
12
13     return Map.of(
14         "approvedVersionId", newVersion.getId());
15 }

```

Together, the four workers — store, fetch, properties, lifecycle — are the content integration’s complete vocabulary. Every content interaction a process needs is one of these four, called through CMIS or the repository’s native API. A platform that implements these four as reusable workers has a content integration that any process can consume by referencing the appropriate job type.

The store worker must set the document’s *type* and *metadata* at creation — not leave it for a later step to classify. A document created in the repository without its taxonomy metadata is an unclassified document, and an unclassified document cannot be retained,

disposed of, or found by type. The taxonomy is not optional metadata to be added “when someone gets around to it”; it is the document’s identity, and it must be set at birth.

102.4 Document governance as a regulatory obligation

In a regulated institution, document governance is not a best practice; it is a *regulatory obligation*. A bank must be able to show, for any regulated document, where it is stored, who has accessed it, how long it will be retained, and when it will be disposed of. An ECM system provides these answers; a process that stores documents in its own variables, in shared drives, or in email attachments provides none of them. The integration discipline this chapter describes — the process carries a reference, the ECM system holds and governs the content — is the only architecture that lets the institution answer a regulator’s questions about its documents.

Worked example: the document fetched at the start and held throughout

Problem. A 12-step lending process needs a customer’s identity document at step 10, for a verification check. The team builds the process to *fetch* the document at step 1 — “so it is ready” — and carry its content in a process variable through steps 1 to 10. Assess this against the chapter’s mechanism.

Setup. We consider what carrying the content from step 1 to step 10 costs, and what the fetch operation is for.

Working. The process needs the document’s *bytes* only at step 10. By fetching at step 1 and holding the content in a variable, the team carries the full document through nine steps that do not use it — and during those nine steps the content sits in the process state, the history, and every backup, exactly the inflation [Chapter 101](#)’s principle warns against:

content needed at step 10 $\neq$ content carried from step 1.

The fetch operation exists precisely so that content is retrieved *at the step that needs it* — step 10 — not hoarded from the start.

Discussion. The team treated “fetch early so it is ready” as prudence, but it is the opposite, and the worked example shows why the *timing* of the fetch is part of the mechanism, not a detail. The process should carry the document *reference* — the identifier — from the moment it knows it, because a reference is a few bytes and costs nothing to carry; what it must not do is carry the *content*. The fetch job worker of [Listing 102.1](#) is built to be called at step 10, the step that actually needs the bytes, so the content lives in a process variable for the duration of one step rather than ten. Fetching at step 1 converts a lightweight, reference-carrying process into one that hauls document content through most of its length, inflating state and history for nine steps to save nothing — the document is no more “ready” as a variable than as a reference the worker can resolve in milliseconds. The lesson refines the part’s principle: it is not only *what* the process carries — a reference, not the document — but *when* it fetches: at the step that needs the content, and not before.

Practical next steps

In a document-handling process, check *when* content is fetched. The process should carry a document reference throughout and call the fetch operation only at the step that needs the

bytes. Fetching early and holding the content in a variable inflates the process state for every step in between, and gains nothing a reference does not already give.

Chapter in brief

A Camunda process reaches an ECM system through ordinary connectors and job workers, but content repositories offer a standard interface worth using: *CMIS* (Content Management Interoperability Services), an open standard with a common model and API for content operations, portable across FileNet, OpenText, and others. Where the repository exposes CMIS and the operations are standard, the integration speaks CMIS; where more is needed, it speaks the native API. A content integration is built from four recurring operations — *store*, *fetch*, *read/update properties*, and *advance life-cycle* — each an ordinary connector or job worker. And the content must be fetched at the step that needs it, not hoarded from the start, or the process inflates its state carrying bytes it is not yet using.

Governing Documents Across the Process Estate

103.1 Beyond one process

The first two chapters treated one process integrating with one content repository. This chapter widens to the *estate*: the governance concerns that arise when many processes, across an institution, all handle documents — because content carries regulatory weight, and an estate that integrates with content carelessly fails in ways a single process never reveals.

103.2 Retention, disposition, and the regulated document

The first governance concern is *retention* — and it is where process and content most consequentially meet.

A regulated institution does not keep documents forever, and does not keep them for arbitrary periods. Each class of document has a *retention rule*: a mortgage file kept for a defined number of years after the mortgage closes; a claims file for a period set by regulation; identity documents under data-protection limits. And retention has a second half — *disposition* — the rule that a document, once its retention period ends, is *destroyed*, because keeping personal data longer than permitted is itself a breach.

The crucial architectural point is that retention and disposition are the *ECM system's* job, not the process's. The ECM system is built to apply retention rules, hold documents under legal hold, and dispose of them on schedule. A process must therefore not become a place documents accumulate outside that regime — which is the reference principle again, now seen from the compliance side: documents held in process variables, in process history, in engine backups, are documents *outside* the ECM system's retention and disposition, and an institution that cannot say where all its copies of a regulated document are cannot honestly claim to dispose of it.

Retention and disposition are the ECM system's responsibility, and a process must not undermine them. A document copied into process variables, process history, or engine backups is a copy *outside* the ECM system's retention regime — it will not be disposed of on schedule, and it may not even be locatable. For a regulated institution this is a compliance failure: keeping personal data beyond its permitted period is a breach, and “we also had copies in the workflow engine” is not a defence. Hold references; let the ECM system own the document's lifecycle.

103.3 Access, audit, and classification across the estate

Three further governance concerns complete the picture.

Access control. A regulated document is not visible to everyone, and the ECM system enforces who may see each document. A process integration must *respect* that control, not circumvent it — a process step that fetches a document on behalf of a user must honour whether that user is entitled to it, rather than becoming a side channel that serves content the ECM system would have refused.

Audit. The ECM system records what happened to a document — who viewed it, who changed it, when. The process records what happened in the process. A well-governed estate keeps these consistent: when a process advances a document's lifecycle, the action is auditable on both sides, and the two records can be reconciled.

Classification consistency. Across an estate, the same kind of document should be classified the same way — the same document type, the same metadata properties — so that a query for “all identity documents” means something. An estate where each process integrates with the ECM system on its own terms, inventing its own document types and metadata, produces a content store no one can query coherently. Estate-level governance means a *shared* content model: agreed document types, agreed metadata, used consistently by every process.

103.4 Retention and disposition in code

The retention rules the taxonomy defines are enforced by the ECM system, but a process can *trigger* disposition. Listing 103.1 is a worker that queries the repository for documents whose retention period has expired — documents eligible for disposition — and returns them for a governed review-and-dispose process.

Listing 103.1. A retention-query worker: it finds documents whose retention period has expired, returning them for a governed disposition process.

```
1 @JobWorker(type = "find-expired-documents")
2 public Map<String, Object> findExpired(ActivatedJob job) {
3     String query = "SELECT cmis:objectId, cmis:name "
4         + "FROM bank:regulated_document "
5         + "WHERE bank:retentionEndDate < TIMESTAMP '"
6         + Instant.now().toString() + "' "
7         + "AND bank:dispositionStatus = 'PENDING'";
8
9     List<String> ids = repository.query(query).stream()
10         .map(r -> r.getPropertyValue("cmis:objectId"))
11         .collect(Collectors.toList());
12
13     return Map.of(
14         "expiredDocumentIds", ids,
15         "expiredCount",      ids.size());
16 }
```

Listing 103.2 is the disposition worker: it permanently removes a document from the repository, after governance approval and an audit record.

Listing 103.2. A disposition worker: it permanently removes a document from the repository after governance approval.

```
1 @JobWorker(type = "dispose-document")
2 public void dispose(ActivatedJob job) {
3     String docId = job.getVariable("documentId");
4     Document doc = repository.getById(docId);
5
6     auditService.recordDisposition(
7         docId,
8         doc.getProperty("bank:docType"),
```

```

9      job.getVariable("approverIdentity"),
10      Instant.now());
11
12      doc.delete(true); // allVersions = true
13  }

```

Document disposition is irreversible. A disposition worker must never run without the governance process having completed — the human review, the approval, the audit record. A worker that deletes documents on a schedule without governance approval is not automating disposition; it is automating destruction.

103.5 Document governance as a regulatory obligation

In a regulated institution, document governance is not a best practice; it is a *regulatory obligation*. A bank must be able to show, for any regulated document, where it is stored, who has accessed it, how long it will be retained, and when it will be disposed of. An ECM system provides these answers; a process that stores documents in its own variables, in shared drives, or in email attachments provides none of them. The integration discipline this chapter describes — the process carries a reference, the ECM system holds and governs the content — is the only architecture that lets the institution answer a regulator’s questions about its documents.

Worked example: the document that could not be disposed

Problem. A bank receives a data-protection request: a former customer asks that their personal data be erased, as their retention period has ended. The bank’s ECM system dutifully disposes of the customer’s documents. Months later, an audit finds copies of those same identity documents still present — in the process history and backups of three lending processes that had, years earlier, loaded the documents into process variables. Assess the failure.

Setup. We trace where the copies came from and why disposition did not reach them.

Working. The ECM system disposed of the documents it held — that part worked. But three lending processes had, years earlier, loaded the document *content* into process variables, and that content propagated into each process’s history and backups. Those copies are *outside* the ECM system’s retention regime:

ECM disposition → ECM copies destroyed;

copies in process history / backups → untouched.

The ECM system cannot dispose of what it does not hold, and it does not hold — does not even know about — the copies sitting in the workflow engine’s data layer. The bank told the regulator the data was erased; it was not.

Discussion. This is the warn box’s failure realised, and it is a serious one — a data-protection breach, not an inconvenience. The root cause is the violation, years earlier, of the part’s governing principle: three processes held document *content* in process variables rather than *references*, and in doing so created copies of regulated personal data outside the one system — the ECM system — whose job is to track and dispose of it. Disposition is only as complete as the institution’s knowledge of where its copies are, and a process that swallows documents creates copies the ECM system has no way to reach. The fix is architectural and must be retrospective: the reference principle stops new copies being made, and an estate

that has already violated it must hunt down the document content sitting in process history and backups and bring it under disposition — or it cannot honestly answer a data-protection request. The lesson closes the part: integrating with content is not only an operational matter of storing and fetching documents; it is a governance matter, and the reference principle — the process holds a pointer, the ECM system holds and governs the document — is what keeps a document estate disposable, auditable, and compliant. A process platform that ignores it does not merely carry too much data; it quietly makes the institution unable to keep its regulatory promises.

Practical next steps

Audit your process estate for document content held outside the ECM system — in process variables, history, and backups. Each such copy is outside retention and disposition and is a compliance exposure. Establish the reference principle as estate-wide architecture, and bring existing content copies under the ECM system's governance or delete them.

Chapter in brief

Across a process estate, content integration is a governance matter. *Retention and disposition* — keeping each class of document for its defined period and destroying it after — are the ECM system's responsibility, and a process must not undermine them: content copied into process variables, history, or backups is outside the ECM system's retention regime, will not be disposed of on schedule, and is a compliance breach waiting to be found. *Access control* must be respected, not circumvented; *audit* records on the process and ECM sides must stay reconcilable; and *classification* must be consistent across the estate, through a shared content model, or the content store cannot be coherently queried. The reference principle is what keeps a document estate disposable, auditable, and compliant.

PART XVII

Integrating with Core Banking Systems

The Core Banking System as a System of Record

104.1 Why core banking deserves a part

Part VI treated connectors — the mechanism by which a process reaches an external system. This part, and the four that follow it, treat the *systems* themselves: the specific categories of platform a financial hyperautomation process must integrate with, and the disciplines each category demands. The first, and the most important for a bank, is the *core banking system*.

The core banking system deserves a part of its own because it is not just another integration. It is, for a bank, *the system of record* — **Chapter 25** introduced that term, and nowhere does it matter more. The core banking system holds the accounts, the balances, the ledger, the deposits and loans — the authoritative record of the bank's financial reality. Every hyperautomation process a bank builds will, sooner or later, touch the core. An institution that integrates well with its core banking system has a sound foundation; one that integrates badly has built its processes on sand.

104.2 What a core banking system is, and what it is not

A core banking system runs the fundamental account-keeping of a bank: it maintains customer accounts, processes the transactions that move money in and out of them, keeps the ledger, calculates interest, and holds the deposit and loan records. It is the bank's financial heart.

The crucial framing for an architect — the one this whole part rests on — is what the core banking system *is* relative to the orchestration platform. The core is the *system of record*: it owns the authoritative truth about accounts and balances. The Camunda platform is *not* a second home for that truth; it is the *orchestrator* that coordinates the processes which *use* the core. When an onboarding process opens an account, the account is created *in the core*; the process orchestrates the opening, but the core owns the account. When a payment process debits a balance, the balance lives *in the core*; the process orchestrates the payment, but the core owns the balance.

This is **Chapter 25's** anti-pattern stated for the most important case: the orchestration platform must never become a *shadow ledger*, accumulating its own copy of balances and account state that then drifts out of step with the core. The core banking system holds the financial truth; the platform holds process state and references to that truth. An architect who blurs this line has created the most dangerous integration defect a bank can have — two systems that disagree about money.

Figure 104.1 draws the core as system of record and the platform as orchestrator.

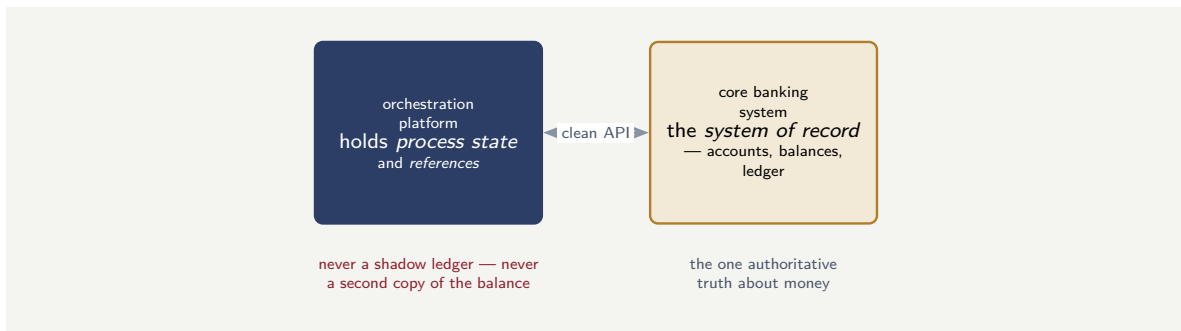


Figure 104.1. The core banking system is the bank’s system of record — it owns the authoritative truth about accounts, balances, and the ledger. The orchestration platform coordinates processes that use the core; it holds process state and references, never a shadow copy of the balance.

104.3 The spectrum of core banking systems

Core banking systems are not all alike, and the kind a bank runs shapes the integration profoundly. They sit on a spectrum from the traditional to the cloud-native.

At the traditional end are the long-established platforms — *Oracle FLEXCUBE*, the core banking offerings within *SAP*’s banking suite, and the established *Temenos* core — that have run banks for decades. They are comprehensive and proven, deeply embedded, and often architected before the API-first era; integrating with them can mean working with older interface styles alongside their modern APIs.

At the cloud-native end are the newer platforms — *Mambu* the most prominent — built from the start as cloud-native, API-first, composable systems. Integration with these is, by design, through clean modern APIs.

In between, the established vendors have modernised: *Temenos Transact*, the current *Temenos* core, is offered as a cloud-native, API-first platform with fully consumable APIs, used by over a thousand banks. The traditional cores have added API layers over their established engines.

The architect’s first task in any core banking integration is therefore to know *which* core the bank runs and *where on this spectrum* it sits — because that determines whether the integration is through clean modern APIs, through older interface styles, or, most commonly, through a mixture. The next chapters treat the specific systems.

104.4 The integration in code: a worker against the core

The principle that the core banking system is the *system of record* — and the process must not duplicate its data — is concrete in the shape of the worker that integrates with it. Listing 104.1 is a job worker that reads an account balance from the core.

Listing 104.1. A job worker reading from the core banking system. It fetches the balance at the step that needs it and returns it for that decision — it does not copy the balance into lasting process state.

```

1 @JobWorker(type = "fetch-account-balance")
2 public Map<String, Object> balance(ActivatedJob job) {
3     String accountId = job.getVariable("accountId");
4
5     // the core is the system of record -- read, don't copy
6     AccountView account = coreBanking.getAccount(accountId);
7
8     // return it for the decision at hand, not to store
9     return Map.of(

```

```

10     "availableBalance", account.availableBalance(),
11     "balanceAsOf",     account.asOfTimestamp());
12 }

```

Two things in the worker enforce the chapter’s principle. It returns a `balanceAsOf` timestamp alongside the figure, so the process knows the balance is a *reading taken at a moment*, not a durable truth — a balance is only current as of when it was read. And it returns the balance for the *decision at hand*; a disciplined process uses it in the step that needs it and does not carry it as lasting state, because the moment it does, the process holds a copy that the core will move on from. A *write* to the core — posting a transaction, opening an account — is the same worker shape but demands more, as Listing 104.2 shows.

Listing 104.2. A job worker posting a transaction to the core. The write carries an idempotency key, so a retried job does not post the transaction twice.

```

1  @JobWorker(type = "post-transaction")
2  public Map<String, Object> post(ActivatedJob job) {
3      String txnRef = job.getVariable("transactionRef");
4
5      // the txnRef is the idempotency key: a retry of this
6      // job posts the same transaction, never a second one
7      PostingResult result = coreBanking.postTransaction(
8          txnRef,
9          job.getVariable("accountId"),
10         job.getVariable("amount"));
11
12     return Map.of("postingId", result.postingId());
13 }

```

The write worker carries an *idempotency key* — the `transactionRef` — and this is not optional. A job worker may, under Camunda’s at-least-once delivery, run twice for one logical step; a read run twice is harmless, but a *posting* run twice is a duplicate transaction — real money moved twice. The idempotency key makes the core itself recognise the second attempt as a repeat of the first and refuse to post it again. Writing to a core banking system without an idempotency key is the single most dangerous omission in core banking integration.

Any worker that *writes* to a core banking system — posts a transaction, transfers funds, opens an account — must carry an idempotency key the core honours. Camunda delivers a job at least once, so a write worker can run twice for one step, and a duplicated write to a core banking system is duplicated money movement. The key — a transaction reference unique to the logical operation — lets the core recognise and reject the repeat. A read worker without one is merely inefficient; a write worker without one is a financial incident waiting for its first retry.

104.5 The anti-corruption discipline for core banking

The core banking system’s data model is its own: decades of accumulated structure, vendor-specific field names, legacy encodings. A process that binds directly to that model inherits all of it, and every change the vendor makes to the core’s API ripples into every process. The anti-corruption discipline requires a deliberate translation layer: the worker maps the core’s fields into the process’s own vocabulary, and the process model upstream of the worker never sees a vendor-specific field name. The translation costs a small amount of code; the coupling it prevents costs a large amount of rework every time the core’s API changes.

Worked example: the shadow ledger that drifted

Problem. A bank builds a Camunda lending process. To make the process fast and to avoid calling the core banking system repeatedly, the design team decides the process will keep its own copy of each borrower's account balance as a process variable, updated whenever the process itself moves money, and read from that variable rather than calling the core. For six months it works well. Then reconciliation finds that for 1,400 loan accounts, the balance the lending platform believes and the balance the core banking system holds *disagree* — sometimes by large amounts. Explain how the design produced this, and what it should have done.

Setup. We consider what happens to the platform's copy of a balance when something *other than the lending process* changes the real balance in the core.

Working. The platform's copy of a balance is updated only when *the lending process itself* moves money. But a borrower's account balance in the core banking system is changed by many things the lending process never sees: a payment received through the payments system, a fee applied by the core, a manual adjustment by a branch, interest posted by the core's overnight batch. Every one of those changes the *real* balance in the core and does *not* change the platform's copy:

real balance (core) $\neq$ platform's copy, growing apart with every unseen change.

Over six months, across 1,400 accounts, the unseen changes accumulate and the two balances drift apart. The platform built a *shadow ledger*, and a shadow ledger drifts the moment anything but the platform touches the truth.

Discussion. The design team optimised for speed and produced the single most dangerous core banking integration defect: two systems that disagree about money. The worked example shows why the shadow ledger is not a performance shortcut but a correctness failure waiting for time to pass. The team's mental model was that the lending process *owns* the balance — that if the process keeps its copy updated whenever *it* moves money, the copy stays correct. But the balance is not the lending process's to own: it is the core banking system's, and the core is changed by the payments system, by fees, by branches, by overnight interest — a dozen actors the lending process has no visibility of. The platform's copy was current only with respect to the process's own actions and stale with respect to everyone else's, so it drifted, silently, until reconciliation found 1,400 disagreements. The correct design is the chapter's central principle: the core banking system is the system of record for the balance, so the process *reads the balance from the core* when it needs it, and never keeps an authoritative copy. If repeated core calls are a genuine performance concern, the answer is a deliberately-managed, clearly-non-authoritative cache with a defined freshness — explicitly a convenience copy, never trusted for a financial decision — not a shadow ledger the process treats as truth. The lesson is the one the part opens with: the core owns the money, the platform orchestrates; an architecture that lets the platform believe its own balance has built two ledgers, and two ledgers about money will always, eventually, disagree.

Practical next steps

Audit your bank's hyperautomation processes for any place a balance, an account status, or a ledger figure is *stored* in the platform and *read back as truth*. Each one is a shadow ledger that will drift. The core banking system owns those figures; the process should read them from the core when it needs them, and treat any local copy as an explicitly non-authoritative convenience, never as the truth.

Chapter in brief

The core banking system — the platform that runs a bank’s accounts, balances, and ledger — is the most important integration a hyperautomation process makes, and it deserves a part of its own. The defining principle is that the core is the *system of record*: it owns the authoritative truth about money, while the orchestration platform coordinates the processes that use the core and holds only process state and references. The platform must never become a *shadow ledger* with its own copy of balances, because that copy drifts the moment anything else touches the core. Core banking systems span a spectrum from traditional — Oracle FLEXCUBE, SAP, the established Temenos core — to cloud-native and API-first — Mambu, Temenos Transact — and an architect’s first task is to know which the bank runs.

Integrating with Temenos and Oracle FLEXCUBE

105.1 The established cores

This chapter treats integration with two of the most widely-deployed core banking platforms in the world: *Temenos* — in its current form, *Temenos Transact* — and *Oracle FLEXCUBE*. Both are comprehensive, long-established cores running large numbers of banks, and an architect in an established bank is more likely to be integrating with one of these than with anything else.

105.2 Temenos Transact

Temenos is the most widely-used core banking platform in the world, relied on by over a thousand banks across many countries, covering retail, corporate, treasury, wealth, and payments. Its current form, *Temenos Transact*, is offered as a cloud-native, cloud-agnostic platform — available as SaaS, on cloud, or on premise — and, importantly for integration, it exposes a rich set of *fully consumable APIs*.

For an architect, the API-first nature of Temenos Transact is the good news: a Camunda process integrates with it, in principle, through clean modern API calls — the REST-style connectors of [Part VI](#) — to perform the banking operations the process needs. Temenos also provides a developer sandbox, an API gateway for centralising access control and traffic management, and a marketplace ecosystem.

The realistic qualification is that Temenos is a *comprehensive and complex* platform. Its functional richness means its API surface is large, and its data model — the way it represents accounts, customers, arrangements — has its own structure and its own terminology that an integrating process must learn. Integration with Temenos is genuinely API-based and modern; it is not trivial, because the platform it exposes is genuinely large.

105.3 Oracle FLEXCUBE

Oracle FLEXCUBE is a comprehensive banking platform from Oracle, designed for the full complex requirements of a financial institution — core banking and beyond, into areas such as customer relationship management, wealth, and risk. It is deployed widely, particularly internationally, and is a proven, deeply-capable platform.

FLEXCUBE integration carries the characteristics of an established, Oracle-ecosystem platform. It provides APIs for modern integration, and a Camunda process integrates with it through them. But, as [Chapter 86](#) noted of Oracle platforms generally, FLEXCUBE is part of the broader Oracle technology world, and integrations with it have historically often gone *through* Oracle's integration tooling and middleware. An architect integrating with FLEXCUBE should establish early which interface styles the bank's particular FLEXCUBE deploy-

ment and version actually offer — modern APIs, older message-based interfaces, or a mixture — because established cores frequently present all of these at once.

Tip

With an established core — Temenos Transact, Oracle FLEXCUBE — do not assume the integration style; establish it. These platforms are API-capable, but their specific deployment and version may present a mixture of modern APIs and older message-based or batch interfaces, and their data models are large and vendor-specific. The first task of a core banking integration is a concrete inventory: which interfaces this bank's core actually exposes, and what its data model calls the things your process needs.

105.4 The orchestration layer between core and channels

A pattern recurs in real integrations with established cores, and it is the architecture this manuscript has argued for throughout, applied here. Banks integrating Temenos or FLEXCUBE with their channels and their wider system estate very commonly introduce a *middleware orchestration layer* between the core and everything else — a layer that handles protocol transformation, retries, message enrichment, and routing, so that the core's specific interfaces are not exposed raw to every consumer.

This is precisely [Chapter 25](#)'s anti-corruption layer, and the Camunda platform's place relative to it should be clear. The Camunda process orchestrates the *business* flow — the onboarding journey, the lending process. Where the bank has an integration middleware layer in front of the core, the process calls the core *through* it; where it does not, the anti-corruption discipline says the process should still integrate with the core through clean, stable, business-meaningful interfaces — its connectors and a deliberate integration design — and never bind its process models directly to FLEXCUBE's or Temenos's raw internal representations. The core's data model and quirks stop at the integration boundary; the process model stays clean.

105.5 The two integrations in code

The difference between an API-first core and one reached through middleware is concrete in the worker code. [Listing 105.1](#) integrates with Temenos Transact: because Transact is API-first, the worker is a clean REST call, and its one real job is *translation* — turning Temenos's response into the process's own business-meaningful variables.

Listing 105.1. A job worker calling Temenos Transact. The core is API-first, so the call is a clean REST request; the worker's task is to translate the Temenos response into the process's own terms.

```
1 @JobWorker(type = "open-deposit-account")
2 public Map<String, Object> openAccount(ActivatedJob job) {
3     // Temenos Transact: a clean, API-first REST call
4     TemenosResponse r = temenosClient.post(
5         "/api/v1.0.0/holdings/accounts",
6         Map.of("customerId", job.getVariable("customerId"),
7             "productId", job.getVariable("productCode"),
8             "currency", job.getVariable("currency")));
9
10    // anti-corruption: translate, do not pass raw through
11    return Map.of(
12        "accountId", r.path("body.accountReference"),
13        "accountStatus", mapStatus(r.path("body.status")));
14 }
```


Listing 105.2 integrates with FLEXCUBE in a bank where FLEXCUBE sits behind Oracle integration middleware. The worker does not call FLEXCUBE directly; it calls the *middleware*, which owns the protocol translation to the core. The worker’s shape is the same — the difference is what it points at.

Listing 105.2. A job worker integrating with FLEXCUBE through an integration-middleware layer. The worker calls the middleware endpoint; the middleware owns the translation to the core.

```
1 @JobWorker(type = "open-deposit-account")
2 public Map<String, Object> openAccount(ActivatedJob job) {
3     // FLEXCUBE reached through the integration layer --
4     // the worker calls the middleware, not the core
5     MiddlewareResponse r = integrationLayer.invoke(
6         "flexcube.account.create",
7         Map.of("customerNo", job.getVariable("customerId"),
8             "acctClass", job.getVariable("productCode"),
9             "ccy", job.getVariable("currency")));
10
11     return Map.of(
12         "accountId", r.field("accountNumber"),
13         "accountStatus", mapStatus(r.field("acctStat")));
14 }
```

The two workers make the chapter’s point precise. They are the *same shape* — a job worker, taking process variables, calling out, translating the result back — and the same business operation, “open a deposit account.” What differs is only the *target*: Temenos Transact reached directly as the API-first platform it is, FLEXCUBE reached through the Oracle integration middleware its deployment sits behind. And both workers do the anti-corruption translation — mapping the core’s fields and status codes into the process’s own terms — so that whichever core the bank runs, the process model upstream of the worker is identical. The worker absorbs the difference between the cores; the process never sees it.

105.6 The anti-corruption discipline for core banking

The core banking system’s data model is its own: decades of accumulated structure, vendor-specific field names, legacy encodings. A process that binds directly to that model inherits all of it, and every change the vendor makes to the core’s API ripples into every process. The anti-corruption discipline requires a deliberate translation layer: the worker maps the core’s fields into the process’s own vocabulary, and the process model upstream of the worker never sees a vendor-specific field name. The translation costs a small amount of code; the coupling it prevents costs a large amount of rework every time the core’s API changes.

Worked example: binding the process to the core's data model

Problem. A bank integrates a Camunda onboarding process directly with Temenos Transact. To move quickly, the team maps the Temenos account and customer data structures — field names, codes, nested structures and all — straight into the Camunda process variables, and the process’s gateways and forms reference those Temenos-shaped structures directly. The onboarding process ships. Two years later, the bank undertakes a Temenos version upgrade that changes parts of the data model. Assess the position the onboarding process is in.

Setup. We consider what the onboarding process depends on, and what the Temenos upgrade changes.

Working. The onboarding process variables, gateways, and forms all reference the Temenos data structures *directly* — the process is bound to Temenos’s specific representation of accounts and customers. The Temenos upgrade changes parts of that representation. Therefore every place the process touched a changed structure must itself be changed:

core data model changes $\Rightarrow$ the process model must change with it.

The onboarding process — a business process, owned by a business function — must be re-opened, re-tested, and re-deployed not because the onboarding *business* changed but because the *core’s data model* did. The process’s stability is hostage to the core’s release cycle.

Discussion. The team’s shortcut — map the core’s structures straight into the process — saved time at the start and created a permanent coupling that the worked example exists to expose. By binding the process model directly to Temenos’s data representation, the team made the business process depend on a technical detail it should never have known about: the core’s internal data model. The consequence surfaces at the upgrade, when a change that is purely the core vendor’s — a revised data structure — forces a change to the bank’s *business* onboarding process, with all the reopening, re-testing, and re-deployment that implies, and the risk of disturbing a process that had nothing wrong with it. This is exactly the leakage [Chapter 27’s](#) anti-corruption layer exists to prevent. The correct design maps the Temenos structures, at the integration boundary, into the process’s *own* clean, stable, business-meaningful representation — “the customer,” “the account,” in the bank’s terms — so the process model references only that. Then a Temenos upgrade changes the mapping inside the integration layer, and the onboarding process model is untouched. The lesson is the chapter’s discipline: an established core has a large, vendor-specific data model, and that model must stop at the integration boundary. A process bound directly to the core’s data structures is a process whose stability belongs to the core vendor’s release schedule — and a bank does not want its onboarding process upgraded every time Temenos is.

Practical next steps

For any process integrating with Temenos, FLEXCUBE, or another established core, check whether the process model references the core’s data structures directly or its own business-meaningful representation. Direct references mean the core’s next version upgrade becomes a change to your business processes. Map the core’s model into a clean representation at the integration boundary, and keep the process model bound only to that.

Chapter in brief

Temenos Transact and Oracle FLEXCUBE are two of the most widely-deployed core banking platforms, and an architect in an established bank most often integrates with one of them. Temenos Transact is cloud-native and API-first with fully consumable APIs, though comprehensive and complex with a large API surface and data model. FLEXCUBE is a deeply-capable Oracle-ecosystem platform whose integrations historically often went through Oracle middleware. With either, the integration style must be established, not assumed — modern APIs, older interfaces, or a mixture. Banks commonly place a middleware orchestration layer between the core and its consumers; whether or not one exists, the anti-corruption discipline holds: the core’s large, vendor-specific data model must stop at the integration boundary and never be bound directly into the process model.

Integrating with Mambu and SAP

106.1 Two contrasting cores

This chapter treats two further core banking platforms that an architect meets, and they make an instructive contrast: *Mambu*, a cloud-native challenger built API-first from the ground up, and *SAP*'s banking platform, the core-banking capability within the vast *SAP* enterprise ecosystem. They sit at nearly opposite points of the spectrum [Chapter 104](#) described, and integrating with each has a different character.

106.2 Mambu and the cloud-native, composable core

Mambu is a cloud-native banking platform, built from the start as an API-first, cloud-based system, and it has become prominent particularly with fintechs, neobanks, and banks pursuing a modern, composable architecture.

For integration, Mambu's nature is the defining fact: it was *designed* to be integrated with. Its API is its primary interface, not an addition layered over an older engine. Mambu is associated with the idea of *composable banking* — the bank assembled from best-of-breed components connected by APIs, rather than delivered as one monolithic suite. For a Camunda platform, this is the most congenial kind of core to integrate with: clean modern APIs, a system that expects to be one component among many, an architecture that shares the manuscript's own assumptions.

The discipline here is not about overcoming an awkward interface — Mambu's is clean — but about *architecture*. In a composable, Mambu-centred bank, the question of what orchestrates the composed components becomes central, and that is exactly the Camunda platform's role: the process engine is the orchestrator that turns a set of composable banking components — Mambu for the core, other services around it — into end-to-end business processes. Mambu provides the core capability cleanly; the Camunda process composes it with everything else into a journey.

106.3 SAP banking in the enterprise ecosystem

SAP's banking platform is a different proposition. *SAP* is one of the largest enterprise-software ecosystems in the world, and a bank running *SAP* for banking is typically running it amid a broader *SAP* estate — *SAP* for finance, for the general ledger, for other enterprise functions.

Integration with *SAP* banking therefore has the character of integrating with the *SAP world*, not only a core banking product. The architect must account for *SAP*'s own integration technologies and interface conventions, and for the fact that the banking data may be connected to a wider *SAP* data and process landscape. As with the Oracle case of [Chapter 105](#), the practical first step is to establish concretely how this bank's *SAP* banking deployment exposes the operations a process needs — through modern APIs, through *SAP*'s integration tooling, or a mixture — and to integrate through a clean boundary regardless.

Figure [106.1](#) contrasts the Mambu and *SAP* integration profiles.

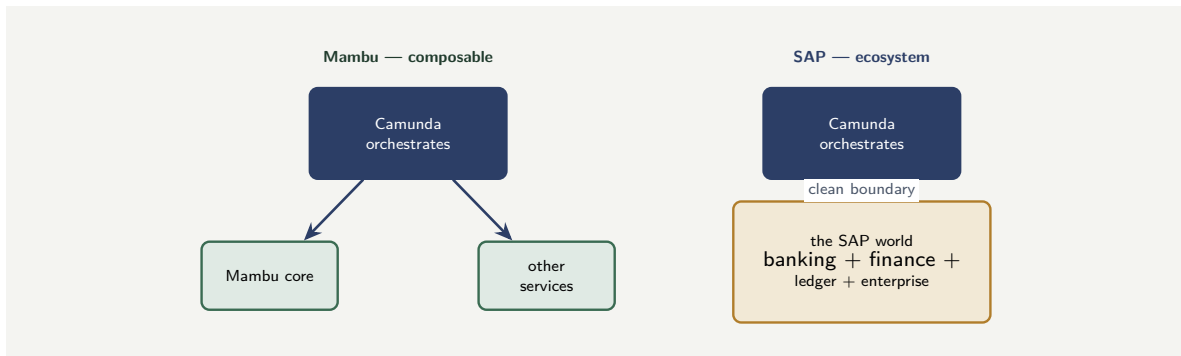


Figure 106.1. Two contrasting cores. Mambu is a clean, cloud-native, composable component the Camunda process orchestrates among others. SAP banking sits inside a vast enterprise ecosystem, integrated through a clean boundary onto the SAP world. The orchestration role is the same; the integration character differs.

106.4 The two integrations in code

The worked example below argues that one business process should serve both cores through a core-agnostic contract. The code makes that concrete. Both integrations are job workers of type `create-deposit-account` — the same contract the process calls — and they differ only behind it.

Listing 106.1 integrates with Mambu. Because Mambu is cloud-native and API-first, the worker is a clean, modern REST call, about as direct as a core banking integration gets.

Listing 106.1. A job worker creating an account in Mambu. The cloud-native, API-first core is reached by a clean, direct REST call.

```

1  @JobWorker(type = "create-deposit-account")
2  public Map<String, Object> create(ActivatedJob job) {
3      // Mambu: cloud-native, API-first -- a clean REST call
4      MambuResponse r = mambuClient.post(
5          "/deposits",
6          Map.of("productTypeKey", job.getVariable("productCode"),
7                "accountHolderKey", job.getVariable("customerId"),
8                "accountHolderType", "CLIENT"));
9
10     return Map.of(
11         "accountId", r.field("encodedKey"),
12         "accountStatus", mapStatus(r.field("accountState")));
13 }

```

Listing 106.2 fulfils the *same contract* against SAP. The worker calls into the SAP world — here through a published SAP integration service — and translates the result back into the process's terms. The method type is identical to the Mambu worker's; the process cannot tell which is deployed.

Listing 106.2. A job worker fulfilling the same `create-deposit-account` contract against SAP banking, reached through an SAP integration service.

```

1  @JobWorker(type = "create-deposit-account")
2  public Map<String, Object> create(ActivatedJob job) {
3      // SAP: reached through an SAP integration service
4      SapResponse r = sapIntegration.call(
5          "BankAccountCreate",
6          Map.of("BusinessPartner", job.getVariable("customerId"),
7                "ProductCategory", job.getVariable("productCode")));

```

```

8
9     return Map.of(
10         "accountId",    r.get("BankAccountID"),
11         "accountStatus", mapStatus(r.get("AccountStatus")));
12 }

```

The two listings are the worked example’s lesson in code. They share a job type — `create-deposit-account` — which is the *core-agnostic contract*: the business process has one service task that says “create a deposit account,” and it neither knows nor cares which of these two workers is wired behind it. The Mambu worker and the SAP worker are completely different inside — a direct cloud API call versus a call into the SAP ecosystem — but each ends by translating its core’s response into the same two variables, `accountId` and `accountStatus`, in the process’s own vocabulary. That shared output is what makes them interchangeable. The group builds the deposit-opening process once; each subsidiary deploys the worker for its core; and the process model is identical in both banks.

106.5 The anti-corruption discipline for core banking

The core banking system’s data model is its own: decades of accumulated structure, vendor-specific field names, legacy encodings. A process that binds directly to that model inherits all of it, and every change the vendor makes to the core’s API ripples into every process. The anti-corruption discipline requires a deliberate translation layer: the worker maps the core’s fields into the process’s own vocabulary, and the process model upstream of the worker never sees a vendor-specific field name. The translation costs a small amount of code; the coupling it prevents costs a large amount of rework every time the core’s API changes.

Worked example: the same process, two cores

Problem. A banking group owns two subsidiary banks. Bank M runs Mambu, cloud-native and API-first. Bank S runs SAP banking within a broad SAP estate. The group wants to deploy the *same* Camunda deposit-account-opening process to both. A team proposes to build the process once, hard-wired to one core’s specifics, and “adjust” it for the other. Assess this, and describe the sound approach.

Setup. We consider what differs between the two integrations and what does not, and how the process should be structured to span both.

Working. What is the *same* for both banks is the *business process*: the deposit-account-opening journey — gather details, verify identity, run checks, create the account, confirm — is identical group business logic. What *differs* is only the integration: Bank M’s account creation is a clean Mambu API call; Bank S’s is a call into the SAP world through its interfaces. So:

business process: identical; core integration: different per bank.

A process hard-wired to one core’s specifics entangles the identical business logic with one bank’s integration detail, so the “adjustment” for the other bank means picking that entanglement apart. The sound structure separates the two: one business process model, calling account-creation through a clean, core-agnostic interface — a service task whose *contract* is “create a deposit account” — backed by two different integration implementations, one for Mambu, one for SAP.

Discussion. The team’s proposal repeats, across two cores, the coupling mistake the previous chapter warned of. The deposit-opening *business process* is group logic and is genuinely identical for both subsidiaries; the only real difference is that one core is reached by a Mambu

API call and the other through the SAP world. A process hard-wired to one core's specifics fuses the shared business logic to one bank's integration, so deploying to the second bank is not deployment but disentanglement-and-rebuild — and the group ends up maintaining two divergent copies of what should be one process. The sound design applies the anti-corruption discipline as a *contract*: the process model calls “create a deposit account” through a clean, core-agnostic service interface, and behind that interface sit two integration implementations — a Mambu one and an SAP one. The business process is built and owned once; each bank supplies the integration behind the contract. The lesson generalises beyond this group: the business process and the core integration are different things with different owners and different rates of change, and keeping them separate — the process bound to a core-agnostic contract, the core specifics behind it — is what lets one process serve many banks, and lets a bank change its core without rewriting its processes. Mambu and SAP could hardly be more different as integration targets, and that is exactly why the process must not be welded to either.

Practical next steps

If your group runs more than one core banking system, check whether shared business processes are built once against a core-agnostic contract or separately per core. Separate copies per core are the same process maintained many times and diverging. Define a clean “what the process needs from the core” contract, build the business process once against it, and put each core's specifics in its own integration implementation behind it.

Chapter in brief

Mambu and SAP banking sit near opposite ends of the core banking spectrum. Mambu is cloud-native and API-first, built to be integrated with, associated with composable banking — the bank assembled from API-connected best-of-breed components, which the Camunda process orchestrates into journeys. SAP banking sits within the vast SAP enterprise ecosystem, and integrating with it has the character of integrating with the SAP world, through its own integration technologies. Despite the contrast, the discipline is one: the business process is the same regardless of core, so it should be built once against a clean, core-agnostic contract, with each core's integration specifics behind it — which is what lets one process serve many banks and a bank change its core without rewriting its processes.

Core Banking Integration Patterns

107.1 The recurring patterns

The previous chapters treated specific core banking systems. This chapter gathers the integration *patterns* that recur across all of them — because while Temenos, FLEXCUBE, Mambu, and SAP differ, the shapes of a sound integration between a Camunda process and a core banking system are common.

107.2 Read, command, and the events the core emits

A process interacts with a core banking system in three distinct ways, and an architect should design each deliberately.

A *read* retrieves information from the core: fetch the account, check the balance, get the customer record. Reads are the most frequent interaction, and the discipline is the system-of-record discipline of [Chapter 104](#) — read from the core when the process needs the truth, and do not hoard an authoritative copy.

A *command* tells the core to do something: open an account, post a transaction, place a hold. Commands change the bank's financial reality, and they demand the strongest discipline. A command must be *idempotent* in effect or guarded against duplication — because, as [Chapter 34](#)'s job semantics made clear, a process step can be retried, and a payment posted twice because a step retried is a real loss of money. Every command to a core banking system must be designed so that a retry cannot double the effect.

An *event* is the core telling the process something happened: a payment was received, an account state changed. Where the core can emit events, an inbound connector lets a process react to them — the core becomes not only something the process calls but something the process can listen to.

Figure [107.1](#) sets out the read, command, and event patterns.

107.3 The core is slow, fallible, and sometimes batch

A final pattern is the realistic accommodation of what core banking systems are actually like.

A core banking system can be *slow*: an established core under load, reached through layers of middleware, may answer in hundreds of milliseconds or more. A process must accommodate that with deliberate timeouts and, where the core call is genuinely slow, the asynchronous patterns of [Chapter 25](#), rather than blocking a synchronous process on a slow core.

A core banking system can be *unavailable*: it has maintenance windows, it has the overnight processing during which some operations are restricted, it has outages. A process that assumes the core is always reachable will fail during every maintenance window; a process designed for core integration expects unavailability and handles it — waiting, retrying, esca-

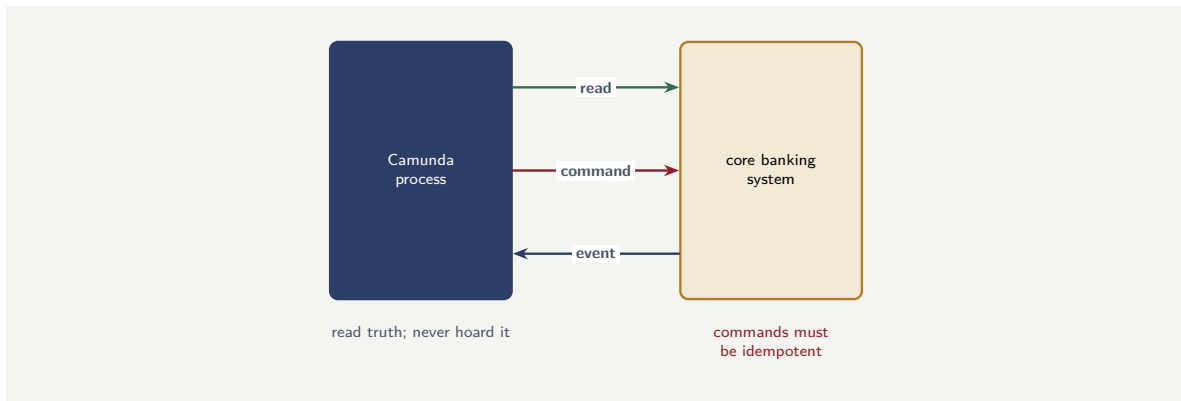


Figure 107.1. The three ways a process interacts with a core banking system: a *read* retrieves the authoritative truth; a *command* changes the bank’s financial reality and must be idempotent against retries; an *event* lets the process react to something the core reports.

lating — rather than breaking.

And a core banking system is, in part, *batch*. Many cores still run significant work as overnight batch — interest, statements, certain postings. A process may need to wait for the batch, or initiate work from its results. This connects directly to **Part X**’s treatment of batch processing patterns: a core banking integration that ignores the core’s batch nature has integrated with only half the core.

A core banking system is slow, periodically unavailable, and partly batch-driven — and a process that integrates with it as though it were a fast, always-on, real-time API will fail predictably: it will block on slow calls, break during every maintenance window, and miss the core’s overnight batch entirely. Design core banking integration for what the core actually is: deliberate timeouts, asynchronous patterns for slow calls, explicit handling of maintenance windows, and a defined relationship with the core’s batch cycle.

107.4 The anti-corruption discipline for core banking

The core banking system’s data model is its own: decades of accumulated structure, vendor-specific field names, legacy encodings. A process that binds directly to that model inherits all of it, and every change the vendor makes to the core’s API ripples into every process. The anti-corruption discipline requires a deliberate translation layer: the worker maps the core’s fields into the process’s own vocabulary, and the process model upstream of the worker never sees a vendor-specific field name. The translation costs a small amount of code; the coupling it prevents costs a large amount of rework every time the core’s API changes.

Worked example: the process that broke every night

Problem. A bank’s Camunda account-servicing process calls the core banking system synchronously for every operation, with default connector settings. The process works well during business hours. But operations notice that every night, between about 1 a.m. and 3 a.m., a large number of account-servicing process instances fail with errors, and staff spend the first

hour of each morning clearing them. Explain what is almost certainly happening.

Setup. We consider what a core banking system is doing between 1 a.m. and 3 a.m. and how the process's design responds to it.

Working. The window between 1 a.m. and 3 a.m. is the classic *overnight batch* window for a core banking system: the core is running interest calculation, statement generation, and end-of-day processing, during which it is heavily loaded and some operations are restricted or the core is partly unavailable. The account-servicing process calls the core *synchronously*, assuming a fast, always-available real-time response. During the batch window the core does not provide that:

core in overnight batch + process assumes always-on core $\Rightarrow$ nightly failures.

Every instance that runs during the window calls a core that cannot answer as expected, and fails — producing the nightly pile of errors that staff clear each morning.

Discussion. The process was designed for the core as the team wished it were — a fast, always-on, real-time API — rather than the core as it is, and the worked example shows that the gap between the two has a precise, recurring cost: a pile of failed instances every single night. The core banking system's overnight batch is not an anomaly or an outage; it is a *known, scheduled, permanent* characteristic of how the core works, and a core banking integration that does not account for it has simply not finished the integration design. The fixes follow the chapter's patterns. The process should know about the batch window: operations that can wait should be designed to wait — to defer past the window rather than fail in it — using the timer and asynchronous patterns the manuscript has covered. Operations that genuinely must run overnight must be designed for a core that is restricted during that time, with appropriate retry and escalation rather than naive synchronous calls. And the integration should treat core unavailability as expected, not exceptional. The lesson is the chapter's closing warning made concrete: a core banking system is slow, periodically unavailable, and partly batch-driven, and those are not edge cases to be surprised by — they are the core's nature. A process integrated with the core's reality runs cleanly through the night; a process integrated with a wished-for always-on core generates a pile of failures at 1 a.m. with the reliability of a scheduled job.

Practical next steps

For each process integrating with your core banking system, establish what the core does during its overnight batch window and whether the process is designed for it. Processes failing nightly are processes that assumed an always-on core. Design core integration for the core's real availability profile — its batch window, its maintenance windows — with deferral, asynchronous patterns, and explicit handling rather than naive synchronous calls.

Chapter in brief

Across all core banking systems, the integration patterns recur. A process interacts with the core in three ways: *reads* retrieve the authoritative truth and must not hoard it; *commands* change the bank's financial reality and must be idempotent against retries, since a retried step must never double a posting; and *events* let the process react to what the core reports. And a sound integration accommodates what a core banking system actually is — slow, periodically unavailable, and partly batch-driven — with deliberate timeouts, asynchronous patterns, explicit handling of maintenance and batch windows,

and a defined relationship with the core's overnight cycle. A process that integrates with a wished-for always-on core fails predictably and nightly.

PART XVIII

Integrating with Insurance Systems

The Policy Administration System as a System of Record

108.1 The insurer's core

The previous part treated the core banking system — the bank's system of record. This part treats its counterpart for an insurer: the *policy administration system*, the platform that holds the authoritative record of an insurer's policies, and the related claims and underwriting systems an insurance hyperautomation process must integrate with.

The framing is the same as for core banking, and it is worth stating directly because it governs the whole part. For an insurer, the policy administration system is *the system of record*: it owns the authoritative truth about which policies exist, what they cover, what they cost, and their status. The Camunda platform orchestrates the insurer's processes — new-business, underwriting, claims, servicing — but it does not own the policy. The policy lives in the policy administration system; the process coordinates the work *around* the policy.

108.2 The insurer's system landscape

An insurer's core systems are, in practice, a small landscape rather than a single platform, and an architect should know its shape.

The *policy administration system* (PAS) is the centre: it holds the policies, processes new business, handles endorsements and renewals, and keeps the authoritative policy record. The *claims system* handles the claims lifecycle — notification, assessment, settlement — though in some insurers it is a module of the same suite as the PAS, and in others a separate platform. The *underwriting* capability — the rules and the risk assessment for accepting and pricing risk — may again be part of the suite or a separate system. And around these sit rating engines, billing systems, and reinsurance systems.

Several vendors supply these as integrated suites. *Oracle* offers insurance-specific platforms within its financial-services portfolio — policy administration and related capabilities for life, health, and general insurance. Other major suites come from the established insurance-technology vendors. And, as in banking, there are newer cloud-native and API-first insurance platforms alongside the established suites.

The architect's first task — exactly as in [Chapter 104](#) — is to map this landscape concretely for the specific insurer: which systems exist, which are one suite and which are separate, and which holds the authoritative record of what.

Figure [108.1](#) draws the insurer's core landscape.

108.3 Why insurance integration is distinctive

Insurance integration has a character of its own, different from banking, and an architect should understand why.

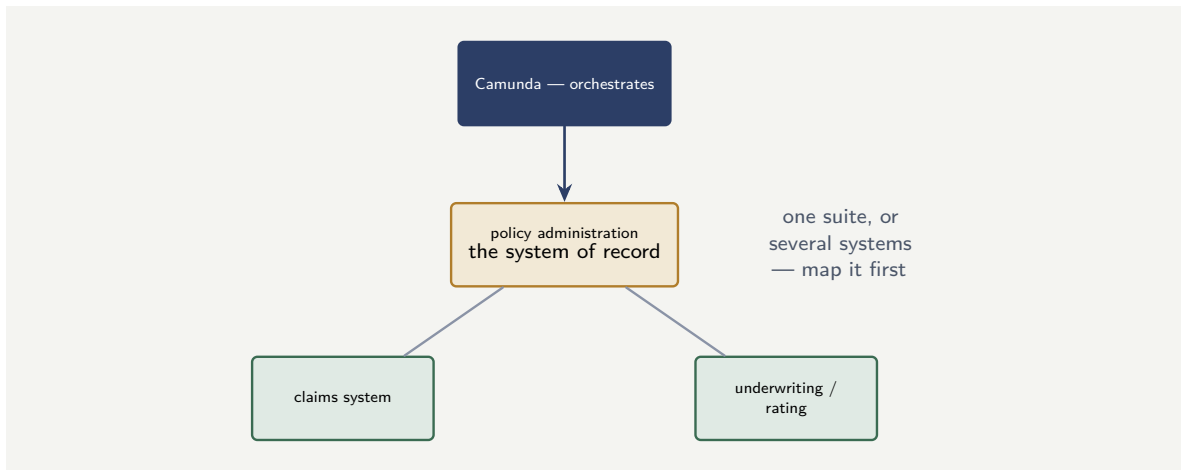


Figure 108.1. An insurer’s core system landscape. The policy administration system is the system of record at the centre; claims and underwriting may be modules of the same suite or separate platforms. The Camunda platform orchestrates the processes around them.

A bank’s core transactions are short and frequent — a payment, a balance check. An insurer’s core entity, the *policy*, is *long-lived*: a policy may run for years or decades, accumulating endorsements, renewals, claims, and changes across its life. The process landscape reflects this: underwriting a new policy, processing a claim against an existing one, renewing a policy at the end of its term are all long-running, and they all revolve around a policy record that itself persists for years. Insurance is, even more than banking, a domain of *long-running processes around long-lived records* — which is exactly what [Chapter 3](#) said BPMN and a process engine are built for.

108.4 The integration in code

The policy-administration system is reached the same way as the core banking system of [Chapter 104](#): a job worker that reads from the system of record, returns a reference, and never duplicates the master data.

Listing 108.1. A job worker reading a policy from the policy administration system — the insurance counterpart of the core banking read-worker.

```

1  @JobWorker(type = "fetch-policy-details")
2  public Map<String, Object> fetch(ActivatedJob job) {
3      String policyRef = job.getVariable("policyReference");
4
5      PolicyView policy = policyAdmin.getPolicy(policyRef);
6
7      return Map.of(
8          "policyStatus", policy.status(),
9          "productCode",  policy.productCode(),
10         "sumInsured",   policy.sumInsured(),
11         "asOfDate",     policy.asOfTimestamp());
12 }
  
```

The worker returns a `policyStatus` and the key fields the process needs for its current decision, plus an `asOfDate` timestamp — the same “reading taken at a moment” discipline the core banking chapter established. The policy administration system remains the system of record; the process carries only what it needs and never stores a copy.

108.5 Insurance-specific integration concerns

Insurance integration has concerns that banking integration does not share, and they are worth naming. *Long-tail claims*: an insurance claim can remain open for years, and the process that manages it must be designed for a lifetime measured in years, not days. *Actuarial data*: the process touches data that feeds actuarial models, and the integrity of that data is a pricing concern, not merely an operational one. *Regulatory reporting*: insurance regulators require specific, formatted reports on claims, reserves, and policyholder outcomes, and the process must produce the data those reports need. Each of these is a design input, not an afterthought.

Worked example: which system owns the policy status

Problem. An insurer builds a Camunda claims process. During a claim, the process needs to know the policy's status — is it in force, lapsed, cancelled — because a claim against a lapsed policy is handled differently. The claims system the process integrates with has a *policy status* field of its own. The design team plans to read policy status from the claims system, since the process is already talking to it. An architect objects. Explain the issue.

Setup. We ask which system is the *system of record* for policy status, and what the claims system's copy of it actually is.

Working. The authoritative record of a policy — including whether it is in force, lapsed, or cancelled — lives in the *policy administration system*; that is what “system of record for the policy” means. The claims system's *policy status* field is not the authoritative source; it is, at best, a copy the claims system holds for its own convenience, and a copy can be stale — a policy may have lapsed in the PAS since the claims system last synchronised:

PAS policy status = authoritative; claims-system copy = possibly stale.

Reading policy status from the claims system risks deciding a claim on a stale status — treating a lapsed policy as in force, or the reverse.

Discussion. The architect is right, and the issue is the system-of-record discipline of [Chapter 104](#) applied within the insurer's landscape. The design team's reasoning — “the process is already talking to the claims system, so read it there” — optimises for convenience and ignores the question that actually matters: which system *owns* policy status. The policy administration system does; the claims system's status field is a convenience copy, and a convenience copy is exactly the kind of thing that drifts. Deciding a claim — a financially consequential decision — on a possibly-stale status is the insurance equivalent of the shadow ledger. The correct design reads policy status from the policy administration system, the system of record, even though that means the claims process integrates with two systems rather than one. The lesson is the part's governing principle: in an insurer's landscape of several core systems, each piece of authoritative truth has exactly one owning system, and a process must read each truth from its owner — not from whichever system is most convenient to ask. Convenience is not authority.

Practical next steps

For an insurance process your institution runs, list each piece of core information it uses — policy status, coverage, sums insured, claim history — and identify which system is the authoritative owner of each. Where the process reads a piece of information from a system that merely holds a convenience copy of it, it risks deciding on stale data. Read each truth from

its system of record.

108.6 The insurance process estate

An insurer's process estate is distinctive in its *breadth* and its *timescales*. The breadth: an insurer runs processes across underwriting, policy administration, claims, renewals, complaints, and regulatory reporting — more process categories than most banks. The timescales: a claim can remain open for years, a policy for decades, and the processes that manage them must be designed for lifetimes measured in years, not days. These two properties — breadth and longevity — make the insurer's process estate one of the most demanding environments for a hyperautomation platform, and every architectural choice in this chapter must be evaluated against both.

The chapter's principles interact with the platform's operational model in a way worth making explicit. The *deployment* of the artefacts this chapter describes must go through a governed pipeline — versioned, reviewed, tested, and traceable. An artefact deployed by a manual action, outside the pipeline, is an artefact whose presence in production cannot be explained, and in a regulated institution an unexplained artefact is a finding.

The *monitoring* of the artefacts must cover both their functional correctness and their operational health. A process step that executes but produces wrong results is harder to detect than one that fails outright, because it does not raise an error — it raises a consequence, and the consequence may not be discovered until a customer complains or a regulator asks a question. The observability model should therefore include not only error rates and latencies but also *outcome monitoring*: checks that the results the process produces are within expected ranges, flagging anomalies for review.

Chapter in brief

For an insurer, the policy administration system is the system of record — it owns the authoritative truth about policies — and the Camunda platform orchestrates the processes around the policy without owning it. An insurer's core is a small landscape: the policy administration system at the centre, with claims and underwriting either modules of the same suite or separate platforms, supplied by vendors including Oracle's insurance platforms. Insurance integration is distinctive because the policy is a long-lived entity — running for years, accumulating endorsements, renewals, and claims — making insurance a domain of long-running processes around long-lived records. Each piece of authoritative truth has one owning system, and a process must read each from its system of record, not from whichever system is convenient.

Integrating with Oracle Insurance and the Major Suites

109.1 The established insurance platforms

This chapter treats integration with the established insurance platforms an architect most often meets — with *Oracle's* insurance platforms as the worked case, since Oracle offers a prominent and widely-deployed set of insurance-specific systems, and the patterns generalise to the other major suites.

109.2 Oracle insurance platforms

Oracle offers insurance-specific platforms within its broader financial-services portfolio, covering policy administration and related capabilities across life and annuities, health, and general insurance. For an architect, integrating with an Oracle insurance platform has two characteristics worth anticipating.

The first is that, as with Oracle FLEXCUBE in [Chapter 105](#), an Oracle insurance platform sits within the *Oracle technology ecosystem*. Its integration may involve Oracle's integration tooling and middleware, and the architect should establish concretely which interface styles the specific deployment offers — modern APIs, message-based interfaces, batch interfaces, or a mixture.

The second is that insurance platforms are *functionally deep*. A policy administration platform for life insurance, say, encodes an enormous amount of product logic — how each product accrues value, how premiums and benefits are calculated, how the many product variations behave. Its data model is correspondingly rich. An integrating process must learn the parts of that model it touches, and — crucially — must resist the temptation to *replicate* the platform's product logic in the process. The product logic belongs in the insurance platform; the Camunda process orchestrates around it.

109.3 The suite that is several systems

A specific complication of insurance suites deserves attention. An established insurance suite is often, internally, *several systems* — a policy module, a claims module, a billing module — that the vendor sells as a suite but that have their own data stores and their own interfaces, sometimes their own histories from before the vendor assembled the suite.

For an architect, this means “integrate with the insurance suite” is rarely one integration. It is integration with a policy capability, a claims capability, a billing capability — each potentially with its own interface style and its own data model. The integration must be designed per capability, and the architect must not assume that because two capabilities are sold as one suite they integrate the same way or even share a consistent data model.

Tip

An established insurance suite is often, internally, several systems — policy, claims, billing — each with its own interfaces and data model, sometimes its own pre-suite history. “Integrate with the suite” is therefore rarely one integration. Design the integration per capability, establish each one’s interface style and data model separately, and do not assume a single suite presents a single consistent integration surface.

109.4 The Oracle integration in code

Oracle’s insurance platforms are typically reached through their published APIs or through an integration middleware layer. Listing 109.1 is a job worker calling Oracle Health Insurance (OHI) to create a claim record.

Listing 109.1. A job worker creating a claim record in Oracle Health Insurance through its REST API.

```
1 @JobWorker(type = "create-claim-ohi")
2 public Map<String, Object> create(ActivatedJob job) {
3     OhiResponse r = ohiClient.post(
4         "/claims/v1/claims",
5         Map.of("policyRef", job.getVariable("policyRef"),
6             "claimType", job.getVariable("claimType"),
7             "eventDate", job.getVariable("eventDate")));
8
9     return Map.of(
10         "claimId", r.field("claimIdentifier"),
11         "claimStatus", r.field("status"));
12 }
```

109.5 Insurance-specific integration concerns

Insurance integration has concerns that banking integration does not share, and they are worth naming. *Long-tail claims*: an insurance claim can remain open for years, and the process that manages it must be designed for a lifetime measured in years, not days. *Actuarial data*: the process touches data that feeds actuarial models, and the integrity of that data is a pricing concern, not merely an operational one. *Regulatory reporting*: insurance regulators require specific, formatted reports on claims, reserves, and policyholder outcomes, and the process must produce the data those reports need. Each of these is a design input, not an afterthought.

Worked example: the product logic copied into the process

Problem. An insurer builds a Camunda new-business process for a life-insurance product. The Oracle insurance platform that administers the product can calculate the premium for a given applicant. But to make the process’s quote step fast and self-contained, the team re-implements the premium calculation — the rates, the factors, the product rules — inside the Camunda process, in DMN tables and code. The process ships. A year later, the insurer changes the product’s pricing. Assess what now has to happen.

Setup. We consider where the product’s premium logic now lives, and what a pricing change must touch.

Working. After the team’s design, the premium logic exists in *two* places: in the Oracle

insurance platform, which administers the product, and *re-implemented* inside the Camunda new-business process. A pricing change is a change to the product's logic, so it must be made in the platform — *and* in the process's copy:

pricing change $\Rightarrow$ update the platform + update the process's duplicate + keep them consistent.

If the two are not updated in perfect lockstep, the process quotes one premium and the platform administers the policy at another — the insurer's quote and its policy disagree about price.

Discussion. The team's optimisation — re-implement the premium calculation in the process for speed — created the insurance equivalent of the shadow ledger: duplicated authoritative logic that must now be kept consistent forever, and that will eventually drift. The premium calculation is *product logic*, and product logic belongs in the system that administers the product — the Oracle insurance platform. By copying it into the process, the team made every future pricing change a two-place change with a consistency risk, and created the possibility of the worst kind of insurance defect: a quoted premium that does not match the premium the policy is actually issued at. The correct design has the process *call* the insurance platform's premium calculation — the platform owns the product logic, the process orchestrates the quote journey and invokes the calculation when it needs a number. If calling the platform is too slow for an interactive quote, the answer is a deliberate performance design — not duplicating the authoritative logic. The lesson generalises the system-of-record principle from *data* to *logic*: the insurance platform is the system of record not only for the policy but for the product's rules, and a process that copies those rules has built a second, divergent source of truth about how the insurer's own products work.

Practical next steps

Check your insurance processes for any place a product's logic — premium rates, benefit calculations, product rules — has been re-implemented inside the process rather than called from the insurance platform. Each is duplicated authoritative logic that will drift from the platform. The platform owns the product logic; the process should call it, not copy it.

109.6 The insurance process estate

An insurer's process estate is distinctive in its *breadth* and its *timescales*. The breadth: an insurer runs processes across underwriting, policy administration, claims, renewals, complaints, and regulatory reporting — more process categories than most banks. The timescales: a claim can remain open for years, a policy for decades, and the processes that manage them must be designed for lifetimes measured in years, not days. These two properties — breadth and longevity — make the insurer's process estate one of the most demanding environments for a hyperautomation platform, and every architectural choice in this chapter must be evaluated against both.

The chapter's principles interact with the platform's operational model in a way worth making explicit. The *deployment* of the artefacts this chapter describes must go through a governed pipeline — versioned, reviewed, tested, and traceable. An artefact deployed by a manual action, outside the pipeline, is an artefact whose presence in production cannot be explained, and in a regulated institution an unexplained artefact is a finding.

The *monitoring* of the artefacts must cover both their functional correctness and their operational health. A process step that executes but produces wrong results is harder to detect than one that fails outright, because it does not raise an error — it raises a consequence, and the

consequence may not be discovered until a customer complains or a regulator asks a question. The observability model should therefore include not only error rates and latencies but also *outcome monitoring*: checks that the results the process produces are within expected ranges, flagging anomalies for review.

Chapter in brief

Oracle offers prominent insurance-specific platforms across life, health, and general insurance, and integrating with them — as with the other major suites — has two characteristics to anticipate: they sit within the Oracle technology ecosystem, so the interface style must be established concretely; and they are functionally deep, encoding extensive product logic in a rich data model. An established insurance suite is often internally several systems — policy, claims, billing — each with its own interfaces, so “integrate with the suite” is rarely one integration. The governing discipline is that the insurance platform is the system of record not only for policy *data* but for product *logic* — premium and benefit calculation — and a process must call that logic, never re-implement it.

CHAPTER 110

Underwriting, Claims, and the Insurance Process Estate

110.1 The processes around the policy

The previous chapters treated the insurer's systems. This chapter treats the *processes* an insurer's hyperautomation platform orchestrates around those systems — because the integration patterns are clearest when seen through the actual processes that use them.

110.2 New business and underwriting

The *new-business* process takes an application for cover and turns it — or declines to — into a policy. At its heart is *underwriting*: the assessment of the risk and the decision whether to accept it and at what price.

For integration, the new-business process is a clear example of orchestration across several systems. It gathers the application; it calls underwriting systems and rating engines to assess and price the risk; it may call external data sources — the agentic and decision layers of the manuscript are heavily used here; and, when the risk is accepted, it commands the *policy administration system* to create the policy. The new-business process is the orchestrator; the policy is created *in the PAS*; the underwriting decision draws on underwriting systems and on the DMN and predictive-model layers of [Chapter 6](#) and [Chapter 22](#).

110.3 Claims

The *claims* process runs from the first notification of a loss to the settlement or denial of the claim. It is, for many insurers, the process where hyperautomation delivers the most, and the most integration-intensive.

A claims process integrates widely: with the *claims system* for the claim record; with the *policy administration system* to confirm coverage — the [Chapter 108](#) discipline of reading policy truth from the PAS; with payment systems to settle; with fraud-detection capability; with external parties — assessors, repairers, medical providers. The claims process is one of the richest orchestration problems in insurance, and it shows the whole architecture at work: BPMN orchestrating a long-running, multi-system, multi-party, human-and-decision-heavy journey.

Figure [110.1](#) draws the claims process orchestrating across the insurer's systems.

110.4 Servicing and the long life of a policy

Beyond new business and claims, an insurer runs many *servicing* processes across a policy's long life: endorsements, renewals, beneficiary changes, address changes, lapses and reinstatements. Each is a process that changes or acts on the policy in the policy administration system.

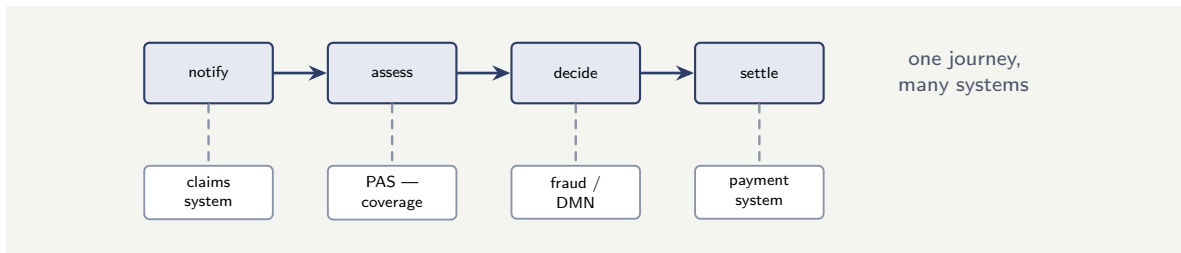


Figure 110.1. The claims process as orchestration across systems. A long-running journey — notify, assess, decide, settle — draws on the claims system, the policy administration system for coverage, fraud and decision capability, and the payment system. The claims process shows the whole architecture at work.

Together, they are the ongoing work of [Chapter 104](#)’s long-lived record — and each follows the same discipline: the PAS owns the policy, the process orchestrates the change.

110.5 A claims-handling process skeleton

A claims process is a concrete example of the chapter’s argument: human tasks for assessment, automated tasks for validation, and a gateway that routes between fast-track and complex-track.

Listing 110.1. A simplified claims-handling BPMN skeleton: an automated validation step, a gateway that routes simple versus complex claims, and a human assessment task for the complex path.

```

1 <bpmn:serviceTask id="validate" name="Validate Claim">
2   <bpmn:extensionElements>
3     <zeebe:taskDefinition type="validate-claim"/>
4   </bpmn:extensionElements>
5 </bpmn:serviceTask>
6 <bpmn:exclusiveGateway id="complexity"/>
7 <bpmn:serviceTask id="fastTrack" name="Auto-Settle"/>
8 <bpmn:userTask id="assess" name="Assessor Review"/>
9 <bpmn:sequenceFlow sourceRef="complexity"
10   targetRef="fastTrack">
11   <bpmn:conditionExpression>
12     =claimAmount < 500 and validationPassed
13   </bpmn:conditionExpression>
14 </bpmn:sequenceFlow>

```

110.6 Insurance-specific integration concerns

Insurance integration has concerns that banking integration does not share, and they are worth naming. *Long-tail claims*: an insurance claim can remain open for years, and the process that manages it must be designed for a lifetime measured in years, not days. *Actuarial data*: the process touches data that feeds actuarial models, and the integrity of that data is a pricing concern, not merely an operational one. *Regulatory reporting*: insurance regulators require specific, formatted reports on claims, reserves, and policyholder outcomes, and the process must produce the data those reports need. Each of these is a design input, not an afterthought.

Worked example: the claim decided before coverage was confirmed

Problem. An insurer's Camunda claims process, to be fast, runs the claim assessment and the fraud check in parallel as soon as a claim is notified, and proceeds to a settlement decision based on their results. Confirming coverage against the policy administration system is modelled as a step that happens *after* the settlement decision, just before payment. For most claims this is fine. But for a batch of claims against recently-lapsed policies, the insurer finds it has *decided to settle* claims that, once coverage was checked, should never have been entertained. Assess the modelling error.

Setup. We consider the order of the steps and what the settlement decision depends on.

Working. The settlement decision is made on the assessment and the fraud check. Coverage confirmation — the PAS check of whether the policy is even in force — is placed *after* the decision. But whether a policy is in force is *logically prior* to any question of settling a claim against it: a claim against a lapsed policy should be declined for that reason, before assessment effort is even spent.

coverage confirmation placed after the decision it should gate.

For recently-lapsed policies, the process reaches a settlement decision on claims it should have declined at the coverage step — a decision made, then contradicted by a check that came too late.

Discussion. The error is one of process *ordering*, and the worked example shows that integration is not only about *which* systems a process calls but *when*. The team parallelised assessment and fraud for speed — reasonable in itself — but placed coverage confirmation after the settlement decision, when coverage is logically a *precondition* of the whole claim: a claim against a policy not in force is not a claim to assess and settle, it is a claim to decline on coverage grounds. By checking coverage too late, the process spends assessment effort on, and reaches settlement decisions about, claims that the PAS would have eliminated up front. The fix is to make coverage confirmation an early gate: read the policy's status from the policy administration system — the [Chapter 108](#) discipline — near the start of the claims process, and let it gate whether the rest of the process runs at all. The lesson is that integrating with the insurer's systems correctly includes integrating with them in the right *order*: the system of record for coverage must be consulted when coverage logically matters — at the start — not as a late formality before payment. A process can call all the right systems and still be wrong if it calls them in the wrong sequence.

Practical next steps

For your insurer's claims process, check where coverage is confirmed against the policy administration system. If it is confirmed late — after assessment, after a settlement decision — the process is doing work on, and reaching decisions about, claims it may have to decline on coverage grounds. Make coverage an early gate, consulted when it logically matters.

110.7 The insurance process estate

An insurer's process estate is distinctive in its *breadth* and its *timescales*. The breadth: an insurer runs processes across underwriting, policy administration, claims, renewals, complaints, and regulatory reporting — more process categories than most banks. The timescales: a claim can remain open for years, a policy for decades, and the processes that manage them

must be designed for lifetimes measured in years, not days. These two properties — breadth and longevity — make the insurer’s process estate one of the most demanding environments for a hyperautomation platform, and every architectural choice in this chapter must be evaluated against both.

The chapter’s principles interact with the platform’s operational model in a way worth making explicit. The *deployment* of the artefacts this chapter describes must go through a governed pipeline — versioned, reviewed, tested, and traceable. An artefact deployed by a manual action, outside the pipeline, is an artefact whose presence in production cannot be explained, and in a regulated institution an unexplained artefact is a finding.

The *monitoring* of the artefacts must cover both their functional correctness and their operational health. A process step that executes but produces wrong results is harder to detect than one that fails outright, because it does not raise an error — it raises a consequence, and the consequence may not be discovered until a customer complains or a regulator asks a question. The observability model should therefore include not only error rates and latencies but also *outcome monitoring*: checks that the results the process produces are within expected ranges, flagging anomalies for review.

Chapter in brief

An insurer’s hyperautomation platform orchestrates the processes around the policy. The new-business process turns an application into a policy through underwriting, calling underwriting systems and rating engines and the DMN and predictive layers, then commanding the policy administration system to create the policy. The claims process — from notification to settlement — is the richest orchestration problem in insurance, integrating with the claims system, the PAS for coverage, fraud capability, payment systems, and external parties. Servicing processes act on the policy across its long life. Across all of them the discipline holds: the PAS owns the policy, the process orchestrates — and the systems must be called not only correctly but in the right order, coverage confirmed when it logically matters.

Insurance Integration Patterns

111.1 What insurance integration shares with banking, and what it does not

This chapter gathers the patterns of insurance-systems integration. Much is shared with core banking integration — the system-of-record discipline, the read/command/event interaction model, the accommodation of slow and batch systems, all of [Chapter 107](#) — and that shared material is not repeated. This chapter treats what is *distinctive* about insurance.

111.2 The long-running process around the long-lived policy

The defining distinctive pattern is the one [Chapter 104](#) named: insurance is a domain of long-running processes around a long-lived record. This shapes integration in concrete ways.

A process may run for a long time — an underwriting case awaiting a medical report, a claim awaiting an assessor — and during that time the *policy it concerns may itself change* in the policy administration system. The discipline this demands is that a long-running insurance process must not capture the policy’s state once, at the start, and assume it holds. It must re-read the policy state from the PAS at the points where current state matters — because in a process that runs for weeks, a policy detail read at the start may be stale by the time it is used. This is the system-of-record discipline extended along the time dimension: read the truth from the system of record not only *from the right place* but *at the right time*.

111.3 The policy lifecycle and correlation

A second distinctive pattern concerns *correlation*. Because a policy is long-lived and many processes touch it across its life, an insurer’s platform runs, at any moment, many process instances — claims, endorsements, renewals — that all relate to the *same policy*. When something happens to a policy, the platform may need to find the running processes that concern it.

This is the correlation machinery of the manuscript’s patterns part applied to the insurance domain: process instances are correlated by the policy they concern, so that an event about a policy — a cancellation, say — can reach the claim process running against that policy. An insurer’s integration design must include this: a clear, consistent way to correlate process instances to the policy, and through the policy to one another.

A long-running insurance process must not read a policy detail once at the start and trust it for the life of the process. A claim or underwriting case may run for weeks, and the policy it concerns can change in the policy administration system meanwhile. Re-read policy state from the PAS at the points where current state matters — the system-of-

record discipline applies along the time dimension, not only across systems. A policy detail captured weeks ago is not current truth.

111.4 Insurance-specific integration concerns

Insurance integration has concerns that banking integration does not share, and they are worth naming. *Long-tail claims*: an insurance claim can remain open for years, and the process that manages it must be designed for a lifetime measured in years, not days. *Actuarial data*: the process touches data that feeds actuarial models, and the integrity of that data is a pricing concern, not merely an operational one. *Regulatory reporting*: insurance regulators require specific, formatted reports on claims, reserves, and policyholder outcomes, and the process must produce the data those reports need. Each of these is a design input, not an afterthought.

Worked example: the renewal that used a stale sum insured

Problem. An insurer runs a Camunda renewal process for property policies. The process begins 60 days before a policy's renewal date: it reads the policy from the policy administration system — including the sum insured — prepares the renewal terms, sends them to the customer, and processes the renewal. One customer, 30 days before renewal, increases their sum insured through a separate endorsement process. When the renewal completes, it renews the policy at the *old* sum insured. Explain the error.

Setup. We trace when the renewal process read the sum insured and what happened to it afterward.

Working. The renewal process read the policy — including the sum insured — once, 60 days before renewal, at the start of the process. The process then runs for 60 days. At day 30, a separate endorsement process changes the sum insured in the policy administration system. The renewal process is holding the value it read at day 60 and never re-reads it:

sum insured read at day 60 $\neq$ sum insured in the PAS at completion.

The renewal completes using the stale day-60 value, renewing the policy at the old sum insured and ignoring the customer's increase.

Discussion. The renewal process integrated with the right system — it read the sum insured from the policy administration system, the correct system of record — but it read it at the wrong *time* relative to how long the process runs, and the worked example shows that this is a genuine and distinctive insurance integration failure. The renewal process runs for 60 days, and a policy is a long-lived record that other processes change; the sum insured the renewal read at the start is simply not guaranteed to be the sum insured at the end. The endorsement at day 30 is not an edge case — customers change their policies, and an insurer runs many processes against the same policy concurrently, exactly the correlation situation this chapter describes. The fix is the chapter's discipline: a long-running insurance process re-reads the policy state that matters from the PAS at the point of use — the renewal process should re-read the sum insured when it finalises the renewal terms, not rely on the value captured two months earlier. Better still, the process should be correlated to the policy so that a significant endorsement could notify the in-flight renewal. The lesson is that the system-of-record discipline has a time dimension: in a domain of long-running processes around long-lived records, reading the truth from the right system is necessary but not sufficient —

it must also be read at the right time, because truth read long ago is no longer truth.

Practical next steps

For each long-running insurance process your institution runs, identify the policy details it reads at the start and uses much later. Any such detail can have changed in the policy administration system in the interim. Re-read policy state from the PAS at the point of use, and consider correlating long-running processes to their policy so that significant changes can reach them.

111.5 The insurance process estate

An insurer's process estate is distinctive in its *breadth* and its *timescales*. The breadth: an insurer runs processes across underwriting, policy administration, claims, renewals, complaints, and regulatory reporting — more process categories than most banks. The timescales: a claim can remain open for years, a policy for decades, and the processes that manage them must be designed for lifetimes measured in years, not days. These two properties — breadth and longevity — make the insurer's process estate one of the most demanding environments for a hyperautomation platform, and every architectural choice in this chapter must be evaluated against both.

The chapter's principles interact with the platform's operational model in a way worth making explicit. The *deployment* of the artefacts this chapter describes must go through a governed pipeline — versioned, reviewed, tested, and traceable. An artefact deployed by a manual action, outside the pipeline, is an artefact whose presence in production cannot be explained, and in a regulated institution an unexplained artefact is a finding.

The *monitoring* of the artefacts must cover both their functional correctness and their operational health. A process step that executes but produces wrong results is harder to detect than one that fails outright, because it does not raise an error — it raises a consequence, and the consequence may not be discovered until a customer complains or a regulator asks a question. The observability model should therefore include not only error rates and latencies but also *outcome monitoring*: checks that the results the process produces are within expected ranges, flagging anomalies for review.

Chapter in brief

Insurance integration shares much with core banking — the system-of-record discipline, the read/command/event model, the accommodation of slow and batch systems — but has distinctive patterns. The defining one is the long-running process around the long-lived policy: a process may run for weeks while the policy it concerns changes in the policy administration system, so it must re-read policy state at the point of use, not capture it once at the start — the system-of-record discipline applies along the time dimension. And because many processes touch the same long-lived policy, an insurer's platform must correlate process instances to the policy, so that an event about a policy can reach the processes running against it.

PART XIX

Integrating with CRM and Master Data

The CRM and the Customer View

112.1 Where the customer lives

The core banking and insurance parts treated the systems that hold the *financial* record — accounts, policies. This part treats the systems that hold the *customer* record: the CRM — customer relationship management — systems, and the *master data management* systems that govern the single, trusted view of who a customer is.

These deserve a part because a hyperautomation process is, very often, a process *about a customer*: onboarding a customer, servicing a customer, selling to a customer. Such a process must know who the customer is, and that knowledge does not come from the core banking or policy system — it comes from the CRM and the master data layer. An institution that integrates well with its customer-data systems gives its processes a sound understanding of who they are acting for; one that does not builds processes that act on a confused or fragmented picture of the customer.

112.2 What a CRM is, in a financial institution

A CRM system manages the institution's relationships with its customers: it holds customer contact information, the history of interactions, the sales pipeline and opportunities, service cases, marketing engagement. It is where the institution's *relationship* with the customer — as distinct from the customer's accounts or policies — is recorded and worked.

The two most prominent CRM platforms a financial institution runs are *Salesforce* and *Microsoft Dynamics 365*. Both are large, cloud-based, API-rich platforms. For an architect, the good news is that both are built to be integrated with — they expose extensive APIs, and a Camunda process integrates with them through the connector mechanisms of [Part VI](#). A CRM is, in integration terms, a far more modern and congenial target than a legacy core banking system.

The integration question is therefore less about overcoming an awkward interface and more about *role*: what is the CRM's place relative to the process platform, and relative to the other systems that also know things about the customer.

112.3 The CRM is one view of the customer, not the only one

The crucial framing is this: the CRM holds *a* view of the customer, an important one — the relationship view — but it is *not the only system that knows the customer*. The core banking system knows the customer as the holder of accounts. The policy administration system knows them as a policyholder. A servicing system knows them as a caller. Each of these systems has its own customer record, and — this is the central problem the part addresses — those records are not automatically consistent with one another.

The same human being is, across an institution's systems, many records: a CRM contact, a core banking customer, a policyholder, each with its own identifier, its own copy of the name and address, its own version of the truth. A hyperautomation process that needs to act on

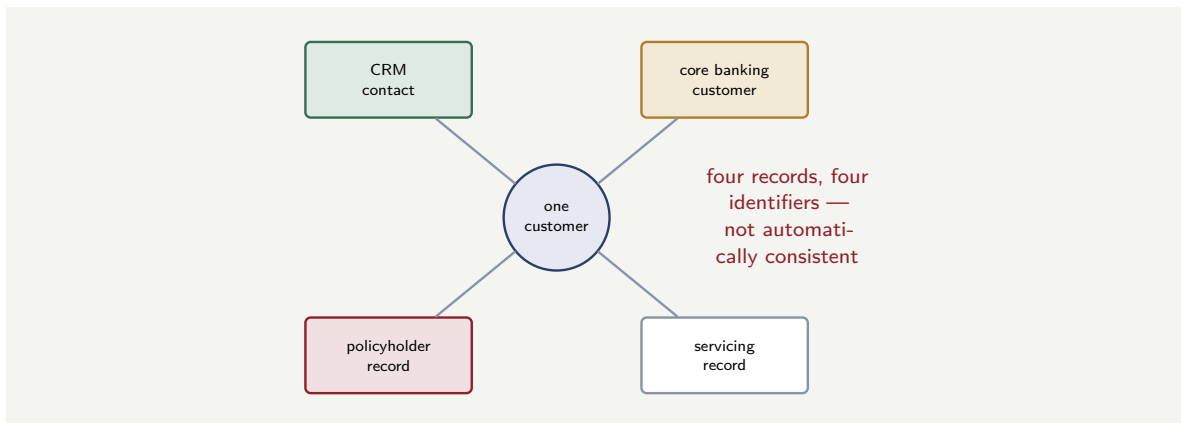


Figure 112.1. One customer, many records. The same person exists across an institution’s systems as a CRM contact, a core banking customer, a policyholder, a servicing record — each with its own identifier and its own copy of the truth, and not automatically consistent with the others.

“the customer” confronts this fragmentation directly. Resolving it is what the master data layer, treated in the next chapter, exists for.

Figure 112.1 draws the fragmentation of the customer across systems.

112.4 Reading the CRM from a process

A process step that needs the customer’s profile reads it from the CRM through a job worker, returning only the fields the current step requires.

Listing 112.1. A job worker fetching a customer profile from the CRM — the process carries the customer reference and reads the profile at the step that needs it.

```

1  @JobWorker(type = "fetch-customer-profile")
2  public Map<String, Object> fetch(ActivatedJob job) {
3      String custId = job.getVariable("customerId");
4
5      CustomerProfile p = crmClient.getCustomer(custId);
6
7      return Map.of(
8          "customerName", p.fullName(),
9          "segment",      p.segment(),
10         "relationshipAge", p.yearsAsCustomer());
11 }

```

112.5 CRM integration and the single customer view

The CRM is the institution’s attempt at a *single customer view*: one place where everything known about a customer is gathered. A process that integrates with the CRM must both *read* from it (to know the customer) and *write* to it (to record the process’s outcome as part of the customer record). The discipline is that the process *enriches* the CRM rather than duplicating it: the process reads the customer’s segment and relationship history from the CRM, uses them in its decisions, and writes the outcome — “application approved,” “complaint resolved” — back, so the CRM’s view of the customer grows with every process that touches them.

Worked example: which system the process should believe

Problem. A bank's Camunda servicing process handles a customer's request to update their address. The customer exists in the CRM (as a contact), in the core banking system (as a customer), and in a policy system (as a policyholder) — three records, three copies of the address. The design team asks a simple-seeming question: when the process updates the address, which system should it update? Their first instinct is "the CRM, since the process is a customer-relationship process." Assess this.

Setup. We consider what updating only one of the three records achieves, and what the customer actually expects.

Working. The customer has one address and expects the institution, as a whole, to know it. But the address is stored in three places — CRM, core banking, policy system. Updating only the CRM record changes one of the three:

update CRM only $\Rightarrow$ CRM correct, core banking and policy still stale.

The customer's statements would still go to the old address; their policy documents would still go to the old address. The process would have recorded the change in the relationship system and left the systems that actually *act* on the address untouched. Updating "the CRM, since this is a relationship process" answers the wrong question — the question is not which single system to pick but how the institution maintains *one* address across all the systems that hold it.

Discussion. The team's instinct exposes the central problem of this part: the customer is fragmented across systems, and an address-change process that picks any *one* system to update has not actually changed the customer's address as far as the institution is concerned — it has changed one of several copies. The customer does not experience the CRM, the core banking system, and the policy system as separate things; they experience "my bank," and they expect a single address change to take effect. The worked example is not yet a solution — the solution is the master data layer of the next chapter — but it sharpens the problem. There is no correct answer to "which single system should the process update," because the premise is wrong: the address is not owned by any one of these three systems in a way that makes the others its copies. The institution has genuine data fragmentation, and the address-change process needs a way to update the customer's address as a *single act* that reaches every system that holds it — which is precisely what master data management exists to provide. The lesson of this chapter is the diagnosis: a process that acts on "the customer" cannot simply pick a customer system, because there is no single customer system — there is a CRM view, a banking view, a policy view, and the process needs the customer-data architecture the rest of this part builds.

Practical next steps

For a customer-facing process your institution runs, list every system that holds a copy of the customer attribute the process reads or changes — name, address, contact details. If there is more than one, the process cannot simply pick a system to trust or update. Note the fragmentation; the next chapter's master data layer is what resolves it.

112.6 Data quality as a process responsibility

A process that reads from and writes to a CRM is also a process that *affects data quality*. Every field the process writes back becomes part of the customer record; every field it reads depends

on the quality of what was written before. A process that writes poorly validated data back to the CRM degrades the institution's customer view for every subsequent process that reads it. The discipline is that every write-back is validated, every field is typed, and the process treats the CRM's data quality as a responsibility, not merely an input.

The chapter's principles interact with the platform's operational model in a way worth making explicit. The *deployment* of the artefacts this chapter describes must go through a governed pipeline — versioned, reviewed, tested, and traceable. An artefact deployed by a manual action, outside the pipeline, is an artefact whose presence in production cannot be explained, and in a regulated institution an unexplained artefact is a finding.

The *monitoring* of the artefacts must cover both their functional correctness and their operational health. A process step that executes but produces wrong results is harder to detect than one that fails outright, because it does not raise an error — it raises a consequence, and the consequence may not be discovered until a customer complains or a regulator asks a question. The observability model should therefore include not only error rates and latencies but also *outcome monitoring*: checks that the results the process produces are within expected ranges, flagging anomalies for review.

Chapter in brief

A hyperautomation process is often a process about a customer, and it must know who the customer is — knowledge that comes from the CRM and the master data layer, not the core financial systems. A CRM — most prominently Salesforce or Microsoft Dynamics 365 — manages the institution's relationship with the customer: contacts, interactions, pipeline, service cases. Both are modern, API-rich, congenial integration targets. But the CRM holds only *one* view of the customer; the core banking system, the policy system, and servicing systems each hold their own customer record, with their own identifiers and their own copies of the truth, not automatically consistent. A process acting on “the customer” confronts this fragmentation directly — and cannot resolve it by simply choosing one system to believe.

Master Data Management and the Golden Record

113.1 The answer to fragmentation

The previous chapter ended on a problem: the customer is fragmented across many systems, each with its own record, and a process acting on “the customer” cannot simply pick one. This chapter treats the answer: *master data management* — MDM — and its central idea, the *golden record*.

113.2 What master data management does

Master data management is the discipline, and the class of system, that creates and maintains a single, trusted, consistent view of an entity — here, the customer — across an institution’s many systems.

An MDM system works by taking the customer records from all the contributing systems — the CRM, the core banking system, the policy system — and resolving them. It performs *matching*: identifying which records, across the different systems, actually refer to the same real person, despite different identifiers and slightly different spellings. It performs *survivorship*: where the matched records disagree — different addresses, different name spellings — determining which value is the trusted one. The result is the *golden record*: the single, reconciled, authoritative view of the customer, assembled from all the fragments.

The most prominent MDM platform in financial services is IBM’s — *IBM InfoSphere Master Data Management* — a multi-domain enterprise MDM system with matching, survivorship, and pre-built adapters for the major CRM and enterprise systems including Salesforce, Microsoft Dynamics, and SAP. Other MDM platforms exist, including offerings associated with the major data and CRM vendors. What they share is the function: turning the fragments into a golden record.

Figure 113.1 draws MDM assembling the golden record from the fragments.

113.3 The process and the golden record

For a hyperautomation process, the golden record is the answer to “who is the customer.” A process that needs to act on a customer integrates with the MDM layer to obtain the golden record — the trusted, reconciled view — rather than guessing from one fragmentary source.

This places the MDM layer firmly in the architecture. The CRM, core banking, and policy systems are *contributing systems*, each authoritative for *its own domain* — the CRM for the relationship, the core for the accounts, the PAS for the policies. The MDM system is authoritative for the *identity and the single view* of the customer. A process reads financial truth from the core, policy truth from the PAS — and *who the customer is* from the MDM golden record. Each system of record in its place; the MDM layer the system of record for customer identity itself.

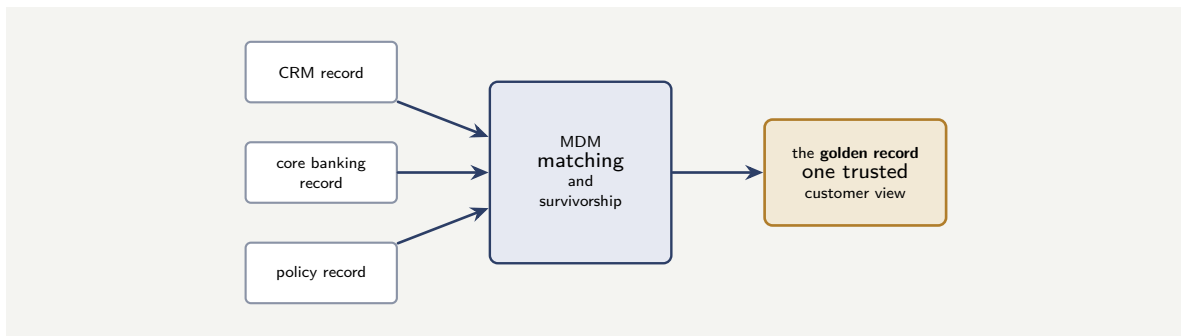


Figure 113.1. Master data management. The MDM system takes the fragmented customer records from the contributing systems, performs matching — identifying which records are the same person — and survivorship — choosing the trusted value where they disagree — to assemble the golden record: the single, reconciled, authoritative customer view.

Tip

The golden record is the system of record for *customer identity* — the single, reconciled answer to “who is this customer,” assembled by the MDM layer through matching and survivorship across all the contributing systems. A process that needs to act on a customer should obtain the customer’s identity and single view from the MDM golden record, not guess it from one fragmentary source such as the CRM alone. The CRM, core, and policy systems remain authoritative each for its own domain; the MDM layer is authoritative for identity.

113.4 CRM integration and the single customer view

The CRM is the institution’s attempt at a *single customer view*: one place where everything known about a customer is gathered. A process that integrates with the CRM must both *read* from it (to know the customer) and *write* to it (to record the process’s outcome as part of the customer record). The discipline is that the process *enriches* the CRM rather than duplicating it: the process reads the customer’s segment and relationship history from the CRM, uses them in its decisions, and writes the outcome — “application approved,” “complaint resolved” — back, so the CRM’s view of the customer grows with every process that touches them.

Worked example: the duplicate customer that became two relationships

Problem. A bank has no MDM layer. A long-standing customer — who holds a current account (in core banking, as “Jonathan A. Smith”) — applies for a new insurance product. The onboarding process, finding no exact match, creates a fresh customer record (in the policy system and the CRM, as “Jon Smith”). The bank now treats one person as two customers. Six months later this causes concrete problems: the customer is sent duplicate and inconsistent communications, a risk assessment misses that the two “customers” are one person with a combined exposure, and a service agent cannot see the full relationship. Explain how an MDM layer would have changed this.

Setup. We consider what happened at onboarding without MDM, and what the matching

function of an MDM layer would have done.

Working. *Without MDM.* The onboarding process searched for an exact match, found none — because “Jon Smith” is not an exact string match for “Jonathan A. Smith” — and created a new customer. The bank now has two records for one person, with nothing reconciling them:

one person → two unlinked customer records.

The consequences — duplicate communications, missed combined exposure, incomplete service view — all flow from the bank genuinely not knowing the two records are one person. *With MDM.* The MDM layer’s *matching* function does not rely on exact string equality; it identifies that “Jonathan A. Smith” and “Jon Smith,” with the same other identifying attributes, are very probably the same person. The onboarding process, consulting the MDM layer, would have been told this is an *existing* customer, and the new product would have been attached to the existing golden record:

one person → one golden record → one relationship.

Discussion. The worked example shows the cost of the fragmentation the previous chapter diagnosed, and the concrete value of the MDM layer. Without MDM, customer identity is decided by whatever ad-hoc matching each process does for itself — here, an exact-match search, which is brittle: “Jon” is not “Jonathan,” so a real returning customer was treated as new, and the bank split one person into two. Every downstream problem — the duplicate mailings, the missed combined exposure, the fractured service view — is a symptom of that one unresolved identity. An MDM layer addresses the problem at its root: it provides matching designed for exactly this — recognising that records which are not string-identical nonetheless refer to the same person — and it maintains the golden record so that there is one authoritative answer to who the customer is. Had the onboarding process consulted the MDM layer, the returning customer would have been recognised, and the new policy attached to the one existing relationship. The lesson is the chapter’s central point: customer identity is too important and too error-prone to be left to each process’s own matching, and the MDM layer exists to be the system of record for identity — so that “is this someone we already know” has one reliable answer, and one person is one customer.

Practical next steps

Ask whether your institution has an MDM layer and whether customer-facing processes consult it to resolve customer identity — or whether each process does its own matching. Ad-hoc per-process matching, especially exact-match matching, splits real customers into duplicates. The MDM golden record is the one place customer identity should be resolved.

113.5 Data quality as a process responsibility

A process that reads from and writes to a CRM is also a process that *affects data quality*. Every field the process writes back becomes part of the customer record; every field it reads depends on the quality of what was written before. A process that writes poorly validated data back to the CRM degrades the institution’s customer view for every subsequent process that reads it. The discipline is that every write-back is validated, every field is typed, and the process treats the CRM’s data quality as a responsibility, not merely an input.

The chapter’s principles interact with the platform’s operational model in a way worth making explicit. The *deployment* of the artefacts this chapter describes must go through a

governed pipeline — versioned, reviewed, tested, and traceable. An artefact deployed by a manual action, outside the pipeline, is an artefact whose presence in production cannot be explained, and in a regulated institution an unexplained artefact is a finding.

The *monitoring* of the artefacts must cover both their functional correctness and their operational health. A process step that executes but produces wrong results is harder to detect than one that fails outright, because it does not raise an error — it raises a consequence, and the consequence may not be discovered until a customer complains or a regulator asks a question. The observability model should therefore include not only error rates and latencies but also *outcome monitoring*: checks that the results the process produces are within expected ranges, flagging anomalies for review.

Chapter in brief

Master data management is the answer to customer fragmentation. An MDM system takes the customer records from all contributing systems — CRM, core banking, policy — and resolves them through *matching* (identifying which records are the same person despite different identifiers and spellings) and *survivorship* (choosing the trusted value where they disagree), producing the *golden record*: the single, reconciled, authoritative customer view. IBM InfoSphere MDM is the prominent platform in financial services. The MDM layer is the system of record for customer *identity*, while the CRM, core, and policy systems remain authoritative each for its own domain — and a process acting on a customer should obtain identity from the golden record, not guess it from one fragmentary source.

Integrating with Salesforce and Microsoft Dynamics

114.1 The two CRM platforms in practice

This chapter treats integration with the two CRM platforms a financial institution most often runs — *Salesforce* and *Microsoft Dynamics 365* — in the practical depth a practitioner needs.

114.2 The CRM as a modern integration target

Both Salesforce and Dynamics 365 are large, mature, cloud-based platforms with extensive APIs, and integrating a Camunda process with either is, in mechanical terms, comfortable: it is API integration through the connector mechanisms of [Part VI](#), against well-documented, modern interfaces. Both platforms also have their own automation and workflow capabilities — and this raises the most important architectural question of the chapter.

114.3 Where the process runs: the CRM's automation, or Camunda

A CRM platform like Salesforce or Dynamics comes with its own workflow, process-automation, and low-code tooling. An institution adopting Camunda for hyperautomation must therefore answer a real question: when a process concerns the customer relationship, does it run in the CRM's own automation, or in Camunda?

The principle for deciding follows the manuscript's whole argument. Automation that is *local to the CRM* — a simple within-CRM task, a CRM-only data update, a piece of relationship-management logic that touches nothing else — can reasonably run in the CRM's own tooling; pulling it into Camunda would be needless. But a process that is genuinely *cross-system* — an onboarding journey that spans the CRM, the core banking system, identity verification, and decisioning; a claims process; a lending process — belongs in Camunda, the orchestrator, even though it touches the CRM. The CRM is then one of the systems the Camunda process integrates with, not the place the end-to-end process lives.

The anti-pattern to avoid is the end-to-end business process implemented inside the CRM's automation tooling because it happened to start with a customer-relationship step. A cross-system process implemented in a CRM's low-code workflow becomes a process locked inside the CRM — invisible to the enterprise orchestration tooling, hard to govern as a whole, exactly the island of [Chapter 17](#). The dividing line: CRM-local automation may live in the CRM; cross-system orchestration lives in Camunda.

Figure [114.1](#) draws the dividing line between CRM-local automation and Camunda orchestration.

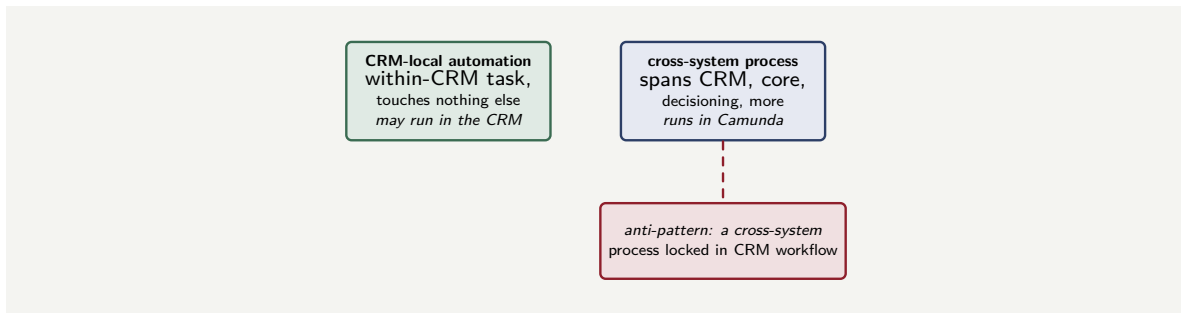


Figure 114.1. Where a process runs. CRM-local automation — a within-CRM task that touches nothing else — may run in the CRM’s own tooling; a cross-system process belongs in Camunda. The anti-pattern is an end-to-end cross-system process locked inside the CRM’s low-code workflow.

114.4 Reading and writing the CRM

When a Camunda process does integrate with the CRM, the interaction follows the read/command/event model of [Chapter 107](#). The process *reads* the CRM for relationship information — the customer’s interaction history, their open service cases. It *commands* the CRM to record things — log an interaction, update a service case, create an opportunity. And where the CRM can emit *events* — a case raised, a customer action — an inbound connector lets a process start or continue in response. The CRM is, in this respect, an external system like any other, integrated through clean connectors, with the CRM remaining authoritative for the relationship data it owns.

114.5 The two integrations in code

Salesforce and Microsoft Dynamics are both reached through REST APIs, and the worker shape is identical — only the endpoint and the field names differ.

Listing 114.1. A job worker updating a Salesforce contact record through the Salesforce REST API.

```

1 @JobWorker(type = "update-crm-contact")
2 public void update(ActivatedJob job) {
3     String sfId = job.getVariable("salesforceContactId");
4     sfClient.patch(
5         "/services/data/v60.0/objects/Contact/" + sfId,
6         Map.of("Description",
7             job.getVariable("caseOutcome")));
8 }
  
```

114.6 CRM integration and the single customer view

The CRM is the institution’s attempt at a *single customer view*: one place where everything known about a customer is gathered. A process that integrates with the CRM must both *read* from it (to know the customer) and *write* to it (to record the process’s outcome as part of the customer record). The discipline is that the process *enriches* the CRM rather than duplicating it: the process reads the customer’s segment and relationship history from the CRM, uses them in its decisions, and writes the outcome — “application approved,” “complaint resolved” — back, so the CRM’s view of the customer grows with every process that touches them.

Worked example: the onboarding process built in the CRM

Problem. A bank's customer-onboarding process begins, naturally, with a customer-relationship step — capturing the prospect in the CRM. Because of that starting point, the team builds the *entire* onboarding process — identity verification, core-banking account creation, credit decisioning, funding — in Salesforce's own low-code workflow tooling, calling out to the other systems from there. It works. A year later the bank's enterprise team, rolling out Camunda-based process observability and standardised process governance across all major processes, finds onboarding is not covered. Assess the situation.

Setup. We consider what kind of process onboarding is, and where the team placed it.

Working. Onboarding is a *cross-system* process: it spans the CRM, identity verification, the core banking system, credit decisioning, and funding. By the chapter's principle, a cross-system process belongs in Camunda, the orchestrator. The team instead built it in Salesforce's low-code workflow, because onboarding *began* with a CRM step. The result:

a cross-system process implemented inside one system's tooling.

When the enterprise rolls out Camunda-based observability and governance for major processes, onboarding — which lives in Salesforce, not Camunda — is invisible to it: it is an island, exactly as [Chapter 17](#) described, outside the enterprise's process tooling and standards.

Discussion. The team made a placement error driven by a superficial cue — the process *starts* with a customer-relationship step, so it felt like a CRM process — and the worked example shows why the starting step is the wrong basis for the decision. What determines where a process belongs is not its first step but its *whole shape*: onboarding spans five systems and is a textbook cross-system orchestration, and a cross-system orchestration belongs in the orchestrator, Camunda, regardless of which system its first step touches. Building it in Salesforce's workflow tooling produced a working process and an enterprise liability: an end-to-end business process locked inside one system, invisible to the enterprise's process observability and ungoverned by its standards — the island of [Chapter 17](#), here created by mistaking a CRM-starting process for a CRM-local one. The correct design runs the onboarding process in Camunda, which orchestrates across the CRM, identity, core banking, decisioning, and funding — with the CRM as one integrated system among five. The CRM's own automation would be reserved for genuinely CRM-local tasks. The lesson is the chapter's dividing line stated sharply: a process is CRM-local or cross-system by its whole reach, not by its first step, and a cross-system process placed in the CRM because it began there is an island the enterprise will later have to discover and rebuild.

Practical next steps

Review the customer-facing processes your institution runs inside its CRM's automation tooling. For each, ask whether it is genuinely CRM-local or actually a cross-system process that began with a CRM step. Cross-system processes built in CRM workflow are islands — invisible to enterprise process tooling. They belong in Camunda, with the CRM as one integrated system.

114.7 Data quality as a process responsibility

A process that reads from and writes to a CRM is also a process that *affects data quality*. Every field the process writes back becomes part of the customer record; every field it reads depends on the quality of what was written before. A process that writes poorly validated data back

to the CRM degrades the institution's customer view for every subsequent process that reads it. The discipline is that every write-back is validated, every field is typed, and the process treats the CRM's data quality as a responsibility, not merely an input.

The chapter's principles interact with the platform's operational model in a way worth making explicit. The *deployment* of the artefacts this chapter describes must go through a governed pipeline — versioned, reviewed, tested, and traceable. An artefact deployed by a manual action, outside the pipeline, is an artefact whose presence in production cannot be explained, and in a regulated institution an unexplained artefact is a finding.

The *monitoring* of the artefacts must cover both their functional correctness and their operational health. A process step that executes but produces wrong results is harder to detect than one that fails outright, because it does not raise an error — it raises a consequence, and the consequence may not be discovered until a customer complains or a regulator asks a question. The observability model should therefore include not only error rates and latencies but also *outcome monitoring*: checks that the results the process produces are within expected ranges, flagging anomalies for review.

Chapter in brief

Salesforce and Microsoft Dynamics 365 are the two CRM platforms a financial institution most often runs, and both are modern, API-rich, comfortable integration targets. The key architectural question is where a process runs: both CRMs have their own automation tooling, and CRM-*local* automation — a within-CRM task touching nothing else — may reasonably run there, but a *cross-system* process belongs in Camunda even when it touches the CRM. A cross-system process built in the CRM's low-code workflow becomes an island, invisible to enterprise process tooling. When a Camunda process does integrate with the CRM, it follows the read/command/event model, with the CRM authoritative for the relationship data it owns.

Customer Data Integration Patterns

115.1 Bringing the customer-data systems together

This final chapter of the part gathers the patterns for integrating the customer-data systems — the CRM and the MDM layer — into a coherent whole, so that a hyperautomation process has a sound, single understanding of the customer it acts for.

115.2 The pattern: identity from MDM, domain data from the owning system

The governing pattern, drawing the part together, is a division of authority that a process must respect.

The *MDM layer* is the system of record for customer *identity*: who the customer is, the single golden-record view, the resolution of the fragments into one person. A process establishes *who* it is acting for by consulting the MDM layer.

Each *contributing system* is the system of record for its own *domain* of customer data: the CRM for relationship and interaction data, the core banking system for the account relationship, the policy system for policies. Once a process knows *who* the customer is — from MDM — it reads each kind of customer information from the system that owns it.

A process about a customer therefore typically does two things in sequence: first, resolve identity through the MDM layer to obtain the golden record and its links; then, using those links, read the specific domain data it needs from each owning system. Identity first, from MDM; domain data second, from the owning systems.

115.3 Keeping customer data consistent across systems

The hardest customer-data pattern is the one the part opened with: when a customer's data *changes* — a new address, a name change — the change must reach every system that holds it, consistently.

There is no trivial solution, but there is a sound architecture. A customer-data change is itself a *process* — and a process is exactly what a hyperautomation platform orchestrates well. A change-of-address process, modelled in Camunda, can orchestrate the update across every system that holds the address: updating the CRM, the core banking system, the policy system, and notifying the MDM layer, as a single governed process with the error handling and compensation of the manuscript's patterns. The fragmentation of customer data is real; the orchestrated update process is how an institution keeps it consistent despite the fragmentation. The platform that caused difficulty in [Chapter 112's](#) worked example — the address updated in only one system — is, correctly built, the very thing that solves it: an orchestrated process that updates them all.

A customer-data change — an address, a name — must reach every system that holds that data, or the institution’s view of the customer fragments further with every change. Do not let a process update one system and leave the others stale. Model the customer-data change as an orchestrated Camunda process that updates every holding system — CRM, core, policy — and the MDM layer, with proper error handling and compensation. The fragmentation is real; an orchestrated update process is how consistency is maintained despite it.

115.4 CRM integration and the single customer view

The CRM is the institution’s attempt at a *single customer view*: one place where everything known about a customer is gathered. A process that integrates with the CRM must both *read* from it (to know the customer) and *write* to it (to record the process’s outcome as part of the customer record). The discipline is that the process *enriches* the CRM rather than duplicating it: the process reads the customer’s segment and relationship history from the CRM, uses them in its decisions, and writes the outcome — “application approved,” “complaint resolved” — back, so the CRM’s view of the customer grows with every process that touches them.

Worked example: identity first, then domain data

Problem. A bank builds a Camunda process that produces a consolidated “customer overview” for a service agent: the customer’s accounts, their policies, their recent interactions, all on one screen. The process is given only a phone number — the number the customer called from. A junior architect proposes the process should query the core banking system, the policy system, and the CRM each by phone number, and combine whatever each returns. A senior architect says the process should go through the MDM layer first. Explain the difference.

Setup. We consider what querying each system independently by phone number yields, versus resolving identity through MDM first.

Working. *Query each system by phone number.* Each of the three systems is searched, independently, for records with that phone number. But the customer’s records across the systems may not all carry the same phone number; some may carry an old one, some none; and the phone number might match the wrong person in one system. The process combines whatever each search happens to return:

three independent searches $\Rightarrow$ possibly partial, possibly mismatched results.

Go through MDM first. The process gives the phone number to the MDM layer, which resolves it to the customer’s *golden record* — the confirmed identity — and the golden record carries the *links* to that customer’s record in each contributing system. The process then reads accounts from core banking, policies from the policy system, interactions from the CRM, each *by the correct, confirmed link*:

resolve identity once via MDM $\Rightarrow$ correct domain data from each system.

Discussion. The senior architect is right, and the worked example captures the part’s governing pattern. The junior architect’s approach treats the phone number as if it were a reliable key into every system — but it is not: the customer-data systems are fragmented, each

with its own identifiers, and a phone number is a weak, possibly-stale, possibly-ambiguous attribute that will not reliably find the right record in all three. Combining three independent weak-key searches produces a customer overview that may be partial (a system where the number did not match) or wrong (a system where it matched someone else) — and a service agent acting on a wrong or partial overview is a real risk. The sound pattern is identity-first: resolve the phone number to a confirmed identity through the MDM layer *once*, obtain the golden record with its authoritative links to each system, and then read each kind of domain data from its owning system by the correct link. This is the division of authority the chapter sets out — the MDM layer is the system of record for identity, and identity must be established before domain data is gathered. The lesson is that a process about a customer has two distinct questions — “who is this” and “what is their data” — and they must be answered in that order, by the right systems: identity from MDM first, domain data from the owning systems second. Skipping the first and using a weak key everywhere is how a consolidated customer view becomes a consolidated guess.

Practical next steps

For any process that gathers data about a customer from multiple systems, check whether it resolves the customer’s identity through the MDM layer first, or queries each system independently by some attribute. Independent attribute-based queries against fragmented systems yield partial or mismatched results. Resolve identity once via MDM, then read domain data from each owning system by the confirmed link.

115.5 Data quality as a process responsibility

A process that reads from and writes to a CRM is also a process that *affects data quality*. Every field the process writes back becomes part of the customer record; every field it reads depends on the quality of what was written before. A process that writes poorly validated data back to the CRM degrades the institution’s customer view for every subsequent process that reads it. The discipline is that every write-back is validated, every field is typed, and the process treats the CRM’s data quality as a responsibility, not merely an input.

The chapter’s principles interact with the platform’s operational model in a way worth making explicit. The *deployment* of the artefacts this chapter describes must go through a governed pipeline — versioned, reviewed, tested, and traceable. An artefact deployed by a manual action, outside the pipeline, is an artefact whose presence in production cannot be explained, and in a regulated institution an unexplained artefact is a finding.

The *monitoring* of the artefacts must cover both their functional correctness and their operational health. A process step that executes but produces wrong results is harder to detect than one that fails outright, because it does not raise an error — it raises a consequence, and the consequence may not be discovered until a customer complains or a regulator asks a question. The observability model should therefore include not only error rates and latencies but also *outcome monitoring*: checks that the results the process produces are within expected ranges, flagging anomalies for review.

Chapter in brief

Integrating the customer-data systems coherently rests on a division of authority: the

MDM layer is the system of record for customer *identity* — the golden record — while each contributing system (CRM, core, policy) is the system of record for its own *domain* of customer data. A process about a customer therefore works in two steps: resolve identity through MDM to obtain the golden record and its links, then read domain data from each owning system. And because customer data is genuinely fragmented, a customer-data change must reach every holding system — best done as an orchestrated Camunda process that updates the CRM, core, policy systems, and MDM layer together, with proper error handling. Identity first from MDM; domain data second from the owners.

PART XX

Integrating with Payment Systems

The Payments Landscape

116.1 Why payments deserve a part

The preceding parts treated the systems that hold a financial institution's records — the core banking system, the policy administration system, the CRM. This part treats a different kind of system: the systems through which money actually *moves*. *Payments* — moving value from one party to another — is, for a bank, among the most important things its processes orchestrate, and the payments landscape is intricate enough to need a part of its own.

Payments deserve dedicated treatment for two reasons. The first is that payments are *consequential*: a payment moves real money, and a payment made wrongly — twice, to the wrong party, for the wrong amount — is a real, immediate loss. The second is that the payments landscape is *a landscape of many parties and standards*, not a single system, and a process that orchestrates payments must integrate with the right part of that landscape in the right way.

116.2 The parties in a card payment

To integrate with payments, an architect must first understand the landscape, and the clearest entry point is the *card payment* — because a card payment, familiar as it is, involves a surprising number of distinct parties.

When a cardholder pays a merchant by card, the transaction passes through: the *cardholder* and their *issuing bank* — the bank that issued the card and holds the cardholder's account; the *merchant* and their *acquiring bank* — the bank that holds the merchant's account and accepts the payment on their behalf; a *payment gateway* and *processor* — the technical infrastructure that carries and processes the transaction; and, connecting the issuing and acquiring sides, the *card scheme* or *network*.

The *card schemes* — Visa and Mastercard the most prominent — are central to the landscape and worth understanding precisely. A card scheme is *not* the cardholder's bank and not the merchant's bank: it is the *network* that connects them. The scheme sets the rules and standards for how card transactions are processed, and it routes the transaction data between the issuing side and the acquiring side. Visa and Mastercard operate the rails; the banks hold the accounts; the scheme is the rulebook and the routing in between.

Figure 116.1 sets out the parties in a card payment.

116.3 Schemes, processors, and PSPs

Three categories of payments organisation recur, and an architect should hold them distinct because they do different things.

A *card scheme* or *network* — Visa, Mastercard — sets the rules and routes transactions between issuers and acquirers, as just described.

A *payment processor* handles the technical work of a payment — authorization, clearing, and settlement — on behalf of an issuer or an acquirer. The processor is the operational engine that carries out the transaction.

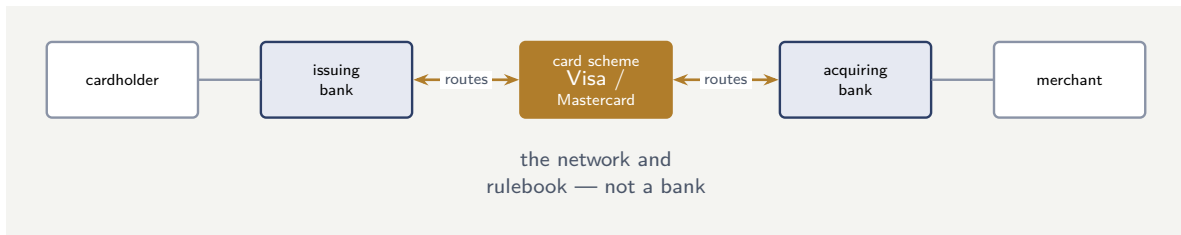


Figure 116.1. The parties in a card payment. The card scheme — Visa, Mastercard — is the network connecting the cardholder’s issuing bank and the merchant’s acquiring bank: it sets the rules and routes the transaction, but it is not a bank and holds no accounts.

A *payment service provider* — a PSP — offers a broader suite of payment services to merchants: payment processing plus fraud detection, support for multiple payment methods, and integration tooling. *PayPal* is a prominent PSP, alongside others such as Stripe and Adyen. A PSP is, for many merchants, the single integration point through which they accept a range of payment types.

For a hyperautomation architect, the practical consequence is that “integrate with payments” is ambiguous until the part of the landscape is named. Integrating with a card scheme, with a processor, with a PSP, or with the bank’s own internal payment systems are different integrations — and the next chapters treat the main ones.

116.4 A payment initiation in code

A payment process begins with a payment-initiation step — a job worker that calls the payment gateway or PSP to start a transaction.

Listing 116.1. A job worker initiating a payment through a payment service provider’s API.

```

1 @JobWorker(type = "initiate-payment")
2 public Map<String, Object> initiate(ActivatedJob job) {
3     PaymentResponse r = pspClient.initiatePayment(
4         job.getVariable("paymentRef"),
5         job.getVariable("amount"),
6         job.getVariable("currency"),
7         job.getVariable("beneficiaryAccount"));
8
9     return Map.of(
10         "transactionId", r.transactionId(),
11         "paymentStatus", r.status());
12 }
  
```

The idempotency discipline of [Chapter 104](#) applies here with full force: the `paymentRef` is the idempotency key, so a retried worker does not initiate a second payment for the same instruction.

116.5 Payment integration and idempotency

Payment integration is the domain where idempotency is most consequential, because a duplicated payment is duplicated money. Every write operation against a payment system — initiating a payment, capturing a charge, issuing a refund — must carry an idempotency key that the downstream system honours, so a retried worker does not create a second financial transaction for the same instruction. The key is typically a business reference — the payment

instruction identifier, the refund reference — that is unique to the logical operation. A payment worker without an idempotency key is not a bug; it is a financial incident waiting for its first retry.

Worked example: the scheme that was mistaken for a bank

Problem. A team building a Camunda payment process describes its design as: “the process sends the payment to Visa, and Visa moves the money from the customer’s account to the merchant’s account.” An architect says this description reveals a misunderstanding that will lead to a wrong integration design. Identify the misunderstanding.

Setup. We compare the team’s description against what a card scheme actually does.

Working. The team’s description has Visa *moving money between accounts*. But a card scheme is not a bank and holds no customer or merchant accounts. The customer’s account is at the *issuing bank*; the merchant’s account is at the *acquiring bank*. What Visa does is *route* the transaction — carry the authorization and clearing data between the issuing and acquiring sides, under its rules — so that the *banks* move the money:

scheme routes the transaction; the banks hold the accounts and move the money.

The team has collapsed the scheme and the banks into one thing, and an integration designed on that collapsed model will be wrong about where the authoritative account state lives, where money actually moves, and what the process is even integrating with.

Discussion. The misunderstanding is small as a sentence and large as a design error, which is why the worked example opens the part on it. “Visa moves the money” treats the card scheme as the system that holds accounts and performs the transfer — but the scheme is the *network and rulebook*, and the accounts and the actual movement of money belong to the issuing and acquiring *banks*. A process designed on the team’s mental model would look for account state and settlement in the wrong place, would misunderstand which party it is actually integrating with, and would not know where the system-of-record discipline of [Chapter 104](#) even applies — because it has not correctly identified the systems. The correct model distinguishes the parties: the scheme routes, the banks hold accounts and move money, the processor does the technical processing, and a PSP — if involved — is the merchant’s broader service provider. An architect integrating a payment process must know *which* of these the process is talking to and what each does, because they are genuinely different systems with different roles. The lesson is the chapter’s: “payments” is a landscape of distinct parties, and an integration cannot be designed until the architect has stopped saying “send it to Visa” and started naming precisely which party does what.

Practical next steps

For any payment process your institution is designing, write down precisely which payments party each integration point talks to — a card scheme, a processor, a PSP, an issuing or acquiring bank, an internal payment system — and what that party does. A design that blurs these into “the payment system” or “Visa” has not yet identified what it is integrating with.

116.6 Payment regulation and compliance

Payment processing is among the most heavily regulated domains in financial services. Anti-money-laundering screening, sanctions checking, transaction monitoring, and regulatory reporting are not optional add-ons; they are mandatory steps in every payment flow. A process

that handles payments must include these steps as modelled, governed activities — not as post-hoc checks — because the regulator expects to see them in the process, to audit their execution, and to hold the institution accountable for their completeness. A payment process without embedded compliance steps is not merely incomplete; it is, in a regulated institution, non-compliant.

The chapter's principles interact with the platform's operational model in a way worth making explicit. The *deployment* of the artefacts this chapter describes must go through a governed pipeline — versioned, reviewed, tested, and traceable. An artefact deployed by a manual action, outside the pipeline, is an artefact whose presence in production cannot be explained, and in a regulated institution an unexplained artefact is a finding.

The *monitoring* of the artefacts must cover both their functional correctness and their operational health. A process step that executes but produces wrong results is harder to detect than one that fails outright, because it does not raise an error — it raises a consequence, and the consequence may not be discovered until a customer complains or a regulator asks a question. The observability model should therefore include not only error rates and latencies but also *outcome monitoring*: checks that the results the process produces are within expected ranges, flagging anomalies for review.

Chapter in brief

Payments — moving value between parties — is among the most consequential things a bank's processes orchestrate, and the payments landscape is intricate enough to need a part of its own. A card payment alone involves the cardholder and issuing bank, the merchant and acquiring bank, gateways and processors, and the card scheme. The *card schemes* — Visa, Mastercard — are the networks: they set the rules and route transactions between issuers and acquirers, but they are not banks and hold no accounts. Three categories recur and must be kept distinct: card schemes (rules and routing), payment processors (the technical authorization, clearing, settlement), and PSPs such as PayPal (a broad payment-services suite for merchants). "Integrate with payments" is ambiguous until the part of the landscape is named.

Authorization, Clearing, and Settlement

117.1 A payment is not one event

The previous chapter set out the parties. This chapter sets out the *stages* — because the single most important thing for a process architect to understand about a payment is that it is *not one event*. A payment, especially a card payment, unfolds in distinct stages separated in time, and a process that treats it as a single instantaneous act will be wrong about money.

117.2 The three stages

A card payment moves through three distinct stages.

Authorization is the first: a check, at the moment of the transaction, that the payment *can* proceed — that the card is valid, that funds or credit are available. Authorization typically places a *hold* on the funds but does not yet move them. It is fast, it happens at the point of sale, and a successful authorization is a *promise*, not a completed transfer.

Clearing is the stage where the transaction details are exchanged between the acquiring and issuing sides through the scheme, and the obligations between the banks are calculated. Clearing typically happens *after* authorization — often in batches, often later the same day or the next.

Settlement is the stage where the money actually moves — where the funds are genuinely transferred between the banks to discharge the obligations that clearing calculated. Settlement is distinct from authorization and happens later, frequently the next day or on a defined settlement cycle.

The gap between these stages is the crucial fact. When a cardholder's payment is authorized at a shop, the money has *not yet moved*; it moves at settlement, perhaps a day later. Authorization, clearing, and settlement are separated in time, and a process that orchestrates payments must model that separation.

Figure 117.1 sets out the three stages and the time between them.

117.3 What the stages mean for a process

This staged nature has direct consequences for a Camunda process that orchestrates a payment.

A process must not treat a successful *authorization* as a completed payment. Authorization is a promise; the process may proceed on the strength of it — many business processes do — but it must know that the money has not yet moved, and it must account for the possibility that the payment does not complete at settlement.

A process that needs to know a payment has *genuinely completed* must wait for, or check, *settlement* — not authorization. A fulfilment process that releases goods on authorization is

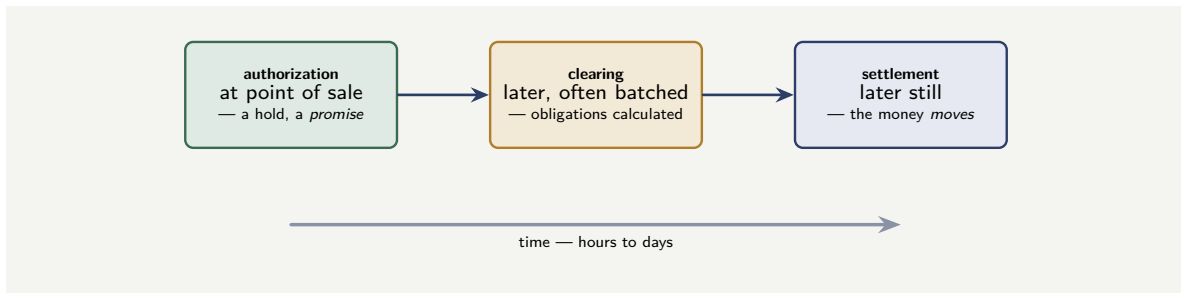


Figure 117.1. A card payment is not one event. Authorization places a hold and makes a promise at the point of sale; clearing later calculates the obligations between banks; settlement, later still, is where the money actually moves. A process that orchestrates payments must model this separation in time.

making a business decision to trust the promise; a process that must not act until money has truly moved must reach the settlement stage.

And because the stages are separated by hours or days, a payment process is inherently a *long-running* process in exactly the sense of [Chapter 3](#): it starts at authorization, then *waits* — the waiting state, the timer and message events of BPMN — for clearing and settlement to occur, and continues when they do. A payment process modelled as a single synchronous sequence has mismodelled the payment.

A successful authorization is a *promise*, not a completed payment — the money has not moved. It moves at *settlement*, which happens later, often the next day. A process that treats authorization as “payment done” will believe money has moved when it has not, and will not handle a payment that authorizes but fails to settle. A process that must know a payment genuinely completed must reach settlement — and because the stages are separated by hours or days, a payment process is inherently long-running, waiting between the stages.

117.4 Payment integration and idempotency

Payment integration is the domain where idempotency is most consequential, because a duplicated payment is duplicated money. Every write operation against a payment system — initiating a payment, capturing a charge, issuing a refund — must carry an idempotency key that the downstream system honours, so a retried worker does not create a second financial transaction for the same instruction. The key is typically a business reference — the payment instruction identifier, the refund reference — that is unique to the logical operation. A payment worker without an idempotency key is not a bug; it is a financial incident waiting for its first retry.

Worked example: the goods released on a promise

Problem. A merchant’s Camunda order-fulfilment process works like this: the customer pays by card, the process receives the *authorization* result, and on a successful authorization the process immediately marks the payment complete and releases the goods for dispatch. For most orders this is fine. But over a period, the merchant finds a number of cases where goods were dispatched and the corresponding payment never settled — the money never

arrived. Explain how the process design allowed this.

Setup. We consider what the process treated as “payment complete” and what that stage actually guarantees.

Working. The process releases goods on a successful *authorization*. But authorization is only the first stage: it is a check and a hold — a *promise* that the payment can proceed — and the money does not move at authorization. The money moves at *settlement*, which happens later. Between authorization and settlement a payment can still fail to complete — for a range of reasons the cardholder or issuer side can introduce. The process treated:

successful authorization = payment complete,

when in truth a successful authorization only means the payment was *promised*. For the orders where settlement later failed, the process had already released the goods on the strength of a promise that was not kept.

Discussion. The fulfilment process mistook a payment’s first stage for its completion, and the worked example shows that this is not a rare edge case but a structural error that will produce losses whenever a payment authorizes and fails to settle. The team’s mental model was that a payment is one event — “did the payment succeed? yes? release the goods” — but a card payment is three stages separated in time, and a successful authorization is a promise, not a transfer of money. The process acted on the promise as though it were the money. The fix depends on the merchant’s risk appetite, and the chapter’s point is that it must be a *deliberate decision*. A merchant may legitimately choose to release goods on authorization — accepting the settlement risk as a cost of speed — but that must be a known, owned decision, with handling for the cases where settlement fails (recovery, write-off). Or the process may wait for settlement before releasing goods, accepting the delay for the certainty. What the process must not do is release goods on authorization *by accident*, because the designers believed authorization *was* the payment. The lesson is the chapter’s warning: authorization, clearing, and settlement are distinct stages, money moves only at settlement, and a payment process must model the stages explicitly — waiting for settlement where settlement is what matters — rather than collapsing a multi-stage, multi-day payment into a single “it succeeded” event.

Practical next steps

For any process that acts on a payment, identify exactly which stage it treats as “payment complete.” If it acts on authorization, confirm that releasing value on a promise is a deliberate, owned business decision with handling for settlement failures — not an accident of believing authorization is the payment. Where genuine completion matters, the process must reach settlement.

117.5 Payment regulation and compliance

Payment processing is among the most heavily regulated domains in financial services. Anti-money-laundering screening, sanctions checking, transaction monitoring, and regulatory reporting are not optional add-ons; they are mandatory steps in every payment flow. A process that handles payments must include these steps as modelled, governed activities — not as post-hoc checks — because the regulator expects to see them in the process, to audit their execution, and to hold the institution accountable for their completeness. A payment process without embedded compliance steps is not merely incomplete; it is, in a regulated institution, non-compliant.

The chapter's principles interact with the platform's operational model in a way worth making explicit. The *deployment* of the artefacts this chapter describes must go through a governed pipeline — versioned, reviewed, tested, and traceable. An artefact deployed by a manual action, outside the pipeline, is an artefact whose presence in production cannot be explained, and in a regulated institution an unexplained artefact is a finding.

The *monitoring* of the artefacts must cover both their functional correctness and their operational health. A process step that executes but produces wrong results is harder to detect than one that fails outright, because it does not raise an error — it raises a consequence, and the consequence may not be discovered until a customer complains or a regulator asks a question. The observability model should therefore include not only error rates and latencies but also *outcome monitoring*: checks that the results the process produces are within expected ranges, flagging anomalies for review.

Chapter in brief

A payment is not one event. A card payment moves through three distinct stages separated in time: *authorization* — a check and a hold at the point of sale, a promise that the payment can proceed, with no money moved; *clearing* — the later, often batched, exchange that calculates the obligations between banks; and *settlement* — later still, where the money actually moves. A process must not treat a successful authorization as a completed payment; if it needs genuine completion it must reach settlement. And because the stages are separated by hours or days, a payment process is inherently long-running — it starts at authorization and waits for clearing and settlement. A process that collapses the stages into one event mismodels the payment.

Integrating with Schemes, Processors, and PayPal

118.1 The integration targets

The previous chapters established the payments landscape and the staged nature of a payment. This chapter treats the practical integration — with the card schemes, with payment processors, and with a PSP such as PayPal — and the message standards an architect meets.

118.2 The message standards

Payments run on *message standards*, and an architect integrating with payments will meet two in particular.

ISO 8583 is the long-established standard for card transaction messages — the format in which card authorization and related messages have been exchanged for decades. It is old, compact, and deeply entrenched in card payments, and an institution's card-payment integration is very likely to involve it, directly or behind a layer.

ISO 20022 is the modern, XML-based message standard, richer and more structured, which is being adopted across the payments world — prominently in cross-border and high-value payments, and increasingly elsewhere. The migration toward ISO 20022 is a long, ongoing industry programme.

For a hyperautomation architect, the practical point is that payment messages have *specific, standardised formats*, and integrating with payments means either handling those formats or, more commonly and more wisely, integrating *through* a layer — a processor's API, a PSP's API, the bank's own payment gateway — that handles the raw message standards, so the Camunda process works with cleaner interfaces. As with the core banking systems of [Chapter 105](#), the process should not bind itself directly to the raw payment message format.

118.3 Integrating with PayPal and PSPs

PayPal is a prominent example of a *payment service provider*, and PSPs are, for many institutions and merchants, the most congenial payments integration target. A PSP like PayPal exposes a modern API through which a process can initiate payments, accept payments, and query their status across a range of payment methods — the PSP itself handling the underlying complexity of schemes, processors, and message standards.

Integrating a Camunda process with a PSP is, mechanically, API integration through the connectors of [Part VI](#). The discipline is the discipline of the whole part: even though the PSP's API is clean, the payment behind it is still staged — a payment initiated through PayPal still authorizes, clears, and settles — and the process must still model the payment as the long-running, staged thing [Chapter 110](#) described. The PSP simplifies the *interface*; it does not abolish the *nature* of a payment.

Tip

A payment service provider such as PayPal gives a process a clean, modern API and hides the underlying complexity of schemes, processors, and message standards like ISO 8583 and ISO 20022 — and integrating through such a layer, rather than binding to raw payment message formats, is the wise default. But a clean PSP API does not change what a payment *is*: the payment behind it still authorizes, clears, and settles over time. The PSP simplifies the interface; the process must still model the payment as staged and long-running.

118.4 The PayPal integration in code

PayPal is API-first and well-documented; a job worker reaches it through its REST API, creating a payment order and capturing it.

Listing 118.1. A job worker creating a PayPal payment order through the PayPal REST API.

```
1 @JobWorker(type = "create-paypal-order")
2 public Map<String, Object> create(ActivatedJob job) {
3     PayPalOrder order = paypalClient.createOrder(
4         job.getVariable("amount"),
5         job.getVariable("currency"),
6         job.getVariable("returnUrl"));
7
8     return Map.of(
9         "paypalOrderId", order.id(),
10        "approvalUrl", order.approvalUrl());
11 }
```

118.5 Payment integration and idempotency

Payment integration is the domain where idempotency is most consequential, because a duplicated payment is duplicated money. Every write operation against a payment system — initiating a payment, capturing a charge, issuing a refund — must carry an idempotency key that the downstream system honours, so a retried worker does not create a second financial transaction for the same instruction. The key is typically a business reference — the payment instruction identifier, the refund reference — that is unique to the logical operation. A payment worker without an idempotency key is not a bug; it is a financial incident waiting for its first retry.

Worked example: the clean API and the assumption it invited

Problem. A team integrates a Camunda process with a PSP — PayPal — to take payments. The PSP's API is clean and modern: one call to initiate a payment, an immediate response. Encouraged by this simplicity, the team models the payment as a single synchronous service task: call the PSP, get the response, treat the payment as done, continue. An architect warns that the clean API has led the team into the very mistake [Chapter 73](#) warned against. Explain.

Setup. We consider what the PSP's clean API does and does not change about the underlying payment.

Working. The PSP's API is clean: it abstracts away the schemes, the processors, and the

raw ISO message standards, so the integration is one tidy API call. But the PSP abstracts the *interface*, not the *payment*. Behind the clean call, the payment still passes through authorization, clearing, and settlement, separated in time. The team's single synchronous service task — call, respond, done — models a payment as one instantaneous event:

clean API $\neq$ instantaneous payment.

The immediate API response reflects, at most, the initiation or authorization of the payment — not its settlement. The team has, exactly as [Chapter 110](#) warned, collapsed a staged, long-running payment into a single “it succeeded” event, misled by the cleanliness of the interface into thinking the payment itself was simple.

Discussion. The worked example shows a subtle trap: a good abstraction can mislead. The PSP's clean API is genuinely valuable — it spares the team from handling ISO 8583, scheme routing, and processor mechanics, and the chapter rightly recommends integrating through such a layer. But the team drew a wrong inference from the clean interface: because the *integration* was simple, they assumed the *payment* was simple, and modelled it as a single synchronous event. The PSP abstracted the complexity of the payments landscape; it did not, and could not, abstract away the fundamental nature of a payment as a staged process unfolding over time. The immediate response to the payment-initiation call is not settlement confirmation, and a process that treats it as such will believe payments are complete when they are still in flight. The correct design uses the PSP's clean API — that part of the team's approach was right — but still models the payment as the staged, long-running process of [Chapter 110](#): initiate the payment through the PSP, then wait, using the PSP's status callbacks or queries, for the payment to genuinely reach settlement before treating it as complete. The lesson is the chapter's tip stated as a caution: a clean PSP API simplifies the interface and should be welcomed, but it must not be allowed to simplify the architect's *model* of the payment. The interface is simple; the payment is still a payment.

Practical next steps

Where your processes integrate with a PSP, check that the clean API has not led to the payment being modelled as a single synchronous event. The PSP abstracts the interface, not the staged nature of the payment. Model the payment as long-running — initiate, then wait for genuine settlement via the PSP's status mechanisms — regardless of how simple the API made initiation look.

118.6 Payment regulation and compliance

Payment processing is among the most heavily regulated domains in financial services. Anti-money-laundering screening, sanctions checking, transaction monitoring, and regulatory reporting are not optional add-ons; they are mandatory steps in every payment flow. A process that handles payments must include these steps as modelled, governed activities — not as post-hoc checks — because the regulator expects to see them in the process, to audit their execution, and to hold the institution accountable for their completeness. A payment process without embedded compliance steps is not merely incomplete; it is, in a regulated institution, non-compliant.

The chapter's principles interact with the platform's operational model in a way worth making explicit. The *deployment* of the artefacts this chapter describes must go through a governed pipeline — versioned, reviewed, tested, and traceable. An artefact deployed by a

manual action, outside the pipeline, is an artefact whose presence in production cannot be explained, and in a regulated institution an unexplained artefact is a finding.

The *monitoring* of the artefacts must cover both their functional correctness and their operational health. A process step that executes but produces wrong results is harder to detect than one that fails outright, because it does not raise an error — it raises a consequence, and the consequence may not be discovered until a customer complains or a regulator asks a question. The observability model should therefore include not only error rates and latencies but also *outcome monitoring*: checks that the results the process produces are within expected ranges, flagging anomalies for review.

Chapter in brief

Payments run on message standards: *ISO 8583*, the long-established, compact card-transaction standard, and *ISO 20022*, the modern XML-based standard being adopted across payments. A process should integrate not with raw message formats but through a layer — a processor's API, a PSP's API, the bank's gateway — that handles them. *PayPal* exemplifies the payment service provider: a clean modern API through which a process initiates and queries payments, with the PSP handling the underlying scheme, processor, and message-standard complexity. Integrating with a PSP is comfortable API integration — but the clean API abstracts the interface, not the payment: the payment behind it still authorizes, clears, and settles over time, and must still be modelled as staged and long-running.

Payment Integration Patterns

119.1 The patterns that protect money

This chapter gathers the integration patterns specific to payments. They share the foundations of the core banking patterns of [Chapter 70](#) — those are not repeated — but payments add patterns of their own, and because payments move money, getting these patterns right is not optional.

119.2 Idempotency: never pay twice

The single most important payment pattern is *idempotency*, and it deserves the emphasis.

A process step, as [Chapter 34](#)'s job semantics established, can be retried — after a timeout, after a worker failure, after an uncertain response. For most operations a retry is harmless. For a *payment*, a retry that causes a *second payment* is a direct, real loss of money: the customer is charged twice, the merchant is paid twice, the institution must detect and reverse it.

The pattern that prevents this is *idempotency*: every payment instruction carries a unique *idempotency key*, and the payment system — the processor, the PSP — is designed so that two instructions with the same key result in *one* payment, not two. If a payment step retries and re-sends the instruction with the same key, the payment system recognises it as the same payment and does not execute it again. A payment integration that does not use idempotency keys is a payment integration that *will*, eventually, pay twice — because retries are not an anomaly, they are a normal part of how a resilient process works.

Figure [119.1](#) draws the idempotency pattern.

119.3 Reconciliation: trust, then verify

A second essential pattern is *reconciliation*. Because a payment is staged and involves several parties, a process's belief about a payment — “this payment settled” — must be *verified* against the payment systems' own records, not merely assumed.

Reconciliation is the discipline of regularly comparing the institution's record of payments against the records from the schemes, processors, and PSPs — and investigating every discrepancy. It is how an institution catches the payment that authorized but did not settle, the settlement that does not match what was expected, the double payment that slipped through. Reconciliation is not a sign of a badly-built process; it is a necessary control in a domain of many parties and staged transactions, and a payment architecture without a reconciliation process is incomplete.

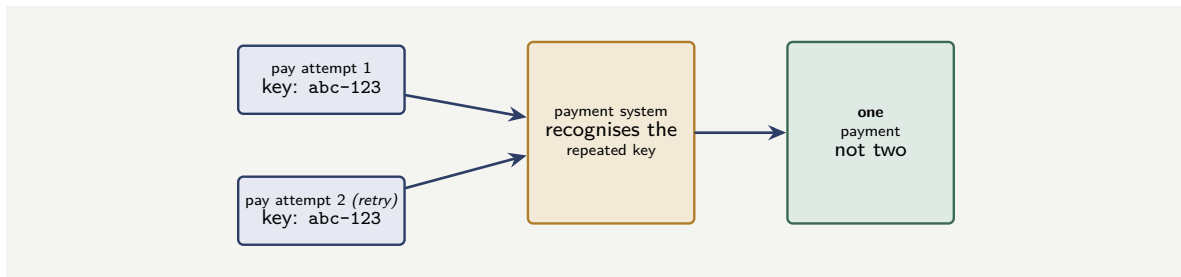


Figure 119.1. Idempotency. Every payment instruction carries a unique idempotency key; a retried instruction re-sends the same key, and the payment system recognises it and executes the payment only once. Without idempotency keys, a retry — a normal part of a resilient process — becomes a double payment.

119.4 The payment process as a long-running, compensable thing

The final pattern draws the part together: a payment process is *long-running* — it spans authorization to settlement — and it must be *compensable*. If a business process makes a payment and then a later step fails, the payment may need to be reversed — a refund, a payment the other way — and that reversal is exactly the *compensation* of the manuscript’s patterns part. A payment is a step that, once done, may need a defined undoing, and a payment process must be built with that compensation modelled, not improvised when a reversal is first needed.

119.5 Payment integration and idempotency

Payment integration is the domain where idempotency is most consequential, because a duplicated payment is duplicated money. Every write operation against a payment system — initiating a payment, capturing a charge, issuing a refund — must carry an idempotency key that the downstream system honours, so a retried worker does not create a second financial transaction for the same instruction. The key is typically a business reference — the payment instruction identifier, the refund reference — that is unique to the logical operation. A payment worker without an idempotency key is not a bug; it is a financial incident waiting for its first retry.

Worked example: the timeout that paid twice

Problem. A bank’s Camunda process makes outbound payments. A payment step calls the payment system; on one occasion the payment system is slow, and the call *times out* before a response is received. The process, following its retry configuration, retries the payment step — calls the payment system again. It turns out the first call had *actually succeeded*; the payment system had made the payment, and only the response was lost. The payment integration does not use idempotency keys. Describe what has happened and the pattern that would have prevented it.

Setup. We trace the two calls and what the payment system did with each.

Working. The first call reached the payment system and the payment was made — but the response was lost to the timeout, so the process does not know this. The process’s retry sends the payment instruction *again*. Without idempotency keys, the payment system has no way to know the second instruction is the same payment as the first; it sees a fresh, valid

instruction and makes the payment *again*:

one intended payment + a timed-out retry + no idempotency key = two real payments.

The customer or counterparty has been paid twice, and the bank must now detect and reverse the duplicate.

Discussion. This is the precise scenario the idempotency pattern exists to prevent, and the worked example shows that it arises not from a bug but from *normal, correct* process behaviour. The process did nothing wrong in retrying: a timed-out call is genuinely ambiguous — the process cannot tell whether the payment succeeded, failed, or never arrived — and retrying is the resilient, correct response, exactly as [Chapter 34](#)’s job semantics describe. The fault is entirely in the *payment integration*’s lack of an idempotency key. Because the instruction carried no key, the payment system could not recognise the retry as a repeat of an already-executed payment, and so it did the only thing it could — treated it as a new payment and executed it. With an idempotency key, the first and the retried instruction would have carried the *same* key, the payment system would have recognised the second as a duplicate of a payment already made, and it would have returned the original result without paying again. The lesson is the chapter’s central pattern stated as an imperative: because resilient processes retry, and retries of a payment are inevitable, every payment instruction must carry an idempotency key, and the payment system must honour it — this is not an enhancement but the baseline of a correct payment integration. A payment integration without idempotency is not a slightly-imperfect integration; it is one that is guaranteed, given enough time and enough timeouts, to pay someone twice.

Practical next steps

Check every payment integration in your institution’s processes for idempotency keys. Any payment instruction sent without one will, eventually — on the first timed-out call that actually succeeded — become a double payment. Idempotency is the baseline of payment integration, not an enhancement. Confirm, too, that reconciliation and payment compensation (reversal) are designed in, not improvised.

119.6 Payment regulation and compliance

Payment processing is among the most heavily regulated domains in financial services. Anti-money-laundering screening, sanctions checking, transaction monitoring, and regulatory reporting are not optional add-ons; they are mandatory steps in every payment flow. A process that handles payments must include these steps as modelled, governed activities — not as post-hoc checks — because the regulator expects to see them in the process, to audit their execution, and to hold the institution accountable for their completeness. A payment process without embedded compliance steps is not merely incomplete; it is, in a regulated institution, non-compliant.

The chapter’s principles interact with the platform’s operational model in a way worth making explicit. The *deployment* of the artefacts this chapter describes must go through a governed pipeline — versioned, reviewed, tested, and traceable. An artefact deployed by a manual action, outside the pipeline, is an artefact whose presence in production cannot be explained, and in a regulated institution an unexplained artefact is a finding.

The *monitoring* of the artefacts must cover both their functional correctness and their operational health. A process step that executes but produces wrong results is harder to detect than one that fails outright, because it does not raise an error — it raises a consequence, and the

consequence may not be discovered until a customer complains or a regulator asks a question. The observability model should therefore include not only error rates and latencies but also *outcome monitoring*: checks that the results the process produces are within expected ranges, flagging anomalies for review.

Chapter in brief

Payments add integration patterns of their own, and because payments move money these are not optional. *Idempotency* is the foremost: since resilient processes retry, every payment instruction must carry a unique idempotency key so that the payment system executes a retried instruction only once — without it, a timed-out-but-successful call followed by a retry becomes a double payment. *Reconciliation* — regularly comparing the institution's payment records against the schemes', processors', and PSPs' own records and investigating discrepancies — is a necessary control in a multi-party, staged domain. And a payment process is long-running and must be *compensable*: a payment may later need a defined reversal, which is the compensation pattern, modelled in advance rather than improvised.

PART XXI

Applications

Customer Onboarding and Identity

120.1 The process at a glance

Customer onboarding is the natural first application, because it is where a banking or insurance relationship begins, where regulatory obligation is at its sharpest, and where the three layers of this manuscript — orchestration, decision, and agent — each have an obvious and well-bounded role. The process takes a prospective customer from an expression of interest to an active, verified, risk-assessed relationship.

Modelled as a journey, onboarding decomposes — in the layered style of [Chapter 21](#) — into a short top-level model that calls a sequence of capability subprocesses: *capture application*, *verify identity*, *screen and risk-rate*, *decision*, and *activate*. Each capability is independently owned and reusable: the *verify identity* capability that serves account opening also serves lending and insurance applications. The top-level journey is deliberately thin — a sequence of call activities and one or two gateways — because the substance lives in the capabilities.

Figure [120.1](#) shows the journey at the top level. The discipline the diagram embodies is that the journey model itself carries almost no logic: it is a sequence of call activities, and the real work — and the real governance — lives one level down in the capabilities.

120.2 Where each layer does its work

Onboarding illustrates the division of labour cleanly. The orchestrator runs the journey, holds the state of a part-completed application across the days a customer may take to supply documents, and places human work on analyst worklists. The *decision* layer — DMN — holds the eligibility rules, the document-checklist logic, and the risk-rating bands: rules that compliance owns, that change on a regulatory cadence, and that must be auditable. The *agent* layer does the unstructured work: reading an uploaded proof of address and judging whether it genuinely evidences the claimed address, reconciling a name that appears in three slightly different forms across three documents, assessing whether a submitted document pack is internally coherent.

The screening step — checking a customer against sanctions and politically-exposed-person lists — deserves a specific note, because it is the step where the agentic temptation is strongest and most dangerous. It is tempting to let an agent “decide” whether a fuzzy name match is a true hit. The disciplined design does not. The agent may *assist* — summarising why two records resemble each other — but the disposition of a screening hit is a deterministic, rule-governed, human-accountable decision, because a wrong clear here is a regulatory breach, not an inefficiency.

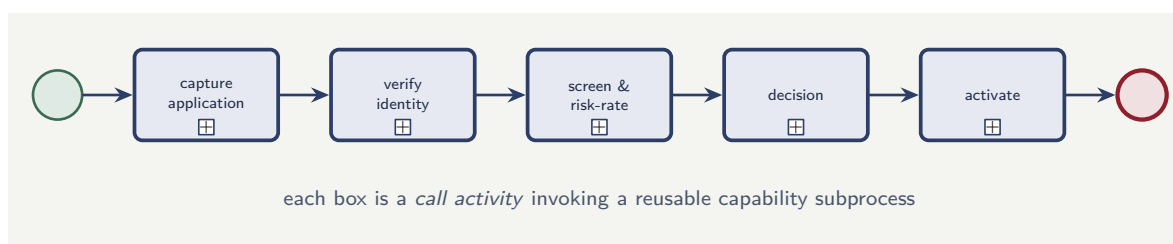


Figure 120.1. The onboarding journey as a thin top-level model. Each call activity (marked with the subprocess symbol) invokes an independently owned, reusable capability — the same *verify identity* capability serves lending and insurance applications too.

Identity verification and sanctions screening are the steps where a confident, plausible, wrong agent does regulatory damage. Keep the agent in an assistive role — extracting, summarising, reconciling — and keep the disposition of every identity and screening decision on a deterministic, human-accountable path. The agent makes the human faster; it does not make the decision.

120.3 Document processing as a sub-pattern

A large part of onboarding — and, as later chapters show, of lending, underwriting, and claims — is *document processing*: turning the identity documents, proofs of address, payslips, and statements a customer uploads into structured, usable data. It recurs often enough across the application chapters to be worth treating once, here, as a named sub-pattern.

The naive view of document processing is that it is a single step: a service task that “reads the documents.” The disciplined view is that it is a small process in its own right, with four distinct stages, because each stage has a different failure mode and a different owner.

The first stage is *classification*: determining what each uploaded file actually is. A customer who was asked for a proof of address uploads a file; the platform must establish whether it is a utility bill, a bank statement, a tenancy agreement, or something irrelevant. This is a judgement over unstructured input — agentic or model work — and its failure mode is miscategorisation.

The second stage is *extraction*: pulling the specific fields the process needs — a name, an address, a date, an amount — from the classified document. Its failure mode is the misread field: a transposed digit, a date in the wrong format, a name captured with an error.

The third stage is *validation*: checking the extracted data for internal coherence and against other sources. Does the name on the proof of address match the name on the application? Is the document recent enough? Is the amount plausible? This is where errors from the first two stages are meant to be caught, and it is the stage teams most often skimp on.

The fourth stage is *the confidence decision*: every extracted field carries a confidence, and the process must decide — as a governed DMN decision — whether the confidence is high enough to use the field automatically or low enough to route the document to a human. This is the containment that makes the first three stages safe: an extraction the platform is not sure of becomes a human task, not a silent guess.

Modelling document processing as these four explicit stages — rather than as one opaque “read the documents” task — means each stage can be measured, each failure mode can be attributed, and the confidence decision gives the process a principled, auditable way to fall back to a human exactly when it should.

Tip

Never model document processing as a single step that “reads the documents.” Model it as classify, extract, validate, and a confidence decision. The confidence decision is the important one: it is the governed point at which a low-confidence extraction becomes a human task instead of a silent error, and a document-processing pipeline without it is one that fails quietly.

120.4 Modelling the long wait and the abandoned application

Onboarding is a long-running process in the sense of [Chapter 3](#): between application and completion the process spends most of its elapsed time *waiting* — for the customer to upload a document, to complete a verification step, to respond to a query. The model must represent those waits explicitly: an intermediate message event for each awaited customer action, with a boundary timer that escalates — a reminder, then a second reminder, then an abandonment path — when the wait runs too long.

The abandonment path matters more than teams expect. A large fraction of real onboarding applications are never completed, and an unmodelled process leaves those half-finished applications as litter: partial customer records, held documents, unresolved screening hits. The disciplined model has an explicit *abandonment* end state with defined clean-up — what is deleted, what is retained and why, what is reported — so that an incomplete application ends as cleanly as a completed one.

120.5 The process as a regulatory exhibit

Each of the application processes this part describes is, in a regulated institution, a potential *regulatory exhibit*: a process a regulator may ask to see, and whose execution record the institution must be able to produce for any past case. This is not an abstract concern; it is a concrete requirement that shapes the process design. Every decision point must be recorded, every human step must be traceable to an authenticated identity, and every automated step must log its inputs and outputs. A process that runs correctly but cannot produce an audit trail of a specific past case has failed its regulatory purpose.

Worked example: the cost of a verification step's error rates

Problem. An onboarding process verifies identity using an automated check that clears or refers each application. The bank processes 200,000 applications a year. The automated check refers 15% of applications to a manual reviewer; of the 85% it clears automatically, 0.5% are later found to be identity failures that should have been referred. A manual review costs \$22; an identity failure that reaches an active account costs the bank an average of \$1,400 in remediation and loss. A vendor offers an improved check that refers only 11% but reduces the missed-failure rate among cleared applications to 0.2%, for an additional \$1.50 per application checked. Is the improved check worth adopting?

Setup. For each check we total two costs: the manual-review cost (volume times referral rate times \$22) and the expected identity-failure cost (volume times clear rate times missed-failure rate times \$1,400). The improved check adds \$1.50 per application. We compare totals.

Working. *Current check.* Referrals:

$$200,000 \times 0.15 = 30,000 \text{ at } \$22 = \$660,000.$$

Cleared applications: $200,000 \times 0.85 = 170,000$. Missed failures:

$$170,000 \times 0.005 = 850 \text{ at } \$1,400 = \$1,190,000.$$

Current total: $\$660,000 + \$1,190,000 = \$1,850,000$.

Improved check. Referrals:

$$200,000 \times 0.11 = 22,000 \text{ at } \$22 = \$484,000.$$

Cleared applications: $200,000 \times 0.89 = 178,000$. Missed failures:

$$178,000 \times 0.002 = 356 \text{ at } \$1,400 = \$498,400.$$

Added per-application fee: $200,000 \times \$1.50 = \$300,000$. Improved total: $\$484,000 + \$498,400 + \$300,000 = \$1,282,400$.

Discussion. The improved check is worth adopting: \$1.28M against \$1.85M, a saving of about \$568,000 a year, comfortably more than the \$300,000 it costs to run. But the worked example is more instructive than its bottom line. Notice where the saving comes from. The reduction in referrals saves \$176,000 — real, but modest. The overwhelming majority of the benefit, nearly \$692,000, comes from cutting the missed-failure rate from 0.5% to 0.2%. The expensive error in an onboarding process is not the application sent for unnecessary review; it is the identity failure that gets through. A team optimising this process by focusing on the visible cost — the referral rate, because manual reviews are a line item someone complains about — would be optimising the wrong number. The disciplined process owner measures and attacks the *invisible* cost, the missed failure, because that is where both the money and the regulatory exposure actually sit.

Practical next steps

For your onboarding process, find the two numbers in the worked example: the referral rate (visible, complained about) and the missed-failure rate among auto-cleared cases (invisible, rarely measured). If you cannot state the second number, that is the measurement gap to close first — you are currently managing the cheap error and ignoring the expensive one.

120.6 Data quality as a process responsibility

A process that reads from and writes to a CRM is also a process that *affects data quality*. Every field the process writes back becomes part of the customer record; every field it reads depends on the quality of what was written before. A process that writes poorly validated data back to the CRM degrades the institution's customer view for every subsequent process that reads it. The discipline is that every write-back is validated, every field is typed, and the process treats the CRM's data quality as a responsibility, not merely an input.

The chapter's principles interact with the platform's operational model in a way worth making explicit. The *deployment* of the artefacts this chapter describes must go through a governed pipeline — versioned, reviewed, tested, and traceable. An artefact deployed by a manual action, outside the pipeline, is an artefact whose presence in production cannot be explained, and in a regulated institution an unexplained artefact is a finding.

The *monitoring* of the artefacts must cover both their functional correctness and their operational health. A process step that executes but produces wrong results is harder to detect than one that fails outright, because it does not raise an error — it raises a consequence, and the consequence may not be discovered until a customer complains or a regulator asks a question. The observability model should therefore include not only error rates and latencies but also *outcome monitoring*: checks that the results the process produces are within expected ranges, flagging anomalies for review.

Chapter in brief

Customer onboarding is the natural first application: a clear journey — capture, verify, screen and risk-rate, decide, activate — decomposed into reusable capability subprocesses. The orchestrator holds the long-running state; DMN holds eligibility, checklist, and risk-rating rules; agents do the unstructured extraction and reconciliation. Identity and sanctions decisions stay deterministic and human-accountable, with the agent only assisting. The model must represent the long customer waits and an explicit, clean abandonment path. The expensive error in onboarding is the missed identity failure that reaches an active account — the invisible cost — not the visible cost of unnecessary manual reviews.

Lending and Credit Origination

121.1 The process at a glance

Credit origination — turning a loan application into a funded, booked loan — is among the highest-value processes in a bank and one of the best suited to hyperautomation, because it combines a clear deterministic skeleton, several genuinely predictive decisions, and document-heavy steps that need unstructured reasoning. It is also a process where the cost of getting the decision wrong is asymmetric and quantifiable, which makes it an excellent setting for disciplined modelling.

The journey decomposes into capabilities: *capture and validate application, verify identity and income, assess affordability, assess credit risk, decision and price, produce and manage offer, and disburse and book*. As in [Chapter 120](#), the top-level journey is a thin sequence of call activities; the substance is in the capabilities, several of which — identity verification especially — are shared with onboarding.

121.2 The decisions and where they live

Lending makes the layered decision pattern of [Chapter 22](#) concrete. There are three distinct decisions in origination, and they belong in three different places.

Affordability is partly rule, partly fact. Whether a customer can afford a repayment depends on verified income and committed outgoings against policy ratios. The ratios and the policy are DMN rules; the income figure may come from an agent reading payslips and bank statements. The affordability *decision* is a DMN evaluation; the affordability *inputs* may be agent-extracted.

Credit risk — the probability the loan will not be repaid — is genuinely predictive. It is a model, trained on historical lending outcomes, returning a calibrated probability of default. It is not a rule, and forcing it into a decision table would be pretending a prediction is a policy.

The lending decision and the price — approve or decline, and at what rate and limit — is policy. It consumes the affordability result and the credit-risk score, applies the bank's risk appetite, regulatory limits, and pricing policy, and produces the actual decision. This belongs in DMN: it is the deterministic, owned, auditable step that turns a prediction into an action, exactly the pattern [Chapter 22](#) argued for.

Tip

Keep the credit-risk *model* and the lending *policy* strictly separate, even though both feel like “the credit decision.” The model predicts default probability and is owned by data science and model risk. The policy decides what to do with that probability and is owned by credit risk and the business. Tangling them means a pricing-policy tweak requires a model redeployment, and a model update silently changes lending policy. Separated, each is governed by the people accountable for it.

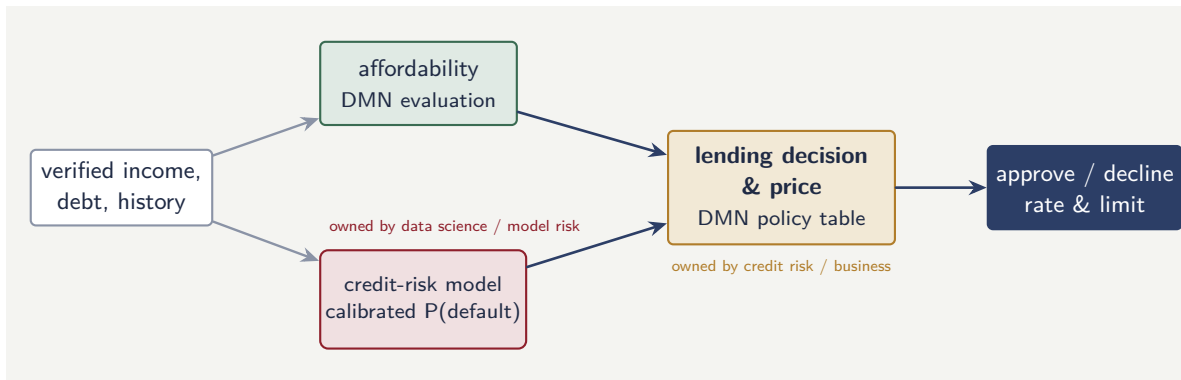


Figure 121.1. The layered lending decision. Affordability is a DMN evaluation; credit risk is a predictive model; and the lending decision and price is the deterministic, owned DMN policy that turns the prediction into an action. Prediction and policy have different owners.

Figure 121.1 draws the layered pattern for lending. The model predicts; the policy table decides; and the diagram’s owner labels make visible the point of the separation — each kind of logic is governed by the people accountable for it.

121.3 The offer as a long-running, interruptible subprocess

A feature of lending that the model must handle well is the *offer*: once the bank approves, it issues an offer the customer may accept, reject, ignore, or accept after negotiation, and the offer has a validity period. This is a textbook long-running, interruptible subprocess. The model represents it with an event-based gateway waiting for whichever comes first: the customer’s acceptance, the customer’s rejection, or the expiry timer. Each outcome routes differently — acceptance to disbursement, rejection to a closed end state, expiry to a lapsed end state with possible re-offer.

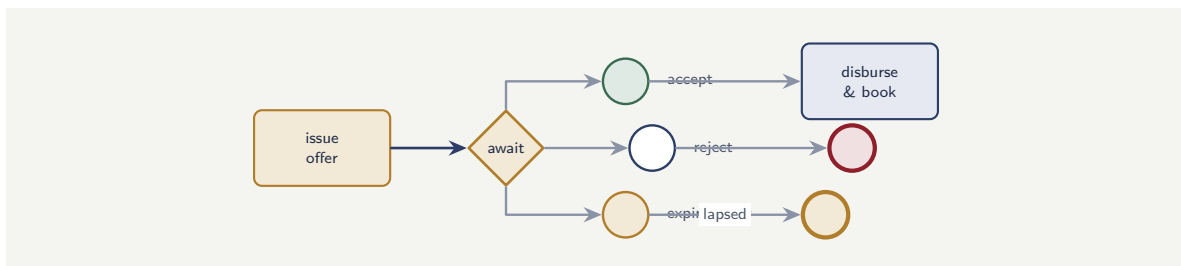


Figure 121.2. The lending offer as an event-based gateway. The process waits for whichever occurs first — acceptance, rejection, or the expiry timer — and each outcome routes to a distinct, clean end state.

Modelling the offer explicitly does two things. It makes the offer’s economics measurable — acceptance rates, time-to-accept, lapse rates all become process metrics — and it ensures that an ignored offer reaches a clean, defined end state rather than leaving an approved-but-unfunded application in limbo. Chapter 125 returns to these metrics as part of process observability.

121.4 The process as a regulatory exhibit

Each of the application processes this part describes is, in a regulated institution, a potential *regulatory exhibit*: a process a regulator may ask to see, and whose execution record the institution must be able to produce for any past case. This is not an abstract concern; it is a concrete requirement that shapes the process design. Every decision point must be recorded, every human step must be traceable to an authenticated identity, and every automated step must log its inputs and outputs. A process that runs correctly but cannot produce an audit trail of a specific past case has failed its regulatory purpose.

Worked example: the asymmetric cost of a lending cut-off

Problem. A bank's origination process approves a personal loan when the predicted probability of default is below a cut-off. At the current cut-off, of every 1,000 applications, 700 are approved. Among approved loans, the realised default rate is 4%. A defaulted loan costs the bank an average of \$8,000 (net of recovery); a performing loan earns the bank an average of \$1,100 in lifetime margin. The credit team proposes loosening the cut-off so that 770 of every 1,000 are approved; they estimate the 70 newly-approved marginal applications will default at 11%. Does loosening the cut-off increase or decrease profit per 1,000 applications?

Setup. Profit is earnings on performing loans minus losses on defaulted loans. We evaluate the current book and the proposed book per 1,000 applications. For the proposed book we treat the original 700 as unchanged and add the 70 marginal loans at their own default rate.

Working. *Current book* — 700 approved, 4% default. Defaults: $700 \times 0.04 = 28$; performing: 672.

$$\text{Profit} = 672 \times \$1,100 - 28 \times \$8,000 = \$739,200 - \$224,000 = \$515,200.$$

Marginal 70 loans — 11% default. Defaults: $70 \times 0.11 = 7.7$; performing: 62.3.

$$\text{Marginal profit} = 62.3 \times \$1,100 - 7.7 \times \$8,000 = \$68,530 - \$61,600 = \$6,930.$$

Proposed book total:

$$\$515,200 + \$6,930 = \$522,130.$$

Loosening the cut-off increases profit per 1,000 applications by $\$522,130 - \$515,200 = \$6,930$ — the marginal contribution, which is positive.

Discussion. The marginal expansion is barely profitable — \$6,930 across 70 loans, under \$100 a loan — and that thin margin is the real lesson. At an 11% default rate, the marginal loans are very close to the point where each defaulted loan's \$8,000 loss cancels the margin earned on the performing ones. A small error in the 11% estimate flips the sign: if the marginal cohort actually defaults at 13%, the marginal profit becomes negative and loosening destroys value. This is why a lending cut-off is not a dial to be turned by intuition. The decision depends precisely on the predicted default rate of the *marginal* applicant — not the average applicant — and on the asymmetry between an \$8,000 loss and an \$1,100 gain, which means roughly one default must be avoided for every seven performing loans just to break even. The disciplined process ties the cut-off to the credit-risk model's calibration at the margin (Chapter 6), revisits it as the model and the economy change, and treats it as a governed policy parameter in the DMN decision, not a number someone adjusts quietly.

Practical next steps

For one lending product, obtain the realised default rate of your most recently approved decile — the marginal applicants, not the book average — and the average loss-given-default and the average margin on a performing loan. Compute whether your marginal decile is profitable. If you cannot get the marginal default rate, your cut-off is being set without the one number that determines whether it is right.

Chapter in brief

Credit origination is a high-value process well suited to hyperautomation, decomposed into capabilities from capture through to disburse-and-book. It makes the layered decision pattern concrete: affordability is a DMN evaluation on possibly agent-extracted inputs; credit risk is a genuinely predictive model; the lending decision and price is the deterministic, owned DMN policy that turns the prediction into an action. The model and the lending policy are kept strictly separate. The offer is modelled as a long-running, interruptible subprocess with an event-based gateway. A lending cut-off must be set on the *marginal* applicant's default rate and the asymmetric cost of default, as a governed policy parameter.

Payments, Disputes, and Exceptions

122.1 The process at a glance

Payments processing differs from the application-shaped processes of the two preceding chapters in a way that shapes its modelling. Onboarding and lending are relatively low-volume, long-running, and decision-heavy. The core payment — moving money from one account to another — is extremely high-volume, very short-running, and, on the happy path, almost decision-free. The interesting hyperautomation is not in the happy path of a payment; it is in the *exceptions* and the *disputes*, which are lower in volume but rich in process, decision, and unstructured reasoning.

This chapter therefore treats payments as two distinct processes. The *payment execution* process is high-volume and lean: validate, screen, post, confirm. The *exception and dispute* process is the hyperautomation-rich one: it handles failed payments, recalls, fraud holds, and customer disputes, and it is where orchestration, decision services, and agents earn their place.

Figure 122.1 shows the split that the rest of the chapter develops: a lean execution path for the overwhelming majority of payments, and a deliberate branch to a process-rich exception path for the minority that need genuine work.

122.2 Why the happy path is barely a process

It is worth stating plainly that not every process should be heavily hyperautomated, and payment execution is the example. A straightforward domestic payment, with a valid instruction, sufficient funds, and no screening hit, should pass through a short, fast, deterministic path with no agent anywhere near it. Putting reasoning into the hot path of a payment adds latency and cost and risk for no benefit, because there is nothing to reason about.

The discipline here is to model the happy path as deliberately minimal and to make the *decision to leave the happy path* the first significant decision in the process. A DMN decision examines the payment and routes it: the overwhelming majority to straight-through execution, the minority with a screening hit, a validation failure, an unusual pattern, or insufficient funds to the exception process. This is the same lean-skeleton-with-an-exit pattern that [Chapter 21](#)'s decomposition principle implies: keep the common path thin, and branch deliberately to the rich process only when there is genuine work to do.

Resist the urge to instrument the high-volume payment hot path with agents or heavy decision logic “so it is consistent with the rest of the platform.” Latency and cost scale with volume; an agent call added to a path that runs millions of times a day is millions of agent calls. The hot path stays lean and deterministic; the intelligence belongs in the

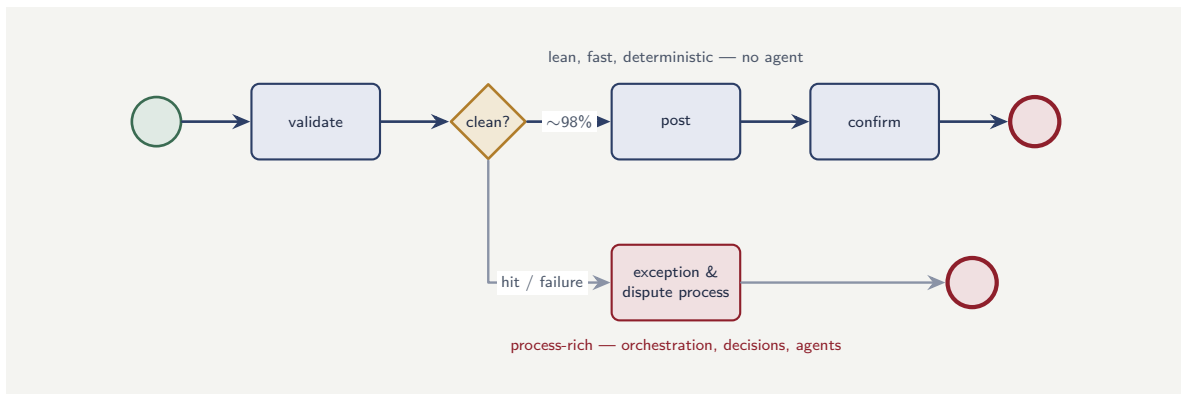


Figure 122.1. Payments as two processes. The high-volume happy path is deliberately minimal; the first real decision is whether to leave it. Intelligence belongs in the lower-volume exception and dispute process, not the hot path.

lower-volume exception process where there is something to be intelligent about.

122.3 The exception process: where the layers work

The exception and dispute process is genuinely process-rich, and it shows the three layers cooperating.

A *disputed transaction* — a customer asserting that a payment was unauthorised, or that goods were not received — is a case in the sense of [Chapter 2](#): it is registered, investigated, decided, and resolved, it is long-running because it waits on the customer and sometimes on another bank, and it is exception-prone. The orchestrator runs that case. DMN holds the rules: the scheme rules and time limits that govern what kind of dispute can be raised and by when, the thresholds that decide whether a provisional credit is given. An agent does the unstructured work: reading the customer’s free-text description of what happened, reading the merchant’s evidence, and assessing whether the dispute narrative is coherent and which scheme reason code it corresponds to — a genuinely unstructured judgement that a rule cannot make.

And the disposition — whether the customer’s dispute is upheld, whether the bank bears the loss or recovers it from the merchant or another bank — is a deterministic, rule-and-policy decision informed by the agent’s assessment, never the agent’s decision alone. The pattern is the recurring one of this manuscript: agent reasons, deterministic step disposes, human is accountable for the consequential outcomes.

122.4 Idempotency: the discipline of doing it exactly once

Payments raise a problem more sharply than any other process in this manuscript, and it is worth a section because the discipline it demands applies wherever a process has external effects. The problem is this: a hyperautomation platform is a distributed system, distributed systems retry failed operations, and a payment must not be made twice.

Consider a service task that instructs the core banking system to move money. The job worker sends the instruction. Then the network connection drops before the worker receives confirmation. The worker does not know whether the payment was made. [Chapter 21](#)’s job

mechanism will, correctly, retry the job — that is what makes the platform resilient. But if the original instruction did in fact succeed, the retry makes the payment a second time. Resilience and correctness appear to be in conflict.

The resolution is *idempotency*: designing the operation so that performing it twice has the same effect as performing it once. The payment instruction carries a unique identifier — derived from the process instance and the step, so it is the same on every retry. The receiving system records the identifiers of instructions it has already executed. When the retried instruction arrives, the receiving system recognises the identifier, knows it has already acted, and returns the original result without moving money again. The retry is now safe: it either completes a payment that had not happened, or harmlessly re-confirms one that had.

Idempotency is not a property the platform has automatically; it is a property each externally-effecting operation must be designed to have, and the design spans the boundary. The orchestrated process must generate stable identifiers; the integration layer must carry them; the receiving system, or the anti-corruption layer of [Chapter 25](#) standing in front of it, must honour them. Where a legacy system of record cannot itself de-duplicate, the anti-corruption layer must, keeping a record of executed instruction identifiers so that it can absorb a retry the legacy system would otherwise act on twice.

Any process step with an irreversible external effect — moving money, binding cover, dispatching a payment instruction — must be idempotent, because the platform *will* retry it. A step that is not idempotent is a step that will, eventually, be performed twice when a retry coincides with a successful original. Treat idempotency as a mandatory property of every externally-effecting service task, designed and tested, not as a quality that resilience provides for free. Resilience without idempotency is duplication.

122.5 Payment integration and idempotency

Payment integration is the domain where idempotency is most consequential, because a duplicated payment is duplicated money. Every write operation against a payment system — initiating a payment, capturing a charge, issuing a refund — must carry an idempotency key that the downstream system honours, so a retried worker does not create a second financial transaction for the same instruction. The key is typically a business reference — the payment instruction identifier, the refund reference — that is unique to the logical operation. A payment worker without an idempotency key is not a bug; it is a financial incident waiting for its first retry.

Worked example: provisional credit and the cost of a dispute policy

Problem. A bank handles 50,000 card disputes a year. Policy gives an immediate provisional credit to the customer while a dispute is investigated. Investigation upholds the customer in 80% of cases (the provisional credit becomes permanent and is recovered from the merchant or scheme) and rejects the dispute in 20% (the provisional credit must be reversed). Reversing a provisional credit succeeds in only 70% of rejected cases — in the other 30% the customer has spent the funds and the bank cannot recover, losing the disputed amount, which averages \$240. The bank considers a stricter policy: give provisional credit only to disputes

an agent-assisted triage assesses as likely to be upheld, which would withhold credit from the 20% that are ultimately rejected — but the triage is imperfect and would also wrongly withhold credit from 5% of the genuinely-upheld disputes, each generating a complaint that costs \$60 to handle. What does the stricter policy save or cost?

Setup. We cost the current policy's unrecoverable losses, then the stricter policy's residual losses plus its wrongful-withholding complaint cost, and compare. Rejected disputes are $50,000 \times 0.20 = 10,000$; upheld are 40,000.

Working. *Current policy.* Rejected disputes where the credit cannot be reversed:

$$10,000 \times 0.30 = 3,000 \text{ at } \$240 = \$720,000 \text{ lost.}$$

Stricter policy. Suppose the triage correctly withholds credit from all 10,000 rejected disputes — then none of those become unrecoverable losses, saving the \$720,000. But it wrongly withholds from 5% of the 40,000 upheld disputes:

$$40,000 \times 0.05 = 2,000 \text{ complaints at } \$60 = \$120,000.$$

Net effect of the stricter policy:

$$\$720,000 \text{ saved} - \$120,000 \text{ complaint cost} = \$600,000 \text{ net saving.}$$

Discussion. On the arithmetic the stricter policy saves \$600,000, and the temptation is to adopt it. The worked example is constructed to show why the arithmetic is not the whole decision. The calculation made a flattering assumption — that triage correctly withholds credit from *all* rejected disputes — which a real, imperfect triage will not achieve; relax it and the saving shrinks. More importantly, the \$120,000 complaint cost is only the *handling* cost of wrongly withholding credit from 2,000 customers with genuine disputes. It does not price the customer harm: a customer whose money was genuinely taken, who is then refused provisional credit because an agent-assisted triage guessed wrong, experiences real detriment, and in many jurisdictions provisional credit during a dispute is a regulatory expectation, not a discretionary kindness. This is a case where a process change that is positive on a cost spreadsheet may be impermissible on conduct and regulatory grounds. The disciplined process owner presents the \$600,000 alongside the conduct analysis and the regulatory position, and lets an accountable human — not the spreadsheet — decide.

Practical next steps

For your disputes process, separate the two cost lines the worked example distinguishes: the recoverable-loss cost of your current credit policy, and the customer-harm and regulatory exposure of withholding credit. Most institutions can quantify the first and have never written down the second. Writing the second down is the precondition for making this decision honestly.

122.6 Payment regulation and compliance

Payment processing is among the most heavily regulated domains in financial services. Anti-money-laundering screening, sanctions checking, transaction monitoring, and regulatory reporting are not optional add-ons; they are mandatory steps in every payment flow. A process that handles payments must include these steps as modelled, governed activities — not as post-hoc checks — because the regulator expects to see them in the process, to audit their execution, and to hold the institution accountable for their completeness. A payment process

without embedded compliance steps is not merely incomplete; it is, in a regulated institution, non-compliant.

The chapter's principles interact with the platform's operational model in a way worth making explicit. The *deployment* of the artefacts this chapter describes must go through a governed pipeline — versioned, reviewed, tested, and traceable. An artefact deployed by a manual action, outside the pipeline, is an artefact whose presence in production cannot be explained, and in a regulated institution an unexplained artefact is a finding.

The *monitoring* of the artefacts must cover both their functional correctness and their operational health. A process step that executes but produces wrong results is harder to detect than one that fails outright, because it does not raise an error — it raises a consequence, and the consequence may not be discovered until a customer complains or a regulator asks a question. The observability model should therefore include not only error rates and latencies but also *outcome monitoring*: checks that the results the process produces are within expected ranges, flagging anomalies for review.

Chapter in brief

Payments is two processes, not one: a high-volume, lean, near-decision-free execution path, and a lower-volume, process-rich exception and dispute process. The happy path is modelled as deliberately minimal, with the decision to leave it as the first real decision — intelligence belongs in the exception process, not the hot path. The dispute process is a long-running case where the orchestrator runs the flow, DMN holds the scheme rules and thresholds, and an agent reads and assesses the unstructured dispute narrative — with disposition deterministic and human-accountable. Process changes that look positive on cost, like withholding provisional credit, must be weighed against conduct and regulatory obligations.

Insurance Underwriting and Claims

123.1 Two processes at the core of an insurer

Where a bank's defining processes are onboarding, lending, and payments, an insurer's are *underwriting* — deciding whether to offer cover and at what price — and *claims* — deciding what to pay when a covered loss occurs. They are the insurance counterparts of credit origination and dispute resolution, and they show the same hyperautomation patterns with an insurance shape. This chapter treats both, because they are mirror images: underwriting prices a risk before it is taken on, claims settles a loss after it has occurred, and the data and decisions of one inform the other.

123.2 Underwriting: pricing a risk

The underwriting journey decomposes into *capture and validate application, enrich and verify, assess risk, decide and price, and issue*. Its structure echoes lending closely, and so does its decision architecture.

The *risk assessment* is partly predictive and partly rule-based. For many lines, a predictive model estimates expected claims cost — the insurance analogue of a default-probability model — from the characteristics of the risk. For other lines, or other parts of the same risk, a rating manual expresses the price as explicit factors and loadings: a set of DMN tables. The mature underwriting process uses both, and the layered pattern of [Chapter 22](#) governs: the model predicts expected cost, and a DMN-expressed pricing policy turns that prediction, together with the rating rules and the insurer's margin and appetite, into the actual premium.

The agentic work in underwriting is the unstructured enrichment: reading a submitted survey report, extracting risk-relevant facts from free-text application answers, assessing whether a commercial risk's description is consistent with the cover requested. As everywhere, the agent enriches and assesses; the price is a deterministic decision.

Referral is underwriting's characteristic exception. A risk that falls outside the automated appetite — too large, too unusual, a combination the rules do not cover — is referred to a human underwriter. The model makes referral an explicit, defined path: a DMN decision identifies the referral, the process routes to a user task with the assembled context, and the human underwriter's decision re-enters the flow. A well-modelled underwriting process is not one that automates every risk; it is one that automates the standard risks cleanly and routes the genuinely exceptional ones to a human with everything they need.

Tip

Measure your automated underwriting appetite as a deliberate, governed number, not an emergent one. The fraction of risks that can be priced straight-through is a policy

choice balancing efficiency against the cost of a mispriced automated decision. Make it explicit in the DMN referral logic, monitor it, and revisit it — rather than letting it drift as rules accrete.

123.3 Claims: settling a loss

The claims journey — *notify, triage, validate cover, assess, decide, settle* — is where insurance hyperautomation is richest, because claims are document-heavy, judgement-heavy, exception-prone, and the setting for most insurance fraud.

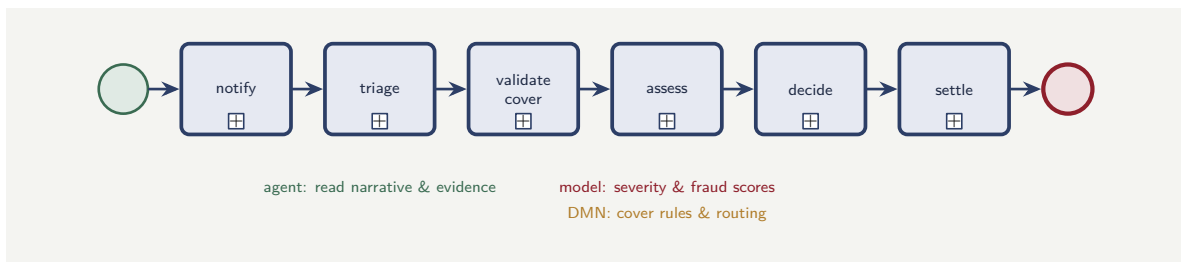


Figure 123.1. The claims journey as a sequence of capability subprocesses. The orchestrator runs the long-running claim; agents read the unstructured evidence; models estimate severity and fraud; and DMN holds the cover rules and routing decisions.

Figure 123.1 shows the journey and where each layer does its work. Each layer has clear work. The orchestrator runs the long-running claim, holding its state through the days or months between notification and settlement. DMN holds the cover rules — is this loss covered by this policy, within limits, subject to which excess — which are genuine rules and must be auditable. Predictive models estimate severity (the likely cost of the claim, which drives reserving) and fraud propensity. Agents do the heavy unstructured work that is the essence of claims: reading the first-notification narrative, the photographs, the adjuster’s report, the repair estimate, and assessing coherence — does the described incident, the documented damage, and the claimed amount form a consistent whole?

Fraud detection sits within claims and combines the layers characteristically. A predictive model scores fraud propensity; an agent reads the claim narrative and the evidence and assesses specific inconsistencies; a DMN decision combines those signals with hard rules to route the claim — to fast settlement, to standard handling, or to investigation. The decision to *deny* a claim for suspected fraud is never the model’s and never the agent’s; it is a deterministic, human-accountable decision, because a wrongful denial is both a conduct failure and, often, a legal one.

Figure 123.2 draws the fraud-routing pattern. It is the layered decision of Chapter 22 with an extra input: the model and the agent both feed the DMN decision, which combines two probabilistic signals with deterministic rules to produce a routing.

123.4 Reserving and the loop back to underwriting

A claims process does one thing beyond settling the individual claim that deserves its own treatment: it *reserves*. From the moment a loss is notified, the insurer must set aside its best estimate of what the claim will ultimately cost — the reserve — and that estimate is itself a decision the hyperautomated process makes, repeatedly, as the claim develops.

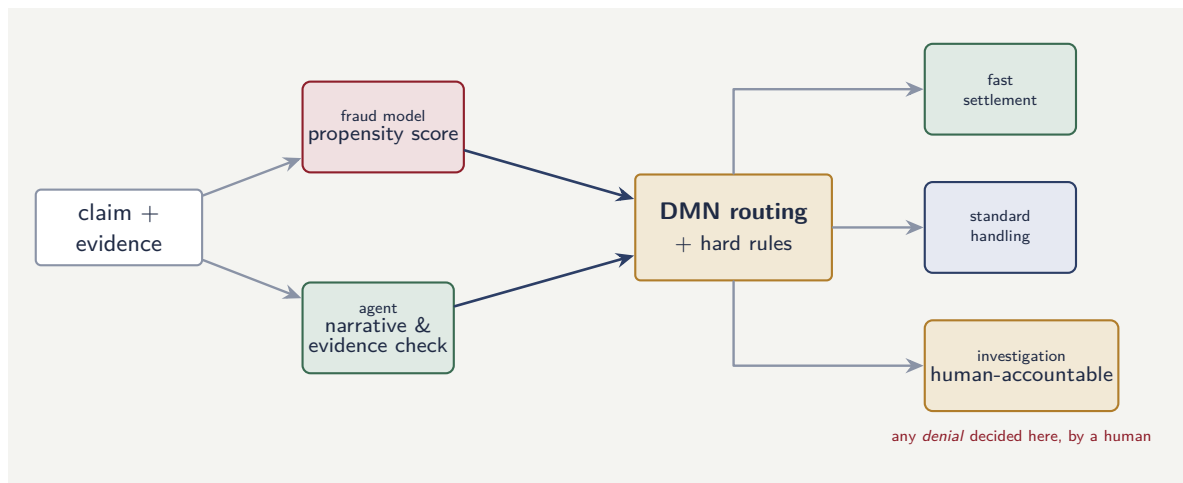


Figure 123.2. Fraud routing in claims combines all three layers. The model scores propensity, the agent assesses narrative and evidence, and a DMN decision combines both with hard rules to route the claim. The decision to deny is always deterministic and human-accountable.

Reserving is a predictive task, and it fits the layered pattern. A severity model estimates the likely ultimate cost from what is known at notification and updates the estimate as more is learned. The reserve the insurer actually books, though, is not simply the model's number: it is a governed decision that may apply prudential margins, case-specific adjustments by an adjuster, and the insurer's reserving policy. The model predicts; a governed decision sets the booked reserve. **Chapter 22's** insistence on calibration is especially sharp here, because — as this chapter's worked example shows — a miscalibrated severity model does not merely misroute a case; it makes the insurer's financial statements wrong.

The deeper point is that claims data closes a loop back to underwriting. The claims a book of policies generates are the realised outcomes of the risks the underwriting process priced. A severity model's errors, a fraud model's hit rate, the patterns in what is actually claimed — all of this is evidence about whether the underwriting of **Chapter 123's** first half priced those risks correctly. A mature insurer treats the two processes as a single feedback system: claims experience flows back as data that recalibrates the underwriting models and, sometimes, revises the underwriting policy. The hyperautomation platform, because it records every decision in both processes as governed, queryable data, is what makes that loop measurable rather than anecdotal.

Tip

Treat underwriting and claims as one feedback system, not two separate processes that happen to share a customer. The claims a book generates are the graded examination of how it was underwritten. An insurer whose platform can connect a claim's realised cost back to the risk assessment that priced it has turned its claims process into a continuous source of underwriting improvement — one of the most valuable things the shared, governed data of a hyperautomation platform makes possible.

123.5 Insurance-specific integration concerns

Insurance integration has concerns that banking integration does not share, and they are worth naming. *Long-tail claims*: an insurance claim can remain open for years, and the process that manages it must be designed for a lifetime measured in years, not days. *Actuarial data*: the process touches data that feeds actuarial models, and the integrity of that data is a pricing concern, not merely an operational one. *Regulatory reporting*: insurance regulators require specific, formatted reports on claims, reserves, and policyholder outcomes, and the process must produce the data those reports need. Each of these is a design input, not an afterthought.

Worked example: the reserving consequence of a severity model

Problem. An insurer's claims process sets an initial reserve — the amount set aside to pay a claim — using a severity model at the point of notification. The insurer notifies 20,000 motor claims a year. The model predicts an average severity of \$3,200 per claim; the realised average cost, once claims close, turns out to be \$3,680. The insurer holds reserves equal to the model's prediction. Capital rules require the insurer to hold the full reserve as it is set. If holding reserve capital costs the insurer 6% a year and claims remain open an average of 9 months, what is the annual cost of the reserving error, and what is the consequence the cost figure understates?

Setup. The reserving error per claim is the gap between realised and predicted severity. The under-reserved amount across the book is that gap times volume. The carrying cost is the cost of capital applied to the average amount, pro-rated for the 9-month average claim duration. We then consider what a pure carrying-cost figure leaves out.

Working. Severity under-estimate per claim:

$$\$3,680 - \$3,200 = \$480.$$

Across the book of 20,000 claims:

$$20,000 \times \$480 = \$9,600,000 \text{ of under-reserving.}$$

If instead the insurer reserved correctly, it would carry an extra \$9.6M; the cost of capital on that, for the 9-month (0.75-year) average duration, is

$$\$9,600,000 \times 0.06 \times 0.75 = \$432,000.$$

So the *carrying cost* of reserving correctly rather than under-reserving is about \$432,000 a year.

Discussion. A naive reading inverts the lesson: it notes that under-reserving “saves” \$432,000 of capital carrying cost and treats the model's optimism as a happy accident. That reading is dangerous and wrong. The \$432,000 is not a saving; it is the small, visible carrying cost, and it is swamped by the consequences a carrying-cost figure does not capture. A reserve is the insurer's own measurement of money it already owes; under-reserving by \$9.6M does not make the liability smaller, it makes the insurer's accounts *wrong* — it overstates current profit, understates true liabilities, and the \$9.6M gap surfaces later as adverse reserve development that must be funded out of capital at the worst possible time. For a regulated insurer that is a solvency and reporting failure, not a financing optimisation. The real lesson

is about the severity model: a model running 15% light (\$480/\$3,200) on average is miscalibrated in exactly the way [Chapter 22](#) warned of, and the fix is to recalibrate the model so the reserve it drives is right — not to enjoy the illusory saving of a reserve that is wrong.

Practical next steps

For one claims line, compare your severity model's predicted average against the realised average on recently-closed claims. If they differ materially, you have a reserving error of that gap times your claim volume sitting in your accounts. Quantify it, and treat it as a model-calibration defect to fix — the subject of [Chapter 126](#) — not as a financing question.

123.6 The insurance process estate

An insurer's process estate is distinctive in its *breadth* and its *timescales*. The breadth: an insurer runs processes across underwriting, policy administration, claims, renewals, complaints, and regulatory reporting — more process categories than most banks. The timescales: a claim can remain open for years, a policy for decades, and the processes that manage them must be designed for lifetimes measured in years, not days. These two properties — breadth and longevity — make the insurer's process estate one of the most demanding environments for a hyperautomation platform, and every architectural choice in this chapter must be evaluated against both.

The chapter's principles interact with the platform's operational model in a way worth making explicit. The *deployment* of the artefacts this chapter describes must go through a governed pipeline — versioned, reviewed, tested, and traceable. An artefact deployed by a manual action, outside the pipeline, is an artefact whose presence in production cannot be explained, and in a regulated institution an unexplained artefact is a finding.

The *monitoring* of the artefacts must cover both their functional correctness and their operational health. A process step that executes but produces wrong results is harder to detect than one that fails outright, because it does not raise an error — it raises a consequence, and the consequence may not be discovered until a customer complains or a regulator asks a question. The observability model should therefore include not only error rates and latencies but also *outcome monitoring*: checks that the results the process produces are within expected ranges, flagging anomalies for review.

Chapter in brief

Underwriting and claims are the insurer's defining processes — pricing a risk before it is taken on, settling a loss after it occurs. Underwriting decomposes like lending: predictive models estimate expected cost, DMN rating tables and pricing policy turn the prediction into a premium, agents enrich unstructured submissions, and referral is an explicit governed path. Claims is the richest insurance process: the orchestrator runs the long-running claim, DMN holds cover rules, models estimate severity and fraud, and agents assess the coherence of narratives and evidence. Fraud routing combines all three layers; denial is always deterministic and human-accountable. A miscalibrated severity model produces wrong reserves — an accounting and solvency failure, not a financing saving.

Servicing and the Ongoing Relationship

124.1 The process after the process

The applications of the preceding four chapters share a shape: each takes a case from initiation to a terminal outcome — an account opened, a loan booked, a dispute resolved, a claim settled. But the customer relationship does not end at those outcomes. After onboarding comes years of *servicing*: the changes, requests, life events, and interactions that make up the ongoing relationship between the customer and the institution. Servicing is the process after the process, and it is often the least well modelled, because it has no single clean shape.

Servicing is not one process but a large family of them: a change of address, a request for a payment holiday, a beneficiary change, a complaint, a product switch, a policy alteration, a hardship notification, a bereavement. Each is its own small process. What they share is that they are individually low-volume but collectively enormous, they are triggered by the customer at unpredictable times, and they vary from the trivial to the highly sensitive. Modelling servicing well is a different challenge from modelling a single high-value journey: it is the challenge of handling *variety* at scale.

124.2 The intake problem and the routing decision

The defining problem of servicing is intake. A customer makes contact — a message, a call, a form, a chat — and the institution must work out what they actually want and route it to the right process. A lending application announces itself; a servicing request arrives as “I need to sort out my account.”

This is where an agent has a natural and well-bounded role. An intake agent reads the unstructured customer contact and classifies it: what kind of request is this, what product does it concern, does it carry any signal of vulnerability or urgency. That classification is exactly the open-ended, unstructured judgement [Chapter 23](#) identified as agentic work. But — the recurring discipline — the agent *classifies and routes*; it does not *handle*. Its output is the input to a DMN routing decision that selects the appropriate servicing process, and the servicing process itself has whatever deterministic structure and human accountability its sensitivity requires.

Figure [124.1](#) draws the intake pattern, including the red path that records the discipline of the warning below: when the agent is uncertain, the routing sends the case to a human.

The intake agent will sometimes misclassify, and a misclassified sensitive request — a bereavement read as a routine account closure, a hardship notification read as a simple payment query — is a serious harm, not a minor inefficiency. Build the routing so that any signal of vulnerability, distress, or complaint escalates to a human-handled path

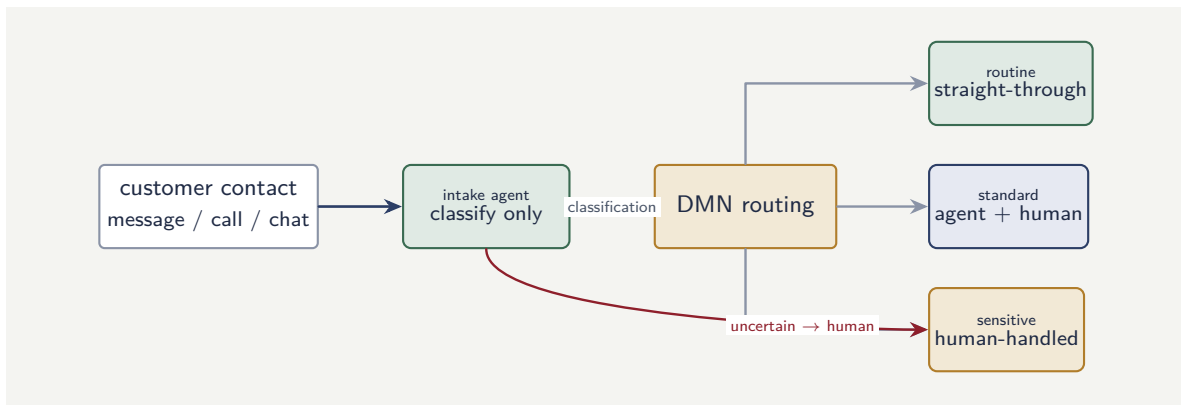


Figure 124.1. Servicing intake. The agent classifies the unstructured contact but never handles it; a DMN decision routes to routine, standard, or sensitive handling. Agent uncertainty biases deliberately toward the human-handled path.

even when the agent is uncertain. The agent’s uncertainty should bias toward human handling, never away from it.

124.3 Vulnerability and the limits of automation

Servicing is where a hyperautomation programme meets, most directly, the question of where automation should stop. Many servicing interactions are routine and benefit straightforwardly from straight-through processing: an address change, a duplicate-statement request, a standing-order amendment. Others touch customers in vulnerable circumstances — financial hardship, bereavement, illness, the aftermath of fraud — and these are precisely the interactions where a fast, automated, efficient process is the wrong answer.

A vulnerable customer needs a human, time, and judgement. The role of the hyperautomation platform for these interactions is not to handle them but to *recognise* them and route them well: to detect the signal of vulnerability at intake, to assemble the relevant context for the human who will handle the case, and to remove the routine work *around* the sensitive interaction so the human can give it their full attention. This is a genuine and valuable role — but it is a supporting role. The disciplined servicing design treats the identification of interactions that should *not* be fully automated as one of its primary requirements, and [Chapter 127](#) returns to this as a matter of fair treatment and conduct.

124.4 The process as a regulatory exhibit

Each of the application processes this part describes is, in a regulated institution, a potential *regulatory exhibit*: a process a regulator may ask to see, and whose execution record the institution must be able to produce for any past case. This is not an abstract concern; it is a concrete requirement that shapes the process design. Every decision point must be recorded, every human step must be traceable to an authenticated identity, and every automated step must log its inputs and outputs. A process that runs correctly but cannot produce an audit trail of a specific past case has failed its regulatory purpose.

Worked example: sizing the servicing automation opportunity

Problem. An institution handles 1,200,000 servicing interactions a year across many request types. An analysis classifies them: 65% are routine requests that could be fully automated, 25% are standard requests that need some human involvement, and 10% are sensitive interactions that should remain human-handled with full attention. Currently every interaction is handled manually at an average cost of \$14. Full automation of a routine request would cost \$1.20; a standard request handled with agent assistance plus human involvement would cost \$8; sensitive interactions stay at \$14 — but the institution decides to spend an extra \$6 of handling time on each sensitive interaction to improve the quality of care. What is the net change in annual servicing cost?

Setup. We compute the current total cost, then the proposed total cost across the three categories, including the deliberate extra spend on sensitive interactions, and take the difference.

Working. *Current cost.*

$$1,200,000 \times \$14 = \$16,800,000.$$

Proposed cost. Routine (65%): 780,000 interactions at \$1.20:

$$780,000 \times \$1.20 = \$936,000.$$

Standard (25%): 300,000 interactions at \$8:

$$300,000 \times \$8 = \$2,400,000.$$

Sensitive (10%): 120,000 interactions at \$14 + \$6 = \$20:

$$120,000 \times \$20 = \$2,400,000.$$

Proposed total:

$$\$936,000 + \$2,400,000 + \$2,400,000 = \$5,736,000.$$

Net change:

$$\$5,736,000 - \$16,800,000 = -\$11,064,000,$$

a reduction of about \$11.06M a year.

Discussion. The headline saving is large — roughly two-thirds of the servicing cost base — but the worked example is built to make a subtler point about *where* the saving comes from and what it buys. Almost all of the reduction is generated by the 65% of routine interactions, whose unit cost falls from \$14 to \$1.20. The 10% of sensitive interactions are deliberately made *more* expensive: an extra \$6 each, \$720,000 a year of additional spend, a choice the institution made on purpose. That is the design philosophy of this chapter expressed as a number. Hyperautomation in servicing is not uniform cost reduction; it is the reallocation of effort — stripping cost out of the routine so that human attention, and even additional spend, can be concentrated on the interactions that genuinely need a person. An institution that treated all 1,200,000 interactions as a single number to be minimised would automate the sensitive 10% too, save a little more, and do real harm. The disciplined institution segments first, automates the routine aggressively, and spends part of the proceeds on doing the sensitive work better.

Practical next steps

Segment one month of your servicing interactions into the three categories of the worked example: routine, standard, and sensitive. The exercise is valuable even before any automation, because most institutions do not know the proportions — and the proportion of sensitive interactions, in particular, is the number that should govern how much human capacity you protect rather than remove.

Chapter in brief

Servicing is the process after the process: a large family of individually low-volume, collectively enormous request types making up the ongoing relationship. Its defining challenge is variety, and its defining problem is intake — working out what an unstructured customer contact actually wants. An intake agent classifies and routes but never handles, feeding a DMN routing decision. Servicing is where automation must know its limits: vulnerable- circumstance interactions should be recognised and routed to human handling, not automated. Hyperautomation in servicing is not uniform cost reduction but the reallocation of effort — stripping cost from the routine to concentrate human attention on the interactions that need it.

PART XXII

Operations

Observability and Process Monitoring

125.1 Knowing what the platform is doing

A hyperautomation platform that has been built, integrated, and put into production is not finished; it is now *running*, and a running platform must be observed. Observability — the ability to know, at any moment and after any event, what the platform is doing and why — is not an operational afterthought. It is the property that distinguishes a platform an institution can responsibly run from one it is merely hoping works.

The case for observability in hyperautomation is stronger than for ordinary software, for two reasons. First, the platform makes consequential decisions about real customers — who is lent to, what claim is paid, which transaction is held — and the institution must be able to account for those decisions. Second, the platform contains predictive models and agents, components that degrade quietly: a model drifts, an agent’s accuracy slips, and nothing crashes. Without observability designed to catch quiet degradation, the first sign of a problem is a customer complaint or a regulatory finding, which is far too late.

125.2 The three things to observe

It helps to separate what a hyperautomation platform must observe into three distinct layers, because they answer different questions and are watched by different people.

Technical observability answers “is the platform healthy?” — the familiar territory of latency, error rates, throughput, queue depths, resource use, and availability. It is watched by platform engineers, and the standards of [Chapter 24](#) apply: it is the cloud platform’s operational telemetry.

Process observability answers “are the processes performing?” — and this is the layer unique to an orchestrated platform and the most valuable. Because every process is an explicit model, the orchestrator can report, for every process, how many instances are running, how long they take, where they wait, where they fail, how often each path is taken, and which SLAs are being met. Every state and transition that [Chapter 2](#) counted is a live metric. This layer is watched by process owners, and it makes a process owner’s accountability real: they can see their process performing, not guess.

Decision observability answers “are the decisions sound?” — the behaviour of the DMN decisions, the predictive models, and the agents. It tracks the distribution of decisions (what fraction of applications are being approved, what fraction of claims routed to investigation), the scores models produce, the assessments agents make, and how these change over time. It is watched jointly by the business and by risk, and it is the layer that catches the quiet degradation of a model or an agent.

Figure [125.1](#) stacks the three layers. The arrangement is deliberate: technical observability at the base is familiar from any software system, but the two layers above it — process and

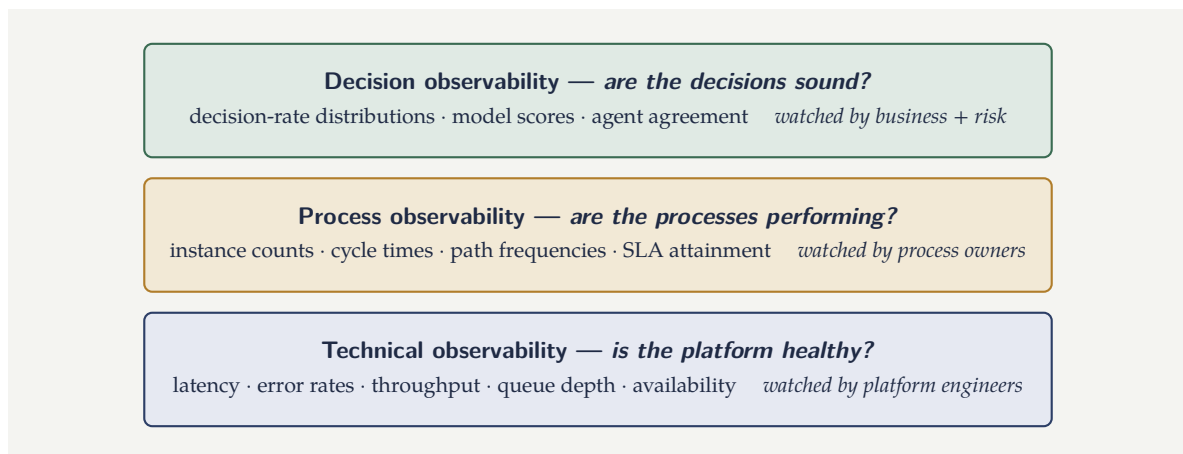


Figure 125.1. The three layers of observability. Each answers a different question and is watched by different people. Process and decision observability are the orchestrated platform’s distinctive advantage — and the layer that catches the quiet degradation of a model or an agent.

decision observability — are what an orchestrated hyperautomation platform can do and a pile of bots cannot.

Tip

Build process and decision observability *into* the platform from the start, not as a later analytics project. Because every process is an explicit model and every decision an explicit step, the platform already has the raw material — it knows every state, transition, and decision. The work is to expose it as live, owned dashboards. A hyperautomation platform that cannot show its process and decision metrics has discarded its main advantage over the pile of bots it replaced.

125.3 From observation to action

Observation has value only if it drives action, and the platform’s observability should be wired to response.

Some responses are automated. An SLA timer that is about to breach escalates the case — a behaviour modelled directly in BPMN as a boundary event. A queue depth that crosses a threshold triggers the elastic scaling of [Chapter 24](#). A decision-distribution shift beyond a defined band raises an alert.

Other responses are human and deliberate. A model whose score distribution has drifted is referred to the model-risk process of [Chapter 17](#). An agent whose assessments are diverging from human reviewers’ is investigated. A process whose cycle time is climbing is examined by its owner. The point is that observability is not a wall of dashboards nobody acts on; it is a defined set of signals, each with a defined owner and a defined response — the operational nervous system of the platform.

125.4 The end-to-end trace

The three layers of observation just described — technical, process, and decision — are each watched in aggregate: latencies across all instances, cycle times across a process, decision

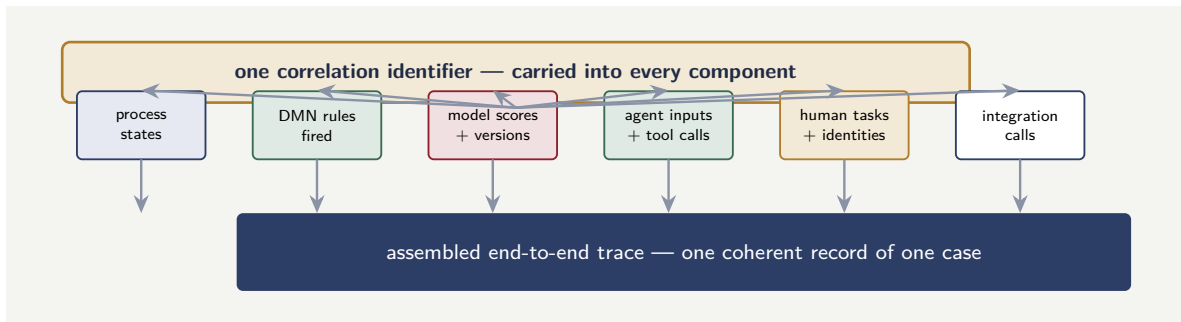


Figure 125.2. The end-to-end trace. A single correlation identifier, established at the start of the process instance and carried into every component, is what lets the records scattered across the orchestrator, decision services, model services, agent runtime, and integration layer be gathered into one coherent account of a single case.

rates across a population. But a running platform also needs the opposite of aggregate: the ability to take a *single* case and reconstruct, completely, everything that happened to it. This is the *end-to-end trace*, and it is the observability counterpart of the explainability requirement that [Chapter 18](#) will develop.

A single declined loan, a single investigated claim, a single disputed payment — for any one of them, the platform should be able to assemble one coherent record: the process instance and every state it passed through, the DMN decisions it triggered and the rules that fired, the model scores it received with their versions, the agent invocations with their inputs and tool calls, the human tasks and who completed them, and every integration call to a system of record. The record spans every layer and every component, and it is assembled for one case.

Figure 125.2 shows how the trace is assembled. The correlation identifier at the top is the single design feature that makes it possible: without it, each component knows its own part of the story and there is no key by which the parts can be joined.

The difficulty is that these components are distinct — the orchestrator, the decision services, the model services, the agent runtime, the integration layer — and each naturally records its own activity in its own place. The end-to-end trace requires that all of them share a *correlation identifier*: a single identifier, established when the process instance begins and carried into every downstream call, so that the records scattered across the components can later be gathered by that common key. A platform that does not propagate a correlation identifier from the first step has components that each know their own part of the story and no way to assemble the whole.

The end-to-end trace earns its cost in three ways. It is the raw material of explainability: [Chapter 127](#)'s per-case decision record is exactly this trace, viewed through a regulatory lens. It is the fastest route to diagnosis when a single case behaves wrongly: rather than inspecting five components separately, an investigator reads one assembled story. And it is what makes the aggregate observability *actionable* — when decision observability flags a drift, the investigation that follows works by examining the end-to-end traces of the affected cases.

Tip

Establish a correlation identifier at the start of every process instance and require every component — decision services, model services, agent runtime, integration calls — to carry it and stamp it on everything they record. Retrofitting correlation across com-

ponents that were built without it is painful and never quite complete. Designed in from the start, it costs almost nothing and is the single feature that turns a collection of separately-logging components into a platform whose every case can be told as one story.

125.5 Operations as a first-class concern

Operations is not an afterthought bolted on after the platform is built; it is a first-class concern designed in from the start. A platform that was designed for development convenience and never for operational visibility will be, in production, a black box: running but unobservable, processing but unmeasurable, failing but not detectably so. The operational disciplines this chapter describes — monitoring, alerting, incident management, capacity planning — must be part of the platform's design from the first sprint, not a project phase added after go-live when the first production incident reveals their absence.

Worked example: detecting model drift from decision rates

Problem. A claims process routes claims to investigation using a fraud model and a DMN threshold. Historically the process routes 8% of claims to investigation, and this rate has been stable. Over the most recent month, of 9,000 claims, 1,080 were routed to investigation. The investigations team can handle 800 investigations a month within its staffing. The platform's decision observability flags any monthly routing rate that deviates from the 8% baseline by more than 2 percentage points. Is the recent month flagged, what is the operational consequence, and what should the response be?

Setup. We compute the recent routing rate, compare it with the 8% baseline and the 2-point band, and compare the resulting investigation volume with the team's capacity.

Working. Recent-month routing rate:

$$\frac{1,080}{9,000} = 0.12, \quad \text{that is } 12\%.$$

Deviation from the baseline:

$$12\% - 8\% = 4 \text{ percentage points},$$

which exceeds the 2-point band — so the month is **flagged**.

Investigation volume against capacity: 1,080 routed against a capacity of 800:

$$1,080 - 800 = 280 \text{ investigations beyond capacity.}$$

Discussion. The flag is correct and the operational consequence is immediate: 280 claims a month are being routed to an investigations team that cannot absorb them, so a backlog is forming and will grow until the cause is addressed. But the worked example's real lesson is in how the response must *not* be chosen. The fastest way to clear the backlog is to raise the DMN threshold so that fewer claims are routed — and that would be exactly the wrong first move, because it treats the symptom and may hide a real problem. The 12% rate has at least three quite different possible causes. It could be genuine: fraud has actually risen, and the process is correctly catching more of it — in which case the answer is more investigation capacity, not a higher threshold. It could be model drift: the fraud model has degraded and

is over-routing, the quiet degradation this chapter warned about — in which case the answer is the model-risk process of [Chapter 126](#). Or it could be a data problem: an upstream change has shifted the inputs the model sees. Decision observability has done its job by raising the flag; it has not told the institution which of the three is true. The disciplined response is to investigate the cause before touching the threshold, and to add interim capacity so that genuine fraud is not waved through while the diagnosis proceeds.

Practical next steps

For one model-driven decision in your institution, find its baseline decision rate — the fraction approved, routed, or referred — and check whether anyone would currently notice if that rate moved by 4 points. If no one is watching the decision rate, the platform has decision logic running unobserved, and a drifting model could run for months before anyone knew.

Chapter in brief

A running hyperautomation platform must be observed, and observability is the property separating a platform an institution can responsibly run from one it hopes works — doubly so because models and agents degrade quietly. Observe three layers: technical (is the platform healthy?), process (are the processes performing?), and decision (are the decisions sound?). Process and decision observability are the orchestrated platform's unique advantage and must be built in from the start. Observability has value only wired to response — some automated, some human — with every signal owned. A shifted decision rate is a flag, not a diagnosis: investigate the cause before adjusting the threshold.

Testing, Validation, and Model Risk

126.1 Three things that must be shown to work

Before a hyperautomation process carries real customers, and continuously after, the institution must be able to show that it works. “Works” here has three distinct meanings, because a hyperautomation process has three kinds of component, and each is shown to work differently.

The *process model* must be shown to behave correctly: every path it can take leads where it should, every exception is handled, every SLA timer fires as intended. This is process testing, and it is largely deterministic — Chapter 3 noted that every distinct path through a BPMN model is a test case.

The *decision logic* must be shown to be correct: the DMN tables are complete and consistent, the rules express the intended policy, the boundaries are right. This is decision testing, and Chapter 4’s completeness and consistency checks are its foundation.

The *predictive models and agents* must be shown to be fit for use — and this is different in kind, because their correctness is statistical and probabilistic rather than absolute. This is the domain of *model risk*, and it is where most of the difficulty and most of the regulatory attention lie.

126.2 Testing the deterministic parts

The deterministic parts of a hyperautomation process — the BPMN flow and the DMN decisions — can be tested with confidence, and the institution should exploit that.

A process model is tested by exercising every path. Because the model is explicit, the paths are enumerable, as Chapter 3’s worked example showed: every routing branch, every boundary event, every exception flow is a defined test case. A process that passes a test of every path is genuinely known to behave correctly, which is a stronger guarantee than most software offers.

A DMN decision is tested by checking its formal properties — completeness, consistency — which tooling can do exhaustively, and by exercising it against a library of cases with known correct outcomes. Because a decision table is a finite, explicit object, this testing can be thorough.

The lesson is to draw the line clearly: the deterministic parts of the platform can and must be tested to a high standard, and that testing is tractable precisely because BPMN and DMN are explicit, finite, enumerable artefacts. The effort that this tractability saves should be redirected to the parts that are genuinely hard — the models and the agents.

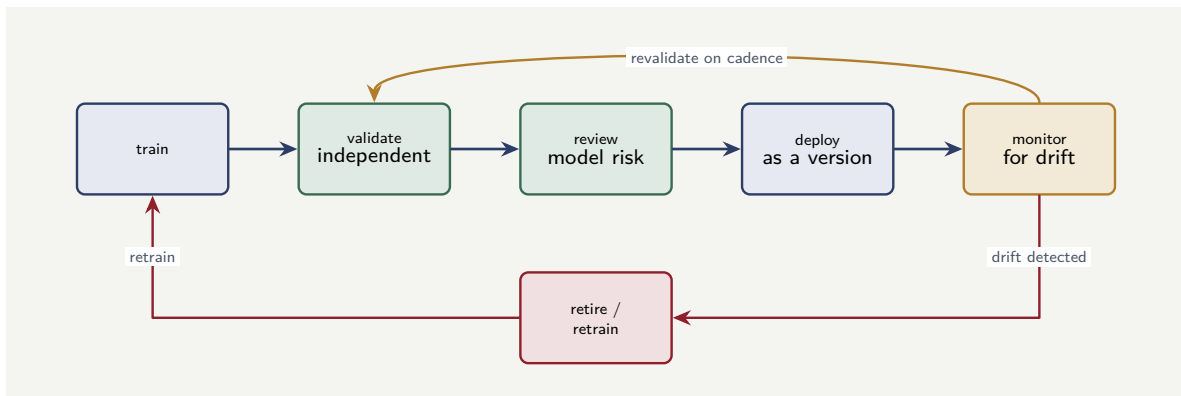


Figure 126.1. The model lifecycle. A model is trained, independently validated, reviewed by model risk, deployed as a version, and monitored — and the loop back from monitoring to revalidation and retraining is what keeps a deployed model from decaying silently.

126.3 Model risk: validating the probabilistic parts

A predictive model cannot be tested in the deterministic sense. It will be wrong on some cases; that is in its nature. The discipline of *model risk* is not about eliminating that wrongness but about understanding it, bounding it, governing it, and detecting when it grows.

Model risk runs across the model lifecycle that [Chapter 22](#) sketched. Before deployment, a model is *validated*: its accuracy is measured on data it was not trained on; its *calibration* is checked, in the sense of [Chapter 22](#)’s worked example; its behaviour across customer groups is examined for fairness, the subject of [Chapter 127](#); its stability and its failure modes are characterised. The validation is performed, importantly, by people independent of those who built the model, and it produces a documented opinion on whether and how the model may be used. After deployment, the model is *monitored* — the decision observability of [Chapter 125](#) — for drift in its inputs and degradation in its performance, and it is *revalidated* periodically and whenever monitoring raises a concern.

Figure [126.1](#) draws the lifecycle as the loop it really is. The forward path — train, validate, review, deploy — is the part teams remember; the feedback paths from monitoring back to revalidation and retraining are the part they forget, and they are exactly what the warning below is about.

A predictive model that has been deployed and never revisited is a growing, unmeasured risk. The world the model was trained on drifts away from the world it now operates in — customer behaviour changes, the economy moves, upstream data shifts — and the model’s accuracy silently decays. A model with no owner, no monitoring, and no revalidation schedule is not an asset; it is an incident waiting to be discovered. Every model in production has a named owner and a revalidation cadence, or it does not stay in production.

126.4 Validating an agent

An agent is the hardest component to validate, because its task is open-ended and its output is a judgement. It cannot be validated by enumerating paths, and it cannot be validated by a single accuracy number the way a classifier can. Yet it must be validated, because [Chapter 23](#)

showed what an unvalidated agent costs.

Agent validation rests on a few principles. The agent is validated *on its bounded task*, not in general — the narrow scope that [Chapter 23](#) insisted on is what makes validation possible at all. It is validated against *human judgement*: a sample of cases is assessed both by the agent and by qualified humans, and the agent’s agreement with the humans — and the nature and cost of its disagreements — is measured. It is validated for its *failure behaviour*: not just how often it is wrong, but how wrong, how confidently, and whether its errors are the kind the containment of [Chapter 23](#) will catch. And it is *monitored* continuously against the human reviewers who handle the cases it does not, so that a drift in its agreement rate is caught. The agent, like the model, is never validated once; it is validated continuously, because the thing it reasons about keeps changing.

126.5 The Java discipline for a Camunda platform

The Java code behind a Camunda platform is production code in a regulated institution, and it must be held to production standards: code review, static analysis, dependency scanning, and the testing disciplines this part describes. A worker that “works” but has not been reviewed, tested, and scanned is technical debt from the moment it is deployed, and in a regulated institution technical debt is also regulatory debt — a regulator who asks “how do you ensure the quality of your automation code” expects an answer, and “it works in the demo” is not one.

Worked example: the sample size to validate an agent

Problem. A team wants to validate a claims-assessment agent by having qualified human assessors independently assess a sample of the agent’s cases and measuring the agent’s error rate — the fraction on which it disagrees with the human consensus. The team needs the estimated error rate to be accurate to within roughly ± 2 percentage points. As a standard approximation, the margin of error of an estimated proportion at this confidence level is about $1/\sqrt{n}$, where n is the sample size. How many cases must be human-assessed, and what does the answer imply about the cost of validation?

Setup. We set the margin-of-error approximation $1/\sqrt{n}$ equal to the target precision of 0.02 and solve for n . We then reflect on what sampling that many cases costs and how often it must be repeated.

Working. Set the margin of error to the target:

$$\frac{1}{\sqrt{n}} = 0.02.$$

Solve for $\sqrt{n}$:

$$\sqrt{n} = \frac{1}{0.02} = 50.$$

So

$$n = 50^2 = 2,500 \text{ cases.}$$

To estimate the agent’s error rate to within ± 2 points, about 2,500 cases must be independently assessed by qualified humans.

Discussion. The number is larger than teams expect, and confronting it honestly is the point of the exercise. Validating the agent to a useful precision requires 2,500 cases assessed by qualified human assessors — and those are exactly the experienced people whose scarcity

motivated automating the task in the first place. Two consequences follow. First, validation is not free and not quick; it is a real, recurring cost that must be in the business case for the agent, alongside its compute cost. Second — and this is the deeper point — validation is not a one-off. [Chapter 23](#) and this chapter both stressed that an agent must be monitored continuously because the world it reasons about drifts, which means the 2,500-case exercise recurs on a cadence, not once. If the team wanted tighter precision, the cost rises sharply: ± 1 point would need $1/\sqrt{n} = 0.01$, hence $n = 10,000$ cases — four times the effort for twice the precision, because precision improves only with the square root of sample size. The disciplined response is to choose a validation precision that is genuinely needed rather than reflexively tight, to budget the recurring human-assessment cost explicitly, and — where possible — to use the human reviewers who already handle the agent’s non-cleared cases as a continuous, lower-cost validation stream rather than commissioning a separate exercise each time.

Practical next steps

For any agent or model you intend to validate, decide the precision you actually need and compute the resulting sample size. Then identify who will do the human assessment and how often. If the recurring human-assessment cost is not in the business case, the business case is incomplete — and the gap is usually large enough to change the decision.

Chapter in brief

A hyperautomation process has three kinds of component that must be shown to work differently. The deterministic parts — the BPMN flow and the DMN decisions — can be tested thoroughly because they are explicit, finite, and enumerable. The predictive models are governed by model risk: independent validation of accuracy, calibration, fairness, and stability before deployment, then monitoring and revalidation, because an unvisited model decays silently. Agents are hardest — validated on their bounded task against human judgement, for their failure behaviour, and continuously. Validation is a real, recurring cost that must be in the business case.

Regulation, Fairness, and Explainability

127.1 Why this chapter is not optional

Everything the preceding chapters built — the orchestration, the decisions, the models, the agents — operates inside a regulated industry, and a hyperautomation platform that cannot satisfy its regulatory obligations cannot be put into production, however elegant its architecture or compelling its economics. This chapter treats three obligations that bear most directly on hyperautomation: the decisions the platform makes must be *explainable*, they must be *fair*, and the whole platform must be *accountable*. These are not constraints to be worked around. They are design requirements, and a platform built without them is a platform built to fail its first serious examination.

127.2 Explainability: the platform's strong suit

A regulated financial institution must, for any decision it makes about a customer, be able to say why. Why was this loan declined, this claim routed to investigation, this premium loaded, this dispute rejected. The obligation is both regulatory and, increasingly, a matter of customers' rights.

Here the architecture of this manuscript has a genuine advantage, and it is worth claiming plainly. A decision made by an orchestrated process is *explainable by construction*. The process is an explicit model, so the path the case took is known. The decisions are explicit DMN steps, so — [Chapter 4](#) — the exact rule that fired on the exact inputs is recorded. The predictive model's contribution is a versioned, attributed score with its inputs as recorded — [Chapter 6](#). The agent's contribution is a traced, schema-bounded assessment — [Chapter 23](#). The orchestrator assembled all of this into a per-case record. The explanation of a decision is therefore not reconstructed after the fact; it is assembled from what the platform already recorded. An institution running a pile of bots cannot do this. An institution running an orchestrated hyperautomation platform can, and should treat the capability as one of the main returns on the architecture.

Figure [127.1](#) draws explainability as a property each layer contributes.

The one component that complicates the picture is the predictive model, which may be accurate without being transparent. This is why [Chapter 22](#) required the model service to provide an attribution — an indication of which features drove the score — and why the layered pattern matters: because the *decision* is a DMN step consuming the score, the action taken is always explainable as a rule, even when the score feeding it is from a complex model. The model explains its score as best it can; the table explains the decision exactly.

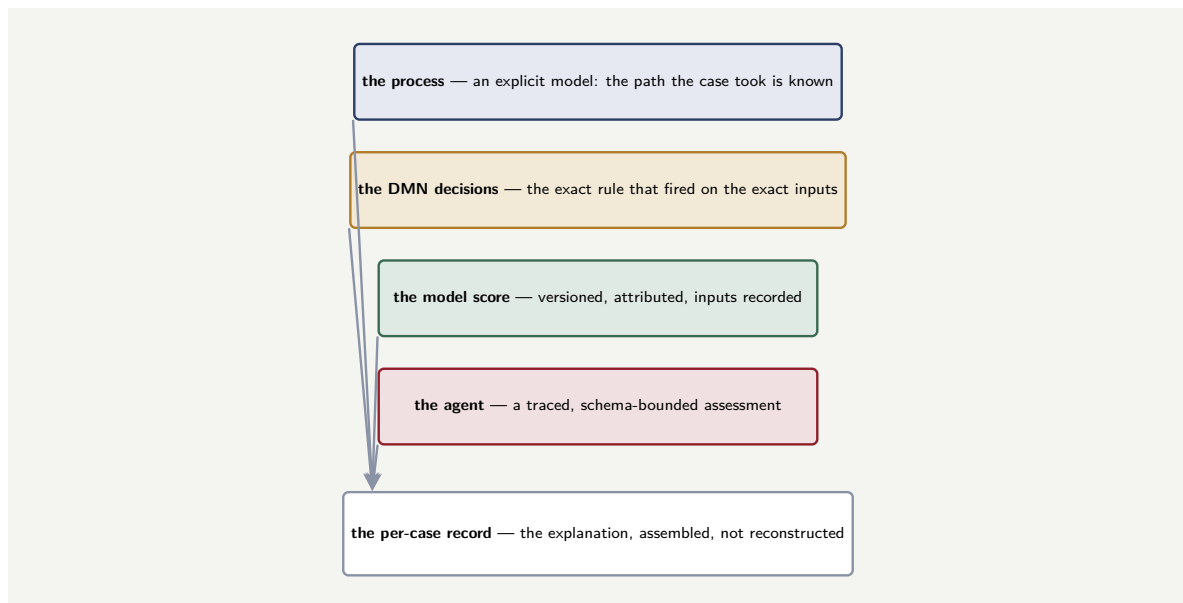


Figure 127.1. Explainability by construction. Each layer of the platform records its own contribution — the process its path, the DMN steps their fired rules, the model its attributed score, the agent its traced assessment — and the orchestrator assembles them into a per-case record. The explanation is not reconstructed after the fact; it is already there.

Tip

Treat the per-case decision record as a product of the platform, designed deliberately: for any decision, it should be possible to produce, quickly and without archaeology, the process path, the rules that fired, the model versions and scores, the agent assessments, and the human actions and their accountable identities. If producing that record is hard, explainability has not been designed in — and it is far cheaper to design in than to retrofit under examination.

127.3 Fairness: a property that must be measured

A hyperautomation platform makes decisions at scale, and a platform that makes decisions at scale can deliver *unfairness* at scale. Fairness is therefore not a hope but a property that must be defined, measured, and monitored.

The risk enters principally through the predictive models. A model trained on historical data learns the patterns in that data, and if the historical data reflects past unfairness — if certain groups were treated less favourably — the model can learn and then perpetuate that pattern, even when the protected characteristic itself is never an input, because other features may stand in for it. This is not a hypothetical; it is a well-understood failure mode, and in financial services it is also frequently unlawful.

Addressing it is part of the model risk discipline of [Chapter 126](#). Fairness is examined at validation: the model's decisions and errors are measured across customer groups, against a fairness definition the institution has chosen deliberately and can defend. It is monitored in production through the decision observability of [Chapter 16](#): decision rates across groups are watched for divergence. And it is governed: a model that cannot be shown to be fair, on the institution's defined and defensible criterion, does not go into production. The layered

pattern helps here too — because the final decision is an explicit DMN policy, the institution has an explicit, inspectable place where its fairness constraints are expressed, rather than a fairness property buried unrecoverably inside a model.

127.4 Accountability: a human at every consequential decision

The deepest regulatory principle for hyperautomation is also the simplest to state: for every consequential decision, a specific human being is accountable. Automation does not dilute accountability, and “the model decided” or “the agent decided” is not an answer a regulated institution may give.

This principle has been present throughout the manuscript, and this chapter only names it. It is why [Chapter 23](#) placed a human task on the path of every consequential agentic decision. It is why [Chapter 26](#) insisted that every action link to an accountable identity. It is why the disposition of a screening hit ([Chapter 120](#)), a fraud denial ([Chapter 14](#)), and a sensitive servicing interaction ([Chapter 124](#)) were all kept on human-accountable paths. The platform’s job is not to remove the accountable human but to support them: to bring them the right cases with the right context, to handle the routine so their attention goes to what matters, and to record what they decided and why. A hyperautomation platform is, in the end, a machine for letting a finite number of accountable humans govern an effectively infinite number of cases — and the accountability is the point, not an obstacle to it.

127.5 The supervisory relationship

A regulated institution does not merely comply with rules in the abstract; it maintains a *relationship* with one or more supervisors, and a hyperautomation platform changes the texture of that relationship in ways worth anticipating.

A supervisor examining a traditional, manual or bot-automated process is largely dependent on what the institution chooses to show: documents, reconstructed descriptions, sampled cases. A supervisor examining a hyperautomation platform of the kind this manuscript describes encounters something different — a platform whose processes are explicit models, whose decisions are recorded with the rules that fired, whose models are versioned and validated, and whose every action links to an accountable identity. This is, for the well-prepared institution, an advantage: it can answer supervisory questions directly, from the platform, rather than through a laborious and uncertain reconstruction.

But the same transparency raises the supervisory expectation. A platform that *can* show exactly how every decision was made will be expected to. The institution can no longer treat the inner workings of a process as unobservable; it has made them observable, and observability cuts both ways. This is not a reason to avoid building observable platforms — the alternative, an unobservable platform, fails a serious examination outright. It is a reason to ensure that what the platform reveals is something the institution is content to have revealed: that the decisions are sound, the models validated, the fairness measured, the accountability real. The disciplined institution treats the platform’s transparency as a standing invitation to keep its own house in order, and walks the regulatory reference of this manuscript’s appendices on its own initiative, on a cadence, rather than waiting for a supervisor to walk it for them.

A hyperautomation platform makes an institution's processes legible to its supervisors in a way a pile of bots never could. An institution that builds such a platform and then governs it poorly has not hidden its problems — it has documented them. Build the platform observable, and ensure that what the observability shows is a platform genuinely worth showing.

127.6 Operations as a first-class concern

Operations is not an afterthought bolted on after the platform is built; it is a first-class concern designed in from the start. A platform that was designed for development convenience and never for operational visibility will be, in production, a black box: running but unobservable, processing but unmeasurable, failing but not detectably so. The operational disciplines this chapter describes — monitoring, alerting, incident management, capacity planning — must be part of the platform's design from the first sprint, not a project phase added after go-live when the first production incident reveals their absence.

Worked example: a fairness gap in approval rates

Problem. A lending model's decisions are reviewed for fairness across two customer groups, A and B, who are equally represented in the applicant pool. Among group A applicants, 72% are approved; among group B applicants, 66% are approved. The institution's chosen fairness criterion requires that approval rates across groups not differ by more than 4 percentage points *after* controlling for genuine creditworthiness. An analysis that controls for verified income, existing debt, and credit history finds that these legitimate factors account for 3.5 points of the gap. Does the model pass the institution's fairness criterion, and what should happen next?

Setup. The raw gap is the difference in approval rates. The criterion applies to the *residual* gap — the part not explained by legitimate creditworthiness factors. We compute the raw gap, subtract the explained portion, and compare the residual with the 4-point threshold.

Working. Raw approval-rate gap:

$$72\% - 66\% = 6 \text{ percentage points.}$$

Portion explained by legitimate creditworthiness factors: 3.5 points. Residual, unexplained gap:

$$6 - 3.5 = 2.5 \text{ percentage points.}$$

The residual gap of 2.5 points is below the institution's 4-point threshold, so on this criterion the model **passes**.

Discussion. The model passes, but the worked example exists to show why "passes" must not end the conversation. Three cautions belong with the result. First, the pass depends entirely on the claim that legitimate factors explain 3.5 of the 6 points — and that figure is the output of an analysis that itself rests on assumptions: that verified income, existing debt, and credit history are themselves untainted by historical unfairness, and that they were measured well. If any of those factors is itself a partial proxy for group membership, the "explained" portion is overstated and the true residual is larger. Second, a 2.5-point residual that passes a 4-point threshold is still a 2.5-point residual: real group B applicants are being declined at a rate the institution's own analysis cannot fully justify, and "within threshold" is a reason

to monitor closely, not a reason to stop looking. Third, the criterion and the threshold are themselves choices the institution made and must defend; a regulator may apply a different fairness definition, and the institution should know how the model fares under more than one. The disciplined response to a pass is to record the analysis and its assumptions, to monitor the residual gap over time through the decision observability of [Chapter 125](#), and to treat any widening as a model-risk trigger — not to file the result and move on.

Practical next steps

For one model-driven decision in your institution, compute the raw decision- rate gap between two customer groups. Then ask the harder question: how much of that gap can your institution actually *explain*, with evidence, by legitimate factors — and how confident is it that those factors are not themselves proxies? The honest answer to the second half of that question is usually less comfortable than the first.

Chapter in brief

A hyperautomation platform operates inside a regulated industry, and explainability, fairness, and accountability are design requirements, not constraints to work around. Explainability is the architecture's strong suit: an orchestrated decision is explainable by construction, assembled from the process path, the DMN rules that fired, the versioned model scores, the traced agent assessments, and the human actions. Fairness is a property that must be defined, measured at validation, and monitored in production, because models can learn historical unfairness. Accountability is the deepest principle: for every consequential decision a specific human is accountable, and the platform exists to support that human, not replace them.

CHAPTER 128

The Economics of Hyperautomation

128.1 Why a cost chapter

A hyperautomation programme is justified by its economics, and yet the economics are routinely mis-estimated — in both directions. They are over-estimated when a business case counts only the visible processing saving and ignores the costs of building, integrating, validating, and running the platform. They are under-estimated when an institution, daunted by the build cost, never accounts for the compounding cost of *not* modernising. This chapter is about estimating the economics honestly.

128.2 The four costs

A hyperautomation platform has four categories of cost, and a business case that omits any of them is wrong.

The *build cost* is the one-off cost of designing the process models, the decision logic, the agents, the integrations, and the platform itself — the architecture of Parts II and IV. It is visible and usually estimated, though the integration component ([Chapter 25](#)) is routinely underestimated.

The *run cost* is the ongoing cost of operating the platform: the cloud infrastructure of [Chapter 24](#), and — distinctively for hyperautomation — the per-invocation cost of the agentic layer, which unlike classical compute can be a material variable cost that scales with volume.

The *governance cost* is the ongoing cost of the disciplines this manuscript has insisted on: the model validation and revalidation of [Chapter 126](#), including its recurring human-assessment component; the monitoring of [Chapter 125](#); the fairness analysis of [Chapter 127](#). This cost is real, recurring, and almost always missing from the first draft of a business case.

The *change cost* is the ongoing cost of keeping the platform current — [Chapter 17](#)'s operating model — as processes, rules, models, regulation, and the connected systems all evolve.

The two costs most often missing from a hyperautomation business case are the governance cost and the variable agentic run cost. Omitting governance makes the platform look cheaper than it is and, worse, tempts the institution to skimp on the validation and monitoring that keep it safe. Omitting the variable agentic cost produces a business case that is accurate at pilot volume and badly wrong at production volume. Put both in the model from the first draft.

Figure [128.1](#) sets out the four costs and the two most often missing.

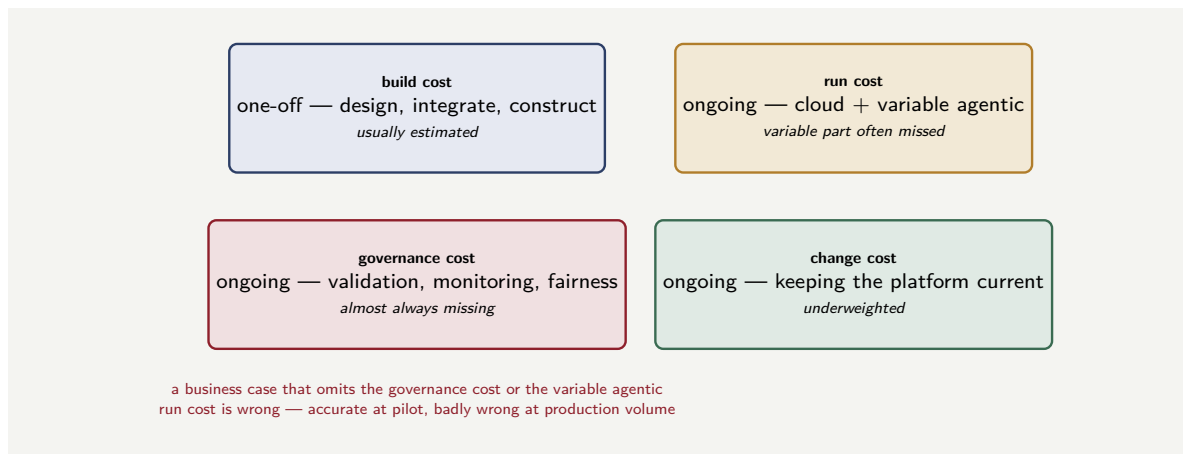


Figure 128.1. The four costs of a hyperautomation platform. Only the one-off build cost is reliably estimated; the three ongoing costs — run, governance, and change — are routinely understated, and a business case that omits the governance cost or the variable agentic run cost is wrong.

128.3 The benefit side, honestly

The benefits of hyperautomation are real but must be counted with the same discipline as the costs.

The *processing saving* — less manual handling — is the visible benefit, and [Chapter 1](#)'s worked example showed how to size it. It is real, but it is rarely the whole story and is often not the largest part.

The *error and loss reduction* is frequently larger and almost always under-counted: the missed identity failures of [Chapter 120](#), the mispriced risks of [Chapter 123](#), the unrecoverable dispute losses of [Chapter 122](#). These are the invisible costs of the current process, and reducing them is often where the real money is.

The *speed and experience benefit* — faster decisions, shorter cycle times — converts into competitive advantage and customer retention. It is harder to quantify and should be estimated conservatively rather than omitted.

The *capacity reallocation* benefit — [Chapter 124](#)'s theme — is that human effort removed from routine work is redeployed to judgement, exceptions, and vulnerable customers. This is a genuine benefit even when it is not a headcount saving, and a mature business case says so honestly rather than dressing it as a cost cut.

128.4 The cost of not modernising

The hardest number in the business case is the cost of the status quo, because it is a cost the institution is already paying and has stopped noticing. The unmodernised process has a rising cost curve: its manual effort does not scale economically with volume, its error costs persist, its cycle times lose customers, its undocumented logic is a growing operational risk, and the aging systems beneath it cost ever more to maintain. A business case that compares the hyperautomation investment against a flat “do nothing” baseline is comparing it against a fiction. The honest comparison is against the *rising* cost of the status quo — and against that curve, the case for modernisation is usually far stronger than a flat-baseline comparison makes it look.

128.5 Phasing the investment

A hyperautomation programme is rarely affordable, or wise, as a single all-at-once investment. It is phased — and how it is phased is itself an economic decision, because the sequence determines when value arrives, when risk is taken, and whether the programme sustains the support it needs to finish.

The maturity progression of [Chapter 1](#) already implies the shape of the phasing within a single process: model the process first, add predictive decisions next, add contained agents last. The economic argument reinforces the order. Modelling the process delivers the governance and the observability and often a substantial processing saving on its own — real value, at the lowest risk, because it adds no probabilistic component. Adding prediction and then agents delivers further value but adds the governance cost and the model risk that Part XIV has detailed. Front-loading the lower-risk, value-delivering work funds and de-risks the harder work that follows.

Across processes, the phasing question is which process to hyperautomate first. The instinct is to start with the largest, because that is where the biggest saving sits. The instinct is usually wrong. The first process should be chosen to *build the platform and the capability* — the orchestration core, the integration patterns, the operating model, the team — as much as to deliver a saving. A good first process is large enough to matter and to prove the case, structured enough to model cleanly, and not so entangled with the institution's most fragile systems that the programme's first delivery becomes its hardest. The biggest, most tangled process is often the worst first choice and the right third or fourth one, once the platform and the team exist.

Tip

Sequence the programme so that value and learning arrive early and risk is taken late. Within a process, model before you add prediction, and add prediction before you add agents. Across processes, choose a first process that builds the platform and the team — substantial, but not the most entangled. A programme that takes its hardest risk first, before the capability exists to manage it, is a programme that struggles to earn its second phase of funding.

128.6 Operations as a first-class concern

Operations is not an afterthought bolted on after the platform is built; it is a first-class concern designed in from the start. A platform that was designed for development convenience and never for operational visibility will be, in production, a black box: running but unobservable, processing but unmeasurable, failing but not detectably so. The operational disciplines this chapter describes — monitoring, alerting, incident management, capacity planning — must be part of the platform's design from the first sprint, not a project phase added after go-live when the first production incident reveals their absence.

Worked example: the full-cost business case for a process

Problem. An insurer evaluates hyperautomating a process. The visible processing saving is estimated at \$4.0M a year. The build cost is \$5.0M one-off. The run cost is \$0.8M a year. The governance cost — model validation, monitoring, fairness analysis — is \$0.6M a year.

The change cost is \$0.4M a year. Separately, the team estimates that reducing process errors will avoid \$1.5M a year of losses. Evaluate the annual net benefit and the simple payback period, first counting only the visible saving against all four costs, and then including the error-reduction benefit.

Setup. Annual net benefit is annual benefits minus annual costs; annual costs are run plus governance plus change. Simple payback is the build cost divided by the annual net benefit. We do this twice: once with the processing saving alone, once including error reduction.

Working. Total annual cost:

$$\$0.8\text{M} + \$0.6\text{M} + \$0.4\text{M} = \$1.8\text{M}.$$

Counting only the visible processing saving. Annual net benefit:

$$\$4.0\text{M} - \$1.8\text{M} = \$2.2\text{M}.$$

Simple payback:

$$\frac{\$5.0\text{M}}{\$2.2\text{M per year}} \approx 2.3 \text{ years}.$$

Including the \$1.5M error-reduction benefit. Annual benefits become $\$4.0\text{M} + \$1.5\text{M} = \$5.5\text{M}$. Annual net benefit:

$$\$5.5\text{M} - \$1.8\text{M} = \$3.7\text{M}.$$

Simple payback:

$$\frac{\$5.0\text{M}}{\$3.7\text{M per year}} \approx 1.4 \text{ years}.$$

Discussion. The two calculations tell importantly different stories, and the gap between them is the chapter's argument in numbers. Counting only the visible processing saving, the programme pays back in about 2.3 years — a respectable but not commanding case, the kind that competes uneasily for funding and that a cautious committee might defer. Including the error-reduction benefit, payback falls to about 1.4 years — a clearly compelling case. The \$1.5M of avoided losses was not a minor adjustment; it shifted the decision. And note what the disciplined version did *not* do: it did not omit the governance and change costs to flatter the case. Those costs — \$1.0M a year combined — are real, and a business case that hid them would look better still and would be dishonest, and would tempt the institution to under-fund the validation and monitoring that keep the platform safe. The honest business case counts all four costs *and* all the benefits, including the invisible error-reduction benefit and, properly, the rising cost of the status quo this chapter's previous section described — a factor that, if included here, would strengthen the case further still. The discipline cuts both ways: it adds costs the optimistic case omits and adds benefits the pessimistic case omits, and only the case that does both is worth putting in front of a board.

Practical next steps

Take any hyperautomation business case in your institution — or build a quick one — and check it for the four costs and the four benefits this chapter names. Most first-draft business cases are missing the governance cost, the variable agentic run cost, the error-reduction benefit, or the rising-baseline comparison. Adding the missing terms rarely leaves the conclusion unchanged.

Chapter in brief

Hyperautomation economics are routinely mis-estimated in both directions. A platform has four costs — build, run (including variable agentic cost), governance, and change — and a case omitting any is wrong; governance and variable agentic cost are the ones most often missing. The benefits are the visible processing saving, the usually-larger and under-counted error and loss reduction, the speed and experience benefit, and capacity reallocation. The hardest and most important number is the rising cost of *not* modernising: the honest comparison is against that rising curve, not a flat do-nothing baseline. A business case worth presenting counts all four costs and all the benefits.

The Operating Model and the Road Ahead

129.1 The platform is never finished

A recurring theme of this manuscript, stated first in [Chapter 1](#), is that a hyperautomated process is not a project with an end date but a living artefact. This final chapter takes that theme seriously and asks what it implies: if the platform is never finished, what operating model sustains it, and where is the whole field heading?

A hyperautomation platform drifts the moment it goes live. The business changes its products and policies. Regulation changes its requirements. The models drift as the world moves away from their training data. The connected systems are themselves modernised and replaced. Customer behaviour shifts. A platform that is not actively sustained does not stay still — it decays, and its decay is mostly invisible until it produces an incident. The operating model is the answer to the question: who keeps it alive, and how.

129.2 Who owns what

A sustainable hyperautomation platform has clear, durable ownership, and the ownership structure follows the architecture.

Each *process* has a *process owner* from the business, accountable for what the process achieves and for the metrics that [Chapter 125](#)'s observability surfaces. Each *decision* — each DMN model — has an owner from the function whose policy it expresses: credit risk, underwriting, pricing, compliance. Each *predictive model* has an owner accountable for its performance and a model-risk function accountable for its governance, per [Chapter 126](#). Each *agent* has an owner accountable for its bounded task and its validation. The *platform* itself — the orchestration core, the cloud infrastructure, the integration fabric — has a platform engineering owner. And the whole forms a *capability* with overall ownership that can balance the parts.

This is more ownership roles than an institution typically expects, and that is the point. [Chapter 2](#) said ownership is a property of a model, decided when the model is created. The operating model is where that principle becomes an organisational reality: every component of the platform has a named owner, and an unowned component is a defect to be fixed, not a detail to be tolerated.

Figure [129.1](#) draws the ownership structure mapped onto the platform's components.

Tip

A practical test of operating-model health: pick any decision the platform made last week — a declined loan, an investigated claim — and ask who is accountable for each component that contributed to it: the process, each decision, each model, each agent. If every component has a name attached, the operating model is real. If the answer is

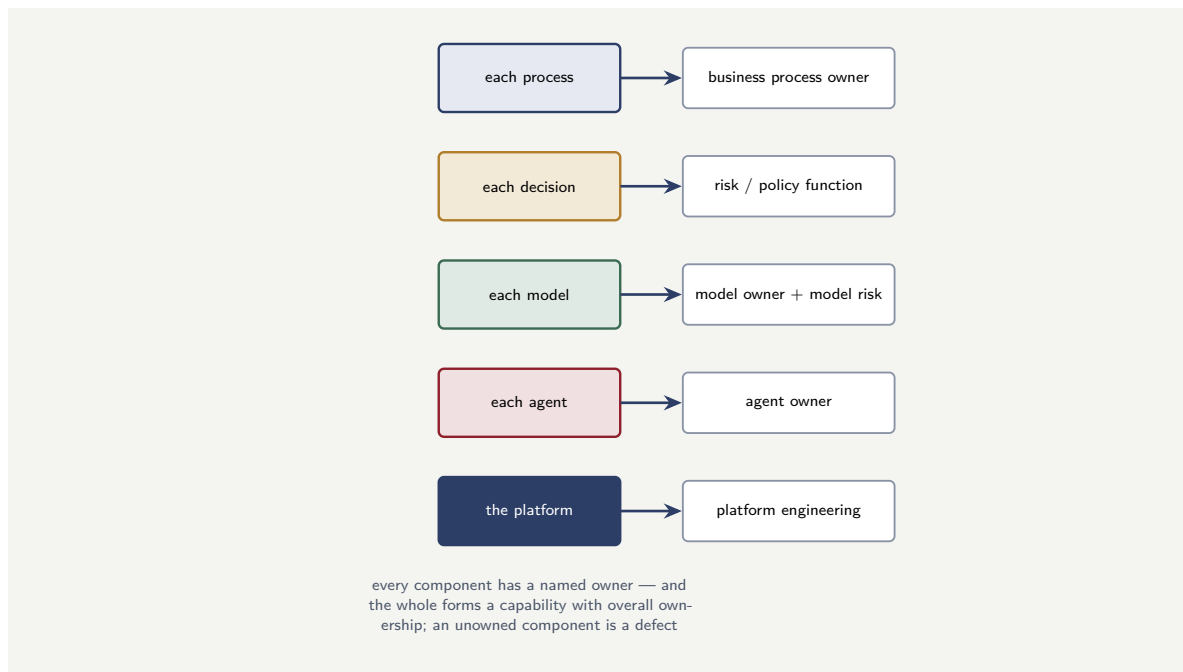


Figure 129.1. Ownership follows the architecture. Each process, decision, model, agent, and the platform itself has a named, durable owner from the function accountable for it. An unowned component is a defect to be fixed — and the whole forms a capability with overall ownership that can balance the parts.

vague for any component, that component is ungoverned, and ungoverned components are where incidents come from.

129.3 The team and its skills

The operating model needs people, and the skill set of a hyperautomation team is distinctive because the platform is distinctive.

It needs *process and domain expertise*: people who understand the banking or insurance processes deeply enough to model them well — the modelling discipline of Part I is a skill, not an afterthought. It needs *orchestration and integration engineering*: the platform skills of Part II. It needs *data science and model risk*: the people who build and the independent people who validate the predictive components. It needs *agentic engineering*: a genuinely new specialism concerned with designing, bounding, and validating agents, the subject of [Chapter 23](#). And it needs *risk, compliance, and conduct expertise* embedded in the team rather than consulted at the end — because, as Part XIV has argued throughout, governance designed in is cheap and governance retrofitted is expensive.

The important organisational point is that these skills work *together*, continuously, on a platform that is never finished — not in a sequence of hand-offs on a project that ends. The operating model is a standing capability.

129.4 How hyperautomation programmes fail

A manuscript that has spent twenty chapters describing how to do hyperautomation well owes the reader a frank account of how it goes wrong, because the failure modes are recognisable, recurring, and mostly avoidable once named. Five account for the great majority of troubled programmes.

The first is *automating before modelling* — the maturity-progression error of [Chapter 1](#). The institution adds bots, then models, then agents to a process it never made explicit, and ends with capability it cannot govern, observe, or explain. The symptom is a programme that delivers automations but cannot answer, for any individual case, how the decision was made.

The second is *the ungoverned agent* — the failure [Chapter 23](#) exists to prevent. Under delivery pressure, the deterministic adjudication step is dropped, or the agent is given a tool with external consequence, and an agent's confident, plausible errors begin reaching real effects. The symptom is often invisible until an incident, which is precisely what makes it dangerous.

The third is *the underfunded operating model* — the failure quantified in this chapter's worked example. The build is funded; the sustaining capability is not; the governance work is silently starved; and the platform decays while appearing healthy. The symptom is models drifting unvalidated and agents unmonitored, surfacing eventually as a regulatory finding.

The fourth is *the integration underestimate* — the failure [Chapter 25](#) warned of. The orchestration and the agents are treated as the hard part and the connection to the systems of record as a detail; the integration then consumes far more time and money than planned, and the programme stalls. The symptom is a working process model that cannot go live because it cannot reach the core systems clearly.

The fifth is *the wrong first process* — the phasing error of [Chapter 128](#). The programme begins with the largest, most entangled process, before the platform and the team exist to manage it; the first delivery becomes the hardest; and the programme loses support before it has proved itself. The symptom is a first phase that runs long, costs more than promised, and exhausts the organisation's patience.

What unites the five is that none is a technology failure. Each is a failure of sequence, of discipline, or of honest accounting — which is the manuscript's argument in its starkest form. The technologies work. The programmes that fail do so because they skipped the modelling, dropped the containment, underfunded the governance, underestimated the integration, or sequenced the work to take their hardest risk first.

None of the common ways a hyperautomation programme fails is a failure of the technology. Each — automating before modelling, the ungoverned agent, the underfunded operating model, the integration underestimate, the wrong first process — is a failure of discipline or of honest accounting. They are listed here so that they can be recognised early, while they are still cheap to correct, rather than diagnosed late from the incident they cause.

129.5 The road ahead

It is worth closing by looking forward, with appropriate caution about predicting a fast-moving field.

The clear direction of travel is that the *agentic layer will become more capable*. Agents will handle a wider range of unstructured tasks, more reliably, and the boundary of what can be

automated will move. This makes the manuscript's central discipline *more* important, not less: as agents grow more capable, the temptation to give them more scope and less containment will grow, and the institutions that succeed will be the ones that scale the governance — the containment, the deterministic adjudication, the human accountability — at the same pace as the capability.

A second direction is that *orchestration and intelligence will converge* in tooling: the modelling of processes, decisions, and agentic steps will sit in increasingly unified environments. The conceptual distinction this manuscript has drawn — between the deterministic skeleton and the probabilistic components within it — will remain essential even as the tools merge, because it is the distinction that governance rests on.

A third is that *regulation will continue to develop* specifically around automated decision-making and artificial intelligence in financial services. Institutions that have built explainability, fairness, and accountability in — as design requirements, the way Part XIV urged — will adapt to new regulation far more cheaply than those that treated governance as optional.

The thread through all three is the thread of the whole manuscript. Hyperautomation in banking and insurance is not, in the end, a technology problem. The technologies — orchestration, predictive models, agents, the cloud — are necessary, and assembling them well is real work. But what determines whether an institution succeeds is whether it has treated the work as the construction of a governed capability: explicit models, contained intelligence, deterministic adjudication, measured fairness, observable operation, and a named accountable human at every consequential decision. An institution that builds that can adopt each new capability as it arrives, safely. An institution that builds clever automation without the governance has built something it cannot trust, cannot explain, and cannot keep — and no amount of additional capability will fix that. The discipline is the deliverable.

Worked example: the steady-state effort to sustain a platform

Problem. An institution runs a hyperautomation platform with 12 process models, 30 DMN decision models, 8 predictive models, and 5 agents. From experience, sustaining each component costs, per year, roughly: 0.2 person-years per process model, 0.1 per decision model, 0.5 per predictive model (including validation and monitoring), and 0.8 per agent (including continuous validation). Platform engineering adds a fixed 4 person-years. The institution has budgeted 12 person-years a year to “maintain” the platform. Is that budget adequate, and what is the consequence if it is not?

Setup. We sum the per-component sustaining effort across all four component types, add the fixed platform-engineering effort, and compare the total with the 12-person-year budget.

Working. Process models: $12 \times 0.2 = 2.4$ person-years. Decision models: $30 \times 0.1 = 3.0$ person-years. Predictive models: $8 \times 0.5 = 4.0$ person-years. Agents: $5 \times 0.8 = 4.0$ person-years. Platform engineering: 4.0 person-years (fixed).

Total steady-state sustaining effort:

$$2.4 + 3.0 + 4.0 + 4.0 + 4.0 = 17.4 \text{ person-years.}$$

Against a budget of 12 person-years, there is a shortfall of

$$17.4 - 12 = 5.4 \text{ person-years.}$$

Discussion. The budget is short by 5.4 person-years — about a third of the true requirement — and the worked example exists to show what that shortfall actually does, because

it does not announce itself. A 31% under-funding does not stop the platform; the processes keep running, the decisions keep being made, the customers keep being served. What it does is silently starve the least visible work, and the least visible work is exactly the governance: the model revalidation of [Chapter 126](#), the monitoring of [Chapter 125](#), the fairness analysis of [Chapter 127](#), the continuous agent validation. Those are the activities that have no immediate customer waiting and no immediate deadline, so they are the activities that quietly do not happen when the team is 5.4 person-years short. The platform appears healthy and is in fact decaying — models drifting unvalidated, agents unmonitored — and the decay surfaces, eventually, as an incident or a regulatory finding. Notice, too, where the effort concentrates: the predictive models and agents together account for 8 of the 17.4 person-years, nearly half, although they are only 13 of the 55 components. The intelligent components are cheap to invoke and expensive to *sustain*, and an operating model — and a budget — that does not reflect that asymmetry is set up to under-govern exactly the components that most need governing. The disciplined institution sizes the sustaining budget bottom-up from the components it actually runs, funds the governance work explicitly so it cannot be the silent casualty of a shortfall, and treats the sustaining cost as a permanent cost of the capability — because, as this chapter began by saying, the platform is never finished.

Practical next steps

Inventory the components of your current or planned platform — processes, decisions, models, agents — and estimate the per-component annual sustaining effort, separating the governance portion explicitly. Compare the bottom-up total with whatever “maintenance” budget has been assumed. If there is a shortfall, identify which governance activities it will silently defund — and recognise that those are the activities whose absence becomes your next incident.

Chapter in brief

A hyperautomation platform is never finished; it drifts from the moment it goes live, and an operating model must sustain it. Every component — each process, decision, model, and agent — and the platform itself has a named, durable owner; an unowned component is a defect. The team is a standing, multi-skilled capability — process and domain, orchestration and integration, data science and model risk, agentic engineering, and embedded risk and conduct — working together continuously, not in project hand-offs. The road ahead — more capable agents, converging tooling, developing regulation — makes the manuscript’s discipline more important, not less. Hyperautomation is not a technology problem; the governed capability is the deliverable, and sustaining it is a permanent, honestly-budgeted cost.

PART XXIII

Camunda and Data

The Data Architecture of a Camunda Platform

130.1 A process platform is a data platform

Every preceding part of this manuscript has treated Camunda as a thing that *runs processes*. This part treats it as a thing that *produces data* — because every process instance, as it runs, generates a stream of data about what happened, and that data, used well, is one of the platform’s most valuable outputs. A bank that orchestrates its lending on Camunda is also, whether it realises it or not, generating a complete, structured, timestamped record of how every loan was decided — and that record is the raw material of process improvement, of regulatory evidence, of analytics.

This chapter sets out the data architecture: where a Camunda platform’s data lives, how it moves, and the structures it lives in. The chapters that follow treat what is done with it — process mining, the analytics of Optimize, and streaming the data onward through Kafka.

130.2 Two stores: runtime and history

The first and most important fact about a Camunda 8 platform’s data is that it lives in *two* distinct stores, with two distinct purposes, and confusing them is the root of several architectural mistakes.

The *primary store* is the engine’s own. [Chapter 27](#) described it: Zeebe’s replicated, append-only event log, holding the live execution state of every running instance. It is optimised for one thing — low-latency local access, so the engine can advance instances fast — and it is deliberately *not* a general-purpose database. It is not where an analyst queries, not where a report is run, not where history is browsed. It is the engine’s working memory, and it is kept lean precisely so the engine stays fast.

The *secondary store* is where the data goes to be *used*. It is a general-purpose, queryable data store — in current Camunda 8, Elasticsearch or OpenSearch — and it holds the running and historical process data in a form built for searching, visualising, and analysing. Operate’s views, Tasklist’s worklists, the query APIs of [Chapter 36](#) and [Chapter 38](#), the analytics of Optimize — all of these read the secondary store, not the engine’s primary store.

The separation is a deliberate *separation of concerns*, and it is the single idea this chapter most wants to fix. The primary store serves execution; the secondary store serves observation and analysis. Keeping them apart means that heavy analytical querying — a year’s claims pulled for a report — places its load on the secondary store and cannot slow the engine that is deciding live loans. An architecture that tried to query the engine’s primary store directly for analytics would be loading the execution path with reporting work, and [Chapter 27](#)’s lean, fast engine would not stay lean or fast.

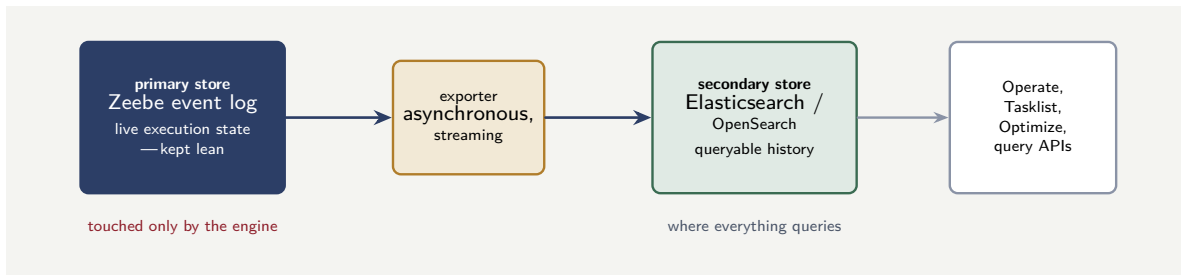


Figure 130.1. The two stores. The engine’s primary store — Zeebe’s event log — holds live execution state and is kept lean for fast execution; the exporter streams records to the secondary store, a queryable history that Operate, Tasklist, Optimize, and the query APIs all read. Analytics never touches the engine.

130.3 The exporter: the bridge between the stores

Data gets from the primary store to the secondary store by a component called the *exporter*. As the engine processes instances, it writes a stream of records — every state change, every token movement, every variable update — and the exporter consumes that stream and writes it into the secondary store.

The exporter is worth understanding as an architectural element, not just a configuration. It is the *bridge*, and it has three properties that matter. It is *asynchronous* — the engine does not wait for the exporter; it processes instances at its own pace and the exporter follows — so exporting never slows execution. It is *streaming* — it processes the record stream continuously, so the secondary store is kept close to current, not refreshed in batches. And it is, in a sense, the *seam* at which the platform’s data becomes available to everything outside the engine: Operate, Tasklist, Optimize, and — as a later section shows — any other data sink the institution chooses to add.

A platform engineer configures the exporter, and two configuration concerns recur. The first is *what* is exported: not every record need go to the secondary store, and filtering the export — by record type, by process, even by variable — keeps the secondary store’s volume manageable. The second is *retention*: the secondary store would grow without bound if nothing removed old data, so a retention policy deletes data past a configured age, and that age is a real decision balancing the cost of storage against the period over which history must remain queryable.

Tip

Hold the two-store model clearly in mind: the engine’s primary store serves *execution* and is kept lean; the secondary store serves *observation and analysis* and is where everything queries. Never reason about analytics, reporting, or history as if they touched the engine — they touch the secondary store, fed by the exporter. An architecture that understands this separation can run heavy analytics without ever threatening the engine’s execution speed.

Figure 130.1 draws the two-store model and the exporter bridge between them.

130.4 The shape of the data

The data in the secondary store has a shape worth knowing, because process mining and analytics both work on it directly.

At its core, the data is a set of *event records*: each is a timestamped fact about a process instance — this instance started, this token entered this activity, this activity completed, this variable took this value, this user task was assigned to this person, this incident was raised. The records are organised, in the secondary store, into indices by the kind of thing they describe — process instances, user tasks, variables, incidents, and so on.

Two derived structures matter especially. The *process instance* record is the spine: for each instance, when it started, which definition and version it ran, its current or final state, when it ended. And the *flow-node* records — one per activity an instance visited — are, taken together, the *event log* that process mining consumes: every instance, every step it took, every step's timestamp. That event log is not a by-product the platform incidentally keeps; it is, as the next chapter shows, a complete and faithful record of how the institution's processes actually ran, and it is extracted from the secondary store, not reconstructed.

130.5 Data as a platform capability

The data the platform produces is not a by-product; it is a *capability*. Every process instance generates a stream of facts: what happened, when, to whom, with what outcome. That stream, properly captured and governed, feeds operational dashboards, management reporting, regulatory submissions, and the machine-learning models that improve the processes themselves. A platform that runs processes but does not capture their data in a governed, queryable form has built half a platform: it automates the work but not the institution's ability to learn from it.

Worked example: the cost of querying the wrong store

Problem. An analytics team needs a daily report over the previous day's completed loan applications — on a busy day, 30,000 instances, each with around 40 variables. They have two possible designs. In design A, the report queries the engine's primary store directly. In design B, it queries the secondary store. The engine's primary store is sized for execution and can serve the report's heavy scan only by diverting resources from instance processing; benchmarking shows design A's daily report would, while it runs, reduce the engine's instance-processing throughput by 35% for its 20-minute duration. The platform processes, on average, 50 loan-decision instances per second. Quantify the execution work design A displaces, and state the architectural lesson.

Setup. The displaced execution work is the throughput reduction, applied to the platform's instance rate, over the report's duration. We compute it, and contrast with design B.

Working. The report runs for 20 minutes — 1,200 seconds. During that time, design A reduces the engine's throughput by 35%. At a normal 50 instances per second, the lost capacity is:

$$50 \times 0.35 = 17.5 \text{ instances per second of lost throughput.}$$

Over the 1,200-second report:

$$17.5 \times 1,200 = 21,000 \text{ instances' worth of processing capacity displaced.}$$

Under design B, the report queries the secondary store, which exists for exactly this; the en-

engine's throughput is unaffected, and the displaced capacity is 0.

Discussion. Design A displaces 21,000 instances' worth of the engine's processing capacity every day — and on a busy morning, that displacement is not abstract: it is real loan applications processed more slowly, real customers waiting longer, because a *report* was competing with the *engine* for the same store. The worked example is built to make one architectural point unmissable. The two-store model is not an implementation detail to be aware of; it is a separation that exists precisely so that the heavy, scanning, analytical workload and the fast, latency-sensitive execution workload never compete — and design A throws that separation away. The instinct behind design A is usually “the data is right there in the engine, why add a hop?” — and the answer is that the hop, the exporter writing to the secondary store, is what *buys* the isolation. The secondary store is not a redundant copy; it is the analytical workload's own home, sized and tuned for scanning, where a 20-minute report is just a 20-minute report and not a 35% tax on the bank's lending throughput. The rule is absolute: analytics, reporting, mining, and history query the secondary store, always; the engine's primary store is touched only by the engine.

Practical next steps

Find out, for one analytical query or report your institution runs against its Camunda platform, which store it actually queries. If anything reports against the engine's primary store directly, it is taxing execution to serve reporting — and the fix is to point it at the secondary store, which exists for exactly that purpose.

Chapter in brief

A Camunda platform produces data as well as running processes, and that data lives in two stores. The engine's *primary store* — Zeebe's event log — holds live execution state and is kept lean for low-latency execution. The *secondary store* — Elasticsearch or OpenSearch — holds running and historical data in queryable form, and is what Operate, Tasklist, the query APIs, and Optimize all read. The *exporter* is the asynchronous, streaming bridge between them, configured for what it exports and for data retention. The secondary store's event records — process instances and the flow-node steps they visited — are the event log process mining consumes. Analytics always queries the secondary store; the engine's primary store is touched only by the engine.

Process Mining

131.1 Seeing how processes actually run

A process model is a statement of how a process is *meant* to run. The event log of the previous chapter is a record of how it *actually* ran. *Process mining* is the discipline that works on the second to check it against the first — and the gap between intention and reality is, very often, where the most valuable insight in a hyperautomation programme lies.

The premise of process mining is simple and powerful. Every process instance leaves, in the secondary store, a trace: the sequence of activities it visited, each with a timestamp. Thousands or millions of those traces, taken together, are a complete factual record of how a process behaves in aggregate — which paths are common and which rare, where instances wait, where they loop, how long each step really takes, how the reality differs from the model on the wall. Process mining is the set of techniques that turns that mass of traces into that picture.

131.2 What process mining reveals

Process mining answers questions that a process model alone cannot, because the model says what *should* happen and mining shows what *does*.

It reveals the *real process*, as flown. *Process discovery* — mining's most striking technique — takes the event log and reconstructs, from the traces alone, the actual process model: the paths instances really took, in the proportions they took them. Set beside the designed model, the discovered model shows where reality has diverged — a path everyone forgot the process could take, a loop no one modelled, a step that is, in practice, always skipped.

It reveals the *bottlenecks*. By aggregating the timestamps, mining shows where instances spend their time — and almost always, the time is not where the team assumed. A step the team worried about turns out to be fast; a step no one thought about turns out to be where every instance waits two days.

It reveals *conformance*. *Conformance checking* compares the event log against the intended model and quantifies the deviation: how many instances took an unauthorised path, skipped a required control, ran steps out of the modelled order. For a regulated institution this is not idle analytics — it is evidence of whether the process, as actually run, conformed to the process as governed.

It reveals *variance*. Two instances of the same process can have very different histories, and mining shows the spread — the fast cases and the slow, the simple paths and the tangled — so that improvement effort goes to the variance that matters.

Figure 131.1 draws process mining as the transformation it is: event log in, four kinds of insight out.

131.3 Process mining and the hyperautomation cycle

Process mining has a particular place in a hyperautomation programme, and it is worth naming because it closes a loop the rest of the manuscript opened.

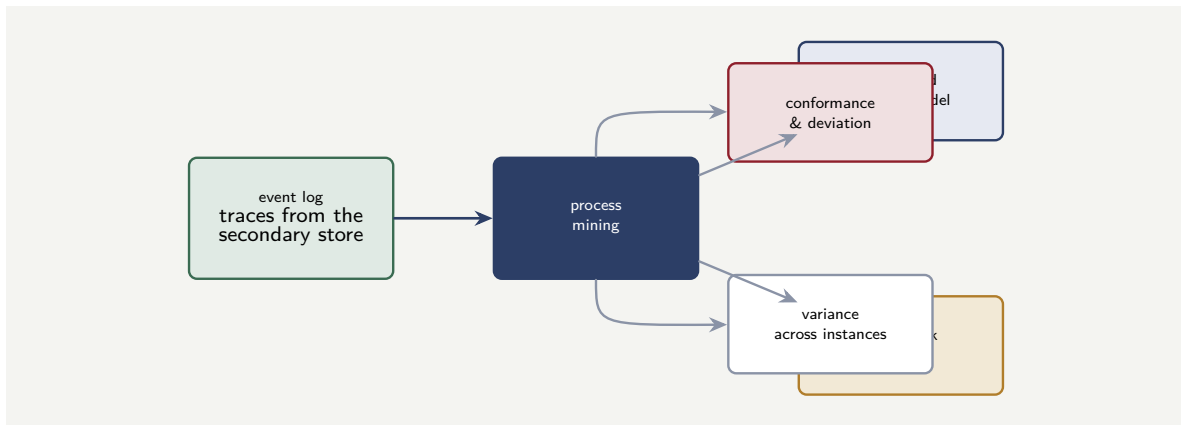


Figure 131.1. Process mining. The event log — the traces of how instances actually ran, drawn from the secondary store — is the input. Mining turns it into a discovered model of the real process, a bottleneck analysis, a conformance check against the intended model, and a picture of variance across instances.

Chapter 2 argued that a process must be modelled before it can be automated. Process mining adds the converse: once a process is running and automated, mining shows how it is *really* behaving, and that picture is the input to the next round of improvement. The cycle is: model the process, automate it, mine how it actually runs, find where reality diverges from intent or where the time is really lost, improve the model, and automate the improvement. Process mining is the *measurement* step of that cycle — without it, a hyperautomation programme automates, then guesses at how well it worked; with it, the programme automates, then *sees*.

This connects directly to the observability of Chapter 125. Where that chapter’s observability is largely about the *operational* present — what is running, what is stuck, now — process mining is about the *analytical* aggregate — how the process behaves across thousands of instances over time. The two are complementary: observability runs the platform day to day; mining improves it cycle over cycle.

131.4 Data as a platform capability

The data the platform produces is not a by-product; it is a *capability*. Every process instance generates a stream of facts: what happened, when, to whom, with what outcome. That stream, properly captured and governed, feeds operational dashboards, management reporting, regulatory submissions, and the machine-learning models that improve the processes themselves. A platform that runs processes but does not capture their data in a governed, queryable form has built half a platform: it automates the work but not the institution’s ability to learn from it.

Worked example: the bottleneck that was not where anyone looked

Problem. A lending operation believes its slow step is credit assessment, and is about to spend on automating it further. Before doing so, the team mines six months of event log — 280,000 completed loan applications. The mining shows the average end-to-end time is

4.2 days, and breaks it down by step. Credit assessment, the suspected culprit, averages 0.3 days. The largest single contributor turns out to be a wait between *document submitted* and *document reviewed* — averaging 2.6 days. The proposed credit-assessment automation would have cost \$400,000 and, even if it halved that step, would have saved 0.15 days per application. Addressing the document-review wait — by reallocating reviewer capacity — is estimated to cut that wait to 0.6 days. Compare what mining-led and assumption-led improvement deliver.

Setup. For each option we compute the reduction in average end-to-end time. The assumption-led option halves the 0.3-day credit step; the mining-led option cuts the 2.6-day document wait to 0.6 days.

Working. *Assumption-led — automate credit assessment.* Reduction in average end-to-end time:

$$0.3 \times 0.5 = 0.15 \text{ days saved per application,}$$

for an outlay of \$400,000.

Mining-led — address the document-review wait. Reduction:

$$2.6 - 0.6 = 2.0 \text{ days saved per application.}$$

The mining-led improvement delivers

$$\frac{2.0}{0.15} \approx 13 \text{ times}$$

the reduction in end-to-end time — and by reallocating existing capacity rather than building automation, at a fraction of the \$400,000.

Discussion. The mining-led improvement is roughly thirteen times more effective than the one the team was about to fund on the strength of an assumption — and the worked example is built to show that this is the *normal* result of mining, not a lucky one. Teams' beliefs about where their processes are slow are formed from anecdote, from the steps that *feel* effortful, from whichever step generated the last complaint — and those beliefs are reliably wrong, because the human steps that feel effortful are rarely the same as the waits where instances actually sit. Credit assessment *felt* like the bottleneck because it is visibly complex work; the document-review wait was invisible precisely because waiting is invisible — no one is doing anything during it, so no one notices it. Mining makes the invisible wait visible, because the event log's timestamps do not have opinions: they simply record that instances sat, on average, 2.6 days between two events. The lesson is the one this part most wants to land: a hyperautomation programme that improves its processes from assumption spends its money where the work *looks* hard, and a programme that improves from mining spends it where the time *actually* goes — and the gap between those two, as the thirteen-fold figure shows, is the difference between an expensive disappointment and a cheap transformation. Mine before you spend.

Practical next steps

Before the next process-improvement investment your institution is considering, mine the event log of the process in question and find where the end-to-end time actually goes. Compare that with where the team *assumes* it goes. If the two agree, the investment is well aimed; if they disagree — and they usually do — the mining has just redirected the spend to where it will actually help.

Chapter in brief

A process model says how a process should run; the event log records how it did. Process mining works on the event log — the traces of how instances actually ran, from the secondary store — to reveal what the model alone cannot. Process discovery reconstructs the real process from the traces; bottleneck analysis shows where the time actually goes; conformance checking quantifies deviation from the intended model; variance analysis shows the spread across instances. Mining is the measurement step of the hyperautomation cycle — model, automate, mine, improve — and it reliably shows that the real bottleneck is not where teams assume. Mine before you spend.

Camunda Optimize

132.1 From raw event log to managed insight

The previous chapter treated process mining as a discipline. This chapter treats the Camunda component built to deliver much of that discipline as a managed, ongoing capability: *Optimize*. [Chapter 28](#) introduced Optimize among the components; this chapter gives it the depth its place in the data part warrants.

The distinction between raw process mining and Optimize is the distinction between a technique and a product. Process mining, in principle, can be done with general analytical tools against the event log. Optimize is the component that does it *for* a Camunda platform, continuously, with the process awareness built in: it knows what a process instance is, what a flow node is, what an SLA is, and it presents process analytics in those terms rather than as generic data.

132.2 What Optimize is for

Optimize serves the people who own and improve processes — the process owners of [Chapter 2](#), the operations managers, the analysts — with three kinds of capability.

It provides *reports*: configurable analyses of a process's behaviour — how long instances take, how many took each path, how often an SLA was met, how a duration trends over weeks. A report is the answer to a specific question about a process, kept current as new data arrives.

It provides *dashboards*: collections of reports, arranged for a role. A claims operations manager's dashboard might show, together, the current throughput, the SLA-attainment rate, the bottleneck step, and the trend in average handling time — the process's health, at a glance, continuously updated.

It provides *alerts*: a report can have a threshold, and Optimize can notify someone when the threshold is crossed — when average cycle time rises past a limit, when SLA attainment falls below a target. This is the analytical counterpart of the operational alerting of [Chapter 125](#): not “an instance is stuck now” but “this process's performance has drifted past where it should be.”

Underlying all three, Optimize draws on the same event log of [the data architecture chapter](#) — it reads the secondary store — so everything it shows is grounded in how the processes actually ran, not in how they were meant to.

Figure [132.1](#) draws Optimize's three capabilities, all grounded in the event log.

132.3 Optimize and decision analysis

One capability of Optimize deserves singling out, because it connects to a thread running through the whole manuscript: Optimize can analyse not only processes but *decisions*.

[Chapter 4](#) made the DMN decision a first-class, governed artefact, and [Chapter 125](#) asked for *decision observability* — watching the distribution of decision outcomes over time. Optimize is where that is delivered analytically. It can report on a DMN decision's behaviour: how

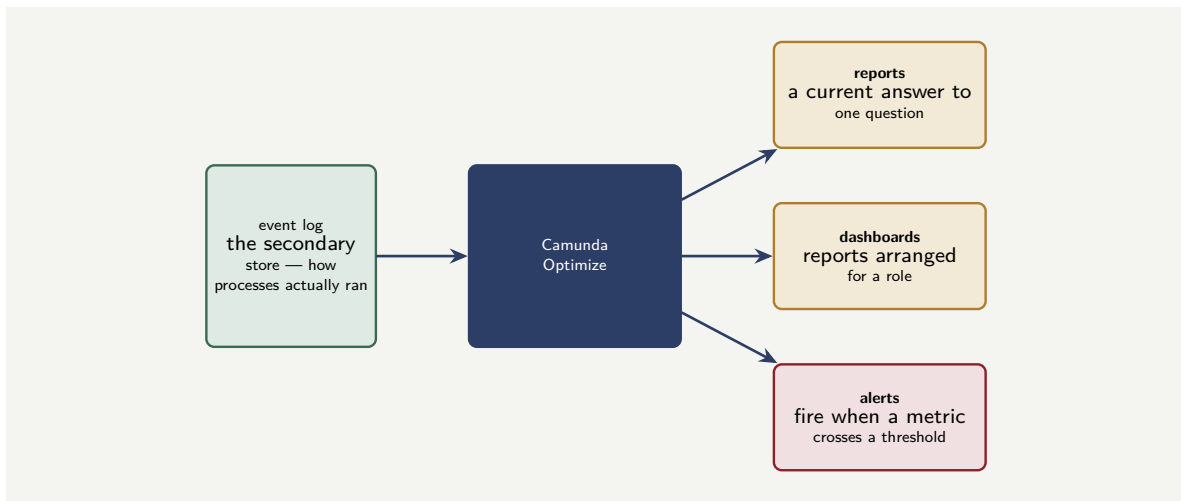


Figure 132.1. Camunda Optimize. Reading the same event log as process mining, Optimize delivers three managed, continuous capabilities: reports that answer a specific question and stay current, dashboards that arrange reports for a role, and alerts that fire when a metric crosses a threshold.

often each outcome was reached, how the mix of outcomes is trending, whether the rate of one outcome is drifting. A lending decision that begins approving a steadily rising fraction of applications, or a fraud decision whose flag rate is creeping, is exactly the kind of drift [Chapter 125](#) and [Chapter 127](#) warned must be watched — and Optimize’s decision analysis is the instrument that watches it.

Optimize, like everything that reads the secondary store, sees only what the exporter exported. If the export was filtered — variables excluded, processes omitted — to control the secondary store’s size, then those variables and processes are not available to Optimize’s analysis, however much an analyst later wishes they were. The export filter and the analytical need must be decided together: an export tuned only for storage economy can quietly make a later, important analysis impossible.

132.4 The cost of Optimize, honestly

A platform engineer should know that Optimize is not free, and the manuscript is direct about it. Optimize reads the process data from the secondary store and writes its own analytical indices back, and that is real additional load on the secondary-store infrastructure — enabling Optimize measurably reduces the throughput the same hardware can sustain for the core platform.

This is not a reason to forgo Optimize; the analytical capability is worth a great deal. It is a reason to *plan* for it: to size the secondary-store infrastructure for the analytical load as well as the operational one, to consider running Optimize’s analytics against isolated infrastructure so its load does not compete with the core platform, and to use the export filtering of [the data architecture chapter](#) to keep Optimize’s data volume proportionate to what the analysis genuinely needs. Optimize is a capability with a cost, and a platform that switches it on without planning the cost will find its core throughput quietly diminished.

132.5 Data as a platform capability

The data the platform produces is not a by-product; it is a *capability*. Every process instance generates a stream of facts: what happened, when, to whom, with what outcome. That stream, properly captured and governed, feeds operational dashboards, management reporting, regulatory submissions, and the machine-learning models that improve the processes themselves. A platform that runs processes but does not capture their data in a governed, queryable form has built half a platform: it automates the work but not the institution's ability to learn from it.

Worked example: the report that watched a decision drift

Problem. A bank's lending platform uses a DMN decision for automatic approval of small loans. When the decision was approved by the model-risk function, its approval rate was 62%. The bank sets up, in Optimize, a report on the decision's approval rate, with an alert if the rate moves outside a band of 57% to 67% — five points either side of the approved baseline. Eight months later, the alert fires: the approval rate has reached 68%. An investigation finds that an upstream change in how an input variable was populated had gradually shifted the decision's behaviour. Without the Optimize report, the drift would have continued undetected. If the approval rate had reached 80% before a person happened to notice, on a monthly volume of 40,000 small-loan decisions, how many additional loans per month would have been approved beyond the intended-baseline rate, and what is the lesson?

Setup. The excess approvals at the undetected 80% rate, versus the 62% baseline, is the rate difference applied to the monthly volume. We compute it, and contrast with what the alert actually caught.

Working. Excess approval rate, had drift reached 80% undetected:

$$80\% - 62\% = 18 \text{ percentage points.}$$

On 40,000 small-loan decisions a month:

$$40,000 \times 0.18 = 7,200 \text{ additional loans approved per month}$$

beyond what the model-risk-approved decision was intended to approve.

By contrast, the Optimize alert fired at 68% — 6 points above baseline — catching the drift when the monthly excess was:

$$40,000 \times 0.06 = 2,400 \text{ loans,}$$

and, more to the point, catching it at all, in month eight, rather than letting it run.

Discussion. Without the Optimize report, a decision approved to run at 62% could have drifted to 80% and approved 7,200 extra loans a month — a substantial, unintended expansion of the bank's lending risk, produced not by any deliberate decision but by an upstream change quietly shifting an input. The worked example's lesson is about the particular danger of *drift*. The decision did not fail; no incident was raised; every individual loan approval was the decision faithfully executing its rules. The decision's *rules* never changed — what changed was the population of inputs flowing into them, and so the decision's *behaviour in aggregate* drifted while every individual execution remained correct. That is exactly the kind of problem that is invisible to operational monitoring — there is no stuck instance, no error — and visible only to the analytical, aggregate watching that Optimize provides. The report

on the decision's approval rate, with its alert band, was a deliberate instrument pointed at exactly this risk, and it did its job: it caught a drift, in month eight, that no operational alert and no human spot-check would have caught. The lesson connects the data part back to [Chapter 127](#)'s model risk: a decision's correctness is not established once, at approval; it must be *watched*, because the world feeding the decision changes even when the decision does not — and Optimize's decision reports are how the watching is actually done.

Practical next steps

For one consequential DMN decision your platform runs, set up an Optimize report on its outcome rates, with an alert band around the rates the decision was approved to produce. The report costs little to build, and it is the instrument that catches a decision drifting — silently, with no error, no incident — away from the behaviour its model-risk approval was based on.

Chapter in brief

Optimize is the Camunda component that delivers process analytics as a managed, continuous capability, reading the same secondary-store event log as process mining but with process awareness built in. It provides reports — current answers to specific questions about a process — dashboards that arrange reports for a role, and alerts that fire when a metric crosses a threshold. It analyses decisions as well as processes, which is how the decision observability of [Chapter 125](#) is delivered: watching a DMN decision's outcome rates drift. Optimize sees only what the exporter exported, and it places real additional load on the secondary store — a capability with a cost that must be planned, not a free switch.

Streaming Process Data with Kafka

133.1 When the platform's data must go further

Operate, Tasklist, and Optimize all consume the Camunda platform's data within the platform's own boundary. But an institution's appetite for that data rarely stops at the platform's edge. The enterprise data warehouse wants it. The fraud-analytics team wants the event stream in real time. A customer-notification service wants to know the instant a claim is settled. A regulatory-reporting system wants a feed of decisions. The process data must go *further* — out of the Camunda platform and into the institution's wider data landscape — and this chapter treats how, with *Apache Kafka* as the worked example.

Kafka is the natural vehicle because it is, in most large institutions, already the backbone of exactly this kind of data movement: a distributed, durable, high-throughput event-streaming platform that many systems publish to and subscribe from. The question this chapter answers is how a Camunda platform's process data gets onto that backbone, and the disciplines that keep the result clean.

133.2 Two ways data reaches Kafka

There are two architecturally distinct ways process data flows from Camunda to Kafka, and they serve different needs.

The first is the *custom exporter*. The [data architecture chapter](#) described the exporter as the bridge from the engine's primary store to the secondary store — and noted that the exporter mechanism is extensible. An institution can write a *custom exporter* that consumes the same engine record stream and publishes it to Kafka. This gives Kafka the *complete, low-level record stream* — every state change, every token movement — as it happens. It is the right choice when a downstream system needs the full firehose of platform events with minimal latency.

The second is the *deliberate business event*, published from within the process. [Chapter 34](#) described a process emitting its significant business events — “loan disbursed,” “claim settled” — and [Chapter 33](#)'s connectors are how. A process can have a service task, or a message-throw event, whose job is to publish a well-named business event to a Kafka topic. This gives Kafka a *curated stream of meaningful business facts* — not every low-level record, but the events the institution has decided matter.

The distinction is important and is a design decision. The custom exporter gives Kafka *everything, raw*; the in-process business event gives Kafka *the significant facts, curated*. A fraud-analytics system that needs to see every variable change wants the exporter feed; a customer-notification service that needs to know when a claim is settled wants the curated business event. Many institutions run both, for their different consumers.

Figure [133.1](#) draws both paths onto the Kafka backbone.

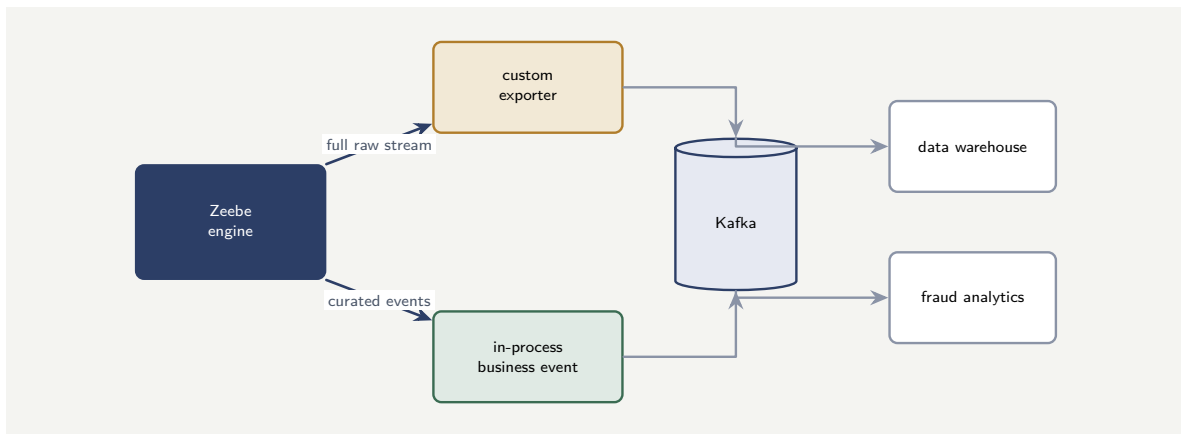


Figure 133.1. Two ways process data reaches Kafka. A custom exporter publishes the complete, raw engine record stream; an in-process business event publishes a curated stream of significant facts. Both land on the Kafka backbone, from which the institution’s wider systems consume.

133.3 The disciplines of streaming process data

Streaming process data onto an enterprise backbone carries disciplines, and three matter most.

The first is *the event is a contract*. Once a Camunda platform publishes a “claim-settled” event to a Kafka topic, other systems — systems the Camunda team may not even know about — consume it and depend on its shape. The event’s schema is therefore a published contract, versioned and evolved with the same care [Chapter 25](#)’s integration contracts demanded. A process team that changes the shape of a streamed event casually has broken, silently, every consumer of it.

The second is *data protection crosses the boundary too*. The process data streamed to Kafka carries customer information, and once it is on the enterprise backbone it is far more widely reachable than it was inside the Camunda platform. The minimisation discipline of Part IV’s security chapter applies with full force at this boundary: the streamed events should carry the data their consumers genuinely need and not the entire process state, and sensitive fields may need masking or omission before they are published. A careless stream is a data-protection breach with a very wide blast radius.

The third is *the stream does not replace the engine’s truth*. Kafka carries a *copy* of the process data for other systems to use; the engine remains the authoritative source of the process’s actual state. A downstream system must not be architected to believe that, because it saw an event on Kafka, it may treat the process as being in a certain state and act authoritatively on that — the event is a notification, and anything that needs the authoritative state must ask the engine, through the APIs of [Chapter 36](#). The stream informs; the engine remains the truth.

A business event streamed to Kafka is a published contract: unknown downstream systems depend on its shape, so it must be schema-versioned like any integration contract. It carries customer data onto a widely-reachable backbone, so data minimisation and masking apply at the streaming boundary with full force. And it is a *copy* — a notification — not the authoritative state: a downstream system that needs the truth of a process’s state must ask the engine, not infer it from a Kafka event. Treating a streamed event as a private message, as unprotected data, or as authoritative truth are three dis-

tinct and serious mistakes.

133.4 Data as a platform capability

The data the platform produces is not a by-product; it is a *capability*. Every process instance generates a stream of facts: what happened, when, to whom, with what outcome. That stream, properly captured and governed, feeds operational dashboards, management reporting, regulatory submissions, and the machine-learning models that improve the processes themselves. A platform that runs processes but does not capture their data in a governed, queryable form has built half a platform: it automates the work but not the institution's ability to learn from it.

Worked example: the curated event versus the raw fire-hose

Problem. A bank wants its customer-notification service to send a message the moment a claim is settled. Two designs are proposed. In design A, the notification service subscribes to the *raw exporter stream* on Kafka — every engine record from every process — and filters, in its own code, for the records that indicate a claim settlement. In design B, the claims process publishes a single curated *claim-settled* business event to a dedicated Kafka topic, and the notification service subscribes to just that topic. The platform produces, across all processes, about 9,000 engine records per second; claim settlements occur about 4 times per second. Compare what each design's notification service must process, and identify the better design.

Setup. We compare the volume of Kafka messages the notification service must consume and inspect under each design, and the proportion of that volume that is actually relevant.

Working. *Design A — subscribe to the raw firehose.* The notification service must consume and inspect every engine record to find the settlements:

9,000 records per second consumed,

of which the relevant ones are

4 per second,

so the fraction of consumed volume that is relevant is

$$\frac{4}{9,000} \approx 0.044\%.$$

The service does 99.96% of its consumption work on records it discards.

Design B — subscribe to the curated event. The notification service consumes only the *claim-settled* topic:

4 messages per second,

every one of them relevant — 100% useful consumption.

Discussion. Design A makes the notification service consume 9,000 messages a second to act on 4 of them; design B makes it consume exactly the 4. The 2,250-fold difference in consumption volume is the visible result, but the worked example's lesson is architectural. Design A is the *wrong tool for the need*: the raw exporter stream exists for consumers that genuinely want the full low-level firehose — a fraud-analytics system modelling every variable change — and pointing a simple notification service at it forces that service to take on the

entire platform's event volume, to re-implement the knowledge of which low-level records mean "settlement," and to scale with the whole platform's throughput rather than with its own modest need. Design B matches the tool to the need: the claims process, which knows exactly when a settlement has occurred, publishes one clear business event saying so, and the notification service consumes a stream that is small, entirely relevant, and stable. The two-path model of this chapter is not two equivalent options — it is two tools for two different needs, and the design skill is matching them. A consumer that wants *a specific business fact* should receive a *curated business event*; only a consumer that genuinely needs *everything* should drink from the raw exporter firehose. Design A's error — a small, specific need wired to the raw stream — is among the most common mistakes in streaming process data, and it produces a fragile, oversized consumer where a simple one would have done.

Practical next steps

For each system in your institution that consumes Camunda process data from Kafka, ask whether it wants *a specific business fact* or *the whole event stream*. Any system with a specific need that is subscribed to the raw exporter firehose is carrying the whole platform's volume to serve a small purpose — and it should instead consume a curated business event the process publishes deliberately.

Chapter in brief

An institution's appetite for process data rarely stops at the platform's edge; Kafka is the usual vehicle for streaming it into the wider data landscape. Data reaches Kafka two ways: a custom exporter publishes the complete, raw engine record stream for consumers that need everything; an in-process business event publishes a curated stream of significant facts for consumers that want specific events. Matching the path to the consumer's need is the design skill — a specific need wired to the raw firehose is a common and costly mistake. Three disciplines govern streaming: a streamed event is a versioned contract, data minimisation applies with full force at the boundary, and the stream is a notification copy — the engine remains the authoritative truth.

PART XXIV

Large Language Models and RAG with Camunda

Large Language Models as a Process Capability

134.1 The model as a step, not a centre

The manuscript has referred to large language models throughout — the agentic layer of [Chapter 23](#), the agent connector of [Chapter 33](#) — but always glancingly, as one capability among many. This part treats them directly. It is, deliberately, a part and not a single chapter, because using large language models well within a Camunda platform has enough depth — the model itself, the retrieval that grounds it, and the wider agentic patterns — to need three chapters. This first chapter treats the large language model as what, in a governed platform, it must be: a *step* in a process, not the centre of one.

That framing is the whole argument, and it is worth stating sharply before any detail. A large language model is a powerful, general, probabilistic component: give it text, it produces plausible text. The temptation, when such a component arrives, is to build the system *around* it — to make the model the thing that decides, and the process merely its errand-runner. This manuscript's position, argued from [Chapter 23](#) onward and made concrete here, is the reverse: the process is the centre — explicit, governed, auditable — and the model is a contained step within it, used for the specific kinds of work it is genuinely good at and held away from the kinds it is not.

134.2 What a language model is good at, and what it is not

Using a language model well as a process step begins with an honest account of its capabilities, because the failures of model-in-process designs almost always trace to using the model for something it cannot reliably do.

A language model is genuinely good at a recognisable family of tasks. It is good at *transforming unstructured text* — summarising a long document, extracting structured fields from a free-text claim description, classifying a customer message by intent, rephrasing dense policy language into plain terms. It is good at *drafting* — producing a first version of a letter, a summary, an explanation, for a human to review. It is good at *interpreting language* — understanding what a customer's free-text complaint is actually about. These are the tasks where a language model, used as a process step, adds real value that deterministic code could not.

A language model is *not* reliable at a different family of tasks, and a governed platform must not use it for them. It is not reliable as a *calculator* — it does not compute, it produces plausible-looking numbers. It is not reliable as a *source of fact* — asked a question about the institution's policy or a customer's account, it will produce a fluent answer that may be wholly invented. It is not reliable as a *decision-maker* of record — its outputs are probabilistic, not rule-grounded, and a regulated decision must be grounded in rules. And it is not *deterministic* — the same input may yield different outputs, which for an auditable process is a serious property to manage.

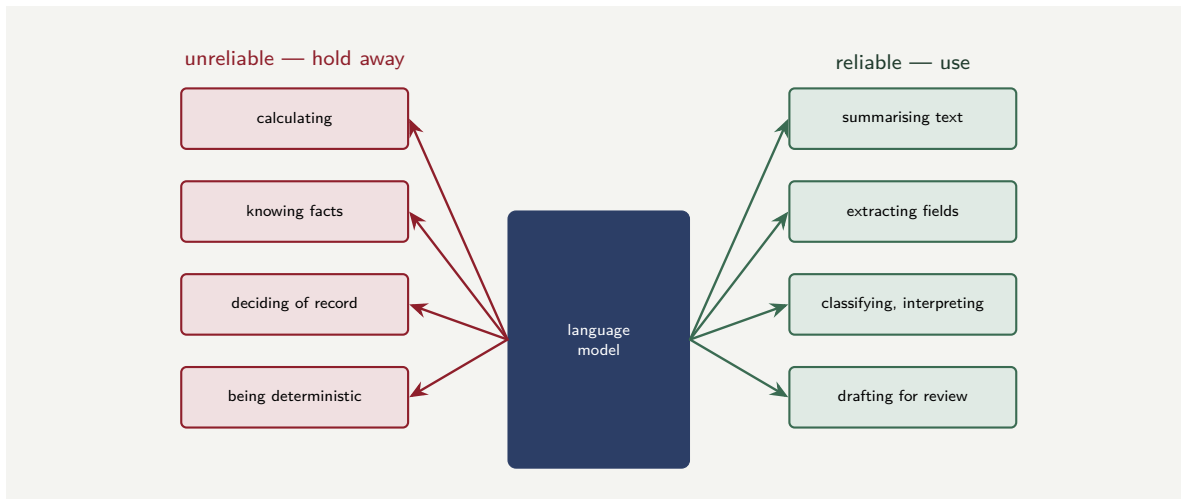


Figure 134.1. What a language model is reliable for, and what it is not. It is genuinely good at transforming, interpreting, and drafting text; it is unreliable as a calculator, a source of fact, a decision-maker of record, or a deterministic component. The discipline is to match the model to the task family it is reliable for.

The discipline that follows is simple: use the model for the transformation, interpretation, and drafting it is good at, and never for the calculation, fact-retrieval, or decision-of-record it is not. The next chapter’s retrieval addresses the fact problem; the deterministic-adjudication discipline of [Chapter 23](#) addresses the decision problem.

Figure 134.1 draws the capability boundary — the task families on each side of it.

A large language model does not compute, does not know facts, and does not decide — it produces plausible text. Used as a process step for summarising, extracting, classifying, or drafting, it is genuinely valuable. Used as a calculator, a source of fact about the institution or a customer, or the decision-maker of record, it will produce fluent, confident, and wrong output — and “fluent and confident” is exactly what makes the error dangerous in a financial process. Match the model to the task family it is reliable for, and hold it away from the rest.

134.3 The model as a contained step in Camunda

Concretely, a language model becomes a process step through the connector mechanism of [Chapter 33](#): an outbound connector to a model-serving endpoint — the institution’s own hosted model, or a model service reached over the network. The service task that uses it is, in BPMN terms, an ordinary service task; what makes it a *contained* step is the discipline around it, and that discipline is [Chapter 23](#)’s four containments, applied here specifically to a language model.

The task is *bounded*: the model is asked to do one defined thing — “extract these six fields from this claim description,” “classify this message into one of these five intents” — not given an open-ended brief. The *output is structured*: the model is asked to return its result in a defined shape — a JSON object with named fields — which the process can then validate, rather than free prose the process must then parse hopefully. The output is *adjudicated*: a deterministic step after the model checks the model’s output before the process acts on it

— the extracted fields are validated, the classification is checked against the permitted set, and an output that fails the check is routed to a human rather than used. And a *human is accountable* for any consequential outcome the model’s work feeds into.

A language model used this way — bounded task, structured output, deterministic adjudication, accountable human — is a contained, governed process step. A language model wired in without that discipline — handed an open brief, returning free text the process trusts and acts on — is the ungoverned component [Chapter 23](#) and [Chapter 45](#)’s antipattern chapter both warned against, and no amount of the model’s capability makes that wiring safe.

134.4 AI within governance, not outside it

The integration of AI capabilities — language models, agents, retrieval systems — into a regulated process must happen *within* the process’s governance, not outside it. An LLM call from inside a governed process step is auditable: the input, the output, the model version, the timestamp are all part of the process record. An LLM call from an ungoverned script, triggered by a cron job or a manual action, has none of these properties. The process model is the governance frame, and every AI capability must sit inside it, as a step whose inputs and outputs the institution can show, explain, and defend.

Worked example: the model asked to do arithmetic

Problem. A team builds a claims-intake step that uses a language model to read a free-text claim description and extract structured data. Pleased with how well it works, they extend the prompt to also ask the model to *compute* the total claimed amount by summing the individual amounts mentioned in the description. Testing on 500 claims, the extraction of individual amounts is correct on 99% of claims, but the model’s *summed total* is wrong on 7% of claims — it makes plausible-looking arithmetic errors. The process uses that total to route claims: claims under \$5,000 are fast-tracked. The operation handles 180,000 claims a year. How many claims a year are mis-routed by the model’s arithmetic errors, and what is the correct design?

Setup. The mis-routed claims are those where the model’s summed total is wrong — 7% of claims — and the error crosses the \$5,000 routing threshold. We compute the affected volume and state the fix.

Working. Claims a year with a wrong model-computed total:

$$180,000 \times 0.07 = 12,600 \text{ claims with an arithmetic error.}$$

Each is a claim whose routing — fast-track or not — was decided on a wrong number. Even if only a fraction of those errors happen to cross the \$5,000 threshold, the design has put 12,600 claims a year onto a routing decision made from an unreliable figure.

Discussion. The design fails on exactly the boundary this chapter drew. The model’s *extraction* of the individual amounts — a text-transformation task — was 99% correct, because extraction is in the family of things a language model does reliably. The model’s *summation* of those amounts — arithmetic — was wrong 7% of the time, because arithmetic is in the family of things a language model does not do reliably: it does not compute, it produces a plausible-looking number, and a plausible-looking sum is right most of the time and confidently wrong the rest. The team’s mistake was natural — the model was extracting the amounts anyway, so asking it to also add them up felt like a free extension — but it crossed the model from a task it is reliable at to a task it is not, in a single sentence of prompt. The fix is equally simple

and is the chapter's whole discipline: let the model do the extraction it is good at — pull out the individual amounts as structured data — and let *deterministic code* do the arithmetic. Summing a list of numbers is a line of code that is correct 100% of the time; it should never have been delegated to a probabilistic component. The lesson generalises beyond arithmetic: whenever a model-in-process design is extended, the extension must be checked against the capability boundary — is this new thing still in the family the model is reliable for? — because the boundary is crossed not by big architectural decisions but by small, reasonable-seeming additions to a prompt.

Practical next steps

Audit one language-model step in your platform against the capability boundary. List everything the model is being asked to do in its prompt, and for each, ask: is this transformation, interpretation, or drafting — which the model is reliable for — or is it calculation, fact-retrieval, or decision — which it is not? Anything in the second group should be moved to deterministic code or a governed decision, and the model's prompt narrowed to what it is genuinely good at.

Chapter in brief

A large language model in a Camunda platform must be a contained step, not the centre: the process is the governed, auditable whole, and the model is one component within it. A model is reliable at transforming unstructured text — summarising, extracting, classifying — and at drafting and interpreting language. It is not reliable as a calculator, a source of fact, a decision-maker of record, or a deterministic component. Used as a step, the model is contained by [Chapter 23](#)'s four disciplines: a bounded task, a structured output, a deterministic adjudication of that output, and an accountable human. The capability boundary is crossed by small, reasonable-seeming prompt extensions — so every extension must be re-checked against it.

Retrieval-Augmented Generation in the Process

135.1 The fact problem, and what solves it

The previous chapter named, among the things a language model is not reliable at, being a *source of fact*: asked about the institution's policy or a customer's situation, a model produces a fluent answer that may be entirely invented. For a bank or insurer this is a disqualifying flaw — a process step that confidently states invented facts about a customer's cover or a product's terms is worse than no step at all.

Retrieval-augmented generation — RAG — is the technique that addresses this flaw, and this chapter treats it as a process capability. The idea is precise: rather than asking the model to answer from its own trained parameters — where the institution's actual policies and the customer's actual situation are not, and cannot reliably be — the system first *retrieves* the relevant facts from a trustworthy source, and then asks the model to produce its answer *grounded in those retrieved facts*. The model's job shifts from “know the answer” — which it cannot — to “compose an answer from these supplied facts” — which is the language task it is good at.

135.2 How RAG works, as a process capability

RAG, reduced to its essentials, is a sequence of steps, and seeing it as a sequence is the key to building it into a process correctly.

First, *the question is received* — a customer's free-text query, a request to summarise a policy's relevant terms, whatever the process needs answered. Second, *the relevant facts are retrieved*: the system searches a trustworthy knowledge source — the institution's actual policy documents, the customer's actual account data, the genuine product terms — for the material relevant to the question. This retrieval is itself often done with a *vector search*, which finds documents by semantic similarity rather than exact keyword match, but the mechanism matters less than the principle: the facts come from a real, governed source. Third, *the model is asked to answer*, given both the question and the retrieved facts, and instructed to ground its answer in those facts and not to go beyond them. Fourth — and this is the discipline a governed platform adds — *the answer is checked* before the process uses it.

The crucial property is the third step's grounding. A well-constructed RAG prompt does not ask the model “what is this customer's cover?” — it supplies the customer's actual policy text and asks “based on the following policy text, what does it say about this customer's cover?” The model is no longer being asked to know; it is being asked to read and summarise supplied material, which is the reliable text-transformation task of the previous chapter.

Figure 135.1 draws RAG as the four-step sequence it is, with the governed knowledge source feeding the retrieval.

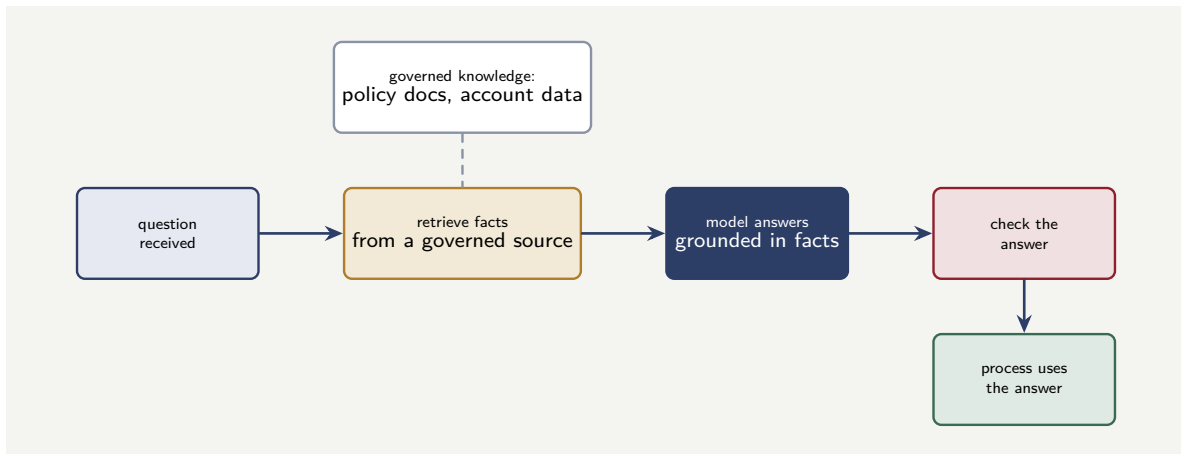


Figure 135.1. Retrieval-augmented generation as a process sequence. The question is received; relevant facts are retrieved from a governed knowledge source; the model answers grounded in those facts rather than from its own parameters; the answer is checked before the process uses it.

135.3 RAG modelled in Camunda

A great strength of building RAG inside a Camunda process is that the sequence above becomes *explicit BPMN*, and explicitness is governability.

Each step of the RAG sequence is a modelled element. The retrieval is a service task — often a connector to a vector-search service over the institution’s knowledge base. The generation is a service task using the language-model connector of the previous chapter. The check is a step — a deterministic validation, or itself a modelled human task for a consequential answer. And the flow between them, the routing on a failed check, the path to a human when the answer cannot be trusted — all of it is on the diagram, a reviewer can see it, the audit record captures it.

This is the manuscript’s recurring argument applied to RAG: a RAG capability built as opaque application code is an ungoverned thing — no one can see what it retrieves, how it grounds, whether it checks. The same RAG capability built as an explicit Camunda process is governed — the retrieval source is a named service task, the grounding is visible, the check is a modelled step, and the whole is auditable. RAG is not exempt from [Chapter 2](#)’s principle that the process must be an explicit, shared, readable model; it is another process, and it belongs on the diagram.

135.4 The disciplines RAG still needs

RAG addresses the fact problem, but it does not, on its own, make a language-model step safe, and three disciplines remain.

The retrieved facts must come from a *governed, current source*. RAG is only as trustworthy as its knowledge base: if the policy documents it retrieves from are out of date, the model will faithfully ground its answer in stale facts and be confidently wrong in a new way. The knowledge source is itself a governed artefact, and keeping it current is part of the platform’s discipline.

The answer must still be *checked*. RAG reduces invention but does not eliminate it — a model can still misread the retrieved facts, or answer beyond them. The check step of the sequence is not optional, and for a consequential answer the check should include a human.

And RAG does not make the model a *decision-maker*. A RAG step can tell a customer what their policy says; it must not, on its own, *decide* their claim. The decision remains where [Chapter 23](#) and [Chapter 4](#) put it — in a governed, deterministic decision, which a RAG step may inform but never replaces.

135.5 AI within governance, not outside it

The integration of AI capabilities — language models, agents, retrieval systems — into a regulated process must happen *within* the process's governance, not outside it. An LLM call from inside a governed process step is auditable: the input, the output, the model version, the timestamp are all part of the process record. An LLM call from an ungoverned script, triggered by a cron job or a manual action, has none of these properties. The process model is the governance frame, and every AI capability must sit inside it, as a step whose inputs and outputs the institution can show, explain, and defend.

Worked example: grounded versus ungrounded answers

Problem. An insurer builds a step that answers customers' free-text questions about their own policy cover. Two designs are tested on 1,000 real questions, each with a known correct answer. Design A asks the language model the question directly, relying on the model's own knowledge. Design B retrieves the customer's actual policy document and asks the model to answer grounded in it. Design A produces a confidently-stated *wrong* answer on 22% of questions — the model inventing plausible cover details. Design B produces a wrong answer on 3%. The insurer answers 500,000 such questions a year. A wrong answer about cover is a potential mis-selling or conduct issue. Compare the two designs' wrong-answer volumes.

Setup. The wrong-answer volume is the error rate times the annual question volume, for each design. We compute both and consider the difference.

Working. *Design A — ungrounded.* Wrong answers a year:

$$500,000 \times 0.22 = 110,000 \text{ confidently wrong answers about cover.}$$

Design B — RAG, grounded. Wrong answers a year:

$$500,000 \times 0.03 = 15,000.$$

RAG reduces the wrong-answer volume by

$$110,000 - 15,000 = 95,000 \text{ a year (an 86\% reduction).}$$

Discussion. RAG cuts the confidently-wrong-answer volume from 110,000 to 15,000 a year — and for an insurer, where a wrong statement about a customer's cover is a conduct and mis-selling exposure, that 95,000-answer reduction is the difference between a capability that is a serious liability and one that is largely sound. But the worked example carries two lessons, not one. The first is the obvious one: grounding works, because it moves the model from the task it fails at — knowing the customer's specific cover — to the task it succeeds at — reading a supplied policy document and summarising what it says. The second lesson is in design B's residual 3%. RAG did not reach zero. Fifteen thousand wrong answers a year remain, because a model given the right document can still misread it, or answer slightly beyond it. That residual is exactly why the chapter insists the check step is not optional and why, for a consequential answer, the check should include a human. Design B is the right

architecture, but “the right architecture” is not the same as “done”: it still needs the checking discipline around it. The deeper point is that RAG is a large, necessary improvement and not a complete solution — it changes the model’s wrong-answer rate from disqualifying to manageable, and the platform’s checking discipline must then manage what remains. An institution that adopts RAG and believes the fact problem thereby solved has done 86% of the work and declared 100% — and the last 3% is the part that, unchecked, still reaches customers.

Practical next steps

If your platform has a language-model step that answers questions about policies, products, or customer situations, establish whether it is grounded — retrieving real facts and answering from them — or ungrounded, answering from the model’s own parameters. An ungrounded step is producing confident invented answers at a rate that, multiplied by your volume, is almost certainly a conduct concern. RAG is the architecture that fixes it — and the check step after it is what handles the remainder RAG does not.

Chapter in brief

A language model is not a reliable source of fact — retrieval-augmented generation, RAG, is the technique that addresses this. Rather than asking the model to answer from its own parameters, RAG first retrieves the relevant facts from a governed, trustworthy source, then asks the model to answer grounded in those facts — shifting the model from “know the answer” to the reliable task of “compose an answer from these supplied facts.” RAG is a four-step sequence — receive, retrieve, generate-grounded, check — and built in Camunda each step is explicit BPMN, which makes the capability governable. RAG still needs a current knowledge source, a check on every answer, and the recognition that it informs decisions but never replaces a governed one.

Prompts, Context, and the Cost of Language Models

136.1 The operational realities

The two preceding chapters treated the language model as a capability and RAG as the technique that grounds it. This chapter treats the operational realities of running language models in a process at scale — the things a platform team discovers once a model-in-process design moves from a demonstration to a production system handling real volume: the prompt as a governed artefact, the management of context, latency, cost, and failure.

These are not glamorous concerns, and a hyperautomation programme excited by what a model *can do* is tempted to treat them as details. They are not details. They are the difference between a model capability that is sustainable in production and one that is slow, expensive, unpredictable, and quietly ungoverned.

136.2 The prompt is a governed artefact

The instruction given to a language model — the *prompt* — determines what the model does, and it must therefore be treated with the same seriousness as any other thing that determines process behaviour.

A prompt is, in effect, logic. The prompt that tells a model how to classify a customer message, what categories to use, how to handle an ambiguous case — that prompt *is* the classification logic, as surely as a DMN table would be. And so it must be governed as logic is: it is versioned, it is reviewed, it is tested, and it is changed deliberately, not edited casually by whoever is nearest. A platform where prompts live as string literals scattered in application code, edited without review, is a platform whose process behaviour can be changed by anyone with commit access and no one watching — the fat-process-model antipattern of [Chapter 45](#), in a new guise.

The disciplined practice keeps prompts as managed artefacts: versioned alongside the process model, reviewed when changed, and tested — because a prompt change can shift the model's behaviour as much as a code change can, and an unreviewed, untested prompt change is an unreviewed, untested change to the process.

Tip

Treat a prompt as governed logic, not as a string. The prompt determines what the model does, so a prompt change is a behaviour change — version it, review it, test it, alongside the process model. A platform whose prompts are scattered, unversioned string literals edited without review has a process whose behaviour anyone can change and no one is watching.

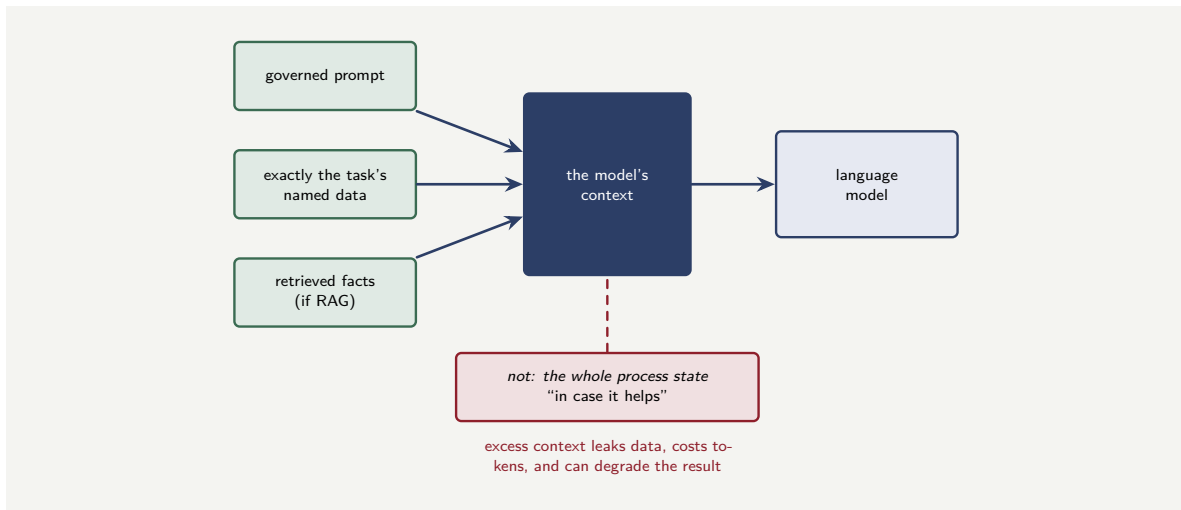


Figure 136.1. Constructing the model’s context. The context is assembled deliberately from the governed prompt, exactly the task’s named data, and — in a RAG step — the retrieved facts. The whole process state poured in “in case it helps” leaks data, costs tokens on every call, and can degrade the result.

136.3 Context: what the model is told

A language model answers based on the *context* it is given — the prompt, plus whatever data the step supplies, plus, in a RAG step, the retrieved facts. Managing that context well is a real discipline, and it pulls in two directions.

The model needs *enough* context: a model asked to extract fields from a claim needs the claim description; a RAG step needs the retrieved documents. Too little context, and the model cannot do the task.

But the model should not be given *too much*, and for two reasons that a financial platform must weigh. The first is the data-minimisation principle of Part IV’s security chapter: the context sent to a model-serving endpoint — especially an externally-hosted one — is customer data leaving the process, and it should be the minimum the task needs, not the whole process state poured into the prompt for convenience. The second is that context has a *cost* — the next section’s concern — and an oversized context is a directly larger bill and a slower response.

The discipline is the data-flow discipline of [Chapter 3](#), applied to the model step: the step is given exactly the named data the task requires, and the construction of the model’s context is as deliberate as the construction of any other step’s inputs.

Figure 136.1 draws the deliberate construction of a model step’s context.

136.4 Latency, cost, and failure

Three operational properties of a language-model step must be designed for, not discovered.

Latency. A call to a language model is not instant — it is, typically, far slower than a deterministic service task, and slower for a longer output. A model step sits on the process’s critical path, and [Chapter 9](#)’s latency-budget thinking applies directly: the model step’s latency must fit the process’s budget, and a model step on a synchronous, customer-waiting path needs particular care, including a timeout and a fallback for when the model is slow.

Cost. A language model is charged for, typically in proportion to the volume of text in and out — the *tokens*. At a demonstration’s volume the cost is invisible; at a production process’s

volume — hundreds of thousands of instances — it is a real and recurring line item, and one that scales with every instance. The cost must be estimated before a model step is committed to, and it is directly controlled by the context discipline of the previous section: a leaner context is a smaller bill on every one of those instances.

Failure. A model-serving endpoint can be slow, can be unavailable, can return a malformed or unusable output. The model step is, in the end, a service task calling an external system, and every failure-handling discipline of [Chapter 41](#) applies: the call needs a timeout, transient failures are retried, and — critically — the process model needs a defined path for when the model step cannot produce a usable result. A process that assumes the model step always succeeds will stall the first time the model service has a bad afternoon.

136.5 AI within governance, not outside it

The integration of AI capabilities — language models, agents, retrieval systems — into a regulated process must happen *within* the process's governance, not outside it. An LLM call from inside a governed process step is auditable: the input, the output, the model version, the timestamp are all part of the process record. An LLM call from an ungoverned script, triggered by a cron job or a manual action, has none of these properties. The process model is the governance frame, and every AI capability must sit inside it, as a step whose inputs and outputs the institution can show, explain, and defend.

Worked example: the token cost of a careless context

Problem. A claims platform has a model step that classifies the intent of a customer's message. The step needs the customer's message — about 150 tokens — and a prompt of about 100 tokens. But the step, as built, also includes in the model's context the customer's entire claim history and full policy document — about 4,000 additional tokens — on the vague reasoning that “more context might help.” None of those 4,000 tokens is relevant to classifying the *intent of a message*. The model service charges \$0.50 per million input tokens. The platform classifies 2,000,000 messages a year. Compute the annual cost of the unnecessary context, and identify the deeper problem.

Setup. The unnecessary input is the 4,000 tokens of irrelevant context per call. The annual cost is that, times the call volume, times the per-token price. We then consider the non-cost harm.

Working. Unnecessary tokens per call: 4,000. Across 2,000,000 calls a year:

$$4,000 \times 2,000,000 = 8,000,000,000 \text{ unnecessary input tokens a year.}$$

At \$0.50 per million tokens:

$$\frac{8,000,000,000}{1,000,000} \times \$0.50 = \$4,000 \text{ a year}$$

spent transmitting, on every classification, a claim history and policy document that play no part in classifying a message's intent.

Discussion. Four thousand dollars a year for context that does nothing is the measurable waste, and on its own it is reason enough to fix the step — but the worked example is really about two deeper problems the careless context creates. The first is the data-protection one. Those 4,000 tokens are the customer's entire claim history and full policy document, and they are being transmitted to a model-serving endpoint — on every single message classification,

two million times a year — for no reason. That is a large, needless, repeated export of sensitive customer data across the process boundary, and the data-minimisation principle of Part IV’s security chapter is violated not once but two million times annually. The second problem is that the careless context probably makes the classification *worse*, not better: a model asked to classify a short message’s intent, but handed 4,000 tokens of unrelated history, has been given a haystack to find a needle in, and models can be distracted by irrelevant context just as people can. The “more context might help” reasoning is wrong on every count — it costs money, it leaks data, and it can degrade the result. The fix is the chapter’s discipline: the step’s context is constructed deliberately to contain exactly what the task needs — here, the message and the prompt, 250 tokens — and nothing else. The context of a model step is not a place to pour in everything available on the hope it helps; it is a deliberately constructed input, governed like any other.

Practical next steps

For one language-model step in your platform, examine exactly what goes into the model’s context, token by token, and for each part ask whether the *specific task* genuinely needs it. The parts that fail that test are costing money on every call, leaking customer data across the boundary on every call, and possibly degrading the result — and trimming the context to what the task needs fixes all three at once.

Chapter in brief

Running language models in a process at scale has operational realities that are not details. The prompt is governed logic — it determines the model’s behaviour, so it must be versioned, reviewed, and tested like a DMN table, not scattered as unversioned string literals. The model’s context must be constructed deliberately: enough for the task, but no more, because excess context leaks customer data across the boundary, costs money on every call, and can degrade the result. A model step has real latency — it must fit the process’s latency budget — a real, volume-scaling token cost that must be estimated up front, and real failure modes that need a timeout, retries, and a defined fallback path. These realities are designed for, not discovered.

PART XXV

Agentic AI

What an Agent Is, and Why It Needs a Process

137.1 From a model to an agent

The previous part treated the large language model as a process step: a component that, given input, produces output, once. An *agent* is more than that, and this part treats the difference — because the difference is exactly where the governance challenge of agentic AI lies.

Chapter 23 introduced the agentic layer; this part gives it the extended treatment its importance now warrants. The manuscript has placed this part deliberately last among the technical parts, because understanding agentic AI well depends on everything before it: the process discipline of Part I, the architecture of Part IV, the Camunda mechanics of Part V, and the language-model treatment of the previous part. An agent, properly understood, is not a new and separate thing to bolt onto a platform — it is a particular, demanding use of everything the manuscript has already built.

137.2 The defining characteristics of an agent

An agent differs from a plain model step in a few specific, related ways, and naming them precisely is the foundation of governing them.

An agent is *goal-directed*. A model step performs a defined transformation — extract these fields, classify this message. An agent is given a *goal* — “assess this claim,” “resolve this customer’s query” — and works toward it, where the path to the goal is not fixed in advance.

An agent *reasons in a loop*. A model step runs once. An agent typically runs in a cycle: it considers the goal and its situation, decides on a next action, takes it, observes the result, and reconsiders — looping until it judges the goal met. This reasoning loop is the agent’s defining mechanism.

An agent *uses tools*. A model step transforms text. An agent, in its loop, can invoke *tools* — it can call a service to look something up, to perform an action, to fetch data — and chooses, in its reasoning, which tool to use and when.

An agent has, in some designs, *memory* — it carries context across the steps of its loop, and sometimes across separate runs.

These characteristics make an agent far more capable than a single model step — and far more demanding to govern. A model step does one bounded thing; an agent pursues a goal through a self-directed sequence of tool-using reasoning steps, and *self-directed* is the word that should make a banking or insurance architect pay close attention.

Figure 137.1 draws the reasoning loop that defines an agent.

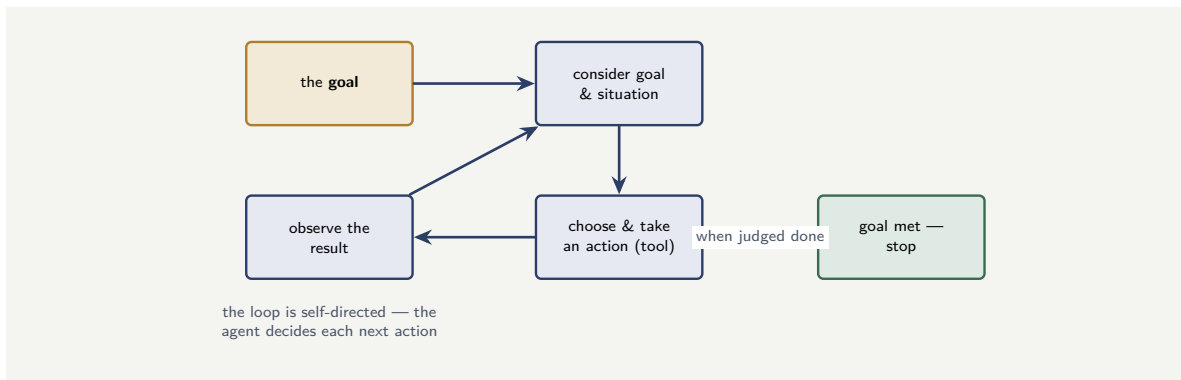


Figure 137.1. An agent’s reasoning loop. Given a goal, the agent cycles — consider the goal and situation, choose and take an action with a tool, observe the result, reconsider — until it judges the goal met. The loop is self-directed: the agent decides each next action, which is what makes it powerful and what makes it demanding to govern.

137.3 The spectrum of agent autonomy

It is tempting to treat “agent” as a single thing — a system either is an agent or is not. The reality a practitioner must work with is a *spectrum*: agents differ in *how much autonomy* they are given, and where a particular agent sits on that spectrum is one of the most consequential design decisions in an agentic process.

At the lowest-autonomy end is the agent that *proposes* but does not act. It reasons over a goal, decides what should be done, and produces a recommendation — but a human, or a deterministic downstream step, actually takes the action. The agent’s autonomy is confined to *thinking*; the *doing* is held outside it. A claims-assessment agent that reads a claim, reasons about it, and proposes a settlement figure for a human adjuster to approve is at this end.

In the middle is the agent that *acts within bounds*. It is permitted to take real actions itself — call tools, update systems — but only within an explicitly delimited envelope: certain tools, certain value limits, certain case types. Inside the envelope it acts autonomously; at the envelope’s edge it must stop, or escalate. An agent that can itself approve claims up to a modest value but must hand larger ones to a human is here.

At the highest-autonomy end is the agent given broad latitude to pursue a goal through whatever sequence of actions it judges fit, with few predefined bounds. This is the most capable configuration and, for a regulated institution, the most dangerous — and, as the rest of this part argues, the one a bank or insurer should almost never choose for a consequential process.

The practitioner’s discipline is to recognise that autonomy is a *dial*, not a *switch*, and to set the dial deliberately, per agent, as low as the task permits. The question is never simply “shall we use an agent” but “how much autonomy shall this agent have, and what holds the rest.” An institution that sets every agent to high autonomy because high autonomy is the most capable setting has confused the dial’s maximum with its correct position.

Figure 137.2 sets out the autonomy spectrum.

137.4 Why an agent needs a process

Here is the central argument of this part, and it follows directly from those characteristics. An agent, left to itself, is goal-directed, looping, tool-using, and self-directed — which means

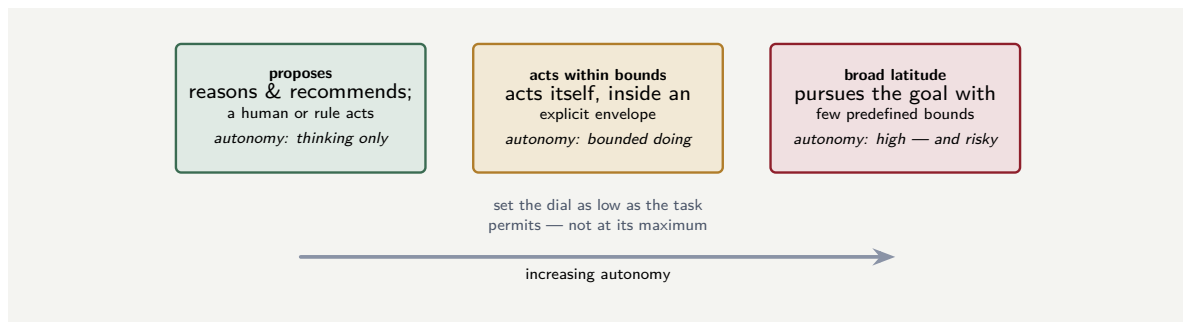


Figure 137.2. Agent autonomy is a spectrum, not a switch. An agent may merely *propose*, leaving the action to a human or a rule; may *act within an explicit envelope*; or may be given *broad latitude*. The practitioner sets the dial deliberately, per agent, as low as the task permits.

an agent left to itself is a component that decides its own actions, takes them, and continues until it decides to stop. In a domain of cat-photo captioning, that autonomy is charming. In a domain where the tools move money, bind insurance cover, and close claims — and where every action is subject to regulation — an unconstrained autonomous agent is not charming; it is an unbounded, unaccountable actor inside the bank.

The manuscript’s answer, argued from [Chapter 23](#) and made the centre of this part, is that an agent must run *inside a process*. The process is what supplies the agent with everything its raw autonomy lacks: a *boundary* on what it may do, a defined set of *tools* rather than open capability, *adjudication* of its conclusions before they take effect, *accountability* through a named human at every consequential point, and an *audit record* of what it did. The agent supplies the capability — the flexible, goal-directed reasoning; the process supplies the governance. Neither is sufficient alone: an agent without a process is ungoverned, and a process without the agent cannot do the flexible reasoning the hardest cases need.

This is not a constraint grudgingly imposed on agents. It is the architecture that makes agents *usable* in a regulated institution at all. An institution that cannot run agents inside processes cannot responsibly run agents; an institution that can has a way to use the most powerful automation capability available while keeping it as governed as everything else.

An agent is goal-directed, loops, uses tools, and is self-directed — which means an unconstrained agent decides and takes its own actions until it decides to stop. In a domain where the tools move money and bind cover, an autonomous agent without a process around it is an unbounded, unaccountable actor inside the institution. The process is not an optional wrapper; it is what supplies the boundary, the defined toolset, the adjudication, the accountability, and the audit record that an agent’s raw autonomy lacks. An institution that cannot run agents inside processes cannot responsibly run agents.

137.5 AI within governance, not outside it

The integration of AI capabilities — language models, agents, retrieval systems — into a regulated process must happen *within* the process’s governance, not outside it. An LLM call from inside a governed process step is auditable: the input, the output, the model version, the timestamp are all part of the process record. An LLM call from an ungoverned script, triggered by a cron job or a manual action, has none of these properties. The process model is

the governance frame, and every AI capability must sit inside it, as a step whose inputs and outputs the institution can show, explain, and defend.

Worked example: the autonomous agent and the bounded one

Problem. Two teams build a claims-assistance agent. Team A builds an *autonomous* agent: it is given the goal “resolve this claim” and broad access — it can read systems, write to systems, move money, and close cases, and it loops until it judges the claim resolved. Team B builds a *process-embedded* agent: it runs inside a Camunda claims process, is given a bounded task within that process — “assess this claim’s evidence and recommend a disposition” — has read-only tools, and its recommendation is adjudicated by a deterministic decision and, above a threshold, a human. Both are tested on 10,000 claims. Team A’s agent resolves 94% of claims correctly and, on 6%, takes a wrong consequential action — pays a claim that should have been declined, closes a claim that needed investigation. Team B’s agent produces a wrong *recommendation* on a similar 6%, but the adjudication catches them. Compare the consequential errors that actually reach the world.

Setup. The errors that reach the world are wrong *actions* — not wrong internal recommendations the platform catches. We compute them for each design.

Working. *Team A — autonomous agent.* On 6% of 10,000 claims, the agent itself takes a wrong consequential action, with nothing between its decision and the effect:

$$10,000 \times 0.06 = 600 \text{ wrong consequential actions reach the world.}$$

Team B — process-embedded agent. The agent produces a wrong recommendation on 6% — 600 claims — but the agent takes no consequential action itself; its recommendation goes to a deterministic adjudication and, for consequential dispositions, a human. The wrong recommendations are caught there. Wrong consequential actions reaching the world:

$$\approx 0 \quad (\text{the adjudication and human catch the bad recommendations}).$$

Discussion. Both agents are, in raw reasoning quality, *identical* — each is wrong about 6% of the time, because they are built on the same underlying capability. The difference of 600 wrong actions versus essentially none is not a difference in the agents; it is entirely a difference in the *architecture around* them. Team A’s autonomous agent has its 6% error rate connected directly to consequential tools — so 6% of the time, money moves wrongly or a claim is wrongly closed, with nothing in between. Team B’s process-embedded agent has the identical 6% error rate, but its errors are wrong *recommendations*, not wrong actions, and they meet a deterministic adjudication and a human before anything consequential happens — so the same 6% reasoning error produces, at the world, almost no wrong actions at all. This is the whole argument of the part in one comparison. The governance of an agent is not about making the agent’s reasoning better — both teams’ agents reason identically. It is about ensuring that when the agent’s reasoning is wrong, as it sometimes will be, the error is caught by the process around it before it becomes a consequence. Team A built a capable agent and shipped its error rate straight to customers’ claims and the bank’s money. Team B built an equally capable agent and placed it inside a process that catches what it gets wrong. The agent is the same; the architecture is everything.

Practical next steps

For any agent your institution is building or considering, ask the team A versus team B question: when the agent's reasoning is wrong — and it will be, at some rate — what happens? If the answer is “a wrong action occurs,” the agent is autonomous and the architecture is missing. If the answer is “the process catches a wrong recommendation before it becomes an action,” the agent is embedded and governed. The agent's error rate is not the design question; what the architecture does with that error rate is.

Chapter in brief

An agent differs from a model step in being goal-directed, reasoning in a loop, using tools, and being self-directed — it pursues a goal through a sequence of actions it chooses itself. That makes it powerful and, in a regulated domain, demanding to govern: an unconstrained agent is an unbounded, unaccountable actor inside the institution. The answer is that an agent must run inside a process, which supplies what its autonomy lacks — a boundary, a defined toolset, adjudication of its conclusions, accountability through named humans, and an audit record. Two agents of identical reasoning quality differ entirely in the errors that reach the world according to whether a process catches their wrong conclusions: the agent's error rate is not the design question; what the architecture does with it is.

Agentic BPMN: Modelling the Agent in the Process

138.1 Making the agent a modelled thing

The previous chapter argued that an agent must run inside a process. This chapter treats how — concretely, in Camunda — the agent becomes a *modelled* element of a BPMN process, through the capability Camunda calls *agentic BPMN*.

The principle behind agentic BPMN is the manuscript’s oldest and most consistent: a thing that is modelled explicitly is a thing that can be governed, reviewed, and audited; a thing that runs as opaque code cannot be. [Chapter 2](#) applied that principle to processes, [Chapter 4](#) to decisions, and agentic BPMN applies it to agents. It makes the agent — its goal, its tools, its reasoning loop, its boundaries — elements of the BPMN model, so that the agent is as explicit, as reviewable, and as auditable as any other part of the process.

138.2 The elements of an agentic process

An agentic process — a process with an agent in it — has, beyond ordinary BPMN, a recognisable set of elements, and understanding them is understanding how the agent is governed by the model.

The *agent task* is the element where the agent does its reasoning. It is configured with the agent’s *goal* — the bounded task it is to pursue, expressed as model configuration a reviewer can read — and with the *model* that powers the agent’s reasoning. The goal is not buried in code; it is on the diagram.

The *tools* available to the agent are declared — as a defined set, in the model. The agent, in its reasoning, may use these tools and only these. Crucially, each tool is itself typically a modelled element — a service task, a connector, a sub-process — so a tool the agent invokes is a tool the process governs. An agent given a “look up policy” tool is given a modelled, governed lookup; it is not given open access to the policy system.

The *reasoning loop* — the agent’s cycle of decide, act, observe — runs within the agent task, but its boundaries are modelled: how many iterations it may run, what conditions end it, what happens if it does not converge. The loop is bounded by the model, not left to run until the agent itself decides to stop.

The *human-in-the-loop boundary* is a modelled element: a point where the agent’s work passes to a human task — for a decision, for an approval, for a disposition the agent may recommend but not make. [Chapter 7](#)’s accountable human is, in agentic BPMN, a user task on the diagram.

And the *adjudication* of the agent’s output — the deterministic check of [Chapter 23](#) — is a modelled step: a business-rule task, a gateway, a decision that examines what the agent produced before the process acts on it.

Figure [138.1](#) draws an agentic process: the agent task with its modelled tools, the adjudica-

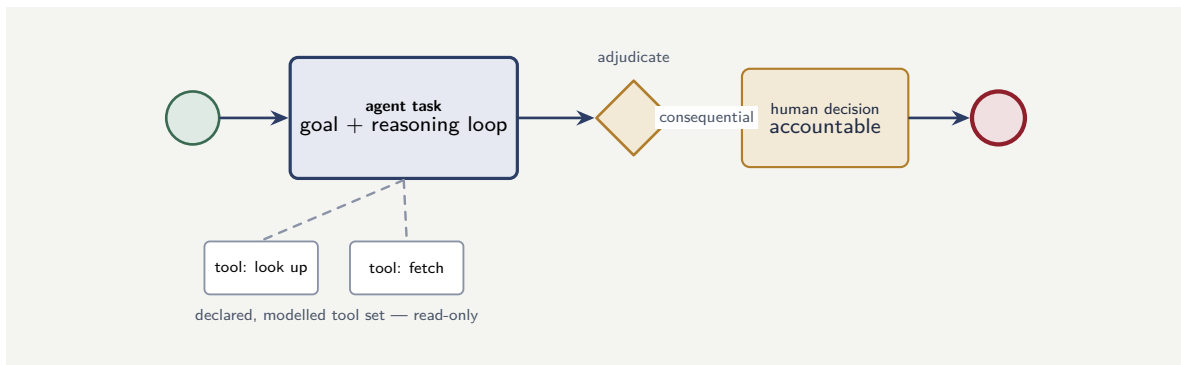


Figure 138.1. An agentic process in BPMN. The agent task — carrying the goal and the reasoning loop — uses a declared, modelled set of read-only tools. Its output is adjudicated, and a consequential outcome passes to an accountable human. Every element is on the diagram, reviewable and auditable.

tion, the human-in-the-loop boundary, the deterministic flow around it all.

138.3 Why modelling the agent matters

The value of agentic BPMN is that it makes every one of [Chapter 23](#)’s containments *visible and enforced by the model* rather than hoped for in code.

A reviewer looking at an agentic process can *see* the agent’s goal — and challenge it if it is too broad. They can *see* the agent’s tools — and ask why a particular tool is in the set, or insist a write-capable tool is removed. They can *see* where the human-in-the-loop boundary is — and require one where it is missing. They can *see* the adjudication — and check it is genuinely deterministic. None of this is possible when an agent is opaque code; all of it is routine when the agent is modelled.

And the audit record, drawn from the executed model, captures the agent’s operation as it captures any process: which goal the agent pursued, which tools it invoked, what it recommended, how the adjudication ruled, which human decided. An agentic process is, after the fact, as accountable as any other process — because it *is* a modelled process, with an agent as one governed element.

138.4 BPMN as a regulated artefact

In banking and insurance, a BPMN process model is not documentation — it is a *regulated artefact*. A regulator may ask to see the process, and the model must be the answer: accurate, current, and executable. This means the model must be maintained as a source of truth, versioned, reviewed before deployment, and never allowed to diverge from what the engine actually runs. A model that is drawn once and forgotten is worse than no model at all, because it gives the false assurance of governance while the real process drifts.

Worked example: the tool that should not have been in the set

Problem. A claims-assessment agent is built as an agentic process. In the model, the agent task’s declared tool set has five tools: *look up policy*, *look up claim history*, *fetch the evidence documents*, *check the fraud watchlist*, and — added late in development for “efficiency” — *update*

the claim status. The first four are read-only; the fifth can write. A model review, examining the agentic process before it goes live, sees the five tools on the diagram. What should the review conclude about the fifth tool, and what does this illustrate about agentic BPMN?

Setup. We assess each tool against the containment principle: an agent's tools should be the minimum its bounded task requires, and a consequential write capability should not be a tool the agent invokes at its own discretion. We then draw the lesson.

Working. The agent's bounded task is to *assess a claim and recommend a disposition*. Assessment requires *reading*: the policy, the history, the evidence, the watchlist — the four read-only tools are exactly what the task needs. *Updating the claim status*, however, is not assessment — it is a consequential action, and it is precisely the kind of action that, per [Chapter 23](#) and the previous chapter, must not be a tool the agent invokes at its own discretion. The agent should *recommend* a disposition; the claim status should be updated by a *deterministic step after the adjudication*, not by the agent reaching for a write tool mid-reasoning. The review's conclusion: the fifth tool must be removed from the agent's set, and the claim-status update made a modelled deterministic step downstream of the adjudication.

Discussion. The review caught a genuine and dangerous defect — an agent given a write-capable, consequential tool it could invoke autonomously — and the worked example's point is *how* it caught it: the tool was *visible on the diagram*. The fifth tool was added late, for “efficiency,” by someone who reasoned that since the agent knows the disposition it might as well update the status — exactly the kind of small, plausible-seeming extension that, in opaque agent code, no one would ever see and no review could ever catch. Because the agent was built as agentic BPMN, its tool set was an explicit, declared list of five elements on the model, and a reviewer running their eye down that list could see, plainly, that one of the five was a write tool that did not belong — and challenge it before the agent ever ran. This is the entire value of modelling the agent. An autonomous agent with a quietly-added write tool is a latent incident; the same agent built as agentic BPMN has that write tool sitting visibly in a reviewable list, where the ordinary discipline of model review removes it. Agentic BPMN does not make agents safe by magic — it makes them safe by making their containments *visible*, and visible containments are containments a review can actually check.

Practical next steps

For any agentic process your institution runs or plans, find the agent task's declared tool set and read it as a reviewer. For each tool, ask two questions: does the agent's bounded task genuinely need it, and can it *write* or cause a consequential effect? Any tool that fails the first question is scope the agent should not have; any write-capable tool should almost certainly be a deterministic step after adjudication, not a tool the agent invokes itself.

138.5 Tools and memory, examined more closely

The agent task's tools and its memory deserve a closer look than the previous sections gave them, because they are the two elements through which an agent most directly touches — and can most directly endanger — the institution.

Tools, examined closely. A tool is anything the agent can invoke during its reasoning loop: a retrieval of information, a lookup against a system, a calculation, and — the dangerous category — an action with an effect. The discipline already stated is that the tool set is declared and minimal. The deeper point is that tools should be classified, in the model and in the reviewer's mind, by a single sharp question: is this tool *read-only*, or can it *change something*.

A read-only tool — look up a policy, retrieve a document, fetch a balance — can be given to an agent relatively freely, because the worst an agent can do by misusing it is reason from information it should not have weighted. A tool that *writes* — posts a payment, updates a record, sends a binding communication — is a different matter entirely: through a write-capable tool, an agent's reasoning error becomes a real-world consequence directly, with no deterministic step or human between the agent's judgement and the effect. The strong default is that write-capable effects are *not* agent tools at all — they are deterministic steps the process performs *after* the agent's output has been adjudicated. The agent proposes the payment; the process, after adjudication, posts it.

Memory, examined closely. Chapter 23 noted that an agent may have memory — it carries context across the steps of its loop, and sometimes across separate runs. Memory makes an agent more capable, and it also introduces two specific hazards a practitioner must govern. The first is a *data hazard*: an agent that remembers across runs is accumulating data, and accumulated data — about customers, about cases — is subject to every classification, minimisation, and retention rule of Chapter 26. Agent memory is a data store, and an ungoverned agent memory is an ungoverned data store. The second is a *correctness hazard*: an agent that carries memory across cases can let one case's context bleed into another's reasoning — deciding case B partly on something it saw in case A. For a regulated institution, where each case must be decided on its own merits, that bleed is a serious fairness and correctness problem. The discipline is to make agent memory *deliberate and bounded*: scoped to a single case unless there is a specific, governed reason for it to persist further, and treated as the governed data asset it is.

An agent's memory is a data store, and an agent's write-capable tools are a path from its reasoning straight to real-world effect — both must be governed, not assumed benign. Default to read-only tools; make write-capable effects deterministic steps after adjudication, not tools the agent invokes. Scope agent memory to a single case unless a specific governed reason requires persistence, both because cross-case memory can leak one case's context into another's decision, and because accumulated agent memory is subject to every data-classification and retention rule the institution has.

Chapter in brief

Agentic BPMN makes the agent a modelled element of a Camunda process, so it can be governed, reviewed, and audited like any other. An agentic process has recognisable elements: the agent task carrying the goal and reasoning loop; a declared, modelled set of tools the agent may use and only those; a model-bounded reasoning loop; a human-in-the-loop boundary as a user task; and a deterministic adjudication of the agent's output. Modelling the agent makes Chapter 23's containments visible and enforced rather than hoped for — a reviewer can see the goal, the tools, the human boundary, and the adjudication, and challenge what is wrong. Agentic BPMN makes agents safe not by magic but by making their containments visible to review.

Multi-Agent Processes and the Future

139.1 When one agent becomes several

The preceding chapters of this part treated a single agent, embedded in a process. This final chapter treats what happens when an institution wants *several* agents — and what the responsible path forward looks like as agentic capability advances. It is the manuscript's last technical chapter, and it is, deliberately, as much about discipline as about capability.

The appeal of multiple agents is real. A complex claim might be assessed by a *document agent* that interprets the evidence, a *policy agent* that determines what is covered, and a *fraud agent* that weighs the indicators — each specialised, each good at its part. The question is how several agents are combined, and there are two answers, one disciplined and one not.

139.2 Two ways to combine agents

The undisciplined way is to let the agents *talk to each other freely* — a cluster of autonomous agents, passing work and messages among themselves, negotiating, delegating, the overall behaviour emerging from their interaction. This is, in agentic form, exactly the unconstrained choreography that [Chapter 34](#) warned against, and the warning is far sharper here. A cluster of freely-interacting autonomous agents has *no single artefact describing what it does*: the behaviour emerges from agents' interactions, no one modelled it, no one can review it, no one can audit it, and when it does something wrong no one can fully reconstruct why. For a regulated institution this is not an architecture; it is a hazard.

The disciplined way is the one this whole part has built toward: *orchestrate the agents with a process*. Each agent is a contained, modelled agent task, in the agentic-BPMN sense of the previous chapter. A Camunda process *orchestrates* them — it invokes the document agent, then the policy agent, then the fraud agent; it routes between them on defined logic; it adjudicates their combined output; it brings in the human at the modelled boundary. The agents supply specialised reasoning; the process supplies the explicit, governed, auditable structure that combines them. No behaviour emerges unmodelled, because the process *is* the model of how the agents combine.

This is [Chapter 34](#)'s orchestration-versus-choreography lesson, reaching its final and most important application. Orchestrate the agents; do not let them choreograph themselves. The multi-agent process is governed precisely because a process, not the agents' own emergent interaction, determines what happens.

A cluster of autonomous agents interacting freely — passing work, negotiating, delegating among themselves — has no single artefact describing its behaviour: it cannot be reviewed, audited, or fully explained, and when it errs no one can reconstruct why. For

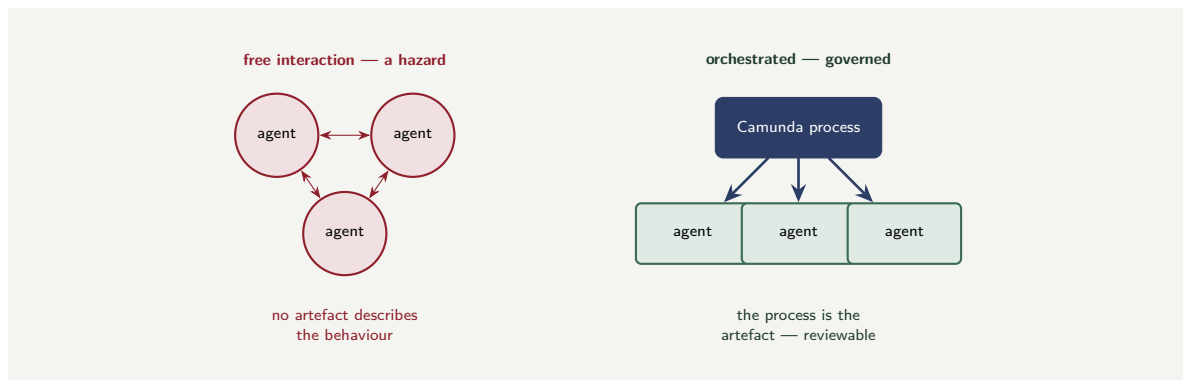


Figure 139.1. Two ways to combine agents. Letting agents interact freely produces emergent behaviour no artefact describes — a hazard for a regulated institution. Orchestrating them with an explicit Camunda process, each agent a contained task, keeps the multi-agent system governed: the process is the reviewable artefact.

a regulated institution that is a hazard, not an architecture. Combine multiple agents by *orchestrating* them with an explicit Camunda process — each agent a contained, modelled task, the process determining how they combine. Orchestrate the agents; never let them choreograph themselves.

Figure 139.1 contrasts the free-interaction hazard with the orchestrated, governed alternative.

139.3 How agents fail, and how to observe them

A part on agentic AI would be incomplete without treating, directly, how agents *fail* — because an agent is a probabilistic component, it will fail, and a practitioner who has not thought through the failure modes cannot design the containment or the observability for them.

Agents fail in characteristic ways, and naming them is the first defence. An agent can be *confidently wrong*: it produces an answer that is fluent, plausible, and incorrect, with no signal that it is incorrect — the failure mode the manuscript has returned to throughout, and the most dangerous because it does not announce itself. An agent can *loop*: its reasoning cycle does not converge, and it continues taking actions without reaching its goal, until something stops it. An agent can *drift from its goal*: across a long reasoning sequence it gradually ends up pursuing something subtly different from what it was asked. An agent can *misuse a tool*: invoke a tool with inputs that are wrong, or invoke it when it should not. And a multi-agent arrangement can fail in the emergent ways the previous sections warned of.

Against these failure modes, the agentic process needs deliberate *containment* and deliberate *observability*. The containments are those this part has already argued for — the bounded loop that caps a non-converging agent, the declared tool set that limits misuse, the adjudication that catches the confidently-wrong output, the human boundary. The *observability* is the part this section adds: an agentic process must be instrumented so that an operator can *see* what the agent did. The agent’s trajectory — the sequence of reasoning steps and tool calls it took to reach its output — should be captured, not discarded. When an agentic process produces a surprising result, the question “what did the agent actually do” must have an answer, and that answer is the recorded trajectory.

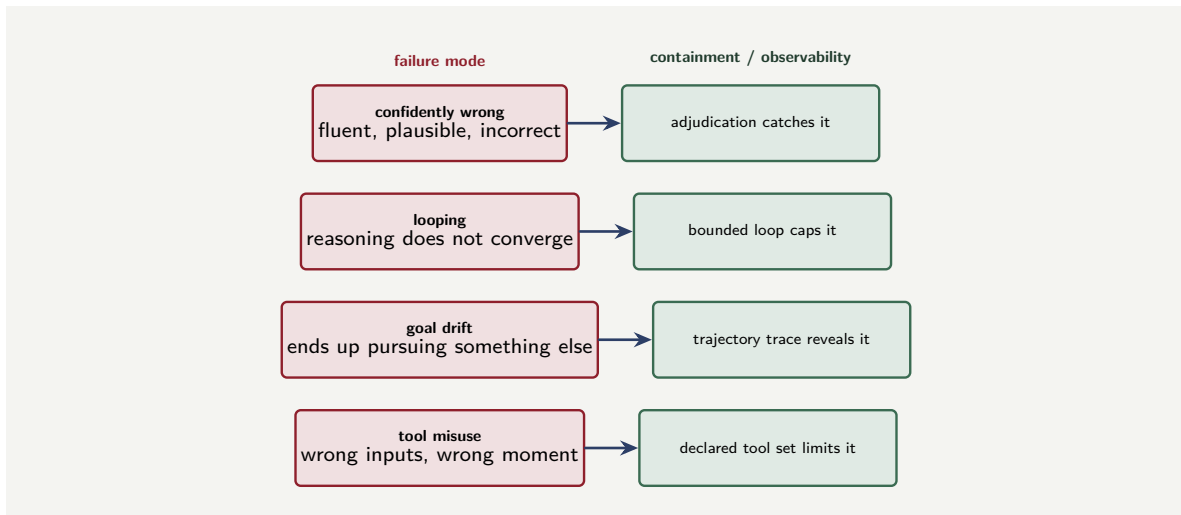


Figure 139.2. How agents fail, and what answers each failure. An agent can be confidently wrong, can loop, can drift from its goal, or can misuse a tool — and each failure mode is met by a specific containment or by the observability of a recorded trajectory. An agent whose trajectory is invisible is ungoverned.

This connects agentic AI back to [Chapter 125](#)’s observability and [Chapter 126](#)’s testing. An agent is not exempt from either. It is observed — its trajectory traced, its outcomes monitored for drift in quality the way a model’s outputs are monitored. And it is tested — not with the deterministic certainty of a rule, but as [Chapter 126](#) described for probabilistic components: evaluated over many cases, its behaviour characterised, its failure rate measured and watched. An agent whose trajectory is invisible and whose quality is unmonitored is an ungoverned agent, however well its surrounding process is modelled.

Figure 139.2 sets the characteristic agent failure modes beside what contains or reveals each.

Worked example: the agent that drifted, and the trace that showed it

Problem. An insurer runs an agentic process in which an agent assesses complex claims and proposes settlements, with a human adjudicating. Over a quarter, adjusters notice that the agent’s proposed settlements have gradually become systematically slightly low — not wrong enough to reject case by case, but, in aggregate, a drift. Two agentic processes are compared. Process A records the agent’s full trajectory — every reasoning step and tool call — for each case. Process B records only the agent’s final proposed figure. Assess how each insurer can respond.

Setup. We consider what each insurer can do to find the cause of the drift, given what each process recorded.

Working. The drift is a goal-drift or reasoning failure mode: the agent is gradually pursuing something subtly different from a correct settlement. *Process A* recorded the full trajectory of every case. The insurer can examine the trajectories — across many cases, see *which reasoning step or which tool result* the agent’s figures turn on, and locate where the drift enters: perhaps a retrieval tool returning stale comparison data, perhaps the agent systematically

under-weighting a factor.

recorded trajectory $\Rightarrow$ the drift can be located and fixed.

Process B recorded only final figures. The insurer can confirm the drift *exists* — the figures are low — but has no record of *how* the agent reached any figure:

only outcomes recorded $\Rightarrow$ the drift is visible but not diagnosable.

Process B's insurer is reduced to guessing, or to rebuilding the agent blind.

Discussion. The worked example makes concrete why agent observability is not optional. Both insurers' agentic processes might be equally well modelled — bounded loops, declared tools, adjudication, a human boundary — and both detected the drift, because the adjudicating humans noticed the aggregate pattern. But detection is not diagnosis. Process A recorded each agent's *trajectory*, so its insurer can do what the failure mode demands: trace *where* in the reasoning the drift enters, and fix that specific cause. Process B recorded only outcomes, so its insurer knows only that something is wrong, with no way to see the agent's reasoning — the agent is a black box that has started misbehaving, and a black box cannot be debugged. The lesson is the section's central one: an agent will exhibit characteristic failure modes — here, drift — and the containments alone (which caught nothing, because each individual case was within tolerance) are not enough; the process must also be *observable*, the trajectory recorded, so that when a failure mode appears it can be diagnosed and not merely detected. An institution that records only what its agents *concluded*, and not how, has built agents it can watch fail but cannot understand — and that is the practical difference between an agent that is governed and one that is merely surrounded by a tidy model.

Practical next steps

For any agentic process your institution runs, check whether the agent's *trajectory* — its reasoning steps and tool calls — is recorded, or only its final output. If only the output, the institution can detect its agents failing but cannot diagnose why. Instrument the agent's trajectory, and monitor agent output quality over time the way a predictive model's quality is monitored — an agent is observed and tested, not trusted.

139.4 The trajectory, and the discipline that survives it

Agentic capability is advancing quickly, and a manuscript that pretended to predict its destination would be foolish. What the manuscript can do, and does here, is identify which of its principles are *robust to that advance* — the disciplines that will still hold however capable agents become.

The capability of agents will grow: they will reason better, use tools more deftly, handle longer and more complex goals, make fewer errors. None of that advance changes the principles of this part. A more capable agent is still probabilistic, still occasionally and confidently wrong, still — in a regulated domain — something whose consequential actions must be adjudicated and whose accountability must rest with a named human. A better agent narrows its error rate; it does not earn the right to be unbounded. The containments of [Chapter 23](#) are not a temporary scaffolding to be removed when agents are good enough — they are the permanent architecture of using a probabilistic component in a domain that demands governance, and they hold at any level of agent capability.

What *will* change is how much a well-governed agentic process can do. As agents improve, the bounded tasks they can be trusted with will widen, the adjudication may grow more

sophisticated, the human-in-the-loop boundary may move — but it moves by deliberate, evidenced decision, governed like every other change to a regulated process, not by quietly removing the boundary because the agent seems good now. The institution that thrives with agentic AI is not the one that grants its agents the most autonomy; it is the one that has the most disciplined process architecture around them, and can therefore *safely* extend what its agents do as the evidence supports it.

139.5 The manuscript's argument, completed

This is the last technical chapter, and the agentic case completes the manuscript's single, consistent argument.

From [Chapter 1](#), the argument has been this: hyperautomation in banking and insurance is powerful and necessary, and it is safe only when it is *governed* — when the process is an explicit, modelled, auditable artefact, when decisions are deterministic and owned, when probabilistic components are contained, and when a named human is accountable for every consequential outcome. Every part has applied that argument: to processes, to decisions, to the cloud, to integration, to the Camunda mechanics, to the Java, to the data, to language models. The agent is the argument's hardest test — the most autonomous, most capable, most tempting-to-unleash component the manuscript has treated — and the argument holds: the agent, too, is governed by being a contained, modelled element of an explicit process, and it is *usable* in a regulated institution precisely because it can be.

That is the manuscript's claim, now complete. The technologies will go on changing — Camunda will release new versions, models will grow more capable, agentic patterns will mature. But the discipline does not change: explicit processes, governed decisions, contained probabilistic components, accountable humans, an honest audit record. An institution that holds to that discipline can adopt every advance safely; an institution that abandons it under the excitement of a new capability is, however modern its technology, building the un-governed automation this manuscript was written to argue against.

Worked example: orchestrated agents versus an agent free-for-all

Problem. An insurer wants three specialised agents — document, policy, and fraud — to jointly assess complex claims. Two architectures are proposed. Architecture A lets the three agents interact freely: they message each other, delegate, and converge on a joint assessment, which is then applied. Architecture B orchestrates them with a Camunda process: the process invokes each agent as a contained agent task, combines their outputs through a deterministic adjudication, and routes consequential cases to a human. A regulator later investigates a sample of 50 complex-claim assessments and, for each, asks the insurer to explain exactly how the assessment was reached and who was accountable. For how many of the 50 can each architecture answer?

Setup. An architecture can answer if there exists an artefact — a model, an audit record — that records how the assessment was reached and who was accountable. We assess each.

Working. *Architecture A — free-interacting agents.* The joint assessment emerged from the three agents' own interaction — their messages, delegations, and negotiation. No process modelled that interaction; no single artefact describes it; the audit record, at best, has fragments of what each agent did but not a governed account of how they combined or who was

accountable. For the regulator's 50 cases, the insurer can give a full, governed explanation of:

0 of 50.

Architecture B — orchestrated agents. Each assessment was produced by an explicit Camunda process: the model shows how the three agents were invoked and combined, the adjudication is a modelled step, the accountable human is a modelled user task, and the audit record — drawn from the executed model — captures all of it. For all 50 cases:

50 of 50.

Discussion. The contrast — 0 against 50 — is the manuscript's whole argument in its final form. Both architectures use the same three capable agents; both, on an ordinary day, produce assessments. Architecture A might even produce *slightly better* assessments, because freely-interacting agents can be more flexible than an orchestrated sequence — and that is exactly the seduction the manuscript has warned against from the start. Architecture A's flexibility is bought by abandoning the one thing a regulated institution cannot abandon: a governed, explicit, auditable account of how a consequential decision was reached and who is accountable for it. When the regulator asks — and in banking and insurance the regulator always, eventually, asks — architecture A cannot answer, for a single one of its cases, because the assessment emerged from an agent interaction that no artefact records. Architecture B answers for all fifty, because it kept the discipline: the agents are contained, the process is explicit, the adjudication and the accountable human are modelled, the audit record is honest. The lesson, and the manuscript's closing claim, is that the goal is never the most autonomous agents — it is the most *governable* agentic processes. An institution that remembers this can adopt every advance in agentic AI safely, and answer for all fifty cases; an institution that forgets it, dazzled by what free agents can do, builds something impressive that cannot answer for one.

Practical next steps

If your institution is moving toward multiple agents, establish now — before the agents are built — whether they will be orchestrated by an explicit process or left to interact freely. If the plan is free interaction, ask the regulator's question of it: when a consequential assessment is later questioned, what artefact explains how it was reached and who was accountable? If there is no good answer, the architecture is wrong, and the disciplined alternative — orchestrate the agents with a Camunda process — is the subject of this entire part.

Chapter in brief

Multiple agents are combined in one of two ways. Letting them interact freely — messaging, delegating, negotiating — produces emergent behaviour no artefact describes, which cannot be reviewed, audited, or explained: a hazard for a regulated institution. Orchestrating them with an explicit Camunda process — each agent a contained, modelled task, the process determining how they combine — keeps the multi-agent system governed. As agentic capability advances, the containments of **Chapter 23** do not become obsolete: a more capable agent is still probabilistic and still needs adjudication and accountability. The goal is never the most autonomous agents but the most governable agentic processes — the institution that holds that discipline can adopt every

advance safely.

PART XXVI

Security

The Security Posture of a Process Platform

140.1 Why security deserves a part of its own

[Chapter 35](#) treated authentication and authorization — how a caller proves who it is, and what it is then allowed to do. That chapter was necessary and is not repeated here. But security is larger than authentication, and for a banking or insurance platform it is large enough to need a part of its own. This part treats security as a *whole posture*: the full surface the platform exposes, the data it holds and how that data is protected, the secrets it depends on, and the threats a financial platform specifically faces.

The premise of the part is that a hyperautomation platform is an unusually *concentrated* thing to secure. It orchestrates the institution's most sensitive processes — lending, payments, claims — it holds, in its process state, a great deal of customer data, it has reach into the institution's most critical systems through its connectors and workers, and it can, through those same connectors, *act*: move money, bind cover, close cases. A platform that orchestrates the bank is a platform whose compromise compromises the bank. That concentration is why security earns a dedicated part.

140.2 The attack surface of the platform

Securing the platform begins with knowing its *attack surface* — the full set of points at which something outside could attempt to reach in. A platform team that has not enumerated the surface cannot have secured it, because security is only as good as its most overlooked entry point.

The platform's attack surface has several distinct parts. There are the *APIs* — the Process API, the Task API of [Chapter 36](#) and [Chapter 38](#) — through which clients start processes, complete tasks, query state. There are the *inbound connectors* of [Chapter 33](#) — webhooks, queue listeners — each of which is, by design, a point at which the outside world triggers the platform. There are the *web applications* — Operate, Tasklist — through which people sign in and interact. There are the *worker connections* — the points at which the institution's worker applications connect to the cluster. And there is the *infrastructure* itself — the Kubernetes layer, the cluster's network, the secondary store.

Each of these is a way in, and each must be secured deliberately. The discipline is enumeration first: list every point at which the platform can be reached, and then, for each, establish how it is authenticated, what it permits, and what would happen if it were compromised. A surface that has been enumerated can be secured; a surface that has not been enumerated has, almost certainly, a forgotten entry point, and the forgotten entry point is the one an attacker finds.

Figure [140.1](#) draws the surface — the points to enumerate and then secure one by one.

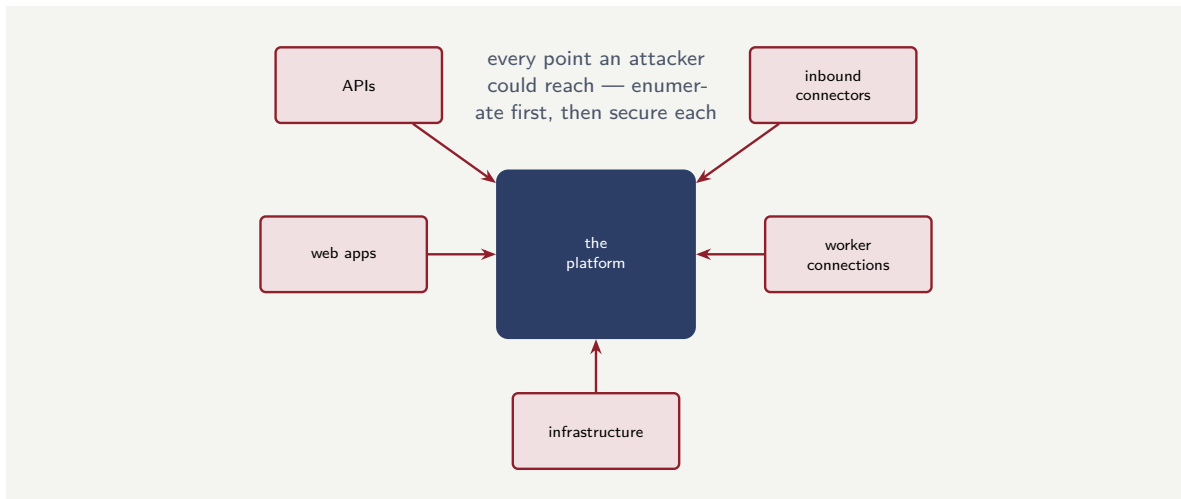


Figure 140.1. The attack surface of a Camunda platform. The APIs, inbound connectors, web applications, worker connections, and the infrastructure itself are all points at which the outside can reach in. Security begins with enumerating every one — the forgotten entry point is the one an attacker finds.

140.3 Defence in depth

A secure platform does not rely on a single barrier. It practises *defence in depth*: multiple, independent layers of protection, so that the failure or bypass of any one layer does not expose the platform, because another layer still stands.

The principle matters because every single barrier can fail. An authentication mechanism can have a flaw; a credential can leak; a network rule can be misconfigured. A platform that depends on *one* barrier is a platform one failure away from compromise. A platform with layered defences — network isolation *and* authentication *and* authorization *and* data protection *and* monitoring — can suffer the failure of one layer and still not be breached, because the attacker who slips past one layer meets the next.

Concretely, for a Camunda platform, the layers include: the *network* — the platform deployed in a private network, not exposed to the public internet, reachable only through controlled ingress; *authentication* — every caller, human or system, proving identity, per [Chapter 35](#); *authorization* — every authenticated caller limited to what they specifically may do; *data protection* — the data itself encrypted and minimised, the subject of the next chapter; and *monitoring* — the platform watched, so that an attempt that does get through is seen. No one of these is the platform's security; the platform's security is all of them, layered, so that no single failure is fatal.

Tip

Never let the platform's security rest on a single barrier. Every individual control — an authentication mechanism, a network rule, a credential — can fail or be bypassed. Defence in depth means layering independent controls — network isolation, authentication, authorization, data protection, monitoring — so the failure of any one is caught by the next. A platform secured by one strong barrier is one failure away from breach; a platform secured in depth is not.

140.4 Least privilege, everywhere

One principle runs through every layer and deserves to be stated as the part's spine: *least privilege*. Every identity on the platform — every human user, every worker application, every connector, every system that calls in — should have exactly the privileges its role genuinely requires, and not one privilege more.

The reason is the containment of damage. [Chapter 35](#)'s discussion of the shared service identity made the point in one place; least privilege generalises it. When an identity is compromised — and over a long enough time, some identity will be — the damage an attacker can do is exactly the privilege that identity held. An identity scoped to precisely its role limits the breach to that role's reach. An identity with broad privileges “to be safe,” or “because it was easier,” hands an attacker that breadth.

Least privilege is unglamorous and is constant work — every new worker, every new connector, every new user is a small act of deciding the minimum it needs — and it is the single most effective security discipline a platform has, because it does not try to prevent every compromise (which is impossible); it ensures that a compromise, when it happens, is *small*.

140.5 Security as a continuous discipline

Security is not a feature deployed once; it is a *continuous discipline*. Credentials rotate, tokens expire, vulnerabilities are discovered, and the threat landscape shifts. A platform whose security was configured at deployment and never revisited is a platform whose security degrades with every passing month. The discipline is periodic review: rotate secrets on a defined schedule, audit token lifetimes, review the permissions granted to each role, scan for dependencies with known vulnerabilities, and test the platform's authentication flow regularly. A security posture that is not actively maintained is not a posture at all; it is a snapshot that is growing staler by the day.

Worked example: the blast radius of over-privilege

Problem. A platform has a worker that performs one task: it reads a customer's address from the core banking system. Two configurations are compared. In configuration A, the worker's identity has been given broad access to the core banking system — read and write, across all customer data — because that was the standing service account available and using it was quickest. In configuration B, the worker's identity is scoped to exactly its need: read-only, and only the customer-address data. The worker's credential is later compromised by an attacker. Compare what the attacker can do with each.

Setup. The damage from a compromised credential is exactly the privilege that credential held. We enumerate it for each configuration.

Working. *Configuration A — broad access.* The attacker holds a credential with read and write access to all customer data in the core banking system. They can:

read every customer's data; *alter* every customer's data.

The blast radius is the entire core banking customer dataset, including the ability to change it. *Configuration B — least privilege.* The attacker holds a credential that can read customer-address data and nothing else. They can:

read customer addresses — and do nothing else at all.

The blast radius is one category of data, read-only.

Discussion. The same compromise — one leaked worker credential — is, in configuration A, a catastrophe touching every customer’s data with the power to alter it, and in configuration B, a contained and far less severe incident: the exposure of one category of data, with no ability to change anything. The worked example is built to show that the *difference is entirely a decision made before the compromise*, when the worker’s identity was scoped. Configuration A’s breadth was not needed — the worker reads addresses; it never writes anything and never touches non-address data — it was taken “because that was the account available” and “because it was quickest,” and those two phrases are how over-privilege almost always enters a platform: not through a decision to be insecure, but through a decision to be quick. The attacker who compromises configuration A’s credential is handed the breadth the platform team handed the worker for convenience. Configuration B’s worker does exactly the same job — reads addresses — with an identity that can do exactly that and nothing more, and so the same compromise is contained to exactly that. Least privilege does not make the worker less capable at its job; it makes the worker’s credential less valuable to an attacker. The discipline’s whole logic is here: a platform cannot guarantee no credential ever leaks, but it can guarantee, in advance, that a leaked credential is worth as little as possible — and that guarantee is made one scoped identity at a time, by declining the broad, convenient access every time the narrow, exact access would do.

Practical next steps

Take one worker or connector identity in your platform and compare what it *can* do against what its job actually *requires*. Any privilege it holds but does not use is blast radius handed to whoever compromises it. Scope it down to exactly its need — and make “exactly its need” the default for every new identity, because least privilege is built one identity at a time.

Chapter in brief

Security deserves a dedicated part because a hyperautomation platform is an unusually concentrated thing to secure — it orchestrates the institution’s most sensitive processes, holds much customer data, and can act on critical systems. Securing it begins with enumerating the full attack surface: APIs, inbound connectors, web applications, worker connections, and the infrastructure. The platform is then secured by defence in depth — multiple independent layers, so the failure of one is caught by the next — never by a single barrier. Running through every layer is least privilege: every identity scoped to exactly its role’s need, so that the inevitable eventual compromise of some credential is contained to as little as possible.

Protecting the Data the Platform Holds

141.1 The platform is full of sensitive data

The previous chapter treated the platform's perimeter and its identities. This chapter treats what is *inside*: the data. A Camunda platform running a bank's processes holds, at any moment, a great deal of highly sensitive information — customer identities, account details, financial positions, the substance of lending and claims decisions — and protecting that data is a distinct security discipline from defending the perimeter.

The data is held in more places than a casual view suggests, and naming them is the start of protecting them. It is in the *process variables* of every running instance, held in the engine. It is in the *secondary store*, where the history of every instance lives. It is in the *audit record*. It is in *backups*. It moves through *connectors* to and from external systems, and — as **the data part** described — it can be *streamed* onward to Kafka. Data protection that secures only one of these places has secured the data in one place and left it exposed in the others.

Figure 141.1 draws the six places the platform's sensitive data lives.

141.2 Encryption: in transit and at rest

The baseline of data protection is *encryption*, and it has two forms, both required.

Encryption in transit protects data while it moves — between a client and the platform's APIs, between a worker and the cluster, between the engine and the secondary store, between a connector and an external system. Every one of those connections carries sensitive data across a network, and every one must be encrypted, so that data intercepted in transit is unreadable. For a platform, this means every connection uses transport encryption — no plaintext link anywhere in the topology, internal links included, because an attacker who reaches the internal network must still not be able to read what crosses it.

Encryption at rest protects data while it is stored — the engine's broker storage, the secondary store's storage, the backups. Data at rest, encrypted, is data that an attacker who obtains the storage — a stolen disk, a copied volume, an exposed backup — still cannot read. Every store the platform writes to should be encrypted at rest, and the backups especially, because backups are deliberately copied to separate places and a backup is sensitive data sitting outside the platform's main defences.

Encryption is the baseline because it is the protection that holds even when other defences fail: an attacker who breaches the perimeter, who obtains a disk, who intercepts a link, still meets unreadable data. It is necessary and it is not sufficient — the rest of this chapter is what encryption alone does not do.

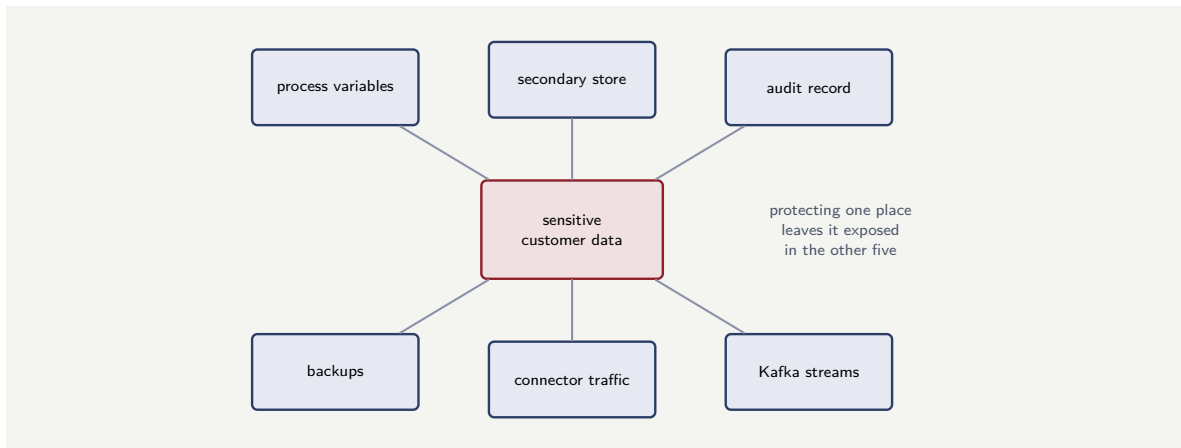


Figure 141.1. Where the platform’s sensitive data lives. The same customer data is held in process variables, the secondary store, the audit record, backups, connector traffic, and Kafka streams. Data protection that secures only one of these places has left the data exposed in the rest.

141.3 Data minimisation: the data not held cannot leak

The most powerful data-protection discipline is not a protection applied to data — it is *holding less data*. The manuscript has invoked data minimisation in many chapters; this is where it is stated as the security principle it fundamentally is.

The logic is simple and absolute: *data the platform does not hold cannot be breached, cannot leak, cannot be misused*. Every piece of sensitive data the platform holds is a piece of data that must be protected, that appears in backups, that widens the consequence of any breach. Data the platform never took on, or held only as long as it genuinely needed and then discarded, is data that carries no risk because it is not there.

For a Camunda platform this has concrete applications, each met earlier in the manuscript and gathered here. A process should carry, in its variables, the data the process genuinely needs — not everything available, poured in for convenience ([the worker chapter](#)’s over-returning worker, [the completing-tasks chapter](#)’s over-stuffed completion). A process should reference large or especially sensitive data — documents, full records — by an identifier into a controlled store, rather than copying it into process variables. A connector or a Kafka stream should carry the minimum its consumer needs, not the whole process state. And the platform’s *retention* should discard process data when the genuine need for it — operational, regulatory — has ended, rather than holding everything forever.

Data minimisation is the discipline that makes every other data protection *smaller in scope* — there is less to encrypt, less in the backup, less in the breach — and it is the one protection that cannot fail, because data that was never held has nothing to fail about.

The most reliable protection for a piece of sensitive data is not to hold it. Every variable a process carries, every field a connector passes, every record copied into the platform is data that must then be encrypted, appears in every backup, and widens every breach. Before adding data to a process, a connector, or a stream, ask whether the platform genuinely needs it — and reference large or sensitive data by an identifier rather than copying it in. Data minimisation is the one data protection that cannot fail, because data never held cannot leak.

141.4 Data and the audit record

A particular tension deserves its own treatment: the platform's *audit record* must be complete enough to satisfy a regulator, and minimal enough to satisfy data protection, and these pull against each other.

The audit record exists, as the manuscript has stressed throughout, so that the institution can later say exactly what happened and who was accountable. That requires the record to capture the substance of decisions, the identities of the people who made them, the data the decisions rested on. But that same record is then a large, long-retained store of sensitive data — and a store that is retained for years, by regulatory necessity, is a store whose protection must hold for years.

The discipline is to be *deliberate* about what the audit record holds. It must hold what genuine accountability requires — the decision, the decider, the time, the inputs that mattered. It should not hold what mere convenience swept in — the over-stuffed completion's UI state, the worker's unread raw text. An audit record disciplined to exactly what accountability needs is both a better audit record — precise, searchable, not buried in noise — and a smaller data-protection liability. The completeness a regulator needs and the minimisation data protection needs are not, in the end, opposed: both are served by an audit record that holds exactly what accountability requires and nothing else.

141.5 Data as a platform capability

The data the platform produces is not a by-product; it is a *capability*. Every process instance generates a stream of facts: what happened, when, to whom, with what outcome. That stream, properly captured and governed, feeds operational dashboards, management reporting, regulatory submissions, and the machine-learning models that improve the processes themselves. A platform that runs processes but does not capture their data in a governed, queryable form has built half a platform: it automates the work but not the institution's ability to learn from it.

Worked example: the document copied into a thousand places

Problem. A claims process handles a customer's uploaded identity documents. The process, as built, reads each document's full content into a process variable so later steps can access it. The document averages 2 MB. The process runs 300,000 times a year, and process data is retained for 7 years. The document content, held as a process variable, is therefore present in: the engine's state while the instance runs, the secondary store, every backup, and the audit record — and remains in the retained history for 7 years. A proposed change instead stores each document once in a dedicated, access-controlled document store and carries only a reference — a few bytes — in the process. Assess the data-protection difference.

Setup. We consider how widely the sensitive document content is spread under each design, and what that means for breach exposure.

Working. *Current design — content in a process variable.* The 2 MB document is copied into the process state and therefore propagates into the secondary store, the backups, and the audit record, and stays in the 7-year retained history. Over a year:

$$300,000 \times 2 \text{ MB} = 600 \text{ GB of identity-document content,}$$

spread across the engine, the secondary store, every backup, and the audit record — and held for 7 years, so on the order of

$$7 \times 600 \text{ GB} \approx 4.2 \text{ TB}$$

of the most sensitive category of customer data, replicated into every one of the platform's data stores. *Proposed design — a reference.* The document content lives in *one* access-controlled store. The process, the secondary store, the backups, and the audit record carry only a few-byte reference. The sensitive content exists in exactly one controlled place.

Discussion. The current design has taken the institution's most sensitive category of customer data — identity documents — and copied it into every data store the platform has, and into seven years of backups and history. A breach of any one of those stores — the secondary store, an exposed backup, the audit archive — is now a breach of identity documents, because the documents are in all of them. The proposed design confines the document content to one store, built for exactly that purpose, with access control suited to it; every other part of the platform holds only a reference, and a reference is not sensitive — a breach of the secondary store or a backup exposes references, not documents. The worked example's lesson is the chapter's central one made concrete: *where* sensitive data lives is a security decision, and copying it for convenience is a security decision made carelessly. The current design did not intend to spread identity documents across four stores and seven years of backups — it intended to make the documents easy for later steps to reach — but the effect is the same as if it had intended it, because the data is now there to be breached. The reference design keeps the documents in one place that can be properly defended and properly access-controlled, and keeps everywhere else free of them. Data minimisation, at its most concrete, is this: hold the sensitive thing in one controlled place, and let everything else hold a pointer.

Practical next steps

For one process in your platform that handles documents or large sensitive records, check whether it carries their full content in process variables or a reference to a controlled store. Content in variables propagates the sensitive data into the secondary store, every backup, and the retained audit history. Moving to a reference confines the sensitive data to one defensible place — one of the highest-value data-protection changes a platform can make.

Chapter in brief

A Camunda platform holds much sensitive data — in process variables, the secondary store, the audit record, backups, and connector and stream traffic. Protecting it has three disciplines. *Encryption*, in transit and at rest, is the baseline that holds even when other defences fail. *Data minimisation* is the most powerful protection, because data not held cannot leak: a process should carry only what it needs and reference large or sensitive data rather than copying it in. And the *audit record* must be deliberate — complete enough for accountability, minimal enough for data protection, which are reconciled by holding exactly what accountability requires and no convenience extras.

Secrets, Threats, and the Secure Operating Discipline

142.1 The security that is never finished

The two preceding chapters treated the platform's perimeter and its data — both, in a sense, things that can be *set up* securely. This final security chapter treats the parts of security that are never set up once and finished: the management of *secrets*, the specific *threats* a financial platform faces, and the ongoing *operating discipline* that keeps a platform secure not on its launch day but in its third year.

142.2 Secrets: the keys to everything

A Camunda platform depends on *secrets* — credentials, keys, tokens: the worker applications' credentials for the cluster and for downstream systems, the connectors' credentials for external services, the keys for encryption, the tokens for authentication. These secrets are, collectively, the keys to the platform, and how they are managed is a security discipline in its own right because a mismanaged secret undoes every other protection.

The disciplines of secret management are few and absolute. A secret is *never* placed in source code, in a configuration file committed to a repository, or in a process model — because a repository's history is permanent, and a secret committed once is a secret leaked forever, retrievable from the history even after it is removed. A secret lives in a dedicated *secrets-management service* — a vault built for the purpose — and is *injected* into the component that needs it at runtime, from that service, so the secret exists in the running component's memory but never in its artefacts. A secret is *rotatable* — changed periodically, and changeable quickly if it is suspected compromised — which a secret hard-coded into a dozen places is not. And access to each secret is itself *least-privilege*: a component can retrieve the secrets it needs and no others.

This discipline has appeared, in passing, in many chapters — the `application.yaml` that holds a placeholder not a credential, the connector that fetches its credential from the secrets store. This chapter states it as the security principle it is: the secrets are the keys to everything, and a platform that manages them casually has, in effect, no security, because the casually-managed secret is the way past all the rest.

A secret committed to a repository — in code, in a configuration file, in a process model — is leaked permanently: a repository's history is forever, and the secret is retrievable from it even after deletion. Secrets live in a dedicated secrets-management service and are injected at runtime, never stored in any artefact. A platform that manages its secrets casually has no real security, however well-defended its perimeter — because the leaked secret is the key that opens every other lock.

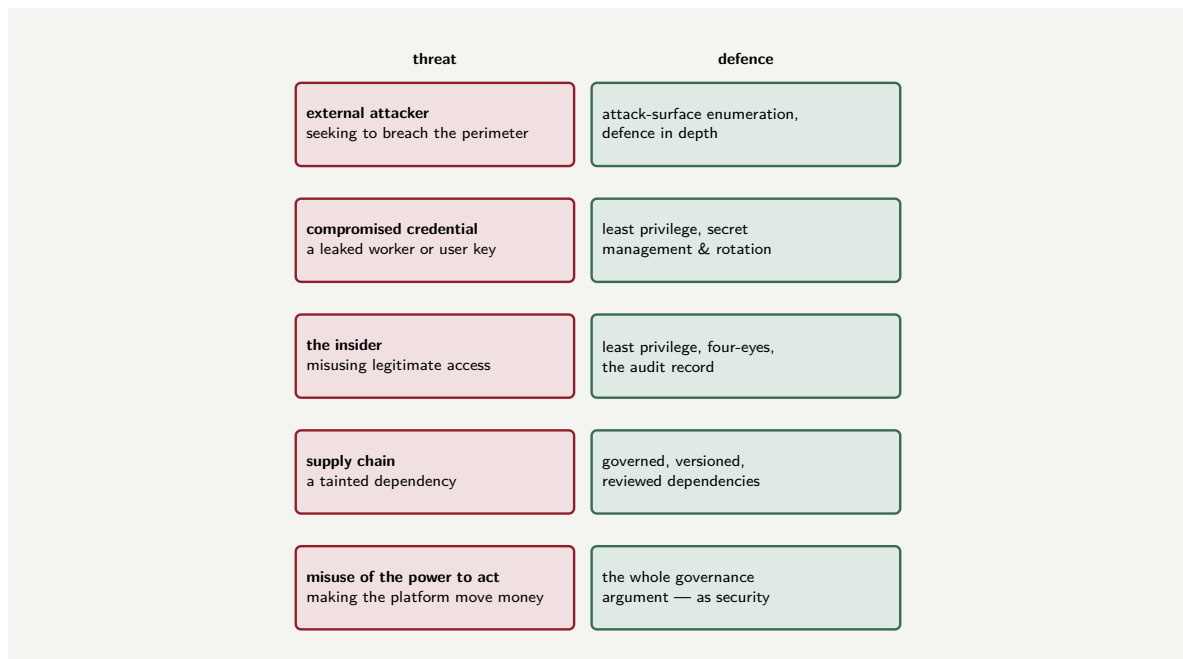


Figure 142.1. The threats a financial process platform faces, and the defence each calls for. Naming the threats makes the defence concrete — from the external attacker met by defence in depth, to the misuse of the platform’s own power to act, met by the whole of the manuscript’s governance argument.

142.3 The threats a financial platform faces

Security is sharpened by naming the *specific threats* a platform faces, because a defence designed against named threats is concrete where a defence against “insecurity” in general is vague. A Camunda platform in a bank or insurer faces several distinct threats worth naming.

There is the *external attacker* seeking to breach the perimeter — to reach the APIs, the inbound connectors, the infrastructure — and the defences against this are the attack-surface enumeration and defence in depth of the first chapter of this part.

There is the *compromised credential* — a worker’s, a connector’s, a user’s credential obtained by an attacker — and the defence is the least privilege that makes a compromised credential worth as little as possible, and the secret management that makes credentials hard to obtain and quick to rotate.

There is the *insider* — a person with legitimate access who misuses it — and the defences are least privilege again, the four-eyes controls of [Chapter 30](#), and above all the *audit record*, which ensures an insider’s actions are attributable and seen.

There is the *supply-chain threat* — a vulnerability or a malicious change in a component the platform depends on, a connector, a library — and the defence is governing what the platform is built from: known, versioned, reviewed dependencies, and the infrastructure-as-code of the previous part.

And there is the threat specific to this platform’s nature: the *misuse of the platform’s own power to act*. The platform can move money and bind cover; an attacker who can make it do so — by compromising a process, a worker, an agent — can cause direct financial harm. The defence is every governance discipline this manuscript has argued: contained agents, adjudicated decisions, accountable humans, idempotent and least-privileged workers. The platform’s power to act is its value and its danger, and the manuscript’s whole argument for governance is, seen from here, a security argument.

Figure 142.1 sets the named threats beside their defences.

142.4 The secure operating discipline

Security is not a state the platform is launched in; it is a discipline the platform is operated with. Three ongoing practices keep a launched platform secure.

The first is *patching*. The platform and its dependencies have vulnerabilities discovered over time, and the fixes must be applied — promptly, which means the platform must be *patchable* without disruption, which is itself a reason for the resilient, replicated topology of the infrastructure part. A platform running a years-old unpatched version is a platform with known, published holes.

The second is *monitoring for the security event*. The observability of Chapter 125 watches the platform's health; security monitoring watches for the signs of attack — the unusual access pattern, the authentication failures, the connector behaving strangely. A platform that is watched sees an attack in progress; a platform that is not, learns of the breach afterward.

The third is the *drill*. Just as the backup must be tested by a real restore, the platform's response to a security incident must be tested by a real exercise — so that, when an incident happens, the institution executes a practised response rather than improvising. A security capability that has never been exercised is, like an untested backup, an assumption.

142.5 Security as a continuous discipline

Security is not a feature deployed once; it is a *continuous discipline*. Credentials rotate, tokens expire, vulnerabilities are discovered, and the threat landscape shifts. A platform whose security was configured at deployment and never revisited is a platform whose security degrades with every passing month. The discipline is periodic review: rotate secrets on a defined schedule, audit token lifetimes, review the permissions granted to each role, scan for dependencies with known vulnerabilities, and test the platform's authentication flow regularly. A security posture that is not actively maintained is not a posture at all; it is a snapshot that is growing staler by the day.

Worked example: the secret in the repository

Problem. A platform team, under deadline pressure, builds a connector to a payment-rail system. To get it working quickly, an engineer places the rail's API credential directly in a configuration file and commits it to the team's source repository, intending to "move it to the vault later." Later does not come; the credential stays in the committed file for eight months. The repository is internal but accessible to several hundred employees across the institution. The credential grants the ability to submit payments to the rail. Assess the exposure this created, and what the discipline required.

Setup. We consider who could reach the credential, for how long, what it permitted, and why removing it later would not undo the exposure.

Working. For the eight months the credential sat in the committed file, it was readable by everyone with access to the repository:

several hundred employees, for 8 months,

and the credential permitted the submission of payments — a directly financial capability. Crucially, even when the credential is eventually noticed and removed from the current files,

it remains in the repository's *history*:

retrievable from the commit history — removal does not undo the leak.

The only true remedy is to treat the credential as compromised and *rotate* it — issue a new one and invalidate the old.

Discussion. The committed credential was a serious security incident from the moment it was committed, even though nothing visibly went wrong for eight months — and the worked example is built to make several points the discipline depends on. First, the exposure is not “an engineer saw a credential”; it is that a payment-submitting capability was readable by several hundred people for eight months, any one of whom could have copied it, and the institution has no way to know whether any did. Second, the “move it to the vault later” intention — entirely sincere — is exactly how this incident type always happens: not through a decision to be insecure, but through a deadline and a deferral, and the deferral that is never undone. The discipline's rule is absolute precisely because the well-intentioned exception is the failure mode: a secret is *never* committed, not even temporarily, because “temporarily” becomes eight months and the repository's history makes even “temporarily” permanent. Third, and most important: removing the credential from the current files does *not* fix the incident, because the repository's history retains it forever — the only real remedy is to assume the credential is compromised and rotate it, which is itself only possible because a well-managed secret is rotatable. The whole of secret management is visible in this one example: the secret should have been in the vault from the first moment, injected at runtime, never in any committed artefact — and the reason is not procedural fussiness but that a committed secret is a permanently leaked secret, and the keys to a payment rail are not a thing a bank can afford to leak.

Practical next steps

Have your platform team check, honestly, whether any secret — a credential, a key, a token — currently sits in a configuration file, a process model, or source code, committed to a repository. Any that does should be treated as compromised: rotated, and moved to the secrets-management service. And establish the rule, without the well-intentioned exception, that a secret is never committed even temporarily — because the repository's history makes “temporarily” permanent.

Chapter in brief

Some security is never set up once and finished. *Secrets* — the credentials and keys that are the platform's keys to everything — must live in a dedicated secrets-management service, be injected at runtime, never appear in any committed artefact (a repository's history makes a leak permanent), and be rotatable. The platform faces specific, nameable *threats*: the external attacker, the compromised credential, the insider, the supply chain, and the misuse of the platform's own power to act — against which the manuscript's whole governance argument is, in the end, a security argument. And security is an ongoing *operating discipline*: prompt patching, monitoring for the security event, and a tested incident-response drill — because security is how a platform is operated, not a state it is launched in.

PART XXVII

Security and API Token Management with Camunda

Authenticating to Camunda: the Token Model

143.1 Why API token management deserves a part

The Security part treated security as a whole posture — identity, access, data, the agentic surface. This part narrows to one specific, concrete, and operationally critical subject: *how the things that call Camunda prove who they are*, and how the *tokens* that carry that proof are managed.

This deserves a part of its own because, in a real Camunda 8 platform, almost *everything* that interacts with the engine does so over an API, and every one of those API calls must be authenticated. The Java applications that start processes, the job workers that take work, the operations consoles, the custom worklists, the connectors, the monitoring tools — each is a client, and each must authenticate. The mechanism that makes that work is a *token*, and an institution that manages those tokens well has a secure, operable platform; one that manages them badly has either a platform that does not work or a platform full of security holes. Digital API token management is not a side topic — it is the daily reality of running Camunda.

143.2 The token model

Camunda 8's API authentication rests on a token model that follows the industry-standard *OAuth* approach, and a practitioner must understand its shape.

The model has these elements. There is a *client* — any application or component that wants to call Camunda. There is an *authorization server* — the component that issues tokens. And there is the *resource* — the Camunda API the client wants to reach: the Zeebe API to start processes, the Operate API to query them, the Tasklist API for human tasks.

The flow is as follows. The client does not send a password with every API call. Instead, the client first *authenticates to the authorization server* — presenting its credentials — and receives, in return, an *access token*. The client then includes that token with each API call to Camunda, and Camunda, trusting tokens from its authorization server, accepts the call. The token is the proof of identity that travels with the request.

The access token is, concretely, a *JWT* — a JSON Web Token: a compact, signed token that carries, inside it, the claims about who the client is and what it may do. All Camunda 8 API requests are authenticated this way: generate a JWT, and include it in each request, conventionally in an authorization header as a *bearer* token.

Figure 143.1 draws Camunda's token model.

143.3 Why tokens, not passwords

It is worth being clear why the model uses tokens at all, rather than having each client simply send a username and password with every call — because the reasons are the reasons the



Figure 143.1. Camunda’s OAuth token model. The client authenticates to the authorization server with its credentials (1) and receives an access token (2); it then includes that token — a signed JWT — with each call to the Camunda API (3). The token is the proof of identity that travels with the request.

model is sound.

A token is *short-lived*. Unlike a password, which is valid until changed, an access token has a defined, limited validity period — it expires. A token that leaks is therefore dangerous only until it expires, where a leaked password is dangerous until someone notices and changes it.

A token is *scoped*. The token carries, in its claims, exactly what the client is permitted to do — which Camunda APIs, which permissions — so a token is not a master key but a key cut for a specific door.

A token is *verifiable without a shared secret at the door*. Because the JWT is signed by the authorization server, Camunda can verify the token is genuine without itself holding the client’s credentials — the credentials stay between the client and the authorization server.

And the model *centralises* the issuing of identity. There is one place — the authorization server — that decides who gets a token and what it permits, rather than credentials scattered across every client-to-Camunda relationship.

Tip

Camunda 8 authenticates API calls with tokens, not passwords, and the reasons are the security properties to preserve. A token is *short-lived* — it expires, so a leaked token is dangerous only briefly; *scoped* — it carries exactly what the client may do, so it is not a master key; and *verifiable by signature* — Camunda checks it without holding the client’s credentials. An institution that undermines these properties — by making tokens long-lived, broadly scoped, or by treating them like static passwords — has kept the token mechanism while discarding the security it exists to provide.

143.4 Security as a continuous discipline

Security is not a feature deployed once; it is a *continuous discipline*. Credentials rotate, tokens expire, vulnerabilities are discovered, and the threat landscape shifts. A platform whose security was configured at deployment and never revisited is a platform whose security degrades with every passing month. The discipline is periodic review: rotate secrets on a defined schedule, audit token lifetimes, review the permissions granted to each role, scan for dependencies with known vulnerabilities, and test the platform’s authentication flow regularly. A security posture that is not actively maintained is not a posture at all; it is a snapshot that is growing staler by the day.

Worked example: the token treated as a password

Problem. A team integrates an application with Camunda 8. The application obtains an access token, and the team — finding token expiry inconvenient — configures the authorization server to issue this client a token valid for ten years, then stores that single long-lived token in the application’s configuration file and uses it for every call, indefinitely. A security architect says the team has dismantled the token model. Explain.

Setup. We compare what the team has created against the three security properties of a token.

Working. A ten-year token stored statically in a configuration file and used indefinitely has none of the properties that make a token a token. It is not *short-lived* — ten years is, for security purposes, forever:

a ten-year token that leaks = a ten-year compromise.

It is stored statically in a configuration file, so it is exposed exactly as a password in a configuration file would be — the credential-in-config problem the security part warned against. And it is used as a single, permanent, fixed string presented on every call. In every operative respect:

the team has turned the access token back into a password,

and a long-lived password in a config file is precisely what the token model was designed to replace. The team kept the *form* of a token — a JWT in a header — and discarded its *substance*.

Discussion. The team treated token expiry as an inconvenience to engineer away, and in doing so dismantled the security the token model exists to provide — the worked example’s point. A token’s short life is not a flaw to be worked around; it is the feature that bounds the damage of a leak. By issuing a ten-year token, the team made a single leaked string a decade-long compromise; by storing it statically in a config file, they exposed it exactly as a password would be exposed; by using it unchanged forever, they turned a dynamic, expiring credential into a static, permanent one. The result is a JWT that is a token in name only and a long-lived password in every way that matters. The correct design embraces the token model rather than defeating it: the application holds its *client credentials* — the means to *obtain* a token — and uses them to request a *fresh, short-lived* access token from the authorization server, automatically, whenever it needs one or the current one nears expiry. The expiry the team found inconvenient is handled by the client requesting a new token, not by making the token eternal — and the next chapter treats exactly that lifecycle. The lesson is the chapter’s tip: the token model’s value is in tokens being short-lived, scoped, and dynamically obtained, and an institution that engineers those properties away to avoid the bother of token refresh has paid the cost of an OAuth model and kept the security of a static password.

Practical next steps

Check how your applications hold their Camunda credentials. If any holds a long-lived access *token* as a static string, it has turned a token back into a password. An application should hold its *client credentials* and use them to obtain fresh, short-lived tokens dynamically — the next chapter treats that lifecycle. A long-lived token in a config file is a security hole wearing a JWT.

Chapter in brief

In a Camunda 8 platform almost everything interacts over an API, and every API call must be authenticated — which makes API token management a daily operational reality, not a side topic. Camunda's model follows OAuth: a *client* authenticates to an *authorization server* with its credentials, receives a short-lived *access token* — a signed JWT — and includes that token with each call to a Camunda API, conventionally as a bearer token. Tokens are used rather than passwords for concrete reasons: a token is short-lived, so a leak is bounded; scoped, so it is not a master key; and verifiable by signature, so Camunda need not hold the client's credentials. An institution that makes tokens long-lived or static keeps the mechanism while discarding the security it exists to provide.

API Clients, Credentials, and the Token Lifecycle

144.1 From the model to the practice

The previous chapter set out Camunda's token model. This chapter treats the practice: how *API clients* are created, what *credentials* they hold, and how the *token lifecycle* — obtaining, using, refreshing tokens — is managed in a running platform.

144.2 The API client and its credentials

To let an application call Camunda, an institution creates an *API client* — a registered identity for that application, defined in Camunda's administration tooling: the Camunda Console for the managed offering, or Camunda Identity for a self-managed platform.

Creating an API client produces a set of *client credentials*, and the central pair is the *client ID* and the *client secret*. The client ID names the client; the client secret is the secret it uses to prove it is that client. Together they are what the application presents to the authorization server to obtain a token — they are the application's *means of obtaining identity*, distinct from the tokens themselves.

Two practical facts about client credentials matter enormously.

The first concerns the *client secret*: when an API client is created, the client secret is shown *once*, and only once. It must be captured at creation and stored securely; it cannot be retrieved again afterward. An institution that does not capture it must create a new client.

The second concerns *scope*. When the API client is created, it is granted *permissions* — which Camunda components and operations it may use: Zeebe, Operate, Tasklist, Optimize, and what within each. The client's tokens will be valid only for what its permissions allow. This is where the *scoping* of [Chapter 140](#) is actually set: the API client is given the permissions it genuinely needs, and no more.

Figure [144.1](#) distinguishes the long-lived credentials from the short-lived tokens.

144.3 The token lifecycle

With credentials in hand, the running application manages a continuous *token lifecycle*, and understanding it is the heart of this chapter.

The application *obtains* a token: it sends its client ID and client secret to the authorization server — the OAuth *client-credentials* grant, the flow for an application authenticating as itself rather than on behalf of a user — and receives an access token. The application *uses* the token: it includes it with each Camunda API call. The token *expires*: after its validity period, it is no longer accepted. And so the application *refreshes*: as the current token nears or reaches expiry, the application obtains a new one, the same way, and continues.

The crucial practical point is that this lifecycle should be *handled automatically by the client*,

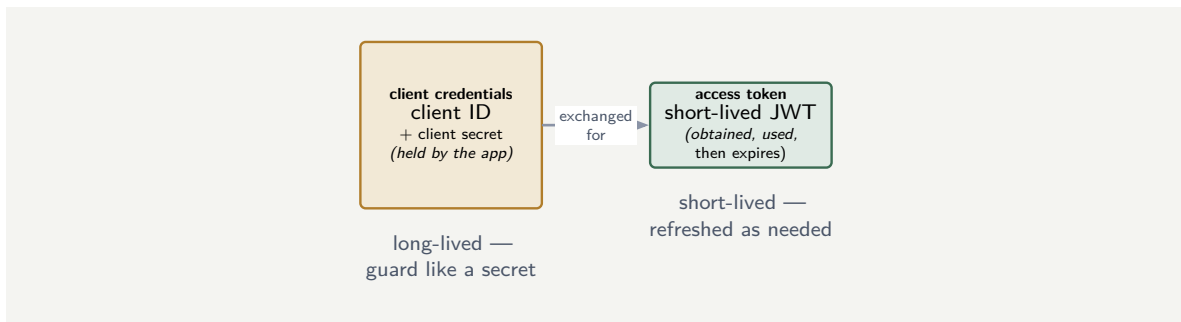


Figure 144.1. Credentials versus tokens. The client credentials — client ID and client secret — are long-lived and held by the application; they are exchanged with the authorization server for short-lived access tokens. The two are distinct things, managed differently: the credentials guarded as a secret, the tokens refreshed as they expire.

not managed by hand. Camunda's own client libraries do exactly this: a Camunda client, given the client credentials, contains a credentials provider that obtains tokens, attaches them to requests, and refreshes them on expiry — the application code simply makes API calls and the token lifecycle is handled beneath it. An institution building a custom client should build the same: the credentials configured once, the token lifecycle automatic. Token management done right is token management the application's developers rarely have to think about, because the client library is doing it.

144.4 The token lifecycle in code

A machine client obtains an access token through the client-credentials flow — a direct exchange of client ID and secret for a token. The token has a defined lifetime, and a well-built client refreshes it before expiry.

Listing 144.1. Obtaining an access token through the client-credentials flow: the machine client exchanges its credentials for a time-limited token.

```

1 // client-credentials flow: machine-to-machine
2 HttpResponse<String> r = httpClient.send(
3     HttpRequest.newBuilder()
4         .uri(URI.create(tokenEndpoint))
5         .header("Content-Type",
6             "application/x-www-form-urlencoded")
7         .POST(HttpRequest.BodyPublishers.ofString(
8             "grant_type=client_credentials"
9             + "&client_id=" + clientId
10            + "&client_secret=" + clientSecret
11            + "&audience=" + audience))
12         .build(),
13     HttpResponse.BodyHandlers.ofString());
14
15 String accessToken = parseToken(r.body());
16 // use before expiry; refresh, do not cache indefinitely
  
```

The token returned has a `expires_in` field, typically 300 seconds. A production client must track that expiry and refresh before it elapses — a token used past its lifetime is rejected. Never cache a token indefinitely.

144.5 Security as a continuous discipline

Security is not a feature deployed once; it is a *continuous discipline*. Credentials rotate, tokens expire, vulnerabilities are discovered, and the threat landscape shifts. A platform whose security was configured at deployment and never revisited is a platform whose security degrades with every passing month. The discipline is periodic review: rotate secrets on a defined schedule, audit token lifetimes, review the permissions granted to each role, scan for dependencies with known vulnerabilities, and test the platform's authentication flow regularly. A security posture that is not actively maintained is not a posture at all; it is a snapshot that is growing staler by the day.

Worked example: the secret that was never captured

Problem. A team creates a Camunda 8 API client for a new application. In the administration tooling, the client is created, and the screen displays the client ID and the client secret. The team copies the client ID, notes the client secret is “in the tooling, we can get it later,” and closes the screen. Two days later, configuring the application, they return to retrieve the client secret — and cannot find it anywhere in the tooling. Explain what has happened and what the team must now do.

Setup. We recall the chapter's first practical fact about the client secret.

Working. The client secret, as the chapter stated, is shown *once* — at the moment the API client is created — and is *not retrievable afterward*. The team's assumption — “it is in the tooling, we can get it later” — is exactly the assumption the one-time display is designed to defeat:

client secret displayed once + not captured $\Rightarrow$ not recoverable.

The client ID, which the team did copy, is still available — it is not secret. But the client secret is gone, and an API client whose secret no one holds cannot authenticate. The team's only remedy is to *create a new API client*, capture its secret this time, and grant it the same permissions — and, ideally, remove the unusable old one.

Discussion. The team walked into a small but instructive trap, and the worked example exists to make the chapter's first practical fact concrete and memorable. The one-time display of the client secret is not a quirk of the tooling; it is a deliberate security design. A secret that can be retrieved from a screen at any later time is a secret sitting permanently in that screen, visible to anyone who can reach it; a secret shown once and never again exists, after that moment, *only where the team deliberately stored it* — which is exactly the control an institution wants over a credential. The team's error was treating the secret like the client ID — a value to look up later — when the secret is the genuinely sensitive half of the credential pair and behaves accordingly. The cost here was small: a new client must be created, which is inconvenient but not damaging. The deeper lesson is what the episode teaches about handling the secret *once captured*. The client secret must, at the moment of creation, be captured and placed directly into the institution's secrets-management service — the vault of [Chapter 142](#) — not pasted into a document, an email, or a chat message while “we sort out where it goes.” The one-time display gives the institution exactly one clean opportunity to route the secret straight from creation into proper custody, and the disciplined practice is to take that opportunity. A team that treats the secret casually at creation either loses it, as here, or — worse — leaves it somewhere insecure.

Practical next steps

When your institution creates a Camunda API client, treat the moment of creation as the one chance to capture the client secret — and route it directly into your secrets-management service, not into a document or a message. If a secret has been lost, create a fresh client, capture the secret properly this time, and remove the unusable one. The one-time display is a control; use it as one.

Chapter in brief

To let an application call Camunda, an institution creates an *API client* — a registered identity, defined in the Camunda Console or Camunda Identity — which yields *client credentials*: a client ID and a client secret. Two facts matter: the client secret is shown only once at creation and must be captured then; and the client is granted scoped permissions, which is where least-privilege is actually set. With credentials in hand, the application runs a continuous *token lifecycle* — obtaining a token via the OAuth client-credentials grant, using it on each call, and refreshing it on expiry — and this lifecycle should be handled automatically by the client library, not by hand. Credentials are long-lived and guarded; tokens are short-lived and refreshed.

Token Management as a Security Discipline

145.1 Tokens across the platform's life

The previous chapters treated the token model and the token lifecycle of a single client. This chapter treats token management as an ongoing *security discipline* across the whole platform — because a real Camunda 8 platform has many API clients, and the security of the platform is, in significant part, the security of how all those clients' credentials and tokens are managed.

145.2 The credential is the asset to protect

The first discipline follows from [Chapter 143](#)'s distinction: the short-lived token is not the main thing to protect — the long-lived *client credential*, especially the client secret, is. A leaked token expires soon; a leaked client secret can be exchanged for fresh tokens indefinitely, until the secret is revoked.

Therefore the client secret of every API client must be treated as a first-class secret: stored in the institution's secrets-management service, never in source code, never in a plain configuration file, never in a document or a message — the discipline of [Chapter 142](#), applied specifically and without exception to Camunda client secrets. The secret is injected into the application at runtime from the vault; it does not live in the application's artefacts.

145.3 Rotation, revocation, and the departing client

A client secret, like any credential, must be *rotatable* and *revocable*, and the discipline must include both.

Rotation: client secrets should be changed periodically, and an institution should be able to rotate the secret of any API client — create a new secret, move the application to it, retire the old — without disruption. A secret that has never been rotated and cannot be is a growing risk.

Revocation: when a client should no longer have access — an application decommissioned, a secret suspected compromised — its API client must be promptly *disabled or deleted*, so its credentials can obtain no further tokens. An API client that outlives the application it was created for is an unused door left unlocked.

This is the joining of token management to the platform's wider lifecycle. When an application is retired, retiring its API client is part of the work — just as, for human identity, [Chapter 48](#)'s discipline held that a departing employee's tasks and access must be handled. A platform accumulates API clients over years, and without a discipline of revoking the ones no longer needed, it accumulates live credentials for applications that no longer exist.

A Camunda platform accumulates API clients over its life, and each one's client secret is a live credential that can obtain tokens until it is revoked. Without a deliberate discipline, the platform ends up with secrets for decommissioned applications still valid, secrets that have never been rotated, and no clear inventory of which client can do what. Treat every client secret as a vaulted first-class secret; rotate secrets periodically; and revoke an API client the moment the application it served is retired. An unrevoked client is an unlocked door to a building no one uses any more.

145.4 Scope, inventory, and least privilege

The final discipline is governing what the platform's API clients *can do*.

Every API client is created with *scoped permissions*, and the discipline of [Chapter 140](#) — least privilege — applies directly: an API client should be granted exactly the Camunda permissions its application genuinely needs, and no more. A job worker that only completes jobs does not need permission to deploy processes; a monitoring client that only reads does not need permission to write. The blast radius of a leaked credential is set by the scope of the client it belongs to, so narrow scopes are narrow blast radii.

And the institution needs an *inventory*: a known, maintained record of which API clients exist, which application each serves, what each is permitted to do, and when each secret was last rotated. An institution that cannot list its API clients cannot govern them — cannot know which to revoke, which are over-scoped, which hold stale secrets. Token management as a security discipline rests, in the end, on the institution being able to see its whole population of API clients and hold each to the standards this part has set.

145.5 Security as a continuous discipline

Security is not a feature deployed once; it is a *continuous discipline*. Credentials rotate, tokens expire, vulnerabilities are discovered, and the threat landscape shifts. A platform whose security was configured at deployment and never revisited is a platform whose security degrades with every passing month. The discipline is periodic review: rotate secrets on a defined schedule, audit token lifetimes, review the permissions granted to each role, scan for dependencies with known vulnerabilities, and test the platform's authentication flow regularly. A security posture that is not actively maintained is not a posture at all; it is a snapshot that is growing staler by the day.

Worked example: the worker credential that could deploy

Problem. A bank's Camunda platform has a job worker — a small application whose only job is to take a particular type of job, do the work, and complete it. When its API client was created, the team, to keep things simple, granted it *full* permissions across Zeebe, including the permission to deploy new process definitions. A year later, the worker's client secret is compromised through a leak. Assess the consequence, and what the discipline of this chapter would have changed.

Setup. We consider what the leaked credential can do, given the scope the API client was granted.

Working. The leaked client secret can be exchanged for tokens, and those tokens carry whatever the API client was permitted to do. The client was granted *full* Zeebe permissions,

including *deploying process definitions*. So an attacker with the leaked secret can do far more than the worker ever did:

worker's actual need: complete one job type;

worker's granted scope: full Zeebe, including deploy;

leaked-credential blast radius: full Zeebe, including deploy.

An attacker could deploy a malicious process definition — a grave compromise. Had the API client been scoped to *least privilege* — permission only to take and complete its one job type — the same leak would have given the attacker only the ability to do that worker's narrow job:

least-privilege blast radius: complete one job type — contained.

Discussion. The damage from the leak was determined not by the leak itself but by a decision made a year earlier — the scope granted to the API client — and the worked example shows that token management's security is made, or lost, at client-creation time. The team granted full permissions "to keep things simple," and that convenience set the blast radius of every future compromise of that credential to the maximum: a worker that only ever needed to complete one kind of job was handed, and so its leaked secret handed an attacker, the power to deploy arbitrary process definitions onto the bank's engine. The least-privilege discipline of [Chapter 140](#), applied at the moment the API client was scoped, would have confined the client — and therefore the leak — to exactly the worker's real need, turning a grave compromise into a contained one. The lesson draws the part together. Token management is not only the lifecycle mechanics of the previous chapters; it is the *governance* of what every API client may do. Each client must be scoped to least privilege at creation, every client secret vaulted and rotatable and revocable, and the whole population of clients inventoried and reviewed — because a Camunda platform's API-authentication security is the sum of these disciplines applied to every client, and the weakest client, the over-scoped one with the stale unvaulted secret, is the one an attacker will find. A platform is as secure as its least-well-managed API client.

Practical next steps

Inventory your Camunda platform's API clients. For each, check three things: is it scoped to least privilege, or granted more than its application needs; is its client secret held in the secrets-management service and rotatable; and is it still needed, or a candidate for revocation. The over-scoped client with the stale, unvaulted secret is your platform's weakest point — find it before an attacker does.

Chapter in brief

Token management is an ongoing security discipline across a Camunda platform's many API clients. The asset to protect is the long-lived *client secret*, not the short-lived token: a leaked secret obtains fresh tokens until revoked, so every secret must be vaulted as a first-class secret, never in code or plain config. Secrets must be *rotatable* and API clients promptly *revoked* when their application is retired — or the platform accumulates live credentials for applications that no longer exist. Every client must be *scoped to least privilege*, since a leaked credential's blast radius is its client's scope, and the institution must *inventory* its clients to govern them at all. A platform's API-authentication

security is the sum of these disciplines applied to every client.

PART XXVIII

Ad-hoc Processes

When the Path Is Not Known in Advance

146.1 The limit of the structured process

Every process this manuscript has modelled, until now, has had a *structure* known in advance: a defined sequence, defined branches, defined paths from start to end. That structure is the source of most of what the manuscript has praised — it is what makes a process reviewable, governable, auditable. A structured process is a process whose every possible path was decided, on the diagram, before any instance ever ran.

But not all work is like that. Some work in a bank or insurer is genuinely *not* a fixed sequence. A complex claim investigation may require some combination of evidence-gathering steps, in an order that depends on what each step reveals, with some steps repeated and some skipped — and which steps, and in what order, cannot be known when the model is drawn, because it depends on the specifics of the case. A wealth-management review, a complex-dispute resolution, a bespoke underwriting case: these are work where the *set* of possible activities is known, but the *path* through them is decided as the case unfolds.

This part treats that kind of work, and the BPMN mechanism for it: the *ad-hoc subprocess*. The part is deliberately placed near the end of the manuscript, because ad-hoc work sits at the frontier of process automation — it is where the structured-process discipline meets work that resists structure, and where, as the next chapter shows, the agentic capability of [Part XXV](#) finds one of its most natural homes.

146.2 The ad-hoc subprocess

An *ad-hoc subprocess* is a special kind of embedded subprocess — marked, in BPMN, with a tilde — that contains a set of activities which are *not connected by a fixed sequence*. Its inner activities have no mandatory start-to-end flow between them. Instead, the activities can be executed in any order, some of them repeated, some skipped entirely — and which activities run, and when, is decided as the subprocess executes rather than fixed when it is modelled.

This is a genuine departure from everything before it. A normal subprocess contains a flow: this step, then that step. An ad-hoc subprocess contains a *repertoire*: here are the activities that *may* be performed; which of them are performed, and in what order, is determined at runtime. The model defines the *set of possible activities* — and that is what keeps the ad-hoc subprocess governable, a point the next section develops — but it does not define the path.

An ad-hoc subprocess is given two things that shape its execution. It is given a way to decide *which activities to activate* — which of its repertoire to run — and it is given a *completion condition*, a boolean expression evaluated as each activity completes, which determines when the ad-hoc subprocess as a whole is finished. When the completion condition becomes true, the subprocess ends, whatever activities have or have not run.

Figure [146.1](#) draws the ad-hoc subprocess — a bounded repertoire of activities with no

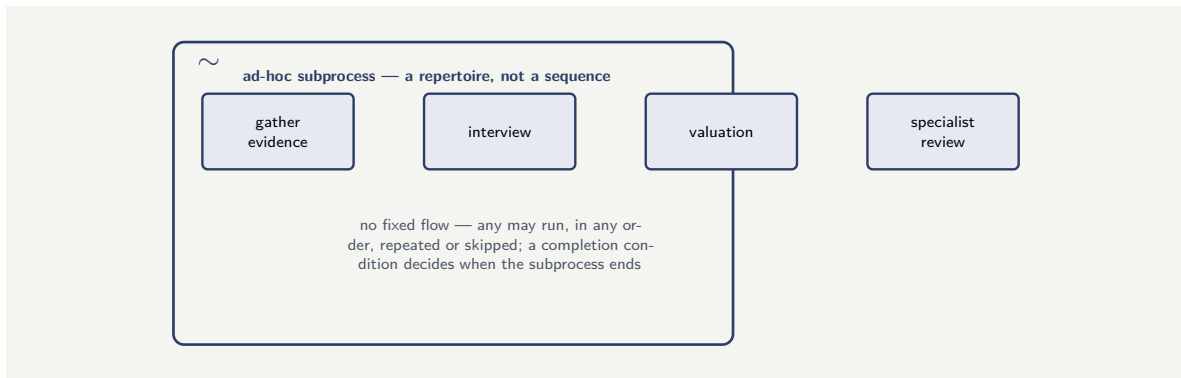


Figure 146.1. An ad-hoc subprocess, marked with a tilde. Its inner activities are a *repertoire*, not a sequence — unconnected by fixed flow. Which run, in what order, and how often is decided at runtime; a completion condition determines when the subprocess as a whole is finished.

fixed flow between them.

146.3 What the ad-hoc subprocess governs, and what it does not

The ad-hoc subprocess is, at first sight, in tension with everything this manuscript has argued. The book has praised structure — the explicit, fixed, reviewable path — and an ad-hoc subprocess deliberately abandons the fixed path. It is worth being precise about what is given up and what is kept, because the ad-hoc subprocess is governable, and understanding why is the heart of this chapter.

What the ad-hoc subprocess gives up is the *fixed sequence*. It does not predetermine the order of work, because the work's order genuinely cannot be predetermined.

What the ad-hoc subprocess *keeps* — and this is the crucial part — is the *bounded repertoire*. The set of activities the ad-hoc subprocess may perform is defined, explicitly, in the model. The ad-hoc subprocess is not “do anything”; it is “do some combination of *these defined activities*.” Each activity in the repertoire is itself a modelled element — a service task, a user task, a call activity — governed exactly as any other modelled activity is. The ad-hoc subprocess flexes the *order*; it does not abandon the *boundary*.

This is what makes the ad-hoc subprocess a governable tool rather than an escape from governance. A reviewer of an ad-hoc subprocess cannot review the path — there is no fixed path — but they *can* review the repertoire: they can see exactly which activities the subprocess may perform, challenge any that should not be in the set, and confirm that each activity is itself properly governed. And the completion condition is reviewable too — a reviewer can see the rule by which the subprocess decides it is done. The ad-hoc subprocess is governed at the level of *the set of possible activities and the rule for stopping*, rather than the level of the path — and for work whose path genuinely cannot be fixed, that is exactly the right level at which to govern.

Tip

An ad-hoc subprocess gives up the fixed sequence but keeps the bounded repertoire: it defines, explicitly, the set of activities it may perform, and each is a governed modelled

element. Review it at the level it governs — the repertoire and the completion condition — not the level it does not. The question for a reviewer is not “is the path correct?” but “is every activity in this repertoire one the subprocess should be able to perform, and is the rule for completing it sound?”

146.4 When to use an ad-hoc subprocess, and when not

The ad-hoc subprocess is a sharp tool, and the discipline of using it well is mostly the discipline of *not* using it where a structured process would do.

The temptation, once the ad-hoc subprocess is known, is to reach for it whenever a process is complex — to model a merely complicated process as an ad-hoc repertoire because drawing all its branches is tedious. This is a mistake. A process that *has* a knowable structure — however many branches — should be modelled with that structure, because the structure is governable, reviewable, and analysable in ways an ad-hoc repertoire is not. The ad-hoc subprocess is not a shortcut around modelling a complicated flow; using it so throws away governance the work did not need to lose.

The ad-hoc subprocess is right only when the path is *genuinely* unknowable in advance — when it depends, irreducibly, on what unfolds during the case. A complex claim investigation where each piece of evidence determines what to gather next; a bespoke advisory case; an exception-handling process for situations too varied to enumerate. The test is honest: *could* this process’s path be drawn, if the modeller were willing to draw all the branches? If yes, draw them — it is a structured process. If no — if the path truly depends on runtime specifics no diagram could anticipate — then the work is genuinely ad-hoc, and the ad-hoc subprocess is the honest way to model it.

146.5 Applying the chapter in practice

The principles this chapter describes are most valuable when applied deliberately and reviewed periodically. A team that applies them once, at initial design, captures their value at that moment; a team that returns to them at each major change, each annual review, and each post-incident analysis captures their value continuously. The platform evolves; the principles hold; the specific choices they inform must evolve with the platform.

Worked example: the structured process modelled as ad-hoc

Problem. A team is modelling a loan-exception process — the handling of applications that fall outside the normal automated flow. The process is complicated: there are 9 distinct exception types, and each has a defined handling procedure, several of which share steps. Finding the full structured model tedious to draw, the team instead models it as an ad-hoc subprocess: a repertoire of all the handling activities, with the operator choosing which to run for each case. Six months on, the model-risk function asks two questions: how many distinct paths can a loan exception take, and is each path correct? Assess what the team can answer, and what they have given up.

Setup. We consider whether the ad-hoc design can answer the model-risk questions, and contrast with what a structured model of the same work would have provided.

Working. *The ad-hoc design.* The model defines a repertoire of handling activities, but no fixed paths — the path for each case is whatever the operator assembles at runtime. To the question “how many distinct paths can a loan exception take?”, the ad-hoc model’s answer is:

undefined — the paths are not in the model,

and to “is each path correct?”:

cannot be answered — there is no enumerated set of paths to check.

A structured model of the same work. The 9 exception types each have a *defined* handling procedure — the team said so. A structured model would have 9 (or so) explicit paths, and model-risk could count them and check each:

9 enumerable, reviewable paths.

Discussion. The team reached for the ad-hoc subprocess to avoid the tedium of drawing a structured model, and in doing so gave up exactly what model-risk needs — and the worked example’s lesson is that this was an *unnecessary* loss, because the work was not actually ad-hoc. The decisive fact is in the problem statement: there are 9 exception types and *each has a defined handling procedure*. Work whose handling procedures are defined is work whose paths are knowable — it is a structured process, merely a complicated one, and the tedium of drawing nine procedures is not a reason to abandon structure; it is just the work of modelling. By modelling it as ad-hoc, the team made a complicated-but-structured process *look* like genuinely ad-hoc work, and so threw away the path enumeration, the per-path review, and the conformance checking that the structured model would have given for free. The ad-hoc subprocess is the right tool for work whose path *cannot* be known; it is the wrong tool for work whose path is merely tedious to draw, and the test that separates the two is the honest question of the previous section. Had the team asked it — “could these paths be drawn?” — the answer, with 9 defined procedures, was plainly yes, and the structured model was the correct choice. The lesson generalises: the ad-hoc subprocess must be reserved for genuine ad-hoc work, because using it for structured work does not save effort so much as discard governance.

Practical next steps

If your institution uses, or is considering, an ad-hoc subprocess for some piece of work, apply the honest test: could the work’s paths be drawn, if a modeller were willing to draw all the branches? If yes, it is a structured process and should be modelled as one, however tedious — the governance is worth the drawing. Reserve the ad-hoc subprocess for work whose path genuinely cannot be enumerated in advance.

Chapter in brief

Some banking and insurance work is genuinely not a fixed sequence — a complex investigation, a bespoke case — where the set of possible activities is known but the path through them is decided as the case unfolds. The *ad-hoc subprocess*, marked with a tilde, is the BPMN mechanism: it contains a *repertoire* of activities not connected by fixed flow, which may run in any order, repeated or skipped, with a completion condition deciding when it ends. It gives up the fixed sequence but keeps the bounded repertoire — each

activity a governed modelled element — so it is reviewed at the level of the repertoire and the completion rule. It must be reserved for genuinely ad-hoc work; using it for merely complicated but structured work discards governance the work need not have lost.

Ad-hoc Work, Agents, and the Edge of Automation

147.1 Where ad-hoc work and agents meet

The previous chapter established the ad-hoc subprocess as the mechanism for work whose path cannot be fixed in advance. This chapter treats what is, in many ways, the manuscript's final synthesis: the meeting of ad-hoc work and the agentic capability of [Part XXV](#). For the ad-hoc subprocess raises a question it does not itself answer — if the path is not fixed, then *who* or *what* decides which activity from the repertoire runs next? — and one of the most natural answers, in a modern platform, is an agent.

This is the point at which several of the manuscript's threads converge, and it is a fitting place for the technical content to end, because it is the frontier: ad-hoc work is process automation reaching toward the kind of flexible, judgement-driven work that was, until recently, considered beyond automation entirely.

147.2 Who decides the next activity

An ad-hoc subprocess needs something to decide, as it runs, which activities of its repertoire to activate. There are several possibilities, and they form a progression worth seeing as a whole.

The decider can be a *human*. The ad-hoc subprocess presents its repertoire to an expert — a senior claims investigator, a wealth adviser — and the expert chooses, case by case, which activity to perform next, drawing on their own judgement. This is the most established model, and for genuinely expert work it is often exactly right: the ad-hoc subprocess gives the expert a governed repertoire and a recorded history of what was done, while leaving the judgement of *what to do* where it belongs — with the expert.

The decider can be *rules*. The ad-hoc subprocess can be driven by decision logic — a DMN decision of [Chapter 4](#) evaluated as the case develops — which selects the next activity from the repertoire based on the case's current state. This suits ad-hoc work whose “unstructuredness” is really a large, data-dependent branching that rules can express even though a fixed flow cannot.

And the decider can be an *agent*. The agentic capability of [Part XXV](#) — an agent that reasons toward a goal, choosing actions from a defined toolset — maps remarkably naturally onto the ad-hoc subprocess. The ad-hoc subprocess's *repertoire of activities* is, almost exactly, the agent's *toolset*. The agent, given the goal of the ad-hoc work — “investigate this claim,” “resolve this case” — reasons about the case and chooses, from the repertoire, which activity to activate next, observes the result, and chooses again, until the completion condition is met. The ad-hoc subprocess and the agent fit together so well because they are, in a sense, the same idea reached from two directions: a bounded set of possible actions, with the path through them decided dynamically.

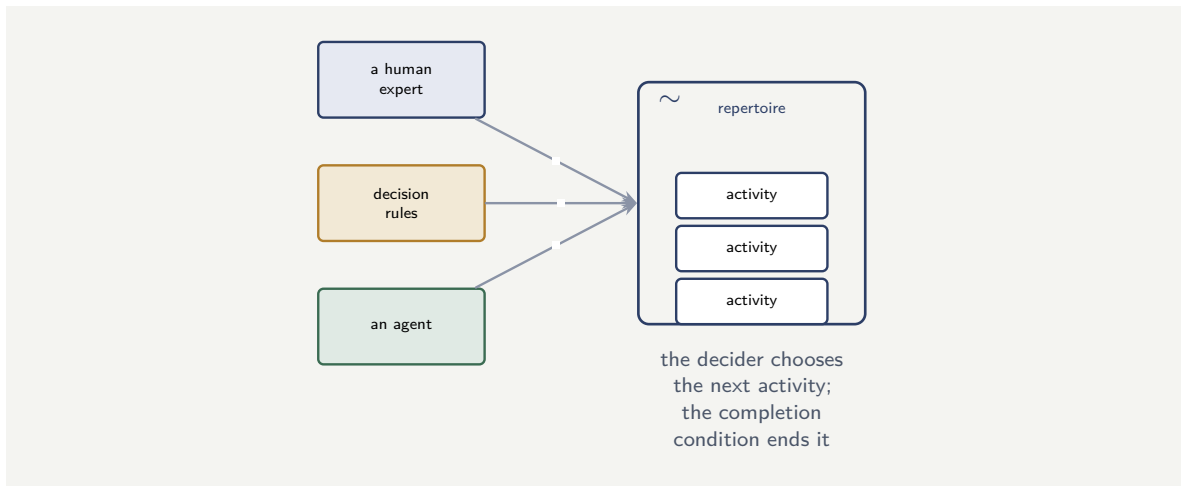


Figure 147.1. Three deciders for an ad-hoc subprocess. A human expert, decision rules, or an agent can each drive the choice of which activity from the repertoire to run next. The agent fits especially naturally: the repertoire is, almost exactly, the agent’s toolset.

Figure 147.1 draws the three deciders, each choosing from the same governed repertoire.

147.3 The governance of agent-driven ad-hoc work

An agent driving an ad-hoc subprocess is a powerful combination — and exactly because it is powerful, it must be governed with the full discipline of [Part XXV](#). It is worth being explicit, because this combination could, misused, be the most ungoverned thing in the manuscript: an agent choosing freely what to do, in a process with no fixed path.

It is not ungoverned, provided the disciplines hold — and they are the disciplines the manuscript has already built. The ad-hoc subprocess’s *repertoire* is the agent’s *bounded toolset* — the agent may activate only the defined activities, exactly the containment of [Chapter 23](#). Each activity in the repertoire is a governed modelled element, and any with a consequential effect is itself adjudicated and accountable, exactly as [Part XXV](#)’s agentic BPMN required. The completion condition bounds the work. And the audit record captures what the agent did — which activities it chose, in what order, to what result — so the ad-hoc, agent-driven work is, after the fact, as accountable as any structured process.

The synthesis, then, is this: an agent driving an ad-hoc subprocess is governed not by a fixed path — there is none — but by the bounded repertoire, the governed activities, the completion condition, the adjudication of consequential actions, the accountable human, and the audit record. It is [Part XXV](#)’s contained agent and the previous chapter’s governed repertoire, fitted together — and fitted together, they govern even work that has no path.

An agent driving an ad-hoc subprocess — choosing freely which activities to run, in a process with no fixed path — is governed only if the disciplines hold: the repertoire is the agent’s bounded toolset, every activity is a governed element, consequential activities are adjudicated and have an accountable human, the completion condition bounds the work, and the audit record captures it all. Drop those disciplines and the combination is the most ungoverned thing a platform could run — an agent doing what it likes inside the institution. Keep them, and even pathless work is fully governed.

147.4 The edge of automation, and the discipline that holds there

This is the manuscript's last technical chapter, and the ad-hoc, agent-driven process is a fitting place to end, because it is the *edge of automation* — the point where process automation reaches into work that has, until very recently, been considered the exclusive domain of human judgement: the complex investigation, the bespoke case, the situation no flow could anticipate.

The manuscript's claim, made first in [Chapter 1](#) and tested in every part since, holds even here, at the edge. The claim was never that automation should be limited to simple, structured work — it was that automation, of whatever work, is safe and valuable only when it is *governed*: when there is an explicit, bounded model of what may happen; when consequential decisions are deterministic and owned; when probabilistic components are contained; when a named human is accountable; when an honest audit record is kept. Structured processes meet that claim through their fixed, reviewable paths. Ad-hoc processes meet it through their bounded, reviewable repertoires. Agent-driven ad-hoc processes meet it through the union of the agentic containments and the ad-hoc repertoire. The *form* of the governance changes as the work becomes less structured; the *fact* of the governance does not.

That is the manuscript's final word. The technologies will continue to advance — Camunda will release new versions, agents will grow more capable, new patterns will emerge for work even less structured than today's ad-hoc cases. An institution that holds to the discipline — explicit and bounded models, governed decisions, contained probabilistic components, accountable humans, an honest audit record — can adopt every one of those advances, safely, including the ones that reach further into work once thought beyond automation. An institution that abandons the discipline, seduced by the capability, builds something powerful that it cannot explain, cannot govern, and cannot answer for — and in banking and insurance, the inability to answer for what was done is not a technical shortcoming but the end of the institution's licence to operate. Hyperautomation, governed, is one of the most powerful things a financial institution can build. Governed is the word that the whole of this manuscript has been written to insist upon.

Worked example: governing an investigation without a path

Problem. An insurer builds a complex-claim investigation as an agent-driven ad-hoc subprocess. The repertoire has 7 activities: gather documents, interview the claimant, commission a valuation, check the fraud watchlist, request a specialist opinion, recalculate the reserve, and *settle the claim*. An agent, given the goal “investigate and resolve this claim,” chooses activities from the repertoire as the case develops. A model-risk reviewer examines the design before it goes live. Six of the seven activities are read-or-assess activities; the seventh, *settle the claim*, moves money. What should the reviewer conclude, and how does this show the governance of pathless work?

Setup. We assess the repertoire against the agentic containment discipline — in particular, whether a consequential, money-moving activity should be one the agent may activate at its own discretion.

Working. The six read-or-assess activities — gather, interview, commission, check, request, recalculate — are activities whose effects are informational; an agent activating them at its discretion is the agent doing its investigative work, and that is the intended use. The seventh, *settle the claim*, moves money — it is a consequential action, and per [Part XXV](#)'s disci-

pline it must not be an activity the agent activates autonomously. The reviewer's conclusion:

6 activities: appropriate for the agent's discretion,

1 activity (settle): must be gated — adjudicated, with an accountable human.

The fix is not to remove *settle* from the process, but to ensure that when the agent's investigation concludes a claim should be settled, that settlement passes through a deterministic adjudication and a human decision — not the agent reaching for the settle activity itself.

Discussion. The reviewer can do this review — can find the one dangerous activity in the repertoire — precisely because the ad-hoc subprocess governs at the level of the repertoire, and the repertoire is an explicit, reviewable list of seven activities on the model. There is no fixed path to review, and the reviewer does not need one; what they need, and have, is the bounded set of things the agent can do, and running their eye down that set they can see that six are safe for agent discretion and one is not. This is the synthesis of the manuscript's last two parts working exactly as intended: the ad-hoc subprocess of the previous chapter contributes the governed repertoire; **Part XXV**'s agentic discipline contributes the rule that a consequential, money-moving action is never the agent's to take autonomously. Together they govern a process that has no path at all — an agent investigating a claim however the case leads — and govern it well: the agent has real freedom to investigate, drawing on six activities in whatever order the case demands, and the one activity that could do harm is gated behind adjudication and a human. The worked example is the manuscript's argument completed. Pathless, agent-driven, genuinely ad-hoc work — the edge of what can be automated — is not beyond governance. It is governed, like everything before it, by a bounded and explicit model of what may happen, by the adjudication of consequential actions, and by an accountable human — and an institution that holds those disciplines can automate even this, and still answer for every claim it settles.

Practical next steps

If your institution is building agent-driven or ad-hoc work, review its repertoire as the model-risk reviewer did: list every activity the work can perform, and for each ask whether it is informational — safe for an agent's or an operator's discretion — or consequential, in which case it must be gated behind adjudication and an accountable human. The repertoire is reviewable even when the path is not, and that review is how pathless work is kept governed.

Chapter in brief

An ad-hoc subprocess needs something to decide which activity runs next, and the deciders form a progression: a human expert, decision rules, or an agent. The agent fits especially naturally — the ad-hoc repertoire is, almost exactly, the agent's bounded toolset. An agent driving an ad-hoc subprocess is powerful and must carry the full discipline of **Part XXV**: the repertoire bounds the agent, every activity is governed, consequential activities are adjudicated with an accountable human, the completion condition bounds the work, the audit record captures it. This is the edge of automation — work once thought beyond it — and the manuscript's claim holds even here: the form of governance changes as work becomes less structured, but the fact of it does not. Governed is the word the whole manuscript insists upon.

CHAPTER A

A Modelling Reference for BPMN and DMN

This appendix is a compact reference for the modelling notations the manuscript relies on. It is not a substitute for the BPMN and DMN specifications; it collects, in one place, the restricted subset [Chapter 3](#) and [Chapter 4](#) recommend for disciplined financial-services modelling, with a note on when each element is the right choice.

A.1 The recommended BPMN subset

A financial-services modelling standard should permit a small, declared set of BPMN elements and forbid the rest. The set below is sufficient for almost every process in this manuscript.

A.1.1 Events

Start event. Begins a process instance. A process may have more than one — a different start event for each distinct way the process can be initiated — but each instance begins at exactly one.

End event. Completes a path of the process. A model should have a distinct, named end event for each materially different outcome — approved, declined, abandoned, lapsed — because the end event is how the process records, and observability reports, how an instance concluded.

Intermediate message event. Waits, during the process, for something external to arrive. The natural representation of every point where the process depends on a customer action, a third party, or another system.

Intermediate timer event. Waits for a duration or until a date. Used on its own to model a deliberate delay, and — far more often — as a boundary event to model an SLA.

Boundary event. Attached to the edge of an activity, expressing that the activity may be interrupted: a boundary timer for an SLA, a boundary message for an external cancellation, a boundary error for a failure. The boundary event is the single most useful construct for modelling the exception-prone reality of financial processes.

A.1.2 Activities

User task. Work for a human, placed on a worklist. The modelled point where human judgement and accountability enter the process. Every consequential decision that this manuscript keeps human-accountable appears in a model as a user task.

Service task. Automated work — an integration call, a rule evaluation, an agent invocation — executed through the job-and-worker mechanism of [Chapter 21](#). From the model's point of view an agent invocation and a simple integration call are both service tasks; the difference is in what the job worker does.

Business rule task. A decision delegated to a DMN model. Distinguished from a service task because it makes explicit, in the diagram, that a governed decision is being taken at this

point — which a reviewer reading the model should be able to see at a glance.

Call activity. Invokes another process model as a subprocess. The mechanism of decomposition: it is how a journey model is built from capability models, and how a model stays readable and a subprocess stays independently governable.

A.1.3 Gateways

Exclusive gateway. Chooses exactly one outgoing path based on a condition. The workhorse of routing. Its conditions should be exhaustive — every possible case leads somewhere — and a default path guards against the case the modeller did not foresee.

Parallel gateway. Splits flow into concurrent paths and, in its joining form, waits for all of them. Used when genuinely independent work — two integration calls that do not depend on each other — can proceed at once, as [Chapter 25](#)’s latency worked example recommended.

Event-based gateway. Waits for whichever of several events occurs first. The natural model of “the customer accepts, the customer rejects, or the offer expires — whichever happens first,” the pattern [Chapter 121](#) used for the lending offer.

A.1.4 Connecting and grouping

Sequence flow connects elements and carries the order of execution. *Pools and lanes* group activities by participant, showing which actor — customer, staff, automated service, agent — is responsible for each part of the model.

A.2 What the BPMN subset deliberately omits

A disciplined standard is defined as much by what it forbids as by what it permits. The manuscript’s recommended subset omits the more exotic BPMN constructs — complex and inclusive gateways, the rarer event types, ad-hoc subprocesses, and the like — not because they are invalid but because each one bought into the standard costs readability and reviewability for a small gain in expressive economy. A model a business stakeholder cannot read and a risk reviewer cannot review has lost the property that justified using BPMN at all. When a modeller feels the need for an exotic element, that feeling is usually a signal that the process should be decomposed differently, not that the standard should be widened.

A.3 The DMN reference

A.3.1 The decision table

A DMN decision table represents one decision. Each row is a rule; the input columns are the conditions under which the rule applies; the output columns are the results the rule produces. The table is read row by row: when the inputs satisfy a row’s conditions, that row’s outputs are the decision.

A.3.2 Input ranges and the partition discipline

[Chapter 4](#)’s worked example showed the central discipline of building a decision table: the input ranges across the rules must *partition* the input space — they must be contiguous, non-overlapping, and exhaustive. Contiguous and exhaustive means every possible input is covered by some rule; non-overlapping means no input is covered by more than one. The standard tool for achieving this with numeric inputs is the half-open interval: ranges of the form $[a, b)$ placed end to end leave neither gaps nor overlaps at the boundaries, which is exactly

where casual table-building goes wrong.

A.3.3 Hit policy

The hit policy declares what the table does when more than one rule matches. *Unique* requires that rules never overlap, so at most one matches; it is the safest choice for a regulated decision, because the DMN tooling then enforces non-overlap and a contradiction between two rules becomes impossible by construction. *First* takes the first matching rule in order; *Collect* aggregates all matching rules. Both have legitimate uses, but a financial-services standard should default to *Unique* and require a documented reason for any other choice.

A.3.4 The decision requirements diagram

A real decision is rarely a single table. A decision requirements diagram shows how decisions depend on one another and on input data: a final *pricing* decision depending on a *risk tier* decision and a *discount eligibility* decision, each its own table, each fed by named inputs. The diagram makes the decision logic itself a modelled, governable structure — the decision-side counterpart of the layered process model.

A.4 A modelling checklist

The following checklist consolidates the modelling discipline of Parts I and II into questions a modeller, or a reviewer, can ask of any model.

Is the model at a declared *level* — value stream, process, or executable — and readable at that level? Does every *path* from a start event reach an end event, and is every distinct path a defined test case? Is every *exception* — every way the happy path can be left — modelled explicitly, including abandonment and failure? Does every consequential *decision* live in a DMN model invoked by a business rule task, not in a gateway condition or service-task code? Do the input ranges of every decision table *partition* the input space? Is every *long wait* an explicit message or event-based gateway with an escalation timer? Is every *agent* invocation bounded in scope, bounded in tools, adjudicated by a deterministic step, and — where consequential — followed by a human task? Does the model have a named *process owner*, and does every decision, model, and agent it uses have a named owner? A model that answers all of these affirmatively is one that can be governed.

CHAPTER B

Implementation Patterns

This appendix collects, as named patterns, the recurring solutions the manuscript has used across its application chapters. A pattern is a solution shape worth recognising and reusing; naming them makes them easier to apply deliberately and to review.

B.1 The lean skeleton with a deliberate exit

Context. A process has a very high-volume, low-complexity happy path and a lower-volume, high-complexity exception path — the shape of payments in [Chapter 122](#).

Pattern. Model the happy path as a deliberately minimal, fast, deterministic sequence. Make the first significant step a DMN decision that routes the case: the overwhelming majority to the lean path, the minority with genuine complexity to a rich exception process. Keep intelligence — agents, heavy decision logic — out of the high-volume path entirely and concentrate it in the exception process.

Why. Latency and cost scale with volume. Intelligence added to a path that runs millions of times a day is millions of expensive invocations for no benefit, because the happy path has nothing to reason about.

B.2 The layered decision

Context. A process must act on something that is partly a prediction and partly a policy — lending in [Chapter 121](#), fraud routing in [Chapter 123](#).

Pattern. A predictive model produces a calibrated score. A DMN decision table consumes that score, together with hard rules and policy constraints, and produces the action. The model and the table have different owners and different review cycles.

Why. It places prediction and policy where each can be governed by the people accountable for it, keeps the action explainable as a rule even when the score is from a complex model, and lets the model be retrained and the policy be amended independently.

B.3 The contained agent

Context. A process step requires open-ended judgement over unstructured information — the consistency check in [Chapter 23](#), the claims assessment in [Chapter 123](#), the intake classification in [Chapter 124](#).

Pattern. Place the agent as a service task with four containments: a bounded task and a defined output schema; a minimal, governed toolset; a deterministic adjudication step — a DMN table or a rule — consuming its output; and, for consequential decisions, a human task on the path. Trace the agent's inputs, tool calls, and output against the process instance.

Why. The agent's capacity for confident, plausible error is its defining risk. Containment ensures that error cannot reach an external consequence ungoverned, and the trace makes the agent's contribution observable and explainable.

B.4 The long-running wait

Context. A process must wait — hours to months — for a customer, a third party, or another system, surviving restarts and remaining interruptible. Present in onboarding ([Chapter 120](#)), lending offers ([Chapter 121](#)), and claims ([Chapter 123](#)).

Pattern. Model the wait as an intermediate message event or an event-based gateway. Attach a boundary timer that escalates — reminder, then second reminder, then a defined abandonment or lapse path — when the wait runs too long. Ensure each terminal outcome, including the abandonment path, has defined clean-up.

Why. The orchestrator persists every waiting instance, so the wait costs nothing while it lasts and survives platform restarts. Modelling the wait explicitly makes every SLA a visible timer and ensures incomplete cases end cleanly rather than becoming litter.

B.5 The reusable capability

Context. Several end-to-end journeys need the same sub-process — identity verification serves onboarding, lending, and insurance applications alike.

Pattern. Model the shared sub-process as a self-contained capability model with its own owner and version. Each journey invokes it through a call activity. Change the capability once; every journey that calls it benefits, and none is affected unless the capability's interface changes.

Why. It gives the process estate changeability with bounded blast radius, removes duplicated logic, and means a shared concern — identity, in the example — is reviewed and governed once rather than many times.

B.6 The anti-corruption boundary

Context. The platform must integrate with legacy systems of record whose data models and quirks would, if exposed directly, rigidify the new platform — [Chapter 25](#).

Pattern. Place a translation tier between the legacy systems and the orchestration platform. Expose the legacy systems to the platform through clean, stable, business-meaningful interfaces. Absorb all legacy translation, protocol adaptation, and quirks inside the tier.

Why. Process models and decision tables stay free of legacy detail, and because the platform couples to the clean interface rather than the legacy system, that system can be modernised or replaced with the change absorbed inside the tier and the process models untouched.

B.7 Observe, signal, respond

Context. A running platform's processes, decisions, models, and agents must be watched, and watching has value only if it drives action — [Chapter 125](#).

Pattern. Expose process metrics (instance counts, cycle times, path frequencies, SLA attainment) and decision metrics (decision-rate distributions, score distributions, agent agreement rates) as live, owned dashboards. Define, for each metric, a signal threshold, an owner, and a response — some automated (escalation, scaling, alerting), some human (referral to model risk, investigation by a process owner).

Why. Models and agents degrade silently; without designed observation wired to response, the first sign of a problem is a complaint or a regulatory finding. A signal with no owner and

no response is not observability.

B.8 Using the patterns

These patterns are not independent; a real application chapter combines several. Claims processing ([Chapter 123](#)) uses the long-running wait for the claim lifecycle, the layered decision for fraud routing, the contained agent for evidence assessment, the reusable capability for identity steps shared with onboarding, the anti-corruption boundary for the policy administration system, and observe-signal-respond for monitoring the fraud model. Recognising the patterns is what lets an architect assemble a new application quickly and a reviewer check it systematically — and every one of them is, at bottom, the same idea the manuscript has argued from its first page: make the process explicit, contain the intelligence, adjudicate deterministically, and keep an accountable human in the loop.

CHAPTER C

A Regulatory and Governance Reference

This appendix collects the governance obligations the manuscript has treated across Part XIV into a single reference. It is organised as a set of questions an institution should be able to answer about any hyperautomated process, with a note on where in the architecture each answer is found. It is not legal advice and not jurisdiction-specific; it is a structuring of the questions a serious internal or supervisory review will ask.

C.1 Explainability questions

For any decision the platform made, can the institution produce the process path the case took? The answer is found in the orchestrator's audit record ([Chapter 21](#)): because the process is an explicit model, the sequence of states the instance passed through is recorded, not reconstructed.

Can it produce the exact rule that determined the outcome? The answer is found in the DMN decision trace ([Chapter 4](#)): the orchestration engine records, for each business rule task, which rule in which table fired and on what inputs.

Can it produce the model version and score that contributed, and the inputs the model saw? The answer is found in the decision service contract ([Chapter 22](#)): a predictive decision service returns a versioned, attributed score, and the inputs-as-recorded are captured alongside it.

Can it produce the agent's assessment and the reasoning trace behind it? The answer is found in the agent invocation record ([Chapter 23](#)): the agent's inputs, tool calls, intermediate steps, and schema-bounded output are traced against the process instance.

Can it produce the human actions and the accountable identities? The answer is found in the identity model ([Chapter 26](#)): every action, human or automated, links to an accountable identity.

If the institution can answer all five quickly and without archaeology, explainability has been designed in. If any answer requires reconstruction, the platform has an explainability gap that a review will find.

C.2 Fairness questions

Has the institution chosen, and can it defend, a definition of fairness for each model-driven decision? Fairness is not a single objective notion; the institution must select a criterion deliberately ([Chapter 127](#)) and be able to justify the choice.

Was the model's behaviour across customer groups measured at validation, and against that criterion? This is part of the model risk discipline ([Chapter 126](#)): fairness is examined before deployment, by people independent of those who built the model.

Is the decision-rate divergence across groups monitored in production? This is decision observability ([Chapter 125](#)): a fairness property is not established once but watched continuously.

Can the institution show that the factors it treats as legitimate explanations of a decision-rate gap

are not themselves proxies for group membership? This is the hardest question, raised by [Chapter 127](#)'s worked example, and the honest answer is usually less comfortable than the institution would like.

C.3 Accountability questions

For every consequential decision the platform makes, is there a named, specific human who is accountable? "The model decided" is not an acceptable answer for a regulated institution.

Is that human supported by the platform — brought the right case with the right context — rather than merely nominally responsible? A human who is accountable but cannot realistically exercise judgement, because the platform does not give them what they need, is an accountability fiction.

Does every component of the platform — each process, decision, model, and agent — have a named owner? This is the operating model ([Chapter 17](#)): an unowned component is a governance gap.

Is the disposition of every high-stakes decision — a sanctions hit, a fraud denial, a sensitive servicing interaction — on a deterministic, human-accountable path rather than an agent's or a model's unaided output? This is the recurring discipline of Parts III and IV.

C.4 Model and agent governance questions

Does every predictive model in production have a named owner, a documented validation, and a revalidation cadence? A model without all three is, in the words of [Chapter 126](#), an incident waiting to be discovered.

Is every model monitored for drift, and is there a defined response when drift is detected? Monitoring without a wired response is not governance.

Is every agent bounded in scope and tools, adjudicated by a deterministic step, and validated continuously against human judgement? The four containments of [Chapter 23](#) and the continuous validation of [Chapter 126](#) are not optional hardening; they are the conditions under which an agent may be in a regulated process at all.

Are tools with external consequence — moving money, binding cover, changing a system of record — kept out of agents' reach entirely? This is the agentic security surface of [Chapter 26](#).

C.5 Resilience and operational questions

Does every process have explicit recovery-time and recovery-point objectives, agreed with the business? These are the resilience requirements of [Chapter 24](#).

Has the platform's recovery behaviour been tested — including by deliberate failure injection — rather than assumed? Recovery that has not been tested is recovery that is hoped for.

Is the platform described as code, so that every change is reviewable, auditable, and reproducible? A platform assembled by hand cannot answer the question "how do you know it is configured as approved?"

Is the sustaining budget — including the governance portion — sized bottom-up from the components actually in production? [Chapter 17](#)'s worked example showed how a plausible-looking maintenance budget can silently under-fund the governance work and let the platform decay invisibly.

C.6 Using this reference

The questions above are deliberately phrased so that a vague or reconstruction-dependent answer is itself the finding. An institution that walks this reference before a supervisory review, rather than during one, will discover its own gaps while they are still cheap to close. That is the recurring economic argument of Part XIV restated as a procedure: governance designed in and checked early is inexpensive; governance retrofitted under examination is not. The reference is most useful applied not once but on a cadence, to a sample of real decisions the platform has actually made — because, as [Chapter 17](#) insisted, the platform is never finished, and neither is the work of governing it.

Glossary

This glossary collects the terms used throughout the manuscript that bridge the vocabularies of process engineering, machine learning, and financial services. Definitions are deliberately short and are framed as the manuscript uses the term, not as a general dictionary would.

Agent

A software component given a task and a set of tools that decides for itself, reasoning step by step, how to use the tools to complete the task. Distinguished from a predictive model, which answers a fixed question with no latitude in method.

Agentic layer

The reasoning layer of the architecture: the agent runtime and the governed tools agents may invoke. One of the three layers, alongside orchestration and the cloud.

Anti-corruption layer

A deliberate translation tier between the institution's legacy systems of record and the orchestration platform, exposing the legacy systems through clean, stable, business-meaningful interfaces and absorbing their quirks.

Boundary event

A BPMN event attached to the edge of an activity, expressing that something — a timer, a message, an error — may interrupt that activity. The natural way to model SLAs and exceptions.

BPMN

Business Process Model and Notation: the standard, graphical, and executable notation used to model the flow of a process. The language of orchestration in this manuscript.

Calibration

The property of a predictive score that its numerical value matches the observed frequency of the outcome: among cases scored 0.8, about 80% genuinely have the outcome. Required wherever a score is treated as a probability.

Call activity

A BPMN element that invokes another process model as a subprocess. The mechanism of decomposition: large processes are built from independently owned, reusable subprocess models.

Capability model

In the layered modelling style, a self-contained, separately owned and versioned process model — verify identity, assess affordability — reused across several end-to-end journeys.

Containment

The set of mechanisms — bounded scope, bounded tools, deterministic adjudication, human accountability — that surround an agent in a regulated process so that its capacity for confident error cannot reach an external consequence ungoverned.

Decision service

Any component, rule-based or predictive, that the orchestrator invokes with inputs to obtain a structured decision result. DMN evaluations and predictive models are both decision services behind a common contract.

Decision table

The core DMN construct: a tabular decision in which each row is a rule, input columns are conditions, and output columns are results. Readable by non-technical experts and formally checkable.

Decomposition

The architectural practice of building the process estate as layered journey, capability, and utility models rather than a flat sprawl, giving changeability with bounded blast radius.

DMN

Decision Model and Notation: the standard notation for modelling decisions separately from process flow. Governs rule-based, auditable, business-owned decisions.

Drift

The gradual divergence of the world a predictive model now operates in from the world its training data represented, causing the model's accuracy to decay silently. Detected by monitoring, addressed by revalidation.

Event-based gateway

A BPMN gateway that waits for whichever of several events occurs first — the natural way to model “wait for the customer's response or the deadline, whichever comes first.”

Exclusive gateway

A BPMN gateway that chooses exactly one outgoing path based on a condition. The workhorse of process routing.

Explainability

The property that, for any decision the platform made, the institution can say why. An orchestrated decision is explainable by construction, assembled from the process path, the rules that fired, the model scores, the agent assessments, and the human actions.

Hit policy

The rule governing what a DMN decision table does when several rules match an input. Unique, which forbids overlap, is the safest default for regulated decisions.

Hyperautomation

The practice of treating a business process as an explicit, executable, governable model and orchestrating against it every capability — rules, models, agents, and humans. Distinguished sharply from RPA-at-scale.

Job

A unit of automated work created by the orchestration engine when a token reaches a service task, picked up and completed by a job worker. The mechanism that lets the engine and the automated work scale and fail independently.

Job worker

A separate process that picks up jobs from the orchestration engine, performs the work, and

reports completion. The component whose queue-rather-than-fail behaviour makes the platform degrade gracefully.

Journey model

In the layered modelling style, a thin top-level model describing an end-to-end customer journey — onboarding, borrowing, claiming — as a short sequence of call activities to capability models.

Least privilege

The security principle that every actor, human or machine, has only the access its role genuinely requires. Limits the blast radius of any single compromised credential.

Model risk

The discipline of understanding, bounding, governing, and monitoring the inevitable wrongness of a predictive model: validation, calibration and fairness checks, monitoring, and revalidation.

Orchestration core

The component that holds the process model and runs it, owning process state, flow control, task distribution, and the audit record — and deliberately owning no business logic.

Process model

A precise, shared, executable description of how a unit of work moves from initiation to completion: the steps, decisions, actors, events, and data.

Process owner

The named individual from the business accountable for what a process is supposed to achieve and for its observed performance metrics.

Recovery-point objective

The maximum amount of in-flight state an institution will tolerate losing when a process's platform fails. A resilience requirement the architecture is built and tested against.

Recovery-time objective

The maximum time an institution will tolerate before a process is running again after a failure. A resilience requirement the architecture is built and tested against.

Service task

A BPMN task representing automated work — an integration call, a rule evaluation, an agent invocation — handled through the job mechanism.

Shadow system of record

The damaging anti-pattern in which the orchestration platform accumulates its own drifting copy of balances, policy details, or reserves, which then disagrees with the authoritative system.

System of record

An authoritative institutional system holding the truth about an entity — the core banking system for accounts, the policy administration system for policies. The platform references and instructs these but does not duplicate them.

Token

The representation of a running process instance's position: a token sits on a model element

and moves along sequence flows as elements complete. The set of token positions is the instance's persisted state.

User task

A BPMN task representing work for a human, placed on a worklist. The modelled point at which human judgement and accountability enter a process.

Worked example

The recurring element closing each chapter of this manuscript: a concrete, numerical version of a problem from the chapter, worked through as problem, setup, working, and discussion.

A closing note. The manuscript has argued one idea from many angles: that hyperautomation in banking and insurance succeeds or fails on discipline, not on technology. The orchestrator, the decision engine, the predictive models, the agents, and the cloud are all necessary, and assembling them is real engineering. But the deliverable is a *governed capability* — explicit process models, contained intelligence, deterministic adjudication, measured fairness, observable operation, and a named accountable human at every consequential decision. An institution that builds that can adopt each new advance safely as it arrives. An institution that builds clever automation without the governance has built something it can neither trust nor keep. The discipline is the deliverable.